淘晶驰串口屏开发手册

发行版本 *1.1.0-2026-05-13 17:26:41*

TJCFAE

2026-05-13 17:26:41

淘晶驰教程

1 快速入门 **32**

1.1 下载和安装上位机软件 . . . 32
1.2 上位机基本功能介绍 . . . 43
1.2.1 1. 菜单栏 . . . 47
1.2.1.1 文件菜单 . . . 47
1.2.1.2 工具菜单 . . . 49
1.2.1.3 设置菜单 . . . 50
1.2.1.4 帮助菜单 . . . 51
1.2.1.5 关于菜单 . . . 52
1.2.2 2. 工具栏 . . . 53
1.2.3 3. 工具箱 . . . 56
1.2.3.1 控件列表 . . . 57
1.2.4 4. 资源文件窗口 . . . 59
1.2.4.1 图片资源 . . . 59
1.2.4.2 图片资源右键菜单 . . . 60
1.2.4.3 字库资源 . . . 61
1.2.4.4 字库资源右键菜单 . . . 61
1.2.4.5 动画资源 . . . 63
1.2.4.6 动画资源右键菜单 . . . 64
1.2.4.7 视频资源 . . . 65
1.2.4.8 视频资源右键菜单 . . . 65
1.2.4.9 音频资源 . . . 66
1.2.4.10 音频资源右键菜单 . . . 66
1.2.5 5. 界面 . . . 67
1.2.5.1 界面窗口 . . . 67
1.2.5.2 program.s 详解 . . . 68
1.2.6 6. 特殊控件窗口 . . . 69
1.2.7 7. 输出窗口 . . . 70
1.2.8 8. 事件编辑窗口 . . . 71
1.2.8.1 页面事件详解 . . . 71
1.2.8.2 控件事件详解 . . . 72
1.2.9 9. 页面窗口 . . . 75
1.2.10 10. 属性窗口 . . . 76
1.2.11 11. 调试窗口 . . . 77
1.2.12 12. 下载窗口 . . . 85
1.2.13 13. 设备 . . . 86
1.2.13.1 设置窗口 . . . 86
1.2.13.2 设备窗口 . . . 86
1.2.13.3 显示窗口 . . . 88
1.2.13.4 工程窗口 . . . 89

1.3 到手测试 . . . 90
1.3.1 普通 TTL/232 接口的串口屏 . . . 90
1.3.2 FPC 软排接口 . . . 91
1.3.3 485 接口的串口屏 . . . 96
1.3.4 USB 转 TTL 工具 . . . 99
1.4 创建工程 . . . 105
1.4.1 演示工程素材下载 . . . 105
1.4.1.1 新建一个工程 . . . 105
1.4.1.2 制作开机页面 . . . 109
1.4.1.3 制作主页面 1 . . . 115
1.4.1.4 报错：字库 ID 无效 . . . 122
1.4.1.5 创建字库和导入字库 . . . 125
1.4.1.5.1 创建字库-相关链接 . . . 132
1.4.1.6 按钮控件美化 . . . 133
1.4.1.7 通过代码修改控件属性 . . . 136
1.4.1.8 安装串口驱动 . . . 138
1.4.1.8.1 串口驱动下载 . . . 143
1.4.1.9 通过串口下载工程到串口屏 . . . 144
1.4.1.10 使用 TFT 文件下载助手 (TFTFileDownload) 通过串口下载工程到串口屏 . . . 150
1.4.1.11 通过 SD 卡下载工程到串口屏 . . . 155
1.4.1.11.1 如何输出 TFT 生产文件 . . . 155
1.4.1.11.2 如何使用读卡器 . . . 156
1.4.1.11.3 格式化 SD 卡为 fat32 . . . 157
1.4.1.11.4 使用 SD 卡下载工程到串口屏 . . . 159
1.4.1.12 修改设备型号 . . . 160
1.4.1.13 RTC 相关 . . . 161
1.4.1.13.1 校准 RTC 时钟 . . . 161
1.4.1.13.2 RTC 电池型号 . . . 162
1.4.1.14 配置亮度 . . . 163
1.4.1.15 配置串口波特率 . . . 165
1.4.1.15.1 配置串口波特率-相关链接 . . . 166
1.4.1.16 配置休眠 . . . 167
1.4.1.17 跨页面操作变量 . . . 168
1.4.1.17.1 跨页面操作变量-相关链接 . . . 168
1.4.1.18 页面滑动切换 . . . 169
1.5 代码编写 . . . 172
1.5.1 main 函数在哪里 . . . 172
1.5.2 代码写在哪里 . . . 172
1.5.2.1 program.s . . . 172
1.5.2.2 控件事件 . . . 174
1.5.2.2.1 页面控件事件 . . . 174
1.5.2.2.2 按下和弹起事件 . . . 175
1.5.2.2.3 滑动事件 . . . 175
1.5.2.2.4 定时事件 . . . 176
1.5.2.2.5 播放完成事件 . . . 176
1.5.3 函数调用 . . . 176

2 串口屏调试 177
2.1 模拟器调试基础知识 . . . 177
2.1.1 在没有串口屏的情况下进行调试 . . . 177
2.1.1.1 模拟器调试示例一 . . . 178
2.1.1.2 模拟器调试示例二 . . . 181
2.2 和 USB 转 TTL 的接线说明 . . . 187
2.3 和单片机的接线说明 . . . 192
2.4 联机调试基础知识 . . . 195
2.4.1 模拟器 (电脑) 与串口屏实物连接 . . . 195
2.4.2 模拟器 (电脑) 与单片机连接 . . . 202
2.4.2.1 模拟器连接 stm32 . . . 202

2.4.2.2 模拟器连接 arduino . . . 204
2.4.3 串口屏与单片机连接 . . . 211
2.4.3.1 串口屏连接 stm32 . . . 213
2.4.3.2 串口屏连接 arduino mega 2560 . . . 215
2.4.3.3 串口屏连接 arduino mega uno . . . 217
2.4.3.4 串口屏与单片机连接说明 1 . . . 218
2.4.3.5 串口屏与单片机连接说明 2 . . . 220
2.4.3.6 串口屏与单片机连接说明 3 . . . 221
2.4.3.7 串口屏与单片机连接说明 4 . . . 221
2.4.3.8 串口屏与单片机连接说明 5（串口解析） . . . 221
2.5 串口屏通讯协议 . . . 224
2.5.1 单片机发送数据给串口屏 . . . 224
2.5.1.1 使用淘晶驰协议控制串口屏 . . . 224
2.5.1.2 单片机使用其他协议控制串口屏 . . . 227
2.5.2 串口屏发送数据给单片机 . . . 228
2.5.2.1 定义发送的数据格式 . . . 228
2.5.2.2 下发数值格式的数据 . . . 228
2.5.2.3 下发字符串格式的数据 . . . 230
2.5.2.4 《串口屏发送数据给单片机例程》下载 . . . 232
2.5.2.5 单片机如何解析串口屏发出的数据 . . . 232
2.6 串口助手软件 (sscom) 和屏幕联调 . . . 233
2.6.1 sscom 和屏幕联调-样例工程下载 . . . 239
2.7 串口助手软件 (sscom) 和串口屏模拟器联调 1 . . . 240
2.8 串口助手软件 (sscom) 和串口屏模拟器联调 2 . . . 247
2.9 其他串口助手 . . . 251
2.9.1 串口助手软件 (uartassist) 和屏幕联调 . . . 251
2.9.2 串口助手软件 (xcom) 和屏幕联调 . . . 256
2.9.3 串口助手软件 (llcom) 和屏幕联调 . . . 262
2.10 与 stm32 单片机联调 . . . 267
2.10.1 stm32 说明 . . . 267
2.10.1.1 stm32 通讯-如何下载工程到串口屏 . . . 268
2.10.1.2 stm32 单片机与串口屏的接法 . . . 270
2.10.1.3 单片机与模拟器如何连接 . . . 271
2.10.2 stm32 HAL 库开发 . . . 275
2.10.2.1 使用 stm32cubemx 创建工程 . . . 275
2.10.2.2 使用 stm32cubeide(HAL 库) 开发 . . . 279
2.10.2.2.1 为什么要在 begin 和 end 之间插入代码 . . . 292
2.10.2.3 使用 keil5（MDK）开发 . . . 293
2.10.2.4 下载《stm32 与串口屏双向通讯》例程 . . . 300
2.11 与 TI（德州仪器）单片机联调 . . . 301
2.11.1 与 TI（德州仪器）单片机联调准备工作 . . . 301
2.11.1.1 TI（德州仪器）单片机-如何下载工程到串口屏 . . . 301
2.11.1.2 TI（德州仪器）单片机与串口屏的接法 . . . 303
2.11.1.3 相关软件安装和基础教程学习 . . . 304
2.11.2 与 TI（德州仪器）单片机联调 2 . . . 305
2.11.3 与 TI（德州仪器）单片机联调可能出现的问题 . . . 313
2.11.3.1 下载后屏幕没有变化 . . . 313
2.11.3.2 缺少 ti_msp_dl_config.c 和 ti_msp_dl_config.h . . . 313
2.11.4 TI（德州仪器）工程下载 . . . 315
2.12 与 arduino 联调 . . . 316
2.12.1 arduino uno 与串口屏通讯 . . . 316
2.12.1.1 arduino uno 与串口屏通讯工程下载 . . . 316
2.12.1.2 arduino uno 与串口屏的连接方式 . . . 318
2.12.1.3 arduino uno 代码 . . . 319
2.12.2 arduino mega2560 与串口屏通讯 . . . 322
2.12.2.1 arduino mega2560 与串口屏通讯工程下载 . . . 322
2.12.2.2 arduino mega2560 与串口屏的连接方式 . . . 324
2.12.2.3 arduino mega2560 代码 . . . 325

2.12.3 米思齐 Mixly arduino mega2560 与串口屏通讯 . . . 328
2.12.3.1 米思齐 Mixly arduino mega2560 与串口屏通讯工程下载 . . . 328
2.12.3.2 米思齐 Mixlyarduino mega2560 与串口屏的连接方式 . . . 330
2.12.3.3 米思齐 Mixly arduino mega2560 代码 . . . 331
2.12.4 ESP32C3 与串口屏通讯 . . . 333
2.12.4.1 ESP32C3 与串口屏通讯工程下载 . . . 333
2.12.4.2 ESP32C3 与串口屏的连接方式 . . . 335
2.12.4.3 ESP32C3 代码 . . . 336
2.12.5 ESP32S3(源地) 与串口屏通讯 . . . 339
2.12.5.1 ESP32S3(源地) 与串口屏通讯工程下载 . . . 339
2.12.5.2 ESP32S3(源地) 与串口屏的连接方式 . . . 341
2.12.5.3 ESP32S3(源地) 代码 . . . 342
2.12.6 ESP32S3(合宙) 与串口屏通讯 . . . 345
2.12.6.1 ESP32S3(合宙) 与串口屏通讯工程下载 . . . 345
2.12.6.2 ESP32S3(合宙) 与串口屏的连接方式 . . . 347
2.12.6.3 ESP32S3(合宙) 代码 . . . 348
2.12.7 ESP32S3(嘉立创) 与串口屏通讯 . . . 351
2.12.7.1 ESP32S3(嘉立创) 与串口屏通讯工程下载 . . . 351
2.12.7.2 ESP32S3(嘉立创) 与串口屏的连接方式 . . . 353
2.12.7.3 ESP32S3(嘉立创) 代码 . . . 354
2.12.8 树莓派 pico 使用 arduino 与串口屏通讯 . . . 357
2.12.8.1 树莓派 pico 与串口屏通讯工程下载 . . . 357
2.12.8.2 树莓派 pico 使用 arduino 与串口屏通讯的连接方式 . . . 359
2.12.8.3 树莓派 pico 使用 arduino 与串口屏通讯代码 . . . 360
2.12.9 arduino 通讯演示工程下载 . . . 363
2.13 与 MicroPython 联调 . . . 364
2.13.1 树莓派 pico 使用 MicroPython 与串口屏通讯 . . . 364
2.13.1.1 树莓派 pico 使用 MicroPython 与串口屏通讯工程下载 . . . 364
2.13.1.2 树莓派 pico 使用 MicroPython 与串口屏通讯的连接方式 . . . 366
2.13.1.3 树莓派 pico 使用 MicroPython 与串口屏通讯代码 . . . 367
2.13.2 MicroPython 通讯演示工程下载 . . . 370
2.14 与 python 联调 . . . 371
2.14.1 python 发送数据给屏幕 . . . 371
2.14.1.1 python 发送数据给屏幕工程下载 . . . 371
2.14.1.2 python 代码 . . . 371
2.14.2 python 获取电脑状态 . . . 373
2.14.2.1 python 获取电脑状态工程下载 . . . 373
2.14.2.2 python 获取电脑状态代码 . . . 373
2.14.3 python 获取奥运奖牌 . . . 376
2.14.3.1 python 获取奥运奖牌通讯工程下载 . . . 376
2.14.3.2 python 获取奥运奖牌代码 . . . 376
2.14.4 python 通讯演示工程下载 . . . 378

3 所有控件详解 381
3.1 认识控件 . . . 381
3.1.1 控件是什么 . . . 381
3.1.2 控件种类有多少 . . . 383
3.1.3 控件事件是什么 . . . 384
3.1.4 控件属性解析 . . . 385
3.1.5 控件属性读写 . . . 388
3.1.6 控件属性详解 . . . 389
3.1.6.1 type 属性 . . . 389
3.1.6.2 id 属性 . . . 389
3.1.6.3 objname 属性 . . . 389
3.1.6.4 vscope 属性 . . . 389
3.1.6.5 txt 属性 . . . 390
3.1.6.6 val 属性 . . . 390
3.1.6.7 font 属性 . . . 390

3.1.7 控件属性-相关链接 . . . 390
3.1.8 控件属性-控件 id 对照表 . . . 391
3.2 页面控件 . . . 392
3.2.1 如何新建一个页面 . . . 392
3.2.2 前初始化事件 . . . 395
3.2.3 后初始化事件 . . . 396
3.2.4 页面按下事件 . . . 396
3.2.5 页面弹起事件 . . . 396
3.2.6 页面离开事件 . . . 396
3.2.7 页面控件-使用详解 . . . 397
3.2.7.1 设置页面的背景图片 . . . 397
3.2.7.2 切换其他页面的背景图片 . . . 402
3.2.7.3 设置页面滑动翻页 . . . 403
3.2.8 页面控件-样例工程下载 . . . 404
3.2.9 页面控件-相关链接 . . . 404
3.2.10 页面控件-属性详解 . . . 404
3.3 文本控件 . . . 405
3.3.1 文本控件-使用详解 . . . 405
3.3.1.1 给文本控件配置键盘 . . . 405
3.3.1.2 文本控件如何跨页面使用 . . . 406
3.3.1.3 文本和数字之间相互转换 . . . 406
3.3.1.4 在文本控件后面追加文本 . . . 406
3.3.1.5 删除文本控件后面的几个字符 . . . 406
3.3.1.6 删除文本控件前面几个字符 . . . 407
3.3.1.7 清空文本控件 . . . 407
3.3.1.8 修改文本控件显示文字的大小和样式 . . . 407
3.3.1.9 通过名称组的方式来设置 t0 控件的属性 . . . 408
3.3.1.10 批量通过名称组的方式读取掉电存储区的数据来设置文本控件的属性 . . . 408
3.3.1.11 如何实现文本输入和密码显示星号 . . . 408
3.3.1.12 单片机如何对文本控件赋值 . . . 408
3.3.1.13 文本控件-如何换行 . . . 408
3.3.1.14 文本数据-显示不完全 . . . 409
3.3.1.15 单片机赋值文本控件，屏幕返回 1C FF FF FF。 . . . 409
3.3.2 文本控件-c 语言示例 . . . 409
3.3.3 文本控件-样例工程下载 . . . 410
3.3.4 文本控件-相关链接 . . . 410
3.3.5 文本控件-属性详解 . . . 410
3.4 滚动文本控件 . . . 412
3.4.1 滚动文本控件-使用详解 . . . 412
3.4.1.1 给滚动文本控件配置键盘 . . . 412
3.4.1.2 修改滚动文本控件显示文字的大小 . . . 413
3.4.1.3 滚动文本控件显示文本信息 . . . 413
3.4.1.4 通过名称组的方式来设置 g0 控件的属性 . . . 413
3.4.1.5 批量通过名称组的方式来设置 g0 控件的属性 . . . 414
3.4.1.6 修改滚动文本的滚动方向 . . . 414
3.4.1.7 让滚动文本停止滚动 . . . 414
3.4.2 滚动文本控件-c 语言示例 . . . 414
3.4.3 滚动文本控件-样例工程下载 . . . 414
3.4.4 滚动文本控件-相关链接 . . . 414
3.4.5 滚动文本控件-属性详解 . . . 415
3.5 数字控件 . . . 417
3.5.1 数字控件-使用详解 . . . 417
3.5.1.1 给数字控件配置键盘 . . . 417
3.5.1.2 数字控件赋值 . . . 418
3.5.1.3 数字和文本之间相互转换 . . . 418
3.5.1.4 数字控件如何跨页面使用 . . . 418
3.5.1.5 修改数字控件显示文字的大小和样式 . . . 419
3.5.1.6 通过名称组的方式来设置 n0 控件的属性 . . . 419

3.5.1.7 批量通过名称组的方式来设置数字控件的属性 . . . 419
3.5.1.8 使用数字控件显示 RTC 时钟数据 . . . 420
3.5.1.9 屏幕如何显示小数 . . . 420
3.5.1.10 单片机如何对数字控件赋值 . . . 421
3.5.2 数字控件-c 语言示例 . . . 421
3.5.3 数字控件-样例工程下载 . . . 421
3.5.4 数字控件-相关链接 . . . 421
3.5.5 数字控件-属性详解 . . . 421
3.6 虚拟浮点数控件 . . . 423
3.6.1 虚拟浮点数控件-使用详解 . . . 423
3.6.1.1 给虚拟浮点数控件配置键盘 . . . 423
3.6.1.2 虚拟浮点数控件赋值 . . . 424
3.6.1.3 虚拟浮点数控件赋值举例 . . . 424
3.6.1.4 修改虚拟浮点数控件显示文字的大小和样式 . . . 426
3.6.1.5 通过名称组的方式来设置 x0 控件的属性 . . . 427
3.6.1.6 批量通过名称组的方式来设置虚拟浮点数控件的属性 . . . 427
3.6.1.7 串口屏-如何显示小数 . . . 427
3.6.1.7.1 使用虚拟浮点数控件显示小数 . . . 427
3.6.1.7.2 使用文本控件显示小数 . . . 427
3.6.2 浮点数控件-c 语言示例 . . . 427
3.6.3 虚拟浮点数计算问题 . . . 428
3.6.3.1 虚拟浮点数加法 . . . 428
3.6.3.2 虚拟浮点数减法 . . . 428
3.6.3.3 虚拟浮点数乘法 . . . 428
3.6.3.4 虚拟浮点数除法 . . . 428
3.6.3.5 浮点数发送给单片机 . . . 429
3.6.4 虚拟浮点数控件-样例工程下载 . . . 429
3.6.5 虚拟浮点数控件-相关链接 . . . 429
3.6.6 虚拟浮点数控件-属性详解 . . . 429
3.7 按钮控件 . . . 431
3.7.1 按钮控件-使用详解 . . . 431
3.7.1.1 通过按钮控件跳转页面 . . . 431
3.7.1.2 修改按钮控件显示文字的大小和样式 . . . 432
3.7.1.3 让按钮在开始和停止之间切换 . . . 433
3.7.1.4 按钮禁用触摸功能 . . . 434
3.7.1.5 通过按钮控件发出串口数据 . . . 435
3.7.1.6 使用按钮实现长按功能 . . . 435
3.7.1.7 使用按钮实现双击功能 . . . 435
3.7.1.8 使用按钮在多个状态之间切换 . . . 436
3.7.1.9 使用按钮在多个状态之间切换 2 . . . 436
3.7.2 按钮控件-样例工程下载 . . . 436
3.7.3 按钮控件-相关链接 . . . 437
3.7.4 按钮控件-属性详解 . . . 437
3.8 进度条控件 . . . 439
3.8.1 进度条控件-使用详解 . . . 439
3.8.1.1 搭配定时器控件实现进度条自动增长并在进度为 100 时跳转页面 . . . 439
3.8.1.2 使用定时器控件实现进度条和数值变量之间同步 . . . 439
3.8.1.3 让进度条从右往左 (反方向) . . . 439
3.8.2 进度条控件-c 语言示例 . . . 440
3.8.3 进度条控件-样例工程下载 . . . 440
3.8.4 进度条控件-相关链接 . . . 440
3.8.5 进度条控件-属性详解 . . . 440
3.9 图片控件 . . . 442
3.9.1 图片控件-使用详解 . . . 442
3.9.1.1 修改图片控件显示不同的图片 . . . 442
3.9.2 图片控件-c 语言示例 . . . 442
3.9.3 图片控件-样例工程下载 . . . 442
3.9.4 图片控件-相关链接 . . . 443

3.9.5 图片控件-属性详解 . . . 443
3.10 切图控件 . . . 444
3.10.1 切图控件-使用详解 . . . 445
3.10.1.1 修改切图控件显示不同的图片 . . . 445
3.10.1.2 使用切图实现透明效果 . . . 446
3.10.2 切图控件-样例工程下载 . . . 446
3.10.3 切图控件-相关链接 . . . 446
3.10.4 切图控件-属性详解 . . . 446
3.11 触摸热区控件 . . . 448
3.11.1 触摸热区控件-使用详解 . . . 448
3.11.1.1 通过指令触发触摸热区控件，实现代码的重复调用 . . . 448
3.11.1.2 实现一个隐藏按钮功能，当点击的次数足够多时，进入后台页面 . . . 448
3.11.2 触摸热区控件-样例工程下载 . . . 449
3.11.3 触摸热区控件-相关链接 . . . 450
3.11.4 触摸热区控件-属性详解 . . . 450
3.12 触摸捕捉控件 . . . 451
3.12.1 触摸捕捉控件-使用详解 . . . 452
3.12.1.1 触摸捕捉控件触发顺序 . . . 452
3.12.1.2 触摸捕捉控件实现双态按钮之间的互斥（双态按钮 bt0-bt5 的 id 号是连续的） . . . 452
3.12.1.3 触摸捕捉控件实现屏保功能 . . . 453
3.12.1.4 触摸捕捉控件实现蜂鸣器播放 . . . 453
3.12.1.4.1 点击任意位置都让蜂鸣器鸣叫 . . . 453
3.12.1.4.2 点击有控件的位置让蜂鸣器鸣叫, 点击空白位置不鸣叫 . . . 453
3.12.1.4.3 只有点击了按钮控件才会鸣叫, 点击其他控件不会鸣叫 . . . 453
3.12.2 触摸捕捉控件-样例工程下载 . . . 454
3.12.3 触摸捕捉控件-相关链接 . . . 454
3.12.4 触摸捕捉控件-属性详解 . . . 454
3.13 指针控件 . . . 455
3.13.1 指针控件-使用详解 . . . 455
3.13.1.1 控制指针控件的转动角度 . . . 455
3.13.1.2 跨页面给指针控件赋值 . . . 455
3.13.2 指针控件-c 语言示例 . . . 456
3.13.3 指针控件-样例工程下载 . . . 457
3.13.4 指针控件-相关链接 . . . 457
3.13.5 指针控件-属性详解 . . . 457
3.14 曲线波形控件 . . . 459
3.14.1 曲线波形控件-使用详解 . . . 459
3.14.1.1 通过指令添加曲线波形控件 s0 数据 . . . 459
3.14.1.2 曲线控件添加的纵坐标值范围是多少 . . . 459
3.14.1.3 曲线控件如何表示负数 . . . 461
3.14.2 曲线波形控件-常见问题 . . . 464
3.14.2.1 使用 add 指令时屏幕没有任何反应 . . . 464
3.14.3 曲线波形控件-样例工程下载 . . . 464
3.14.4 曲线波形控件-相关链接 . . . 464
3.14.5 曲线波形控件-属性详解 . . . 464
3.15 滑块控件 . . . 466
3.15.1 滑块控件-使用详解 . . . 466
3.15.1.1 滑块控件修改亮度 . . . 466
3.15.1.2 滑块控件修改音量大小 . . . 468
3.15.1.3 发送滑块的数据 . . . 468
3.15.2 滑块控件-注意事项 . . . 469
3.15.3 滑块控件-样例工程下载 . . . 469
3.15.4 滑块控件-相关链接 . . . 469
3.15.5 滑块控件-属性详解 . . . 469
3.16 定时器控件 . . . 471
3.16.1 定时器控件-使用详解 . . . 472
3.16.1.1 关闭定时器 . . . 472
3.16.1.2 打开定时器 . . . 472

3.16.1.3 定时器最大时间只能设置为 65534(65 秒多), 怎么计时更长时间 . . . 472
3.16.1.4 通过定时器定时刷新 RTC 到数字控件上 . . . 473
3.16.1.5 两秒后跳转到其他页面 . . . 473
3.16.1.6 让 n0 控件每隔 1 秒自动增加 1 . . . 474
3.16.1.7 使用定时器配合图片控件循环播放图片 . . . 475
3.16.1.8 每个页面限制 12 个定时器，不够用怎么办 . . . 475
3.16.1.9 同一页面两个时间相同的定时器，谁先执行 . . . 476
3.16.2 定时器控件-样例工程下载 . . . 476
3.16.3 定时器控件-相关链接 . . . 476
3.16.4 定时器控件-属性详解 . . . 476
3.17 变量控件 . . . 478
3.17.1 变量控件-样例工程下载 . . . 479
3.17.2 变量控件-相关链接 . . . 479
3.17.3 变量控件-属性详解 . . . 479
3.18 双态按钮控件 . . . 480
3.18.1 双态按钮控件-使用详解 . . . 480
3.18.1.1 修改双态按钮控件显示文字的大小和样式 . . . 480
3.18.1.2 让双态按钮在开始和停止之间切换 . . . 480
3.18.1.3 双态按钮实现按键锁定功能 . . . 481
3.18.1.4 为什么按下的状态会把弹起的事件也执行了 . . . 481
3.18.2 双态按钮-样例工程下载 . . . 481
3.18.3 双态按钮-相关链接 . . . 481
3.18.4 双态按钮控件-属性详解 . . . 482
3.19 复选框控件 . . . 484
3.19.1 复选框控件-使用详解 . . . 484
3.19.1.1 在按钮控件中判断复选框是否选中 . . . 484
3.19.2 复选框控件-样例工程下载 . . . 485
3.19.3 复选框控件-相关链接 . . . 485
3.19.4 复选框控件-属性详解 . . . 485
3.20 单选框控件 . . . 487
3.20.1 单选框控件-使用详解 . . . 487
3.20.1.1 1、单选框互斥效果 . . . 487
3.20.2 单选框控件-样例工程下载 . . . 487
3.20.3 单选框控件-相关链接 . . . 487
3.20.4 单选框控件-属性详解 . . . 488
3.21 二维码控件 . . . 489
3.21.1 二维码控件-使用详解 . . . 489
3.21.1.1 修改二维码显示内容 . . . 489
3.21.1.2 显示微信二维码或者公众号二维码 . . . 489
3.21.2 二维码控件-c 语言示例 . . . 489
3.21.3 二维码控件-样例工程下载 . . . 490
3.21.4 二维码控件-相关链接 . . . 490
3.21.5 二维码控件-属性详解 . . . 490
3.22 状态开关控件 . . . 491
3.22.1 状态开关控件-使用详解 . . . 492
3.22.1.1 修改状态开关的显示文本为打开/关闭 . . . 492
3.22.1.2 仅显示按钮，不显示文本 . . . 492
3.22.1.3 如何修改状态开关控件显示的字体大小和样式 . . . 493
3.22.2 状态开关控件-样例工程下载 . . . 494
3.22.3 状态开关控件-相关链接 . . . 494
3.22.4 状态开关控件-属性详解 . . . 494
3.23 下拉框控件 . . . 496
3.23.1 下拉框控件-使用详解 . . . 496
3.23.1.1 下拉框控件赋值 . . . 496
3.23.1.2 将 t0.txt 动态添加到下拉框控件 . . . 496
3.23.1.3 删除下拉框中指定的某一行数据 . . . 496
3.23.1.4 让下拉框缩回 . . . 497
3.23.1.5 获取下拉框被当前被选中的值 . . . 497

3.23.1.6 发送下拉框被当前被选中的值 . . . 497
3.23.1.7 将下拉框置顶 . . . 497
3.23.1.8 当点击某个下拉框时让其他下拉框缩回 . . . 498
3.23.1.9 下拉框被其他下拉框挡住 . . . 498
3.23.1.10 如何修改下拉框控件显示的字体大小和样式 . . . 498
3.23.2 下拉框控件-样例工程下载 . . . 498
3.23.3 下拉框控件-相关链接 . . . 498
3.23.4 下拉框控件-属性详解 . . . 498
3.24 选择文本控件 . . . 501
3.24.1 选择文本控件-使用详解 . . . 501
3.24.1.1 将 t0.txt 动态添加到选择文本控件 . . . 501
3.24.1.2 删除选择文本中指定的某一行数据 . . . 501
3.24.1.3 在弹起事件里面获取到的 val/txt 属性和显示的值不符 . . . 502
3.24.1.4 获取选择文本被当前被选中的值 . . . 502
3.24.1.5 如何修改选择文本控件显示的字体大小和样式 . . . 502
3.24.2 选择文本控件-样例工程下载 . . . 502
3.24.3 选择文本控件-相关链接 . . . 502
3.24.4 选择文本控件-属性详解 . . . 502
3.25 滑动文本控件 . . . 505
3.25.1 滑动文本控件-使用详解 . . . 505
3.25.1.1 给滑动文本控件配置键盘 . . . 505
3.25.1.2 给滑动文本控件添加新行 . . . 506
3.25.1.3 让滑动文本控件每次都位于最新行 . . . 506
3.25.1.4 滑动文本控件翻页 . . . 506
3.25.1.5 如何修改滑动文本控件显示的字体大小和样式 . . . 506
3.25.2 滑动文本控件-c 语言示例 . . . 506
3.25.3 滑动文本控件-样例工程下载 . . . 506
3.25.4 滑动文本控件-相关链接 . . . 507
3.25.5 滑动文本控件-属性详解 . . . 507
3.26 数据记录控件 . . . 509
3.26.1 数据记录控件-使用详解 . . . 510
3.26.1.1 数据记录控件有哪些方法 . . . 510
3.26.1.2 数据记录控件插入一条数据 . . . 511
3.26.1.3 数据记录控件插入数字控件 . . . 512
3.26.1.4 数据记录控件插入 rtc 时间信息 . . . 513
3.26.1.5 数据记录控件删除选中的数据 . . . 515
3.26.1.6 数据记录控件清空数据 . . . 515
3.26.1.7 数据记录控件更新被选中的数据 . . . 515
3.26.1.8 数据记录控件修改某一行的背景色和字体色 . . . 515
3.26.1.9 数据记录翻页功能实现 . . . 515
3.26.1.10 数据记录控件回到顶部, 回到底部, 回到中间 . . . 516
3.26.1.11 获取数据记录某一行中某一个格 . . . 516
3.26.1.12 数据记录超过 12 列 . . . 516
3.26.1.13 数据记录控件由 3 列修改为 4 列或更多列 . . . 516
3.26.1.14 如何修改数据记录控件显示的字体大小和样式 . . . 516
3.26.1.15 隐藏数据记录控件表头 . . . 516
3.26.2 数据记录控件-常见问题 . . . 517
3.26.2.1 数据记录控件变黑报错，黑屏 . . . 517
3.26.2.1.1 File configuration data does not match Component configuration data. It is recommended to delete this file. The system will recreate the correct data file:sd0/xxx.data . . . 517
3.26.2.1.2 提示 file lost . . . 519
3.26.2.1.3 多个数据记录打开同一个路径下文件导致黑屏 . . . 519
3.26.2.1.4 数据记录 path 属性赋值注意事项 . . . 519
3.26.2.1.5 数据记录超过 8 个的解决方法 . . . 520
3.26.2.2 将数据记录控件保存在内存中 . . . 520
3.26.3 数据记录控件-样例工程下载 . . . 521
3.26.4 数据记录控件-相关链接 . . . 522

3.26.5 数据记录控件-属性详解 . . . 522
3.27 文件浏览器控件 . . . 524
3.27.1 文件浏览器控件-使用详解 . . . 524
3.27.1.1 判断当前被选中文件的类型 . . . 524
3.27.1.2 获取当前被选中文件的完整路径 . . . 524
3.27.1.3 如何修改文件浏览器控件显示的字体大小和样式 . . . 525
3.27.1.4 文件浏览器控件有哪些方法 . . . 525
3.27.2 文件浏览器控件-常见问题 . . . 526
3.27.2.1 1. 文件浏览器中文件名称显示不正常（乱码） . . . 526
3.27.3 文件浏览器控件-样例工程下载 . . . 526
3.27.4 文件浏览器控件-相关链接 . . . 526
3.27.5 文件浏览器控件-属性详解 . . . 526
3.27.6 文件浏览器控件-方法详解 . . . 528
3.28 文件流控件 . . . 529
3.28.1 文件流控件-使用详解 . . . 530
3.28.1.1 文件流关键属性 . . . 530
3.28.1.1.1 val-文件流当前数据指针 . . . 530
3.28.1.1.2 qty-文件大小 . . . 530
3.28.1.1.3 en-文件打开状态 . . . 530
3.28.1.2 文件流控件有哪些方法 . . . 531
3.28.1.2.1 open-打开文件 . . . 531
3.28.1.2.2 read-从文件流读取数据 . . . 532
3.28.1.2.3 文件流读取-示例 . . . 532
3.28.1.2.4 write-将数据写入文件流 . . . 532
3.28.1.2.5 文件流写入-示例 . . . 533
3.28.1.2.6 close-关闭文件流 . . . 533
3.28.1.2.7 文件流关闭-示例 . . . 533
3.28.1.2.8 find-按关键字查询并定位文件流指针 . . . 533
3.28.1.2.9 文件流查找-示例 . . . 533
3.28.2 文件流控件-常见问题 . . . 534
3.28.2.1 在电脑模拟器上是正常打开的，但是下载到串口屏后总是提示打开失败 . . . 534
3.28.2.2 在电脑模拟器上读取到的数据和串口屏上的数据不一致 . . . 534
3.28.2.3 文件流控件如何跨页面调用同一个文件 . . . 534
3.28.3 文件流控件-样例工程下载 . . . 534
3.28.4 文件流控件-相关链接 . . . 535
3.28.5 文件流控件-属性详解 . . . 535
3.28.6 文件流控件-方法详解 . . . 535
3.29 动画控件 . . . 536
3.29.1 动画控件-使用详解 . . . 536
3.29.1.1 开机动画制作演示 . . . 536
3.29.1.2 在动画播放完成后跳转页面 . . . 547
3.29.2 动画控件-样例工程下载 . . . 548
3.29.3 动画控件-相关链接 . . . 548
3.29.4 动画控件-属性详解 . . . 548
3.30 视频控件 . . . 549
3.30.1 视频控件-使用详解 . . . 549
3.30.2 视频控件-c 语言示例 . . . 556
3.30.3 视频控件-常见问题 . . . 556
3.30.3.1 v0.vid 初始值无效: 视频 ID 无效 . . . 556
3.30.3.2 视频控件-视频无法播放 . . . 557
3.30.4 视频控件-样例工程下载 . . . 557
3.30.5 视频控件-相关链接 . . . 557
3.30.6 视频控件-属性详解 . . . 557
3.31 音频控件 . . . 559
3.31.1 音频控件-使用详解 . . . 559
3.31.1.1 打开或关闭音频 . . . 565
3.31.1.2 设置音频是否循环播放 . . . 565
3.31.1.3 音频控件和 play 指令区别 . . . 565

3.31.1.4 如何使用外部音频播放 . . . 566
3.31.2 音频控件-常见问题 . . . 566
3.31.2.1 wav0.vid 初始值无效: 音频 ID 无效 . . . 566
3.31.2.2 屏幕使用外部音频播放无声音 . . . 566
3.31.3 音频控件-样例工程下载 . . . 566
3.31.4 音频控件-相关链接 . . . 567
3.31.5 音频控件-属性详解 . . . 567
3.32 外部图片控件 . . . 568
3.32.1 外部图片控件-使用详解 . . . 568
3.32.2 外部图片控件-常见问题 . . . 570
3.32.2.1 外部图片显示不正常 . . . 570
3.32.2.2 外部图片有透明部分，但是 jpg 不支持透明 . . . 572
3.32.3 外部图片控件-样例工程下载 . . . 572
3.32.4 外部图片控件-相关链接 . . . 572
3.32.5 外部图片控件-属性详解 . . . 573
3.33 关于控件属性的说明 . . . 574

4 指令集 575
4.1 常用指令集 . . . 575
4.1.1 page-页面跳转指令 . . . 575
4.1.1.1 page-示例 1 . . . 576
4.1.1.2 page-示例 2 返回上一个页面 . . . 577
4.1.1.3 page-应用实例 . . . 577
4.1.1.4 page-c 语言示例 . . . 577
4.1.1.5 page 指令-相关链接 . . . 577
4.1.1.6 page 指令-样例工程下载 . . . 577
4.1.2 prints-从串口打印一个变量/常量 . . . 578
4.1.2.1 prints-示例 1 . . . 578
4.1.2.2 prints-示例 2 . . . 579
4.1.2.3 prints-示例 3 . . . 581
4.1.2.4 prints-示例 4 . . . 582
4.1.2.5 prints-示例 5 . . . 584
4.1.2.6 prints-示例 6 . . . 584
4.1.2.7 prints-示例 7 . . . 584
4.1.2.8 prints-示例 8 . . . 584
4.1.2.9 prints-示例 9 . . . 584
4.1.2.10 prints-发送日期时间 . . . 584
4.1.2.11 prints 指令-样例工程下载 . . . 585
4.1.3 printh-从串口打印 16 进制 . . . 586
4.1.3.1 printh-示例 1 . . . 586
4.1.3.2 printh-示例 2: 发送 3 个 0xff . . . 588
4.1.3.3 printh-示例 3 . . . 589
4.1.3.4 printh-示例 4 . . . 590
4.1.3.5 printh 指令-样例工程下载 . . . 591
4.1.4 click-激活控件的按下/弹起事件 . . . 592
4.1.4.1 click-示例 1 . . . 592
4.1.4.2 click-示例 2 . . . 593
4.1.4.3 click-c 语言示例 . . . 593
4.1.4.4 click 指令-相关链接 . . . 594
4.1.4.5 click 指令-样例工程下载 . . . 594
4.1.5 vis-隐藏/显示控件 . . . 595
4.1.5.1 vis-示例 1 . . . 595
4.1.5.2 vis-示例 2 . . . 596
4.1.5.3 vis-示例 3 . . . 596
4.1.5.4 vis-示例 4 . . . 597
4.1.5.5 vis-示例 5 . . . 597
4.1.5.6 vis-示例 6 . . . 598
4.1.5.7 vis 指令-相关链接 . . . 598

4.1.5.8 vis 指令-样例工程下载 . . . 598
4.1.6 tsw-控件触摸使能 . . . 599
4.1.6.1 tsw-示例 1 . . . 599
4.1.6.2 tsw-示例 2 . . . 600
4.1.6.3 tsw-示例 3 . . . 600
4.1.6.4 tsw-示例 4 . . . 601
4.1.6.5 tsw-示例 5 . . . 601
4.1.6.6 tsw-示例 6 . . . 602
4.1.6.7 tsw-使用详解 . . . 602
4.1.6.7.1 tsw-实现按键锁定功能 . . . 602
4.1.6.8 如何实现跨页面使能或失能控件 . . . 603
4.1.6.9 tsw 指令-样例工程下载 . . . 603
4.1.7 randset-随机数范围设置 . . . 604
4.1.7.1 randset-示例 1 . . . 604
4.1.7.2 randset-示例 2 . . . 604
4.1.7.3 randset 指令-样例工程下载 . . . 605
4.1.7.4 rand 指令-相关链接 . . . 605
4.1.8 covx-变量类型转换 . . . 606
4.1.8.1 covx-示例 1 . . . 606
4.1.8.2 covx-示例 2 . . . 607
4.1.8.3 covx-示例 3 . . . 608
4.1.8.4 covx-示例 4 . . . 608
4.1.8.5 covx-示例 5 . . . 609
4.1.8.6 covx 指令-相关链接 . . . 609
4.1.8.7 covx 指令-样例工程下载 . . . 609
4.1.9 strlen-字符串变量字符长度测试 . . . 610
4.1.9.1 strlen-示例 . . . 610
4.1.9.2 strlen 指令-样例工程下载 . . . 611
4.1.10 btlen-字符串变量字节长度测试 . . . 612
4.1.10.1 btlen-示例 1 . . . 612
4.1.10.2 btlen-示例 2 . . . 612
4.1.10.3 btlen-c 语言示例 . . . 613
4.1.10.4 btlen 指令-样例工程下载 . . . 613
4.1.11 substr-字符串截取 . . . 614
4.1.11.1 substr-示例 . . . 614
4.1.11.2 substr 指令-样例工程下载 . . . 615
4.1.12 spstr-字符串分割 . . . 616
4.1.12.1 spstr-示例 1 . . . 616
4.1.12.2 spstr-示例 2 . . . 616
4.1.12.3 spstr-示例: 截取换行符 . . . 617
4.1.12.4 spstr-示例: 获取名人名言左右两边的数据 . . . 618
4.1.12.5 spstr-示例: 截取冒号和句号之间的数据 . . . 618
4.1.12.6 spstr-示例: 分割浮点数 . . . 619
4.1.12.7 spstr-示例: 解析 json 格式 . . . 619
4.1.12.8 spstr 指令-样例工程下载 . . . 619
4.1.13 touch_j-触摸校准 . . . 620
4.1.13.1 touch_j-示例 . . . 620
4.1.13.2 touch_j 指令-样例工程下载 . . . 621
4.1.14 add-往曲线控件添加数据 . . . 622
4.1.14.1 add-示例 1 . . . 622
4.1.14.2 add-示例 2 . . . 623
4.1.14.3 add-示例 3 . . . 625
4.1.14.4 add-c 语言示例 . . . 626
4.1.14.5 add 指令-相关链接 . . . 626
4.1.14.6 add 指令-样例工程下载 . . . 626
4.1.15 addt-曲线数据透传指令 . . . 627
4.1.15.1 addt-示例 1 . . . 627
4.1.15.2 在调试窗口中使用 addt 指令 . . . 628

4.1.15.3 addt 指令-样例工程下载 . . . 630
4.1.15.4 addt-c 语言示例 . . . 630
4.1.15.5 addt 指令-相关链接 . . . 630
4.1.16 cle-清除曲线控件中的数据 . . . 631
4.1.16.1 cle-示例 1 . . . 631
4.1.16.2 cle-示例 2 . . . 632
4.1.16.3 cle 指令-样例工程下载 . . . 632
4.1.16.4 cle-c 语言示例 . . . 632
4.1.17 rest-复位串口屏 . . . 633
4.1.17.1 rest-示例 1 . . . 633
4.1.17.2 rest 指令-样例工程下载 . . . 633
4.2 高级指令 . . . 634
4.2.1 wepo-写入数据到掉电存储空间 . . . 634
4.2.1.1 wepo-示例 1 . . . 634
4.2.1.2 wepo-示例 2 . . . 635
4.2.1.3 wepo-示例 3 . . . 636
4.2.1.4 wepo-应用实例 . . . 636
4.2.1.5 wepo-问答测试 . . . 638
4.2.1.6 wepo 指令-样例工程下载 . . . 638
4.2.1.7 wepo 指令-相关链接 . . . 639
4.2.2 repo-从掉电存储空间读取数据 . . . 640
4.2.2.1 repo-示例 1 . . . 640
4.2.2.2 repo-示例 2 . . . 641
4.2.2.3 repo-应用实例 . . . 641
4.2.2.4 repo 指令-样例工程下载 . . . 642
4.2.2.5 repo 指令-相关链接 . . . 642
4.2.3 wept-通过串口透传数据到掉电存储空间 . . . 643
4.2.3.1 wept-示例 . . . 643
4.2.3.2 wept-c 语言示例 . . . 644
4.2.3.3 wept 指令-样例工程下载 . . . 645
4.2.3.4 wept 指令-相关链接 . . . 645
4.2.4 rept-从掉电存储空间读取数据并透传发送到串口 . . . 646
4.2.4.1 rept-示例 1 . . . 646
4.2.4.2 rept 指令-样例工程下载 . . . 647
4.2.4.3 rept 指令-相关链接 . . . 647
4.2.5 beep-蜂鸣器鸣叫 . . . 648
4.2.5.1 beep-示例 1 . . . 648
4.2.5.2 beep-示例 2 . . . 649
4.2.5.3 beep-c 语言示例 . . . 649
4.2.5.4 beep 指令-样例工程下载 . . . 649
4.2.6 cfgpio-扩展 IO 模式配置 . . . 650
4.2.6.1 cfgpio-示例 1: . . . 650
4.2.6.2 cfgpio-示例 2: . . . 651
4.2.6.3 cfgpio-示例 3: . . . 651
4.2.6.4 cfgpio-示例 4: . . . 652
4.2.6.5 cfgpio 指令-样例工程下载 . . . 652
4.2.6.6 cfgpio 指令-相关链接 . . . 653
4.2.7 setlayer-运行中改变控件图层顺序 . . . 654
4.2.7.1 setlayer-示例 1 . . . 654
4.2.7.2 setlayer-示例 2 . . . 654
4.2.7.3 setlayer-示例 3 . . . 655
4.2.7.4 setlayer 指令-样例工程下载 . . . 656
4.2.8 move-控件移动 . . . 657
4.2.8.1 move-示例 . . . 657
4.2.8.2 move 指令-样例工程下载 . . . 658
4.2.9 play-音频播放 . . . 659
4.2.9.1 play-示例 1 . . . 659
4.2.9.2 play-示例 2 . . . 660

4.2.9.3 play-c 语言示例 . . . 660
4.2.9.4 play 指令-样例工程下载 . . . 660
4.2.9.5 play 指令-相关链接 . . . 661
4.3 文件操作指令 . . . 662
4.3.1 twfile-单片机发送文件给串口屏 . . . 662
4.3.1.1 twfile-示例 1 . . . 662
4.3.1.2 twfile-示例 2 . . . 663
4.3.2 rdfile-串口屏发送文件给单片机 . . . 664
4.3.2.1 rdfile-示例 1 . . . 664
4.3.2.2 rdfile-示例 2 . . . 664
4.3.2.3 rdfile-示例 3 . . . 664
4.3.2.4 rdfile 指令-样例工程下载 . . . 665
4.3.3 newfile-创建文件 . . . 666
4.3.3.1 newfile-示例 1 . . . 666
4.3.3.2 newfile-示例 2 . . . 667
4.3.3.3 newfile-示例 3 . . . 667
4.3.3.4 newfile-应用实例 . . . 668
4.3.3.5 newfile 指令-样例工程下载 . . . 668
4.3.4 delfile-删除文件 . . . 669
4.3.4.1 delfile-示例 1 . . . 669
4.3.4.2 delfile-示例 2 . . . 670
4.3.4.3 delfile-示例 3 . . . 670
4.3.4.4 delfile 指令-样例工程下载 . . . 671
4.3.5 refile-重命名文件 . . . 672
4.3.5.1 refile-示例 1 . . . 672
4.3.5.2 refile-示例 2 . . . 672
4.3.5.3 refile-示例 3 . . . 673
4.3.5.4 refile 指令-样例工程下载 . . . 674
4.3.6 findfile-查找文件 . . . 675
4.3.6.1 findfile-示例 1 . . . 675
4.3.6.2 findfile-示例 2 . . . 675
4.3.6.3 findfile-示例 3 . . . 676
4.3.6.4 findfile-应用实例 . . . 677
4.3.6.5 findfile 指令-样例工程下载 . . . 677
4.3.7 newdir-创建文件夹 . . . 678
4.3.7.1 newdir-示例 1 . . . 678
4.3.7.2 newdir-示例 2 . . . 679
4.3.7.3 newdir-示例 3 . . . 679
4.3.7.4 newdir 指令-样例工程下载 . . . 680
4.3.8 deldir-删除文件夹 . . . 681
4.3.8.1 deldir-示例 1 . . . 681
4.3.8.2 deldir-示例 2 . . . 681
4.3.8.3 deldir-示例 3 . . . 682
4.3.8.4 deldir 指令-样例工程下载 . . . 683
4.3.9 redir-重命名文件夹 . . . 684
4.3.9.1 redir-示例 1 . . . 684
4.3.9.2 redir-示例 2 . . . 684
4.3.9.3 redir 指令-样例工程下载 . . . 685
4.3.10 finddir-查找文件夹 . . . 686
4.3.10.1 finddir-示例 1 . . . 686
4.3.10.2 finddir-示例 2 . . . 686
4.3.10.3 finddir-示例 3 . . . 687
4.3.10.4 finddir 指令-样例工程下载 . . . 688
4.4 不常用指令 . . . 689
4.4.1 doevents-强制刷新屏幕 . . . 689
4.4.1.1 doevents-示例 . . . 689
4.4.1.2 doevents 指令-相关链接 . . . 689
4.4.1.3 doevents 指令-样例工程下载 . . . 690

4.4.2 sendme-发送当前页面 ID 号到串口 . . . 691
4.4.2.1 sendme-示例 . . . 691
4.4.2.2 sendme 指令-样例工程下载 . . . 691
4.4.3 get-带格式获取变量值/常量值 . . . 692
4.4.3.1 get-示例 1 . . . 692
4.4.3.2 get-示例 2 . . . 693
4.4.3.3 get-示例 3 . . . 694
4.4.3.4 get-示例 4 . . . 695
4.4.3.5 get-c 语言示例 . . . 696
4.4.3.6 get 指令-样例工程下载 . . . 697
4.4.4 ref-重绘控件 . . . 698
4.4.4.1 ref-示例 1 . . . 698
4.4.4.2 ref-示例 2 . . . 698
4.4.4.3 ref 指令-样例工程下载 . . . 699
4.4.5 ref_stop-暂停屏幕刷新 . . . 700
4.4.5.1 ref_stop-示例 1 . . . 700
4.4.5.2 ref_stop 指令-样例工程下载 . . . 700
4.4.6 ref_star-恢复屏幕刷新 . . . 701
4.4.6.1 ref_star-示例 1 . . . 701
4.4.6.2 ref_star 指令-样例工程下载 . . . 701
4.4.7 com_stop-暂停串口指令执行 . . . 702
4.4.7.1 com_stop-示例 . . . 702
4.4.7.2 com_stop-c 语言示例 . . . 702
4.4.7.3 com_stop 与 ref_stop 的区别 . . . 702
4.4.7.4 com_stop 指令-相关链接 . . . 703
4.4.7.5 com_stop 指令-样例工程下载 . . . 703
4.4.8 com_star-恢复串口指令执行 . . . 704
4.4.8.1 com_star-示例 . . . 704
4.4.8.2 com_star-c 语言示例 . . . 704
4.4.8.3 com_star 指令-相关链接 . . . 704
4.4.8.4 com_star 指令-样例工程下载 . . . 705
4.4.9 code_c-清空串口指令缓冲区 . . . 706
4.4.9.1 code_c-示例 . . . 706
4.4.9.2 code_c 指令-样例工程下载 . . . 706
4.4.9.3 code_c-c 语言示例 . . . 706
4.5 CRC 校验指令 . . . 707
4.5.1 crcrest-复位 crc 初始值 . . . 707
4.5.1.1 crcrest-示例 . . . 707
4.5.1.2 crcrest 指令-相关链接 . . . 708
4.5.1.3 crcrest 指令-样例工程下载 . . . 708
4.5.2 crcputs-crc 校验一个变量/常量 . . . 709
4.5.2.1 crcputs-示例 1 . . . 709
4.5.2.2 crcputs-示例 2 . . . 709
4.5.2.3 crcputs-示例 3 . . . 710
4.5.2.4 crcputs-示例 4 . . . 711
4.5.2.5 crcputs 指令-相关链接 . . . 711
4.5.2.6 crcputs 指令-样例工程下载 . . . 712
4.5.3 crcputh-crc 校验一组 Hex . . . 713
4.5.3.1 crcputh-示例 1 . . . 713
4.5.3.2 crcputh-示例 2 . . . 713
4.5.3.3 crcputh-示例 3 . . . 714
4.5.3.4 crcputh 指令-相关链接 . . . 715
4.5.3.5 crcputh 指令-样例工程下载 . . . 715
4.5.4 crcputu-crc 校验一段串口缓冲区数据 . . . 716
4.5.4.1 crcputu-示例 1 . . . 716
4.5.4.2 crcputu-示例 2 . . . 716
4.5.4.3 crcputu 指令-相关链接 . . . 717
4.5.4.4 crcputu 指令-样例工程下载 . . . 717

4.6 GUI 绘图指令 . . . 718
4.6.1 cls-清屏指令 . . . 718
4.6.1.1 cls-示例 1 . . . 718
4.6.1.2 cls-示例 2 . . . 718
4.6.1.3 cls-c 语言示例 . . . 719
4.6.1.4 cls 指令-相关链接 . . . 719
4.6.1.5 cls 指令-样例工程下载 . . . 719
4.6.2 pic-刷图指令 . . . 720
4.6.2.1 pic-示例 1 . . . 720
4.6.2.2 pic-示例 2 . . . 720
4.6.2.3 pic-c 语言示例 . . . 721
4.6.2.4 pic 指令-样例工程下载 . . . 721
4.6.3 picq-切图指令 . . . 722
4.6.3.1 picq-示例 1 . . . 722
4.6.3.2 picq-c 语言示例 . . . 723
4.6.3.3 picq 指令-样例工程下载 . . . 723
4.6.4 xpic-高级切图指令 . . . 724
4.6.4.1 xpic-示例 . . . 724
4.6.4.2 xpic-c 语言示例 . . . 725
4.6.4.3 xpic 指令-样例工程下载 . . . 725
4.6.5 xstr-写字指令 . . . 726
4.6.5.1 xstr-示例 . . . 726
4.6.5.2 xstr-c 语言示例 . . . 727
4.6.5.3 xstr 指令-样例工程下载 . . . 727
4.6.5.4 xstr 指令-相关链接 . . . 728
4.6.6 fill-区域填充指令 . . . 729
4.6.6.1 fill-示例 1 . . . 729
4.6.6.2 fill-示例 2 . . . 730
4.6.6.3 fill-c 语言示例 . . . 730
4.6.6.4 fill 指令-相关链接 . . . 730
4.6.6.5 fill 指令-样例工程下载 . . . 730
4.6.7 line-画线指令 . . . 731
4.6.7.1 line-示例 1 . . . 731
4.6.7.2 line-示例 2 . . . 732
4.6.7.3 line-c 语言示例 . . . 732
4.6.7.4 line 指令-样例工程下载 . . . 733
4.6.7.5 line 指令-相关链接 . . . 733
4.6.8 draw-画空心矩形 . . . 734
4.6.8.1 draw-示例 1 . . . 734
4.6.8.2 draw-示例 2 . . . 735
4.6.8.3 draw-c 语言示例 . . . 735
4.6.8.4 draw 指令-相关链接 . . . 735
4.6.8.5 draw 指令-样例工程下载 . . . 735
4.6.9 cir-画空心圆 . . . 736
4.6.9.1 cir-示例 1 . . . 736
4.6.9.2 cir-示例 2 . . . 737
4.6.9.3 cir-c 语言示例 . . . 737
4.6.9.4 cir 指令-相关链接 . . . 737
4.6.9.5 cir-演示工程下载链接: . . . 737
4.6.10 cirs-画实心圆 . . . 738
4.6.10.1 cirs-示例 1 . . . 738
4.6.10.2 cirs-示例 2 . . . 739
4.6.10.3 cirs-c 语言示例 . . . 739
4.6.10.4 cirs 指令-相关链接 . . . 739
4.6.10.5 cirs 指令-样例工程下载 . . . 739
4.6.10.6 cirs 指令-演示工程下载链接: . . . 739
4.7 HMI 颜色代号表 . . . 740
4.8 关于废弃指令的说明 . . . 740

5 系统变量 **741**
5.1 常用变量 . . . 741
5.1.1 sys0-sys2 默认变量 . . . 741
5.1.2 dp-当前页面 ID . . . 743
5.1.2.1 dp-示例 1，跳转页面 . . . 743
5.1.2.2 dp-示例 2，发送当面页面编号到串口 . . . 743
5.1.2.3 dp-示例 3，获取当前页面编号 . . . 744
5.1.2.4 dp-示例 4: 返回上一页 . . . 744
5.1.2.5 dp-示例 5: 刷新当前页面 . . . 747
5.1.2.6 dp-c 语言示例 . . . 747
5.1.2.7 dp-相关链接 . . . 747
5.1.2.8 dp-样例工程下载 . . . 748
5.1.3 volume-系统音量 . . . 749
5.1.3.1 volume-示例 1 . . . 749
5.1.3.2 volume-示例 2 . . . 749
5.1.3.3 volume-示例 3 . . . 750
5.1.3.4 volume-示例 4 . . . 751
5.1.3.5 volume-示例 5 . . . 752
5.1.3.6 volume-相关链接 . . . 753
5.1.3.7 volume-样例工程下载 . . . 753
5.1.4 dims-上电默认背光亮度值 . . . 754
5.1.4.1 dims-示例 1 . . . 754
5.1.4.2 dims-示例 2 . . . 755
5.1.4.3 dims-示例 3 . . . 755
5.1.4.4 dims-示例：通过滑块设置亮度 . . . 756
5.1.4.5 dims-c 语言示例 . . . 757
5.1.4.6 dims-相关链接 . . . 758
5.1.4.7 dims-样例工程下载 . . . 758
5.1.5 dim-当前背光亮度值 . . . 759
5.1.5.1 dim-示例 1 . . . 759
5.1.5.2 dim-示例 2 . . . 760
5.1.5.3 dim-示例 3 . . . 760
5.1.5.4 dim-示例：息屏，灭屏，设置 . . . 761
5.1.5.5 dim-示例：设置最高亮度 . . . 761
5.1.5.6 dim-示例：通过滑块设置亮度 . . . 762
5.1.5.7 dim-c 语言示例 . . . 763
5.1.5.8 dim-相关链接 . . . 764
5.1.5.9 dim-样例工程下载 . . . 764
5.1.6 bauds-上电默认波特率值 . . . 765
5.1.6.1 bauds-示例 1 . . . 765
5.1.6.2 bauds-示例 2 . . . 766
5.1.6.3 bauds-相关链接 . . . 767
5.1.7 baud-当前波特率值 . . . 768
5.1.7.1 baud-示例 1 . . . 768
5.1.7.2 baud-示例 2 . . . 769
5.1.7.3 baud-相关链接 . . . 770
5.1.8 ussp-无串口数据自动睡眠时间 . . . 771
5.1.8.1 ussp-示例 1 . . . 771
5.1.8.2 ussp-示例 2 . . . 771
5.1.8.3 ussp-c 语言示例 . . . 772
5.1.9 thsp-无触摸操作自动睡眠时间 . . . 773
5.1.9.1 thsp-示例 1 . . . 773
5.1.9.2 thsp-示例 2 . . . 773
5.1.9.3 thsp-c 语言示例 . . . 774
5.1.10 thup-睡眠模式下触摸自动唤醒开关 . . . 775
5.1.10.1 thup-示例 1 . . . 775
5.1.10.2 thup-示例 2 . . . 775
5.1.11 usup-睡眠模式下串口数据自动唤醒开关 . . . 777

5.1.11.1 usup-示例 1 . . . 777
5.1.11.2 usup-示例 2 . . . 777
5.1.12 wup-睡眠唤醒后刷新页面设置 . . . 779
5.1.12.1 wup-示例 1 . . . 779
5.1.12.2 wup-示例 2 . . . 779
5.1.12.3 wup-示例 3 . . . 780
5.1.13 sleep-睡眠 . . . 781
5.1.13.1 sleep 模式与关闭亮度的功耗区别 . . . 781
5.1.13.2 sleep-示例 1 . . . 781
5.1.13.3 sleep-c 语言示例 . . . 782
5.1.13.4 sleep-相关链接 . . . 782
5.1.13.5 sleep-样例工程下载 . . . 782
5.1.14 lowpower-睡眠模式设定 . . . 783
5.1.14.1 lowpower-示例 1 . . . 783
5.1.14.2 lowpower-示例 2 . . . 783
5.1.15 bkcmd-串口指令执行状态数据返回 . . . 785
5.1.15.1 bkcmd-示例 1 . . . 785
5.1.15.2 bkcmd-示例 2 . . . 786
5.1.15.3 bkcmd-示例 3 . . . 786
5.1.15.4 bkcmd-示例 4 . . . 787
5.1.15.5 bkcmd-相关链接 . . . 788
5.1.16 delay-延时 . . . 789
5.1.16.1 delay-示例 1 . . . 789
5.1.16.2 delay-示例 2 . . . 789
5.1.16.3 delay-样例工程下载 . . . 790
5.1.17 rand-随机数 . . . 791
5.1.17.1 rand-示例 . . . 791
5.1.17.2 rand-相关链接 . . . 792
5.1.17.3 rand-样例工程下载 . . . 792
5.1.18 crcval-crc 校验结果 . . . 793
5.1.18.1 crcval-示例 1 . . . 793
5.1.18.2 crcval-相关链接 . . . 793
5.1.18.3 crcval-样例工程下载 . . . 794
5.1.19 rtc0~rtc6-RTC 时钟变量 . . . 795
5.1.19.1 rtc-示例 1 . . . 795
5.1.19.2 rtc-示例 2 . . . 797
5.1.19.3 rtc-示例 3 . . . 797
5.1.19.4 rtc-c 语言示例 . . . 800
5.1.19.5 rtc-样例工程下载 . . . 801
5.1.20 pio0~pio7-扩展 IO 端口 . . . 802
5.1.20.1 pio-示例 . . . 802
5.1.20.2 pio-相关链接 . . . 803
5.1.20.3 pio-样例工程下载 . . . 803
5.1.21 pwm4~pwm7-扩展 IO 占空比 . . . 804
5.1.21.1 pwm-示例 . . . 804
5.1.21.2 pwm-相关链接 . . . 805
5.1.21.3 pwm-样例工程下载 . . . 805
5.1.22 pwmf-PWM 输出的频率 . . . 806
5.1.22.1 pwf-示例 . . . 806
5.1.22.2 pwmf-相关链接 . . . 807
5.1.22.3 pwmf-样例工程下载 . . . 807
5.1.23 audio0~audio1-音频通道控制 . . . 808
5.1.23.1 audio-示例 1 . . . 808
5.1.23.2 audio-示例 2 . . . 808
5.1.23.3 audio-示例 3 . . . 809
5.1.23.4 audio-相关链接 . . . 810
5.1.24 串口 HMI 颜色代号表 . . . 811
5.1.24.1 hmi_color-示例 1 . . . 811

5.2 不常用变量 . . . 812
5.2.1 scache-串口发送缓冲区使能 . . . 812
5.2.1.1 scache-打开串口发送缓冲区使能示例 . . . 812
5.2.2 thc-触摸绘图时的画笔色 . . . 814
5.2.2.1 thc-示例 1 . . . 814
5.2.2.2 thc-示例 2 . . . 814
5.2.3 thdra-触摸绘图功能开关 . . . 815
5.2.3.1 thdra-示例 1 . . . 815
5.2.3.2 thdra-示例 2 . . . 815
5.2.4 tch0~tch3-实时触摸坐标 . . . 817
5.2.4.1 tch0~tch3-样例工程下载 . . . 817
5.2.5 addr-设备地址 . . . 818
5.2.5.1 addr-示例 . . . 818
5.2.6 spax-字符显示横向间距 . . . 823
5.2.6.1 spax-示例 . . . 823
5.2.7 spay-字符显示纵向间距 . . . 824
5.2.7.1 spay-示例 . . . 824
5.2.8 sendxy-实时发送触摸坐标功能开关 . . . 825
5.2.8.1 sendxy-示例 1 . . . 825
5.2.8.2 sendxy-示例 2 . . . 825

6 书写语法 827
6.1 赋值操作 . . . 827
6.1.1 字符串属性赋值 . . . 827
6.1.2 数值属性赋值 . . . 831
6.1.3 字符串属性和数值属性相互转换 . . . 833
6.1.4 赋值操作-相关链接 . . . 835
6.1.5 赋值操作-样例工程下载 . . . 835
6.2 运算操作 . . . 836
6.2.1 数值类型变量运算操作 . . . 836
6.2.2 字符串类型变量运算操作 . . . 838
6.2.3 位运算 . . . 841
6.2.3.1 按位与 . . . 842
6.2.3.2 按位或 . . . 843
6.2.3.3 按位异或 . . . 843
6.2.3.4 按位取反 . . . 844
6.2.3.5 按位同或 . . . 844
6.2.3.6 按位左移 . . . 845
6.2.3.7 按位右移 . . . 845
6.2.4 运算操作-相关链接 . . . 846
6.3 跨页面赋值，全局变量操作 . . . 847
6.3.1 全局变量-常规使用 . . . 847
6.3.2 全局变量-特殊控件 . . . 850
6.3.3 不同页面的相同名称的控件是什么关系 . . . 850
6.3.4 全局变量-c 语言示例 . . . 851
6.3.5 全局变量-sscom 串口工具发送示例 . . . 851
6.3.6 全局变量-样例工程下载 . . . 853
6.4 HMI 逻辑语句 . . . 854
6.4.1 if 语句 . . . 854
6.4.1.1 if-示例 1: . . . 854
6.4.1.2 if-示例 2: . . . 855
6.4.1.3 if-示例 3: . . . 855
6.4.1.4 if-示例 4: . . . 856
6.4.1.5 if-示例 5: . . . 857
6.4.2 while 语句 . . . 859
6.4.2.1 while-示例 1: . . . 859
6.4.2.2 while-示例 2: . . . 859
6.4.2.3 while-示例 3: . . . 860

6.4.3 for 语句 . . . 862
6.4.3.1 for-示例 1: . . . 862
6.4.3.2 for-示例 2: . . . 862
6.4.3.3 for-示例 3: . . . 863
6.5 数组/名称组使用说明 . . . 865
6.5.1 什么是名称组, 名称组和数组什么关系 . . . 865
6.5.2 名称组的使用 . . . 865
6.5.3 怎么使用名称组 . . . 868
6.5.4 名称组应用-实现数组 . . . 871
6.5.5 名称组常见操作方式 . . . 871
6.5.5.1 批量读取掉电存储空间数据 . . . 871
6.5.5.2 批量隐藏控件 n0 ~ n99 . . . 872
6.5.5.3 批量清空文本控件 t0 ~ t5 . . . 872
6.5.5.4 批量将文本控件 t0 ~ t5 赋值为“淘晶驰” . . . 872
6.5.5.5 批量将数字控件 n0 ~ n5 赋值为 0 . . . 872
6.5.6 名称组使用-样例工程下载 . . . 873

7 串口屏返回数据格式 874
7.1 bkcmd 非 0 时的通知格式 . . . 874
7.1.1 00 FF FF FF 无效指令 . . . 874
7.1.2 01 FF FF FF 指令成功执行 . . . 875
7.1.3 02 FF FF FF 控件 ID 无效 . . . 876
7.1.4 03 FF FF FF 页面 ID 无效 . . . 877
7.1.5 04 FF FF FF 图片 ID 无效 . . . 878
7.1.6 05 FF FF FF 字库 ID 无效 . . . 879
7.1.7 06 FF FF FF 文件操作失败 . . . 882
7.1.8 09 FF FF FF CRC 校验失败 . . . 883
7.1.9 11 FF FF FF 波特率设置无效 . . . 884
7.1.10 12 FF FF FF 曲线控件 ID 号或通道号无效 . . . 885
7.1.11 1A FF FF FF 变量名称无效 . . . 886
7.1.12 1B FF FF FF 变量运算无效 . . . 887
7.1.13 1C FF FF FF 赋值操作失败 . . . 888
7.1.14 1D FF FF FF 掉电存储空间操作失败 . . . 889
7.1.14.1 掉电存储空间超出范围 . . . 889
7.1.14.2 赋值异常 . . . 889
7.1.15 1E FF FF FF 参数数量无效 . . . 890
7.1.16 1F FF FF FF IO 操作失败 . . . 891
7.1.17 20 FF FF FF 转义字符使用错误 . . . 892
7.1.18 23 FF FF FF 变量名称太长 . . . 895
7.2 其他数据返回格式 . . . 896
7.2.1 24 FF FF FF 串口缓冲区溢出 . . . 896
7.2.2 0X65 控件点击事件返回 . . . 897
7.2.3 0X66 当前页面的 ID 号返回 . . . 898
7.2.4 0X67 触摸坐标数据返回 . . . 899
7.2.5 0X68 睡眠模式触摸事件 . . . 900
7.2.6 0X70 字符串变量数据返回 . . . 901
7.2.7 0X71 数值变量数据返回 . . . 902
7.2.8 0X86 设备自动进入睡眠模式 . . . 903
7.2.9 0X87 设备自动唤醒 . . . 904
7.2.10 0X88 系统启动成功 . . . 905
7.2.10.1 1.65.0 版本以前 . . . 905
7.2.10.2 1.65.0 版本以后 . . . 905
7.2.11 0X89 开始 SD 卡升级 . . . 906
7.2.12 0XFD 透传数据完成 . . . 907
7.2.13 0XFE 数据透传就绪 . . . 908

8 串口屏高级应用详解 909
8.1 系统键盘 . . . 909

8.1.1 键盘基础知识 . . . 909
8.1.1.1 系统键盘的调用方式 . . . 909
8.1.1.2 键盘解锁和重置 . . . 911
8.1.2 键盘的实现原理 . . . 912
8.1.2.1 系统键盘-常用控件和变量说明 . . . 912
8.1.2.2 其他跳转到键盘页面的方法 . . . 913
8.1.2.3 键盘如何获取到原始控件的值 . . . 914
8.1.2.4 输入完成后如何赋值给原始控件 . . . 915
8.1.3 自定义键盘 . . . 916
8.1.3.1 进入键盘时自动将输入框清空 . . . 916
8.1.3.1.1 进入键盘时自动将输入框清空-样例工程下载 . . . 916
8.1.3.2 键盘页面默认为拼音输入 . . . 917
8.1.3.2.1 键盘页面默认为拼音输入-样例工程下载 . . . 918
8.1.3.3 键盘页面默认为英文大写 . . . 919
8.1.3.3.1 键盘页面默认为英文大写-样例工程下载 . . . 920
8.1.3.4 如何让键盘的光标不闪烁 . . . 921
8.1.3.4.1 让键盘的光标不闪烁-样例工程下载 . . . 921
8.1.3.5 键盘页面限制输入数据的值 . . . 922
8.1.3.5.1 键盘页面限制输入数据的值-样例工程下载 . . . 922
8.1.3.6 在键盘页面按下 OK 键时将参数通过串口发送出去 . . . 923
8.1.3.6.1 在键盘页面按下 OK 键时将参数通过串口发送出去-样例工程下载 . . . 927
8.1.3.7 键盘只能输入 32 个字符需要改成能输入更多的字符 . . . 928
8.1.3.7.1 键盘只能输入 32 个字符需要改成能输入更多的字符-样例工程下载 . . . 928
8.1.3.8 不同控件在系统键盘设置好参数存储到用户存储区 . . . 928
8.1.3.8.1 在系统键盘设置好参数存储到用户存储区-样例工程下载 . . . 931
8.1.3.9 键盘输入回车 . . . 932
8.1.3.9.1 键盘输入回车-样例工程下载 . . . 932
8.1.3.10 进入键盘时显示不同的备注信息 . . . 932
8.1.3.10.1 进入键盘时显示不同的备注信息-样例工程下载 . . . 935
8.1.3.11 自定义好看的键盘 . . . 935
8.1.3.11.1 《自定义好看的键盘》下载 . . . 935
8.1.3.12 UTF8 编码下的汉字拼音输入法键盘 . . . 936
8.1.3.12.1 控件如何配置 UTF8 键盘 . . . 938
8.1.4 系统键盘常见问题 . . . 940
8.1.4.1 系统键盘无法用输入法打出汉字 . . . 940
8.1.4.2 系统键盘无法输入字符 . . . 941
8.1.4.3 控件调用系统键盘输入值后，控件显示的数据没有改变 . . . 942
8.1.4.4 控件调用系统键盘输入值后，其他控件又回到默认值 . . . 942
8.2 主动解析模式应用详解 . . . 943
8.2.1 主动解析基本知识 . . . 943
8.2.1.1 淘晶驰字符串指令和主动解析 . . . 943
8.2.1.2 主动解析原理 . . . 944
8.2.1.3 主动解析基本步骤 . . . 944
8.2.1.4 串口数据解析模式系统变量 recmod . . . 945
8.2.1.5 串口缓冲区数据大小系统变量 usize . . . 946
8.2.1.6 串口缓冲区数据组 u[index] . . . 947
8.2.1.7 串口缓冲区数据拷贝指令 ucopy . . . 948
8.2.1.8 删除缓冲区前面 N 字节数据 udelete . . . 950
8.2.1.9 如何退出主动解析 . . . 951
8.2.2 解析定长 hex 格式指令-自定义协议 . . . 952
8.2.2.1 例 1：帧头为 0x55, 帧尾为 3 个 0xff, 每次传输灯的序号和对应的状态 . . . 952
8.2.2.1.1 定长 hex-示例 1-样例工程下载 . . . 953
8.2.2.2 例 2：帧头为 0x55，帧尾为 3 个 0xff，一次性传输 4 个灯的状态 . . . 954
8.2.2.2.1 定长 hex-示例 2-样例工程下载 . . . 955
8.2.2.3 例 3：帧头为 0x33，帧尾为 0d 0a，每次传输灯的序号和对应的状态 . . . 956
8.2.2.3.1 定长 hex-示例 3-样例工程下载 . . . 957
8.2.2.4 例 4：没有帧头，帧尾为 0d 0a，每次传输灯的序号和对应的状态 . . . 958
8.2.2.4.1 定长 hex-示例 4-样例工程下载 . . . 959

8.2.2.5 例 5：帧头为 0x33，没有帧尾，每次传输灯的序号和对应的状态 960
8.2.2.5.1 定长 hex-示例 5-样例工程下载 961
8.2.2.6 例 6：没有帧头，帧尾为 3 个 0xff，每次传输 4 个灯的状态 962
8.2.2.6.1 定长 hex-示例 6-样例工程下载 963
8.2.2.7 例 7：帧头为 0x55，没有帧尾，每次传输 4 个灯的状态 964
8.2.2.7.1 定长 hex-示例 7-样例工程下载 965
8.2.2.8 例 8：没有帧头也没有帧尾，每次传输 4 个灯的状态 966
8.2.2.8.1 定长 hex-示例 8-样例工程下载 967
8.2.2.9 例 9：一个字节有 8 位，其实可以用 1 个字节传输 8 个灯的开关状态 968
8.2.2.9.1 定长 hex-示例 9-样例工程下载 969
8.2.2.10 例 10：帧头为 0x33，无帧尾，使用 modbus_crc16 校验，一次性传输 4 个灯的状态 970
8.2.2.10.1 定长 hex-示例 10-样例工程下载 971
8.2.2.11 例 11：有两种帧头为 0x47 和 0x4E，帧尾为 0x0d，0x0a，每帧长度 22 字节 . . . 972
8.2.2.11.1 定长 hex-示例 11-样例工程下载 973
8.2.2.12 例 101：帧头为 55 aa，定长 hex-点灯 1 974
8.2.2.12.1 定长 hex-点灯 1-样例工程下载 975
8.2.2.13 例 102：帧头为 55 aa，定长 hex-点灯 2 976
8.2.2.13.1 定长 hex-点灯 2-样例工程下载 977
8.2.2.14 例 103：定长 hex-主动解析样例工程 978
8.2.2.14.1 定长 hex-主动解析样例工程-样例工程下载 979
8.2.2.15 例 104：电化学氧气模组 SC-03-O2-主动解析样例工程 980
8.2.2.15.1 电化学氧气模组 SC-03-O2-样例工程下载 981
8.2.2.16 例 1000：某 bms 串口协议 1 982
8.2.2.16.1 定长 hex-某 bms 串口协议 1-样例工程下载 988
8.2.3 解析不定长 hex 格式指令-自定义协议 989
8.2.3.1 不定长-帧头为 0x55，帧尾为 3 个 0xff 989
8.2.3.1.1 不定长 hex-示例 1-样例工程下载 990
8.2.3.2 帧头为 0xFF，帧尾为 0d 0a 991
8.2.3.2.1 不定长 hex-示例 2-样例工程下载 992
8.2.3.3 传输不定长的字符串 993
8.2.3.3.1 不定长 hex-示例 3-样例工程下载 994
8.2.3.4 解析不定长 hex 协议-海陵科 LD-2410 生命存在感应模组 995
8.2.3.4.1 海陵科生命存在感应模组 hex 协议解析下载 998
8.2.4 解析字符串格式指令 999
8.2.4.1 解析 AT 指令 999
8.2.4.1.1 解析 AT 指令-样例工程下载 1002
8.2.4.2 接收 json 数据字符串 1003
8.2.4.2.1 json 解析-样例工程下载 1004
8.2.4.3 解析不定长字符串 (以回车换行结尾) 1005
8.2.4.3.1 解析不定长字符串 (以回车换行结尾)-样例工程下载 1005
8.3 HMI 下载协议详解/OTA 升级 1006
8.3.1 未知串口号和波特率 1006
8.3.2 已知串口号和波特率 1006
8.3.3 【下载步骤 1：联机操作】 1006
8.3.3.1 【下载步骤 1：流程图】 1007
8.3.3.2 如果未开启主动解析模式 1007
8.3.3.3 如果开启了主动解析模式 1008
8.3.3.4 联机数据说明 1008
8.3.4 【下载步骤 2：开始下载】 1008
8.3.4.1 【下载步骤 2：流程图】 1009
8.3.5 开源 TFT 下载工具下载 1009
8.3.5.1 开源 TFT 下载工具-代码示例 1009
8.3.5.1.1 c# 实现的下载工具 1009
8.3.5.1.2 python 实现的下载工具 1011
8.3.5.1.3 串口屏实现的下载工具 1015
8.4 串口指令增加 CRC 校验 1016
8.4.1 结束符不同 1016
8.4.2 CRC16 校验算法 1016

8.4.3 3.CRC16 校验码写入方式 . . . 1017
8.5 程序中使用 CRC 校验数据 . . . 1018
8.5.1 使用 crc 校验的步骤 . . . 1018
8.5.1.1 初始化 crc 校验 . . . 1018
8.5.1.2 使用 crcputh/crcputs/crcputu 将需要的数据放入校验 . . . 1018
8.5.1.3 获取 crc 计算结果 . . . 1018
8.5.2 crc 校验示例 . . . 1019
8.5.3 串口屏计算出来的 crc 结果和单片机计算出来的结果不同如何排查 . . . 1021
8.5.4 crc 校验-样例工程下载 . . . 1021
8.6 运行中串口传输文件到内存或 SD 卡 . . . 1022
8.7 SD 卡读写文件流程 . . . 1024
8.7.1 1. 检查文件是否存在 . . . 1024
8.7.2 2. 打开文件 . . . 1026
8.7.3 3. 读、写、查找 . . . 1028
8.7.4 4. 关闭文件 . . . 1028
8.7.5 文件流控件-注意事项 . . . 1029
8.7.5.1 新建文件时为什么要初始化文件 . . . 1029
8.7.5.2 没有初始化文件会怎么样 . . . 1029
8.7.5.3 如果我不想在屏幕里初始化怎么办呢 . . . 1029
8.7.6 SD 卡读写文件流程-样例工程下载 . . . 1029
8.8 拓展 IO 资料 . . . 1030
8.8.1 拓展 IO 简介 . . . 1030
8.8.2 拓展 IO 简单使用 . . . 1032
8.8.3 拓展板原理图 . . . 1041
8.8.4 sleep 睡眠模式与拓展 IO . . . 1041
8.8.5 外部 IO-样例工程下载 . . . 1042
8.8.6 拓展 IO-相关链接 . . . 1042
8.9 获取控件自身的属性 . . . 1043
8.9.1 获取控件自身的属性-样例工程下载 . . . 1043
8.10 读写用户存储区 . . . 1044
8.10.1 读写用户存储区例程-样例工程下载 . . . 1044
8.11 获取串口屏设备唯一序列号 . . . 1045
8.11.1 通过串口获取 . . . 1045
8.11.2 新方法-通过指令获取 . . . 1045
8.11.3 获取设备唯一序列号例程下载 . . . 1046
8.12 数据记录文件结构分析 . . . 1047
8.12.1 数据记录文件结构分析详解 . . . 1047
8.12.1.1 数据记录文件结构分析-样例工程下载 . . . 1051
8.12.2 数据记录文件转换-python 代码 . . . 1052
8.12.3 数据记录文件转换-js 代码 . . . 1057
8.12.4 下载《数据记录文件解析》例程 . . . 1065

9 资料下载 1066
9.1 上位机下载 . . . 1066
9.1.1 旧版本上位机下载 . . . 1066
9.1.1.1 下载 V1.67.6 USART HMI 软件（最新） . . . 1066
9.1.1.2 下载 V1.67.6 USART HMI 软件 (免安装版) . . . 1067
9.1.1.3 下载 V1.67.5 USART HMI 软件 . . . 1067
9.1.1.4 下载 V1.67.5 USART HMI 软件 (免安装版) . . . 1067
9.1.1.5 下载 V1.67.3 USART HMI 软件 . . . 1067
9.1.1.6 下载 V1.67.3 USART HMI 软件 (免安装版) . . . 1067
9.1.1.7 下载 V1.67.2 USART HMI 软件 . . . 1067
9.1.1.8 下载 V1.67.2 USART HMI 软件 (免安装版) . . . 1067
9.1.1.9 下载 V1.67.1 USART HMI 软件 . . . 1067
9.1.1.10 下载 V1.67.1 USART HMI 软件 (免安装版) . . . 1067
9.1.1.11 下载 V1.66.3 USART HMI 软件 . . . 1068
9.1.1.12 下载 V1.66.3 USART HMI 软件 (免安装版) . . . 1068
9.1.1.13 下载 V1.66.2 USART HMI 软件 . . . 1068

9.1.1.14 下载 V1.66.1 USART HMI 软件 . . . 1068
9.1.1.15 下载 V1.65 USART HMI 软件 . . . 1068
9.1.1.16 下载 V1.64 USART HMI 软件 . . . 1069
9.1.1.17 下载 V1.63 USART HMI 软件 . . . 1070
9.1.1.18 下载 V1.61 USART HMI 软件 . . . 1071
9.1.1.19 下载 V1.60 USART HMI 软件 . . . 1072
9.1.1.20 下载 V0.59 USART HMI 软件 . . . 1073
9.1.1.21 下载 V0.57 USART HMI 软件 . . . 1074
9.1.1.22 下载 V0.56 USART HMI 软件 . . . 1075
9.1.1.23 下载 V0.55 USART HMI 软件 . . . 1076
9.1.1.24 下载 V0.54 USART HMI 软件 . . . 1077
9.1.1.25 下载 V0.52 USART HMI 软件 . . . 1078
9.1.1.26 下载 V0.51 USART HMI 软件 . . . 1079
9.1.1.27 下载 V0.50 USART HMI 软件 . . . 1080
9.1.1.28 下载 V0.49 USART HMI 软件 . . . 1081
9.1.1.29 下载 V0.47 USART HMI 软件 . . . 1082
9.1.1.30 下载 V0.46 USART HMI 软件 . . . 1083
9.1.1.31 下载 V0.45 USART HMI 软件 . . . 1084
9.1.1.32 下载 V0.44 USART HMI 软件 . . . 1085
9.1.1.33 下载 V0.42 USART HMI 软件 . . . 1086
9.1.1.34 下载 V0.41 USART HMI 软件 . . . 1087
9.1.1.35 下载 V0.40 USART HMI 软件 . . . 1088
9.1.1.36 下载 V0.39 USART HMI 软件 . . . 1089
9.1.1.37 下载 V0.37 USART HMI 软件 . . . 1090
9.1.1.38 下载 V0.36 USART HMI 软件 . . . 1091
9.1.1.39 下载 V0.35 USART HMI 软件 . . . 1092
9.1.1.40 下载 V0.34 USART HMI 软件 . . . 1093
9.2 开发文档下载 . . . 1094
9.3 常用工具下载 . . . 1095
9.3.1 sscom5.13.1 串口调试助手下载 . . . 1095
9.3.2 虚拟串口工具 vspd 下载 . . . 1096
9.3.3 TFT 文件下载助手 (TFTFileDownload) . . . 1097
9.3.4 文件透传工具 (SerialFileUp) . . . 1098
9.3.5 VC++ 运行库合集 . . . 1100
9.3.6 Microsoft .net Framework 3.5 . . . 1101
9.3.6.1 如何查看 windows 版本 . . . 1101
9.3.7 数据记录控件数据转 excel 文件工具 (DataBox) . . . 1103
9.3.8 常见串口驱动下载 . . . 1104
9.3.9 powertoys 下载 . . . 1105
9.3.9.1 PowerToys0.97.2-x64 . . . 1105
9.3.9.2 PowerToys0.95.1-x64(旧版本) . . . 1105
9.3.9.3 PowerToys0.90.1-x64(旧版本) . . . 1105
9.3.10 解压缩工具 7zip 下载 . . . 1106
9.3.11 python 安装包 . . . 1107
9.4 标准出厂工程样例 . . . 1109
9.5 官方样例工程 . . . 1111
9.5.1 多语言切换样例工程 . . . 1111
9.5.1.1 多语言切换-样例工程下载 . . . 1111
9.5.2 串口屏弹窗样例工程 . . . 1112
9.5.2.1 串口屏弹窗-样例工程下载 . . . 1112
9.5.3 正计时和倒计时样例工程 . . . 1113
9.5.3.1 正计时和倒计时-样例工程下载 . . . 1113
9.5.4 串口屏跨页面计时样例工程 . . . 1114
9.5.4.1 串口屏跨页面计时-样例工程下载 . . . 1114
9.5.5 RGB 转换样例工程 . . . 1115
9.5.5.1 RGB565 转 RGB888 . . . 1116
9.5.5.2 RGB888 转 RGB565 . . . 1117
9.5.5.3 RGB 转换-样例工程下载 . . . 1119

9.5.6 自定义曲线样例工程 . . . 1120
9.5.6.1 自定义曲线-样例工程下载 . . . 1120
9.5.7 51 单片机样例工程 . . . 1121
9.5.7.1 《51 单片机》-样例工程下载 . . . 1121
9.5.8 简易串口助手样例工程 . . . 1122
9.5.8.1 简易串口助手-样例工程下载 . . . 1122
9.5.9 进制转换样例工程 . . . 1123
9.5.9.1 16 进制转 10 进制 . . . 1123
9.5.9.2 10 进制转 16 进制 . . . 1124
9.5.9.3 16 进制转 2 进制 . . . 1125
9.5.9.4 2 进制转 16 进制 . . . 1126
9.5.9.5 10 进制转 2 进制 . . . 1127
9.5.9.6 2 进制转 10 进制 . . . 1128
9.5.9.6.1 《进制转换》-样例工程下载 . . . 1129
9.5.10 自定义屏幕保护样例工程/低功耗/睡眠休眠替代 . . . 1130
9.5.10.1 自定义屏幕保护-样例工程下载 . . . 1130
9.5.11 虚拟浮点数与文本控件互相转换样例工程 . . . 1131
9.5.11.1 虚拟浮点数与文本控件互相转换-样例工程下载 . . . 1131
9.5.12 大小端数据互相转换样例工程 . . . 1132
9.5.12.1 大小端数据互相转换-样例工程下载 . . . 1132
9.5.13 显示时间和设置时间样例工程 . . . 1133
9.5.13.1 多页面显示时间和使用选择文本控件设置时间-样例工程下载 . . . 1133
9.5.14 串口屏实现通讯心跳样例工程 . . . 1134
9.5.14.1 串口屏实现通讯心跳-样例工程下载 . . . 1135
9.5.15 运行中发送图片到串口屏样例工程 . . . 1136
9.5.15.1 运行中发送图片到串口屏-样例工程下载 . . . 1136
9.5.16 环形进度条样例工程 . . . 1137
9.5.16.1 环形进度条-样例工程下载 . . . 1139
9.5.17 使用存储卡跨页面记录波形数据-文件流控件样例工程 . . . 1140
9.5.17.1 使用存储卡跨页面记录波形数据-样例工程下载 . . . 1142
9.5.18 X 系列给其他屏幕下载工程 . . . 1143
9.5.18.1 为什么要将生成的 tft 文件后缀名改为 dwn . . . 1144
9.5.18.2 X 系列给其他屏幕下载工程-样例工程下载 . . . 1144
9.6 UI 样例工程 . . . 1145
9.6.1 AGV 小车 UI(X2_800x480) . . . 1145
9.6.2 BMS 控制器 UI(T1_240x320) . . . 1145
9.6.3 UPS 储能电源 UI(X2_800x480) . . . 1146
9.6.4 按摩仪 UI(T1_320x240) . . . 1146
9.6.5 按摩仪蓝色 UI(T1_320x240) . . . 1147
9.6.6 按摩椅 UI(X3_800x480) . . . 1147
9.6.7 按摩椅 UI(X5_1024x600) . . . 1148
9.6.8 便携式检测仪 UI(K0_320x240) . . . 1148
9.6.9 叉车充电器 UI(X2_480x272) . . . 1149
9.6.10 超声波清洗机 UI(X5_800x480) . . . 1150
9.6.11 炒菜机 UI(T1_480x272) . . . 1151
9.6.12 车载冰箱 UI(T1_480x320) . . . 1152
9.6.13 充电桩行业 UI(T1_320x240) . . . 1153
9.6.14 抽油烟机 UI(T1_480x320) . . . 1154
9.6.15 灯光控制器 UI(T1_480x320) . . . 1155
9.6.16 电梯屏 UI(X5_1024x600) . . . 1156
9.6.17 电子智能储物柜 UI(X2_800x480) . . . 1157
9.6.18 房车控制面板 UI(X5_800x480) . . . 1158
9.6.19 光伏逆变器 UI(T1_480x320) . . . 1159
9.6.20 焊接机行业 UI(X3_800x480) . . . 1160
9.6.21 户外电源 UI(T1_320x240) . . . 1160
9.6.22 会议系统设备 UI(X2_800x480) . . . 1161
9.6.23 机箱副屏 UI(X2_480x272) . . . 1161
9.6.24 交流充电桩 UI(T1_320x240) . . . 1162

9.6.25 交流充电桩 UI(T1_480x320) . . . 1163
9.6.26 进度条控件 UI(X5_1024x600) . . . 1164
9.6.27 咖啡机 UI1(T1_480x320) . . . 1165
9.6.28 咖啡机 UI2(T1_480x320) . . . 1166
9.6.29 咖啡机 UI3(T1_480x272) . . . 1167
9.6.30 咖啡机 UI4(X5_800x480) . . . 1167
9.6.31 考勤机 UI(T1_320x240) . . . 1168
9.6.32 空调线控器 UI(X5_480x480) . . . 1169
9.6.33 空气检测仪 UI(T1_480x320) . . . 1170
9.6.34 空气炸锅 UI(T1_480x272) . . . 1171
9.6.35 螺丝机 UI(X5_800x480) . . . 1173
9.6.36 毛巾烘干机 UI(T1_480x320) . . . 1174
9.6.37 美容仪器 UI(X2_800x480) . . . 1175
9.6.38 美容仪器 UI(X2_800x600) . . . 1176
9.6.39 跑步机 UI(X5_1024x600) . . . 1177
9.6.40 泡茶机 UI(T1_480x320) . . . 1178
9.6.41 喷码机 UI(T1_480x272) . . . 1179
9.6.42 破壁机 UI(T1_480x272) . . . 1179
9.6.43 汽车充电桩 UI(X5_800x480) . . . 1180
9.6.44 取暖器 UI(T1_480x320) . . . 1181
9.6.45 热水壶 UI(T1_320x240) . . . 1182
9.6.46 热水器 UI(T1_480x272) . . . 1182
9.6.47 商用烤箱 UI(X3_800x480) . . . 1183
9.6.48 摄影灯 UI(K0_320x240) . . . 1183
9.6.49 数字功放一体机 UI(X3_800x480) . . . 1184
9.6.50 水质检测仪 UI(X2_800x480) . . . 1184
9.6.51 睡眠呼吸机 UI(T1_480x320) . . . 1185
9.6.52 微波炉 UI(T1_480x320) . . . 1186
9.6.53 温度控制器 UI(X5_480x480) . . . 1187
9.6.54 物流高拍仪 UI(X5_1024x600) . . . 1188
9.6.55 消防主机 UI(X5_800x480) . . . 1189
9.6.56 新风系统 UI(X5_800x480) . . . 1190
9.6.57 血压计 UI(X2_800x480) . . . 1190
9.6.58 液体灌装机 UI(X5_800x480) . . . 1193
9.6.59 医疗美容 UI(X3_800x480) . . . 1194
9.6.60 医疗美容 UI(X5_480x272) . . . 1195
9.6.61 医疗仪器输液泵 UI(T1_480x320) . . . 1196
9.6.62 照明行业 UI(X5_480x272) . . . 1197
9.6.63 照明行业 UI(X5_800x480) . . . 1197
9.6.64 制冰机 UI(X3_480x272) . . . 1198
9.6.65 制氧机 UI(X2_800x480) . . . 1199
9.6.66 智能称重一体机 UI(X2_800x480) . . . 1199
9.6.67 智能家居 UI(X5_480x480) . . . 1200
9.6.68 智能马桶 UI(T1_320x240) . . . 1201
9.6.69 智能燃气灶 UI(T1_480x272) . . . 1201
9.6.70 智能洗碗机 UI(X2_800x480) . . . 1202
9.6.71 智能洗衣机 UI(T1_480x272) . . . 1202
9.6.72 智能消毒柜 UI(X2_800x480) . . . 1203
9.6.73 智能足浴盆 UI(X2_480x320) . . . 1204
9.6.74 中频治疗仪 UI(X5_480x272) . . . 1205
9.6.75 自动贩卖机 (X5 10 寸 1024600) . . . 1206
9.7 图片资源下载 . . . 1208
9.7.1 按钮图片资源下载 . . . 1208
9.7.2 背景图片资源下载 . . . 1209
9.8 免费字体下载 . . . 1211
9.8.1 思源黑体下载 . . . 1211
9.8.1.1 win10 和 win11 建议使用 otf 字体 . . . 1211
9.8.1.2 win7 建议使用 ttf 字体 . . . 1211

9.8.1.3 数码管字体 . . . 1211
9.8.2 免费中文字体列表 . . . 1211
9.8.3 字体下载相关链接 . . . 1211
9.9 声音资源下载 . . . 1212
9.10 原理图下载 . . . 1213
9.10.1 T1/X2 系列 COF 裸屏测试小板/背板原理图 . . . 1213
9.10.2 USB 转 TTL 转接板 . . . 1214
9.10.3 IO 拓展板 . . . 1215
9.11 网友提供应用样例 . . . 1216
9.11.1 中国象棋 . . . 1216
9.11.2 推箱子游戏 . . . 1218
9.11.3 多功能测量仪 . . . 1219
9.11.4 全键盘输入、密码登录 . . . 1220
9.11.5 简易计算器 . . . 1221
9.11.6 打兔子，无需单片机支持 . . . 1222
9.11.7 电压数据采集及系统参数设置 . . . 1223
9.11.8 串口屏控制 3 路继电器 . . . 1224
9.11.9 九宫格全英文输入（支持串口打印字符） . . . 1225
9.11.10 酷炫 Loading 进度条界面 . . . 1226
9.11.11 86 面板智能家居 . . . 1227
9.11.12 简洁圆形数字键盘（支持全局输入） . . . 1228
9.11.13 多种控件效果 . . . 1229
9.12 游戏工程 . . . 1230
9.12.1 中国象棋游戏工程 . . . 1230
9.12.2 推箱子游戏工程 . . . 1230
9.12.3 兔了个兔 . . . 1230
9.12.4 打地鼠 . . . 1233
9.12.5 掷骰子赌大小游戏 . . . 1234
9.12.6 猜拳大通关 . . . 1236
9.12.7 flappy Bird . . . 1237
9.13 淘晶驰资料合集 . . . 1238
9.13.1 网盘链接 1 . . . 1238
9.13.2 网盘链接 2 . . . 1238

10 视频教程 **1239**
10.1 基础入门 . . . 1239
10.2 控件详解 . . . 1239
10.3 视频对应的工程下载链接 . . . 1240
10.4 其他教程 . . . 1240

11 常见问题 **1241**
11.1 硬件相关 . . . 1241
11.1.1 ESD 能过多少 V，EMI 性能如何 . . . 1241
11.1.2 串口通讯线最长能接多少米 . . . 1242
11.1.3 通讯口电压多少 V，是否可以直接接单片机 . . . 1243
11.1.4 屏幕通电后不断的闪烁 (供电不足导致不断重启) . . . 1244
11.1.5 喇叭相关问题 . . . 1245
11.1.5.1 觉得喇叭声音小怎么办 . . . 1245
11.1.5.2 喇叭接口型号 . . . 1246
11.1.5.3 喇叭的正负极 . . . 1246
11.1.5.4 喇叭插上有电流声 . . . 1247
11.1.6 关于串口工具 (cp2102/ft232/ch340/pl2303) . . . 1248
11.1.6.1 为什么不建议用 ch340 . . . 1248
11.1.6.2 为什么不建议用 pl2303 . . . 1248
11.1.6.3 不建议使用的其他串口工具 . . . 1248
11.1.6.4 建议使用哪些串口工具 . . . 1248
11.1.7 如何修改控件显示的字体 . . . 1251
11.1.8 串口驱动相关问题 . . . 1254

11.1.8.1 如何查看串口驱动是否正确安装 . . . 1254
11.1.8.2 如何安装串口驱动 . . . 1254
11.1.9 串口电平状态 . . . 1255
11.1.10 关于一上电就关闭屏幕背光 . . . 1256
11.1.11 结构兼容性 . . . 1257
11.1.12 拓展 IO 相关问题 . . . 1258
11.1.12.1 如何初始化 IO 口 . . . 1258
11.1.13 如何修改设备型号 . . . 1259
11.1.14 屏幕边框为什么不对称 . . . 1261
11.1.15 外部供电接法 . . . 1262
11.1.16 如何判断屏幕是否支持蜂鸣器 . . . 1263
11.1.17 如何判断屏幕是否支持 RTC 实时时钟 . . . 1264
11.1.18 为什么 232 通讯不能超过 256000 . . . 1265
11.1.19 如何判断屏幕是否支持喇叭 . . . 1267
11.1.20 串口屏常用接口型号说明 . . . 1268
11.1.20.1 XH2.54*4pin 接口说明 . . . 1268
11.1.20.1.1 XH2.54*4pin 实物图片 . . . 1268
11.1.20.1.2 XH2.54*4pin 连接器母座说明 . . . 1269
11.1.20.1.3 XH2.54*4pin 连接器端子说明 . . . 1271
11.1.20.2 喇叭（ph1.25*2pin）接口说明 . . . 1271
11.1.20.2.1 ph1.25*2pin 实物图片 . . . 1271
11.1.20.2.2 ph1.25*2pin 连接器母座说明 . . . 1272
11.1.20.2.3 ph1.25*2pin 连接器端子说明 . . . 1272
11.1.20.3 外部 IO（FPC1.0 * 10pin）接口说明 . . . 1272
11.1.20.3.1 FPC1.0 * 10pin 连接器说明 . . . 1272
11.1.20.3.2 FPC1.0 * 10pin 排线说明 . . . 1274
11.1.20.4 大尺寸高电压串口屏供电接口（HY2.0 8pin）接口说明 . . . 1274
11.1.20.4.1 HY2.0 8pin 实物图片 . . . 1274
11.1.20.4.2 HY2.0 8pin 连接器母座说明 . . . 1274
11.1.20.4.3 HY2.0 8pin 连接器端子说明 . . . 1276
11.1.20.4.4 ph1.25*4pin 连接器母座说明 . . . 1276
11.1.20.4.5 ph1.25*4pin 连接器端子说明 . . . 1279
11.2 结构设计相关 . . . 1280
11.2.1 户外使用说明 . . . 1280
11.3 软件相关 . . . 1281
11.3.1 设计好的工程有哪些下载方式 . . . 1281
11.3.1.1 通过串口下载 . . . 1281
11.3.1.2 通过 SD 卡下载 . . . 1281
11.3.2 tft 文件串口/SD 卡下载失败如何解决 . . . 1282
11.3.3 调试或下载时联机失败 . . . 1283
11.3.4 指定字库 . . . 1284
11.3.4.1 指定字库-相关链接 . . . 1294
11.3.5 哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别? . . . 1295
11.3.6 txt 属性和 val 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 . . . 1297
11.3.6.1 字符串属性和数值属性-相关链接 . . . 1298
11.3.7 运行中控件属性被修改后，离开页面再回来，不希望属性回到初始值怎么操作 . . . 1299
11.3.7.1 跨页面操作-相关链接 . . . 1299
11.3.8 SD 卡有哪些作用 . . . 1300
11.3.9 串口屏卡顿的原因 . . . 1301
11.3.10 文本控件或按钮控件不显示我输入的字符内容或显示不全 . . . 1302
11.3.11 单片机发指令给屏幕没有反应 . . . 1305
11.3.12 如何配置亮度, 主动解析, 波特率 . . . 1306
11.3.12.1 亮度, 主动解析, 波特率-相关链接 . . . 1306
11.3.13 系统内置的触摸键盘怎么修改风格 . . . 1307
11.3.14 滑块或进度条控件默认是横向，为什么我设为竖向后没有变化，还是显示横向呢 . . . 1309
11.3.15 单片机和串口屏通信为什么单片机上电后要延时一会再发指令，延时多久合适 . . . 1311
11.3.16 单片机上电后为什么要先发一次 0x00 0xff 0xff 0xff 给屏幕 . . . 1312
11.3.17 屏幕地址怎么设置，怎么使用 . . . 1313

11.3.18 注释乱码或者不显示 . . . 1314
11.3.19 如何更改控件的前后图层关系 . . . 1315
11.3.20 如何设计多语言界面 . . . 1316
11.3.21 K 系列和 T 系列如何实现透明效果 . . . 1317
11.3.22 单片机如何接收和解析串口屏数据 . . . 1319
11.3.23 如何对掉电存储空间进行初始化 . . . 1320
11.3.24 转义字符相关 . . . 1321
11.3.24.1 回车换行 . . . 1321
11.3.24.2 引号 . . . 1321
11.3.25 使用 sscom 时，sscom 可以接收到串口数据，但是 sscom 发送的数据屏幕接收不到 . . . 1323
11.3.26 如何实现按位取反 . . . 1324
11.3.26.1 计算 1 字节按位取反 . . . 1324
11.3.26.2 计算 2 字节按位取反 . . . 1324
11.3.26.3 计算 4 字节按位取反 . . . 1324
11.3.26.4 按位取反-相关链接 . . . 1324
11.3.27 如何实现按位同或运算 . . . 1325
11.3.27.1 位运算-相关链接 . . . 1325
11.3.28 页面和页面控件的区别 . . . 1326
11.3.29 休眠替代方法 . . . 1328
11.3.30 淘晶驰串口屏支持哪些通讯格式 . . . 1329
11.3.31 dim 和 dims 的区别 . . . 1330
11.3.31.1 什么时候使用 dim，什么时候使用 dims . . . 1330
11.3.32 baud 和 bauds 的区别 . . . 1331
11.3.33 能不能用 printf 一次性发送多条指令 . . . 1332
11.3.34 如何显示和计算百分比 . . . 1333
11.3.34.1 屏幕单纯显示百分比，由单片机计算好后发给屏幕显示 . . . 1333
11.3.34.2 屏幕计算百分比 . . . 1333
11.3.35 内存文件存储区相关 . . . 1334
11.3.35.1 什么是内存文件存储区 . . . 1334
11.3.35.2 如何使用内存文件存储区 . . . 1336
11.3.36 页面名称乱码 . . . 1337
11.3.37 为什么 wepo 写入后不允许立刻读取或者断电 . . . 1338
11.3.38 常用英语 . . . 1339
11.3.39 页面跳转后, 控件怎么保持原来的状态 . . . 1340
11.3.40 串口屏重启后, 控件怎么保持原来的状态 . . . 1341
11.4 错误提示 . . . 1342
11.4.1 编译报错：XXX 初始值无效 . . . 1342
11.4.1.1 字库 ID 无效 . . . 1342
11.4.1.2 图片 ID 无效 . . . 1344
11.4.1.3 动画 ID 无效 . . . 1346
11.4.1.4 音频 ID 无效 . . . 1348
11.4.1.5 视频 ID 无效 . . . 1350
11.4.1.6 图片转换工具 . . . 1352
11.4.1.6.1 初始值无效-相关链接 . . . 1352
11.4.2 单片机发指令给屏幕，屏幕返回 1A FF FF FF 或 1C FF FF FF 四个字节的 HEX 数据 . . . 1353
11.4.3 串口连接不上屏幕 . . . 1354
11.4.4 串口屏黑屏原因 . . . 1355
11.4.5 变量名称无效 zstr . . . 1356
11.4.6 file version is too low . . . 1358
11.4.7 报错:file is too large for destination device . . . 1359
11.4.8 在做字库的时候有部分字体无法选择 . . . 1360
11.4.8.1 字库创建-相关链接 . . . 1360
11.4.9 如何解决调用系统键盘给控件赋值无效 . . . 1361
11.4.10 软件输出生产文件，出现提示框拒绝访问 . . . 1362
11.4.11 window7 无法打开软件安装包 . . . 1364
11.4.12 用串口下载程序，成功但显示 Update failed check error . . . 1365
11.4.13 串口屏开机时死机/不断的闪烁/不断重启 . . . 1366
11.4.14 调试时可以正常显示，但是下载进屏幕就无法使用 . . . 1367

11.4.15 视频 IDO 文件方向与当前工程显示方向不一致 . . . 1368
11.4.16 提示语句错误：无前括号 . . . 1369
11.4.17 提示转义字符使用错误 . . . 1370
11.4.18 编译时提示变量名称无效 . . . 1371
11.4.19 编译时提示无效指令 . . . 1372
11.4.20 提示 Show Picture Error . . . 1373
11.4.21 下载一半失败后下载不进去 . . . 1375
11.4.22 输入中文时上位机卡死 . . . 1376
11.4.23 no find cpu model . . . 1377
11.4.24 提示：请先选择控件 . . . 1378
11.4.25 提示：没有控件可以粘贴 . . . 1379
11.4.26 明明型号是对的，但是烧录的时候提示 Model does not match . . . 1380
11.4.27 MAC 电脑下拷贝文件时提示有多个 TFT 文件 . . . 1381
11.4.28 错误的资源文件或者资源文件已经受损 . . . 1382
11.4.29 烧录文件提示 open file error . . . 1384
11.5 其他 . . . 1385
11.5.1 如何查看编译时输出的信息 . . . 1385
11.5.1.1 运行内存占用情况 . . . 1385
11.5.1.2 FLASH（存储空间）占用情况 . . . 1386
11.5.2 不同页面之间的同名控件有什么联系 . . . 1386
11.5.2.1 如何同步两个不同页面的控件 . . . 1386
11.5.3 在定时器 0 里面加入使能定时器 1 的语句为什么系统不执行 . . . 1387
11.5.4 大小端模式 . . . 1388
11.5.5 如何删除控件 . . . 1389
11.5.6 使用串口下载工程速度慢怎么办 . . . 1391
11.5.7 printf() 重定向之后，发送命令和结束符 . . . 1392
11.5.8 实现自己的 printf 函数 . . . 1393
11.5.9 单片机如何控制屏幕 . . . 1394
11.5.9.1 单片机如何控制屏幕 (文本控件) . . . 1394
11.5.9.2 单片机如何控制屏幕 (数字控件) . . . 1394
11.5.9.3 串口屏如何显示小数 . . . 1394
11.5.9.4 单片机如何控制屏幕-相关链接 . . . 1395
11.5.10 虚拟 sd 卡文件夹如何打开 . . . 1396
11.5.11 如何实现页面滑动切换 . . . 1397
11.5.12 如何设置页面背景 . . . 1399
11.5.13 HMI 文件和 TFT 文件的关系 . . . 1400
11.5.14 界面旋转/翻转/方向 . . . 1401
11.5.15 串口屏如何制作弹窗 . . . 1402
11.5.15.1 弹窗实现方法 1 . . . 1402
11.5.15.2 弹窗实现方法 2 . . . 1402
11.5.15.3 串口屏如何制作弹窗-样例工程 . . . 1402
11.5.16 utf8 字库下如何只选择汉字 . . . 1403
11.5.16.1 UTF8 字库-相关链接 . . . 1404
11.5.17 TFT 文件如何下载到串口屏中 . . . 1405
11.5.17.1 通过 SD 卡下载 TFT 文件到串口屏 . . . 1405
11.5.17.2 通过串口下载 TFT 文件到串口屏 . . . 1405
11.5.18 只有屏/tft 文件, 或者源 HMI 工程丢了, 能反编译出 HMI 文件吗 . . . 1413
11.5.19 如何导入导出 page 文件 . . . 1414
11.5.20 屏幕保护制作 . . . 1416
11.5.21 二维码控件是 gb2312 还是 utf8 编码 . . . 1417
11.5.22 修改工程编码为 utf8 或者 GB2312 . . . 1418
11.5.23 文本数据显示不完全 . . . 1419
11.5.24 键盘无法输入中文 . . . 1421
11.5.25 单片机如何判断串口屏有没有接入 . . . 1422
11.5.26 实现按键锁定功能 . . . 1423
11.5.27 串口屏的串口缓冲区大小 . . . 1424
11.5.28 主动解析下如何提取负数 . . . 1425
11.5.28.1 signed int8 . . . 1425

11.5.28.2 signed int16 . . . 1425
11.5.28.3 signed int32 . . . 1425
11.5.29 带外壳产品安装示意图 . . . 1426
11.5.30 显示比例不对，正方形变成长方形，圆型变成椭圆 . . . 1427
11.5.30.1 3.9 寸屏 320*320 . . . 1427
11.5.30.2 4 寸屏 480*480 . . . 1427
11.5.30.3 4.3 寸屏 800*480 . . . 1428
11.5.30.4 7 寸屏规格 800*480 . . . 1428
11.5.30.5 7 寸屏规格 1024*600 . . . 1429
11.5.30.6 10.1 寸屏 1024*600 . . . 1429
11.5.30.7 原因说明 . . . 1430
11.5.31 发送中文汉字相关问题 . . . 1431
11.5.31.1 keil 编译的程序“中文 (汉字) 显示不全、串口发送中文汉字接收后乱码缺失”问题 1431
11.5.31.2 arduino 发送中文时乱码 . . . 1431
11.5.31.3 单片机发送中文到屏幕上，但是屏幕出现乱码 . . . 1431
11.5.32 为什么关闭小板上的电源时，RX 灯会亮起 . . . 1432
11.5.33 变量控件名和页面名称可以一样吗 . . . 1433
11.5.34 如何使用 TFT 文件下载助手 (TFTFileDownload) 下载工程 . . . 1435
11.5.35 单片机 3.3V 的串口可以直接接串口屏的串口吗 . . . 1438
11.5.36 如何安装新字体 . . . 1439
11.5.36.1 如何安装新字体-相关链接 . . . 1441
11.5.37 如何导入字库 . . . 1442
11.5.37.1 如何导入字库-相关链接 . . . 1447
11.5.38 如何导入图片 . . . 1449
11.5.38.1 如何导入图片-相关链接 . . . 1453
11.5.39 如何导入动画 . . . 1455
11.5.39.1 如何导入动画-相关链接 . . . 1459
11.5.40 如何导入视频 . . . 1461
11.5.40.1 如何导入视频-相关链接 . . . 1466
11.5.41 如何导入音频 . . . 1467
11.5.41.1 如何导入音频-相关链接 . . . 1472
11.5.42 串口屏连接单片机开发板注意事项 . . . 1474
11.5.43 字符编码相关详解——串口怎么发送字符和汉字 . . . 1479
11.5.43.1 为什么要使用字符编码 . . . 1479
11.5.44 为什么转换后的图片/视频/音频体积变大了 . . . 1482
11.5.45 如何测量屏幕的尺寸大小 . . . 1483
11.5.46 MAC 电脑如何开发淘晶驰串口屏 . . . 1484
11.5.47 数据记录文件结构分析 1 . . . 1485
11.5.48 哪些指令和操作会消耗 flash 寿命 . . . 1486
11.5.48.1 bauds-配置掉电不丢失的波特率 . . . 1486
11.5.48.2 volume-配置掉电不丢失的音量 . . . 1486
11.5.48.3 dims-配置掉电不丢失的亮度 . . . 1486
11.5.48.4 wepo-将数据保存在串口屏内 . . . 1487
11.5.48.5 wept-通过串口将数据透传入串口屏内 . . . 1487

12 产品选型和规格书 1488
12.1 产品选型 . . . 1488
12.1.1 产品命名规则 . . . 1488
12.1.2 功能选型 . . . 1490
12.1.2.1 功能选型-相关资料 . . . 1491
12.1.3 X2 系列 . . . 1492
12.1.3.1 X2 系列 4.0 寸 ~7.0 寸 . . . 1492
12.1.4 X3 系列 . . . 1493
12.1.4.1 X3 系列 4.3 寸 ~7.0 寸 . . . 1493
12.1.5 X5 系列 . . . 1494
12.1.5.1 X5 系列 4.0 寸 ~5.0 寸 . . . 1494
12.1.5.2 X5 系列 7.0 寸 ~10.1 寸 . . . 1494
12.1.6 K0 系列 . . . 1496

12.1.6.1 K0 系列 2.2 寸 ~3.2 寸 . . . 1496
12.1.6.2 K0 系列 3.5 寸 ~7.0 寸 . . . 1496
12.1.7 T1 系列 . . . 1498
12.1.7.1 T1 系列 1.8 寸 ~2.8 寸 . . . 1498
12.1.7.2 T1 系列 3.5 寸 ~5.0 寸 . . . 1498
12.1.8 T0 系列（即将停产）不推荐新项目选型 . . . 1500
12.1.8.1 T0 系列 2.2 寸 ~3.5 寸 . . . 1500
12.1.8.2 T0 系列 4.3 寸 ~7.0 寸 . . . 1500

13 电赛专题 **1503**
13.1 电赛快速入门 . . . 1503
13.1.1 1、串口屏 . . . 1503
13.1.2 2、调试工具 . . . 1503
13.1.3 3、连接电脑 . . . 1504
13.1.4 4、上电 . . . 1505
13.1.5 5、串口屏资料下载 . . . 1505
13.1.6 6、驱动安装 . . . 1505
13.1.7 7、开发软件安装 . . . 1506
13.2 上位机使用入门 . . . 1507
13.2.1 1、新建工程 . . . 1507
13.2.2 2、文本控件显示数据 . . . 1508
13.2.3 3、动态修改文本 . . . 1515
13.2.4 4、下载工程 . . . 1518
13.3 串口屏调试入门 . . . 1520
13.3.1 sscom 调试串口屏实物 . . . 1520
13.3.2 模拟器接单片机 . . . 1522
13.3.3 串口屏接单片机 . . . 1522

备注： 本文档适用于淘晶驰所有的串口屏系列，包括 X5，X3，X2，K0，T1，T0 系列。

开发文档下载 (page 1094)

上位机下载 (page 1066)

本次资料中心更新时间：2026-05-13 17:26:41 ，版本号：1.1.0-2026-05-13 17:26:41

---

1 快速入门

1.1 下载和安装上位机软件

1. 下载上位机软件 USART HMI

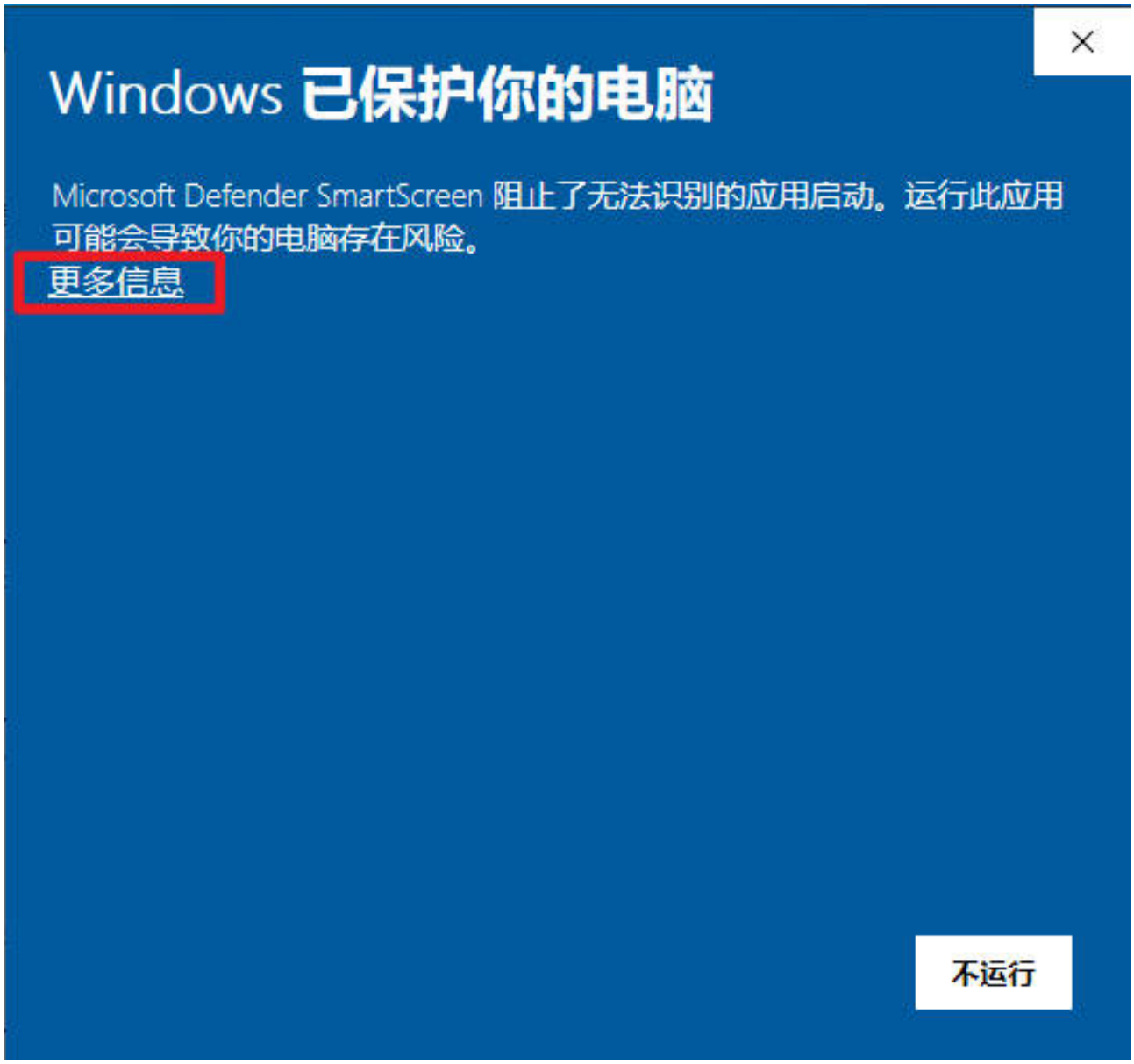
打开这个链接下载上位机开发软件 http://wiki.tjc1688.com/download/usart_hmi.html

---

2. 右键解压下载的压缩包，得到 exe 格式的可执行文件，右键点击安装

USAR
p_1
打开(O)
以管理员身份运行(A)
上传到迅雷云盘
通过 Code 打开
兼容性疑难解答(Y)
固定到"开始"屏幕(P)

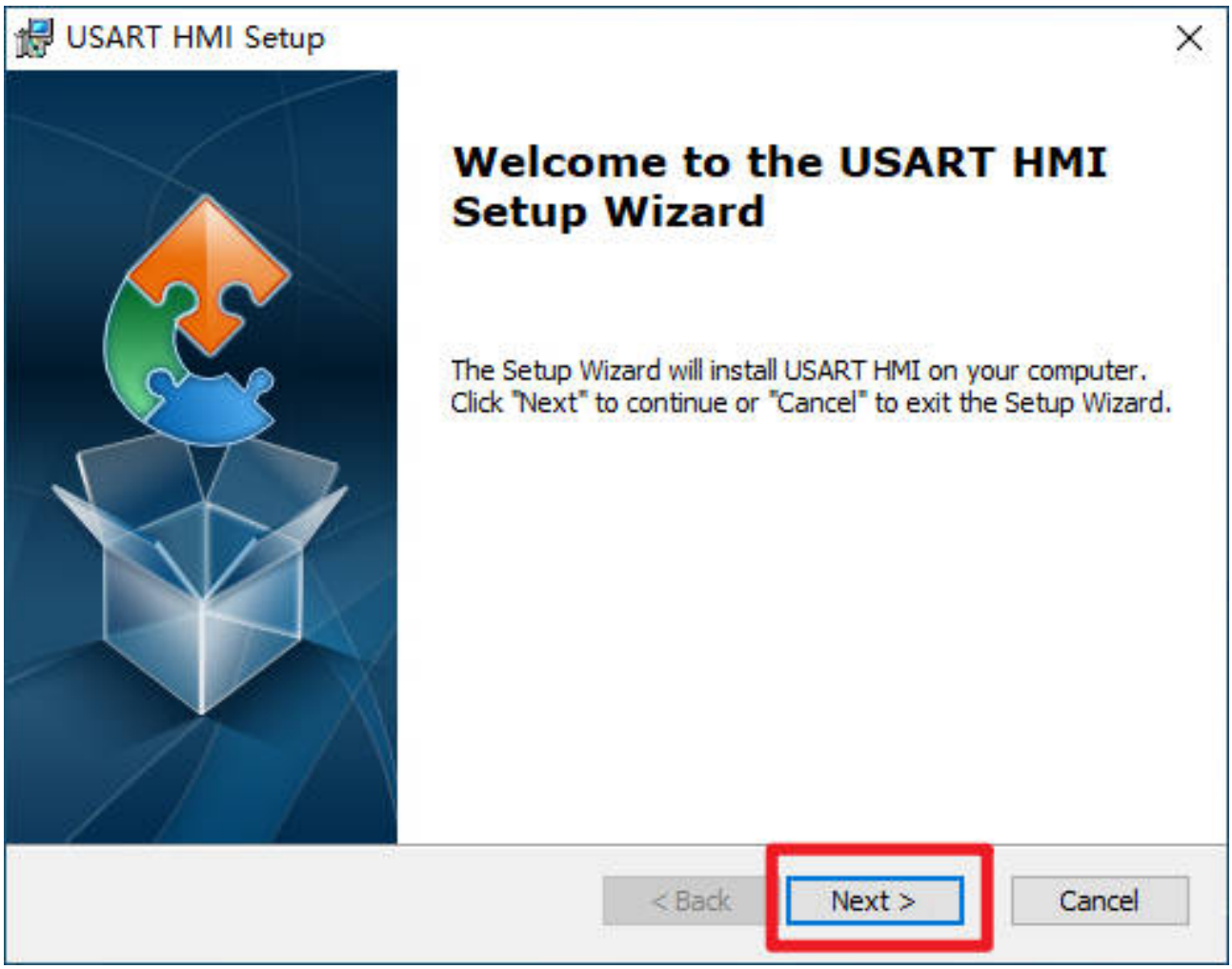
3. 点击更多信息


Windows 已保护你的电脑
Microsoft Defender SmartScreen 阻止了无法识别的应用启动。运行此应用可能会导致你的电脑存在风险。
更多信息
不运行

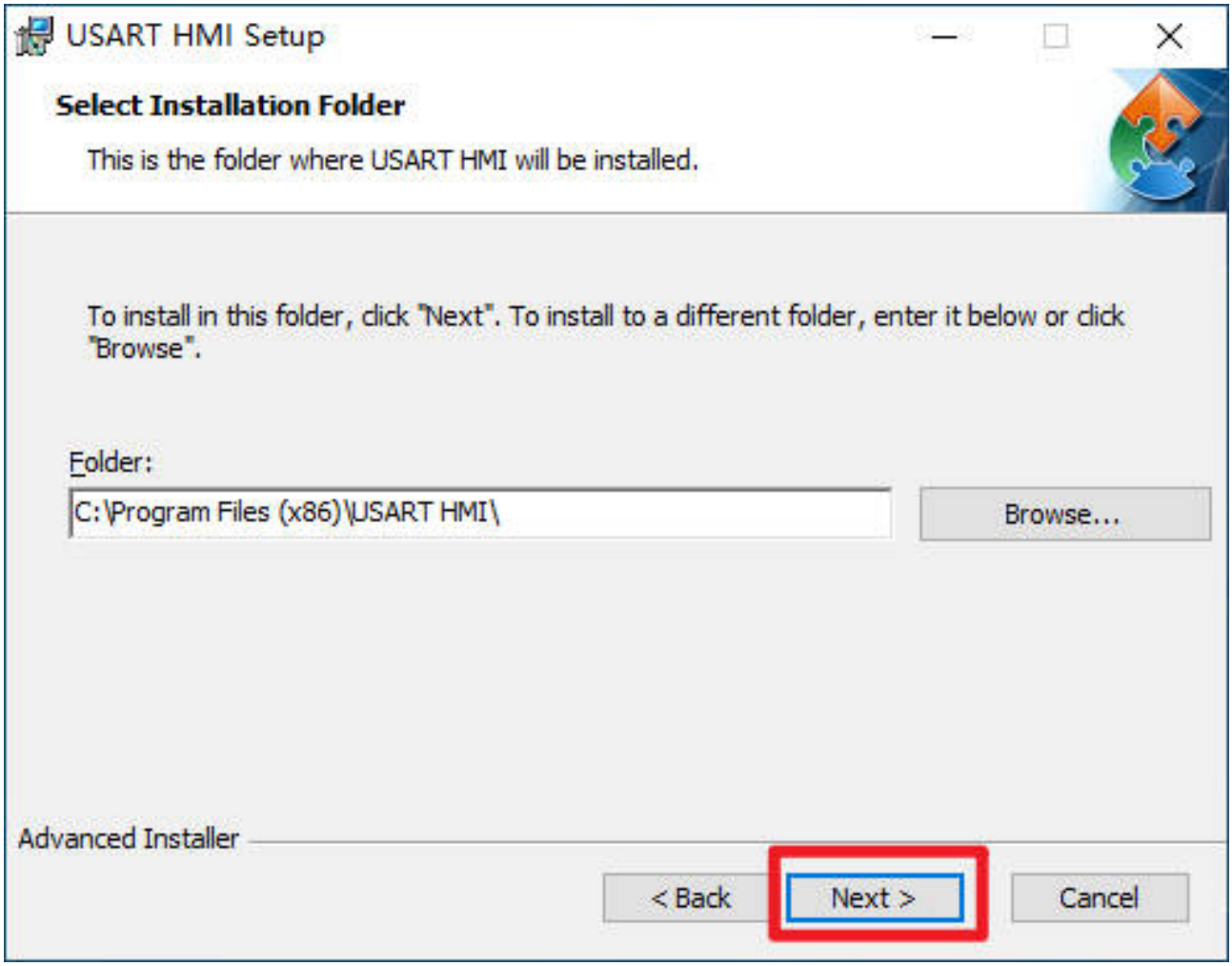
4. 点击仍要运行

Windows 已保护你的电脑
Microsoft Defender SmartScreen 阻止了无法识别的应用启动。运行此应用可能会导致你的电脑存在风险。
应用: USARTHMIsetup_1.64.1.exe
发行者: 发布者未知
仍要运行
不运行

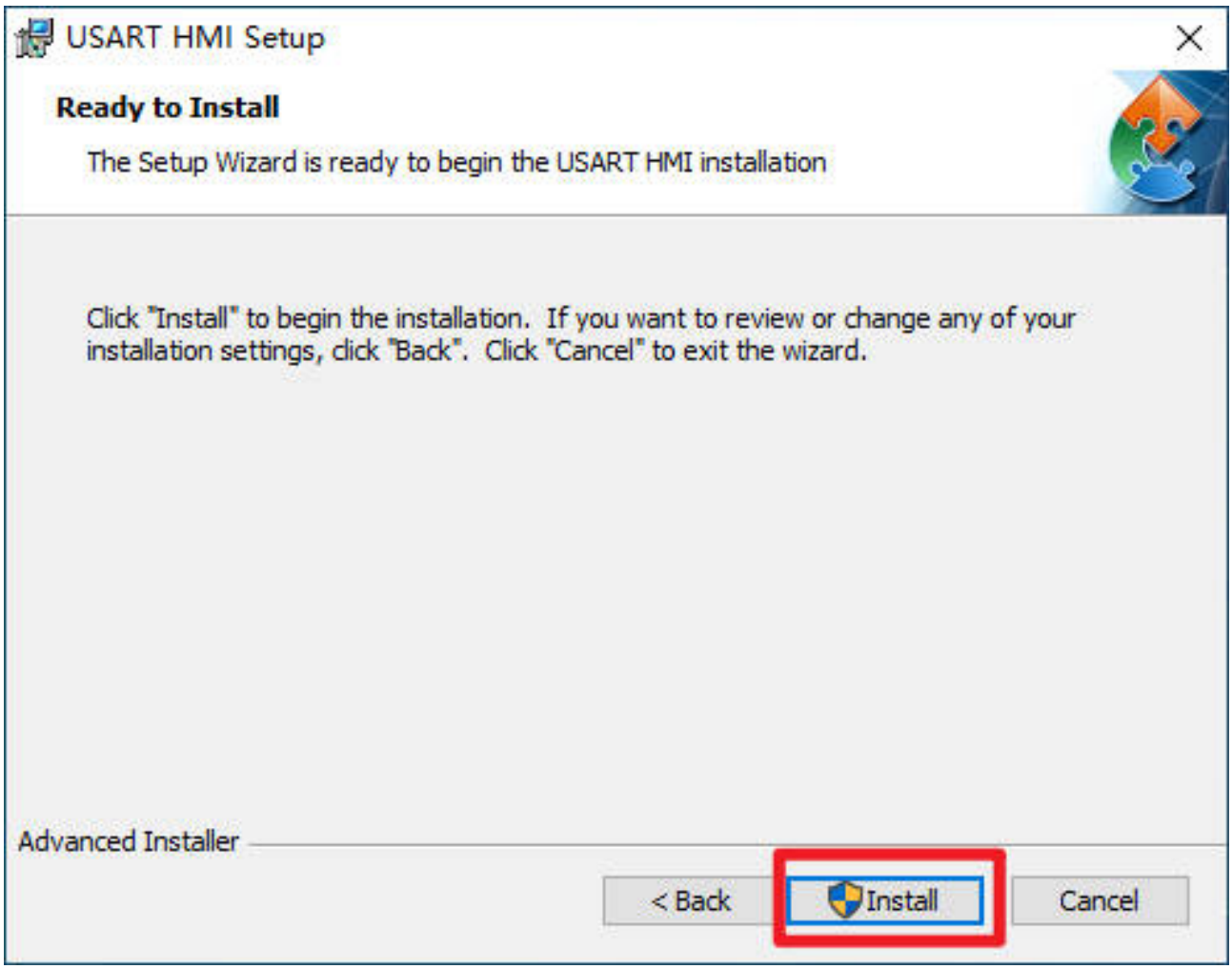
5. 点击 Next


USART HMI Setup
Welcome to the USART HMI Setup Wizard
The Setup Wizard will install USART HMI on your computer. Click "Next" to continue or "Cancel" to exit the Setup Wizard.
< Back
Next >
Cancel

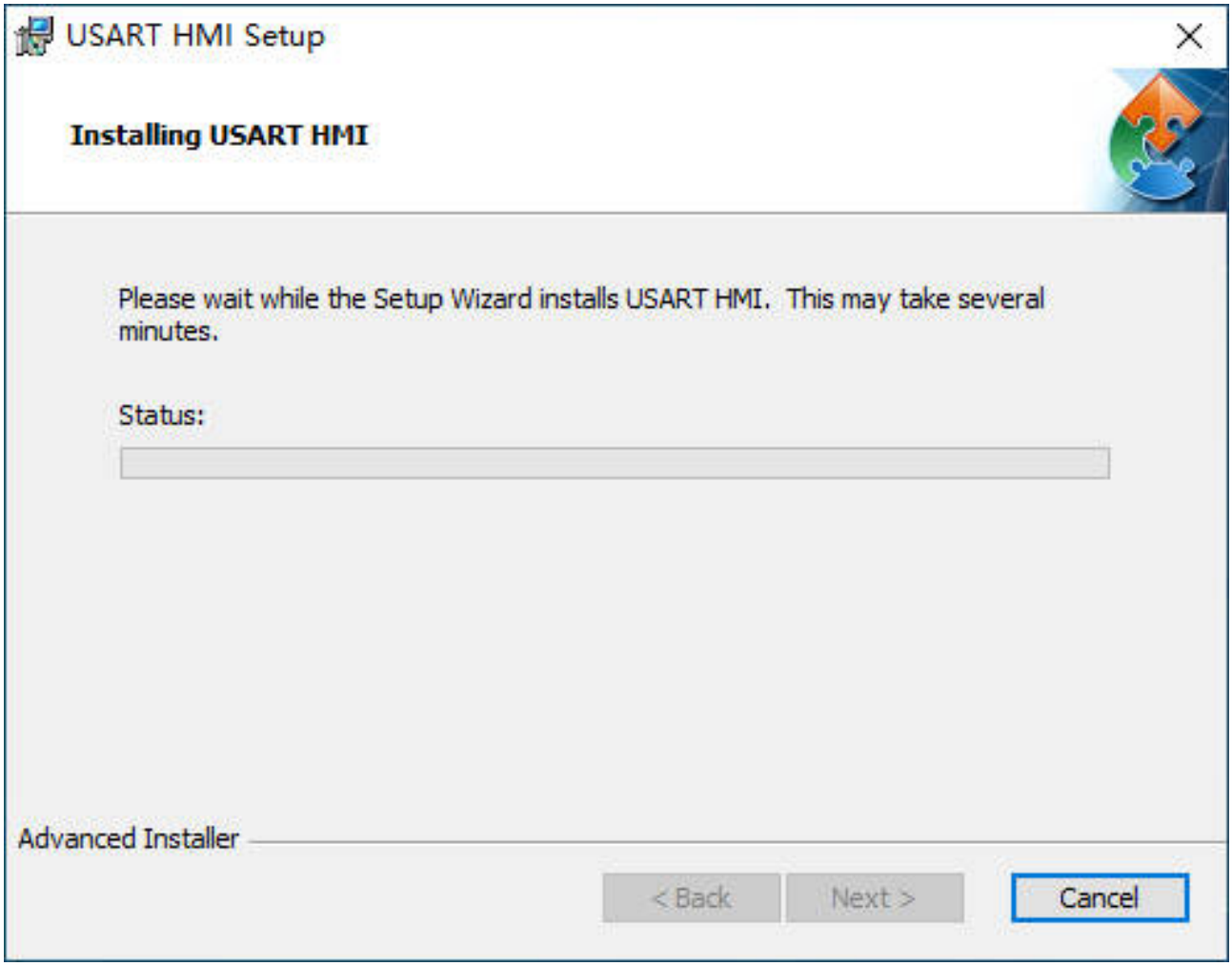
6. 此处建议使用默认的安装地址即可，点击 Next



7. 点击 Install


USART HMI Setup
Ready to Install
The Setup Wizard is ready to begin the USART HMI installation
Click "Install" to begin the installation. If you want to review or change any of your installation settings, click "Back". Click "Cancel" to exit the wizard.
Advanced Installer
< Back
Install
Cancel

8. 等待程序安装完成


USART HMI Setup
Installing USART HMI
Please wait while the Setup Wizard installs USART HMI. This may take several minutes.
Status:
Advanced Installer
< Back
Next >
Cancel

注意： win10 安装.NET Framework 3.5 目前需要联网，网上的离线安装包大部分都无法成功安装，联网在线安装是最快最稳定的方法

9. 部分电脑因为没有.NET Framework 3.5 环境，需要联网安装.NET Framework 3.5 环境，点击下载并安装此功能

如果需要离线安装请查看

Microsoft .net Framework 3.5 (page 1101)

10. 等待程序自动下载并安装

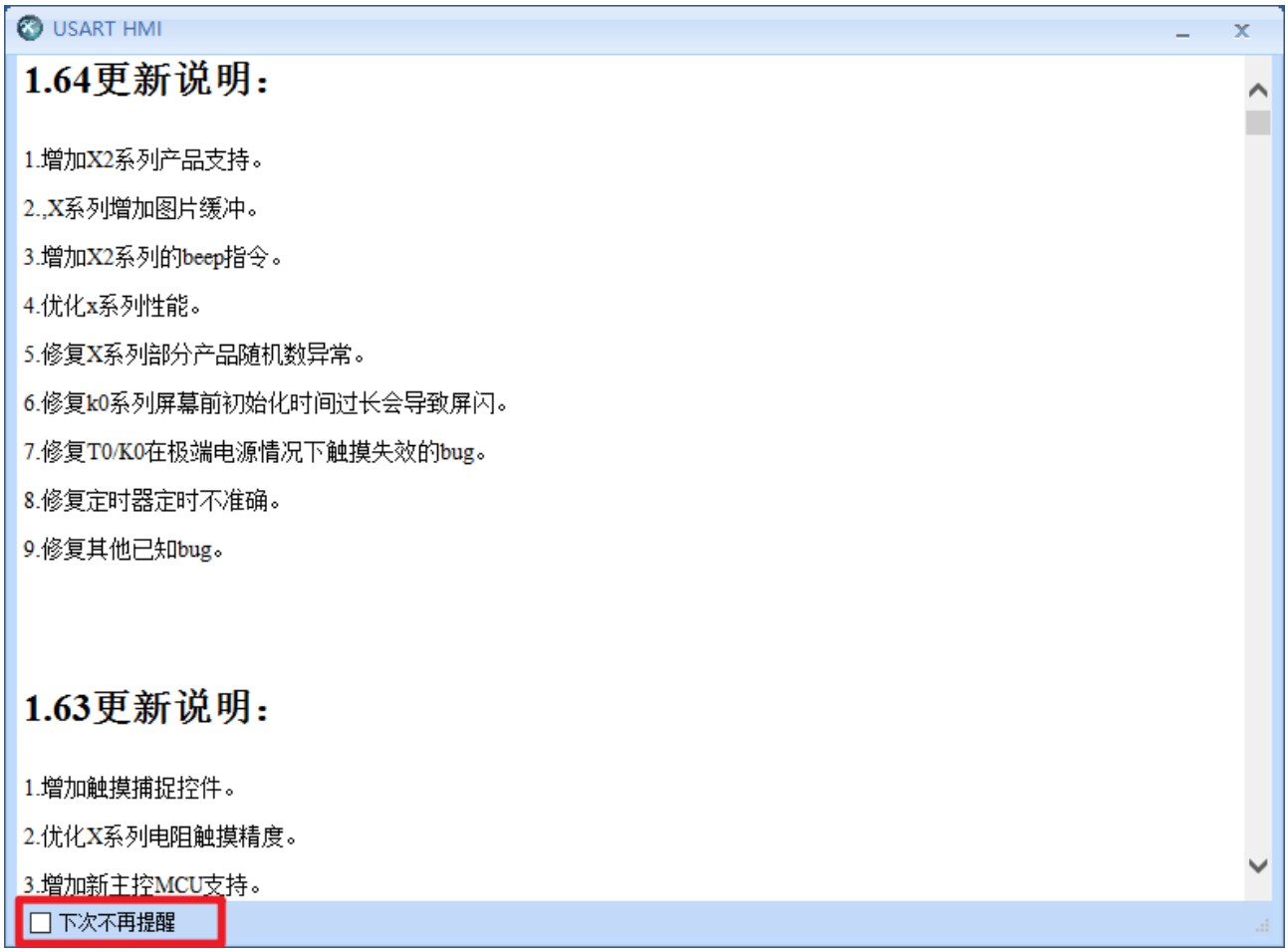
11. 安装完毕点击关闭

Windows 功能
已成功安装以下功能:
.NET Framework 3.5 (包括 .NET 2.0 和 3.0)
你可能需要重新启动那些需要使用此项功能的应用。
关闭

1.2 上位机基本功能介绍

双击桌面上的 USART HMI 图标即可打开上位机

小技巧： 首次打开时会弹出更新说明，可以点击左下角的下次不再提醒，永久关闭更新说明提醒框



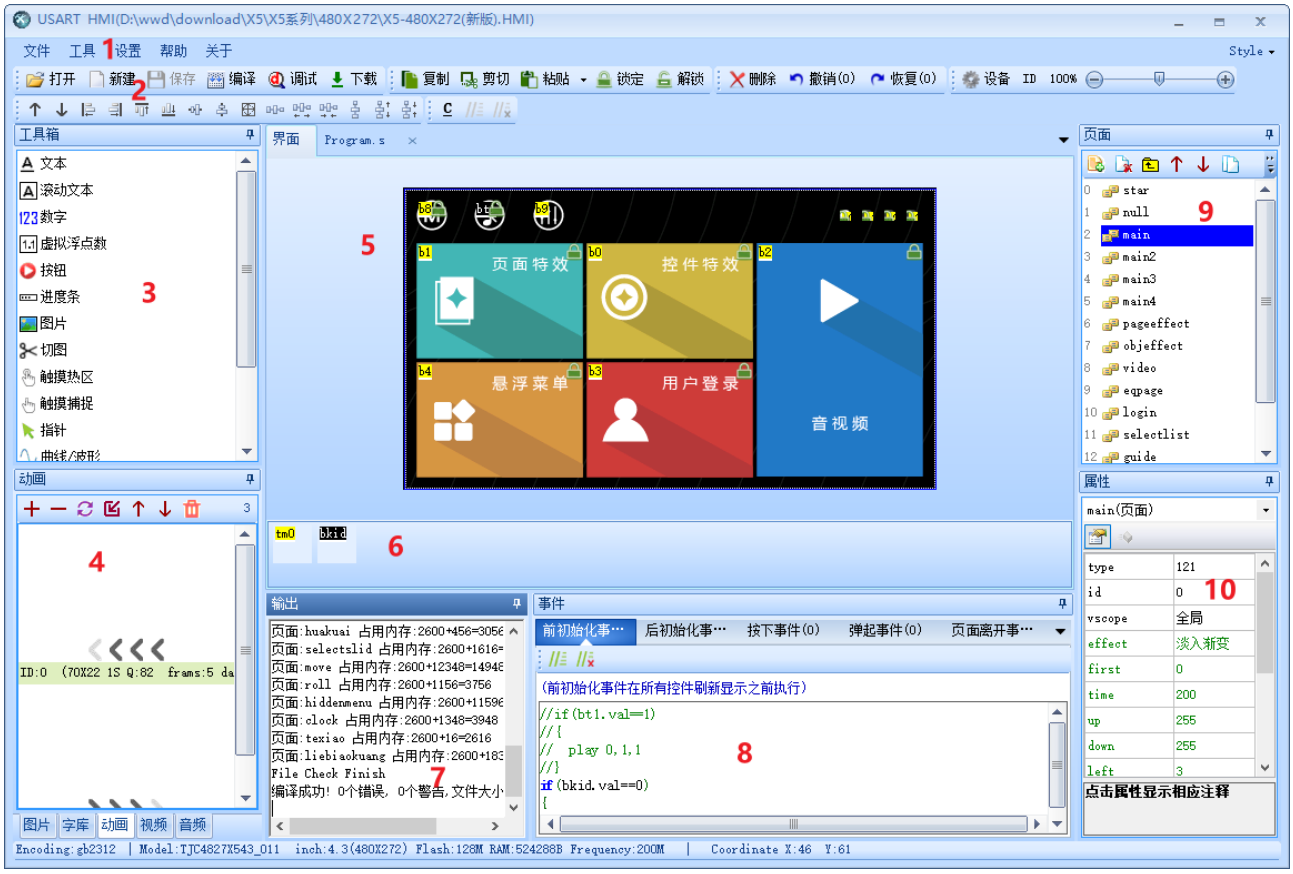
可以通过主界面上的提示来知道当前是否为最新版本，建议所有用户都更新到最新版本

USART HMI

版本：1.67.2 (已是最新版本)

提示： USART HMI 软件为淘晶驰智能串口屏集成开发环境。该集成开发环境包含串口屏界面设计，调试等界面开发软件的所有功能。下方截图使用的工程可以从淘晶驰资料中心-资料下载-标准出场工程样例-X5 系列出厂样例中找到

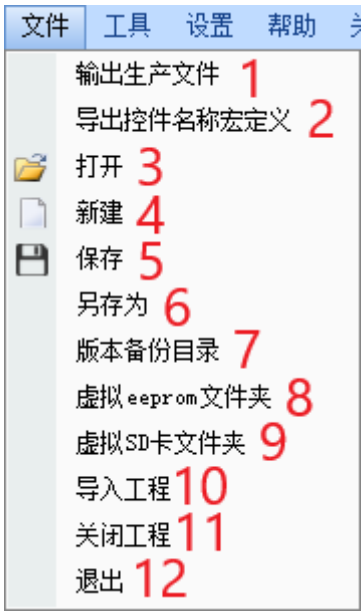
集成开发环境大致分为几大功能区，如下图所示。



提示： 以下部分较多，可以选择性观看，建议先看到手测试 (page 90) 以及创建工程 (page 105)

1.2.1 1. 菜单栏

1.2.1.1 文件菜单



1. 输出生产文件

该菜单用于输出编译生成的.tft 类型文件。.tft 文件用于做 TF 卡升级或外部设备串口升级使用。

1.1 生产输出的.tft 文件，与项目工程文件同名。

1.2 在弹出的《输出生产文件》窗口窗口中，我们可以选择一个指定的目录用于存放输出的.tft 文件。

1.3 选择《输出》按钮。将对项目进行编译后，生成新的.tft 文件。并打开《输出文件夹》。

注意: 如果《输出文件夹》已经存在同名的.tft 文件，新生成的文件会覆盖旧文件。

注意: 如果项目编译失败，将不会打开《输出文件夹》。请留意输出窗口的错误提示。

1.4 《仅打开输出文件夹》。不编译当前项目工程，直接打开《输出文件夹》。

小技巧: 技巧：如果仅仅是想打开上次编译生成的文件，可以执行该操作。

---

2. 导出控件名称宏定义
导出页面编号和控件编号的宏定义 (.h 文件, 用于导入单片机工程中使用)

3. 打开
打开一个.HMI 工程文件。

4. 新建
新建一个.HMI 工程文件。

5. 保存
保存当前正在编辑的工程文件。

6. 另存为
将当前编辑的工程文件，另存为一个副本。

7. 版本备份目录
打开 backup 文件夹。backup 文件夹下保存的工程文件为使用新版本 USART HMI 软件第一次打开旧版本工程文件之前自动备份的工程文件。

8. 虚拟 eeprom 文件夹
打开 eeprom 文件夹。eeprom 文件夹保存的文件是 USART HMI 软件在模拟运行工程中生成的虚拟 eeprom 文件。

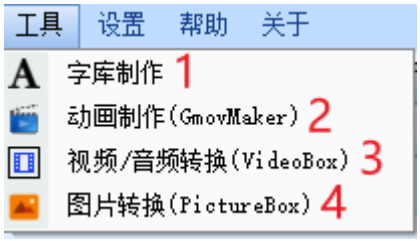
9. 虚拟 SD 卡文件夹
打开虚拟 SD 卡文件夹。虚拟 SD 卡文件夹保存的文件是 USART HMI 软件在模拟运行工程中使用到 SD 卡内的文件。

10. 导入工程
将会导入另一个工程的所有页面。

11. 关闭工程
关闭当前正在编辑的工程文件。

12. 退出
退出 USART HMI 集成开发环境。

1.2.1.2 工具菜单



2.1 字库制作

《字库制作》工具是 USART HMI 开发环境使用的专用字库制作工具。

参考：

创建字库和导入字库 (page 125)

指定字库 (page 1284)

在做字库的时候有部分字体无法选择 (page 1360)

编译报错：*XXX* 初始值无效 (page 1342)

utf8 字库下如何只选择汉字 (page 1403)

报错:*file is too large for destination device* (page 1359)

2.2 动画制作 (GmovMaker)

《动画制作》工具是 USART HMI 开发环境使用的专用动画制作工具。

参考：

动画控件 (page 536)

2.3 视频/音频转换 (VideoBox)

《视频/音频转换》工具是 USART HMI 开发环境使用的专用视频/音频制作工具。

参考：

视频控件 (page 549)

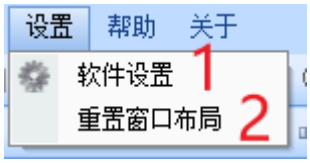
2.4 图片转换

《图片转换》工具是 USART HMI 开发环境使用的用于外部图片控件的工具

参考：

外部图片控件 (page 568)

1.2.1.3 设置菜单



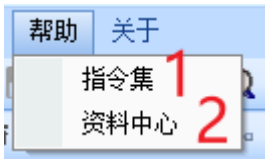
3.1 软件设置

在该选项打开的窗口中，可以根据个人喜好对开发环境的各种参数进行配置。

3.2 重置窗口布局

将开发环境的各项配置参数，窗口布局恢复到默认状态。

1.2.1.4 帮助菜单



4.1 指令集

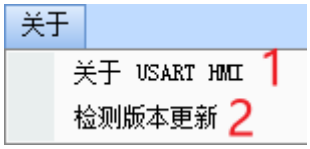
该菜单可以打开一个本地文档，该文档包含串口屏的关键文档描述。方便无法连接互联网的客户查阅资料。

> **注意：** 强烈建议客户使用在线版本资料中心的资料。因为离线版本并没有包含本资料中心的所有资源文档，同时可能由于资料更新不及时出现部分功能描述与软件功能不一致。

4.2 USART HMI 资料中心

该菜单会链接到在线网页版本的 USART HMI 资料中心。

1.2.1.5 关于菜单



1.5.1 关于 USART HMI

该菜单用于查看软件版本信息。

1.5.2 检测版本更新

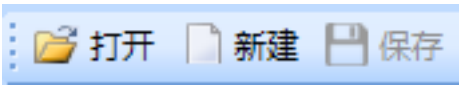
该菜单用于连接服务器，检测软件是否有新版本。

1.2.2 2. 工具栏

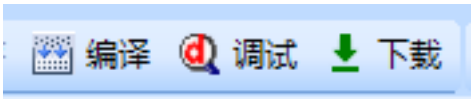
1. 文件操作快捷菜单。

如下图所示，快捷操作功能同文件操作菜单。

分别为打开工程、新建工程、保存当前工程。



2. 项目操作菜单



2.1 编译

对当前的工程文件进行编译操作。并将结果显示在输出窗口中。

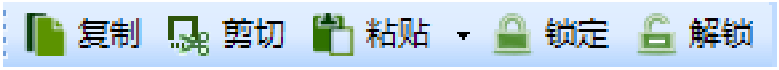
2.2 调试

对编译成功的项目进行模拟运行操作。该功能可以不需要借助实物串口屏快速的验证项目工程的功能是否符合预期需求。具体调试操作请参考调试窗口。

2.3 下载

将编译生成的.tft 文件下载到实物串口屏中。

3. 控件编辑菜单



3.1 控件复制

对界面编辑区中处于选中状态的按钮复制到粘帖板中。也可以使用 Crtl+C 快捷键执行相同操作。

3.2 控件剪切

对界面编辑区中处于选中状态的按钮剪切到粘帖板中。也可以使用 Crtl+X 快捷键执行相同操作。

3.3 控件粘帖

将剪切板中的控件粘帖到界面编辑区。也可以使用 Crtl+V 快捷键执行相同操作。

3.4 控件锁定

选择控件以后，通过点击锁定/解锁按纽来实现控件的锁定/解锁功能。控件被锁定以后，将再控件的右上角显示一个《锁》图标。被锁定的控件，不能通过鼠标拖动控件修改控件位置，和鼠标拖动修改控件的大小。但是仍然可以选中控件后通过修改属性区域的 x，y，w，h 属性来调整控件的位置和大小。该功能能有效防止防止因误操作修改了已经布局好的界面。

3.5 控件解锁

解锁被锁定的控件。

4. 控件操作

4.1 控件删除

对界面编辑区中处于选中状态的控件执行删除操作。

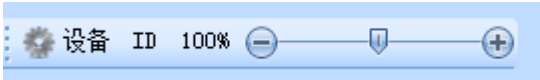
4.2 控件撤销

撤销上一次在界面编辑区的操作。

4.3 控件恢复

恢复上一次在界面编辑区的操作。

5. 界面设置



5.1 设备按钮

打开设备选择窗口。可以配置产品系列，型号以及屏幕显示方向等。具体请参考本章节设备窗口 (page 86)。

5.2 ID 按钮

点击该按钮，可以切换显示/不显示界面区域控件名称。

5.3 缩放比例

可以对界面区域的界面编辑内容进行放大/缩小调节。范围为 10%-600%。方便界面布局的时候，精确摆放控件的位置和调节控件的大小。

小技巧： 可以点击当前的百分比数值，快速将显示比例调整为默认的 100%。

---

6. 控件图层调节

图层调节，主要用于在多个控件存在重叠区域时的现实层级关系。一个页面上的所有控件都拥有不同的图层，可以通过图层调节按钮来调节控件的图层关系。

6.1 该按钮↑用于将控件置于所有控件最顶层。

6.2 该按钮↓用于将控件置于所有控件最顶层。

注意： T0，K0 系列串口屏，只有在页面初始化的时候，具体图层功能。再触摸屏运行过程中，低层级的控件刷新后，将被置于顶层。X 系列串口屏具有图层保持功能，在运行过程中能一直保持图层级别。

注意： 使用图层调节按钮，调节控件的图层优先级功能会导致控件 ID 号变化，如果需要用到名称组等功能，需要特别小心！

7. 控件对齐

7.1 左对齐

选中多个控件，蓝色的控件为主控件，所有控件以此控件为基础进行左对齐。

7.2 右对齐

选中多个控件，蓝色的控件为主控件，所有控件以此控件为基础进行右对齐。

7.3 上对齐

选中多个控件，蓝色的控件为主控件，所有控件以此控件为基础进行上对齐。

7.4 下对齐

选中多个控件，蓝色的控件为主控件，所有控件以此控件为基础进行下对齐。

8. 控件大小

8.1 使宽度相等

选中多个控件，蓝色的控件为主控件，所有控件以此控件为基础进行控件宽度对齐。

8.2 使高度相等

选中多个控件，蓝色的控件为主控件，所有控件以此控件为基础进行控件高度对齐。

8.3 使大小相等

选中多个控件，蓝色的控件为主控件，所有控件以此控件为基础进行控件大小对齐。

9. 控件水平间距

9.1 使水平间距相等

选中多个控件，蓝色的控件为主控件，所有控件以此控件为基础使水平间距相等。

9.2 增加水平间距

选中多个控件，蓝色的控件为主控件，所有控件以此控件为基础增加水平间距。

9.3 减小水平间距

选中多个控件，蓝色的控件为主控件，所有控件以此控件为基础减小水平间距。

10. 控件垂直间距

10.1 使垂直间距相等

选中多个控件，蓝色的控件为主控件，所有控件以此控件为基础使垂直间距相等。

10.2 增加垂直间距

选中多个控件，蓝色的控件为主控件，所有控件以此控件为基础增加垂直间距。

10.3 减小垂直间距

选中多个控件，蓝色的控件为主控件，所有控件以此控件为基础减小垂直间距。

1.2.3 3. 工具箱

工具箱
文本
滚动文本
数字
虚拟浮点数
按钮
进度条
图片
切图
触摸热区
触摸捕捉
指针
曲线/波形
滑块
定时器
变里
双态按钮
复选框
单选框
二维码

工具箱窗口中列有串口屏支持的所有控件列表。用户需要使用各种控件是直接将对应的控件拖放放界面编辑区域。各个控件的详细使用方法请参考控件详解 (page 1239) 章节。

1.2.3.1 控件列表

文本控件 (page 405)

滚动文本控件 (page 412)

数字控件 (page 417)

虚拟浮点数控件 (page 423)

按钮控件 (page 431)

进度条控件 (page 439)

图片控件 (page 442)

切图控件 (page 444)

触摸热区控件 (page 448)

触摸捕捉控件 (page 451)

指针控件 (page 455)
曲线波形控件 (page 459)
滑块控件 (page 466)
定时器控件 (page 471)
变量控件 (page 478)
双态按钮控件 (page 480)
复选框控件 (page 484)
单选框控件 (page 487)
二维码控件 (page 489)
状态开关控件 (page 491)
下拉框控件 (page 496)
选择文本控件 (page 501)
滑动文本控件 (page 505)
数据记录控件 (page 509)
文件浏览器控件 (page 524)
文件流控件 (page 529)
动画控件 (page 536)
视频控件 (page 549)
音频控件 (page 559)
外部图片控件 (page 568)

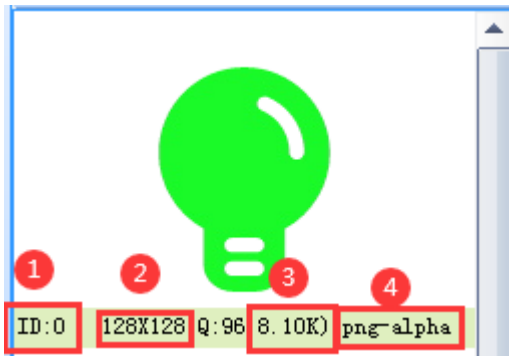
1.2.4 4. 资源文件窗口

资源文件窗口存放工程所调用到资源。其中动画，音频仅 x 系列支持，视频仅 x5 系列支持。

1.2.4.1 图片资源

图片资源：t0 t1 系列支持 jpg bmp；k0 系列支持 jpg bmp；x 系列支持 jpg png bmp；

如图所示, 可以使用相关按钮进行图片添加，删除，替换，插入，上移，下移，全删。



1. 图片 id

控件背景调用图片使用到的 id。

2. 图片分辨率

最小图片分辨率为：2 * 2 像素

最大图片分辨率为当前屏幕的分辨率，例如 TJC8048X550_011C，分辨率为 800 * 480，则最大的图片分辨率也是 800 * 480，也就是全屏图片

3. 图片压缩质量

压缩质量越大，图片显示效果越好，但是占用的空间会更大，x5,x3,t1 系列支持图片压缩储存，k0,t0 系列不支持图片压缩储存

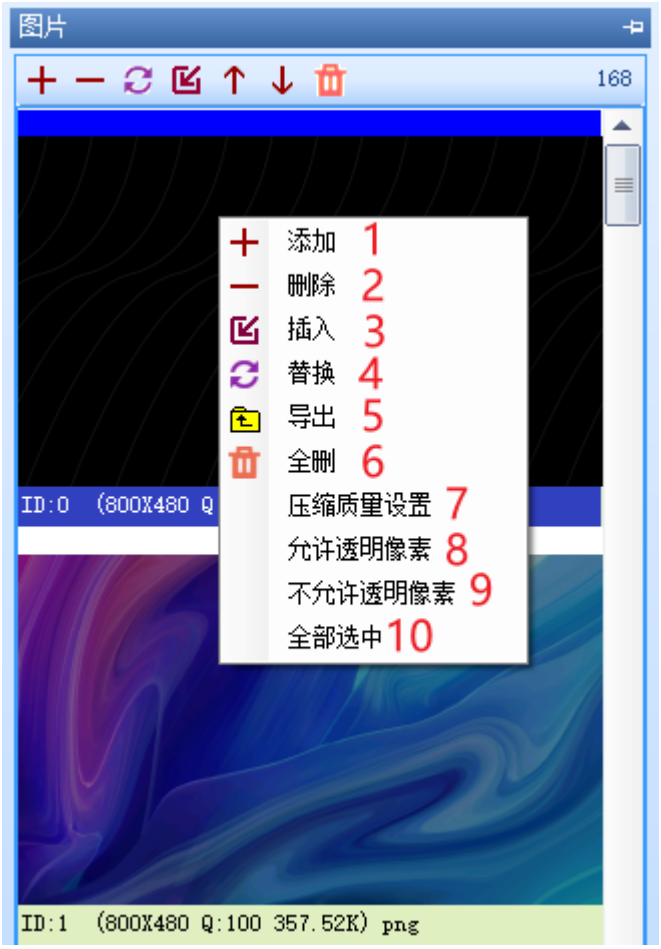
4. 图片大小

导入后的图片占用的空间可能变大也有可能变小

5. 图片格式

导入图片的图片格式。

1.2.4.2 图片资源右键菜单



1~6 为添加，删除，插入，替换，导出，全删

7. 压缩质量设置

仅 x3、x5 系列支持，当导入图片后该图片占用的存储空间大于 400KB 时，请调低压缩质量，否则可能导致图片无法显示。

8. 允许透明像素

仅 x 系列支持。

9. 不允许透明像素

仅 x 系列支持。

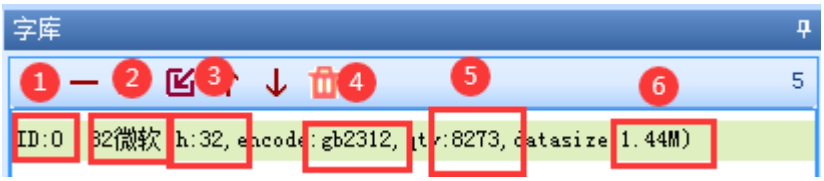
10. 全部选中

选择所有图片。

1.2.4.3 字库资源

小技巧: 字库：可以通过工具菜单中的《字库制作》工具制作

如图所示, 可以使用相关按钮进行字库添加，删除，替换，插入，上移，下移，全删。



1. 字库 id
控件字库调用字库使用到的 id。

2. 字库名称
生成字库的时候进行命名。

3. 字体大小
该字库的字体大小。

4. 字体字符编码
该字库使用的字符编码。

5. 字符数量
字库所包含的字符数量。

6.flash 空间
字库所占 flash 空间大小。

1.2.4.4 字库资源右键菜单

1~6 为添加，删除，插入，替换，导出，全删

7. 全部选中
选择所有字库。

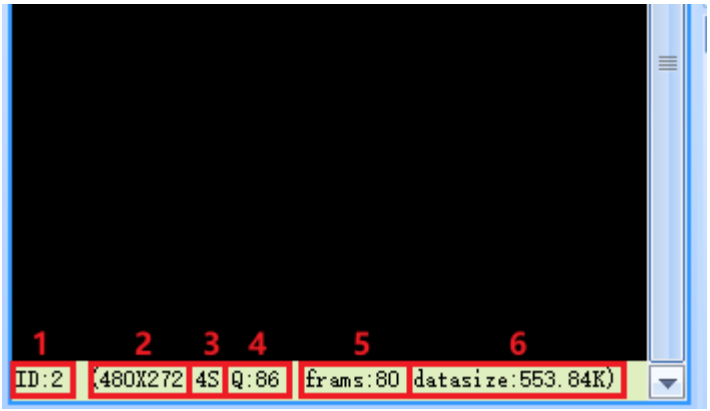
参考:
创建字库和导入字库 (page 125)

指定字库 (page 1284)
在做字库的时候有部分字体无法选择 (page 1360)
编译报错：*XXX* 初始值无效 (page 1342)
utf8 字库下如何只选择汉字 (page 1403)
报错:*file is too large for destination device* (page 1359)

1.2.4.5 动画资源

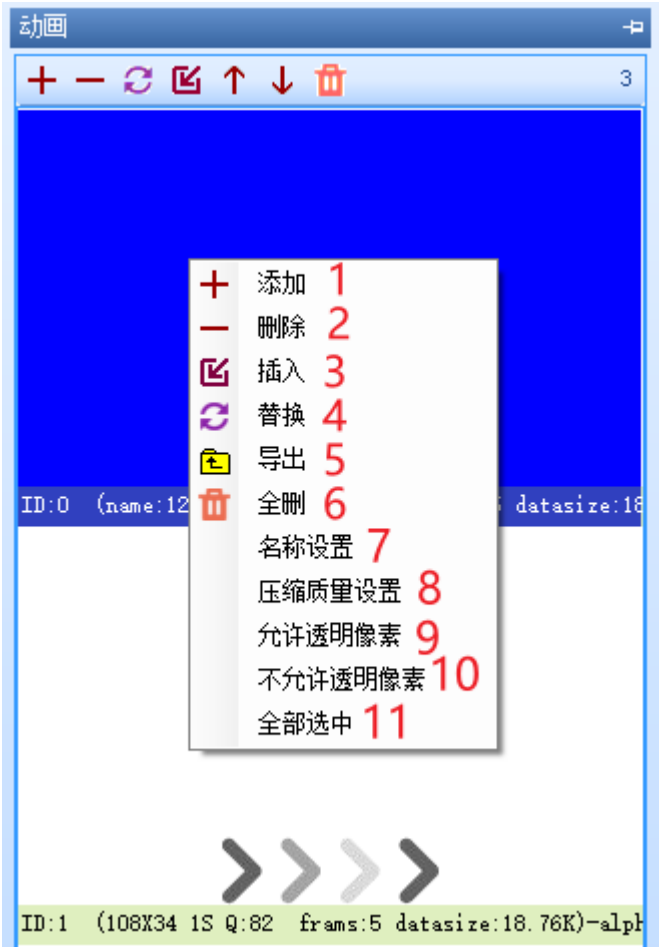
动画：可以通过工具菜单中的动画制作，仅 x3 和 x5 系列支持

如图所示, 可以使用相关按钮进行动画添加，删除，替换，插入，上移，下移，全删。



1. 动画 id，动画控件调用动画使用到的 id。
2. 动画分辨率，导入动画分辨率超出屏幕显示支持最大分辨率将会报错。
3. 动画时长。
4. 动画压缩质量。
5. 动画帧数。
6. 动画所占 flash 空间大小，转换后的动画占用的空间可能变大也有可能变小。

1.2.4.6 动画资源右键菜单



1~6 为添加，删除，插入，替换，导出，全删

7. 名称设置

相当于设置备注。

8. 压缩质量设置

设置压缩质量，当文件过大时，降低压缩质量可以有效降低文件体积，但是会降低动画的显示效果。

9. 允许透明像素

仅 x 系列支持。

10. 不允许透明像素

仅 x 系列支持。

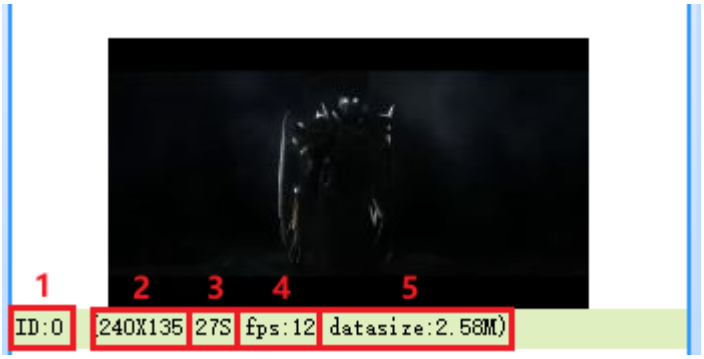
11. 全部选中

选择所有动画。

1.2.4.7 视频资源

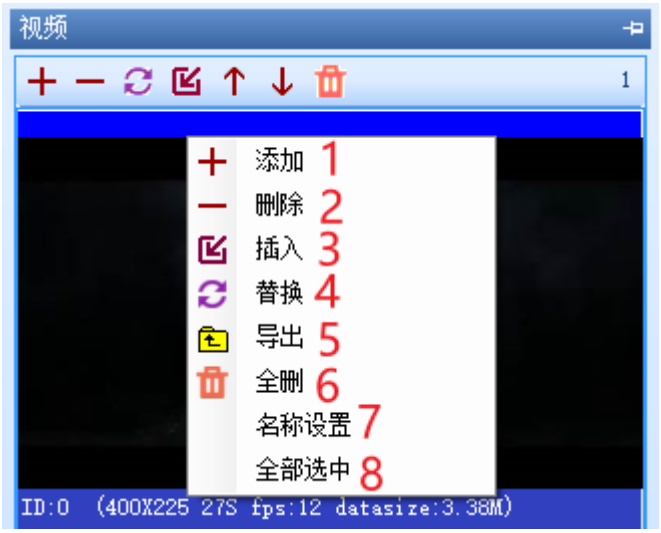
小技巧： 视频：可以通过工具菜单中的视频/音频转换

如图所示, 可以使用相关按钮进行视频添加，删除，替换，插入，上移，下移，全删。



1. 视频 id，视频控件调用视频使用到的 id。
2. 视频分辨率，导入动画分辨率超出屏幕显示支持最大分辨率将会报错。
3. 视频时长。
4. 视频 fps。
5. 视频所占 flash 空间大小，转换后的视频占用的空间可能变大也有可能变小。

1.2.4.8 视频资源右键菜单



1~6 为添加，删除，插入，替换，导出，全删

7. 名称设置
相当于设置备注。

8. 全部选中
选择所有视频。

1.2.4.9 音频资源

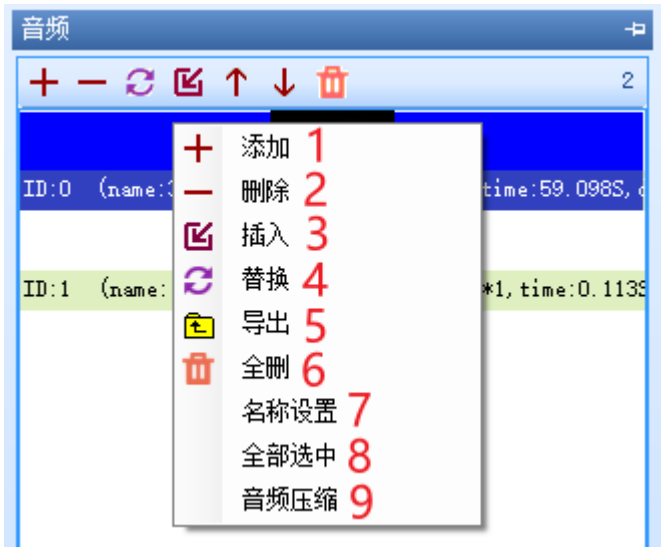
小技巧： 音频：可以通过工具菜单中的《视频/音频》转换

如图所示，可以使用相关按钮进行音频添加，删除，替换，插入，上移，下移，全删。

1. 音频 id，音频控件调用音频使用到的 id。
2. 音频名称，生成音频的时候进行命名。
3. 音频采样率
4. 音频时长。
5. 音频所占 flash 空间大小，转换入后的音频占用的空间可能变大也有可能变小。

注意： 音频格式必须用上位机自带的《视频/音频转换》转换为 wav 格式。

1.2.4.10 音频资源右键菜单



1~6 为添加，删除，插入，替换，导出，全删

7. 名称设置
相当于设置备注。

8. 全部选中
选择所有音频。

9. 音频压缩
修改音频的采样率。

1.2.5 5. 界面

1.2.5.1 界面窗口

界面窗口是 UI 编辑的主要工作区域，该区域可以被分为两部分。

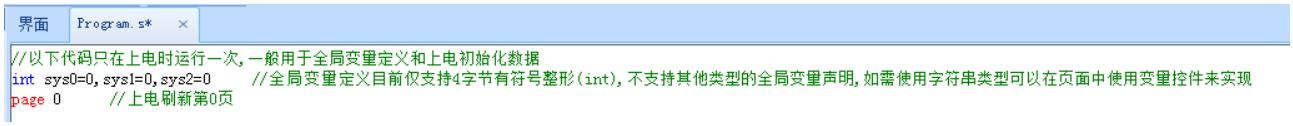
如上图所示，中间一个与当前闲着设备型号分辨率相同的区域为设备运行的可显示区域，超出该范围的为设备运行不可显示区域。

t 系列/.k 系列控件只能放在可显示区域，设置非显示区域软件编译会报错。

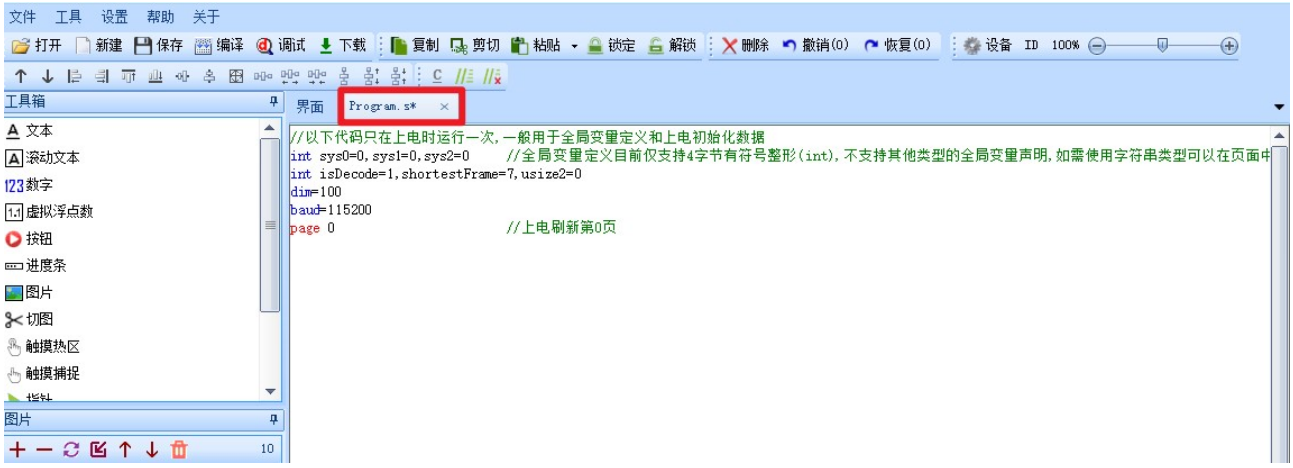
x 系列 x，y 属性可以超出屏幕坐标范围 (-6000~+6000)，但是超出屏幕将被裁减，不会显示出来。

可显示区域，以左上角作为坐标原点。分别向右（X 轴）和向下（Y 轴）增长，增长步进为 1 像素点。

1.2.5.2 program.s 详解



program.s 文件的位置如图所示



可以在 program.s 中使用 int 来定义有符号的 4 字节全局变量（一定是全局变量），不能在这里定义数组或者字符串。

需要使用数组请参考*数组/名称组使用说明* (page 865)

需要使用字符串请使用文本控件或者将变量控件设置为字符串

注意： 全局变量定义必须放在 Program.c 文件中所有代码之前

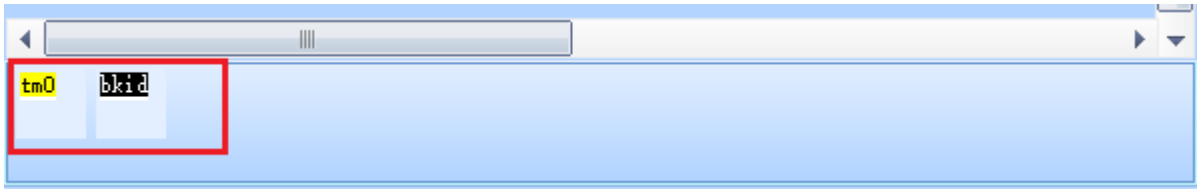
可以在 program.s 配置亮度，波特率，bkcmd 等系统变量

page 指令应写在 program.s 的最后，因为 page 指令后面的代码是不会执行的

如何配置亮度，主动解析，波特率 (page 1306)

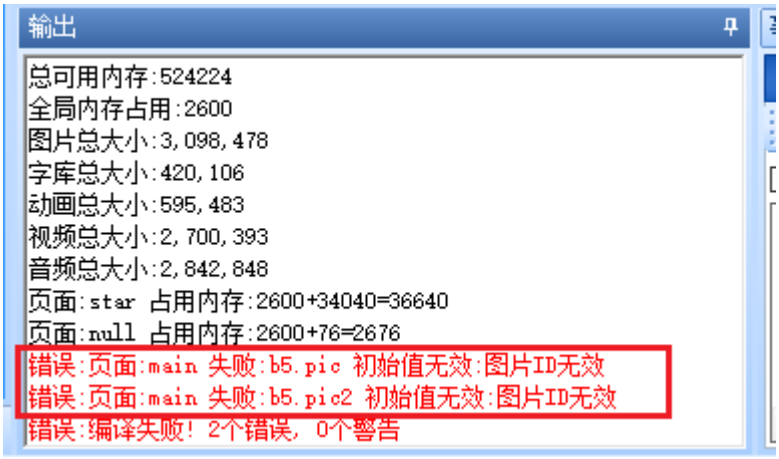
注意： 代码不要写 page 指令后面，page 指令后面的程序是不执行的。

1.2.6 6. 特殊控件窗口



特殊控件窗口位于界面窗口的下方。用于显示当前页面的变量控件、定时器控件等无法在界面区域显示的特殊控件。如果当前页面没有该类无法在界面显示的特殊控件，该窗口可能会被开发环境自动隐藏。

1.2.7 7. 输出窗口



输出窗口主要用于输出编译结果信息。

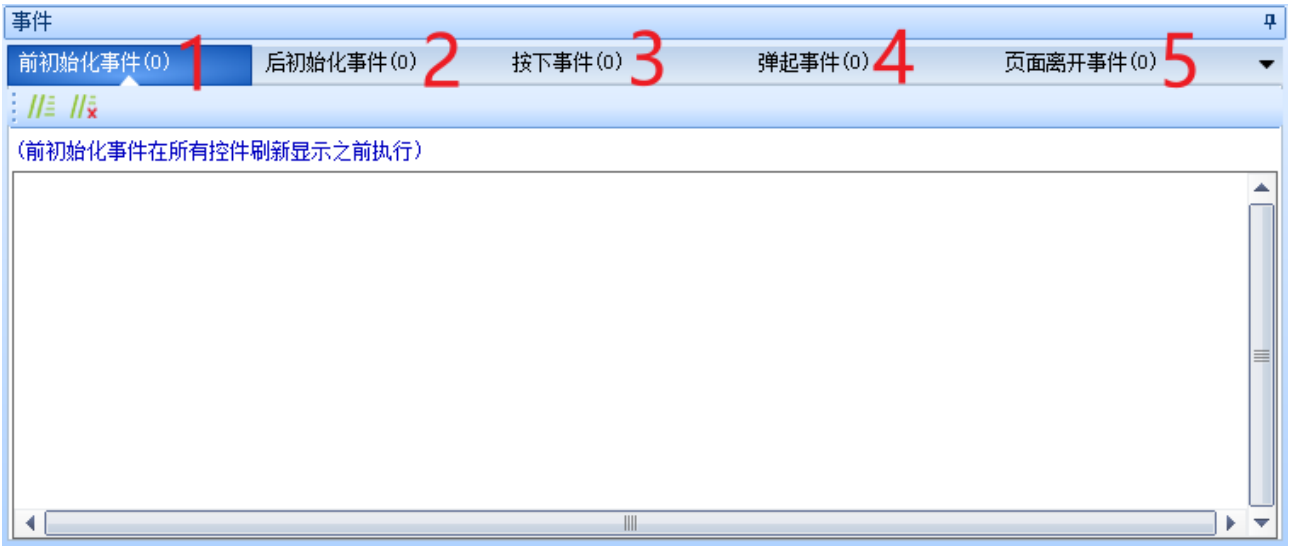
如果项目正常编译成功，窗口中会列出项目工程的各类资源占用详细信息。

如果项目编译失败，窗口中会列出错误信息。如果是事件代码错误，可以双击错误信息可以定位到错误代码位置。如果是提示字库 ID 无效，图片 ID 无效等资源文件错误，请按照提示查找缺少的资源项目，添加对应的资源文件或者修改控件属性不使用该资源。

1.2.8 8. 事件编辑窗口

如上图所示，串口屏有多种触发事件，每种事件都可以根据情况编写不同的代码。可以根据实际功能需求选择对应的事件类型。

1.2.8.1 页面事件详解



1. 前初始化事件

每次在执行 page 命令时，在执行页面刷新操作以前，串口屏会自动执行一次《前初始化事件中》的代码。

需要从其他页面同步数据过来时, 建议把同步代码放到前初始化事件中, 可以避免用户看到数据刷新过程

2. 后初始化事件

每次在执行 page 命令时，在页面刷新操作完成以后，串口屏会自动执行一次《后初始化事件》中的代码。

3. 页面按下事件

显示区域内且没有控件的区域，触摸被按下的瞬间，串口屏会自动执行一次页面的《按下事件》中的代码。

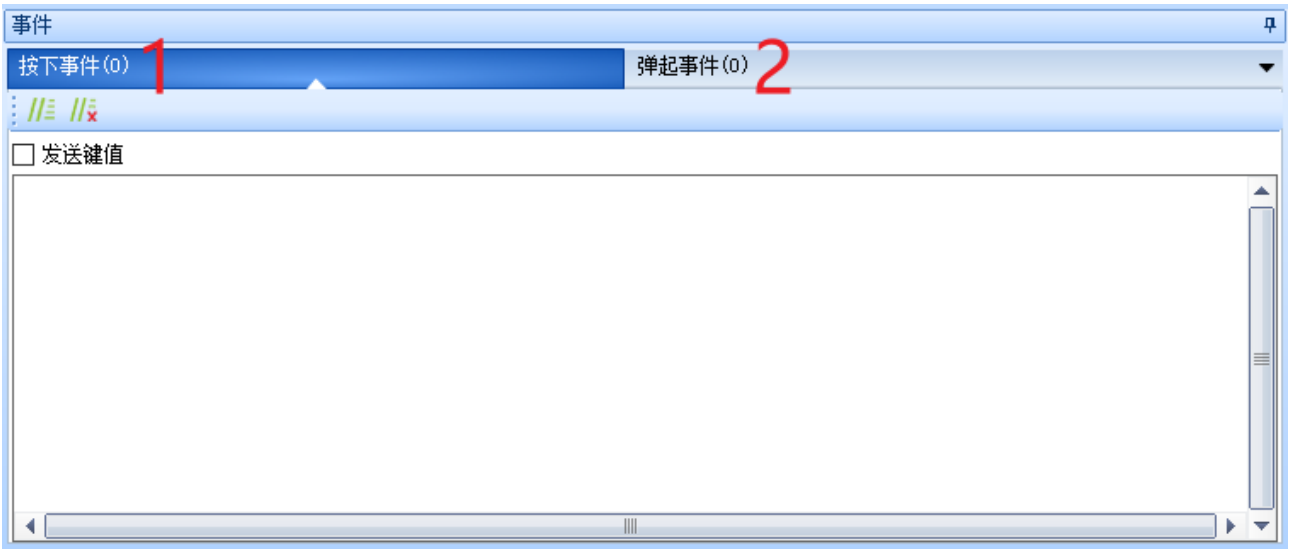
4. 页面弹起事件

显示区域内且没有控件的区域（以触摸按下时的坐标为准），触摸按下以后松开触摸的的瞬间，串口屏会自动执行一次页面的《弹起事件》中的代码。

5. 页面离开事件

每次在执行 page 切换新的页面前，串口屏会自动执行一次当前页面的《页面离开事件》中的代码。

1.2.8.2 控件事件详解

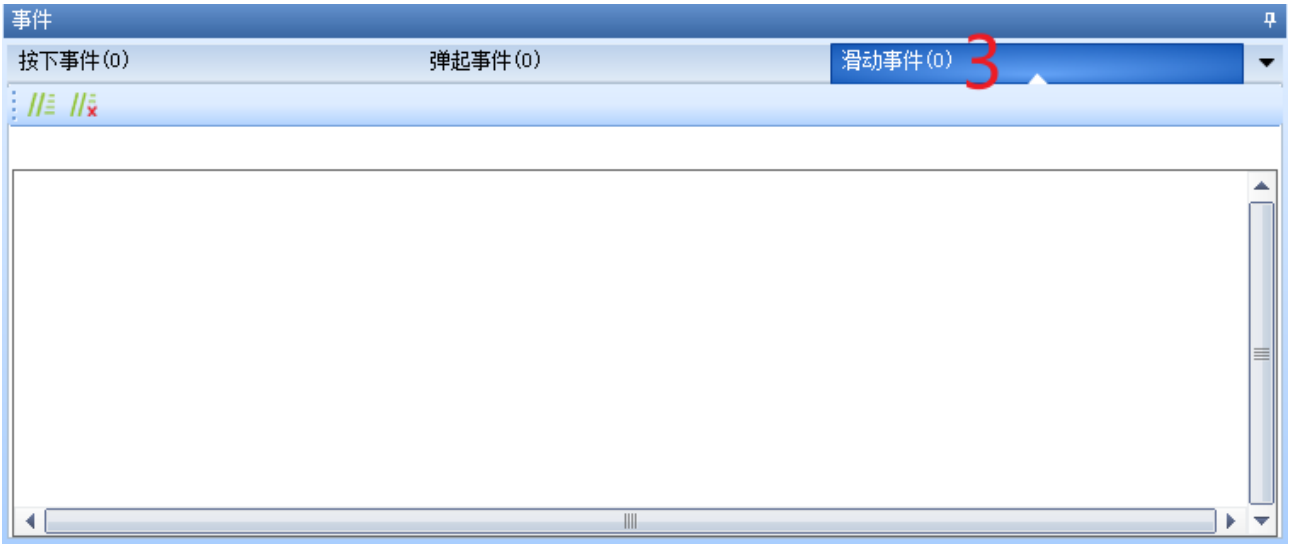


1. 控件按下事件

在控件的显示区域内，触摸被按下的瞬间，串口屏在执行完成控件按下状态刷新完成以后，串口屏会自动执行一次控件的《按下事件》中的代码。

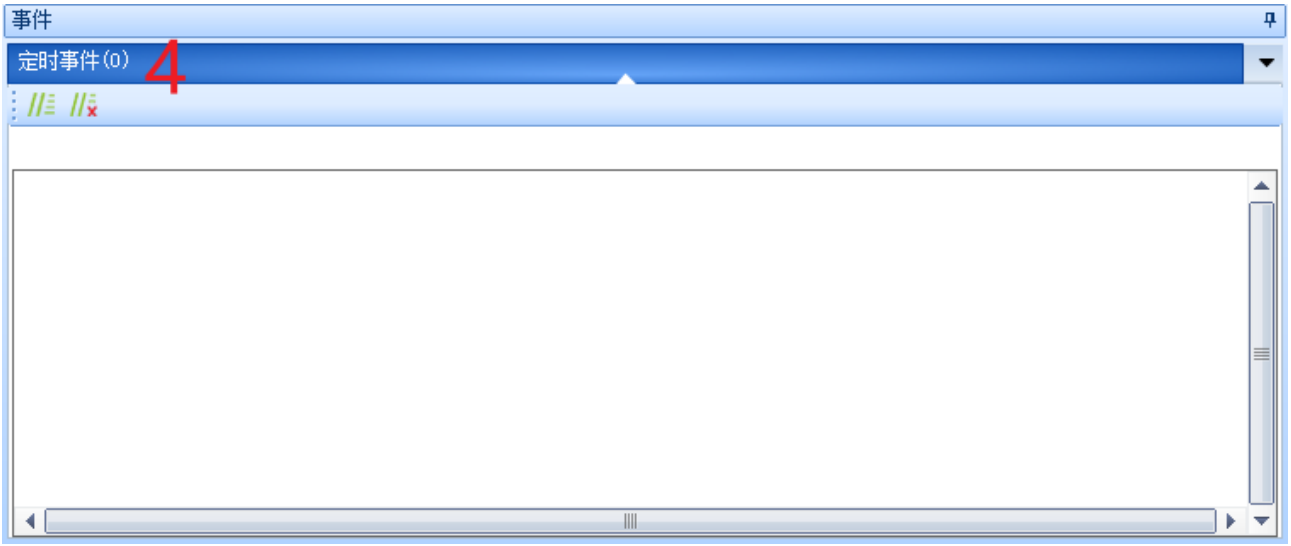
2. 控件弹起事件

在控件的显示区域内（以触摸按下时的坐标为准），触摸按下以后松开触摸的的瞬间，串口屏在执行完成控件弹起状态刷新完成以后，串口屏会自动执行一次控件的《弹起事件》中的代码。



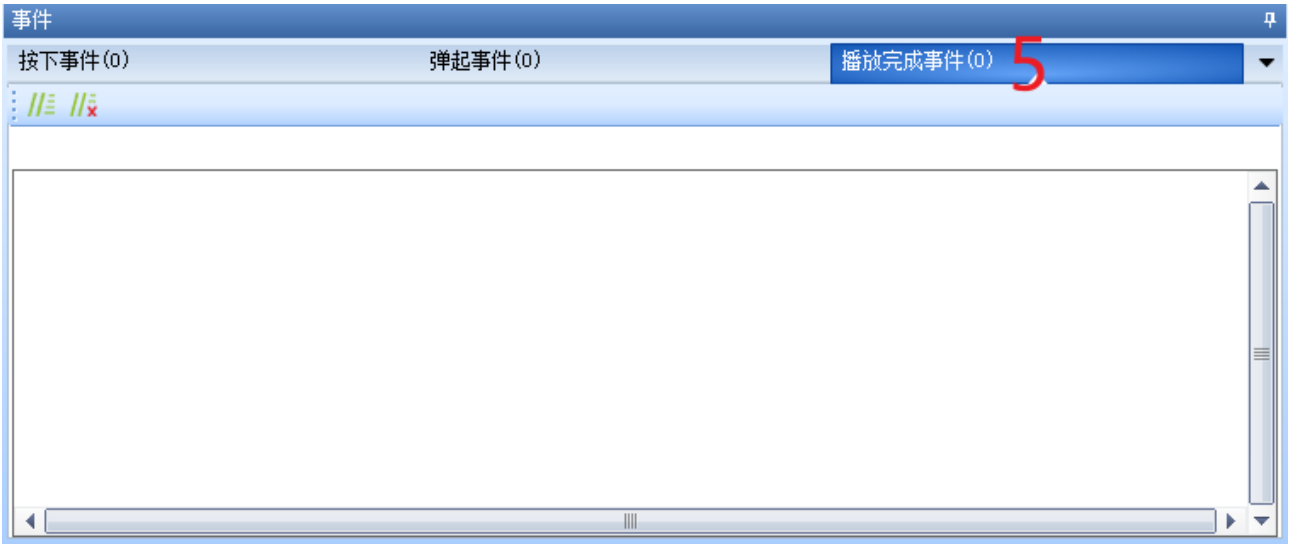
3. 控件滑动事件

在控件的显示区域内（以触摸按下时的坐标为准），触摸按下以后且没有松开的过程中，每次触摸坐标发送改变后，串口屏会自动执行一次控件的《滑动事件》中的代码。



4. 定时事件

定时器控件特有事件定时器设定时间到了以后，串口屏会自动执行一次定时器的《定时事件》中的代码。



5. 播放完成事件

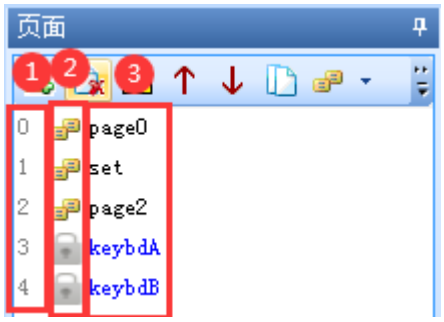
音频控件/视频控件特有控件视频/动画控件播放完成一次视频文件/动画文件以后，串口屏会自动执行一次控件的《播放完成事件》中的代码。

6. 发送键值

每个控件按下事件/弹起事件下面都会有发送键值这个选项，如果勾选该选项，点击控件将会串口发送相应的数据，可方便客户设备快速定位是哪个页面中控件发生点击事件，当然客户也可以通过 printh 加上 prints 指令来自定义串口发送格式。

参考*0X65 控件点击事件返回* (page 897)

1.2.9 9. 页面窗口



1. 页面 id

page 指令、dp 变量将会用到页面 id。

2. 锁

显示页面是否上锁了，上锁的页面无法编辑，如果需要解锁，使用鼠标右键就能解锁了。

3. 页面名称

每个页面的页面名称。page 指令和跨页面操作控件属性将会用到页面名称。

page-页面跳转指令 (page 575)

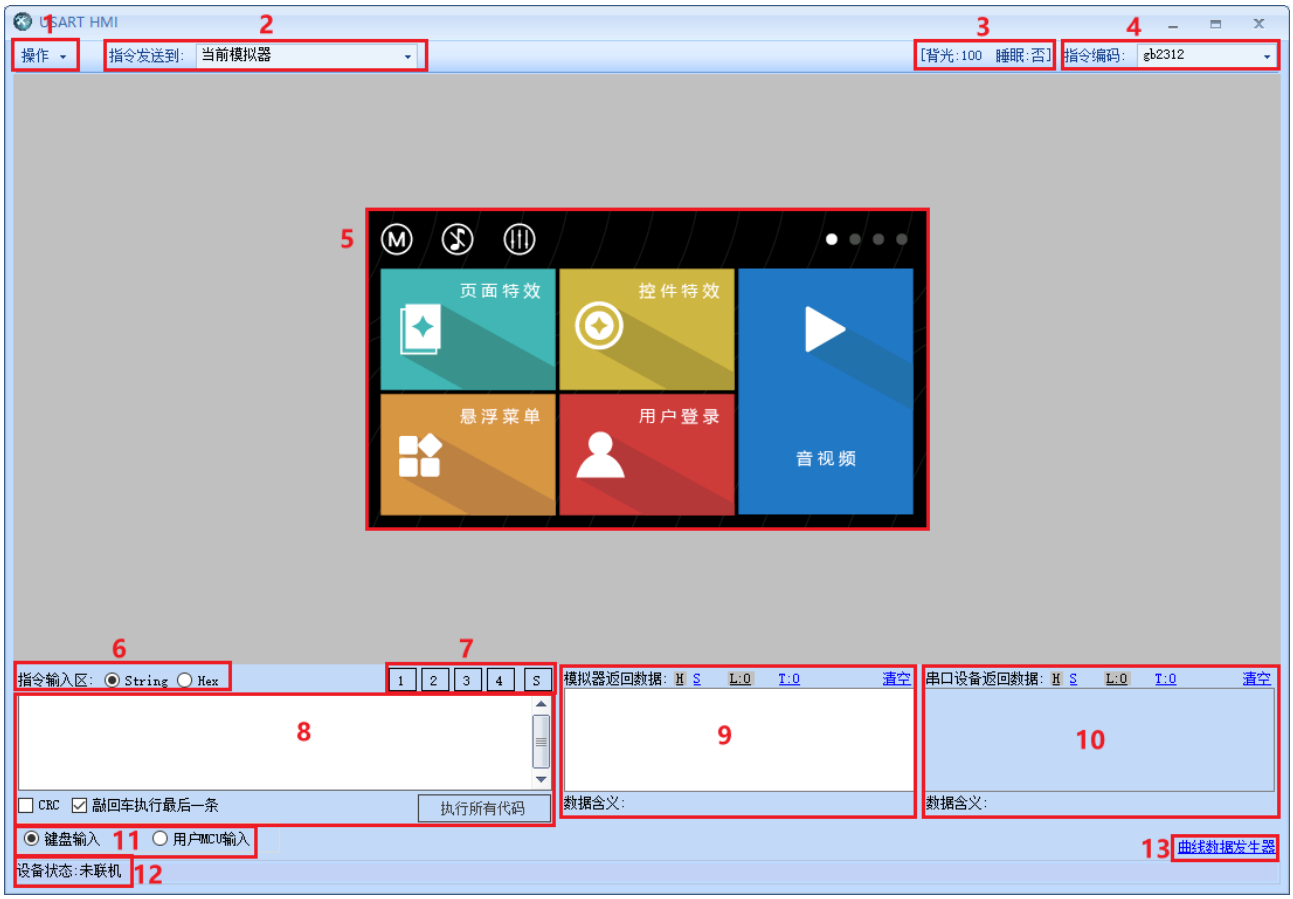
dp-当前页面 ID (page 743)

1.2.10 10. 属性窗口

每一个控件都会有属于它的属性。可以修改控件属性，可以改变控件的外形，位置，功能等属性。点击每一个属性，可以在属性栏的最下方看到对应的注释

1.2.11 11. 调试窗口

点击软件工具栏中的调试将进入调试窗口



1. 操作菜单

1.1 校准设备 RTC 时钟

对电脑正在联机的屏幕中 rtc 时钟进行校准。（仅 k0，x5 支持 rtc）

> **注意：** 使用到屏幕中 rtc 必须装电池，否则重新上电 rtc 值为随机值。电池型号为 cr1220 3v。

1.2 校准模拟器 rtc 时钟

对模拟器中 rtc 时钟进行校准。

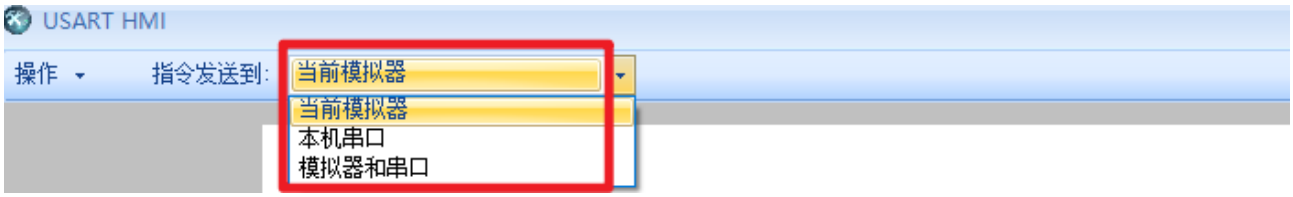
1.3 重启设备

重新启动和电脑联机的屏幕。

1.4 重启模拟器

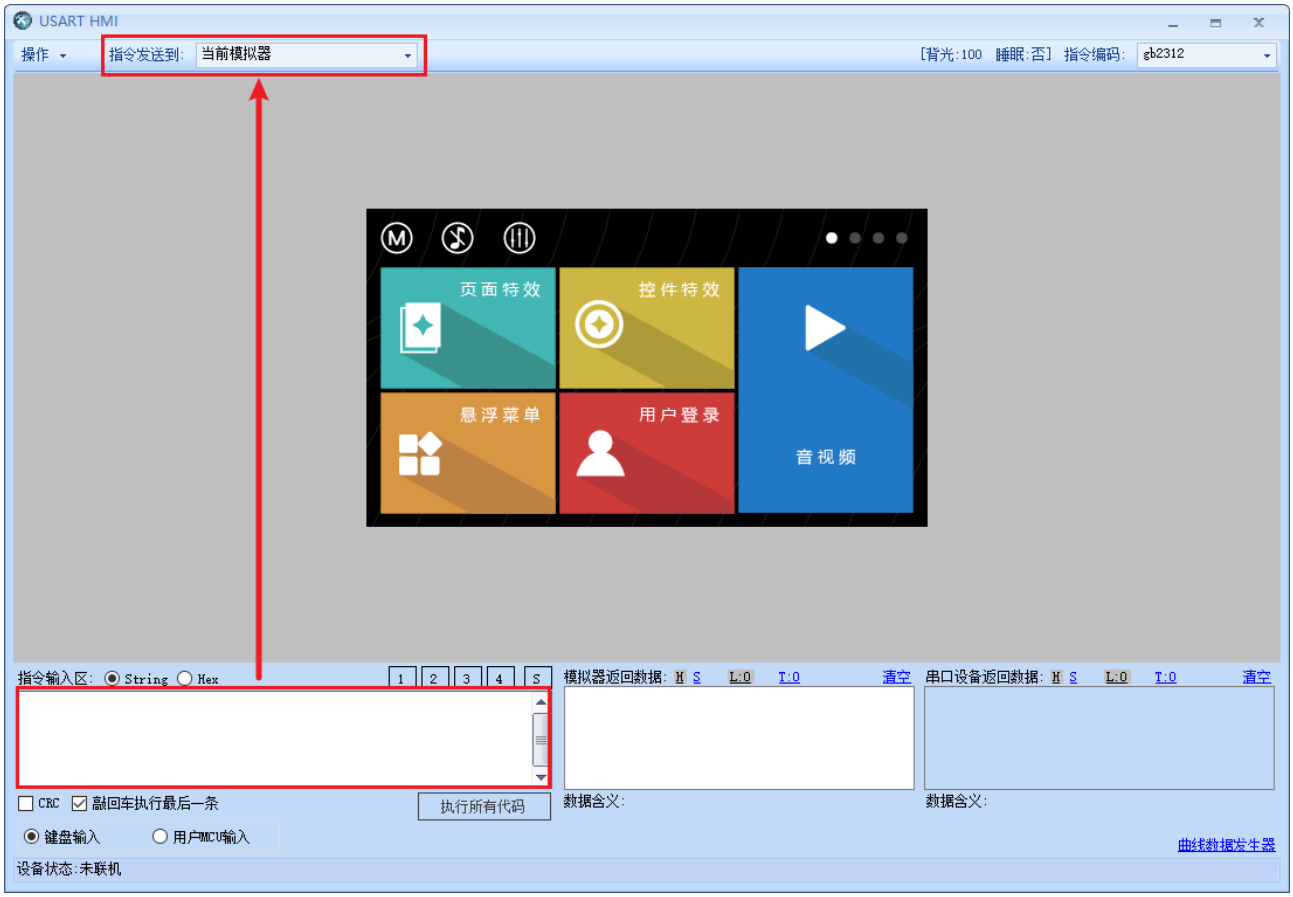
重新启动模拟器。

2. 与串口屏联机进行调试



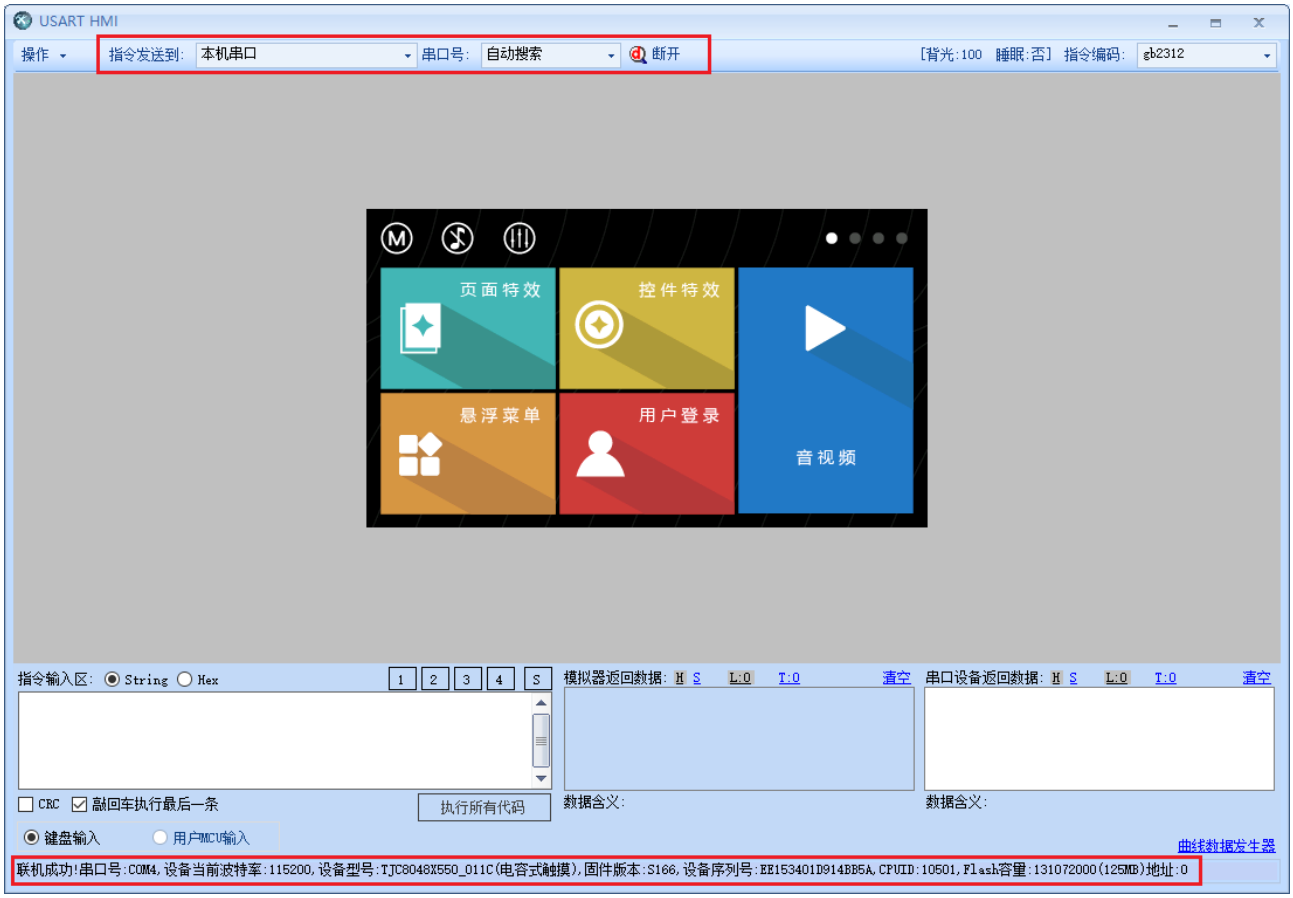
软件支持 3 种调试方式

2.1 当前模拟器



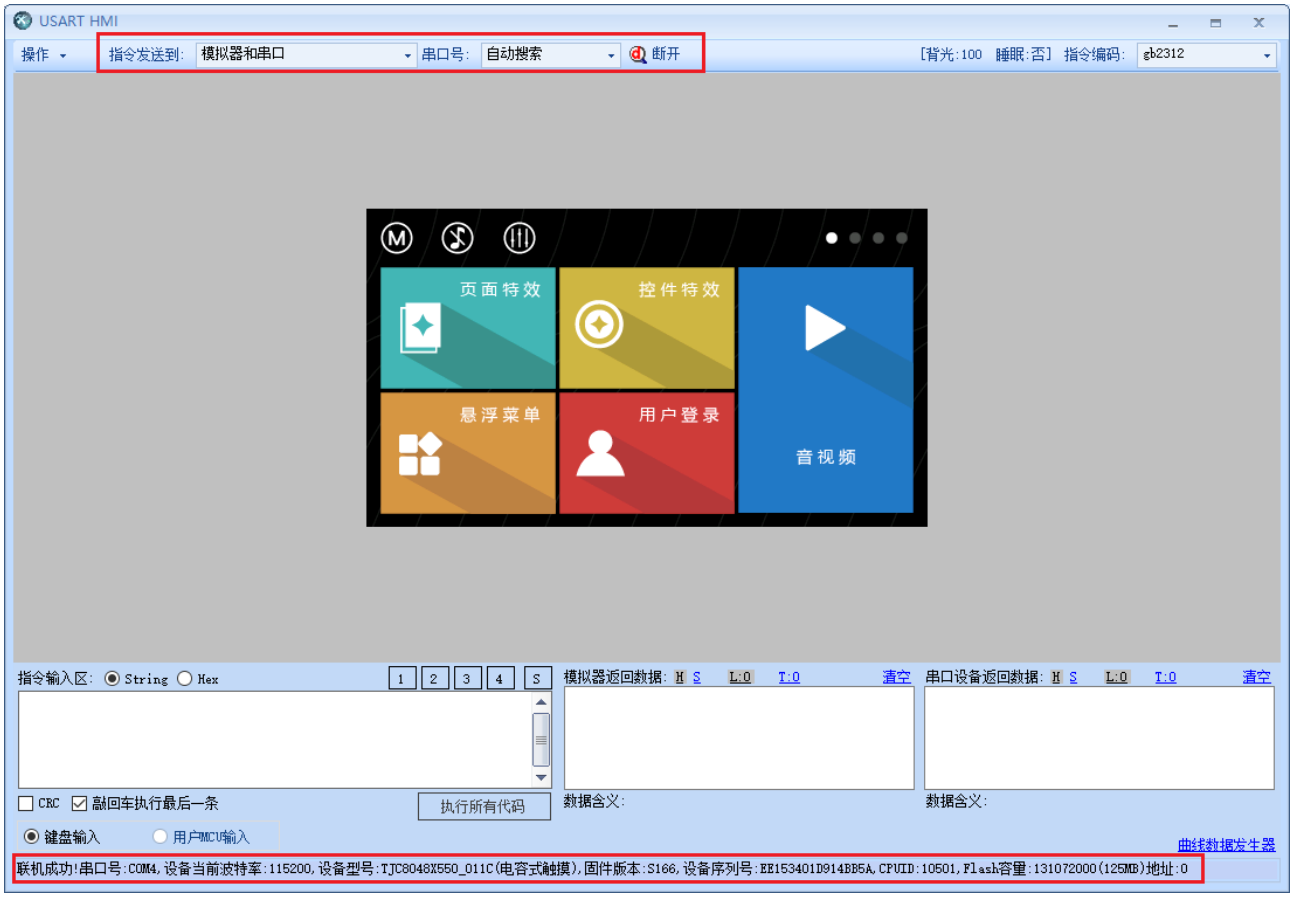
将软件做好的工程在调试界面进行模拟仿真。选择当前模拟器，指令输入区输入指令控制的是模拟器。

2.2 本机串口



将软件做好的工程在调试界面进行模拟仿真。选择本机串口，点击联机按钮，联机成功后，设备状态会出现联机设备型号等设备信息，调试输入区输入指令控制的是设备。

2.3 模拟器和串口

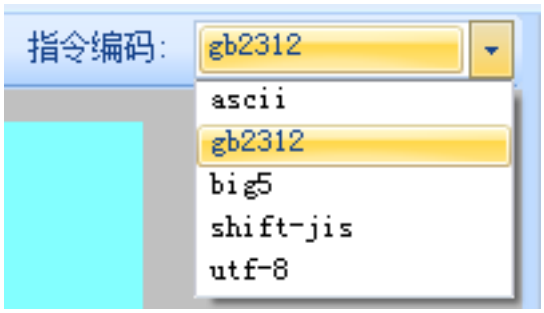


将软件做好的工程在调试界面进行模拟仿真。选择模拟器和串口，点击联机按钮，联机成功后，设备状态会出现联机设备型号等设备信息，调试输入区输入指令控制的是设备和模拟器。能调试看到模拟效果和实际仿真，方便查找问题。

3. 模拟器状态

显示模拟器当前背光亮度，可通过 dim，dims 指令修改。屏幕是否处于睡眠中，通过 sleep 指令直接修改，ussp 指令，thsp 指令间接修改。

4. 指令编码



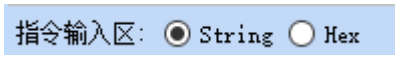
指令输入区发送指令是用什么字符编码发送的。

注意: 字符编码不一致时会出现乱码

5. 模拟器显示区域

模拟工程运行，运行显示界面，鼠标可以模拟触摸。鼠标左键点击触发按下事件，松开左键触发弹起事件。

6. 指令发送格式



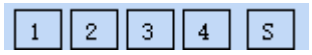
6.1.string

选择 string 发送，指令以字符串形式发送。输入指令后将会默认在后面加上 3 个 16 进制 ff。

6.2.hex

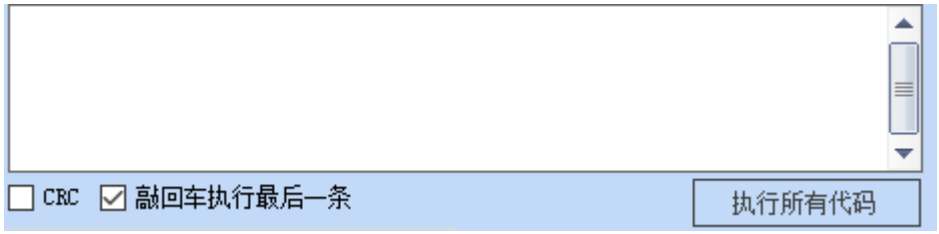
选择 hex 发送，指令以 16 进制形式发送。

7. 指令存储



点击 1,2,3,4 按钮指令输入区将出现保存在里面指令代码。点 S 是将指令输入区中指令代码保存到 1 到 4 按钮中。方便重复的指令调试。

8. 指令输入区



在区域里输入指令，通过执行所有指令按钮，指令成功执行，模拟器显示区域（或串口设备）将会执行相应的操作。

8.1 crc

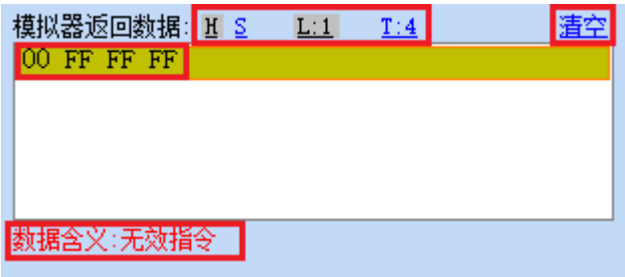
发送的字符串指令将主动加上 crc 校验再发送到模拟器（或串口设备）。默认关闭。

8.2 敲回车执行最后一条

开启后，敲键盘回车能够执行指令输入区的代码；关闭后，敲键盘回车不能够执行指令输入区的代码。默认开启。

注意: 开启或关闭，都需要重新鼠标点击指令输入区；否则不生效。

9. 模拟器返回数据窗口



当模拟界面工程串口发送数据出去的时候，串口发送的数据就会显示在这个窗口。

指令执行失败，将会在模拟器返回数据窗口返回失败原因（可通过 bkcmd 指令开启或关闭）。

9.1 H 显示

发送数据以 16 进制显示

9.2 S 显示

发送数据以字符串显示

9.3 L 显示

串口发送数据以行的形式显示。同一个控件同一个事件下串口发送的数据算一行。

9.4 T 显示

串口发送数据以发送数据先后显示，不分行。并以 16 进制数据显示。

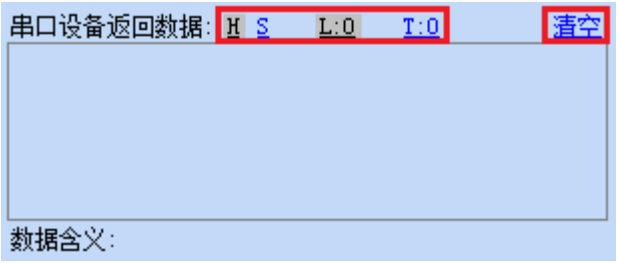
9.5 清空

清空模拟器返回数据窗口里的所有数据

9.6 串口设备返回数据窗口

当串口设备串口发送数据到模拟器，模拟器会将接收到的数据显示在这个窗口。

10. 串口设备返回数据窗口



10.1 H 显示

模拟器接收到数据以 16 进制显示

10.2 S 显示

模拟器接收到数据以字符串显示

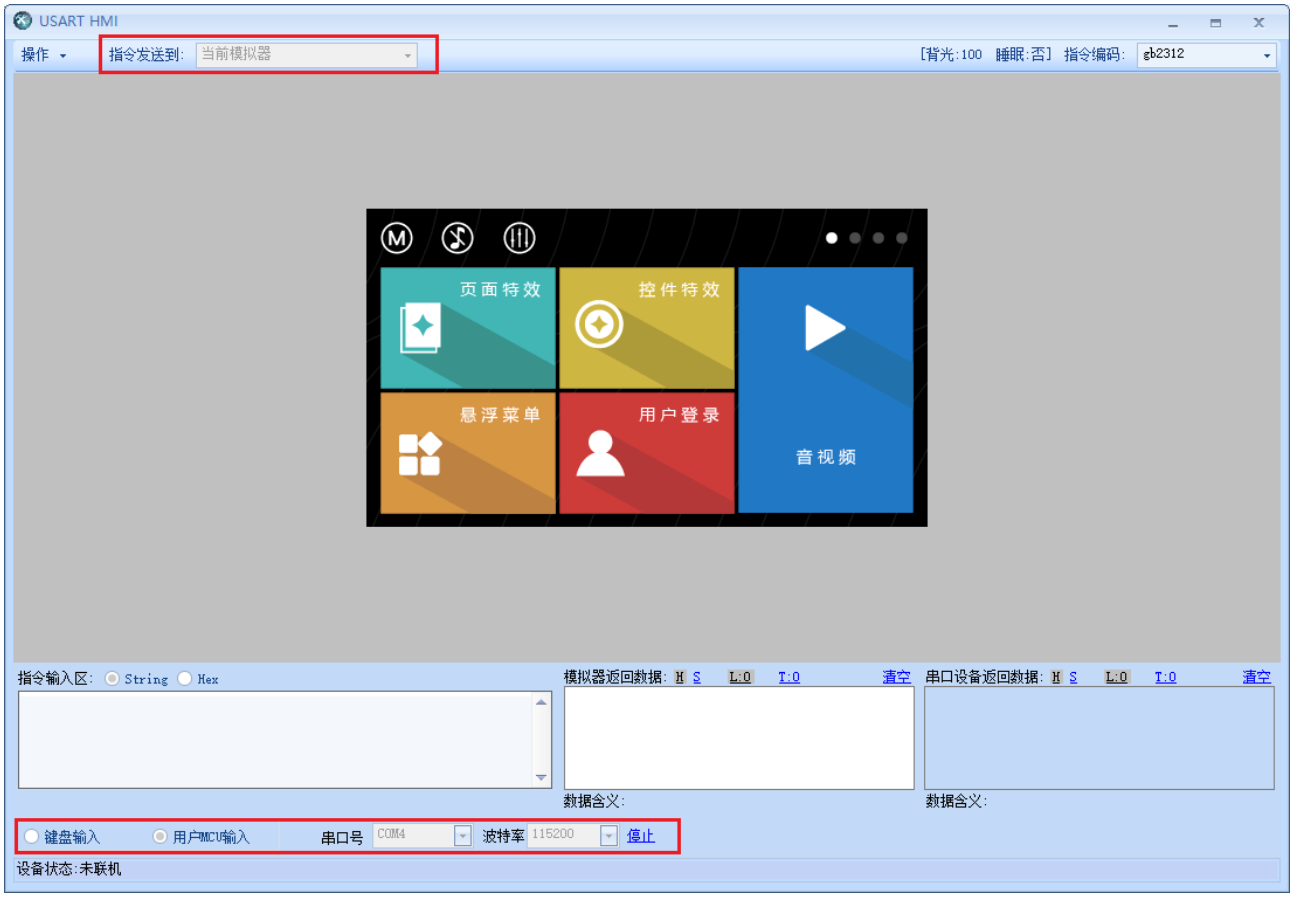
10.3 L 显示

模拟器接收到数据数据以行的形式显示。同一个控件同一个事件下串口屏发送的数据算一行。mcu 每条指令加上 3 个 16 进制结束符算一行。

10.4 T 显示

模拟器接收到数据数据以接收数据先后显示，不分行。并以 16 进制数据显示。

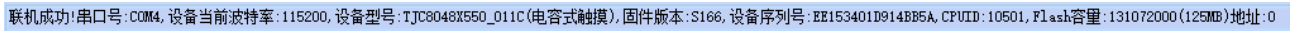
11. 模拟器和 mcu 单片机联机调试



将软件做好的工程在调试界面进行模拟仿真。

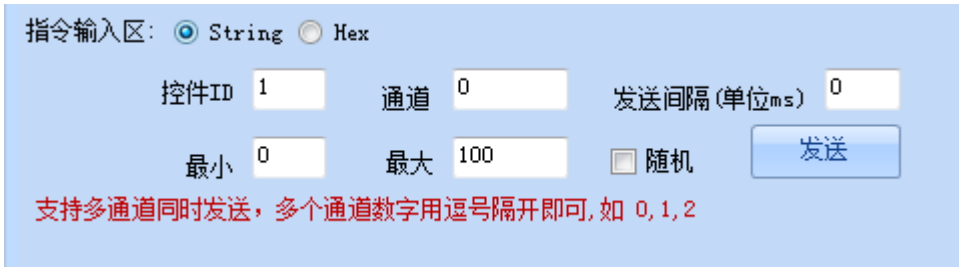
① 指令发送到选择当前模拟器 ② 下面选择用户 mcu 输入 ③ 选择好串口号，波特率，点击开始。联机成功后，就可以 mcu 和模拟器进行调试。

12. 设备状态



指令发送到选择本机串口（模拟器和串口），开始联机，联机成功后会出现如上图所示的内容。

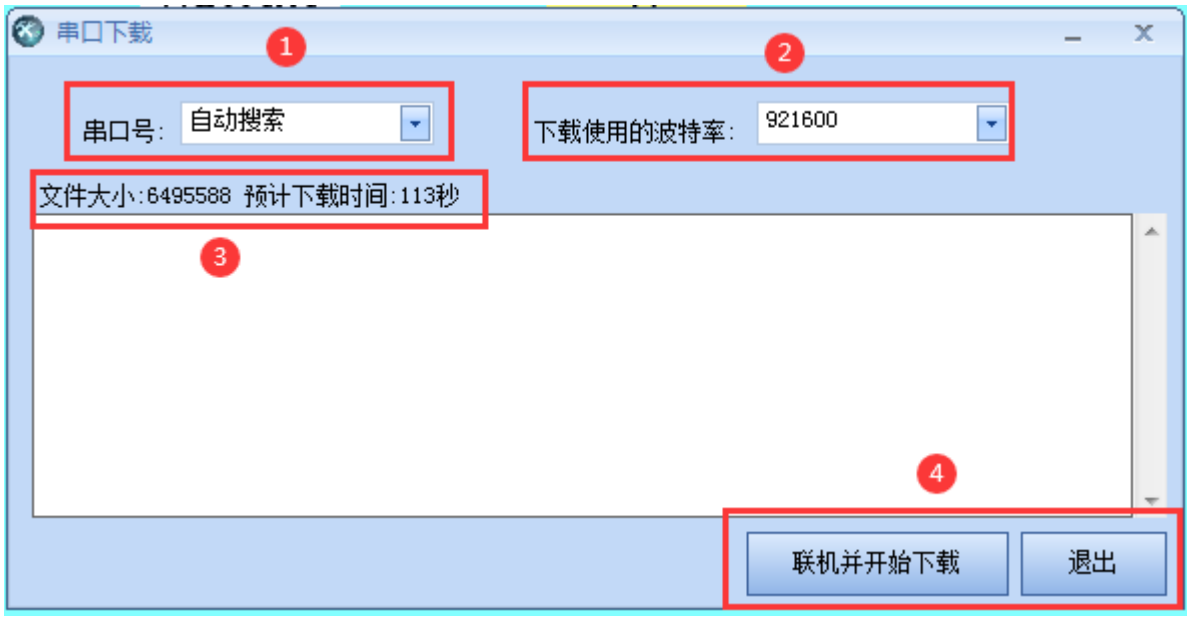
13. 曲线数据发生器



仿真曲线/波形控件每添加一个点会形成的波形，方便用户调试出自己想要波形图。获取它的数据点以及时间间隔。

1.2.12 12. 下载窗口

点击工具栏上的下载按钮，编译无误后会出现下载窗口



1. 串口号

设备和电脑连接串口号，可手动选择，可自动搜索。

2. 下载使用的波特率

串口下载工程速率，范围在 2400~921600。设置高的下载波特率可减少下载时间。

注意： 设备通讯波特率和下载使用的波特率无关，通讯波特率是使用 baud 或者 bauds 来配置的，点击下载进行联机时，上位机会先尝试以各个不同的波特率进行联机，联机成功后，修改为下载波特率进行下载。

注意： 若 usb 转 ttl 芯片不支持过高下载波特率，会出现联机失败。

注意： 通过 232 下载时，因为 232 芯片的原因，下载波特率请勿超过 256000。*为什么 232 通讯不能超过 256000* (page 1265)

注意： 无法通过 485 进行下载请参考*485 接口的串口屏* (page 96)

3. 文件大小和下载时间

整个工程文件大小，以及工程下载到屏预计时间，下载时间和所使用的下载波特率有关。

4. 下载和退出

工程通过串口下载到屏，若工程和设备固件版本不一致，会自动升级为当前工程固件版本。

1.2.13 13. 设备

设备窗口在工具栏中“设备”按钮，点击就可以进入到设备窗口。

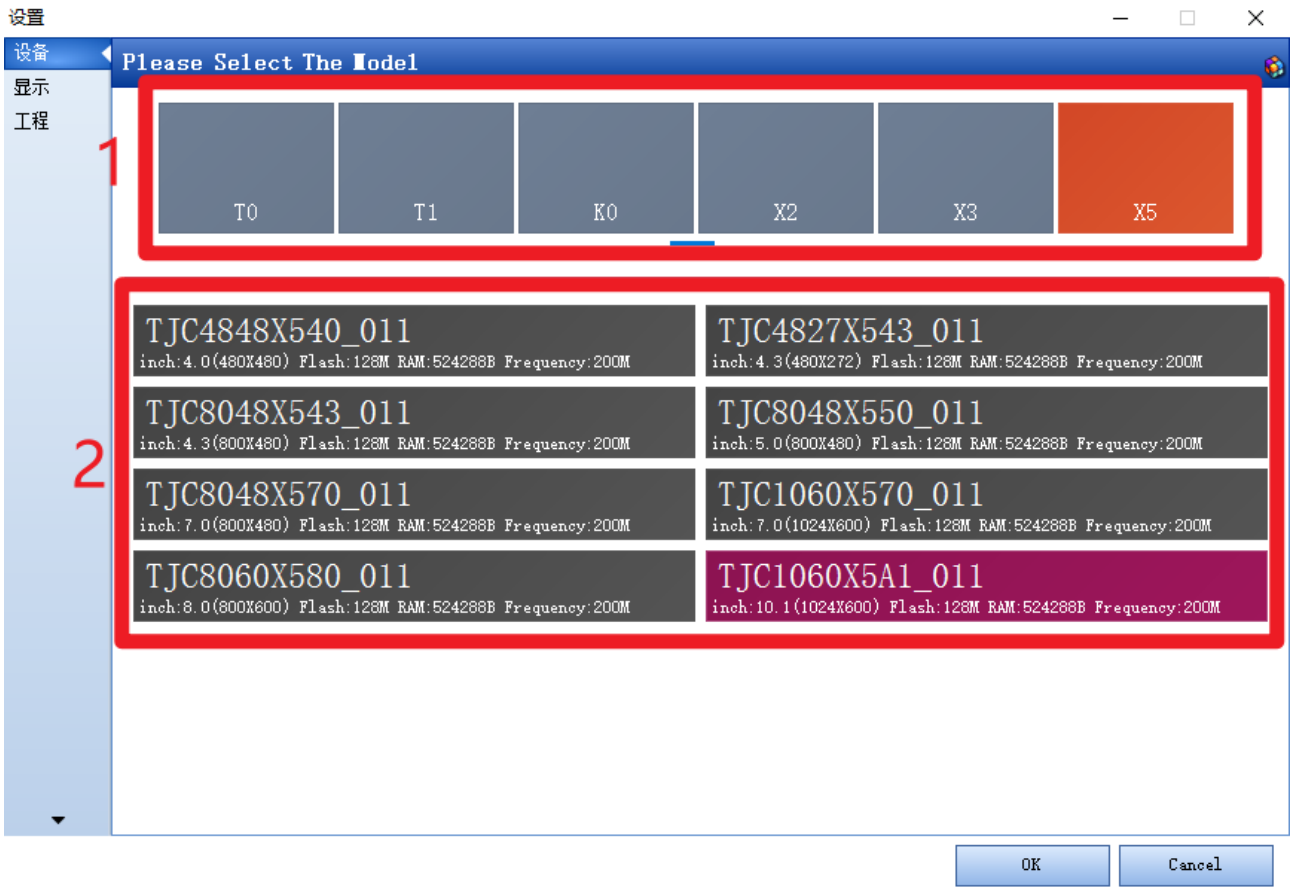
1.2.13.1 设置窗口

1. 设备
可以在此选择串口屏系列和串口屏型号。

2. 显示
设置显示相关的选项

3. 工程
设置工程相关参数

1.2.13.2 设备窗口



1. 产品系列

软件会根据推出产品系列实时更新目前拥有的产品系列，选择产品型号，下方会出现具体的产品型号。

2. 产品型号

选择产品系列会在该窗口出现工程具体型号（产品型号命名）。选择工程具体型号，点击 ok。软件会根据型号自动生成工程窗口。

注意: 下载工程进屏，如果工程型号和屏型号不匹配会出现 model does not match。

1.2.13.3 显示窗口

1. 显示方向

屏幕支持 0 90 180 270 四个显示方向。

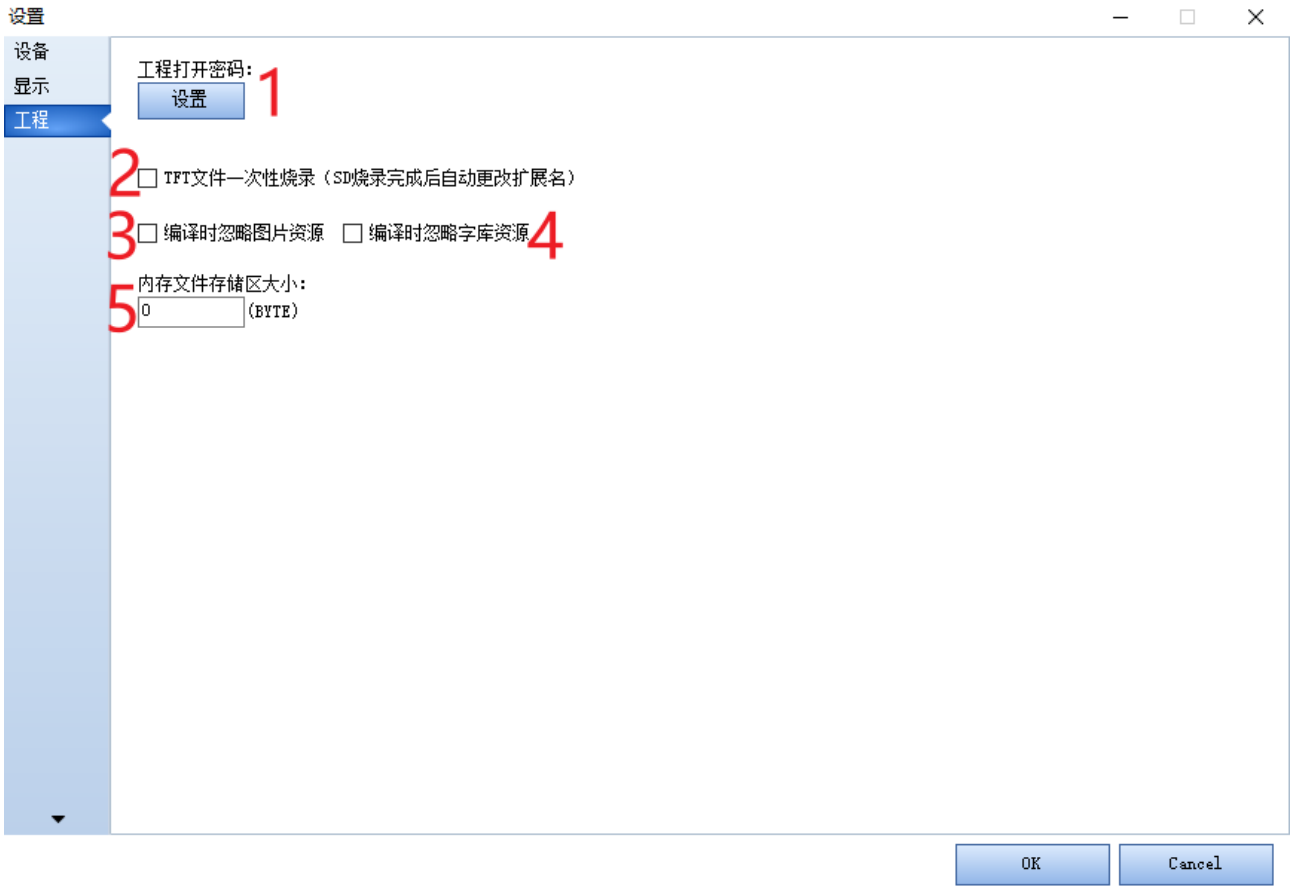
> **注意：** 工程不支持运行中改变显示方向。

2. 字符编码

工程字符编码，目前支持：ascii，gb2312，big5，shift-jis，utf-8。

> **注意：** 字符编码不一致时会出现乱码。

1.2.13.4 工程窗口



1. 工程打开代码

请勿随意设置。设置打开工程是否需要密码，默认是没有密码的（设置密码后如果忘记密码，工程将无法找回，请慎用）。

2.tft 文件一次性烧录

仅 x 系列支持。设置 sd 卡里 tft 文件是否烧录后修改文件名，用于一次性烧录工程到屏上，烧录完毕无需取出 SD 卡。

3. 编译时忽略图片资源

特殊功能使用，请勿勾选。

4. 编译时忽略字库资源

特殊功能使用，请勿勾选。

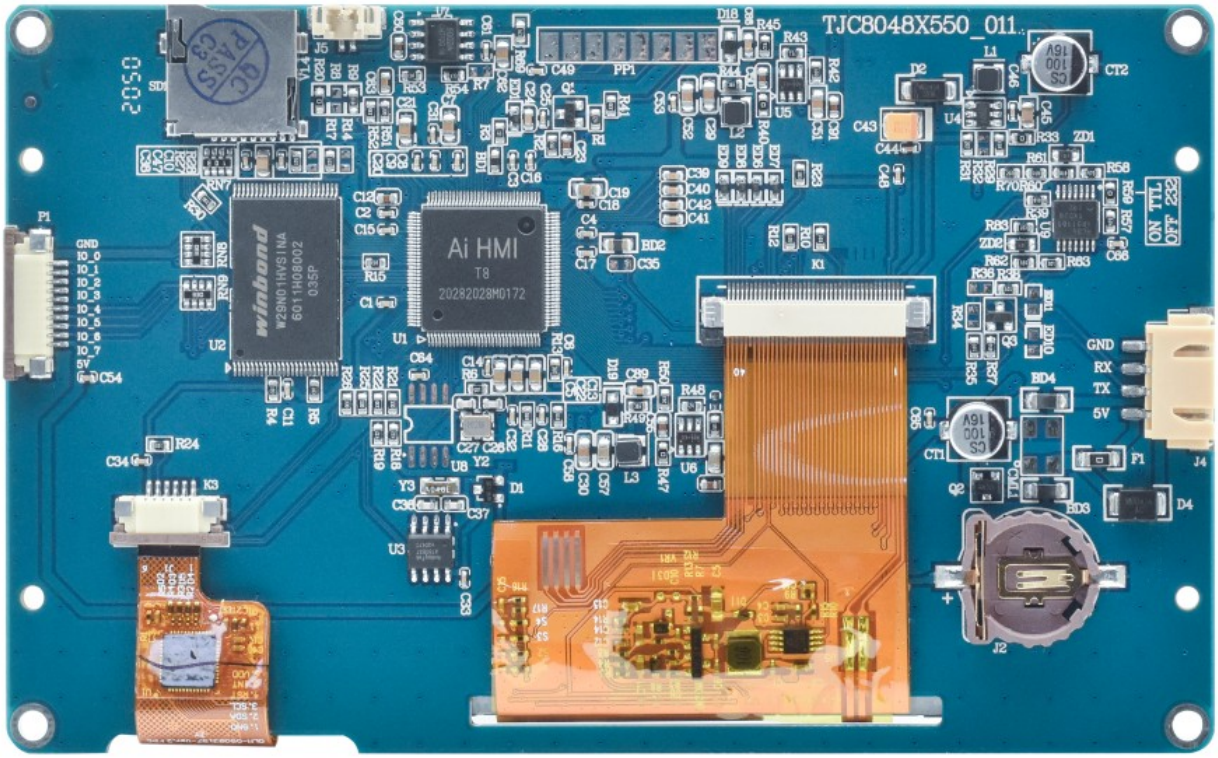
5. 内存文件存储区大小

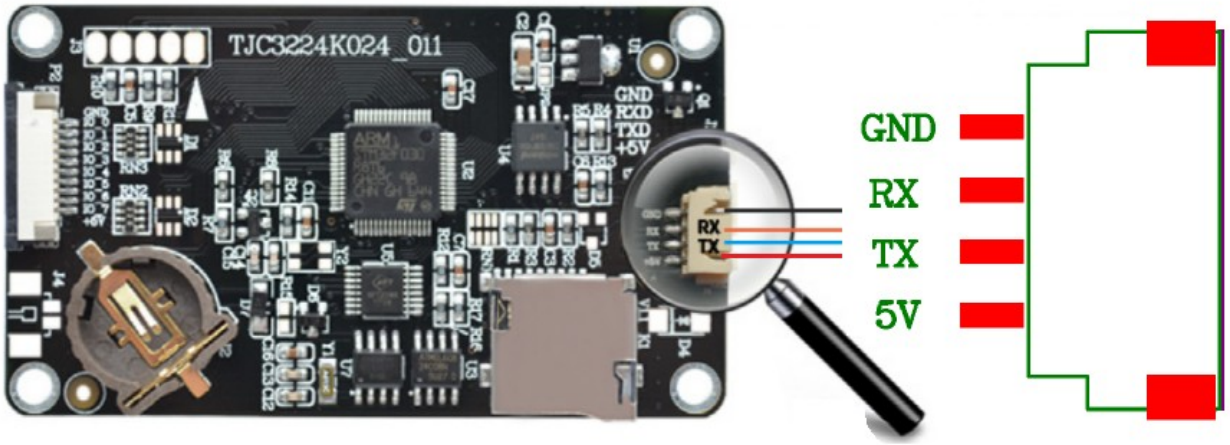
请勿设置。如需使用请参考将数据记录控件保存在内存中 (page 520)

1.3 到手测试

1.3.1 普通 TTL/232 接口的串口屏

目前淘晶驰的标准品接口有 2 种，一种是 XH2.54*4

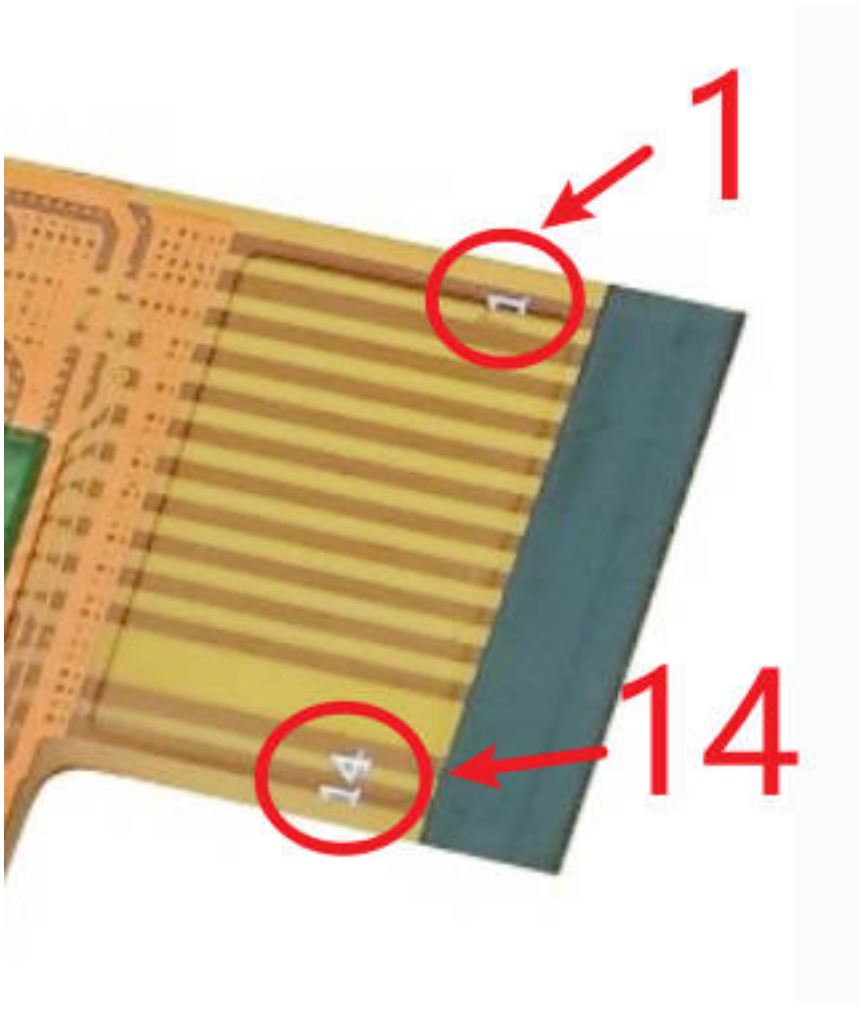




1.3.2 FPC 软排接口

另一种是 FPC 软排接口，FPC 1.0*14，常见于 T1、X2 系列中的 FPC 版本

引脚的顺序如图所示，T1 与 X2 的引脚顺序和引脚定义是一致的



FPC 1.0*14 接口定义如下，正常情况下，仅需接以下 6 个引脚到您的单片机即可

序号	引脚	备注
1	GND	接单片机 GND
2	GND	接单片机 GND
3		
4		
5		
6	UART_RXD	接单片机 TX
7	UART_TXD	接单片机 RX
6		
7		
8		
9		
10		
13	VDD	接 3.3V 供电
14	VDD	接 3.3V 供电

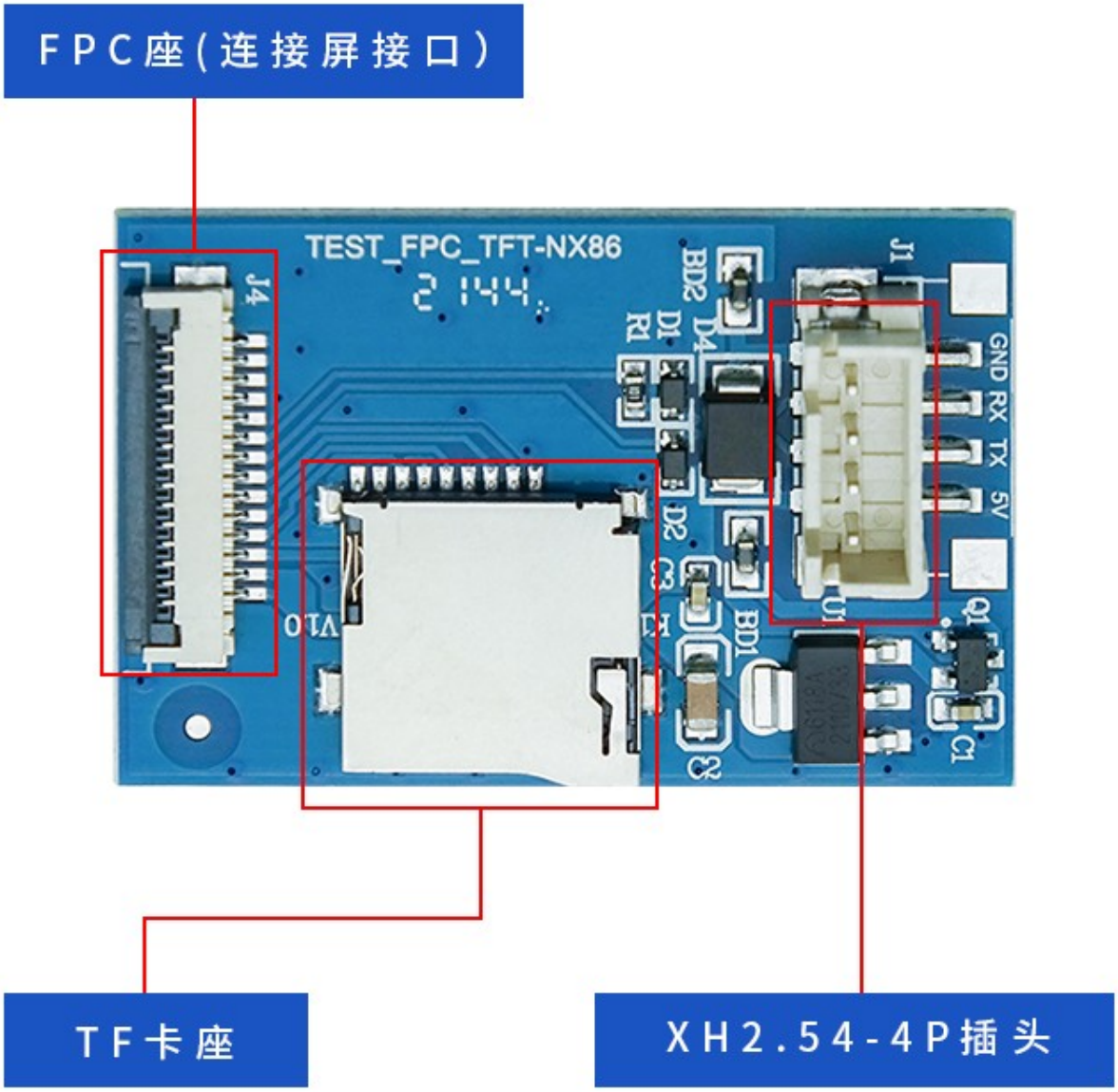
完整的定义如下

表 2: FPC 引脚对照表

序号	引脚	备注
1	GND	接地
2	GND	接地
3	NRST	模组复位，低电平有效，可悬空
4	NC	工厂测试引脚，必须悬空
5	NC	工厂测试引脚，必须悬空
6	UART_RXD	接单片机 TX
7	UART_TXD	接单片机 RX
8	TF_CS	外接 TF 卡
9	TF_MOSI	外接 TF 卡
10	TF_SCK	外接 TF 卡
11	TF_MISO	外接 TF 卡
12	BEEP *	蜂鸣器输出，可悬空
13	VDD	接 3.3V 供电
14	VDD	接 3.3V 供电

注意： No.12 beep 引脚注: 部分型号支持，目前仅 T1 系列使用 HMI C2 芯片的版本支持

FPC 1.0*14 可以通过转接板转换为 XH2.54*4

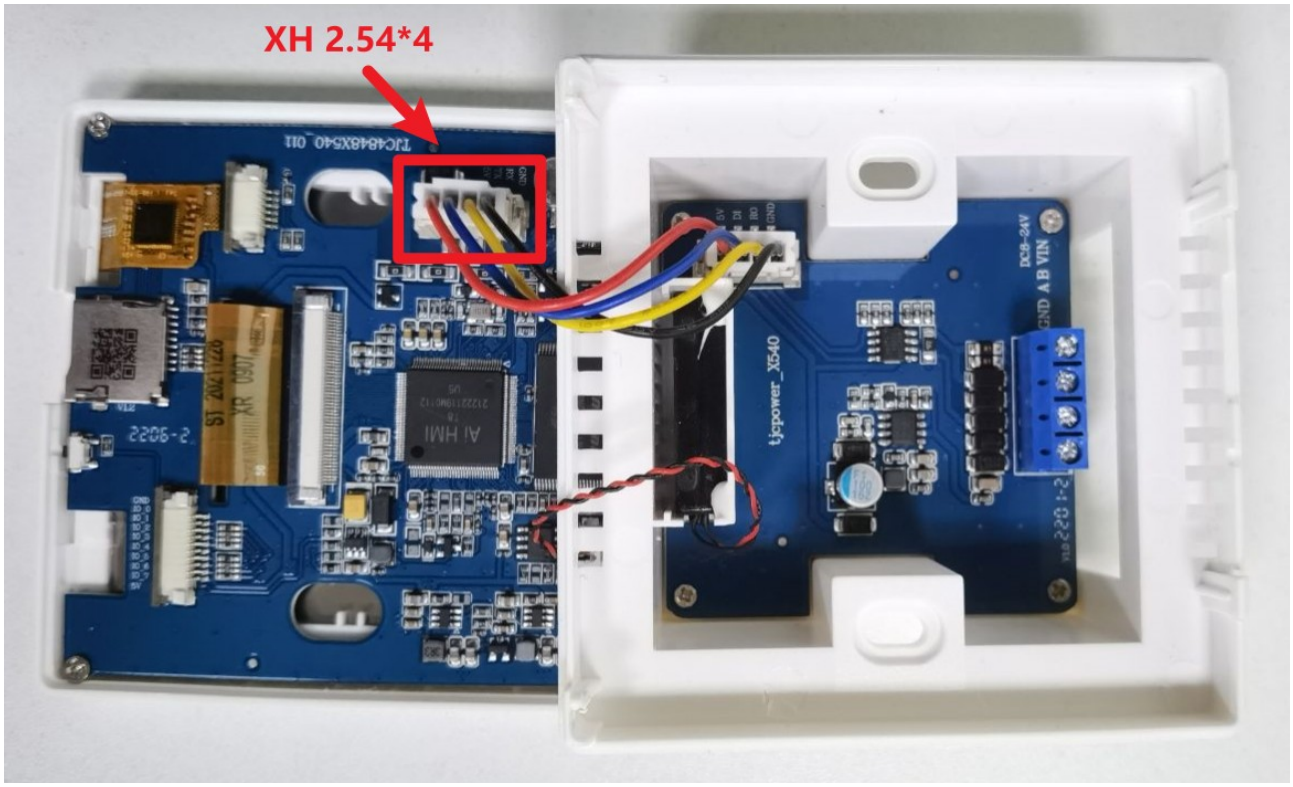


1.3.3 485 接口的串口屏

除此之外还有一款 x5 4 寸宽电压版本，将后盖打开，里面也是 XH2.54*4

x5 4 寸宽电压版本是 485 接口，必须将 485 背板取下才能通过官方上位机（USART HMI）用 ttl 进行下载，如果在 485 模式下，用官方上位机（USART HMI）会无法下载。

需要先通过上位机（USART HMI）导出 tft 文件（左上角文件-输出生产文件），然后使用 TFT 文件下载助手 (TFTFileDownload) 才能通过 485 进行下载

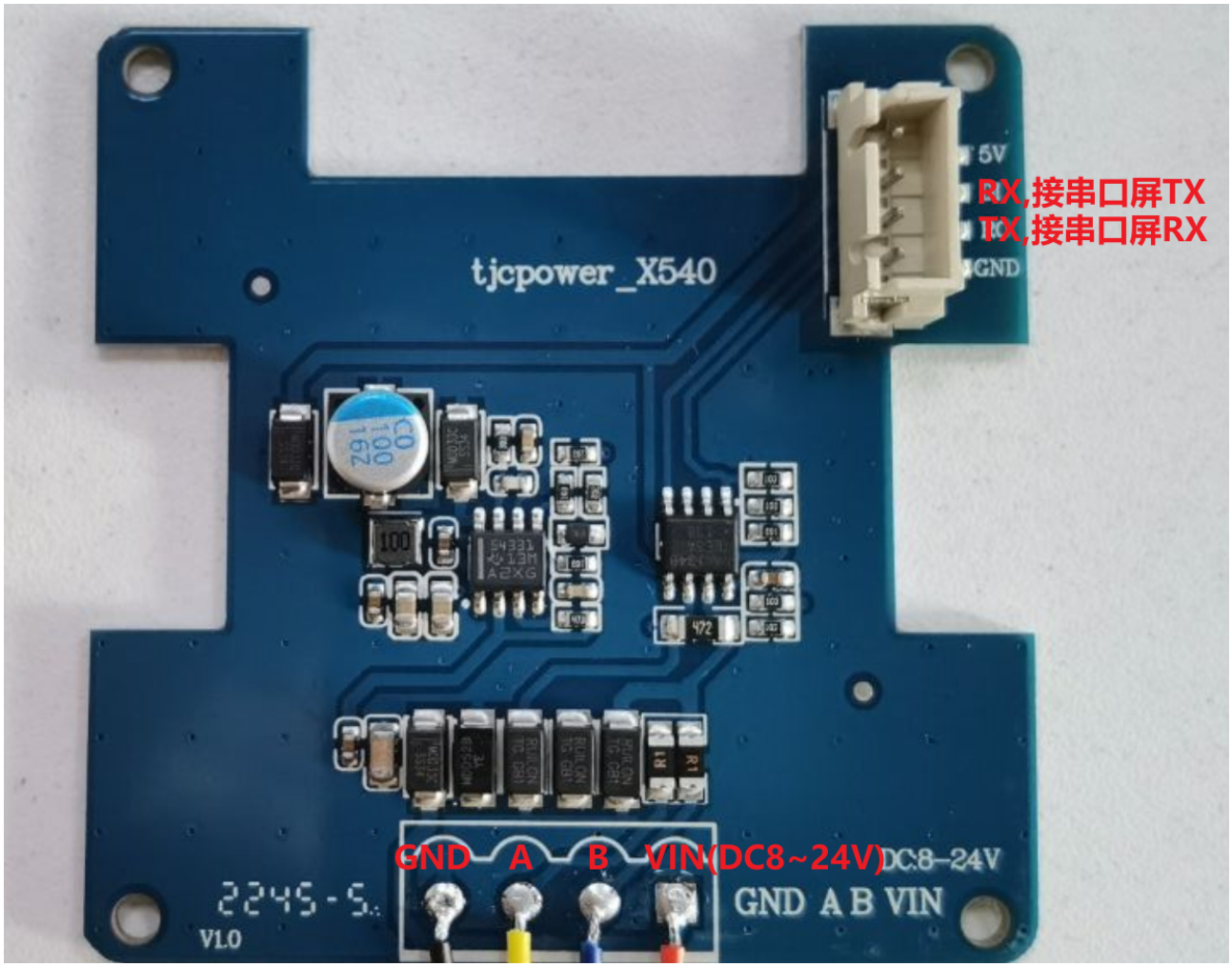


相关链接

TFT 文件如何下载到串口屏中 (page 1405)

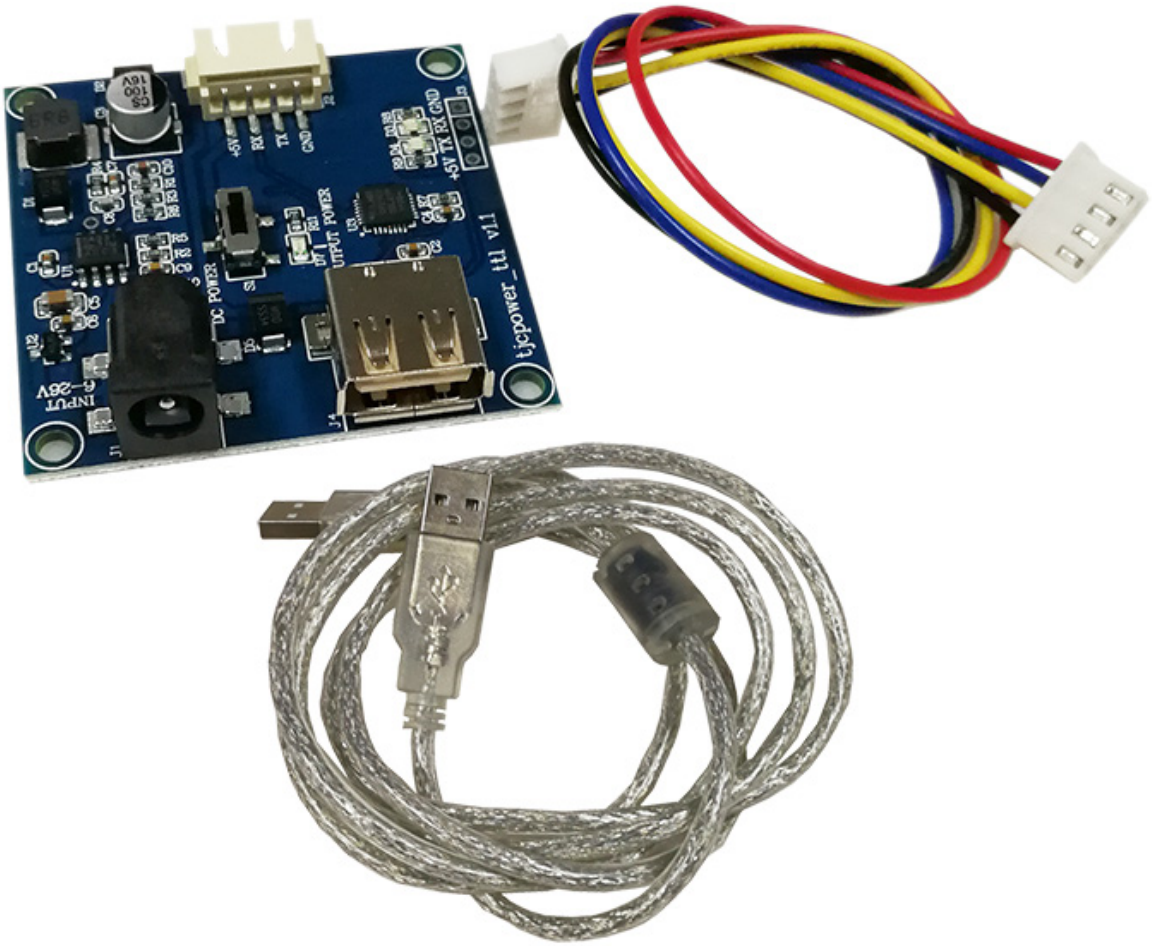
如何输出 TFT 生产文件 (page 155)

x5 4 寸宽电压版本背部接口接口定义如下



1.3.4 USB 转 TTL 工具

建议使用官方的供电和下载工具



注意： 如果您购买的屏大于 5 寸或者有使用喇叭的需求，请再购买一个 12V 电源用于额外供电，否则可能因为电脑 USB 供电不足导致开机启动失败

12V 2A电源

备注： 连接方式如下所示

5V 接串口屏 5V

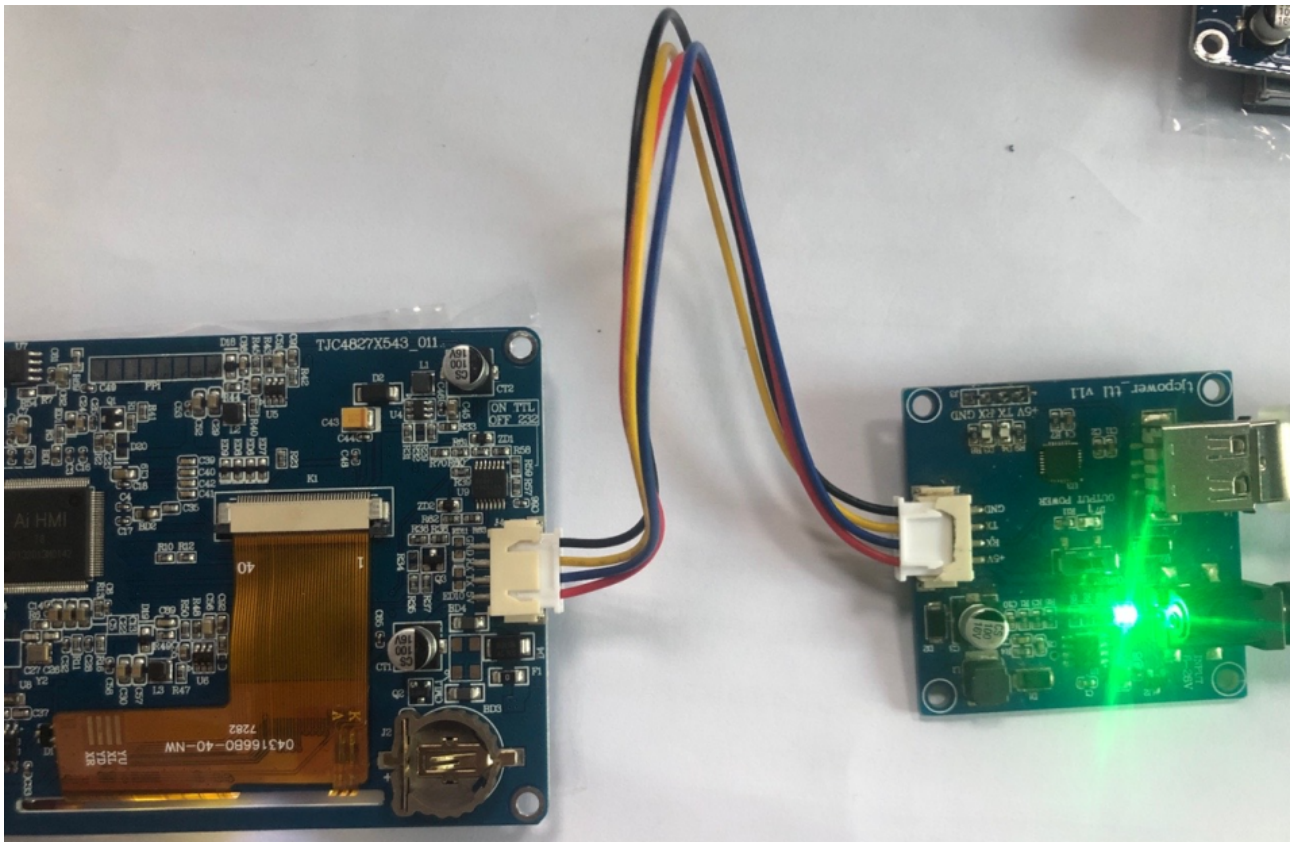
单片机 TX 接串口屏 RX

单片机 RX 接串口屏 TX

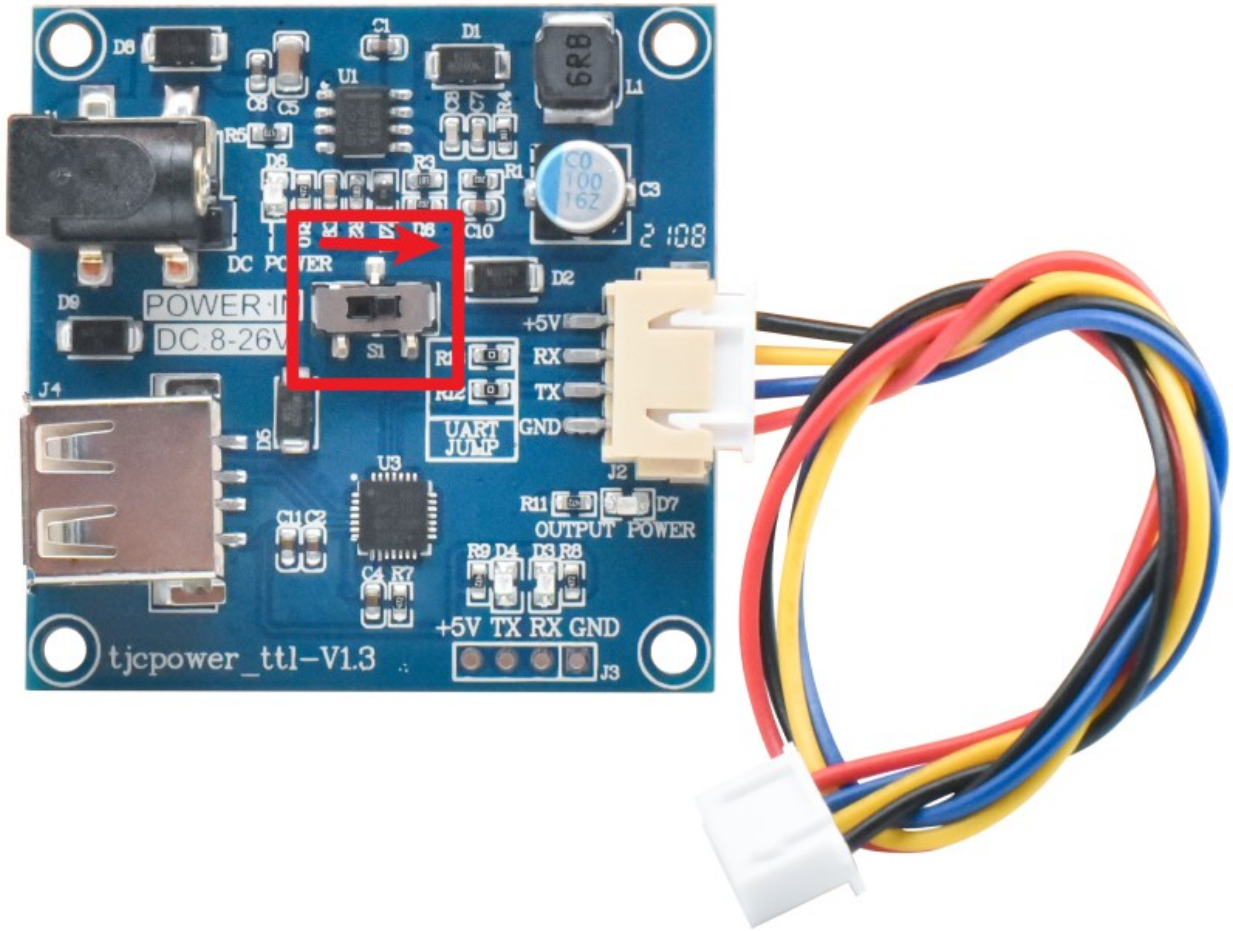
GND 接串口屏 GND

TX 是 Transmit（发送），RX 是 Receive（接收），发送对接收，接收对发送。

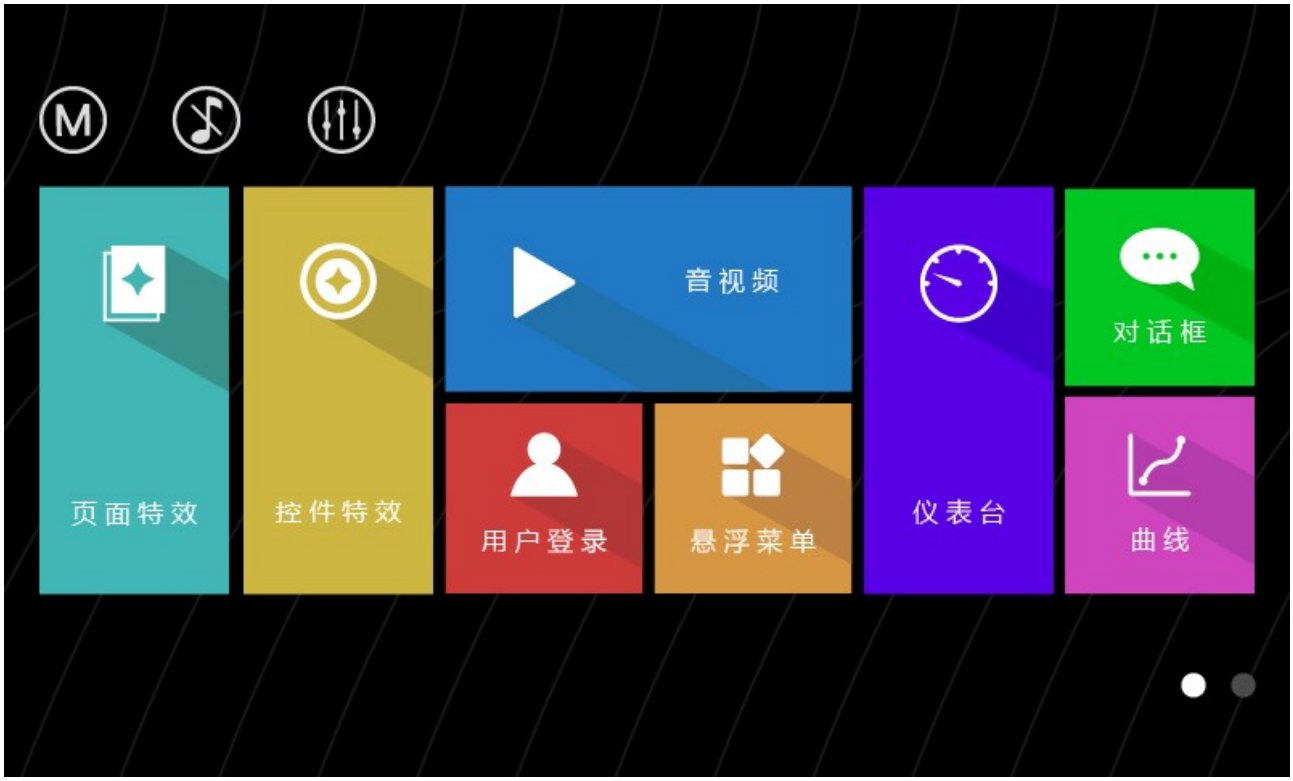
另外说明：有些板子和原理图标注的是 TXD 和 RXD。TX 就是 TXD，RX 就是 RXD。概念都是一样的，可以不做区分。



上电后请确保开关的位置如下图所示（朝向 4pin 接口）



上电后应显示开机界面



其他相关链接

安装串口驱动 (page 138)
通过串口下载工程到串口屏 (page 144)
通过 *SD* 卡下载工程到串口屏 (page 155)

1.4 创建工程

1.4.1 演示工程素材下载

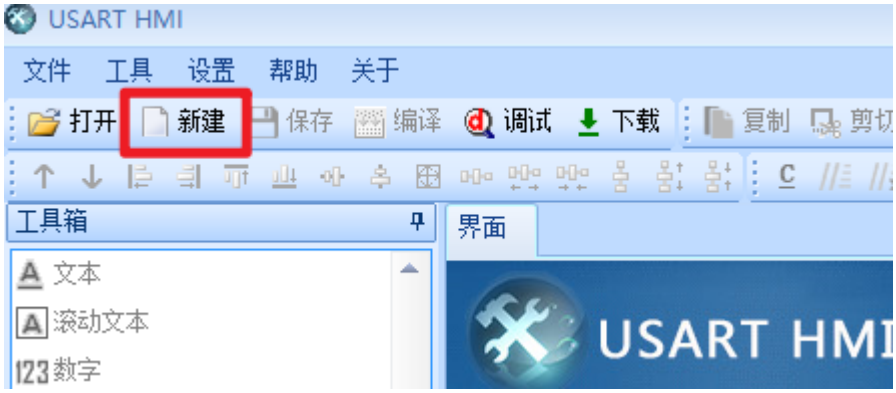
下载链接：

《演示工程素材包》演示工程下载

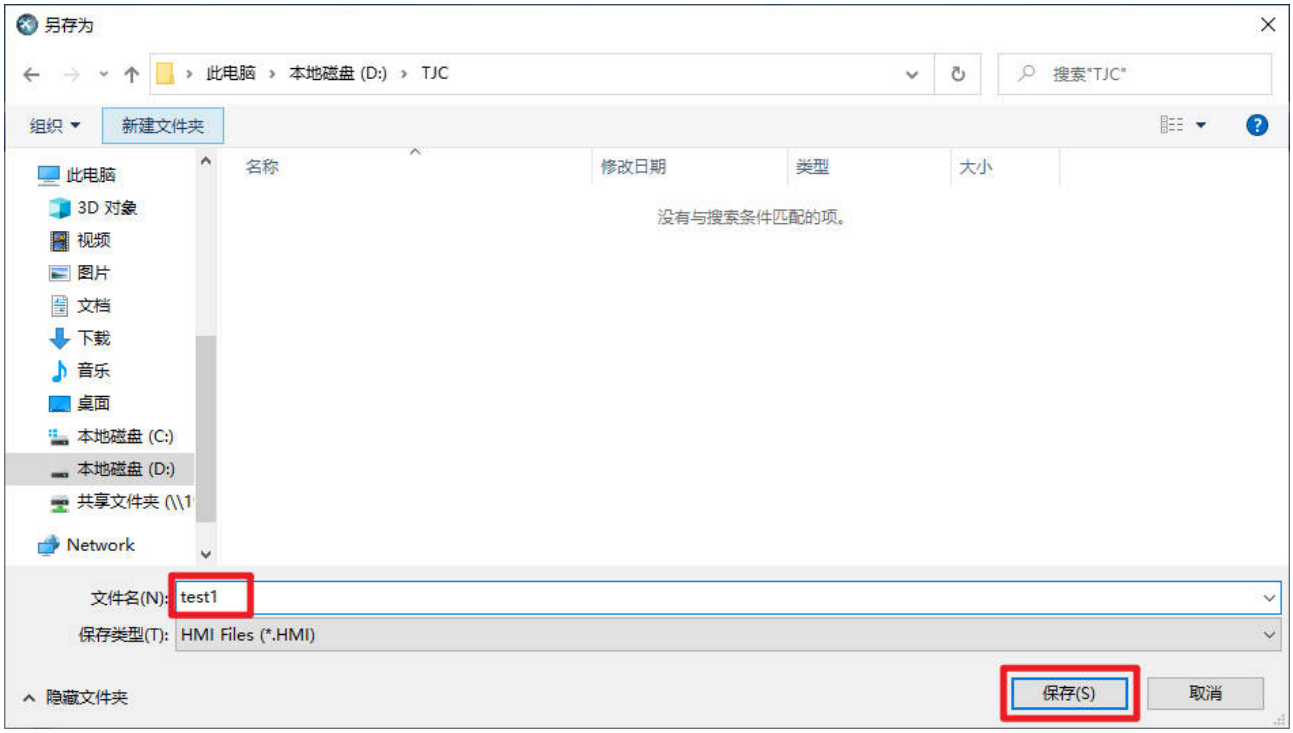
1.4.1.1 新建一个工程

演示工程素材下载 (page 105)

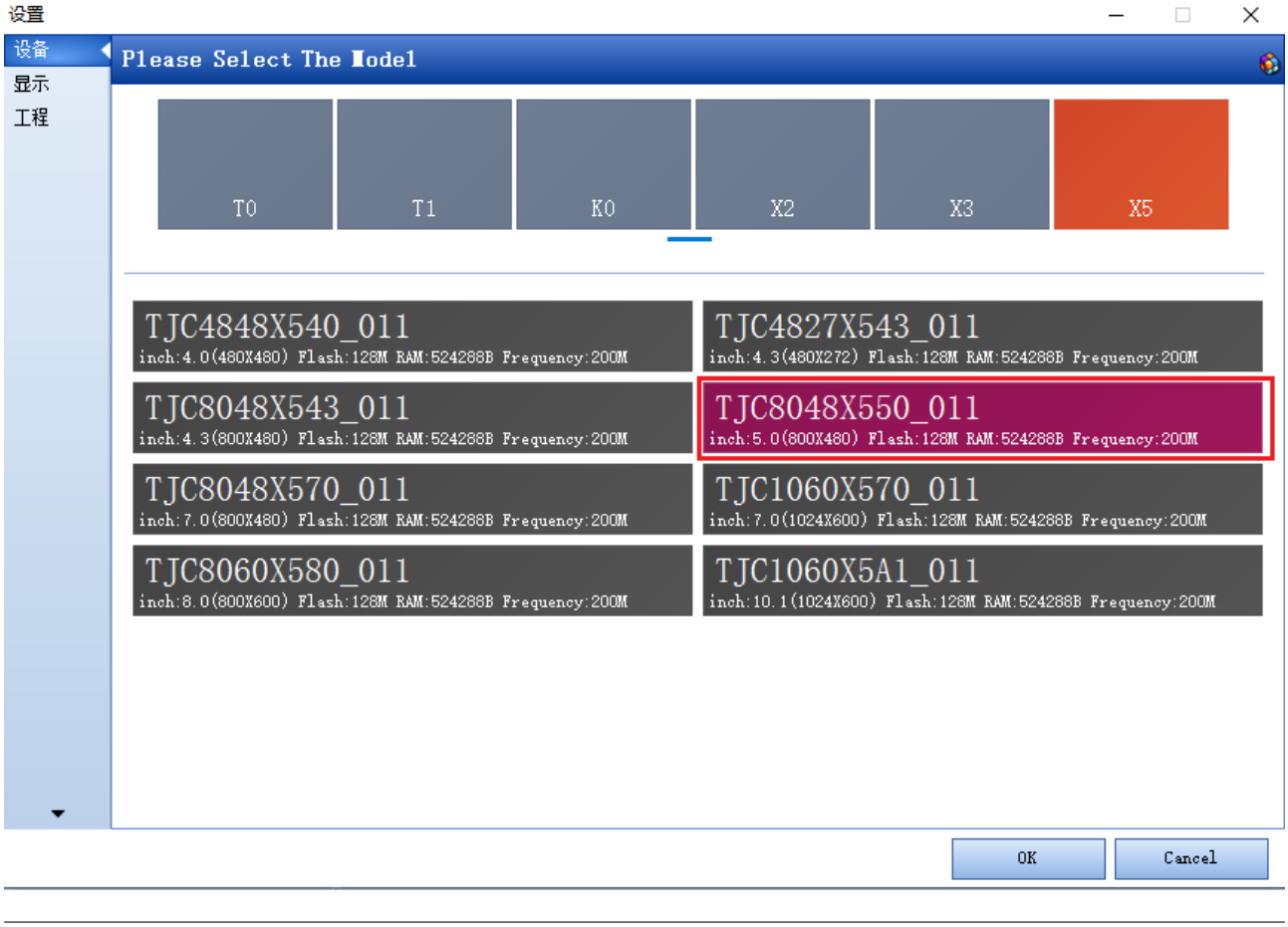
1. 点击新建按钮



2. 选择保存工程的位置，并为工程设置文件名，点击保存



3. 选择串口屏型号，您可以在串口屏的背面看到屏幕的型号，这里以 TJC8048X550_011 来演示

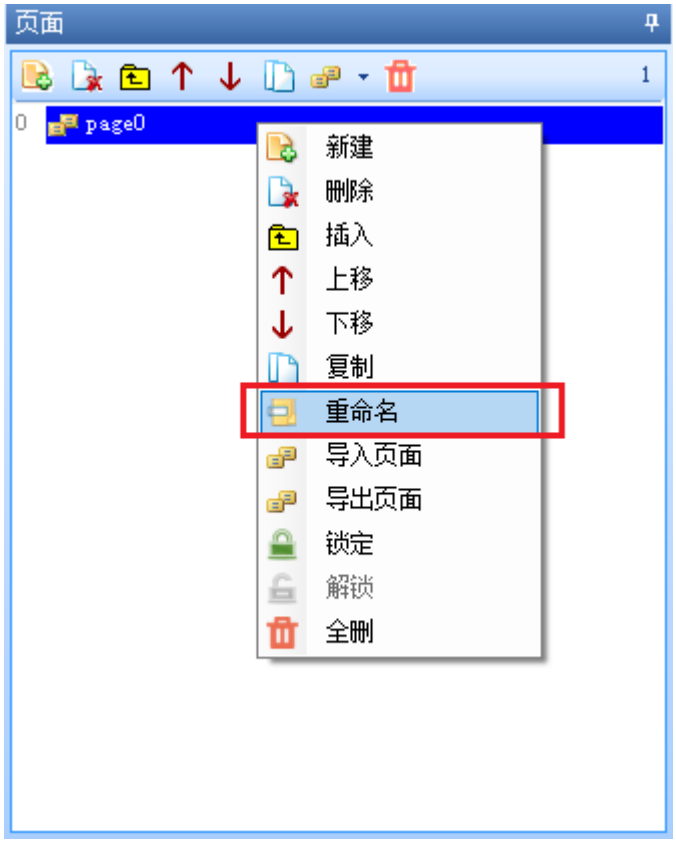


4. 点击左侧的显示选项，选择一个显示方向，并点击 OK，完成工程的创建

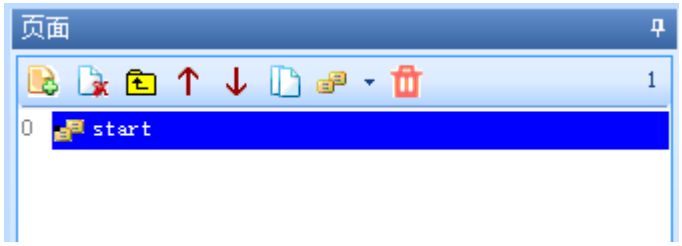
1.4.1.2 制作开机页面

演示工程素材下载 (page 105)

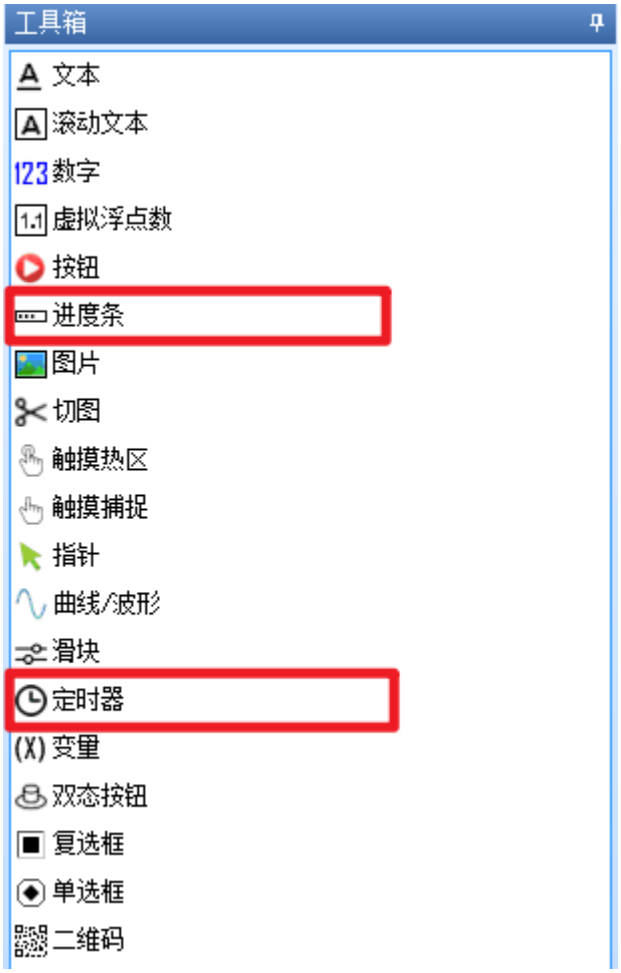
1. 右键选中页面，选择重命名，将页面名称改为 start(你也可以改为其他名称，建议使用英文名称，不建议使用中文或其他非 ascii 字符的文字)



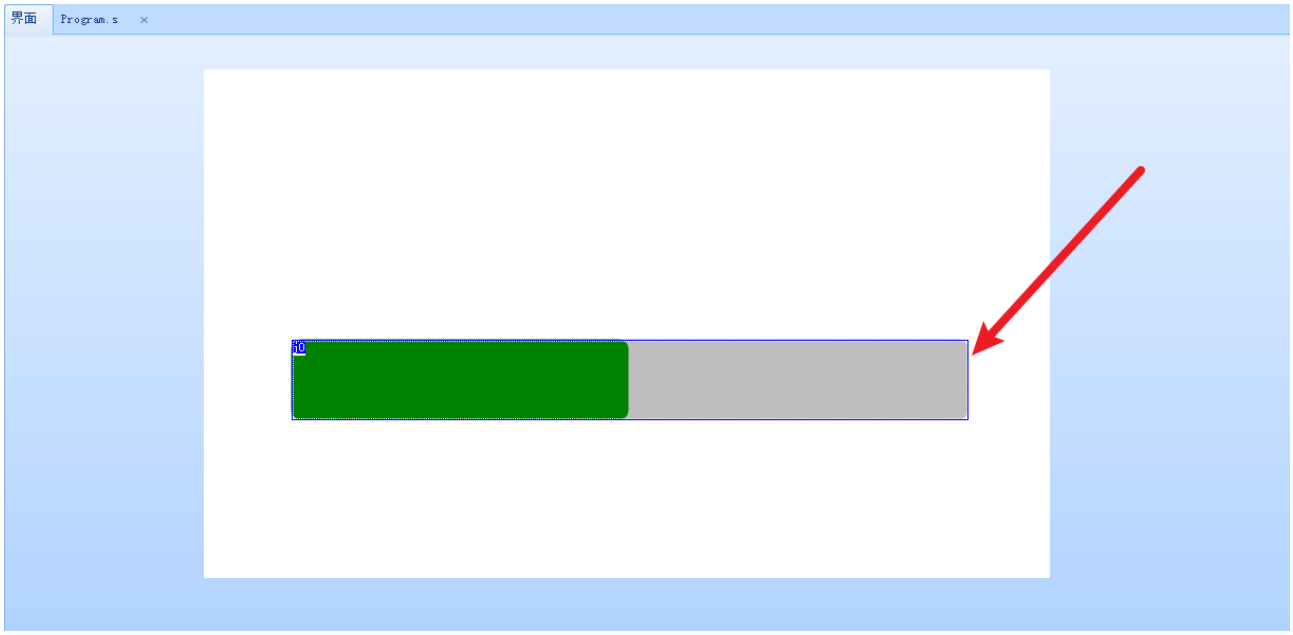
修改完成后如下所示



2. 在当前页面添加一个进度条控件和一个定时器控件

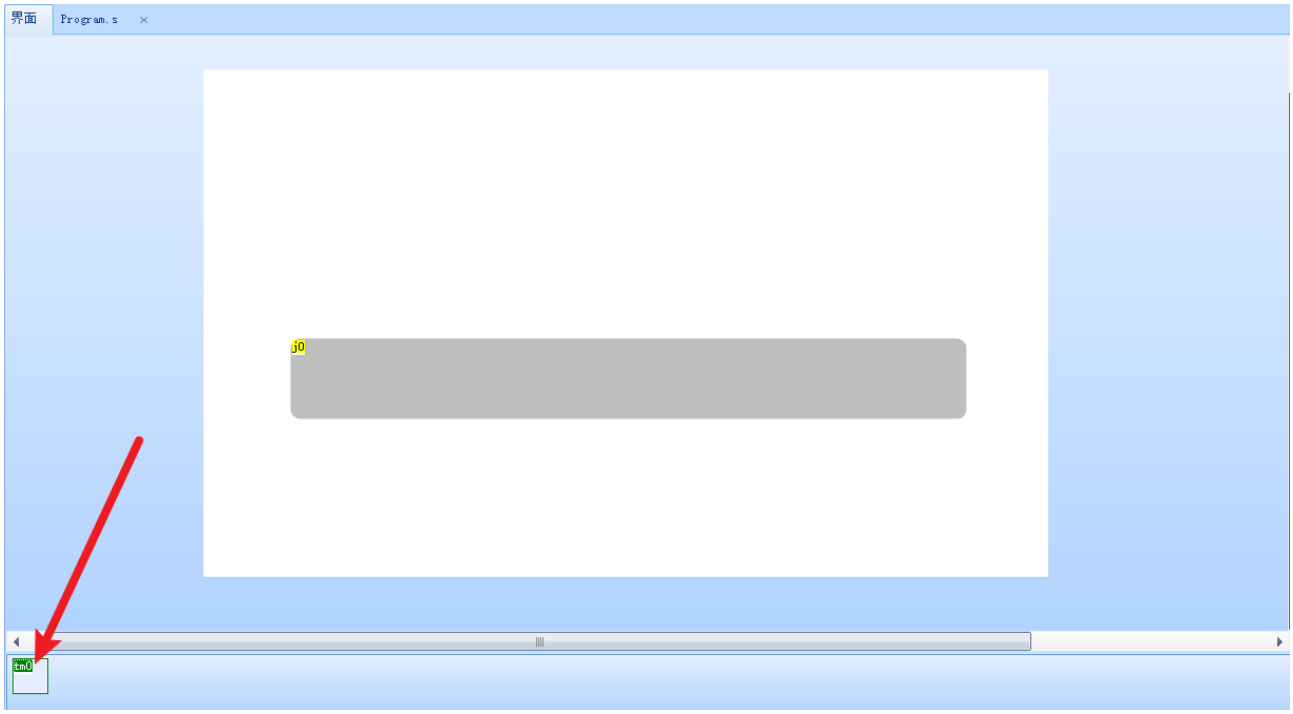


3. 拖动进度条的边缘可以修改进度条的大小，将进度条调整到合适的大小后，拖到合适的地方

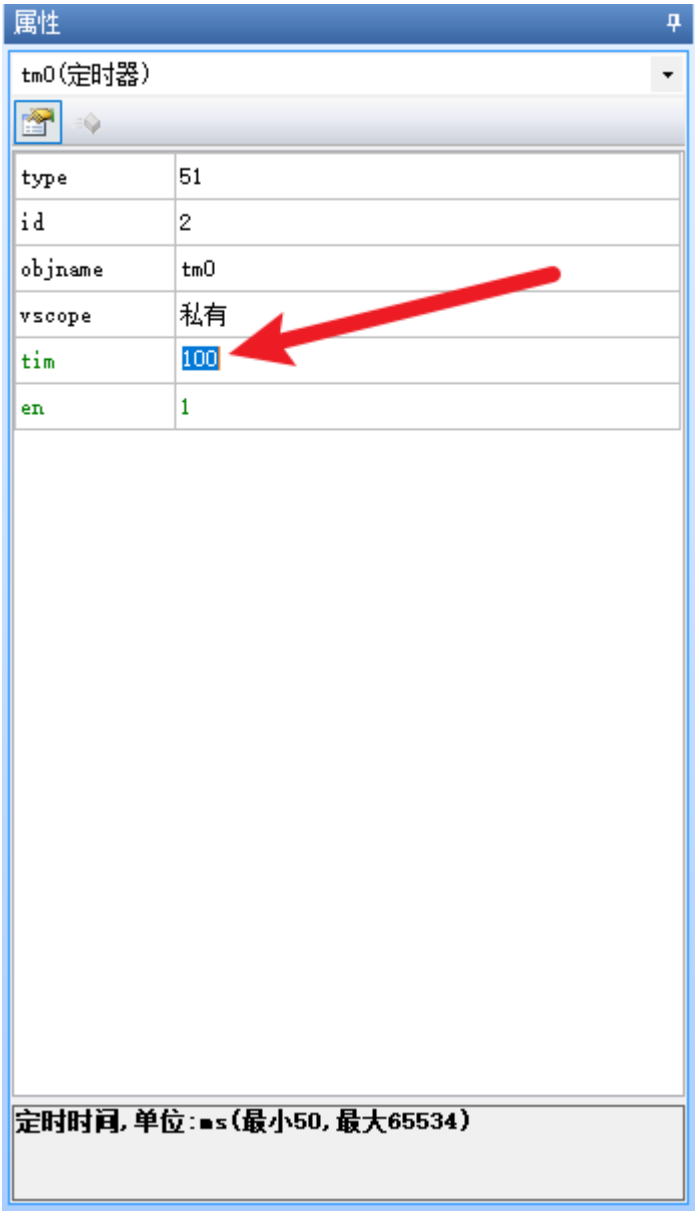


4. 右下角找到进度条控件的属性设置，将 val 改为 0（进度条的进度值百分比）

5. 找到定时器控件（定时器控件位于特殊控件栏）



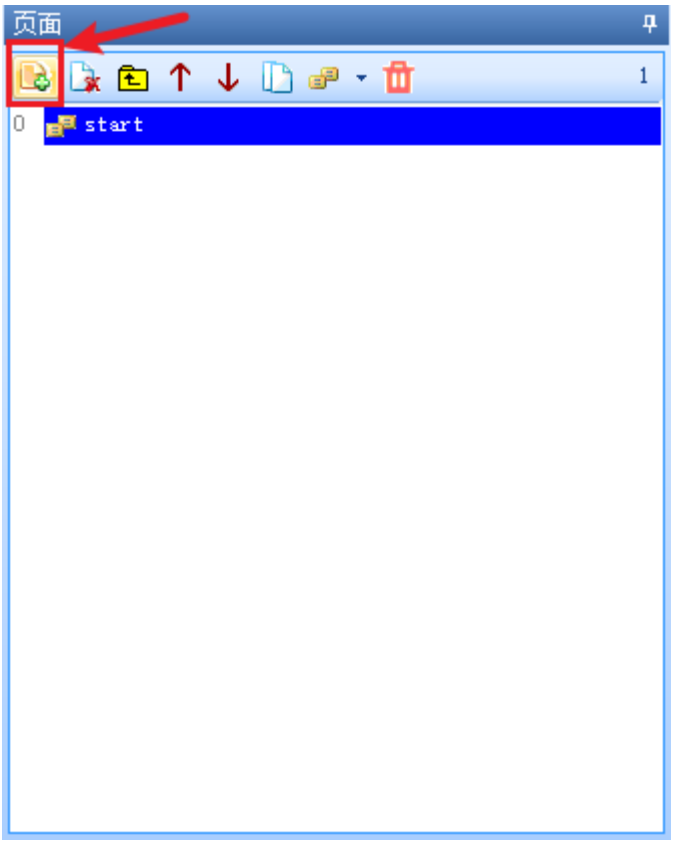
6. 右下角找到定时器控件的属性设置，将 tim 改为 100（即多久执行一次定时器里的代码）



1.4.1.3 制作主页面 1

演示工程素材下载 (page 105)

1. 点击添加按钮，新建一个页面

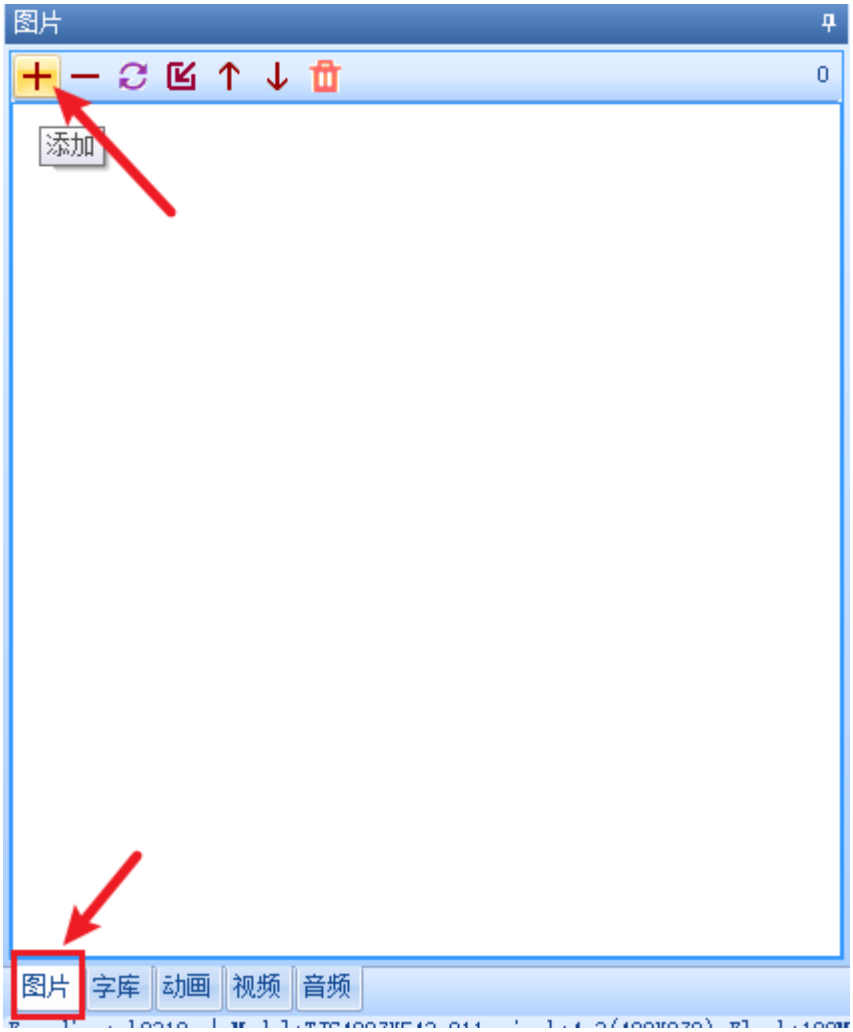


---

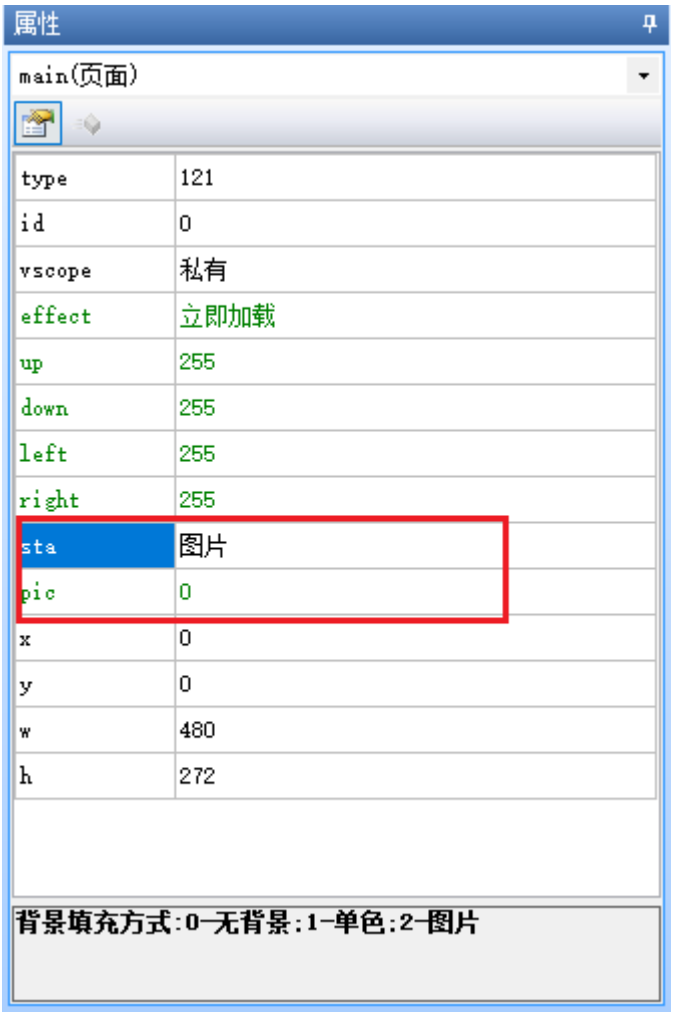
2. 将页面名称改为 main(你也可以改为其他名称，建议使用英文名称，不建议使用中文或其他非 ascii 字符的文字)

---

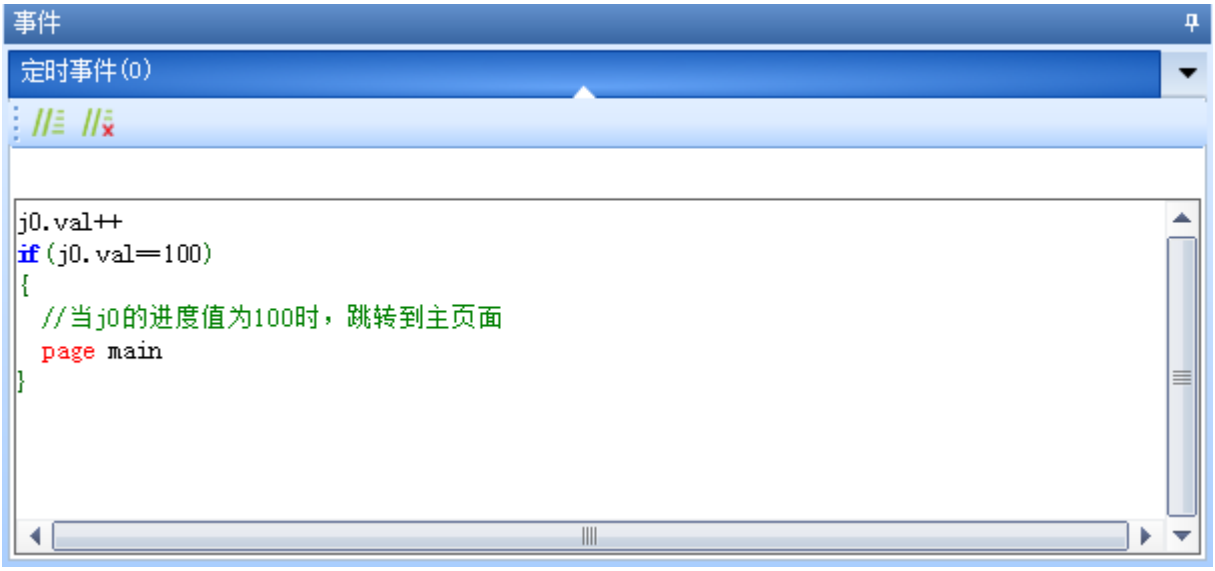
3. 选中图片标签，点击“+”，导入素材-图片-背景 1.png、背景 2.png、背景 3.png，共 3 张分辨率为 800*480 的全屏图片



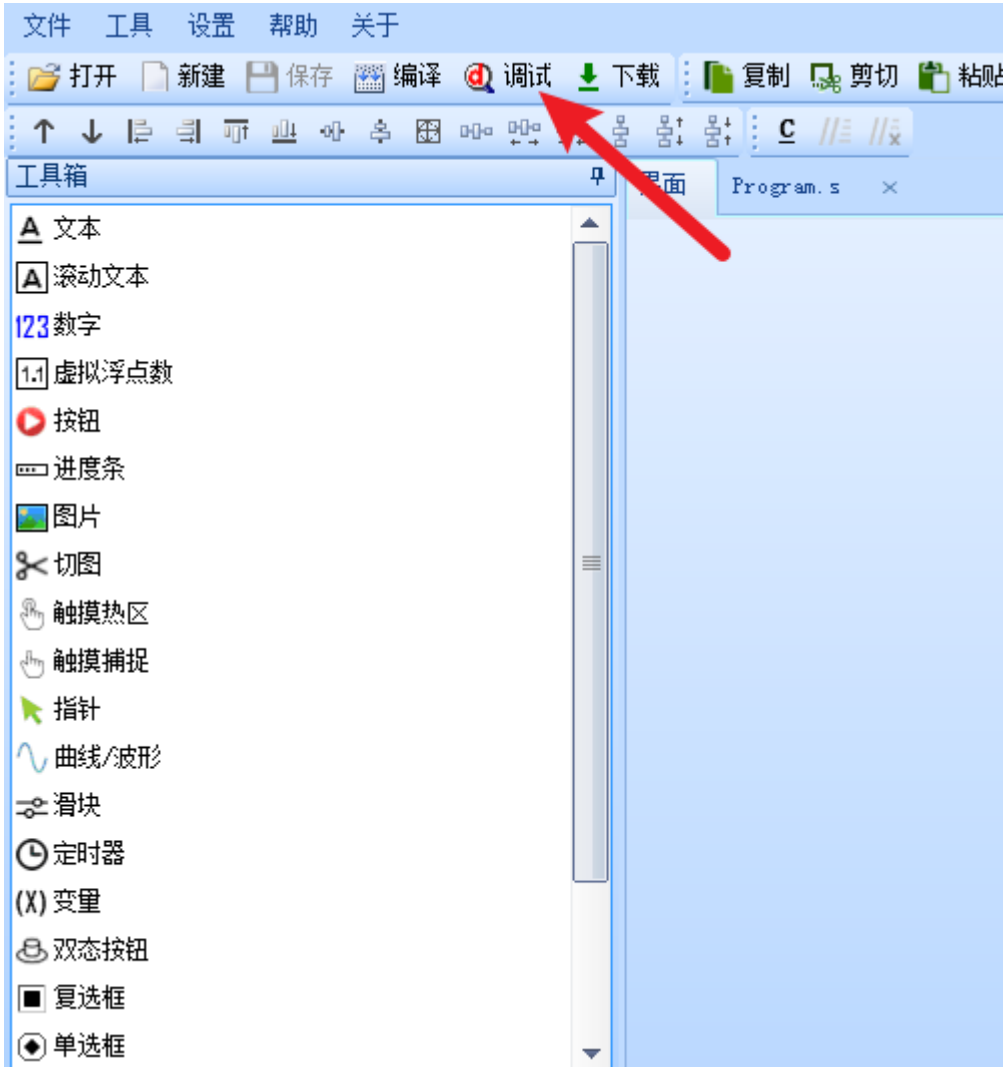
4. 点击页面空白处，设置页面控件的属性，将 sta 设置为图片，pic 选择刚刚导入的背景图片



5. 回到 start 页面，点击 tm0 控件（定时器控件），找到 tm0 控件的定时事件，添加以下代码

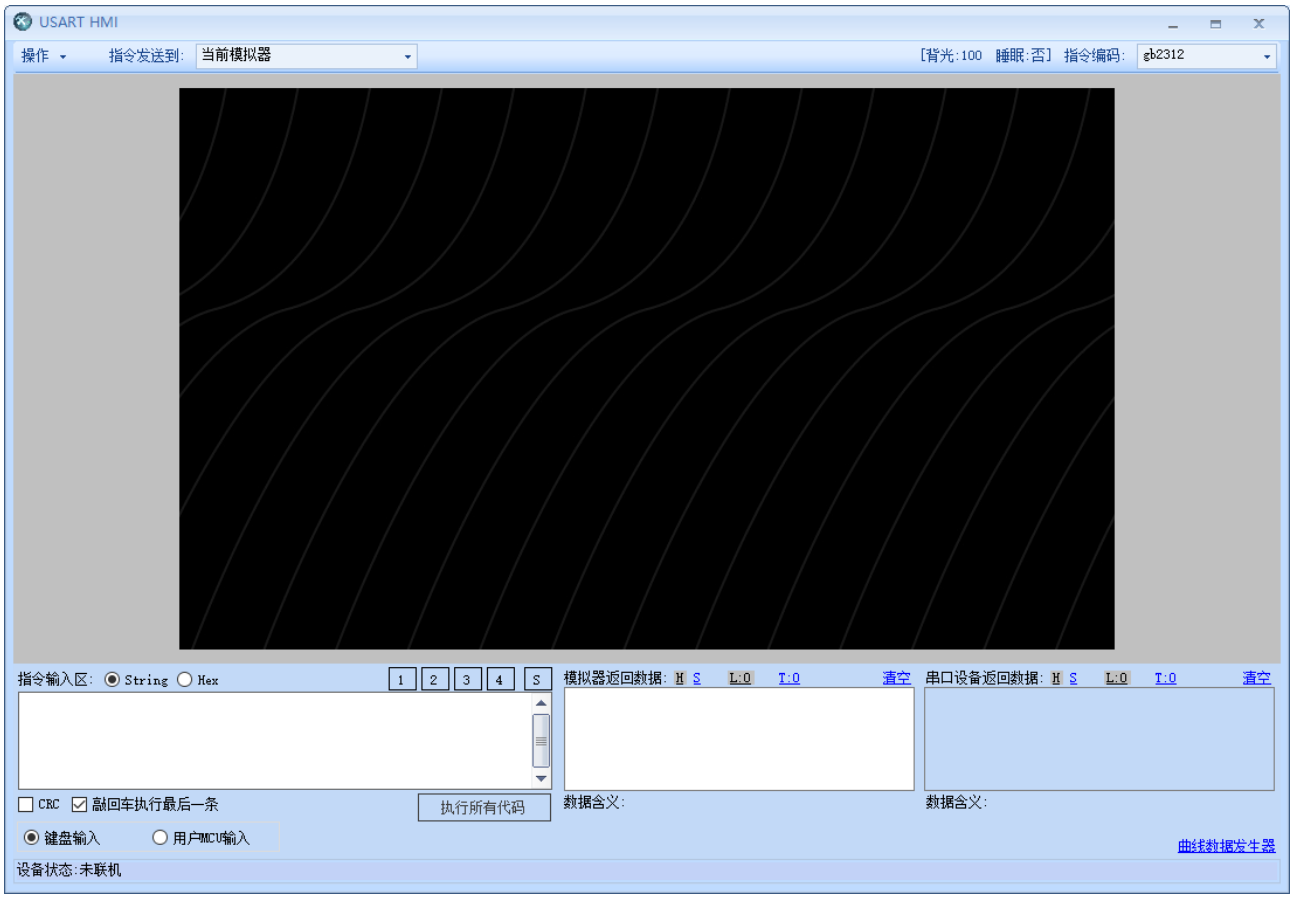


6. 点击调试。


文件
工具
设置
帮助
关于
打开
新建
保存
编译
调试
下载
复制
剪切
工具箱
Program.s
文本
滚动文本
数字
虚拟浮点数
按钮
进度条
图片
切图
触摸热区
触摸捕捉
指针
曲线/波形
滑块
定时器
变里
双态按钮
复选框
单选框

7. 如果没报错，就会自动打开模拟器窗口，如果报错，可以参考错误提示 (page 1342) 解决问题

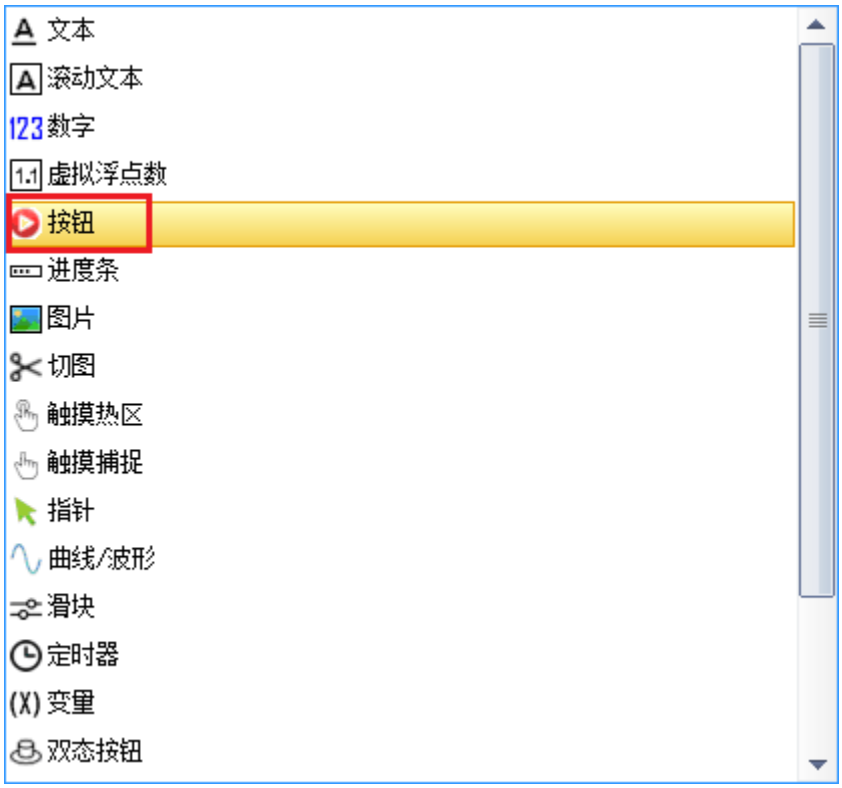
在进度值满后，将会自动跳转到 main 页面



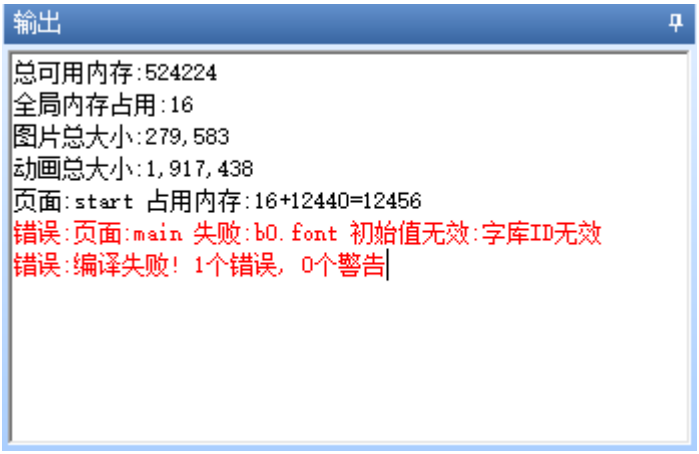
1.4.1.4 报错：字库 ID 无效

演示工程素材下载 (page 105)

1. 切换到 main 页面，点击工具箱的按钮，在 main 页面新建一个按钮控件

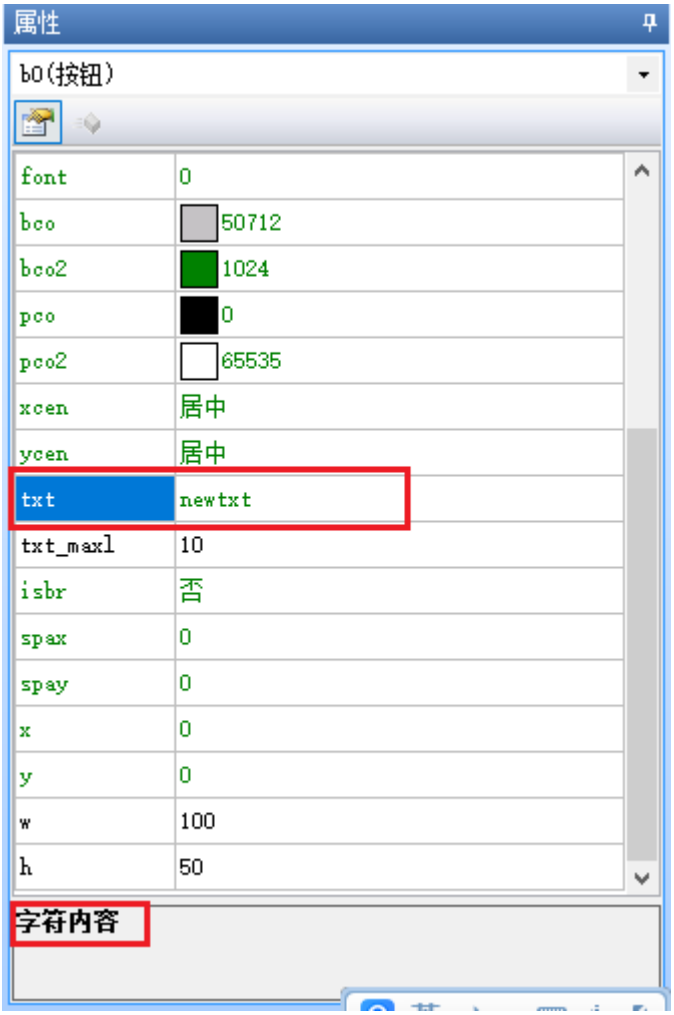


2. 此时，如果直接点击调试，输出框将会提示以下信息”初始值无效: 字库 ID 无效”



---

3. 这是因为按钮上会显示文字，需要调用对应的字库来显示，如果我们没有导入字库，便会提示字库 ID 无效



同理

当调用了图片，但是图片资源不存在时或者没有配置时，会显示图片 ID 无效

当调用了动画，但是动画资源不存在时或者没有配置时，会显示动画 ID 无效

当调用了视频，但是视频资源不存在时或者没有配置时，会显示视频 ID 无效

当调用了音频，但是音频资源不存在时或者没有配置时，会显示音频 ID 无效

1.4.1.5 创建字库和导入字库

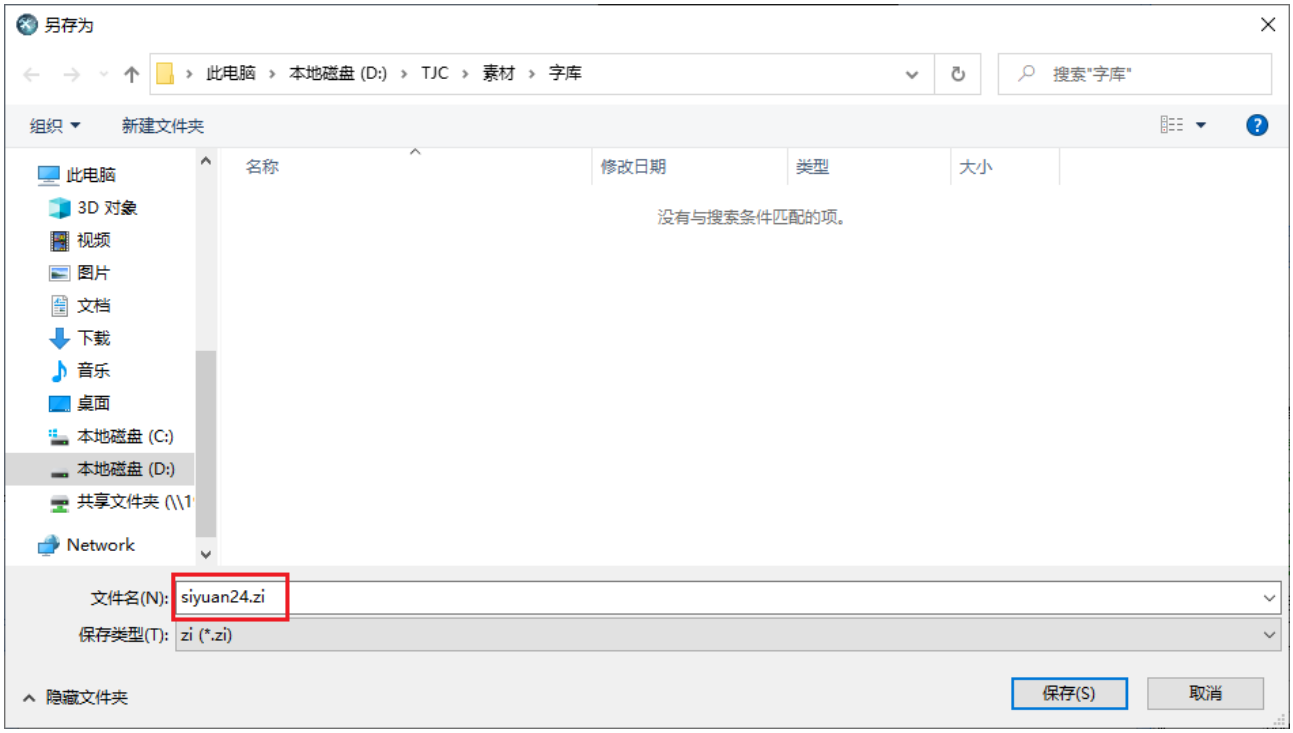
演示工程素材下载 (page 105)

1. 点击工具-字库制作

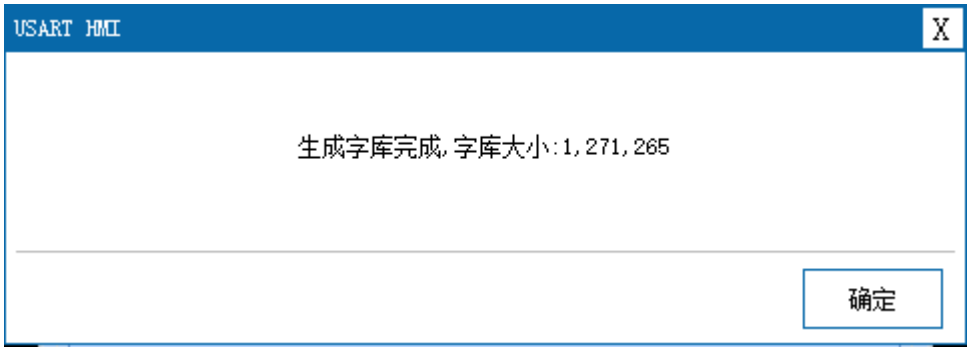
2. 设置如下

字库制作工具
字高: 24
编码: gb2312
字体抗锯齿
字体: 思源黑体 CN Regular
字体加粗
预览区
散
范围: 所有字符
字库名称: siyuan24
生成字库

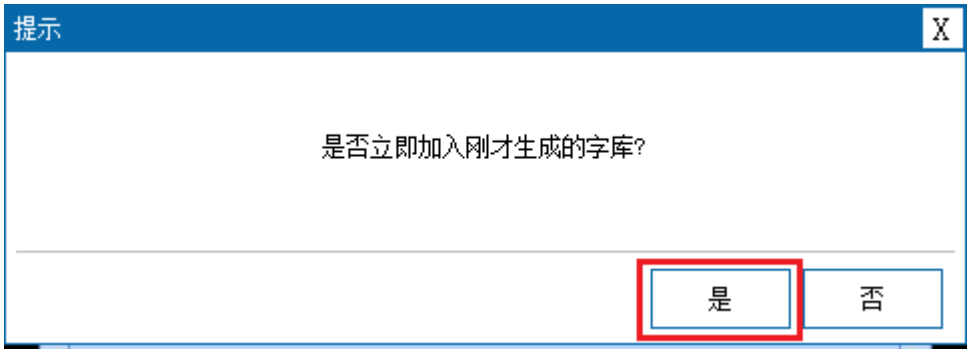
3. 点击生成字库，将字库保存到素材-字库中



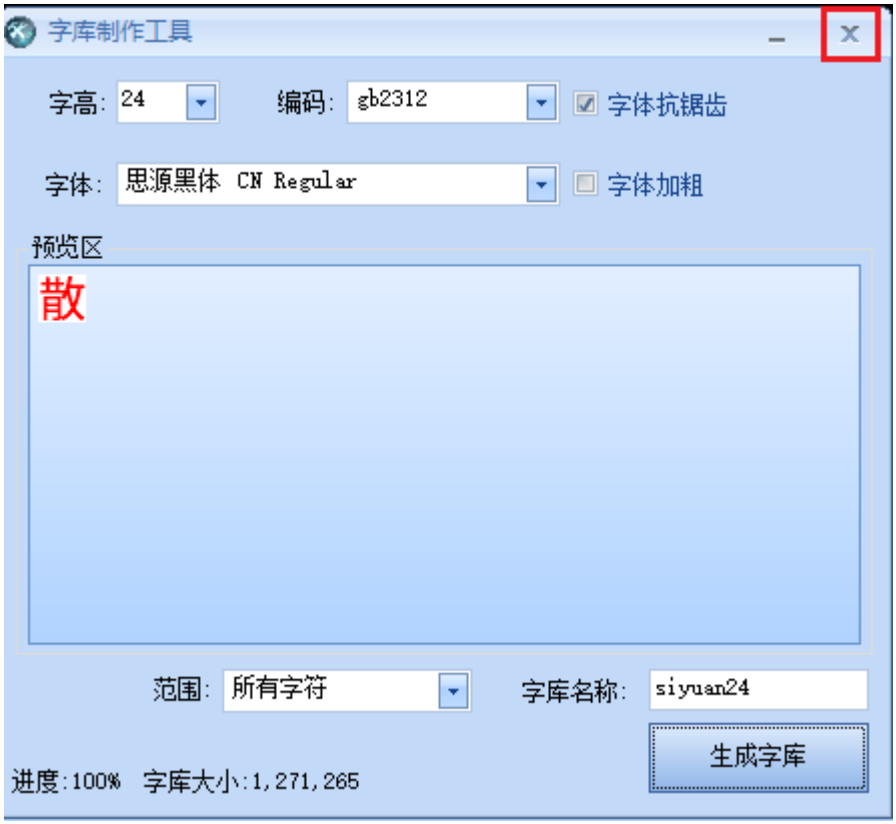
4. 生成完毕，显示字库占用空间，点击确定



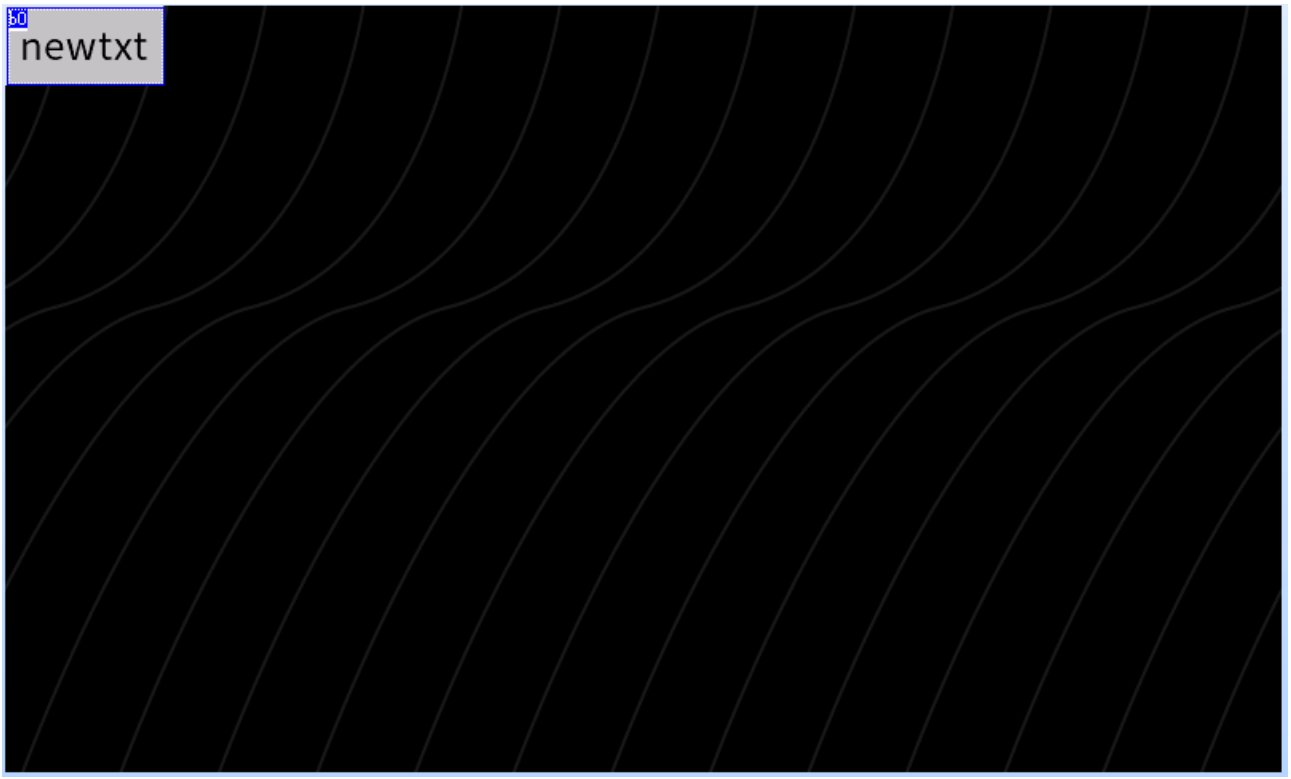
5. 询问是否加入字库，点击“是”



6. 关闭字库制作工具，或者继续制作其他字库

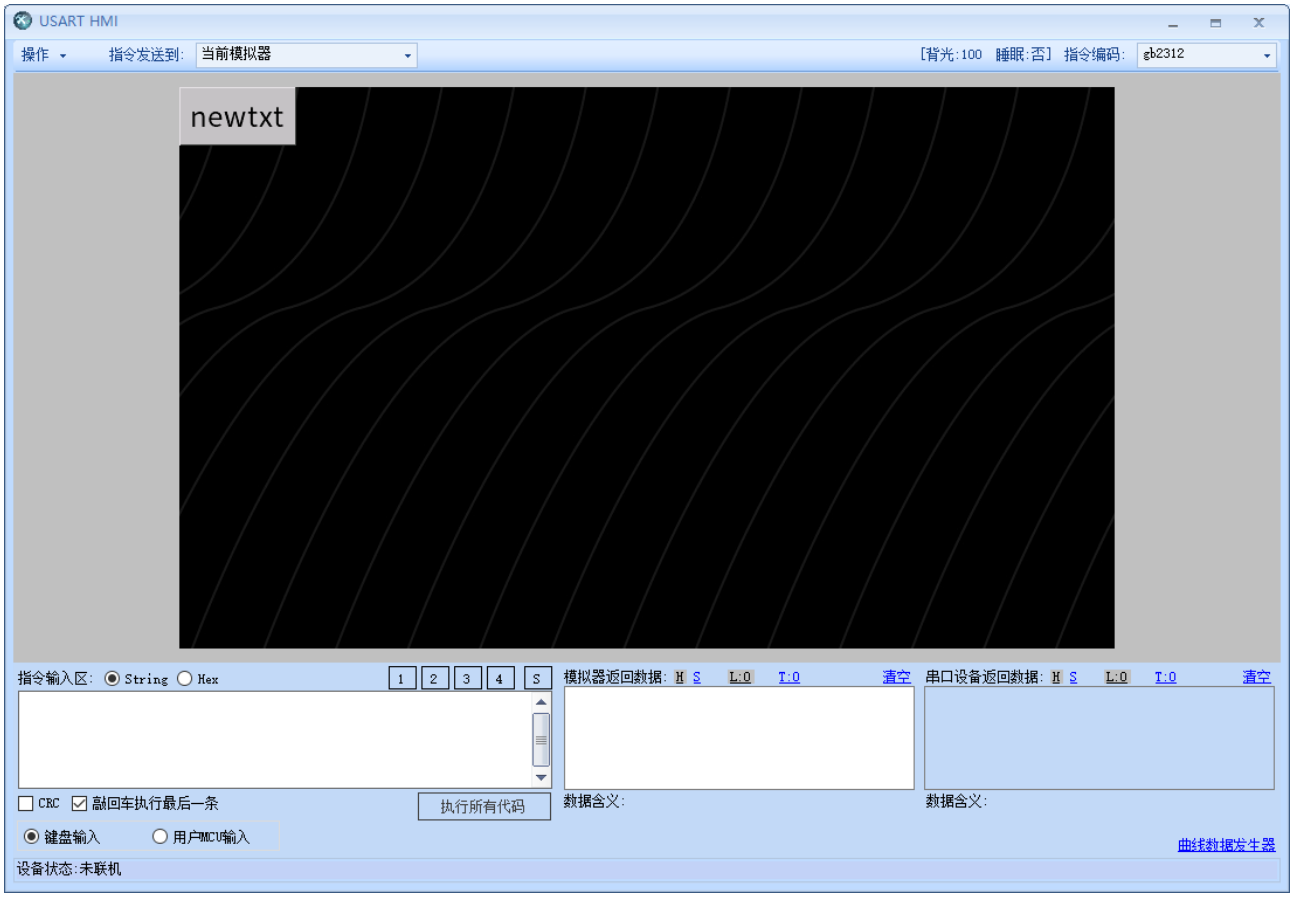


7. 此时按钮控件可以正常显示文字 (需要点击一下控件刷新上面的文字)



---

8. 点击调试也不会报错，可以正常运行



1.4.1.5.1 创建字库-相关链接

指定字库 (page 1284)

在做字库的时候有部分字体无法选择 (page 1360)

编译报错：*XXX* 初始值无效 (page 1342)

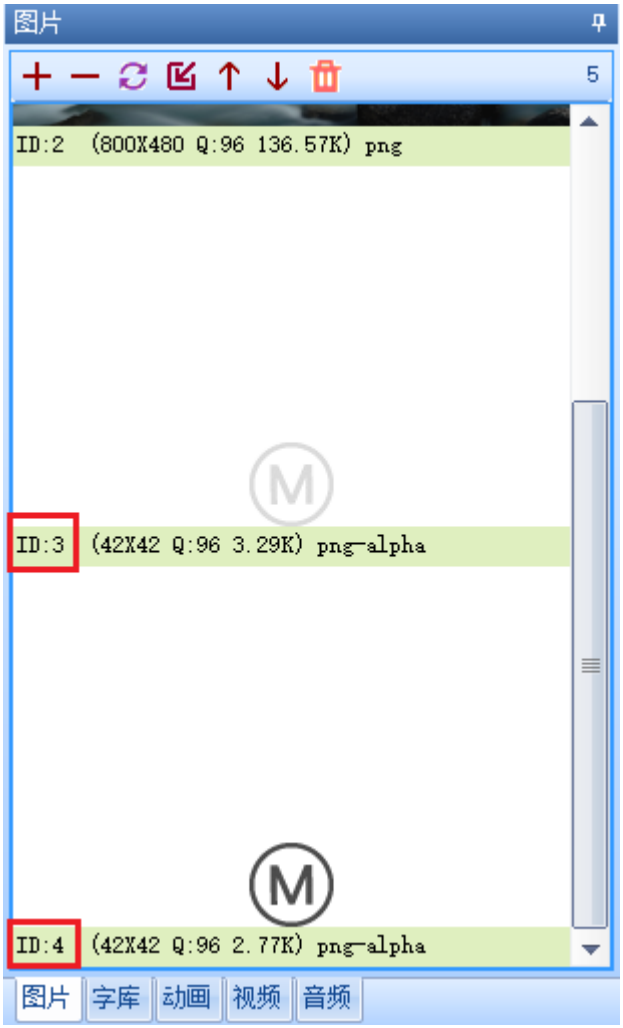
utf8 字库下如何只选择汉字 (page 1403)

报错:*file is too large for destination device* (page 1359)

1.4.1.6 按钮控件美化

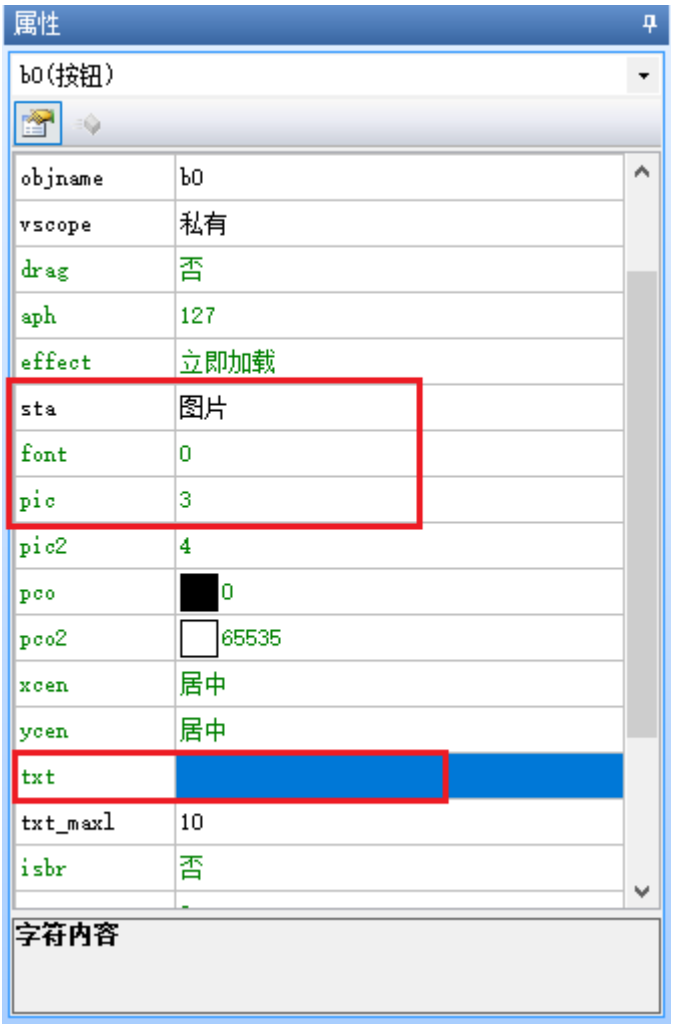
演示工程素材下载 (page 105)

1. 选中图片标签，点击“+”，导入素材-图片-半透明按钮 1.png、半透明按钮 2.png，其图片 ID 分别为 3 和 4

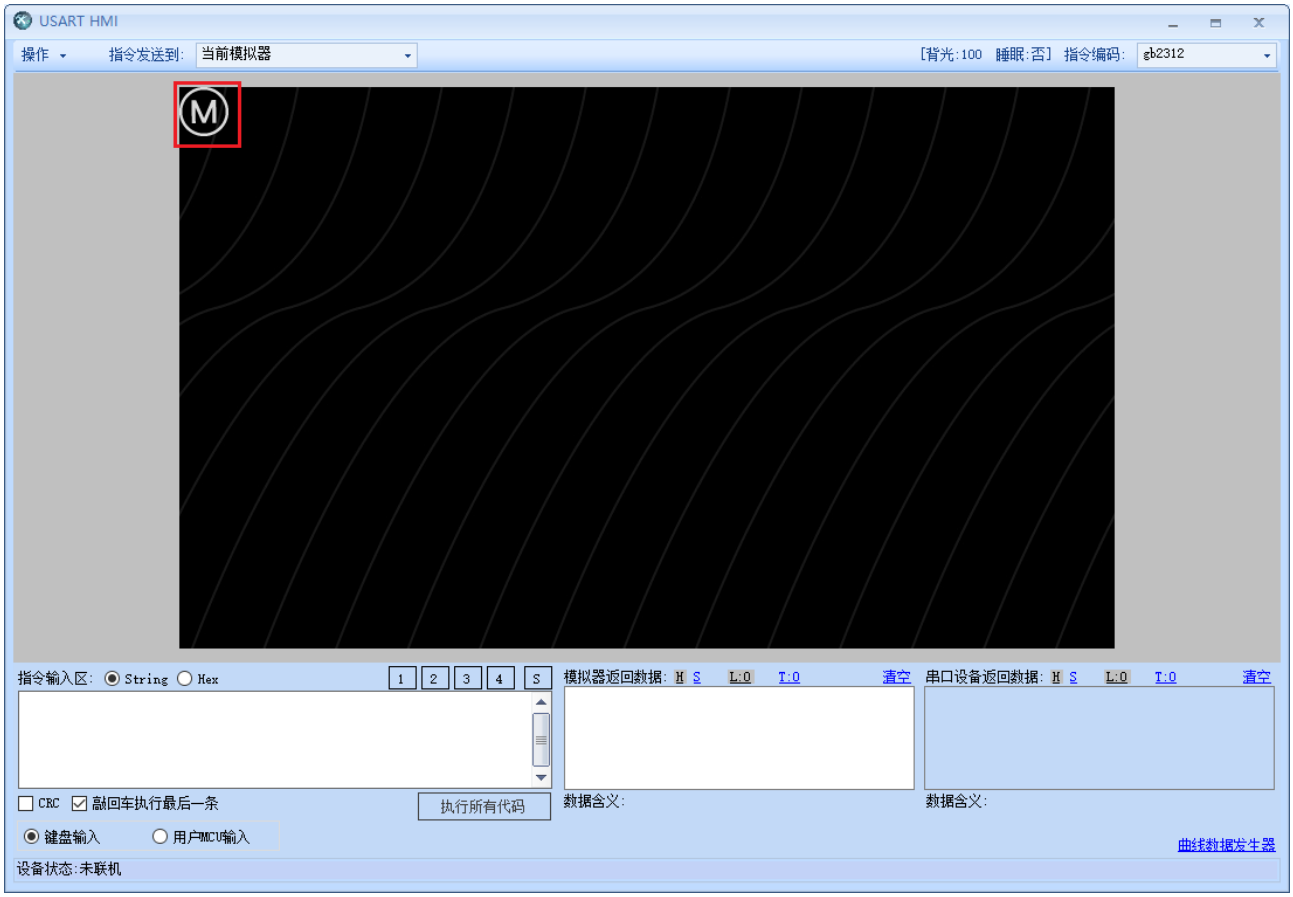


---

2. 选中 b0 按钮控件, 设置其 sta 属性为图片，pic 属性为 3，pic2 属性为 4，并且将 txt 属性清空



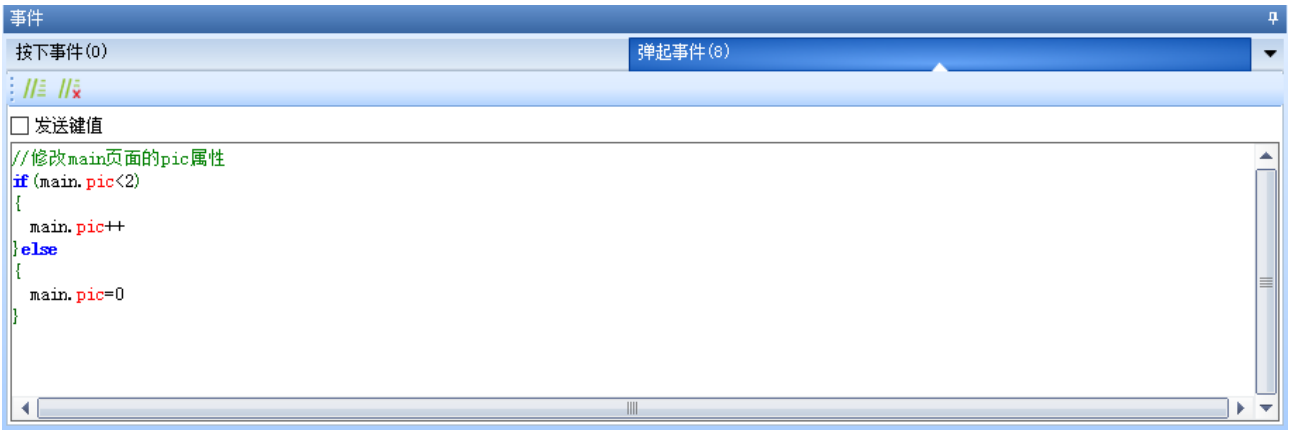
3. 点击调试，可以通过鼠标点击 b0 按钮控件查看按键按下效果



1.4.1.7 通过代码修改控件属性

演示工程素材下载 (page 105)

1. 选中 b0 按钮控件，在弹起事件中编写如下代码

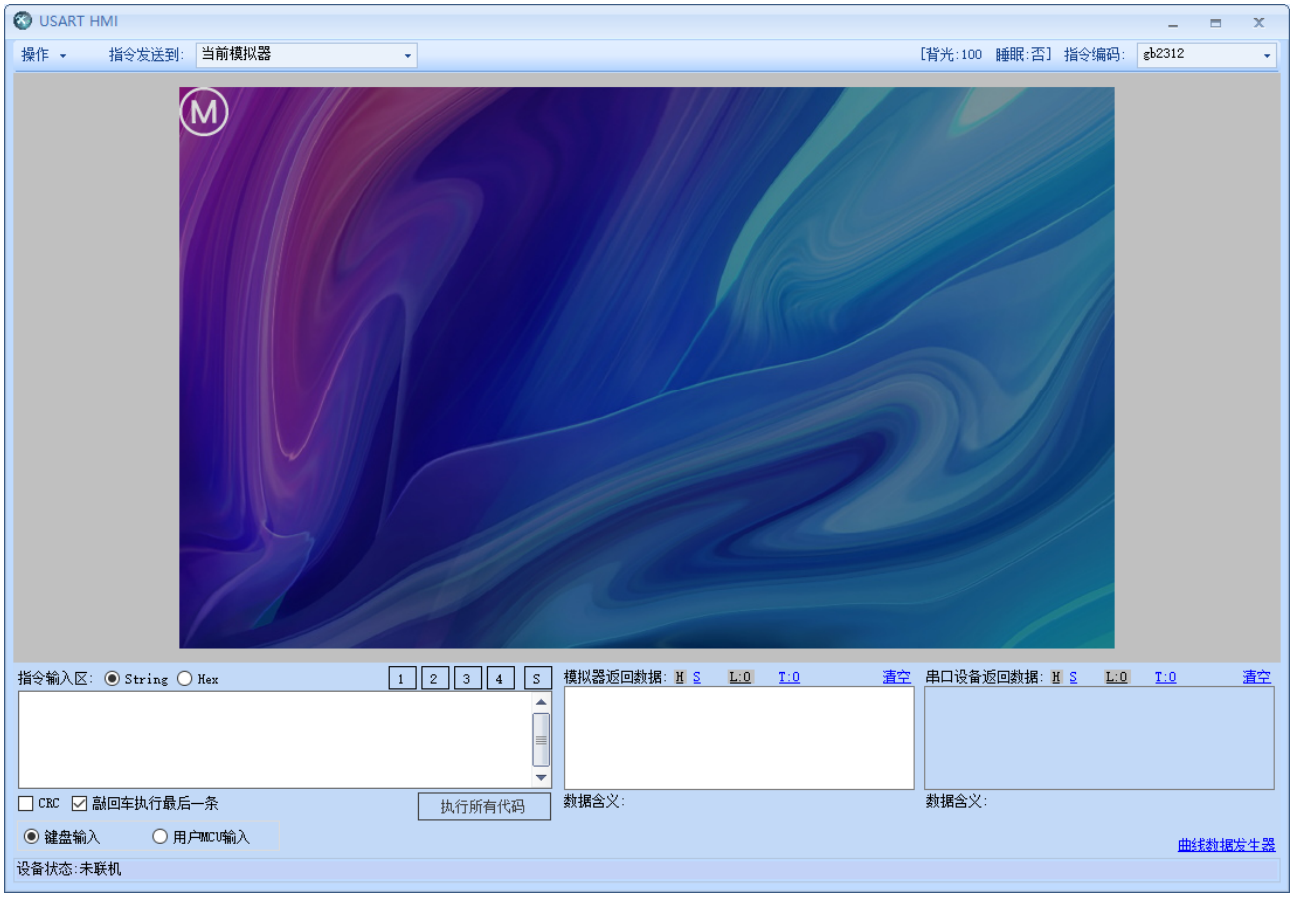


代码如下:

```
if(main.pic<2)
{
    main.pic++
}else
{
    main.pic=0
}
```

---

2. 点击调试，此时可以通过点击 b0 按钮控件，实现切换 main 页面的背景图片



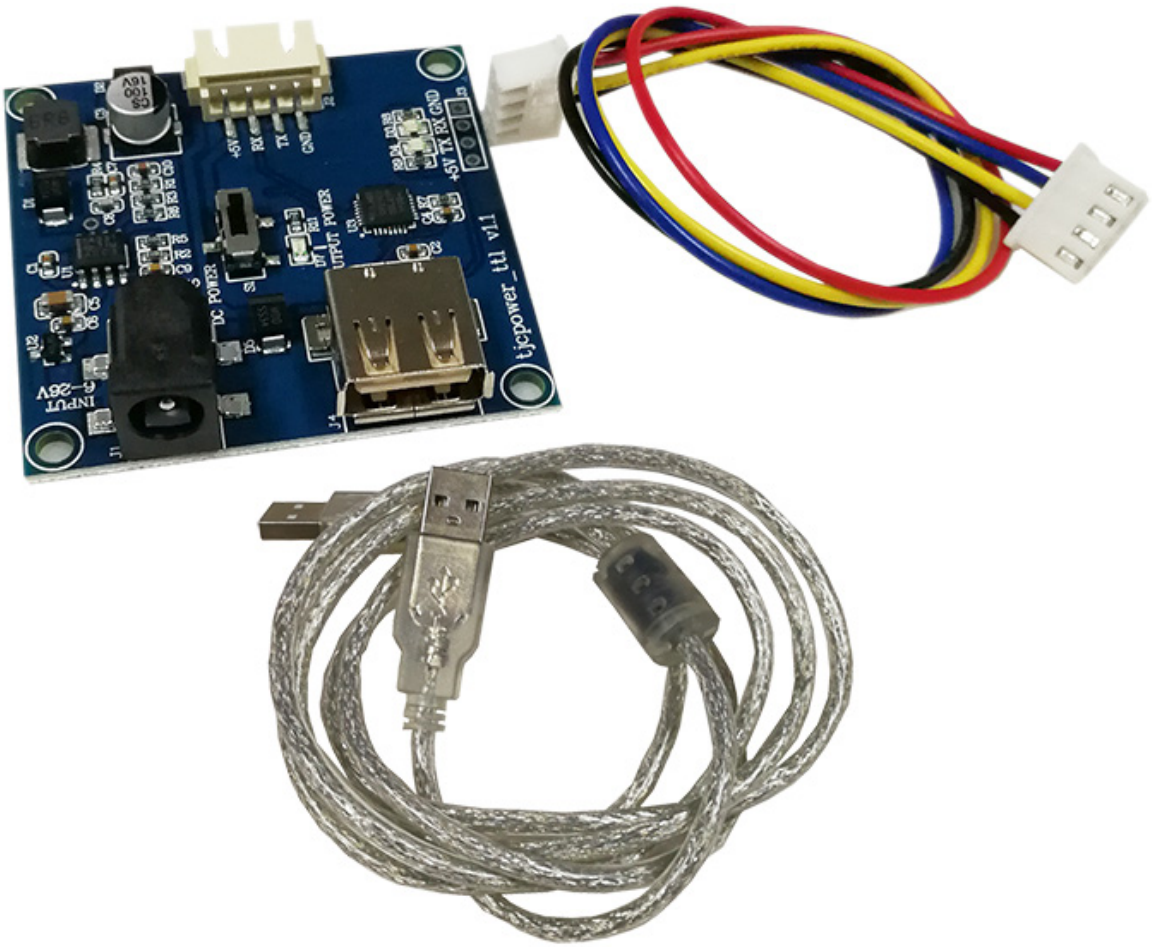
1.4.1.8 安装串口驱动

如果我们想下载到串口屏中查看实际的效果，需要使用到串口工具，推荐使用 cp2102 或者 ft232，不推荐使用 ch340 或者 pl2303

推荐使用官方的串口下载套件

此套件为 USB 转 TTL 工具, 芯片为 CP2102

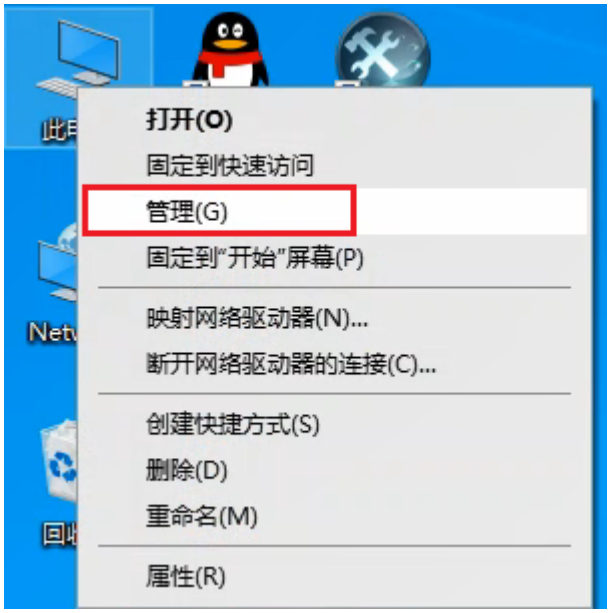
为什么不建议用 *ch340* (page 1248)



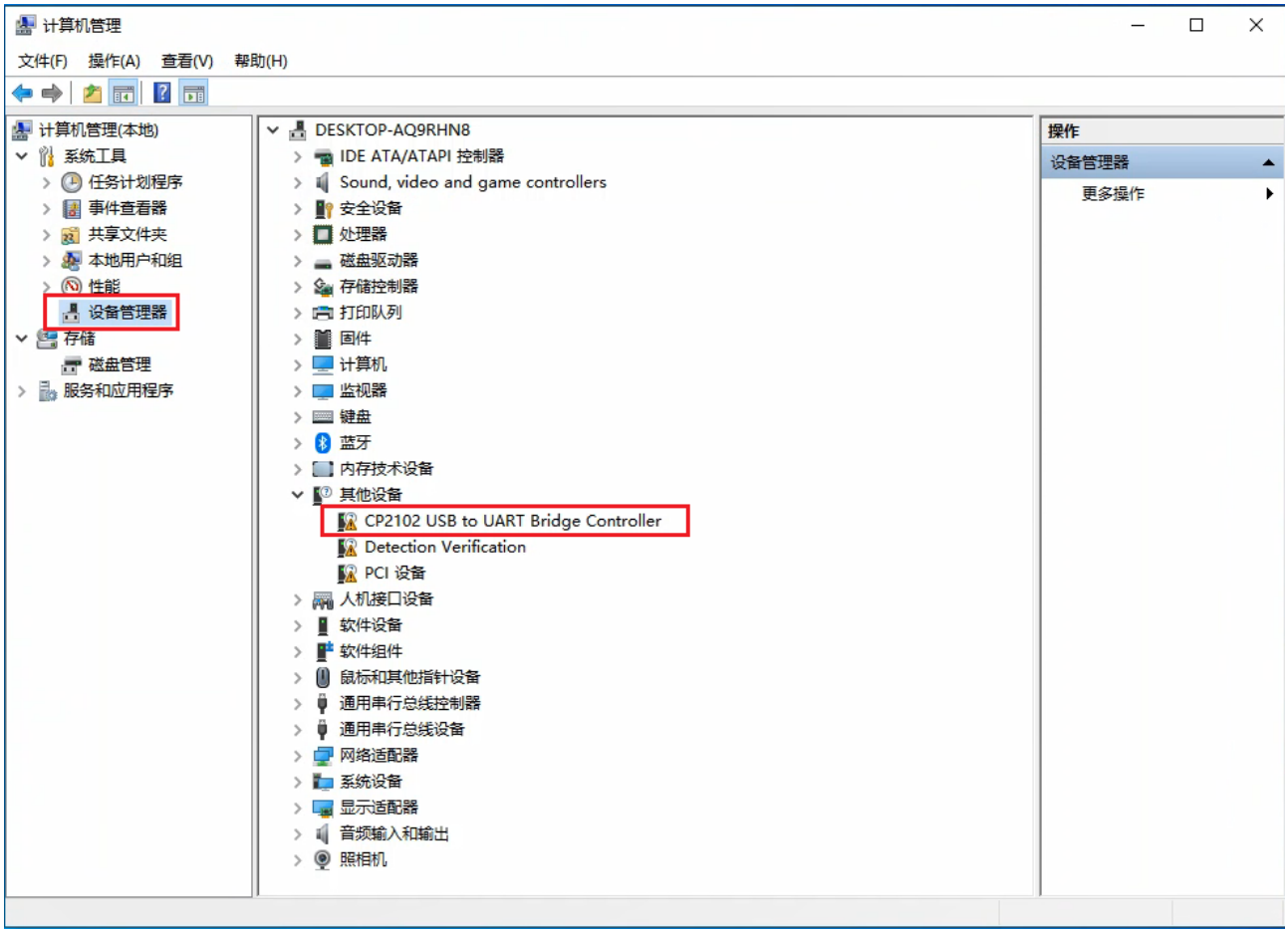
注意： 如果您购买的屏大于 5 寸或者有使用喇叭的需求，请再购买一个 12V 电源用于额外供电，否则可能因为电脑 USB 供电不足导致开机启动失败

12V 2A电源

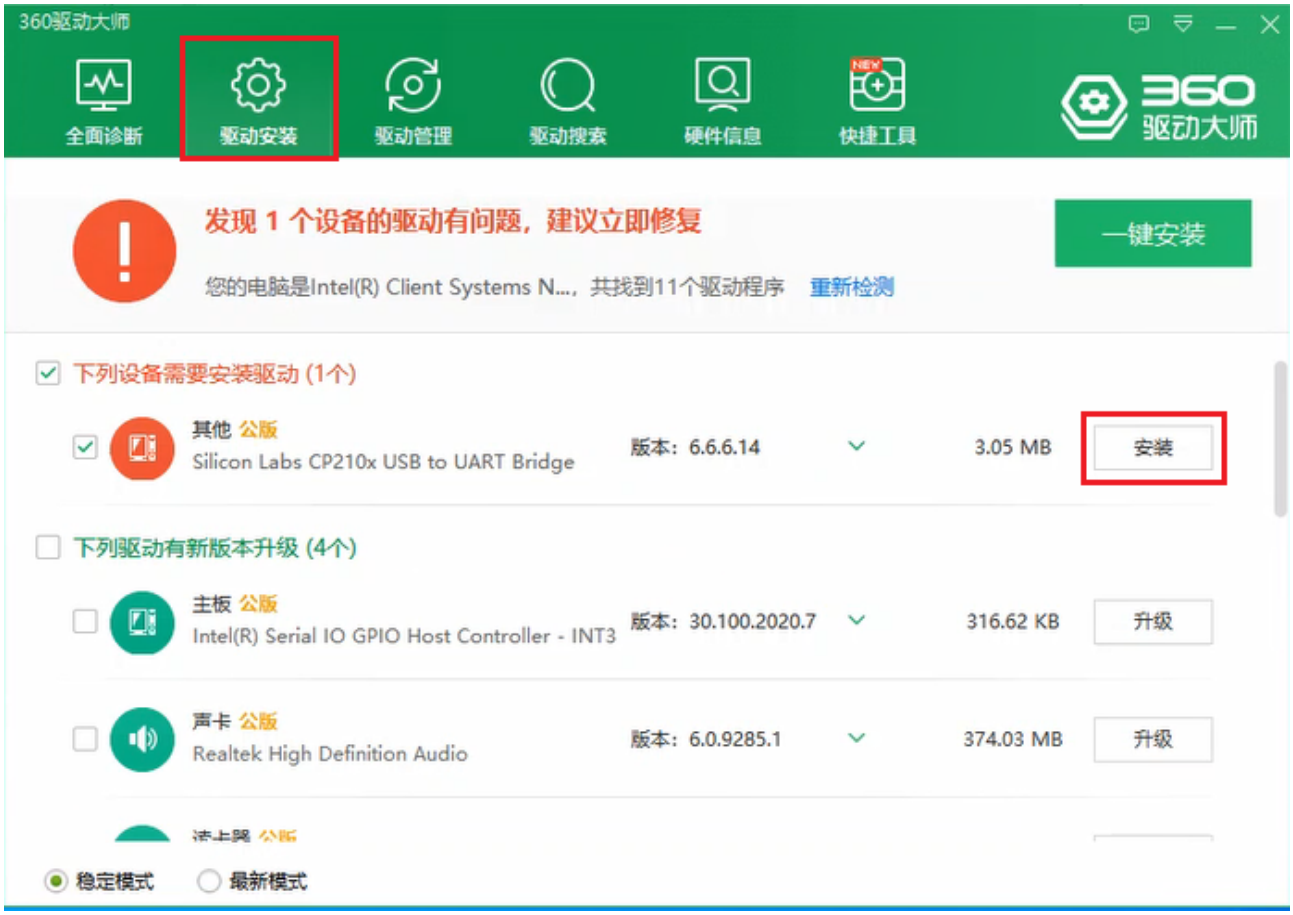
1. 右键此电脑-管理


打开(O)
固定到快速访问
管理(G)
固定到"开始"屏幕(P)
映射网络驱动器(N)...
断开网络驱动器的连接(C)...
创建快捷方式(S)
删除(D)
重命名(M)
属性(R)

2. 可以看到其他设备下有一个 cp2102 的串口工具缺少驱动



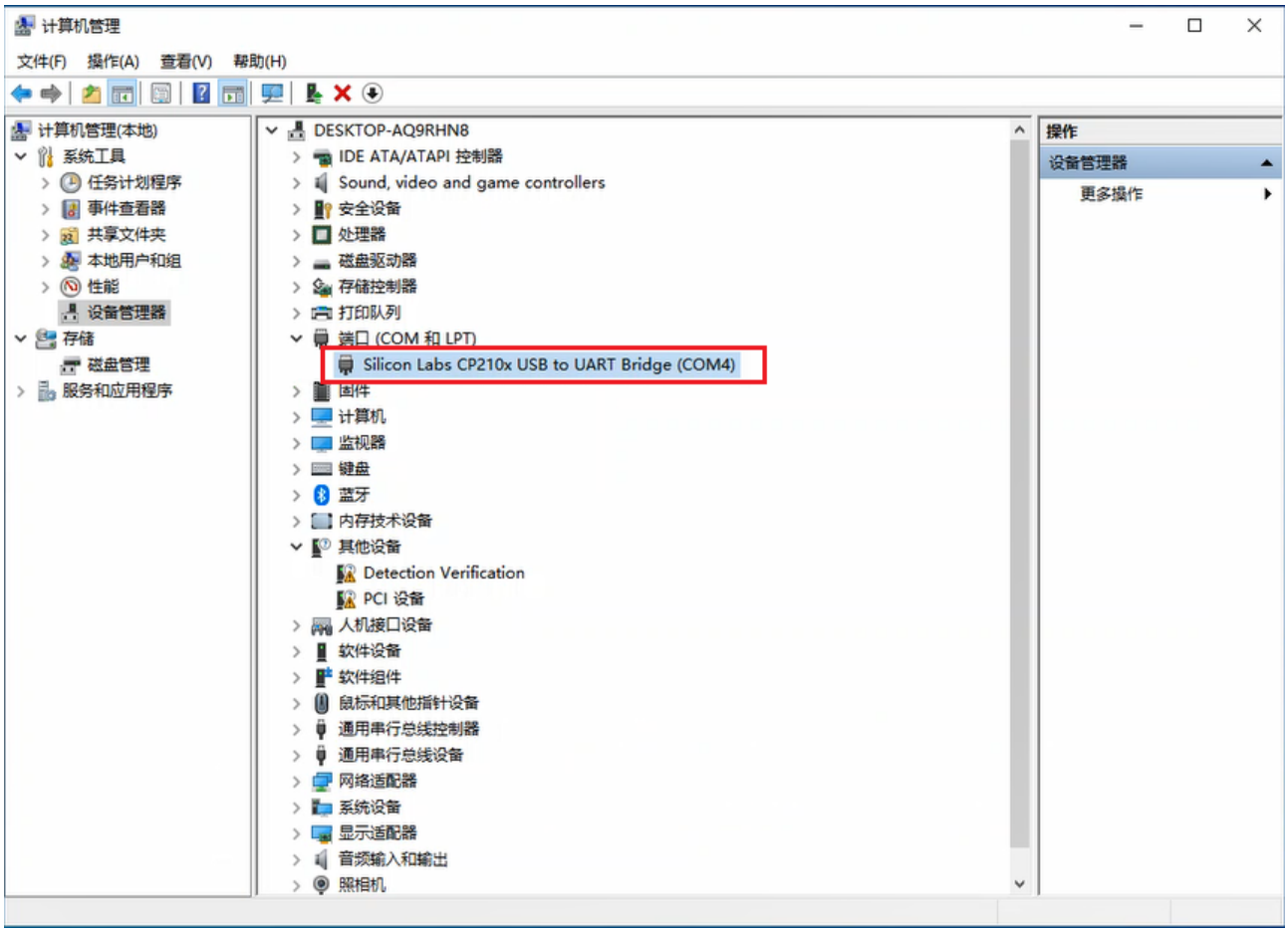
3. 推荐大家使用 360 驱动大师，选择驱动安装，会自动扫描出缺少的驱动，点击安装，即可自动完成安装



您也可以直接下载驱动 (驱动来自网络，淘晶驰不保证其可靠性和安全性)

常见串口驱动下载 (page 1104)

4. 安装完成后，可以在端口中看到 cp210x 设备，具体的端口号是 com4，请记住你自己电脑上对应的端口号



也可以直接下载串口驱动进行安装

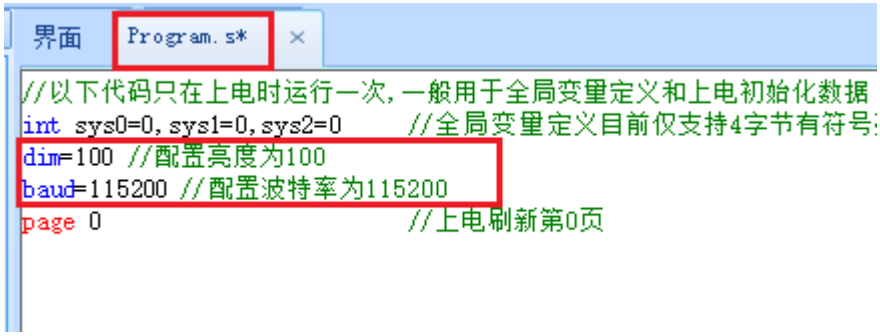
1.4.1.8.1 串口驱动下载

常见串口驱动下载 (page 1104)

1.4.1.9 通过串口下载工程到串口屏

1. 下载前先切换到 program.s 中，配置一下亮度和波特率，请注意，等号两边不要有空格，否则会报错

注意这里的波特率应该和单片机的波特率一致。



2. 连接下载器和屏幕

备注： 连接方式如下所示

5V 接串口屏 5V

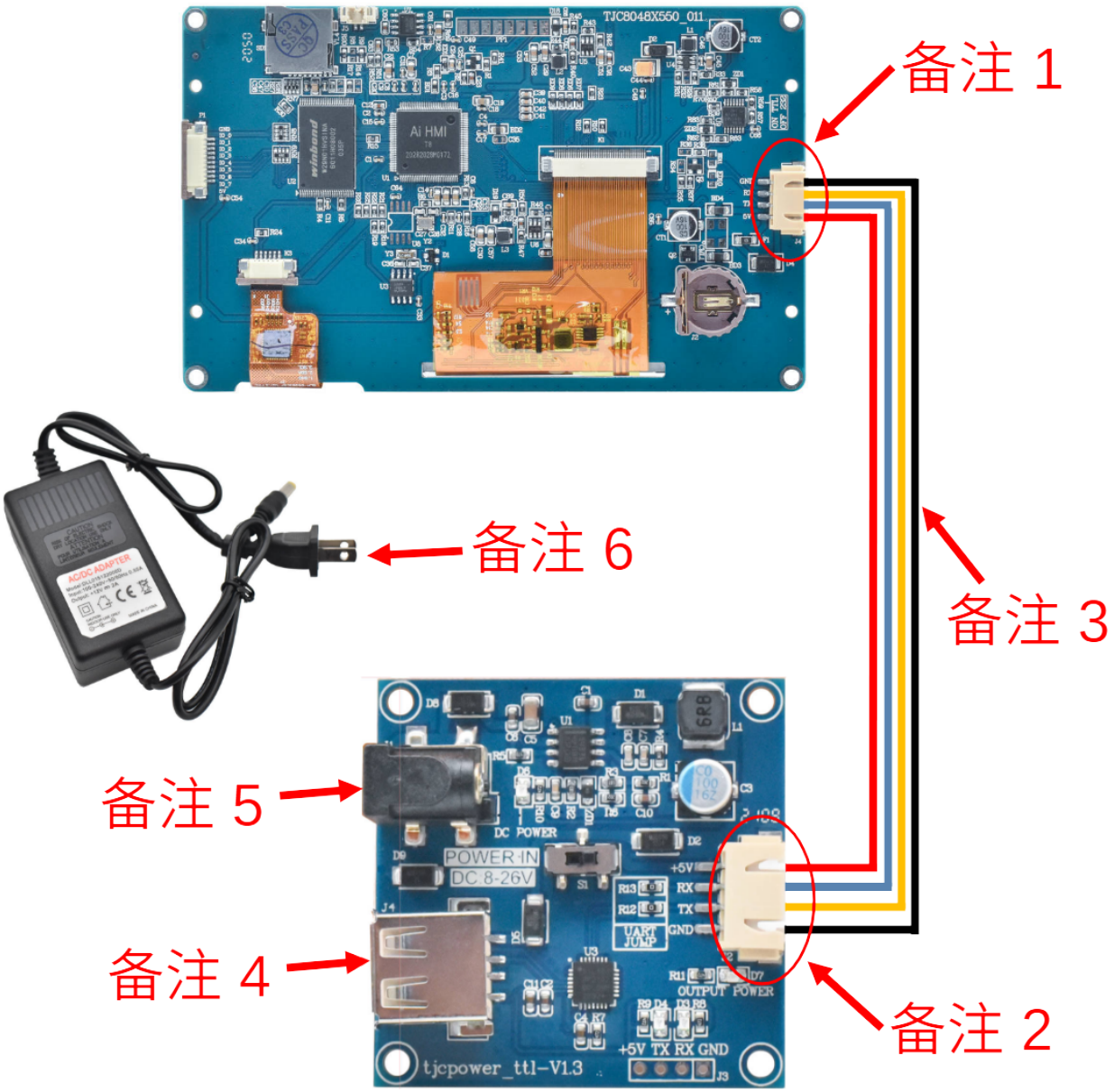
TX 接串口屏 RX

RX 接串口屏 TX

GND 接串口屏 GND

TX 是 Transmit（发送），RX 是 Receive（接收），发送对接收，接收对发送。

另外说明：有些板子和原理图标注的是 TXD 和 RXD。TX 就是 TXD，RX 就是 RXD。概念都是一样的，可以不做区分。



备注 1/2： 5V 接 5V， GND 接 GND， USB 转 TTL 的 TX 接串口屏 RX， USB 转 TTL 的 RX 接串口屏 TX

备注 3： 两边的 GND 要接在一起（共地），才能正常通讯

备注 4： 通过 USB 线接电脑

备注 5： 额外供电口，输入电压支持 8V-26V

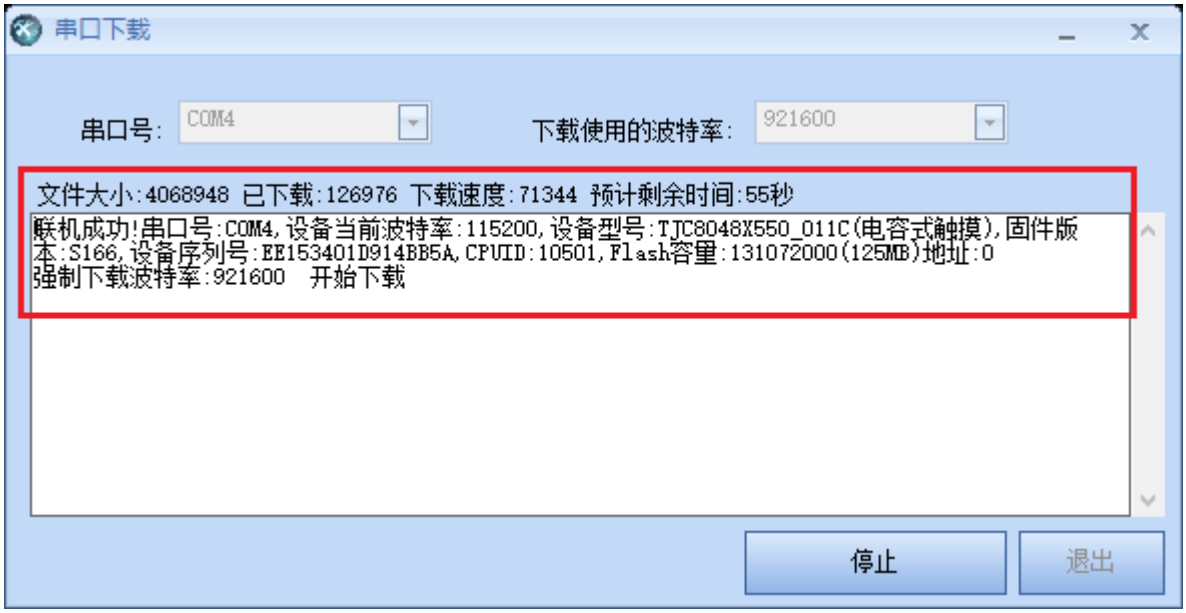
备注 6： 4.3 寸以上的屏幕仅仅使用 USB 线是带不动的，需要使用外部电源供电，否则会无法启动或者不断闪烁重启

3. 点击下载按钮

4. 串口号选择 com4（与设备管理器中的串口号一致），波特率选择 921600，点击联机并开始下载

大家不用担心下载波特率和之前配置的 bauds=115200 有冲突，下载波特率和通讯波特率是分开的，上位机和串口屏之间通过通讯波特率联机成功后，会切换为下载波特率，将整个工程下载完毕后，重新切换回通讯波特率。

5. 联机成功会显示以下信息，等待下载完成即可



---

6. 如果长时间进度没有动，请检查 usb 转 ttl 串口工具，参考：为什么不建议用 *ch340* (page 1248)

X3/X5 系列使用 232 通讯通讯下载时，下载波特率请勿超过 256000，如果还是无法下载，请将屏幕断电重启后，继续将下载波特率调低再尝试下载

使用 485 进行通讯时，需要下载工程时请参考*485 接口的串口屏* (page 96)

不管是使用 ttl 电平/232 电平/485 电平，如果无法通过串口下载, 可以查看下一章节通过 *SD* 卡下载工程到串口屏 (page 155)

1.4.1.10 使用 TFT 文件下载助手 (TFTFileDownload) 通过串口下载工程到串口屏

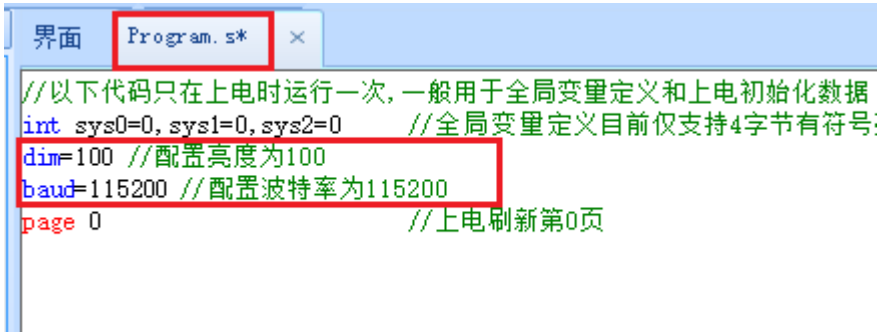
TFT 文件下载助手 (TFTFileDownload) 下载链接

TFT 文件下载助手 (TFTFileDownload) (page 1097)

1. 确认波特率等参数

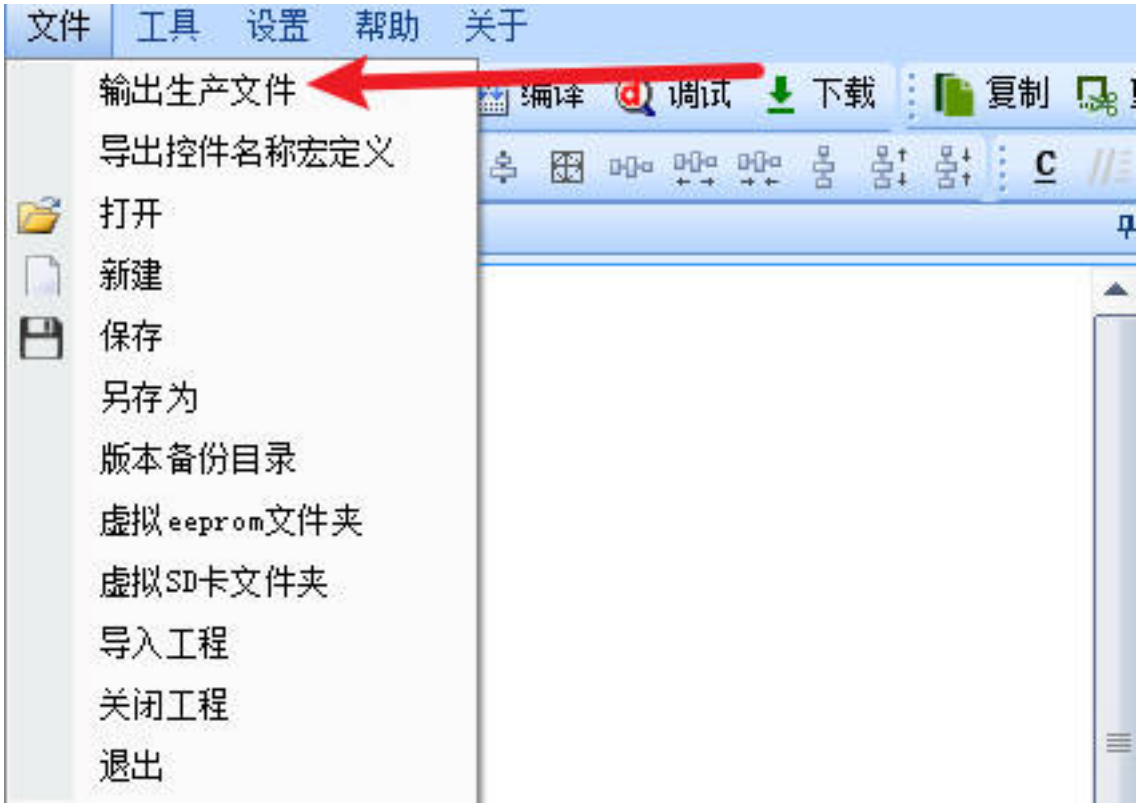
先切换到 program.s 中，配置一下亮度和波特率，请注意，等号两边不要有空格，否则会报错

注意这里的波特率应该和单片机的波特率一致。

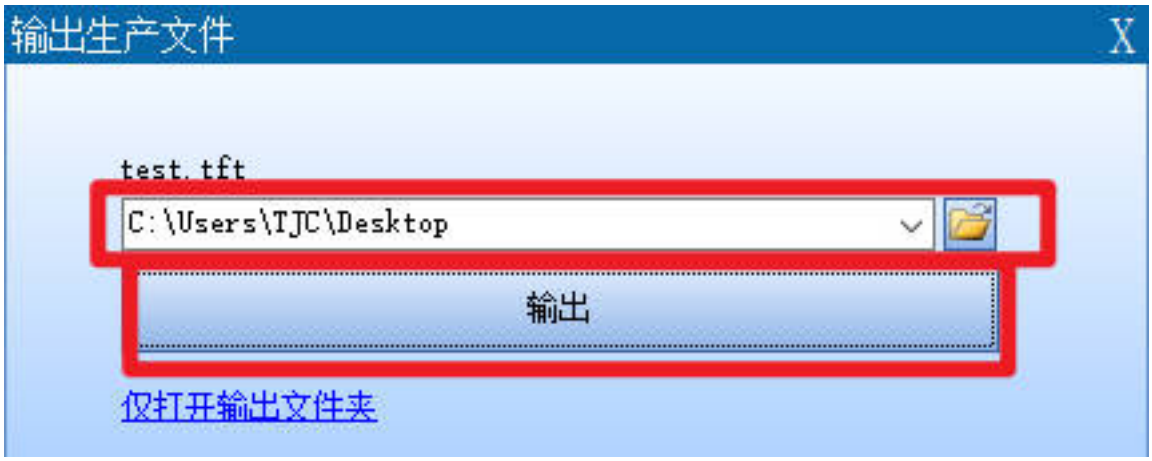


2. 输出 tft 文件

点击上位机左上角的文件-输出生产文件



选择输出位置之后，点击“输出”按钮，即可输出 TFT 文件



3. 连接下载器和屏幕

备注： 连接方式如下所示

5V 接串口屏 5V

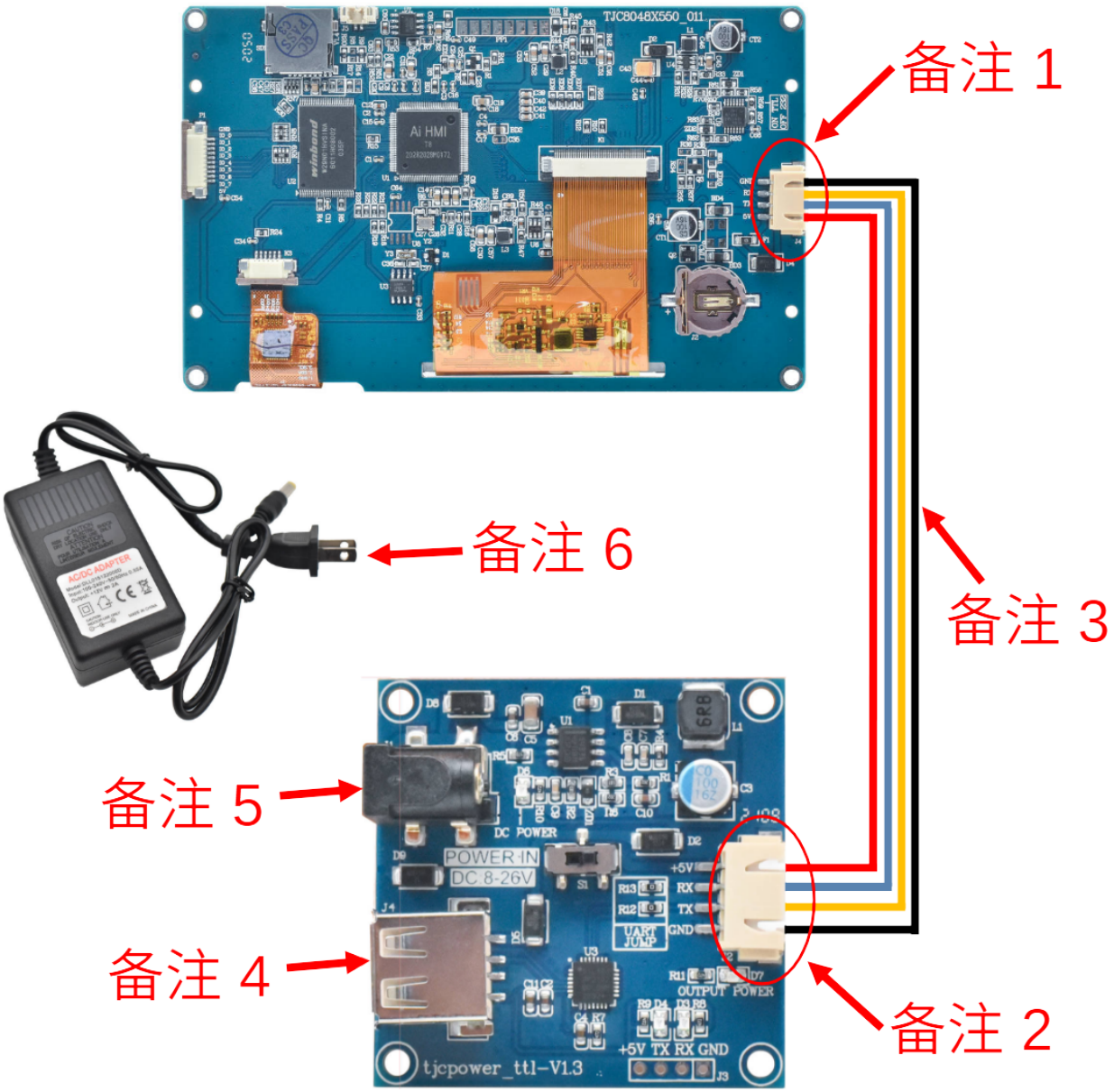
TX 接串口屏 RX

RX 接串口屏 TX

GND 接串口屏 GND

TX 是 Transmit（发送），RX 是 Receive（接收），发送对接收，接收对发送。

另外说明：有些板子和原理图标注的是 TXD 和 RXD。TX 就是 TXD，RX 就是 RXD。概念都是一样的，可以不做区分。



备注 1/2： 5V 接 5V， GND 接 GND， USB 转 TTL 的 TX 接串口屏 RX， USB 转 TTL 的 RX 接串口屏 TX

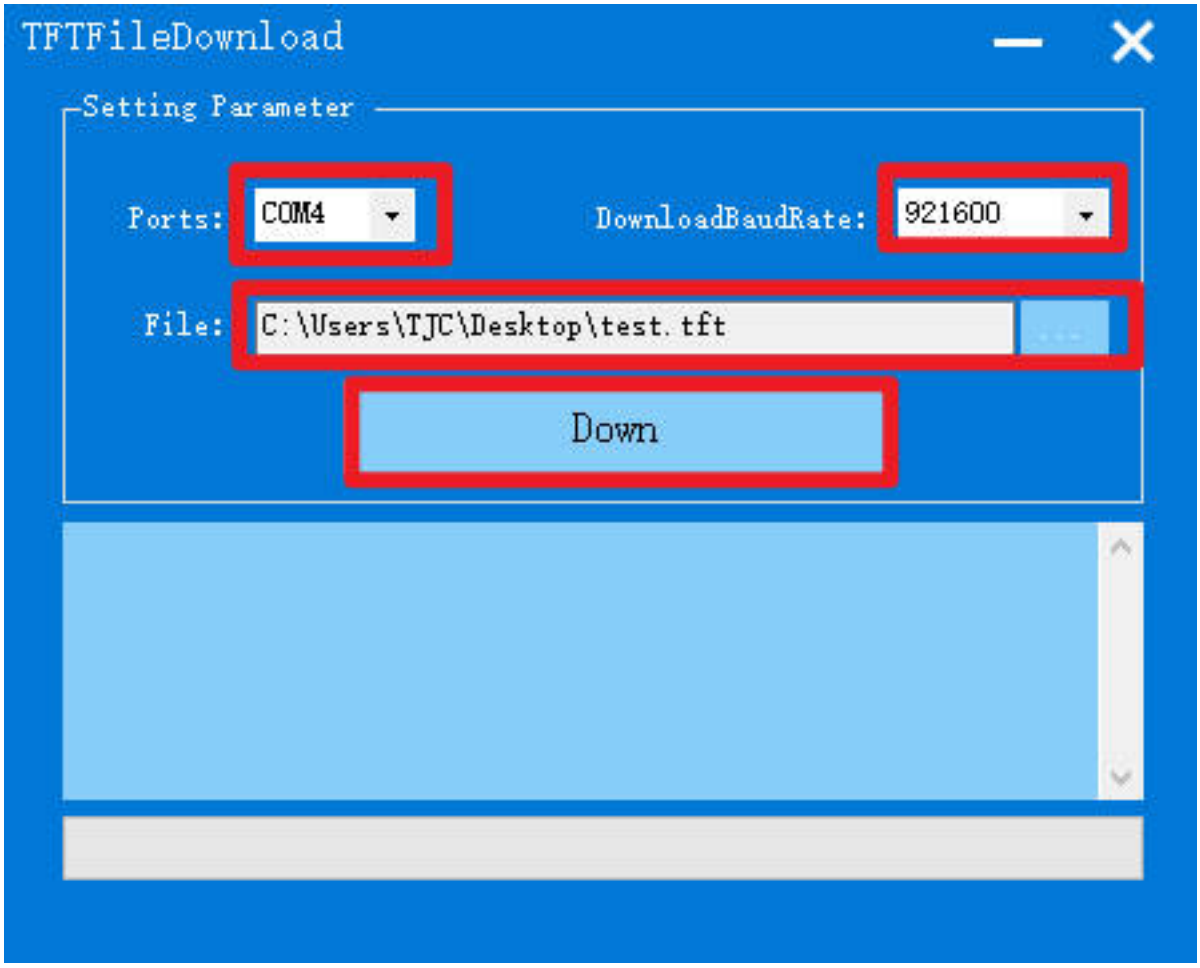
备注 3： 两边的 GND 要接在一起 (共地)，才能正常通讯

备注 4： 通过 USB 线接电脑

备注 5： 额外供电口，输入电压支持 8V-26V

备注 6： 4.3 寸以上的屏幕仅仅使用 USB 线是带不动的，需要使用外部电源供电，否则会无法启动或者不断闪烁重启

4. 分别设置端口号，波特率，并选择对应的 tft 文件后，点击“Down”按钮即可开始下载



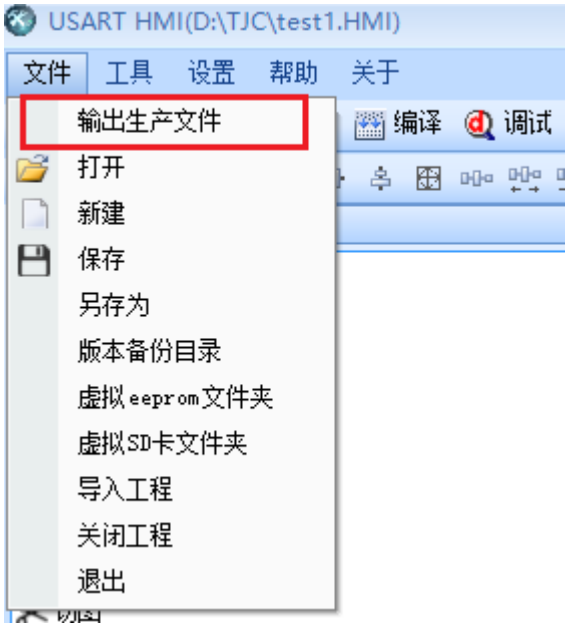
1.4.1.11 通过 SD 卡下载工程到串口屏

通过 microSD/TF 卡下载工程到串口屏

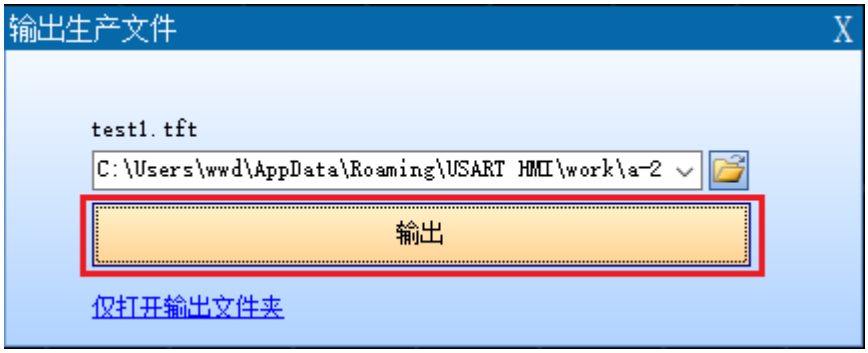
提示： 本方法建议用于批量生产时使用，只需向生产部门提供拷贝了 TFT 文件的 TF 卡即可，不需要向生产部门提供源代码（HMI 文件）

1.4.1.11.1 如何输出 TFT 生产文件

点击文件-输出生产文件



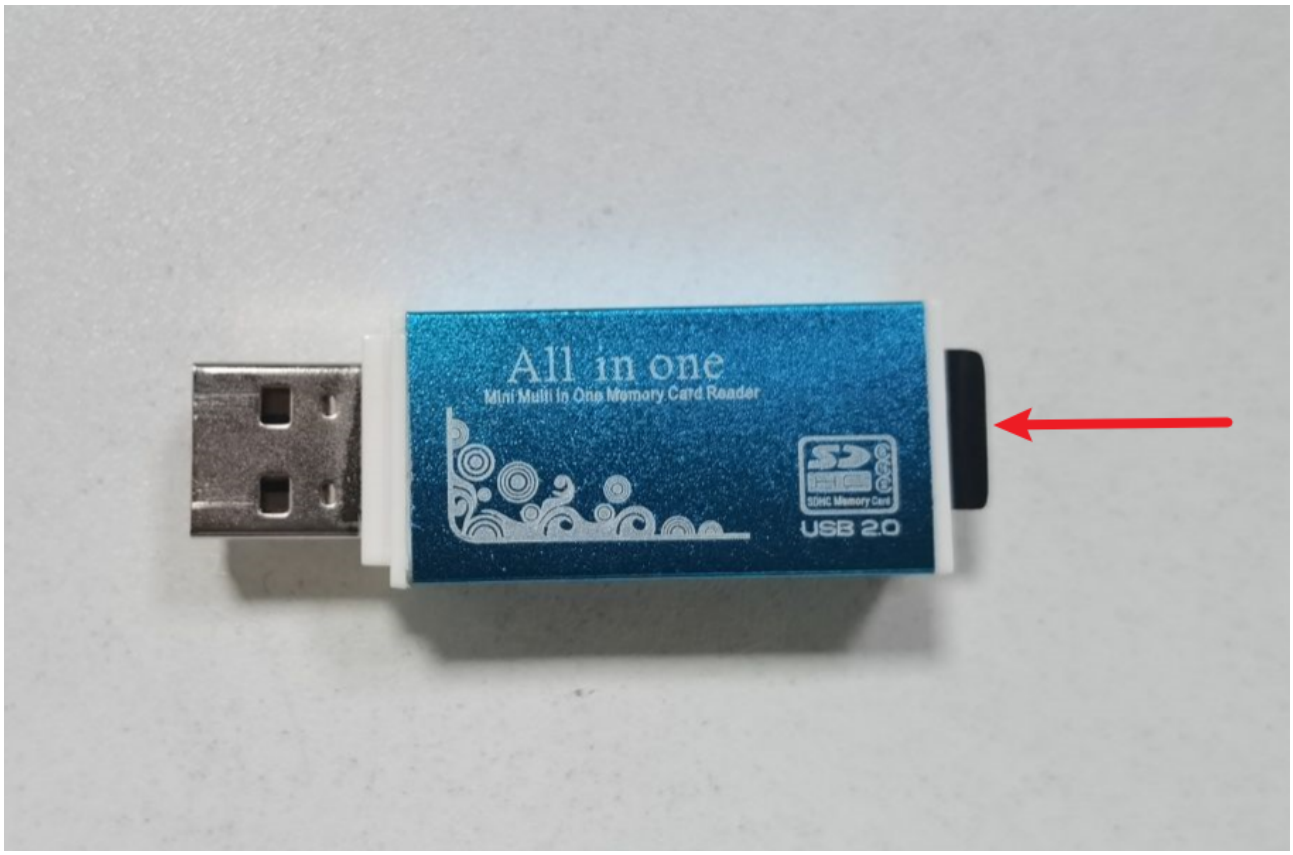
点击输出，如果此时跳出“拒绝访问”，则建议退出电脑上的 360 杀毒软件（或其他杀毒软件），再重新输出即可



输出的是与工程同名的.tft 文件

1.4.1.11.2 如何使用读卡器

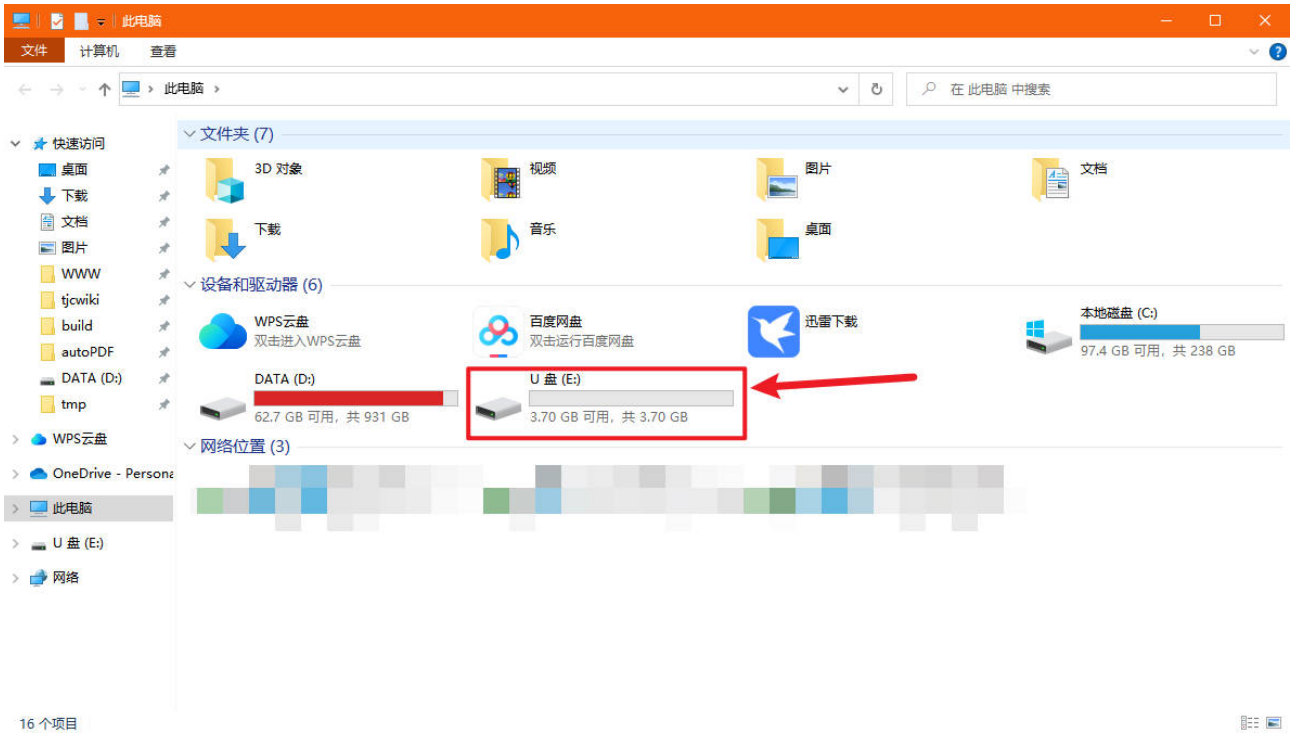
将 microSD 卡插入读卡器，读卡器和 microSD 卡需要自行购买,microSD 卡不能超过 32GB（例如：512M、1GB、2GB、4GB、8GB、16GB、32GB 都是可用的）



将读卡器插入 windows 电脑的 USB 接口

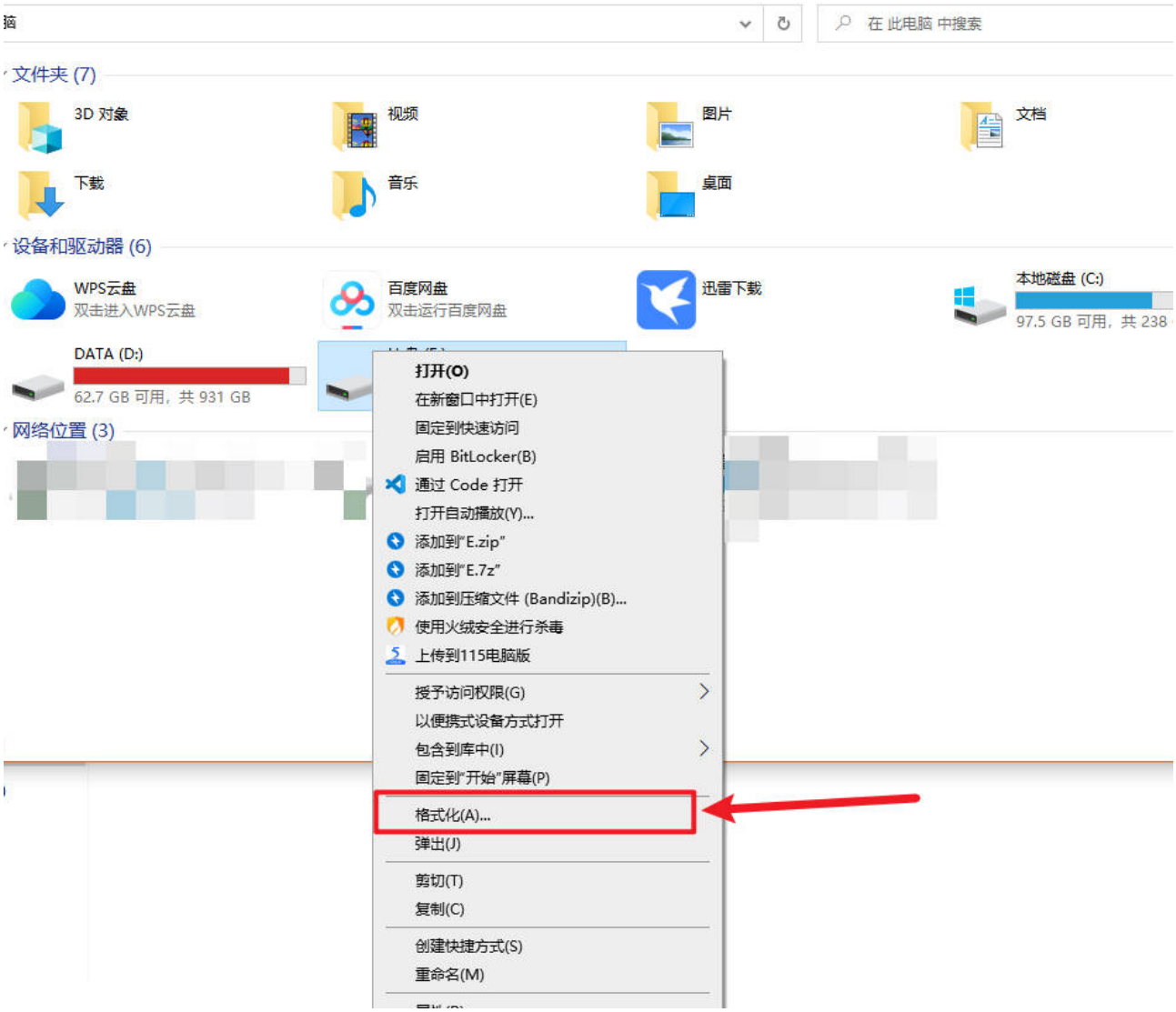
打开“此电脑”

可以在”设备和驱动器”发现刚刚插入的读卡器, 可能你的不是叫做”U 盘”, 也可能叫做可移动硬盘或者其他

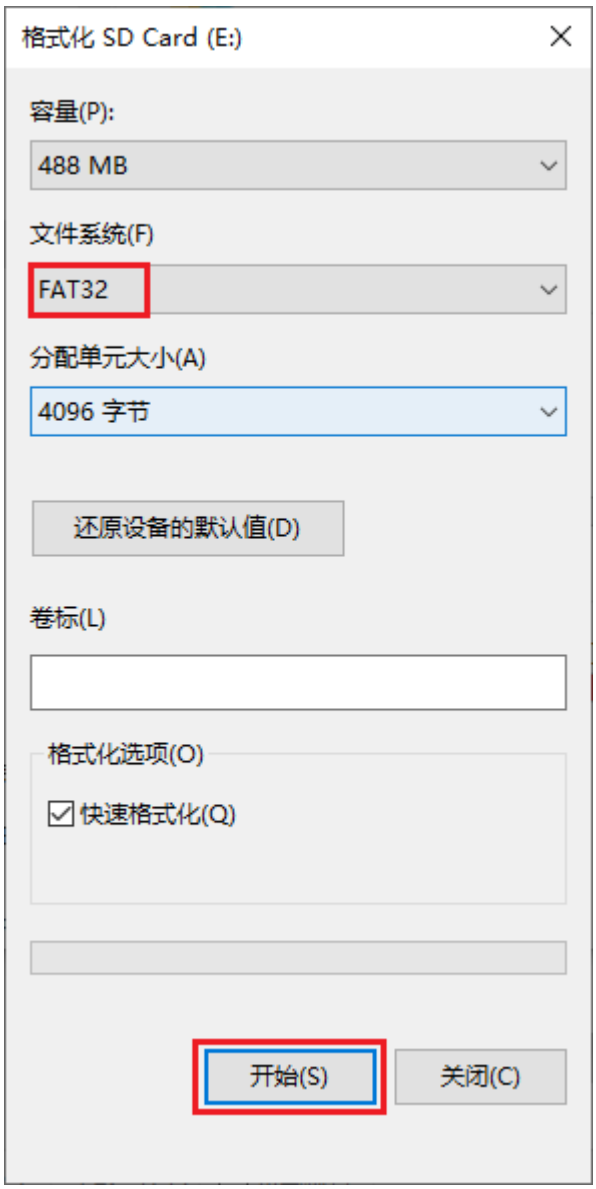


1.4.1.11.3 格式化 SD 卡为 fat32

右键选中盘符, 点击格式化

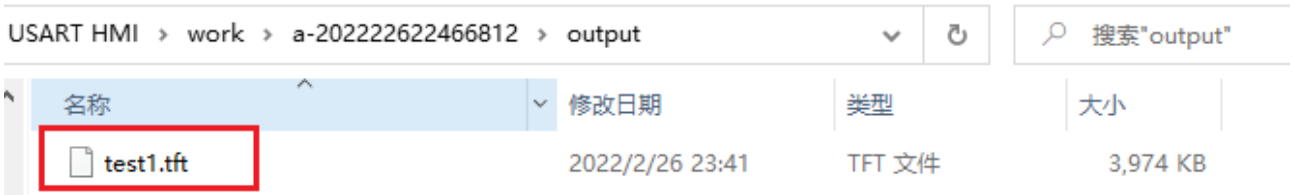


SD 卡下载用于在批量生产时使用，使用前请确保 SD 卡没有隐藏分区（没有做过启动盘），SD 卡容量不超过 32G（例如：512M、1GB、2GB、4GB、8GB、16GB、32GB 都是可用的），且格式化为 fat32 格式



1.4.1.11.4 使用 SD 卡下载工程到串口屏

确保 SD 卡不超过 32GB，将 tft 文件拷贝到 SD 卡根目录，SD 卡根目录有且仅只能有一个 tft 文件，否则会报错

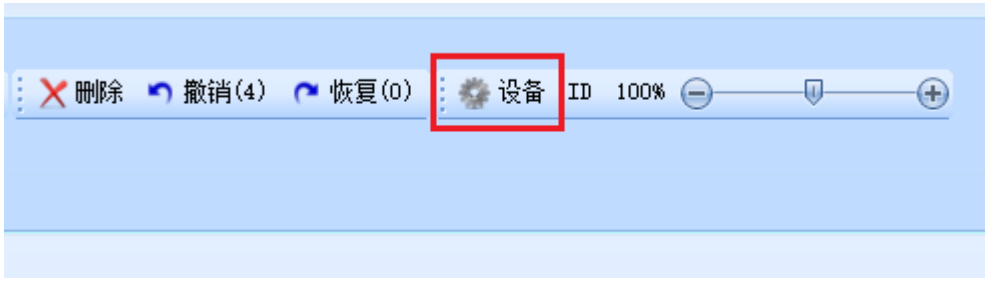


- 将串口屏断电，SD 卡插入串口屏的 SD 卡座中
- 串口屏重新上电，等待烧录完成
- 串口屏断电并取出 SD 卡
- 串口屏重新上电，程序下载完成

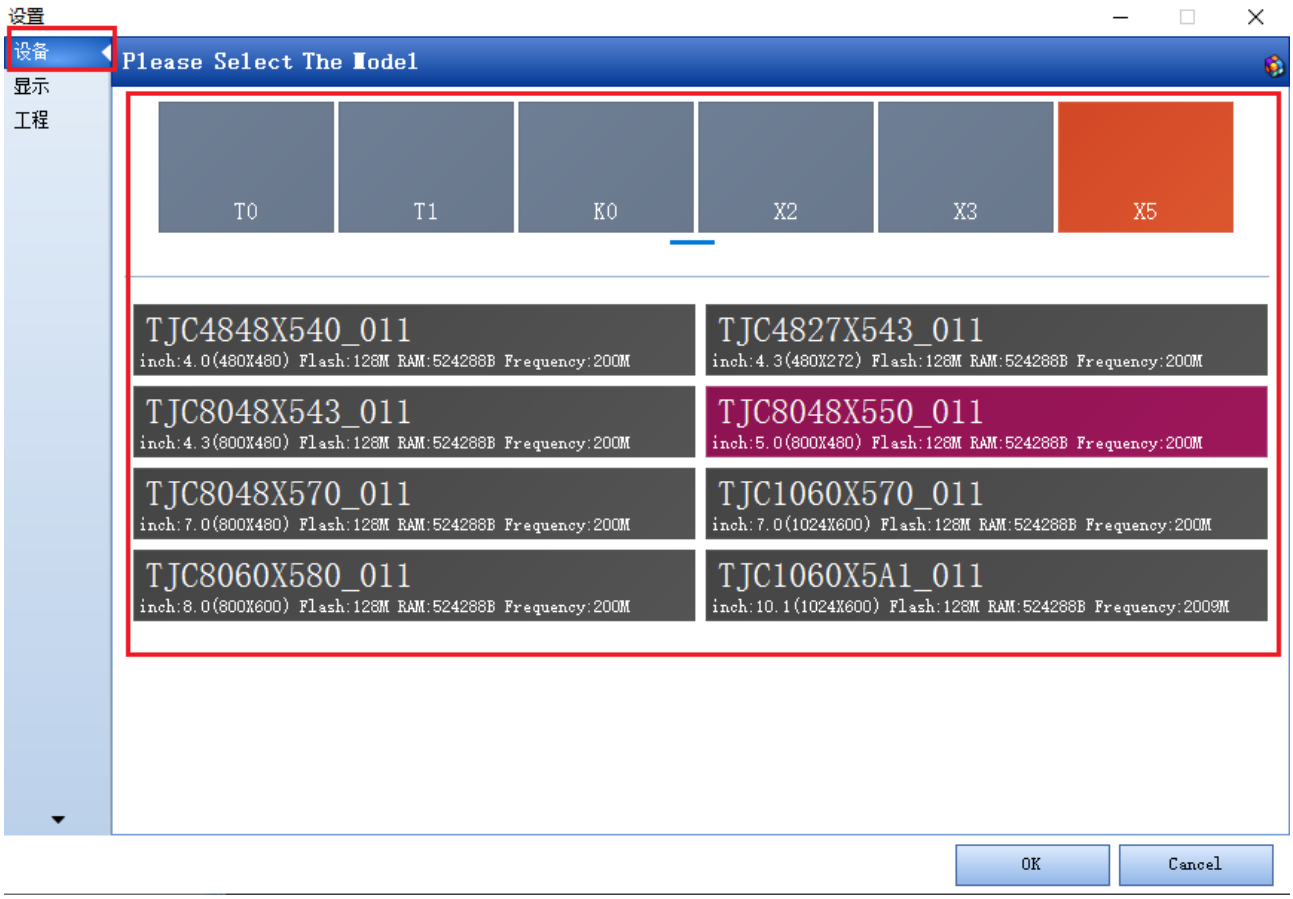
1.4.1.12 修改设备型号

在某些情况下，可能是选错了型号或者需要更换为其他尺寸，但是分辨率相同的型号时，可以直接修改型号

1. 点击顶部菜单栏的“设备”



2. 点击左侧设备，就可以选择自己所需要的型号



3. 重新下载或者输出生产文件即可

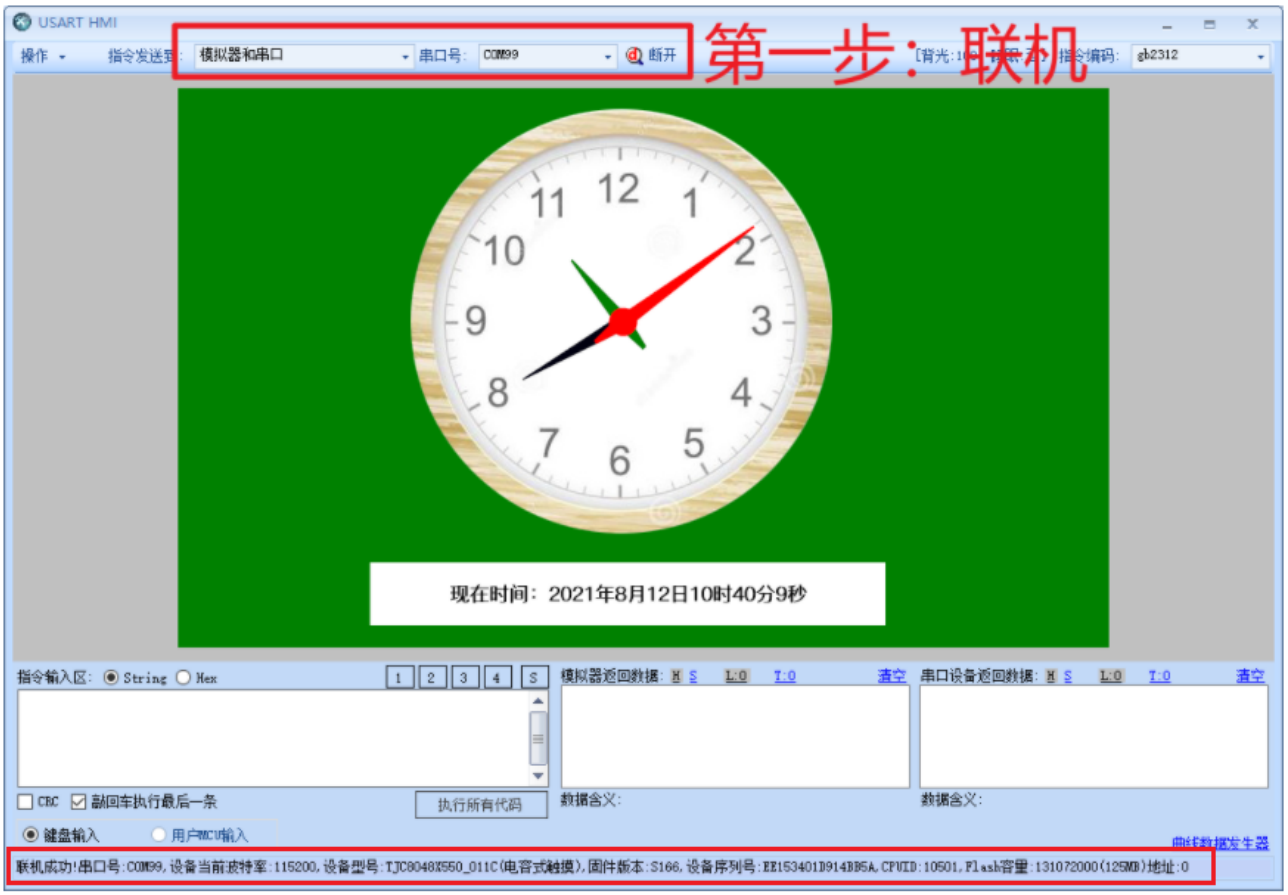
1.4.1.13 RTC 相关

1.4.1.13.1 校准 RTC 时钟

带 RTC 的型号，背部会有一个纽扣电池座。

使用带 RTC 功能的屏幕（K0/X5 系列或其他定制型号），时钟是不会自动同步的，需要手动同步时间

1. 联机成功时，底部会显示联机成功的信息

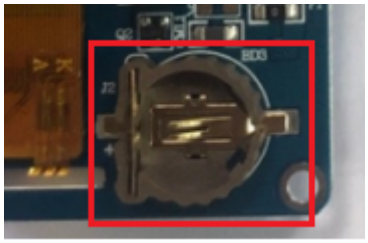


---

2. 点击操作-校准设备 RTC 时钟

1.4.1.13.2 RTC 电池型号

如果要保证时钟断电后还能继续走，请安装背部的 RTC 电池，型号为 CR1220，电池安装时，有字的一面应朝外

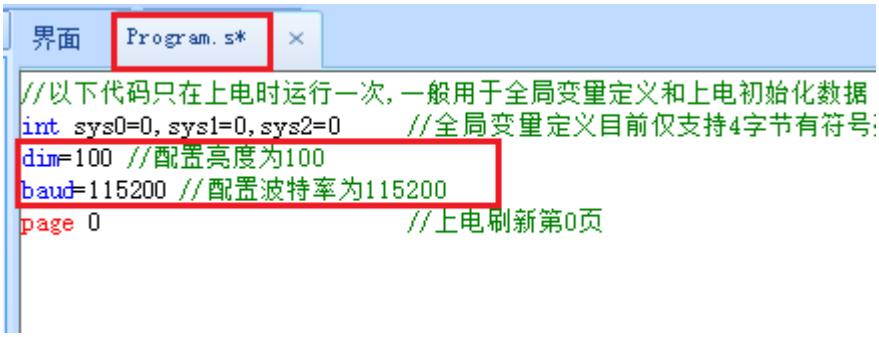


相关链接

rtc0~rtc6-RTC 时钟变量 (page 795)

如何判断屏幕是否支持 RTC 实时时钟 (page 1264)

1.4.1.14 配置亮度



也可在其他控件的事件编辑框中修改亮度。

> **注意：** 请注意：page 是跳转页面的指令，page 指令后面的代码将不会执行

> **注意：** 请注意：page 是跳转页面的指令，page 指令后面的代码将不会执行

> **注意：** 请注意：page 是跳转页面的指令，page 指令后面的代码将不会执行

> **注意：** 请注意：page 是跳转页面的指令，page 指令后面的代码将不会执行

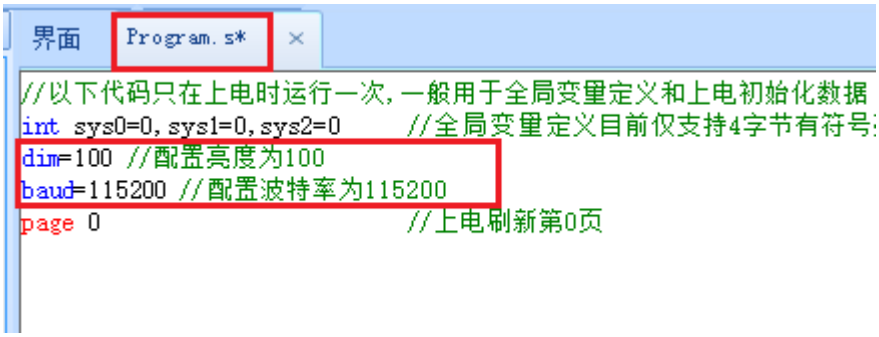
> **注意：** 请注意：page 是跳转页面的指令，page 指令后面的代码将不会执行

> **注意：** 请注意：page 是跳转页面的指令，page 指令后面的代码将不会执行

更多可配置参数详见淘晶驰资料中心-系统变量

淘晶驰资料中心
1.0.1
在文档中搜索
淘晶驰教程
快速入门
控件详解
基本指令集
书写语法
系统变量
常用变量
不常用变量
串口屏返回数据格式
串口屏高级应用详解
资料下载
视频教程
常见问题
产品选型和规格书
产品选型和规格书
» 系统变量
系统变量
注意
所有变量名称使用小写字符
常用变量
dp-当前页面ID
volume-系统音量
dim-当前背光亮度值
baud-当前波特率值
ussp-无串口数据自动睡眠时间
thsp-无触摸操作自动睡眠时间
thup-睡眠模式下触摸自动唤醒开关
usup-睡眠模式下串口数据自动唤醒开关
wup-睡眠唤醒后刷新页面设置
sleep-睡眠
lowpower-睡眠模式设定
bkcmd-串口指令执行状态数据返回
delay-延时
rand-随机数
crcval-crc校验结果
rtc0~rtc6-RTC时钟变量
pio0~pio7-扩展IO端口
pwm4~pwm7-扩展IO占空比
pwmf-PWM输出的频率
dims-上电默认背光亮度值
bauds-上电默认波特率值
audio0~audio1-音频通道控制
串口HMI颜色代号表
不常用变量

1.4.1.15 配置串口波特率



也可在其他控件的事件编辑框中修改波特率。

注意： 请注意：page 是跳转页面的指令，page 指令后面的代码将不会执行

注意： 请注意：page 是跳转页面的指令，page 指令后面的代码将不会执行

注意： 请注意：page 是跳转页面的指令，page 指令后面的代码将不会执行

注意： 请注意：page 是跳转页面的指令，page 指令后面的代码将不会执行

注意： 请注意：page 是跳转页面的指令，page 指令后面的代码将不会执行

注意： 请注意：page 是跳转页面的指令，page 指令后面的代码将不会执行

更多可配置参数详见淘晶驰资料中心-系统变量

1.4.1.15.1 配置串口波特率-相关链接

如何配置亮度, 主动解析, 波特率 (page 1306)

1.4.1.16 配置休眠

界面 Program.s*

```
//以下代码只在上电时运行一次，一般用于全局变量定义和上电初始化数据
int sys0=0,sys1=0,sys2=0      //全局变量定义目前仅支持4字节有符号整形(int),
dim=100 //配置亮度为100
baud=115200 //配置波特率为115200
thsp=30 //无操作自动休眠时间，单位：秒
thup=1  //睡眠模式下自动唤醒开
page 0                        //上电刷新第0页
```

也可在其他控件的事件编辑框中修改休眠参数。

> **注意：** 请注意：page 是跳转页面的指令，page 指令后面的代码将不会执行

更多可配置参数详见淘晶驰资料中心-系统变量

1.4.1.17 跨页面操作变量

多数情况下，我们都是在操作当面页面的控件属性，如果需要操作其他页面的控件属性请按如下书写方式:

[页面].[控件].[属性]=XXX

实例:

给 main 页面的 n0.val 属性赋值 123

```
main.n0.val=123
```

把 set 页面的 t3.txt 赋值给 main 页面的 t0.txt

```
main.t0.txt=set.t3.txt
```

给 set 页面的 t4.txt 赋值”abc”

```
set.t4.txt="abc"
```

注意: 跨页面操作控件属性的时候，不管是读取还是赋值，被操作控件的 vscope 属性必须设置为全局（默认是私有），否则操作会失败

注意: click/vis/tsw 指令无法跨页面触发控件、定时器控件只能在当前页面运行、曲线波形控件只能在当前页面添加数据点。

1.4.1.17.1 跨页面操作变量-相关链接

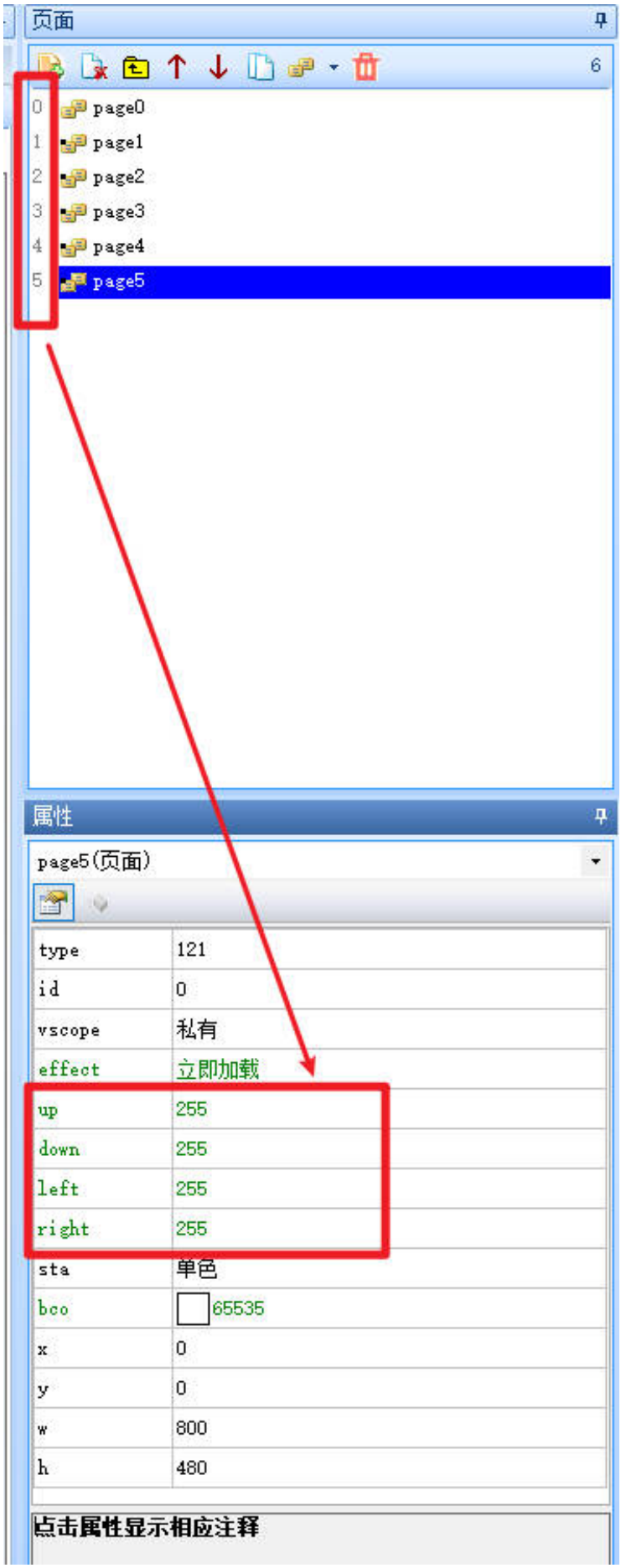
跨页面赋值，全局变量操作 (page 847)

1.4.1.18 页面滑动切换

设置页面的 up/down/left/right 属性, 仅 X 系列支持滑动翻页/划动翻页。

当 up/down/left/right 为 255 时, 代表不翻页。

填写对应的页面 id 号, 则触发相关滑动事件时跳转到相应的页面。


页面
6
0 page0
1 page1
2 page2
3 page3
4 page4
5 page5
属性
page5(页面)
type 121
id 0
vscope 私有
effect 立即加载
up 255
down 255
left 255
right 255
sta 单色
bco 65535
x 0
y 0
w 800
h 480
点击属性显示相应注释

1.5 代码编写

1.5.1 main 函数在哪里

单片机编程时需要在 main 函数内编写主循环，让代码循环执行

但是串口屏内并不需要编写 main 函数和主循环

1.5.2 代码写在哪里

编写代码的位置有以下两个个地方

program.s (page 172)

控件事件 (page 174)

1.5.2.1 program.s

注意:

program.s 主要有以下作用

1. 使用 int 定义整形全局变量

只有在 program.s 中，才可以使用 int 定义全局的数值变量，使用 int 定义的整形变量一定是全局的，int 为 4 字节有符号整形，范围为-2147483648~2147483647。

不允许定义字符串。要存放字符串，请使用文本控件或者将一个变量控件的 sta 设置为“字符串”。

2. 提前初始化其他页面的全局变量。

3. 设置亮度和波特率，参考: *如何配置亮度, 主动解析, 波特率* (page 1306)

4. 输出上电信息到串口*0X88 系统启动成功* (page 905)

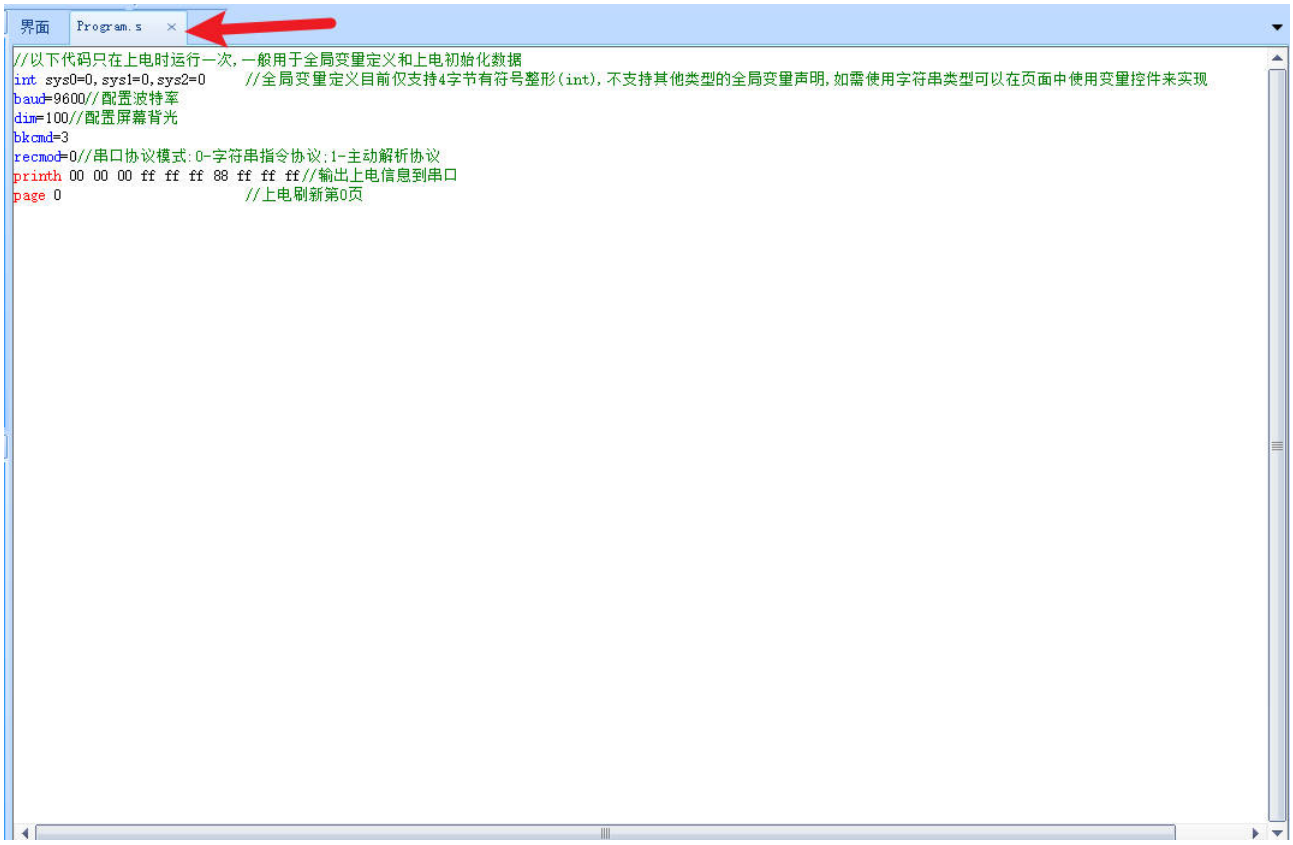
5.program.s 中最后一行通过 page 指令跳转到工程的第一个页面。

提示: program.s 中使用 int 定义的整形均为全局变量，目前仅能定义 int 类型，无法定义其他类型（如字符串，浮点数等），int 定义需放在其他代码之前，否则会报错。

使用 int 定义的整形全局变量与数值控件 (vscope 设置为全局) 以及变量控件 (vscope 设置为全局) 使用上的区别

使用 int 定义的整形全局变量，是一个纯粹的变量, 例如 sys0，可以用 sys0=0 这样的方式直接赋值

使用数值控件或变量控件，它是一个对象, 它有多个属性, 赋值时需要使用 n0.val=0 或者 va0.val=0 这样的方式进行复制 (即需要指明具体的属性)


界面
Program.s
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
int sys0=0,sys1=0,sys2=0 //全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,如需使用字符串类型可以在页面中使用变量控件来实现
baud=9600//配置波特率
dim=100//配置屏幕背光
bkcmd=3
recmod=0//串口协议模式:0-字符串指令协议;1-主动解析协议
printh 00 00 00 ff ff ff 88 ff ff ff//输出上电信息到串口
page 0 //上电刷新第0页

1.5.2.2 控件事件

目前各种控件综合起来被操作的方式有以下种类型：

页面控件事件 (page 174)

按下和弹起事件 (page 175)

滑动事件 (page 175)

定时事件 (page 176)

播放完成事件 (page 176)

1.5.2.2.1 页面控件事件

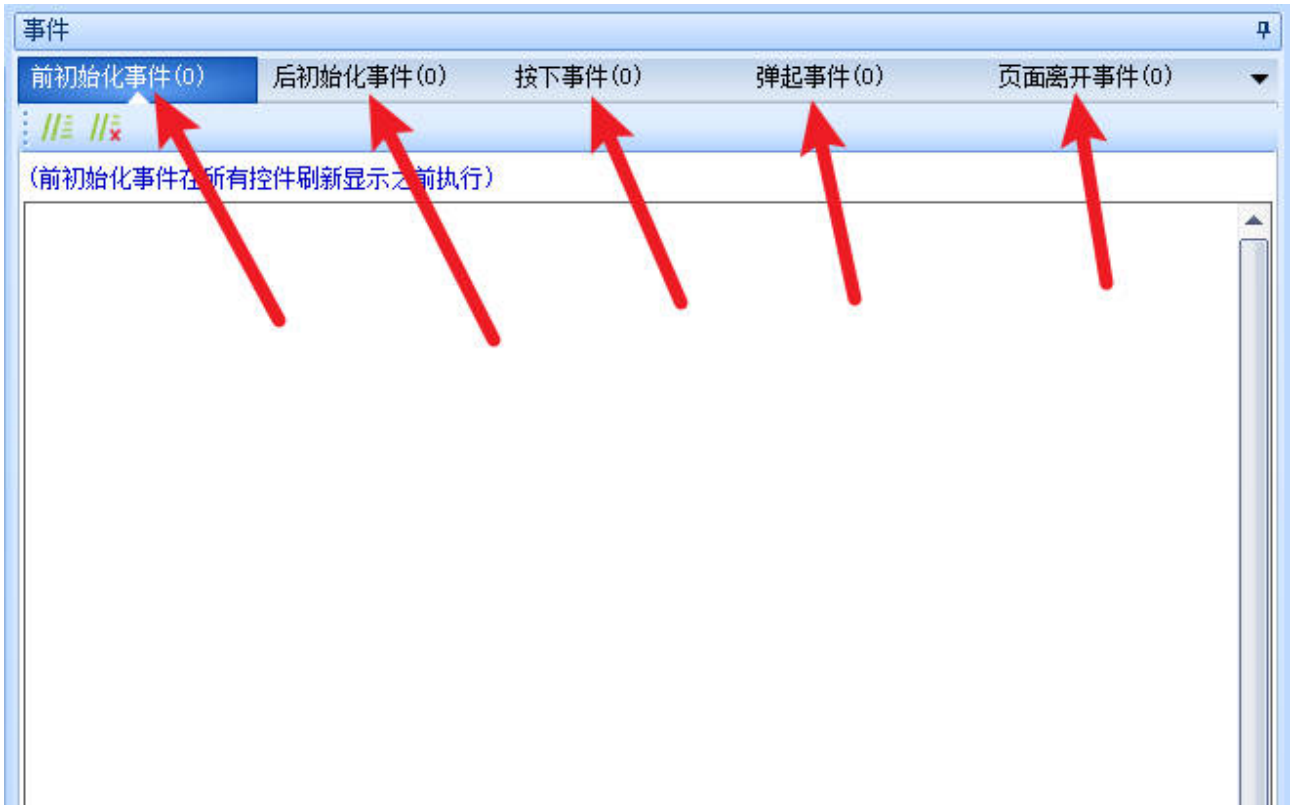
页面控件是一个特殊的控件，新建了一个页面后，会自动创建一个同名的页面控件，页面控件必定与页面名称相同，且页面控件的 ID 必定为 0，即页面控件始终处于最底层

页面前初始化事件是在页面加载前自动执行的

页面后初始化事件是在页面完成后自动执行的

页面离开事件是在执行了跳转页面动作时执行的

点击页面的空白处，即可触发页面控件的按下或弹起事件



1.5.2.2.2 按下和弹起事件

触摸被按下：对应名称叫做【按下事件】

触摸被按下后弹起：对应名称叫做【弹起事件】

例如实现点击按钮跳转页面功能时，将 page 指令（跳转页面）写在按钮的弹起事件中，当手指点击对应的控件并放开时，此时就会触发跳转

1.5.2.2.3 滑动事件

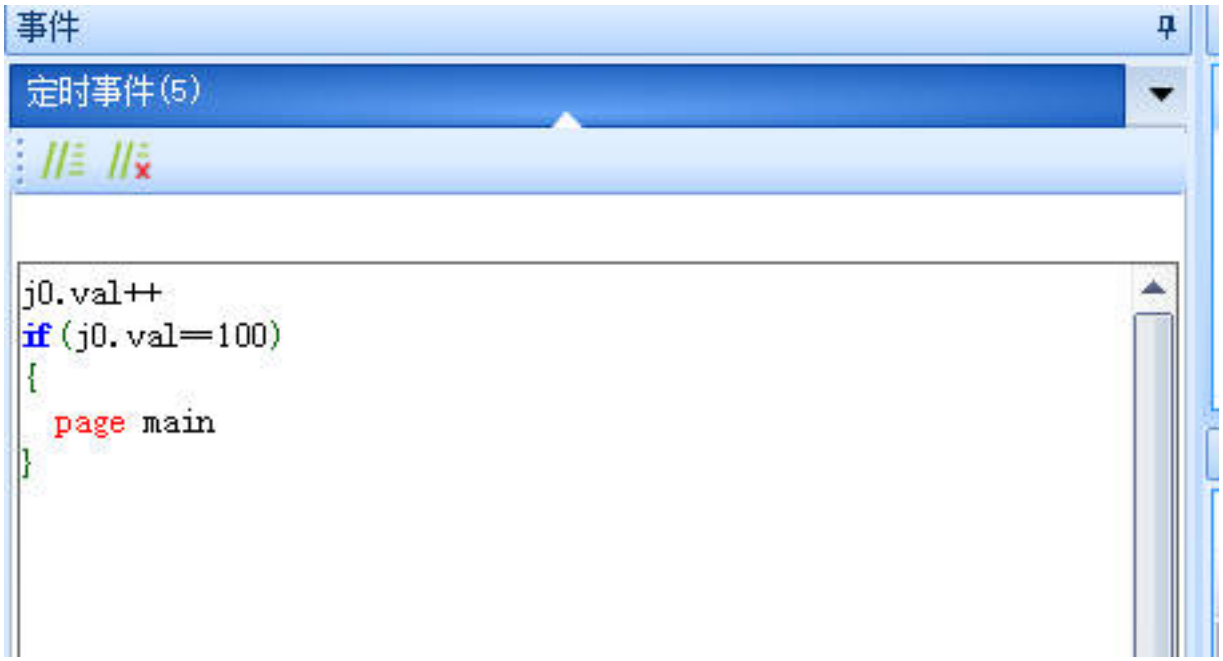
滑块控件被滑动：对应的名称叫做【滑动事件】

例如在滑块的滑动事件和弹起事件中修改串口屏的亮度（全局变量 dim）

1.5.2.2.4 定时事件

定时器定时运行：对应的名称叫做【定时事件】

例如显示开机进度，当进度为 100 时跳转到 main 页面



1.5.2.2.5 播放完成事件

音频、动画、视频播放完成：对应的名称叫做【播放完成事件】

例如开机动画是一段视频或动画时，在播放结束时跳转到主页面

> **注意：** 当播放音频或者视频时，请注意应保证供电充足，否则会导致串口屏供电不足从而重启，参考串口屏开机时死机/不断的闪烁/不断重启 (page 1366)

> **注意：** 写在 page 指令（跳转页面）后面的代码不会被执行

1.5.3 函数调用

1、触摸热区控件可以理解为一个看不见的按钮控件。

2、由于串口屏上没有函数的概念，因此有大量重复的代码需要调用时，可以将代码写在触摸热区内（或其他控件），然后用 click 指令去触发

3、单片机可以发 click 命令来激活相关控件状态。但是单片机需要确保显示屏处于当前界面（此界面包含对应的触摸热区控件）。

4、不允许跨页面 click 控件。如果有需要请将控件复制到相关页面。

参考：触摸热区控件 (page 448)

2 串口屏调试

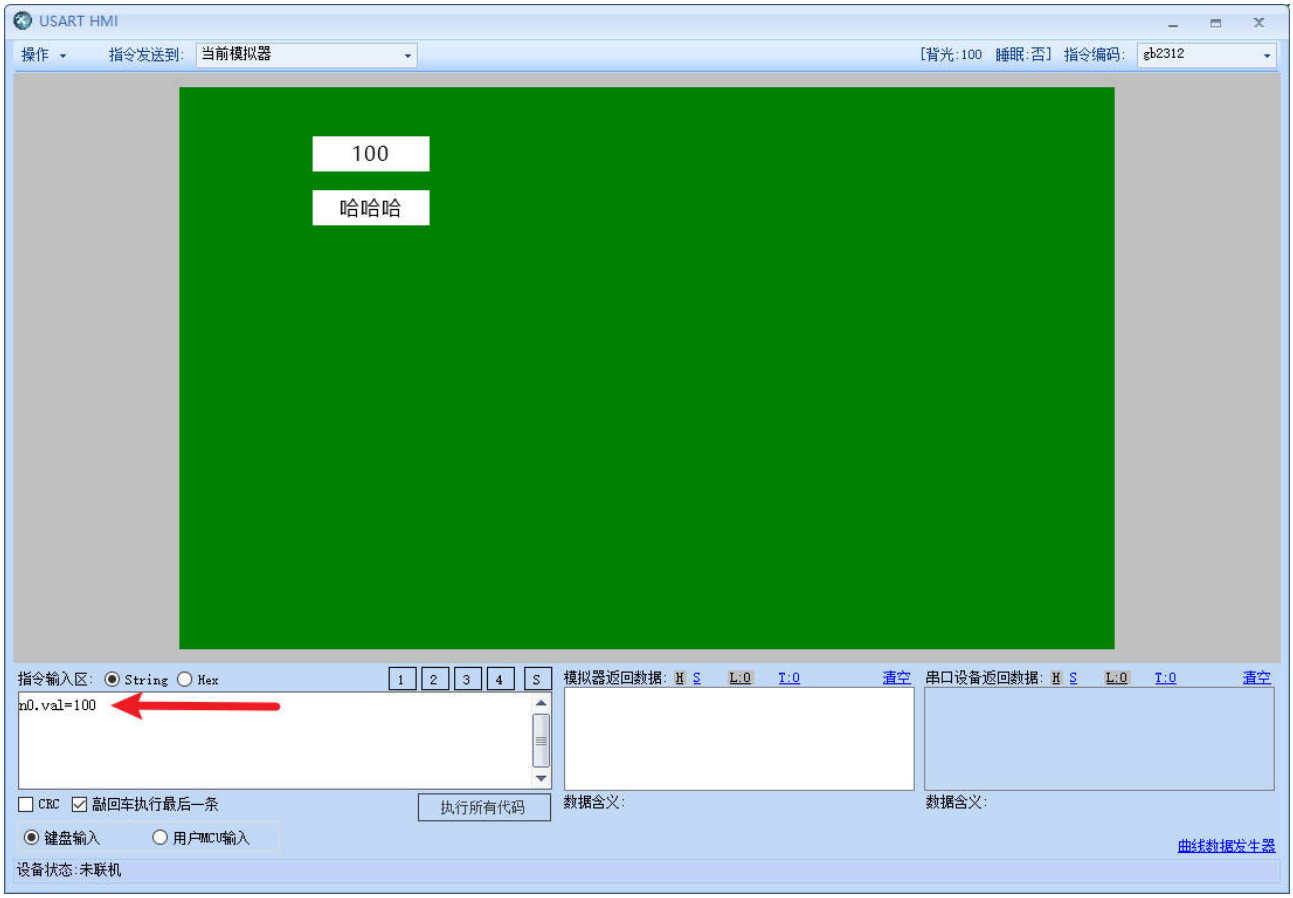
2.1 模拟器调试基础知识

2.1.1 在没有串口屏的情况下进行调试

提示： 淘晶驰串口屏的模拟器功能非常强大，您可以在没有串口屏实物的情况下通过模拟器进行调试

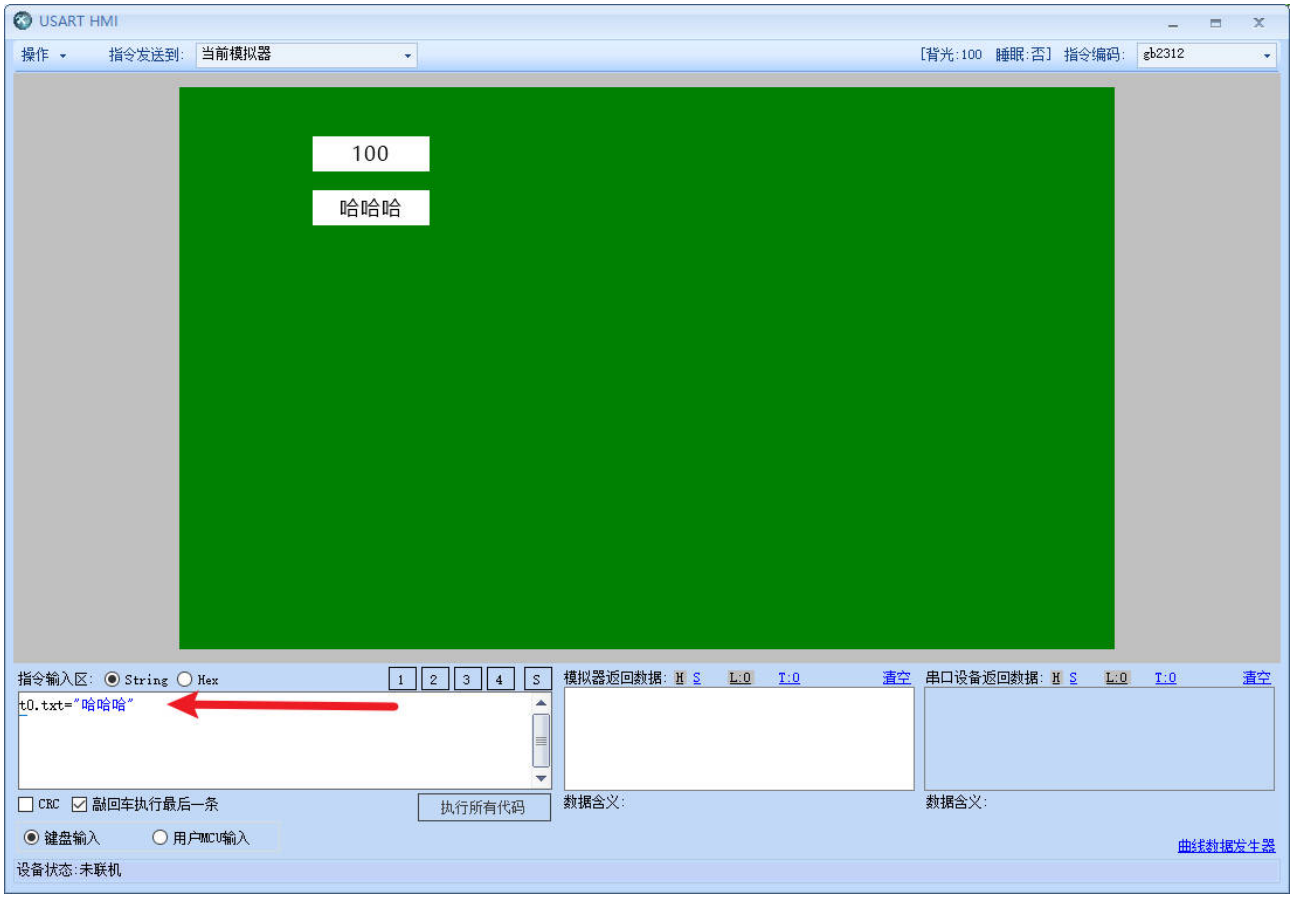
上位机模拟器几乎可以完全模拟真实的设备。

在指令输入区可以直接输入指令，不需要结束符

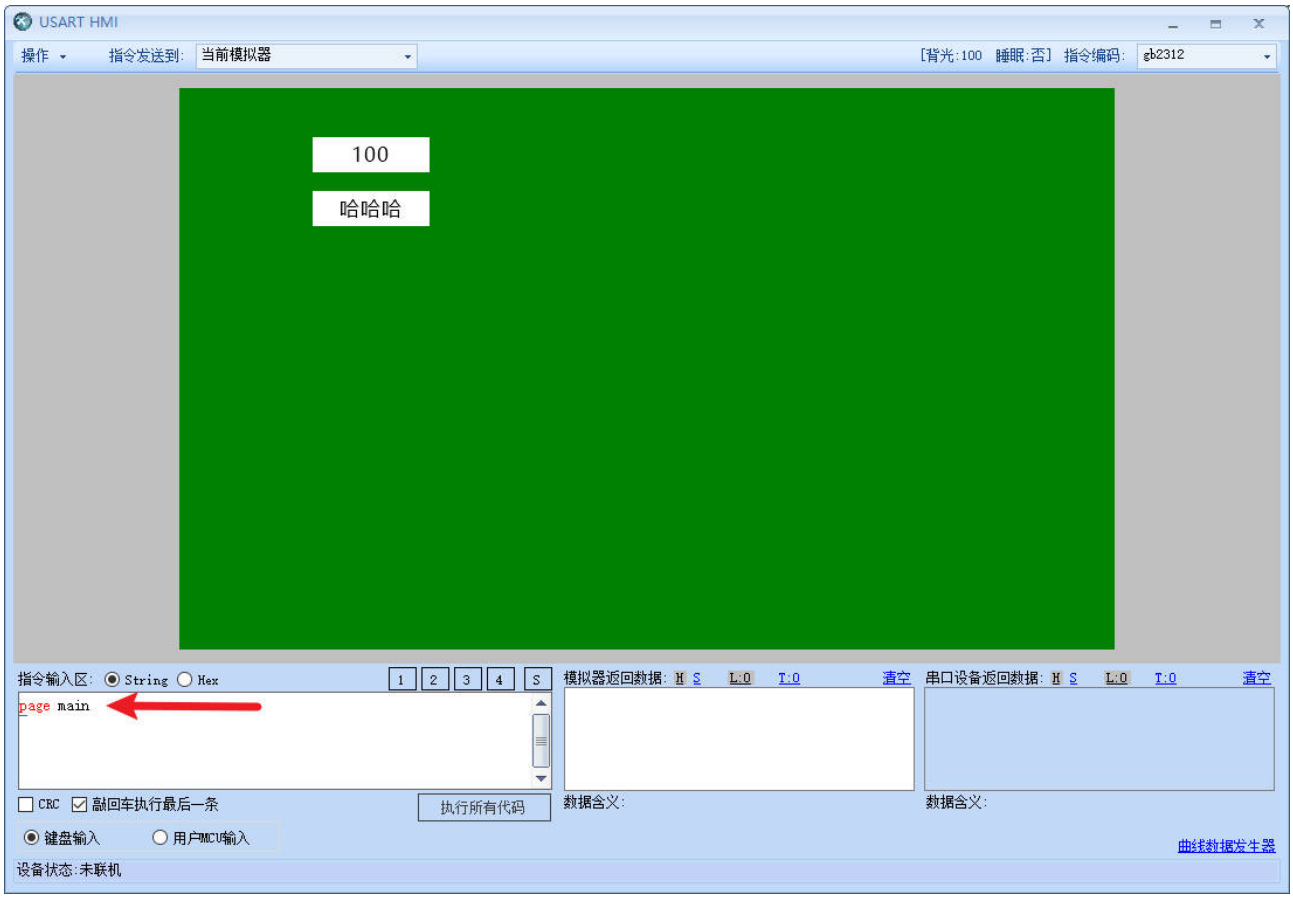


2.1.1.1 模拟器调试示例一

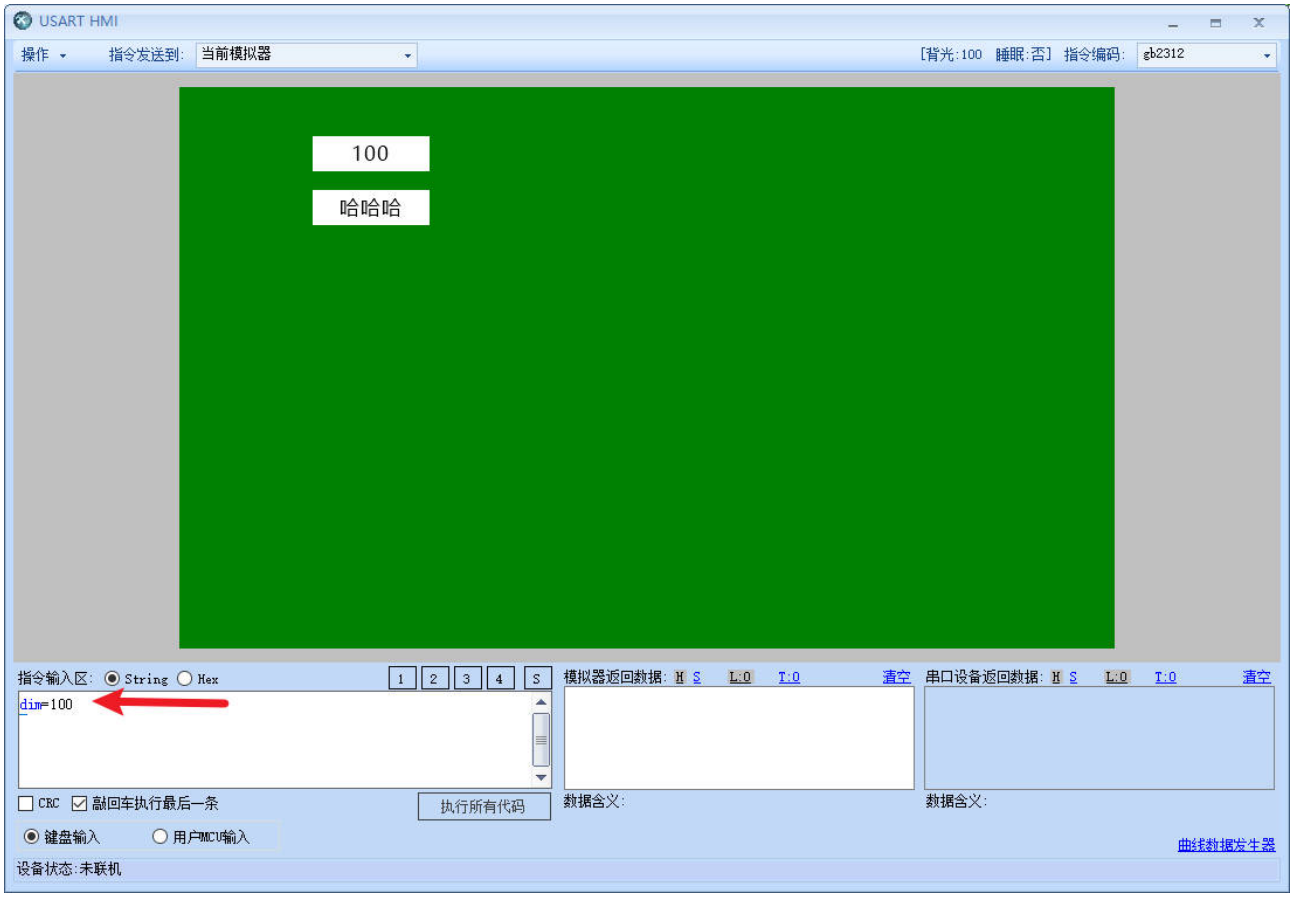
在指令输入区可输入指令给文本赋值，不需要对引号进行转义



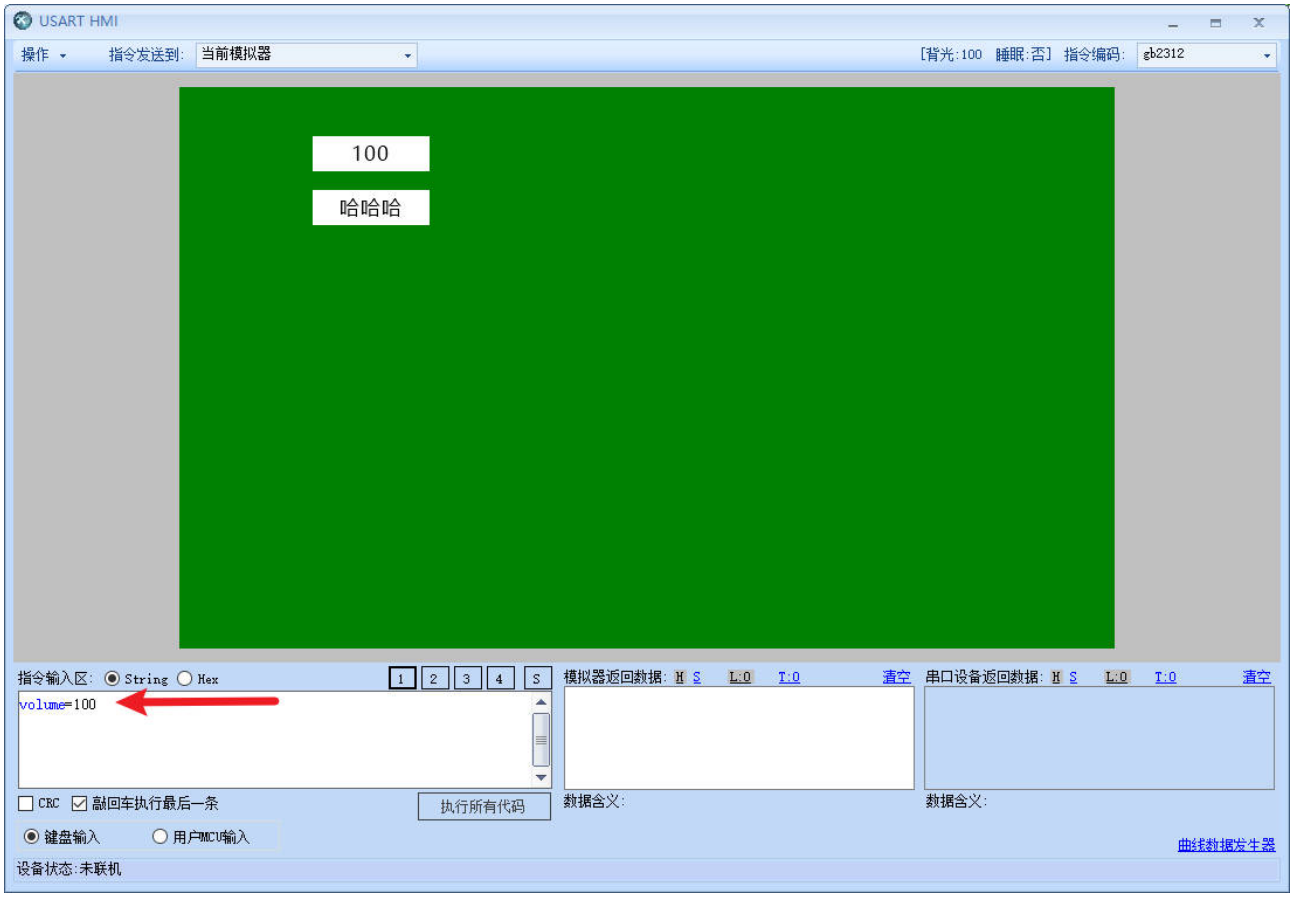
在指令输入区可以使用 page 指令来跳转页面



在指令输入区设置亮度

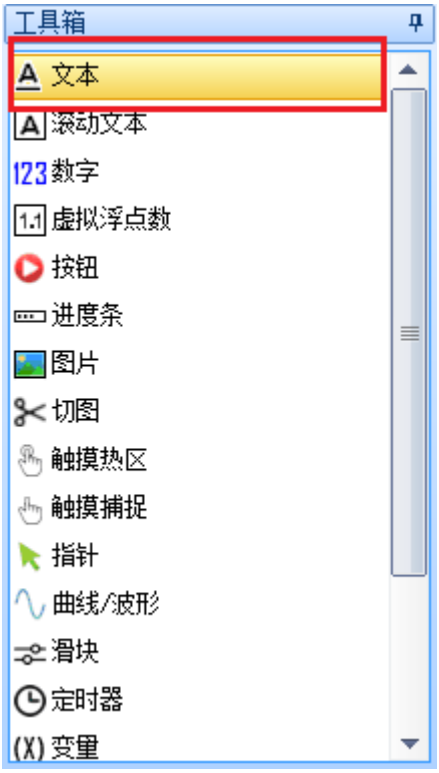

USART HMI
操作
指令发送到:
当前模拟器
[背光:100 睡眠:否] 指令编码:
gb2312
100
哈哈哈
指令输入区:
String
Hex
1
2
3
4
S
dim=100
模拟器返回数据:
H
S
L:0
T:0
清空
串口设备返回数据:
H
S
L:0
T:0
清空
CRC
敲回车执行最后一条
执行所有代码
数据含义:
数据含义:
键盘输入
用户MCU输入
曲线数据发生器
设备状态:未联机

设置音量（仅 x3、x5 支持）

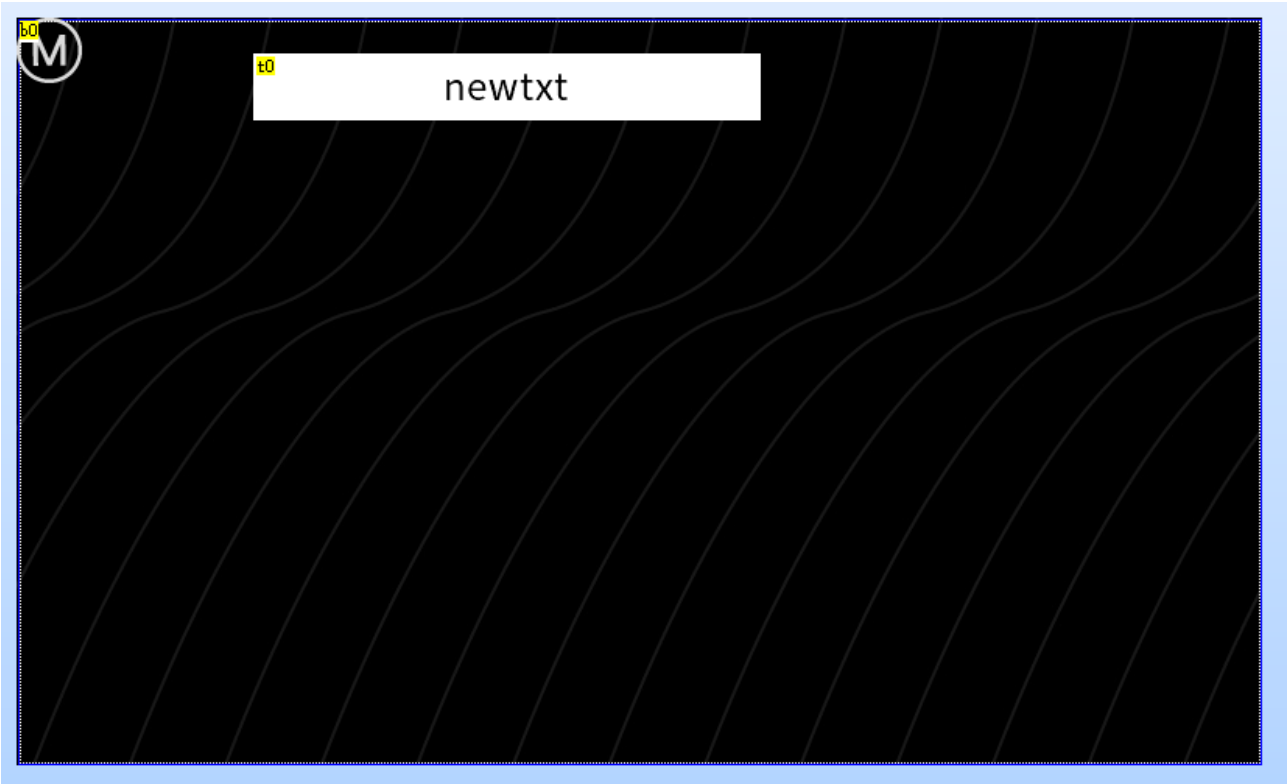


2.1.1.2 模拟器调试示例二

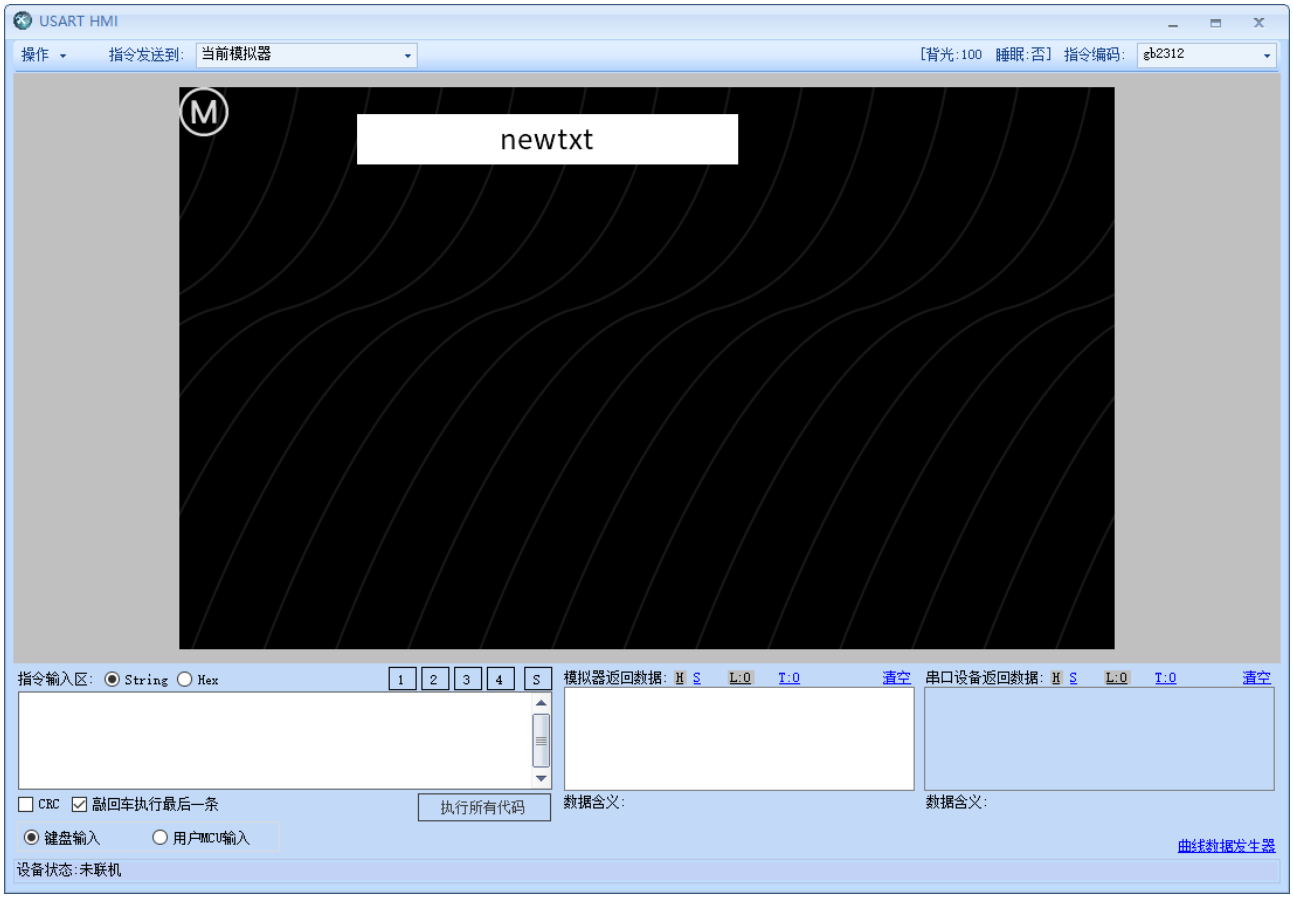
切换到 main 页面, 点击工具箱中的文本, 在 main 页面新建一个文本控件


工具箱
文本
滚动文本
数字
虚拟浮点数
按钮
进度条
图片
切图
触摸热区
触摸捕捉
指针
曲线/波形
滑块
定时器
变量

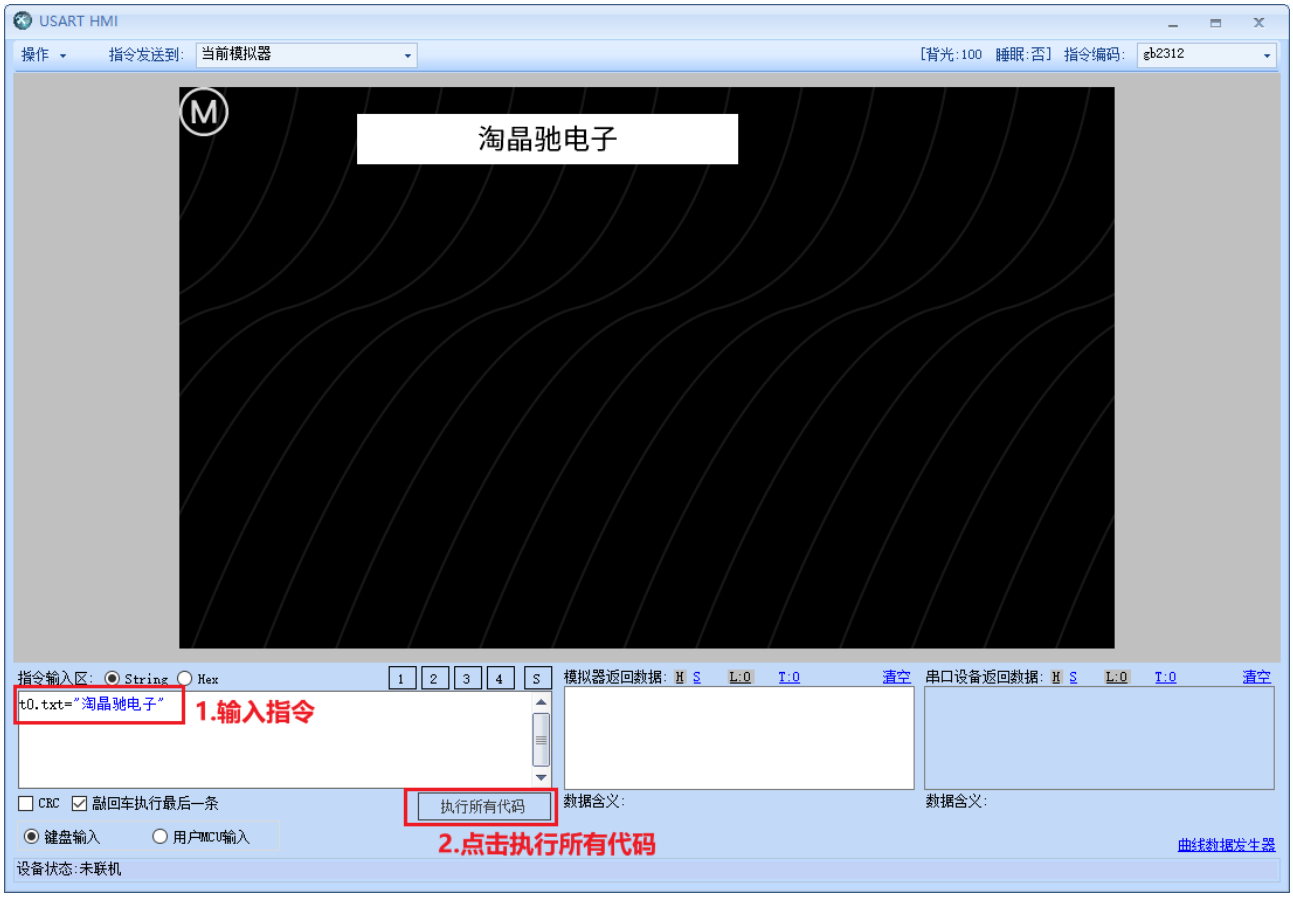
可以通过鼠标移动控件的位置和调整控件的大小



点击调试, 进入调试页面


USART HMI
操作
指令发送到:
当前模拟器
[背光:100 睡眠:否] 指令编码:
gb2312
newtxt
指令输入区:
String
Hex
1
2
3
4
S
模拟器返回数据:
H
S
L:0
T:0
清空
串口设备返回数据:
H
S
L:0
T:0
清空
CRC
敲回车执行最后一条
执行所有代码
数据含义:
数据含义:
键盘输入
用户MCU输入
曲线数据发生器
设备状态:未联机

可以在底部的指令输入区输入指令，并点击执行来实现控件赋值，比如我们让文本控件 t0 显示”淘晶驰电子”



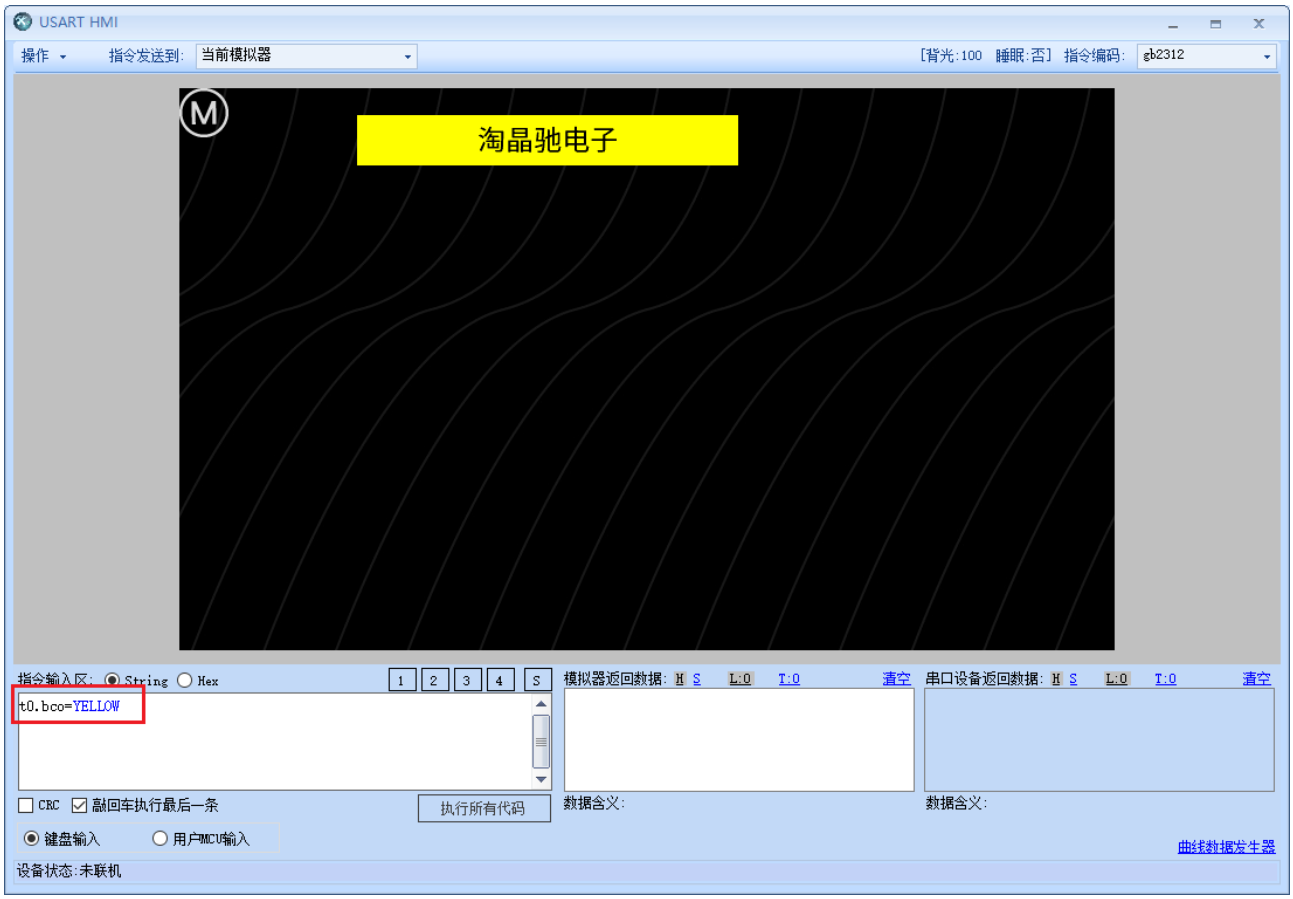
指令如下:

```
t0.txt="淘 晶 驰 电 子"
```

注意: 赋值指令中的引号是英文的”” 而不是中文的 “”

也可以修改文本控件的背景颜色, 可以通过指令修改的是控件的“绿色属性”

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？(page 1295)



指令如下:

```
t0.bco=YELLOW
```

注意: t0.txt=”淘晶驰电子”和 t0.bco=YELLOW 的区别是一个加了引号, 一个没有加引号, 这是因为 txt 是字符串,bco 则是整形, 目前只有 txt,path,dir,filter 是字符串, 其他均为整形

txt 属性和 val 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

2.2 和 USB 转 TTL 的接线说明

串口屏的 TX（发送）接 USB 转 TTL 的 RX（接收）

串口屏的 RX（接收）接 USB 转 TTL 的 TX（发送）

串口屏的 GND 要和 USB 转 TTL 的 GND 连接在一起（共地）

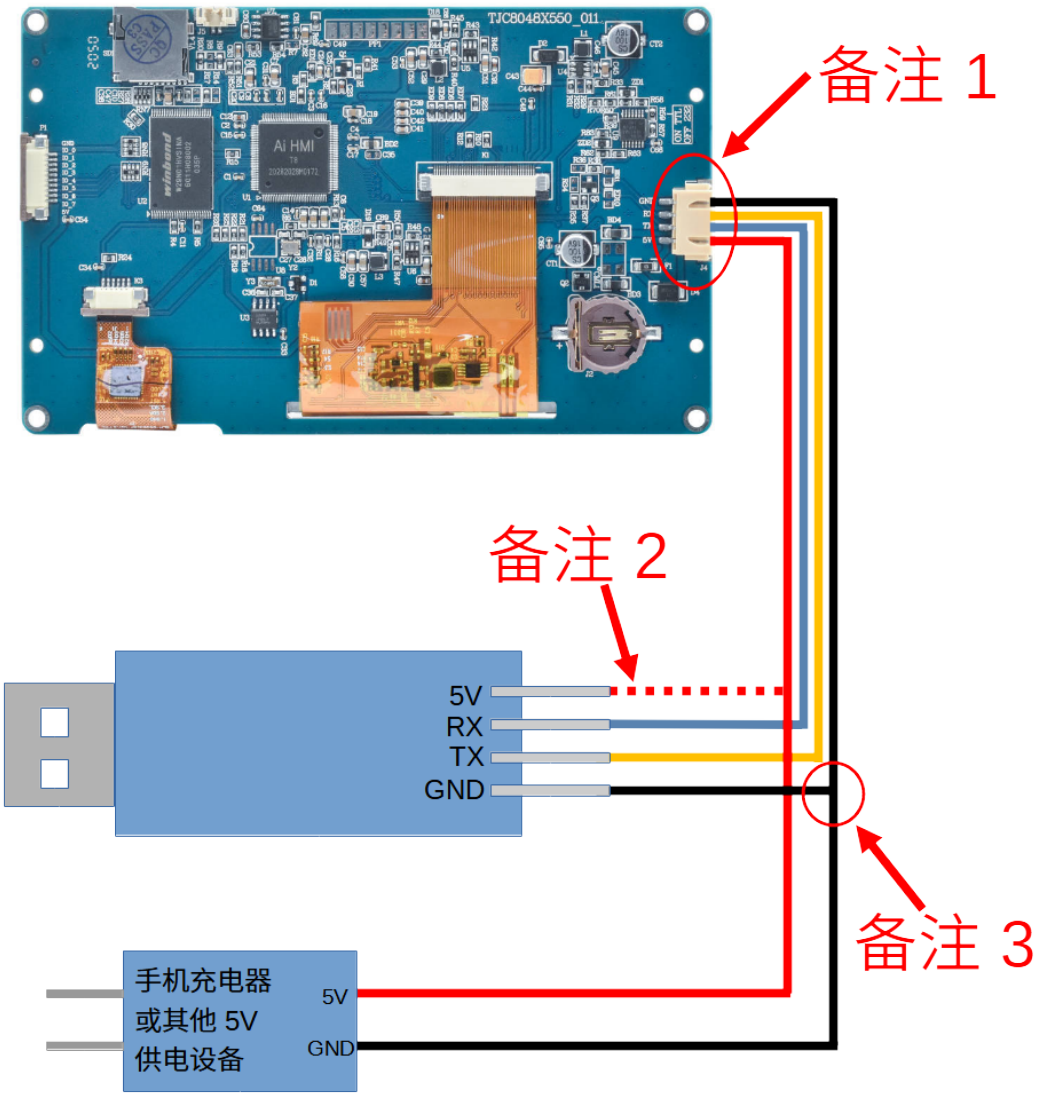
如果上电时发现串口屏不断重启（4.3 寸以上的屏幕以及带喇叭的屏幕），可能是供电不足，这时候需要并联一个电源来提高电流

需要额外找一个电源引出电源正负极，注意正负极之间的电压应该在屏幕的供电范围内

将电源正极接到屏幕的 VCC 上（请注意电压范围）

电源负极接到 GND（共地）

这样就实现了对屏幕的额外供电

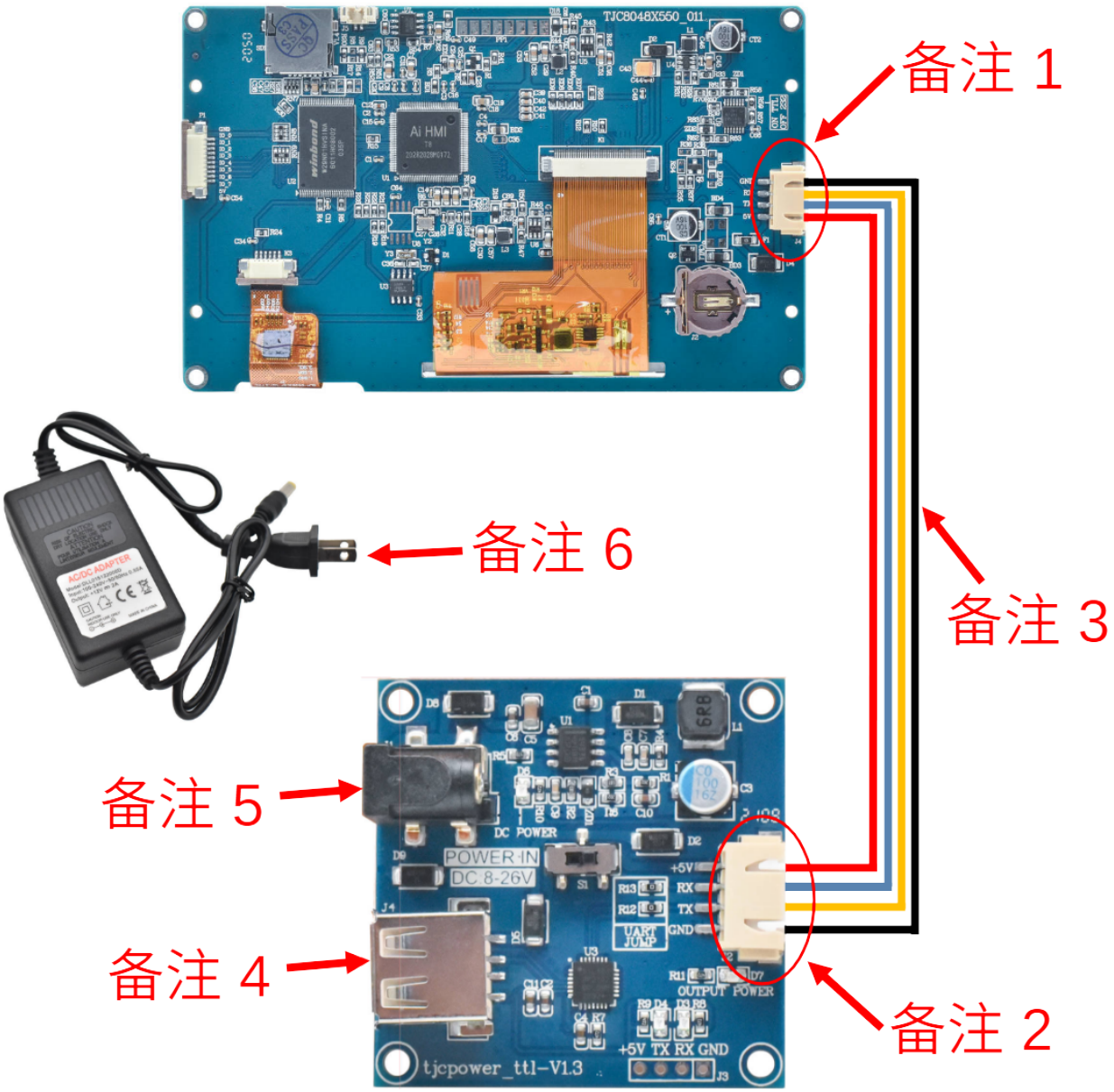


备注 1： 5V 接 5V， GND 接 GND， USB 转 TTL 的 TX 接串口屏 RX， USB 转 TTL 的 RX 接串口屏 TX

备注 2：当接了外部供电时，断开 USB 转 TTL 的 5V，避免外部电流倒灌进入电脑损坏了电脑

备注 3：所有的 GND 要接在一起（共地），才能正常工作和通讯

下图为官方的 USB 转 TTL，可以直接接 DC 直流电源进行供电



备注 1/2： 5V 接 5V， GND 接 GND， USB 转 TTL 的 TX 接串口屏 RX， USB 转 TTL 的 RX 接串口屏 TX

备注 3： 两边的 GND 要接在一起（共地），才能正常通讯

备注 4： 通过 USB 线接电脑

备注 5： 额外供电口，输入电压支持 8V-26V

备注 6： 4.3 寸以上的屏幕仅仅使用 USB 线是带不动的，需要使用外部电源供电，否则会无法启动或者不断闪烁重启

注意： UART（以及 USART）串口只能一对一，串口不允许接多个设备，一个串口只能接一个设备

2.3 和单片机的接线说明

串口屏的 TX（发送）接单片机的 RX（接收）

串口屏的 RX（接收）接单片机的 TX（发送）

串口屏的 GND 要和单片机的 GND 连接在一起（共地）

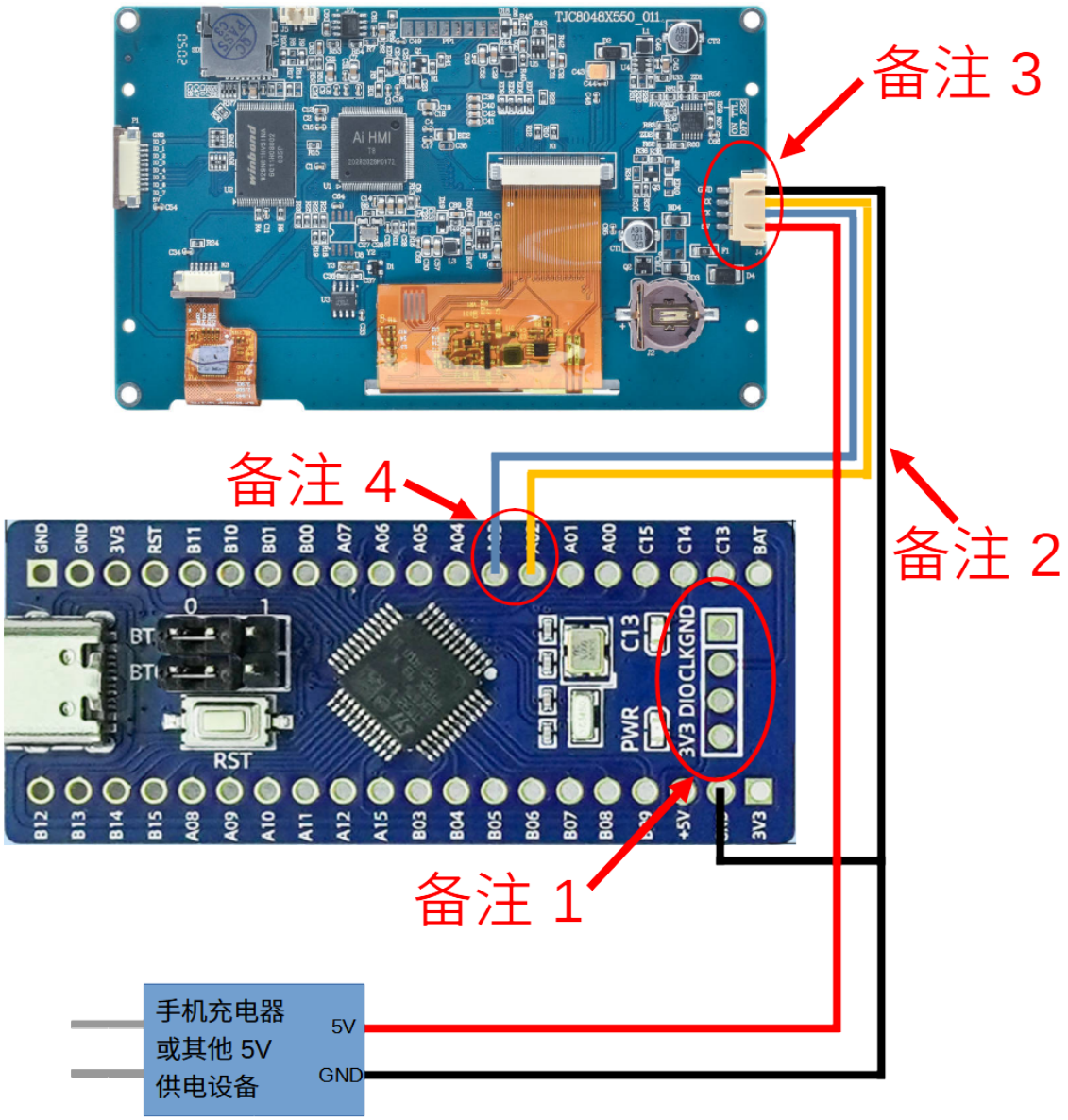
如果上电时发现串口屏不断重启（4.3 寸以上的屏幕以及带喇叭的屏幕），可能是供电不足，这时候需要并联一个电源来提高电流

需要额外找一个电源引出电源正负极，注意正负极之间的电压应该在屏幕的供电范围内

将电源正极接到屏幕的 VCC 上（请注意电压范围）

电源负极接到 GND（共地）

这样就实现了对屏幕的额外供电



备注 1： 给 STM32 下载程序和供电 (3.3V) 还是通过这个接口

备注 2： 两边的 GND 要接在一起 (共地)，才能正常通讯

备注 3： 如果要通过 usb 转 ttl 给屏幕下载程序，要断开和 STM32 的连接，将屏幕接口和 usb 转 ttl 接在一起才能下载

备注 4： STM32 的 TX 接串口屏的 RX， STM32 的 RX 接串口屏的 TX

注意： UART（以及 USART）串口只能一对一，串口不允许接多个设备，一个串口只能接一个设备

如果使用的是正点原子或者野火等厂家出的开发板，请注意检查对应的串口上是否已经接了其他外设或者芯片如果这个串口上已经接了其他的外设或者芯片，可能会照成无法通讯

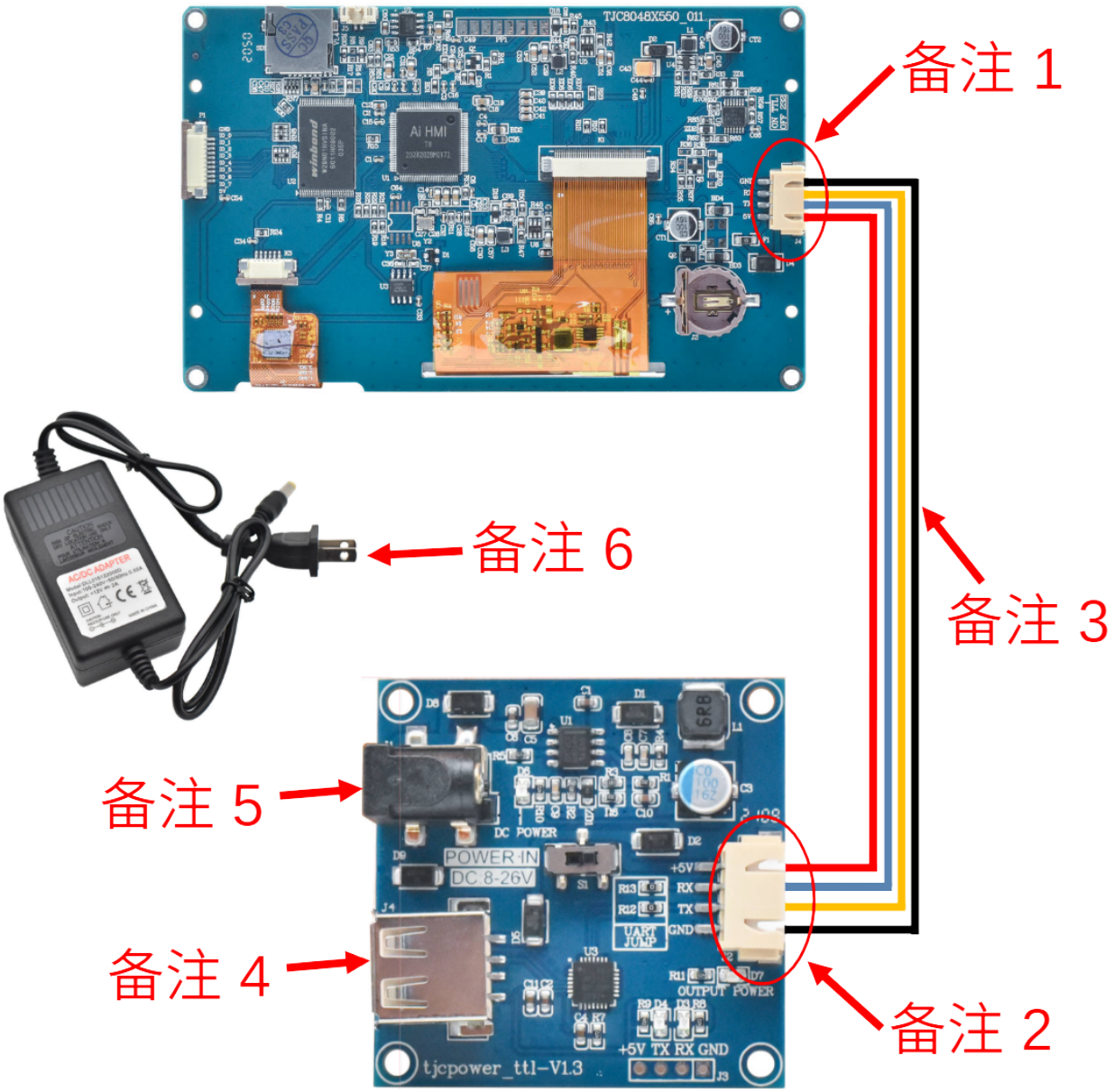
2.4 联机调试基础知识

单片机/串口屏/电脑模拟器如何连接

> **注意:** 任一时间, 只允许一对一连接, 有以下 3 种情况:

2.4.1 模拟器 (电脑) 与串口屏实物连接

连接的接线图如下



备注 1/2： 5V 接 5V， GND 接 GND， USB 转 TTL 的 TX 接串口屏 RX， USB 转 TTL 的 RX 接串口屏 TX

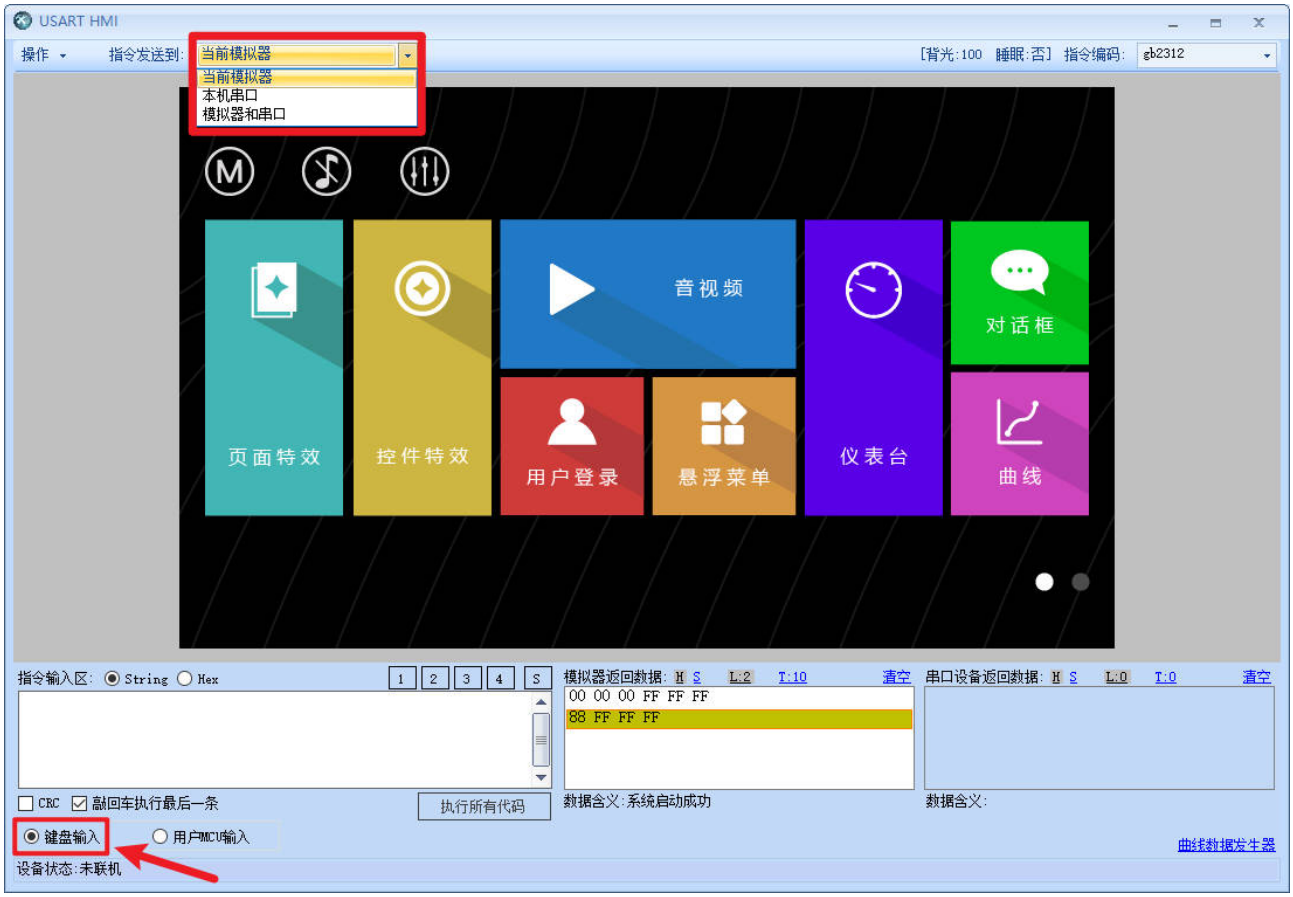
备注 3： 两边的 GND 要接在一起（共地），才能正常通讯

备注 4： 通过 USB 线接电脑

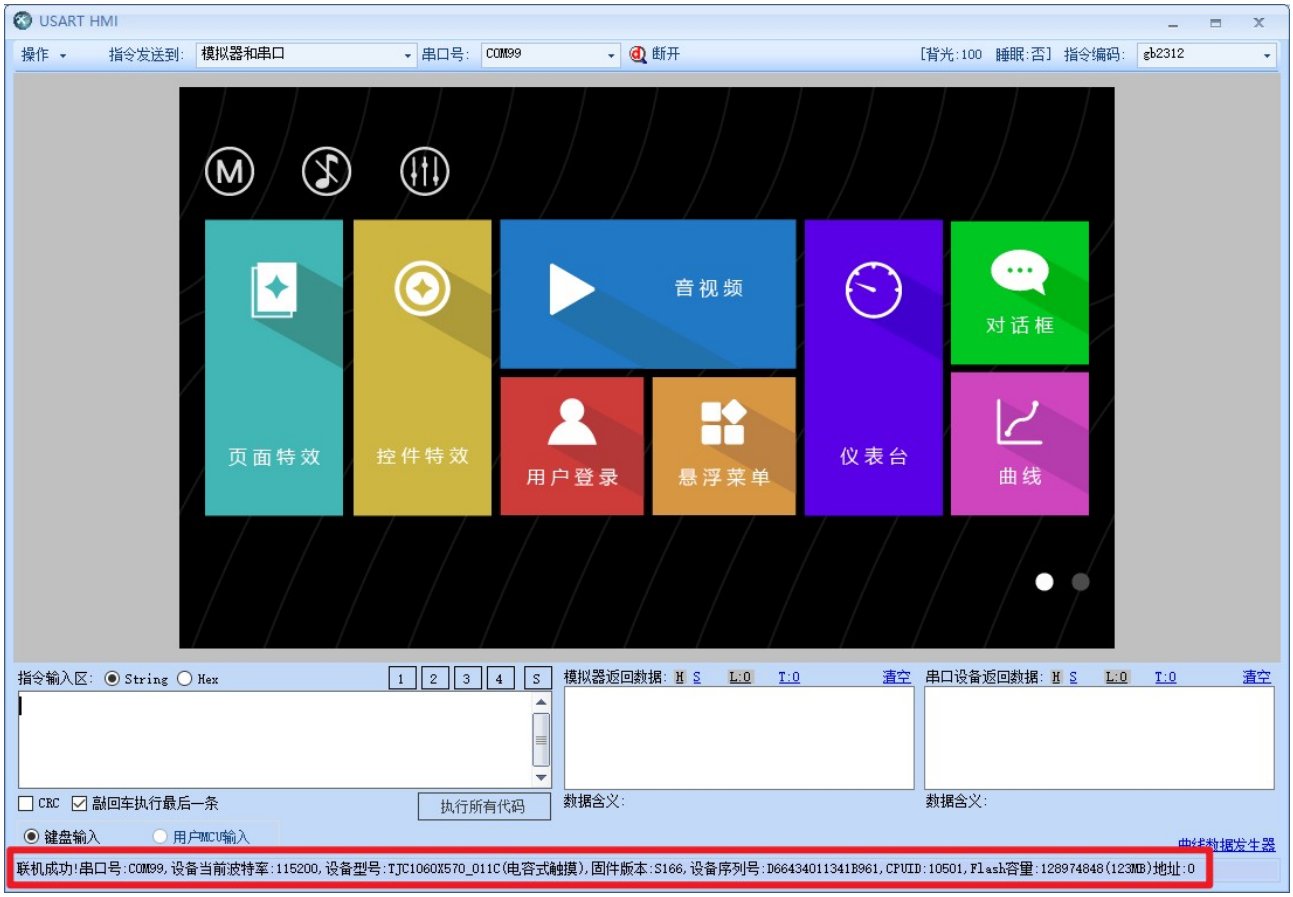
备注 5： 额外供电口，输入电压支持 8V-26V

备注 6： 4.3 寸以上的屏幕仅仅使用 USB 线是带不动的，需要使用外部电源供电，否则会无法启动或者不断闪烁重启

1. 必须保证下面选择的是“键盘输入”，上面才能选择模拟器和串口，与串口屏进行联机



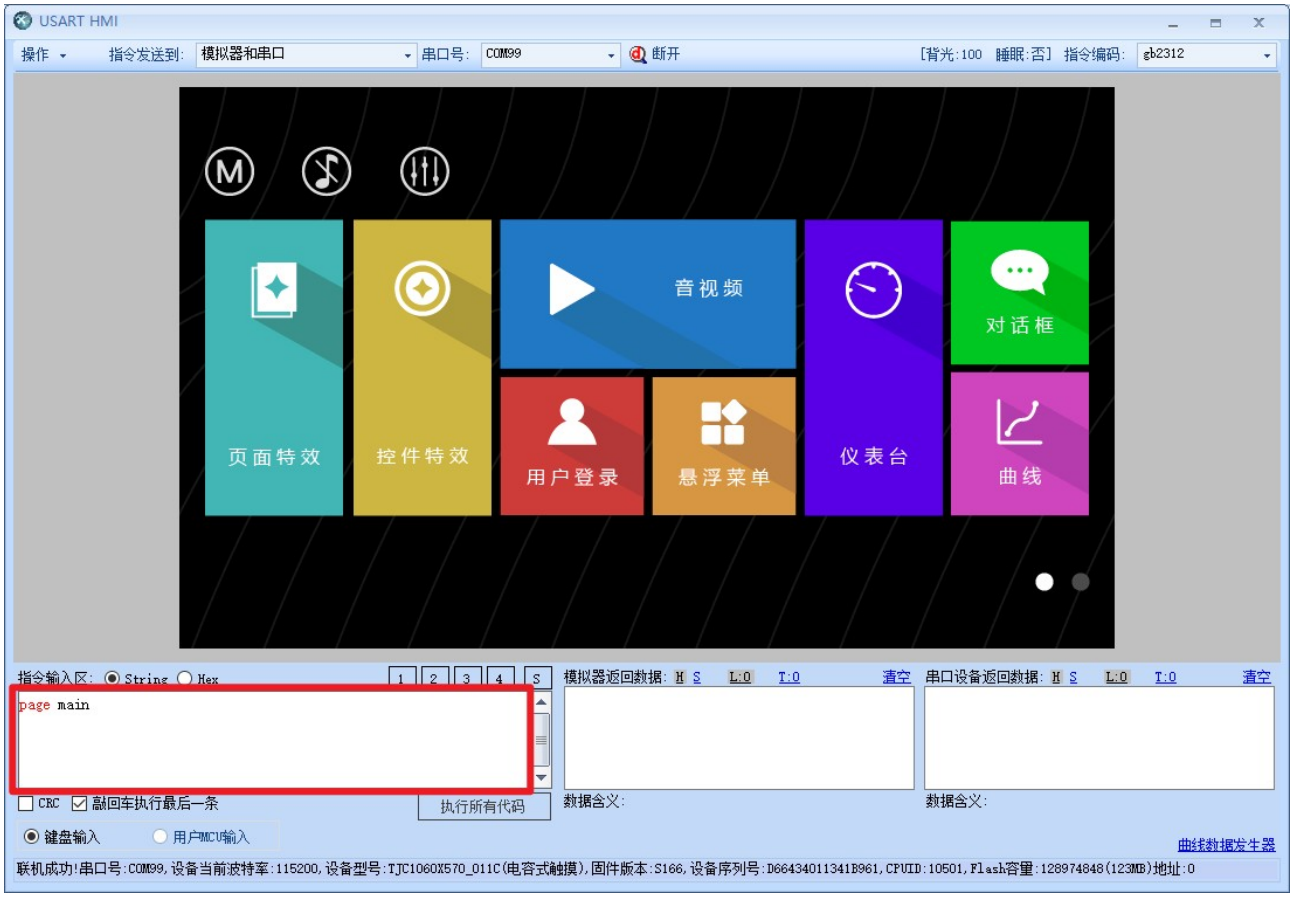
2. 当模拟器和一个串口屏实物连接成功时，底部将会显示联机成功, 以及联机获取到的串口屏参数



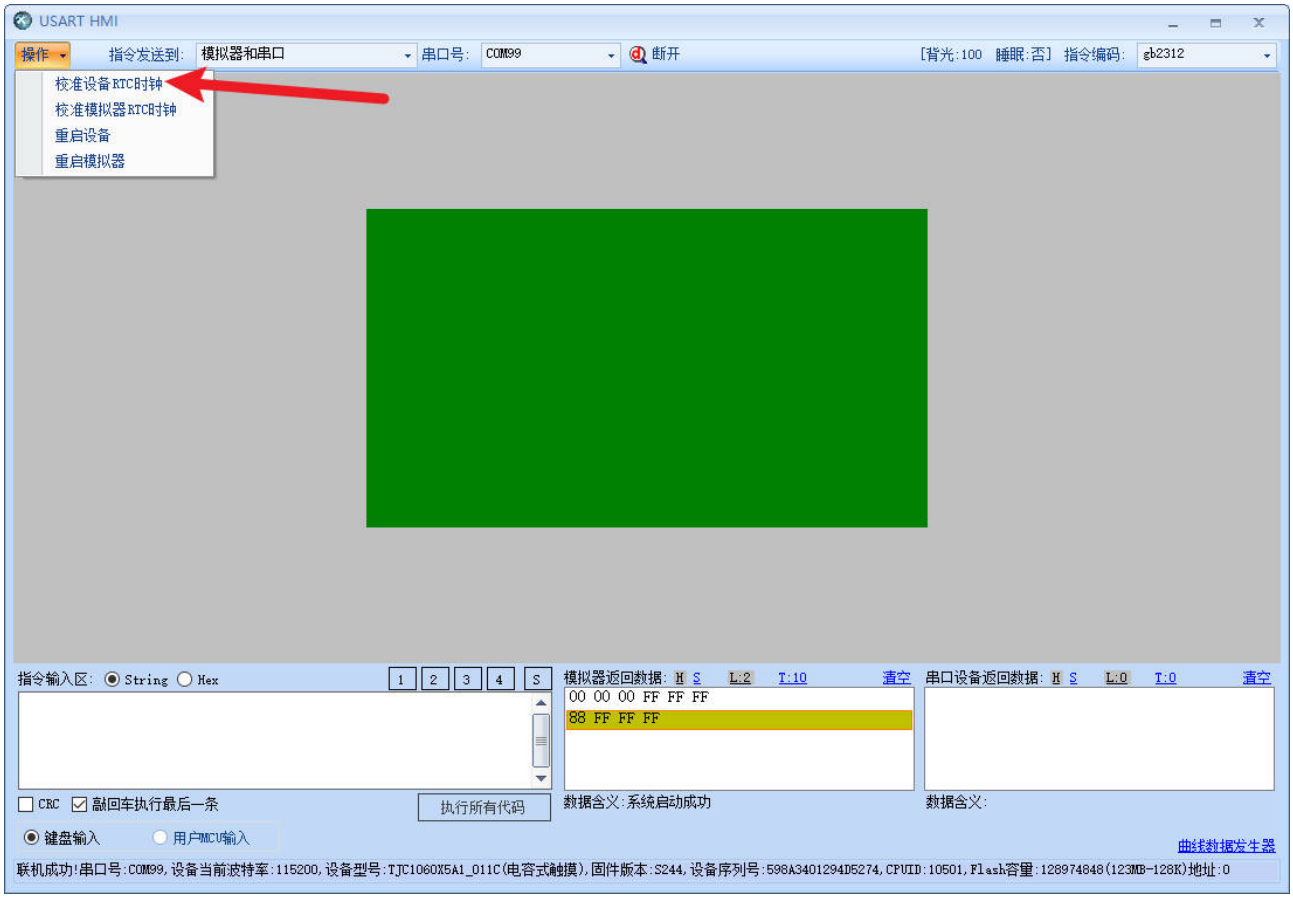
注意: 联机成功后, 串口屏实物会退出主动解析功能, 如果联机后需要使用主动解析功能, 需要手动下发一次 recmod=1, 如果不使用主动解析模式, 请勿下发 recmod=1

USART HMI
操作
指令发送到:
模拟器和串口
串口号:
COM99
断开
[背光:100 睡眠:否] 指令编码:
gb2312
指令输入区:
String
Hex
1
2
3
4
S
recmod=1
模拟器返回数据:
H S
L:0
T:0
清空
串口设备返回数据:
H S
L:0
T:0
清空
CRC
敲回车执行最后一条
执行所有代码
数据含义:
数据含义:
键盘输入
用户MCU输入
曲线数据发生器
联机成功!串口号:COM99,设备当前波特率:115200,设备型号:TJC1060X5A1_011C(电容式触摸),固件版本:S244,设备序列号:598A3401294D5274,CPUID:10501,Flash容量:128974848(123MB-128K)地址:0

3. 此时输入指令，指令会被同时发送到模拟器和串口屏实物上



5. 除此之外, 如果屏幕是带 RTC 的 (例如 X5/K0 系列), 可以校准 RTC(必须是在联机成功的状态下)，校准前请安装串口屏实物背部的 RTC 电池, 型号是 CR1220



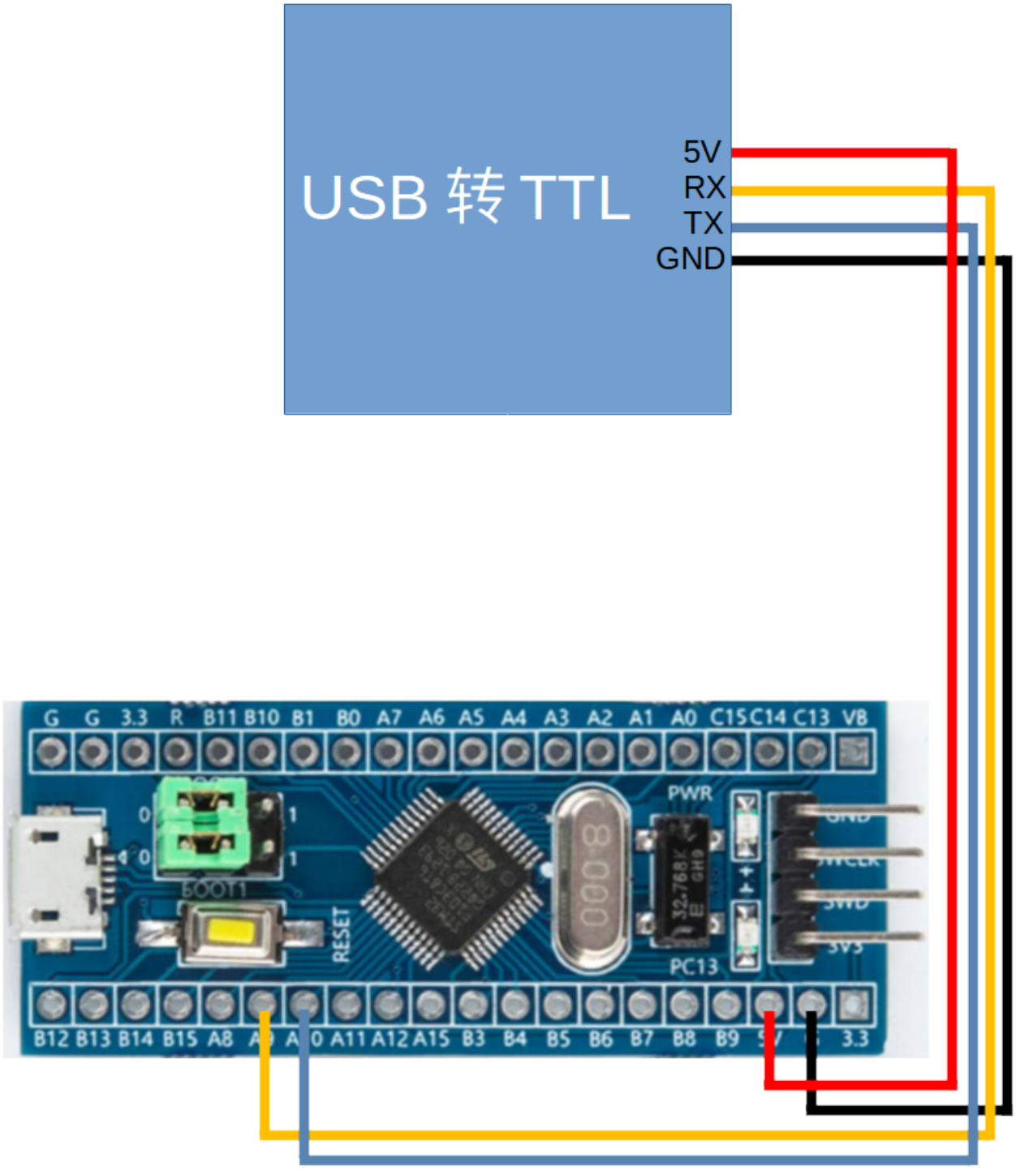
2.4.2 模拟器 (电脑) 与单片机连接

提示： 淘晶驰强大的模拟器功能几乎能够实现与实体屏幕几乎一样的功能，除外部 IO，电阻屏校准等实物特有的少数功能外，几乎都可以在屏幕上完全仿真

您可以提前通过 (电脑上的) 模拟器和单片机之间进行通讯，在屏幕样品快递之前就能完成大部分功能的开发和调试，大大降低开发和选型成本

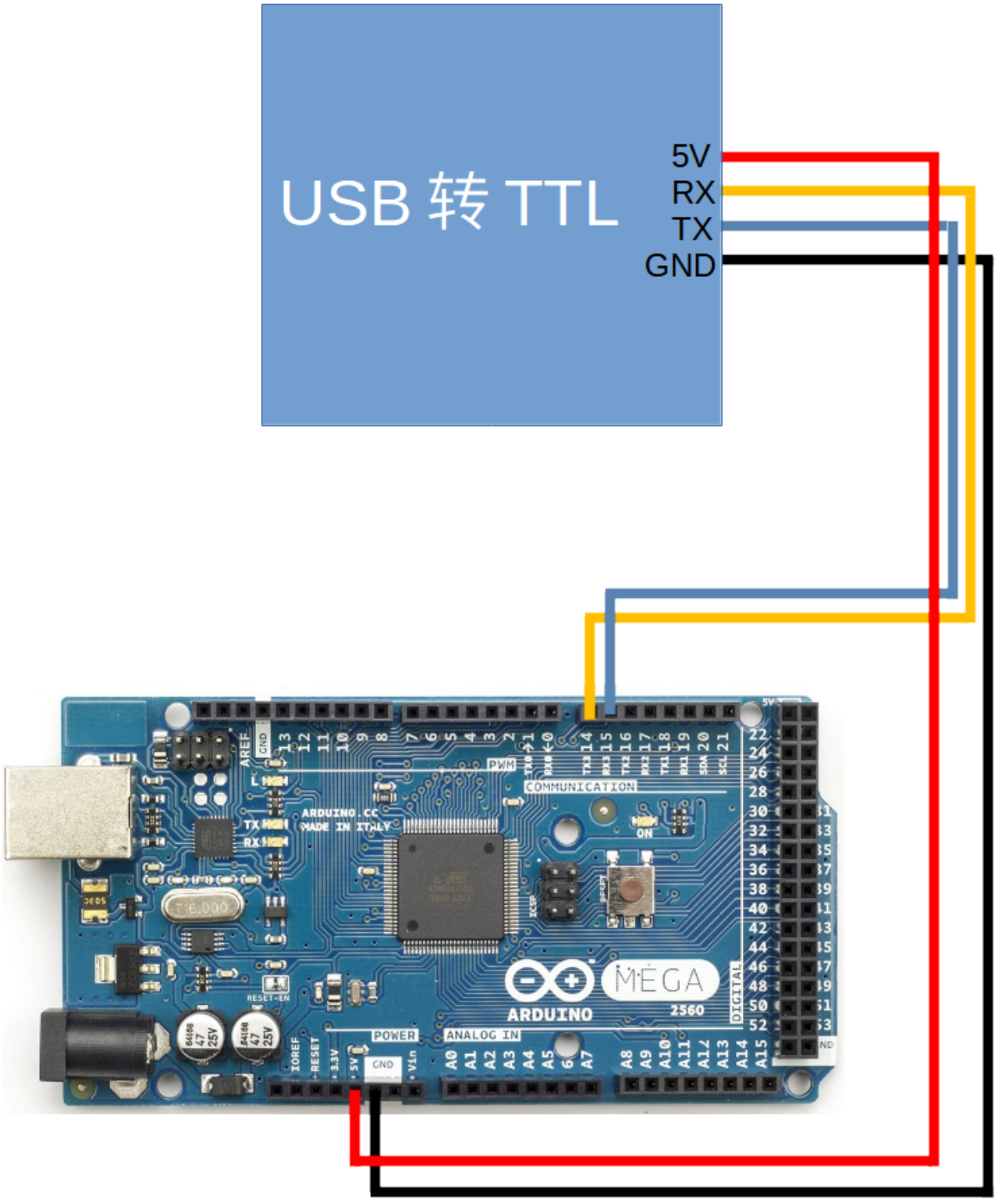
2.4.2.1 模拟器连接 stm32

> **注意：** 请根据自己的需求判断是否需要连接 5V 供电

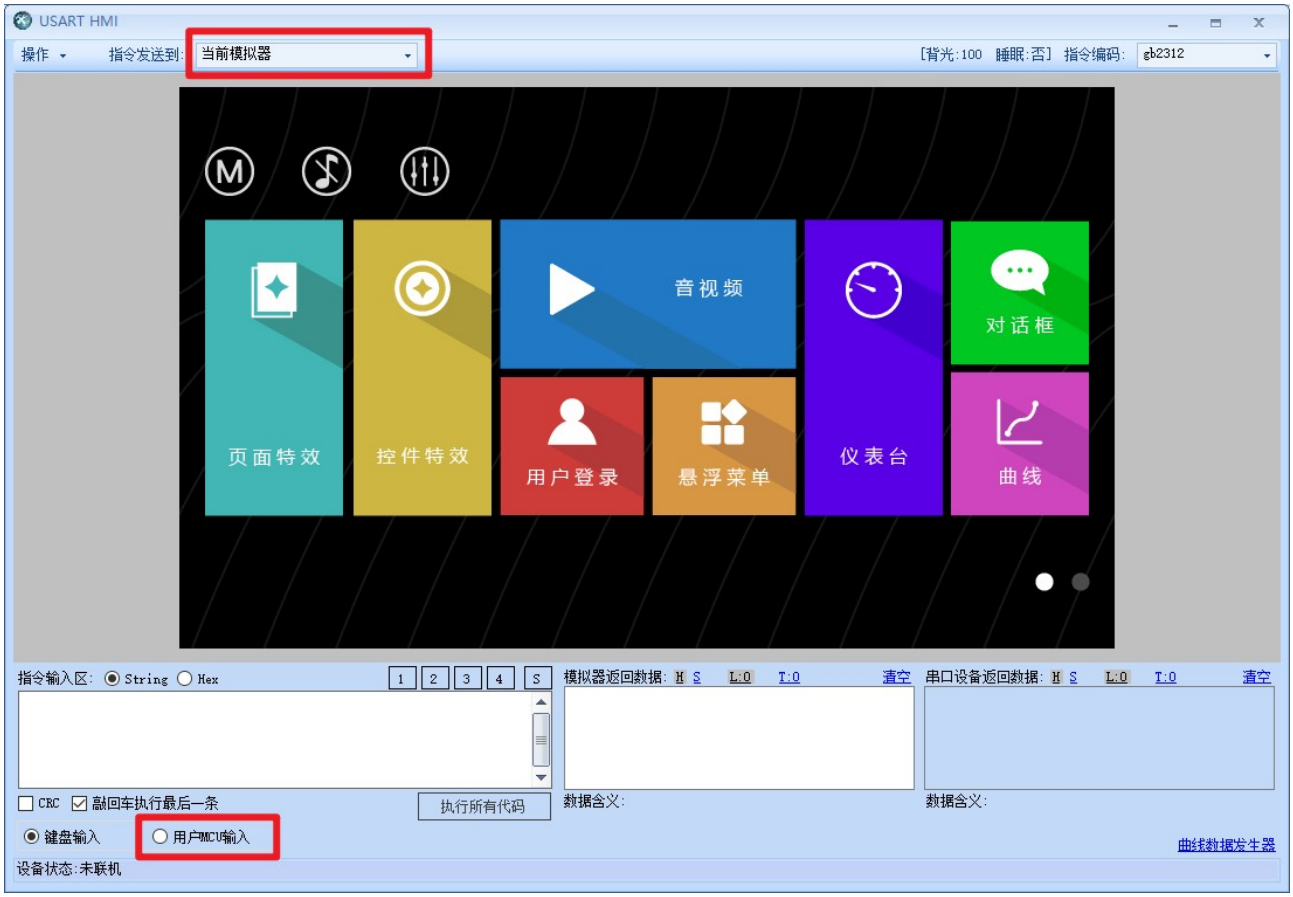

USB 转 TTL
5V
RX
TX
GND
G G 3.3 R B11 B10 B1 B0 A7 A6 A5 A4 A3 A2 A1 A0 C15 C14 C13 VB
PWR
RESET
PC13
B12 B13 B14 B15 A8 A11 A12 A15 B3 B4 B5 B6 B7 B8 B9 3.3

2.4.2.2 模拟器连接 arduino

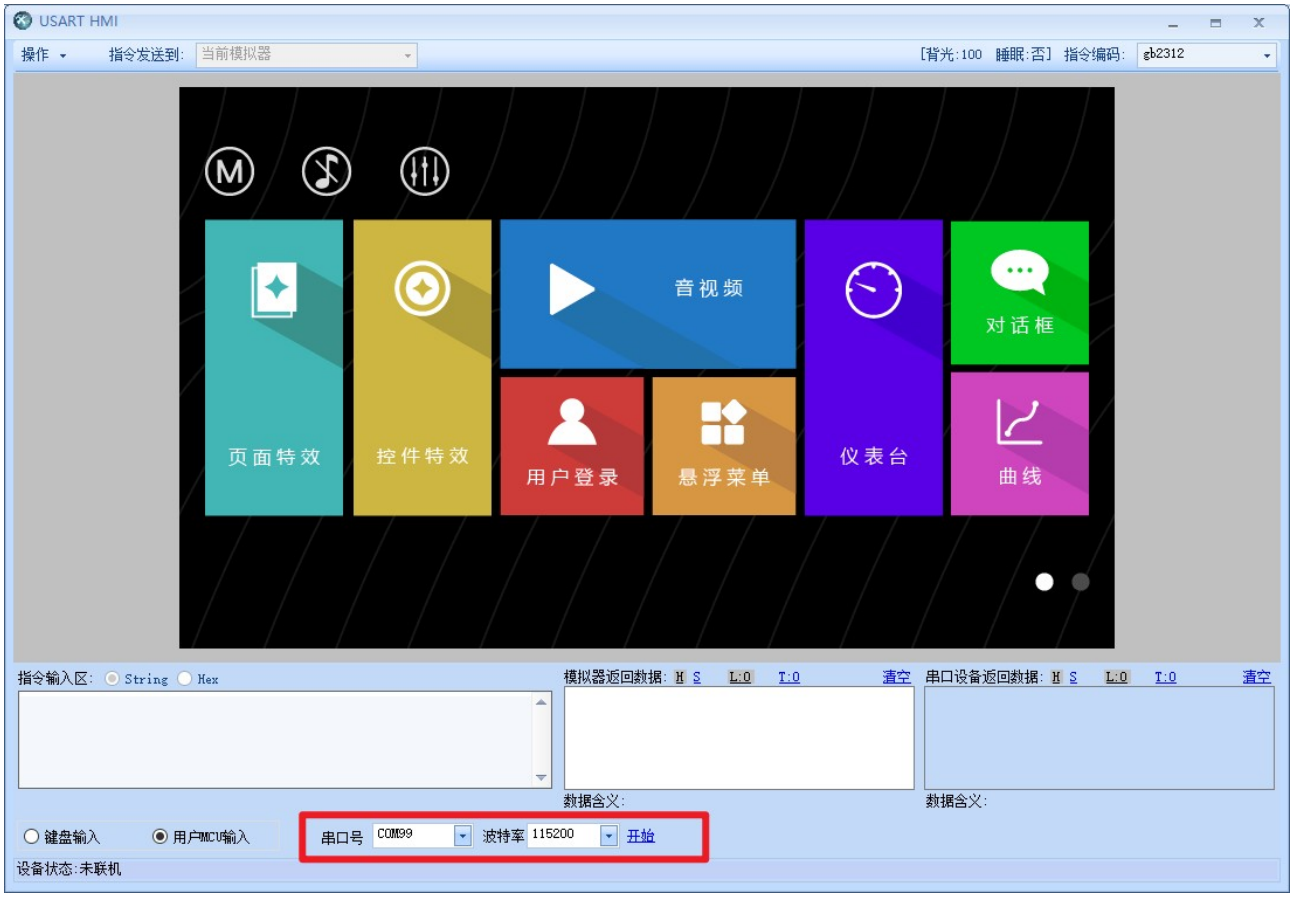
注意: 请根据自己的需求判断是否需要连接 5V 供电


USB 转 TTL
5V
RX
TX
GND
ARDUINO.CC
MADE IN ITALY
COMMUNICATION
PWM
POWER
ANALOG IN
DIGITAL
ARDUINO
MEGA
2560

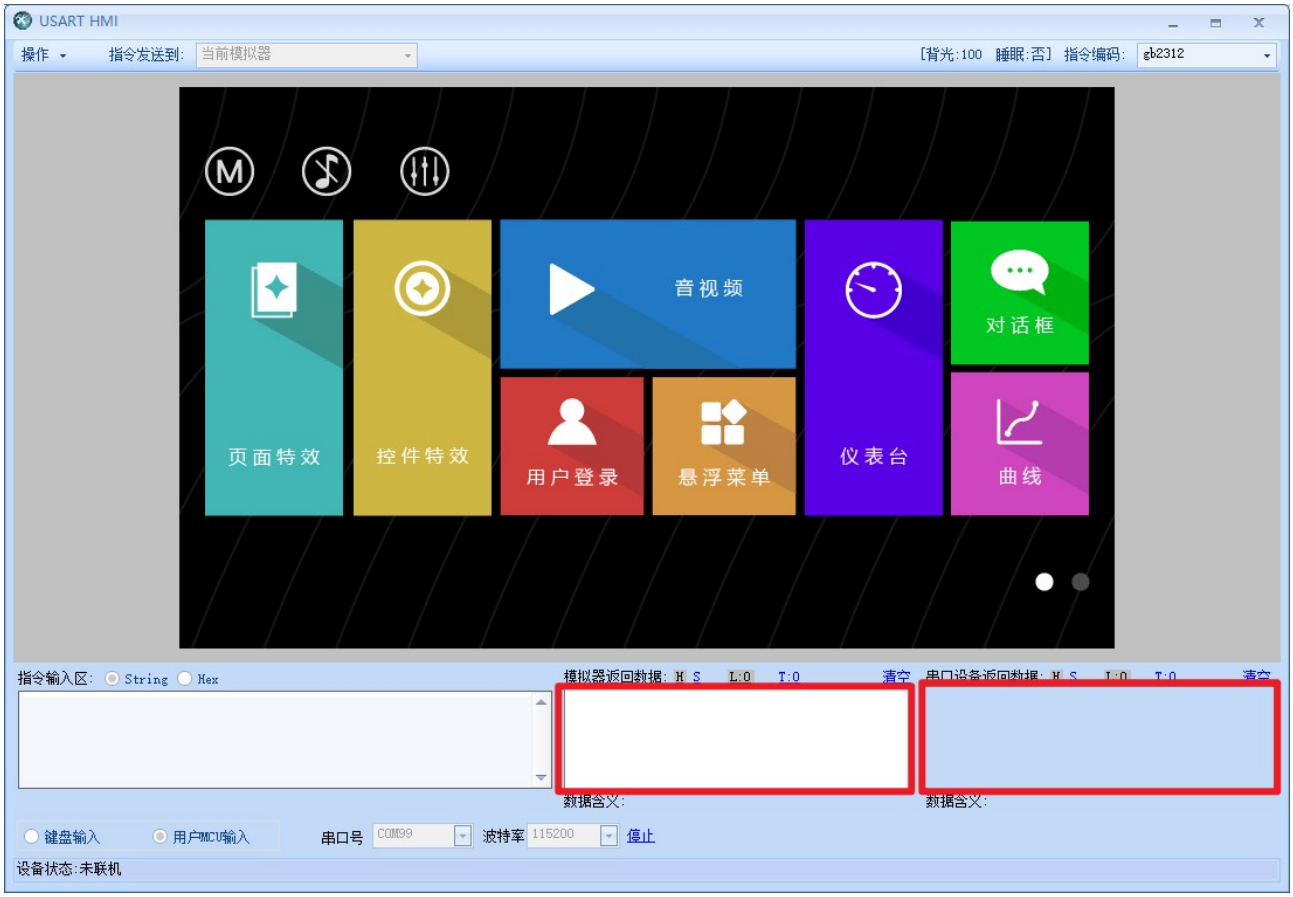
1. 必须保证上面选择的是“当前模拟器”，下面才能选择“MCU 输入”



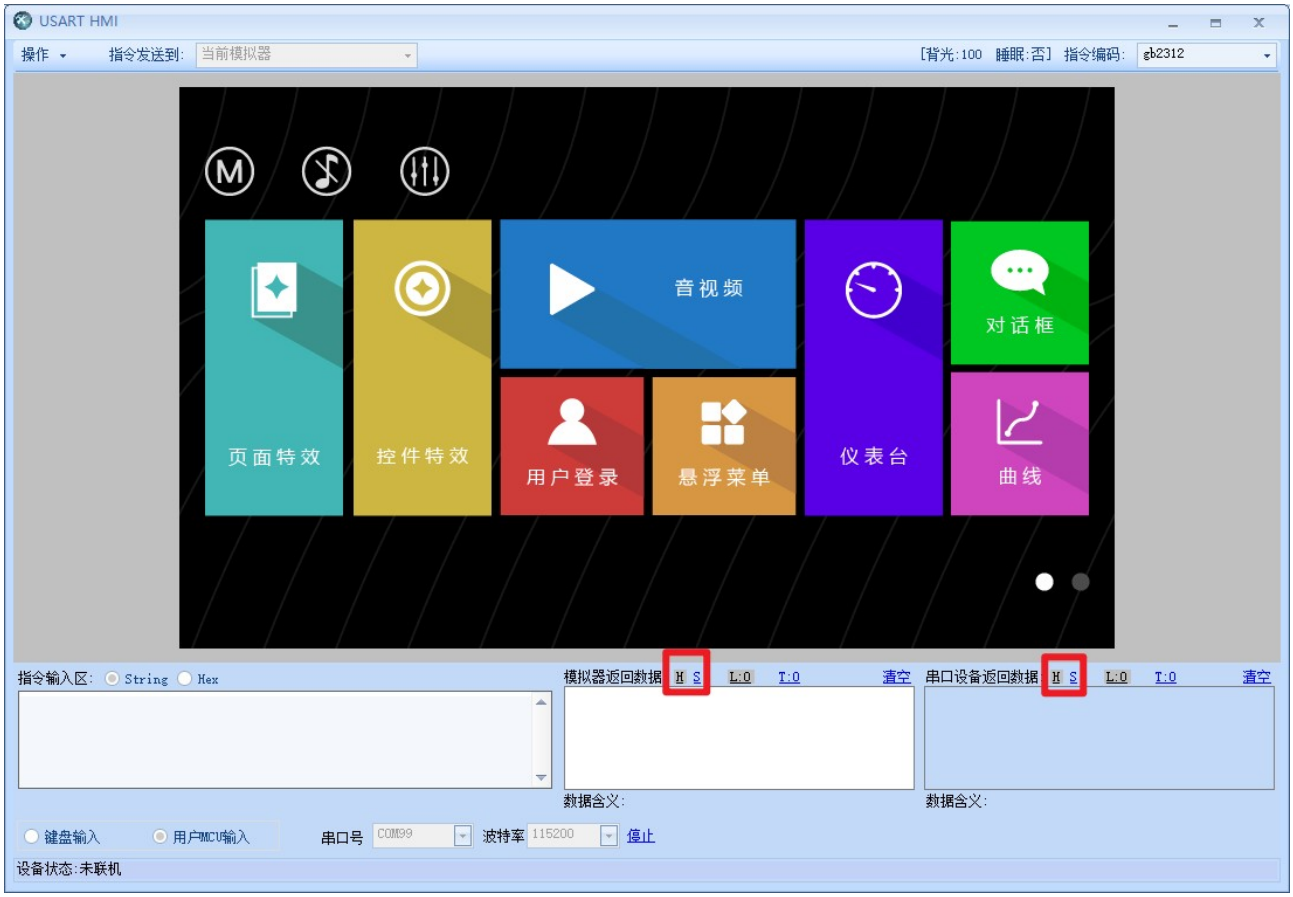
2. 点击开始，就可以直接和单片机进行通讯了



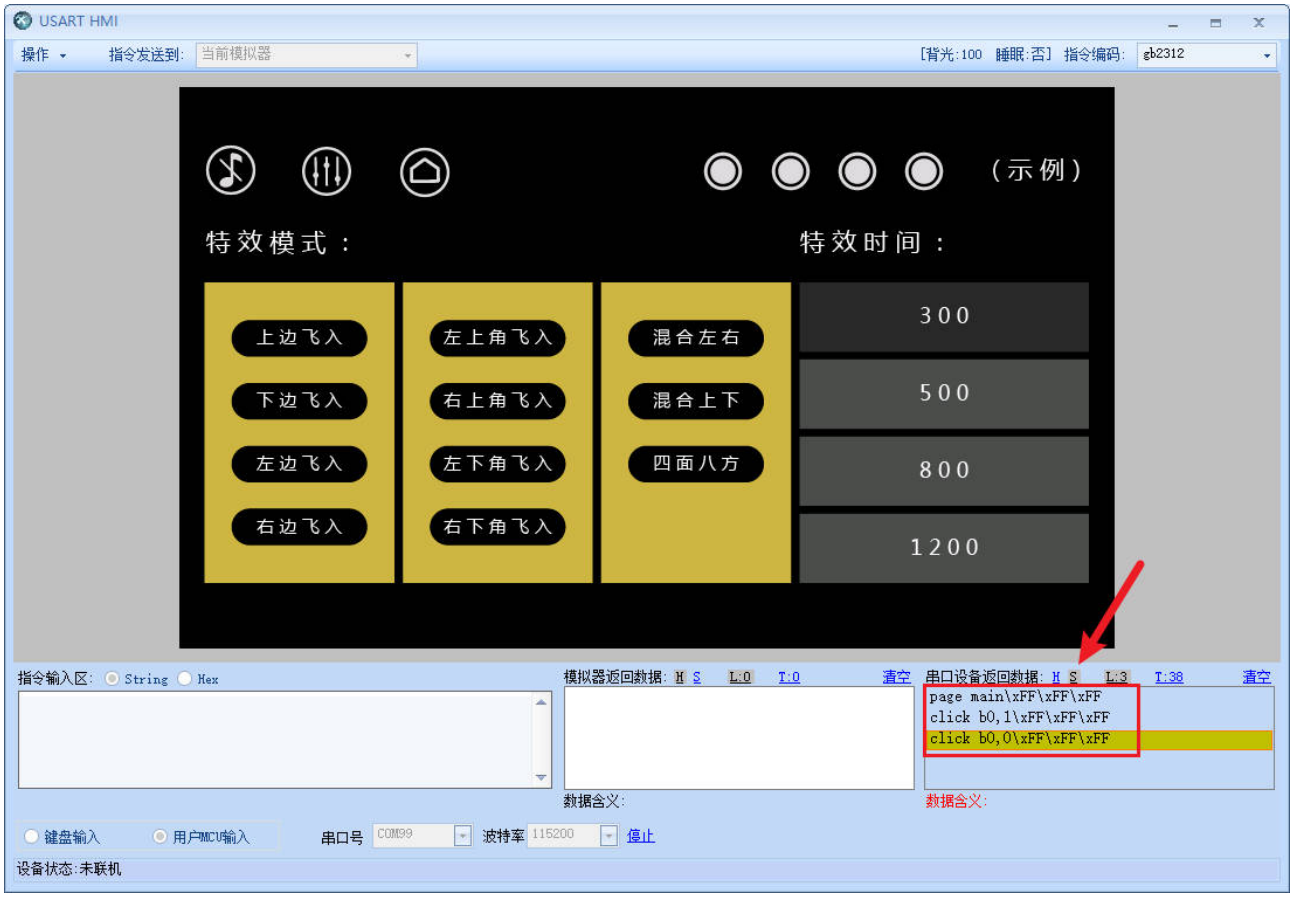
3. 所有调试信息都会显示在红框中，左边的是模拟器发给单片机的数据，右边的是单片机发给屏幕的数据



4.H 代表 hex 模式，S 代表字符串模式



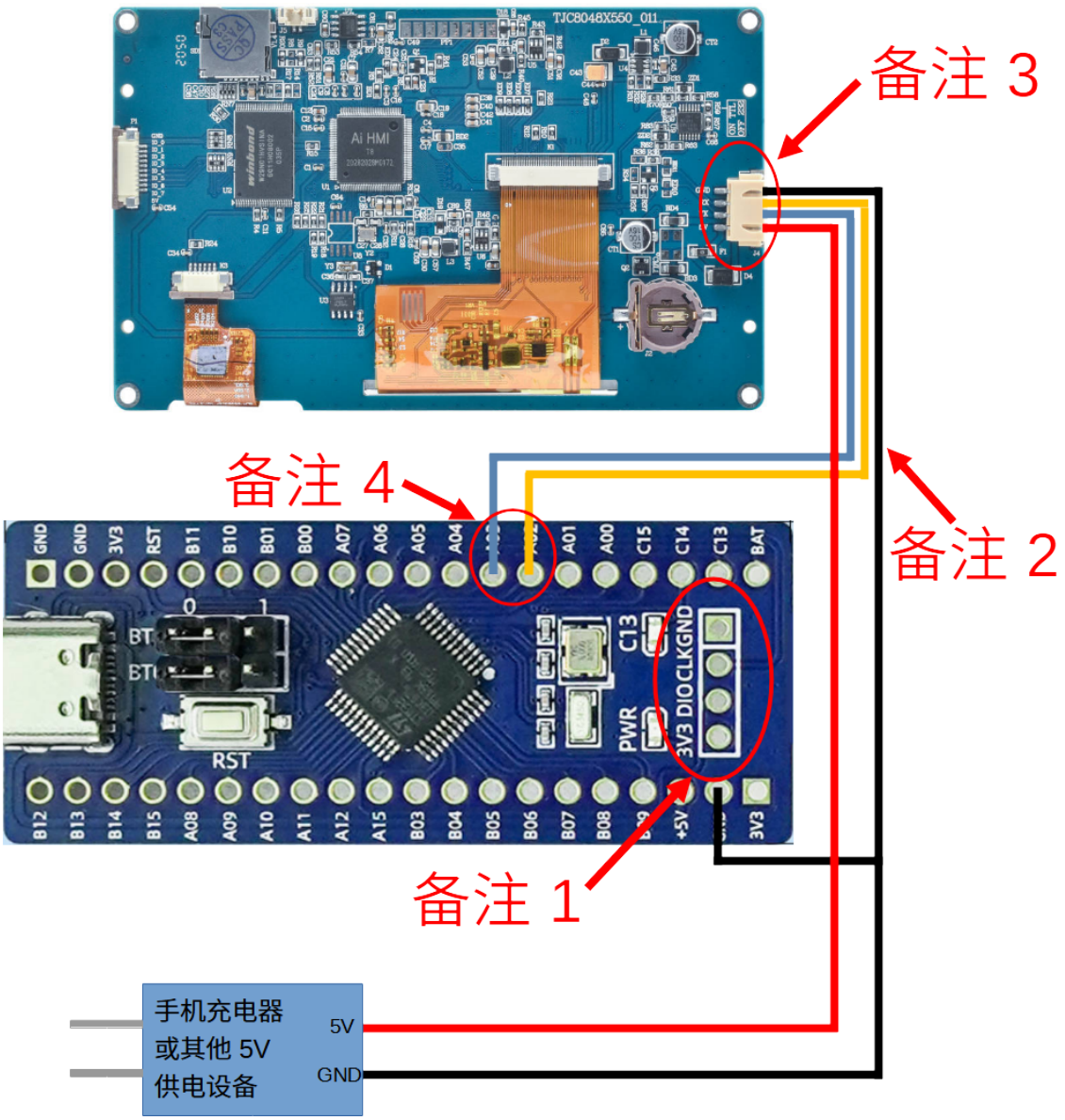
5. 当切换到字符串模式时, 可以清晰的看到单片机发送过来的指令和结束符, 并且模拟器可以实现对指令的响应


USART HMI
操作
指令发送到:
当前模拟器
[背光:100 睡眠:否] 指令编码:
gb2312
(示例)
特效模式：
特效时间：
上边飞入
下边飞入
左边飞入
右边飞入
左上角飞入
右上角飞入
左下角飞入
右下角飞入
混合左右
混合上下
四面八方
300
500
800
1200
指令输入区:
String
Hex
模拟器返回数据:
串口设备返回数据:
page main\xFF\xFF\xFF
click b0,1\xFF\xFF\xFF
click b0,0\xFF\xFF\xFF
数据含义:
键盘输入
用户MCU输入
串口号
COM99
波特率
115200
停止
设备状态:未联机

2.4.3 串口屏与单片机连接

串口屏连接单片机开发板注意事项 (page 1474)

2.4.3.1 串口屏连接 stm32



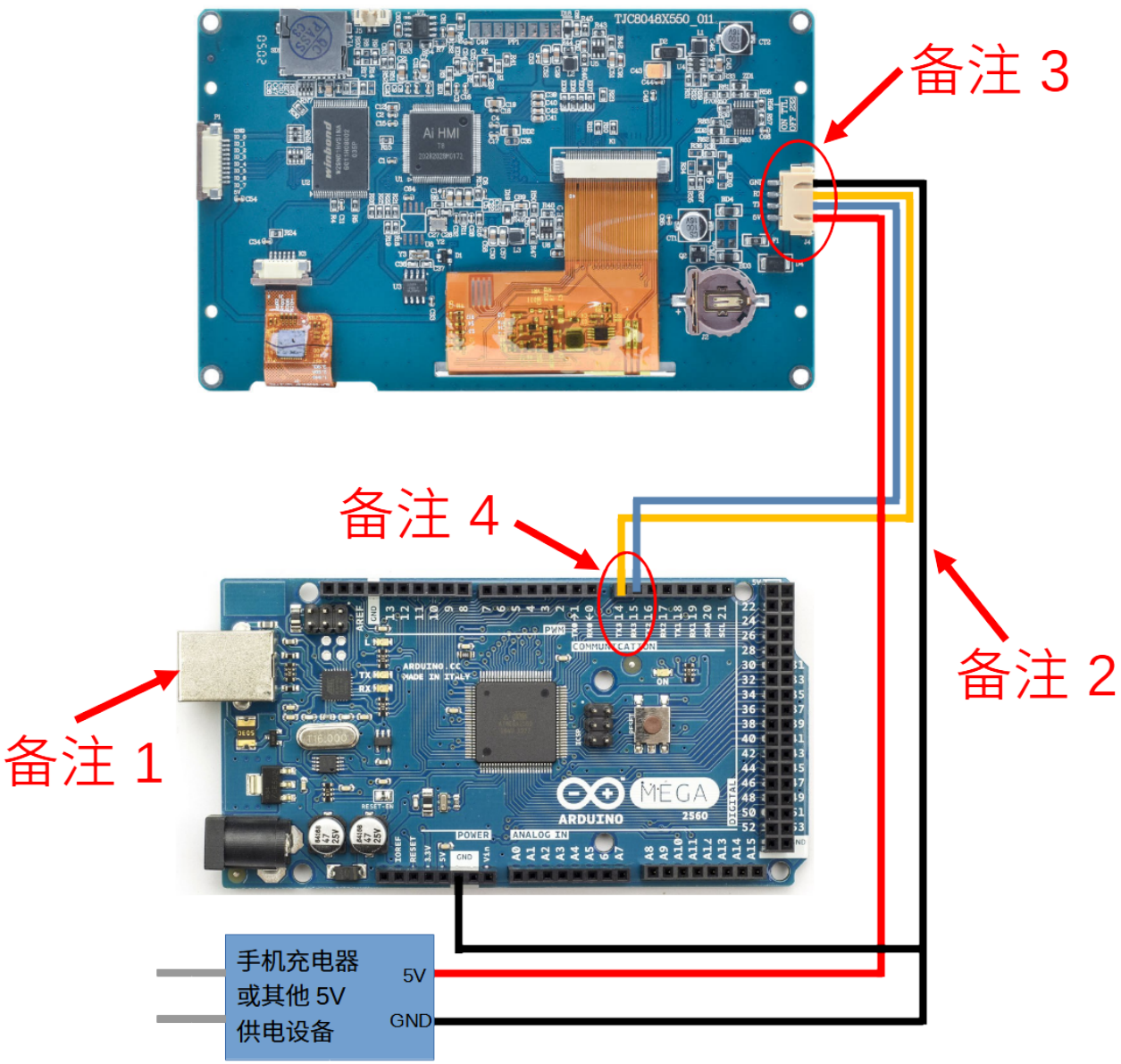
备注 1： 给 STM32 下载程序和供电 (3.3V) 还是通过这个接口

备注 2： 两边的 GND 要接在一起 (共地)，才能正常通讯

备注 3： 如果要通过 usb 转 ttl 给屏幕下载程序，要断开和 STM32 的连接，将屏幕接口和 usb 转 ttl 接在一起才能下载

备注 4： STM32 的 TX 接串口屏的 RX， STM32 的 RX 接串口屏的 TX

2.4.3.2 串口屏连接 arduino mega 2560



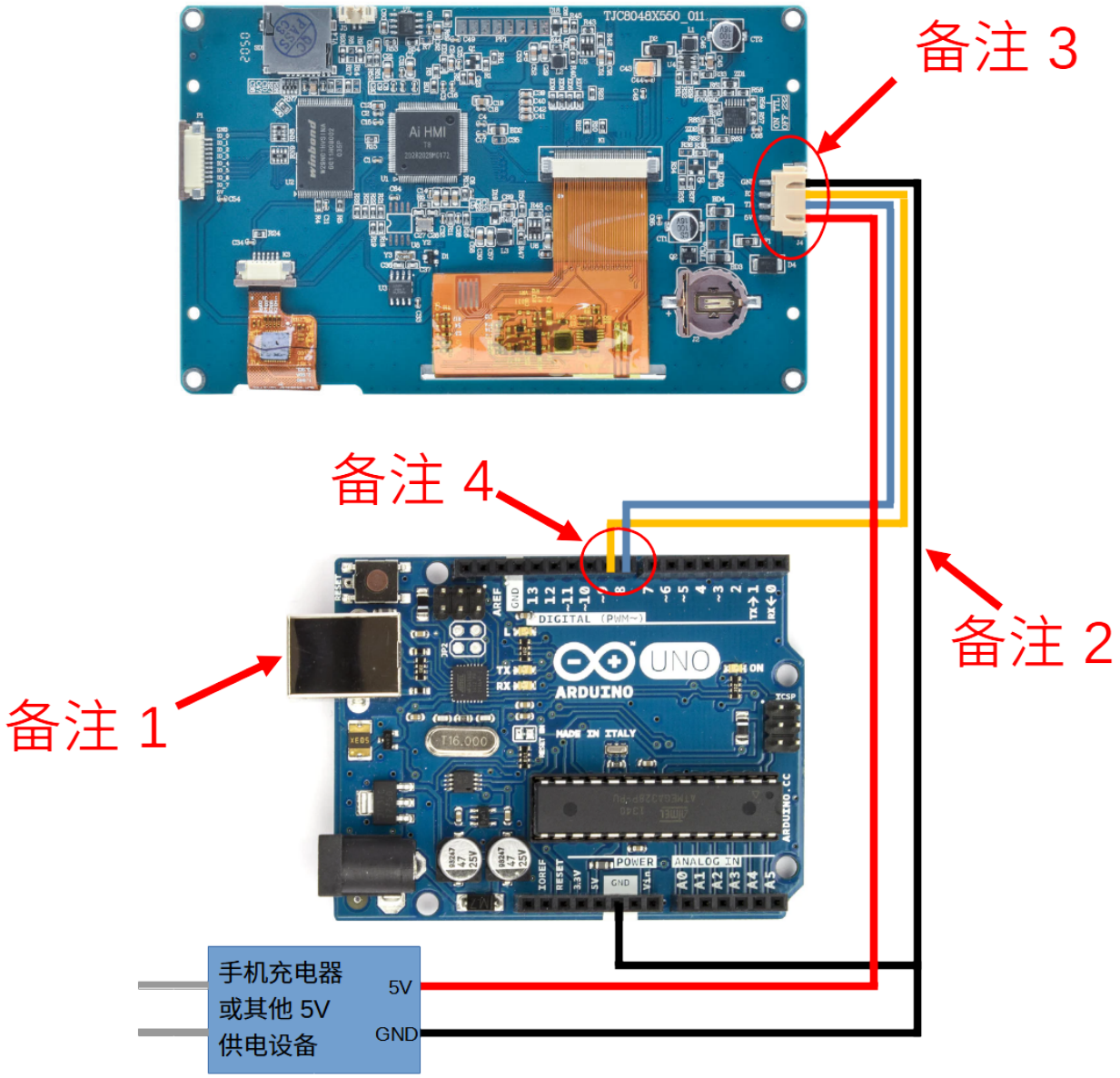
备注 1： 给 arduino 下载程序和供电还是通过这个接口

备注 2： 两边的 GND 要接在一起 (共地)，才能正常通讯

备注 3： 如果要通过 usb 转 ttl 给屏幕下载程序，要断开和 arduino 的连接，将屏幕接口和 usb 转 ttl 接在一起才能下载

备注 4： 因为接的是 TX3 和 RX3，所以请使用 Serial3 进行通讯，arduino 的 TX3 接串口屏的 RX， arduino 的 RX3 接串口屏的 TX

2.4.3.3 串口屏连接 arduino mega uno



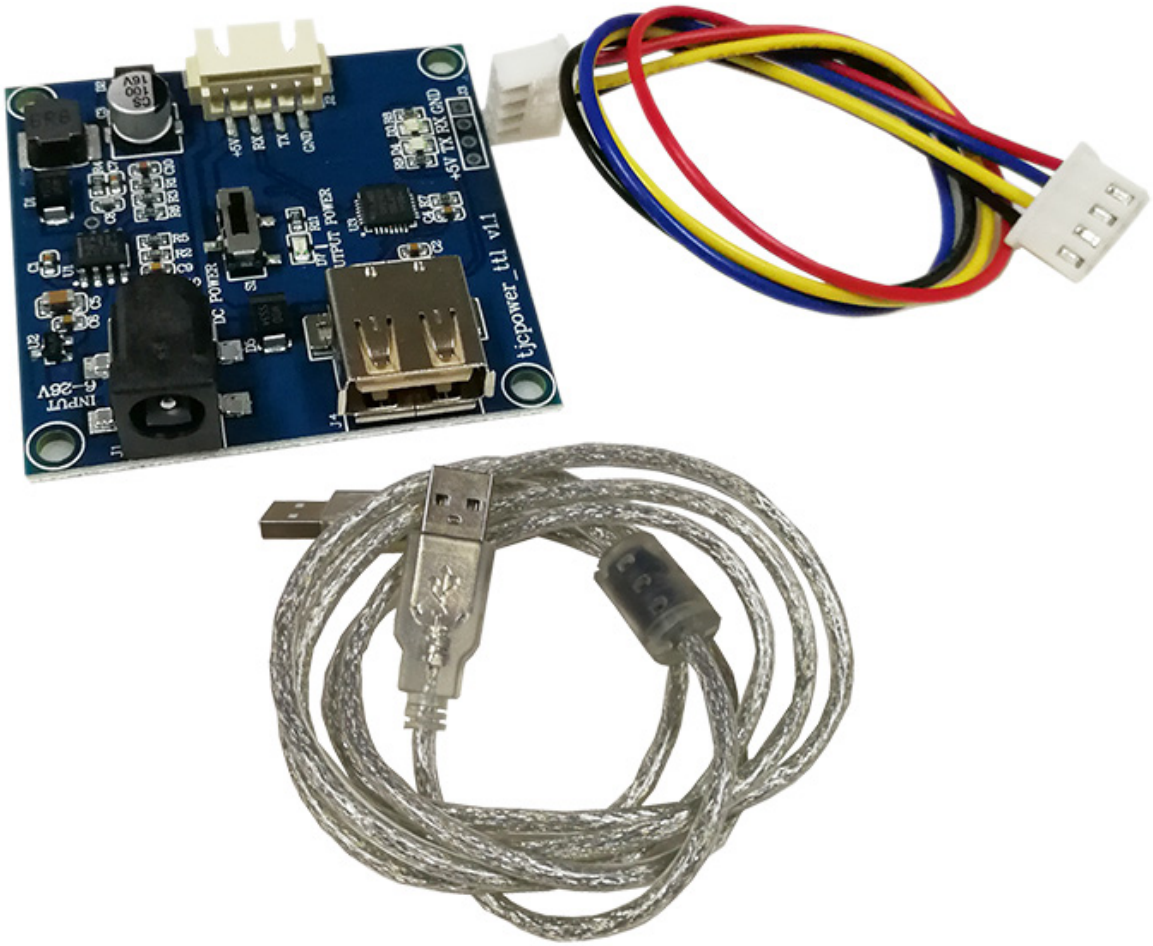
备注 1： 给 arduino 下载程序和供电还是通过这个接口

备注 2： 两边的 GND 要接在一起 (共地)，才能正常通讯

备注 3： 如果要通过 usb 转 ttl 给屏幕下载程序，要断开和 arduino 的连接，将屏幕接口和 usb 转 ttl 接在一起才能下载

备注 4： 因为接的是软串口，所以请使用 SoftwareSerial 进行通讯，arduino 的 TX 接串口屏的 RX， arduino 的 RX 接串口屏的 TX

2.4.3.4 串口屏与单片机连接说明 1



这是淘晶驰官方的 USB 转 TTL 转接板，其作用有以下两种：

1、电脑通过 USB 转 TTL 连接串口屏进行下载或者调试

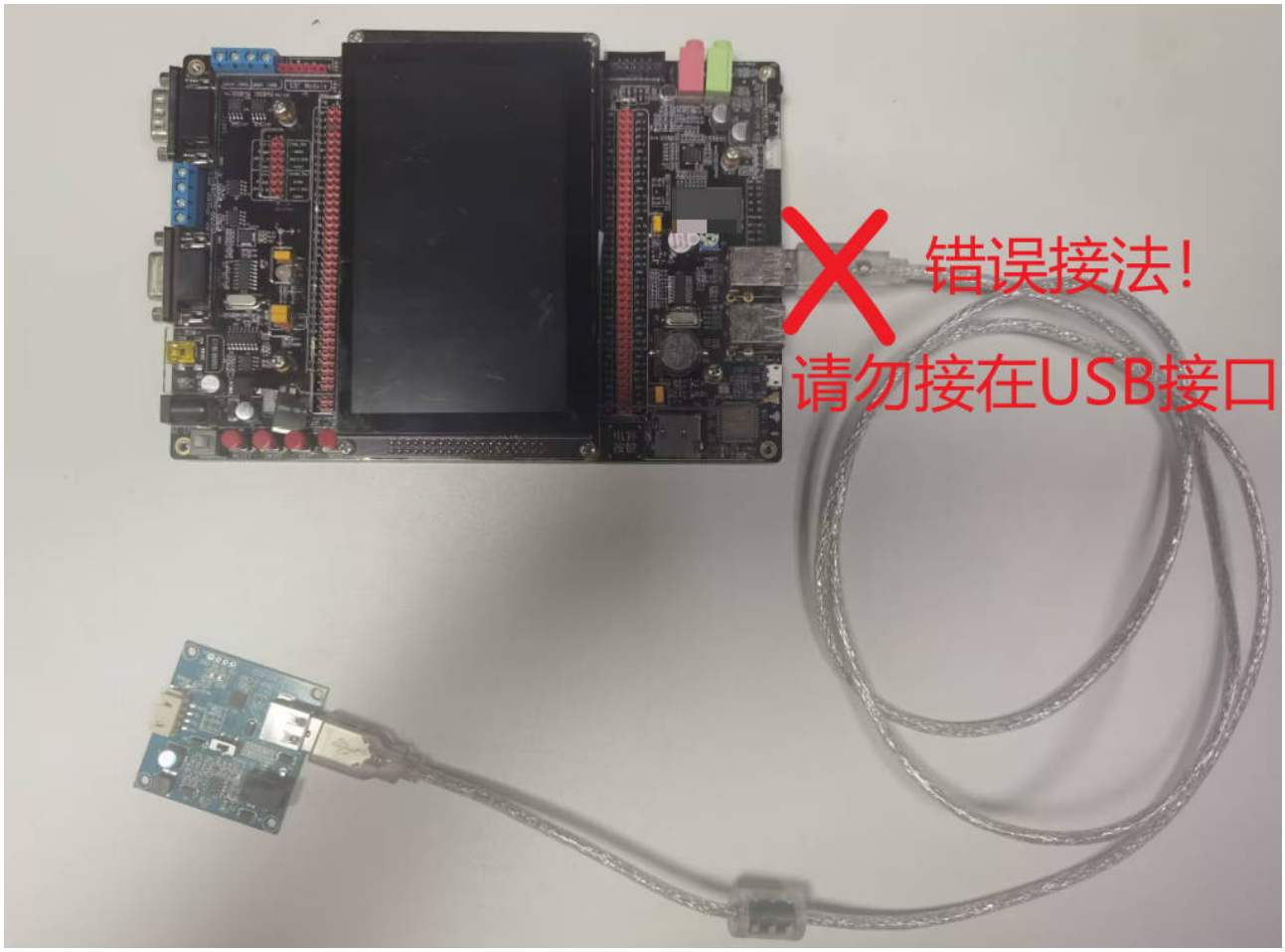
2、电脑通过 USB 转 TTL 连接单片机进行通讯

如果不使用电脑时，请勿通过 USB 转 TTL 工具连接单片机和串口屏，怎么接都不行！

部分客户使用的是正点原子、野火等公司的开发板，看到板子上有 USB 口，就直接把 USB 转 TTL 工具插上去。

或者是开发板有 microUSB 接口或者 typec 接口，把自己的手机充电线插进去，这是万万不行的。

因为这时候你的单片机开发板是接入了一个 USB 设备而不是 USART 设备，除非你去驱动这个 USB 外设先与这个 USB 转 TTL 进行通讯，否则是不能跟屏幕进行通讯的。



正确接法如下:

请查看你的开发板的原理图，找到一个没有连接任何外设的 USART 串口来连接串口屏，一般上面都是标注（USARTn_TXD,USARTn_RXD），大部分开发板的第一个串口 USART0_TXD,USART0_RXD 大概率已经连接了一个 USB 转 TTL 芯片用于连接电脑，请避开这个串口。

备注： 连接方式如下所示

5V 接串口屏 5V

TX 接串口屏 RX

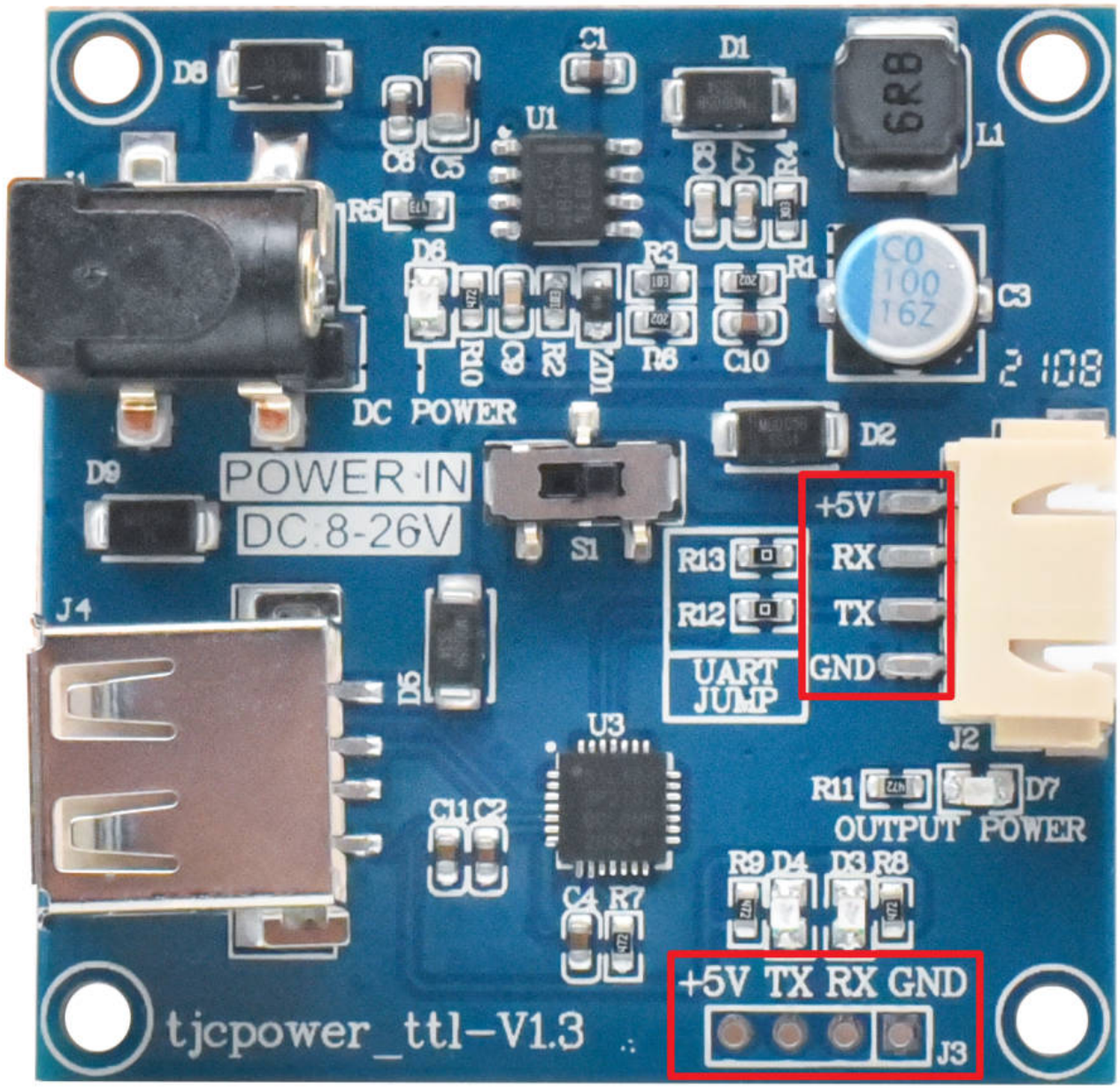
RX 接串口屏 TX

GND 接串口屏 GND

TX 是 Transmit（发送），RX 是 Receive（接收），发送对接收，接收对发送。

另外说明：有些板子和原理图标注的是 TX 和 RX，有些板子和原理图标注的是 TXD 和 RXD。TX 就是 TXD，RX 就是 RXD。概念都是一样的，可以不做区分。

2.4.3.5 串口屏与单片机连接说明 2



上图中圈出来的两个地方，丝印对应的引脚是连通的

也就是说

右侧的 +5V 和底部的 +5V 是连通的

右侧的 TX 和底部的 TX 是连通的

右侧的 RX 和底部的 RX 是连通的

右侧的 GND 和底部的 GND 是连通的

有的客户右侧接口连接串口屏，底部接口用于连接单片机，这样是不行的，因为这个板子是有 USB 转 TTL 芯片的，会造成串口一对多，影响到串口的通讯。

任何情况下请确保单片机的这个 USART 串口上面只有串口屏，没有其他外设。

2.4.3.6 串口屏与单片机连接说明 3

当需要额外供电时，可以另外接 5V 和 GND 到屏幕上给屏供电，但是要将串口屏的 GND 和单片机的 GND 接到一起（也就是共地），单片机的 +5V 供电可以不和屏幕接在一起

然后将单片机的 TX 连接到串口屏的 RX

然后将单片机的 RX 连接到串口屏的 TX

任何情况下请确保单片机的这个 USART 串口上面只有串口屏，没有其他外设。

2.4.3.7 串口屏与单片机连接说明 4

如果不能确认串口屏是否可以接收到单片机的数据，可以把这个工程烧录到串口屏中试试，简易串口助手样例工程 (page 1122)

某些 3.3V 的单片机，可能会无法接收到 X3、X5 系列串口屏的数据，但是串口屏可以接收到单片机的数据

因为 X3、X5 系列串口屏有其他电路用于切换 TTL/232 电平，他们的输出的串口电平是 5V 的，部分 3.3V 的单片机接收到 5V 电压可能无法正确识别

这个时候可以尝试在串口屏的 TX 和单片机的 RX 中间串 1 个 1K 电阻以降低串口电压，让单片机正确识别串口数据。

2.4.3.8 串口屏与单片机连接说明 5（串口解析）

以下为单片机的串口接收程序说明

1、单片机上建议划分 1KB 的 ram 来创建一个串口环形缓冲区。

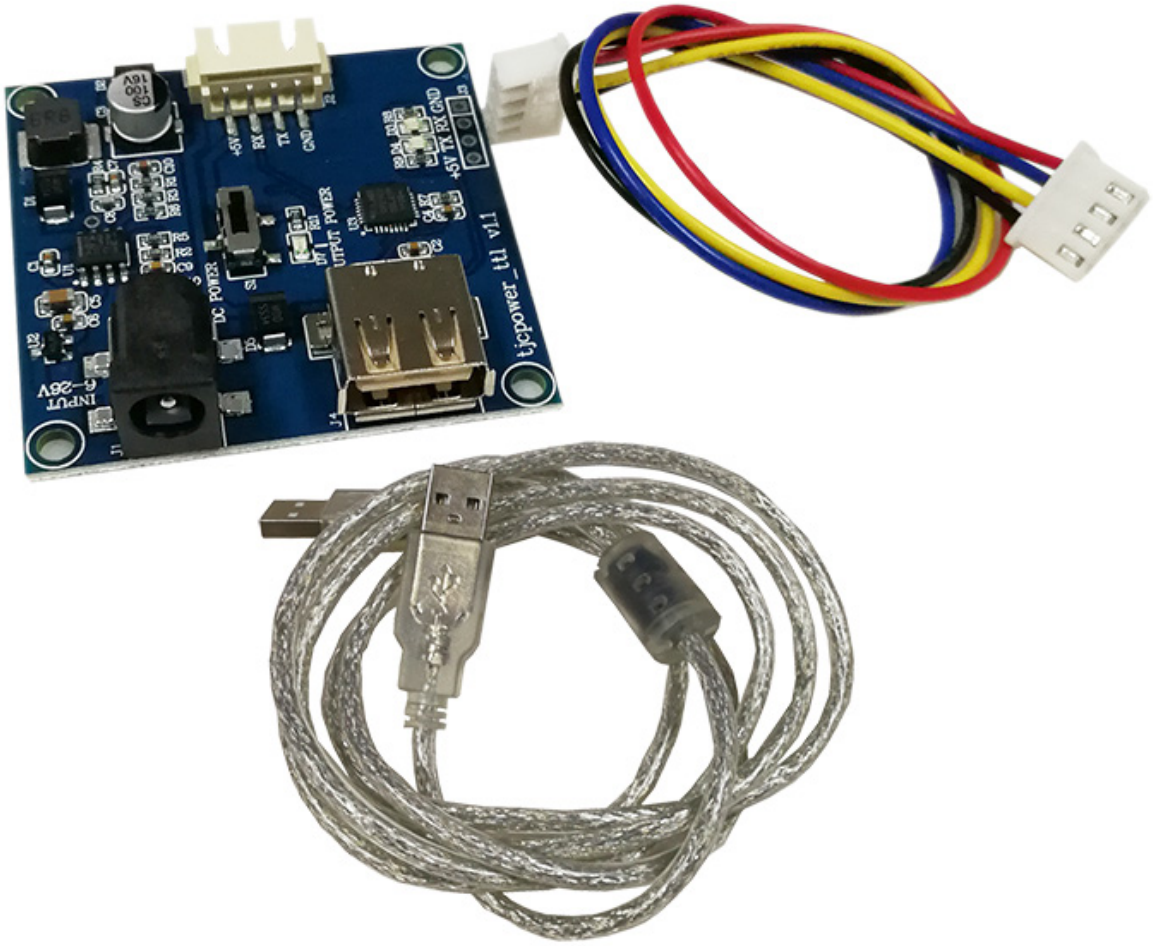
2、推荐使用串口中断模式，在中断事件中将接收到的数据放入串口缓冲区，然后清空中断标志位，退出中断。

3、在 main 循环中解析串口缓冲区中的数据。

任意两个设备之间连接，均是 TX 接对方的 RX，RX 接对方的 TX，GND 接对方的 GND，如果双方都有独立的供电，可以不接 5V 线

需要使用到串口工具，推荐使用 cp2102 或者 ft232，不推荐使用 ch340 或者 pl2303

推荐使用官方的串口下载套件

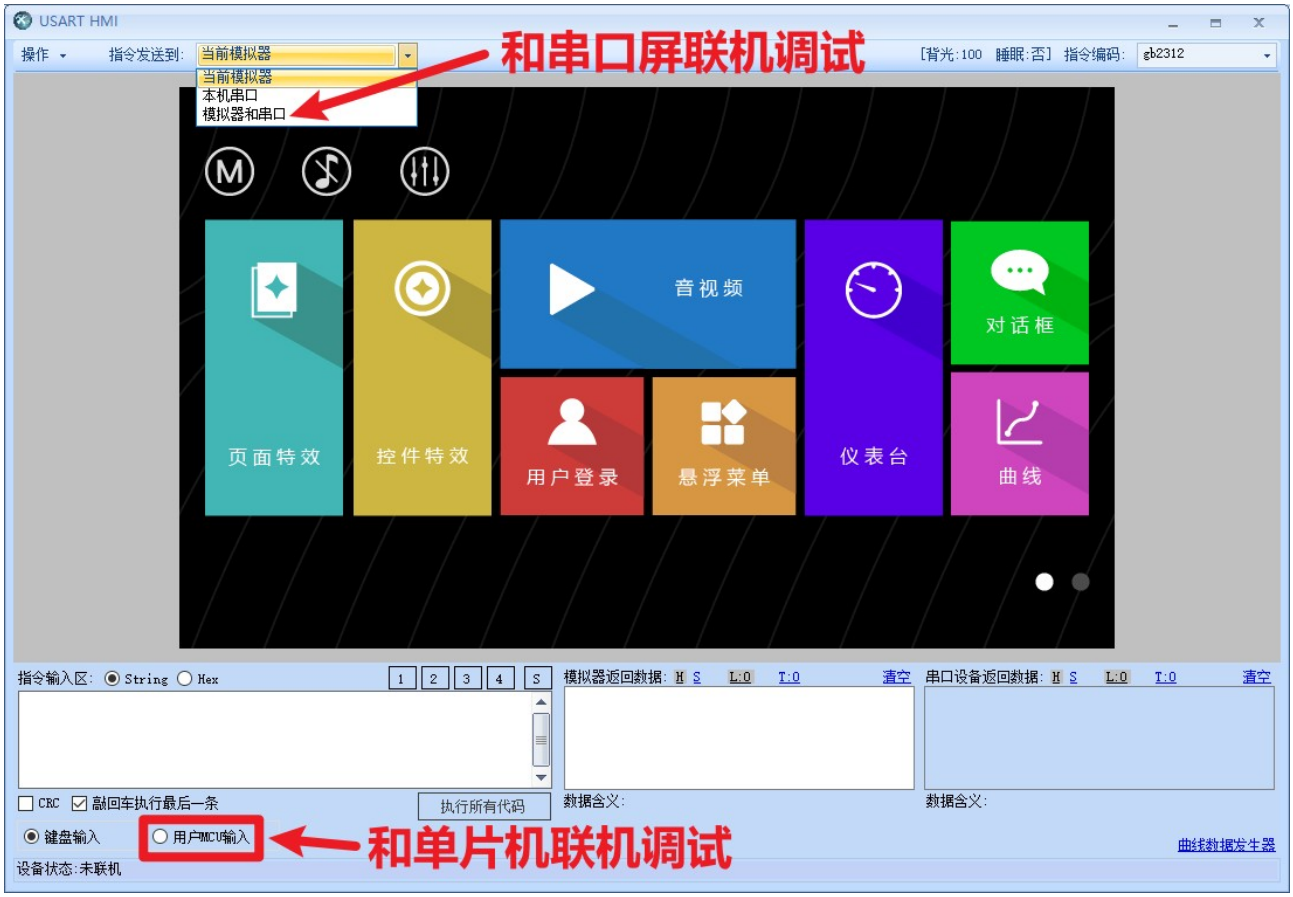


连接期间常见的问题有：

屏幕通电后不断的闪烁(供电不足导致不断重启) (page 1244)

可能会找不到串口驱动，请参考安装串口驱动 (page 138)

模拟器可以和一个串口屏实物连接，也可以和单片机连接，在同一时间二者只能其中选择一个


USART HMI
操作
指令发送到:
当前模拟器
和串口屏联机调试
[背光:100 睡眠:否] 指令编码:
gb2312
当前模拟器
本机串口
模拟器和串口
音视频
对话框
页面特效
控件特效
用户登录
悬浮菜单
仪表台
曲线
指令输入区:
String
Hex
1
2
3
4
S
模拟器返回数据:
H
S
L:0
T:0
清空
串口设备返回数据:
CRC
敲回车执行最后一条
执行所有代码
数据含义:
键盘输入
用户MCU输入
和单片机联机调试
曲线数据发生器
设备状态:未联机

2.5 串口屏通讯协议

2.5.1 单片机发送数据给串口屏

提示： 以下展示的单片机代码均为单片机已经配置了 printf 重定向到串口（即通过 printf 打印的数据就被发送到串口）的情况下使用的。

2.5.1.1 使用淘晶驰协议控制串口屏

例如 main 页面有一个文本控件 t0，我们要让 t0 显示“淘晶驰”，则通过单片机发送 main.t0.txt=”淘晶驰”，然后发送 3 个 0xff 作为结束符，注意，结束符是 16 进制的 0xff

```
printf("main.t0.txt=\"淘 晶 驰\"\xff\xff\xff")
```

使用单片机发出的数据如下所示

```
6D 61 69 6E 2E 74 30 2E 74 78 74 3D 22 CC D4 BE A7 B3 DB 22 FF FF FF
m  a  i  n  .  t  0  .  t  x  t  =  "  淘    晶    驰    "
```

如果是 setting 页面有一个数字控件 n0，我们要让 n0 显示为 100，则通过单片机发送

```
printf("setting.n0.val=100\xff\xff\xff")
```

使用单片机发出的数据如下所示

```
73 65 74 74 69 6E 67 2E 6E 30 2E 76 61 6C 3D 31 30 30 FF FF FF
s  e  t  t  i  n  g  .  n  0  .  v  a  l  =  1  0  0
```

提示： 建议单片机仅操作串口屏的全局变量，例如 main.t0.txt=”淘晶驰”，而不是仅仅使用 t0.txt=”淘晶驰”，这将会减少很多问题。如何将控件设置为全局请参考跨页面赋值，全局变量操作 (page 847)

如果是 program.s 中使用 int 定义的整形，例如 sys0，我们要让 sys0 显示为 123，则通过单片机发送

提示： program.s 中使用 int 定义的整形均为全局变量，目前仅能定义 int 类型，无法定义其他类型（如字符串，浮点数等），int 定义需放在其他代码之前，否则会报错。

```
printf("sys0=123\xff\xff\xff")
```

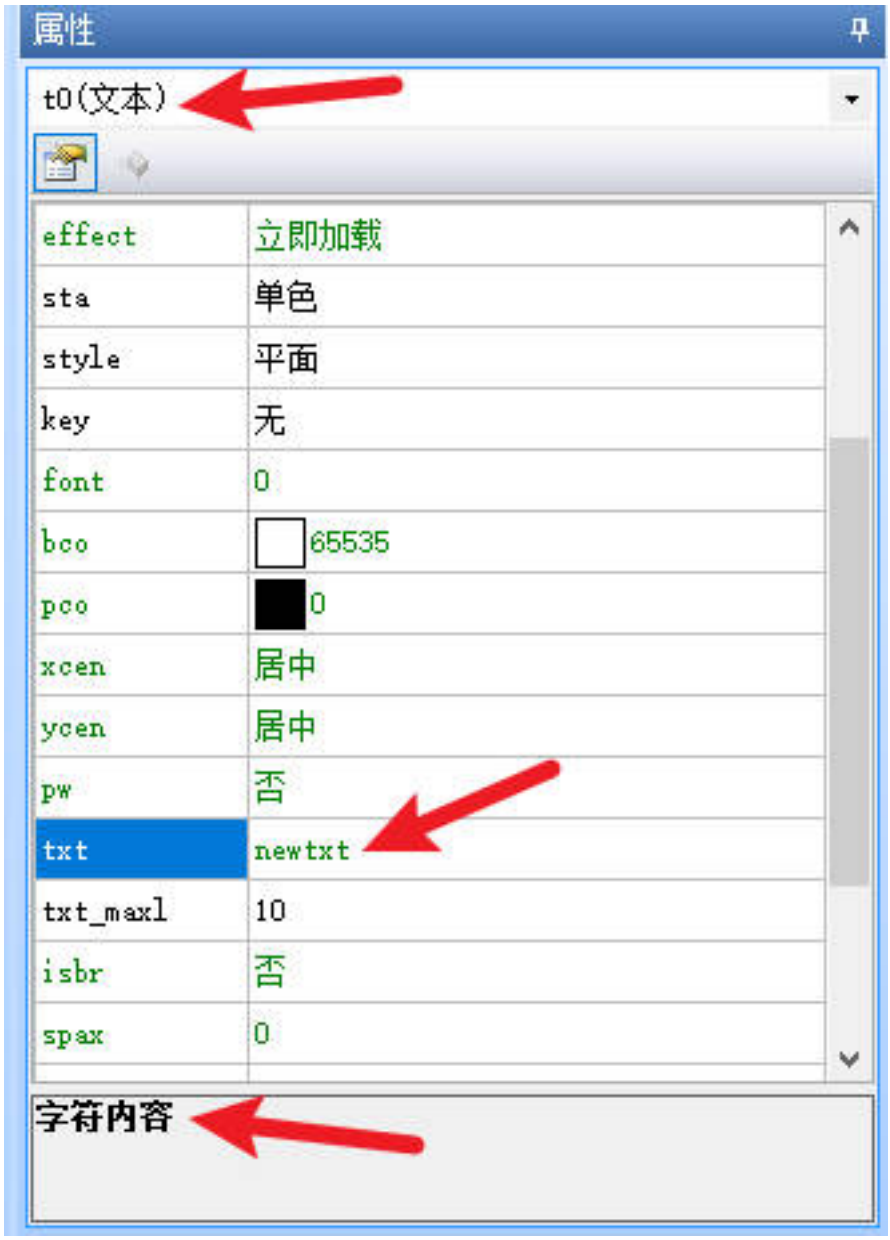
使用单片机发出的数据如下所示

```
H:73 79 73 30 3D 31 32 33 FF FF FF
S:s  y  s  0  =  1  2  3
```

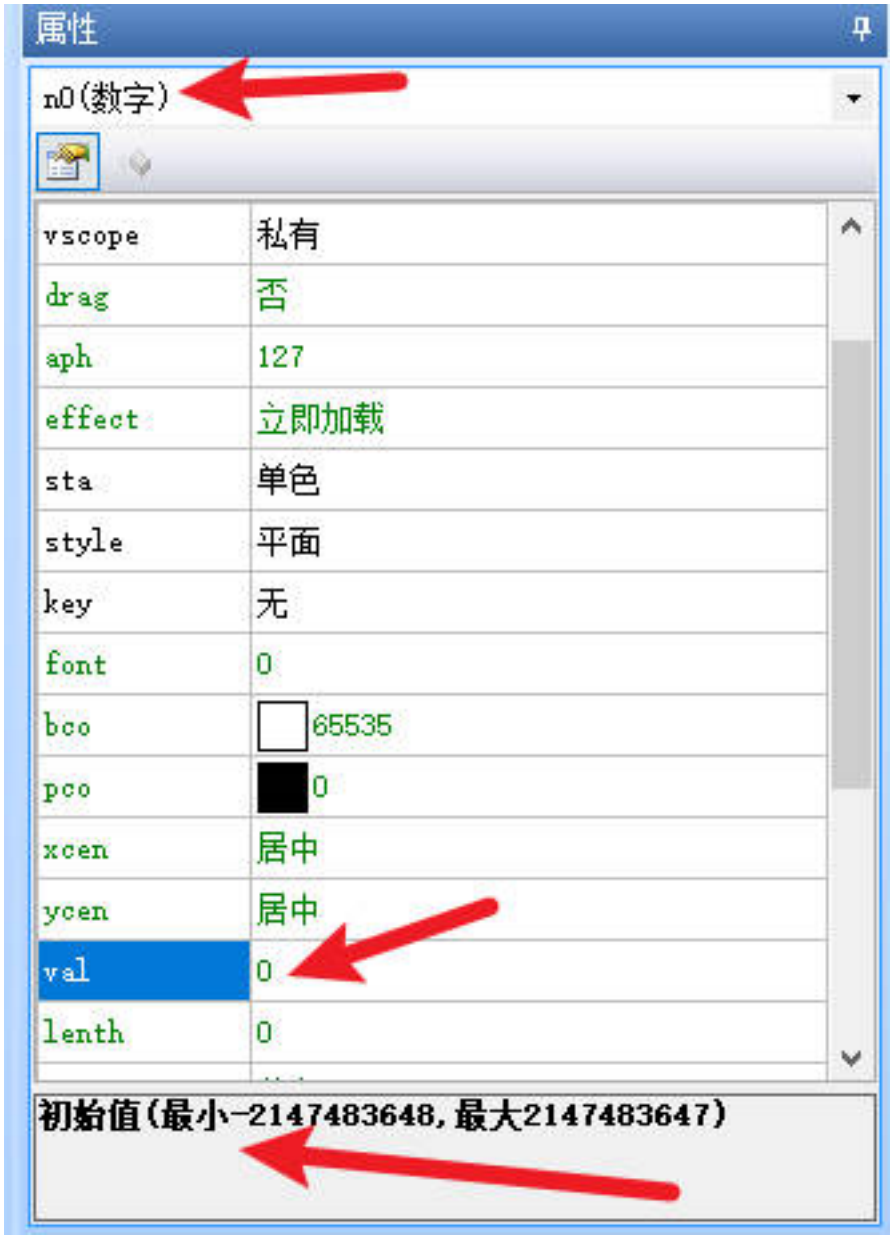
跳转页面时，例如跳转到 main 页面，发送的是 page main，单片机发出的数据如下

```
H:70 61 67 65 20 6D 61 69 6E FF FF FF
S:p  a  g  e     m  a  i  n
```

可以在上位机选中控件后查看控件的属性，选中每个属性后都能在底部看到对应的注释，例如 txt 属性，代表“字符内容”

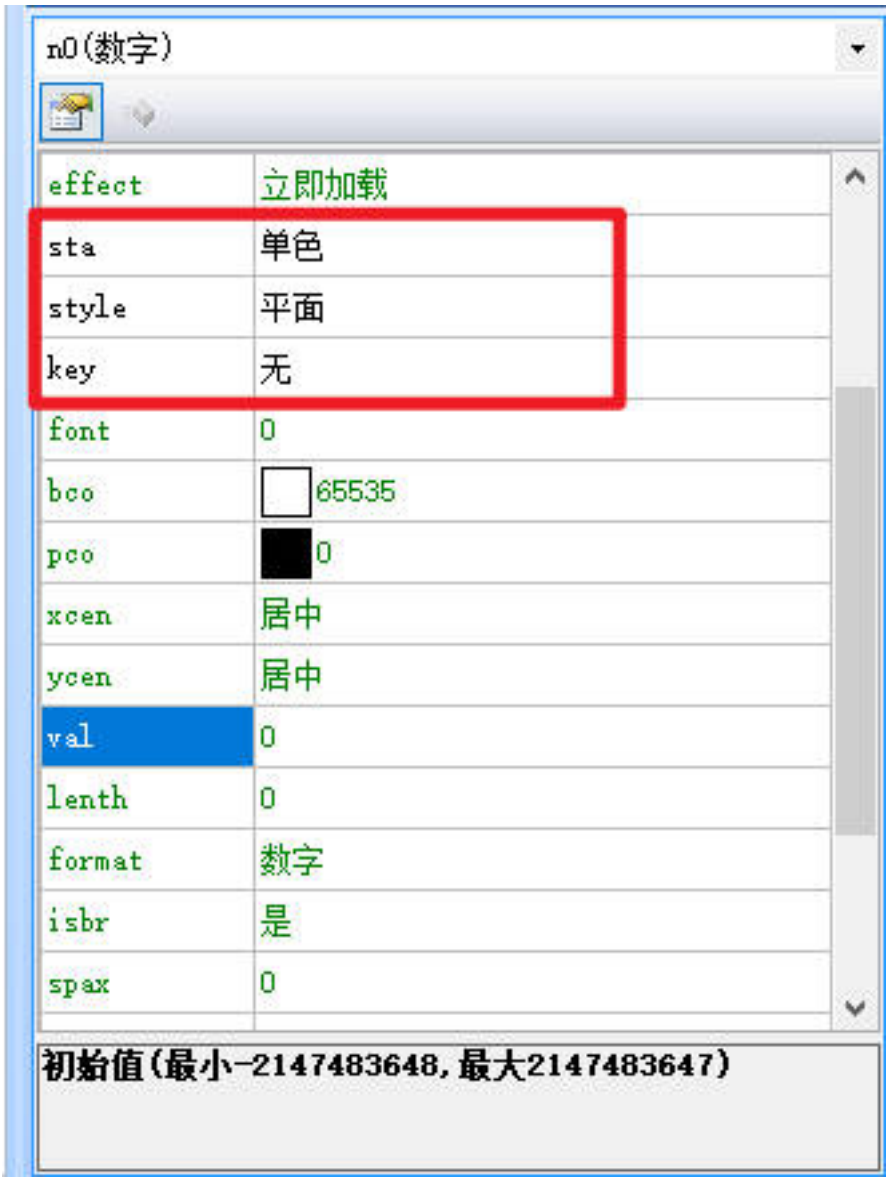


对于数字控件来说，val 属性就代表了数字控件将会显示的数值



并不是每个属性都能通过指令进行修改，只有绿色的属性是可以通过指令修改的，黑色属性是不允许通过指令进行修改的

黑色属性是不可修改或者只能通过上位机进行修改的



objname（控件名称）是比较特殊的属性，不允许通过指令读写，只能在上位机里更改。

2.5.1.2 单片机使用其他协议控制串口屏

请参考主动解析模式应用详解 (page 943)

2.5.2 串口屏发送数据给单片机

2.5.2.1 定义发送的数据格式

淘晶驰没有定义固定的串口屏发送给单片机的数据格式，您需要自己定义格式
其原理相当于在单片机上实现串口屏上的主动解析
我们需要先定义一个符合我们需求的通讯协议
常见的通讯协议的格式如下
帧头 + 帧长度 + 帧内容 + 帧校验 + 帧尾
上面的格式中有些事可以被省略掉的，例如淘晶驰通讯协议是帧内容 + 帧尾（3 个 0xff）
具体需要哪些格式，不需要哪些格式，取决于项目的具体需求
假设使用串口屏控制 4 个 led 灯，串口屏需要下发的参数为灯的状态（开或关）
通讯协议举例（帧头 + 帧内容 + 帧尾）:
当我们做汇报时，要先喊一声“报告!”，提醒其他人我们要开始说话了，这个相当于帧头
接下来我们开始说话，这个相当于帧内容
说完后，我们说一句“完毕!”，提醒大家我们说完了，这个相当于帧尾
这就是我们定义的一个通讯协议的结构，即“帧头 + 帧内容 + 帧尾”

2.5.2.2 下发数值格式的数据

我们以此结构来传输 led 数据
备注:4 个 led 用双态按钮来实现,led 关闭时, 对应的 val 属性是 0，led 开启时, 对应的 val 属性是 1。
帧头为 55
帧尾为 ff ff ff

```
//发出的数据如下
55 01 00 ff ff ff  //含义：1号led关闭
55 04 01 ff ff ff  //含义：4号led打开
55 01 01 ff ff ff  //含义：1号led打开
55 04 00 ff ff ff  //含义：4号led关闭
```

代码示例:

```
//发送1号灯状态
printh 55 01          //发送帧头和灯的编号
prints led1.val,1     //发送led1的数据,长度1字节
printh ff ff ff       //发送帧尾

//发送2号灯状态
printh 55 02          //发送帧头和灯的编号
prints led2.val,1     //发送led2的数据,长度1字节
printh ff ff ff       //发送帧尾

//发送3号灯状态
printh 55 03          //发送帧头和灯的编号
prints led3.val,1     //发送led3的数据,长度1字节
printh ff ff ff       //发送帧尾

//发送4号灯状态
printh 55 04          //发送帧头和灯的编号
```

(续下页)

(接上页)

```
prints led4.val,1    //发送led4的数据,长度1字节
printh ff ff ff      //发送帧尾
```

也可以一次性下发 4 个灯的状态

```
55 01 01 01 01 ff ff ff  //含义：4个灯状态分别为：开、开、开、开
55 00 00 00 00 ff ff ff  //含义：4个灯状态分别为：关、关、关、关
55 00 01 00 01 ff ff ff  //含义：4个灯状态分别为：关、开、关、开
55 01 00 01 00 ff ff ff  //含义：4个灯状态分别为：开、关、开、关
```

代码示例：

```
//一次性下发4个灯的状态
printh 55            //发送帧头
prints led1.val,1    //发送led1的数据,长度1字节
prints led2.val,1    //发送led2的数据,长度1字节
prints led3.val,1    //发送led3的数据,长度1字节
prints led4.val,1    //发送led4的数据,长度1字节
printh ff ff ff      //发送帧尾
```

这里的帧头或者帧尾你可以改为其他你想要的，例如帧头改为 0x33，帧尾用 0d 0a（即回车换行）取代 3 个 0xff

```
33 01 01 01 01 0d 0a  //含义：4个灯状态分别为：开、开、开、开
```

代码示例：

```
//一次性下发4个灯的状态
printh 33            //发送帧头
prints led1.val,1    //发送led1的数据,长度1字节
prints led2.val,1    //发送led2的数据,长度1字节
prints led3.val,1    //发送led3的数据,长度1字节
prints led4.val,1    //发送led4的数据,长度1字节
printh 0d 0a         //发送帧尾
```

也可以去掉帧头或者帧尾

1、去掉帧头

```
01 01 01 01 0d 0a  //含义：4个灯状态分别为：开、开、开、开
```

代码示例：

```
//一次性下发4个灯的状态
prints led1.val,1    //发送led1的数据,长度1字节
prints led2.val,1    //发送led2的数据,长度1字节
prints led3.val,1    //发送led3的数据,长度1字节
prints led4.val,1    //发送led4的数据,长度1字节
printh 0d 0a         //发送帧尾
```

2、去掉帧尾

```
33 01 01 01 01  //含义：4个灯状态分别为：开、开、开、开
```

代码示例：

```
//一次性下发4个灯的状态
printh 33            //发送帧头
prints led1.val,1    //发送led1的数据,长度1字节
prints led2.val,1    //发送led2的数据,长度1字节
prints led3.val,1    //发送led3的数据,长度1字节
prints led4.val,1    //发送led4的数据,长度1字节
```

3、因为每次都是固定下发 4 个数据，因此连帧头帧尾都不要其实问题也不大，但是存在一定的解析错误的风险

```
01 01 01 01  //含义：4个灯状态分别为：开、开、开、开
```

代码示例：

```
//一次性下发4个灯的状态
prints led1.val,1     //发送led1的数据,长度1字节
prints led2.val,1     //发送led2的数据,长度1字节
prints led3.val,1     //发送led3的数据,长度1字节
prints led4.val,1     //发送led4的数据,长度1字节
```

4、如果需要校验，可以使用 modbus_crc16

```
33 01 01 01 01 B0 68   //含义：4个灯状态分别为：开、开、开、开，modbus_crc16校验值为 B0␣
↪68
33 00 00 00 00 20 04   //含义：4个灯状态分别为：关、关、关、关，modbus_crc16校验值为 20␣
↪04
33 00 01 00 01 B0 04   //含义：4个灯状态分别为：关、开、关、开，modbus_crc16校验值为 B0␣
↪04
33 01 00 01 00 20 68   //含义：4个灯状态分别为：开、关、开、关，modbus_crc16校验值为 20␣
↪68
```

代码示例：

```
crcrest 1,0xffff      //复位crc
printh 33             //发送帧头
crcputh 33            //放入帧头
prints led1.val,1     //发送led1的数据,长度1字节
crcputs led1.val,1    //放入led1的数据,长度1字节
prints led2.val,1     //发送led2的数据,长度1字节
crcputs led2.val,1    //放入led2的数据,长度1字节
prints led3.val,1     //发送led3的数据,长度1字节
crcputs led3.val,1    //放入led3的数据,长度1字节
prints led4.val,1     //发送led4的数据,长度1字节
crcputs led4.val,1    //放入led4的数据,长度1字节
prints crcval,2       //发送crc校验结果,长度2字节
```

使用 crc 校验的步骤:

1、复位 crc 校验

2、放入要校验的数据，如果是 printh 就用 crcputh，如果是 prints 就用 crcputs

3、发送校验结果 crcval，长度一定是 2 字节

2.5.2.3 下发字符串格式的数据

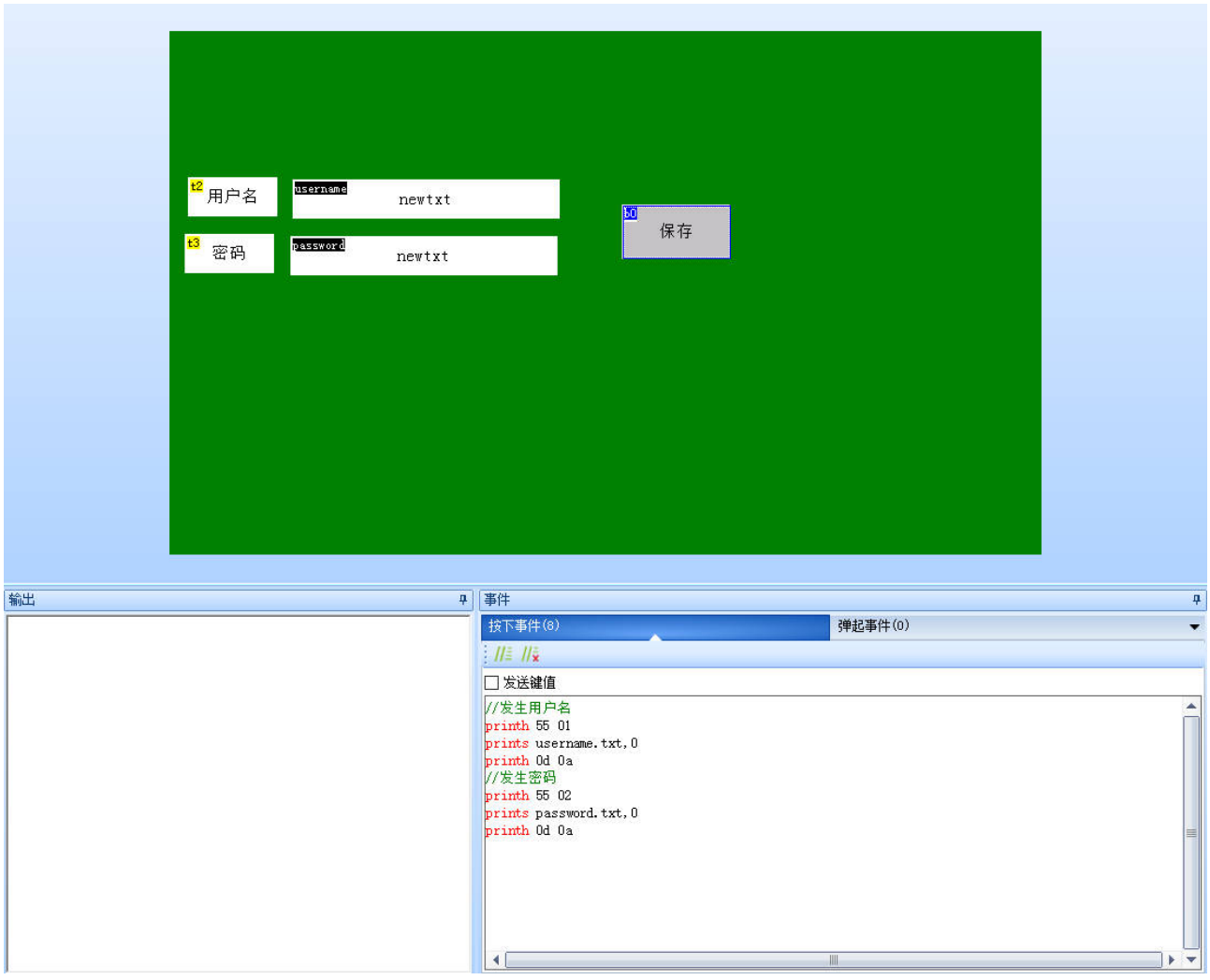
代码示例：

```
//发送用户名 55:帧头，01:代表下发的是用户名
printh 55 01
prints username.txt,0       //发送用户名字符串
printh 0d 0a                //发送帧尾
//发送密码 55:帧头，02:代表下发的是密码
printh 55 02
prints password.txt,0       //发送password字符串
printh 0d 0a                //发送帧尾
```

对于字符串这种长度不固定的数据，建议发送字符串长度方便单片机进行解析

代码示例：

```
//发送用户名 55:帧头，01:代表下发的是用户名
printh 55 01
btlen username.txt,sys0     //检测字符串长度并放入sys0
prints sys0,1               //下发1字节的字符串长度,可以表示0-255个字节,
↪在大部分场景下都是适用的
prints username.txt,0       //发送username字符串
printh 0d 0a                //发送帧尾
//发送密码 55:帧头，02:代表下发的是密码
printh 55 02
btlen password.txt,sys0     //检测字符串长度并放入sys0
prints sys0,1               //下发1字节的字符串长度,可以表示0-255个字节,
↪在大部分场景下都是适用的
prints password.txt,0       //发送password字符串
printh 0d 0a                //发送帧尾
```



2.5.2.4 《串口屏发送数据给单片机例程》下载

演示工程下载链接：

《led 状态下发》演示工程下载

《串口下发用户名和密码》演示工程下载

2.5.2.5 单片机如何解析串口屏发出的数据

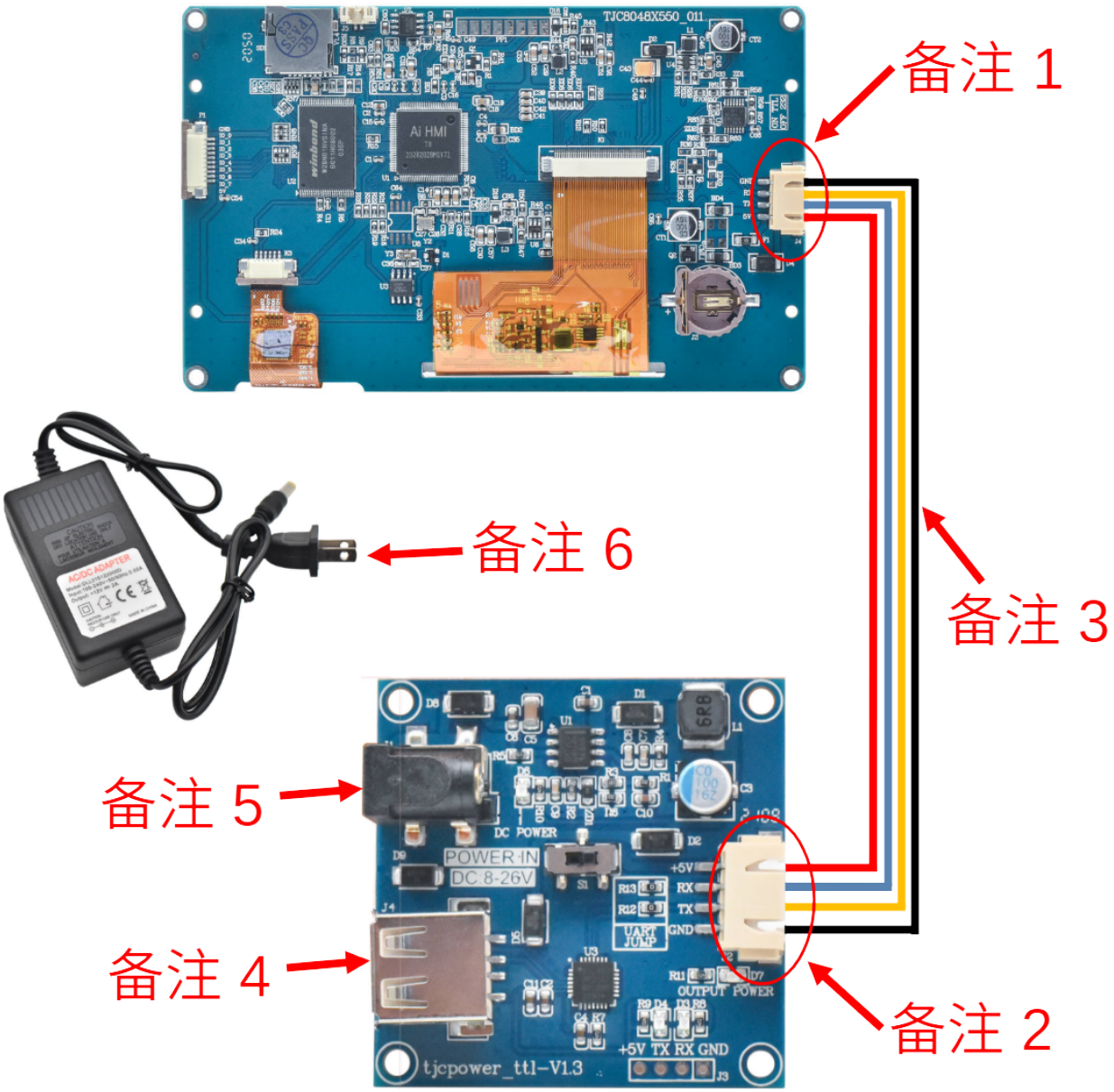
请参考*使用 stm32cubeide(HAL 库) 开发* (page 279)

2.6 串口助手软件 (sscom) 和屏幕联调

串口通讯工具 sscom5.13.1 下载链接：

sscom5.13.1 串口调试助手下载 (page 1095)

连接的接线图如下



备注 1/2： 5V 接 5V， GND 接 GND， USB 转 TTL 的 TX 接串口屏 RX， USB 转 TTL 的 RX 接串口屏 TX

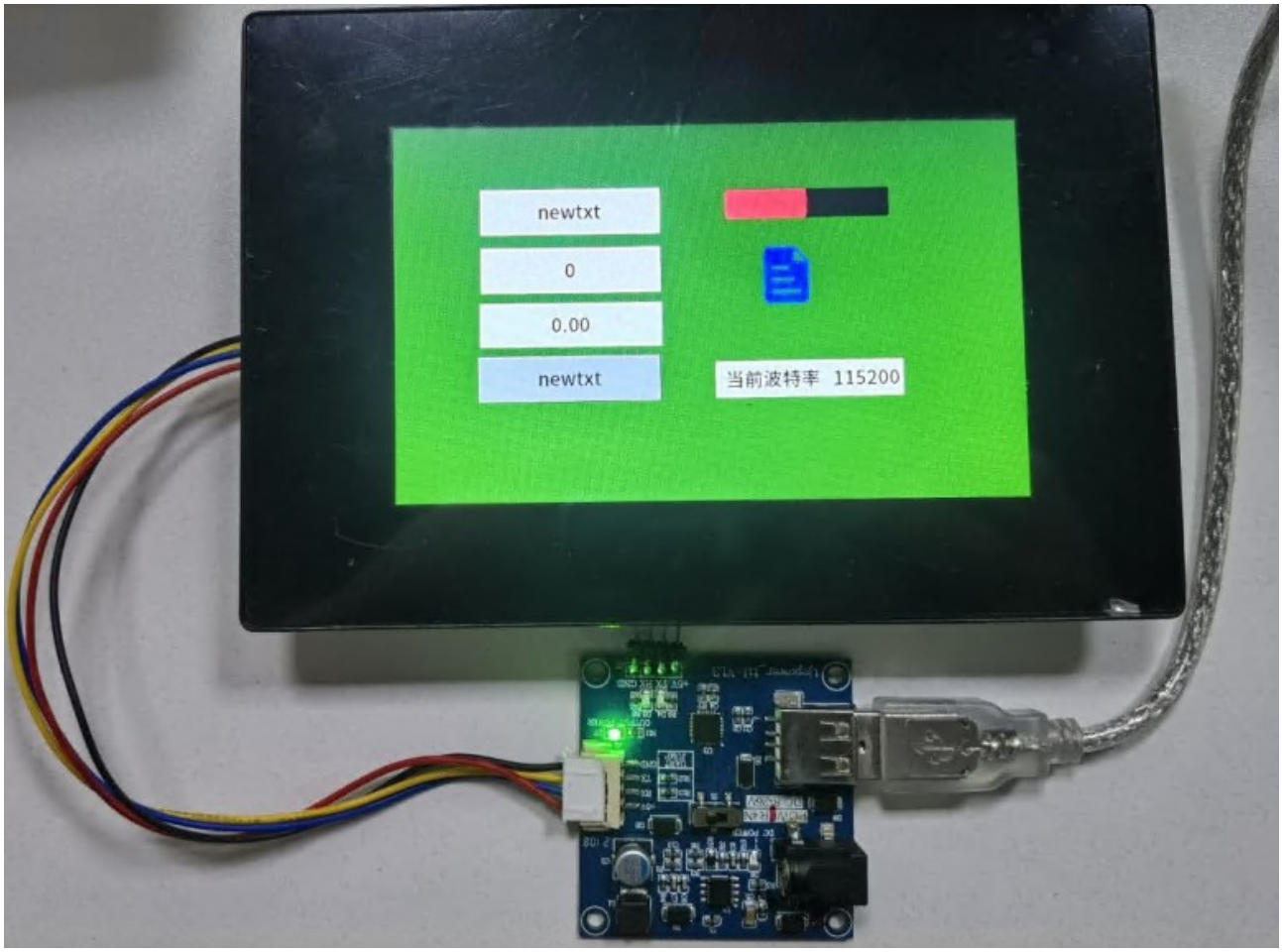
备注 3： 两边的 GND 要接在一起（共地），才能正常通讯

备注 4： 通过 USB 线接电脑

备注 5： 额外供电口，输入电压支持 8V-26V

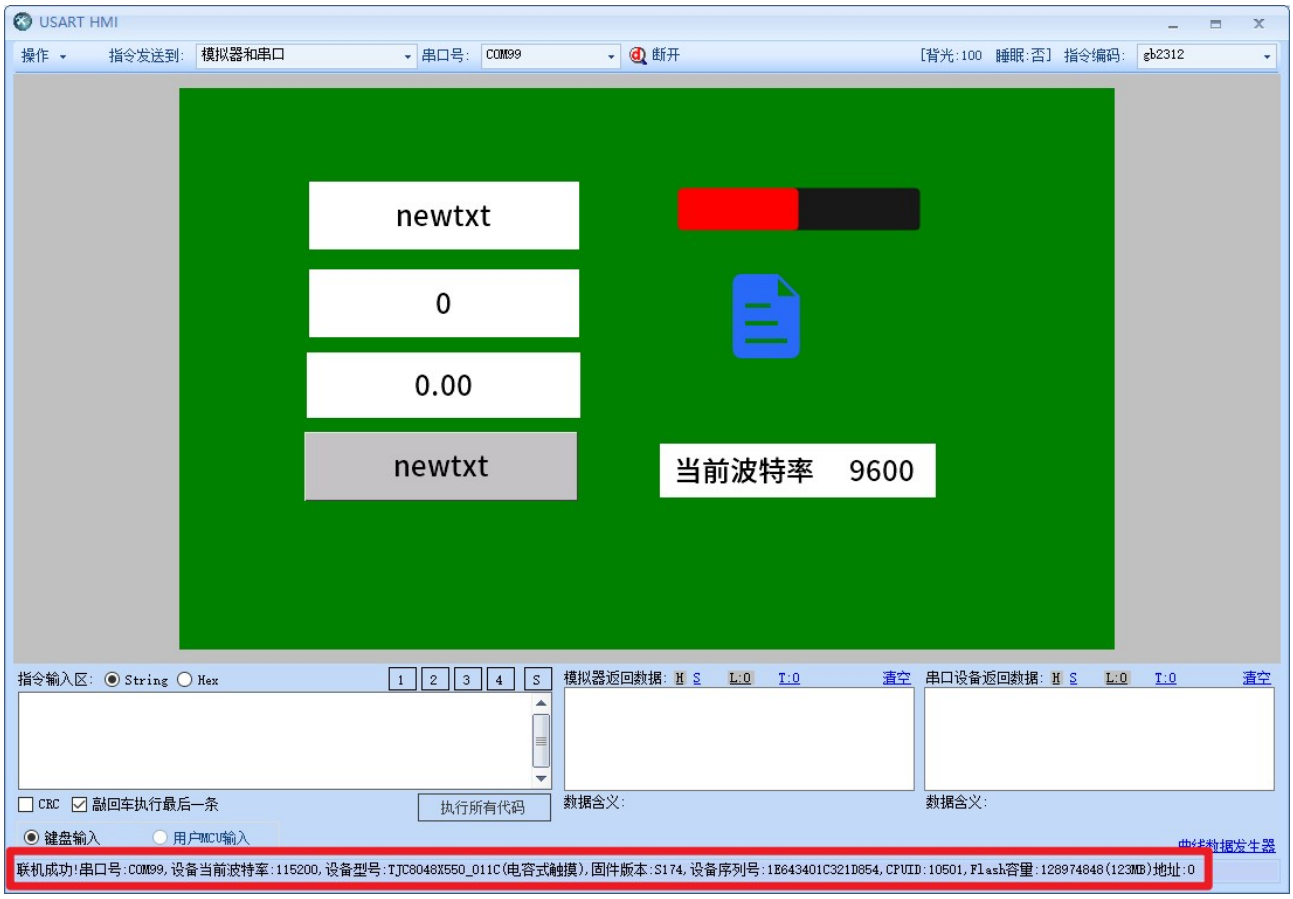
备注 6： 4.3 寸以上的屏幕仅仅使用 USB 线是带不动的，需要使用外部电源供电，否则会无法启动或者不断闪烁重启

使用官方的 USB 转 ttl 连接串口屏如下所示

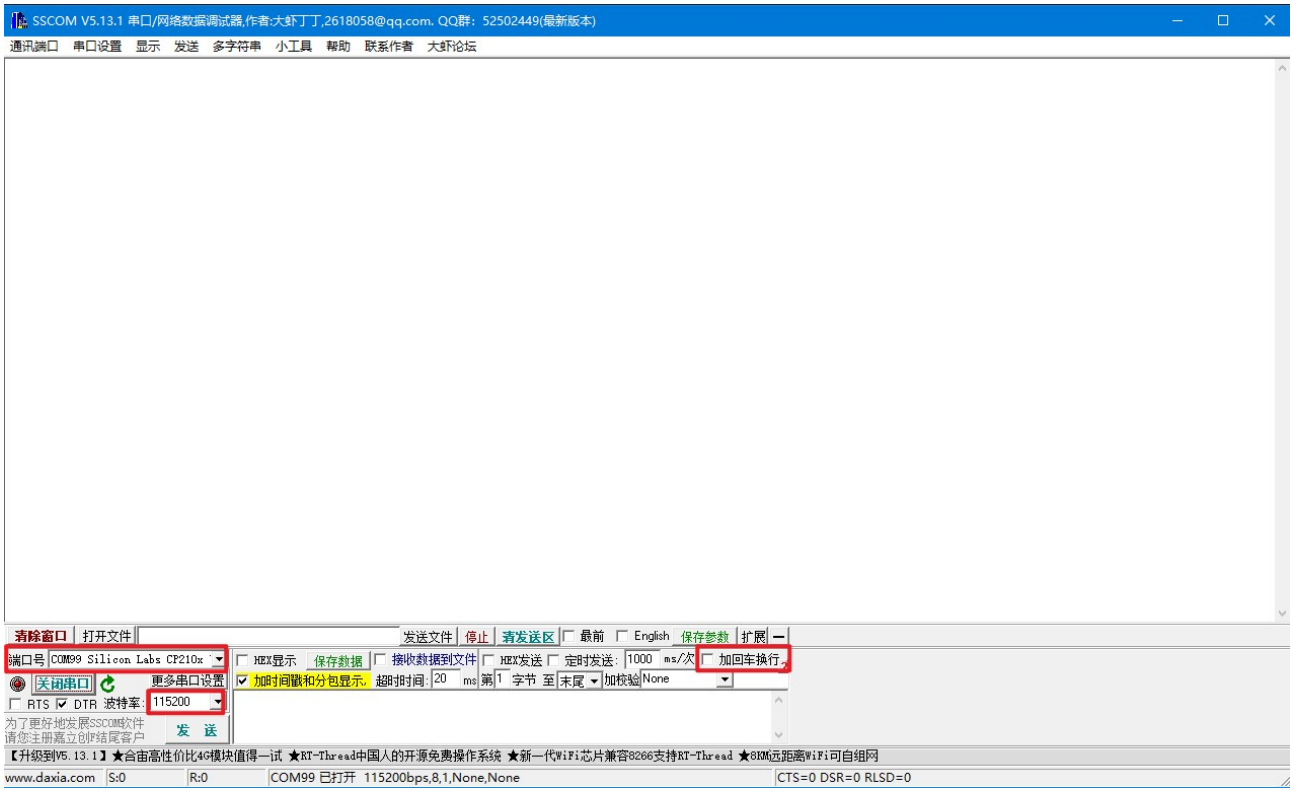


1. 串口屏通过电平转换模块和电脑正确连接，是否连接成功可以先用我们上位联机试一下，这样确保硬件连线没问题。

联机成功后底部会显示联机信息，请记住红圈框起来的串口号和波特率，然后关闭模拟器界面

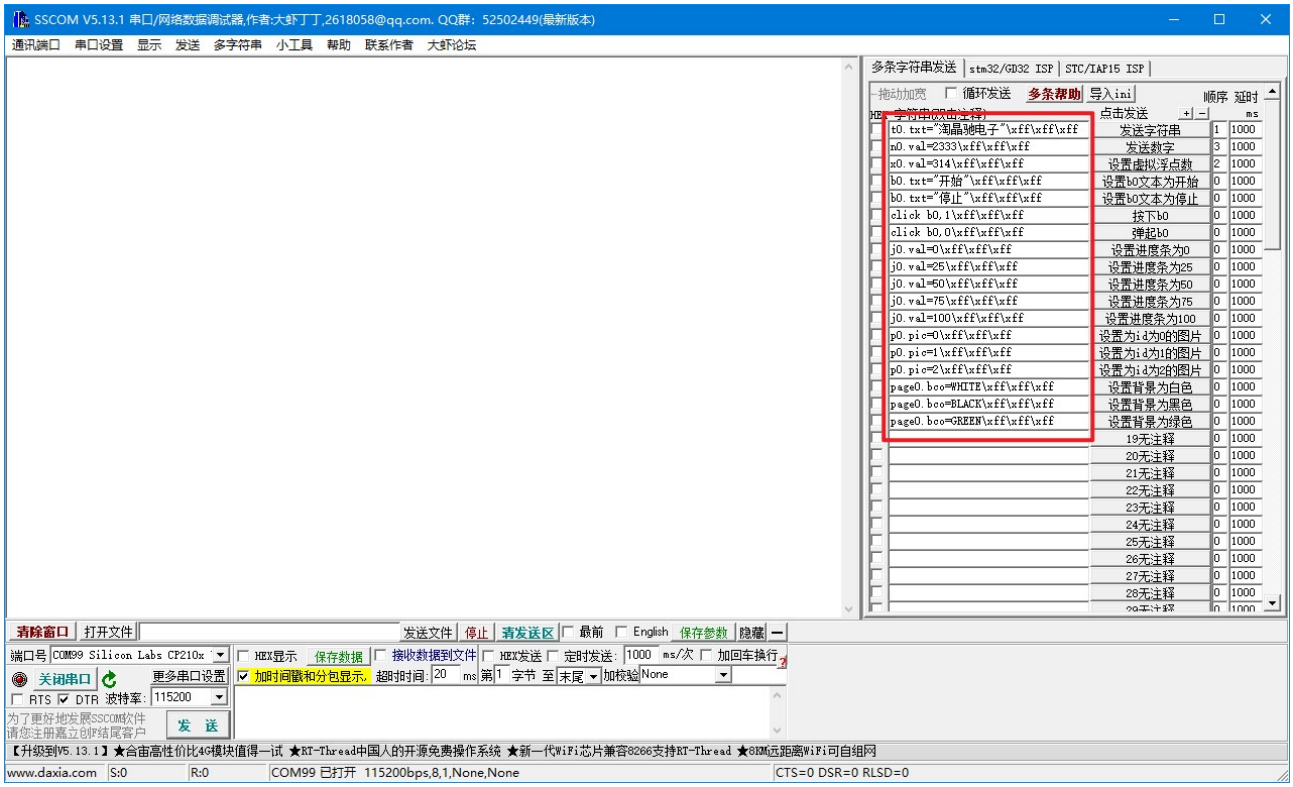


2. 串口助手如下配置，串口助手请使用 SSCOM5.13.1 版本，根据实际情况设置端口号和波特率，不要勾选“加回车换行”



3. 点击拓展

4. 可以看到提前配置好的一些命令



5. 点击右侧对应的按钮进行发送即可

6. 如有其它需求可以模仿提前配置好的命令进行编写

2.6.1 sscom 和屏幕联调-样例工程下载

演示工程下载链接：

《sscom 和屏幕联调》演示工程下载

2.7 串口助手软件 (sscom) 和串口屏模拟器联调 1

串口通讯工具 sscom5.13.1 下载链接：

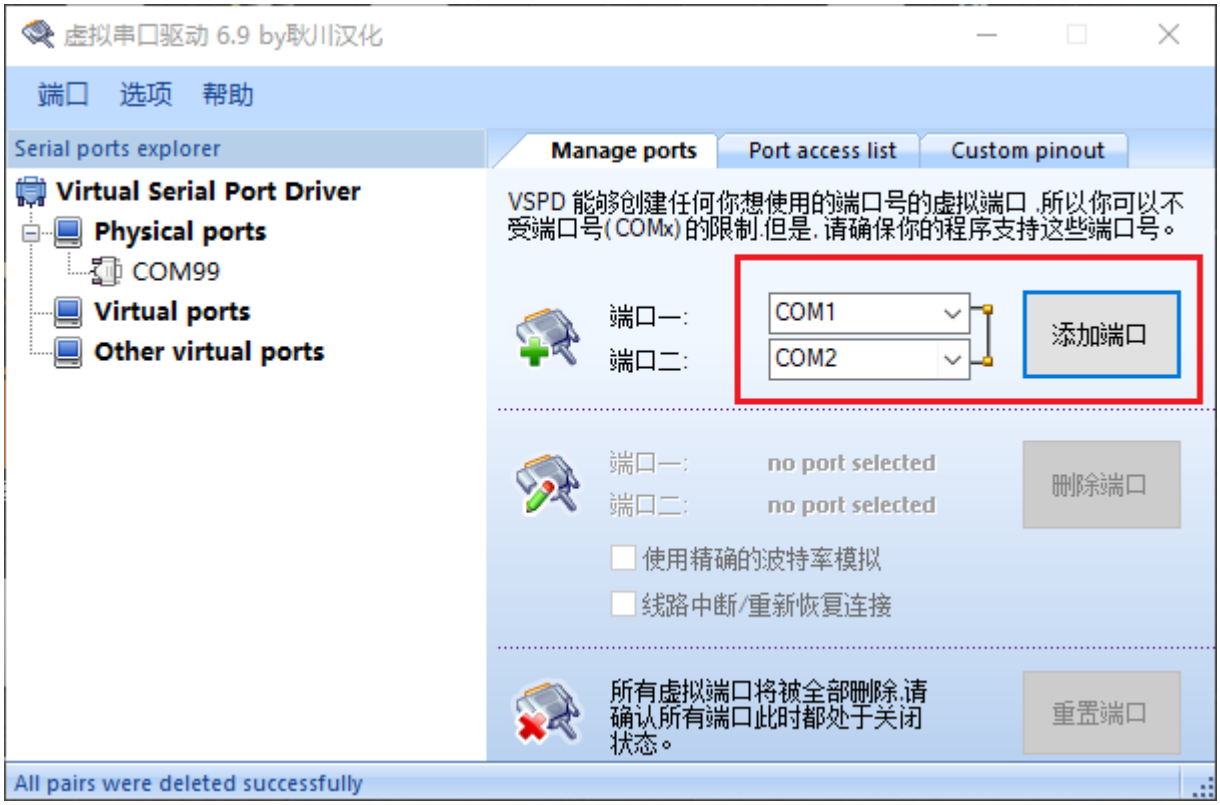
sscom5.13.1 串口调试助手下载 (page 1095)

电脑需要装一个虚拟串口工具 vspd 下载链接：

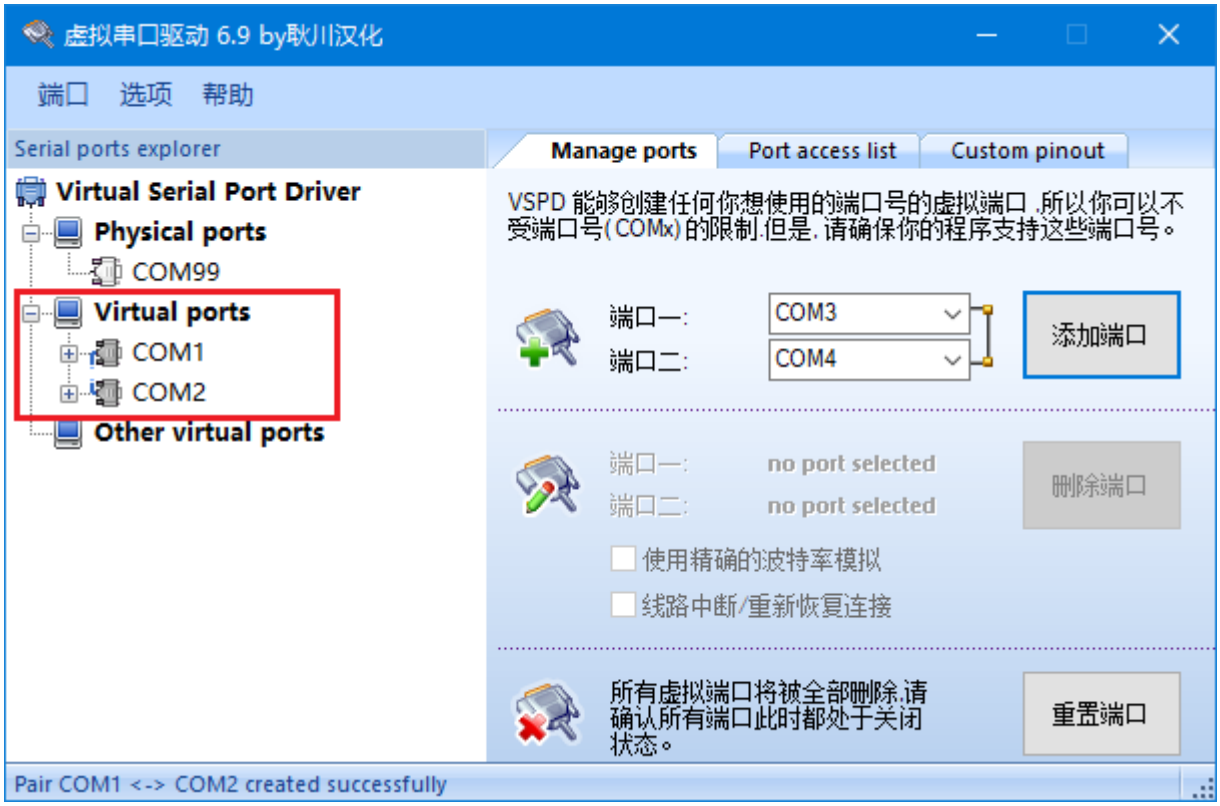
虚拟串口工具 vspd 下载 (page 1096)

打开虚拟串口，添加一对串口，如果已经存在虚拟串口，就不用继续添加了。

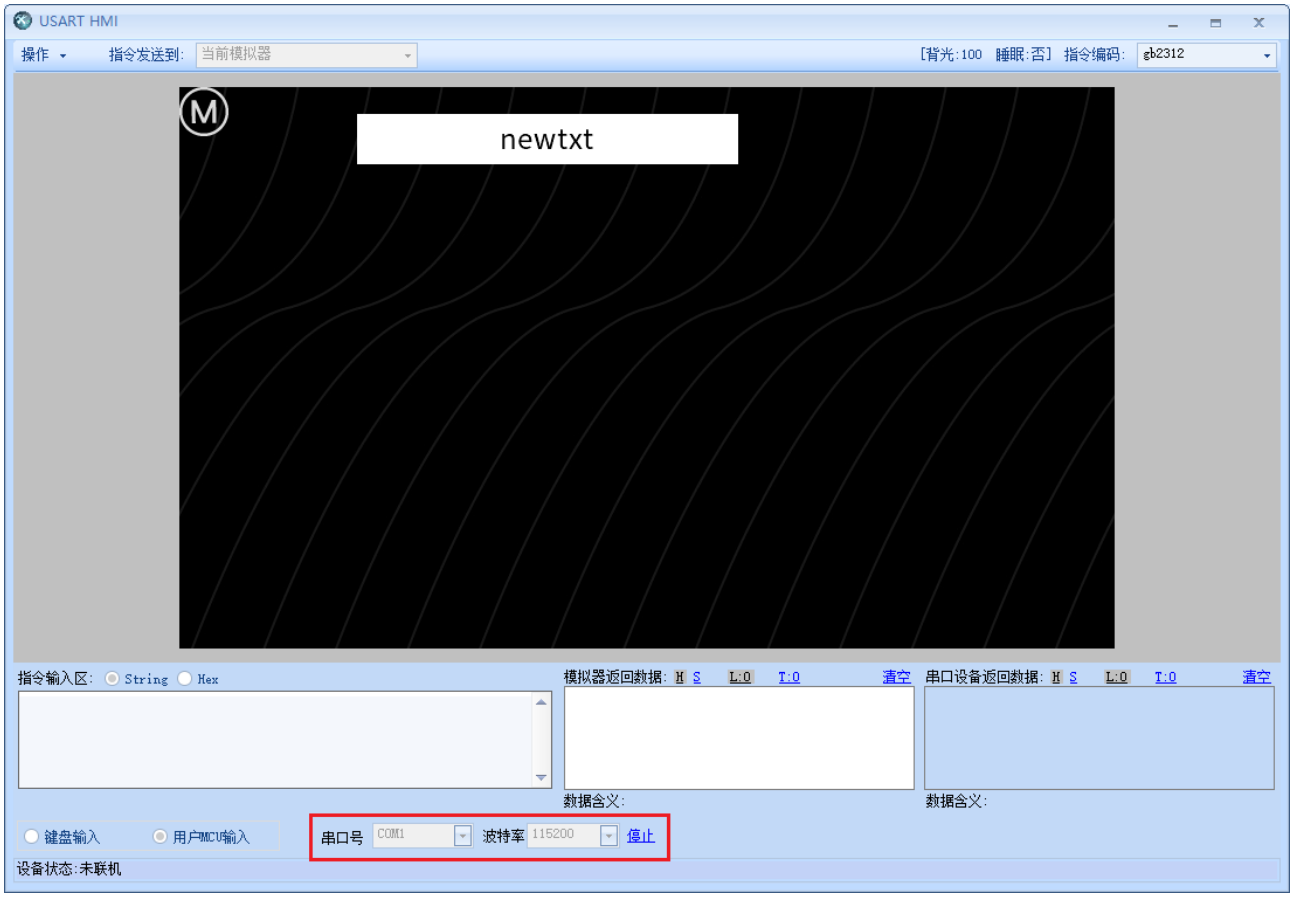
在安装好了 VSPD 之后, 添加一对虚拟串口



添加后如图所示


虚拟串口驱动 6.9 by耿川汉化
端口
选项
帮助
Serial ports explorer
Virtual Serial Port Driver
Physical ports
COM99
Virtual ports
COM1
COM2
Other virtual ports
Manage ports
Port access list
Custom pinout
VSPD 能够创建任何你想使用的端口号的虚拟端口，所以你可以不受端口号(COMx)的限制.但是，请确保你的程序支持这些端口号。
端口一:
端口二:
COM3
COM4
添加端口
端口一:
端口二:
no port selected
no port selected
删除端口
使用精确的波特率模拟
线路中断/重新恢复连接
所有虚拟端口将被全部删除.请确认所有端口此时都处于关闭状态。
重置端口
Pair COM1 <-> COM2 created successfully

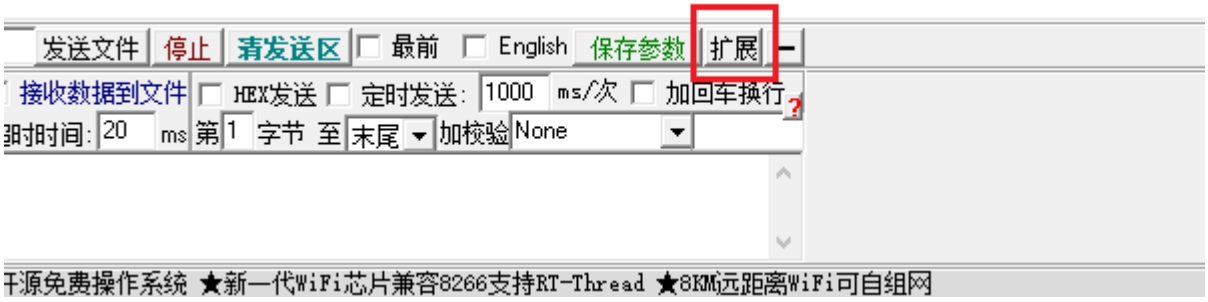
进入串口屏调试界面, 选择用户 mcu 输入, 串口号选择 com1, 波特率 115200, 点击开始



串口助手请使用 SSCOM5.13.1 版本，根据实际情况设置端口号和波特率，不要勾选“加回车换行”

打开 sscom, 串口号选择 com2(与 com1 互为 1 对串口), 波特率 115200, 点击打开串口

点击拓展按钮, 让侧边栏显示出来



将第一栏编辑为如下指令，注意去掉前面的勾选

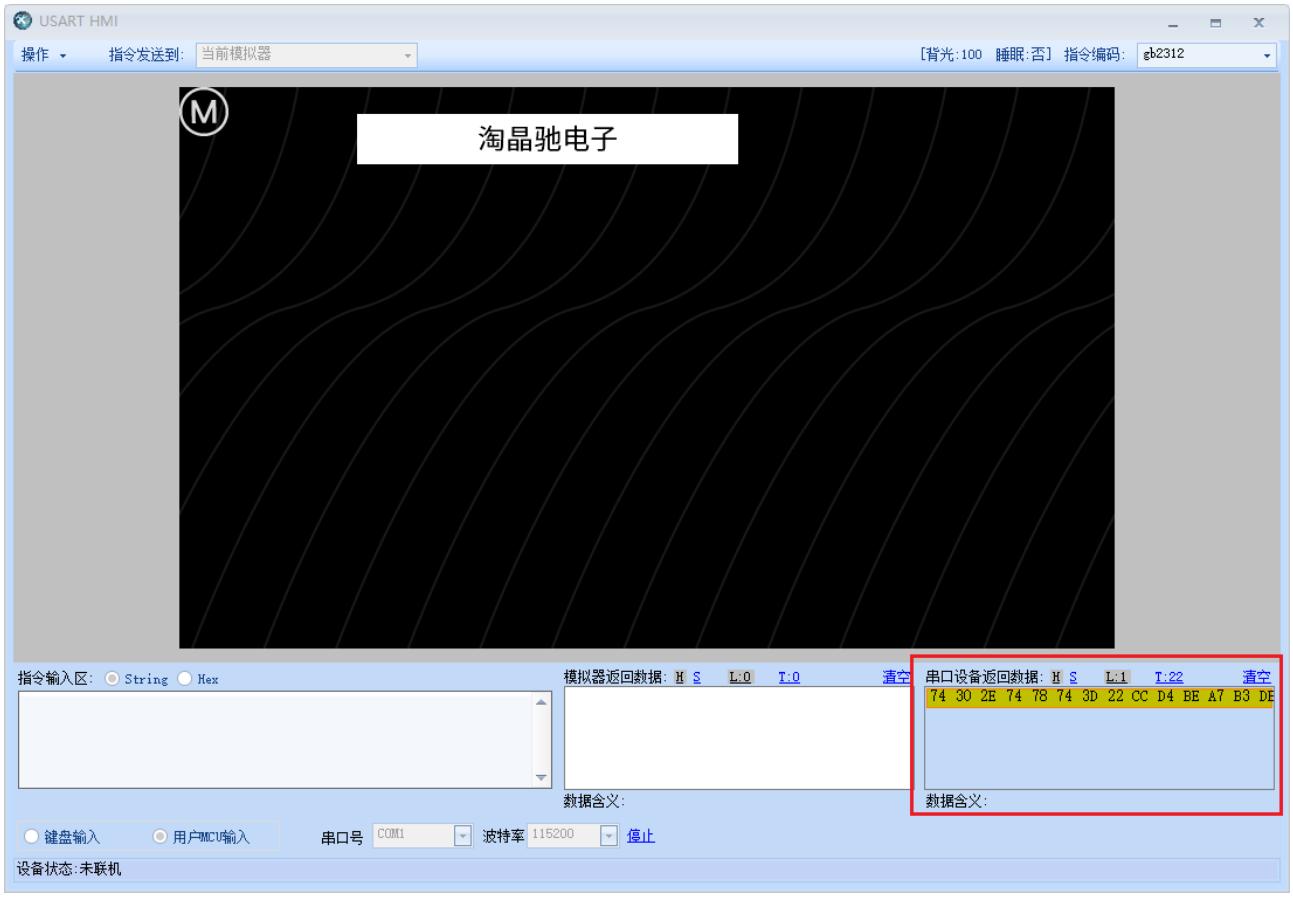
注意： \xff\xff\xff 是结束符

指令如下:

```
t0.txt="淘 晶 驰 电 子"\xff\xff\xff
```

点击右侧按钮即可发送当前指令

此时右下角可以看到接收到了数据, 同时 t0 文本也变成了淘晶驰电子



提示: 用户 mcu 输入功能可以用来连接其他的串口软件或者单片机

2.8 串口助手软件 (sscom) 和串口屏模拟器联调 2

不用单片机，不用屏幕实物，串口助手操作上位模拟器

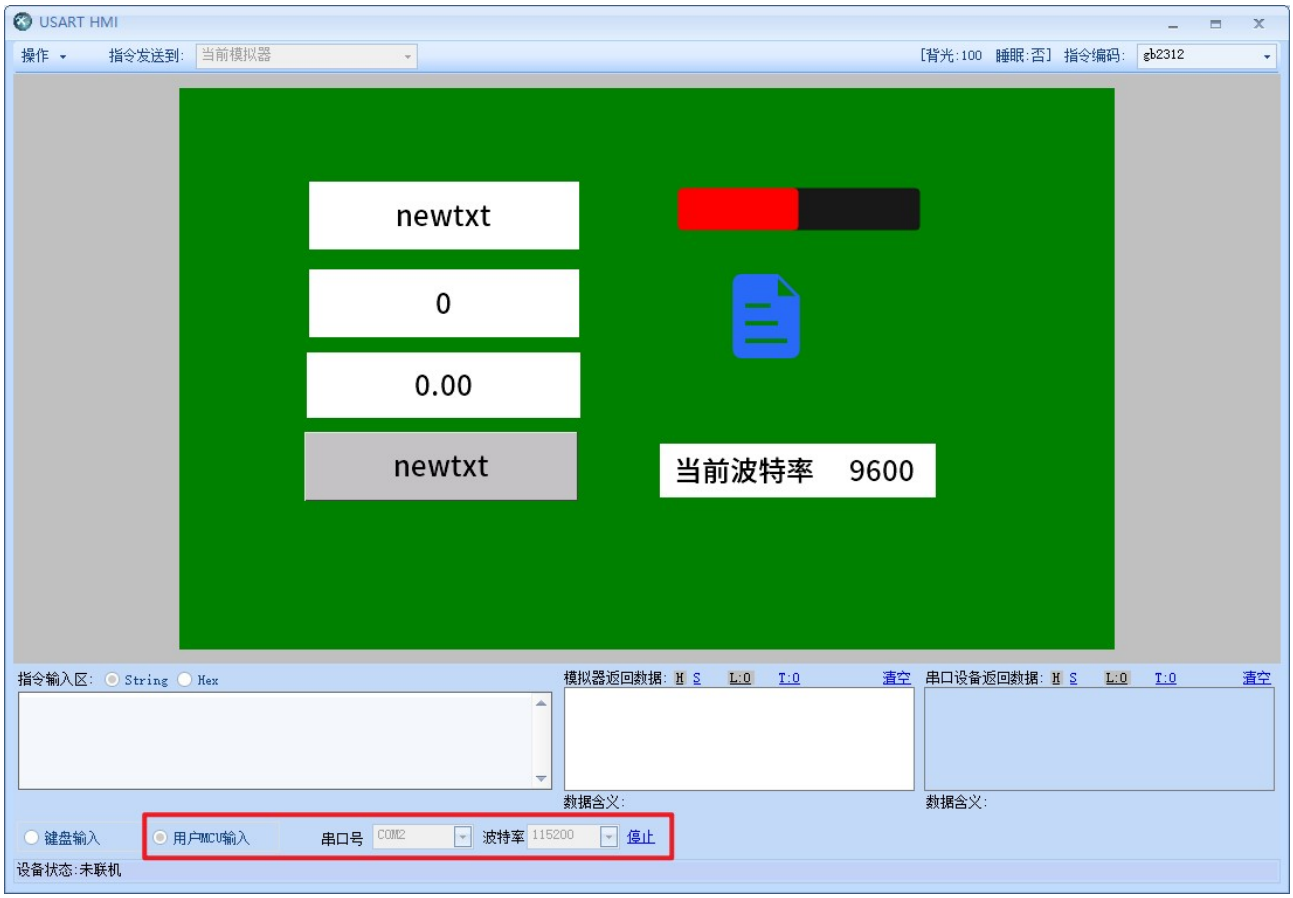
串口通讯工具 sscom5.13.1 下载链接：

sscom5.13.1 串口调试助手下载 (page 1095)

电脑需要装一个虚拟串口工具 vspd 下载链接：

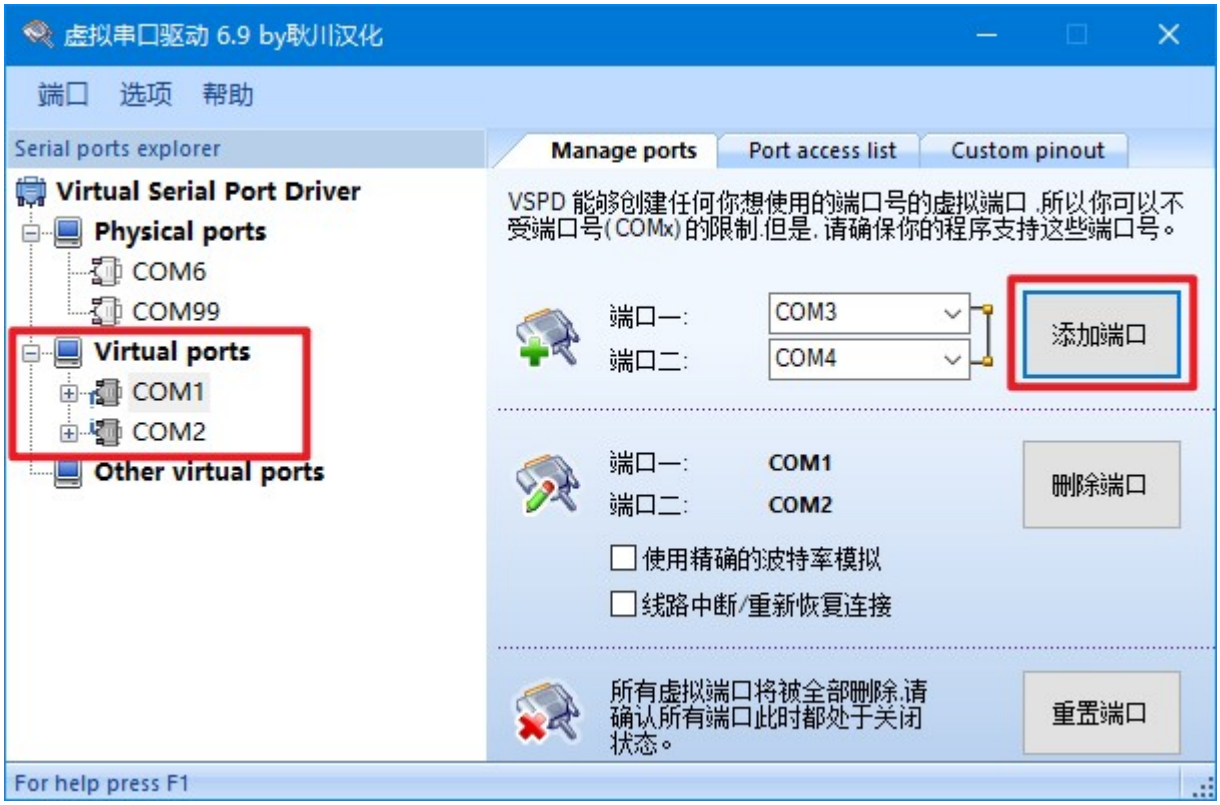
虚拟串口工具 vspd 下载 (page 1096)

2. 打开虚拟串口，添加一对串口，如果已经存在虚拟串口，就不用继续添加了。



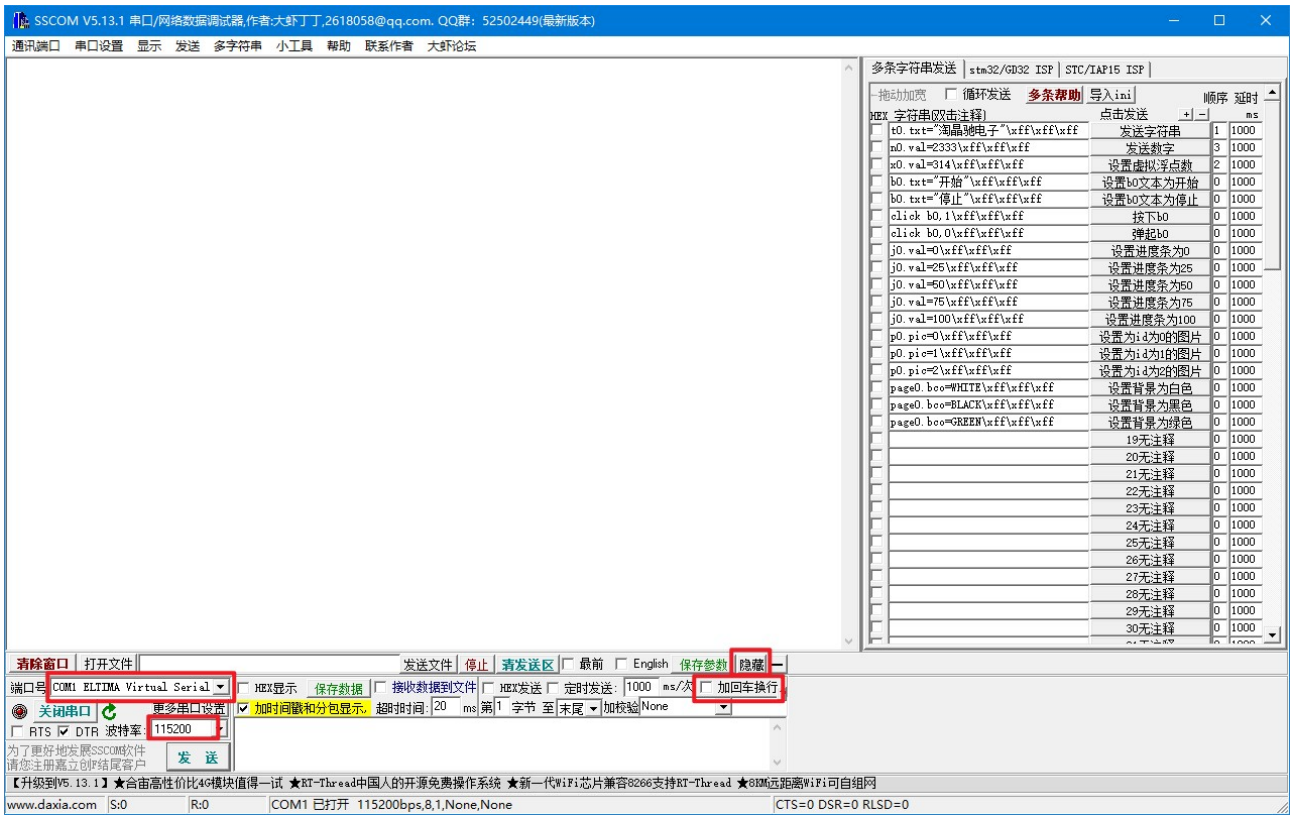
3. 打开串口助手和上位软件（进入上位软件的调试页面）。根据虚拟串口增添的串口来设置串口软件的串口，和上位软件的串口，如图虚拟串口的一对串口是 COM1 和 COM2, 串口助手选择 COM1，那么我们的上位软件必须选择 COM2。

串口助手设置如图：



sscom 软件设置如图：

串口助手请使用 SSCOM5.13.1 版本，根据实际情况设置端口号和波特率，不要勾选“加回车换行”



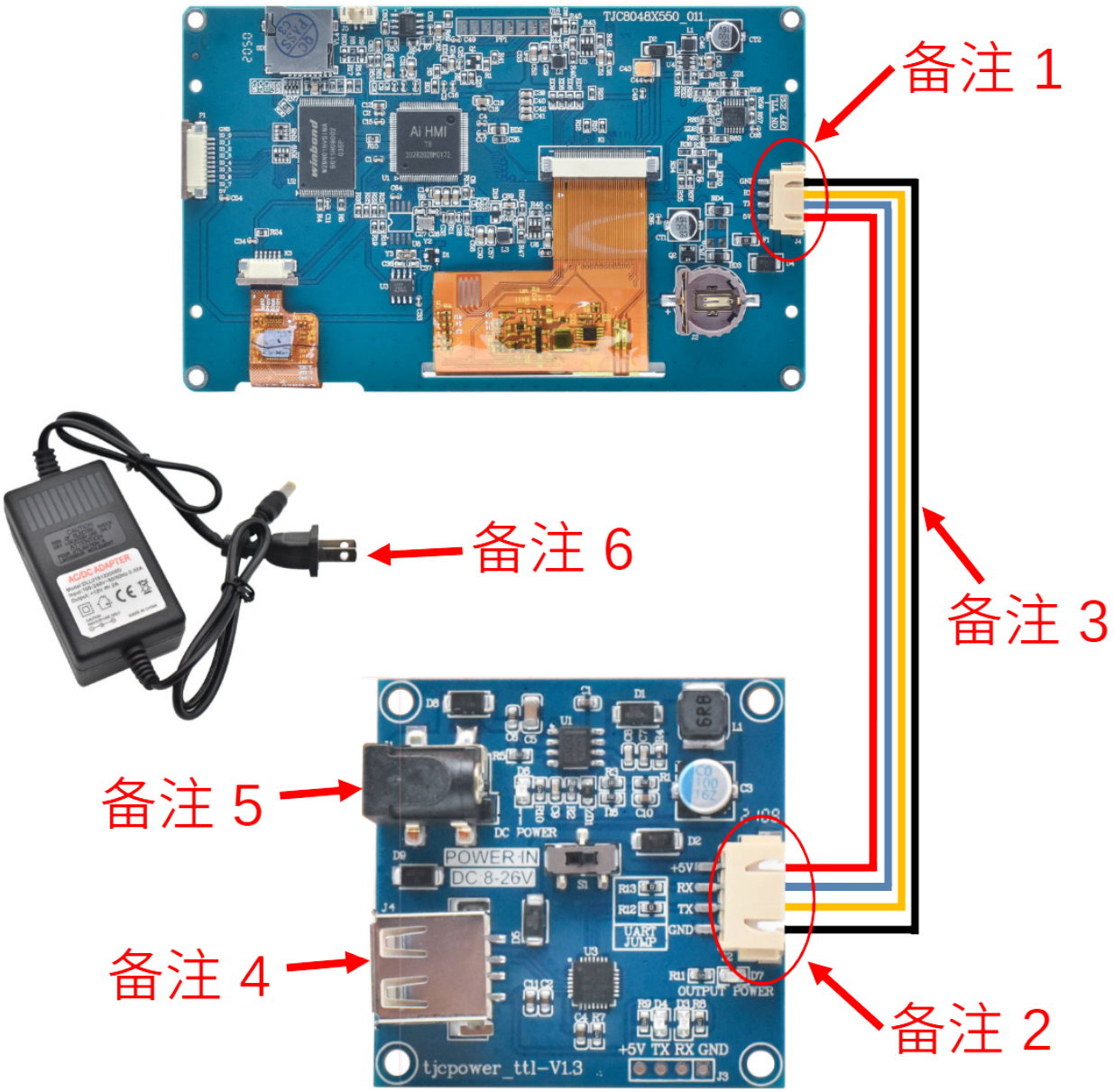
点击右侧对应的按钮进行发送即可

如有其它需求可以模仿提前配置好的命令进行编写

2.9 其他串口助手

2.9.1 串口助手软件 (uartassist) 和屏幕联调

连接的接线图如下



备注 1/2： 5V 接 5V， GND 接 GND， USB 转 TTL 的 TX 接串口屏 RX， USB 转 TTL 的 RX 接串口屏 TX

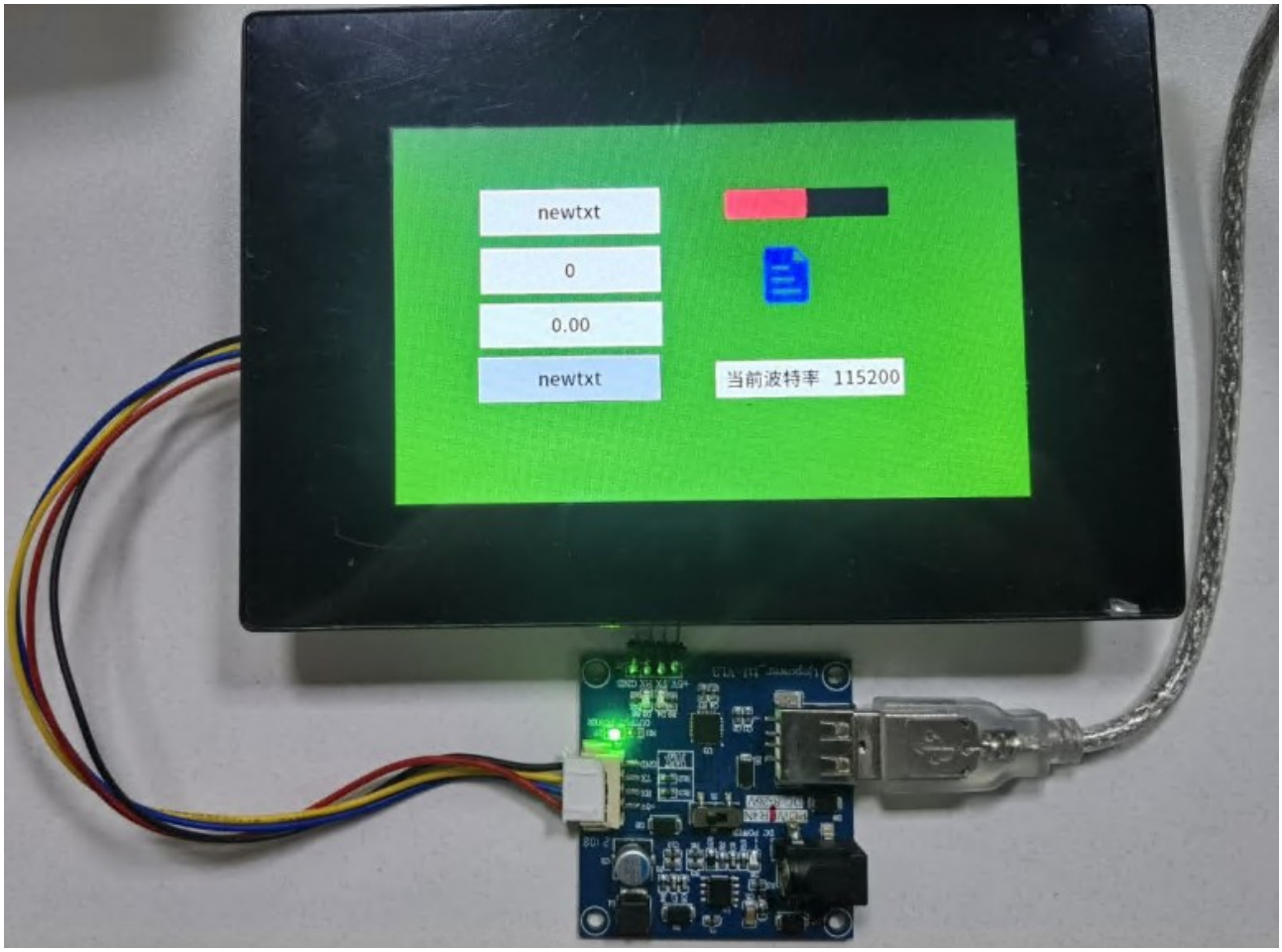
备注 3： 两边的 GND 要接在一起（共地），才能正常通讯

备注 4： 通过 USB 线接电脑

备注 5： 额外供电口，输入电压支持 8V-26V

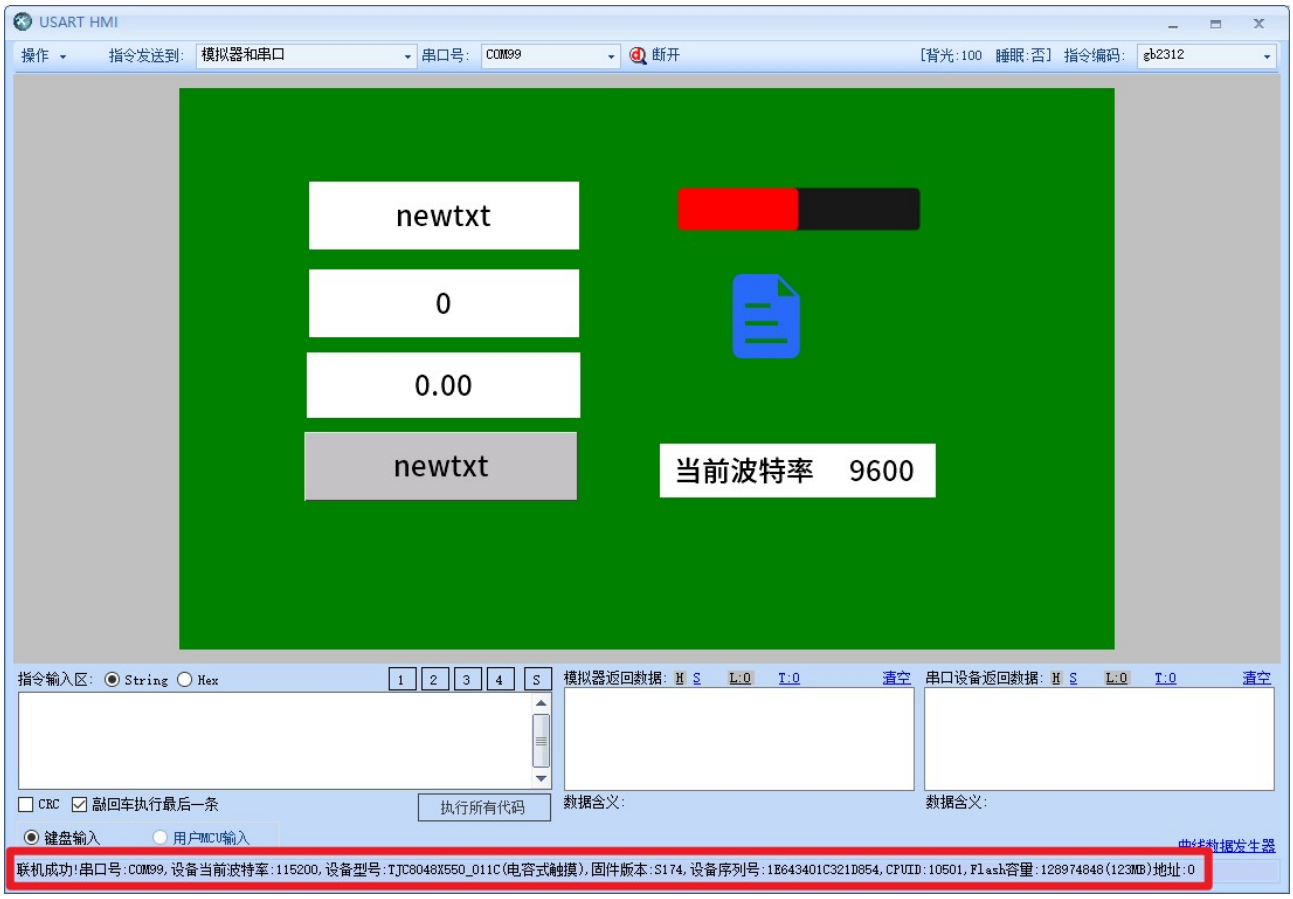
备注 6： 4.3 寸以上的屏幕仅仅使用 USB 线是带不动的，需要使用外部电源供电，否则会无法启动或者不断闪烁重启

使用官方的 USB 转 ttl 连接串口屏如下所示



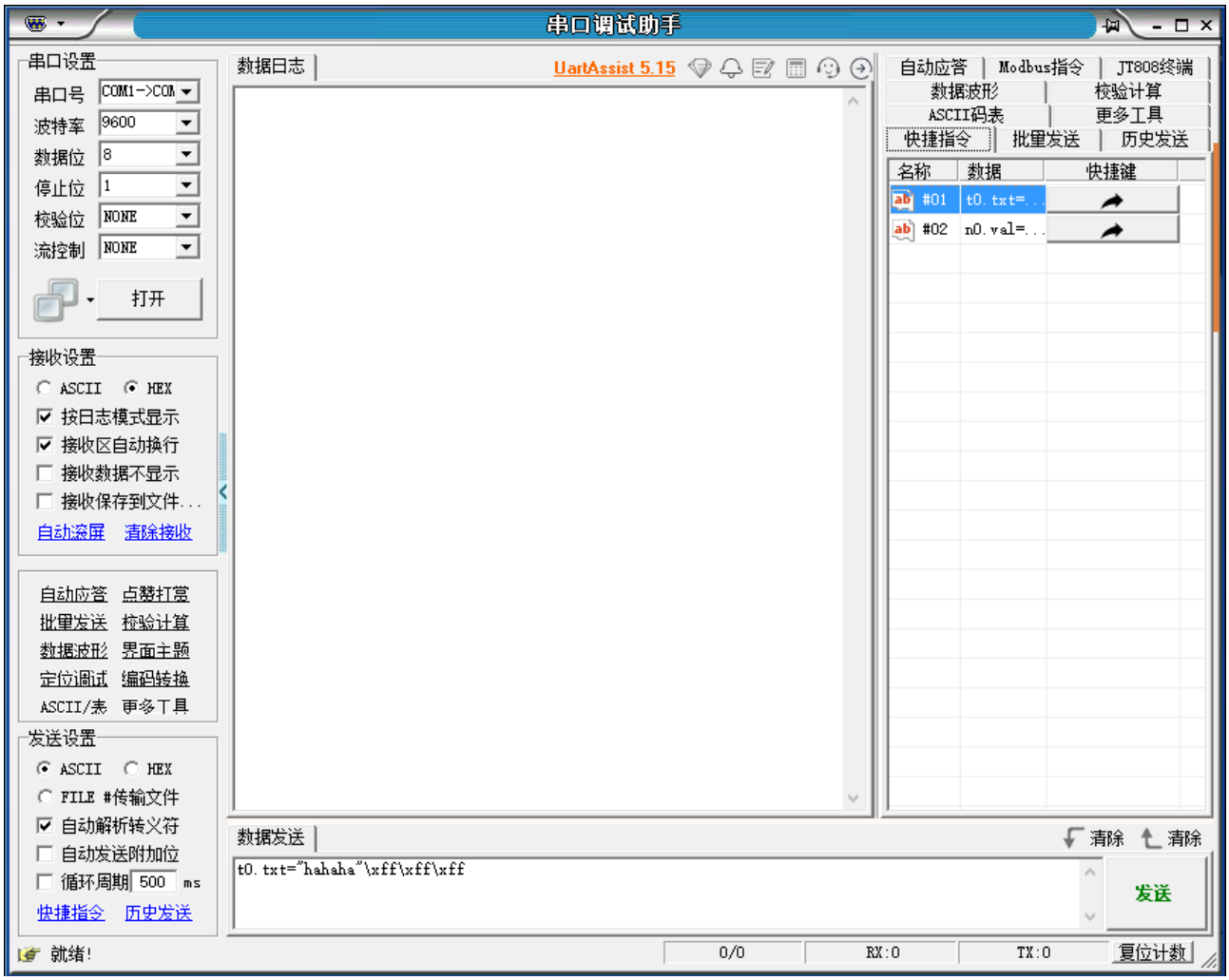
1. 串口屏通过电平转换模块和电脑正确连接，是否连接成功可以先用我们上位联机试一下，这样确保硬件连线没问题。

联机成功后底部会显示联机信息，请记住红圈框起来的串口号和波特率，然后关闭模拟器界面



2. 串口助手如下配置，根据实际情况设置端口号和波特率

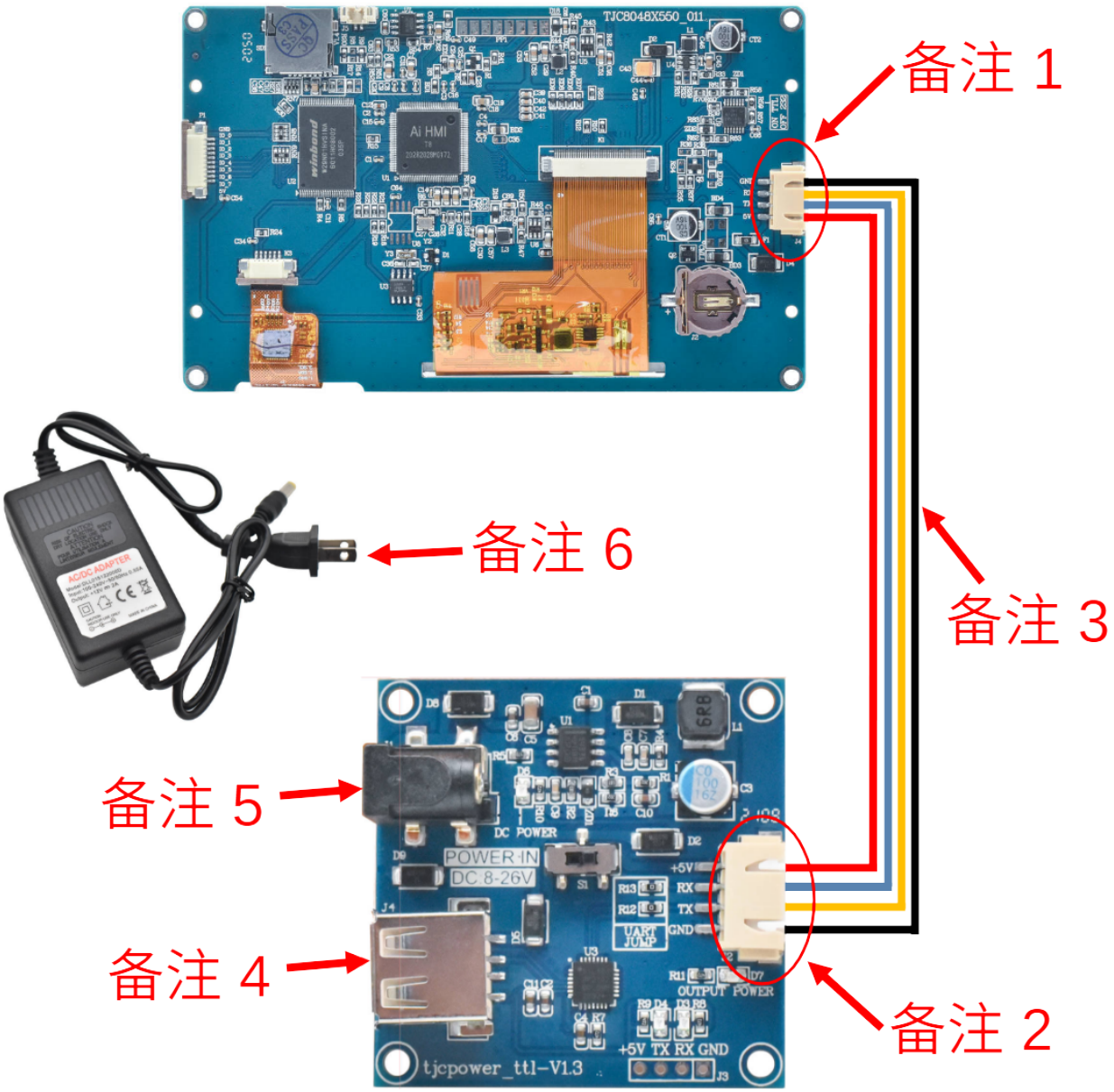
指令的发送格式如下，用法与 sscom 基本一致且功能更加强大



uartassist 下载

2.9.2 串口助手软件 (xcom) 和屏幕联调

连接的接线图如下



备注 1/2： 5V 接 5V， GND 接 GND， USB 转 TTL 的 TX 接串口屏 RX， USB 转 TTL 的 RX 接串口屏 TX

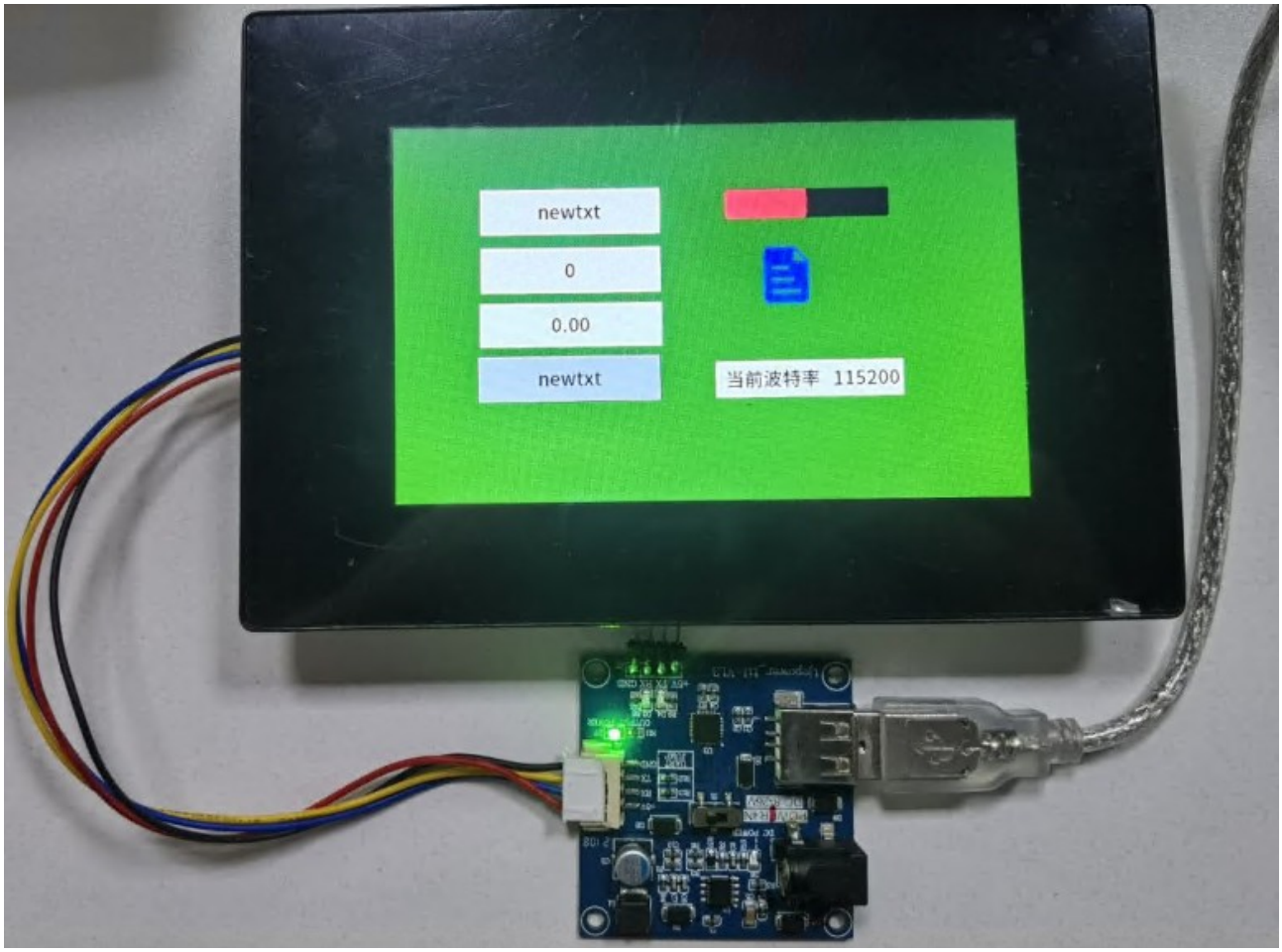
备注 3： 两边的 GND 要接在一起（共地），才能正常通讯

备注 4： 通过 USB 线接电脑

备注 5： 额外供电口，输入电压支持 8V-26V

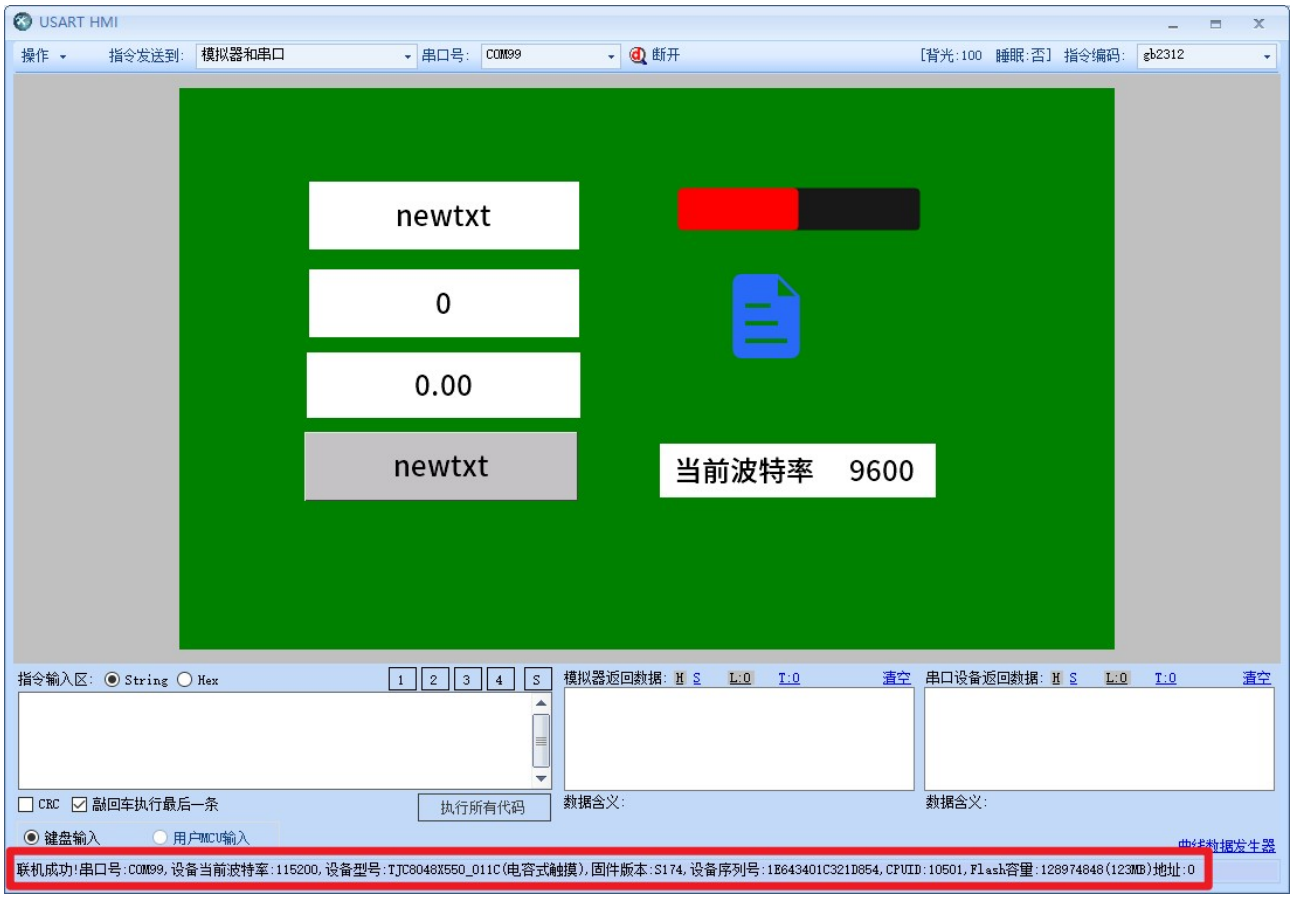
备注 6： 4.3 寸以上的屏幕仅仅使用 USB 线是带不动的，需要使用外部电源供电，否则会无法启动或者不断闪烁重启

使用官方的 USB 转 ttl 连接串口屏如下所示



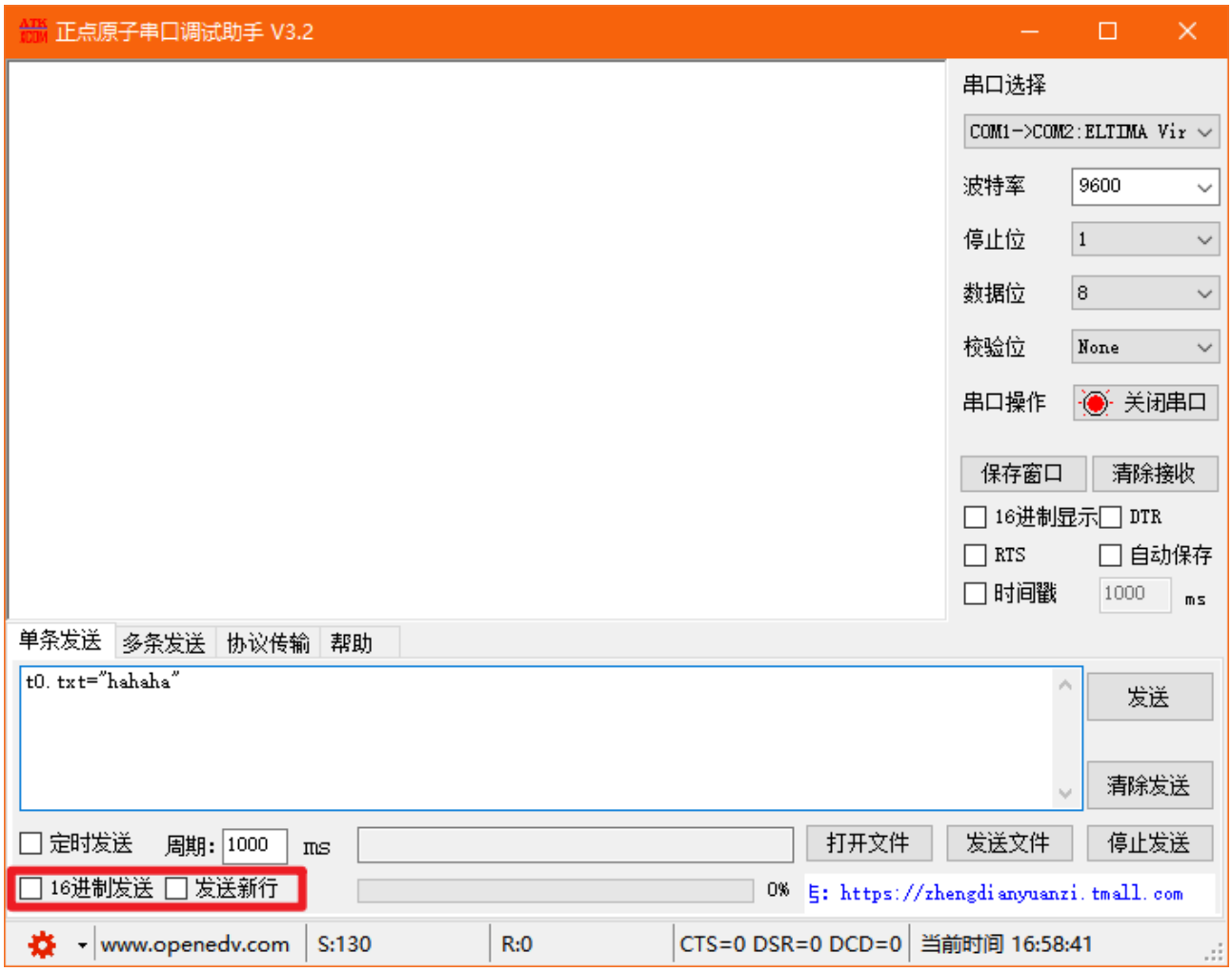
1. 串口屏通过电平转换模块和电脑正确连接，是否连接成功可以先用我们上位联机试一下，这样确保硬件连线没问题。

联机成功后底部会显示联机信息，请记住红圈框起来的串口号和波特率，然后关闭模拟器界面

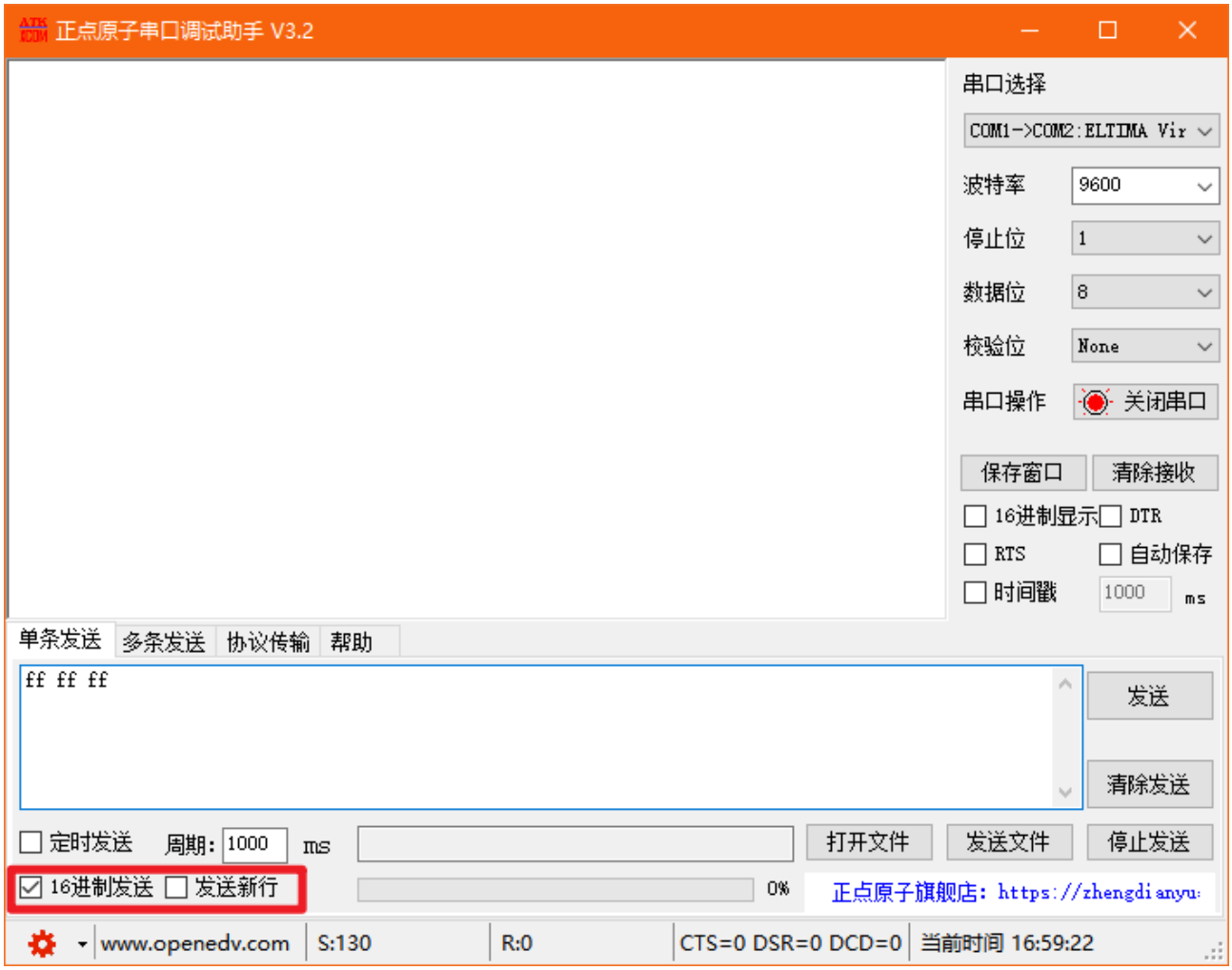


2. 串口助手如下配置，根据实际情况设置端口号和波特率

指令的发送格式如下，不要勾选“16 进制发送”，不要勾选“发送新行”



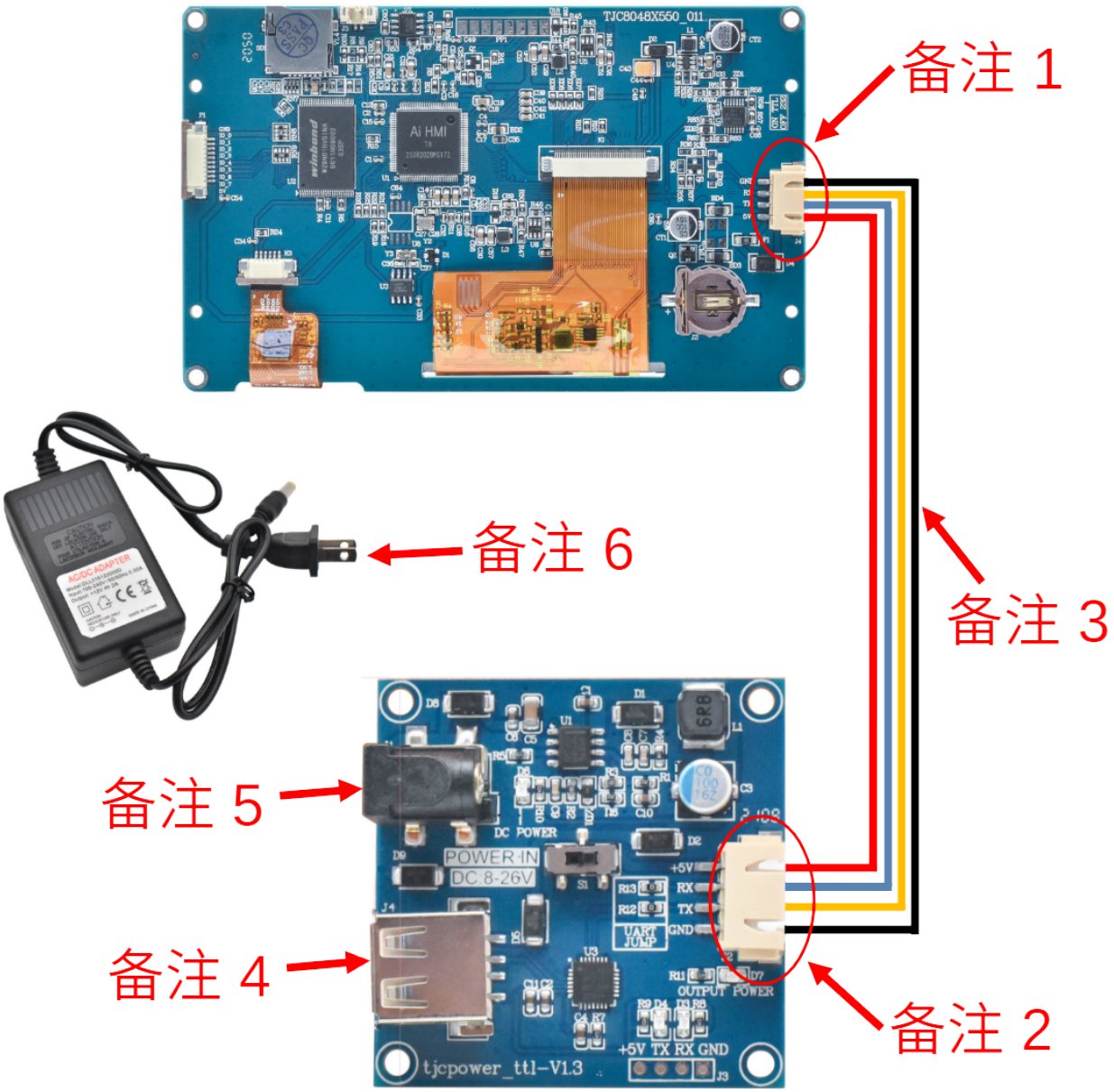
结束符的发送格式如下，勾选“16 进制发送”，不要勾选“发送新行”



注意: 因为 xcom 工具目前为止无法一次性发送完整的指令加结束符, 因此不推荐使用 xcom 进行调试

2.9.3 串口助手软件 (llcom) 和屏幕联调

连接的接线图如下



备注 1/2： 5V 接 5V， GND 接 GND， USB 转 TTL 的 TX 接串口屏 RX， USB 转 TTL 的 RX 接串口屏 TX

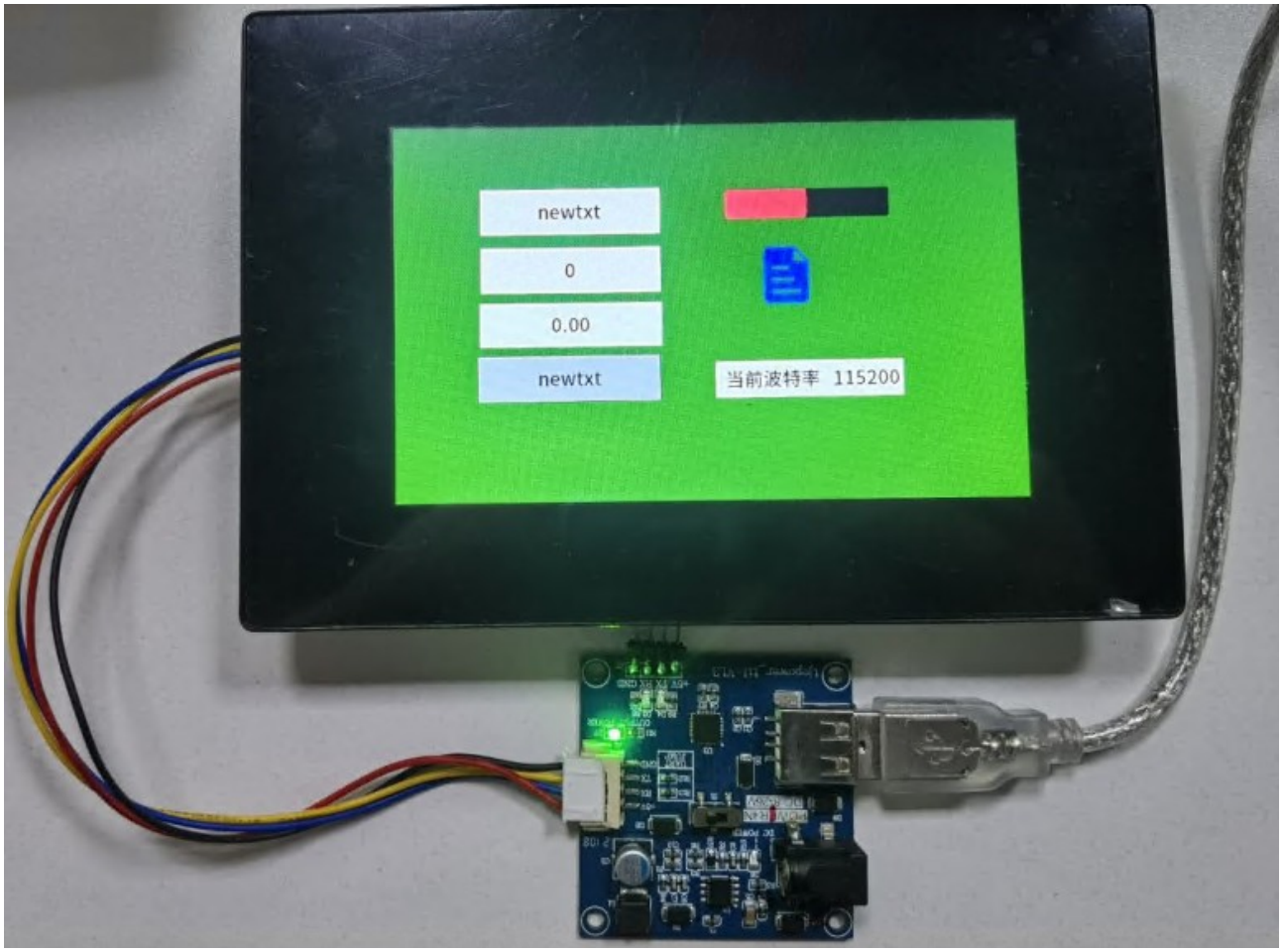
备注 3： 两边的 GND 要接在一起（共地），才能正常通讯

备注 4： 通过 USB 线接电脑

备注 5： 额外供电口，输入电压支持 8V-26V

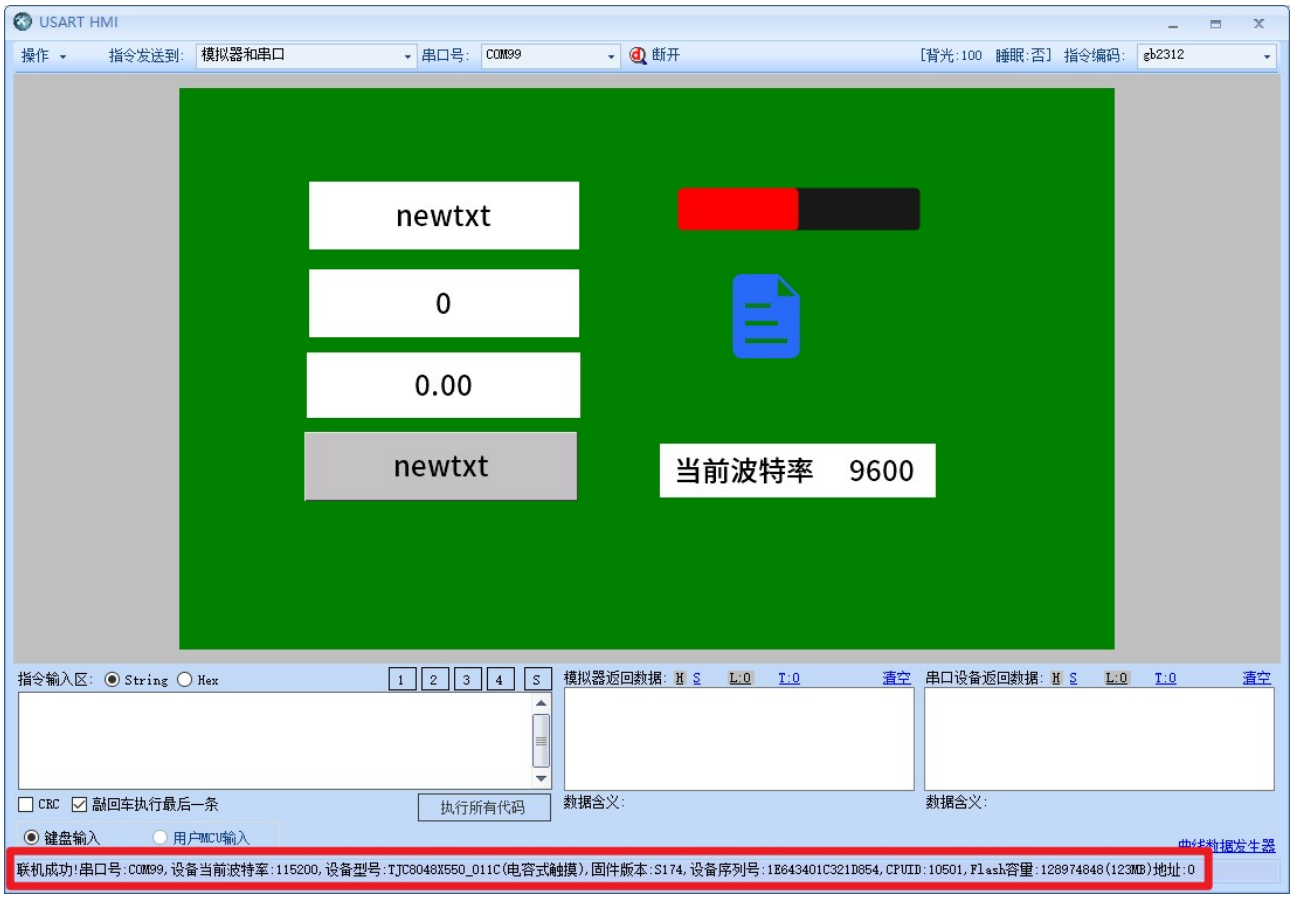
备注 6： 4.3 寸以上的屏幕仅仅使用 USB 线是带不动的，需要使用外部电源供电，否则会无法启动或者不断闪烁重启

使用官方的 USB 转 ttl 连接串口屏如下所示



1. 串口屏通过电平转换模块和电脑正确连接，是否连接成功可以先用我们上位联机试一下，这样确保硬件连线没问题。

联机成功后底部会显示联机信息，请记住红圈框起来的串口号和波特率，然后关闭模拟器界面



2. 串口助手如下配置，根据实际情况设置端口号和波特率

快捷发送配置如下，配置结束符时需要勾选“HEX”，每发送一条指令后需要发送一次结束符

2.10 与 stm32 单片机联调

2.10.1 stm32 说明

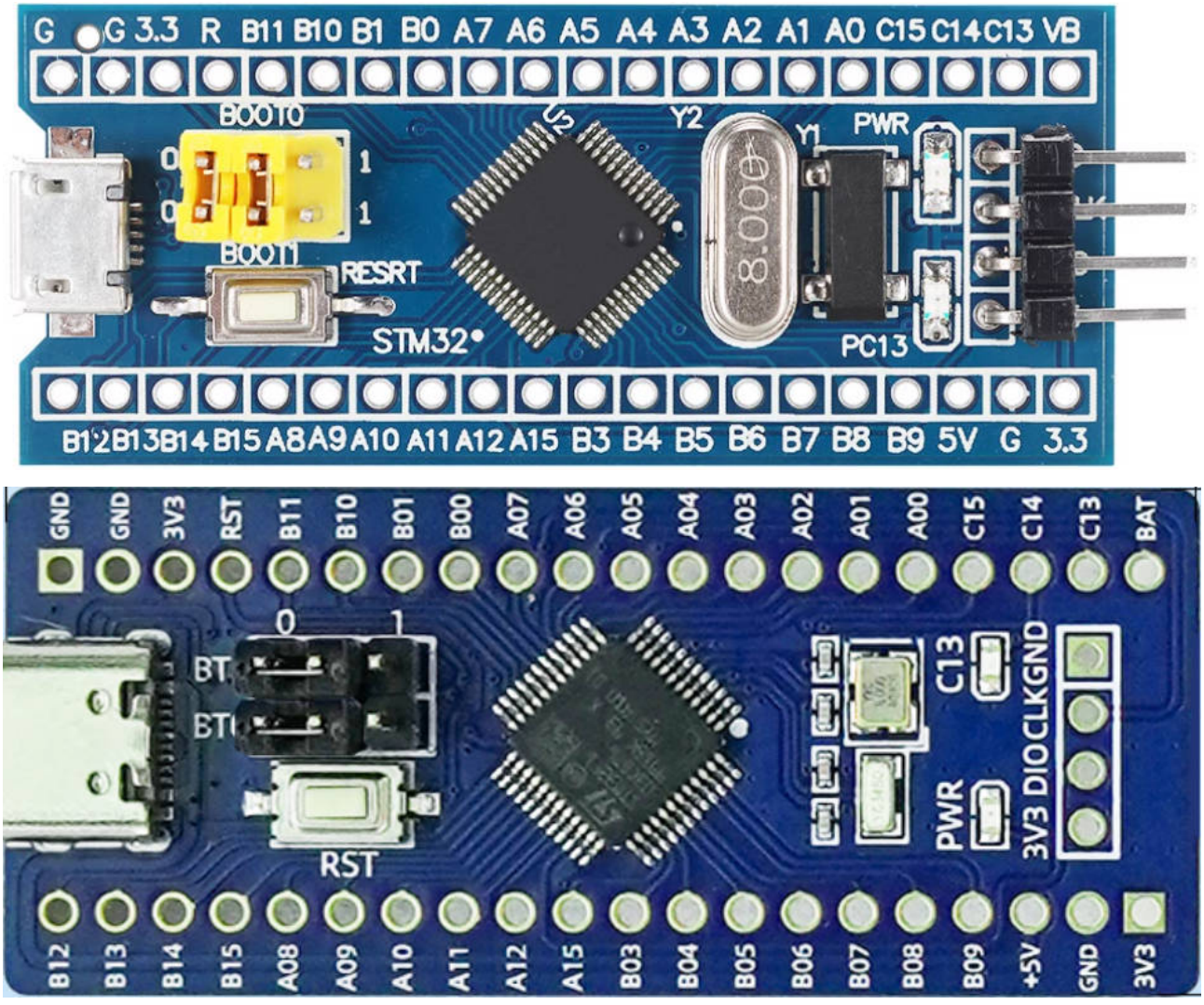
串口屏怎么下载程序

通过串口下载工程到串口屏 (page 144)

使用 TFT 文件下载助手 (TFTFileDownload) 通过串口下载工程到串口屏 (page 150)

通过 SD 卡下载工程到串口屏 (page 155)

使用的单片机型号为 stm32f103c8t6 的最小系统板，兼容淘宝常见款，同时也兼容立创 · 地阔星 STM32F103C8T6 开发板



通过手册我们查找到，在未重映射的情况下，PA2 为 TX，PA3 为 RX。

Table 53. USART2 remapping

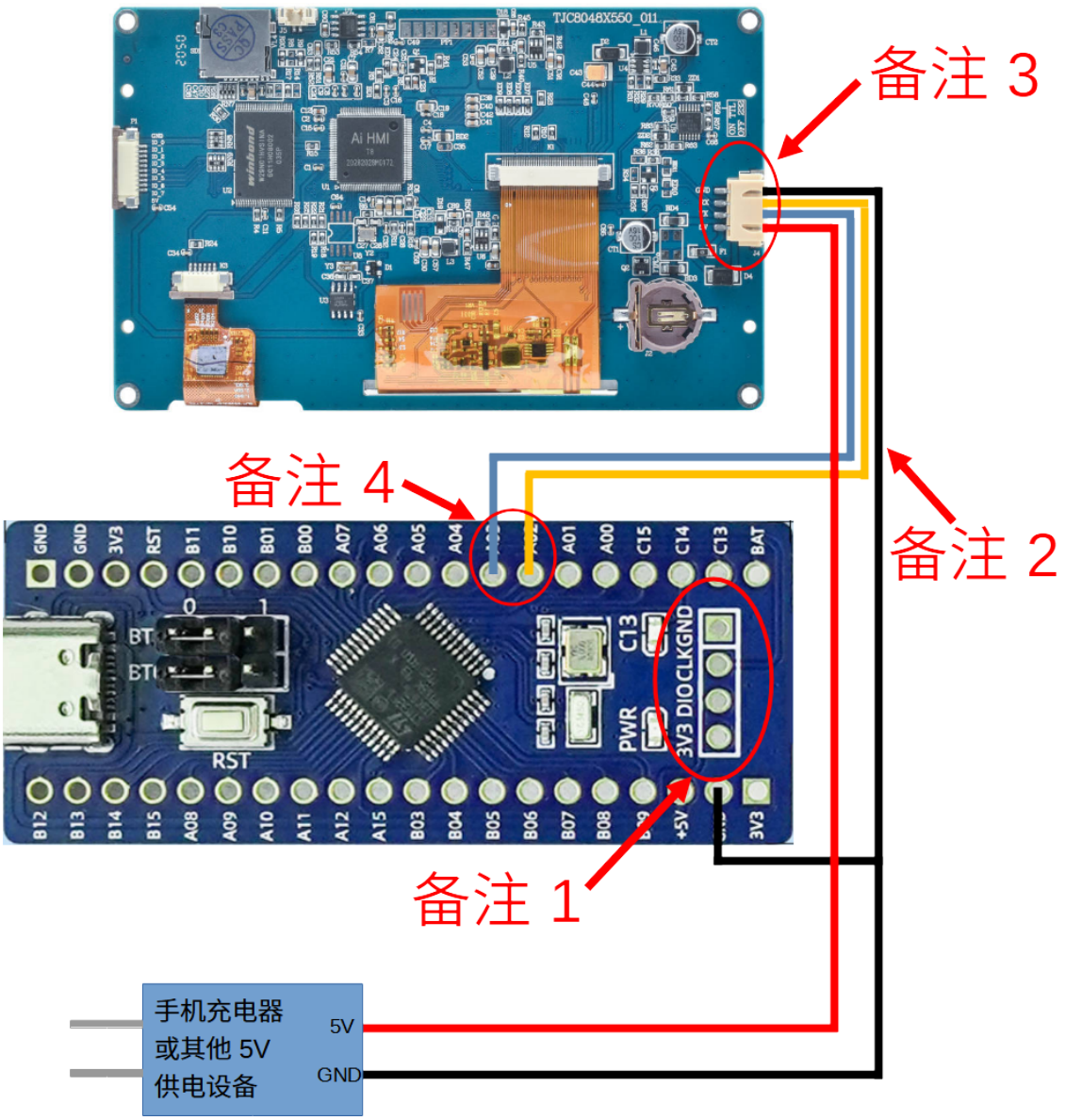
Alternate functions	USART2_REMAP = 0	USART2_REMAP = 1(1)
USART2_CTS	PA0	PD3
USART2_RTS	PA1	PD4
USART2_TX	PA2	PD5
USART2_RX	PA3	PD6
USART2_CK	PA4	PD7

1. Remap available only for 100-pin and 144-pin packages.

2.10.1.1 stm32 通讯-如何下载工程到串口屏

通过串口下载工程到串口屏 (page 144)
通过 *SD* 卡下载工程到串口屏 (page 155)

2.10.1.2 stm32 单片机与串口屏的接法



备注 1： 给 STM32 下载程序和供电 (3.3V) 还是通过这个接口

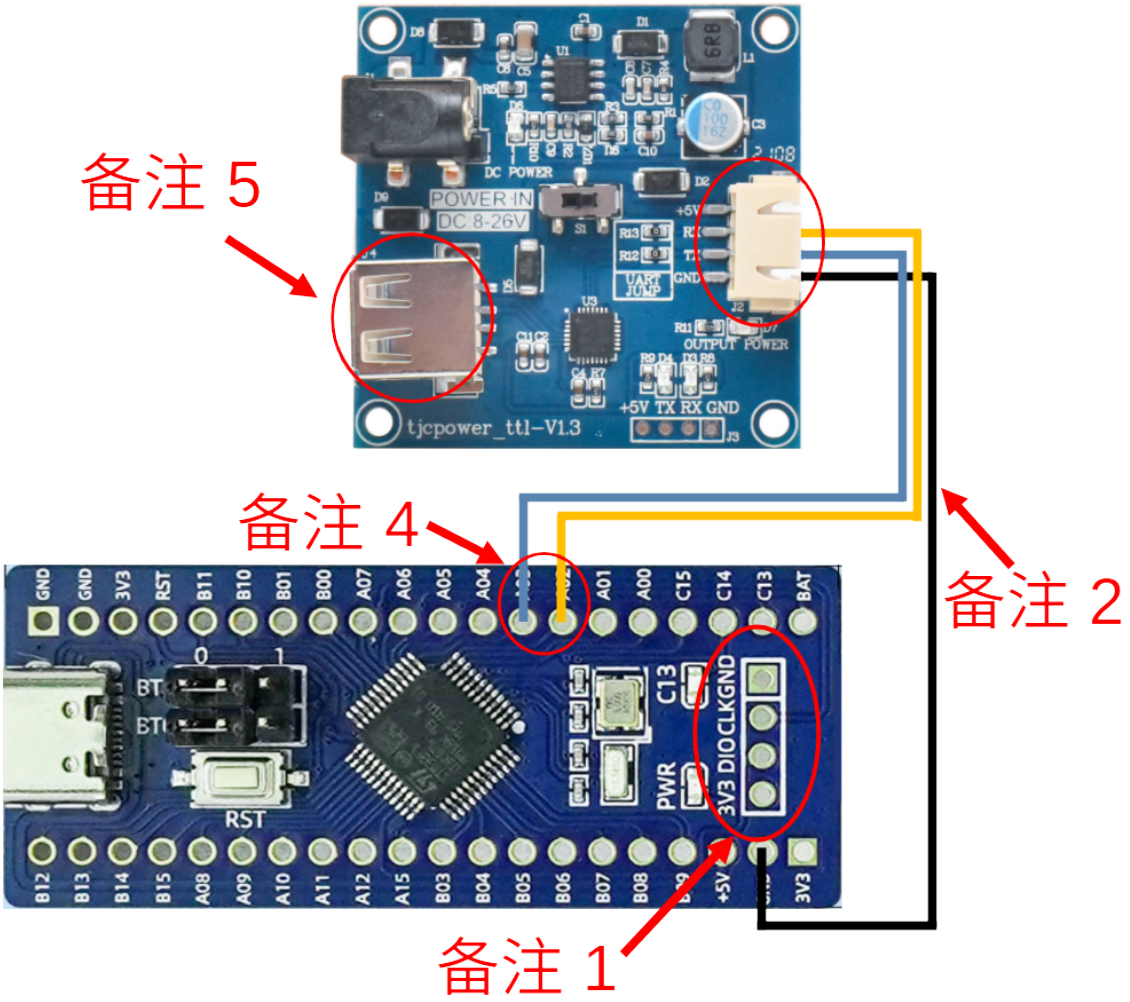
备注 2： 两边的 GND 要接在一起 (共地)，才能正常通讯

备注 3： 如果要通过 usb 转 ttl 给屏幕下载程序，要断开和 STM32 的连接，将屏幕接口和 usb 转 ttl 接在一起才能下载

备注 4： STM32 的 TX 接串口屏的 RX， STM32 的 RX 接串口屏的 TX

2.10.1.3 单片机与模拟器如何连接

将单片机通过 usb 转 ttl 连接到电脑上与电脑联机进行开发
连接方式如下所示



备注 1： 给 STM32 下载程序和供电 (3.3V) 还是通过这个接口

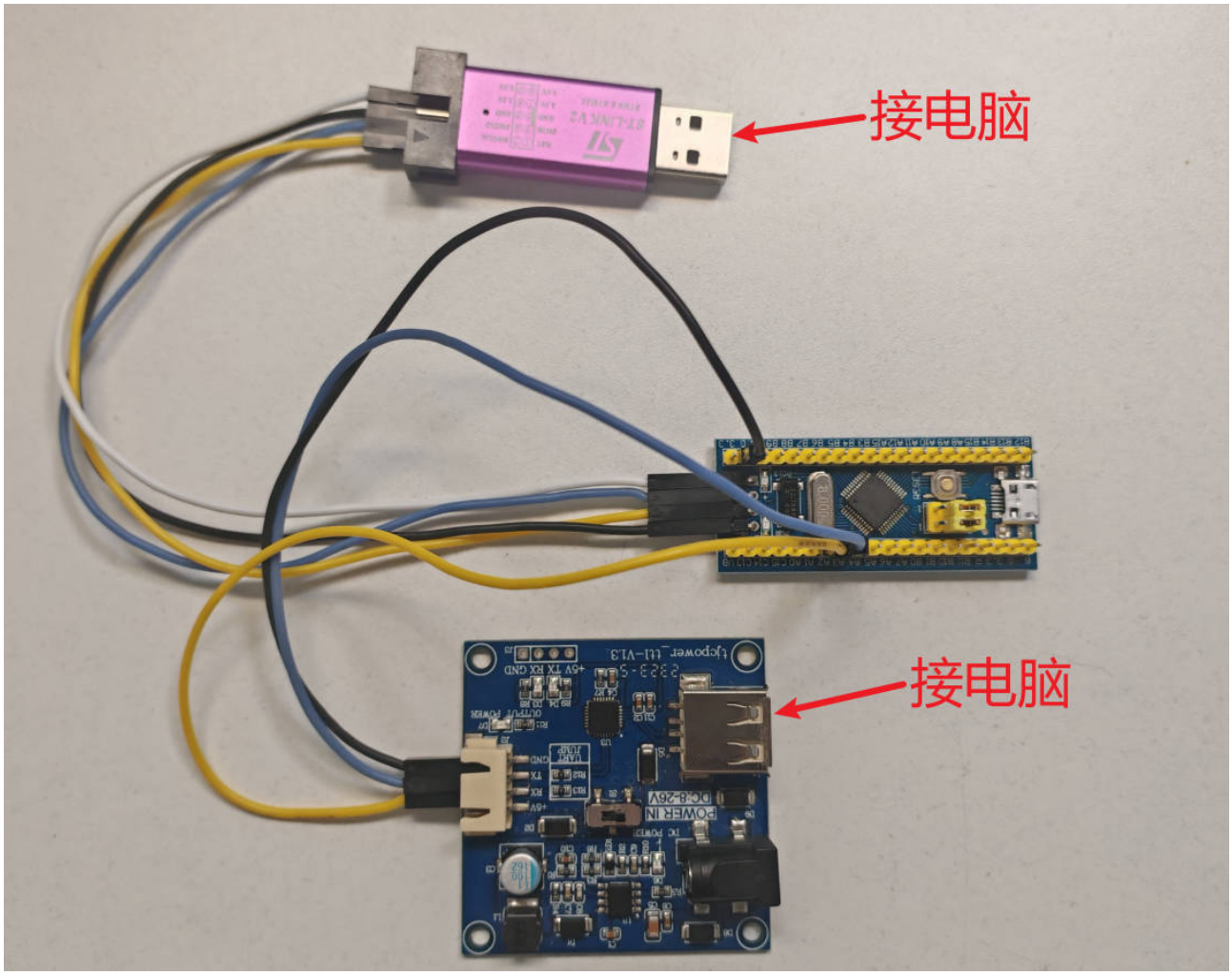
备注 2： 两边的 GND 要接在一起 (共地)，才能正常通讯

备注 3： 如果要通过 usb 转 ttl 给屏幕下载程序，要断开和 STM32 的连接，将屏幕接口和 usb 转 ttl 接在一起才能下载

备注 4： STM32 的 TX 接串口屏的 RX， STM32 的 RX 接串口屏的 TX

备注 5： 通过 usb 线连接到电脑

参考实物接法如下



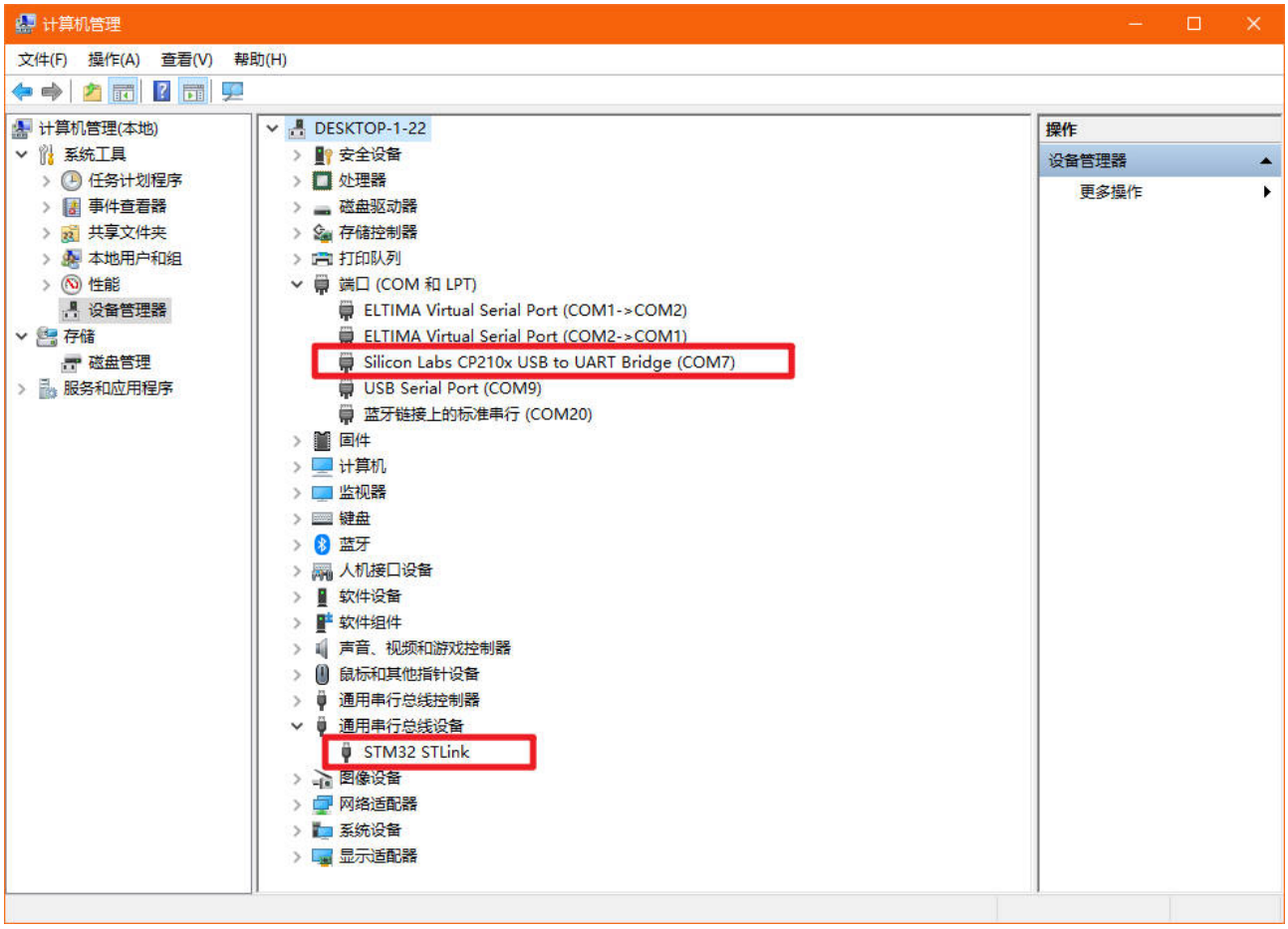
驱动安装完成后，应该能够在设备管理器中看到这两个驱动，驱动安装请参考安装串口驱动 (page 138)

一个是串口驱动，建议使用 cp2102 或者 ft232 芯片的串口工具，不要使用 ch340 或者 pl2303

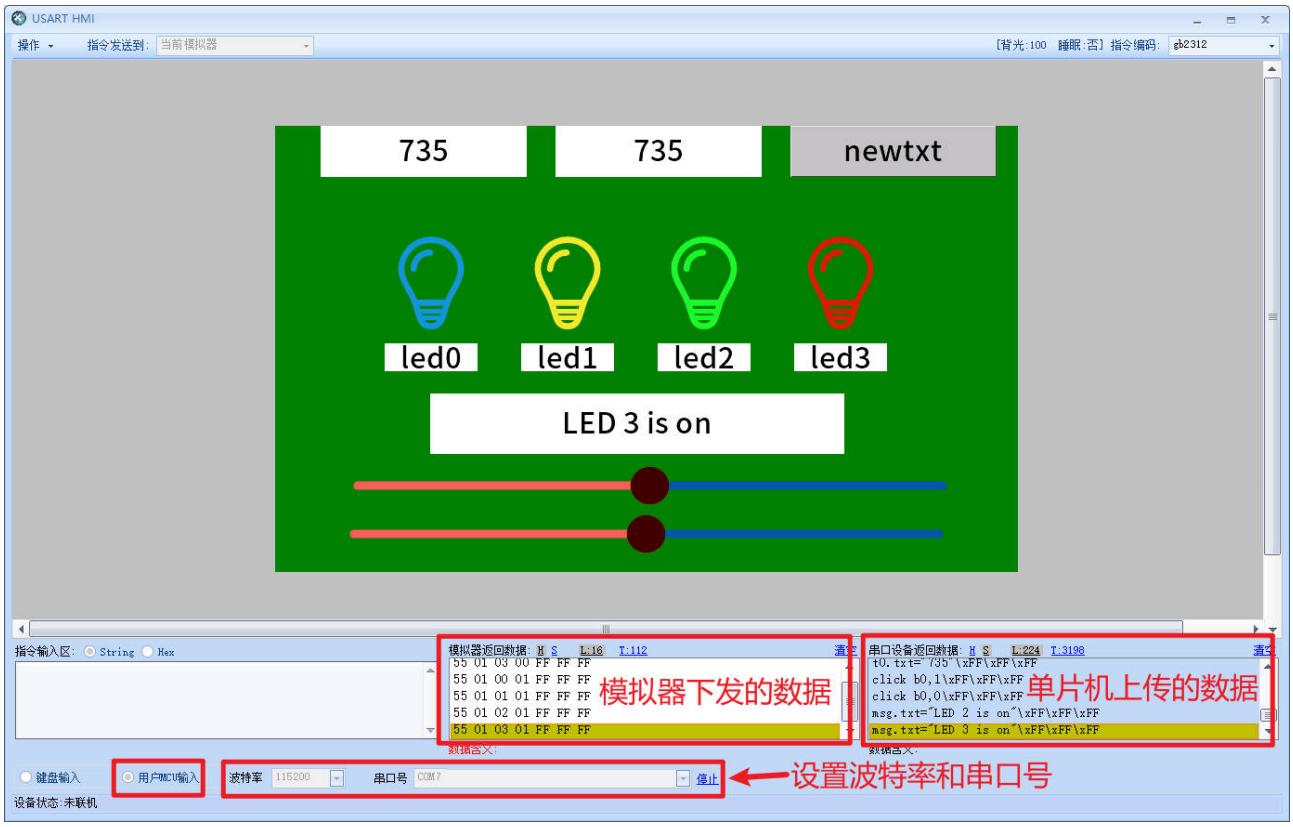
串口工具的串口号是 com7，记住这个串口号，在你的电脑上可能会是其他串口号，需要使用时根据自己电脑的串口号进行选择即可

一个是调试工具的驱动，我这里使用的时 stlink，你也可以使用 jlink 或者 daplink，手里有啥就用啥，但我目前只试过 stlink，不保证其他调试工具能用

这两个驱动是没有感叹号的，如果有感叹号，说明没有正确安装驱动，请通过百度搜索如何安装相应的驱动



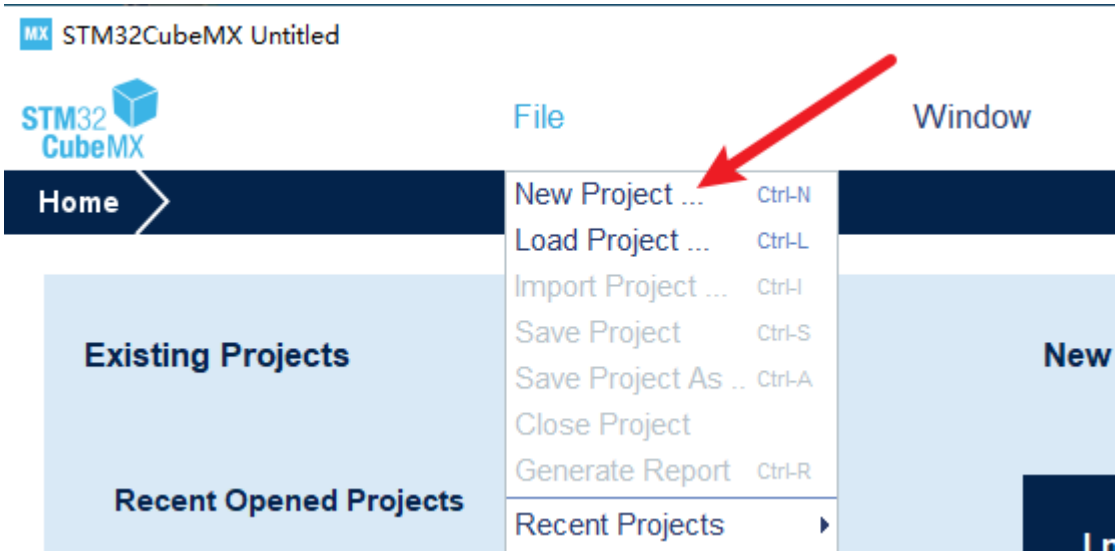
在开发期间, 可以使用模拟器与单片机进行联机调试



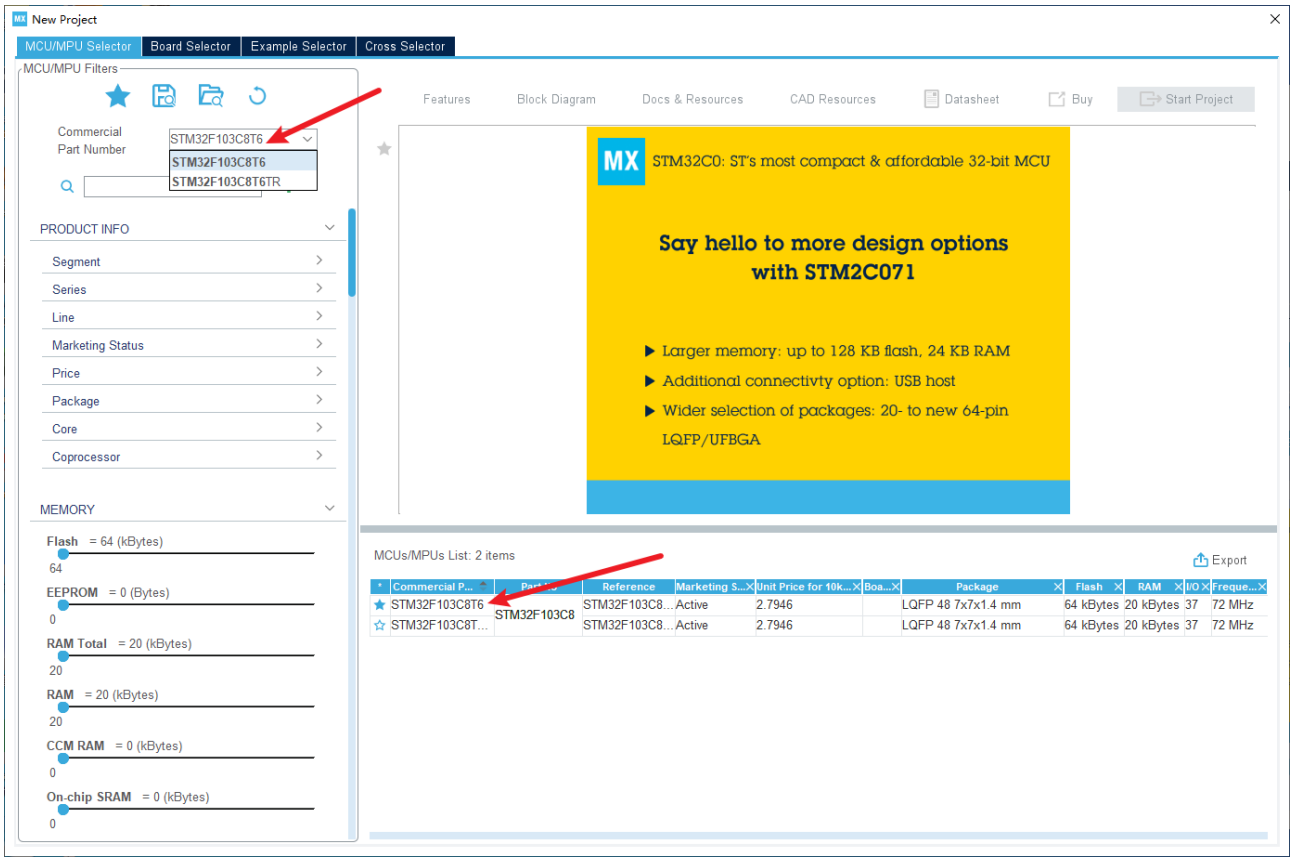
2.10.2 stm32 HAL 库开发

2.10.2.1 使用 stm32cubemx 创建工程

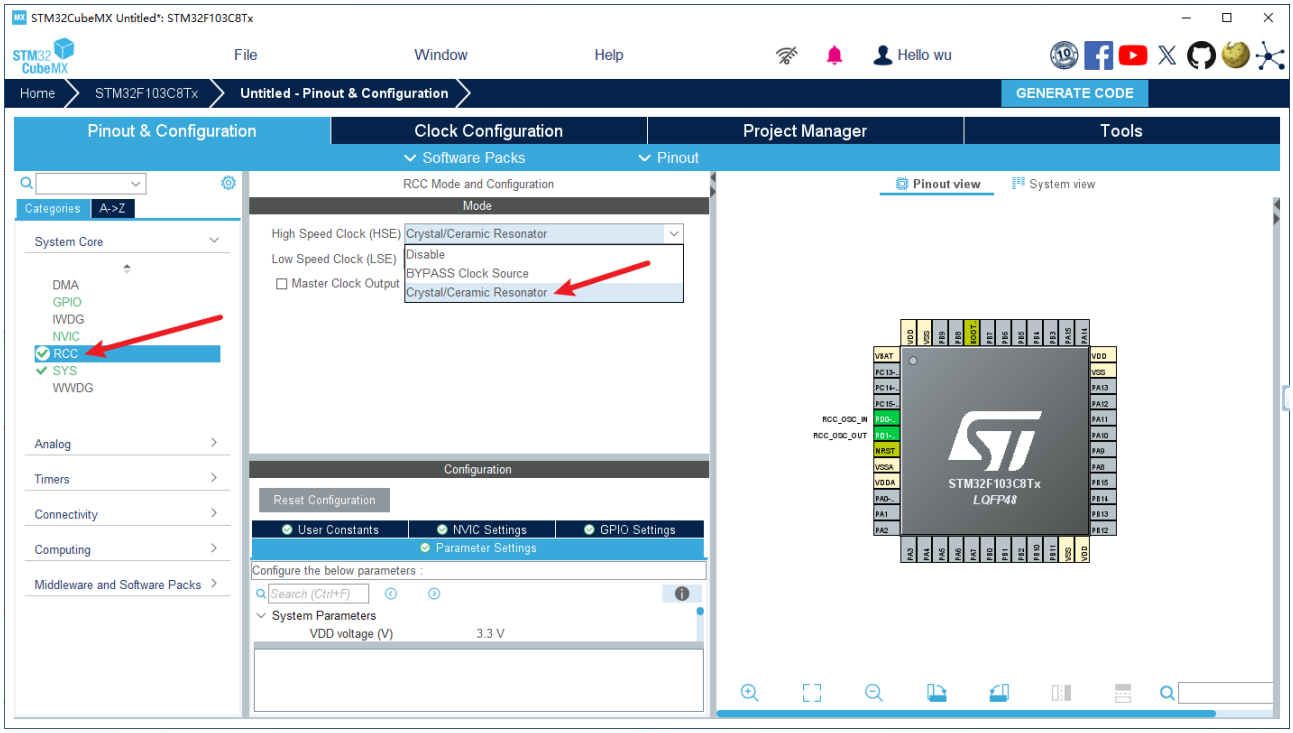
1、新建 cubemx 工程



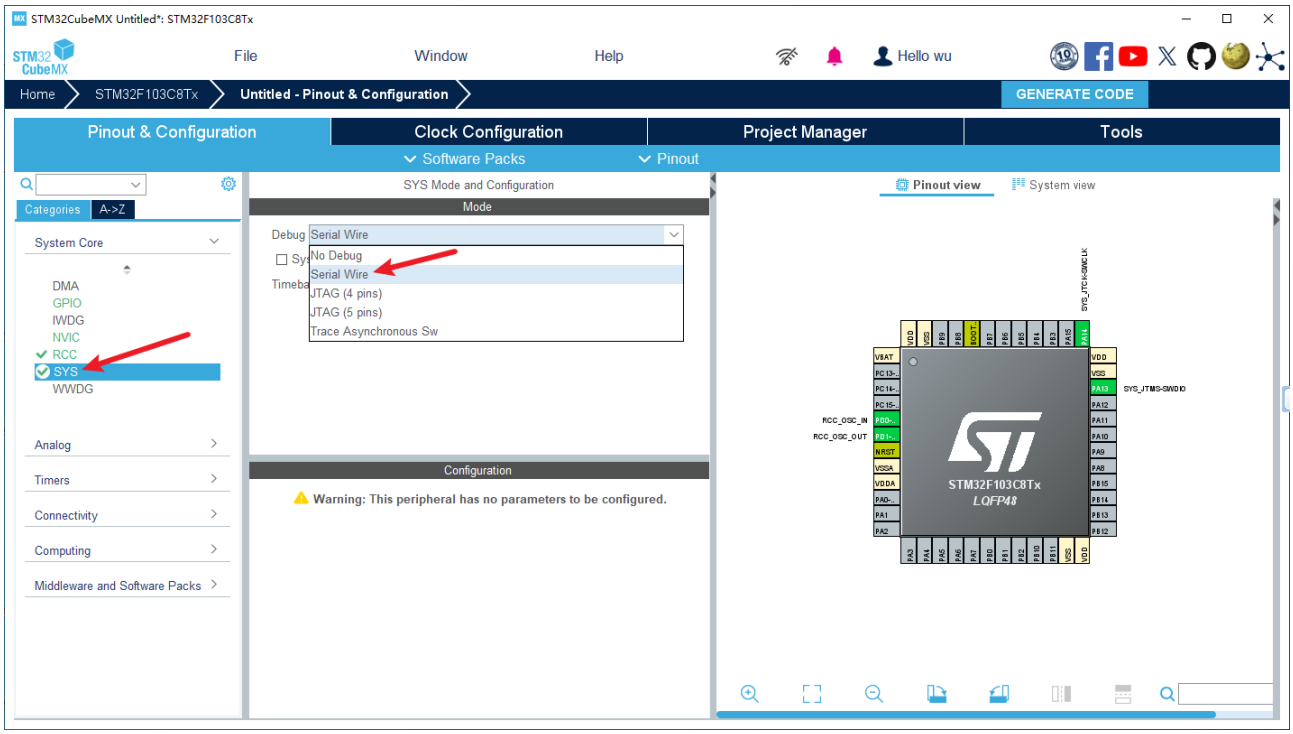
2、选择 stm32f103c8t6，请尽量使用一致的开发板，否则后续的步骤可能会不一致



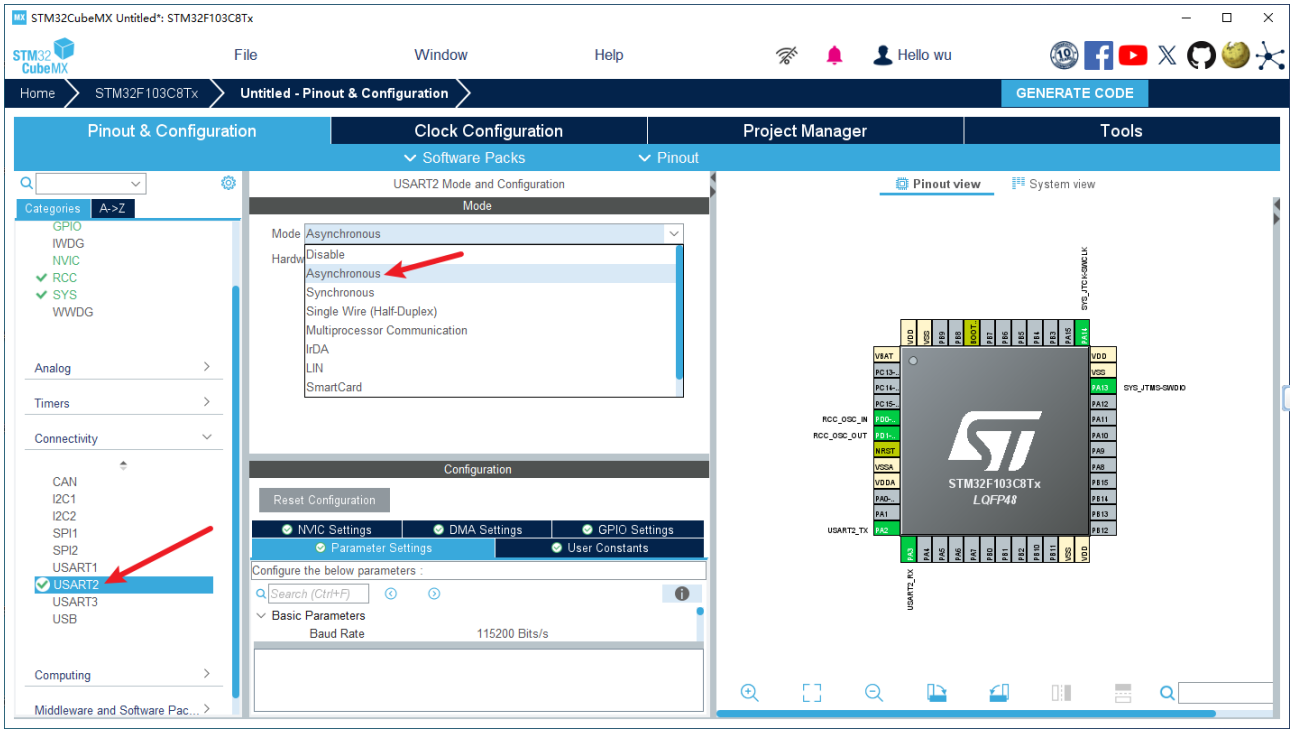
3、点击 RCC，设置外部高速时钟（HSE）



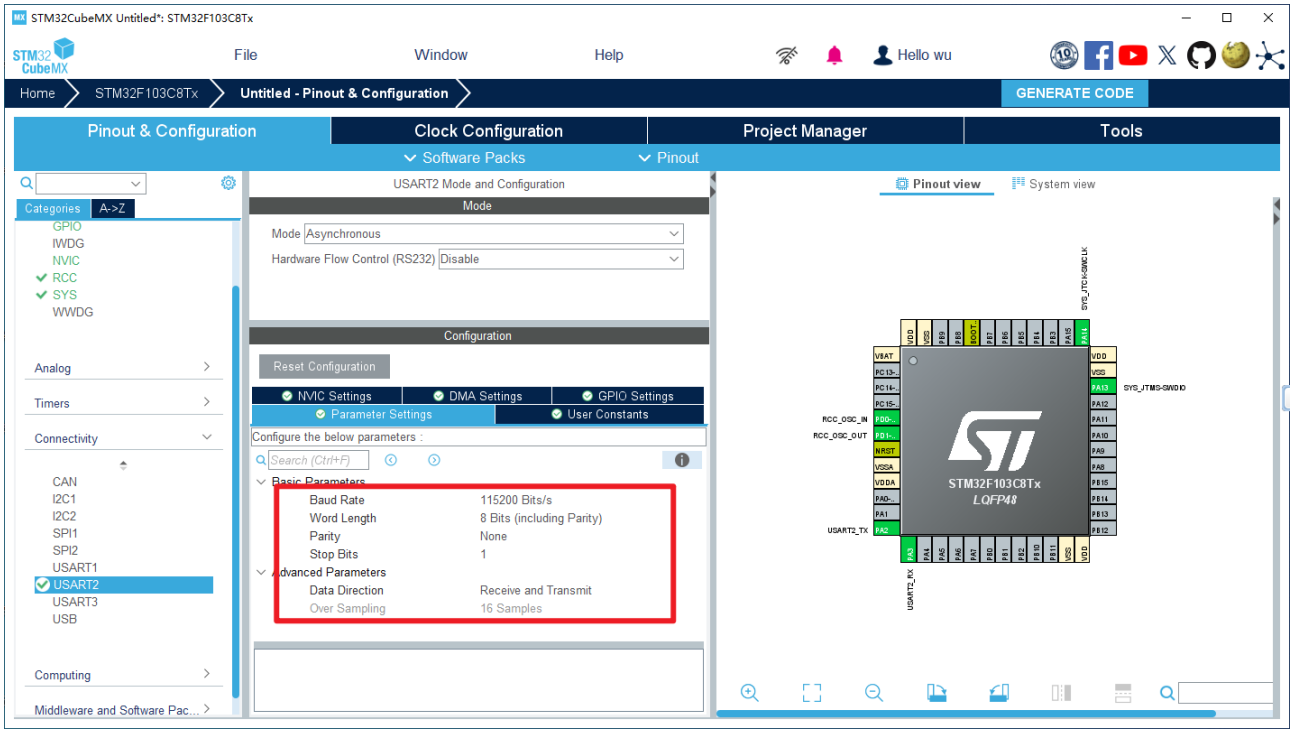
4、点击 SYS，Debug 选择 Serial Wire



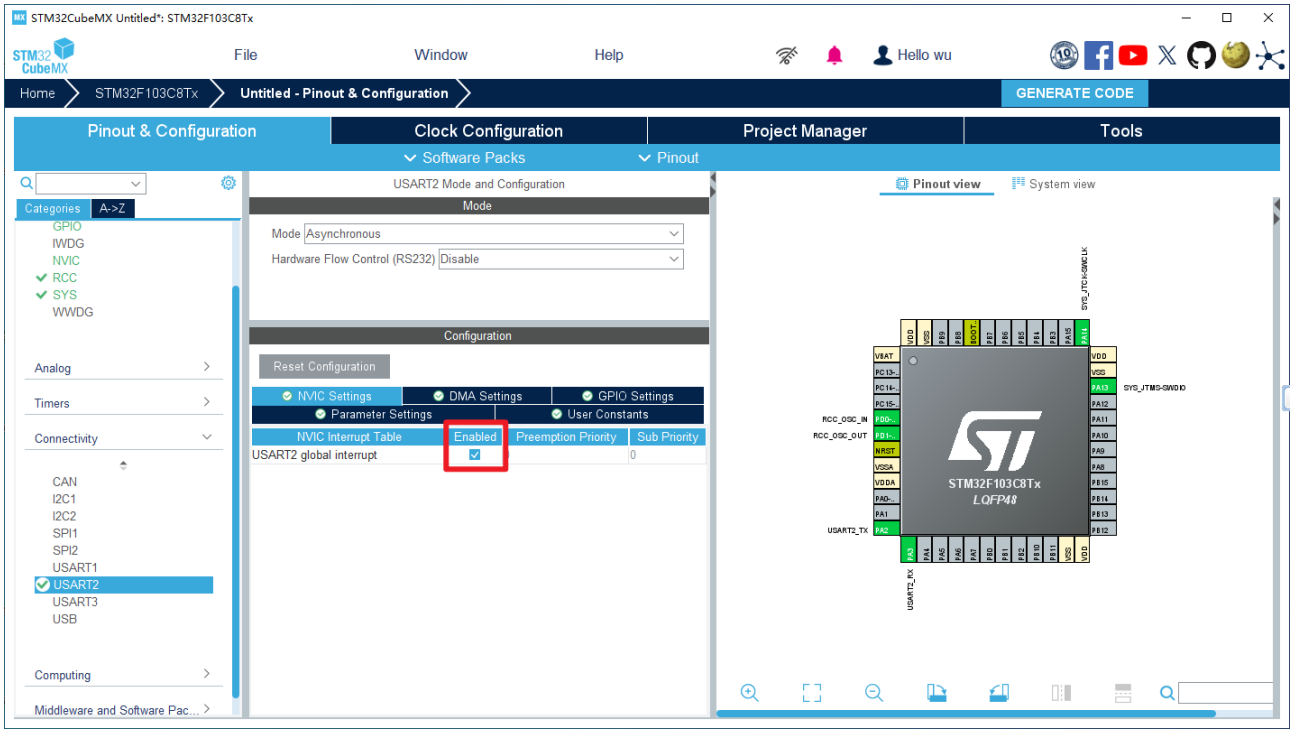
5、USART2 设置为异步（Asynchronous）



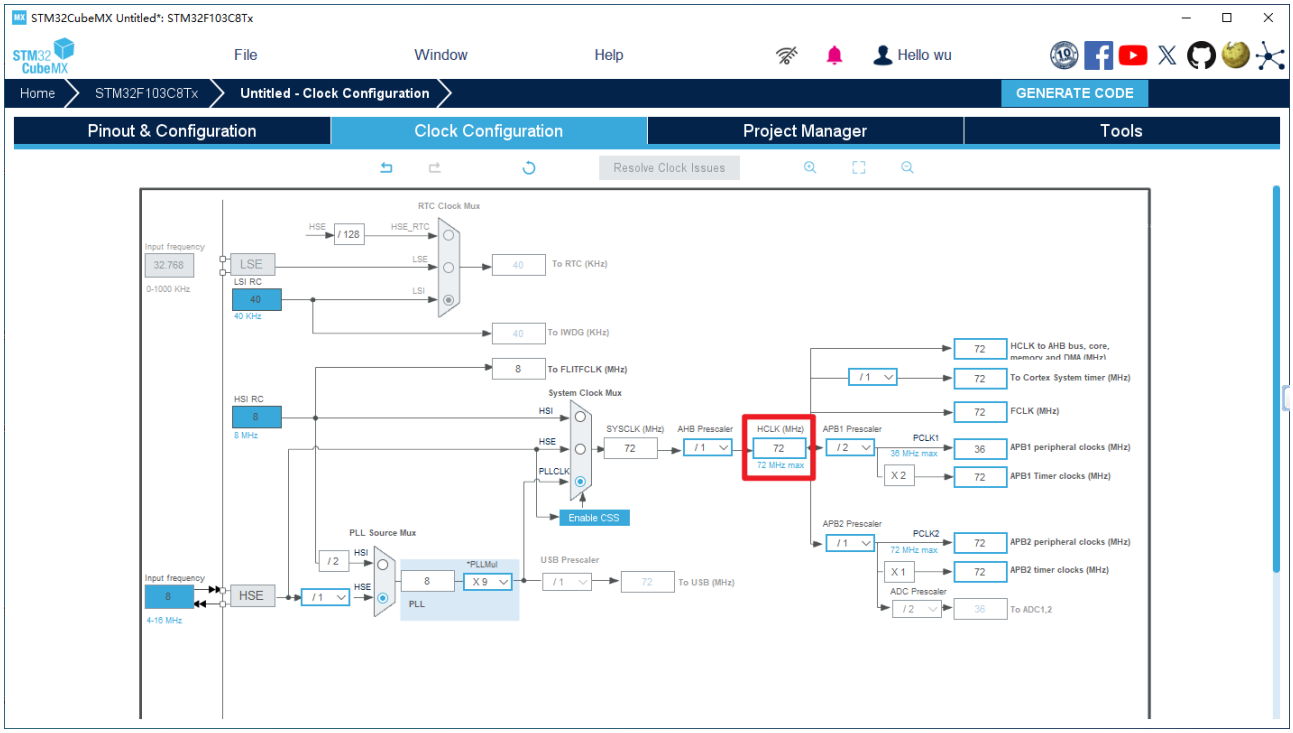
6、波特率和相关的设置使用默认的设置就可以了



7、在 NVIC Settings 里开启 USART2 的中断



8、切换到时钟设置页面，将这里设置为 72，会弹出弹窗，一路同意即可

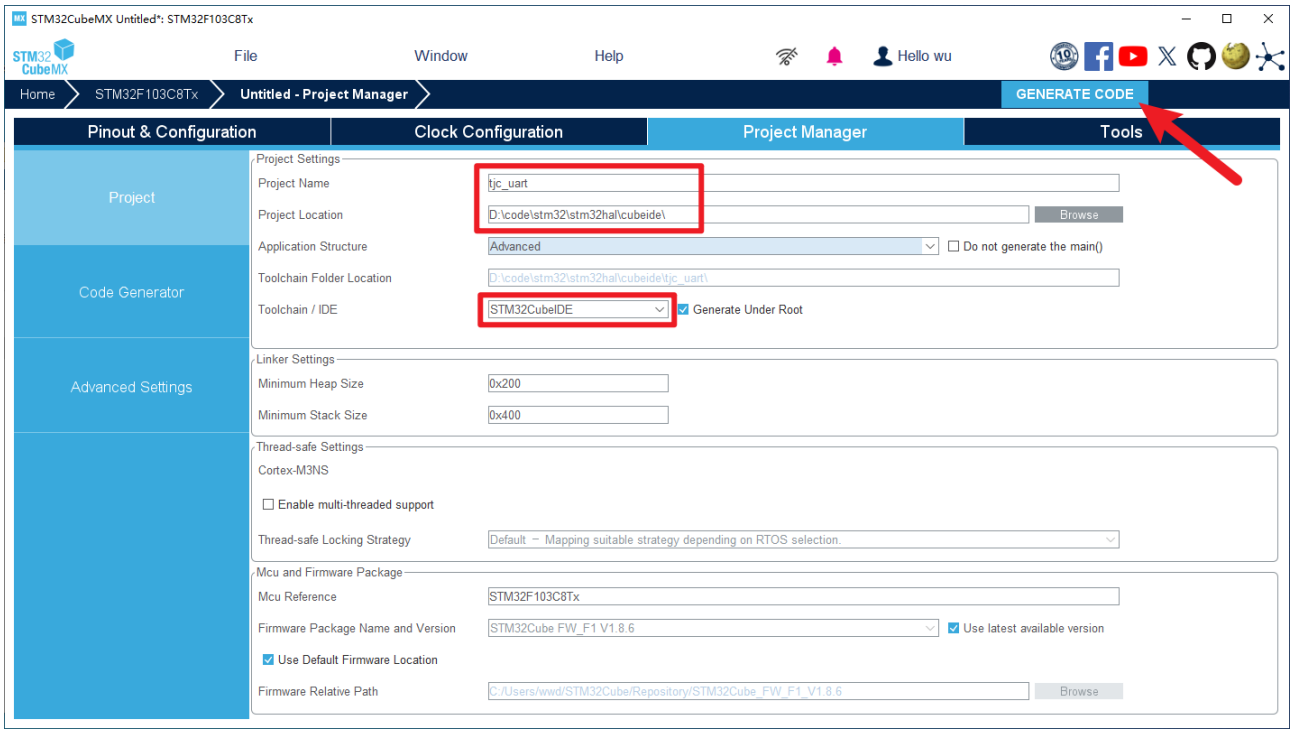


后续使用 stm32cubeide 开发请参考使用 *stm32cubeide(HAL 库) 开发* (page 279)

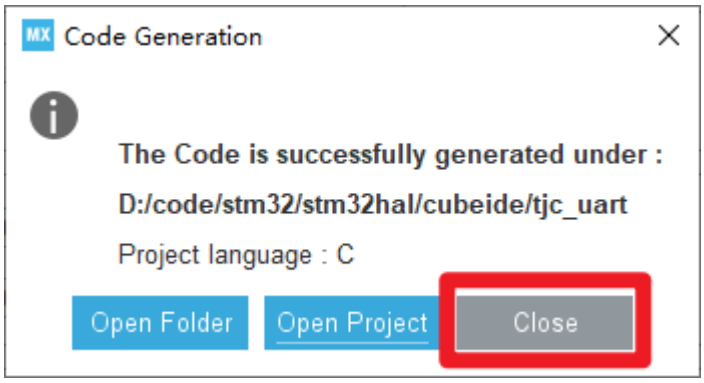
后续使用 keil 开发请参考使用 *keil5（MDK）开发* (page 293)

2.10.2.2 使用 stm32cubeide(HAL 库) 开发

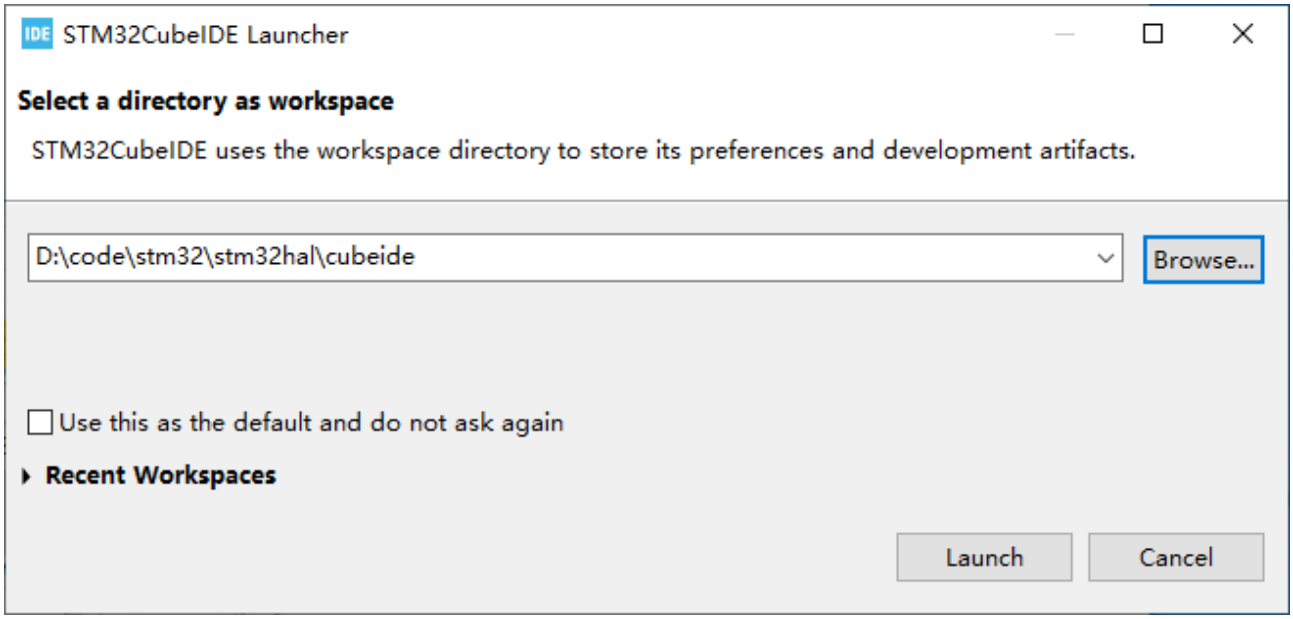
1、切换到 Project Manager 界面，设置 Project Name（工程名称）和 Project Location（工程路径），工程路径需要记下来，等会需要用到，最后点击 GENERATE CODE 来生成代码



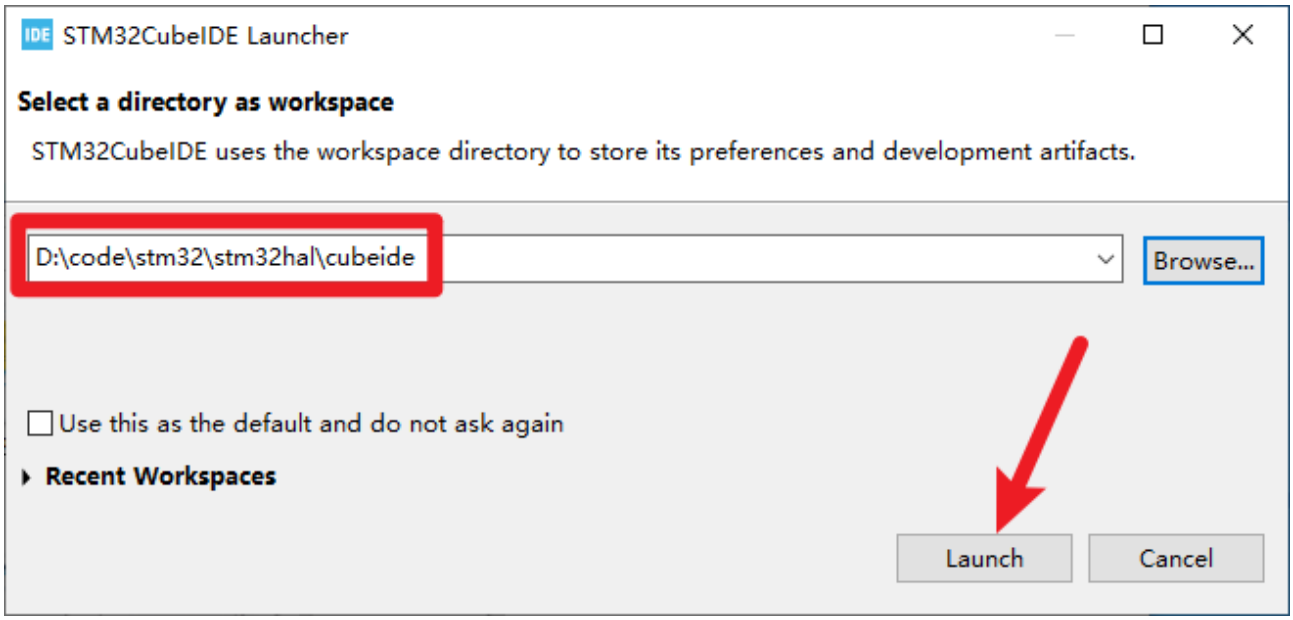
2、不要直接打开工程，点击关闭



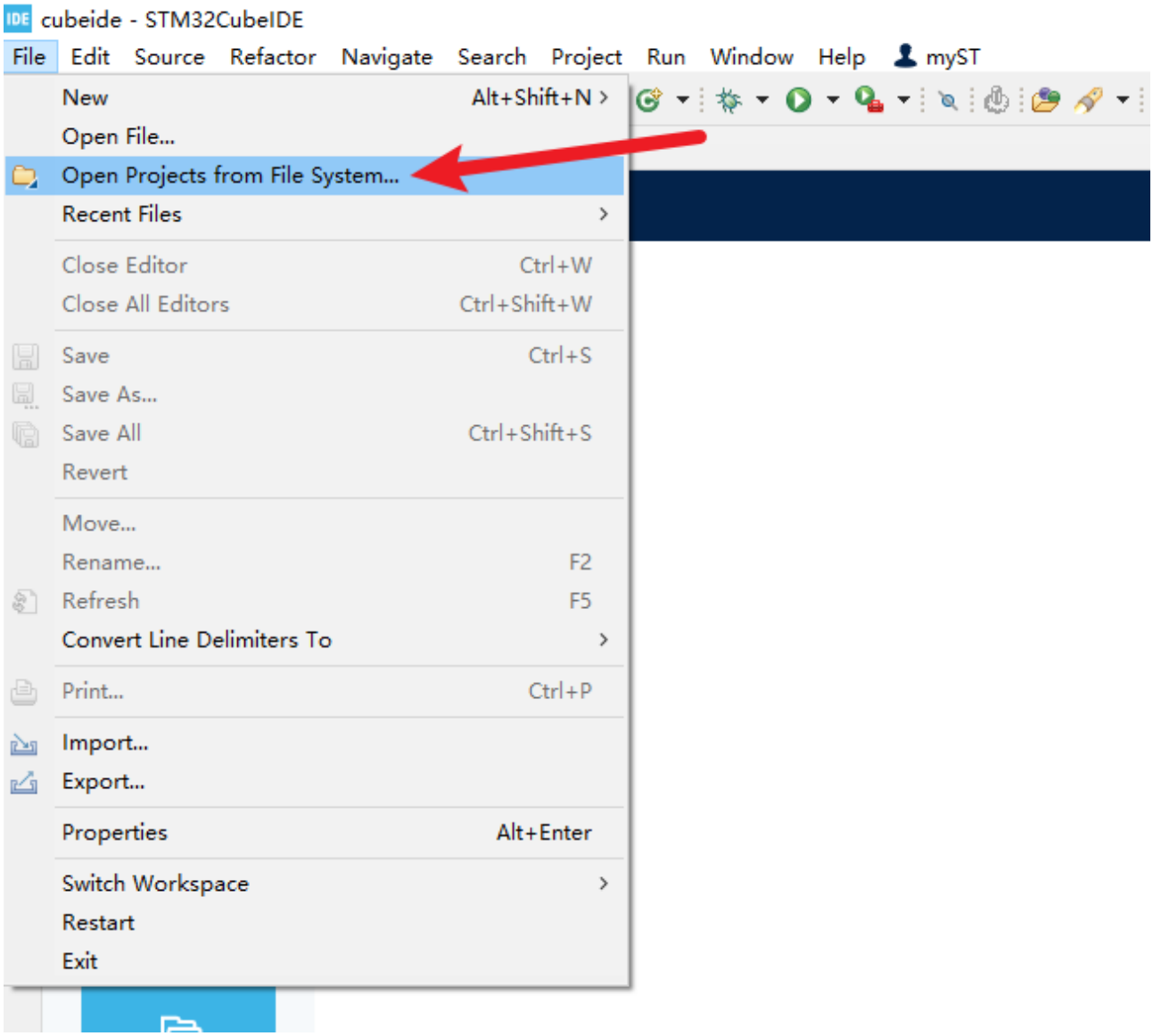
3、打开 stm32cubeide，设置工作空间。



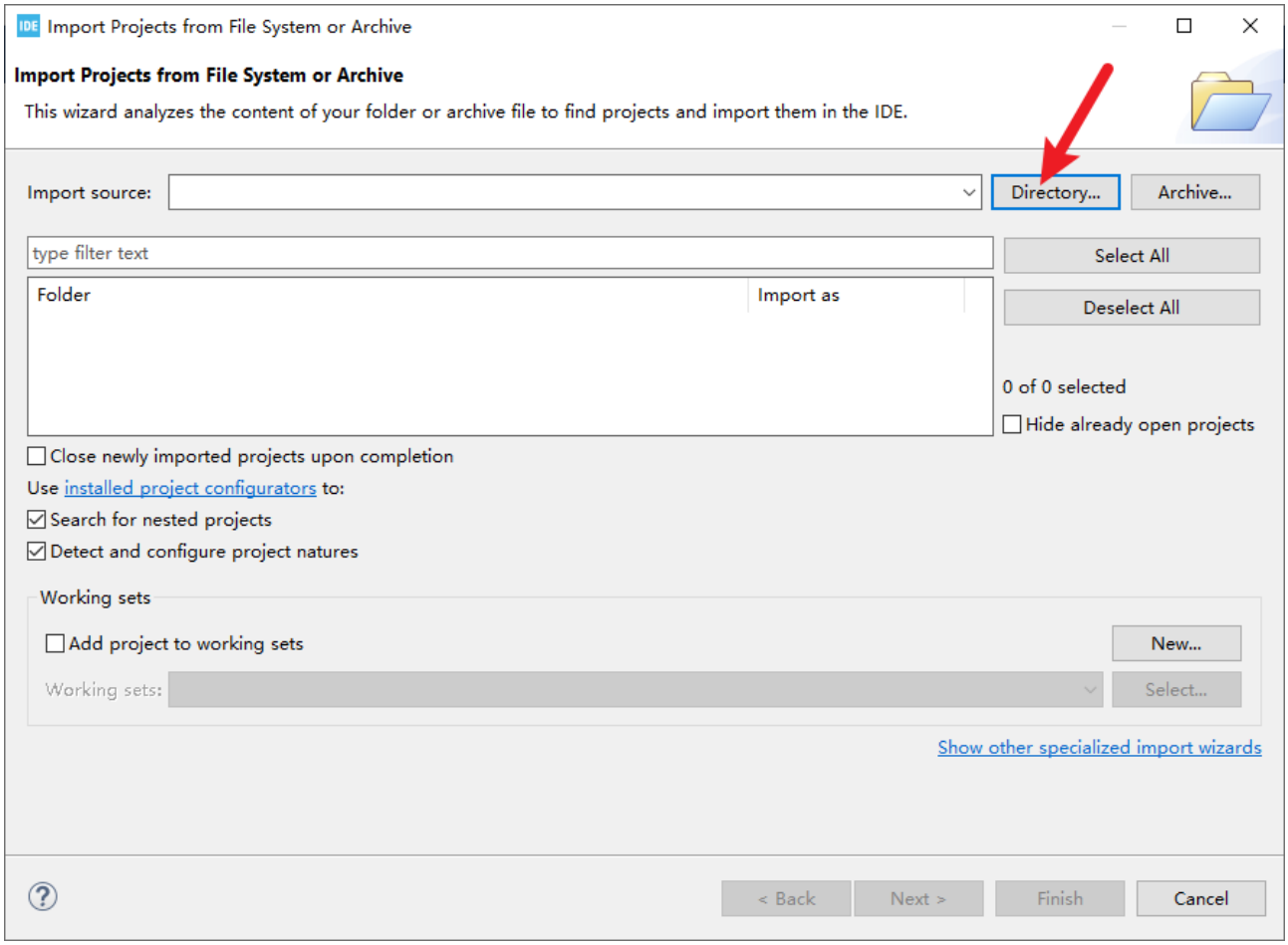
4、工作空间路径就是刚刚记下来的工程路径，点击 launch 打开



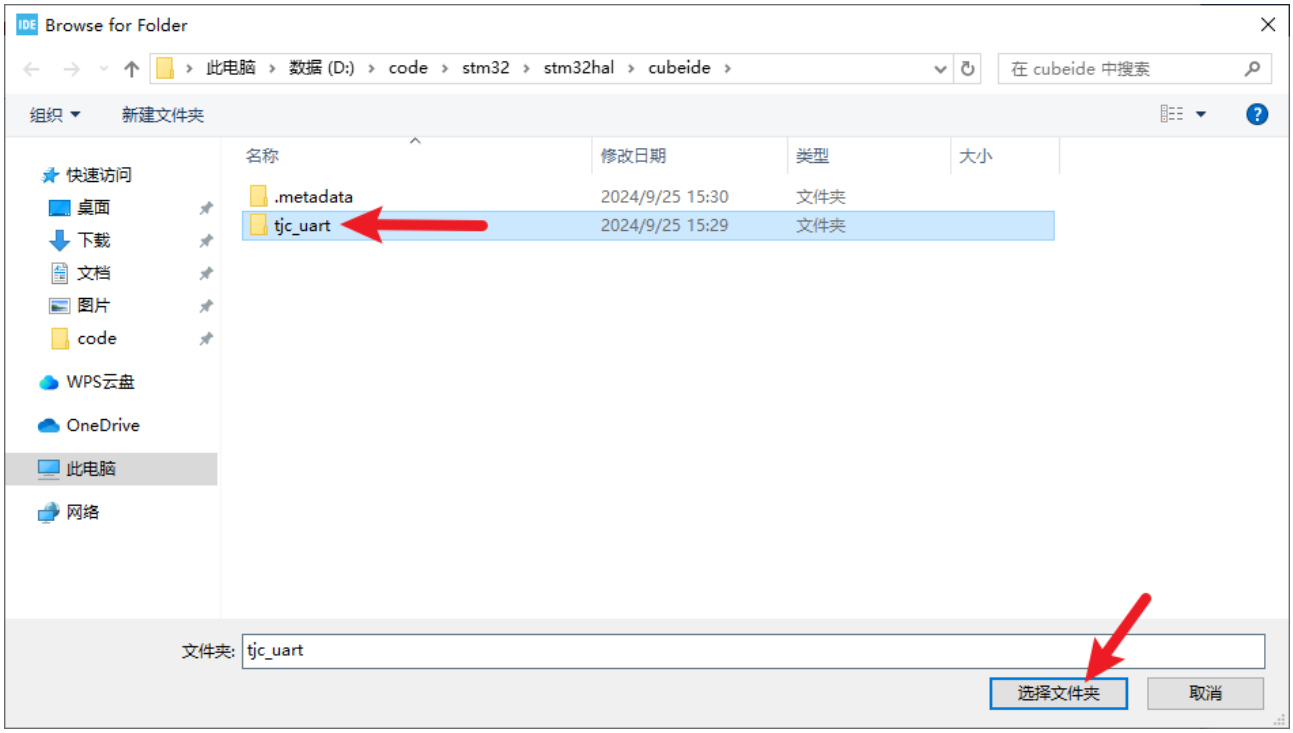
5、打开工程



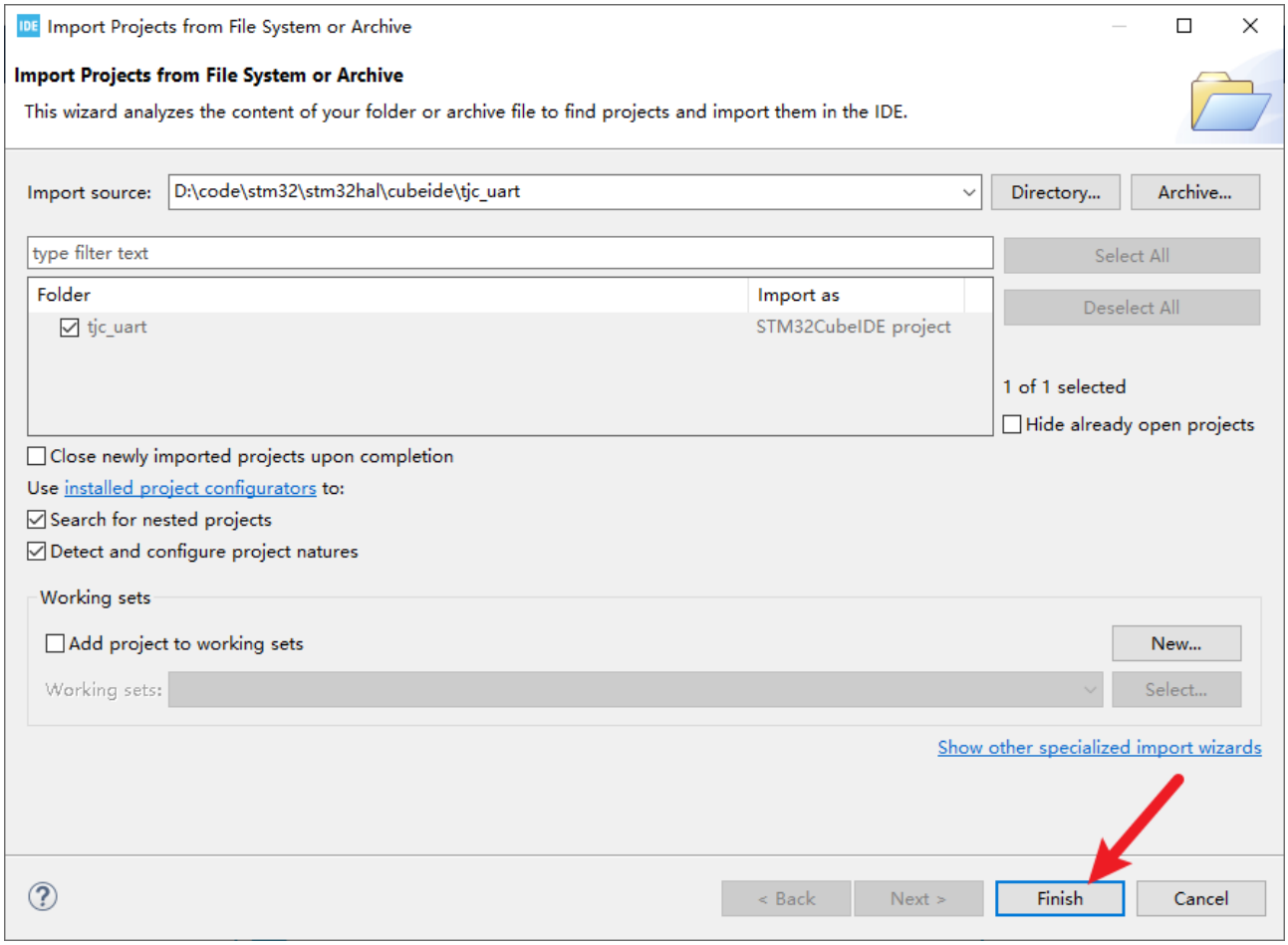
6、点击 Directory，浏览文件夹



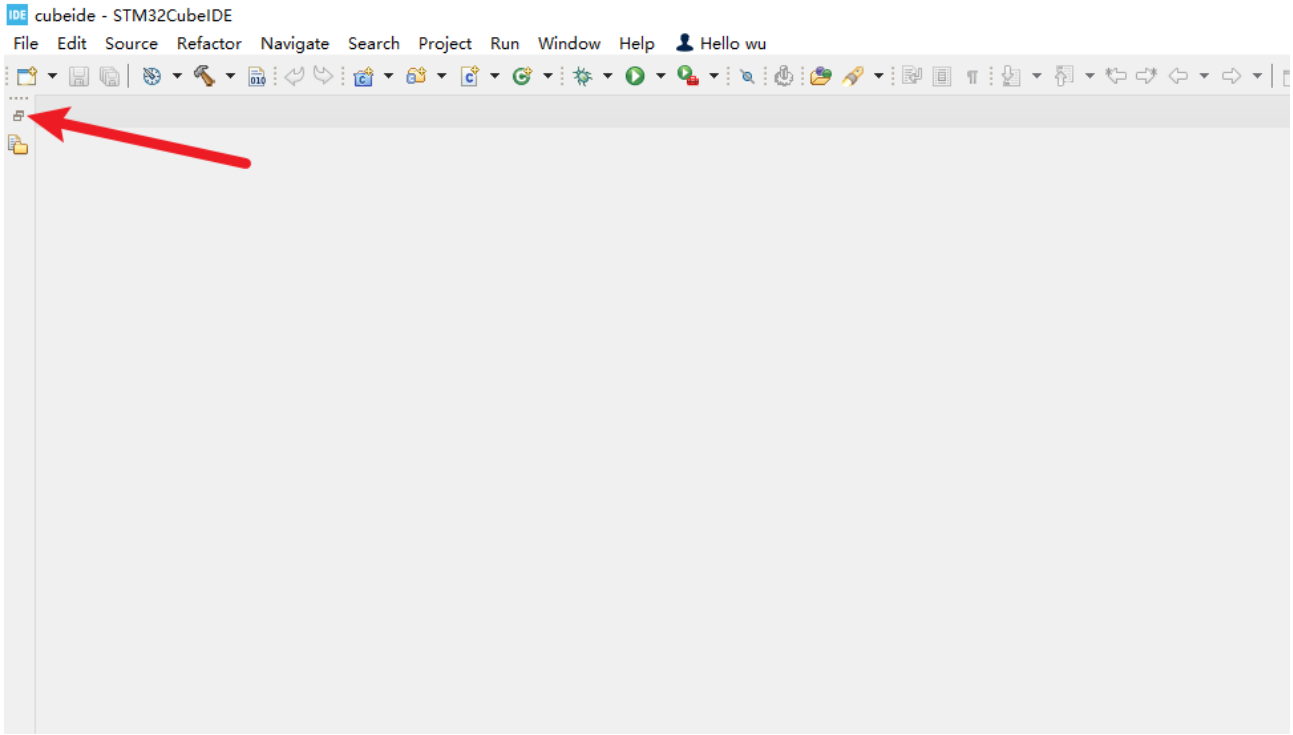
7、找到刚刚生成的工程，点击选择文件夹



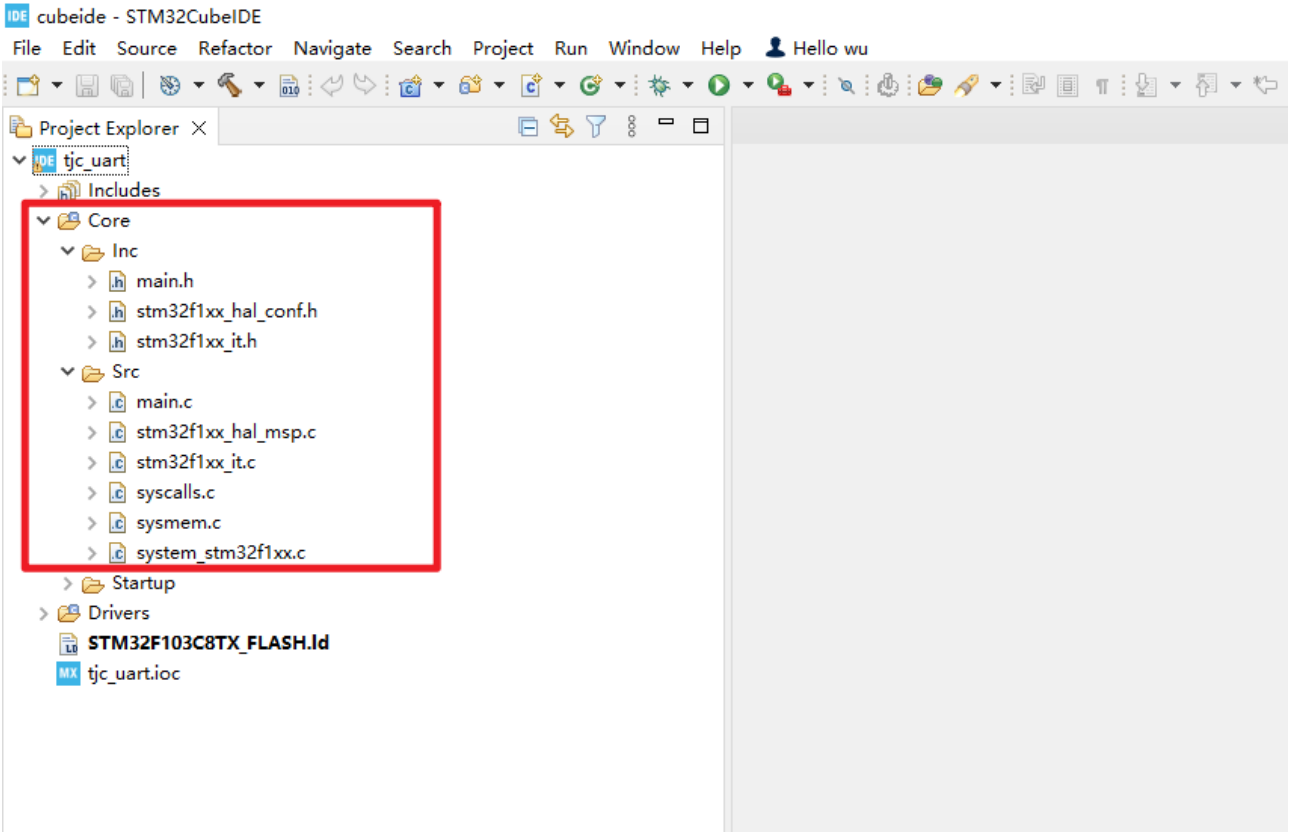
8、点击 finish 就可以打开工程了



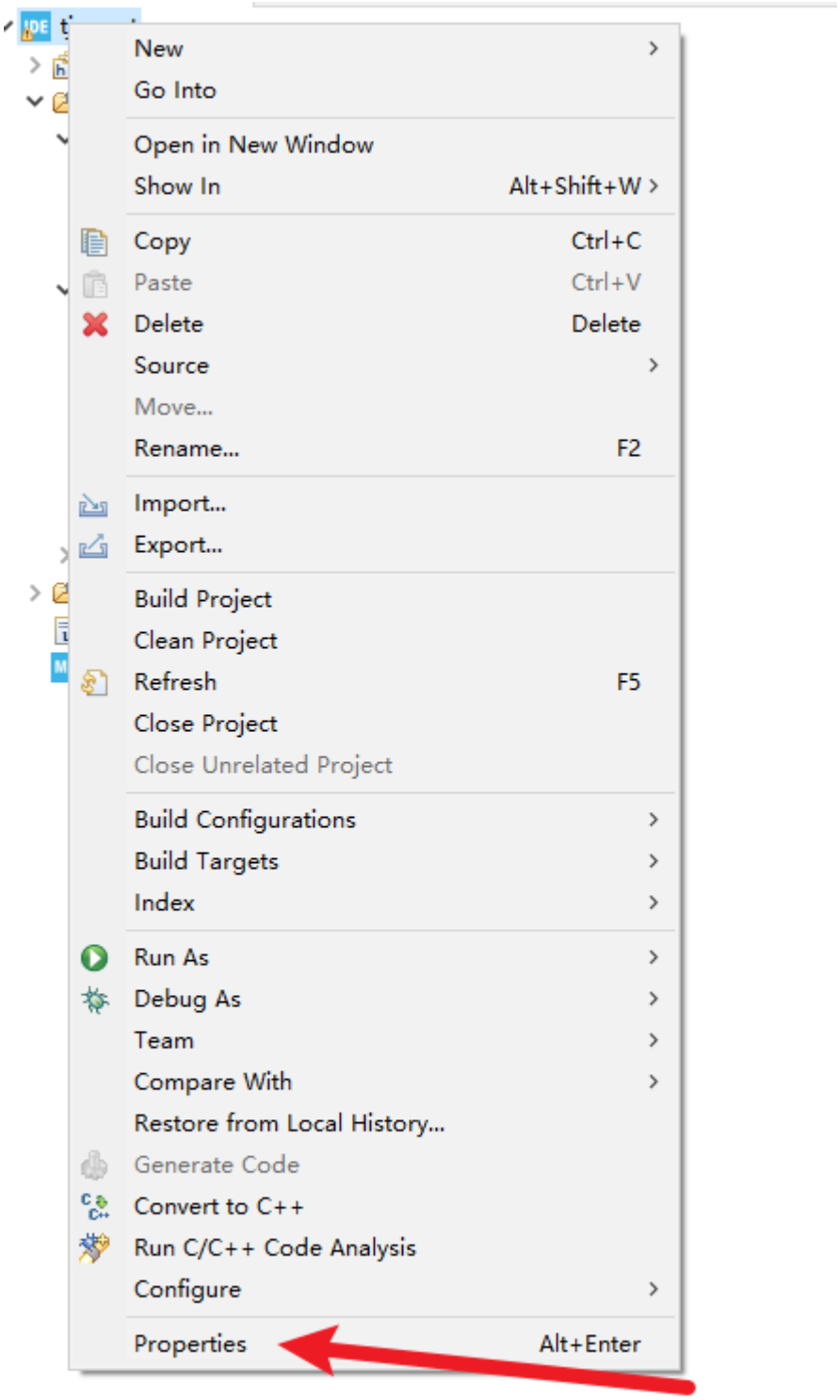
9、点击左上角的目录可以查看整个工程目录



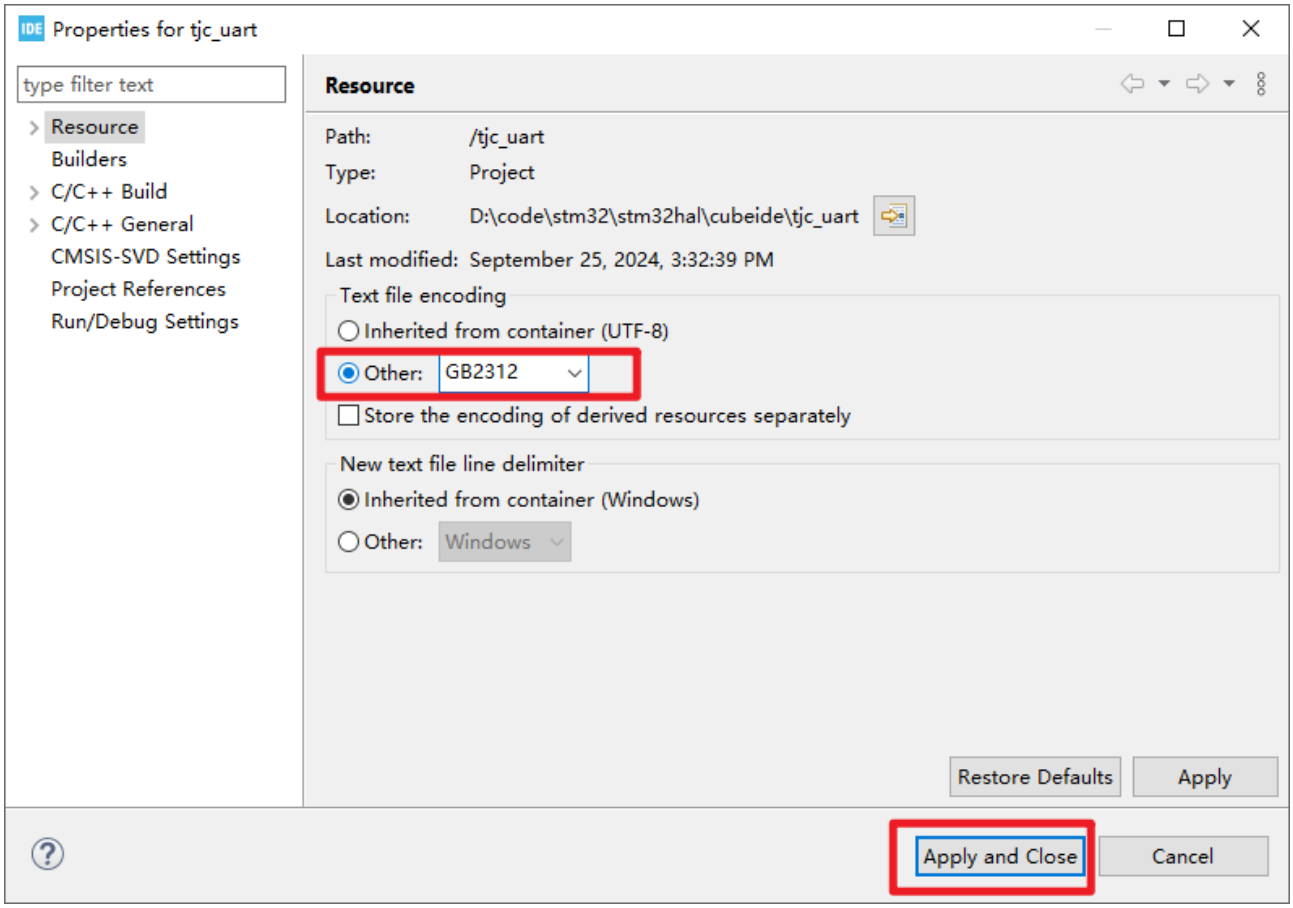
10、Core 里面的 Inc 和 Src 里包含的就是平时经常用到的代码



11、生成代码后请第一时间修改编码（现在不设置，后期修改时中文会乱码）。右键工程，选择 Properties。

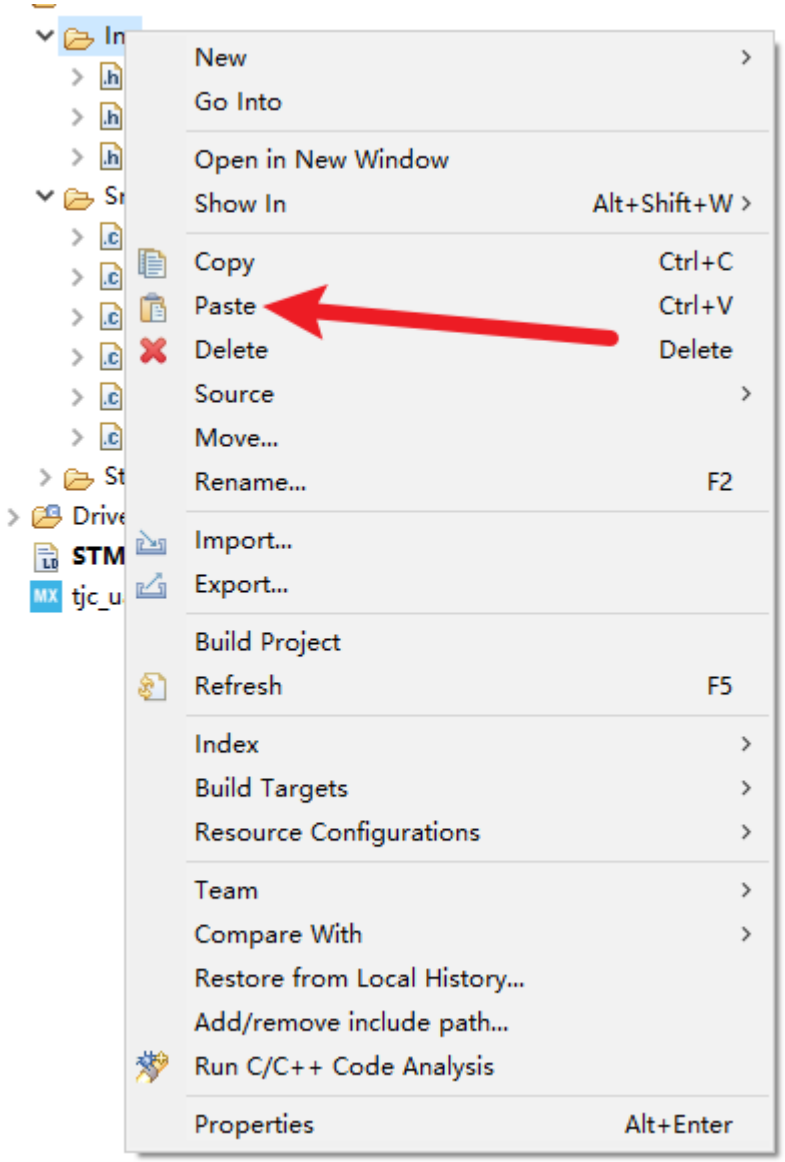


12、在这里就可以设置工程的编码了，根据自己的需求选择合适的编码，这里的编码要和串口屏的编码一致。某些编码不存在，可以手动输入，比如 GB2312，输入完成后点击 Apply and Close 完成设置。

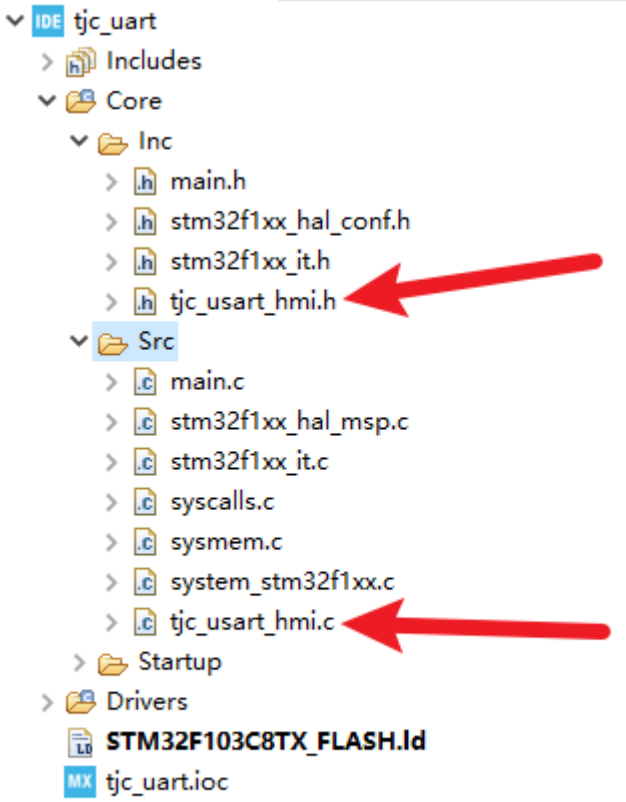


13、将 tjc_usart_hmi.c 复制到 Src 文件夹，将 tjc_usart_hmi.h 复制到 Inc 文件夹（这两个文件会在的工程里提供，需要自行从工程里找到然后导入客户自己的工程中）

下载地址下载《stm32 与串口屏双向通讯》例程 (page 300)



14、复制完成后如图所示



15、切换到 main.c, 在 USER CODE BEGIN Includes 和 USER CODE END Includes 之间插入代码

在 USER CODE BEGIN 2 和 USER CODE END 2 之间初始化环形缓冲区和打开串口接收中断

如有疑问请参考：*为什么要在 begin 和 end 之间插入代码* (page 292)

*main.c

```c
/* USER CODE BEGIN Header */
/**
  ******************************************************************************
  * @file           : main.c
  * @brief          : Main program body
  ******************************************************************************
  * @attention
  *
  * Copyright (c) 2024 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *
  ******************************************************************************
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include "tjc_usart_hmi.h"
#define FRAME_LENGTH 7
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -------------------------------------------------------------*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------*/
UART_HandleTypeDef huart2;
```

*main.c

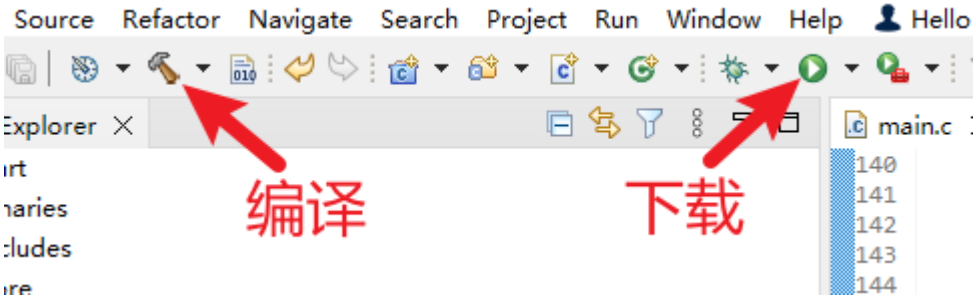
```c
  /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
  HAL_Init();

  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */
  SystemClock_Config();

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_USART2_UART_Init();
  /* USER CODE BEGIN 2 */
    initRingBuffer();       //初始化环形缓冲区
    HAL_UART_Receive_IT(&TJC_UART, RxBuffer, 1);    //打开串口接收中断
    int a = 100;
    char str[100];
    uint32_t nowtime = HAL_GetTick();
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}

/**
  * @brief System Clock Configuration
  * @retval None
  */
void SystemClock_Config(void)
{
  RCC_OscInitTypeDef RCC_OscInitStruct = {0};
  RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
```

16、在 while 循环中编写定时发送和解析的代码

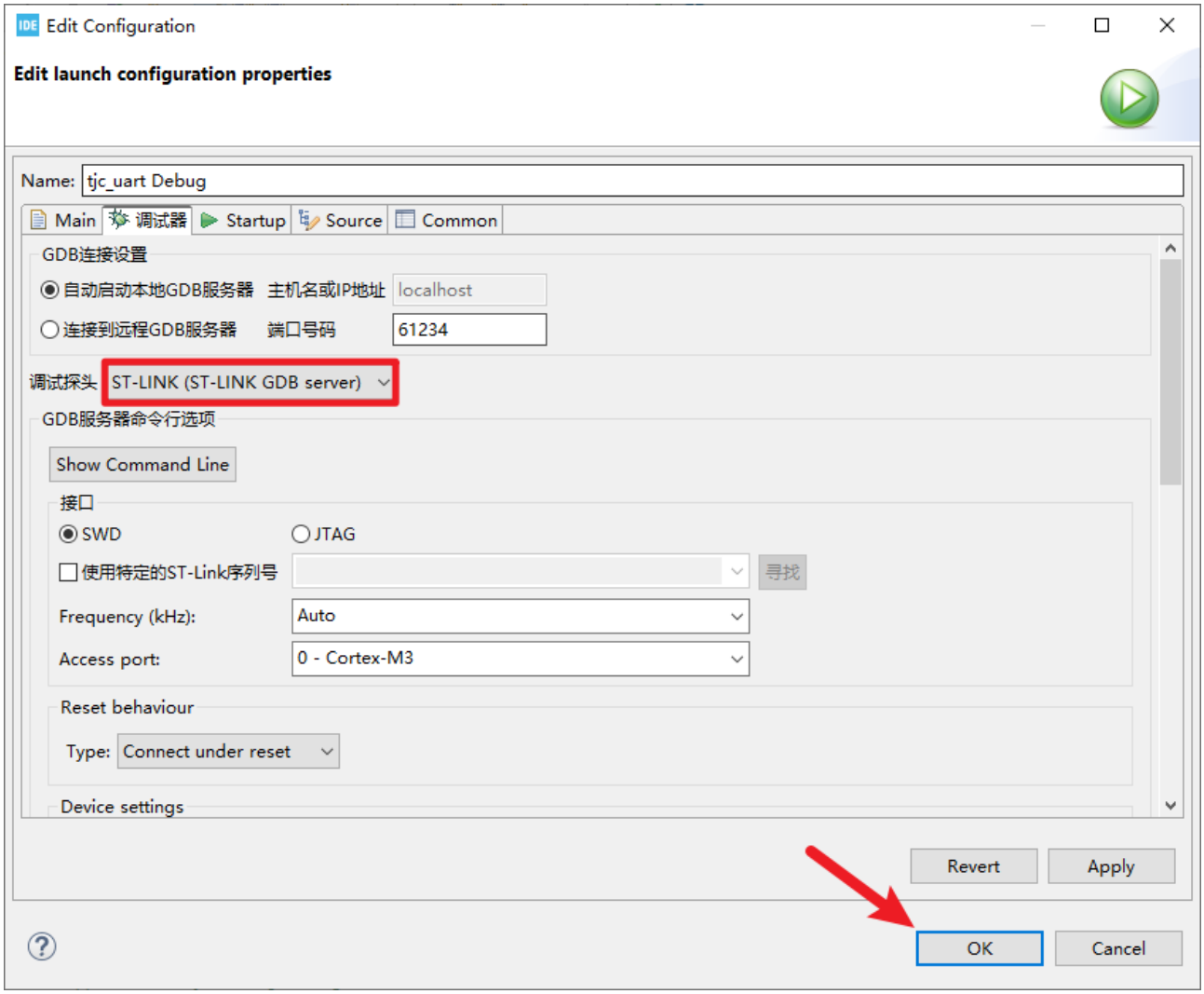
main.c

```
    uint32_t nowtime = HAL_GetTick();
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
    while (1)
    {
        if (HAL_GetTick() - nowtime >= 1000)
        {
            nowtime = HAL_GetTick();

            sprintf(str, "n0.val=%d", a);
            tjc_send_string(str);
            sprintf(str, "t0.txt=\"%d\"\xff\xff\xff", a);
            tjc_send_string(str);
            sprintf(str, "click b0,1\xff\xff\xff");
            tjc_send_string(str);
            HAL_Delay(50);
            sprintf(str, "click b0,0\xff\xff\xff");
            tjc_send_string(str);

            a++;
        }
        /* USER CODE END WHILE */

        /* USER CODE BEGIN 3 */

        //stm32f103的GND接串口屏或串口工具的GND,共地
        //stm32f103的TX2(PA2)接串口屏或串口工具的RX
        //stm32f103的RX2(PA3)接串口屏或串口工具的TX
        //stm32f103的5V接串口屏的5V,如果是串口工具,不用接5V也可以
        //串口数据格式:
        //串口数据帧长度:7字节
        //帧头      参数1     参数2    参数3        帧尾
        //0x55      1字节     1字节     1字节      0xffffff
        //当参数是01时
        //帧头      参数1     参数2    参数3        帧尾
        //0x55       01      led编号   led状态     0xffffff
        //例子1:上位机代码   printh 55 01 01 00 ff ff ff   含义:1号led关闭
        //例子2:上位机代码   printh 55 01 04 01 ff ff ff   含义:4号led打开
        //例子3:上位机代码   printh 55 01 00 01 ff ff ff   含义:0号led打开
        //例子4:上位机代码   printh 55 01 04 00 ff ff ff   含义:4号led关闭
        //当参数是02或03时
        //帧头      参数1     参数2    参数3        帧尾
        //0x55      02/03    滑动值     00     0xffffff
        //例子1:上位机代码   printh 55 02 64 00 ff ff ff   含义:h0.val=100
        //例子2:上位机代码   printh 55 02 00 00 ff ff ff   含义:h0.val=0
        //例子3:上位机代码   printh 55 03 64 00 ff ff ff   含义:h1.val=100
        //例子4:上位机代码   printh 55 03 00 00 ff ff ff   含义:h1.val=0
```

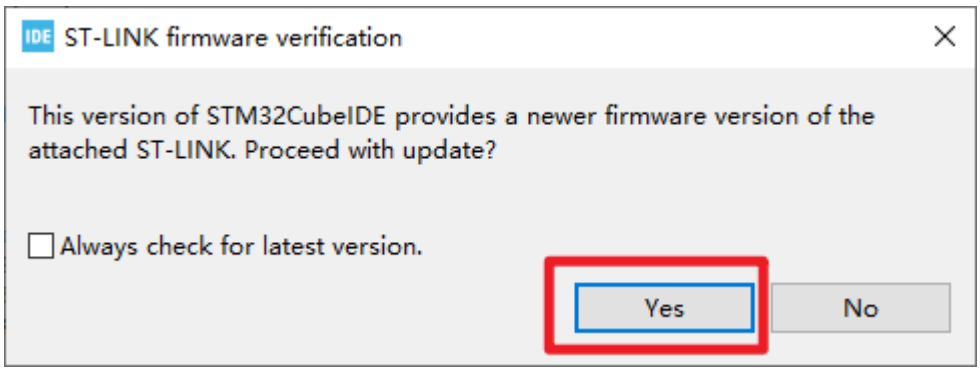
17、这两个按钮用于编译和下载



18、配置你的调试器



19、根据你自己的需求选择，第一次下载时建议更新一下 stlink 的固件



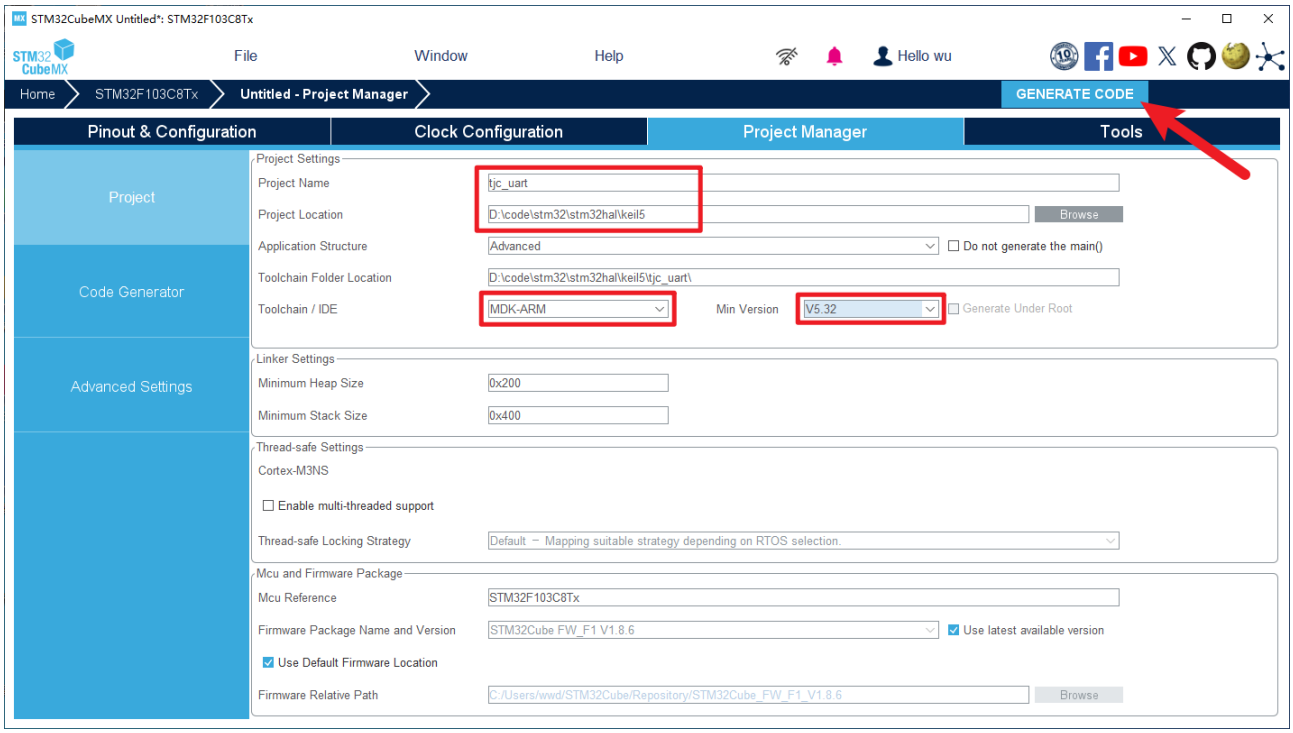
2.10.2.2.1 为什么要在 begin 和 end 之间插入代码

如果不是在 begin 和 end 之间插入的代码，在图形界面重新配置了 STM32 的功能并使用 Generate Code 生成代码时，会导致代码丢失

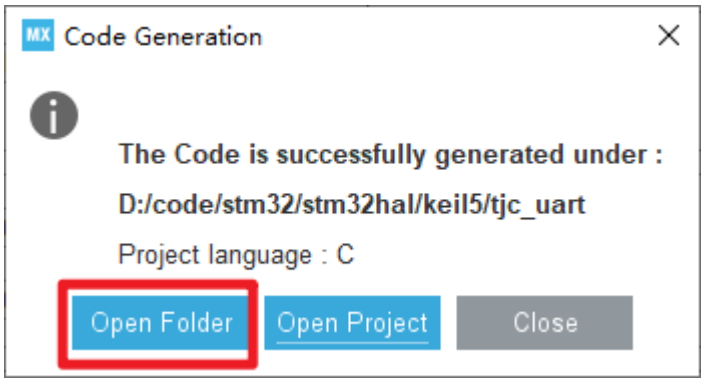
而写在 begin 和 end 之间的代码则不会丢失

2.10.2.3 使用 keil5（MDK）开发

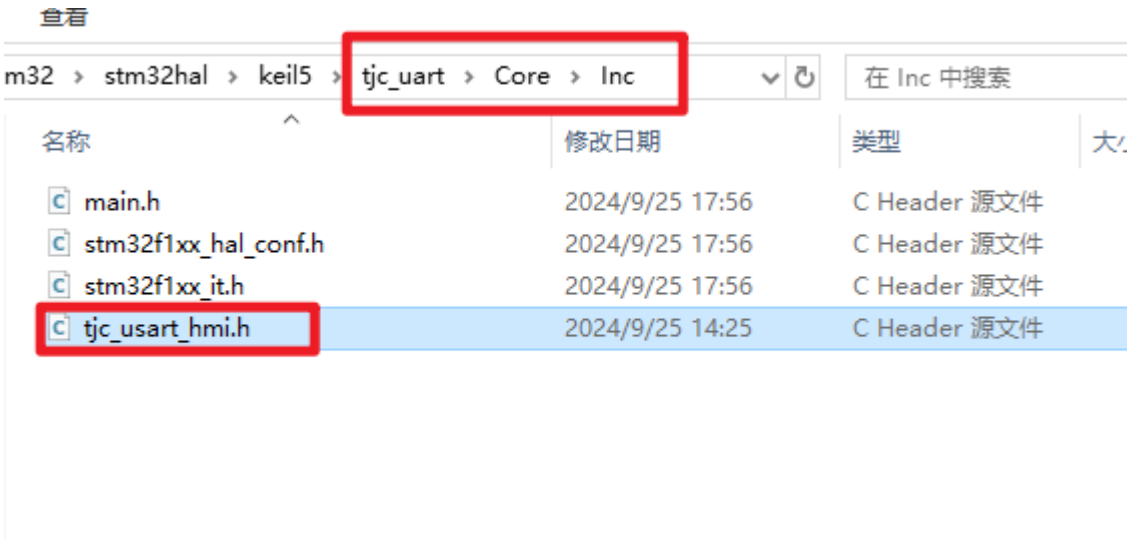
1、切换到 Project Manager 界面，设置 Project Name（工程名称）和 Project Location（工程路径），最后点击 GENERATE CODE 来生成代码。



2、点击打开文件夹

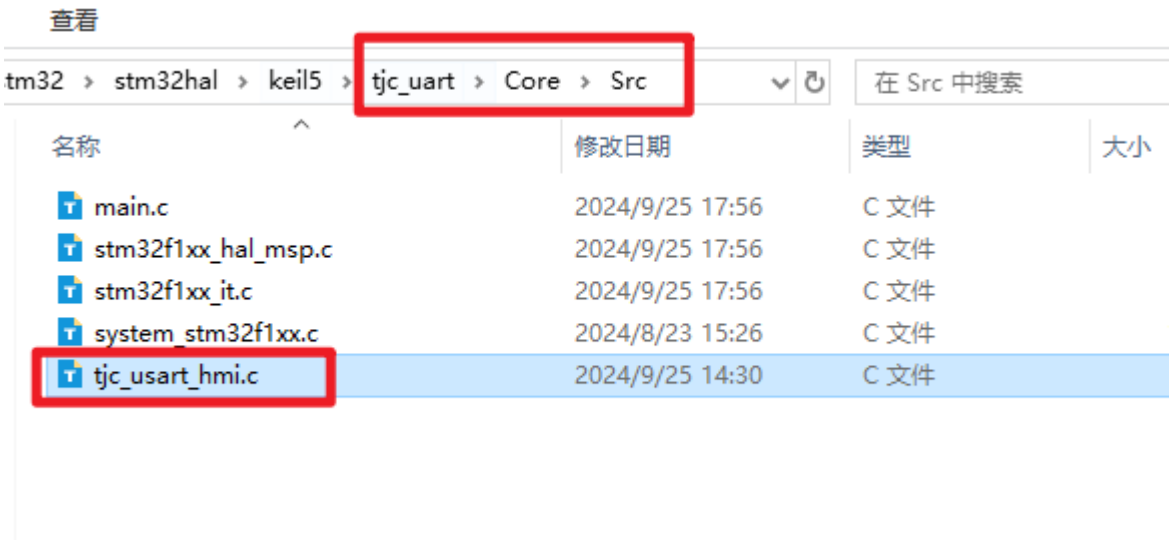


3、将淘晶驰串口屏头文件复制到这个目录下

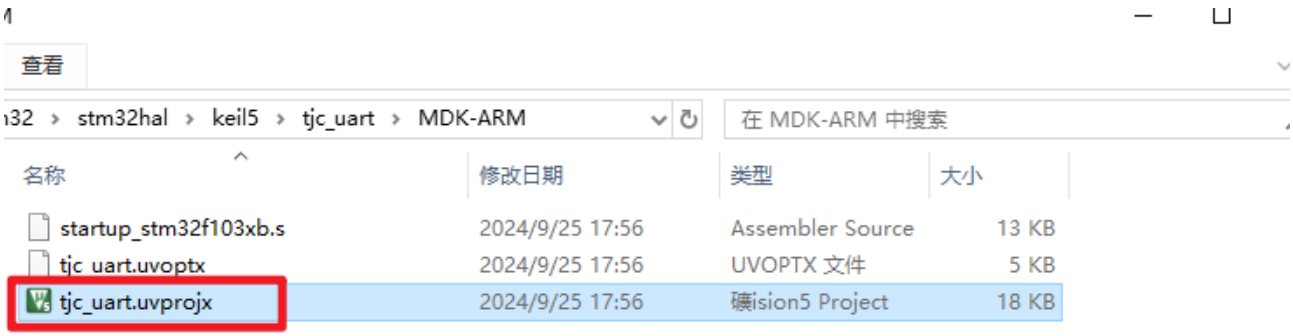


4、将淘晶驰串口屏源文件复制到这个目录下（这两个文件会在工程里提供，需要自行从工程里找到然后导入客户自己的工程中）

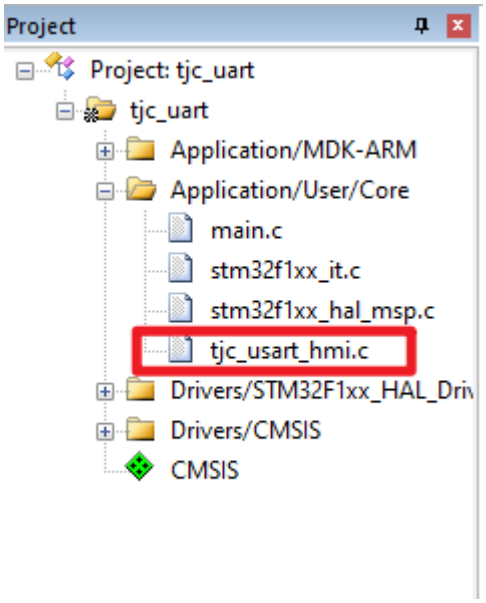
下载地址下载《stm32 与串口屏双向通讯》例程 (page 300)



5、打开工程



6、将刚刚的文件添加到工程文件中



7、添加 include 和 define 定义

```
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------*/
#include "main.h"

/* Private includes ----------------------------------------------------------*/
/* USER CODE BEGIN Includes */
#include "tjc_usart_hmi.h"
#define FRAME_LENGTH 7
/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

```

8、添加初始化和变量定义

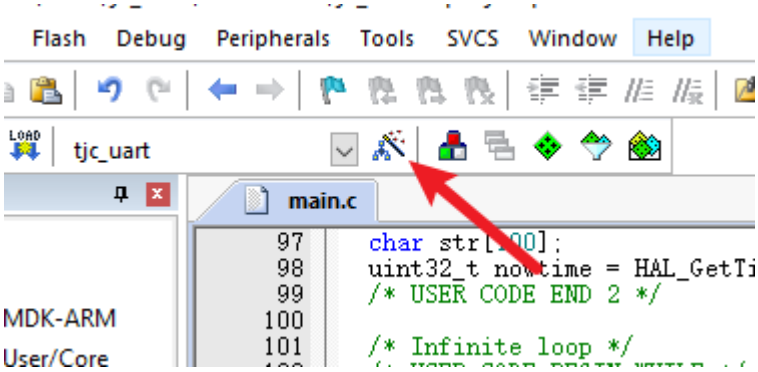
```
  /* Initialize all configured peripherals */
  MX_GPIO_Init();
  MX_USART2_UART_Init();
  /* USER CODE BEGIN 2 */
  initRingBuffer();   //初始化环形缓冲区
  HAL_UART_Receive_IT(&TJC_UART, RxBuffer, 1);  //打开串口接收中断
  int a = 100;
  char str[100];
  uint32_t nowtime = HAL_GetTick();
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
```

9、在 while 里添加串口发送和解析的代码

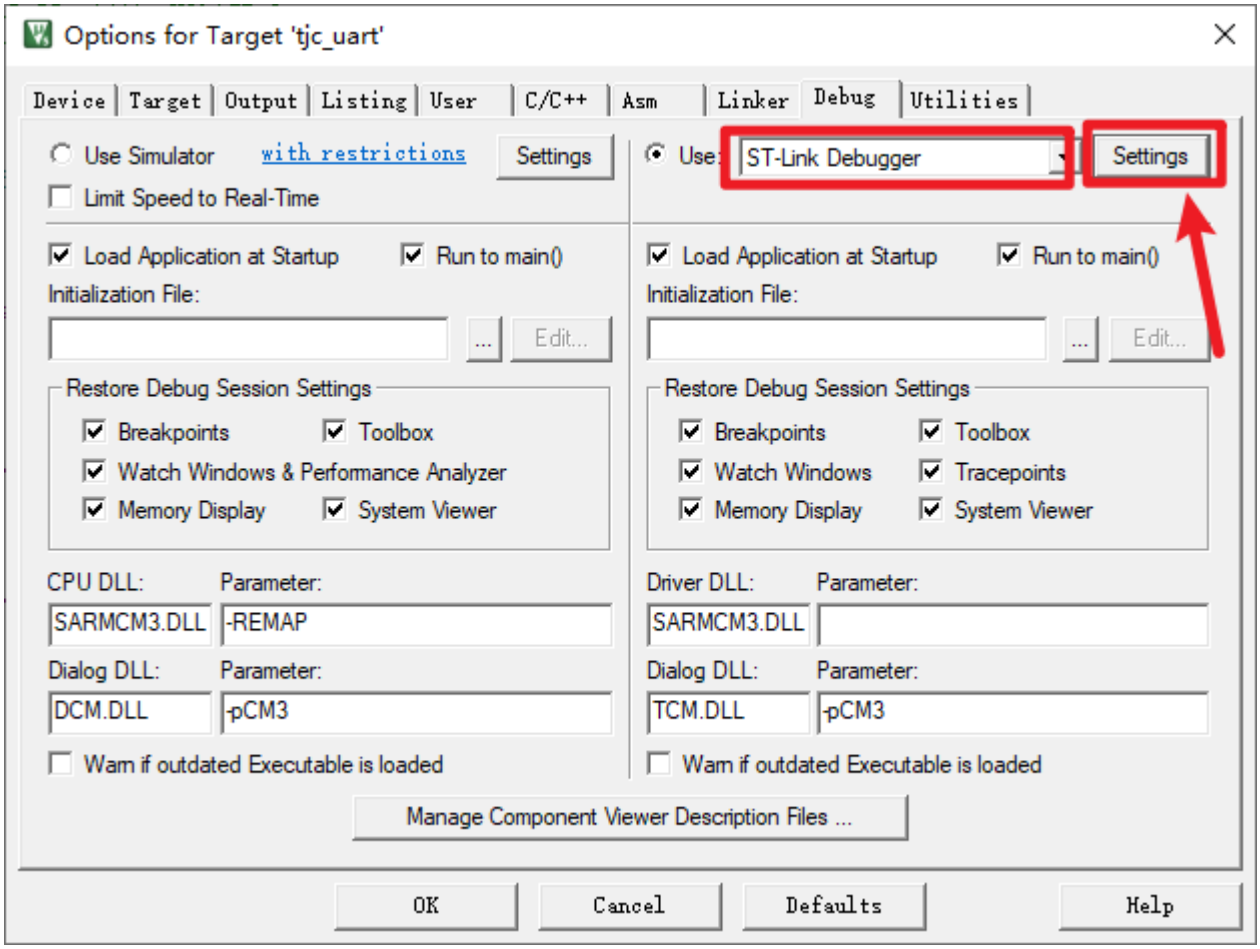
main.c

```
  char str[100];
  uint32_t nowtime = HAL_GetTick();
  /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
while (1)
  {
    if (HAL_GetTick() - nowtime >= 1000)
    {
      nowtime = HAL_GetTick();

      sprintf(str, "n0.val=%d", a);
      tjc_send_string(str);
      sprintf(str, "t0.txt=\"%d\"\xff\xff\xff", a);
      tjc_send_string(str);
      sprintf(str, "click b0,1\xff\xff\xff");
      tjc_send_string(str);
      HAL_Delay(50);
      sprintf(str, "click b0,0\xff\xff\xff");
      tjc_send_string(str);

      a++;
    }
    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */

    //stm32f103的GND接串口屏或串口工具的GND,共地
    //stm32f103的TX2(PA2)接串口屏或串口工具的RX
    //stm32f103的RX2(PA3)接串口屏或串口工具的TX
    //stm32f103的5V接串口屏的5V,如果是串口工具,不用接5V也可以
    //串口数据格式:
    //串口数据帧长度:7字节
    //帧头      参数1     参数2    参数3       帧尾
    //0x55      1字节    1字节     1字节      0xffffff
    //当参数是01时
    //帧头      参数1     参数2    参数3       帧尾
    //0x55      01      led编号   led状态     0xffffff
    //例子1:上位机代码  printh 55 01 01 00 ff ff ff  含义:1号led关闭
    //例子2:上位机代码  printh 55 01 04 01 ff ff ff  含义:4号led打开
    //例子3:上位机代码  printh 55 01 00 01 ff ff ff  含义:0号led打开
    //例子4:上位机代码  printh 55 01 04 00 ff ff ff  含义:4号led关闭
    //当参数是02或03时
    //帧头      参数1     参数2    参数3       帧尾
    //0x55      02/03    滑动值    00     0xffffff
    //例子1:上位机代码  printh 55 02 64 00 ff ff ff  含义:h0.val=100
    //例子2:上位机代码  printh 55 02 00 00 ff ff ff  含义:h0.val=0
    //例子3:上位机代码  printh 55 03 64 00 ff ff ff  含义:h1.val=100
    //例子4:上位机代码  printh 55 03 00 00 ff ff ff  含义:h1.val=0
    // 当串口缓冲区大于等于一帧的长度时
    while (usize >= FRAME_LENGTH)
    {
      // 校验帧头帧尾是否匹配
      if (usize >= FRAME_LENGTH && u(0) == 0x55 && u(4) == 0xff && u(5) == 0xff && u(6) == 0xff)
```

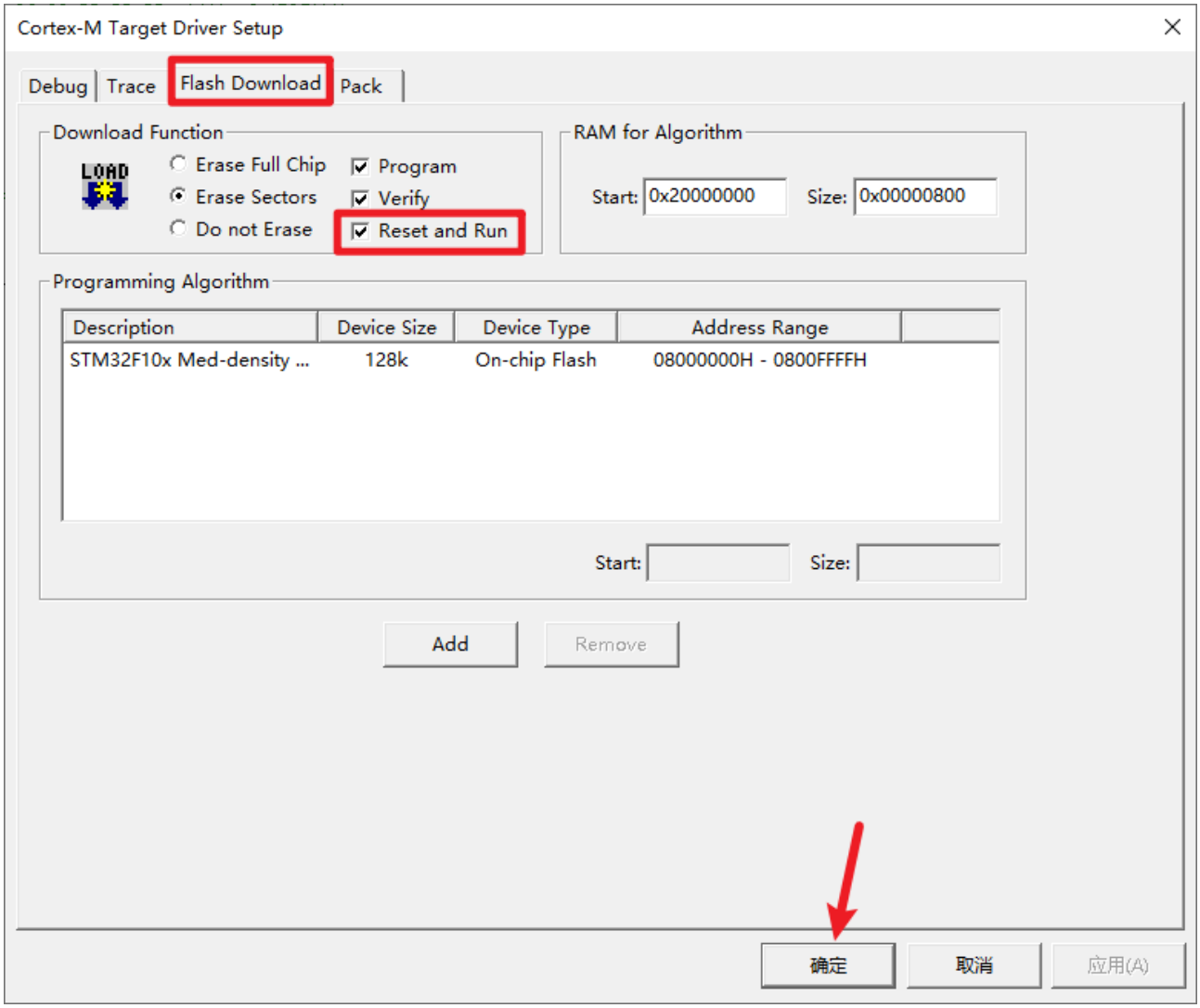
10、点击魔法棒的图标，配置一下调试器



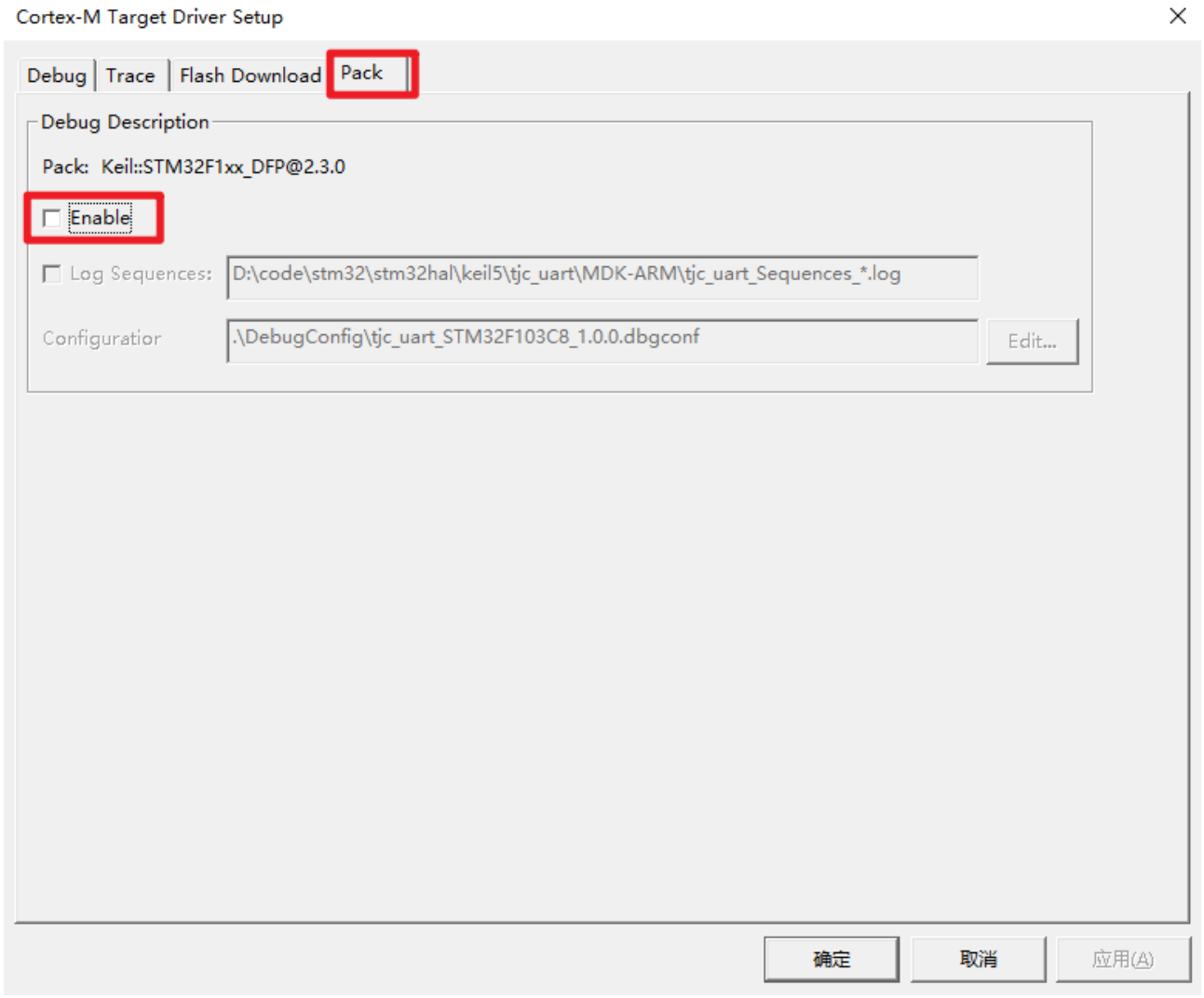
11、选择你的调试器，我这里使用的是 st-link，选择完毕后点击 Settings



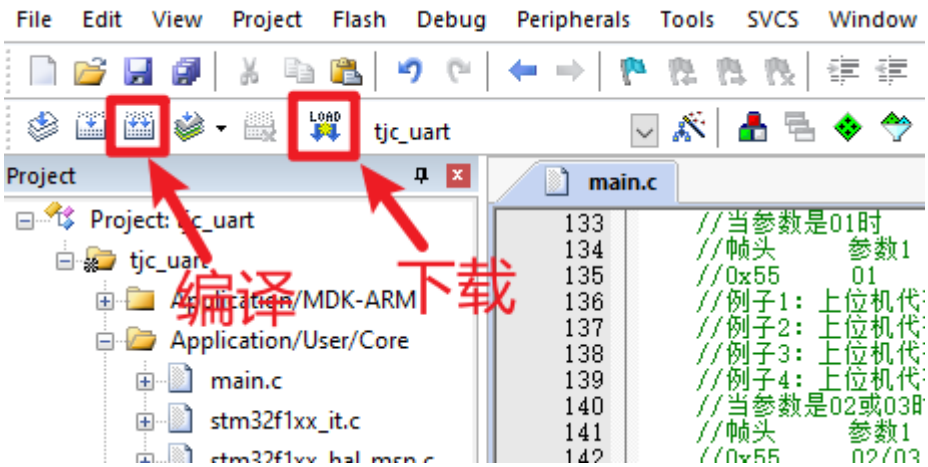
12、切换到 Flash Download 标签，勾选 Reset and Run



13、点击 Pack，取消勾选 Enable，所有设置就完毕了



14、主界面的两个按钮分别是编译和下载



2.10.2.4 下载《stm32 与串口屏双向通讯》例程

本案例包含了 stm32cubeide 工程（HAL 库）以及 keil MDK 工程（标准库）

> **注意：**以下代码仅为演示代码，用于测试显示屏能实现最基本的通信功能，如果您需要在正式产品中进行使用，请根据自己的需求对代码进行相应的优化和修改，或以自己的方式实现相应的功能

演示工程下载链接：

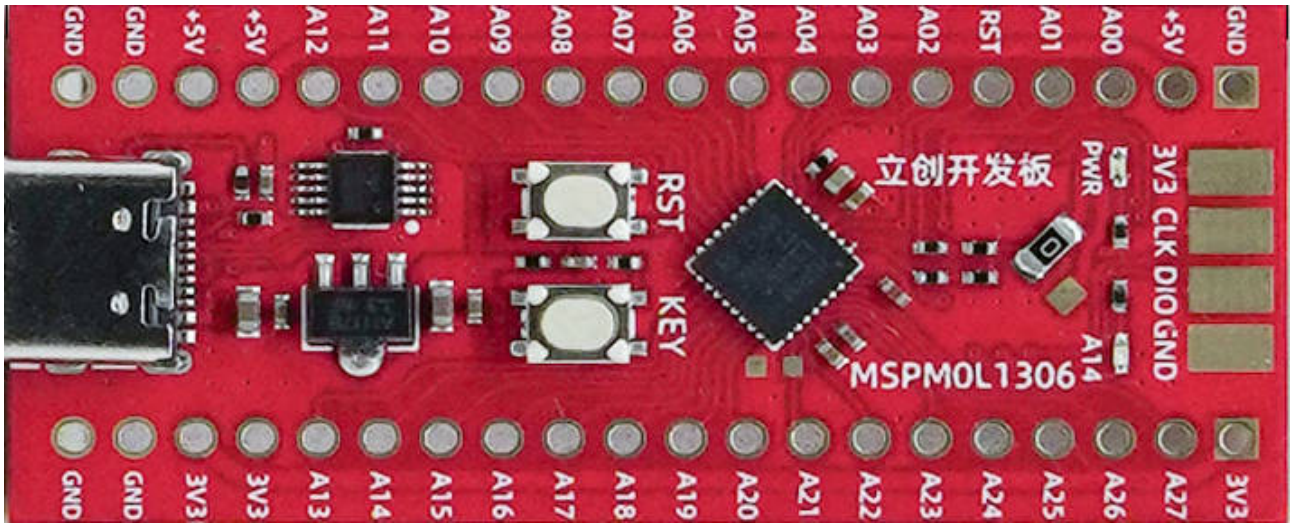
串口屏工程：《上位机 GB2312 编码》演示工程下载

stm32 工程：《stm32 与串口屏双向通讯》演示工程下载

2.11 与 TI（德州仪器）单片机联调

2.11.1 与 TI（德州仪器）单片机联调准备工作

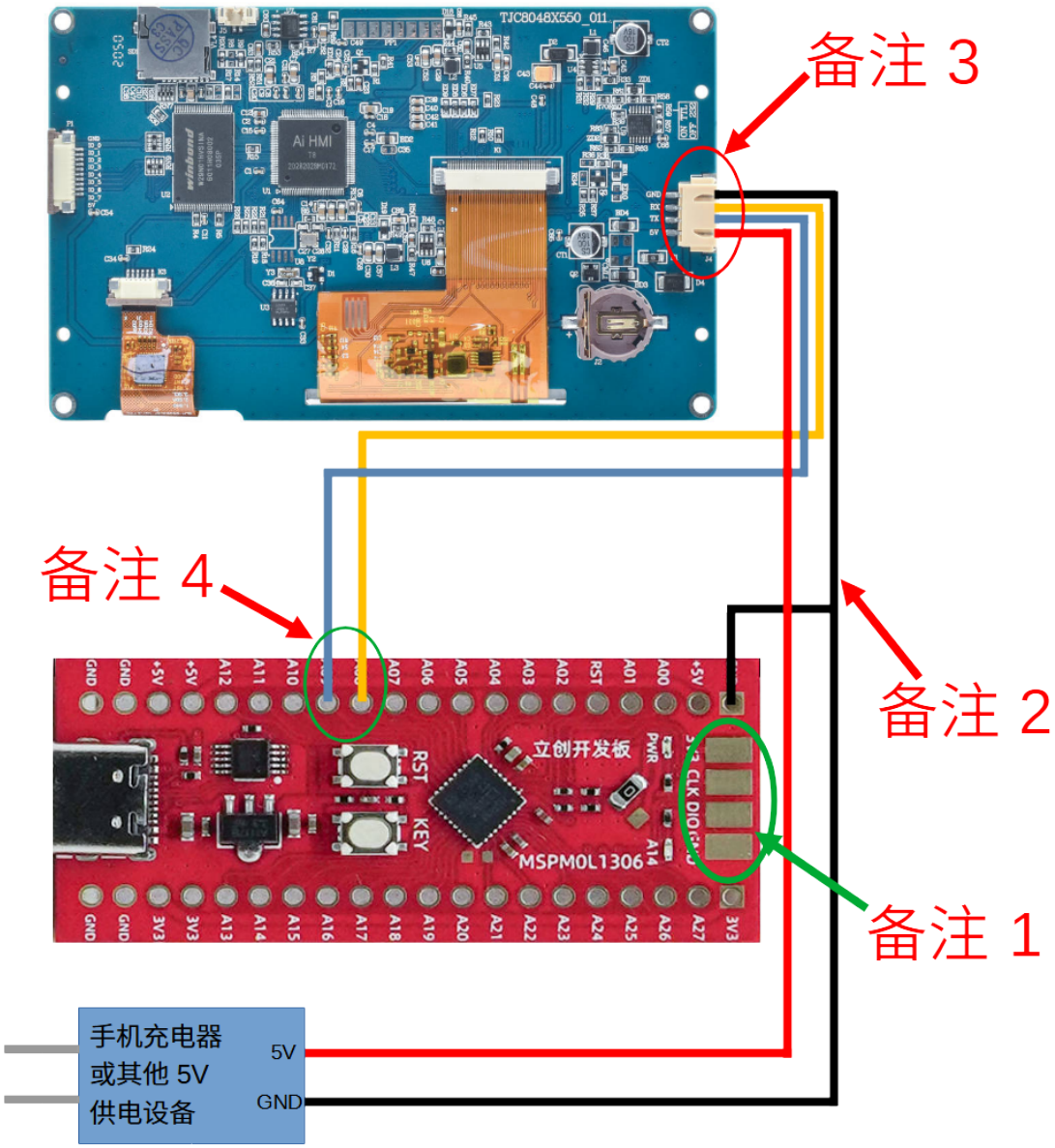
使用的单片机型号为立创 · 地正星-MSPM0L1306 开发板



2.11.1.1 TI（德州仪器）单片机-如何下载工程到串口屏

通过串口下载工程到串口屏 (page 144)
使用 *TFT* 文件下载助手 *(TFTFileDownload)* 通过串口下载工程到串口屏 (page 150)
通过 *SD* 卡下载工程到串口屏 (page 155)

2.11.1.2 TI（德州仪器）单片机与串口屏的接法



备注 1： 给 TI 开发板供电和下载程序还是通过这个接口

备注 2： 两边的 GND 要接在一起 (共地)，才能正常通讯

备注 3： 如果要通过串口给屏幕下载程序，要断开和 TI 开发板的连接，将屏幕接口和 usb 转 ttl 接在一起才能下载

备注 4： TI 的 TX 接串口屏的 RX， TI 的 RX 接串口屏的 TX

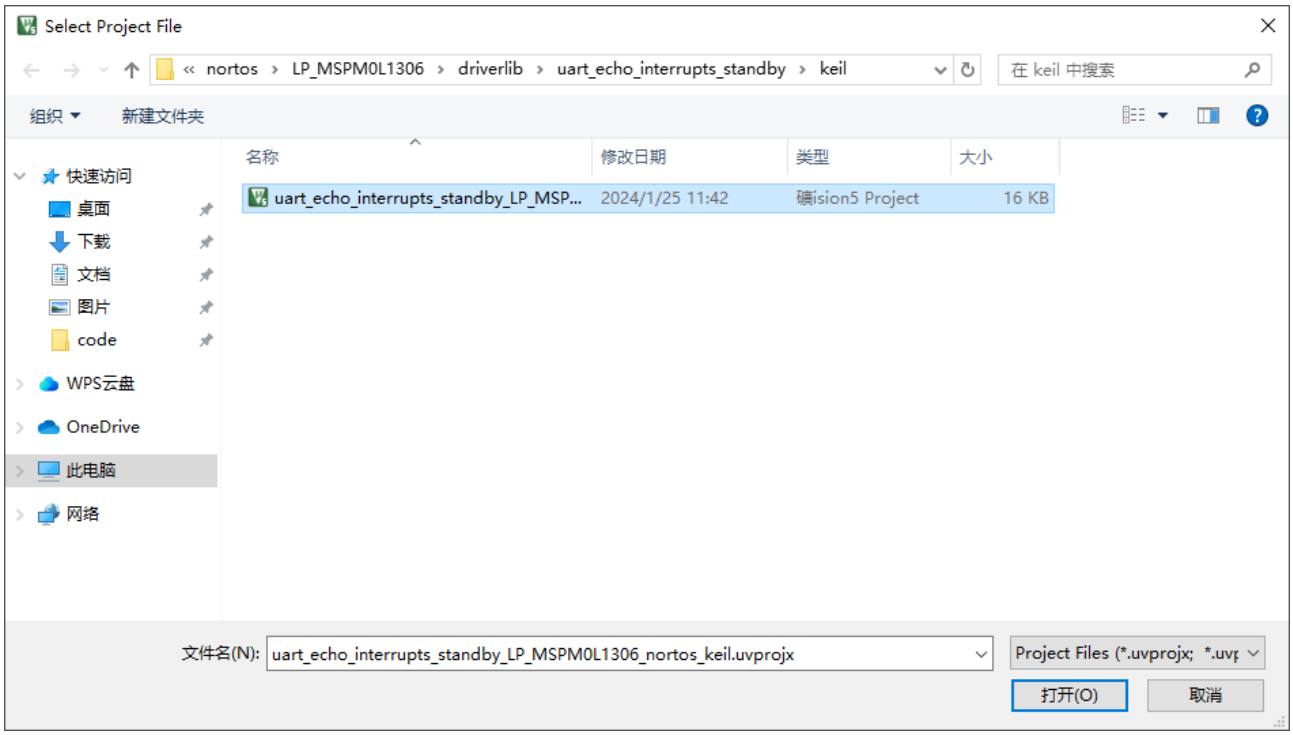
2.11.1.3 相关软件安装和基础教程学习

请先根据“立创开发板”上的教程安装并学习使用 keil 的相关开发课程，熟悉相关操作

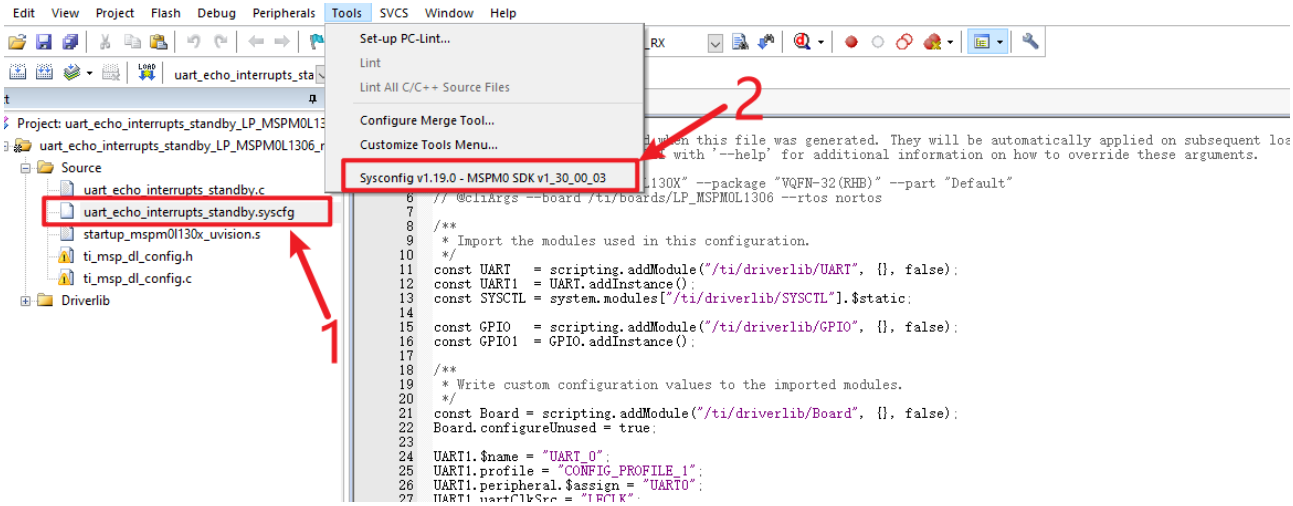
https://wiki.lckfb.com/zh-hans/dzx-mspm0l1306/beginner/install.html

2.11.2 与 TI（德州仪器）单片机联调 2

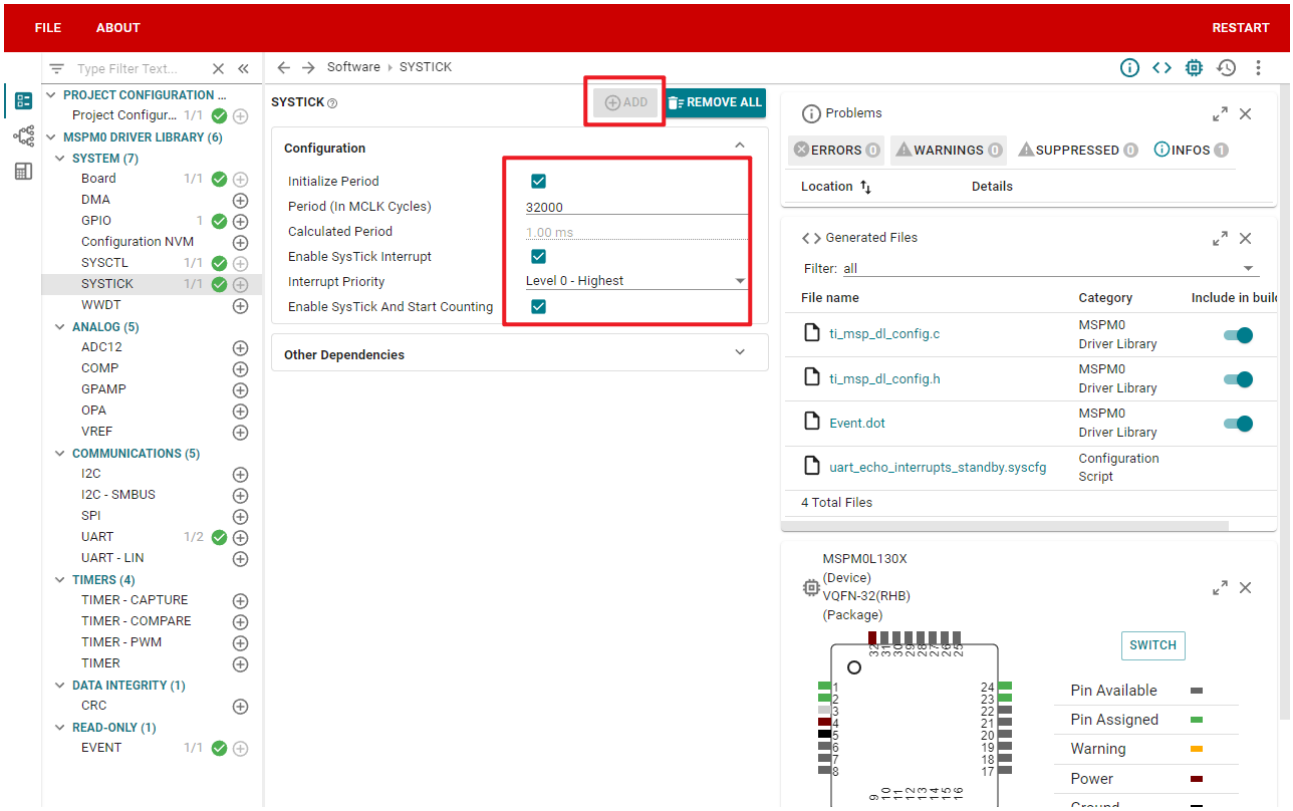
打开工程，其目录为\TI\M0_SDK\mspm0_sdk_1_30_00_03\examples\nortos\LP_MSPM0L1306\driverlib\uart_echo_interrupts_standb



打开 uart_echo_interrupts_standby.syscfg，并保持在当前文件，打开 sysconfig 工具



开启系统滴答定时器，并配置如下



修改串口配置如下：

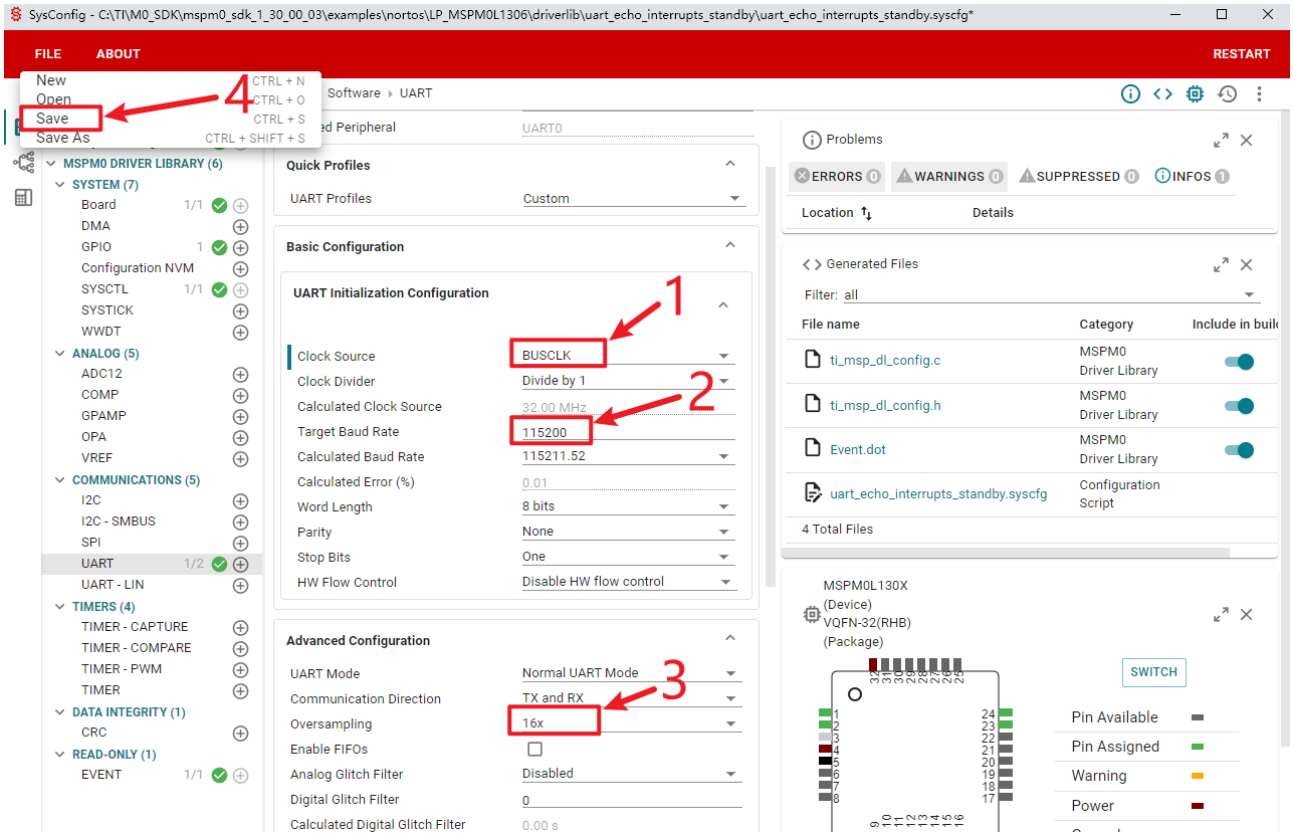
1、时钟源改为 BUSCLK（修改时钟源后会有警告，把后面两个全部修改完之后就不会有警告提示了）

2、波特率改为 115200，

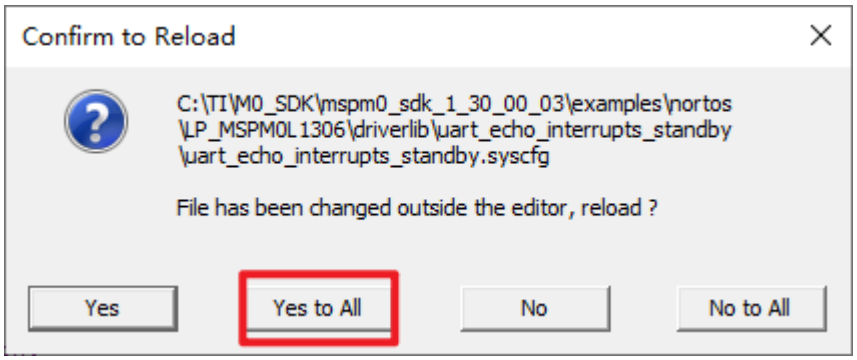
3、采样改为 16x

不要更改其他地方，否则会造成配置变化无法通讯

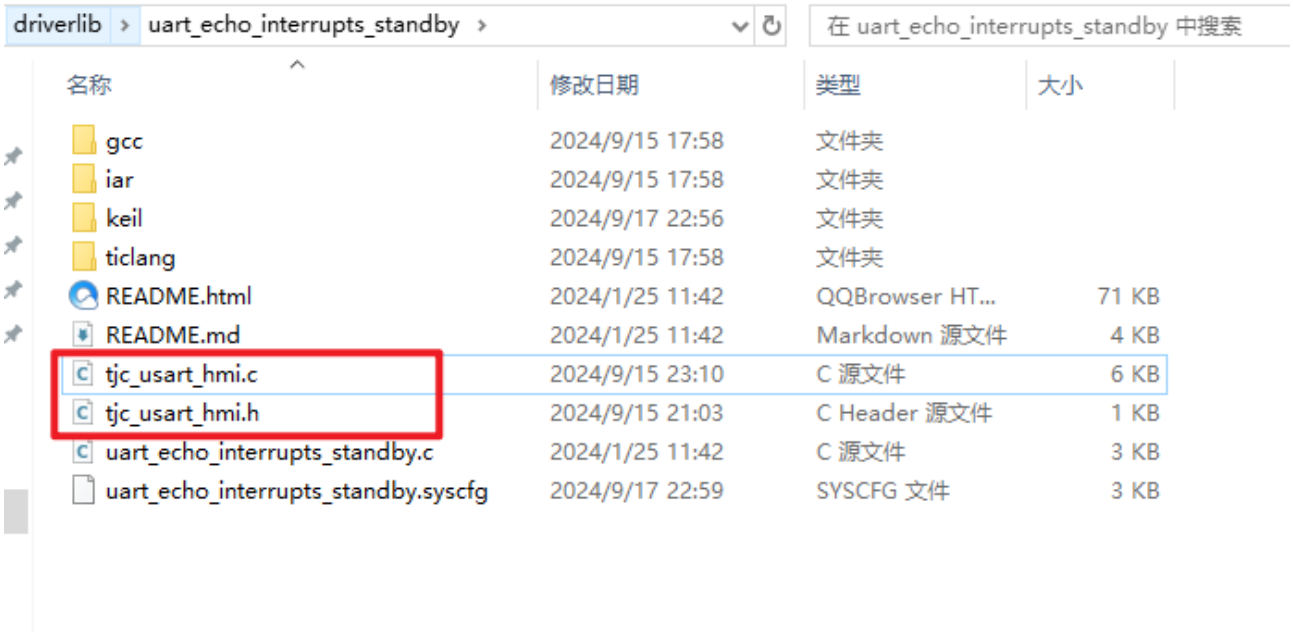
点击 save 保存



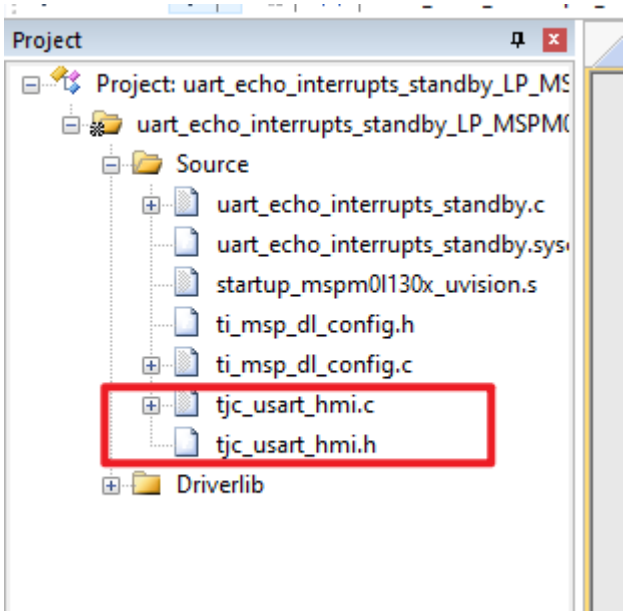
点击 yes to all，更新所有文件，这里可能会出现一个问题，参考缺少 *ti_msp_dl_config.c 和 ti_msp_dl_config.h* (page 313)



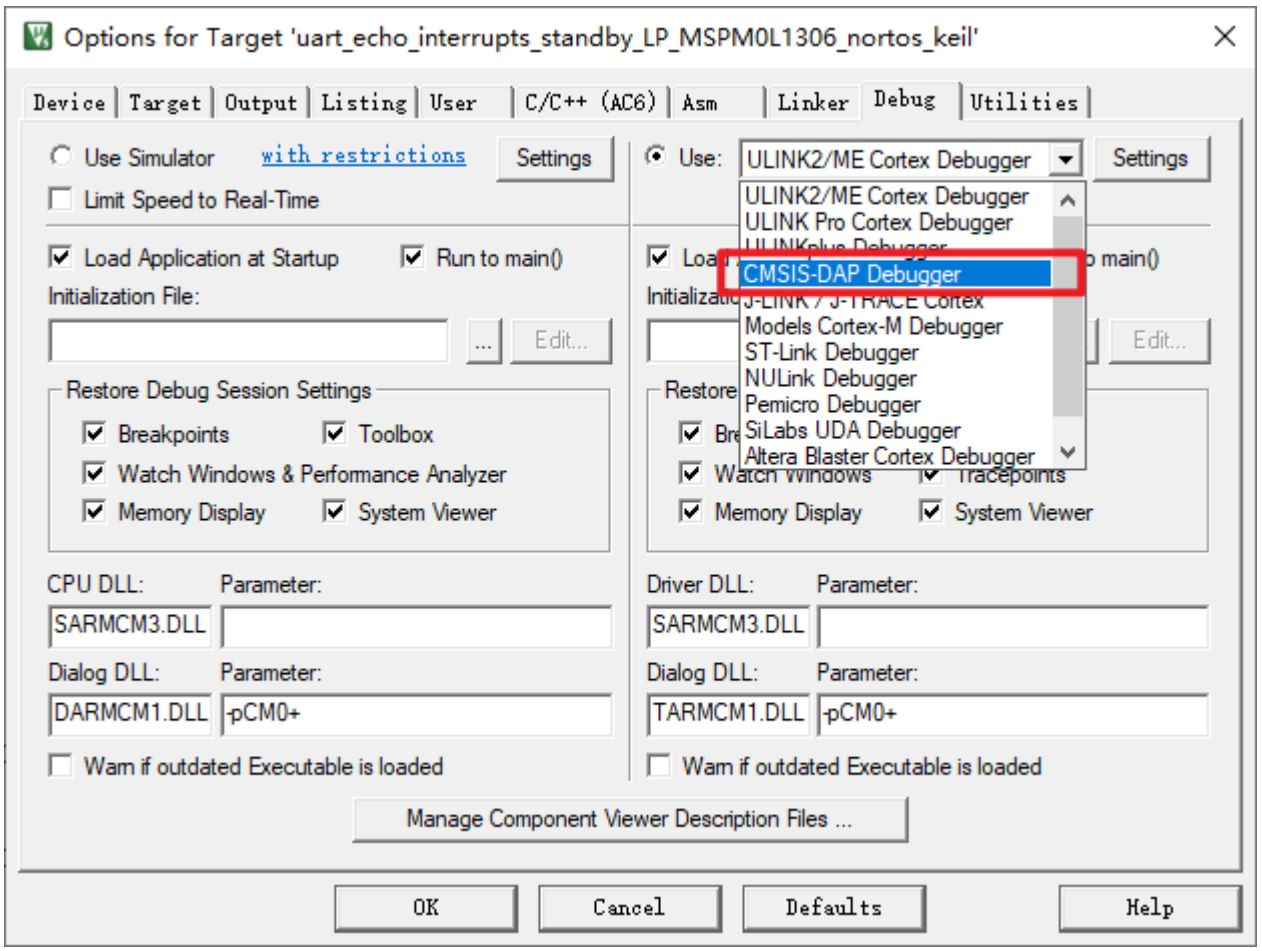
将 tjc_usart_hmi.c 和 tjc_usart_hmi.h 复制到工程目录下



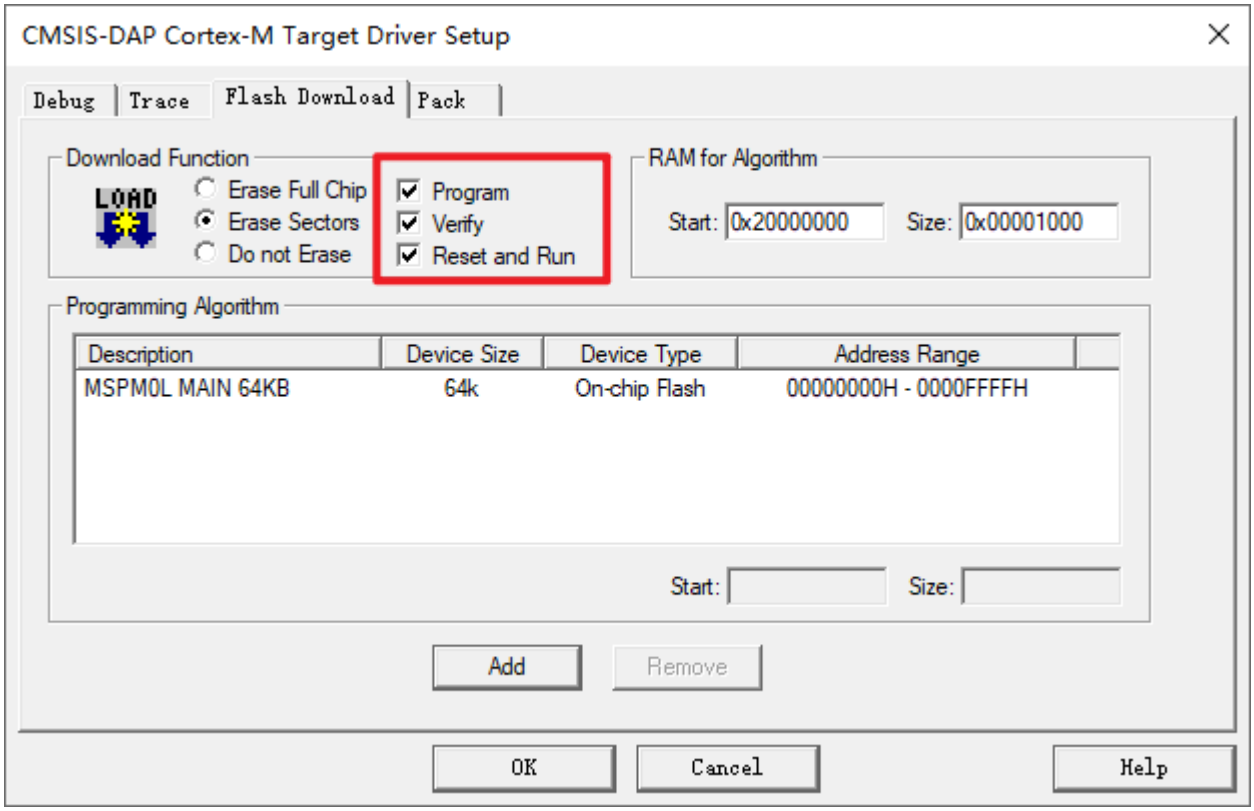
将 tjc_usart_hmi.c 和 tjc_usart_hmi.h 添加到工程中（这两个文件会在后面的工程里提供，需要自行从工程里找到然后导入客户自己的工程中）



配置下载工具，使用的是 daplink，请勿使用 stlink



勾选自动复位



uart_echo_interrupts_standby.c 中的代码如下, 编译下载，即可正常通讯

```
/*
* Copyright (c) 2021, Texas Instruments Incorporated
* All rights reserved.
*
* Redistribution and use in source and binary forms, with or without
* modification, are permitted provided that the following conditions
* are met:
*
* *  Redistributions of source code must retain the above copyright
*    notice, this list of conditions and the following disclaimer.
*
* *  Redistributions in binary form must reproduce the above copyright
*    notice, this list of conditions and the following disclaimer in the
*    documentation and/or other materials provided with the distribution.
*
* *  Neither the name of Texas Instruments Incorporated nor the names of
*    its contributors may be used to endorse or promote products derived
*    from this software without specific prior written permission.
*
* THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS"
* AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO,
* THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR
* PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT OWNER OR
* CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL,
* EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO,
* PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS;
* OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY,
* WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR
* OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE,
* EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
*/

#include "ti_msp_dl_config.h"
#include "tjc_usart_hmi.h"
#include "stdio.h"

#define FRAME_LENGTH 7
volatile uint32_t delay_times = 0;
volatile uint8_t uart_data = 0;
volatile uint32_t now_time = 0;

// 搭配滴答定时器实现的精确ms延时
void delay_ms(unsigned int ms)
{
   delay_times = ms;
   while (delay_times != 0)
      ;
}

int main(void)
{
   SYSCFG_DL_init();
   // 清除串口中断标志
   NVIC_ClearPendingIRQ(UART_0_INST_INT_IRQN);
   // 使能串口中断
   NVIC_EnableIRQ(UART_0_INST_INT_IRQN);
   int a = 100;
   char str[100];
   uint32_t last_time = 0;
   while (1)
   {
      if (now_time - last_time >= 1000)
```

(续下页)

(接上页)

```
    {
        last_time = now_time;
        sprintf(str, "n0.val=%d", a);
        tjc_send_string(str);
        sprintf(str, "t0.txt=\"%d\"\xff\xff\xff", a);
        tjc_send_string(str);
        sprintf(str, "click b0,1\xff\xff\xff");
        tjc_send_string(str);
        delay_ms(50);
        sprintf(str, "click b0,0\xff\xff\xff");
        tjc_send_string(str);

        a++;
    }
    // stm32f103的GND接串口屏或串口工具的GND,共地
    // stm32f103的TX1(PA9)接串口屏或串口工具的RX
    // stm32f103的RX1(PA10)接串口屏或串口工具的TX
    // stm32f103的5V接串口屏的5V,如果是串口工具,不用接5V也可以

    // 串口数据格式:
    // 串口数据帧长度: 7字节
    // 帧头      参数1     参数2    参数3        帧尾
    // 0x55      1字节    1字节     1字节      0xffffff
    // 当参数是01时
    // 帧头      参数1     参数2    参数3        帧尾
    // 0x55      01      led编号  led状态     0xffffff
    // 例子1: 上位机代码  printh 55 01 01 00 ff ff ff  含义: 1号led关闭
    // 例子2: 上位机代码  printh 55 01 04 01 ff ff ff  含义: 4号led打开
    // 例子3: 上位机代码  printh 55 01 00 01 ff ff ff  含义: 0号led打开
    // 例子4: 上位机代码  printh 55 01 04 00 ff ff ff  含义: 4号led关闭

    // 当参数是02或03时
    // 帧头      参数1     参数2    参数3        帧尾
    // 0x55      02/03   滑动值     00     0xffffff
    // 例子1: 上位机代码  printh 55 02 64 00 ff ff ff  含义: h0.val=100
    // 例子2: 上位机代码  printh 55 02 00 00 ff ff ff  含义: h0.val=0
    // 例子3: 上位机代码  printh 55 03 64 00 ff ff ff  含义: h1.val=100
    // 例子4: 上位机代码  printh 55 03 00 00 ff ff ff  含义: h1.val=0

    // 当串口缓冲区大于等于一帧的长度时
    while (usize >= FRAME_LENGTH)
    {
        // 校验帧头帧尾是否匹配
        if (usize >= FRAME_LENGTH && u(0) == 0x55 && u(4) == 0xff && u(5) == 0xff &&␣
↪u(6) == 0xff)
        {
            // 匹配, 进行解析
            if (u(1) == 0x01)
            {
                sprintf(str, "msg.txt=\"led %d is %s\"", u(2), u(3) ? "on" : "off");
                tjc_send_string(str);
                if (u(2) == 0x00)
                {
                    if (u(3) == 0x00)
                    {
                        // DL_GPIO_clearPins(LED1_PORT,LED1_PIN_14_PIN);//输出低电平
                    }
                    else
                    {
                        // DL_GPIO_setPins(LED1_PORT,LED1_PIN_14_PIN);  //输出高电平
                    }
```

(续下页)

(接上页)

```
                }
            }
            else if (u(1) == 0x02)
            {
                // 下发的是h0进度条的信息

                sprintf(str, "msg.txt=\"h0.val is %d\"", u(2));
                tjc_send_string(str);
            }
            else if (u(1) == 0x03)
            {
                // 下发的是h1进度条的信息
                sprintf(str, "msg.txt=\"h1.val is %d\"", u(2));
                tjc_send_string(str);
            }

            udelete(7); // 删除解析过的数据
        }
        else
        {

            // 不匹配删除1字节
            udelete(1);
            break;
        }
    }
  }
}

void SysTick_Handler()
{
  if (delay_times != 0)
  {
    delay_times--;
  }
  now_time++;
}

// 串口的中断服务函数
void UART_0_INST_IRQHandler(void)
{
  // 如果产生了串口中断
  switch (DL_UART_Main_getPendingInterrupt(UART_0_INST))
  {
  case DL_UART_MAIN_IIDX_RX: // 如果是接收中断
    // 接发送过来的数据保存在变量中
    uart_data = DL_UART_Main_receiveData(UART_0_INST);
    // 将保存的数据再发送出去
    // uart0_send_char(uart_data);
    writeRingBuff(uart_data);
    break;

  default: // 其他的串口中断
    break;
  }
}
```

2.11.3 与 TI（德州仪器）单片机联调可能出现的问题

2.11.3.1 下载后屏幕没有变化

检查接线，TX(A08) 接屏幕的 RX，RX(A09) 接屏幕的 TX

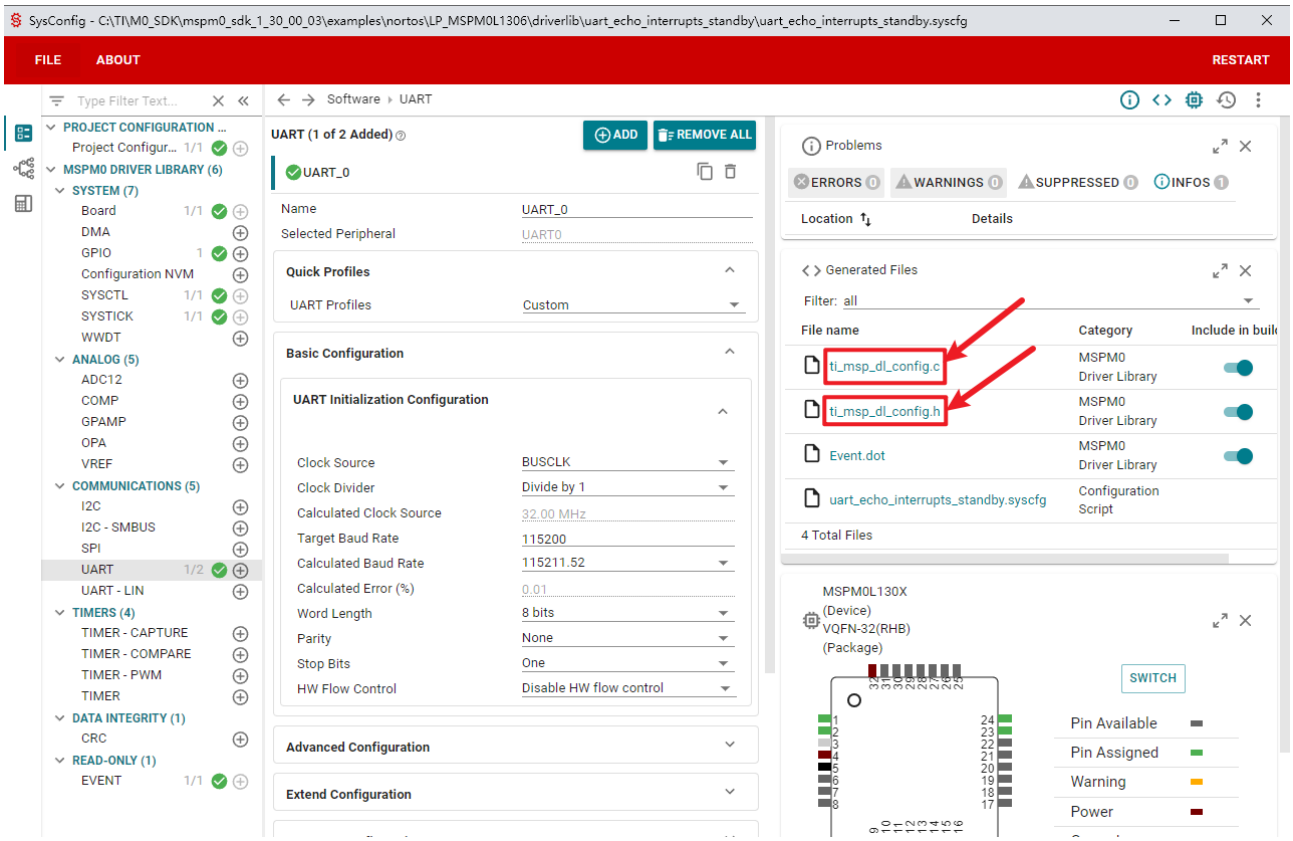
检查波特率是否匹配

检查 TI 工程内是否打开了滴答定时器

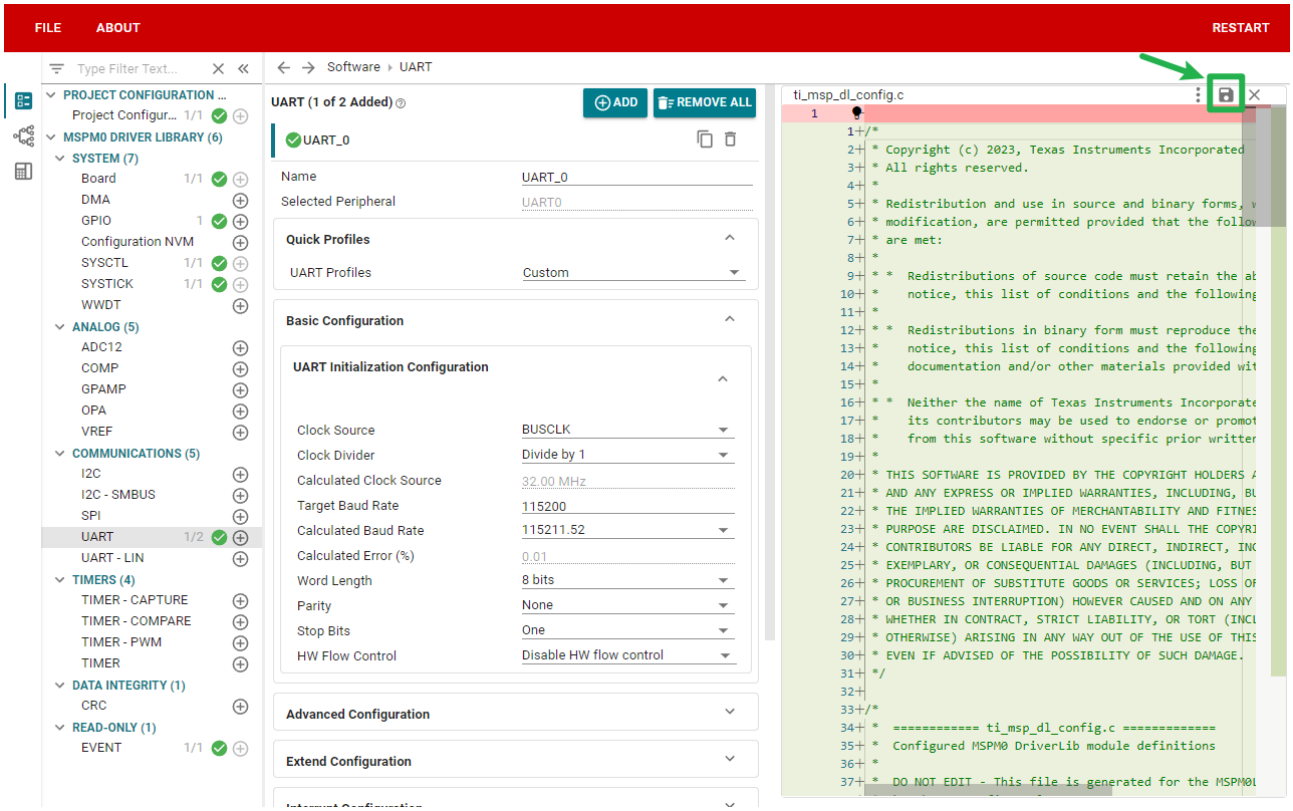
检查是否勾选了下载后自动复位

2.11.3.2 缺少 ti_msp_dl_config.c 和 ti_msp_dl_config.h

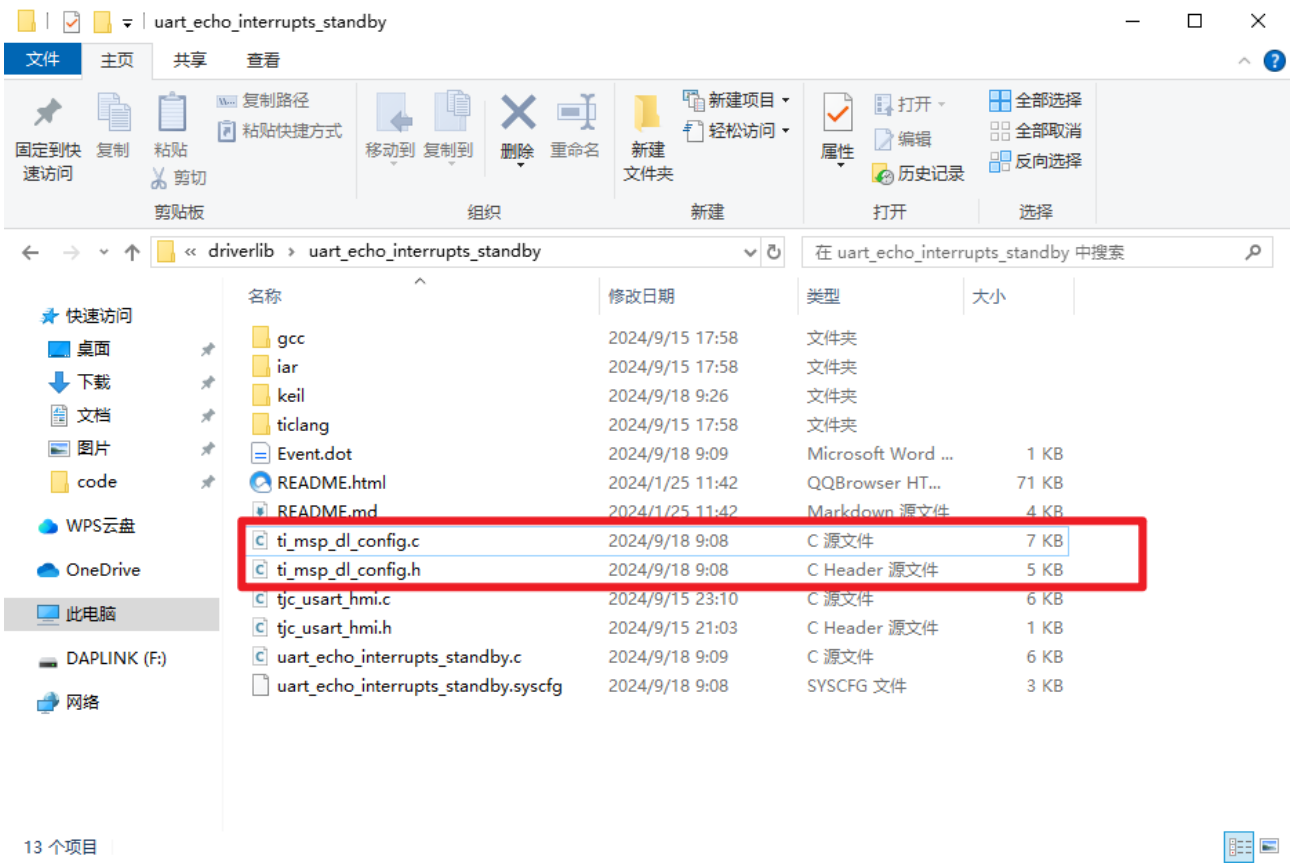
在此界面打开 ti_msp_dl_config.c 和 ti_msp_dl_config.h



分别保存这两个文件到工程目录下即可



注意：不是保存在 keil 文件夹下，是 keil 文件夹的上一级



2.11.4 TI（德州仪器）工程下载

串口屏工程：《上位机 GB2312 编码》演示工程下载

TI 工程：《TI（德州仪器）工程下载》演示工程下载

2.12 与 arduino 联调

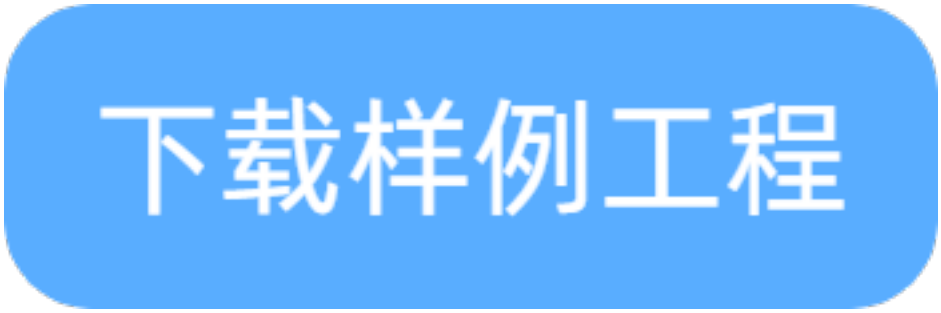
对应的视频教程：arduino 解析协议

提示： 以下的教程均基于 arduino2.0 以上的版本，在低于 arduino2.0 的版本下编译可能会报错

2.12.1 arduino uno 与串口屏通讯

提示： 以下的教程均基于 arduino2.0 以上的版本，在低于 arduino2.0 的版本下编译可能会报错

2.12.1.1 arduino uno 与串口屏通讯工程下载



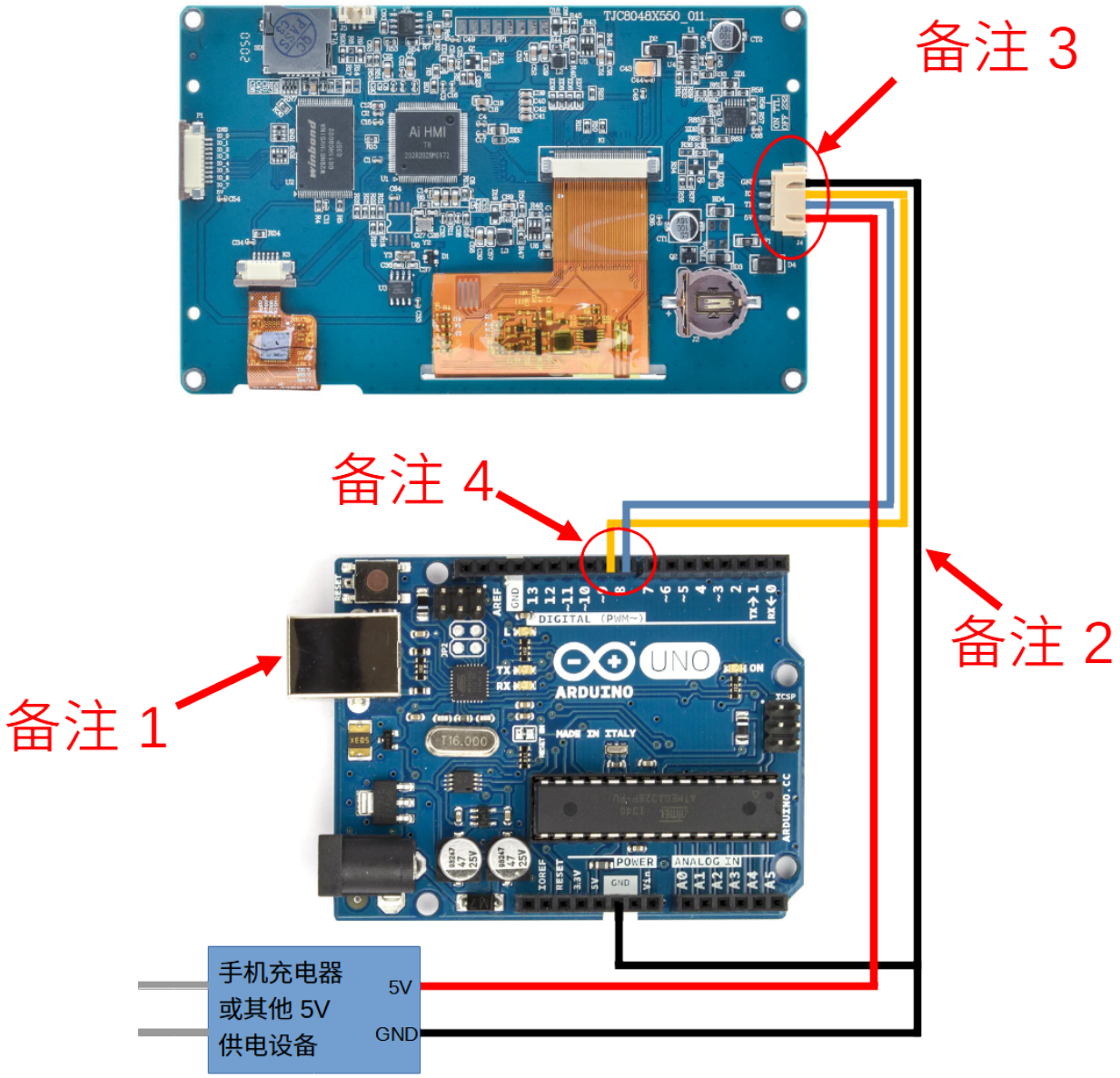
串口屏怎么下载程序

通过串口下载工程到串口屏 (page 144)

使用 *TFT* 文件下载助手 *(TFTFileDownload)* 通过串口下载工程到串口屏 (page 150)

通过 *SD* 卡下载工程到串口屏 (page 155)

2.12.1.2 arduino uno 与串口屏的连接方式



备注 1： 给 arduino 下载程序和供电还是通过这个接口

备注 2： 两边的 GND 要接在一起 (共地)，才能正常通讯

备注 3： 如果要通过 usb 转 ttl 给屏幕下载程序，要断开和 arduino 的连接，将屏幕接口和 usb 转 ttl 接在一起才能下载

备注 4： 因为接的是软串口，所以请使用 SoftwareSerial 进行通讯，arduino 的 TX 接串口屏的 RX， arduino 的 RX 接串口屏的 TX

2.12.1.3 arduino uno 代码

注意: 以下代码仅为演示代码，用于测试显示屏能实现最基本的通信功能，如果您需要在正式产品中进行使用，请根据自己的需求对代码进行相应的优化和修改，或以自己的方式实现相应的功能

注意: 以下代码适用于 arduino uno

arduino uno 和 arduino nano 之类的只有 1 个串口的 arduino, 请使用 SoftwareSerial 库模拟的串口，通讯波特率不要超过 115200

不让用 arduino uno/nano 的硬件串口是因为 arduino 的硬件串口上已经接了其他的芯片，直接接串口屏是无法正常工作的，迫不得已之下只能使用软件串口来模拟

软串口的稳定性不足，在代码比较复杂（任务较多较重的情况下）容易读写出错，还是建议换串口较多的 arduino 开发板

arduino 只支持 utf8 编码，因此需要发送汉字时，在新建串口屏工程时请选择 UTF8 编码，导入字库时也要使用 UTF8 字库，参考*arduino 发送中文时乱码* (page 1431)

```
//arduino的GND接串口屏或串口工具的GND,共地
//arduino的9接串口屏或串口工具的RX
//arduino的8接串口屏或串口工具的TX
//arduino的5V接串口屏的5V,如果是串口工具,不用接5V也可以
//根据自己的实际改为对应的串口,我这里接的是8和9
#include <SoftwareSerial.h>
SoftwareSerial TJC(8,9);
#define FRAME_LENGTH 7

int a;
unsigned long nowtime;
void setup() {
  // put your setup code here, to run once:
  //初始化串口
  TJC.begin(115200);

  //因为串口屏开机会发送88 ff ff ff,所以要清空串口缓冲区
  while (TJC.read() >= 0); //清空串口缓冲区
  TJC.print("page main\xff\xff\xff"); //发送命令让屏幕跳转到main页面
  nowtime = millis(); //获取当前已经运行的时间
}

void loop() {
  // put your main code here, to run repeatedly:
  char str[100];
  if (millis() >= nowtime + 1000) {
    nowtime = millis(); //获取当前已经运行的时间

    //用sprintf来格式化字符串，给n0的val属性赋值
    sprintf(str, "n0.val=%d\xff\xff\xff", a);
    //把字符串发送出去
    TJC.print(str);

    //用sprintf来格式化字符串，给t0的txt属性赋值
    sprintf(str, "t0.txt=\"现在是%d\"\xff\xff\xff", a);
    //把字符串发送出去
    TJC.print(str);
```

(续下页)

(接上页)

```
    //用sprintf来格式化字符串，触发b0的按下事件,直接把结束符整合在字符串中
    sprintf(str, "click b0,1\xff\xff\xff");
    //把字符串发送出去
    TJC.print(str);

    delay(50);  //延时50ms,才能看清楚点击效果

    //用sprintf来格式化字符串，触发b0的弹起事件,直接把结束符整合在字符串中
    sprintf(str, "click b0,0\xff\xff\xff");
    //把字符串发送出去
    TJC.print(str);

    a++;
  }

  //串口数据格式:
  //串口数据帧长度: 7字节
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      1字节    1字节     1字节      0xffffff
  //当参数是01时
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      01      led编号  led状态     0xffffff
  //例子1: 上位机代码  printh 55 01 01 00 ff ff ff  含义: 1号led关闭
  //例子2: 上位机代码  printh 55 01 04 01 ff ff ff  含义: 4号led打开
  //例子3: 上位机代码  printh 55 01 00 01 ff ff ff  含义: 0号led打开
  //例子4: 上位机代码  printh 55 01 04 00 ff ff ff  含义: 4号led关闭

  //当参数是02或03时
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      02/03   滑动值     00     0xffffff
  //例子1: 上位机代码  printh 55 02 64 00 ff ff ff  含义: h0.val=100
  //例子2: 上位机代码  printh 55 02 00 00 ff ff ff  含义: h0.val=0
  //例子3: 上位机代码  printh 55 03 64 00 ff ff ff  含义: h1.val=100
  //例子4: 上位机代码  printh 55 03 00 00 ff ff ff  含义: h1.val=0


  //当串口缓冲区大于等于一帧的长度时
  while (TJC.available() >= FRAME_LENGTH) {
    unsigned char ubuffer[FRAME_LENGTH];
    //从串口缓冲读取1个字节但不删除
    unsigned char frame_header = TJC.peek();
    //当获取的数据是包头(0x55)时
    if (frame_header == 0x55) {
      //从串口缓冲区读取7字节
      TJC.readBytes(ubuffer, FRAME_LENGTH);
      if (ubuffer[4] == 0xff && ubuffer[5] == 0xff && ubuffer[6] == 0xff) {
        if(ubuffer[1] == 0x01)
        {
          //下发的是LED信息
          sprintf(str, "msg.txt=\"led %d is %s\"\xff\xff\xff", ubuffer[2], ubuffer[3] ?
↪"on" : "off");
          TJC.print(str);

        }else if(ubuffer[1] == 0x02)
        {
          //下发的是滑动条h0.val的信息
          sprintf(str, "msg.txt=\"h0.val is %d\"\xff\xff\xff", ubuffer[2]);
          TJC.print(str);

        }else if(ubuffer[1] == 0x03)
        {
```

(续下页)

(接上页)

```
          //下发的是滑动条h1.val的信息
          sprintf(str, "msg.txt=\"h1.val is %d\"\xff\xff\xff", ubuffer[2]);
          TJC.print(str);

        }

      }
    } else {
      TJC.read();  //从串口缓冲读取1个字节并删除
    }
  }
}
```

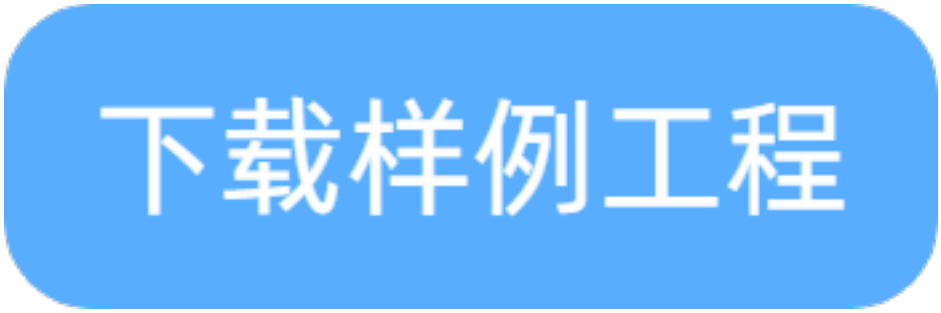
其他参考链接

屏幕通电后不断的闪烁(供电不足导致不断重启) (page 1244)

2.12.2 arduino mega2560 与串口屏通讯

提示： 以下的教程均基于 arduino2.0 以上的版本，在低于 arduino2.0 的版本下编译可能会报错

2.12.2.1 arduino mega2560 与串口屏通讯工程下载



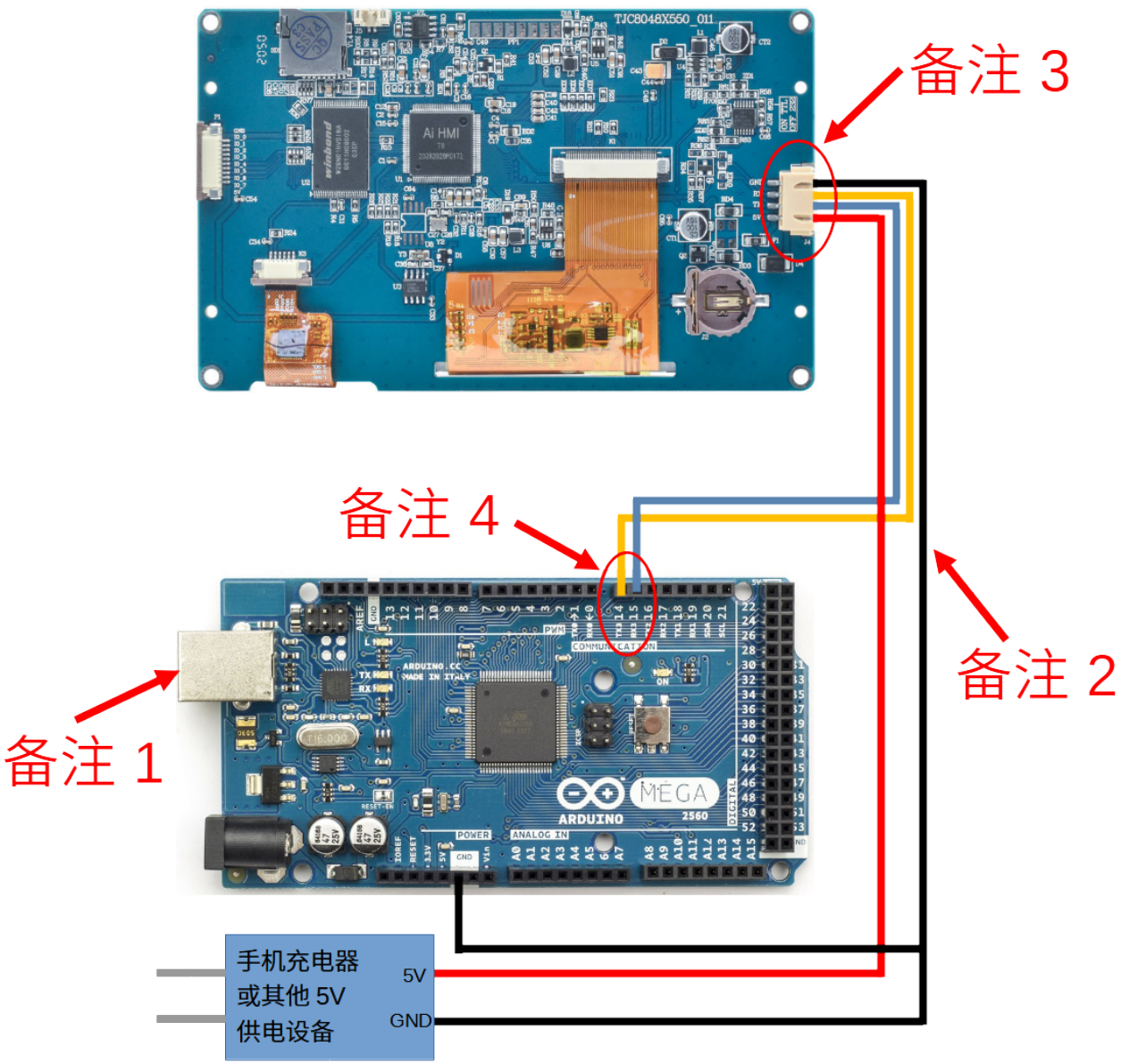
串口屏怎么下载程序

通过串口下载工程到串口屏 (page 144)

使用 *TFT* 文件下载助手 *(TFTFileDownload)* 通过串口下载工程到串口屏 (page 150)

通过 *SD* 卡下载工程到串口屏 (page 155)

2.12.2.2 arduino mega2560 与串口屏的连接方式



备注 1： 给 arduino 下载程序和供电还是通过这个接口

备注 2： 两边的 GND 要接在一起 (共地)，才能正常通讯

备注 3： 如果要通过 usb 转 ttl 给屏幕下载程序，要断开和 arduino 的连接，将屏幕接口和 usb 转 ttl 接在一起才能下载

备注 4： 因为接的是 TX3 和 RX3，所以请使用 Serial3 进行通讯，arduino 的 TX3 接串口屏的 RX， arduino 的 RX3 接串口屏的 TX

2.12.2.3 arduino mega2560 代码

注意: 以下代码仅为演示代码，用于测试显示屏能实现最基本的通信功能，如果您需要在正式产品中进行使用，请根据自己的需求对代码进行相应的优化和修改，或以自己的方式实现相应的功能

注意: 以下代码适用于 arduino mega2560，如果是其他型号的 arduino，请修改第 6 行代码 #define TJC Serial3 为其他串口

arduino 系列请勿使用 Serial(TX0 和 RX0) 与屏幕进行通讯，因为这个串口已经连接到电脑的串口上了，会导致无法和串口屏通讯，请使用 Serial1/Serial2/Serial3，arduino uno 之类的只有 1 个串口的 arduino, 请使用 SoftwareSerial 库模拟的串口，请参考*arduino uno 与串口屏通讯* (page 316)

arduino 只支持 utf8 编码，因此需要发送汉字时，在新建串口屏工程时请选择 UTF8 编码，导入字库时也要使用 UTF8 字库，参考*arduino 发送中文时乱码* (page 1431)

```
//arduino的GND接串口屏或串口工具的GND,共地
//arduino的TX3接串口屏或串口工具的RX
//arduino的RX3接串口屏或串口工具的TX
//arduino的5V接串口屏的5V,如果是串口工具,不用接5V也可以
//根据自己的实际改为对应的串口,我这里接的是TX3和RX3
#define TJC Serial3
#define FRAME_LENGTH 7

int a;
unsigned long nowtime;
void setup() {
  // put your setup code here, to run once:
  //初始化串口
  TJC.begin(115200);

  //因为串口屏开机会发送88 ff ff ff,所以要清空串口缓冲区
  while (TJC.read() >= 0); //清空串口缓冲区
  TJC.print("page main\xff\xff\xff"); //发送命令让屏幕跳转到main页面
  nowtime = millis(); //获取当前已经运行的时间
}

void loop() {
  // put your main code here, to run repeatedly:
  char str[100];
  if (millis() >= nowtime + 1000) {
    nowtime = millis(); //获取当前已经运行的时间

    //用sprintf来格式化字符串，给n0的val属性赋值
    sprintf(str, "n0.val=%d\xff\xff\xff", a);
    //把字符串发送出去
    TJC.print(str);

    //用sprintf来格式化字符串，给t0的txt属性赋值
    sprintf(str, "t0.txt=\"现在是%d\"\xff\xff\xff", a);
    //把字符串发送出去
    TJC.print(str);


    //用sprintf来格式化字符串，触发b0的按下事件,直接把结束符整合在字符串中
    sprintf(str, "click b0,1\xff\xff\xff");
    //把字符串发送出去
    TJC.print(str);
```

(续下页)

(接上页)

```
    delay(50);  //延时50ms,才能看清楚点击效果

    //用sprintf来格式化字符串，触发b0的弹起事件,直接把结束符整合在字符串中
    sprintf(str, "click b0,0\xff\xff\xff");
    //把字符串发送出去
    TJC.print(str);

    a++;
  }

  //串口数据格式:
  //串口数据帧长度: 7字节
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      1字节    1字节     1字节      0xffffff
  //当参数是01时
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      01      led编号  led状态     0xffffff
  //例子1: 上位机代码  printh 55 01 01 00 ff ff ff  含义: 1号led关闭
  //例子2: 上位机代码  printh 55 01 04 01 ff ff ff  含义: 4号led打开
  //例子3: 上位机代码  printh 55 01 00 01 ff ff ff  含义: 0号led打开
  //例子4: 上位机代码  printh 55 01 04 00 ff ff ff  含义: 4号led关闭

  //当参数是02或03时
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      02/03   滑动值     00     0xffffff
  //例子1: 上位机代码  printh 55 02 64 00 ff ff ff  含义: h0.val=100
  //例子2: 上位机代码  printh 55 02 00 00 ff ff ff  含义: h0.val=0
  //例子3: 上位机代码  printh 55 03 64 00 ff ff ff  含义: h1.val=100
  //例子4: 上位机代码  printh 55 03 00 00 ff ff ff  含义: h1.val=0


  //当串口缓冲区大于等于一帧的长度时
  while (TJC.available() >= FRAME_LENGTH) {
    unsigned char ubuffer[FRAME_LENGTH];
    //从串口缓冲读取1个字节但不删除
    unsigned char frame_header = TJC.peek();
    //当获取的数据是包头(0x55)时
    if (frame_header == 0x55) {
      //从串口缓冲区读取7字节
      TJC.readBytes(ubuffer, FRAME_LENGTH);
      if (ubuffer[4] == 0xff && ubuffer[5] == 0xff && ubuffer[6] == 0xff) {
        if(ubuffer[1] == 0x01)
        {
          //下发的是LED信息
          sprintf(str, "msg.txt=\"led %d is %s\"\xff\xff\xff", ubuffer[2], ubuffer[3] ?
↪"on" : "off");
          TJC.print(str);

        }else if(ubuffer[1] == 0x02)
        {
          //下发的是滑动条h0.val的信息
          sprintf(str, "msg.txt=\"h0.val is %d\"\xff\xff\xff", ubuffer[2]);
          TJC.print(str);

        }else if(ubuffer[1] == 0x03)
        {
          //下发的是滑动条h1.val的信息
          sprintf(str, "msg.txt=\"h1.val is %d\"\xff\xff\xff", ubuffer[2]);
          TJC.print(str);
```

(续下页)

(接上页)

```
        }

      }
    } else {
      TJC.read();  //从串口缓冲读取1个字节并删除
    }
  }
}
```

其他参考链接

屏幕通电后不断的闪烁(供电不足导致不断重启) (page 1244)

2.12.3 米思齐 Mixly arduino mega2560 与串口屏通讯

2.12.3.1 米思齐 Mixly arduino mega2560 与串口屏通讯工程下载

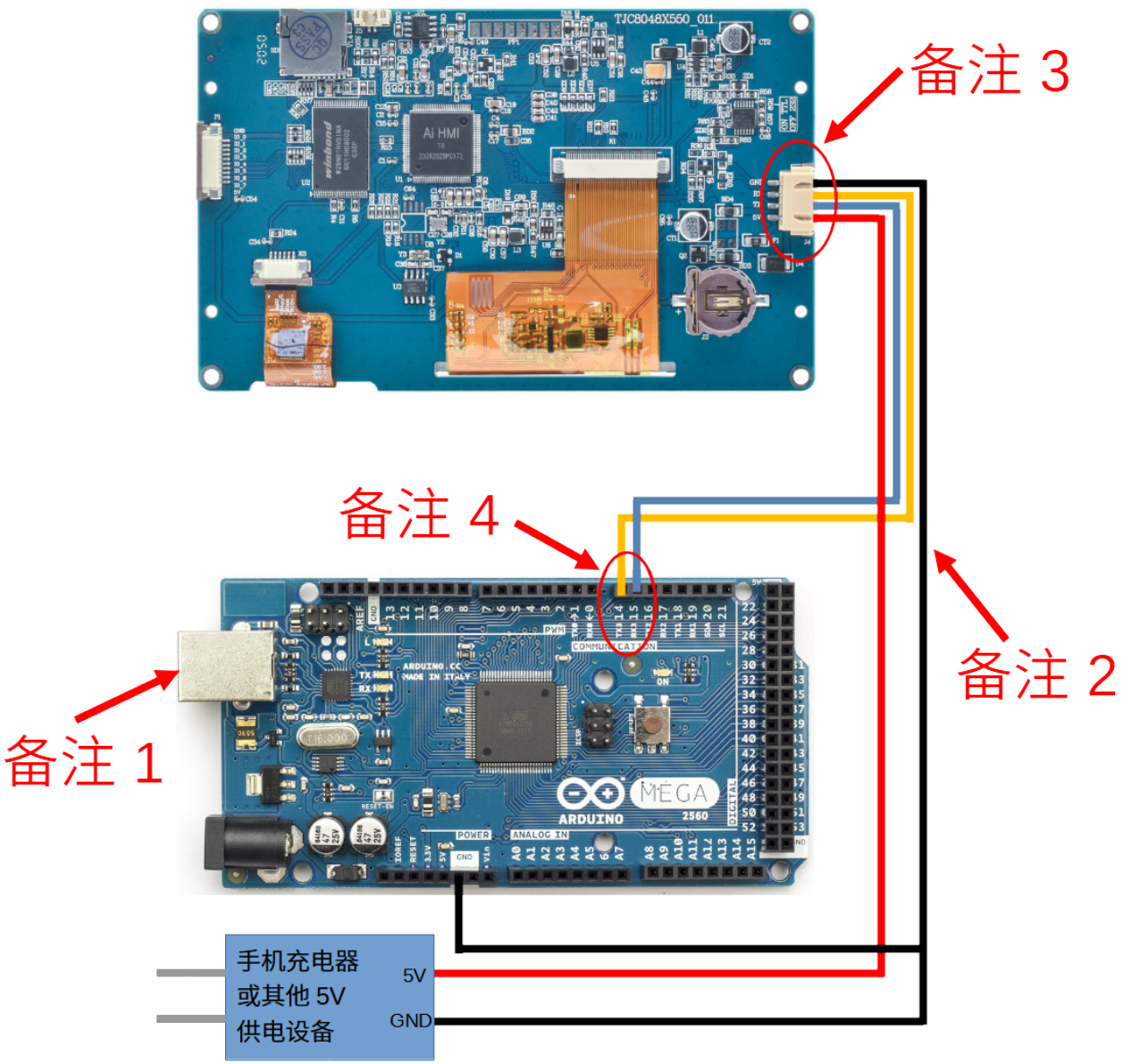
串口屏怎么下载程序

通过串口下载工程到串口屏 (page 144)

使用 *TFT* 文件下载助手 *(TFTFileDownload)* 通过串口下载工程到串口屏 (page 150)

通过 *SD* 卡下载工程到串口屏 (page 155)

2.12.3.2 米思齐 Mixlyarduino mega2560 与串口屏的连接方式



备注 1： 给 arduino 下载程序和供电还是通过这个接口

备注 2： 两边的 GND 要接在一起 (共地)，才能正常通讯

备注 3： 如果要通过 usb 转 ttl 给屏幕下载程序，要断开和 arduino 的连接，将屏幕接口和 usb 转 ttl 接在一起才能下载

备注 4： 因为接的是 TX3 和 RX3，所以请使用 Serial3 进行通讯，arduino 的 TX3 接串口屏的 RX， arduino 的 RX3 接串口屏的 TX

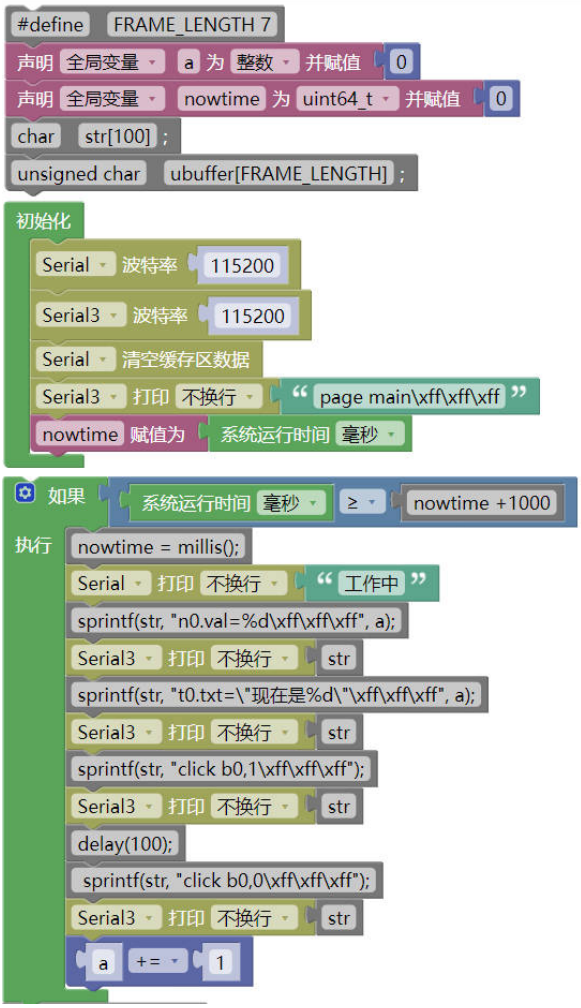
2.12.3.3 米思齐 Mixly arduino mega2560 代码

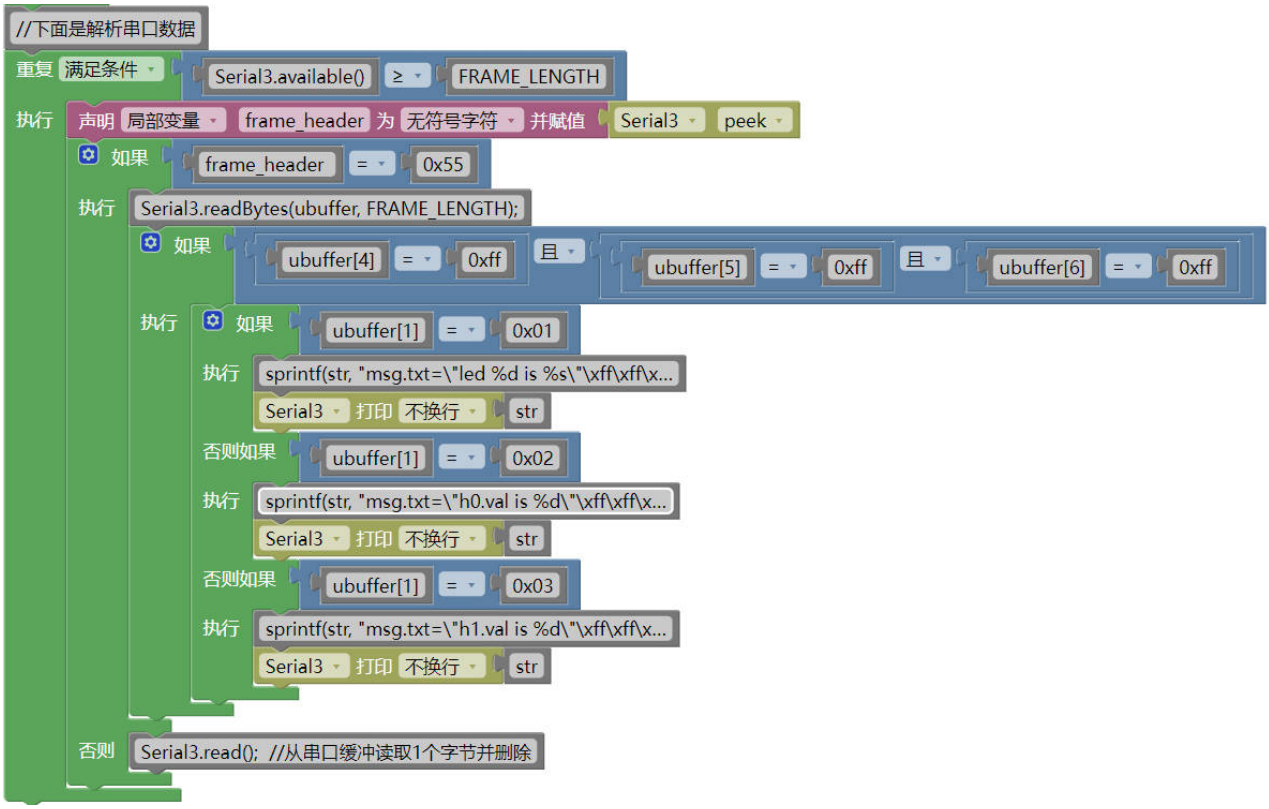
注意： 以下代码仅为演示代码，用于测试显示屏能实现最基本的通信功能，如果您需要在正式产品中进行使用，请根据自己的需求对代码进行相应的优化和修改，或以自己的方式实现相应的功能

注意： 以下代码适用于 arduino mega2560，如果是其他型号的 arduino，请修改 Serial3 为其他串口

arduino 系列请勿使用 Serial(TX0 和 RX0) 与屏幕进行通讯，因为这个串口已经连接到电脑的串口上了，会导致无法和串口屏通讯，请使用 Serial1/Serial2/Serial3，arduino uno 之类的只有 1 个串口的 arduino, 请使用 SoftwareSerial 库模拟的串口，请参考*arduino uno 与串口屏通讯* (page 316)

arduino 只支持 utf8 编码，因此需要发送汉字时，在新建串口屏工程时请选择 UTF8 编码，导入字库时也要使用 UTF8 字库，参考*arduino 发送中文时乱码* (page 1431)





其他参考链接

屏幕通电后不断的闪烁(供电不足导致不断重启) (page 1244)

2.12.4 ESP32C3 与串口屏通讯

使用的是 ESP32C3 合宙经典款（板载 ch343 芯片的）

提示： 以下的教程均基于 arduino2.0 以上的版本，在低于 arduino2.0 的版本下编译可能会报错

2.12.4.1 ESP32C3 与串口屏通讯工程下载

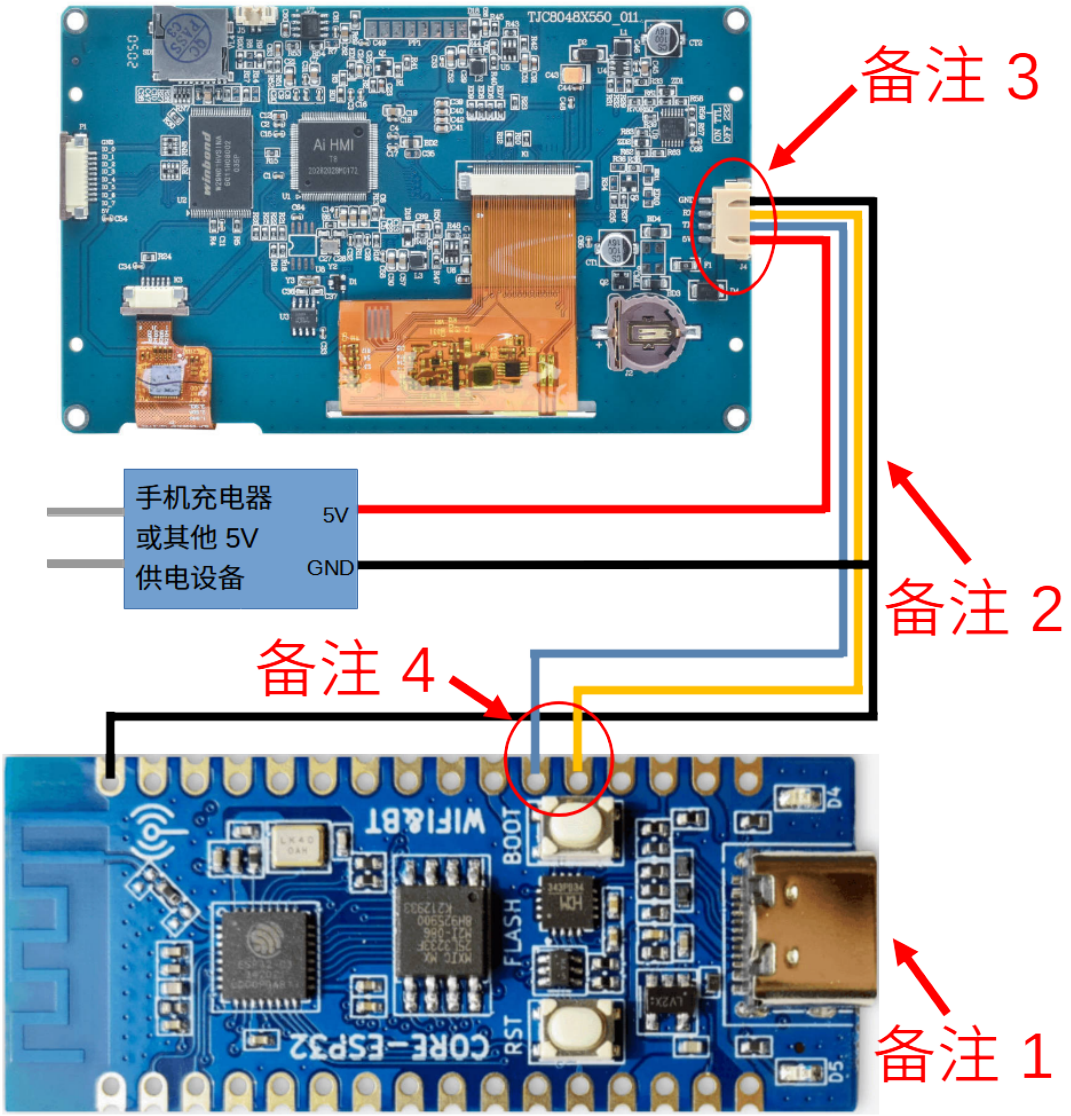
串口屏怎么下载程序

通过串口下载工程到串口屏 (page 144)

使用 *TFT* 文件下载助手 *(TFTFileDownload)* 通过串口下载工程到串口屏 (page 150)

通过 *SD* 卡下载工程到串口屏 (page 155)

2.12.4.2 ESP32C3 与串口屏的连接方式



备注 1： 给 ESP32 下载程序和供电还是通过这个接口

备注 2： 两边的 GND 要接在一起 (共地)，才能正常通讯

备注 3： 如果要通过 usb 转 ttl 给屏幕下载程序，要断开和 ESP32 的连接，将屏幕接口和 usb 转 ttl 接在一起才能下载

备注 4： ESP32 的 TX 接串口屏的 RX， ESP32 的 RX 接串口屏的 TX

2.12.4.3 ESP32C3 代码

注意: 以下代码仅为演示代码，用于测试显示屏能实现最基本的通信功能，如果您需要在正式产品中进行使用，请根据自己的需求对代码进行相应的优化和修改，或以自己的方式实现相应的功能

注意: 以下代码适用于 ESP32C3

arduino 只支持 utf8 编码，因此需要发送汉字时，在新建串口屏工程时请选择 UTF8 编码，导入字库时也要使用 UTF8 字库，参考*arduino 发送中文时乱码* (page 1431)

```
//arduino IDE 上的型号选择AirM2M CORE ESP32C3
//arduino的GND接串口屏或串口工具的GND,共地
//arduino的IO4接串口屏或串口工具的RX
//arduino的IO5接串口屏或串口工具的TX
//arduino的5V接串口屏的5V,如果是串口工具,不用接5V也可以
//根据自己的实际修改对应的TX_Pin和RX_Pin
#define TJC Serial
#define TX_Pin 4
#define RX_Pin 5
#define FRAME_LENGTH 7
#define LED_PIN0 12
#define LED_PIN1 13

int a;
unsigned long nowtime;
void setup() {
  // put your setup code here, to run once:
  //初始化串口
  TJC.begin(115200, SERIAL_8N1, RX_Pin, TX_Pin);
  //TJC.begin(115200);
  //因为串口屏开机会发送88 ff ff ff,所以要清空串口缓冲区
  while (TJC.read() >= 0); //清空串口缓冲区
  TJC.print("page main\xff\xff\xff"); //发送命令让屏幕跳转到main页面
  nowtime = millis(); //获取当前已经运行的时间

  //初始化esp32c3上面的LED灯
  pinMode(LED_PIN0, OUTPUT);
  pinMode(LED_PIN1, OUTPUT);
  digitalWrite(LED_PIN0, LOW);
  digitalWrite(LED_PIN1, LOW);

}

void loop() {
  // put your main code here, to run repeatedly:
  char str[100];
  if (millis() >= nowtime + 1000) {
    nowtime = millis(); //获取当前已经运行的时间

    //用sprintf来格式化字符串，给n0的val属性赋值
    sprintf(str, "n0.val=%d\xff\xff\xff", a);
    //把字符串发送出去
    TJC.print(str);

    //用sprintf来格式化字符串，给t0的txt属性赋值
    sprintf(str, "t0.txt=\"现在是%d\"\xff\xff\xff", a);
    //把字符串发送出去
```

(续下页)

(接上页)

```
    TJC.print(str);


    //用sprintf来格式化字符串，触发b0的按下事件,直接把结束符整合在字符串中
    sprintf(str, "click b0,1\xff\xff\xff");
    //把字符串发送出去
    TJC.print(str);

    delay(50);  //延时50ms,才能看清楚点击效果

    //用sprintf来格式化字符串，触发b0的弹起事件,直接把结束符整合在字符串中
    sprintf(str, "click b0,0\xff\xff\xff");
    //把字符串发送出去
    TJC.print(str);



    a++;
  }

  //串口数据格式:
  //串口数据帧长度: 7字节
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      1字节     1字节     1字节      0xffffff
  //当参数是01时
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      01      led编号  led状态     0xffffff
  //例子1: 上位机代码  printh 55 01 01 00 ff ff ff  含义: 1号led关闭
  //例子2: 上位机代码  printh 55 01 04 01 ff ff ff  含义: 4号led打开
  //例子3: 上位机代码  printh 55 01 00 01 ff ff ff  含义: 0号led打开
  //例子4: 上位机代码  printh 55 01 04 00 ff ff ff  含义: 4号led关闭

  //当参数是02或03时
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      02/03    滑动值     00     0xffffff
  //例子1: 上位机代码  printh 55 02 64 00 ff ff ff  含义: h0.val=100
  //例子2: 上位机代码  printh 55 02 00 00 ff ff ff  含义: h0.val=0
  //例子3: 上位机代码  printh 55 03 64 00 ff ff ff  含义: h1.val=100
  //例子4: 上位机代码  printh 55 03 00 00 ff ff ff  含义: h1.val=0


  //当串口缓冲区大于等于一帧的长度时
  while (TJC.available() >= FRAME_LENGTH) {
    unsigned char ubuffer[FRAME_LENGTH];
    //从串口缓冲读取1个字节但不删除
    unsigned char frame_header = TJC.peek();
    //当获取的数据是包头(0x55)时
    if (frame_header == 0x55) {
      //从串口缓冲区读取7字节
      TJC.readBytes(ubuffer, FRAME_LENGTH);
      if (ubuffer[4] == 0xff && ubuffer[5] == 0xff && ubuffer[6] == 0xff) {
        if(ubuffer[1] == 0x01)
        {
          //下发的是LED信息
          sprintf(str, "msg.txt=\"led %d is %s\"\xff\xff\xff", ubuffer[2], ubuffer[3] ?
↪"on" : "off");
          TJC.print(str);
          if(ubuffer[2]==0x00)
          {
            //设置ESP32C3上的LED灯1
            digitalWrite(LED_PIN0, ubuffer[3]);
```

(续下页)

(接上页)

```
          }else if(ubuffer[2]==0x01)
          {
            //设置ESP32C3上的LED灯2
            digitalWrite(LED_PIN1, ubuffer[3]);
          }

        }else if(ubuffer[1] == 0x02)
        {
          //下发的是滑动条h0.val的信息
          sprintf(str, "msg.txt=\"h0.val is %d\"\xff\xff\xff", ubuffer[2]);
          TJC.print(str);

        }else if(ubuffer[1] == 0x03)
        {
          //下发的是滑动条h1.val的信息
          sprintf(str, "msg.txt=\"h1.val is %d\"\xff\xff\xff", ubuffer[2]);
          TJC.print(str);

        }

      }
    } else {
      TJC.read();  //从串口缓冲读取1个字节并删除
    }
  }
}
```

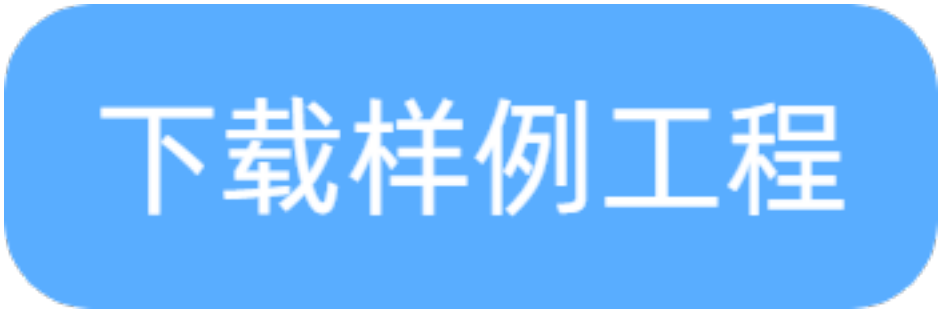
其他参考链接

屏幕通电后不断的闪烁(供电不足导致不断重启) (page 1244)

2.12.5 ESP32S3(源地) 与串口屏通讯

提示： 以下的教程均基于 arduino2.0 以上的版本，在低于 arduino2.0 的版本下编译可能会报错

2.12.5.1 ESP32S3(源地) 与串口屏通讯工程下载



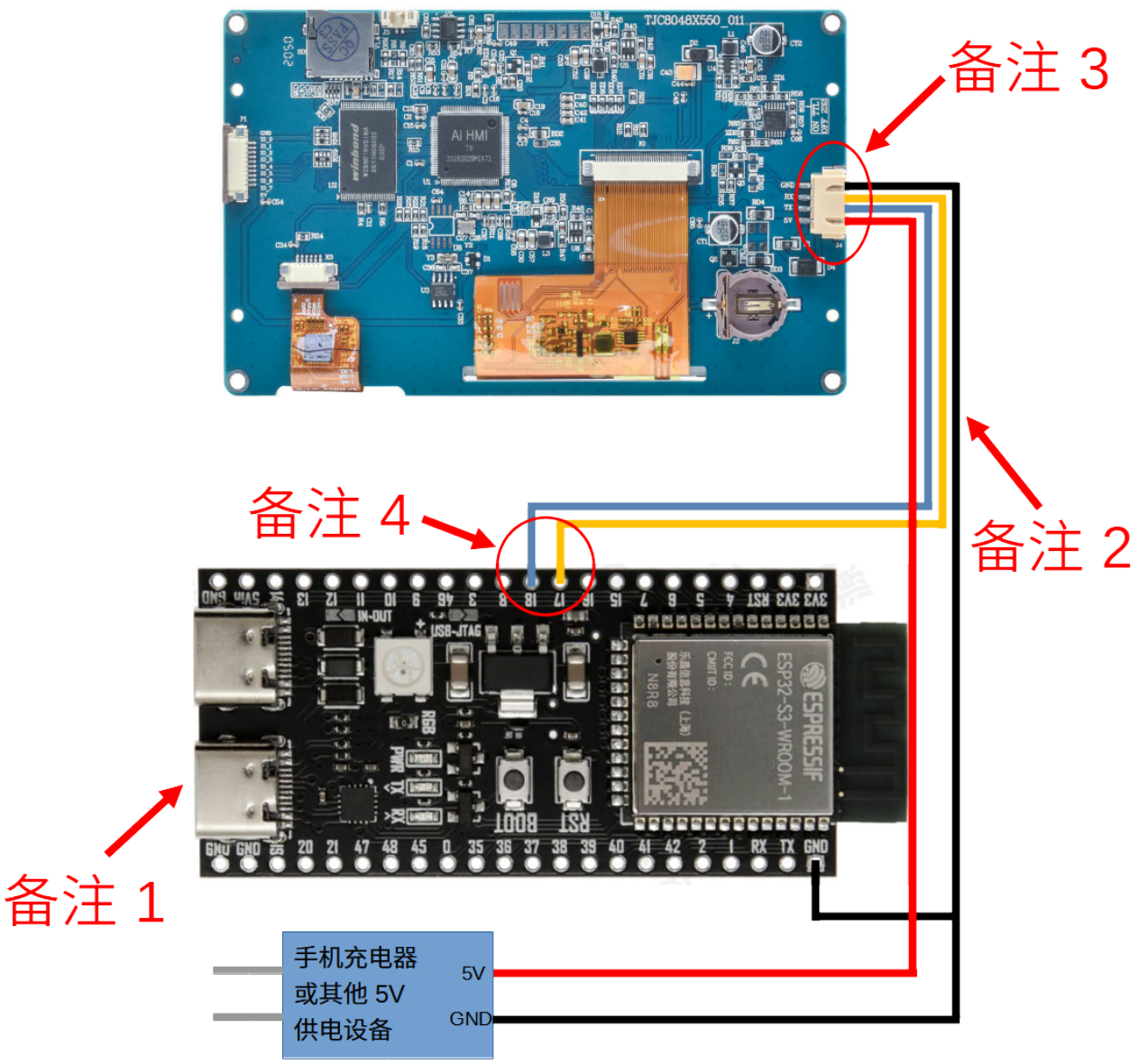
串口屏怎么下载程序

通过串口下载工程到串口屏 (page 144)

使用 *TFT* 文件下载助手 *(TFTFileDownload)* 通过串口下载工程到串口屏 (page 150)

通过 *SD* 卡下载工程到串口屏 (page 155)

2.12.5.2 ESP32S3(源地) 与串口屏的连接方式



备注 1： 给 ESP32 下载程序和供电还是通过这个接口

备注 2： 两边的 GND 要接在一起 (共地)，才能正常通讯

备注 3： 如果要通过 usb 转 ttl 给屏幕下载程序，要断开和 ESP32 的连接，将屏幕接口和 usb 转 ttl 接在一起才能下载

备注 4： ESP32 的 TX 接串口屏的 RX， ESP32 的 RX 接串口屏的 TX

2.12.5.3 ESP32S3(源地) 代码

注意: 以下代码仅为演示代码，用于测试显示屏能实现最基本的通信功能，如果您需要在正式产品中进行使用，请根据自己的需求对代码进行相应的优化和修改，或以自己的方式实现相应的功能

注意: 以下代码适用于 ESP32S3

arduino 只支持 utf8 编码，因此需要发送汉字时，在新建串口屏工程时请选择 UTF8 编码，导入字库时也要使用 UTF8 字库，参考*arduino 发送中文时乱码* (page 1431)

```
//arduino IDE 上的型号选择ESP32S3 Dev Module
//arduino的GND接串口屏或串口工具的GND,共地
//arduino的9接串口屏或串口工具的RX
//arduino的8接串口屏或串口工具的TX
//arduino的5V接串口屏的5V,如果是串口工具,不用接5V也可以
//根据自己的实际修改对应的TX_Pin和RX_Pin
#define TJC Serial1
#define TX_Pin 17
#define RX_Pin 18
#define FRAME_LENGTH 7

int a;
unsigned long nowtime;
void setup() {
  // put your setup code here, to run once:
  //初始化串口
  TJC.begin(115200, SERIAL_8N1, RX_Pin, TX_Pin);

  //因为串口屏开机会发送88 ff ff ff,所以要清空串口缓冲区
  while (TJC.read() >= 0); //清空串口缓冲区
  TJC.print("page main\xff\xff\xff"); //发送命令让屏幕跳转到main页面
  nowtime = millis(); //获取当前已经运行的时间
}

void loop() {
  // put your main code here, to run repeatedly:
  char str[100];
  if (millis() >= nowtime + 1000) {
    nowtime = millis(); //获取当前已经运行的时间

    //用sprintf来格式化字符串，给n0的val属性赋值
    sprintf(str, "n0.val=%d\xff\xff\xff", a);
    //把字符串发送出去
    TJC.print(str);

    //用sprintf来格式化字符串，给t0的txt属性赋值
    sprintf(str, "t0.txt=\"现在是%d\"\xff\xff\xff", a);
    //把字符串发送出去
    TJC.print(str);


    //用sprintf来格式化字符串，触发b0的按下事件,直接把结束符整合在字符串中
    sprintf(str, "click b0,1\xff\xff\xff");
    //把字符串发送出去
    TJC.print(str);

    delay(50);  //延时50ms,才能看清楚点击效果
```

(续下页)

(接上页)

```
    //用sprintf来格式化字符串，触发b0的弹起事件,直接把结束符整合在字符串中
    sprintf(str, "click b0,0\xff\xff\xff");
    //把字符串发送出去
    TJC.print(str);

    a++;
  }

  //串口数据格式:
  //串口数据帧长度: 7字节
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      1字节    1字节     1字节      0xffffff
  //当参数是01时
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      01      led编号  led状态     0xffffff
  //例子1: 上位机代码  printh 55 01 01 00 ff ff ff  含义: 1号led关闭
  //例子2: 上位机代码  printh 55 01 04 01 ff ff ff  含义: 4号led打开
  //例子3: 上位机代码  printh 55 01 00 01 ff ff ff  含义: 0号led打开
  //例子4: 上位机代码  printh 55 01 04 00 ff ff ff  含义: 4号led关闭

  //当参数是02或03时
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      02/03    滑动值     00     0xffffff
  //例子1: 上位机代码  printh 55 02 64 00 ff ff ff  含义: h0.val=100
  //例子2: 上位机代码  printh 55 02 00 00 ff ff ff  含义: h0.val=0
  //例子3: 上位机代码  printh 55 03 64 00 ff ff ff  含义: h1.val=100
  //例子4: 上位机代码  printh 55 03 00 00 ff ff ff  含义: h1.val=0


  //当串口缓冲区大于等于一帧的长度时
  while (TJC.available() >= FRAME_LENGTH) {
    unsigned char ubuffer[FRAME_LENGTH];
    //从串口缓冲读取1个字节但不删除
    unsigned char frame_header = TJC.peek();
    //当获取的数据是包头(0x55)时
    if (frame_header == 0x55) {
      //从串口缓冲区读取7字节
      TJC.readBytes(ubuffer, FRAME_LENGTH);
      if (ubuffer[4] == 0xff && ubuffer[5] == 0xff && ubuffer[6] == 0xff) {
        if(ubuffer[1] == 0x01)
        {
          //下发的是LED信息
          sprintf(str, "msg.txt=\"led %d is %s\"\xff\xff\xff", ubuffer[2], ubuffer[3] ?
↪"on" : "off");
          TJC.print(str);

        }else if(ubuffer[1] == 0x02)
        {
          //下发的是滑动条h0.val的信息
          sprintf(str, "msg.txt=\"h0.val is %d\"\xff\xff\xff", ubuffer[2]);
          TJC.print(str);

        }else if(ubuffer[1] == 0x03)
        {
          //下发的是滑动条h1.val的信息
          sprintf(str, "msg.txt=\"h1.val is %d\"\xff\xff\xff", ubuffer[2]);
          TJC.print(str);

        }
```

(续下页)

(接上页)

```
      }
    } else {
      TJC.read();  //从串口缓冲读取1个字节并删除
    }
  }
}
```

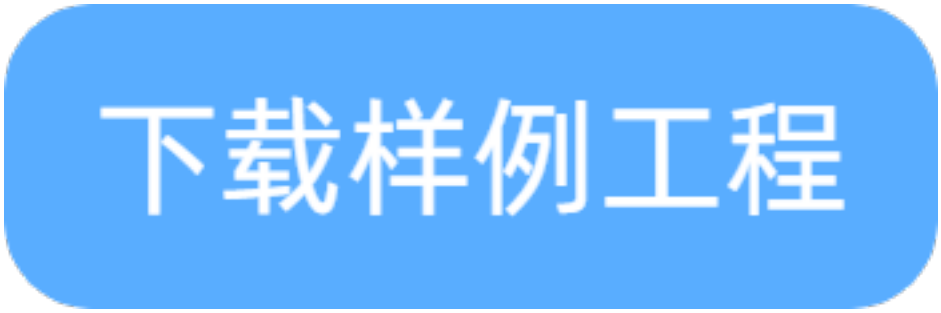
其他参考链接

屏幕通电后不断的闪烁(供电不足导致不断重启) (page 1244)

2.12.6 ESP32S3(合宙) 与串口屏通讯

提示： 以下的教程均基于 arduino2.0 以上的版本，在低于 arduino2.0 的版本下编译可能会报错

2.12.6.1 ESP32S3(合宙) 与串口屏通讯工程下载



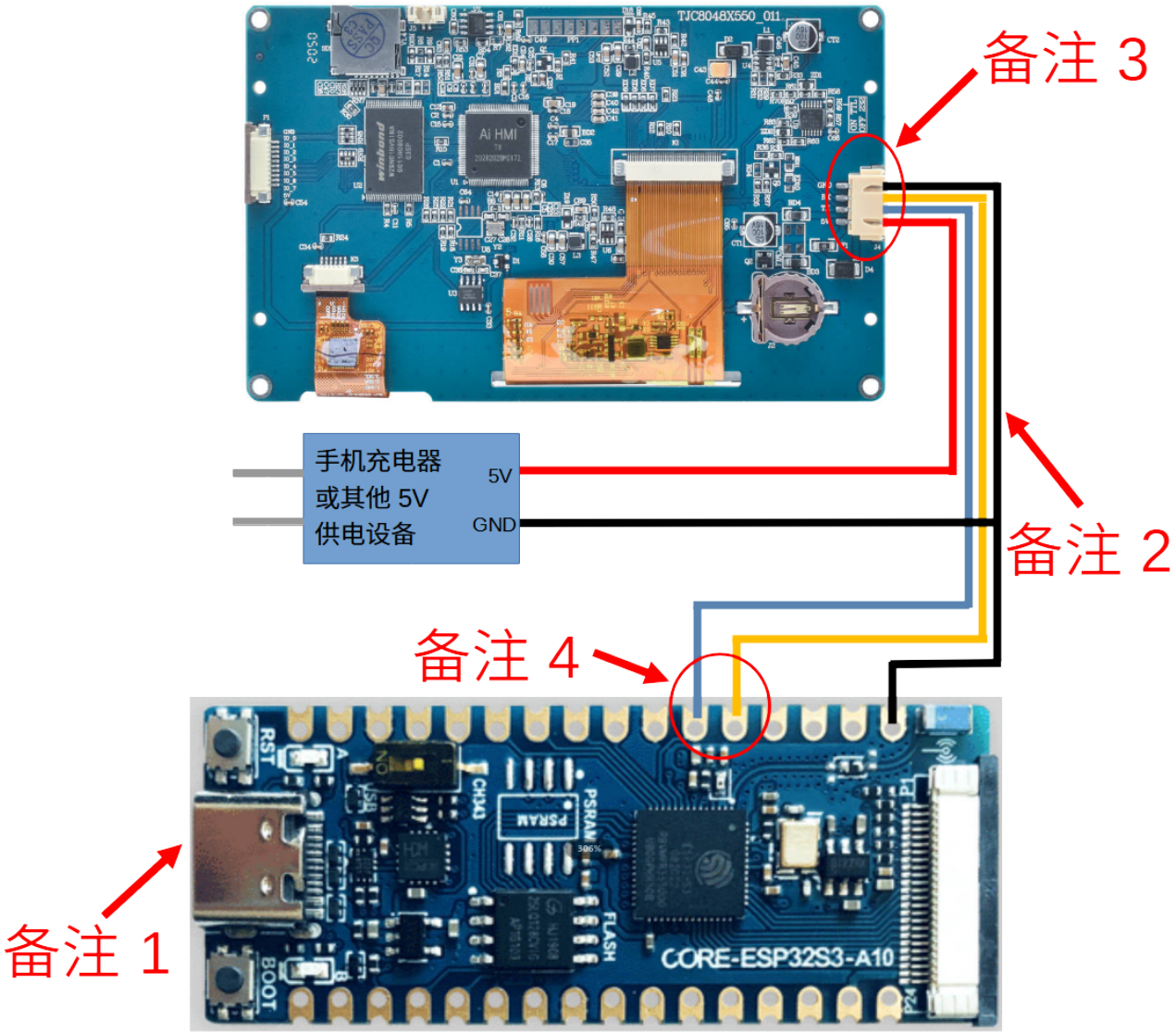
串口屏怎么下载程序

通过串口下载工程到串口屏 (page 144)

使用 *TFT* 文件下载助手 *(TFTFileDownload)* 通过串口下载工程到串口屏 (page 150)

通过 *SD* 卡下载工程到串口屏 (page 155)

2.12.6.2 ESP32S3(合宙) 与串口屏的连接方式



备注 1： 给 ESP32 下载程序和供电还是通过这个接口

备注 2： 两边的 GND 要接在一起 (共地)，才能正常通讯

备注 3： 如果要通过 usb 转 ttl 给屏幕下载程序，要断开和 ESP32 的连接，将屏幕接口和 usb 转 ttl 接在一起才能下载

备注 4： ESP32 的 TX 接串口屏的 RX， ESP32 的 RX 接串口屏的 TX

2.12.6.3 ESP32S3(合宙) 代码

注意: 以下代码仅为演示代码，用于测试显示屏能实现最基本的通信功能，如果您需要在正式产品中进行使用，请根据自己的需求对代码进行相应的优化和修改，或以自己的方式实现相应的功能

注意: 以下代码适用于合宙 ESP32S3

arduino 只支持 utf8 编码，因此需要发送汉字时，在新建串口屏工程时请选择 UTF8 编码，导入字库时也要使用 UTF8 字库，参考*arduino 发送中文时乱码* (page 1431)

```
//arduino IDE 上的型号选择ESP32S3 Dev Module
//arduino的GND接串口屏或串口工具的GND,共地
//arduino的IO4接串口屏或串口工具的RX
//arduino的IO5接串口屏或串口工具的TX
//arduino的5V接串口屏的5V,如果是串口工具,不用接5V也可以
//根据自己的实际修改对应的TX_Pin和RX_Pin
#define TJC Serial
#define TX_Pin 4
#define RX_Pin 5
#define FRAME_LENGTH 7
#define LED_PIN0 10
#define LED_PIN1 11

int a;
unsigned long nowtime;
void setup() {
  // put your setup code here, to run once:
  //初始化串口
  TJC.begin(115200, SERIAL_8N1, RX_Pin, TX_Pin);
  //TJC.begin(115200);
  //因为串口屏开机会发送88 ff ff ff,所以要清空串口缓冲区
  while (TJC.read() >= 0); //清空串口缓冲区
  TJC.print("page main\xff\xff\xff"); //发送命令让屏幕跳转到main页面
  nowtime = millis(); //获取当前已经运行的时间

  //初始化esp32c3上面的LED灯
  pinMode(LED_PIN0, OUTPUT);
  pinMode(LED_PIN1, OUTPUT);
  digitalWrite(LED_PIN0, LOW);
  digitalWrite(LED_PIN1, LOW);

}

void loop() {
  // put your main code here, to run repeatedly:
  char str[100];
  if (millis() >= nowtime + 1000) {
    nowtime = millis(); //获取当前已经运行的时间

    //用sprintf来格式化字符串，给n0的val属性赋值
    sprintf(str, "n0.val=%d\xff\xff\xff", a);
    //把字符串发送出去
    TJC.print(str);

    //用sprintf来格式化字符串，给t0的txt属性赋值
    sprintf(str, "t0.txt=\"现在是%d\"\xff\xff\xff", a);
    //把字符串发送出去
```

(续下页)

(接上页)

```
    TJC.print(str);


    //用sprintf来格式化字符串，触发b0的按下事件,直接把结束符整合在字符串中
    sprintf(str, "click b0,1\xff\xff\xff");
    //把字符串发送出去
    TJC.print(str);

    delay(50);  //延时50ms,才能看清楚点击效果

    //用sprintf来格式化字符串，触发b0的弹起事件,直接把结束符整合在字符串中
    sprintf(str, "click b0,0\xff\xff\xff");
    //把字符串发送出去
    TJC.print(str);



    a++;
  }

  //串口数据格式:
  //串口数据帧长度: 7字节
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      1字节     1字节     1字节      0xffffff
  //当参数是01时
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      01      led编号  led状态     0xffffff
  //例子1: 上位机代码  printh 55 01 01 00 ff ff ff  含义: 1号led关闭
  //例子2: 上位机代码  printh 55 01 04 01 ff ff ff  含义: 4号led打开
  //例子3: 上位机代码  printh 55 01 00 01 ff ff ff  含义: 0号led打开
  //例子4: 上位机代码  printh 55 01 04 00 ff ff ff  含义: 4号led关闭

  //当参数是02或03时
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      02/03    滑动值     00     0xffffff
  //例子1: 上位机代码  printh 55 02 64 00 ff ff ff  含义: h0.val=100
  //例子2: 上位机代码  printh 55 02 00 00 ff ff ff  含义: h0.val=0
  //例子3: 上位机代码  printh 55 03 64 00 ff ff ff  含义: h1.val=100
  //例子4: 上位机代码  printh 55 03 00 00 ff ff ff  含义: h1.val=0


  //当串口缓冲区大于等于一帧的长度时
  while (TJC.available() >= FRAME_LENGTH) {
    unsigned char ubuffer[FRAME_LENGTH];
    //从串口缓冲读取1个字节但不删除
    unsigned char frame_header = TJC.peek();
    //当获取的数据是包头(0x55)时
    if (frame_header == 0x55) {
      //从串口缓冲区读取7字节
      TJC.readBytes(ubuffer, FRAME_LENGTH);
      if (ubuffer[4] == 0xff && ubuffer[5] == 0xff && ubuffer[6] == 0xff) {
        if(ubuffer[1] == 0x01)
        {
          //下发的是LED信息
          sprintf(str, "msg.txt=\"led %d is %s\"\xff\xff\xff", ubuffer[2], ubuffer[3] ?
↪"on" : "off");
          TJC.print(str);
          if(ubuffer[2]==0x00)
          {
            //设置ESP32S3上的LED灯1
            digitalWrite(LED_PIN0, ubuffer[3]);
```

(续下页)

(接上页)

```
          }else if(ubuffer[2]==0x01)
          {
            //设置ESP32S3上的LED灯2
            digitalWrite(LED_PIN1, ubuffer[3]);
          }

        }else if(ubuffer[1] == 0x02)
        {
          //下发的是滑动条h0.val的信息
          sprintf(str, "msg.txt=\"h0.val is %d\"\xff\xff\xff", ubuffer[2]);
          TJC.print(str);

        }else if(ubuffer[1] == 0x03)
        {
          //下发的是滑动条h1.val的信息
          sprintf(str, "msg.txt=\"h1.val is %d\"\xff\xff\xff", ubuffer[2]);
          TJC.print(str);

        }

      }
    } else {
      TJC.read();  //从串口缓冲读取1个字节并删除
    }
  }
}
```

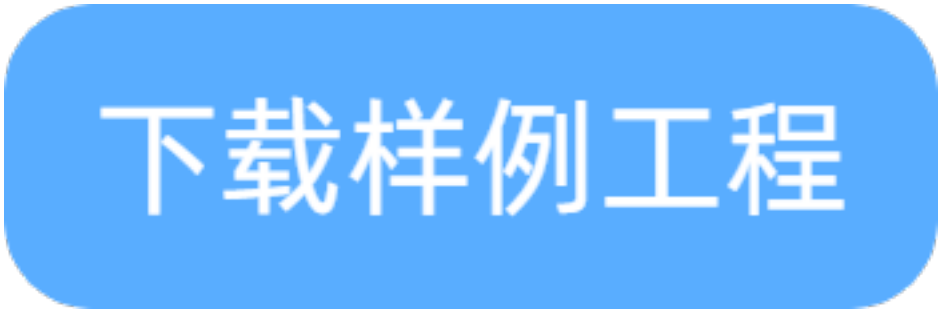
其他参考链接

屏幕通电后不断的闪烁(供电不足导致不断重启) (page 1244)

2.12.7 ESP32S3(嘉立创) 与串口屏通讯

提示： 以下的教程均基于 arduino2.0 以上的版本，在低于 arduino2.0 的版本下编译可能会报错

2.12.7.1 ESP32S3(嘉立创) 与串口屏通讯工程下载



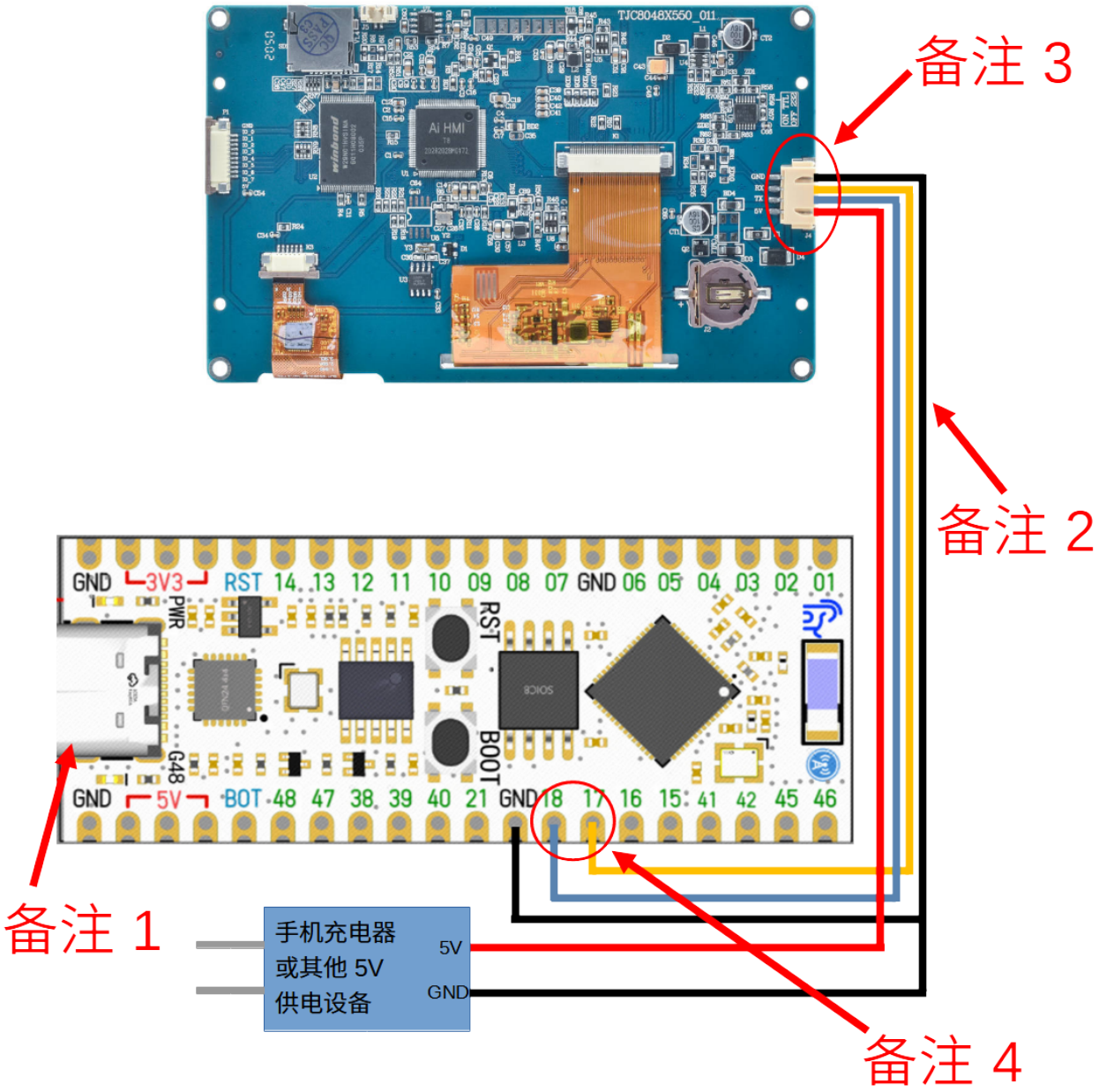
串口屏怎么下载程序

通过串口下载工程到串口屏 (page 144)

使用 *TFT* 文件下载助手 *(TFTFileDownload)* 通过串口下载工程到串口屏 (page 150)

通过 *SD* 卡下载工程到串口屏 (page 155)

2.12.7.2 ESP32S3(嘉立创) 与串口屏的连接方式



备注 1： 给 ESP32 下载程序和供电还是通过这个 type-c 接口

备注 2： 两边的 GND 要接在一起 (共地)，才能正常通讯

备注 3： 如果要通过 usb 转 ttl 给屏幕下载程序，要断开和 ESP32 的连接，将屏幕接口和 usb 转 ttl 接在一起才能下载

备注 4： ESP32 的 TX 接串口屏的 RX， ESP32 的 RX 接串口屏的 TX

2.12.7.3 ESP32S3(嘉立创) 代码

注意: 以下代码仅为演示代码，用于测试显示屏能实现最基本的通信功能，如果您需要在正式产品中进行使用，请根据自己的需求对代码进行相应的优化和修改，或以自己的方式实现相应的功能

注意: 以下代码适用于 ESP32S3

arduino 只支持 utf8 编码，因此需要发送汉字时，在新建串口屏工程时请选择 UTF8 编码，导入字库时也要使用 UTF8 字库，参考*arduino 发送中文时乱码* (page 1431)

```
//arduino IDE 上的型号选择ESP32S3 Dev Module
//arduino的GND接串口屏或串口工具的GND,共地
//arduino的9接串口屏或串口工具的RX
//arduino的8接串口屏或串口工具的TX
//arduino的5V接串口屏的5V,如果是串口工具,不用接5V也可以
//根据自己的实际修改对应的TX_Pin和RX_Pin
#define TJC Serial1
#define TX_Pin 17
#define RX_Pin 18
#define FRAME_LENGTH 7

int a;
unsigned long nowtime;
void setup() {
  // put your setup code here, to run once:
  //初始化串口
  TJC.begin(115200, SERIAL_8N1, RX_Pin, TX_Pin);

  //因为串口屏开机会发送88 ff ff ff,所以要清空串口缓冲区
  while (TJC.read() >= 0); //清空串口缓冲区
  TJC.print("page main\xff\xff\xff"); //发送命令让屏幕跳转到main页面
  nowtime = millis(); //获取当前已经运行的时间
}

void loop() {
  // put your main code here, to run repeatedly:
  char str[100];
  if (millis() >= nowtime + 1000) {
    nowtime = millis(); //获取当前已经运行的时间

    //用sprintf来格式化字符串，给n0的val属性赋值
    sprintf(str, "n0.val=%d\xff\xff\xff", a);
    //把字符串发送出去
    TJC.print(str);

    //用sprintf来格式化字符串，给t0的txt属性赋值
    sprintf(str, "t0.txt=\"现在是%d\"\xff\xff\xff", a);
    //把字符串发送出去
    TJC.print(str);


    //用sprintf来格式化字符串，触发b0的按下事件,直接把结束符整合在字符串中
    sprintf(str, "click b0,1\xff\xff\xff");
    //把字符串发送出去
    TJC.print(str);

    delay(50);  //延时50ms,才能看清楚点击效果
```

(续下页)

(接上页)

```
    //用sprintf来格式化字符串，触发b0的弹起事件,直接把结束符整合在字符串中
    sprintf(str, "click b0,0\xff\xff\xff");
    //把字符串发送出去
    TJC.print(str);

    a++;
  }

  //串口数据格式:
  //串口数据帧长度: 7字节
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      1字节    1字节     1字节      0xffffff
  //当参数是01时
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      01      led编号  led状态     0xffffff
  //例子1: 上位机代码  printh 55 01 01 00 ff ff ff  含义: 1号led关闭
  //例子2: 上位机代码  printh 55 01 04 01 ff ff ff  含义: 4号led打开
  //例子3: 上位机代码  printh 55 01 00 01 ff ff ff  含义: 0号led打开
  //例子4: 上位机代码  printh 55 01 04 00 ff ff ff  含义: 4号led关闭

  //当参数是02或03时
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      02/03    滑动值     00     0xffffff
  //例子1: 上位机代码  printh 55 02 64 00 ff ff ff  含义: h0.val=100
  //例子2: 上位机代码  printh 55 02 00 00 ff ff ff  含义: h0.val=0
  //例子3: 上位机代码  printh 55 03 64 00 ff ff ff  含义: h1.val=100
  //例子4: 上位机代码  printh 55 03 00 00 ff ff ff  含义: h1.val=0


  //当串口缓冲区大于等于一帧的长度时
  while (TJC.available() >= FRAME_LENGTH) {
    unsigned char ubuffer[FRAME_LENGTH];
    //从串口缓冲读取1个字节但不删除
    unsigned char frame_header = TJC.peek();
    //当获取的数据是包头(0x55)时
    if (frame_header == 0x55) {
      //从串口缓冲区读取7字节
      TJC.readBytes(ubuffer, FRAME_LENGTH);
      if (ubuffer[4] == 0xff && ubuffer[5] == 0xff && ubuffer[6] == 0xff) {
        if(ubuffer[1] == 0x01)
        {
          //下发的是LED信息
          sprintf(str, "msg.txt=\"led %d is %s\"\xff\xff\xff", ubuffer[2], ubuffer[3] ?
↪"on" : "off");
          TJC.print(str);

        }else if(ubuffer[1] == 0x02)
        {
          //下发的是滑动条h0.val的信息
          sprintf(str, "msg.txt=\"h0.val is %d\"\xff\xff\xff", ubuffer[2]);
          TJC.print(str);

        }else if(ubuffer[1] == 0x03)
        {
          //下发的是滑动条h1.val的信息
          sprintf(str, "msg.txt=\"h1.val is %d\"\xff\xff\xff", ubuffer[2]);
          TJC.print(str);

        }
```

(续下页)

(接上页)

```
      }
    } else {
      TJC.read();  //从串口缓冲读取1个字节并删除
    }
  }
}
```

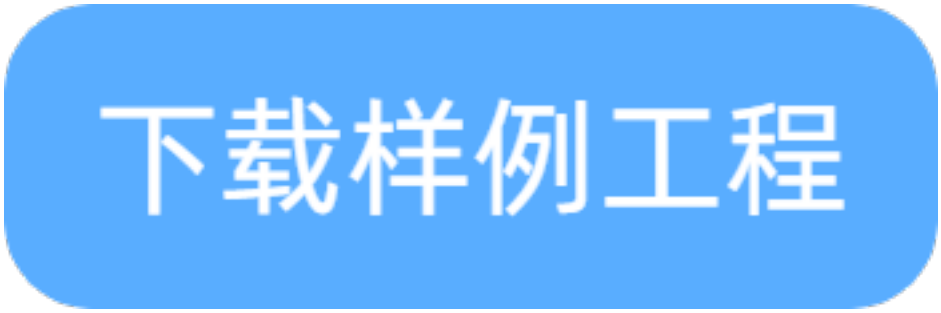
其他参考链接

屏幕通电后不断的闪烁(供电不足导致不断重启) (page 1244)

2.12.8 树莓派 pico 使用 arduino 与串口屏通讯

提示： 以下的教程均基于 arduino2.0 以上的版本，在低于 arduino2.0 的版本下编译可能会报错

2.12.8.1 树莓派 pico 与串口屏通讯工程下载



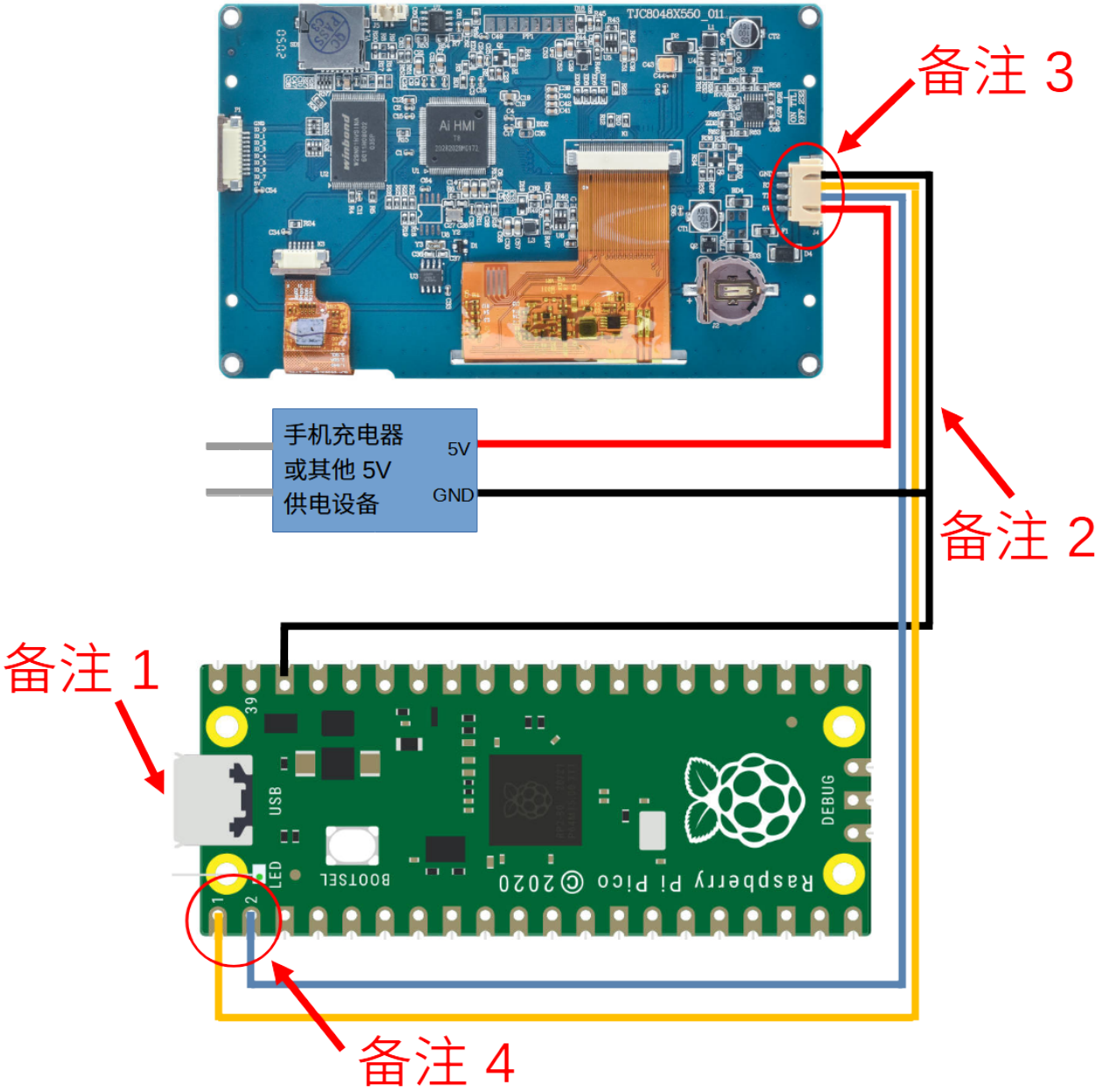
串口屏怎么下载程序

通过串口下载工程到串口屏 (page 144)

使用 *TFT* 文件下载助手 *(TFTFileDownload)* 通过串口下载工程到串口屏 (page 150)

通过 *SD* 卡下载工程到串口屏 (page 155)

2.12.8.2 树莓派 pico 使用 arduino 与串口屏通讯的连接方式



备注 1： 给树莓派下载程序和供电还是通过这个接口

备注 2： 两边的 GND 要接在一起 (共地)，才能正常通讯

备注 3： 如果要通过 usb 转 ttl 给屏幕下载程序，要断开和树莓派的连接，将屏幕接口和 usb 转 ttl 接在一起才能下载

备注 4： 树莓派的 TX 接串口屏的 RX，树莓派的 RX 接串口屏的 TX

2.12.8.3 树莓派 pico 使用 arduino 与串口屏通讯代码

注意: 以下代码仅为演示代码，用于测试显示屏能实现最基本的通信功能，如果您需要在正式产品中进行使用，请根据自己的需求对代码进行相应的优化和修改，或以自己的方式实现相应的功能

注意: 以下代码适用于树莓派 pico

树莓派 pico 第一次下载时会失败，此时需要断开电脑 USB，然后按住 pico 上的 BOOT 按钮，连接电脑 USB，此时就会被识别为一个 U 盘，就可以像其他 arduino 一样正常下载了

arduino 只支持 utf8 编码，因此需要发送汉字时，在新建串口屏工程时请选择 UTF8 编码，导入字库时也要使用 UTF8 字库，参考*arduino 发送中文时乱码* (page 1431)

```
//arduino IDE 上的型号选择Raspberry Pi Pico - Arduino Mbed OS RP2040 Boards
//pico的GND接串口屏或串口工具的GND,共地
//pico的IO0接串口屏或串口工具的RX
//pico的IO1接串口屏或串口工具的TX
//pico的5V接串口屏的5V,如果是串口工具,不用接5V也可以

#define TJC Serial1
#define FRAME_LENGTH 7


int a;
unsigned long nowtime;
void setup() {
  // put your setup code here, to run once:
  //初始化串口
  TJC.begin(115200);

  //因为串口屏开机会发送88 ff ff ff,所以要清空串口缓冲区
  while (TJC.read() >= 0); //清空串口缓冲区
  TJC.print("page main\xff\xff\xff"); //发送命令让屏幕跳转到main页面
  nowtime = millis(); //获取当前已经运行的时间

  //初始化树莓派pico上面的LED灯
  pinMode(LED_BUILTIN, OUTPUT);

}

void loop() {
  // put your main code here, to run repeatedly:
  char str[100];
  if (millis() >= nowtime + 1000) {
    nowtime = millis(); //获取当前已经运行的时间

    //用sprintf来格式化字符串，给n0的val属性赋值
    sprintf(str, "n0.val=%d\xff\xff\xff", a);
    //把字符串发送出去
    TJC.print(str);

    //用sprintf来格式化字符串，给t0的txt属性赋值
    sprintf(str, "t0.txt=\"现在是%d\"\xff\xff\xff", a);
    //把字符串发送出去
    TJC.print(str);
```

(续下页)

(接上页)

```
    //用sprintf来格式化字符串，触发b0的按下事件,直接把结束符整合在字符串中
    sprintf(str, "click b0,1\xff\xff\xff");
    //把字符串发送出去
    TJC.print(str);

    delay(50);  //延时50ms,才能看清楚点击效果

    //用sprintf来格式化字符串，触发b0的弹起事件,直接把结束符整合在字符串中
    sprintf(str, "click b0,0\xff\xff\xff");
    //把字符串发送出去
    TJC.print(str);



    a++;
  }

  //串口数据格式:
  //串口数据帧长度: 7字节
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      1字节    1字节     1字节      0xffffff
  //当参数是01时
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      01      led编号  led状态     0xffffff
  //例子1: 上位机代码  printh 55 01 01 00 ff ff ff  含义: 1号led关闭
  //例子2: 上位机代码  printh 55 01 04 01 ff ff ff  含义: 4号led打开
  //例子3: 上位机代码  printh 55 01 00 01 ff ff ff  含义: 0号led打开
  //例子4: 上位机代码  printh 55 01 04 00 ff ff ff  含义: 4号led关闭

  //当参数是02或03时
  //帧头      参数1     参数2    参数3        帧尾
  //0x55      02/03   滑动值     00    0xffffff
  //例子1: 上位机代码  printh 55 02 64 00 ff ff ff  含义: h0.val=100
  //例子2: 上位机代码  printh 55 02 00 00 ff ff ff  含义: h0.val=0
  //例子3: 上位机代码  printh 55 03 64 00 ff ff ff  含义: h1.val=100
  //例子4: 上位机代码  printh 55 03 00 00 ff ff ff  含义: h1.val=0


  //当串口缓冲区大于等于一帧的长度时
  while (TJC.available() >= FRAME_LENGTH) {
    unsigned char ubuffer[FRAME_LENGTH];
    //从串口缓冲读取1个字节但不删除
    unsigned char frame_header = TJC.peek();
    //当获取的数据是包头(0x55)时
    if (frame_header == 0x55) {
      //从串口缓冲区读取7字节
      TJC.readBytes(ubuffer, FRAME_LENGTH);
      if (ubuffer[4] == 0xff && ubuffer[5] == 0xff && ubuffer[6] == 0xff) {
        if(ubuffer[1] == 0x01)
        {
          //下发的是LED信息
          sprintf(str, "msg.txt=\"led %d is %s\"\xff\xff\xff", ubuffer[2], ubuffer[3] ?
↪"on" : "off");
          TJC.print(str);
          if(ubuffer[2]==0x00)
          {
            //设置树莓派pico上的LED灯1
            digitalWrite(LED_BUILTIN, ubuffer[3]);
          }

        }else if(ubuffer[1] == 0x02)
```

(续下页)

(接上页)

```
        {
          //下发的是滑动条h0.val的信息
          sprintf(str, "msg.txt=\"h0.val is %d\"\xff\xff\xff", ubuffer[2]);
          TJC.print(str);

        }else if(ubuffer[1] == 0x03)
        {
          //下发的是滑动条h1.val的信息
          sprintf(str, "msg.txt=\"h1.val is %d\"\xff\xff\xff", ubuffer[2]);
          TJC.print(str);

        }

      }
    } else {
      TJC.read();  //从串口缓冲读取1个字节并删除
    }
  }
}
```

其他参考链接

屏幕通电后不断的闪烁(供电不足导致不断重启) (page 1244)

2.12.9 arduino 通讯演示工程下载

提示：以下的教程均基于 arduino2.0 以上的版本，在低于 arduino2.0 的版本下编译可能会报错

演示工程下载链接：

串口屏工程：《上位机 UTF8 编码》演示工程下载

arduino 工程：《arduino 通讯》演示工程下载

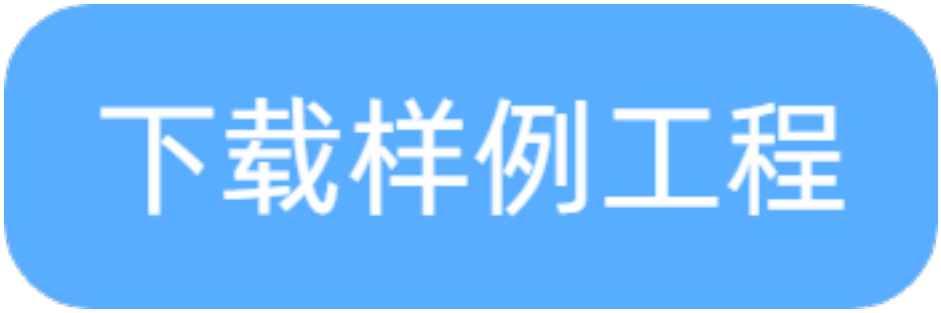
2.13 与 MicroPython 联调

对应的视频教程：arduino 解析协议

2.13.1 树莓派 pico 使用 MicroPython 与串口屏通讯

如何安装树莓派 pico 开发工具和配置请参考 树莓派 Pico 开发软件安装 (Thonny) 及烧录 (flash)

2.13.1.1 树莓派 pico 使用 MicroPython 与串口屏通讯工程下载



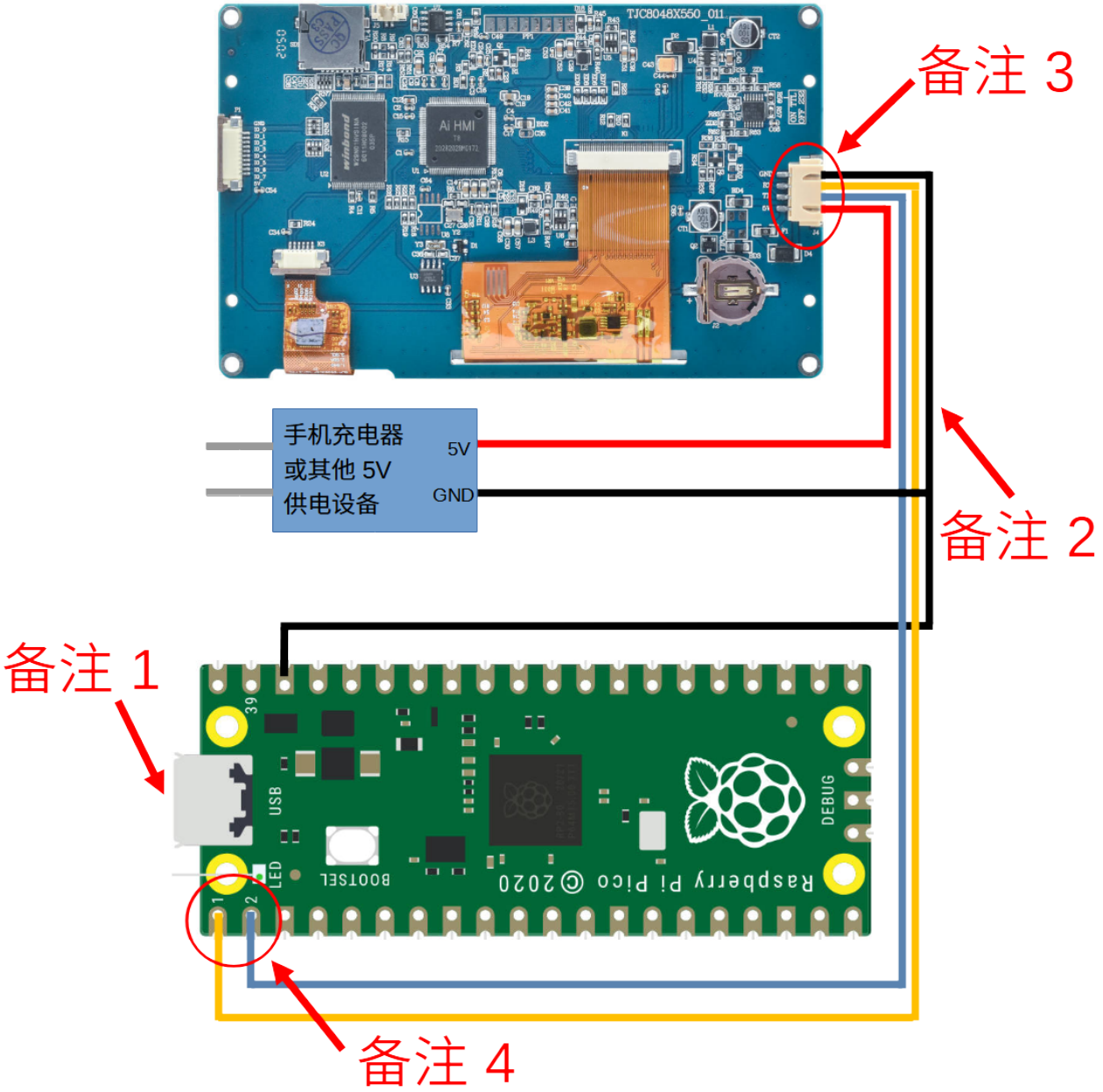
串口屏怎么下载程序

通过串口下载工程到串口屏 (page 144)

使用 *TFT* 文件下载助手 *(TFTFileDownload)* 通过串口下载工程到串口屏 (page 150)

通过 *SD* 卡下载工程到串口屏 (page 155)

2.13.1.2 树莓派 pico 使用 MicroPython 与串口屏通讯的连接方式



备注 1： 给树莓派下载程序和供电还是通过这个接口

备注 2： 两边的 GND 要接在一起 (共地)，才能正常通讯

备注 3： 如果要通过 usb 转 ttl 给屏幕下载程序，要断开和树莓派的连接，将屏幕接口和 usb 转 ttl 接在一起才能下载

备注 4： 树莓派的 TX 接串口屏的 RX，树莓派的 RX 接串口屏的 TX

2.13.1.3 树莓派 pico 使用 MicroPython 与串口屏通讯代码

注意： 以下代码仅为演示代码，用于测试显示屏能实现最基本的通信功能，如果您需要在正式产品中进行使用，请根据自己的需求对代码进行相应的优化和修改，或以自己的方式实现相应的功能

注意： 以下代码适用于树莓派 pico

thonny 只支持 utf8 编码，因此需要发送汉字时，在新建串口屏工程时请选择 UTF8 编码，导入字库时也要使用 UTF8 字库，参考*arduino 发送中文时乱码* (page 1431)

```
# 树莓派pico的GND接串口屏或串口工具的GND,共地
# 树莓派pico的GP0接串口屏或串口工具的RX
# 树莓派pico的GP1接串口屏或串口工具的TX
# 树莓派pico的5V接串口屏的5V,如果是串口工具,不用接5V也可以
import machine
import time

# 一帧的长度
FRAME_LENGTH=7

a=0
nowtime=0

# 这里设置串口 0 的波特率为 115200
uart = machine.UART(0, baudrate=115200)

# 使用 25 号引脚作为 LED 连接引脚
led_pin = machine.Pin(25, machine.Pin.OUT)


# 发送结束符
def sendEnd():
    # 要发送的十六进制数据
    hex_data = [0xff, 0xff, 0xff]

    # 将十六进制数据转换为字节数组并发送
    uart.write(bytearray(hex_data))


# 定义定时器回调函数
def tm0(timer):
    global a
    str = "n0.val={}".format(a)
    uart.write(str)
    sendEnd()

    str = "t0.txt=\"现在是{}\"".format(a)
    uart.write(str)
    sendEnd()


    str = "click b0,1"
    uart.write(str)
    sendEnd()

    time.sleep(0.05)
```

(续下页)

(接上页)

```
    str = "click b0,0"
    uart.write(str)
    sendEnd()
    a+=1


# 创建一个定时器
timer = machine.Timer()


# 初始化定时器，每 1 秒钟触发一次回调函数
timer.init(period=1000, mode=machine.Timer.PERIODIC, callback=tm0)

# ubuffer用于存放串口数据
ubuffer = []

while True:

    # 如果串口有数据，全部存放入ubuffer
    while uart.any():
        data = uart.read()
        if data:
            ubuffer.extend(data)

    # 当ubuffer的长度大于等于一帧的长度时
    if len(ubuffer) >= FRAME_LENGTH:
        # 判断帧头帧尾
        if ubuffer[0] == 0x55 and ubuffer[4] == 0xff and ubuffer[5] == 0xff and
↪ubuffer[6] == 0xff:
            # 如果下发的是led数据
            if ubuffer[1] == 0x01:
                status = ""
                if ubuffer[3] == 0x01:
                    status = "on"
                    if ubuffer[2] == 0x00:
                        led_pin.value(1)
                else:
                    status = "off"
                    if ubuffer[2] == 0x00:
                        led_pin.value(0)
                str = "msg.txt=\"led {} is {}\"".format(ubuffer[2], status)
                uart.write(str)
                sendEnd()
            # 如果下发的是进度条h0的数据
            elif ubuffer[1] == 0x02:
                str = "msg.txt=\"h0.val is {}\"".format(ubuffer[2])
                uart.write(str)
                sendEnd()
            # 如果下发的是进度条h1的数据
            elif ubuffer[1] == 0x03:
                str = "msg.txt=\"h1.val is {}\"".format(ubuffer[2])
                uart.write(str)
                sendEnd()
            # 删除1帧数据
            del ubuffer[:FRAME_LENGTH]


        else:
            # 删除最前面的1个数据
            del ubuffer[0]
```

其他参考链接

屏幕通电后不断的闪烁 (供电不足导致不断重启) (page 1244)

2.13.2 MicroPython 通讯演示工程下载

演示工程下载链接：

串口屏工程：《上位机 UTF8 编码》演示工程下载

MicroPython 工程：《MicroPython 通讯》演示工程下载

2.14 与 python 联调

串口屏怎么下载程序

通过串口下载工程到串口屏 (page 144)

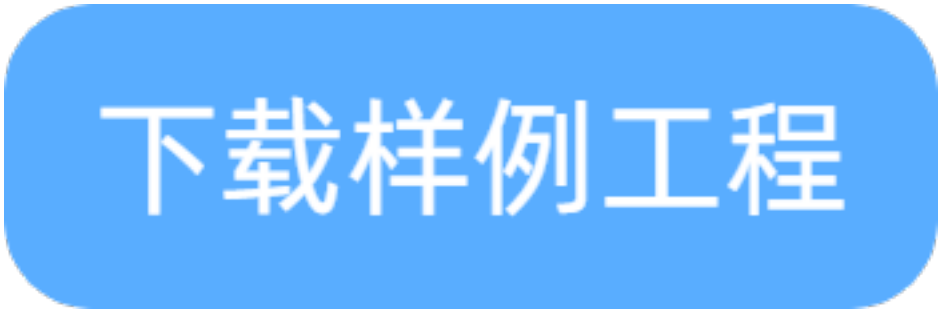
使用 TFT 文件下载助手 (TFTFileDownload) 通过串口下载工程到串口屏 (page 150)

通过 SD 卡下载工程到串口屏 (page 155)

2.14.1 python 发送数据给屏幕

注意: 素材说明: 只要能运行 python 的都能使用，可以在 windows、linux，mac 等操作系统下使用，可用在个人电脑、树莓派、工控机等。

2.14.1.1 python 发送数据给屏幕工程下载



注意: 在 windows 下建议使用 vscode 运行，如果无法通讯时，建议使用管理员权限打开 vscode 再运行，由于 pycharm 需要配置虚拟环境，不建议在 pycharm 下运行，可能会导致无法通讯。

此例程属于 python 代码，仅提供参考，淘晶驰不提供 python 代码的技术支持。

2.14.1.2 python 代码

```
import serial #导入模块
try:
#端口，GNU / Linux上的/ dev / ttyUSB0 等 或 Windows上的 COM1 等
#波特率，标准值：2400 4800 9600 19200 38400 57600 115200 230400 256000 512000 921600
#超时设置,None：永远等待操作，0为立即返回请求结果，其他值为等待超时时间(单位为秒)
# 打开串口，并得到串口对象
    ser=serial.Serial(port="COM9",baudrate=115200,timeout=5)
    print("串口详情参数：", ser)
# 写数据
    #注意：编码应与工程的编码一致（设备-字符编码，如果usarthmi中设置的为utf-
↪8，则这里也填写utf8，如果usarthmi中设置的为GB2312，则这里也填写GB2312也可以填写GBK)
    #result=ser.write("page page0".encode("utf-8"))
    #result=ser.write("page page2".encode("GB2312"))
    result=ser.write("t0.txt=\"淘晶驰电子！\"".encode("GB2312"))
# 发送结束符
    ser.write(bytes.fromhex('ff ff ff'))

    ser.close()#关闭串口
```

(续下页)

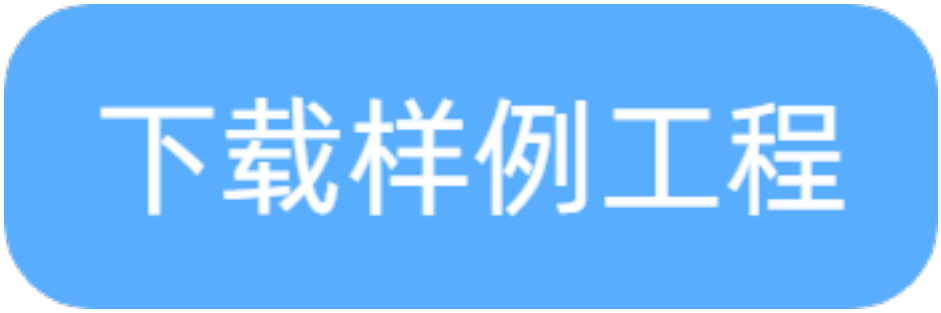
(接上页)

```
except Exception as e:
    print("---异常---: ",e)
```

2.14.2 python 获取电脑状态

注意： 素材说明: python 获取电脑状态目前只在 windows 下验证过

2.14.2.1 python 获取电脑状态工程下载



注意： 在 windows 下建议使用 vscode 运行，如果无法通讯时，建议使用管理员权限打开 vscode 再运行，由于 pycharm 需要配置虚拟环境，不建议在 pycharm 下运行，可能会导致无法通讯。

此例程属于 python 代码，仅提供参考，淘晶驰不提供 python 代码的技术支持。

2.14.2.2 python 获取电脑状态代码

```
import time
import psutil
import serial

time.sleep(3)

# function of Get CPU State;
def getCPUstate(interval=1):
    return (" CPU: " + str(psutil.cpu_percent(interval)) + "%")


# function of Get Memory
def getMemorystate():
    phymem = psutil.virtual_memory()
    line = "Memory: %5s%% %6s/%s" % (
        phymem.percent,
        str(int(phymem.used / 1024 / 1024)) + "M",
        str(int(phymem.total / 1024 / 1024)) + "M"
    )
    return line


def bytes2human(n):
    """
    >>> bytes2human(10000)
    '9.8 K'
    >>> bytes2human(100001221)
    '95.4 M'
    """
    symbols = ('K', 'M', 'G', 'T', 'P', 'E', 'Z', 'Y')
    prefix = {}
```

(续下页)

(接上页)

```
    for i, s in enumerate(symbols):
        prefix[s] = 1 << (i + 1) * 10
    for s in reversed(symbols):
        if n >= prefix[s]:
            value = float(n) / prefix[s]
            return '%.2f %s' % (value, s)
    return '%.2f B' % (n)


def poll(interval):
    """Retrieve raw stats within an interval window."""
    tot_before = psutil.net_io_counters()
    pnic_before = psutil.net_io_counters(pernic=True)
    # sleep some time
    time.sleep(interval)
    tot_after = psutil.net_io_counters()
    pnic_after = psutil.net_io_counters(pernic=True)
    # get cpu state
    cpu_state = getCPUstate(interval)
    # get memory
    memory_state = getMemorystate()
    return (tot_before, tot_after, pnic_before, pnic_after, cpu_state, memory_state)


def refresh_window(tot_before, tot_after, pnic_before, pnic_after, cpu_state, memory_
↪state):

    str = "t0.txt=\"" + time.asctime() + " | " + cpu_state + " | " + memory_state + "\""

    serial.write(str.encode("GB2312"))
    serial.write(bytes.fromhex('ff ff ff'))

    nic_names = pnic_after.keys()
    # nic_names.sort(key=lambda x: sum(pnic_after[x]), reverse=True)

    for name in nic_names:
        # 有线一般是以太网,无线则是WLAN,具体请查看"控制面板\网络和 Internet\网络连接"
        if name == "以太网":
            stats_before = pnic_before[name]
            stats_after = pnic_after[name]
            templ = "%-15s %15s %15s"
            str = "t1.txt=\"上传数据总量" + bytes2human(stats_after.bytes_sent) + "  ␣
↪上传速度:" + bytes2human(stats_after.bytes_sent - stats_before.bytes_sent) + '/s\"'
            serial.write(str.encode("GB2312"))
            serial.write(bytes.fromhex('ff ff ff'))
            str = "t2.txt=\"下载数据总量" + bytes2human(stats_after.bytes_recv) + "  ␣
↪下载速度:" + bytes2human(stats_after.bytes_recv - stats_before.bytes_recv) + '/s\"'
            serial.write(str.encode("GB2312"))
            serial.write(bytes.fromhex('ff ff ff'))


try:
    interval = 0
    portx = "COM9"
    bauds = 115200
    timex = 5
    serial = serial.Serial(port=portx,baudrate=bauds,timeout=timex)
    while 1:
        args = poll(interval)
        refresh_window(*args)
        interval = 1
```

(续下页)

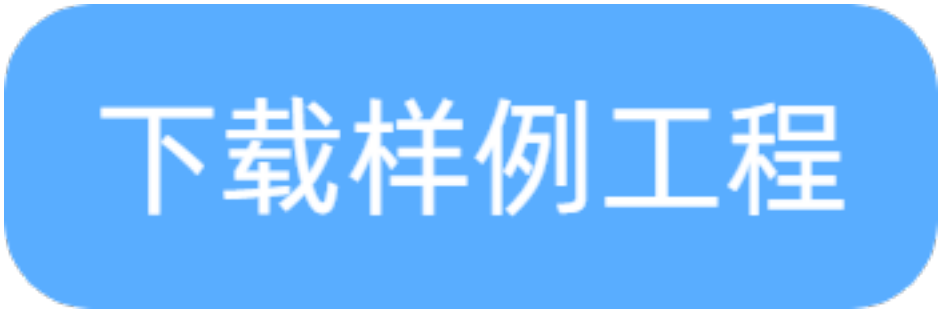
(接上页)

```
except (KeyboardInterrupt, SystemExit):
    pass
```

2.14.3 python 获取奥运奖牌

> **注意：** 素材说明: python 获取奥运奖牌目前只在 windows 下验证过

2.14.3.1 python 获取奥运奖牌通讯工程下载



> **注意：** 在 windows 下建议使用 vscode 运行，如果无法通讯时，建议使用管理员权限打开 vscode 再运行，由于 pycharm 需要配置虚拟环境，不建议在 pycharm 下运行，可能会导致无法通讯。
>
> 此例程属于 python 代码，仅提供参考，淘晶驰不提供 python 代码的技术支持。

2.14.3.2 python 获取奥运奖牌代码

```
# 使用前请安装pyserial库和requests库

import requests
import serial #导入模块
import time
import json
from threading import Timer

# 端口，GNU / Linux上的/ dev / ttyUSB0 等 或 Windows上的 COM1 等
# 波特率，标准值：2400 4800 9600 19200 38400 57600 115200 230400 256000 512000 921600
port="com9"
baud=115200

# 初始化任务时间
task1time=600
↪#每600秒请求一次服务器数据，避免造成服务器负担过大，网络菩萨提供的免费服务器请小力薅，常怀感恩之心
task2time=10    #每10秒向屏幕发送一次

# 初始化金银铜牌数量
gold = 0
silver = 0
bronze = 0

url="https://api.mg-tool.cn/v1/jpb/?country=中国"
def repeat_task1():
    response = requests.get(url)
    if response.status_code == 200:
        data = response.json()  #将响应内容解析为JSON
        print(data)
        # 找到中国的奖牌数据
        china_medal_data = next(item for item in data['data'] if item['country'] == '中国
```

(续下页)

(接上页)

```
↪')

        # 提取中国的金牌、银牌、铜牌数量
        global gold,silver,bronze
        gold = china_medal_data['gold']
        silver = china_medal_data['silver']
        bronze = china_medal_data['bronze']

        print(f"中国的金牌数：{gold}")
        print(f"中国的银牌数：{silver}")
        print(f"中国的铜牌数：{bronze}")

    else:
        print("获取数据失败：",response.status_code)

    Timer(task1time,repeat_task1).start()

def repeat_task2():
    try:
    #超时设置,None：永远等待操作，0为立即返回请求结果，其他值为等待超时时间(单位为秒)
    # 打开串口，并得到串口对象
        ser=serial.Serial(port=port,baudrate=baud,timeout=5)
        print("串口详情参数：", ser)
    # 写数据
        #注意：编码应与工程的编码一致（设备-字符编码，如果usarthmi中设置的为utf-
↪8，则这里也填写utf8，如果usarthmi中设置的为GB2312，则这里也填写GB2312也可以填写GBK)
        # 金牌
        str = "gold.val=%d" %gold
        result=ser.write(str.encode())
        print("发送的数据为："+str)
        # 发送结束符
        ser.write(bytes.fromhex('ff ff ff'))
        # 银牌
        str = "silver.val=%d" % silver
        result = ser.write(str.encode())
        print("发送的数据为："+str)
        # 发送结束符
        ser.write(bytes.fromhex('ff ff ff'))
        # 铜牌
        str = "bronze.val=%d" % bronze
        result = ser.write(str.encode())
        print("发送的数据为：" + str)
        # 发送结束符
        ser.write(bytes.fromhex('ff ff ff'))
        ser.close()#关闭串口

    except Exception as e:
        print("---异常---：",e)

    Timer(task2time, repeat_task2).start()

if __name__ == '__main__':
    repeat_task1()
    repeat_task2()
```

2.14.4 python 通讯演示工程下载

演示工程下载链接：

python 工程：《python 通讯》演示工程下载

高级应用：

python 实现的下载工具 (page 1011)

串口调试软件推荐使用《sscom5.13.1》或者《格西调试精灵》

下载链接:

《sscom5.13.1》

《格西调试精灵》

USB 转 ttl 工具请不要使用 ch340 或者 pl2303 或其他多功能的调试工具（例如同时支持 ttl，232,485，arm 芯片下载调试等功能的工具）

推荐使用 cp2102 芯片或者 ft232 芯片纯粹 USB 转 ttl 工具

3 所有控件详解

3.1 认识控件

提示： 视频教程：认识控件

3.1.1 控件是什么

控件是串口屏想实现相关功能的“功能组件”，需要串口屏显示什么内容就使用相对应的控件。

例如：

显示文本，用【文本控件】

显示图片，用【图片控件】

显示进度，用【进度条控件】

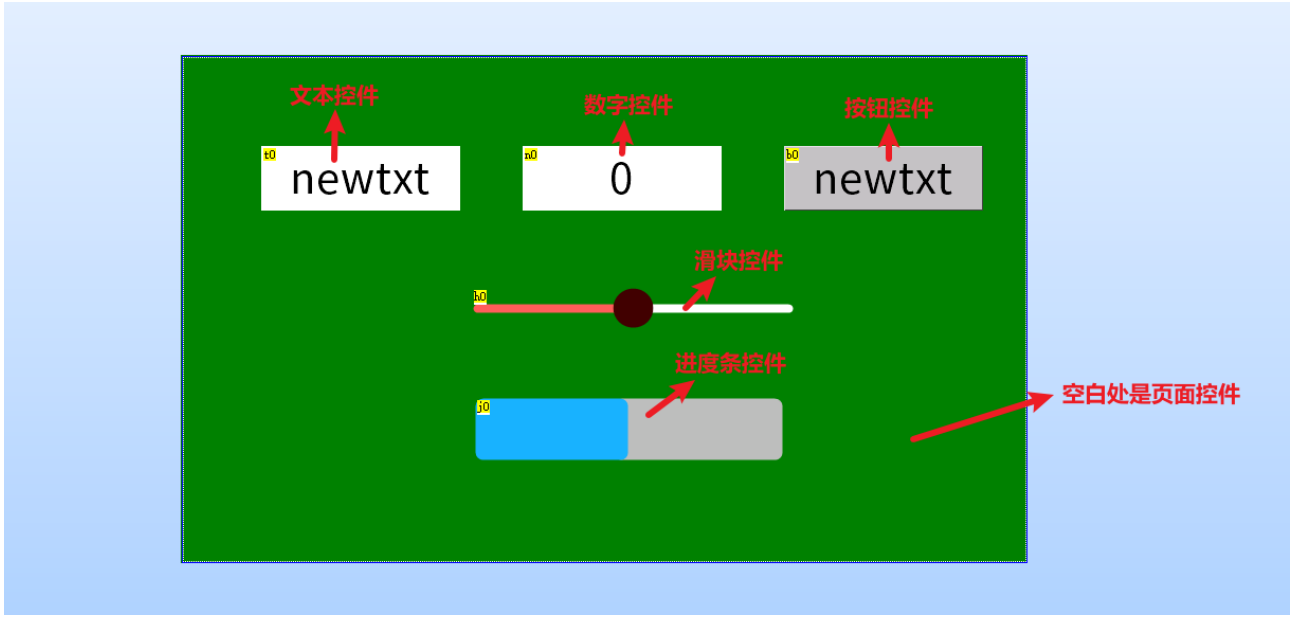
实现按键功能，用【按钮控件】

实现滑动功能，用【滑块控件】

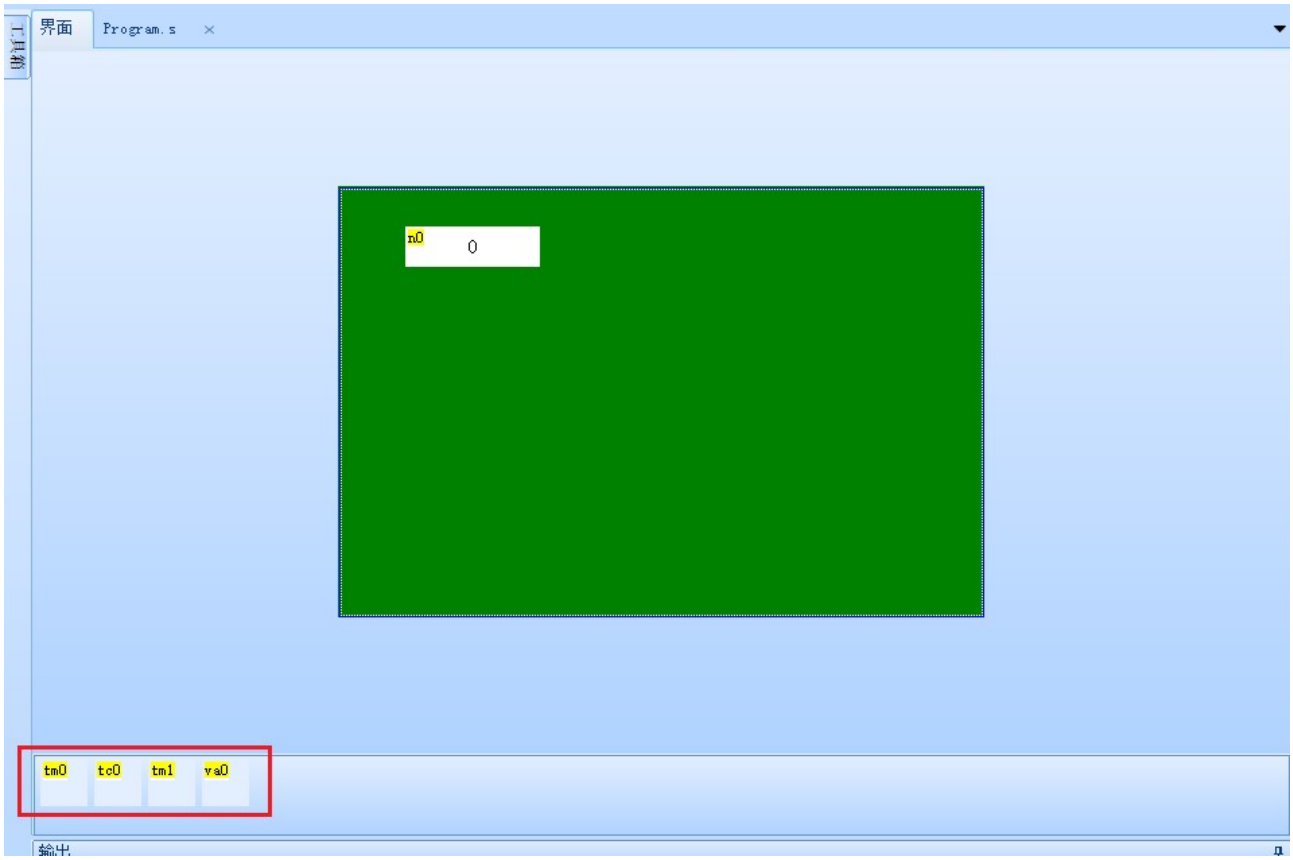
要播放视频，用【视频控件】

••• •••

在串口屏上位机的界面编辑窗口中，一切你能看到的，都是控件



有些控件是看不见的，比如定时器，触摸捕捉等控件，位于底部的特殊控件栏



3.1.2 控件种类有多少

《USART HMI》，目前已有“30 种”控件，具体是哪些控件请打开软件查看控件“工具箱”查阅（控件工具箱位置如下图所示），同时我们根据市场需求会持续增加新控件。

3.1.3 控件事件是什么

控件事件，指的是这个控件被操作时要执行的功能。

目前各种控件综合起来被操作的方式有以下几种类型：

1、触摸被按下：对应名称叫做【按下事件】

2、触摸被按下后弹起：对应名称叫做【弹起事件】

3、滑块控件被滑动：对应的名称叫做【滑动事件】

4、定时器定时运行：对应的名称叫做【定时事件】

5、动画播放完成：对应的名称叫做【播放完成事件】

6、视频播放完成：对应的名称叫做【播放完成事件】

3.1.4 控件属性解析

1、控件属性描述

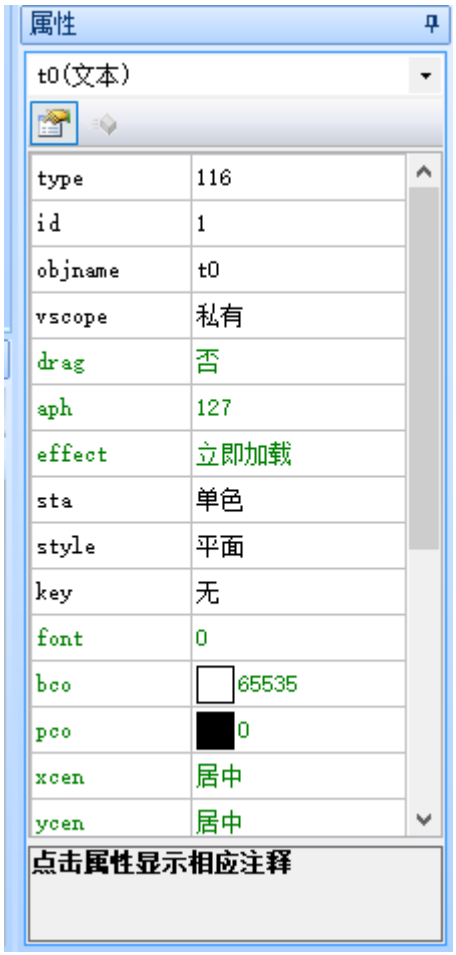
控件属性是控件自己的一些设置项，上面提到想要什么功能就选择对应的控件。

比如想要显示文本，就用文本控件，但是选择文本控件后，显示什么内容？什么字体？字体什么颜色？文本背景什么颜色？字体间距多少等等这些信息怎么设置呢？

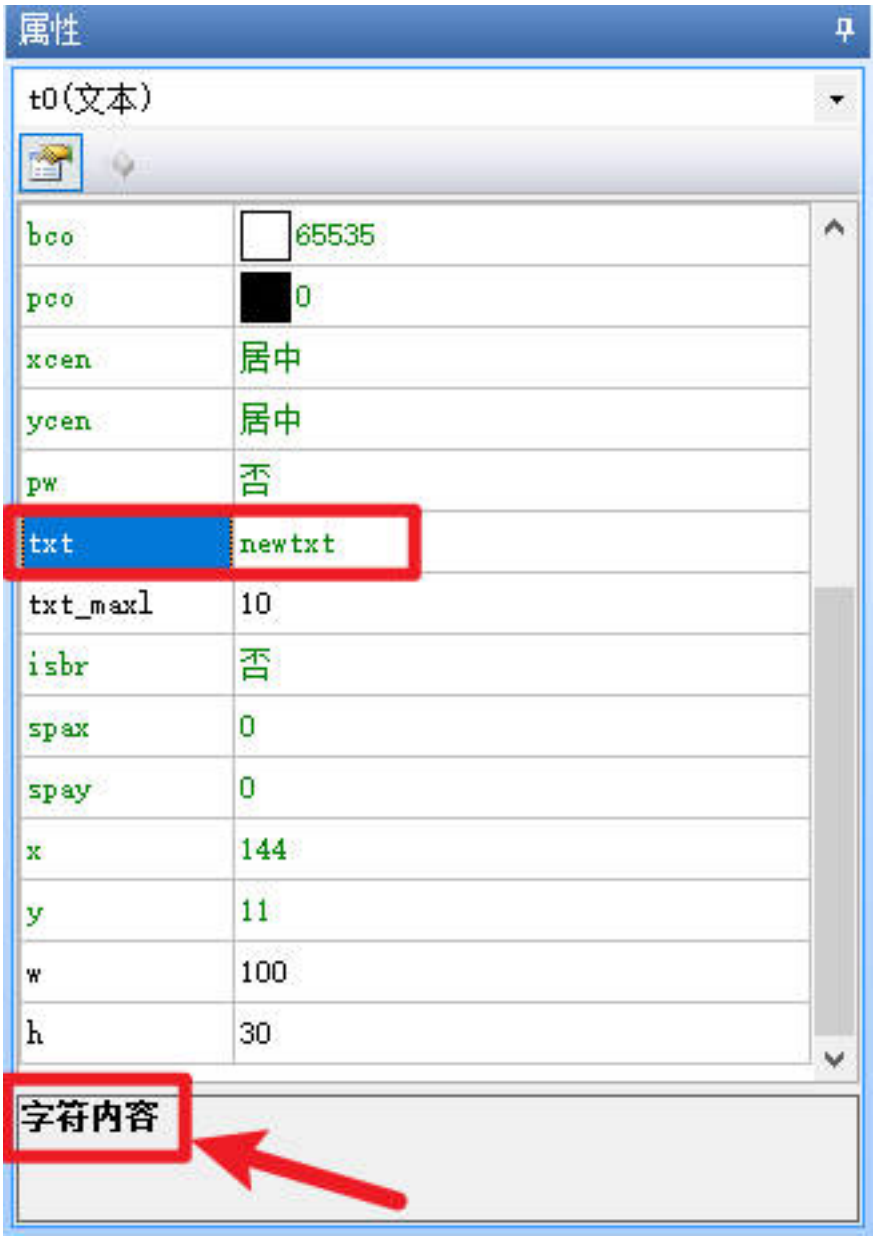
这就需要属性来定义了，这些信息都属于这个文本控件的属性，每个控件都有很多属性可以设置，用来定义他的显示风格。

通过对属性的简单编辑，便可将控件设置成您需要的效果；

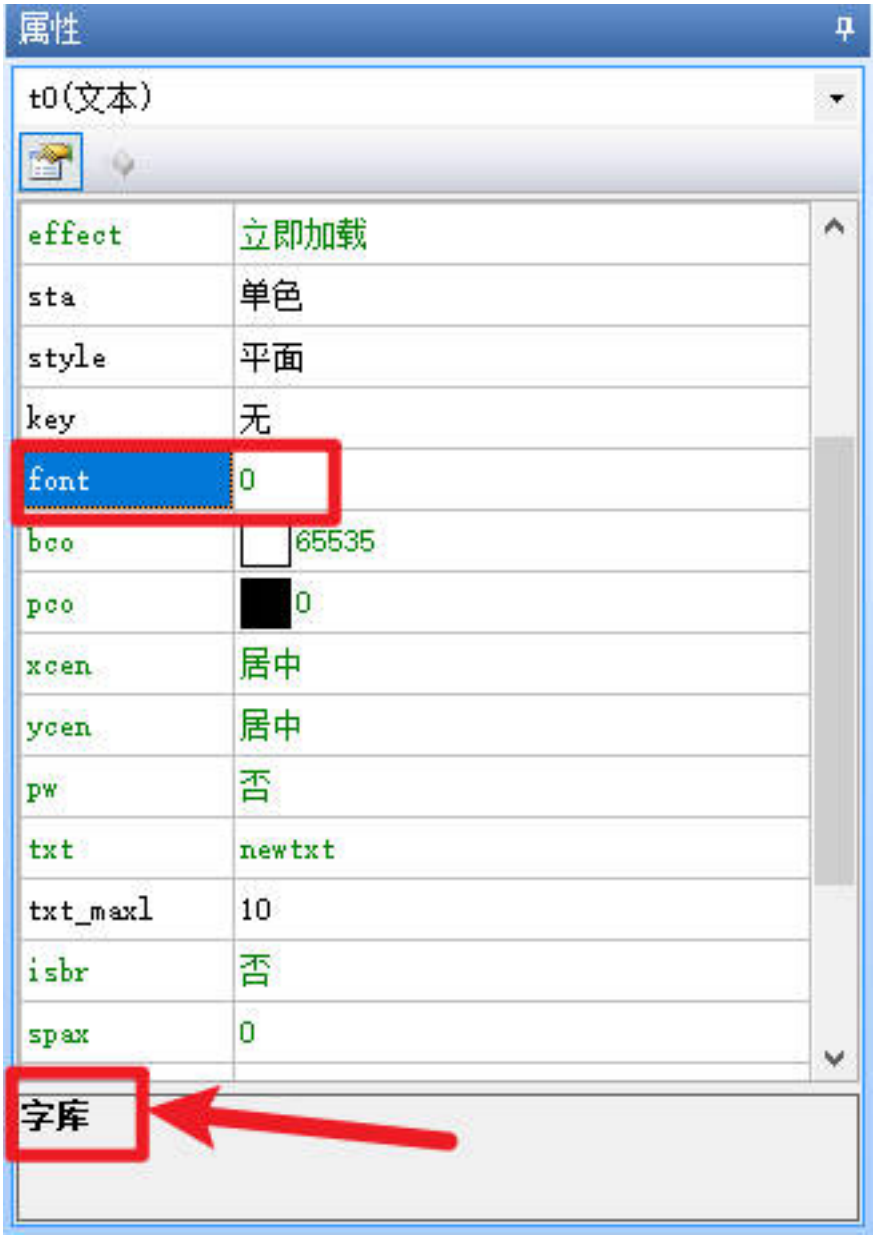
选中任一控件，可以在右下角看到其属性栏



选中属性栏的任一属性，最底部会显示该属性的注释，例如 txt 属性，底部对应显示注释的为 字符内容，即文本控件显示的字符内容

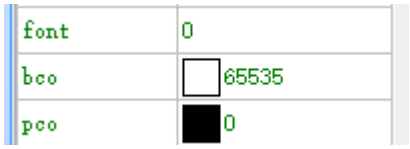


例如 font 属性，底部对应显示注释的为 字库，即文本控件调用的字库



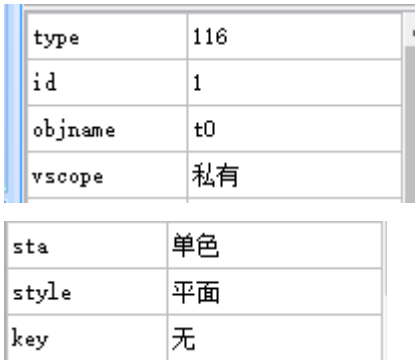
控件属性有 2 种颜色, 分别为绿色和黑色

绿色属性: 控件编辑时可设置 + 运行中可改变，例如字库，字体色，背景色等



font	0
bco	65535
pco	0

黑色属性: 控件编辑时可设置（运行中不可改变，例如 objname：控件名称）或不可更改（例如 type：控件类型）



type	116
id	1
objname	t0
vscope	私有
sta	单色
style	平面
key	无

提示： type 属性不可设置,objname 属性不可读取,id 属性需要通过左上角置顶置底按钮才能进行更改

3.1.5 控件属性读写

```
n0.val=100                //将n0赋值为100
t0.txt="淘晶驰电子"        //文本控件显示淘晶驰电子
prints t0.txt,0           //将t0的文本内容从串口发送出去。
```

更多示例请参考赋值操作 (page 827)

3.1.6 控件属性详解

3.1.6.1 type 属性

控件类型，可读，不可更改，不同类型的控件 type 属性不一样，相同类型的控件 type 属性一样

使用场景：

1、在键盘页面判断跳转过来的控件是什么类型的控件，不同的控件有不同的转换方法，详情请参考键盘如何获取到原始控件的值 (page 914)

2、在触摸捕捉中判断按下去的控件是什么类型

3.1.6.2 id 属性

控件 id，可通过上位机左上角的上下箭头置顶或置底，不能通过指令进行修改

每个页面中最底层都有一个页面控件，其 id 固定为 0，可以设置当前页面的背景图片，背景色等操作

除了页面控件，每个页面最多放 250 个控件，占据 id 1-250 ，每个页面各个控件之间 id 号是唯一的，不会重复

可以理解为控件的图层，id 大的控件会挡住 id 小的控件

使用场景：

通过名称组操作控件，请参考：数组/名称组使用说明 (page 865)

3.1.6.3 objname 属性

控件名称，可通过上位机进行修改，不可通过指令更改，不可读取

例如

```
t0.txt=n0.objname    //错误写法，objname属性不允许读取
n0.objname="n1"    //错误写法，objname属性不允许写入
```

3.1.6.4 vscope 属性

内存占用:0-私有;1-全局; 可通过上位机进行修改，可读取，不可通过指令更改

私有占用只能在当前页面被访问，全局占用可以在所有页面被访问

当设置为私有时，跳转页面后，该控件占用的内存会被释放

重新返回该页面后该控件会恢复到最初的设置，可通过上位机进行修改，不可通过指令更改

3.1.6.5 txt 属性

字符内容，通常出现在需要显示文本字符的控件中，例如文本控件，按钮控件，滚动文本控件等，修改此属性，就可以修改控件上显示的内容，可以理解为 c 语言中的字符串

3.1.6.6 val 属性

数值，通常出现在需要显示数值的控件中，例如数字控件，虚拟浮点数控件等，修改此属性，就可以修改控件上显示的内容吗，数值主要用于加减乘除等计算，可以理解为 c 语言中的有符号整形（signed int）

3.1.6.7 font 属性

控件所调用的字库 id 号，通常出现在需要显示文本字符和数值的控件中，例如文本控件，按钮控件，数字控件，数据记录控件等。导入不同的字库后修改此属性可以让控件显示不同的字体和字高

3.1.7 控件属性-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

赋值操作 (page 827)

3.1.8 控件属性-控件 id 对照表

表 3: 控件 id 对照表

控件类型	type 值
文本	116
滚动文本	55
数字	54
虚拟浮点数	59
按钮	98
进度条	106
图片	112
切图	113
触摸热区	109
触摸捕捉	5
指针	122
曲线波形控件	0
滑块	1
定时器	51
变量	52
双态按钮	53
复选框	56
单选框	57
二维码	58
状态开关	67
下拉框	61
选择文本	68
滑动文本	62
数据记录	66
文件浏览器	65
文件流	63
动画	2
视频	3
音频	4
外部图片	60
页面	121

3.2 页面控件

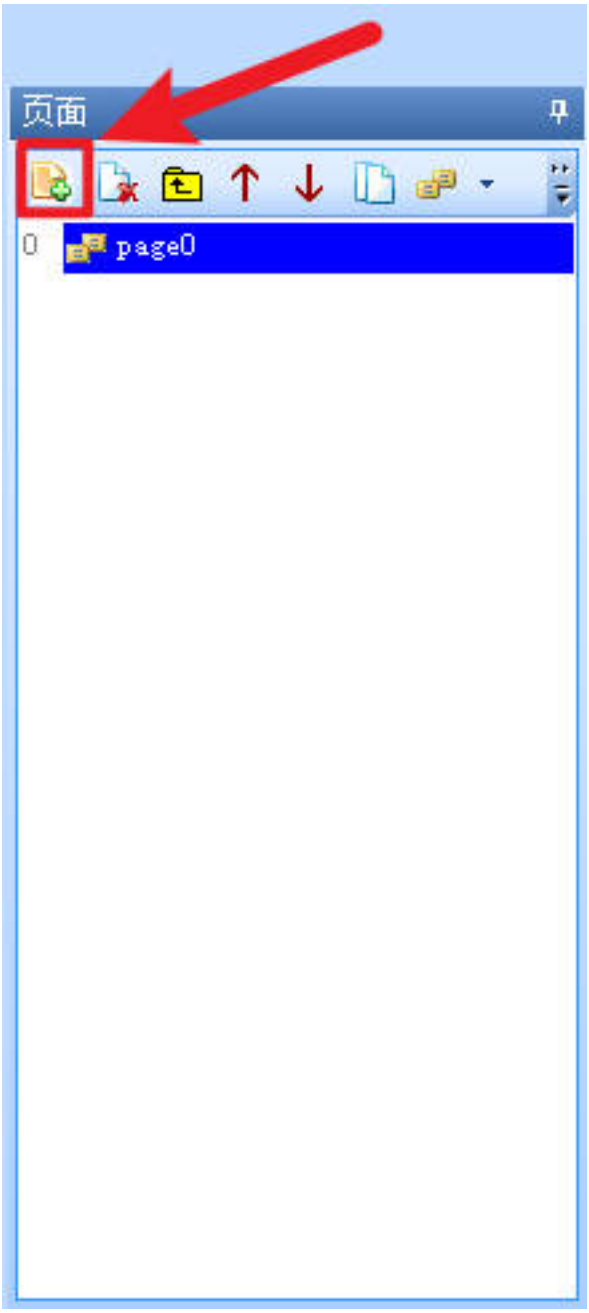
提示： 视频教程：页面控件

提示： 在上位机软件中每新建一个页面，会自动在页面中创建一个页面控件，其 id 固定为 0，页面控件名称与页面名称一致，因此页面内的其他控件不能和页面名称相同，因为这个名称被页面控件使用了

在页面空白处点击鼠标左键，会选中当前页面的页面控件

3.2.1 如何新建一个页面

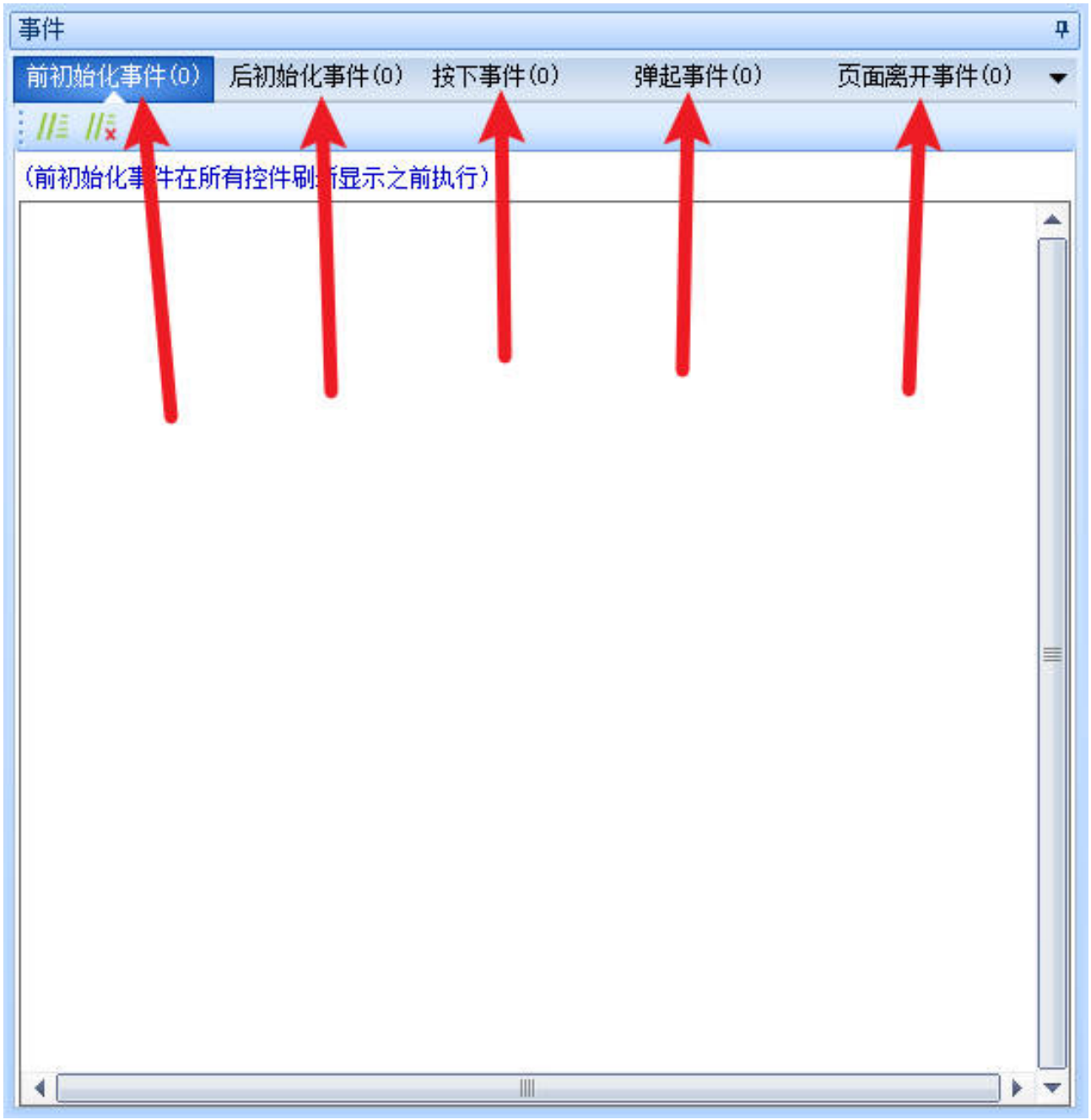
点击此按钮，会新建一个空白页面



例如新建的页面名称为 page1, 会自动在页面中创建一个页面控件 page1(与页面名称相同), 其 id 固定为 0。

页面
0 page0
1 page1
属性
page1(页面)
type 121
id 0
vscope 私有
effect 立即加载
up 255
down 255
left 255
right 255
sta 单色
bco 65535
x 0
y 0
w 800
h 480
控件ID

页面控件拥有以下五个事件



3.2.2 前初始化事件

每次在执行 page 命令时，在执行页面刷新操作以前，串口屏会自动执行一次《前初始化事件》中的代码。

3.2.3 后初始化事件

每次在执行 page 命令时，在页面刷新操作完成以后，串口屏会自动执行一次《后初始化事件》中的代码。

3.2.4 页面按下事件

显示区域内且没有控件的区域，触摸被按下的瞬间，串口屏会自动执行一次页面的《按下事件》中的代码。

3.2.5 页面弹起事件

显示区域内且没有控件的区域（以触摸按下是的坐标为准），触摸按下以后松开触摸的的瞬间，串口屏会自动执行一次页面的《弹起事件》中的代码。

3.2.6 页面离开事件

每次在执行 page 切换新的页面前，串口屏会自动执行一次当前页面的《页面离开事件》中的代码。

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

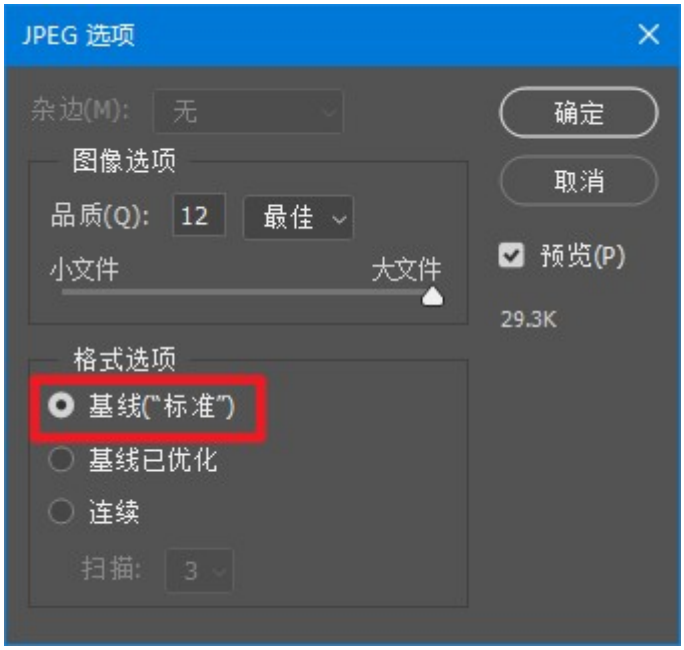
导入音频请参考: 如何导入音频 (page 1467)

3.2.7 页面控件-使用详解

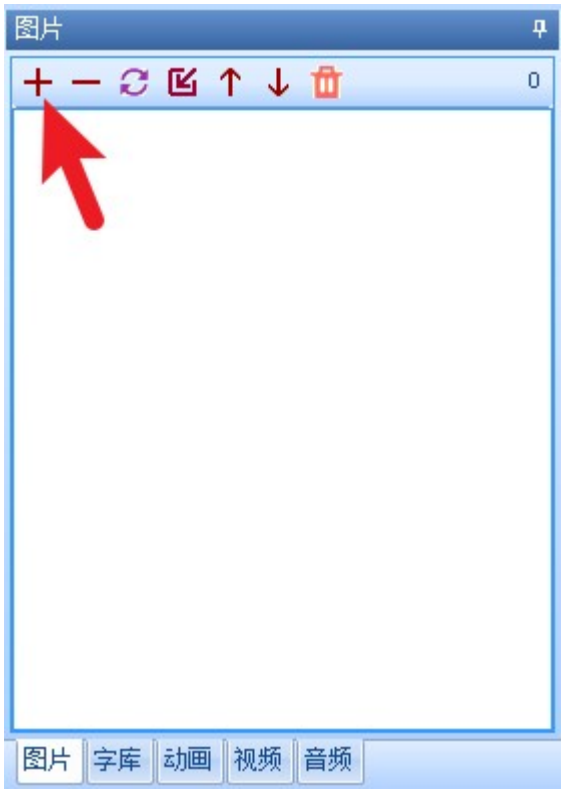
3.2.7.1 设置页面的背景图片

背景图片必须是全屏图片，即背景图片分辨率与要和页面控件的 w、h 大小一致，图片不能带有透明色，否则会出现花屏问题。

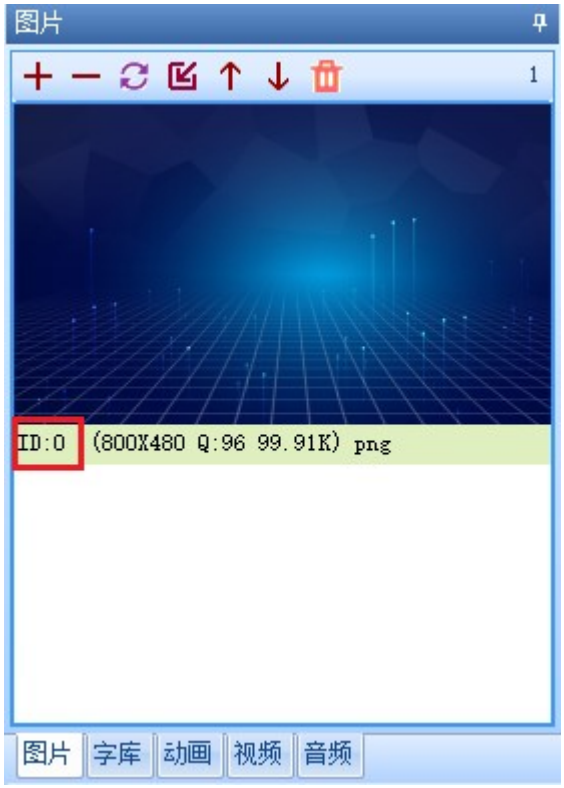
建议使用 jpg 格式，使用 PS 之类的工具输出 jpg 图片时，格式选择基线（“标准”）。



1. 找到图片资源标签，点击加号，导入一张图片

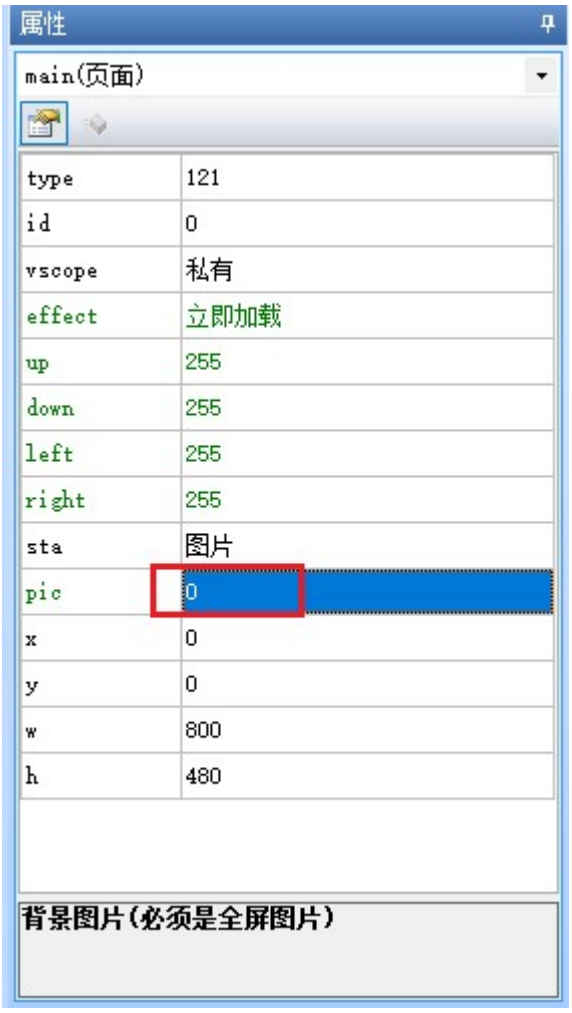


2. 记住图片的 id 号



3. 将页面的 sta 改为图片

4. 设置页面的 pic 属性为 0（对应图片的 id 号）



以上图为例，页面名称为 main，sta 为图片，我们可以通过设置 pic 属性来切换背景图片（背景图片必须是全屏图片，即背景图片分辨率与屏幕的分辨率相同）。

```
main.pic=1
```

还可以通过名称组的方式来设置 pic 属性（页面控件的 id 必定为 0）。

```
b[0].pic=1
```

3.2.7.2 切换其他页面的背景图片

将页面控件的 vscope 设置为全局，这意味着我们可以跨页面来操作页面控件的属性。

```
main.main.pic=1      //第一个main是页面名称，第二个main是页面控件，页面控件与页面名称同名
```

> **注意：** 第一个 main 是页面名称，第二个 main 是页面控件名称

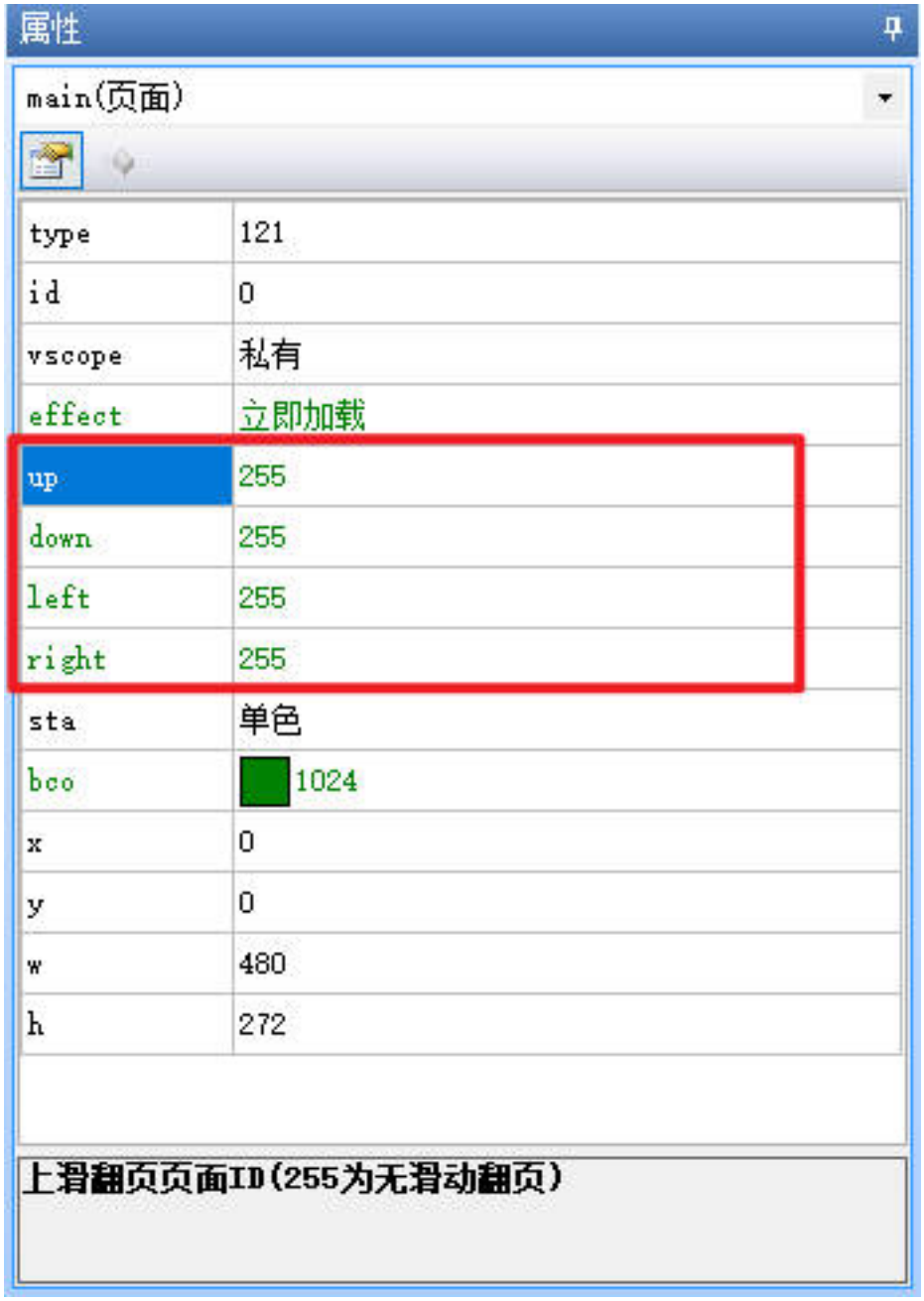
还可以通过名称组的方式来设置页面控件的 pic 属性。

```
p[0].b[0].pic=1
```

3.2.7.3 设置页面滑动翻页

仅 X 系列支持。

可以通过设置页面的 up/down/left/right 属性来设置上划下划左划右划分别跳转到哪个页面。



3.2.8 页面控件-样例工程下载

演示工程下载链接：

《返回前一个页面》演示工程下载

《跨页面传值》演示工程下载

《页面跳转》演示工程下载

3.2.9 页面控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.2.10 页面控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

页面控件的 id 属性比较特殊，是固定为 0 的，无法通过置顶或置底更改。

`vscope` 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

`effect` 属性 -加载特效，仅 x 系列支持，可读，可通过上位机修改，可通过指令修改。在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效。当页面为开机第一个页面时，无法显示特效，可以增加一个全黑的页面，开机时先跳转到这个全黑的页面，然后再跳转到原本的开始页面，就能正常显示特效了。

`up` 属性 -上滑翻页页面 ID(255 为无滑动翻页)，仅 x 系列支持，可读，可通过上位机修改，可通过指令修改。

`down` 属性 -下滑翻页页面 ID(255 为无滑动翻页)，仅 x 系列支持，可读，可通过上位机修改，可通过指令修改。

`left` 属性 -左滑翻页页面 ID(255 为无滑动翻页)，仅 x 系列支持，可读，可通过上位机修改，可通过指令修改。

`right` 属性 -右滑翻页页面 ID(255 为无滑动翻页)，仅 x 系列支持，可读，可通过上位机修改，可通过指令修改。

`sta` 属性 -背景填充方式:0-无背景;1-单色;2-图片; 可读，可通过上位机修改，不可通过指令修改。

`bco` 属性 -背景色，仅 sta 设置为单色时可用。

`pic` 属性 -背景图片 (必须是全屏图片)，仅 sta 设置为图片时可用，不允许为空。

`x` 属性 -控件的 X 坐标。可读，固定为 0，不可修改。

`y` 属性 -控件的 Y 坐标。可读，固定为 0，不可修改。

`w` 属性 -控件的宽度。可读，固定值（根据屏幕分辨率决定），不可修改。

`h` 属性 -控件的高度。可读，固定值（根据屏幕分辨率决定），不可修改。

3.3 文本控件

提示： 视频教程：文本控件

用于在串口屏上显示文本信息，包括数字，字母，符号，汉字和其他各国语言，使用前需要提前导入字库，制作字库请参考创建字库和导入字库 (page 125)

如何修改显示的字体大小：需要提前导入不同大小的字库，需要修改控件显示的字体大小时，通过上位机编辑或者通过指令修改控件的 font 属性即可，请参考如何修改控件显示的字体 (page 1251)

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

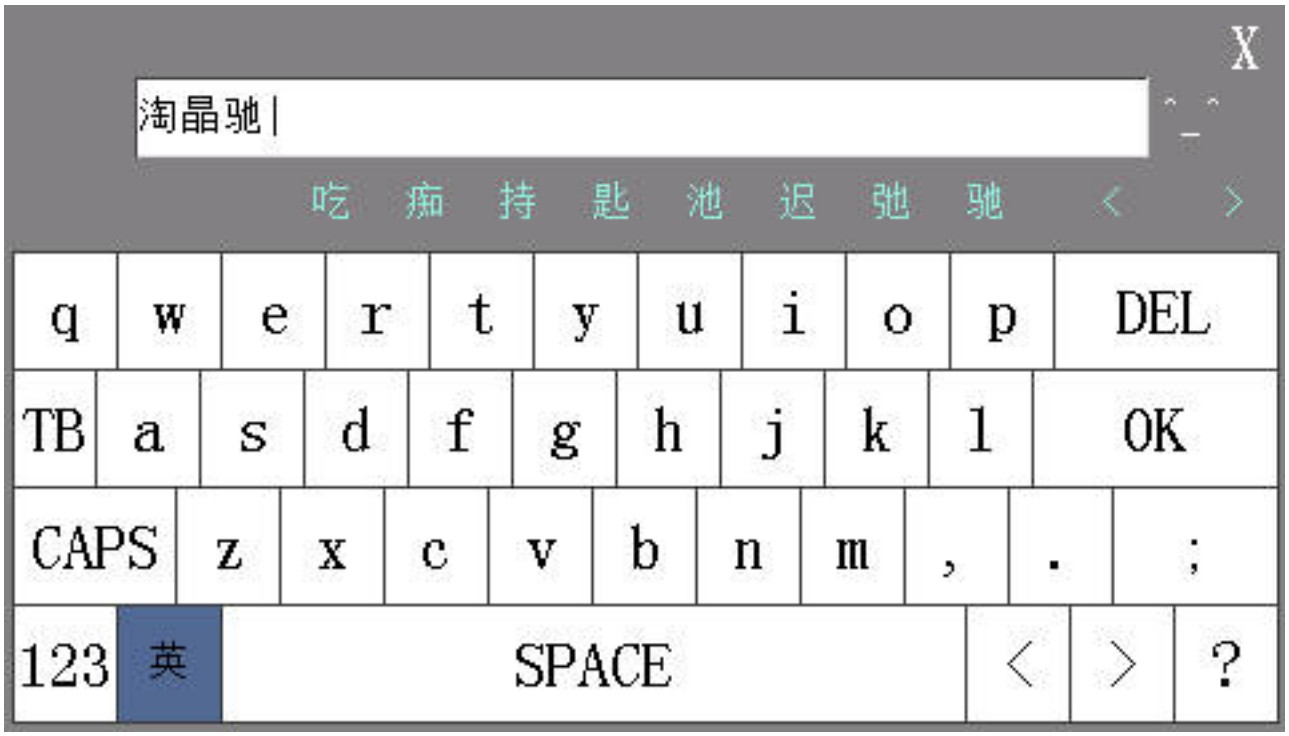
导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.3.1 文本控件-使用详解

3.3.1.1 给文本控件配置键盘



参考：系统键盘的调用方式 (page 909)

3.3.1.2 文本控件如何跨页面使用

假设 page0 页面的 t0 文本控件，如果需要跨页面使用，需要将该控件的 vscope 设置为全局

在其他页面跨页面调用 page0 页面的 t0 文本控件

```
//将当前页面的t0.txt赋值为page0页面的t0.txt
t0.txt=page0.t0.txt

//将page0页面的t0.txt赋值为main页面的t0.txt
page0.t0.txt=main.t0.txt

//将page0页面的t0.txt发送出去
prints page0.t0.txt,0
```

3.3.1.3 文本和数字之间相互转换

covx-变量类型转换 (page 606)

```
//文本控件赋值给数字控件
covx t0.txt,n0.val,0,0

//数字控件赋值给文本控件
covx n0.val,t0.txt,0,0

//可以跨页面转换，前提是对应控件设置为全局
covx main.n0.val,page0.t0.txt,0,0
```

3.3.1.4 在文本控件后面追加文本

```
t0.txt+="哈哈哈"

//将t1.txt追加在t0.txt后面
t0.txt+=t1.txt

//可以跨页面追加文本，前提是对应控件设置为全局
main.t0.txt+=page0.t1.txt
```

3.3.1.5 删除文本控件后面的几个字符

```
//删除1个字符
t0.txt-=1

//删除n0.val个字符
t0.txt-=n0.val

//可以跨页面删除，前提是对应控件设置为全局
main.t0.txt-=main.n0.val
```

注意: 只能从后面开始删除

3.3.1.6 删除文本控件前面几个字符

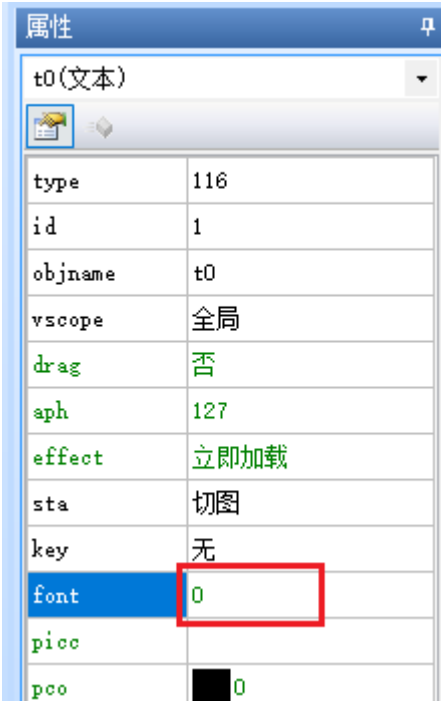
```
//删除前3个字符
substr t0.txt,t0.txt,3,sys0-3

//删除前n0.val个字符
substr t0.txt,t0.txt,n0.val,sys0-n0.val
```

注意： 只能从后面开始删除

3.3.1.7 清空文本控件

```
//将文本赋值为空
t0.txt=""
```

3.3.1.8 修改文本控件显示文字的大小和样式



我们需要提前导入不同大小的字库，需要修改文本控件显示大小时，通过上位机进行编辑或者通过指令进行修改即可，请参考如何修改控件显示的字体 (page 1251)

```
//将t0控件的字体设置为1号字体
t0.font=1


//将main页面的t0控件的字体设置为1号字体（跨页面使用控件需要将该控件设置为全局）
main.t0.font=1
```

3.3.1.9 通过名称组的方式来设置 t0 控件的属性

```
sys0=t0.id
b[sys0].txt="abc"
b[sys0].font=1
```

3.3.1.10 批量通过名称组的方式读取掉电存储区的数据来设置文本控件的属性

假设 t0 存储的起始位置为 100，t1 存储位置 120，t3 存储位置 140，以此类推，t9 存储位置 280

```
sys1=100
for(sys0=t0.id;sys0<=t9.id;sys0++)
{
    repo b[sys0].txt,sys1
    sys1+=20
}
```

3.3.1.11 如何实现文本输入和密码显示星号

1、新建一个文本控件。

2、将文本控件的 vscope 属性设置为全局。

3、设置文本控件的 key 属性，选择你需要的键盘。

4、设置文本控件的 pw 属性为是。

3.3.1.12 单片机如何对文本控件赋值

1、在上位机工程新建一个文本控件，假设为 t0，将程序下载到串口屏上，

2、串口屏串口与单片机串口连接，两者波特率应一致，单片机 RX 接串口屏 TX，单片机 TX 接串口屏 RX。

3、发送指令：单片机串口通过字符串模式发送 t0.txt=”六六六”

//注意 arduino 需要注意编码，因为 arduino 的默认编码是 utf-8，串口屏默认编码是 gb2312，请保证单片机与串口屏的编码相同，否则串口屏会显示乱码, 或者不显示

4、发送结束符：单片机通过 HEX 模式发送 0xff 0xff 0xff

5、此时屏幕上的 t0 控件内的文字变为“六六六”

3.3.1.13 文本控件-如何换行

如果是上位机软件中直接使用多行编辑, 直接按回车可以换行.

在控件的事件中给文本赋值时用\r，例如 t0.txt=”123\r456”

单片机通过串口发送给屏幕时用\r\n，例如 printf(t0.txt=”123\r\n456”)

3.3.1.14 文本数据-显示不完全

1. 文本数据长度不够（检查 txt_maxl 属性，即划给文本控件的字节长度）

GB2312 编码下，一个英文或者数字占 1 个字节，一个中文占 2 个字节

utf-8 编码下，一个英文或者数字占 1 个字节，一个中文占 3 个字节

解决方法：将控件 txt_maxl 改大。

2. 字库中没有某些字，双击相应的字库可以预览字库，显示字库中有哪些字

3. 控件属性 w，h 设置过小导致没办法显示全。

解决方法：将控件属性 w，h 改大。

4. 控件调用字库字符编码和工程设备字符编码不一致。

解决方法：将工程设备字符编码设置和控件调用的字库字符编码一致。

5. 字符编码不支持这个字。

解决方法：p 图片来显示这个字。

3.3.1.15 单片机赋值文本控件，屏幕返回 1C FF FF FF。

可能原因：单片机赋值语句例：(1). printf（“t0.txt=666\xff\xff\xff”）(2) printf（“t0.txt=”666”\xff\xff\xff”）。以上赋值语句都是不正确的。正常情况下屏幕要收到 t0.txt=”666”+ 结束符 (16 进制 3 个 ff) 才能正常赋值。单片机使用上述例句，屏幕获取到但不是正确格式，就会串口返回数据 1C FF FF FF。

解决办法：正确格式是 printf（“t0.txt=\”666\”\xff\xff\xff”）txt 赋值内容需要加双引号（””）双引号发送出去需要加转义字符否则没法正确赋值。

3.3.2 文本控件-c 语言示例

单片机通过串口给文本控件赋值

如果显示的文本不全，请检查控件的 txt_maxl 属性

```
printf("t0.txt=\"abc\"\xff\xff\xff");

printf("t0.txt=\"%d\"\xff\xff\xff",a);

//实现跨页面赋值，需要将main页面的t0控件的vscope属性设置为全局
printf("main.t0.txt=\"%d\"\xff\xff\xff",a);

//文本控件换行
printf("t0.txt=\"abc\r\n123\"\xff\xff\xff");

//这样换行也是可以的
printf("t0.txt=\"abc\\r123\"\xff\xff\xff");

//字符串中带引号{"id":1,"name":"tjc"}
printf("t0.txt=\"{\\\"id\\\":1,\\\"name\\\":\\\"tjc\\\"}\"\xff\xff\xff");
```

3.3.3 文本控件-样例工程下载

演示工程下载链接：

《跨页面赋值 (全局控件)》演示工程下载

《数码管字体等宽显示》演示工程下载

《通过名称组的方式来设置文本控件的属性》演示工程下载

《文本控件赋值》演示工程下载

《文本控件和数字控件相互转换》演示工程下载

《文本控件换行》演示工程下载

《文本控件配置键盘》演示工程下载

《文本控件写入和读取 eeprom》演示工程下载

《修改字体大小》演示工程下载

《用户名和密码登录》演示工程下载

《追加，删除，清空文本》演示工程下载

3.3.4 文本控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

虚拟浮点数与文本控件互相转换样例工程 (page 1131)

3.3.5 文本控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

`vscope` 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

`drag` 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`aph` 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`effect` 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

`sta` 属性 -背景填充方式:0-切图;1-单色;2-图片;3-透明（仅 x 系列支持透明）。可读，可通过上位机修改，不可通过指令修改。

picc 属性 -切图背景 (必须是全屏图片)，sta 为切图时才有这个属性。可读，可通过上位机修改，可通过指令修改。

bco 属性 -背景色，sta 为单色时才有这个属性。可读，可通过上位机修改，可通过指令修改。

style 属性 -显示风格:0-平面;1-边框;2-3D_Down;3-3D_Up;4-3D_Auto，sta 为单色时才有这个属性。可读，可通过上位机修改，不可通过指令修改。

borderc 属性 -边框颜色。当 style 设置为边框时可用。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

borderw 属性边框粗细。当 style 设置为边框时可用。最大值:255。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

pic 属性 -背景图片，sta 为图片时才有这个属性。可读，可通过上位机修改，可通过指令修改。

pco 属性 -字体色。可读，可通过上位机修改，可通过指令修改。

key 属性 -绑定键盘。可读，可通过上位机修改，不可通过指令修改。

font 属性 -控件调用的字库 id，调用不同的字库会显示不同的字体或字号。可读，可通过上位机修改，可通过指令修改。参考：1、创建字库和导入字库 (page 125) 2、指定字库 (page 1284)

xcen 属性 -水平对齐:0-靠左;1-居中;2-靠右。可读，可通过上位机修改，可通过指令修改。

ycen 属性 -垂直对齐:0-靠上;1-居中;2-靠下。可读，可通过上位机修改，可通过指令修改。

pw 属性 -是否显示为密码 (内容仍为实际内容, 仅仅显示出来为 *):0-否;1-是。可读，可通过上位机修改，可通过指令修改。

txt 属性 -字符内容。可读，可通过上位机修改，可通过指令修改。

txt_maxl 属性 -字符最大长度 (即分配内存空间)。可读，可通过上位机修改，不可通过指令修改。

isbr 属性 -是否自动换行:0-否;1-是。可读，可通过上位机修改，可通过指令修改。

spax 属性 -字符横向间距 (最小 0, 最大 255)。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

spay 属性 -字符纵向间距 (最小 0, 最大 255)。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.4 滚动文本控件

提示： 视频教程：滚动文本控件

用于在串口屏上显示文本信息来回滚动效果，使用前需要提前导入字库，制作字库请参考创建字库和导入字库 (page 125)

如何修改显示的字体大小：需要提前导入不同大小的字库，需要修改控件显示的字体大小时，通过上位机编辑或者通过指令修改控件的 font 属性即可，请参考如何修改控件显示的字体 (page 1251)

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

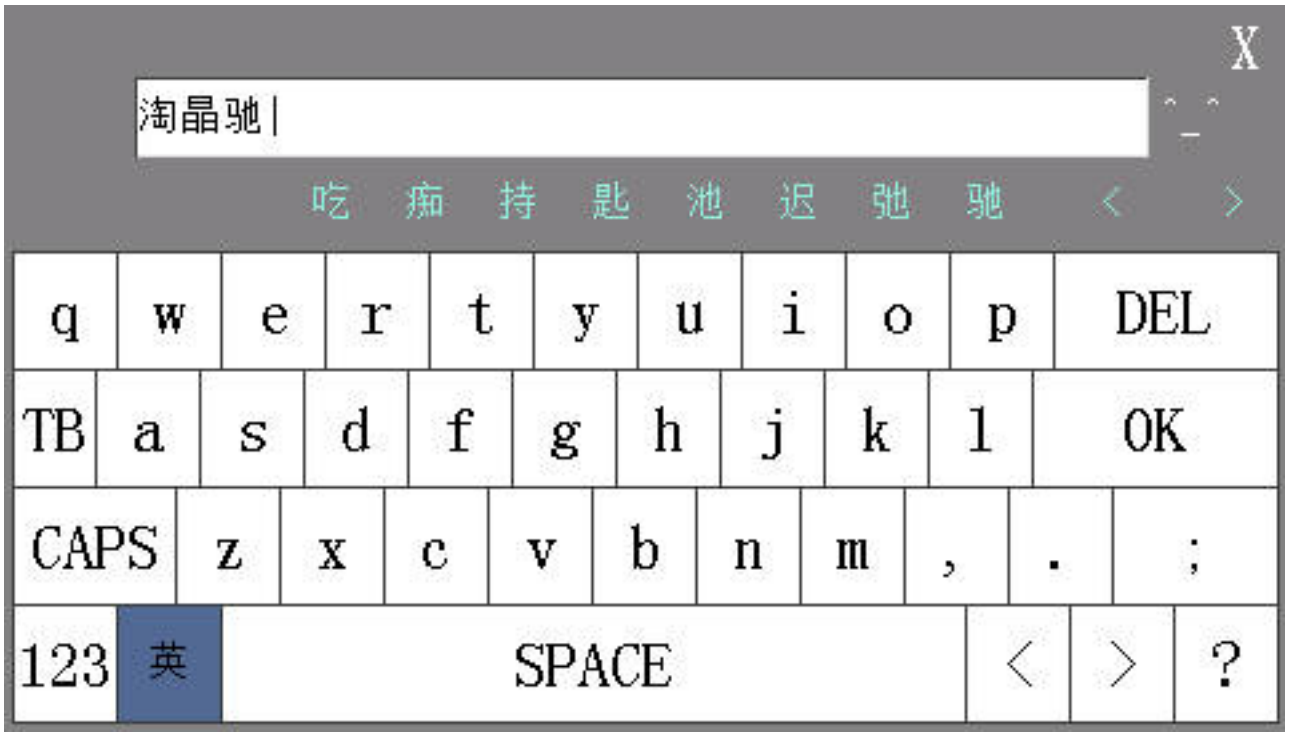
导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

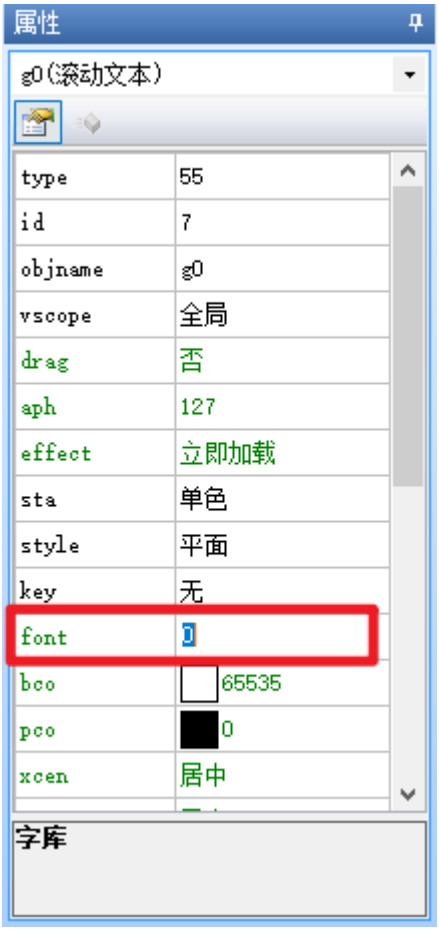
3.4.1 滚动文本控件-使用详解

3.4.1.1 给滚动文本控件配置键盘



参考：系统键盘的调用方式 (page 909)

3.4.1.2 修改滚动文本控件显示文字的大小



我们需要提前导入不同大小的字库，需要修改滚动文本显示大小时，通过上位机进行编辑或者通过指令进行修改即可

```
//将g0控件的字体设置为1号字体
g0.font=1

//将main页面的g0控件的字体设置为1号字体
main.g0.font=1
```

3.4.1.3 滚动文本控件显示文本信息

```
g0.txt="深圳市淘晶驰电子有限公司"
```

3.4.1.4 通过名称组的方式来设置 g0 控件的属性

```
sys0=g0.id
b[sys0].txt="abc"
b[sys0].font=1
```

3.4.1.5 批量通过名称组的方式来设置 g0 控件的属性

假设 g0 存储的起始位置为 100，g1 存储位置 120，g3 存储位置 140，以此类推，g9 存储位置 280

```
sys1=100
for(sys0=g0.id;sys0<=g9.id;sys0++)
{
    repo b[sys0].txt,sys1
    sys1+=20
}
```

3.4.1.6 修改滚动文本的滚动方向

```
//将g0的滚动方向改为从上往下
g0.dir=2
```

3.4.1.7 让滚动文本停止滚动

```
g0.en=0
```

3.4.2 滚动文本控件-c 语言示例

单片机通过串口给滚动文本控件赋值

如果显示的文本不全，请检查控件的 txt_maxl 属性

```
printf("g0.txt=\"深圳市淘晶驰电子有限公司\"\xff\xff\xff");

printf("g0.txt=\"%d\"\xff\xff\xff",a);
```

注意： 滚动文本不建议换行显示，会导致显示效果有问题

3.4.3 滚动文本控件-样例工程下载

演示工程下载链接：

《滚动文本控件显示文本信息》演示工程下载

3.4.4 滚动文本控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.4.5 滚动文本控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

---

type 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

id 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

objname 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

vscope 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

drag 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

aph 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

effect 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

sta 属性 -背景填充方式:0-切图;1-单色;2-图片;3-透明（仅 x 系列支持透明）。可读，可通过上位机修改，不可通过指令修改。

picc 属性 -切图背景 (必须是全屏图片)，sta 为切图时才有这个属性。可读，可通过上位机修改，可通过指令修改。

bco 属性 -背景色，sta 为单色时才有这个属性。可读，可通过上位机修改，可通过指令修改。

style 属性 -显示风格:0-平面;1-边框;2-3D_Down;3-3D_Up;4-3D_Auto，sta 为单色时才有这个属性。可读，可通过上位机修改，不可通过指令修改。

borderc 属性 -边框颜色。当 style 设置为边框时可用。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

borderw 属性边框粗细。当 style 设置为边框时可用。最大值:255。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

pic 属性 -背景图片，sta 为图片时才有这个属性。可读，可通过上位机修改，可通过指令修改。

pco 属性 -字体色。可读，可通过上位机修改，可通过指令修改。

key 属性 -绑定键盘。可读，可通过上位机修改，不可通过指令修改。

font 属性 -控件调用的字库 id，调用不同的字库会显示不同的字体或字号。可读，可通过上位机修改，可通过指令修改。参考：1、创建字库和导入字库 (page 125) 2、指定字库 (page 1284)

xcen 属性 -水平对齐:0-靠左;1-居中;2-靠右。可读，可通过上位机修改，可通过指令修改。

ycen 属性 -垂直对齐:0-靠上;1-居中;2-靠下。可读，可通过上位机修改，可通过指令修改。

dir 属性 -滚动方向:0-从左往右;1-从右往左;2-从上往下;3-从下往上。可读，可通过上位机修改，可通过指令修改。

dis 属性 -滚动幅度像素值 (最小 2, 最大 50)。可读，可通过上位机修改，可通过指令修改。

tim 属性 -滚动周期 (单位 ms, 最小 80, 最大 65534)。可读，可通过上位机修改，可通过指令修改。

en 属性 -滚动使能:0 为关闭,1 为开启。可读，可通过上位机修改，可通过指令修改。

txt 属性 -字符内容。可读，可通过上位机修改，可通过指令修改。

txt_maxl 属性 -字符最大长度 (即分配内存空间)。可读，可通过上位机修改，不可通过指令修改。

isbr 属性 -是否自动换行:0-否;1-是。可读，可通过上位机修改，可通过指令修改。

spax 属性 -字符横向间距 (最小 0, 最大 255)。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

spay 属性 -字符纵向间距 (最小 0, 最大 255)。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.5 数字控件

用于在串口屏上显示整形（int）类型的数据，使用前需要提前导入字库，制作字库请参考创建字库和导入字库 (page 125)

数字控件 val 赋值范围 -2147483648 到 2147483647

如何修改显示的字体大小：需要提前导入不同大小的字库，需要修改控件显示的字体大小时，通过上位机编辑或者通过指令修改控件的 font 属性即可，请参考如何修改控件显示的字体 (page 1251)

> **注意：** 数字控件是可以显示负数的，如果负号没有显示，请检查当前调用的字库是否有”-”

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

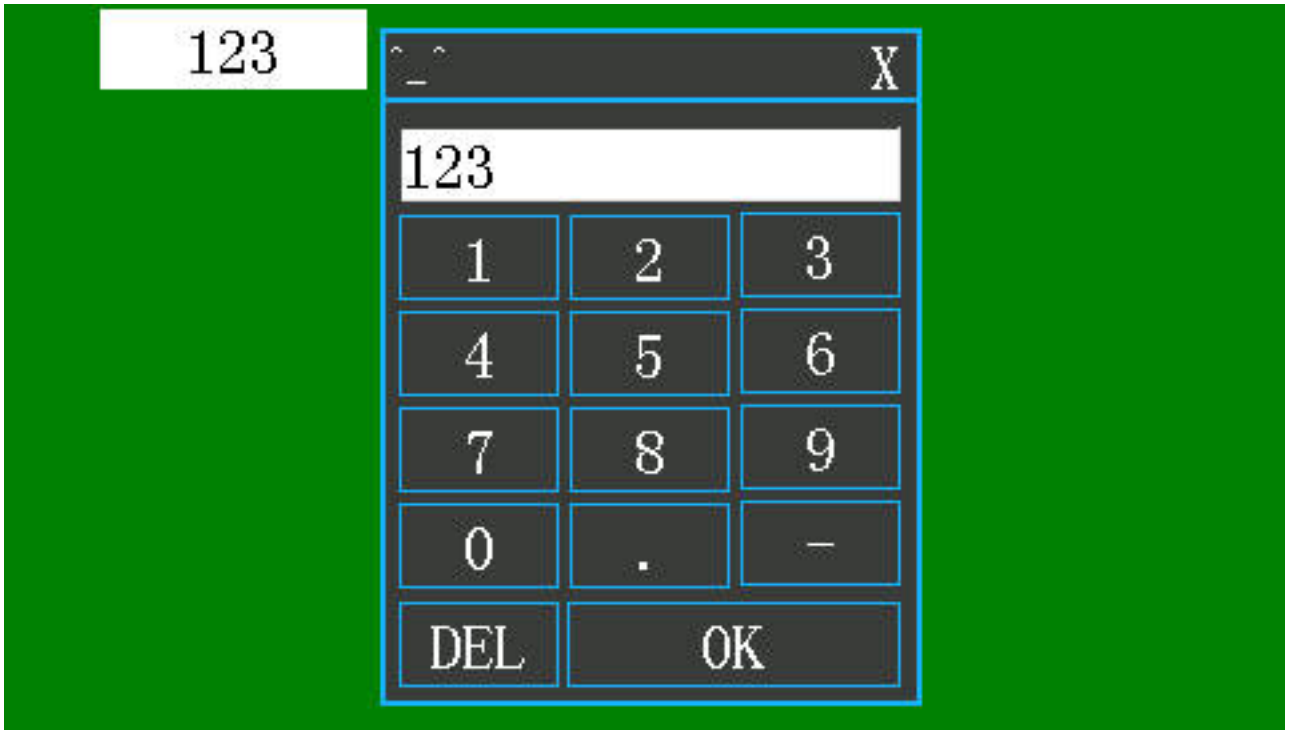
导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.5.1 数字控件-使用详解

3.5.1.1 给数字控件配置键盘



参考：系统键盘的调用方式 (page 909)

3.5.1.2 数字控件赋值

```
//n1赋值为100
n1.val=100

//n2赋值为-999
n2.val=-999

//n0赋值为n1.val
n0.val=n1.val

//加法
n0.val=n1.val+n2.val

//减法
n0.val=n1.val-n2.val

//乘法
n0.val=n1.val*n2.val

//除法
n0.val=n1.val/n2.val

//取余
n0.val=n1.val%n2.val

//跨页面赋值，前提是相关的变量vscope属性设置为全局
main.n0.val=set.n1.val+set.n2.val
```

3.5.1.3 数字和文本之间相互转换

covx-变量类型转换 (page 606)

```
//文本控件赋值给数字控件
covx t0.txt,n0.val,0,0

//数字控件赋值给文本控件
covx n0.val,t0.txt,0,0
```

3.5.1.4 数字控件如何跨页面使用

假设 page0 页面的 n0 数字控件，如果需要跨页面使用，需要将该控件的 vscope 设置为全局

在其他页面跨页面调用 page0 页面的 n0 数字控件

```
//将page1页面的n0.val赋值为page0页面的n0.val
page1.n0.val=page0.n0.val

//将page0页面的n0.val发送出去
prints page0.n0.val,0
```

3.5.1.5 修改数字控件显示文字的大小和样式

我们需要提前导入不同大小的字库，需要修改数字控件显示大小时，通过上位机进行编辑或者通过指令进行修改即可，请参考如何修改控件显示的字体 (page 1251)

```
//将n0控件的字体设置为1号字体
n0.font=1

//将main页面的n0控件的字体设置为1号字体，需要将n0控件的vscope属性设置为全局
main.n0.font=1
```

3.5.1.6 通过名称组的方式来设置 n0 控件的属性

```
sys0=n0.id
b[sys0].val=123
b[sys0].font=1
```

3.5.1.7 批量通过名称组的方式来设置数字控件的属性

假设 n0 存储的起始位置为 100，n1 存储位置 104，n3 存储位置 108，以此类推，n9 存储位置 136

```
sys1=100
for(sys0=n0.id;sys0<=n9.id;sys0++)
{
    repo b[sys0].val,sys1
    sys1+=4
}
```

3.5.1.8 使用数字控件显示 RTC 时钟数据

仅 k0，x5 系列支持，以下代码需要写在页面的前初始化事件以及定时器中定时执行，可以通过设置数字控件的 length 属性来设置显示的位数

```
n0.val=rtc0      //显示年份
n1.val=rtc1      //显示月份
n2.val=rtc2      //显示日期
n3.val=rtc3      //显示小时
n4.val=rtc4      //显示分钟
n5.val=rtc5      //显示秒钟
//显示星期，需要使用文本控件
if(rtc6==0)
{
    t0.txt="星期日"
}else if(rtc6==1)
{
    t0.txt="星期一"
}else if(rtc6==2)
{
    t0.txt="星期二"
}else if(rtc6==3)
{
    t0.txt="星期三"
}else if(rtc6==4)
{
    t0.txt="星期四"
}else if(rtc6==5)
{
    t0.txt="星期五"
}else if(rtc6==6)
{
    t0.txt="星期六"
}
```

3.5.1.9 屏幕如何显示小数

解决方法：使用虚拟浮点数控件或者文本控件。

虚拟浮点数控件 (page 423)

文本控件 (page 405)

数字控件如何在一定情况不显示默认值

解决方法一：数字控件默认值是一直有的，不能像文本控件为空。要实现数字控件默认值为空的话，可以先将数字控件属性 font（字库）设置为不含数字的字库。赋值的时候再将 font（字库）设置为包含数字的字库。

解决方法二：通过一个空白控件挡住，当数字控件不显示为 0 时，将空白控件通过 vis 指令隐藏。

3.5.1.10 单片机如何对数字控件赋值

1、在上位机工程新建一个数字控件，假设为 n0，将程序下载到串口屏上，

2、串口屏串口与单片机串口连接，两者波特率应一致，单片机 RX 接串口屏 TX，单片机 TX 接串口屏 RX。

3、发送指令：单片机串口通过字符串模式发送 n0.val=666

4、发送结束符：单片机通过 HEX 模式发送 0xff 0xff 0xff

5、此时屏幕上的 n0 控件内的文字变为“666”

3.5.2 数字控件-c 语言示例

单片机通过串口给数字控件赋值

```
printf("n0.val=123\xff\xff\xff");

printf("n0.val=%d\xff\xff\xff",a);

//赋值main页面的n0，前提是main页面的n0的vscope设置为全局
printf("main.n0.val=%d\xff\xff\xff",a);
```

注意： 给数字控件赋值必须是整形（int），否则会出错

3.5.3 数字控件-样例工程下载

演示工程下载链接：

《数字控件赋值》演示工程下载

《数字控件显示 RTC 时钟数据》演示工程下载

《数字控件写入和读取 eeprom》演示工程下载

《通过名称组的方式来设置数字控件的属性》演示工程下载

《数字控件示数闪烁》演示工程下载

3.5.4 数字控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.5.5 数字控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

id 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

objname 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

vscope 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

drag 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

aph 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

effect 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

sta 属性 -背景填充方式:0-切图;1-单色;2-图片;3-透明（仅 x 系列支持透明）。可读，可通过上位机修改，不可通过指令修改。

picc 属性 -切图背景 (必须是全屏图片)，sta 为切图时才有这个属性。可读，可通过上位机修改，可通过指令修改。

bco 属性 -背景色，sta 为单色时才有这个属性。可读，可通过上位机修改，可通过指令修改。

style 属性 -显示风格:0-平面;1-边框;2-3D_Down;3-3D_Up;4-3D_Auto，sta 为单色时才有这个属性。可读，可通过上位机修改，不可通过指令修改。

borderc 属性 -边框颜色。当 style 设置为边框时可用。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

borderw 属性边框粗细。当 style 设置为边框时可用。最大值:255。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

pic 属性 -背景图片，sta 为图片时才有这个属性。可读，可通过上位机修改，可通过指令修改。

pco 属性 -字体色。可读，可通过上位机修改，可通过指令修改。

key 属性 -绑定键盘。可读，可通过上位机修改，不可通过指令修改。

font 属性 -控件调用的字库 id，调用不同的字库会显示不同的字体或字号。可读，可通过上位机修改，可通过指令修改。参考：1、创建字库和导入字库 (page 125) 2、指定字库 (page 1284)

xcen 属性 -水平对齐:0-靠左;1-居中;2-靠右。可读，可通过上位机修改，可通过指令修改。

ycen 属性 -垂直对齐:0-靠上;1-居中;2-靠下。可读，可通过上位机修改，可通过指令修改。

val 属性 -初始值 (最小-2147483648, 最大 2147483647)。可读，可通过上位机修改，可通过指令修改。

length 属性 -显示位数 (0 为自动, 最大 15 位)。可读，可通过上位机修改，可通过指令修改。

format 属性 -格式化类型:0-数字;1-货币;2-Hex。可读，可通过上位机修改，可通过指令修改。

isbr 属性 -是否自动换行:0-否;1-是。可读，可通过上位机修改，可通过指令修改。

spax 属性 -字符横向间距 (最小 0, 最大 255)。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

spay 属性 -字符纵向间距 (最小 0, 最大 255)。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.6 虚拟浮点数控件

用于在串口屏上显示浮点型（float）类型的数据，但是本质上仍然是整形（int），使用前需要提前导入字库，制作字库请参考创建字库和导入字库 (page 125)

如何修改显示的字体大小：需要提前导入不同大小的字库，需要修改控件显示的字体大小时，通过上位机编辑或者通过指令修改控件的 font 属性即可，请参考如何修改控件显示的字体 (page 1251)

注意： 虚拟浮点数控件是可以显示负数的，如果负号没有显示，请检查当前调用的字库是否有”-“(减号/负号)

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

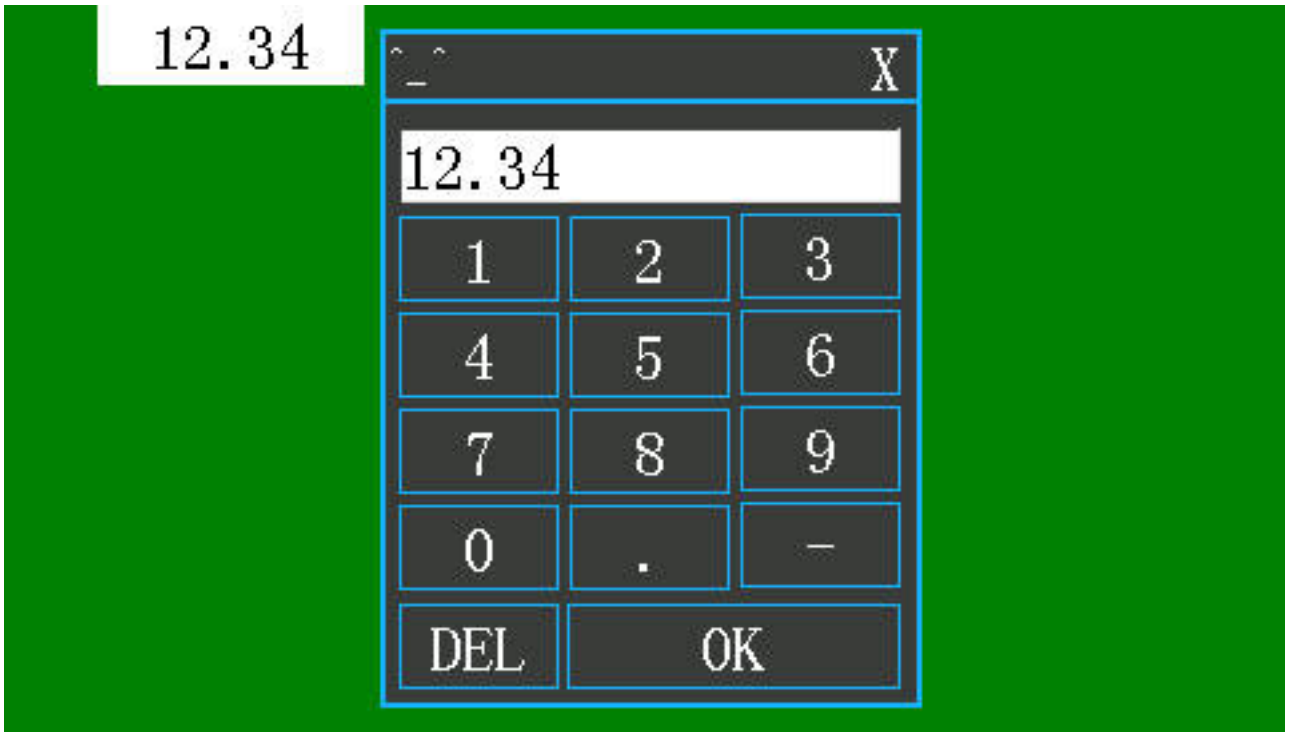
导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.6.1 虚拟浮点数控件-使用详解

3.6.1.1 给虚拟浮点数控件配置键盘



参考：系统键盘的调用方式 (page 909)

3.6.1.2 虚拟浮点数控件赋值

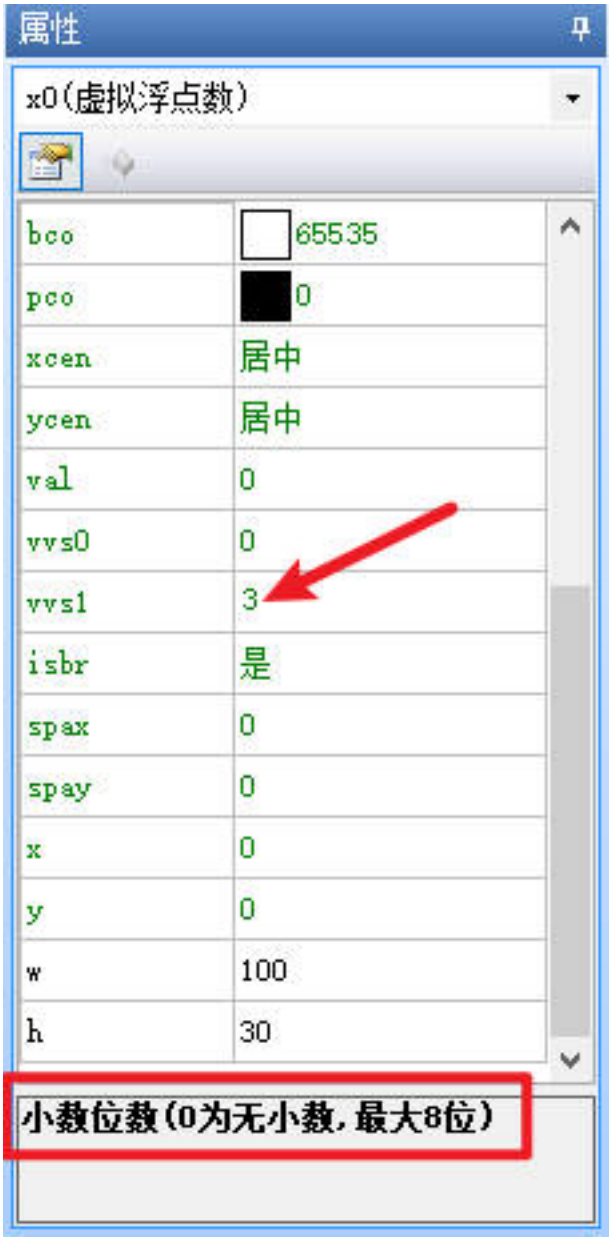
```
//n1赋值为100，具体显示数值取决于控件的vvs0和vvs1属性
x1.val=100

//n2赋值为-999，具体显示数值取决于控件的vvs0和vvs1属性
x2.val=-999

//n0赋值为x1.val
x0.val=x1.val

//有虚拟浮点数的加法，所有参与计算的虚拟浮点数的vvs0和vvs1属性应一致
x0.val=x1.val+x2.val

//减法
x0.val=x1.val-x2.val

//乘法
x0.val=x1.val*x2.val

//取整
x0.val=x1.val/x2.val

//取余
x0.val=x1.val%x2.val

//跨页面赋值，前提是相关的变量vscope属性设置为全局
main.x0.val=set.x1.val+set.x2.val
```

注意: 虚拟浮点数本质上还是整数, 只是额外显示了一个小数点，但是这个小数点其实是假的，虚拟浮点数具体计算请参考虚拟浮点数计算问题 (page 428)

3.6.1.3 虚拟浮点数控件赋值举例

例如显示 114.514

新建一个虚拟浮点数控件，设置 vvs1 属性为 3（设置小数位数为 3）




```
//设置为114514，此时显示114.514
x0.val=114514
```

3.6.1.4 修改虚拟浮点数控件显示文字的大小和样式

我们需要提前导入不同大小的字库，需要修改虚拟浮点数控件显示大小时，通过上位机进行编辑或者通过指令进行修改即可，请参考如何修改控件显示的字体 (page 1251)

```
//将x0控件的字体设置为1号字体
x0.font=1

//将main页面的x0控件的字体设置为1号字体
main.x0.font=1
```

3.6.1.5 通过名称组的方式来设置 x0 控件的属性

```
sys0=x0.id
b[sys0].val=123
b[sys0].font=1
```

3.6.1.6 批量通过名称组的方式来设置虚拟浮点数控件的属性

假设 x0 存储的起始位置为 100，x1 存储位置 104，x3 存储位置 108，以此类推，x9 存储位置 136

```
sys1=100
for(sys0=x0.id;sys0<=x9.id;sys0++)
{
    repo b[sys0].val,sys1
    sys1+=4
}
```

3.6.1.7 串口屏-如何显示小数

3.6.1.7.1 使用虚拟浮点数控件显示小数

1、在上位机工程新建一个虚拟浮点数控件，假设为 x0，设置虚拟浮点数控件的小数点位数 2 位，将程序下载到串口屏上，

2、串口屏串口与单片机串口连接，两者波特率应一致，单片机 RX 接串口屏 TX，单片机 TX 接串口屏 RX。

3、发送指令：单片机串口通过字符串模式发送 x0.val=314

4、发送结束符：单片机通过 HEX 模式发送 0xff 0xff 0xff

5、此时屏幕上的 x0 控件内的文字变为“3.14”

3.6.1.7.2 使用文本控件显示小数

1、在上位机工程新建一个文本控件，假设为 t0，将程序下载到串口屏上，

2、串口屏串口与单片机串口连接，两者波特率应一致，单片机 RX 接串口屏 TX，单片机 TX 接串口屏 RX。

3、发送指令：单片机串口通过字符串模式发送 t0.txt=”3.14”

4、发送结束符：单片机通过 HEX 模式发送 0xff 0xff 0xff

5、此时屏幕上的 t0 控件内的文字变为“3.14”

3.6.2 浮点数控件-c 语言示例

单片机通过串口给虚拟浮点数控件赋值

```
printf("x0.val=123\xff\xff\xff");

printf("x0.val=%d\xff\xff\xff",a);
```

注意： 给虚拟浮点数控件赋值必须是整形（int），否则会出错，虚拟浮点数本质上还是整数

3.6.3 虚拟浮点数计算问题

假如有浮点数 x0，x1，x2

x0.val=1234，小数位数 2 位，屏幕上显示 12.34

x1.val=34567, 小数位数 3 位，屏幕上显示 34.567

3.6.3.1 虚拟浮点数加法

先将所有浮点数的小数位数设置为相等

```
x0.val=x0.val*10
x0.vvs1=x0.vvs1+1
```

此时 x0.val=12340，小数位数 3 位，屏幕上显示 12.340，此时就可以直接相加

```
x2.val=x0.val+x1.val
x2.vvs1=x0.vvs1      //或者x2.vvs1=x1.vvs1
```

3.6.3.2 虚拟浮点数减法

与加法一样，需要将所有浮点数的小数位数设置为相等

```
x2.val=x0.val-x1.val

//或者x2.vvs1=x1.vvs1
x2.vvs1=x0.vvs1
```

3.6.3.3 虚拟浮点数乘法

```
x2.val=x0.val*x1.val
x2.vvs1=x0.vvs1+x1.vvs1
```

如果最后的结果发现小数点里的 0 太多，建议把 x2.val 除以 10 的倍数，同时 x2.vss1 减小相应的值

```
x2.val=x2.val/100
x2.vvs1=x2.vvs1-2
```

3.6.3.4 虚拟浮点数除法

```
x2.val=x0.val/x1.val
x2.vvs1=x0.vvs0-x1.vvs1
```

鉴于精度的问题，建议将被除数多扩大几倍以提高精度

3.6.3.5 浮点数发送给单片机

x0.val=-1234，小数位数 2 位，屏幕上显示-12.34

屏幕：

```
prints x0.val,0
```

此时屏幕发出的是-1234，单片机将这个值保存到 float 类型变量 x 中，此时单片机中的 x=-1234

屏幕：

```
prints x0.vvs1, 0
```

此时屏幕发送 x0 的小数位数，也就是 2，单片机将这个值保存到 u8 类型变量 j 中，此时单片机中的 j=2 单片机：

```
uint32_t j2=1;
for(i=0;i<j;i++)
{
    j2=j2*10;
}

//得到真正的浮点数x
x=x/j2;

//初始化j2，避免后续出问题
j2=1;
```

3.6.4 虚拟浮点数控件-样例工程下载

演示工程下载链接：

《虚拟浮点数》演示工程下载

3.6.5 虚拟浮点数控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

虚拟浮点数与文本控件互相转换样例工程 (page 1131)

3.6.6 虚拟浮点数控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

vscope 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

drag 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

aph 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

effect 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

sta 属性 -背景填充方式:0-切图;1-单色;2-图片;3-透明（仅 x 系列支持透明）。可读，可通过上位机修改，不可通过指令修改。

picc 属性 -切图背景 (必须是全屏图片)，sta 为切图时才有这个属性。可读，可通过上位机修改，可通过指令修改。

bco 属性 -背景色，sta 为单色时才有这个属性。可读，可通过上位机修改，可通过指令修改。

style 属性 -显示风格:0-平面;1-边框;2-3D_Down;3-3D_Up;4-3D_Auto，sta 为单色时才有这个属性。可读，可通过上位机修改，不可通过指令修改。

borderc 属性 -边框颜色。当 style 设置为边框时可用。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

borderw 属性边框粗细。当 style 设置为边框时可用。最大值:255。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

pic 属性 -背景图片，sta 为图片时才有这个属性。可读，可通过上位机修改，可通过指令修改。

pco 属性 -字体色。可读，可通过上位机修改，可通过指令修改。

key 属性 -绑定键盘。可读，可通过上位机修改，不可通过指令修改。

font 属性 -控件调用的字库 id，调用不同的字库会显示不同的字体或字号。可读，可通过上位机修改，可通过指令修改。参考：1、创建字库和导入字库 (page 125) 2、指定字库 (page 1284)

xcen 属性 -水平对齐:0-靠左;1-居中;2-靠右。可读，可通过上位机修改，可通过指令修改。

ycen 属性 -垂直对齐:0-靠上;1-居中;2-靠下。可读，可通过上位机修改，可通过指令修改。

val 属性 -初始值 (最小-2147483648, 最大 2147483647)。可读，可通过上位机修改，可通过指令修改。

vvs0 属性 -整数位数 (0 为自动, 最大 10 位)。可读，可通过上位机修改，可通过指令修改。

vvs1 属性 -小数位数 (0 为无小数, 最大 8 位)。可读，可通过上位机修改，可通过指令修改。

isbr 属性 -是否自动换行:0-否;1-是。可读，可通过上位机修改，可通过指令修改。

spax 属性 -字符横向间距 (最小 0, 最大 255)。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

spay 属性 -字符纵向间距 (最小 0, 最大 255)。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.7 按钮控件

提示： 视频教程：按钮控件

用于在串口屏上显示按钮，使用前需要提前导入字库，制作字库请参考创建字库和导入字库 (page 125)

如何修改显示的字体大小：需要提前导入不同大小的字库，需要修改控件显示的字体大小时，通过上位机编辑或者通过指令修改控件的 font 属性即可，请参考如何修改控件显示的字体 (page 1251)

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

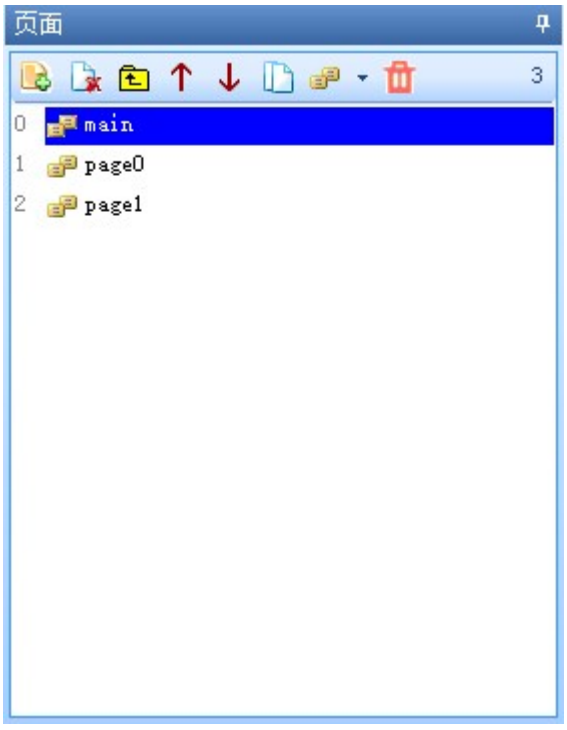
导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.7.1 按钮控件-使用详解

3.7.1.1 通过按钮控件跳转页面



如上图所示，该工程有三个页面，分别为 main，page0，page1，main 页面的页面 id 为 0，page0 页面的页面 id 为 1，page1 页面的页面 id 为 2

在 main 页面新建一个按钮控件，按钮控件的 txt 属性改为“跳转页面”，在按钮控件的弹起事件编写以下代码

```
//推荐此方法
page page1   //跳转到page1页面
```

因为 page1 页面的页面 id 是 2，因此也可以这么写

```
//不推荐此方法，当插入页面导致页面顺序改变时会导致页面跳转逻辑混乱
page 2  //跳转到id为2的页面（也就是page1页面）
```

3.7.1.2 修改按钮控件显示文字的大小和样式

我们需要提前导入不同大小的字库，需要修改按钮控件显示大小时，通过上位机进行编辑或者通过指令进行修改即可，请参考如何修改控件显示的字体 (page 1251)

```
//串口屏控件事件代码（通常写在串口屏控件的按下或弹起事件中）
b0.font=1   //将b0控件的字体设置为1号字体
main.b0.font=1    //
↪将main页面的b0控件的字体设置为1号字体，b0控件的vscope属性必须设置为全局


//单片机发送指令通过串口修改调用的字库id
int font_id = 1;   //字库编码
char tjcstr[100];
sprintf(tjcstr, "main.b0.font=%d\xff\xff\xff",font_id);
printf(tjcstr);   //单片机需要配置printf重定向到串口


//arduino发送指令通过串口修改调用的字库id
int font_id = 1;   //字库编码
char tjcstr[100];
sprintf(tjcstr, "main.b0.font=%d\xff\xff\xff",font_id);
Serial.print(tjcstr);
```

3.7.1.3 让按钮在开始和停止之间切换

```
if(b0.txt=="开始")
{
      b0.txt="停止"
}else
{
      b0.txt="开始"
}
```

3.7.1.4 按钮禁用触摸功能

使用 tsw 指令来实现触摸的使能和失能

```
//禁用b0的触摸
tsw b0,0

//开启b0的触摸
tsw b0,1
```

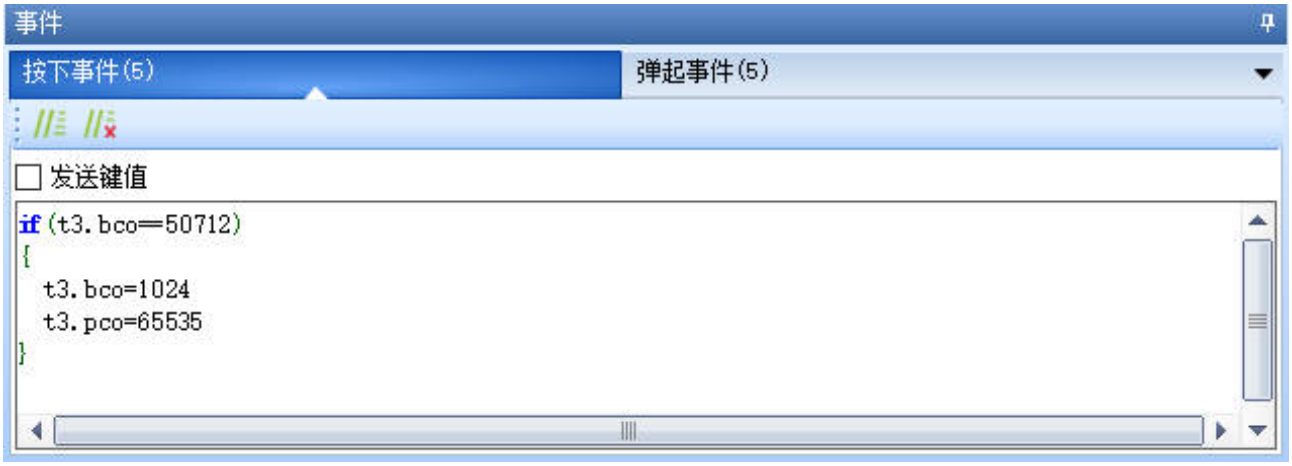
建议使用 tsw 时，配合修改按键的外观（修改颜色或者修改调用的图片）来让用户能够一眼了解按钮是否可用

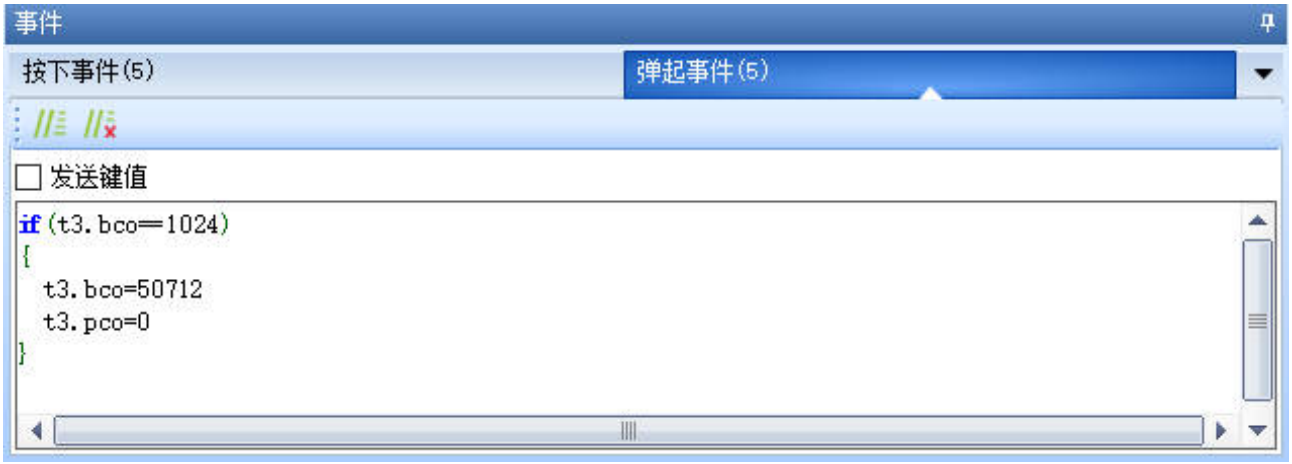
当然也有其他的方法来实现类似的功能

例如：

使用文本控件替代按钮控件

在文本控件的按下或者弹起事件里，判断文本的 bco 或者 pic 属性，只有属性为自己需要时才可触发对应的代码





我们只需修改控件的 bco 属性为其他值，此时不符合条件，不会执行 if 里面的内容，也就相当于“禁用”这个功能了

3.7.1.5 通过按钮控件发出串口数据

```
//将n0.val的值从串口发送出来
prints n0.val,0

//将t0.txt的值从串口发送出来
prints t0.txt,0

//发送回车换行
printh 0d 0a
```

prints-从串口打印一个变量/常量 (page 578)

printh-从串口打印 16 进制 (page 586)

在键盘页面按下 OK 键时将参数通过串口发送出去 (page 923)

3.7.1.6 使用按钮实现长按功能

需配合定时器使用

1. 新建一个定时器 tm0，en 属性设置为 0，tim 属性设置为你想要长按的时间，例如 2 秒，则设置为 2000。

2. 在按钮的按下事件中写上如下代码：

```
//打开定时器
tm0.en=1
```

3. 在按钮的弹起事件中写上如下代码：

```
//弹起事件代表已经松手了，所以让定时器立即停止工作
tm0.en=0
```

4. 在定时器 tm0 的定时事件中先关闭定时器，然后写上想操作的长按事件代码，比如:

```
//长按时间到了,先关闭定时器
tm0.en=0
//接下来可以写自己需要长按后执行的代码，例如跳转到main页面
page main
```

3.7.1.7 使用按钮实现双击功能

需配合定时器使用和数字控件使用

1. 新建一个定时器 tm0，en 属性设置为 0，tim 属性设置双击的间隔，例如 400 毫秒，则设置为 400（定时器定时间隔最低 50ms）。新建一个数字控件 n0，用于记录双击次数

2. 在按钮的按下事件中写上如下代码：

```
//打开定时器
tm0.en=1
n0.val++
if(n0.val>=2)
{
    //触发了双击，跳转页面
    page main
}
```

3. 在定时器 tm0 的定时事件中编写以下代码

```
//关闭定时器
tm0.en=0

//清空n0.val
n0.val=0
```

调试 ok 后可以将数字控件改为变量控件

3.7.1.8 使用按钮在多个状态之间切换

```
if(b0.txt=="状态1")
{
   b0.txt="状态2"
}else if(b0.txt=="状态2")
{
   b0.txt="状态3"
}else if(b0.txt=="状态3")
{
   b0.txt="状态4"
}else if(b0.txt=="状态4")
{
   b0.txt="状态5"
}else
{
   b0.txt="状态1"
}
```

3.7.1.9 使用按钮在多个状态之间切换 2

```
if(b0.txt=="开始")
{
   b0.txt="暂停"
}else if(b0.txt=="暂停")
{
   b0.txt="停止"
}else
{
   b0.txt="开始"
}
```

3.7.2 按钮控件-样例工程下载

演示工程下载链接：

《按钮控件》演示工程下载

《按钮长按发送数据》演示工程下载

《一个按钮同时实现点按和长按》演示工程下载

3.7.3 按钮控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.7.4 按钮控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

---

type 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

id 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

objname 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

vscope 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

drag 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

aph 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

effect 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

sta 属性 -背景填充方式:0-切图;1-单色;2-图片;3-透明（仅 x 系列支持透明）。可读，可通过上位机修改，不可通过指令修改。

picc 属性 -按钮没按下时切图背景 (必须是全屏图片)，sta 为切图时才有这个属性。可读，可通过上位机修改，可通过指令修改。

picc2 属性 -按钮按下时切图背景 (必须是全屏图片)，sta 为切图时才有这个属性。可读，可通过上位机修改，可通过指令修改。

bco 属性 -按钮没按下时背景色，sta 为单色时才有这个属性。可读，可通过上位机修改，可通过指令修改。

bco2 属性 -按钮按下时背景色，sta 为单色时才有这个属性。可读，可通过上位机修改，可通过指令修改。

style 属性 -显示风格:0-平面;1-边框;2-3D_Down;3-3D_Up;4-3D_Auto，sta 为单色时才有这个属性。可读，可通过上位机修改，不可通过指令修改。

borderc 属性 -边框颜色。当 style 设置为边框时可用。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

borderw 属性边框粗细。当 style 设置为边框时可用。最大值:255。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

pic 属性 -按钮没按下时背景图片，sta 为图片时才有这个属性。可读，可通过上位机修改，可通过指令修改。

pic2 属性 -按钮按下时背景图片，sta 为图片时才有这个属性。可读，可通过上位机修改，可通过指令修改。

pco 属性 -按钮没按下时字体色。可读，可通过上位机修改，可通过指令修改。

pco2 属性 -按钮按下时字体色。可读，可通过上位机修改，可通过指令修改。

font 属性 -控件调用的字库 id，调用不同的字库会显示不同的字体或字号。可读，可通过上位机修改，可通过指令修改。参考：1、创建字库和导入字库 (page 125) 2、指定字库 (page 1284)

xcen 属性 -水平对齐:0-靠左;1-居中;2-靠右。可读，可通过上位机修改，可通过指令修改。

ycen 属性 -垂直对齐:0-靠上;1-居中;2-靠下。可读，可通过上位机修改，可通过指令修改。

txt 属性 -字符内容（按钮上显示的文本）。可读，可通过上位机修改，可通过指令修改。

txt_maxl 属性 -控件分配的内存空间。GB2312 下，ascii 字符占用 1 个字节，一个汉字占 2 字节。UTF8 下，ascii 字符占用 1 个字节，一个汉字占 3 字节。可读，可通过上位机修改，不可通过指令修改。

isbr 属性 -是否自动换行:0-否;1-是。可读，可通过上位机修改，可通过指令修改。

spax 属性 -字符横向间距 (最小 0, 最大 255)。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

spay 属性 -字符纵向间距 (最小 0, 最大 255)。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.8 进度条控件

提示： 视频教程：进度条控件

用于在串口屏上显示进度（例如开机进度）

提示： 导入字库请参考: 如何导入字库 (page 1442)
导入图片请参考: 如何导入图片 (page 1449)
导入动画请参考: 如何导入动画 (page 1455)
导入视频请参考: 如何导入视频 (page 1461)
导入音频请参考: 如何导入音频 (page 1467)

3.8.1 进度条控件-使用详解

3.8.1.1 搭配定时器控件实现进度条自动增长并在进度为 100 时跳转页面

新建一个定时器 tm0，en 属性设置为 1，定时器的 tim 属性建议设置为 200，在定时器内写下如下代码

```
if(j0.val<100)
{
   j0.val++
}else
{
   page main
}
```

进度条走完的时间 = 定时器 tim * 进度条总刻度，因此上面的例子中，花费的时间为 200 * 100 = 20000(毫秒)，等于 20 秒

3.8.1.2 使用定时器控件实现进度条和数值变量之间同步

新建一个定时器 tm0，en 属性设置为 1，定时器的 tim 属性建议设置为 100，

在定时器内写下如下代码

```
//当n0.val发生变化时，定时器会不断同步j0.val为n0.val
j0.val=n0.val
```

3.8.1.3 让进度条从右往左 (反方向)

新建一个定时器 tm0，en 属性设置为 1，定时器的 tim 属性建议设置为 100，

在定时器内写下如下代码

```
//当n0.val发生变化时，定时器会不断同步j0.val为100-n0.val
j0.val=100-n0.val
```

当 n0.val=100 时,j0.val=0

当 n0.val=0 时,j0.val=100

3.8.2 进度条控件-c 语言示例

单片机通过串口给进度条控件赋值

```
printf("j0.val=50\xff\xff\xff");

int a = 50;
printf("j0.val=%d\xff\xff\xff",a);
```

注意： 给进度条控件赋值必须是整形（int），否则会出错

3.8.3 进度条控件-样例工程下载

演示工程下载链接：

《进度条控件》演示工程下载

3.8.4 进度条控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 val 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.8.5 进度条控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：*控件属性-控件 id 对照表* (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：*如何更改控件的前后图层关系* (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

`vscope` 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：*跨页面赋值，全局变量操作* (page 847)

`drag` 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`aph` 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`effect` 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

`sta` 属性 -背景填充方式:0-单色;1-图片。可读，可通过上位机修改，不可通过指令修改。

`sta` 属性 -进度方式:0-横向;1-竖向。可读，可通过上位机修改，不可通过指令修改。

`val` 属性 -进度值。可读，可通过上位机修改，可通过指令修改。

dis 属性 -外形圆角半径百分比 (0-100,0 为直角)。可读，可通过上位机修改，可通过指令修改。sta 为单色时可用。

bco 属性 -背景色。可读，可通过上位机修改，可通过指令修改。sta 为单色时可用。

pco 属性 -前景色。可读，可通过上位机修改，可通过指令修改。sta 为单色时可用。

bpic 属性 -背景图片。可读，可通过上位机修改，可通过指令修改。sta 为图片时可用。

ppic 属性 -前景图片。可读，可通过上位机修改，可通过指令修改。sta 为图片时可用。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.9 图片控件

注意： 请勿使用图片控件作为背景图片，正确的背景图片使用方式是-点击页面空白处-设置页面控件的 sta 为图片，将页面控件的 pic 属性设置为你想要显示的背景图片

用于在串口屏上显示图片（显示产品 logo，提示灯图片等）。

支持的格式有 jpg，png，bmp，其中 T 系列和 K 系列不支持 PNG 透明图层，需要类似的效果请参考*K 系列和 T 系列如何实现透明效果* (page 1317)

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.9.1 图片控件-使用详解

3.9.1.1 修改图片控件显示不同的图片

需要修改图片控件显示大小时，通过上位机进行编辑或者通过指令进行修改即可

```
//让图片控件p0显示id为0的图片
p0.pic=0

//让图片控件p1显示id为1的图片
p1.pic=1
```

3.9.2 图片控件-c 语言示例

单片机通过串口给图片控件赋值

```
//让图片控件p0显示id为0的图片
printf("p0.pic=0\xff\xff\xff");

//通过变量a让图片控件p1显示id为1的图片
int a=1;
printf("p1.pic=%d\xff\xff\xff",a);
```

3.9.3 图片控件-样例工程下载

演示工程下载链接：

《图片控件》演示工程下载

《图片控件实现开机动画》演示工程下载

《艺术字图片》演示工程下载

3.9.4 图片控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.9.5 图片控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

---

type 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

id 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

objname 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

vscope 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

drag 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

aph 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

effect 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

pic 属性 -图片 id，可读，可通过上位机修改，可通过指令修改，不允许为空。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.10 切图控件

用于在串口屏上显示切图

提示： 导入字库请参考: 如何导入字库 (page 1442)

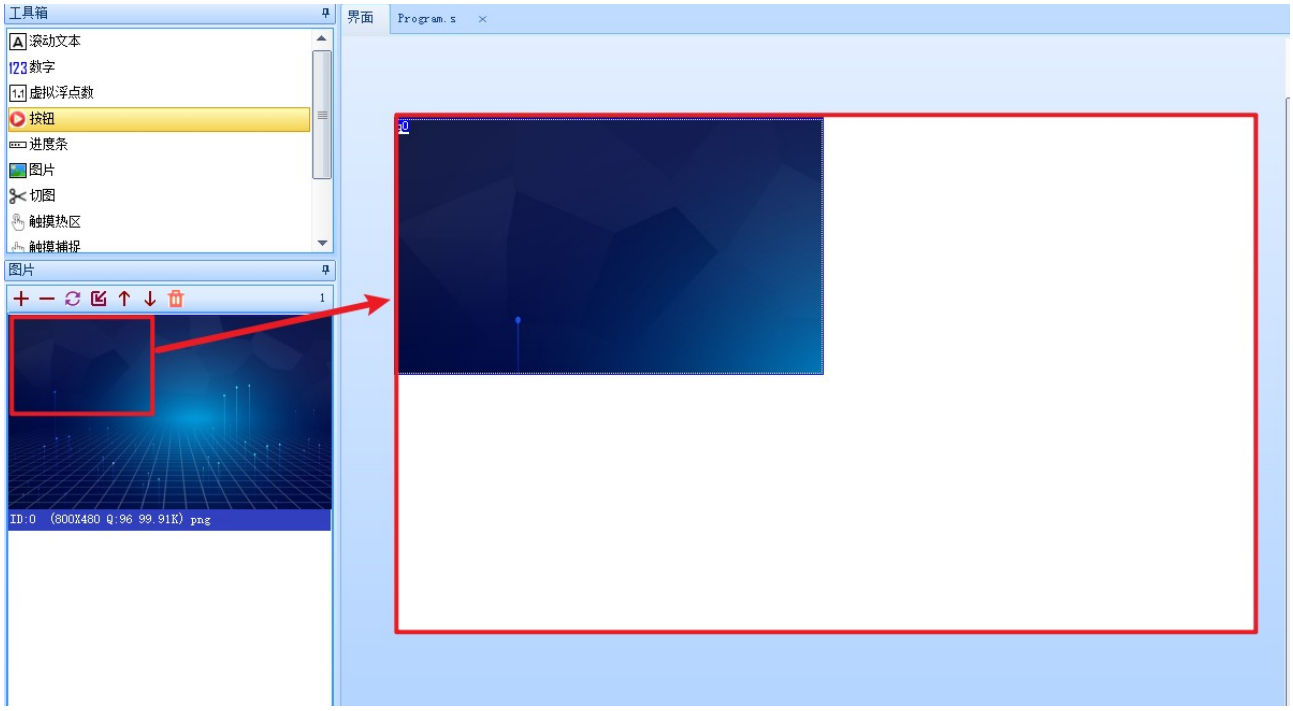
导入图片请参考: 如何导入图片 (page 1449)

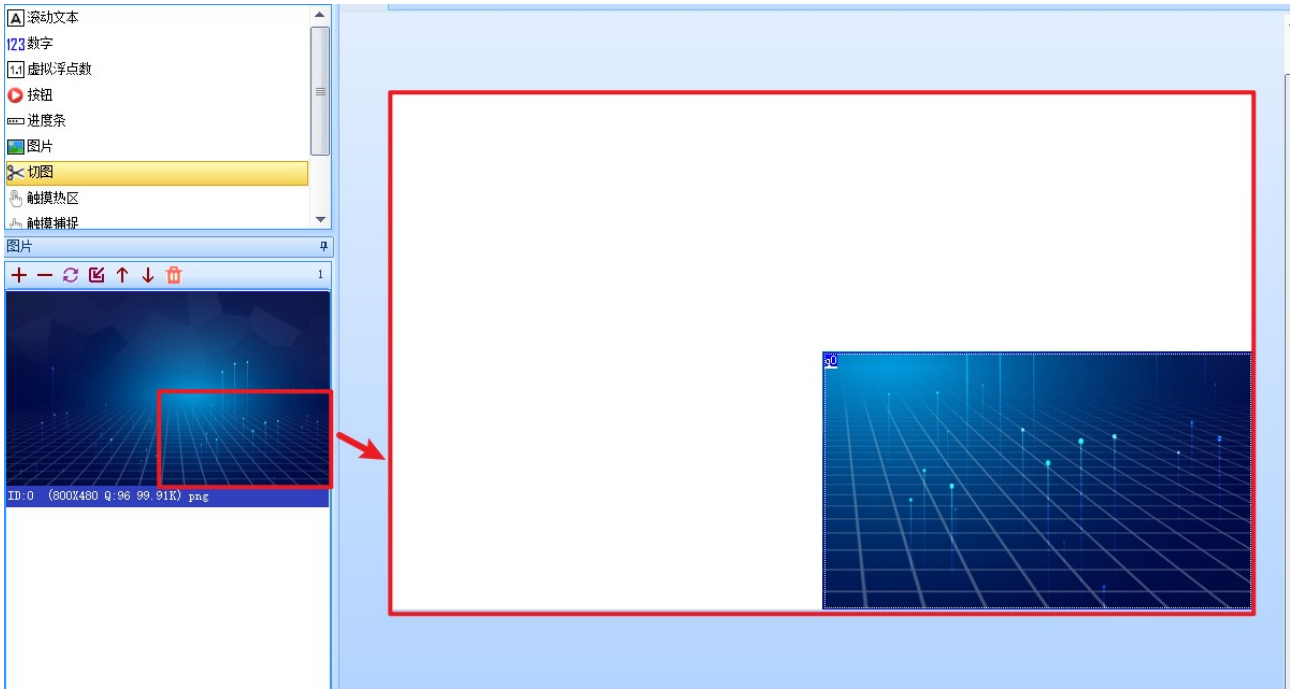
导入动画请参考: 如何导入动画 (page 1455)

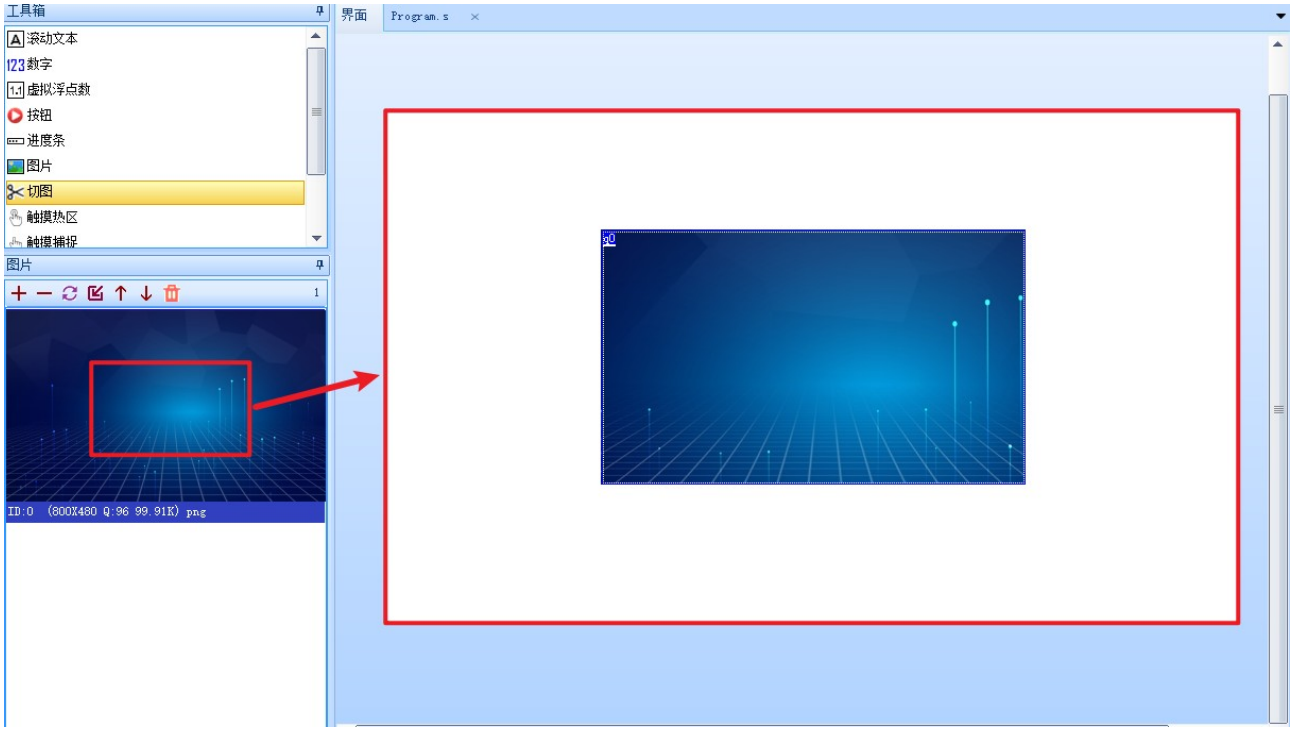
导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

切图控件的作用是将全屏图片的一部分抠出来显示，请注意切图控件 pic 属性对应的一定要是全屏图片







3.10.1 切图控件-使用详解

3.10.1.1 修改切图控件显示不同的图片

我们需要提前导入多张全屏图片，需要修改切图控件显示的图片时，通过上位机进行编辑或者通过指令进行修改即可

```
//让切图控件q0从id为0的图片切出一部分来显示
q0.pic=0
```

(续下页)

(接上页)

```
//让切图控件q1从id为1的图片切出一部分来显示
q1.pic=1
```

3.10.1.2 使用切图实现透明效果

t 系列和 k 系列不支持透明图层，如果要实现圆角矩形的控件，就得靠切图控件或者将按钮控件的 sta 设置为切图来来实现

K 系列和 T 系列如何实现透明效果 (page 1317)

3.10.2 切图控件-样例工程下载

演示工程下载链接：

《切图控件》演示工程下载

3.10.3 切图控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.10.4 切图控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

---

type 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

id 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

objname 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

vscope 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

drag 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

aph 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

effect 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

picc 属性 -切图图片 (必须是全屏图片)。可读，可通过上位机修改，可通过指令修改。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.11 触摸热区控件

提示： 视频教程：触摸热区控件

1、触摸热区控件可以理解为一个看不见的按钮控件

2、由于串口屏上没有函数的概念，因此有大量重复的代码需要调用时，可以将代码写在触摸热区内，然后用 click 指令去触发

3、单片机可以发 click 命令来激活相关控件状态。但是单片机需要确保显示屏处于当前界面（此界面包含对应的触摸热区控件）。

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.11.1 触摸热区控件-使用详解

3.11.1.1 通过指令触发触摸热区控件，实现代码的重复调用

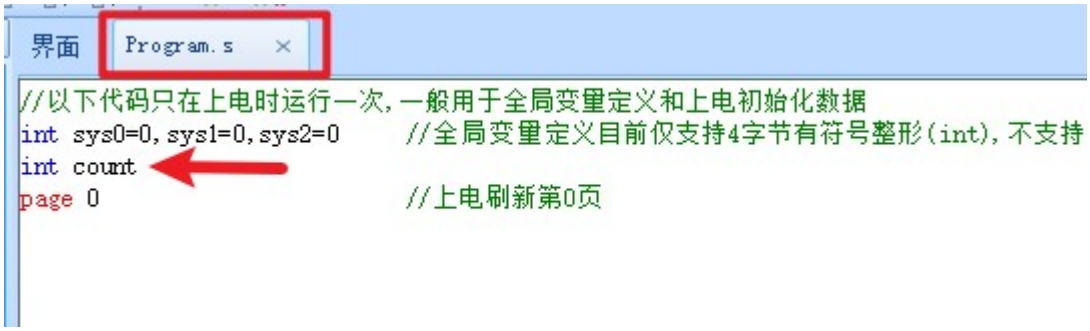
```
//触发触摸热区m0的按下事件
click m0,1

//触发触摸热区m0的弹起事件
click m0,0
```

注意： 请注意不要在控件内部 click 自身，会照成递归深度过大导致不可预见的问题

3.11.1.2 实现一个隐藏按钮功能，当点击的次数足够多时，进入后台页面

首先在 program.s 中定义一个 count 变量

```
int count
```

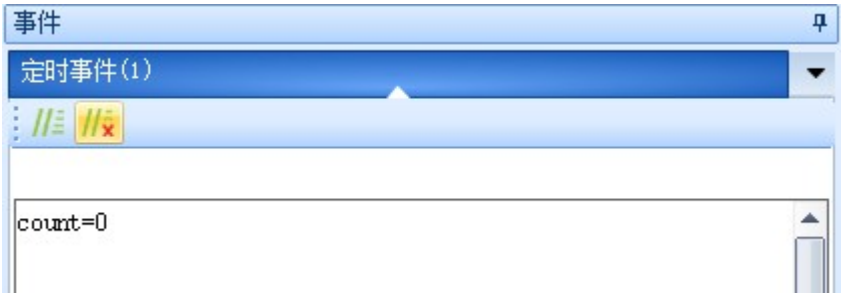


新建一个触摸热区，objname 改为 m_superUser，在触摸热区控件中编写以下代码

```
count++
if(count>5)
{
      //跳转到superUser页面
      page superUser
}
tm_reset.en=1
```

新建一个定时器，objname 改为 tm_reset，en 属性设置为 1，定时器的 tim 属性建议设置为 1000，在定时器中编写以下代码

```
count=0
```



注意： 每次给定时器的 tim 或者 en 属性赋值，都会使得定时器重新计时，因此只要用户每次点击的间隔小于 1000ms，连续 5 次点击，即可进入 superUser 页面，如果超过 1000ms，则 count 将被置零

3.11.2 触摸热区控件-样例工程下载

演示工程下载链接：

《触摸热区控件》演示工程下载

3.11.3 触摸热区控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.11.4 触摸热区控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

`vscope` 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

`x` 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

`y` 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

`w` 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

`h` 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.12 触摸捕捉控件

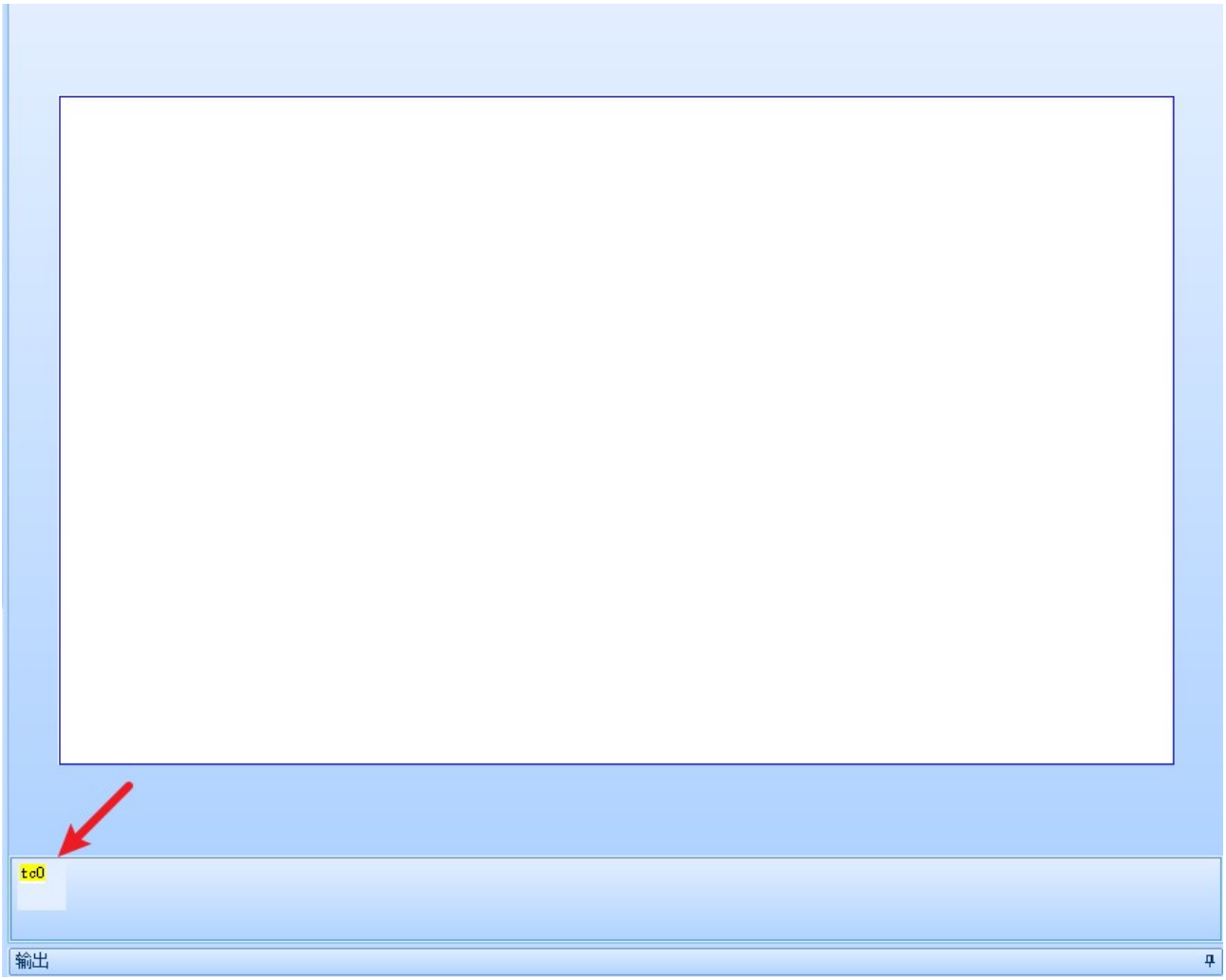
提示： 视频教程：触摸捕捉控件

触摸捕捉控件可以获取到被点击控件的 id。

click 指令触发控件时，不会导致触摸捕捉控件被触发，只有用手去点击屏幕时才会被捕捉。

每个页面虽然能创建多个触摸捕捉控件，但只有 1 个会生效（id 值最高且有事件代码的触摸捕捉控件会生效），因此请勿在同一个页面创建多个触摸捕捉控件。

触摸捕捉控件位于特殊控件栏上。



提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.12.1 触摸捕捉控件-使用详解

3.12.1.1 触摸捕捉控件触发顺序

当一个控件被触发时，触发的顺序如下

按下时：

触摸捕捉按下事件被触发 -> 控件的按下事件被触发

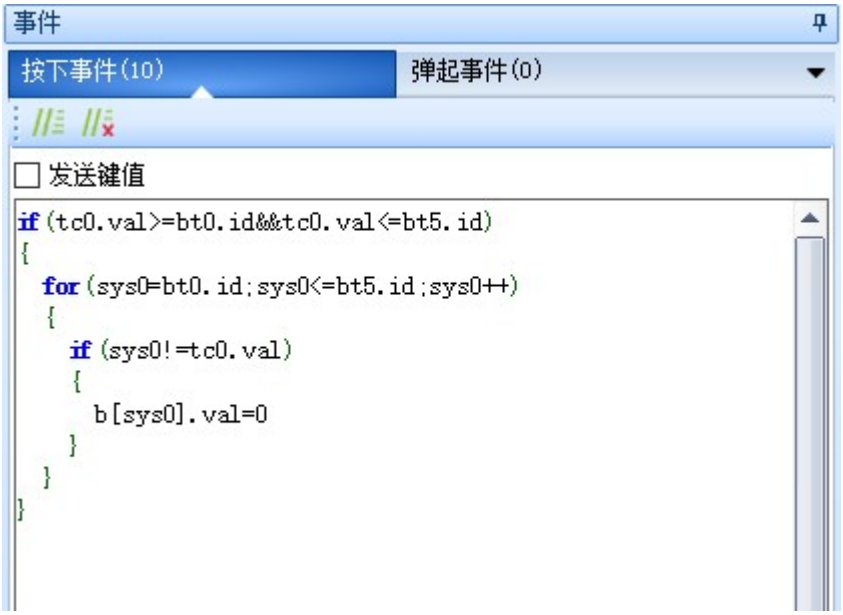
弹起时：

触摸捕捉弹起事件被触发 -> 控件的弹起事件被触发

因此，如果要在触摸捕捉控件中判断控件状态，例如判断双态按钮的状态，请写在控件的弹起事件中

3.12.1.2 触摸捕捉控件实现双态按钮之间的互斥（双态按钮 bt0-bt5 的 id 号是连续的）

```
if(tc0.val>=bt0.id&&tc0.val<=bt5.id)
{
   for(sys0=bt0.id;sys0<=bt5.id;sys0++)
   {
      if(sys0!=tc0.val)
      {
         b[sys0].val=0
      }
   }
}
```



注意： 如果出现报错信息，请检查双态按钮的 id 号是否连续

3.12.1.3 触摸捕捉控件实现屏保功能

需要搭配定时器控件和数字控件实现实现

新建一个触摸捕捉控件 tc0 和数字控件 n0，定时器控件 tm0，tm0 的 en 设置为 1，tim 设置为 1000

因为点击任何控件都会触发触摸捕捉控件，在触摸捕捉控件编写以下代码

```
n0.val=0     //任何触摸操作都会触发n0被清零
```

在定时器中编写以下代码

```
n0.val++
if(n0.val>60)
{
   //超过60秒，跳转到屏保页面
   page screenSaver
}
```

3.12.1.4 触摸捕捉控件实现蜂鸣器播放

需要支持蜂鸣器的屏幕, 如果是支持喇叭的屏幕, 可以用 play 指令来代替

3.12.1.4.1 点击任意位置都让蜂鸣器鸣叫

因为点击任何控件都会触发触摸捕捉控件，在触摸捕捉控件编写以下代码, 此时点击屏幕上的任意位置蜂鸣器都会响

```
beep 50
```

3.12.1.4.2 点击有控件的位置让蜂鸣器鸣叫, 点击空白位置不鸣叫

点击空白位置时是触发了页面控件, 其 id 固定为 0, 因此只要 id 是大于 0 的, 就是存在其他控件

```
if(tc0.val>0)
{
   beep 50
}
```

3.12.1.4.3 只有点击了按钮控件才会鸣叫, 点击其他控件不会鸣叫

判断此时的控件类型, 按键的控件类型是 98, 可以查看控件的 type 属性获得

```
if(b[tc0.val].type==98)
{
   beep 50
}
```

3.12.2 触摸捕捉控件-样例工程下载

演示工程下载链接：

《触摸捕捉控件》演示工程下载

3.12.3 触摸捕捉控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.12.4 触摸捕捉控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

`vscope` 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

`val` 属性 -本次捕捉控件 ID。可读，不可通过上位机进行修改，不可通过指令更改。

3.13 指针控件

提示： 视频教程：指针控件

用于在串口屏上显示时钟或者仪表指针，指针范围的值为 0-360，超过这个范围会返回 1C FF FF FF，即赋值操作失败

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.13.1 指针控件-使用详解

3.13.1.1 控制指针控件的转动角度

```
//让指针控件z0转动到100度的位置
z0.val=100
```

3.13.1.2 跨页面给指针控件赋值

将指针控件设置为全局，才能跨页面赋值

```
//让main页面的指针控件z0转动到100度的位置
main.z0.val=100
```

3.13.2 指针控件-c 语言示例

单片机通过串口给指针控件赋值

```
printf("z0.val=123\xff\xff\xff");

printf("z0.val=%d\xff\xff\xff",a);
```

注意： 给指针控件赋值必须是整形（int），否则会出错

3.13.3 指针控件-样例工程下载

演示工程下载链接：

《指针控件》演示工程下载

《指针缓动》演示工程下载

3.13.4 指针控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.13.5 指针控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

`vscope` 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

`drag` 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`aph` 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`effect` 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

`sta` 属性 -背景填充方式:0-切图;1-单色;2-图片;3-透明（仅 x 系列支持透明）。可读，可通过上位机修改，不可通过指令修改。

`val` 属性 -当前值

`format` 属性 -补偿值

`up` 属性 -尖长 (32767 为自动, 允许负数)

`down` 属性 -尾长 (允许负数)

`left` 属性 -中心偏移

`pco` 属性 -指针颜色

`pco2` 属性 -中心圆颜色

`hig` 属性 -中心圆直径, 最大值:100(0 为无中心圆)

`vvs0` 属性 -尖宽, 最大值:128

`vvs1` 属性 -腰宽, 最大值:128

vvs2 属性 -尾宽, 最大值:128

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.14 曲线波形控件

提示： 视频教程：曲线波形控件

用于在串口屏上显示波形。

因为控件越宽，需要显示的点越多，因此，曲线波形控件占用的内存大小和它的宽度有关

每个页面不能超过 5 个曲线波形控件

只能往当前页面的曲线控件添加数据，不支持跨页面添加曲线控件数据

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.14.1 曲线波形控件-使用详解

3.14.1.1 通过指令添加曲线波形控件 s0 数据

在曲线波形控件上添加曲线的指令请参考

add-往曲线控件添加数据 (page 622)

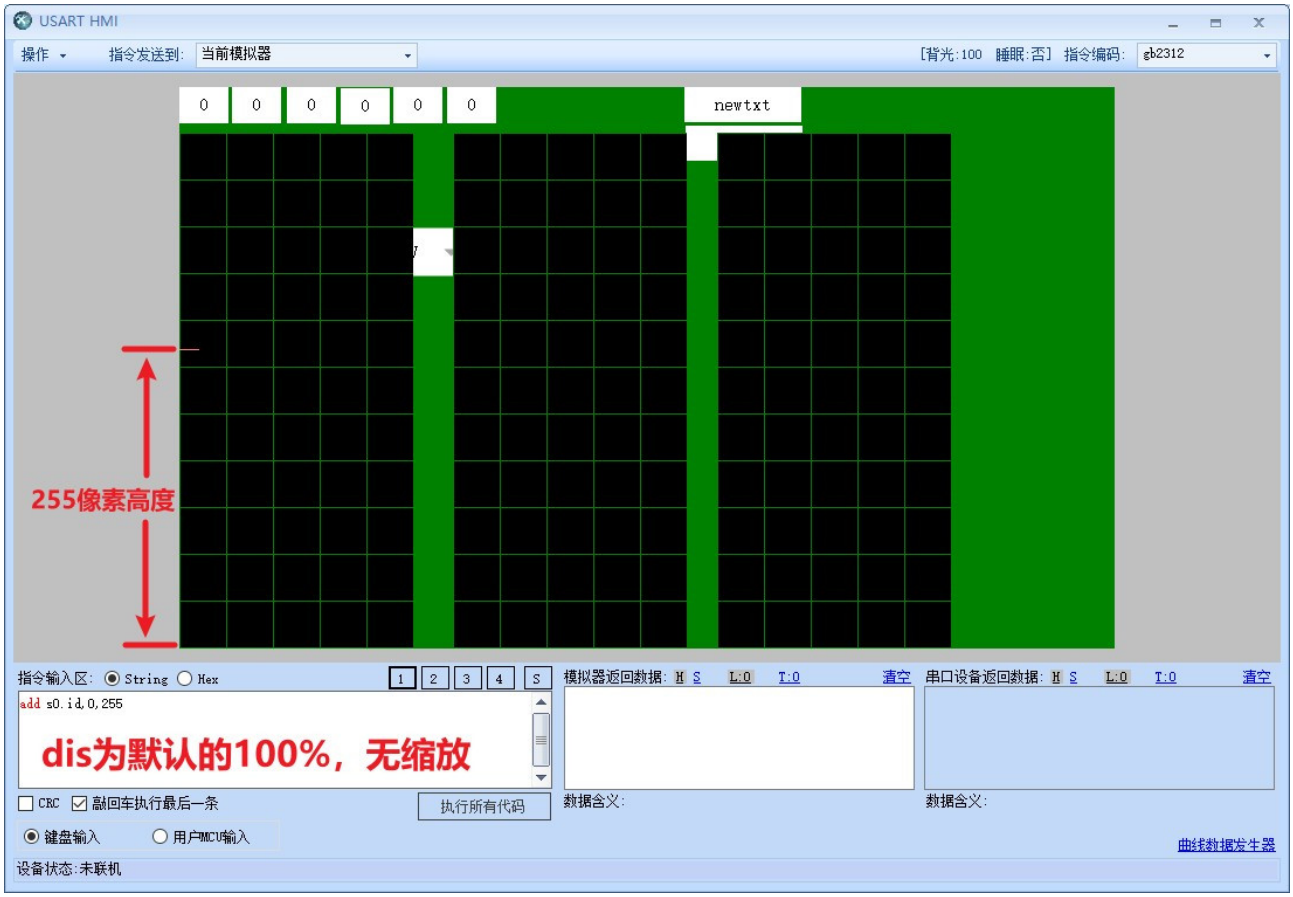
addt-曲线数据透传指令 (page 627)

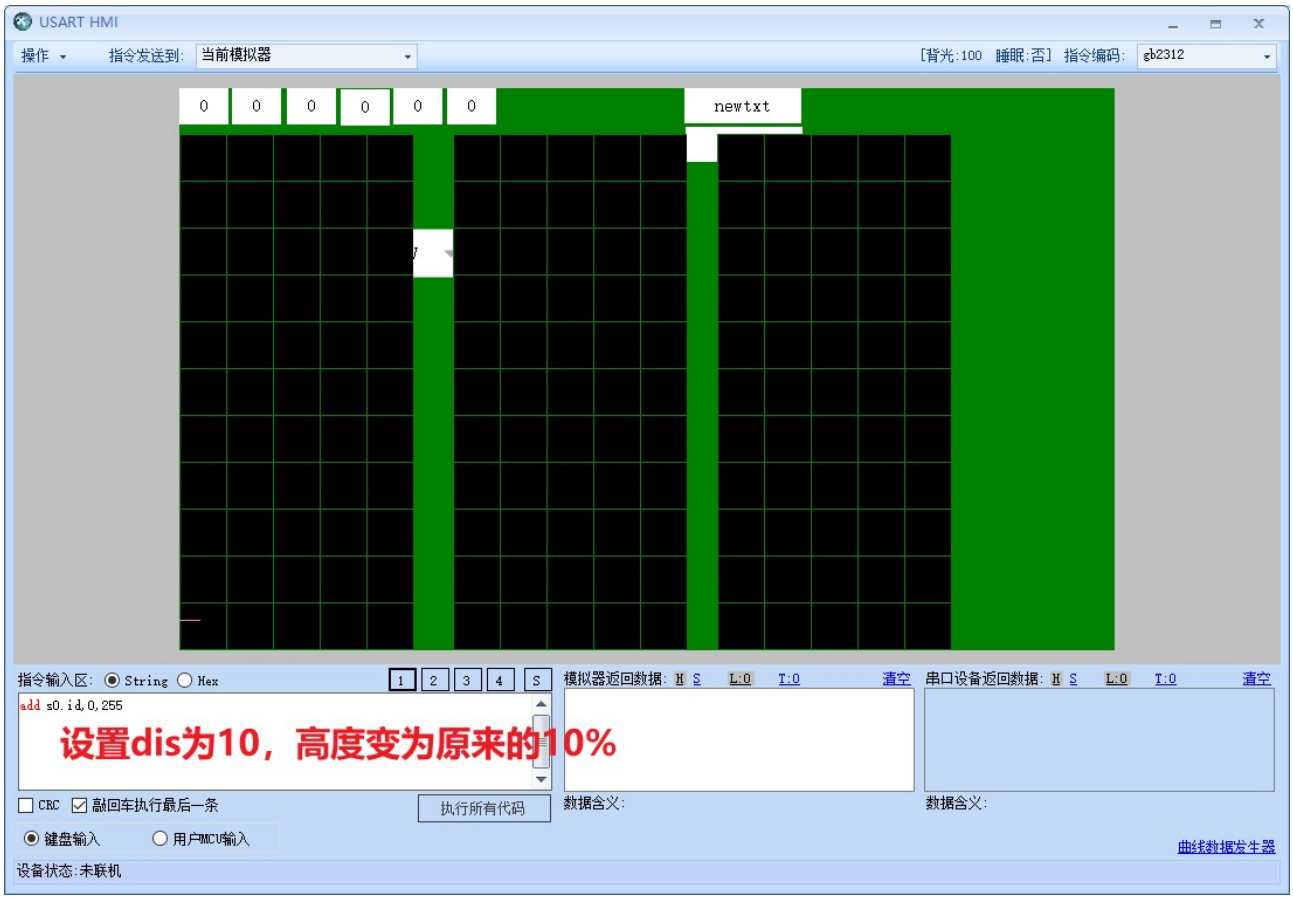
3.14.1.2 曲线控件添加的纵坐标值范围是多少

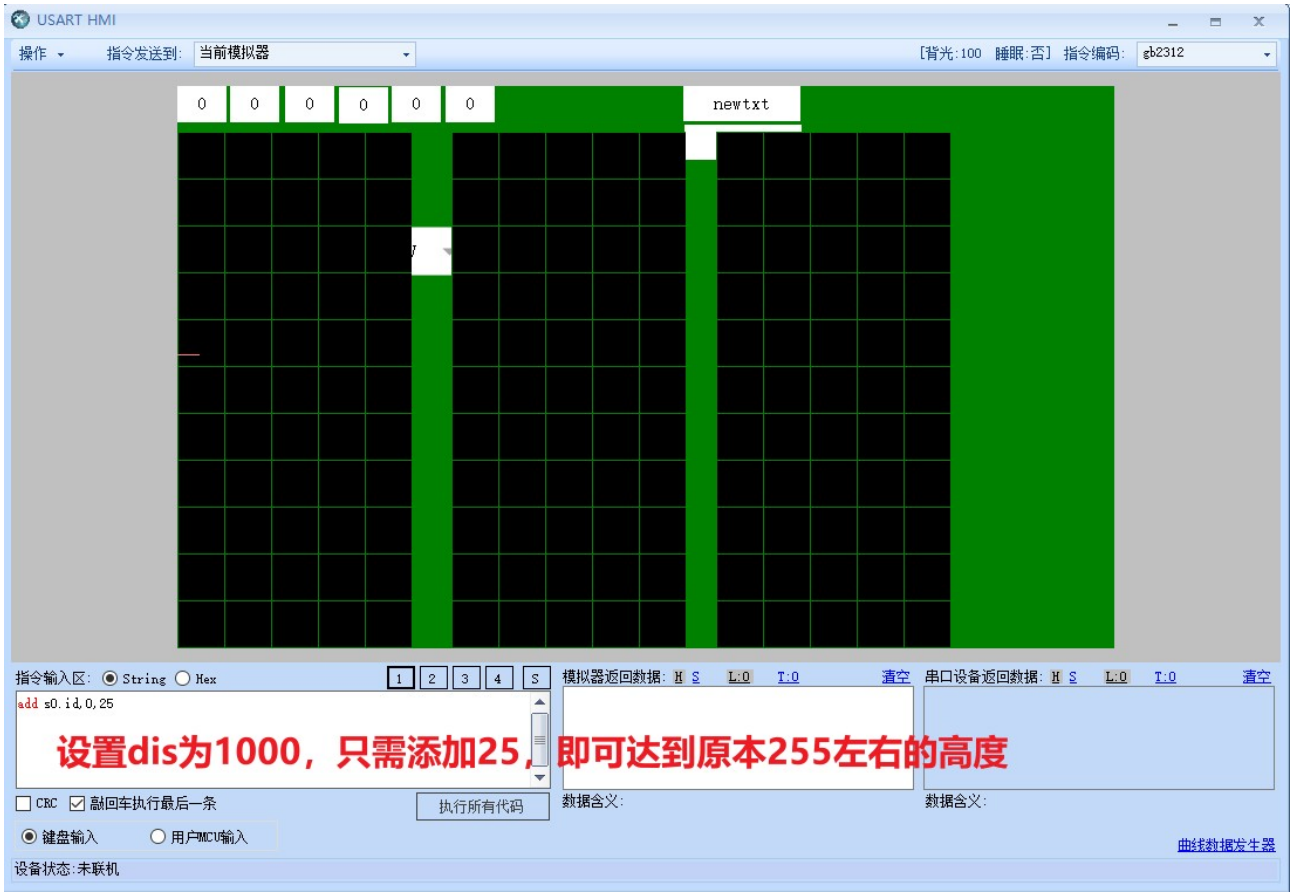
无论是使用 add 还是会用 addt，添加的值范围为 0-255，需要注意的是，最大值不要超过控件高度，即这个值必须小于 255 的同时也要小于控件高度

从曲线控件的底部开始算，在 dis（缩放比例）为默认 100% 的情况下，添加的值是多少，就代表距离曲线控件底部的高度

例如 add s0.id，0，255，会在距离曲线控件 s0 底部 255 个像素的地方开始画点


USART HMI
操作
指令发送到:
当前模拟器
[背光:100 睡眠:否] 指令编码:
gb2312
0 0 0 0 0 0
newtxt
255像素高度
指令输入区:
String
Hex
1 2 3 4 S
add s0.id,0,255
dis为默认的100%，无缩放
CRC
敲回车执行最后一条
执行所有代码
键盘输入
用户MCU输入
模拟器返回数据:
串口设备返回数据:
清空
数据含义:
曲线数据发生器
设备状态:未联机


USART HMI
操作
指令发送到:
当前模拟器
[背光:100 睡眠:否] 指令编码:
gb2312
0 0 0 0 0 0
newtxt
指令输入区:
String
Hex
1 2 3 4 S
add s0.id,0,255
设置dis为10，高度变为原来的10%
CRC
敲回车执行最后一条
执行所有代码
键盘输入
用户MCU输入
模拟器返回数据:
串口设备返回数据:
清空
数据含义:
曲线数据发生器
设备状态:未联机



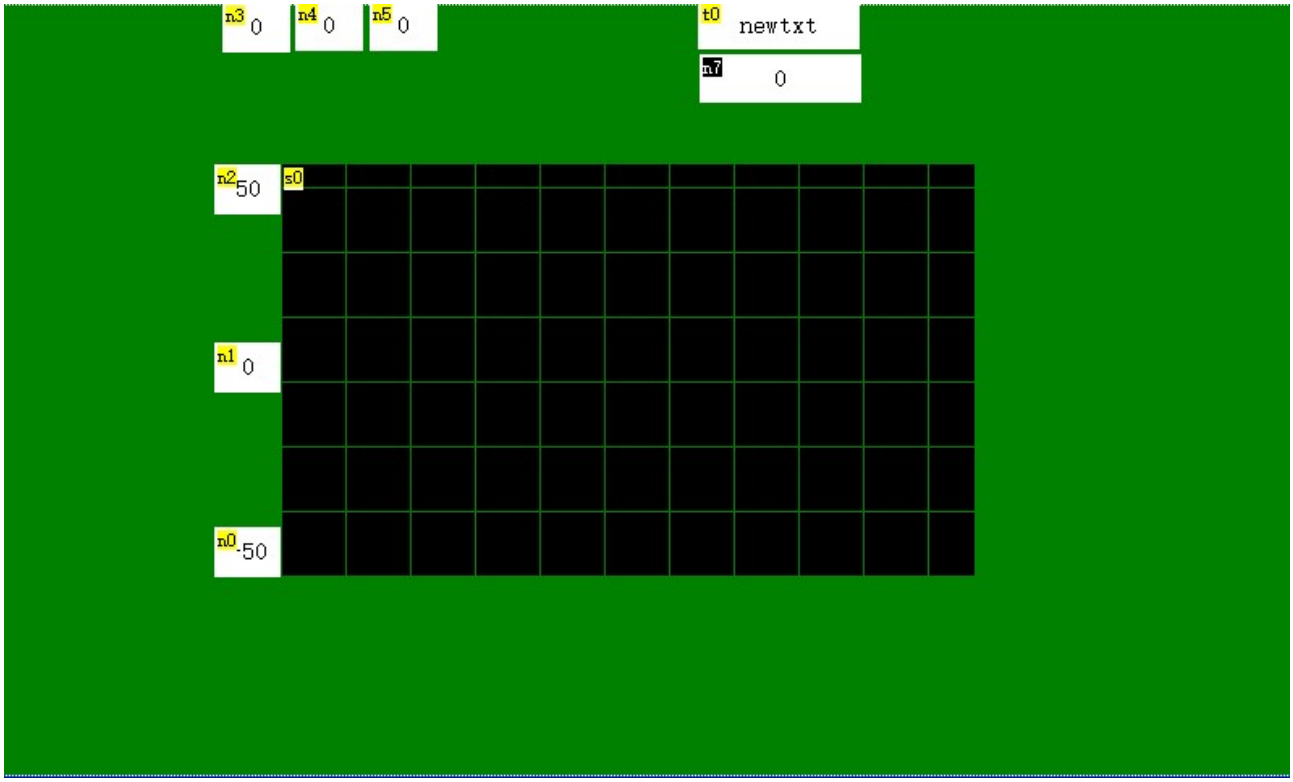
3.14.1.3 曲线控件如何表示负数

曲线控件只能添加 0-255 之间的数据，因此不能添加负数

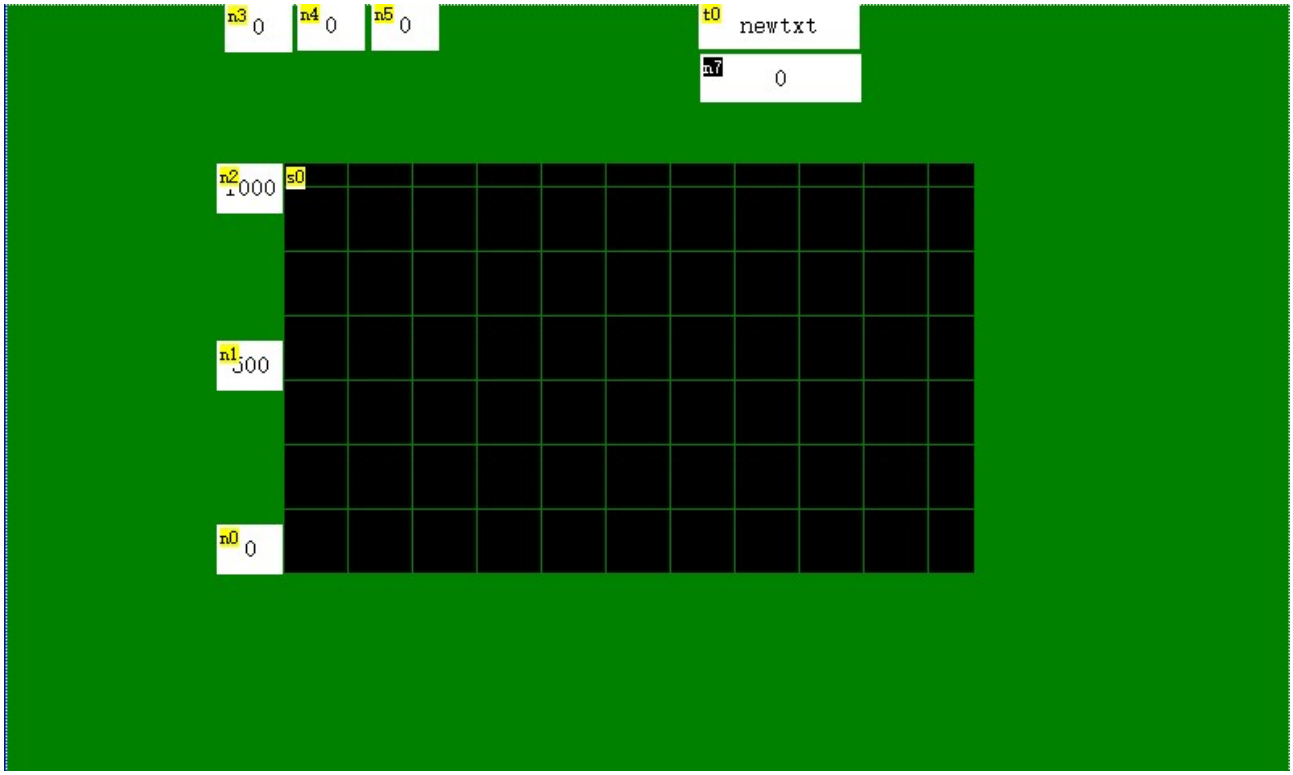
但是曲线控件只是表示一个趋势

用户不会在意你添加进去的值是多少，只会去看坐标

你只需要在曲线控件外部放好对应的刻度标识即可，如下图

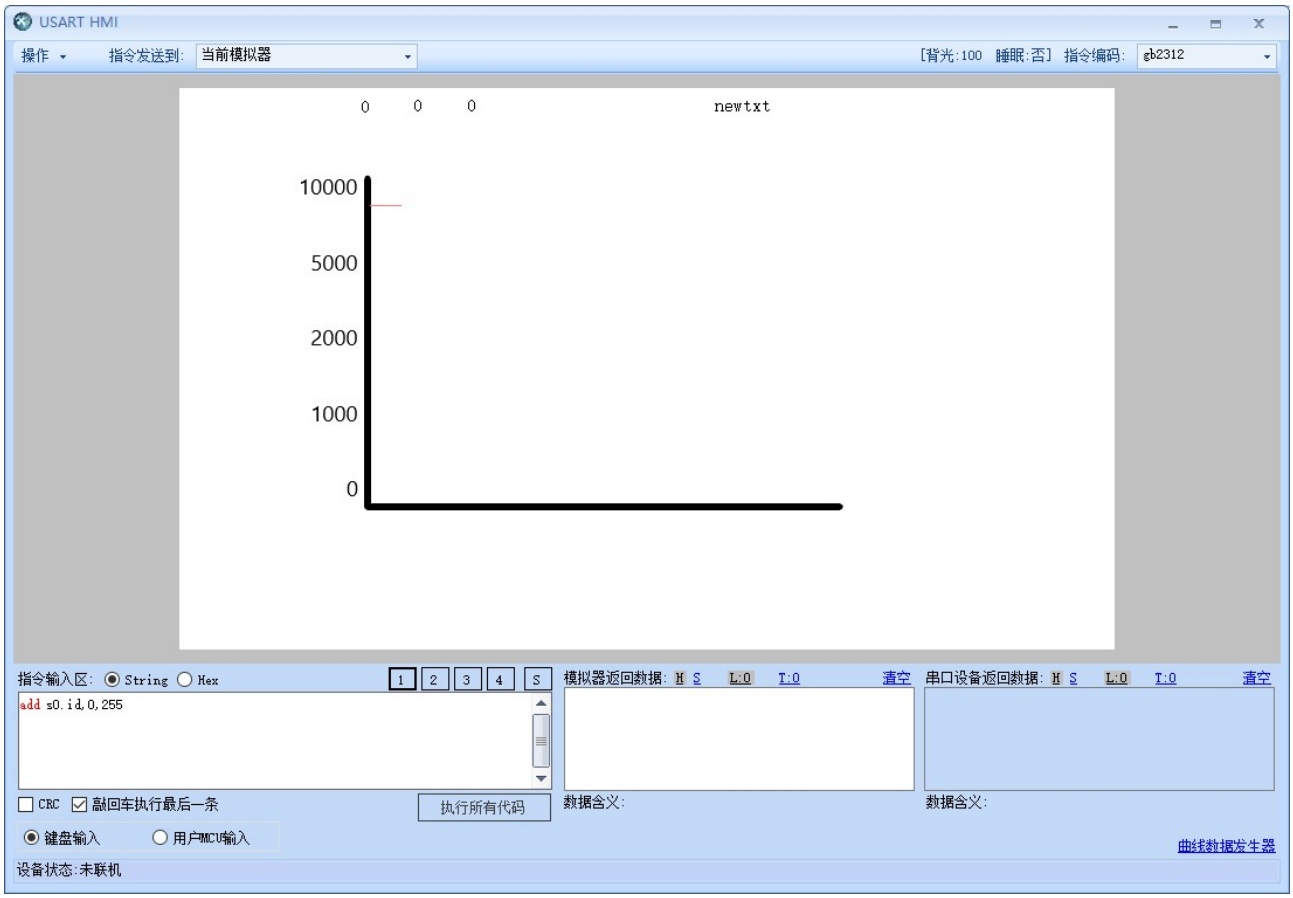


你不需要关心曲线控件里到底添加进去的值是多少，只要关心位置对不对即可，是不是在你需要的位置上，你可以随心所欲的标定你外部的刻度



直接用控件做的话比较丑，可以用 ps 做在背景图片上，甚至刻度也可以不是线性的

10000
5000
2000
1000
0


USART HMI
操作
指令发送到:
当前模拟器
[背光:100 睡眠:否] 指令编码:
gb2312
0 0 0
newtxt
10000
5000
2000
1000
0
指令输入区:
String
Hex
1
2
3
4
S
add s0.id,0,255
模拟器返回数据:
H S
L:0
T:0
清空
串口设备返回数据:
H S
L:0
T:0
清空
CRC
敲回车执行最后一条
执行所有代码
数据含义:
数据含义:
键盘输入
用户MCU输入
曲线数据发生器
设备状态:未联机

3.14.2 曲线波形控件-常见问题

3.14.2.1 使用 add 指令时屏幕没有任何反应

检查 id 号是否正确

检查通道号是否正确

检查线的颜色是否与背景颜色过于接近

> **注意：** 如果只运行了一次 add 指令，只绘制了一两个点，在屏幕上是很难看出来的，至少要连续运行多次 add 指令，才能看到曲线出现

3.14.3 曲线波形控件-样例工程下载

演示工程下载链接：

《曲线波形控件》演示工程下载

《曲线波形控件动态坐标》演示工程下载

《曲线控件方波 + 三角波 + 正弦波 (简易)+ 直线跟随》演示工程下载

《不使用存储卡跨页面记录波形数据 (发送)》演示工程下载

《不使用存储卡跨页面记录波形数据 (接收)》演示工程下载

3.14.4 曲线波形控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.14.5 曲线波形控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

`vscope` 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

`drag` 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`aph` 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

effect 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

sta 属性 -背景填充方式:0-切图;1-单色;2-图片;3-透明。可读，可通过上位机进行修改，不可通过指令更改。

dir 属性 -平推方向:0-从左往右;1-从右往左;2-靠右对齐。可读，可通过上位机进行修改，不可通过指令更改。

ch 属性 -通道数量 (最小 1，最大 4):1-1;2-2;3-3;4-4。可读，可通过上位机进行修改，不可通过指令更改。

bco 属性 -背景色，sta 为单色时可用。可读，可通过上位机进行修改，可通过指令更改。

gdc 属性 -网格颜色，sta 为单色时可用。可读，可通过上位机进行修改，可通过指令更改。

gdw 属性 -网格宽度，sta 为单色时可用，0 为无网格。可读，可通过上位机进行修改，可通过指令更改。

gdh 属性 -网格高度，sta 为单色时可用，0 为无网格。可读，可通过上位机进行修改，可通过指令更改。

pco0 属性 -通道 0 前景色。可读，可通过上位机进行修改，可通过指令更改。

pco0 属性 -通道 1 前景色，ch 至少设置为 2 才可用。可读，可通过上位机进行修改，可通过指令更改。/ pco0 属性 -通道 2 前景色，ch 至少设置为 3 才可用。可读，可通过上位机进行修改，可通过指令更改。

pco0 属性 -通道 3 前景色，ch 至少设置为 4 才可用。可读，可通过上位机进行修改，可通过指令更改。

dis 属性 -数据缩放百分比 (最小 10，最大 1000，即 10%-1000%，缩小到原来的十分之一到放到到原来的十倍)。可读，可通过上位机进行修改，可通过指令更改。

picc 属性 -背景切图图片 (必须是全屏图片)，sta 为图片时可用。可读，可通过上位机进行修改，可通过指令更改。

pic 属性 -背景图片，sta 为图片时可用。可读，可通过上位机进行修改，可通过指令更改。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.15 滑块控件

提示： 视频教程：滑块控件

用于在串口屏上滑动调整数值，例如亮度，音量

提示： 导入字库请参考: 如何导入字库 (page 1442)
导入图片请参考: 如何导入图片 (page 1449)
导入动画请参考: 如何导入动画 (page 1455)
导入视频请参考: 如何导入视频 (page 1461)
导入音频请参考: 如何导入音频 (page 1467)

3.15.1 滑块控件-使用详解

3.15.1.1 滑块控件修改亮度

建议新建一个 setting 页面，用于设置

在 setting 页面的前初始化事件编写以下代码

```
//将亮度赋值给滑块控件
h0.val=dim
```

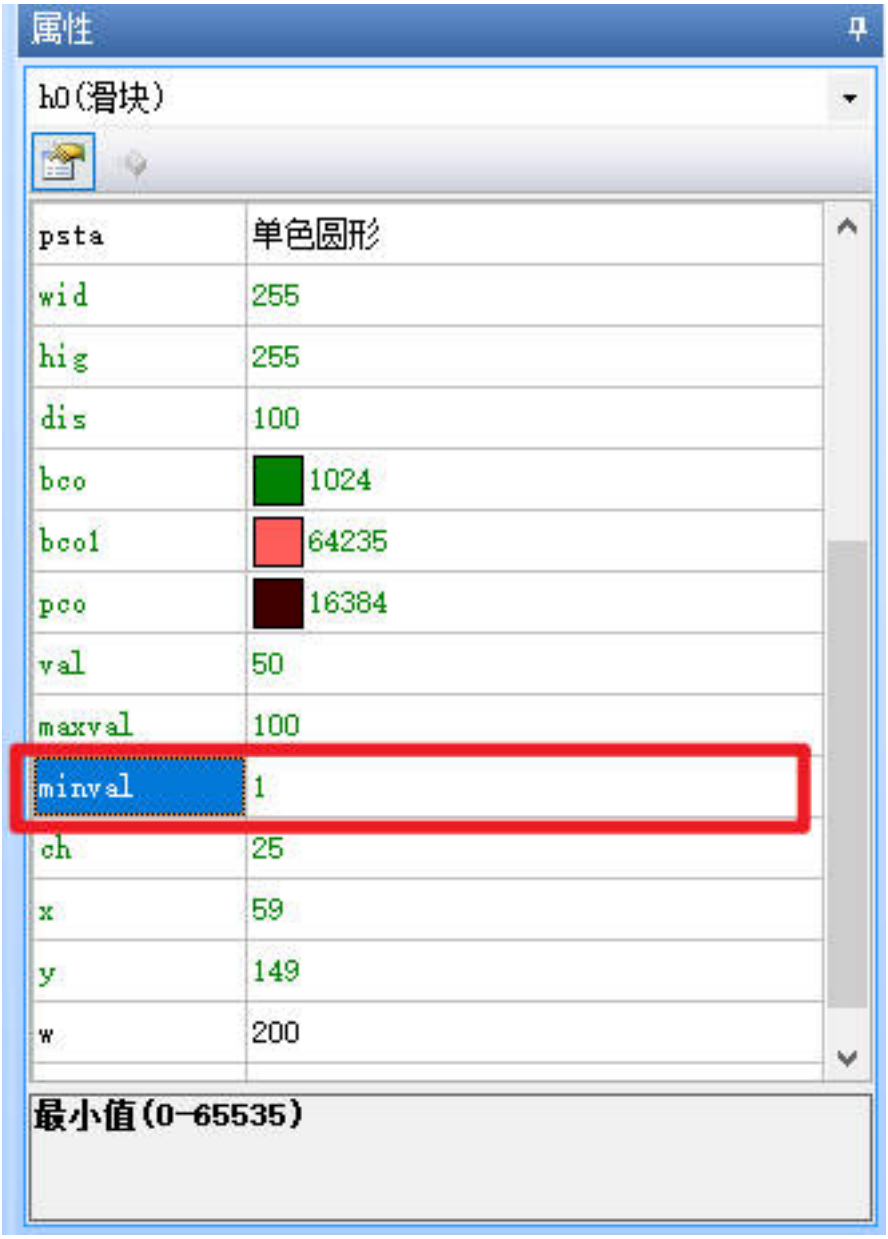
在滑块控件的滑动事件编写以下代码

```
//将滑块的值赋值给当前亮度
dim=h0.val
```

在滑块控件的弹起事件编写以下代码

```
//将滑块的值赋值给当前亮度
dim=h0.val
```

将滑块的 minval 设置为 1，避免将 dims 设置为 0 后，结果重启后发现屏幕不亮，以为屏幕坏了



注意:

1. 不建议新手操作 dims，因为 dims 属性是关机保存的，可能存在将 dims 设置为 0 后，结果重启后发现屏幕不亮，以为屏幕坏了，售后返修发现是亮度设置问题。
2. 请勿在滑动事件里操作 dims，而是应该操作 dim，因为 dims 需要写入 flash 保存，对于 T 系列和 K 系列来说，可能会导致在滑动过程中导致卡顿的现象。
3. 使用滑块调节亮度时，建议滑块的 minval 属性设置为大于 0 的整数，否则可能导致屏幕背光完全关闭（屏幕全黑，看不见任何东西），难以调回原本的亮度。

3.15.1.2 滑块控件修改音量大小

音量 volume 属性仅带喇叭的型号（例如 X3、X5 系列）支持

在页面的前初始化事件编写以下代码

```
//将音量值赋值给滑块控件
h1.val=volume
```

在滑块控件的滑动事件编写以下代码

```
//将滑块的值赋值给音量
volume=h1.val
```

在滑块控件的弹起事件编写以下代码

```
//将滑块的值赋值给音量
volume=h1.val
```

注意: 音量属性是默认关机保存的，设置了，下次开机也是这个值

3.15.1.3 发送滑块的数据

在滑块控件的弹起事件编写以下代码

```
//发送帧头 55 aa，01是标志位，代表下发的是滑块数据
printh 55 aa 01

//发送1字节的滑块值
prints h0.val,1

//发送帧尾
printh 0d 0a
```

3.15.2 滑块控件-注意事项

t1,t0,k1 系列的里面负责滑动的滑块, 建议设计得外部会变化的部分一样大, 才能挡住后面的部分 (模拟器和串口屏的效果不太一样)

3.15.3 滑块控件-样例工程下载

演示工程下载链接:

《RGB 进度条颜色混合》演示工程下载

《滑块控件》演示工程下载

《滑块控件控制亮度》演示工程下载

《滑块控件十个挡位》演示工程下载

《滑动解锁》演示工程下载

3.15.4 滑块控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.15.5 滑块控件-属性详解

控件属性解析 (page 385)

提示: 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

`vscope` 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

`drag` 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`aph` 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`effect` 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

`mode` 属性 -滑行方向:0-横向;1-竖向。可读，可通过上位机修改，不可通过指令修改。

`sta` 属性 -背景填充方式:0-切图;1-单色;2-图片。可读，可通过上位机修改，不可通过指令修改。

`psta` 属性 -游标填充方式:0-单色圆形;1-图片;2-单色四边形。可读，可通过上位机修改，不可通过指令修改。

`wid` 属性 -游标宽度 (255 为自动，0 为无游标)。可读，可通过上位机修改，可通过指令修改。

`hig` 属性 -游标高度 (255 为自动，0 为无游标)。可读，可通过上位机修改，可通过指令修改。

dis 属性 -外形圆角半径百分比 (0-100,0 为直角)。可读，可通过上位机修改，可通过指令修改。

picc 属性 -背景切图图片 (必须是全屏图片)。sta 为切图时可用。可读，可通过上位机修改，可通过指令修改。

picc1 属性 -背景切图图片 (必须是全屏图片)。sta 为切图时可用。可读，可通过上位机修改，可通过指令修改。

pic 属性 -背景图片。sta 为图片时可用。可读，可通过上位机修改，可通过指令修改。

pic1 属性 -背景图片。sta 为图片时可用。可读，可通过上位机修改，可通过指令修改。

pic2 属性 -游标图片。psta 为图片时可用。可读，可通过上位机修改，可通过指令修改。

bco 属性 -背景色。sta 为单色时可用。可读，可通过上位机修改，可通过指令修改。

bco1 属性 -前景色。sta 为单色时可用。可读，可通过上位机修改，可通过指令修改。

pco 属性 -游标颜色。psta 为单色时可用。可读，可通过上位机修改，可通过指令修改。

val 属性 -当前值。可读，可通过上位机修改，可通过指令修改。

maxval 属性 -最大值 (0-65535)。可读，可通过上位机修改，可通过指令修改。仅在滑动时可以限制 val 值。

minval 属性 -最小值 (0-65535)。可读，可通过上位机修改，可通过指令修改。仅在滑动时可以限制 val 值。

ch 属性 -背景宽度百分比 (0-100,0 为无需填充背景)。可读，可通过上位机修改，可通过指令修改。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.16 定时器控件

提示： 视频教程：定时器控件

定时器控件用于定时执行某些代码，或者延时执行某些代码，当定时被使能时，定时器里的代码会定时执行。

注意： 1、每个页面的定时器数量不能超过 12 个。

2、定时器只能在当前页面运行，不可后台运行，如果想要定时器一直运行，每个页面都加个定时器。

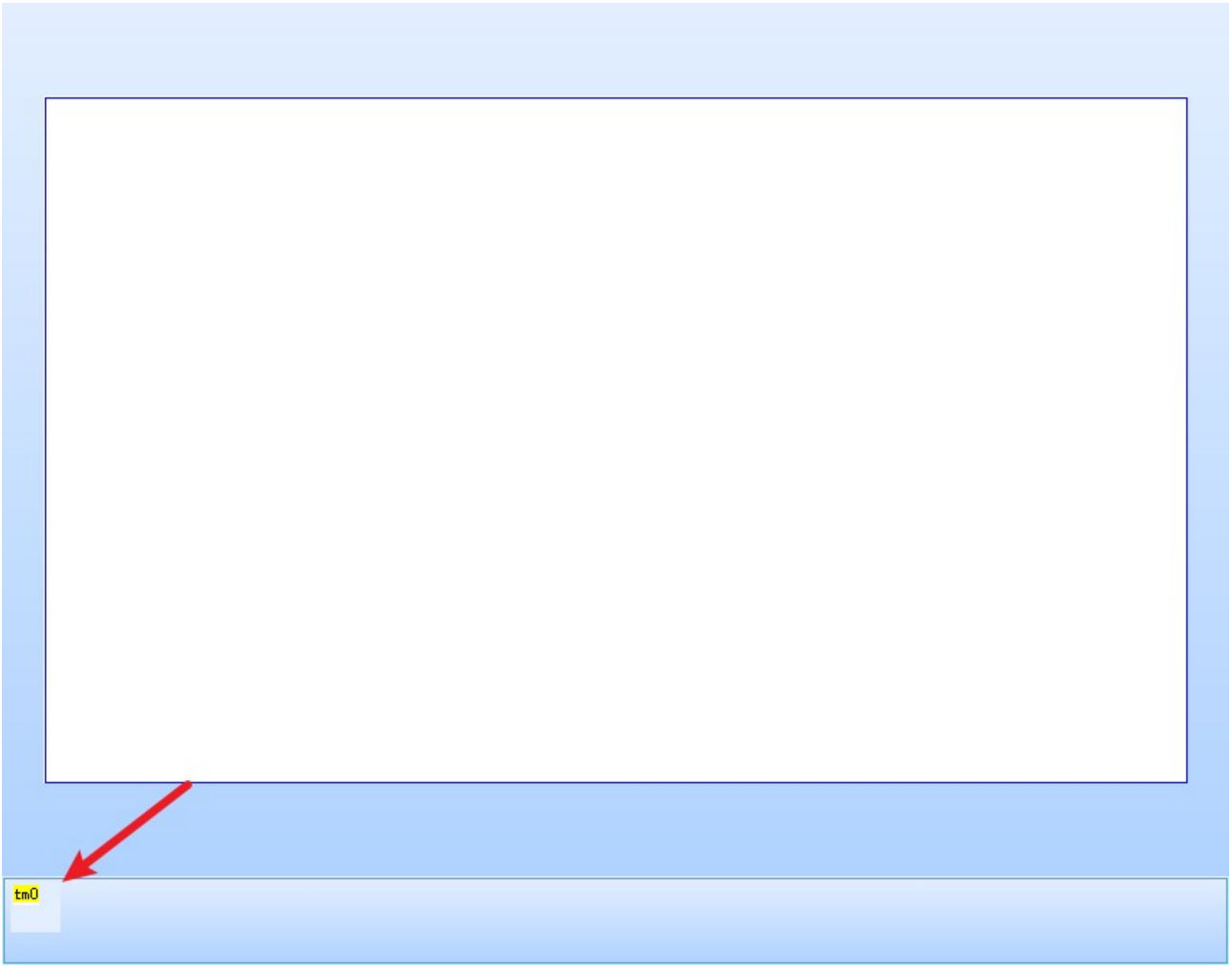
3、定时器最小的定时时间为 50ms，定时器的精度会随代码的复杂程度变化，会有一定的误差。

4、由于每个人的电脑性能不同，模拟器中定时器的效果可能会与实物有较大的差距，电脑上可能会运行得很准，但是串口屏实物上会偏慢，与屏幕内代码的复杂程度有关，请以实物为准。

5、请不要使用定时器来替代 RTC 功能，不带 RTC 的型号，使用定时器来实现是完全不现实的。

6、串口屏进入休眠（睡眠）模式后，定时器会停止运行，退出休眠（睡眠）模式后，定时器会继续运行。

定时器控件位于特殊控件栏上。



提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.16.1 定时器控件-使用详解

3.16.1.1 关闭定时器

```
tm0.en=0
```

3.16.1.2 打开定时器

```
//打开定时器
tm0.en=1
```

注意: 在定时器开启的情况下，对定时器的任意属性赋值（设置 tim 属性或者 en 属性），都会导致定时器重新计时。

例如一个定时时间为 1000ms（1 秒）的定时器 tm0，另一个定时时间为 50ms 的定时器 tm1 中，在定时器 tm1 中不断执行 tm0.en=1，那么 tm0 将永远没有执行的机会

若不想定时器重新计时，请使用以下操作:

```
//判断定时器是否打开，如果没打开，则打开
if(tm0.en!=1)
{
    tm0.en=1
}
```

相关链接

在定时器 0 里面加入使能定时器 1 的语句为什么系统不执行 (page 1387)

3.16.1.3 定时器最大时间只能设置为 65534(65 秒多), 怎么计时更长时间

例: 计时 1 小时

新建一个定时器 tm0，tim 属性设置为 1000（定时器每 1000ms 执行一次定时事件中的代码），新建一个变量 n0，用于计时

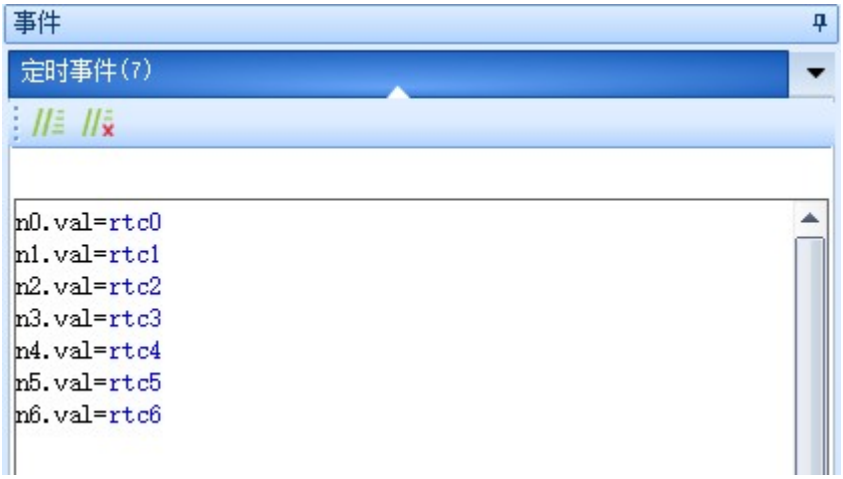
在定时器 tm0 的定时事件编写以下代码

```
n0.val++
//3600秒,即1小时
if(n0.val>=3600)
{
   //计时时间到,复位用于计时的变量
   n0.val=0
   //其他需要执行的代码

}
```

3.16.1.4 通过定时器定时刷新 RTC 到数字控件上

定时器的 tim 设置为 300(定时器每 300ms 执行一次定时事件中的代码)，en 设置为 1(定时器开启)

```
n0.val=rtc0
n1.val=rtc1
n2.val=rtc2
n3.val=rtc3
n4.val=rtc4
n5.val=rtc5
n6.val=rtc6
```



提示： 为什么将定时器设置为 300ms?

因为 300ms 可以相对均匀的显示时间变化，不会占用太多性能（定时器定时时间越短，会越频繁进入定时器，占用更多单片机的性能）

3.16.1.5 两秒后跳转到其他页面

定时器的 tim 设置为 2000(定时器每 2000ms 执行一次定时事件中的代码)，en 设置为 1(定时器开启)

定时器中的代码为

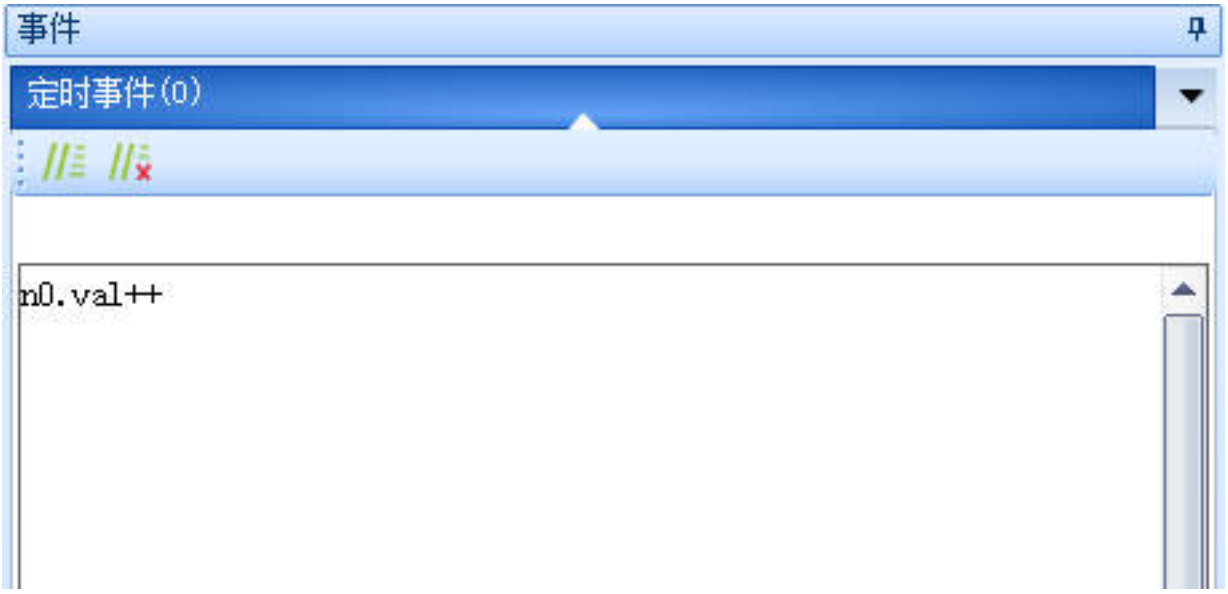
```
page page1    //跳转到page1
```

3.16.1.6 让 n0 控件每隔 1 秒自动增加 1

定时器的 tim 设置为 1000(定时器每 1000ms 执行一次定时事件中的代码)，en 设置为 1(定时器开启)

定时器中的代码为

```
n0.val++
```



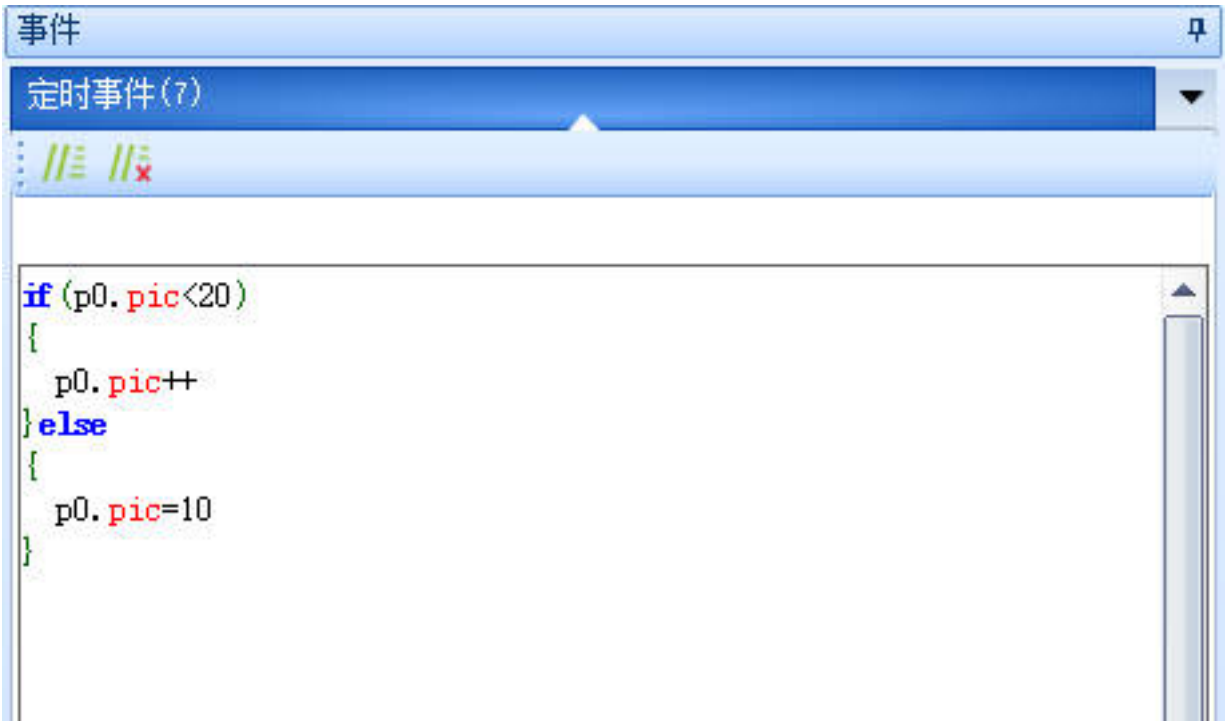
3.16.1.7 使用定时器配合图片控件循环播放图片

定时器的 tim 设置为 100(定时器每 100ms 执行一次定时事件中的代码)，en 设置为 1(定时器开启)

假设图片 id 从 10-20

定时器中的代码为

```
if(p0.pic<20)
{
     p0.pic++
}else
{
     p0.pic=10
}
```



3.16.1.8 每个页面限制 12 个定时器，不够用怎么办

很多时候，多个定时器是可以和在一起的，比如我现在有 3 个定时事件，一个是 10 秒执行一次，一个是 30 秒执行一次，一个是 60 秒执行一次，那需要创建 3 个定时器吗，其实并不需要，只需要创建一个就够了

需要新建一个定时器 tm0，一个数字控件 n0，三个文本控件 t0，t1，t2

tm0 控件的 tim 属性设置为 1000

定时器中的代码为

```
t0.txt=""
t1.txt=""
t2.txt=""
n0.val++
//因为不能在if里计算,我们需要提前计算好,在if中进行判断
sys0=n0.val%10
sys1=n0.val%30
sys2=n0.val%60
if(sys0==0)
{
```

(续下页)

(接上页)

```
    //每10秒需要做的事
    t0.txt="10s"
}
if(sys1==0)
{
    //每30秒需要做的事
    t1.txt="30s"
}
if(sys2==0)
{
    //每60秒需要做的事
    t2.txt="60s"
}
```

3.16.1.9 同一页面两个时间相同的定时器，谁先执行

根据定时器控件 id 大小决定，id 小的先执行

3.16.2 定时器控件-样例工程下载

演示工程下载链接：

《定时器控件》演示工程下载

《超过范围时提示超量程》演示工程下载

《定时器和图片控件实现动画》演示工程下载

3.16.3 定时器控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

显示时间和设置时间样例工程 (page 1133)

串口屏跨页面计时样例工程 (page 1114)

3.16.4 定时器控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

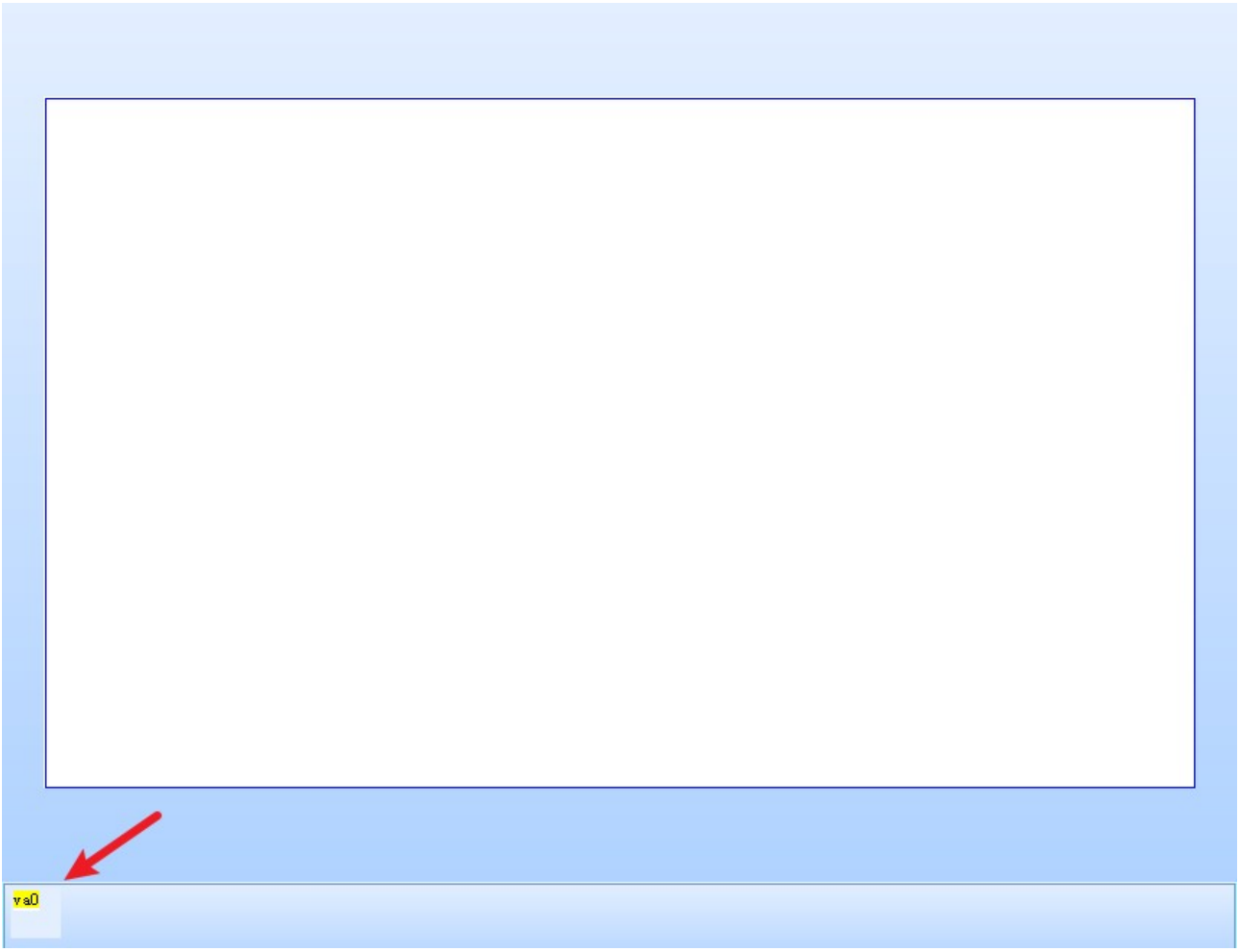
`vscope` 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

`tim` 属性 -定时时间, 单位:ms(最小 50, 最大 65534)，可读，可通过上位机进行修改，可通过指令更改。在定时器开启的情况下，对定时器的 tim 属性赋值，会导致定时器重新计时

`en` 属性 -使能开关:0 为关闭,1 为开启，可读，可通过上位机进行修改，可通过指令更改。在定时器开启的情况下，对定时器的 en 属性赋值，会导致定时器重新计时

3.17 变量控件

变量控件位于特殊控件栏上



变量控件将 sta 设置为数值时，相当于一个看不见的数字控件，使用方法参考数字控件

变量控件将 sta 设置为字符串时，相当于一个看不见的文本控件，使用方法参考文本控件

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.17.1 变量控件-样例工程下载

演示工程下载链接：

《变量控件》演示工程下载

3.17.2 变量控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.17.3 变量控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

`vscope` 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

`sta` 属性 -数据类型：数值或字符串。可通过上位机进行修改，不可通过指令更改。

`val` 属性 -初始值 (最小-2147483648, 最大 2147483647)。当 sta 为数值时可用。可读，可通过上位机进行修改，可通过指令更改。

`txt` 属性 -初始字符内容。当 sta 为字符串时可用。可读，可通过上位机进行修改，可通过指令更改。

`txt_maxl` 属性 -字符最大长度 (即分配内存空间)。当 sta 为字符串时可用。可读，可通过上位机进行修改，不可通过指令更改。

3.18 双态按钮控件

提示： 视频教程：双态按钮控件

用于在串口屏上显示按钮，此按钮类似于自锁开关，使用前需要提前导入字库，制作字库请参考创建字库和导入字库 (page 125)

如何修改显示的字体大小：需要提前导入不同大小的字库，需要修改控件显示的字体大小时，通过上位机编辑或者通过指令修改控件的 font 属性即可，请参考如何修改控件显示的字体 (page 1251)

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.18.1 双态按钮控件-使用详解

3.18.1.1 修改双态按钮控件显示文字的大小和样式

我们需要提前导入不同大小的字库，需要修改双态按钮控件显示大小时，通过上位机进行编辑或者通过指令进行修改即可，请参考如何修改控件显示的字体 (page 1251)

```
//将b0控件的字体设置为1号字体
b0.font=1

//将main页面的b0控件的字体设置为1号字体
main.b0.font=1
```

3.18.1.2 让双态按钮在开始和停止之间切换

```
if(bt0.val==1)
{
   bt0.txt="开始"
}
else
{
   bt0.txt="停止"
}
```

3.18.1.3 双态按钮实现按键锁定功能

实现按键锁定功能 (page 1423)

3.18.1.4 为什么按下的状态会把弹起的事件也执行了

按下和弹起不是指按钮的状态，而是你手指的状态，手指按下去就一定会执行按下事件，手指放开就一定会执行弹起事件

请在按下或弹起事件里进行如下判断

```
if(bt0.val==1)
{
    //此时按键是按下的
}
else
{
    //此时按键是弹起的
}
```

3.18.2 双态按钮-样例工程下载

演示工程下载链接：

《双态按钮控件》演示工程下载

3.18.3 双态按钮-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.18.4 双态按钮控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

---

type 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

id 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

objname 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

vscope 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

drag 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

aph 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

effect 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

sta 属性 -背景填充方式:0-切图;1-单色;2-图片;3-透明（仅 x 系列支持透明）。可读，可通过上位机修改，不可通过指令修改。

picc 属性 -按钮没按下时切图背景 (必须是全屏图片)，sta 为切图时才有这个属性。可读，可通过上位机修改，可通过指令修改。

picc2 属性 -按钮按下时切图背景 (必须是全屏图片)，sta 为切图时才有这个属性。可读，可通过上位机修改，可通过指令修改。

bco 属性 -按钮没按下时背景色，sta 为单色时才有这个属性。可读，可通过上位机修改，可通过指令修改。

bco2 属性 -按钮按下时背景色，sta 为单色时才有这个属性。可读，可通过上位机修改，可通过指令修改。

style 属性 -显示风格:0-平面;1-边框;2-3D_Down;3-3D_Up;4-3D_Auto，sta 为单色时才有这个属性。可读，可通过上位机修改，不可通过指令修改。

borderc 属性 -边框颜色。当 style 设置为边框时可用。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

borderw 属性边框粗细。当 style 设置为边框时可用。最大值:255。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

pic 属性 -按钮没按下时背景图片，sta 为图片时才有这个属性。可读，可通过上位机修改，可通过指令修改。

pic2 属性 -按钮按下时背景图片，sta 为图片时才有这个属性。可读，可通过上位机修改，可通过指令修改。

pco 属性 -按钮没按下时字体色。可读，可通过上位机修改，可通过指令修改。

pco2 属性 -按钮按下时字体色。可读，可通过上位机修改，可通过指令修改。

font 属性 -控件调用的字库 id，调用不同的字库会显示不同的字体或字号。可读，可通过上位机修改，可通过指令修改。参考：1、创建字库和导入字库 (page 125) 2、指定字库 (page 1284)

xcen 属性 -水平对齐:0-靠左;1-居中;2-靠右。可读，可通过上位机修改，可通过指令修改。

ycen 属性 -垂直对齐:0-靠上;1-居中;2-靠下。可读，可通过上位机修改，可通过指令修改。

val 属性 -当前状态 (0 或 1),0 为弹起，1 为按下。可读，可通过上位机修改，可通过指令修改。

txt 属性 -字符内容（按钮上显示的文本）。可读，可通过上位机修改，可通过指令修改。

txt_maxl 属性 -控件分配的内存空间。GB2312 下，ascii 字符占用 1 个字节，一个汉字占 2 字节。UTF8 下，ascii 字符占用 1 个字节，一个汉字占 3 字节。可读，可通过上位机修改，不可通过指令修改。

isbr 属性 -是否自动换行:0-否;1-是。可读，可通过上位机修改，可通过指令修改。

spax 属性 -字符横向间距 (最小 0, 最大 255)。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

spay 属性 -字符纵向间距 (最小 0, 最大 255)。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.19 复选框控件

用于在串口屏上显示复选框

提示: 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.19.1 复选框控件-使用详解

3.19.1.1 在按钮控件中判断复选框是否选中

在按钮控件的按下事件中编写以下代码

```
if(c0.val==1)
{
    //如果c0被选中
}
if(c1.val==1)
{
    //如果c1被选中
}
if(c2.val==1)
{
    //如果c2被选中
}
if(c3.val==1)
{
    //如果c3被选中
}
```

3.19.2 复选框控件-样例工程下载

演示工程下载链接：

《复选框控件》演示工程下载

3.19.3 复选框控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.19.4 复选框控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

`vscope` 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

`drag` 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

aph 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

effect 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

style 属性 -显示风格:0-平面;1-边框;2-3D_Down;3-3D_Up;4-3D_Auto，sta 为单色时才有这个属性。可读，可通过上位机修改，不可通过指令修改。

borderc 属性 -边框颜色。当 style 设置为边框时可用。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

borderw 属性边框粗细。当 style 设置为边框时可用。最大值:255。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

bco 属性 -按钮没按下时背景色，sta 为单色时才有这个属性。可读，可通过上位机修改，可通过指令修改。

pco 属性 -按钮没按下时字体色。可读，可通过上位机修改，可通过指令修改。

val 属性 -当前状态 (0 或 1)，即是否被选中。可读，可通过上位机修改，可通过指令修改。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.20 单选框控件

用于在串口屏上显示单选框，实现单选功能。

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.20.1 单选框控件-使用详解

3.20.1.1 1、单选框互斥效果

单选框之间一般是互斥的，但是这个互斥效果需要自己实现

需要通过触摸捕捉控件，实现单选框之间的互斥（单选框 r0-r5 的 id 号必须是连续的）

在触摸捕捉控件中编写以下代码

```
if(tc0.val>=r0.id&&tc0.val<=r5.id)
{
   for(sys0=r0.id;sys0<=r5.id;sys0++)
   {
      if(sys0!=tc0.val)
      {
         b[sys0].val=0
      }
   }
}
```

注意： 如果出现报错信息，请检查 id 号是否连续

3.20.2 单选框控件-样例工程下载

演示工程下载链接：

《单选框控件》演示工程下载

3.20.3 单选框控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.20.4 单选框控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

---

type 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

id 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

objname 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

vscope 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

drag 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

aph 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

effect 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

bco 属性 -按钮没按下时背景色，sta 为单色时才有这个属性。可读，可通过上位机修改，可通过指令修改。

pco 属性 -按钮没按下时字体色。可读，可通过上位机修改，可通过指令修改。

val 属性 -当前状态 (0 或 1)，即是否被选中。可读，可通过上位机修改，可通过指令修改。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.21 二维码控件

提示： 视频教程：二维码控件

用于在串口屏上显示动态的二维码，如果二维码是固定不变的，或者只有几个，则建议使用图片控件来代替

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.21.1 二维码控件-使用详解

3.21.1.1 修改二维码显示内容

新建一个按钮控件 b0，在 b0 控件的弹起事件中编写以下代码

```
//给qr0赋值字符串
qr0.txt="http://wiki.tjc1688.com"
```

3.21.1.2 显示微信二维码或者公众号二维码

把你的二维码截图，导进串口屏工程中，然后用图片控件来显示

3.21.2 二维码控件-c 语言示例

单片机通过串口给二维码控件赋值

二维码可以用手机扫描打开检查，如果扫描出来的结果不对，请检查控件的 txt_maxl 属性，保证分配的空间能完整放下整个二维码字符串

```
//给qr0赋值字符串
printf("qr0.txt=\"http://wiki.tjc1688.com\"\xff\xff\xff");
```

3.21.3 二维码控件-样例工程下载

演示工程下载链接：

《二维码控件》演示工程下载

3.21.4 二维码控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.21.5 二维码控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

`vscope` 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

`drag` 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`aph` 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`effect` 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

`sta` 属性 -中心 LOGO:0-无;1-有。可读，可通过上位机进行修改，不可通过指令更改

`pic` 属性 -中心 LOGO 图片。仅当 sta 为有时可用。可读，可通过上位机修改，可通过指令修改，不允许为空。

`bco` 属性 -背景色。可读，可通过上位机修改，可通过指令修改。

`pco` 属性 -前景色。可读，可通过上位机修改，可通过指令修改。

`txt` 属性 -二维码中的字符内容

`txt_maxl` 属性 -字符最大长度 (即分配内存空间:0-192)

`x` 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

`y` 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

`w` 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

`h` 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.22 状态开关控件

提示： 视频教程：状态开关控件

状态开关控件仅 X2、X3、X5 系列支持

用于在串口屏上显示状态开关，仅 X 系列支持，其他系列可以用双态按钮代替，使用前需要提前导入字库，制作字库请参考创建字库和导入字库 (page 125)

如何修改显示的字体大小：需要提前导入不同大小的字库，需要修改控件显示的字体大小时，通过上位机编辑或者通过指令修改控件的 font 属性即可，请参考如何修改控件显示的字体 (page 1251)

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.22.1 状态开关控件-使用详解

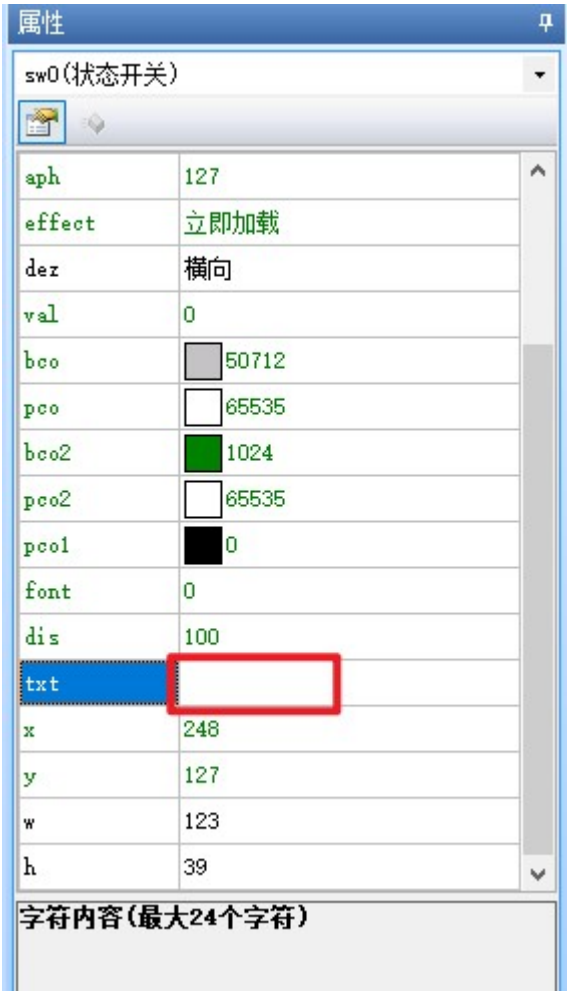
3.22.1.1 修改状态开关的显示文本为打开/关闭

也可以通过指令修改

```
sw0.txt="打开/关闭"
```

3.22.1.2 仅显示按钮，不显示文本

删除 txt 属性的内容



也可以通过指令修改

```
sw0.txt=""
```

3.22.1.3 如何修改状态开关控件显示的字体大小和样式

请参考如何修改控件显示的字体 (page 1251)

3.22.2 状态开关控件-样例工程下载

演示工程下载链接：

《状态开关》演示工程下载

3.22.3 状态开关控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.22.4 状态开关控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

`vscope` 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

`drag` 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`aph` 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`effect` 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

`dez` 属性 -方向:0-横向;1-竖向。可读，可通过上位机修改，不可通过指令修改。

`val` 属性 -当前值。可读，可通过上位机修改，可通过指令修改。

`bco` 属性 -状态 0 背景色。可读，可通过上位机修改，可通过指令修改。

`pco` 属性 -状态 0 圆心色。可读，可通过上位机修改，可通过指令修改。

`bco2` 属性 -状态 1 背景色。可读，可通过上位机修改，可通过指令修改。

`pco2` 属性 -状态 1 圆心色。可读，可通过上位机修改，可通过指令修改。

`pco1` 属性 -字体色。可读，可通过上位机修改，可通过指令修改。

`font` 属性 -字库。可读，可通过上位机修改，可通过指令修改。

`dis` 属性 -外形圆角半径百分比 (0-100,0 为直角)。可读，可通过上位机修改，可通过指令修改。

`txt` 属性 -字符内容 (最大 24 个字符)。可读，可通过上位机修改，可通过指令修改。

`x` 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

`y` 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.23 下拉框控件

下拉框控件仅 X2、X3、X5 系列支持

用于在串口屏上显示文本信息，包括数字，字母，符号，汉字和其他各国语言，使用前需要提前导入字库，制作字库请参考创建字库和导入字库 (page 125)

如何修改显示的字体大小：需要提前导入不同大小的字库，需要修改控件显示的字体大小时，通过上位机编辑或者通过指令修改控件的 font 属性即可，请参考如何修改控件显示的字体 (page 1251)

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.23.1 下拉框控件-使用详解

3.23.1.1 下拉框控件赋值

```
//向下拉框cb0赋值三行,分别是123、456、789
cb0.path="123\r456\r789"

//将t1、t2、t3赋值给下拉框cb0
cb0.path=t1.txt+"\r"+t2.txt+"\r"+t3.txt
```

3.23.1.2 将 t0.txt 动态添加到下拉框控件

```
t0.txt="123"

//向下拉文本cb0的下拉框中添加一个回车换行
cb0.path+="\r"

//将t0.txt添加到下拉文本cb0
cb0.path+=t0.txt
```

3.23.1.3 删除下拉框中指定的某一行数据

以默认添加的下拉框为例，我们删除其中 100V 这行

添加三个文本控件 key，t1，t2，key 的 txt_maxl 设置为 50，t1 和 t2 的 txt_maxl 设置为 100

```
//设置要删除的数据为100V
key.txt="100V"

//在100V后面添加一个回车换行
key.txt+="\r"

//截取100V前面的数据
spstr cb0.path,t1.txt,key.txt,0

//截取100V后面的数据
```

(续下页)

(接上页)

```
spstr cb0.path,t2.txt,key.txt,1

//将100V前面的数据和100V后面的数据重新拼接后赋值给cb0.path
cb0.path=t1.txt+t2.txt
```

3.23.1.4 让下拉框缩回

设置下拉框的 down 属性

```
cb0.down=0
```

3.23.1.5 获取下拉框被当前被选中的值

如果需要在 cb0 选中某一项时执行, 请在 cb0 的弹起事件中写以下代码

```
//判断cb0是弹起状态
if(cb0.down==0)
{
   //将cb0.txt赋值给给t0.txt
   t0.txt=cb0.txt
}
```

3.23.1.6 发送下拉框被当前被选中的值

如果需要在 cb0 选中某一项时执行, 请在 cb0 的弹起事件中写以下代码

```
//判断cb0是弹起状态
if(cb0.down==0)
{
   printh 55 aa    //发送帧头
   prints cb0.txt,0  //发送数据
   printh 0d 0a    //发送帧尾
}
```

3.23.1.7 将下拉框置顶

由于下拉框是可展开的，因此在展开是可能会被其他控件挡住，此时就需要将下拉框置顶

在下拉框的按下事件中写

```
//将下拉框cb0置于最顶层
setlayer cb0,255
```

3.23.1.8 当点击某个下拉框时让其他下拉框缩回

三个下拉框 cb0，cb1，cb2

下拉框 cb0 的按下事件中写

```
cb1.down=0
cb2.down=0
```

下拉框 cb1 的按下事件中写

```
cb0.down=0
cb2.down=0
```

下拉框 cb2 的按下事件中写

```
cb0.down=0
cb1.down=0
```

3.23.1.9 下拉框被其他下拉框挡住

可以用 setlayer 指令将当前下拉框置顶*setlayer-运行中改变控件图层顺序* (page 654)

或者用 vis 指令隐藏其他的下拉框*vis-隐藏/显示控件* (page 595)

3.23.1.10 如何修改下拉框控件显示的字体大小和样式

请参考*如何修改控件显示的字体* (page 1251)

3.23.2 下拉框控件-样例工程下载

演示工程下载链接：

《下拉框控件》演示工程下载

3.23.3 下拉框控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 val 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.23.4 下拉框控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：*控件属性-控件 id 对照表* (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：*如何更改控件的前后图层关系* (page 1315)

objname 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

vscope 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

drag 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

aph 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

effect 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

sta 属性 -背景填充方式:0-切图;1-单色;2-图片;3-透明（仅 x 系列支持透明）。可读，可通过上位机修改，不可通过指令修改。

picc 属性 -切图背景 (必须是全屏图片)，sta 为切图时才有这个属性。可读，可通过上位机修改，可通过指令修改。

bco 属性 -背景色，sta 为单色时才有这个属性。可读，可通过上位机修改，可通过指令修改。

pic 属性 -背景图片，sta 为图片时才有这个属性。可读，可通过上位机修改，可通过指令修改。

pco 属性 -字体色。可读，可通过上位机修改，可通过指令修改。

font 属性 -控件调用的字库 id，调用不同的字库会显示不同的字体或字号。可读，可通过上位机修改，可通过指令修改。参考：1、创建字库和导入字库 (page 125) 2、指定字库 (page 1284)

xcen 属性 -水平对齐:0-靠左;1-居中;2-靠右。可读，可通过上位机修改，可通过指令修改。

ycen 属性 -垂直对齐:0-靠上;1-居中;2-靠下。可读，可通过上位机修改，可通过指令修改。

spax 属性 -字符横向间距 (最小 0, 最大 255)。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

dis 属性 -外形圆角半径百分比 (0-100,0 为直角)

txt 属性 -字符内容 (运行中会根据 val 值的变化自动改变)。可读，可通过上位机修改，可通过指令修改。

txt_maxl 属性 -字符最大长度 (即分配内存空间)。GB2312 下，ascii 字符占用 1 个字节，一个汉字占 2 字节。UTF8 下，ascii 字符占用 1 个字节，一个汉字占 3 字节。可读，可通过上位机修改，不可通过指令修改。

up 属性 -字符框是否显示箭头:0-不显示;1-显示。可读，可通过上位机修改，可通过指令修改。

pco3 属性 -字符框箭头颜色。可读，可通过上位机修改，可通过指令修改。

bco1 属性 -单元格背景色。可读，可通过上位机修改，可通过指令修改。

pco1 属性 -单元格字体色。可读，可通过上位机修改，可通过指令修改。

path 属性 -单元格内容集合 (请使用多行输入, 每行为一条单元格内容)。可读，可通过上位机修改，可通过指令修改。

path_m 属性 -单元格内容集合最大长度 (即分配内存空间)。可读，可通过上位机修改，不可通过指令修改。

dir 属性 -单元格展开方向:0-上;1-下;2-左;3-右。可读，可通过上位机修改，可通过指令修改。

qty 属性 -最大展开单元格数量 (1-254)。可读，可通过上位机修改，可通过指令修改。

vvs0 属性 -展开单元格间距 (最小 0, 最大 255)。可读，可通过上位机修改，可通过指令修改。

val 属性 -当前选中单元格 ID(运行中每次改变会将相应的内容同步到 txt 属性)。可读，可通过上位机修改，可通过指令修改。

bco2 属性 -选中单元格背景色。可读，可通过上位机修改，可通过指令修改。

pco2 属性 -选中单元格字体色。可读，可通过上位机修改，可通过指令修改。

hig 属性 -单元格高度/宽度 (16-255, 上下展开时表示高度; 左右展开时表示宽度)。可读，可通过上位机修改，可通过指令修改。

down 属性 -单元格展开状态:0-收缩;1-展开。可读，可通过上位机修改，可通过指令修改。

mode 属性 -单元格可滑动标识类型:0-无显示;1-仅显示三角形;2-显示背景 + 三角形。可读，可通过上位机修改，可通过指令修改。

wid 属性 -单元格可滑动标识高度/宽度 (8-255, 上下展开时表示高度; 左右展开时表示宽度)。可读，可通过上位机修改，可通过指令修改。

vvs1 属性 -单元格可滑动标识与单元格间距 (0-255)。可读，可通过上位机修改，可通过指令修改。

ch 属性 -滑动惯性力度 (0-32,0 为无惯性)。可读，可通过上位机修改，可通过指令修改。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.24 选择文本控件

选择文本控件仅 X2、X3、X5 系列支持

用于在串口屏上展示语言选择，设置日期等，使用前需要提前导入字库，制作字库请参考创建字库和导入字库 (page 125)

如何修改显示的字体大小：需要提前导入不同大小的字库，需要修改控件显示的字体大小时，通过上位机编辑或者通过指令修改控件的 font 属性即可，请参考如何修改控件显示的字体 (page 1251)

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.24.1 选择文本控件-使用详解

3.24.1.1 将 t0.txt 动态添加到选择文本控件

```
t0.txt="123"

//向下拉文本cb0的选择文本中添加一个回车换行
select0.path+="\r"

//向下拉文本cb0的选择文本中添加t0.txt
select0.path+=t0.txt
```

3.24.1.2 删除选择文本中指定的某一行数据

以默认添加的选择文本为例，我们删除其中 06 这行

添加三个文本控件 key，t1，t2，key 的 txt_maxl 设置为 50，t1 和 t2 的 txt_maxl 设置为 100

```
//设置要删除的数据为06
key.txt="06"

//在06后面添加一个回车换行
key.txt+="\r"

//截取06前面的数据
spstr select0.path,t1.txt,key.txt,0

//截取06后面的数据
spstr select0.path,t2.txt,key.txt,1

//将06前面的数据和06后面的数据重新拼接后赋值给select0.path
select0.path=t1.txt+t2.txt
```

3.24.1.3 在弹起事件里面获取到的 val/txt 属性和显示的值不符

因为滑动文本控件他有个滑动惯性, 你手指离开之后还会继续滑动, 可能这个值就变换了, 在定时器里面不断去获取就好了

也就是在定时器里写

```
t0.txt=select0.txt    //获取选中行的文本
```

或者

```
n0.val=select0.val    //获取选中行的编号
```

3.24.1.4 获取选择文本被当前被选中的值

在进入滑动选择文本所在的页面时, 如果没有滑动选择文本, 将无法获取 select0.txt, 需要对 select0.val 进行赋值后, 才能成功获取 select0.txt

```
select0.val=select0.val
t0.txt=select0.txt
```

3.24.1.5 如何修改选择文本控件显示的字体大小和样式

请参考如何修改控件显示的字体 (page 1251)

3.24.2 选择文本控件-样例工程下载

演示工程下载链接:

《选择文本控件》演示工程下载

3.24.3 选择文本控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.24.4 选择文本控件-属性详解

控件属性解析 (page 385)

提示: 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

vscope 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

drag 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

aph 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

effect 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

sta 属性 -背景填充方式:0-切图;1-单色;2-图片;3-透明（仅 x 系列支持透明）。可读，可通过上位机修改，不可通过指令修改。

picc 属性 -切图背景 (必须是全屏图片)，sta 为切图时才有这个属性。可读，可通过上位机修改，可通过指令修改。

bco 属性 -背景色，sta 为单色时才有这个属性。可读，可通过上位机修改，可通过指令修改。

style 属性 -显示风格:0-平面;1-边框，sta 为单色时才有这个属性。可读，可通过上位机修改，不可通过指令修改。

borderc 属性 -边框颜色。当 style 设置为边框时可用。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

borderw 属性边框粗细。当 style 设置为边框时可用。最大值:255。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

pic 属性 -背景图片，sta 为图片时才有这个属性。可读，可通过上位机修改，可通过指令修改。

pco 属性 -字体色。可读，可通过上位机修改，可通过指令修改。

pco2 属性 -选中项字体色。可读，可通过上位机修改，可通过指令修改。

key 属性 -绑定键盘。可读，可通过上位机修改，不可通过指令修改。

font 属性 -控件调用的字库 id，调用不同的字库会显示不同的字体或字号。可读，可通过上位机修改，可通过指令修改。参考：1、创建字库和导入字库 (page 125) 2、指定字库 (page 1284)

xcen 属性 -水平对齐:0-靠左;1-居中;2-靠右。可读，可通过上位机修改，可通过指令修改。

ycen 属性 -垂直对齐:0-靠上;1-居中;2-靠下。可读，可通过上位机修改，可通过指令修改。

pw 属性 -是否显示为密码 (内容仍为实际内容, 仅仅显示出来为 *):0-否;1-是。可读，可通过上位机修改，可通过指令修改。

hig 属性 -显示记录的行高度 (最小 16, 最大 255)。可读，可通过上位机修改，可通过指令修改。

dis 属性 -选中项加横线:0-否;1-是。可读，可通过上位机修改，可通过指令修改。

pco1 属性 -横线颜色。可读，可通过上位机修改，可通过指令修改。

txt 属性 -当前 val 值指向的文本 (自动随 val 值改变, 只可获取不可设置)。

val 属性 -当前选择项 ID(每改变一次,txt 值会相应变化)。可读，可通过上位机修改，可通过指令修改。

ch 属性 -滑动惯性力度 (0-32,0 为无惯性)。可读，可通过上位机修改，可通过指令修改。

path 属性 -滑动选择项集合 (每行为一项)。可读，可通过上位机修改，可通过指令修改。

path_m 属性 -滑动选择项集合最大长度 (即分配内存空间)。可读，可通过上位机修改，不可通过指令修改。

spax 属性 -字符横向间距 (最小 0, 最大 255)。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.25 滑动文本控件

滑动文本控件仅 X2、X3、X5 系列支持

用于在串口屏上显示文本信息，包括数字，字母，符号，汉字和其他各国语言，使用前需要提前导入字库，制作字库请参考创建字库和导入字库 (page 125)

如何修改显示的字体大小：需要提前导入不同大小的字库，需要修改控件显示的字体大小时，通过上位机编辑或者通过指令修改控件的 font 属性即可，请参考如何修改控件显示的字体 (page 1251)

注意： 有的用户把滑动文本控件当日志显示来用了，输出几百上千字节，也会导致串口屏卡顿。

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

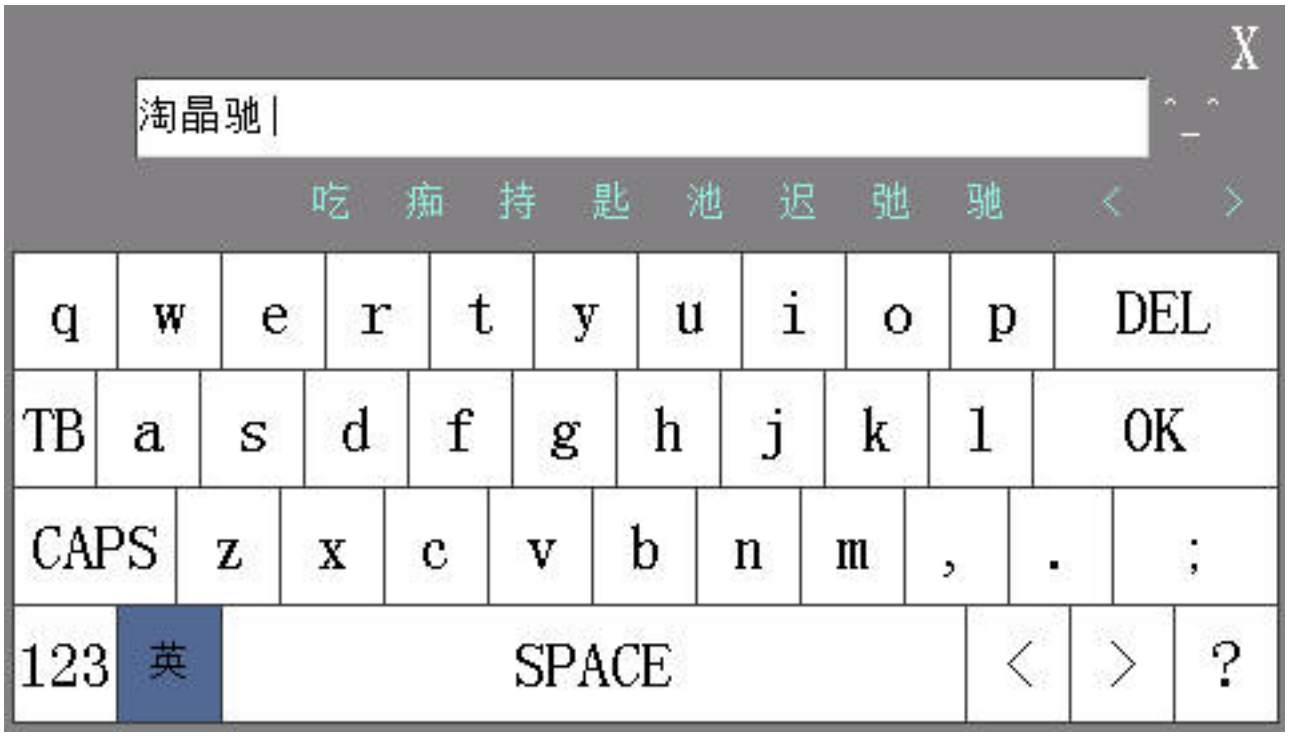
导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.25.1 滑动文本控件-使用详解

3.25.1.1 给滑动文本控件配置键盘



参考：系统键盘的调用方式 (page 909)

3.25.1.2 给滑动文本控件添加新行

```
//添加一个回车换行
slt0.txt+="\r"

slt0.txt+="abc"

//添加一个回车换行
slt0.txt+="\r"

slt0.txt+=t0.txt
```

3.25.1.3 让滑动文本控件每次都位于最新行

添加一个定时器 tm0，tim 属性设置为 1000，en 属性设置为 1

定时器中添加以下代码

```
slt0.val_y=slt0.maxval_y
```

3.25.1.4 滑动文本控件翻页

```
//滑动文本控件调用的字库字高*翻页行数
sys0=32*5
slt0.val_y+=sys0
```

3.25.1.5 如何修改滑动文本控件显示的字体大小和样式

请参考如何修改控件显示的字体 (page 1251)

3.25.2 滑动文本控件-c 语言示例

单片机通过串口给滑动文本控件赋值

如果显示的文本不全，请检查控件的 txt_maxl 属性

```
printf("slt0.txt=\"深圳市淘晶驰\r\n电子有限公司\"\xff\xff\xff");     //
↪滑动文本控件换行显示

printf("slt0.txt=\"%d\"\xff\xff\xff",a);
```

3.25.3 滑动文本控件-样例工程下载

演示工程下载链接：

《滑动文本控件》演示工程下载

3.25.4 滑动文本控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.25.5 滑动文本控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

---

type 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

id 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

objname 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

vscope 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

drag 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

aph 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

effect 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

sta 属性 -背景填充方式:0-切图;1-单色;2-图片;3-透明（仅 x 系列支持透明）。可读，可通过上位机修改，不可通过指令修改。

picc 属性 -切图背景 (必须是全屏图片)，sta 为切图时才有这个属性。可读，可通过上位机修改，可通过指令修改。

bco 属性 -背景色，sta 为单色时才有这个属性。可读，可通过上位机修改，可通过指令修改。

style 属性 -显示风格:0-平面;1-边框;2-3D_Down;3-3D_Up;4-3D_Auto，sta 为单色时才有这个属性。可读，可通过上位机修改，不可通过指令修改。

borderc 属性 -边框颜色。当 style 设置为边框时可用。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

borderw 属性边框粗细。当 style 设置为边框时可用。最大值:255。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

pic 属性 -背景图片，sta 为图片时才有这个属性。可读，可通过上位机修改，可通过指令修改。

pco 属性 -字体色。可读，可通过上位机修改，可通过指令修改。

key 属性 -绑定键盘。可读，可通过上位机修改，不可通过指令修改。

font 属性 -控件调用的字库 id，调用不同的字库会显示不同的字体或字号。可读，可通过上位机修改，可通过指令修改。参考：1、创建字库和导入字库 (page 125) 2、指定字库 (page 1284)

xcen 属性 -水平对齐:0-靠左;1-居中;2-靠右。可读，可通过上位机修改，可通过指令修改。

left 属性 -是否显进度条:0-不显示;1-操作时显示;2-持续显示。可读，可通过上位机修改，可通过指令修改。

ch 属性 -滑动惯性力度 (0-32,0 为无惯性)。可读，可通过上位机修改，可通过指令修改。

txt 属性 -字符内容。可读，可通过上位机修改，可通过指令修改。

txt_maxl 属性 -字符最大长度 (即分配内存空间)。可读，可通过上位机修改，不可通过指令修改。

isbr 属性 -是否自动换行:0-否;1-是。可读，可通过上位机修改，可通过指令修改。

spax 属性 -字符横向间距 (最小 0, 最大 255)。可读，可通过上位机修改，可通过指令修改。

spay 属性 -字符纵向间距 (最小 0, 最大 255)。可读，可通过上位机修改，可通过指令修改。

path_m 属性 -缓冲区大小 (此值的 1/2 表示支持的最大文本行数,0 为自动, 自动情况下此值等于 txt_maxl, 即 txt_maxl 的 1/2 为支持的最大行数)。可读，可通过上位机修改，不可通过指令修改。

maxval_y 属性 -最大纵向滑动值 (运行中根据字符内容自动改变, 只可读取不可设置)

val_y 属性 -当前纵向滑动值 (最小 0, 最大 maxval_y) 可读，不建议通过上位机修改，可通过指令修改。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.26 数据记录控件

数据记录控件仅 X2、X3、X5 系列支持

数据记录控件一定是全局的，无法更改为私有属性，每个工程最多使用 8 个数据记录控件，多了会出现黑屏，目前没有办法支持更多的数据记录控件。

数据记录控件类似于数据库，但是目前只有增删改三个功能，使用前需要提前导入字库，制作字库请参考创建字库和导入字库 (page 125)

数据记录是通过相关方法来添加，修改数据。而不是通过属性的 txt，val 赋值。

数据记录控件会自动创建对应的.data 文件用于记录数据，请勿自己创建文件, 如果自己提前创建了文件, 文件结构是不符合数据记录控件的文件结构, 会导致黑屏报错。

如何修改显示的字体大小：需要提前导入不同大小的字库，需要修改控件显示的字体大小时，通过上位机编辑或者通过指令修改控件的 font 属性即可，请参考如何修改控件显示的字体 (page 1251)

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

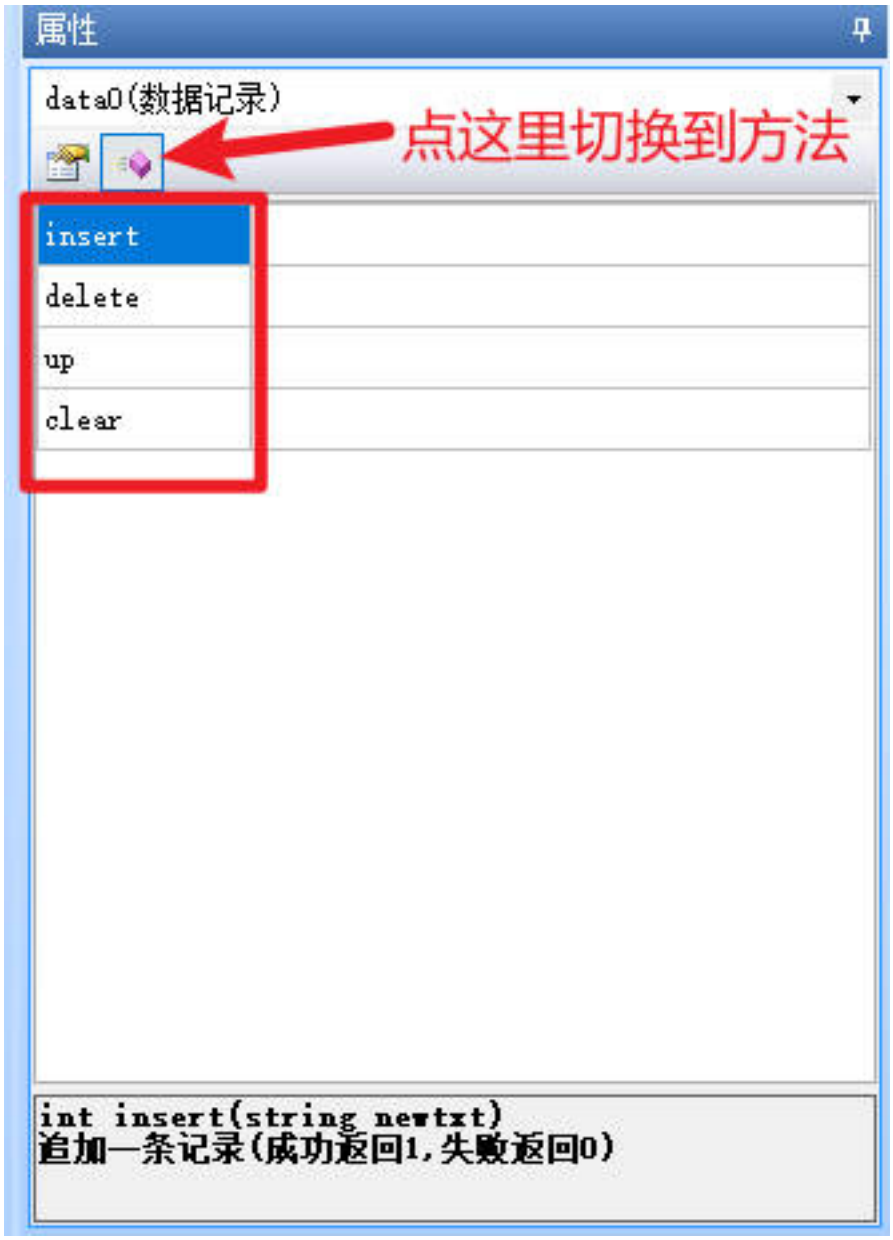
导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.26.1 数据记录控件-使用详解

3.26.1.1 数据记录控件有哪些方法



数据记录控件有 4 个方法，分别为 insert，delete，up，clear

insert： 追加一条记录 (成功返回 1, 失败返回 0)

```
data0.insert(string newtxt)

string newtxt : 新数据(多个字段使用^分割;颜色定义使用<font>标签,如<font b=1024,p=123>)
```

delete: 删除数据 (成功返回 1, 失败返回 0)

```
data0.delete(int startid, int qty)

int startid : 待删除数据起始ID

int qty : 需要删除的条数
```

up： 修改一条记录 (成功返回 1, 失败返回 0)

```
data0.up(string newtxt, int index)

string newtxt : 新数据(多个字段使用^分割;颜色定义使用<font>标签,如<font b=1024,p=123>)

int qty : 需要修改的记录ID
```

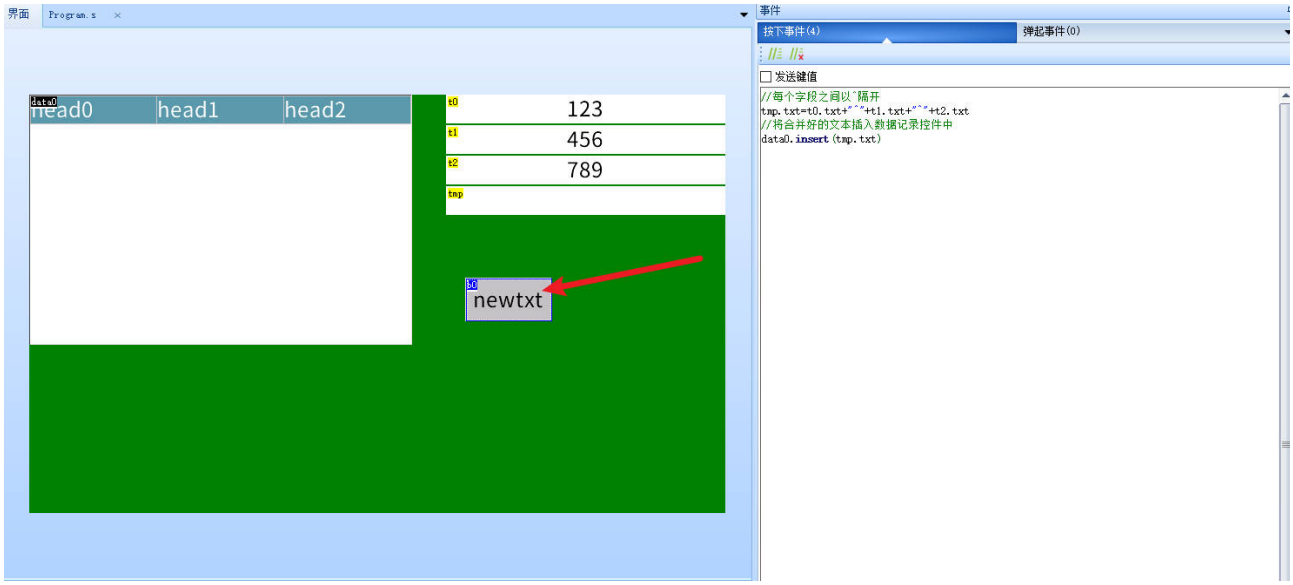
clear： 清除所有数据记录

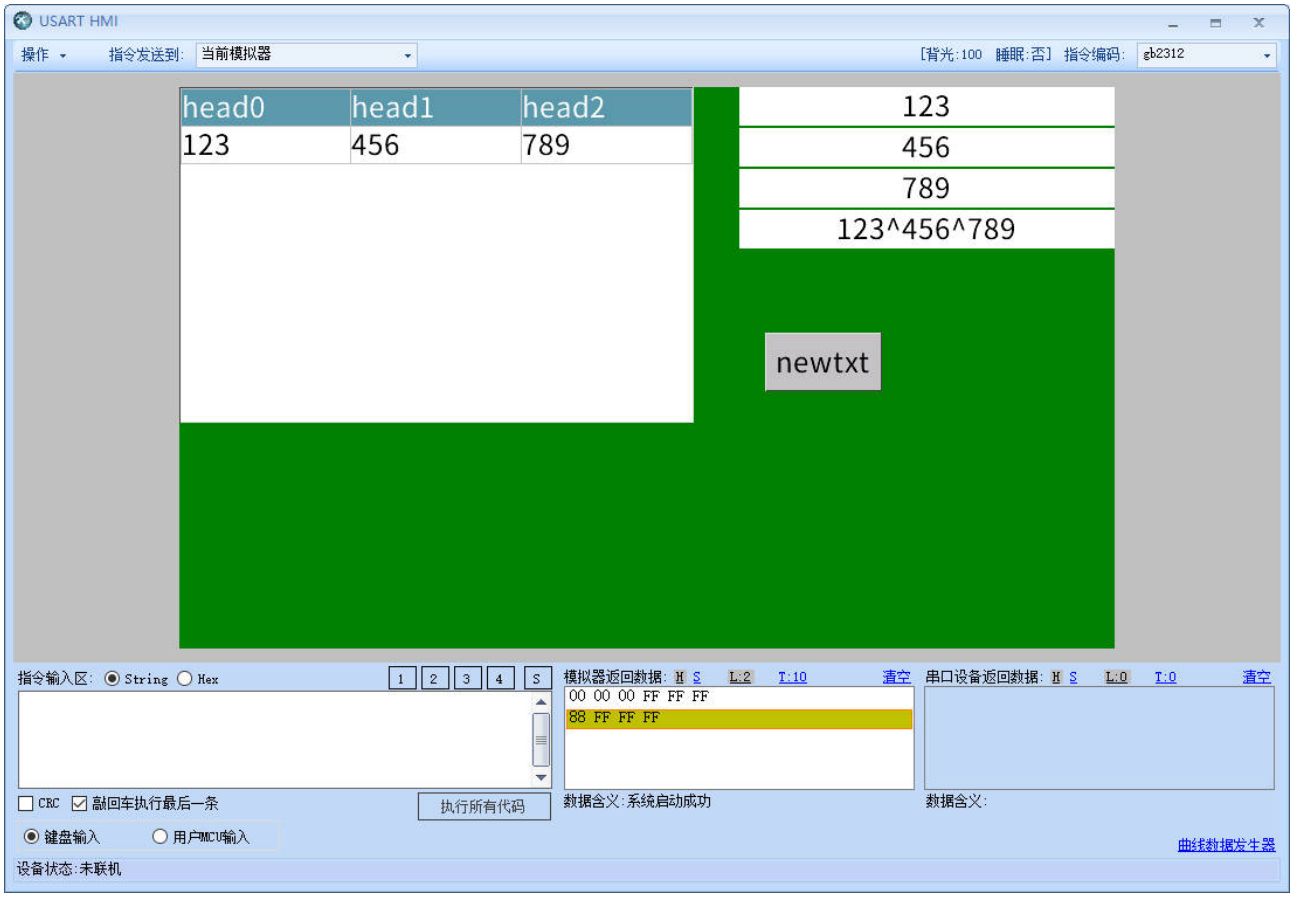
```
//不需要参数
data0.clear()
```

3.26.1.2 数据记录控件插入一条数据

假设文本控件 t0，t1，t2 分别记录了你需要插入数据记录控件的数据, 还需要创建一个文本控件 tmp 来组合这些数据，请注意将 tmp 的 txt_maxl 设置得足够大

```
//每个字段之间以^隔开
tmp.txt=t0.txt+"^"+t1.txt+"^"+t2.txt

//将合并好的文本插入数据记录控件中
data0.insert(tmp.txt)
```



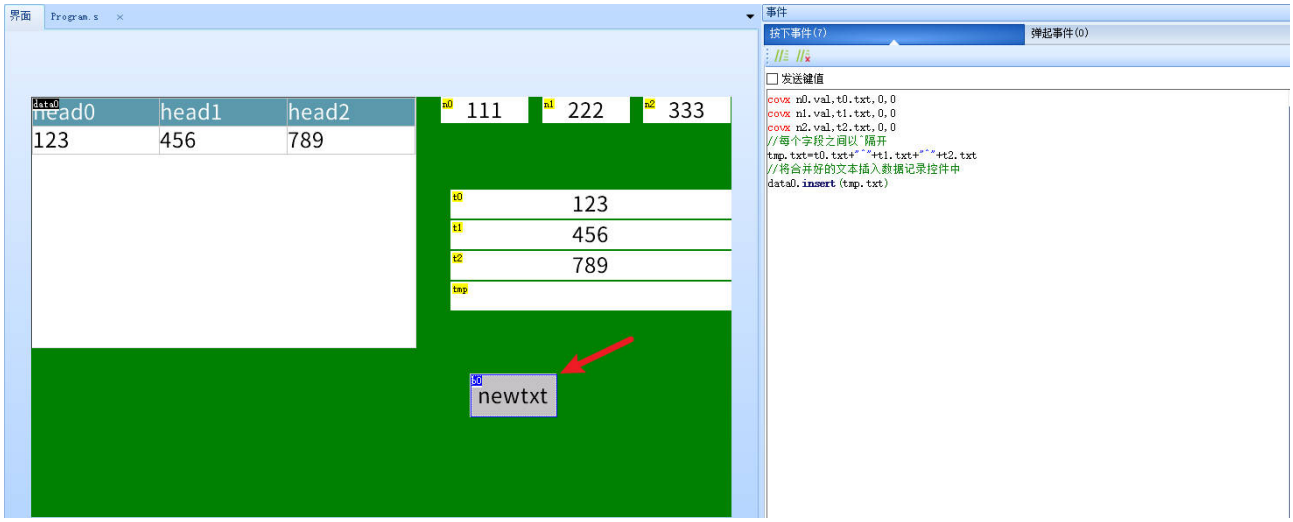


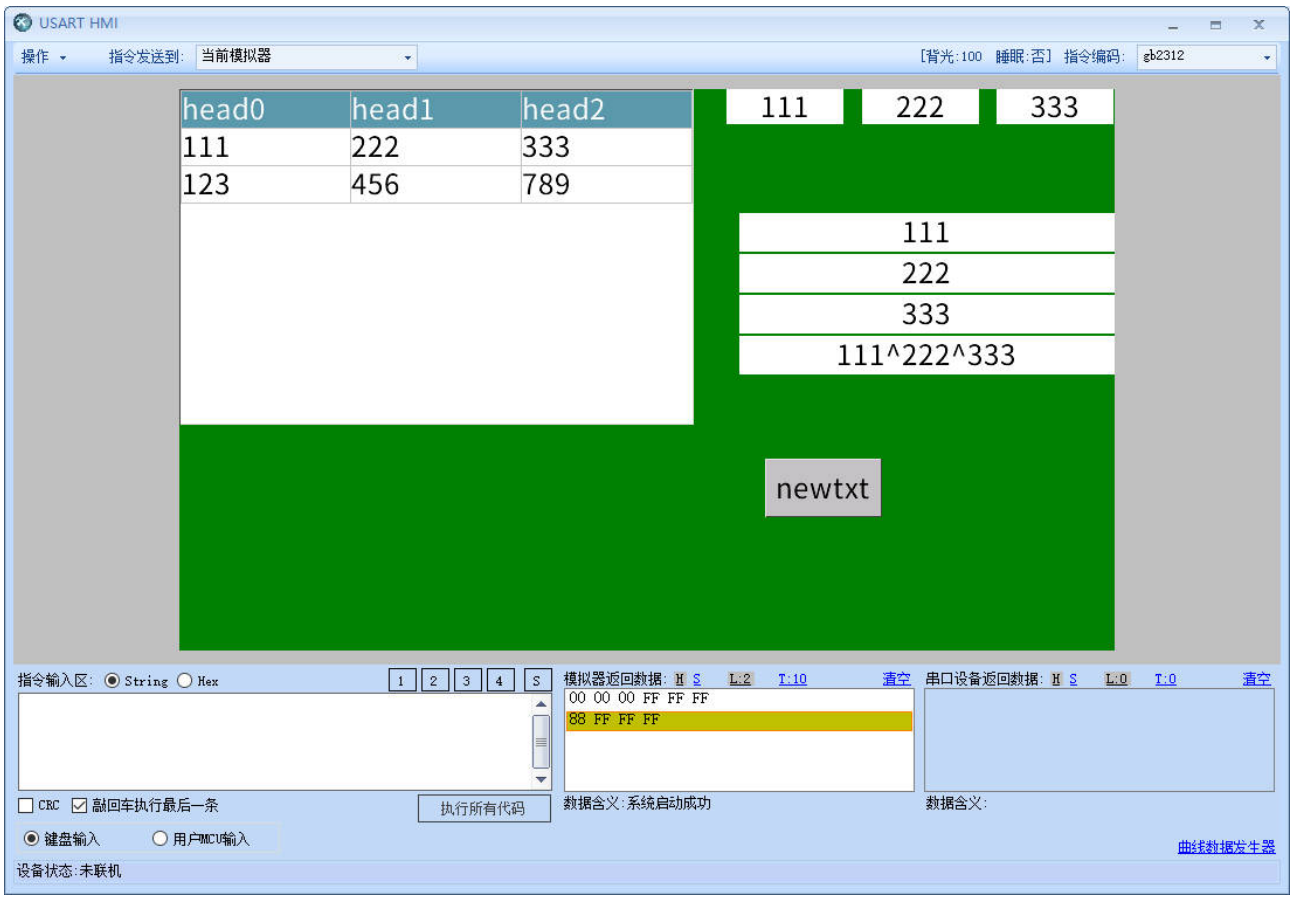
3.26.1.3 数据记录控件插入数字控件

假设数字控件 n0，n1，n2 分别记录了你需要插入数据记录控件的数据，文本控件 t0，t1，t2 用来存储转换后的数据，还需要创建一个文本控件 tmp 来组合这些数据，请注意将 tmp 的 txt_maxl 设置得足够大

用 covx 把数字控件转换为文本控件，再拼接即可

```
covx n0.val,t0.txt,0,0
covx n1.val,t1.txt,0,0
covx n2.val,t2.txt,0,0

//每个字段之间以^隔开
tmp.txt=t0.txt+"^"+t1.txt+"^"+t2.txt

//将合并好的文本插入数据记录控件中
data0.insert(tmp.txt)
```





3.26.1.4 数据记录控件插入 rtc 时间信息

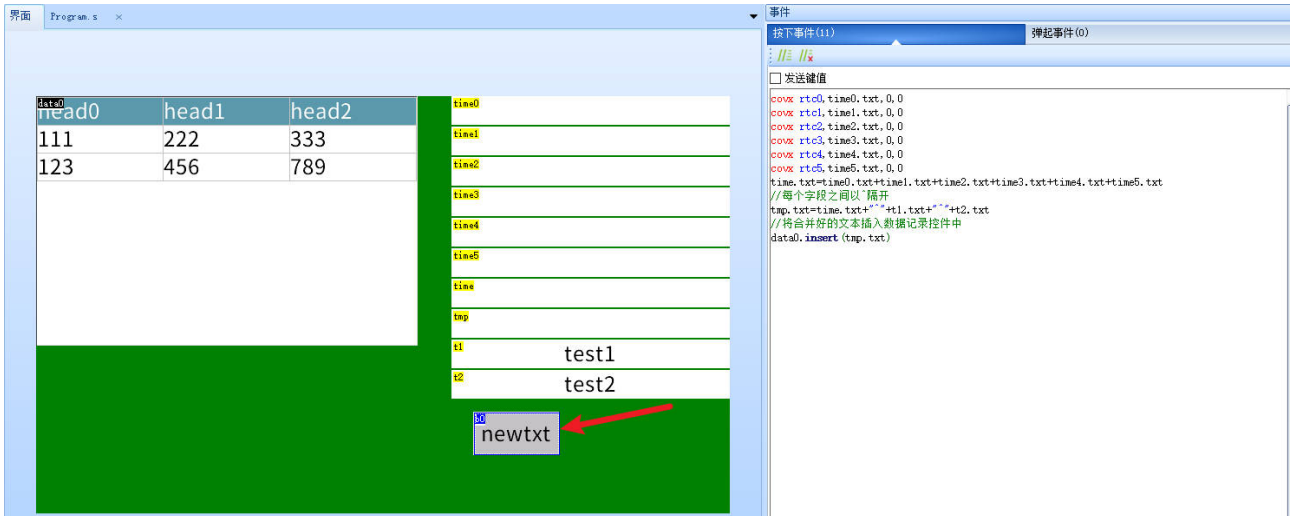
请注意将 tmp 和 time 的 txt_maxl 设置得足够大，rtc 功能仅 x5 和 k0 系列支持

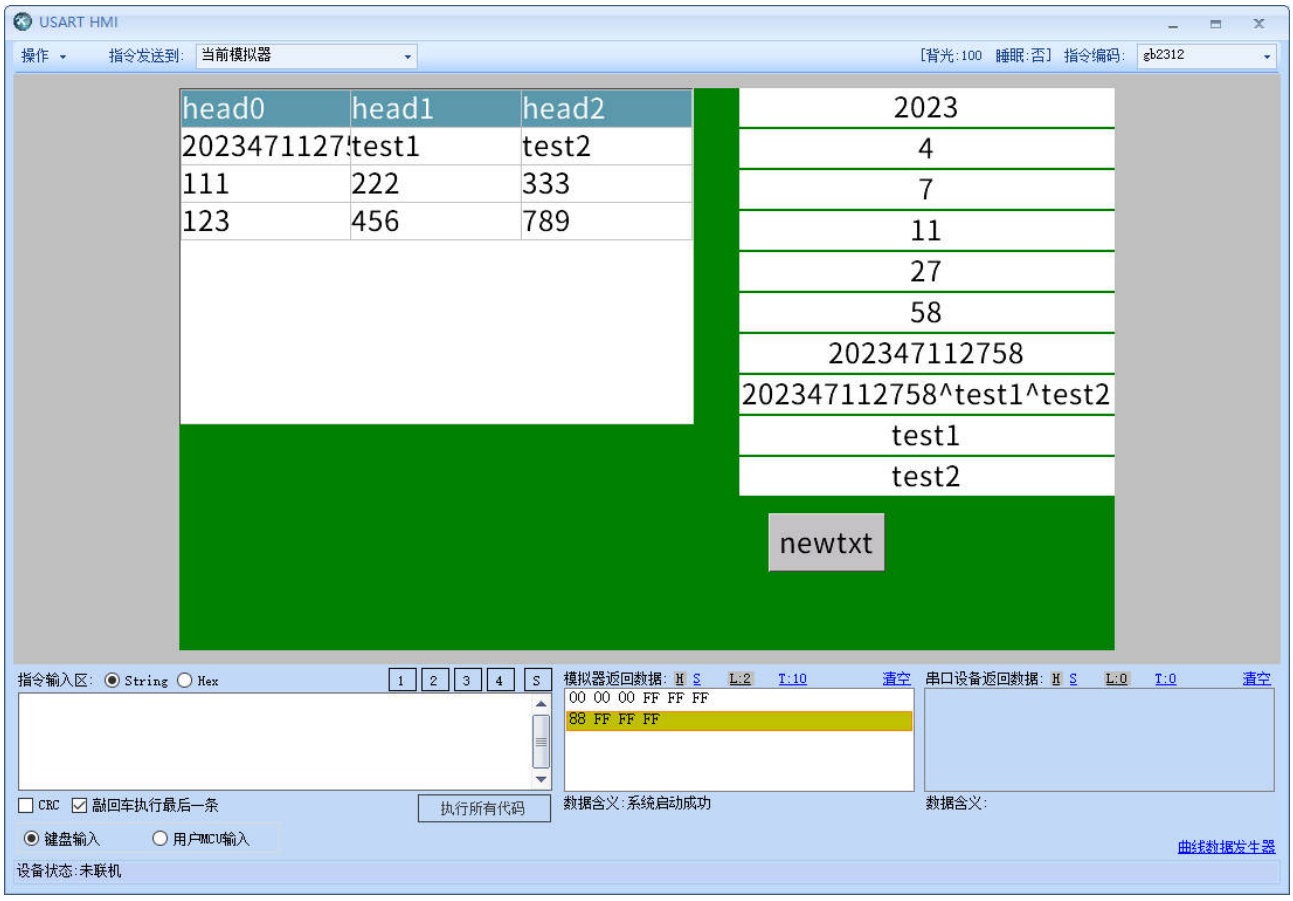
```
covx rtc0,time0.txt,0,0
covx rtc1,time1.txt,0,0
covx rtc2,time2.txt,0,0
covx rtc3,time3.txt,0,0
covx rtc4,time4.txt,0,0
covx rtc5,time5.txt,0,0
time.txt=time0.txt+time1.txt+time2.txt+time3.txt+time4.txt+time5.txt
```

(续下页)

(接上页)

```
//每个字段之间以^隔开
tmp.txt=time.txt+"^"+t1.txt+"^"+t2.txt

//将合并好的文本插入数据记录控件中
data0.insert(tmp.txt)
```





3.26.1.5 数据记录控件删除选中的数据

当配置了数据记录控件的 dis 属性（是否允许触摸选中记录项）时，数据记录控件的 val 属性将会变为选中项所在的行数

```
//删除被选中的行
data0.delete(data0.val,1)
```

3.26.1.6 数据记录控件清空数据

```
//清空数据记录控件
data0.clear()
```

3.26.1.7 数据记录控件更新被选中的数据

```
//每个字段之间以^隔开
tmp.txt=t0.txt+"^"+t1.txt+"^"+t2.txt

//更新被选中的数据
data0.up(tmp.txt,data0.val)
```

3.26.1.8 数据记录控件修改某一行的背景色和字体色

```
//将背景颜色设置为1024（绿色），将字体颜色设置为63488（红色），添加内容为1 2 3
data0.insert("<font b=1024,p=63488>1^2^3")

data0.up("<font b=1024,p=63488>1^2^3",0)
```

```
//（将n1设置的颜色改变选中表格）
covx n1.val,va0.txt,0,0
//请注意va2的长度以及数据记录控件的lenth属性是否足够大
va2.txt="<font b="+va0.txt+">"+t0.txt
//（将原来的内容添加上背景颜色为n1.val）
data0.up(va2.txt,n0.val)
```

《数据记录控件彩色》演示工程下载

3.26.1.9 数据记录翻页功能实现

```
sys0=n*data0.hig  //翻页n行
data0.val_y+=sys0
```

hig: 显示记录的行高度

n: 翻页的行数，n 为正数向后翻页，n 为负数向前翻页

具体例子：向前翻页 5 行

```
sys0=-5*data0.hig
data0.val_y+=sys0
```

具体例子：向后翻页 5 行

```
sys0=5*data0.hig
data0.val_y+=sys0
```

参考工程下载地址

《数据记录控件实现翻页》演示工程下载

3.26.1.10 数据记录控件回到顶部, 回到底部, 回到中间

```
//回到顶部
data0.val_y=0

//回到底部
data0.val_y=data0.maxval_y

//回到中间
data0.val_y=data0.maxval_y/2
```

3.26.1.11 获取数据记录某一行中某一个格

获取数据记录某一行（点击相应数据记录中的记录或者对属性 val 赋值）中某一个格使用到的是 spstr 指令获取

参考：*spstr-字符串分割* (page 616)

3.26.1.12 数据记录超过 12 列

数据记录控件-样例工程下载 (page 521)

3.26.1.13 数据记录控件由 3 列修改为 4 列或更多列

需要修改数据记录控件的 dez 属性，只能在上位机中修改，不能通过指令修改，修改后数据记录控件会显示黑屏和报错信息

此时你需要到 SD 卡或者虚拟 SD 卡的目录下删除原有的.data 文件，重新调试运行后，控件会自动生成新的.data 文件

修改了 length 属性或者 maxval 属性导致的黑屏也是同样的操作进行解决

3.26.1.14 如何修改数据记录控件显示的字体大小和样式

请参考如何修改控件显示的字体 (page 1251)

3.26.1.15 隐藏数据记录控件表头

删除数据记录控件的 dir 属性里的值即可

3.26.2 数据记录控件-常见问题

3.26.2.1 数据记录控件变黑报错，黑屏

3.26.2.1.1 File configuration data does not match Component configuration data. It is recommended to delete this file. The system will recreate the correct data file:sd0/ xxx.data

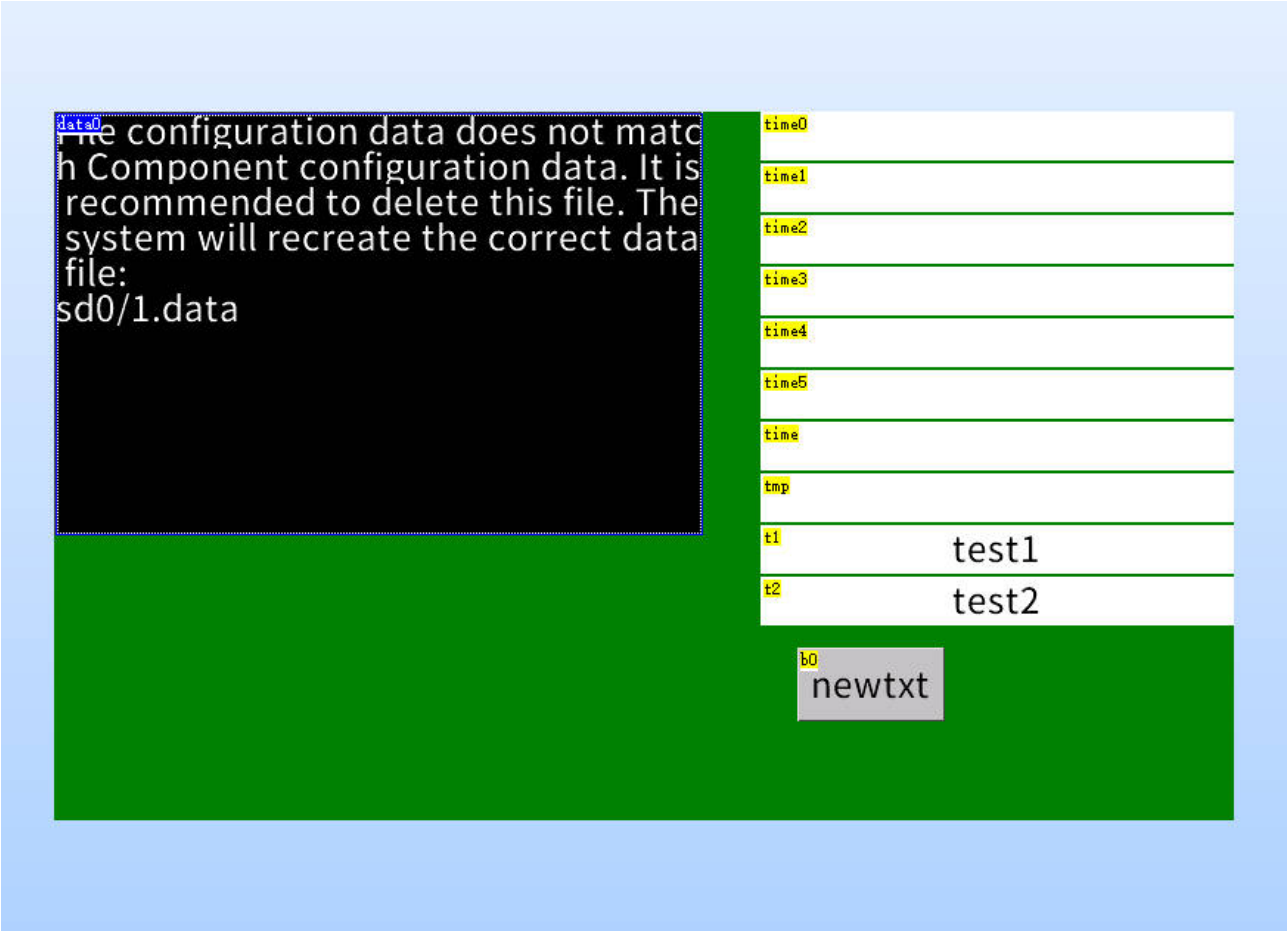
lenth	63
maxval	1000
dez	3

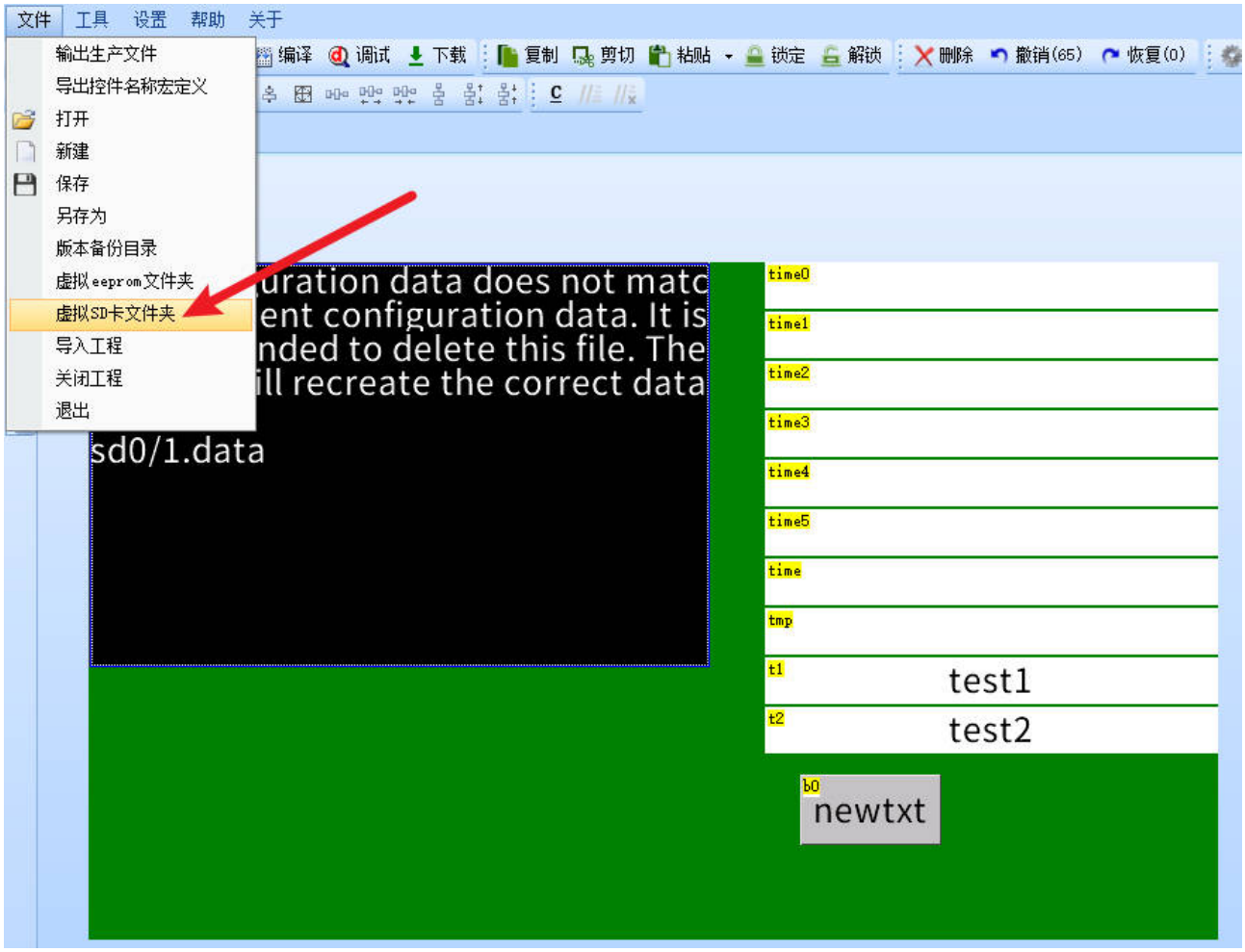
因为记录的字段和所指定的.data 文件中的字段数量不符导致的，有多种解决方法

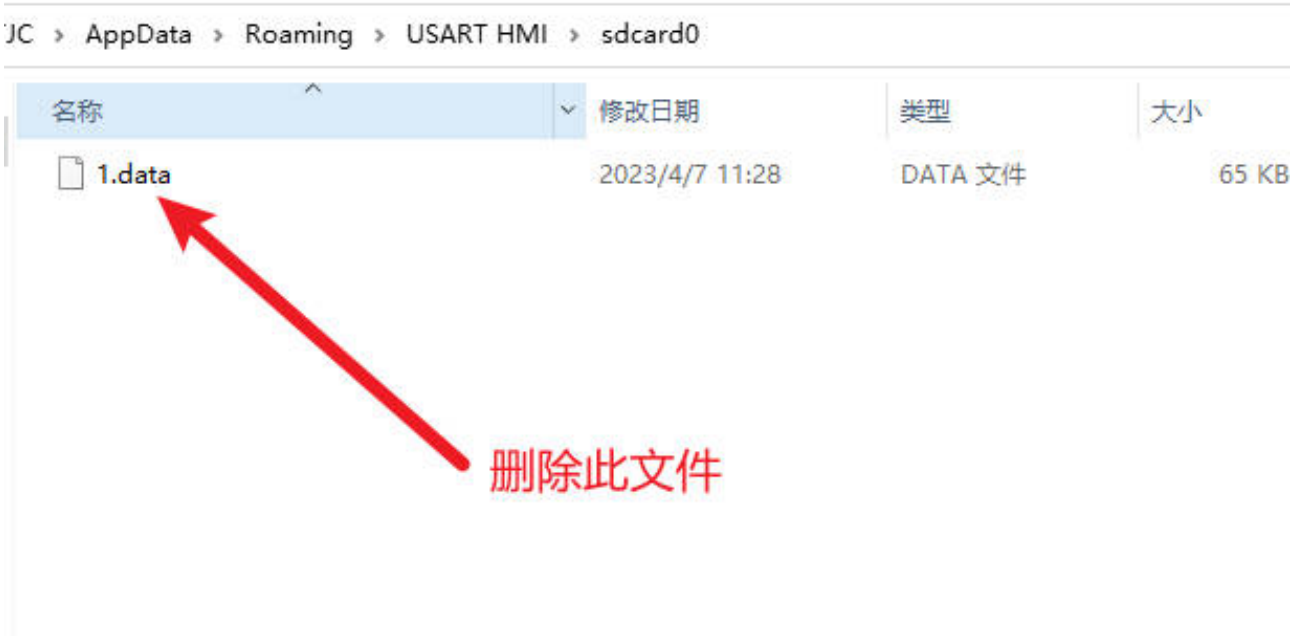
1. 你改动了数据记录控件的 length、maxval、dez 属性，将其恢复即可

2. 把存储卡或者虚拟 sd 卡文件夹中的原本绑定的对应的的.data 文件删除

3. 修改数据记录控件的 path 属性，将其指向一个不存在的文件（将会自动创建），例如原本是 sd0/1.data，改为 sd0/123.data




文件
工具
设置
帮助
关于
输出生产文件
导出控件名称宏定义
打开
新建
保存
另存为
版本备份目录
虚拟eeprom文件夹
虚拟SD卡文件夹
导入工程
关闭工程
退出
编译
调试
下载
复制
剪切
粘贴
锁定
解锁
删除
撤销(65)
恢复(0)
uration data does not matc
ent configuration data. It is
nded to delete this file. The
ill recreate the correct data
sd0/1.data
time0
time1
time2
time3
time4
time5
time
tmp
t1
test1
t2
test2
b0
newtxt


JC › AppData › Roaming › USART HMI › sdcard0
名称
修改日期
类型
大小
1.data
2023/4/7 11:28
DATA 文件
65 KB
删除此文件

3.26.2.1.2 提示 file lost

没插 micro sd(tf) 卡

卡的格式不对（非 fat32 格式）

尝试换一张卡

data 文件不是放在根目录，而是放在某个文件夹下，但是这个文件夹不存在，导致创建 data 文件失败，在代码中应该配合 finddir 和 newdir 来查找和创建文件夹，以保证文件夹存在。

3.26.2.1.3 多个数据记录打开同一个路径下文件导致黑屏

这是因为文件已经被上一个数据记录打开导致新的数据记录无法打开文件。

举例:

page0,page1,page2 各有一个数据记录控件，都同时调用 sd0/1.data ，此时只有 page0 页面是正常的，其他页面的数据记录都会黑屏

我们可以通过以下方法解决

即每次跳转时，将当前页面的数据记录控件指向一个临时的文件，从而解除目标文件的占用状态，再将即将跳转到的页面的数据记录控件指向目标文件

当需要从 page0 跳转到 page1 时，在对应的控件中写下

```
page0.data0.path="sd0/tmp0.data"
page1.data0.path="sd0/1.data"
page page1
```

当需要从 page1 跳转到 page2 时，在对应的控件中写下

```
page1.data0.path="sd0/tmp1.data"
page2.data0.path="sd0/1.data"
page page2
```

当需要从 page2 跳转到 page0 时，在对应的控件中写下

```
page2.data0.path="sd0/tmp2.data"
page0.data0.path="sd0/1.data"
page page0
```

3.26.2.1.4 数据记录 path 属性赋值注意事项

不要拼接时直接赋值给数据记录 path 属性, 要通过一个中间变量拼接好后整体赋值给数据记录 path 属性

```
page1.data0.path="sd0/"+t1.txt+".data" //错误方式
```

```
t0.txt="sd0/"+t1.txt+".data" //正确方式
page1.data0.path=t0.txt
```

3.26.2.1.5 数据记录超过 8 个的解决方法

《数据记录超过 8 个的解决方法》演示工程下载

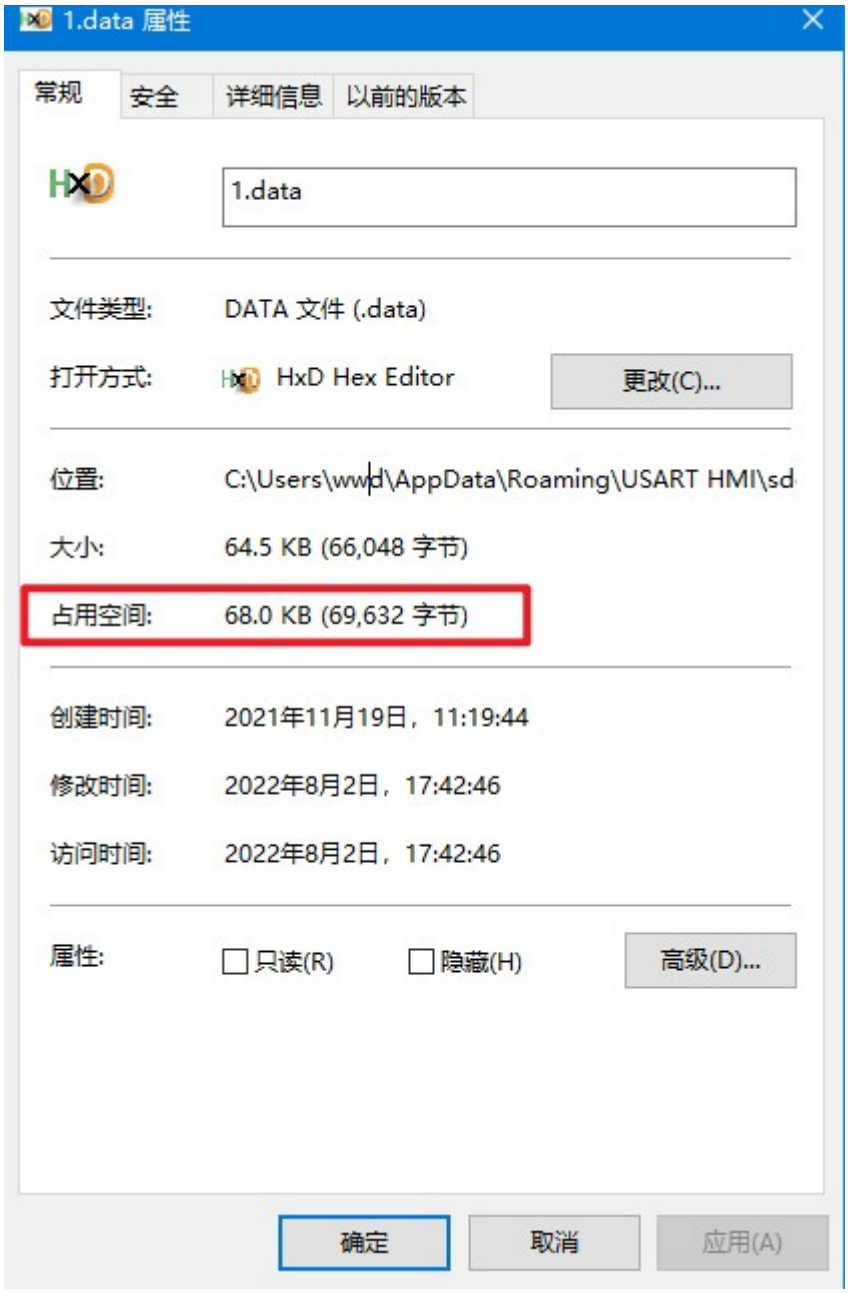
3.26.2.2 将数据记录控件保存在内存中

不需要 sd 卡，但是重启后数据会消失

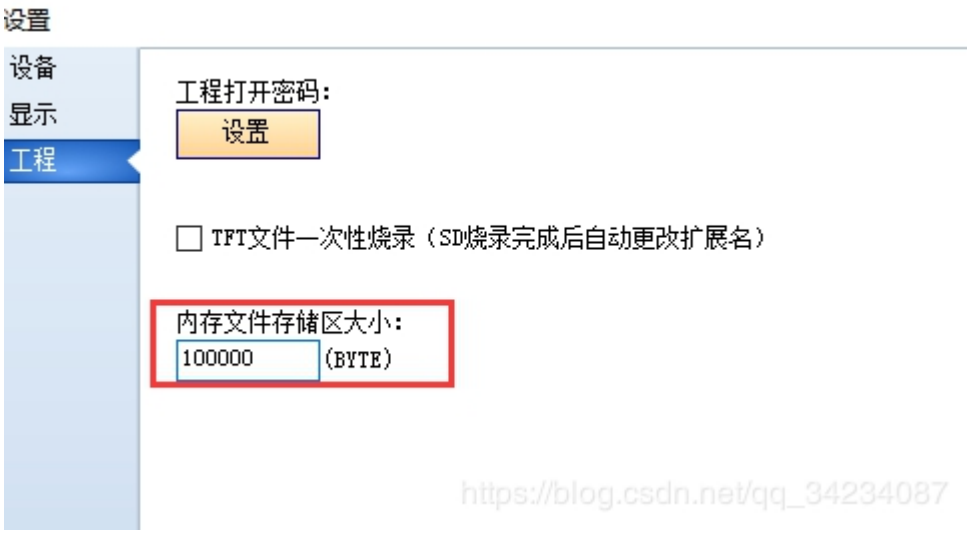
先打开虚拟 SD 卡文件夹，查看默认设置下生成的.data 文件有多大

例如当前绑定的数据记录文件为“sd0/1.data”

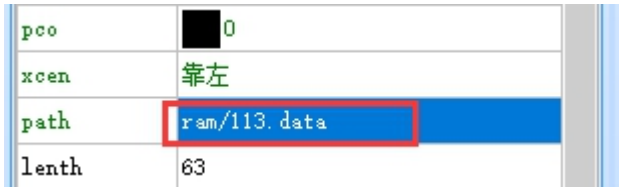
则查看“1.data”文件的体积



设备-工程-内存文件存储区大小设置为 100000Byte(理论上稍大于文件占用空间 69632Byte 几个 KB 就行了，取 100000Byte 是因为方便)，这个值只建议比生成的文件刚好大一点点，如果设置得过大，会占用过多内存，导致使用其他控件时出现问题



将数据记录的位置指定到 ram 中



3.26.3 数据记录控件-样例工程下载

演示工程下载链接：

《使用滑块滑动数据记录》演示工程下载

《数据记录超过 8 个的解决方法》演示工程下载

《数据记录控件彩色》演示工程下载

《数据记录控件超过 12 个字段解决方案》演示工程下载

《数据记录控件打开列数不同的文件》演示工程下载

《数据记录控件获取选中的行》演示工程下载

《数据记录控件实现翻页》演示工程下载

《数据记录例程 1》演示工程下载

《数据记录例程 2》演示工程下载

《数据记录例程 3》演示工程下载

《文件搜索》演示工程下载

3.26.4 数据记录控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

数据记录文件结构分析详解 (page 1047)

3.26.5 数据记录控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

---

type 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

id 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

objname 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

vscope 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

数据记录控件的 vscope 只能是全局

drag 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

aph 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

effect 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

sta 属性 -背景填充方式:0-切图;1-单色;2-图片;3-透明（仅 x 系列支持透明）。可读，可通过上位机修改，不可通过指令修改。

picc 属性 -切图背景 (必须是全屏图片)，sta 为切图时才有这个属性。可读，可通过上位机修改，可通过指令修改。

bco 属性 -背景色，sta 为单色时才有这个属性。可读，可通过上位机修改，可通过指令修改。

pic 属性 -背景图片，sta 为图片时才有这个属性。可读，可通过上位机修改，可通过指令修改。

pco 属性 -字体色。可读，可通过上位机修改，可通过指令修改。

borderc 属性 -边框颜色。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

borderw 属性边框粗细。最大值:255。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

font 属性 -控件调用的字库 id，调用不同的字库会显示不同的字体或字号。可读，可通过上位机修改，可通过指令修改。参考：1、创建字库和导入字库 (page 125) 2、指定字库 (page 1284)

xcen 属性 -水平对齐:0-靠左;1-居中;2-靠右。可读，可通过上位机修改，可通过指令修改。

path 属性 -绑定数据文件路径 (如:”sd0/1.data”)。可读，可通过上位机修改，可通过指令修改。

lenth 属性 -当前绑定数据记录文件中每条记录最大字节数 (最大长度 255 字节)。可读，可通过上位机修改，不可通过指令修改。通过上位机修改需删除原有的.data 文件，否则会黑屏。

maxval 属性 -当前绑定数据记录文件可存入的最大行数, 超出之后的新增记录将循环覆盖老数据。可读，可通过上位机修改，不可通过指令修改。通过上位机修改需删除原有的.data 文件，否则会黑屏。

dez 属性 -当前绑定数据记录文件中设置的字段数量 (最小 1, 最大 12)。可读，可通过上位机修改，不可通过指令修改。通过上位机修改需删除原有的.data 文件，否则会黑屏。

format 属性 -字段宽度自定义 (直接输入字段宽度值, 多个字段使用 ^ 分隔, 如:100^100)。可读，可通过上位机修改，可通过指令修改。

dir 属性 -表头名称自定义 (多个字段使用 ^ 分隔)。可读，可通过上位机修改，可通过指令修改。值为空时会隐藏表头。

mode 属性 -是否允许自动创建文件 (path 路径无效时):0-不允许;1-允许。可读，可通过上位机修改，可通过指令修改。

dis 属性 -是否允许触摸选中记录项:0-不允许;1-允许。可读，可通过上位机修改，可通过指令修改。

order 属性 -显示顺序:0-新数据在前;1-新数据在后。可读，可通过上位机修改，可通过指令修改。

qty 属性 -当前数据文件总记录数。可读，不可通过上位机修改，不可通过指令修改。

spax 属性 -字符横向间距 (最小 0, 最大 255)。可读，可通过上位机修改，可通过指令修改。

hig 属性 -显示记录的行高度 (最小 1, 最大 255)。可读，可通过上位机修改，可通过指令修改。

left 属性 -是否显进度条:0-不显示;1-操作时显示;2-持续显示。可读，可通过上位机修改，可通过指令修改。

gdc 属性 -表格线颜色。可读，可通过上位机修改，可通过指令修改。

gdw 属性 -横向表格线宽度 (0 为关闭)。可读，可通过上位机修改，可通过指令修改。

gdh 属性 -纵向表格线宽度 (0 为关闭)。可读，可通过上位机修改，可通过指令修改。

bco1 属性 -表头背景色。可读，可通过上位机修改，可通过指令修改。

pco1 属性 -表头字体色。可读，可通过上位机修改，可通过指令修改。

bco2 属性 -选中项背景色。可读，可通过上位机修改，可通过指令修改。

pco2 属性 -选中项字体色。可读，可通过上位机修改，可通过指令修改。

val 属性 -当前选中行记录 ID(每变化一次,txt 属性将重新加载此 ID 的行记录内容)。可读，可通过上位机修改，可通过指令修改。

txt 属性 -行记录内容 (val 值代表的行号记录内容，只可读取不可修改)

ch 属性 -滑动惯性力度 (0-32,0 为无惯性)。可读，可通过上位机修改，可通过指令修改。

maxval_y 属性 -最大纵向滑动值 (运行中根据字符内容自动改变, 只可读取不可设置)

val_y 属性 -当前纵向滑动值 (最小 0, 最大 maxval_y)。可读，可通过上位机修改，可通过指令修改。

maxval_x 属性 -最大横向滑动值 (运行中根据字符内容自动改变, 只可读取不可设置)

val_x 属性 -当前横向滑动值 (最小 0, 最大 maxval_x)。可读，可通过上位机修改，可通过指令修改。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.27 文件浏览器控件

文件浏览器控件仅 X2、X3、X5 系列支持

文件浏览器控件用于显示 SD 卡中的文件，仅 X 系列支持，使用前需要提前导入字库（UTF8 字库），制作字库请参考创建字库和导入字库 (page 125)

不建议将文件浏览器控件设置为全局，某些参数无法跨页面设置或者获取，建议仅在需要的页面放一个文件浏览器控件用于获取 SD 卡中的文件和目录信息

如何修改显示的字体大小：需要提前导入不同大小的字库，需要修改控件显示的字体大小时，通过上位机编辑或者通过指令修改控件的 font 属性即可，请参考如何修改控件显示的字体 (page 1251)

注意： 使用 delfile 删除文件后，如果当前页面使用了文件浏览器控件，需要重新 page 到当前页面以重新刷新文件浏览器控件

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.27.1 文件浏览器控件-使用详解

3.27.1.1 判断当前被选中文件的类型

```
spstr fbrowser0.txt,t0.txt,".",1
if(t0.txt=="jpg")
{
    //这是一张jpg图片
}else if(t0.txt=="video")
{
    //这是一个video视频
}else if(t0.txt=="wav")
{
    //这是一首音乐
}else
{
    //这是其他类型
}
```

3.27.1.2 获取当前被选中文件的完整路径

请注意 t1.txt 的 txt_maxl 应配置得足够大，建议配置为 200，否则可能因为长度不够导致路径不完整

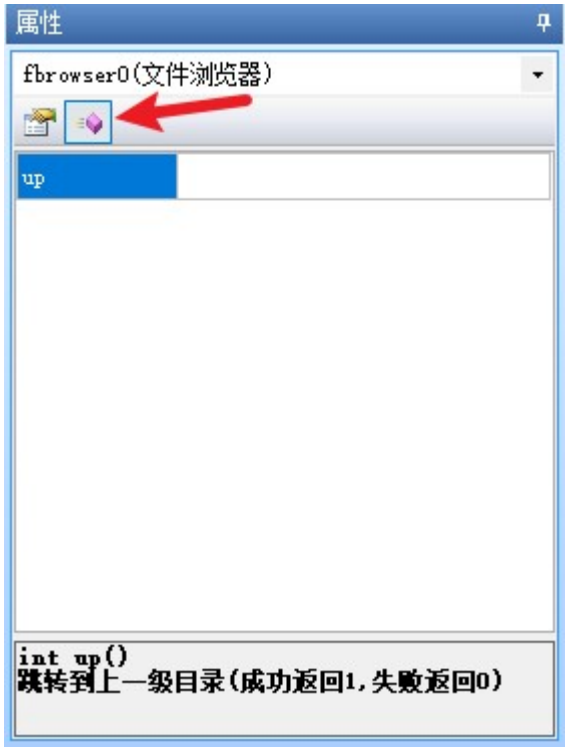
```
t1.txt=fbrowser0.dir+fbrowser0.txt  //获取文件的路径和名称
```

3.27.1.3 如何修改文件浏览器控件显示的字体大小和样式

请参考如何修改控件显示的字体 (page 1251)

3.27.1.4 文件浏览器控件有哪些方法

文件浏览器所有的方法可以点击这里查看



文件浏览器控件有 1 个方法 up

up：跳转到上一级目录 (成功返回 1, 失败返回 0)

使用方法：

新建一个按钮控件，txt 属性改为返回，在弹起事件中编写以下代码

```
fbrowser0.up()
```

注意： 当文件浏览器处于根目录时，便无法返回上一级目录

3.27.2 文件浏览器控件-常见问题

3.27.2.1 1. 文件浏览器中文件名称显示不正常（乱码）

文件名只建议包含数字和英文，不建议包含其他字符，比如中文日文韩文等，因为文件浏览器需要使用 UTF8 编码，且目前输入法不支持 UTF8 拼音输入，因此建议大家使用数字和英文来命名文件夹

3.27.3 文件浏览器控件-样例工程下载

《文件浏览器 v3.0》演示工程下载

《文件浏览器低分辨率 v2.0》演示工程下载

《获取目录下的所有文件》演示工程下载

《文件搜索》演示工程下载

3.27.4 文件浏览器控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.27.5 文件浏览器控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

`vscope` 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

`drag` 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`aph` 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`effect` 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

`sta` 属性 -背景填充方式:0-切图;1-单色;2-图片;3-透明（仅 x 系列支持透明）。可读，可通过上位机修改，不可通过指令修改。

`style` 属性 -显示风格:0-平面;1-边框;2-3D_Down;3-3D_Up;4-3D_Auto，sta 为单色时才有这个属性。可读，可通过上位机修改，不可通过指令修改。

`borderc` 属性 -边框颜色。当 style 设置为边框时可用。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

borderw 属性边框粗细。当 style 设置为边框时可用。最大值:255。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

picc 属性 -切图背景 (必须是全屏图片)，sta 为切图时才有这个属性。可读，可通过上位机修改，可通过指令修改。

bco 属性 -背景色，sta 为单色时才有这个属性。可读，可通过上位机修改，可通过指令修改。

pic 属性 -背景图片，sta 为图片时才有这个属性。可读，可通过上位机修改，可通过指令修改。

font 属性 -控件调用的字库 id，调用不同的字库会显示不同的字体或字号。可读，可通过上位机修改，可通过指令修改。参考：1、创建字库和导入字库 (page 125) 2、指定字库 (page 1284)

pco 属性 -字体色。可读，可通过上位机修改，可通过指令修改。

bco2 属性 -选中项背景色，sta 为单色时才有这个属性。可读，可通过上位机修改，可通过指令修改。

pco2 属性 -选中项字体色。可读，可通过上位机修改，可通过指令修改。

left 属性 -是否显进度条:0-不显示;1-操作时显示;2-持续显示。

ch 属性 -滑动惯性力度 (0-32,0 为无惯性)

dir 属性 -需要遍历的目录路径 (每设置一次将重新刷新一次文件列表)

filter 属性 -文件名关键字过滤 (多个关键字使用 | 分隔, 如:”*.txt|*.data”)

val 属性 -当前选中项序号 (每改变一次,txt 值将重新赋值为新的 val 值指向的文件名)

txt 属性 -当前 val 值指向的文件名 (自动随 val 值改变, 只可获取不可设置), 请勿跨页面获取 (无法跨页面获取)

qty 属性 -遍历结果中的文件总数

dis 属性 -文件名显示行数

spax 属性 -文件名显示横向间距 (最小 0, 最大 255)

spay 属性 -文件名显示纵向间距

maxval_y 属性 -最大纵向滑动值 (运行中根据字符内容自动改变, 只可读取不可设置)

val_y 属性 -当前纵向滑动值 (最小 0, 最大 maxval_y)

psta 属性 -是否支持子文件夹:0-否;1-是

pic1 属性 -文件夹图标

pic2 属性 -默认文件图标

vvs2 属性 -自定义文件图标 (类型/图片 ID, 如:”*.txt/2*.data/3”)

buffsize 属性 -储存遍历结果所使用的缓冲区大小 (用于储存遍历出来的所有文件名, 推荐 32000)

fwarning 属性 -编译时是否提示非 utf-8 编码警告 (此属性仅用于编译配置, 运行中不可访问不可读取):0-不提示;1-提示

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.27.6 文件浏览器控件-方法详解

`up` 方法 - int up() 跳转到上一级目录 (成功返回 1, 失败返回 0)

3.28 文件流控件

提示： 视频教程：文件流控件使用教程一

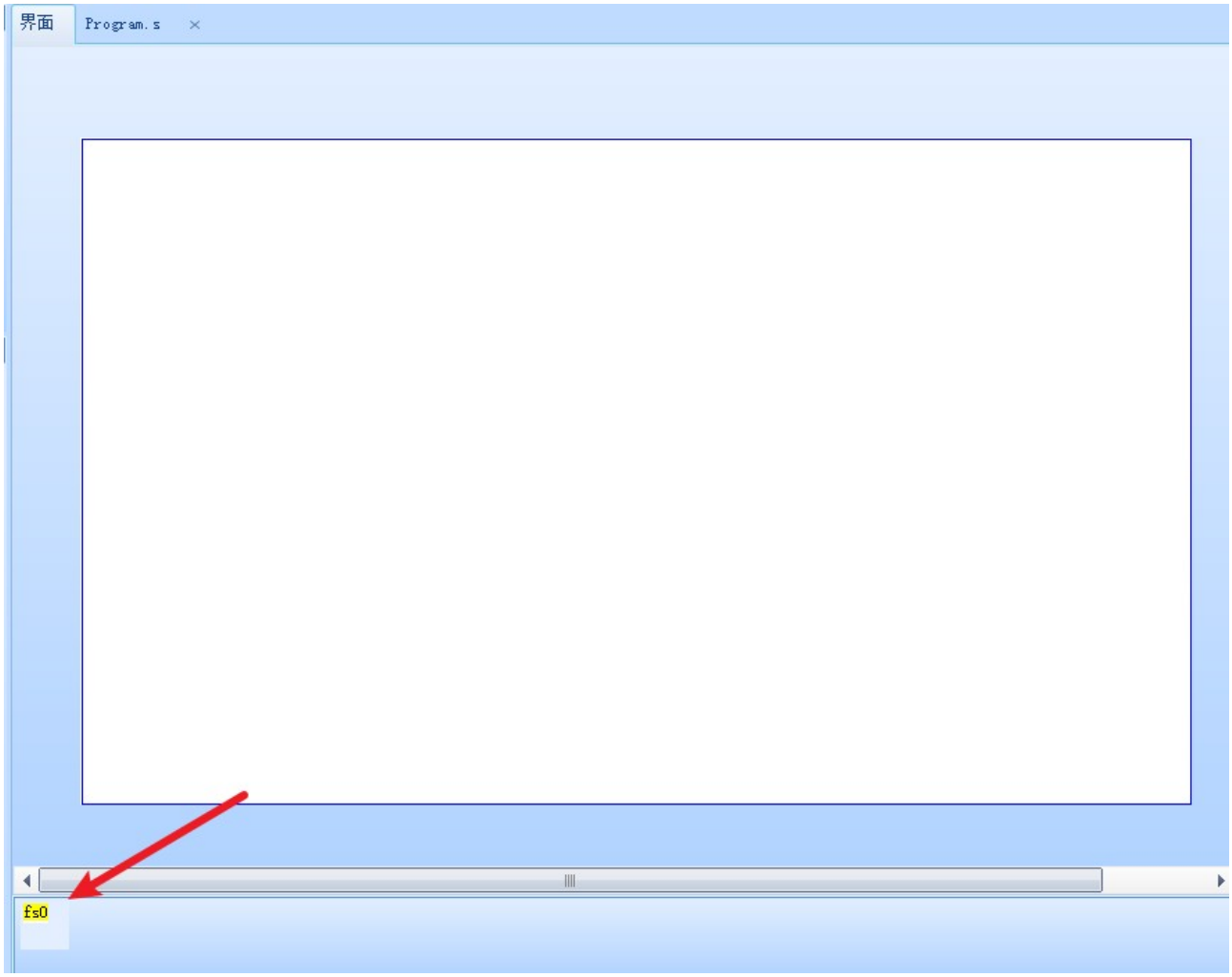
提示： 视频教程：文件流控件使用教程二

提示： 视频教程：文件流控件使用教程三

文件流控件仅 X2、X3、X5 系列支持

文件流控件用于在串口屏上对 SD 卡中的文件进行读写操作，仅 X 系列支持，文件流控件只能设置为私有

文件流控件位于特殊控件栏上



提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.28.1 文件流控件-使用详解

请先查看*SD 卡读写文件流程* (page 1024)

3.28.1.1 文件流关键属性

3.28.1.1.1 val-文件流当前数据指针

1、每次使用 open 方法打开一个文件时，文件流的 val 数据指针会默认变为 0

2、write 方法会基于当前的 val 数据指针的位置写入数据，写入成功后会自动移动 val 数据指针的位置

3、read 方法会基于当前的 val 数据指针的位置读取数据，读取成功后会自动移动 val 数据指针的位置

4、find 方法查找成功后，会自动移动指针到关键字中第一个字符所在的位置

5、find 方法可以查找汉字，如果查找失败，注意检查工程的编码与文件的编码是否一致

3.28.1.1.2 qty-文件大小

例-获取当前文件的大小：

```
n0.val=fs0.qty
```

3.28.1.1.3 en-文件打开状态

例-获取当前文件打开状态，如果是打开的就关闭：

```
if(fs0.en==1)
{
    fs0.close()
}
```

3.28.1.2 文件流控件有哪些方法

文件流控件有 5 个方法，分别为 open，read，write，close，find

open：打开文件 (成功返回 1, 失败返回 0)

read：从当前流读数据 (成功返回 1, 失败返回 0, 从当前数据指针 [val 属性] 位置开始读, 读完以后指针将自动移动相应的长度)

write：将数据写入当前流 (成功返回 1, 失败返回 0, 从当前数据指针 [val 属性] 位置开始写, 写完以后指针将自动移动相应的长度)

close：关闭文件流 (成功返回 1, 失败返回 0, 文件打开读写操作完成后一定要记得关闭文件, 同一个文件在打开后, 关闭之前, 是不能被另外一个文件流控件打开的)

find：按关键字查询并定位文件流指针 (查询成功返回 1, 失败返回 0, 从当前流的当前数据指针 [val 属性] 位置开始查询关键字, 如果查询成功, 数据指针将会移动到关键字中第一个字符处; 如果查询失败保持数据当前指针不变)

3.28.1.2.1 open-打开文件

int open(string path)

成功返回 1, 失败返回 0

path 文件路径如 “sd0/aa.txt”

例：

```
fs0.open("sd0/aa.txt")   //打开sd0下aa.txt
```

因为有返回值，所以可以用一个变量来判断是否打开成功

```
sys0=fs0.open("sd0/aa.txt")
if(sys0==1)
{
    //文件打开成功
}
```

注意： 1、文件打开读写操作完成后一定要记得关闭文件，同一个文件在打开后，关闭之前，是不能被另外一个文件流控件打开的。

2、文件每次打开，文件流属性 val 都会被赋值为 0。

3.28.1.2.2 read-从文件流读取数据

int read(object att,int star,int lenth)

读取成功返回 1，失败返回 0

att 变量名称

star 变量起始地址 (一般为 0)

lenth 读入数据长度

3.28.1.2.3 文件流读取-示例

以下代码将会从文件的 100 地址处读取 10 个字节，读取完毕后，文件流的指针（fs0.val）会自动变为 110

```
fs0.val=100 //设置读取的位置
fs0.read(va2.txt,0,10) //读取10个字节,将数据放入va2.txt
```

注意： 起始地址不为 0 时，若此时文本控件为空文本，即首字符为”\0”，此时是读取成功的，但是文本控件的第一个数据是”\0”，被判断为字符串已经结束了，因此还是显示空白文本。

读取要注意控件属性 (txt_maxl), 读取数据长度超过范围，会读取失败，返回 0，无法显示读取的内容。

读取成功，文件流指针会自动增加。

3.28.1.2.4 write-将数据写入文件流

int write(object att,int star,int lenth)

写入成功返回 1，失败返回 0

att 变量名称

star 变量起始地址 (一般为 0)

lenth 写入数据长度

3.28.1.2.5 文件流写入-示例

以下代码将会从文件的 1000 地址处读取 100 个字节，读取完毕后，文件流的指针（fs0.val）会自动变为 1100

```
fs0.val=1000 //设置写入的位置
fs0.write(va2.txt,0,100) //将va2.txt的字符内容写入当前打开的文件中
```

> **注意：** 写入成功，文件流指针会自动增加
>
> 变量起始地址不为 0 时，写入变量的起始地址，不是文件的起始位置。

```
t0.txt="abcd123456"
fs0.write(t0.txt,2,10)  //此时写入的是cd123456,长度不足的部分被写入0x00
```

3.28.1.2.6 close-关闭文件流

int close()

成功返回 1, 失败返回 0

文件打开读写操作完成后一定要记得关闭文件, 同一个文件在打开后, 关闭之前, 是不能被另外一个文件流控件打开的

3.28.1.2.7 文件流关闭-示例

```
if(fs0.en==1)
{
    fs0.close()
}
```

3.28.1.2.8 find-按关键字查询并定位文件流指针

int find(string key)

成功返回 1，失败返回 0

key 关键字字符串变量/常量

3.28.1.2.9 文件流查找-示例

```
t0.txt="abcd123456"
fs0.write(t0.txt,0,15)  //写入成功后fs0.val变为15
fs0.val=0   //将文件流指针复位，否则查找不到
sys0=fs0.find("123")
if(sys0==1)
{
    n0.val=fs0.val
}
```

查找结果 n0.val=4

> **注意：** 从当前流的当前数据指针 (val 属性) 位置开始查询关键字，如果查询成功，数据指针将会移动到关键字中第一个字符串：如果查询失败保持数据当前指针不变

3.28.2 文件流控件-常见问题

3.28.2.1 在电脑模拟器上是正常打开的，但是下载到串口屏后总是提示打开失败

请按以下步骤进行检查

1、串口屏上是否插入了 SD 卡

2、SD 卡是否超过 32GB，请勿使用超过 32GB 的 SD 卡（例如：512M、1GB、2GB、4GB、8GB、16GB、32GB 都是可用的）

3、SD 卡的文件格式是否是 FAT32，目前只支持 FAT32 格式

3.28.2.2 在电脑模拟器上读取到的数据和串口屏上的数据不一致

电脑上在创建文件时，会自动将整个文件都初始化为 0x00, 但是在串口屏上并不会

这是因为电脑的 cpu 是多线程的且电脑的性能远远高于串口屏，而串口屏是单线程

如果串口屏也自动初始化整个文件，当用户创建比较大的文件，如 1GB 的文件时，串口屏将会因为初始化文件的原因卡住一段时间，可能几分钟到十几分钟不等，取决于 SD 卡性能

如有需求，可以按照下面的方法来初始化文件

```
//新建文件，并初始化整个文件
newfile "sd0/1.txt",4096
for(sys0=0;sys0<=1024;sys0++)
{
    //循环1024次，每次写入4字节，总共写入4096个字节，将0-4095初始化为0x00
    fs0.write(0,0,4)
}
```

3.28.2.3 文件流控件如何跨页面调用同一个文件

文件流控件只能是私有的，不能跨页面调用，因此只能在当前页面调用文件。

需要跨页面调用同一个文件的情况下，请使用以下方法。

用一个全局文本控件记录当前文件流控件调用的文件位置。

用一个全局数字控件记录当前文件流控件的 val 属性。

每个需要用到文件流读写的页面都要添加一个文件流控件。

当跳转到另一个页面时，重新通过全局文本控件打开之前的文件，重新通过全局数字控件赋值给新的文件流控件的 val 属性即可进行读写, 读写完毕后关闭文件流。

以上方法也可以用于一个文件流控件同时操作记录多个文件的名称以及 val 属性。

3.28.3 文件流控件-样例工程下载

演示工程下载链接：

《文件浏览器 v3.0》演示工程下载

3.28.4 文件流控件-相关链接

SD 卡读写文件流程 (page 1024)

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.28.5 文件流控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

`vscope` 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

文件流控件的 vscope 属性仅能为私有。

`val` 属性 -此文件流当前数据指针 (打开文件时恢复为 0, 读写操作过程中自动移动, 支持手动设置)

`qty` 属性 -文件大小 (运行中根据实际打开的文件自动更新, 只可获取不可设置)

`en` 属性 -文件打开状态 (只可获取不可设置)

3.28.6 文件流控件-方法详解

`open` 方法 - int open(string path) 打开文件 (成功返回 1, 失败返回 0)

`read` 方法 - int read(object att,int star,int lenth) 从当前流读数据 (成功返回 1, 失败返回 0, 从当前数据指针 [val 属性] 位置开始读, 读完以后指针将自动移动相应的长度)

`write` 方法 - int write(object att,int star,int lenth) 将数据写入当前流 (成功返回 1, 失败返回 0, 从当前数据指针 [val 属性] 位置开始写, 写完以后指针将自动移动相应的长度)

`close` 方法 - int close() 关闭文件流 (成功返回 1, 失败返回 0, 文件打开读写操作完成后一定要记得关闭文件, 同一个文件在打开后, 关闭之前, 是不能被另外一个文件流控件打开的)

`find` 方法 - int find(string key) 按关键字查询并定位文件流指针 (查询成功返回 1, 失败返回 0, 从当前流的当前数据指针 [val 属性] 位置开始查询关键字, 如果查询成功, 数据指针将会移动到关键字中第一个字符处; 如果查询失败保持数据当前指针不变)

3.29 动画控件

动画控件仅 X2、X3、X5 系列支持

动画控件用于显示 gif 动画。

其他系列不支持动画控件，但是可以使用定时器 + 图片控件实现完全一样的功能。参考图片控件 (page 442) 定时器控件 (page 471)

使用动画前，需要先用动画制作工具将 gif 图片转换为特定格式才能导入串口屏上位机中。

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

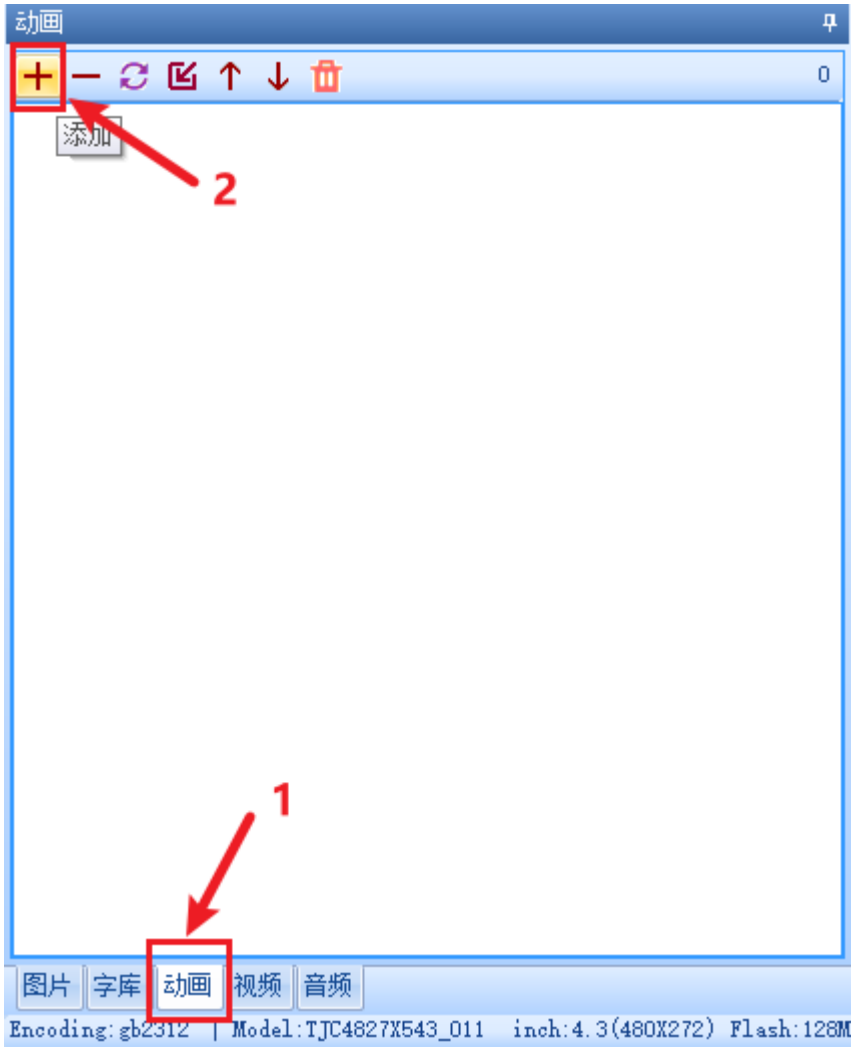
导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

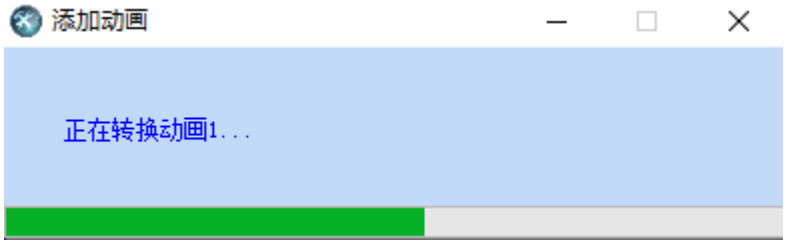
3.29.1 动画控件-使用详解

3.29.1.1 开机动画制作演示

1. 点击左下角动画标签，点击“+”，导入素材

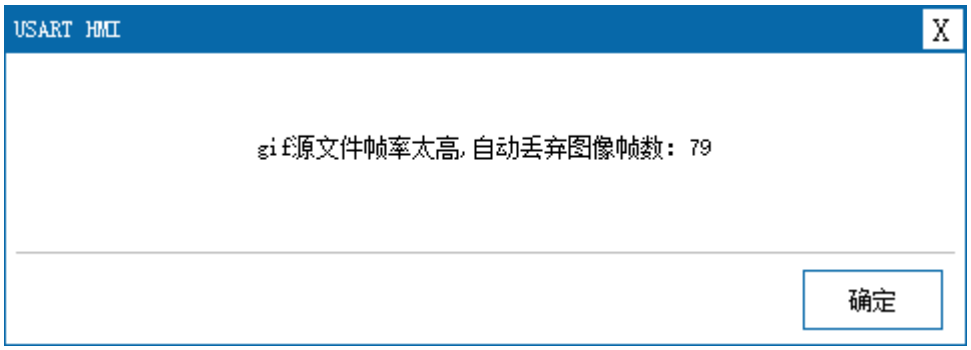

动画
0
添加
2
1
图片
字库
动画
视频
音频
Encoding:gb2312 | Model:TJC4827X543_011 inch:4.3(480X272) Flash:128M

2. 导入一张 gif 动图，并等待动画转换完成



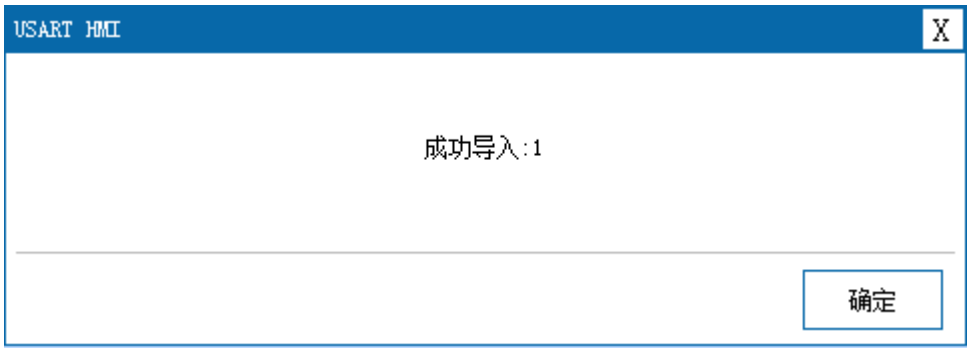
---

3. 如果提示帧率太高，点击确定即可



---

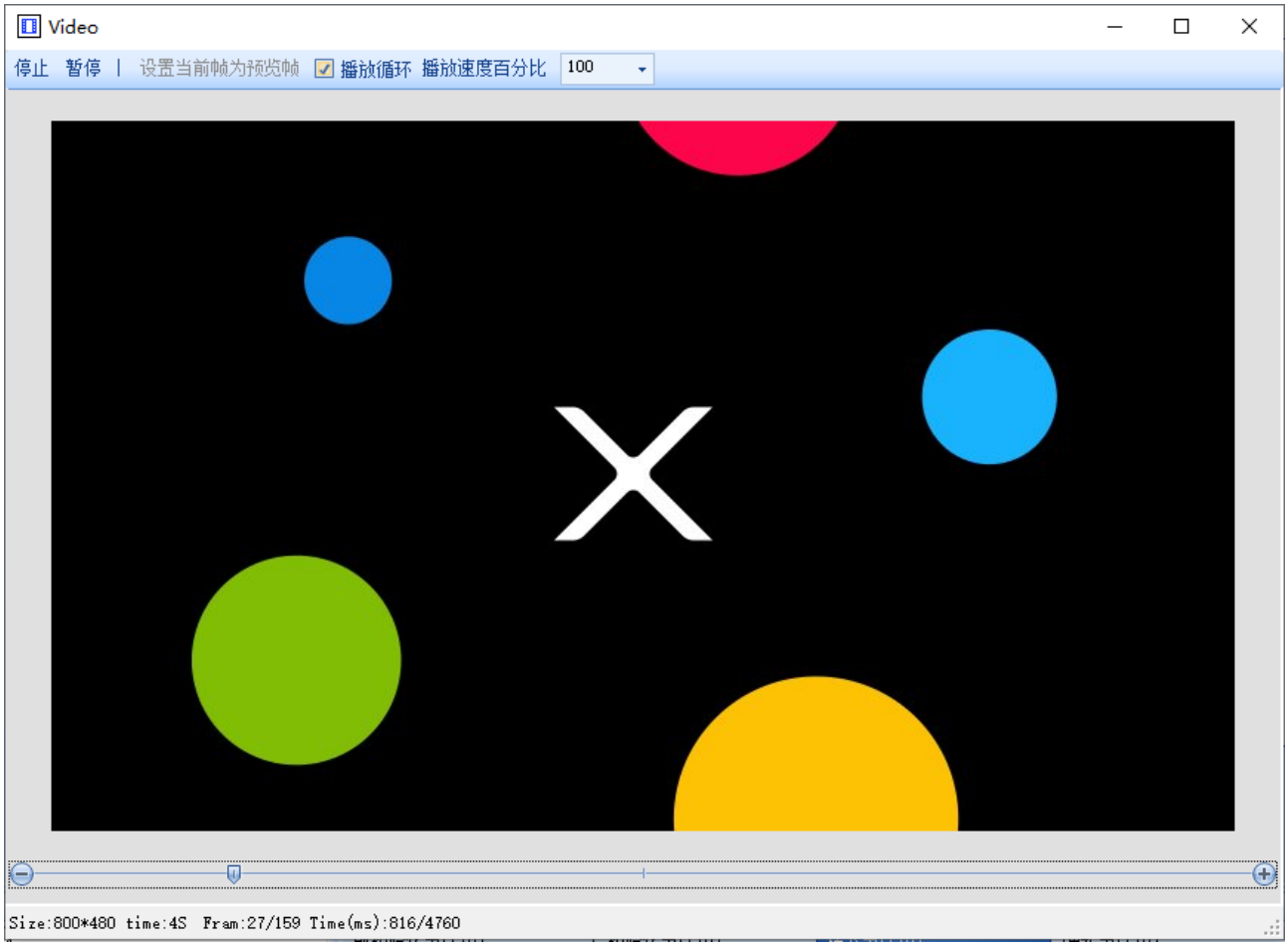
4. 显示成功导入，点击确定



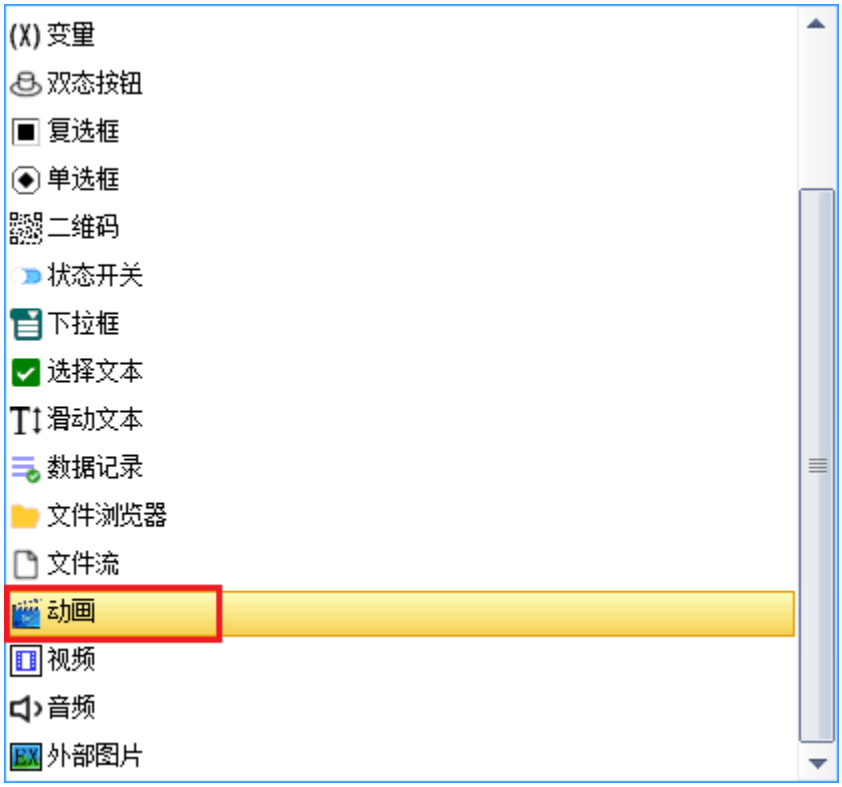
---

5. 此时动画标签就会显示我们刚刚导入成功的动画

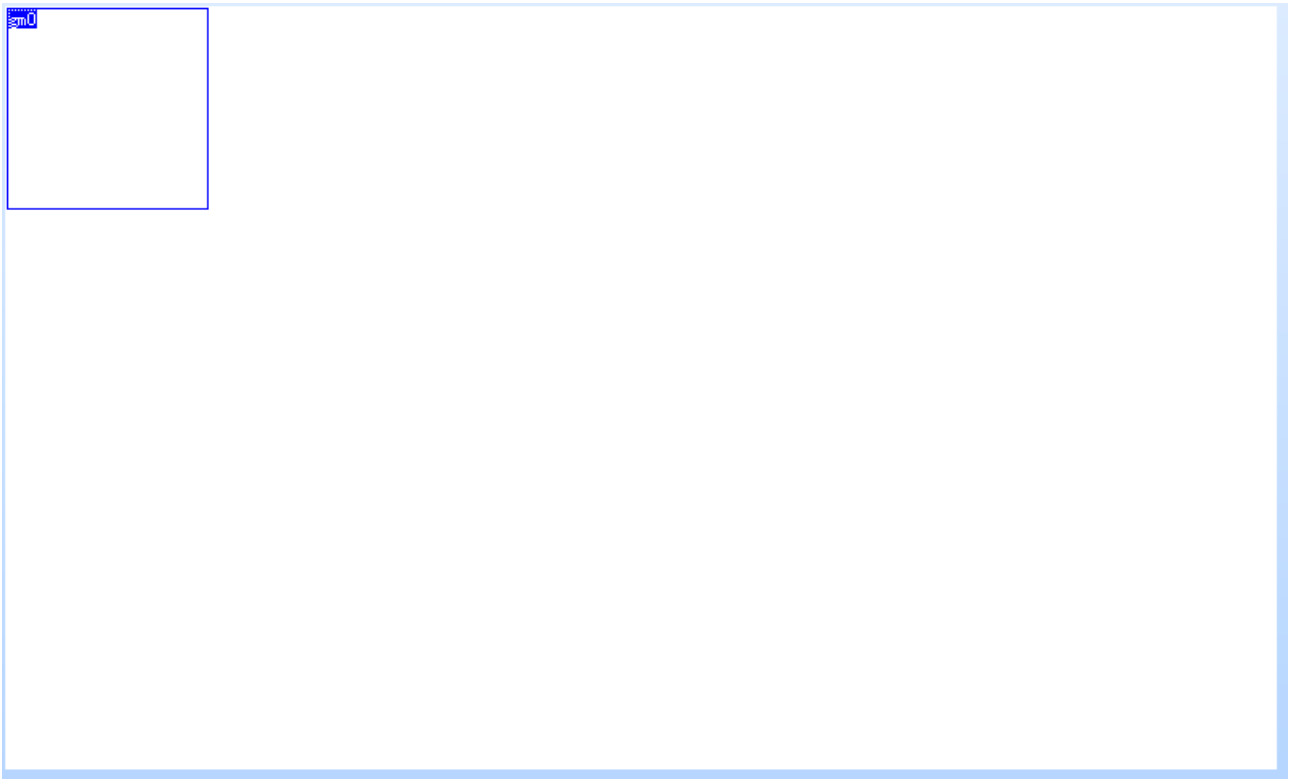
6. 双击动画资源，可以查看动画效果



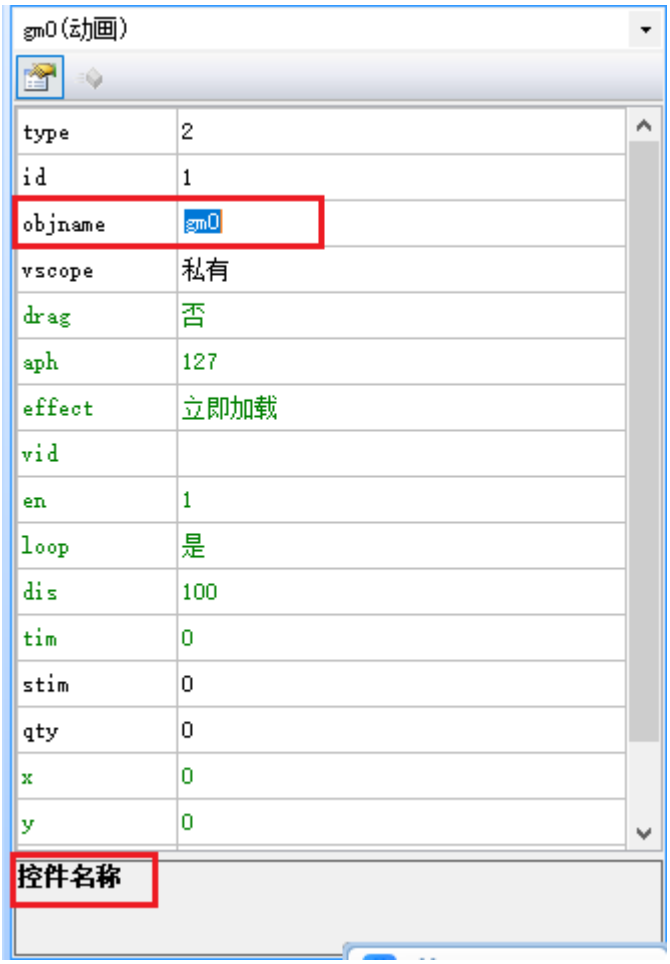
7. 点击工具箱中的动画控件，在界面上新建一个动画控件



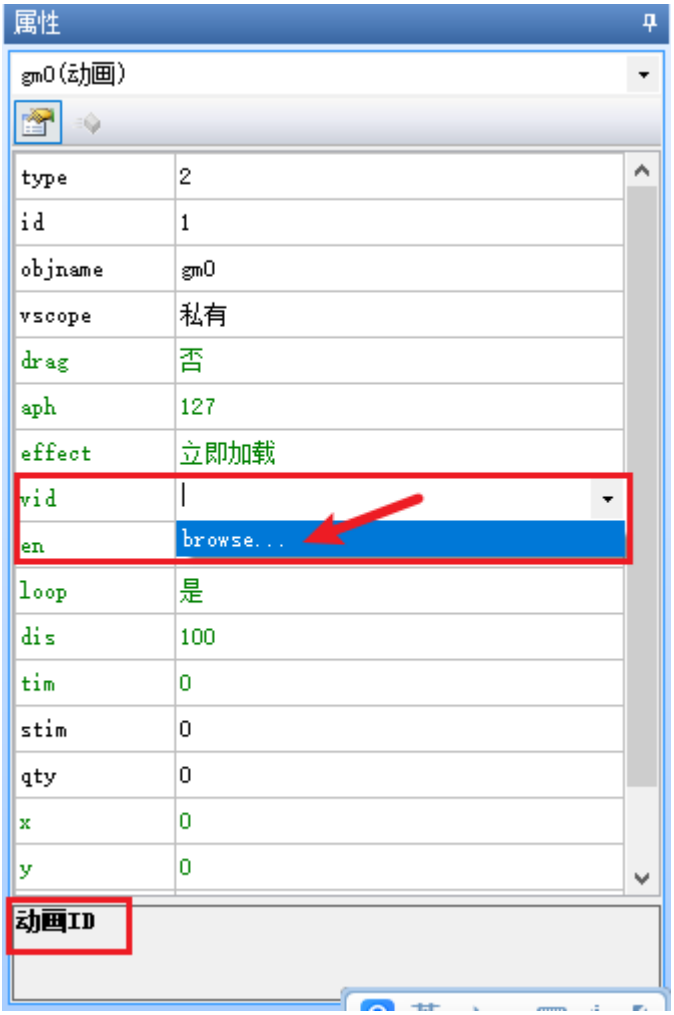
8. 选中动画控件 gm0



9. 选中动画控件的 object-gm0 属性，可以看到底部显示了当前属性的注释，属性分为黑色和绿色，黑色属性不可更改或者只能在上位机中设计时进行更改，绿色属性可以通过代码进行修改



10. 选中动画控件的 vid 属性，并点击 browse..，属性栏底部会显示当前属性的注释



11. 选中刚刚导入的动画，并点击确定

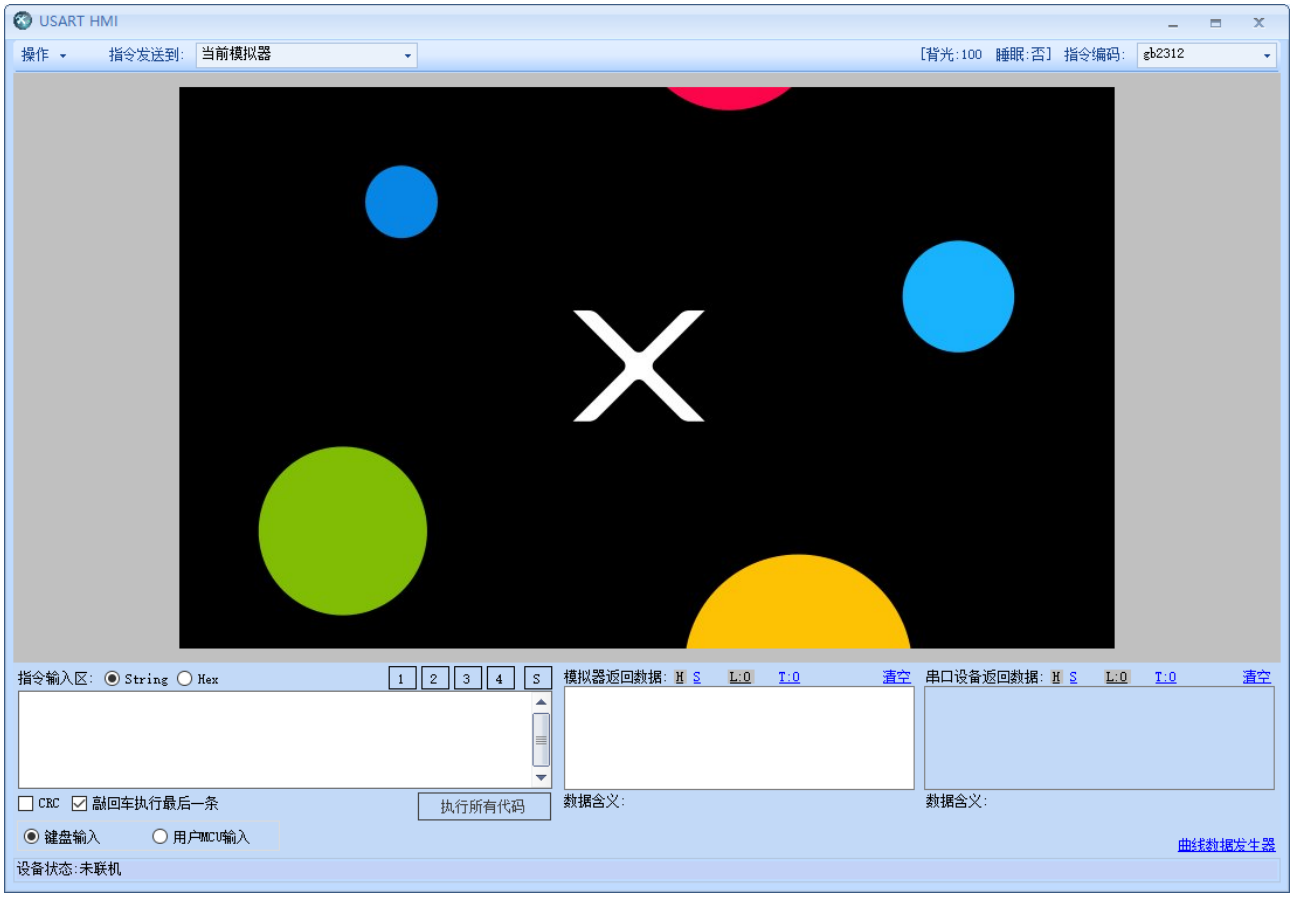
12. 此时动画控件会自适应素材的尺寸，变为全屏

---

13. 点击顶部的调试按钮，查看效果

---

14. 调试效果如下图所示



3.29.1.2 在动画播放完成后跳转页面

在动画控件 gm0 的播放完成事件编写以下代码

```
page main  //跳转到main页面
```

3.29.2 动画控件-样例工程下载

演示工程下载链接：

《动画控件》演示工程下载

3.29.3 动画控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.29.4 动画控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

`vscope` 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

`drag` 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`aph` 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`effect` 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

`vid` 属性 -动画 ID

`en` 属性 -播放状态 (0-停止;1-播放;2-暂停)

`loop` 属性 -循环播放:0-否;1-是

`dis` 属性 -播放速度百分比 (最小 10, 最大 1000)

`tim` 属性 -当前播放时间 (ms)

`stim` 属性 -总时间 (由动画文件决定，不可设置, 运行中可获取)

`qty` 属性 -总帧数 (由动画文件决定，不可设置, 运行中可获取)

`x` 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

`y` 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

`w` 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

`h` 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.30 视频控件

视频控件用于播放视频，仅 X5 系列支持

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

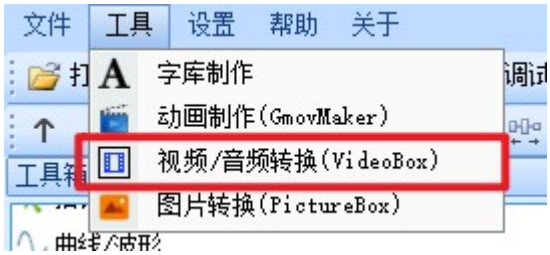
导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.30.1 视频控件-使用详解

使用视频控件前，需要先用音视频转换工具转换视频资源为.video 格式



转换好的视频文件，从左下角的视频资源窗口导入

导入成功后，我们需要关注的是视频文件的 ID 号

视频控件的 vid 属性填写的是对应视频控件的 id 号

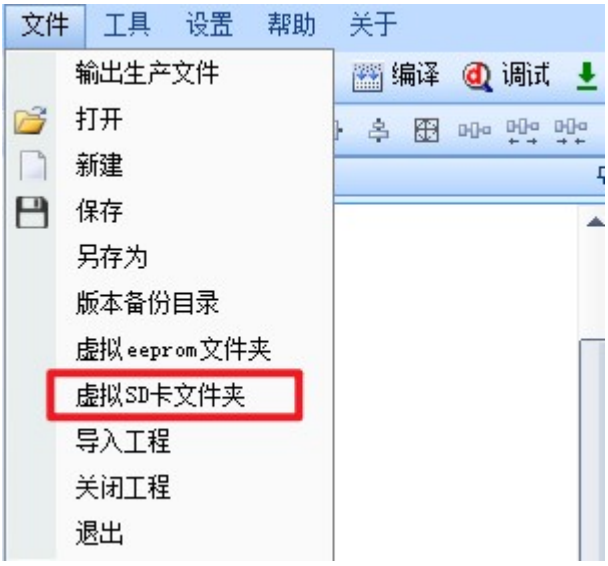
视频的 vid 属性填写了 1，那么调用的就是 id 为 1 的视频

视频控件可以配置 from 属性，来设置从 flash 内部或者从 SD 卡内读取音频资源进行播放，当配置为外部文件时，此时将会从 SD 卡中调用文件，vid 属性将变成 path 属性

请提前将转换好的资源文件复制到 SD 卡或者虚拟 SD 卡文件夹，并且填写正确的 path 属性。

注意： 当需要在上位机的模拟器里调试时，请将资源复制到虚拟 SD 卡文件夹中，需要在串口屏实物上调试时，请将资源复制到 SD 卡里，并插到串口屏上

虚拟 SD 卡文件夹打开方式



SD 卡不能超过 32GB（例如：512M、1GB、2GB、4GB、8GB、16GB、32GB 都是可用的），请格式化为 FAT32 格式

例如放在 SD 卡根目录的 1.video 文件，对应的路径是

```
sd0/1.video
```

放在 SD 卡 video 目录下的 demo.video 文件，对应的路径是

```
sd0/video/demo.video
```

属性
v0(视频)
type 3
id 2
objname v0
vscope 私有
drag 否
aph 127
effect 立即加载
from 外部文件
path sd0/video/demo.video
en 1
loop 是
dis 100
tim 0
stim 0
qty 0
x 0
y 0
w 800
h 480
外部视频文件路径
(如:"ram/0.video"或"sd0/1.video")

3.30.2 视频控件-c 语言示例

单片机通过串口给视频控件赋值

```
int vid_enable=1;
printf("vid0.en=%d\xff\xff\xff",vid_enable);     //开启视频播放
```

3.30.3 视频控件-常见问题

3.30.3.1 v0.vid 初始值无效: 视频 ID 无效

这是因为没有配置 vid 属性导致的，导入视频文件并配置 vid 属性即可

3.30.3.2 视频控件-视频无法播放

使用 usart hmi 上位机时无法播放，但是下载进屏幕可以正常播放

修改 video 控件的 dis 为 99 或者 101 属性（播放速度百分比）

3.30.4 视频控件-样例工程下载

演示工程下载链接：

《视频控件》演示工程下载

3.30.5 视频控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

volume-系统音量 (page 749)

觉得喇叭声音小怎么办 (page 1245)

喇叭接口型号 (page 1246)

喇叭的正负极 (page 1246)

喇叭插上有电流声 (page 1247)

视频 *IDO* 文件方向与当前工程显示方向不一致 (page 1368)

3.30.6 视频控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

`vscope` 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

`drag` 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`aph` 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

`effect` 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

`from` 属性 -播放源:0-内部资源文件;1-外部文件。可读，可通过上位机进行修改，不可通过指令更改。

`vid` 属性 -视频 ID，当 from 为内部资源文件时可用。可读，可通过上位机进行修改，可通过指令更改。

path 属性 -外部视频文件路径 (如:”ram/0.video”或”sd0/1.video”)，当 from 为外部文件时可用。可读，可通过上位机进行修改，可通过指令更改。

en 属性 -播放状态 (0-停止;1-播放;2-暂停)。可读，可通过上位机进行修改，可通过指令更改。

loop 属性 -循环播放:0-否;1-是。可读，可通过上位机进行修改，可通过指令更改。

dis 属性 -播放速度百分比 (最小 10, 最大 1000)。可读，可通过上位机进行修改，可通过指令更改。

tim 属性 -当前播放时间 (ms)。可读，可通过上位机进行修改，可通过指令更改。

stim 属性 -总时间 (由视频文件决定，不可设置, 运行中可获取)。可读，不可通过上位机进行修改，不可通过指令更改。

qty 属性 -总帧数 (由视频文件决定，不可设置, 运行中可获取)。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

3.31 音频控件

音频控件用于播放音频，仅带喇叭的型号（例如 X3、X5 系列）支持

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.31.1 音频控件-使用详解

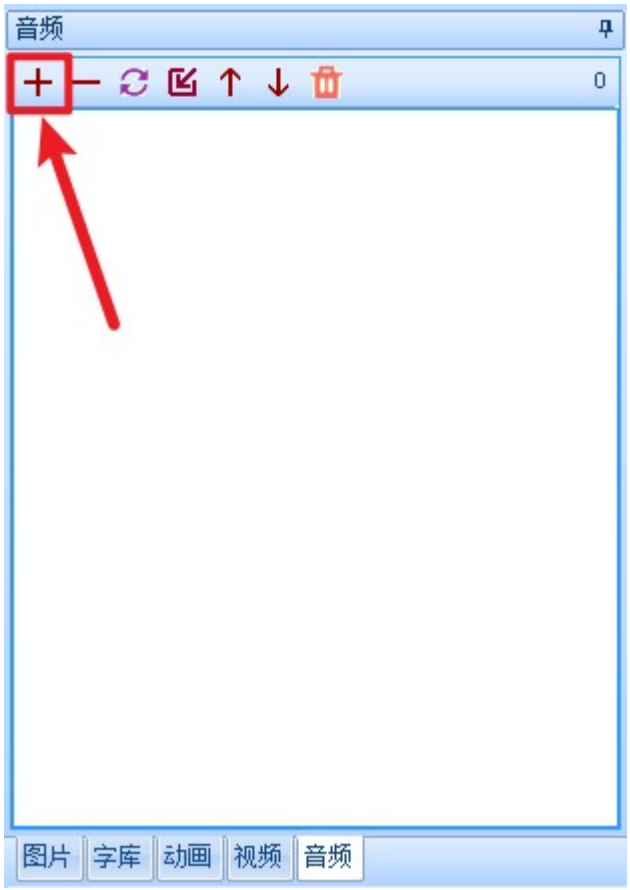
音频控件位于特殊控件栏

使用音频控件前，需要先用音视频转换工具转换音频资源（即使原本就是 wav 格式，如果导不进去，也请再转换一遍）

文件
工具
设置
帮助
关于
字库制作
动画制作(GmovMaker)
视频/音频转换(VideoBox)
图片转换(PictureBox)
调试
下载
工具箱

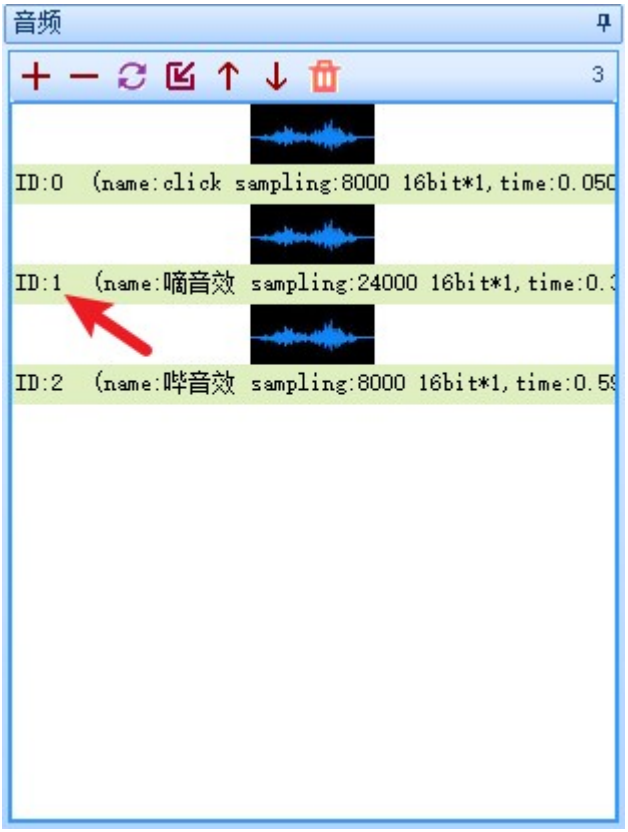
点击这里切换到音频转换

转换好的音频文件，从左下角的音频资源窗口导入



导入成功后，我们需要关注的是音频文件的 ID 号

音频控件的 vid 属性填写的是对应音频控件的 id 号

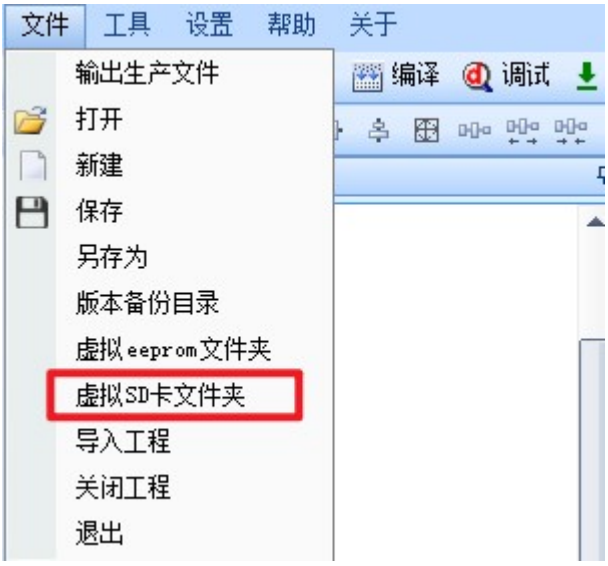


音频控件可以配置 from 属性，来设置从 flash 内部或者从 SD 卡内读取音频资源进行播放，当配置为外部文件时，此时将会从 SD 卡中调用文件，vid 属性将变成 path 属性

请提前将转换好的资源文件复制到 SD 卡或者虚拟 SD 卡文件夹，并且填写正确的 path 属性。

注意： 当需要在上位机的模拟器里调试时，请将资源复制到虚拟 SD 卡文件夹中，需要在串口屏实物上调试时，请将资源复制到 SD 卡里，并将 SD 卡插到串口屏上

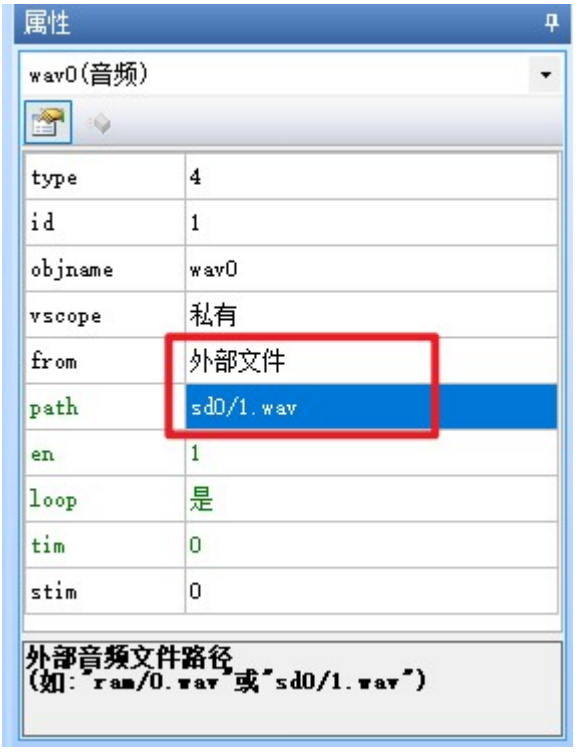
虚拟 SD 卡文件夹打开方式



SD 卡不能超过 32GB（例如：512M、1GB、2GB、4GB、8GB、16GB、32GB 都是可用的），请格式化为 FAT32 格式

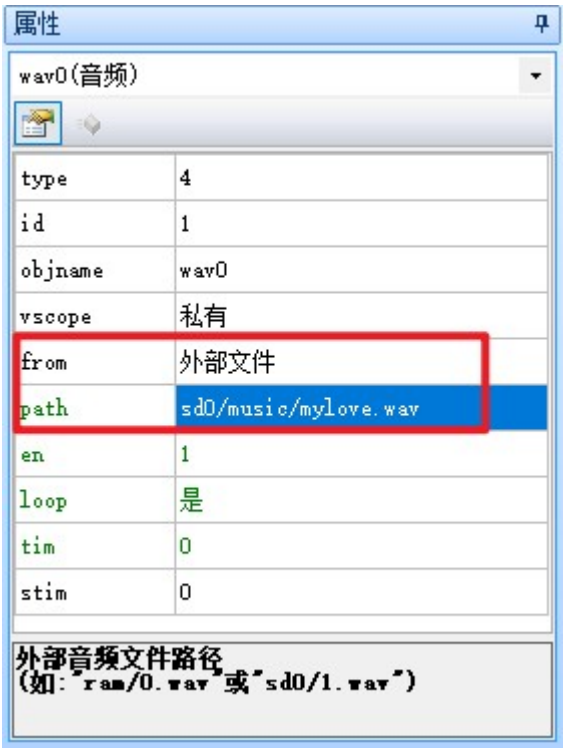
例如放在 SD 卡根目录的 1.wav 文件，对应的路径是

```
sd0/1.wav
```



放在 SD 卡 music 目录下的 mylove.wav 文件，对应的路径是

```
sd0/music/mylove.wav
```



3.31.1.1 打开或关闭音频

```
//串口屏控件事件代码（通常写在串口屏控件的按下或弹起事件中）
wav0.en=1   //开始播放音频
wav0.en=0   //停止播放音频


//单片机发送指令音频控制指令
int sound_sta = 1;   //播放状态
char tjcstr[100];
sprintf(tjcstr, "main.wav0.en=%d\xff\xff\xff",sound_sta);
printf(tjcstr);   //单片机需要配置printf重定向到串口


//arduino发送指令音频控制指令
int sound_sta = 1;   //播放状态
char tjcstr[100];
sprintf(tjcstr, "main.wav0.en=%d\xff\xff\xff",sound_sta);
Serial.print(tjcstr);
```

3.31.1.2 设置音频是否循环播放

```
//串口屏控件事件代码（通常写在串口屏控件的按下或弹起事件中）
wav0.loop=1   //循环播放音频
wav0.loop=0   //不循环播放音频


//单片机发送指令音频控制指令
int sound_loop = 1;   //循环状态
char tjcstr[100];
sprintf(tjcstr, "main.wav0.loop=%d\xff\xff\xff",sound_loop);
printf(tjcstr);   //单片机需要配置printf重定向到串口


//arduino发送指令音频控制指令
int sound_loop = 1;   //循环状态
char tjcstr[100];
sprintf(tjcstr, "main.wav0.loop=%d\xff\xff\xff",sound_loop);
Serial.print(tjcstr);
```

3.31.1.3 音频控件和 play 指令区别

① 音频控件播放音频只能在当前页面播放，不可跨页面播放。play 指令可以在各个页面使用。如果设置各个页面按键音那么使用 play 指令。

② 音频控件可以选择播放外部资源文件 (sd 卡文件)，play 指令只能播放内部资源文件。

③ 音频控件可以获取播放音频的总时间以及当前时间，play 指令无法获取。

④ 音频控件会占用大量内存，使用多个音频控件会导致内存溢出。play 指令不会出现类似问题。

⑤ 音频控件在事件编译窗口会有播放完成事件，play 指令没有。

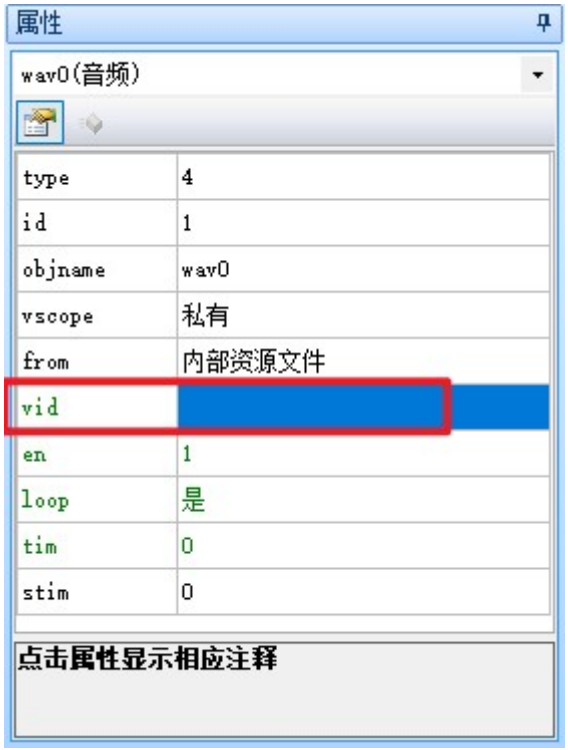
3.31.1.4 如何使用外部音频播放

1. 控件属性 from 设置为 1（外部文件）；2.path 设置为路径（例：sd0/1.wav，注：运行赋值需要加双引号）。

3.31.2 音频控件-常见问题

3.31.2.1 wav0.vid 初始值无效: 音频 ID 无效

这是因为没有配置 vid 属性导致的，导入音频文件并配置 vid 属性即可



3.31.2.2 屏幕使用外部音频播放无声音

1. 屏幕是否识别到 sd 卡里的文件（可以用文件浏览器控件看是否有文件显示出来）；2.sd 卡音频文件名是否和属性 path 设置的文件名是否一致（即 path 设置为 sd0/1.wav，那么 SD 卡根目录音频文件名要为 1.wav，否则无法识别到）；3. 屏幕电路板是否有外接喇叭（sd 卡旁边 j5 接口是否接有喇叭）。4. 检测 volume 指令是否设置为 0（默认为 100）

3.31.3 音频控件-样例工程下载

演示工程下载链接：

《音频控件使用》演示工程下载

《数字播报》演示工程下载

《电话》演示工程下载

3.31.4 音频控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

volume-系统音量 (page 749)

觉得喇叭声音小怎么办 (page 1245)

喇叭接口型号 (page 1246)

喇叭的正负极 (page 1246)

喇叭插上有电流声 (page 1247)

3.31.5 音频控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

`type` 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

`id` 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

`objname` 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

`vscope` 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

`from` 属性 -播放源，内部资源文件或外部文件，可通过上位机进行修改，不可通过指令更改

`vid` 属性 -音频 ID。播放源为内部资源文件时显示。可通过上位机进行修改，可通过指令更改

`path` 属性 -外部音频文件路径 (如:”ram/0.wav”或”sd0/1.wav”)。播放源为外部文件时显示。可通过上位机进行修改，可通过指令更改。

`en` 属性 -播放状态 (0-停止;1-播放;2-暂停)，可通过上位机进行修改，可通过指令更改

`loop` 属性 -循环播放:0-否;1-是，可通过上位机进行修改，可通过指令更改

`tim` 属性 -当前播放时间 (ms)，可通过上位机进行修改，可通过指令更改

`stim` 属性 -总时间 (由音频文件决定，不可更改，运行中可获取)

3.32 外部图片控件

外部图片控件仅 X 系列支持

X2 系列解析外部图片控件会出现问题，请勿使用。

外部图片控件用于显示 SD 卡中的图片，仅 X 系列支持，需要插入 SD 卡

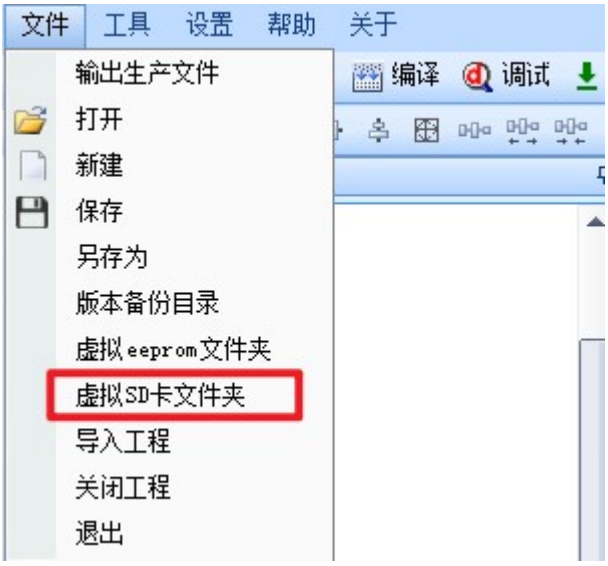
图片格式支持 jpg 和 xi 格式, 直接将图片放在 SD 卡中即可调用

3.32.1 外部图片控件-使用详解

请提前将转换好的资源文件复制到 SD 卡或者虚拟 SD 卡文件夹，并且填写正确的 path 属性。

> **注意:** 当需要在上位机的模拟器里调试时，请将资源复制到虚拟 SD 卡文件夹中，需要在串口屏实物上调试时，请将资源复制到 SD 卡里，并插到串口屏上

虚拟 SD 卡文件夹打开方式



SD 卡不能超过 32GB（例如：512M、1GB、2GB、4GB、8GB、16GB、32GB 都是可用的），请格式化为 FAT32 格式

例如放在 SD 卡根目录的 1.jpg 文件，对应的路径是

```
sd0/1.jpg
```

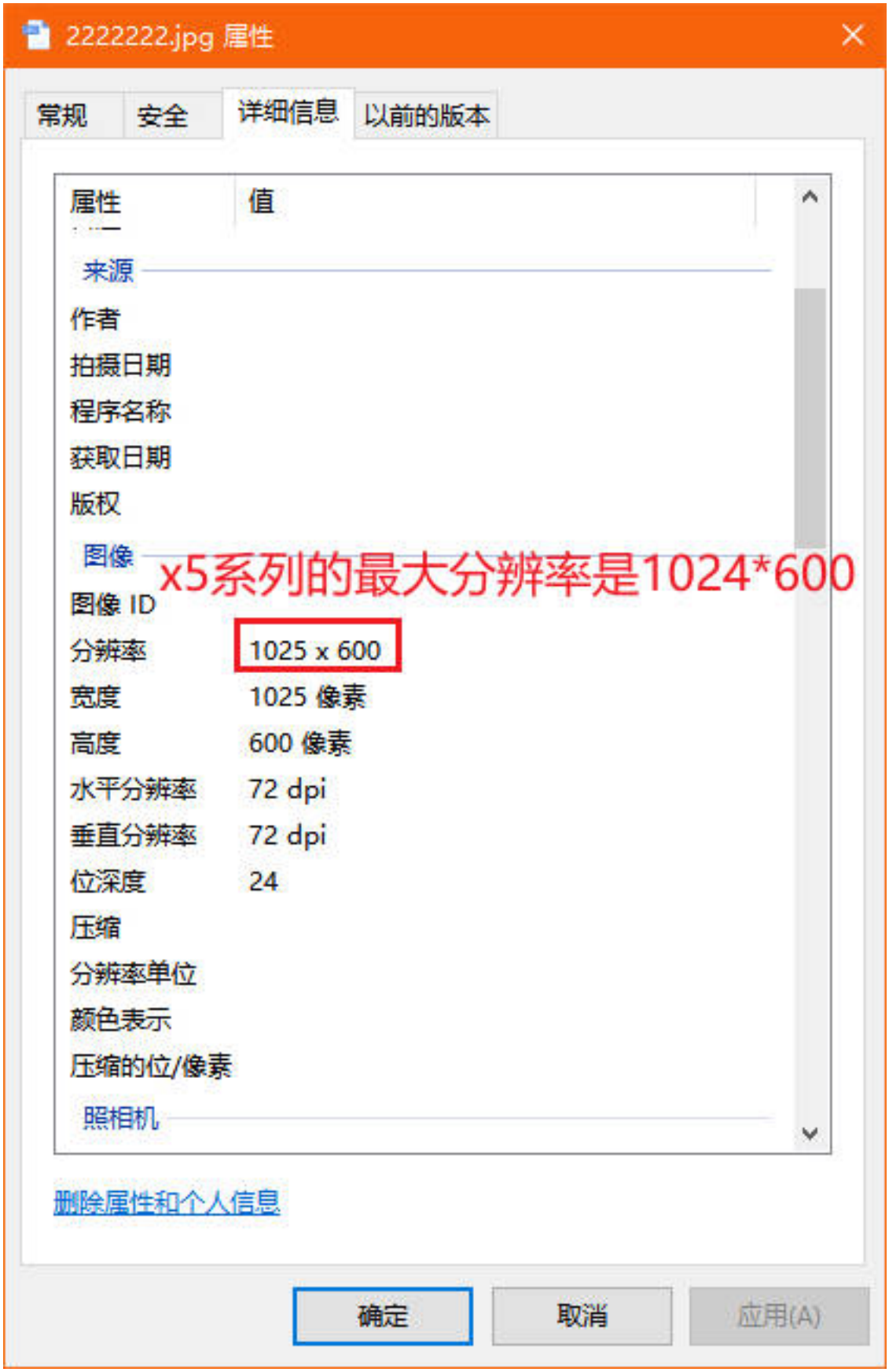
放在 SD 卡 pic 目录下的 aaa.jpg 文件，对应的路径是

```
sd0/pic/aaa.jpg
```

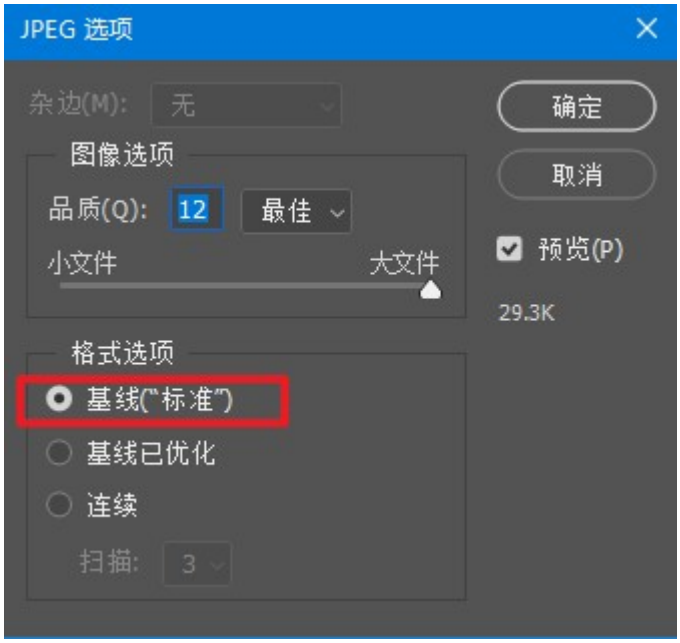
3.32.2 外部图片控件-常见问题

3.32.2.1 外部图片显示不正常

步骤 1、检查图片的分辨率是否超过了屏幕的分辨率导致显示异常



步骤 2、使用 ps 打开 jpg 图片，将该图片另存为，保存时格式选择“基线（“标准”）”



3.32.2.2 外部图片有透明部分，但是 jpg 不支持透明

外部图片使用 jpg 格式时无法显示透明图层

必须使用 png 图片用上位机自带的图片转换工具转为.xi 格式的图片

3.32.3 外部图片控件-样例工程下载

演示工程下载链接：

《外部图片控件》演示工程下载

3.32.4 外部图片控件-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)

txt 属性和 val 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

跨页面赋值，全局变量操作 (page 847)

3.32.5 外部图片控件-属性详解

控件属性解析 (page 385)

提示： 绿色属性可以通过上位机或者串口屏指令进行修改，黑色属性只能在上位机中修改或者不可修改，可通过上位机进行修改指“选中控件后通过属性栏修改控件的属性”

---

type 属性 -控件类型，固定值，不同类型的控件 type 值不同，相同类型的控件 type 值相同，可读，不可通过上位机修改，不可通过指令修改。参考：控件属性-控件 *id* 对照表 (page 391)

id 属性 -控件 ID，可通过上位机左上角的上下箭头置顶或置底，可读，可通过上位机修改左上角的箭头置顶或置地间接修改，不可通过指令修改。参考：如何更改控件的前后图层关系 (page 1315)

objname 属性 -控件名称。不可读，可通过上位机进行修改，不可通过指令更改。

vscope 属性 -内存占用 (私有占用只能在当前页面被访问，全局占用可以在所有页面被访问)，当设置为私有时，跳转页面后，该控件占用的内存会被释放，重新返回该页面后该控件会恢复到最初的设置。可读，可通过上位机进行修改，不可通过指令更改。参考：跨页面赋值，全局变量操作 (page 847)

drag 属性 -是否支持拖动:0-否;1-是。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

aph 属性 -不透明度 (0-127)，0 为完全透明，127 为完全不透明。仅 x 系列支持。可读，可通过上位机修改，可通过指令修改。

effect 属性 -加载特效:0-立即加载;1-上边飞入;2-下边飞入;3-左边飞入;4-右边飞入;5-左上角飞入;6-右上角飞入;7-左下角飞入;8-右下角飞入。仅 x 系列支持，在上位机中设置为立即加载时，无法通过指令变为其他特效，当在上位机中设置为非立即加载的特效时，可以变为立即加载，也可以再改为其他特效

path 属性 -图片文件路径 (如:”ram/0.jpg”或”sd0/1.jpg”)。

x 属性 -控件的 X 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

y 属性 -控件的 Y 坐标。可读，可通过上位机修改，x 系列可通过指令修改，其他系列不可通过指令修改。

w 属性 -控件的宽度。可读，可通过上位机修改，不可通过指令修改。

h 属性 -控件的高度。可读，可通过上位机修改，不可通过指令修改。

提示： 导入字库请参考: 如何导入字库 (page 1442)

导入图片请参考: 如何导入图片 (page 1449)

导入动画请参考: 如何导入动画 (page 1455)

导入视频请参考: 如何导入视频 (page 1461)

导入音频请参考: 如何导入音频 (page 1467)

3.33 关于控件属性的说明

endx：控件右侧横坐标，等于控件起始横坐标 x 加上控件宽度 w

endy：控件底部纵坐标，等于控件起始纵坐标 x 加上控件高度 h

4 指令集

4.1 常用指令集

所有型号的串口屏均支持常用指令，且指令的使用频率较高

4.1.1 page-页面跳转指令

如上图所示，红色框内为页面 ID，绿色框内为页面名称

小技巧： 以上图为例，main 页面的 ID 为 2，因此如果要跳转到 main 页面，可以使用 page 页面名称或者 page 页面 ID

- page main
- page 2
- 也可以对系统变量 dp 赋值来实现跳转页面 (如 dp=2)，系统变量 dp 可以设置可以读取，具体请参看系统变量列表。

注意： page 指令后面的代码将不会被执行，所以千万不要把代码写在 page 指令后面

小心： page 指令后面的代码将不会被执行，所以千万不要把代码写在 page 指令后面

提示： page 指令后面的代码将不会被执行，所以千万不要把代码写在 page 指令后面

重要： page 指令后面的代码将不会被执行，所以千万不要把代码写在 page 指令后面

备注： page 指令后面的代码将不会被执行，所以千万不要把代码写在 page 指令后面

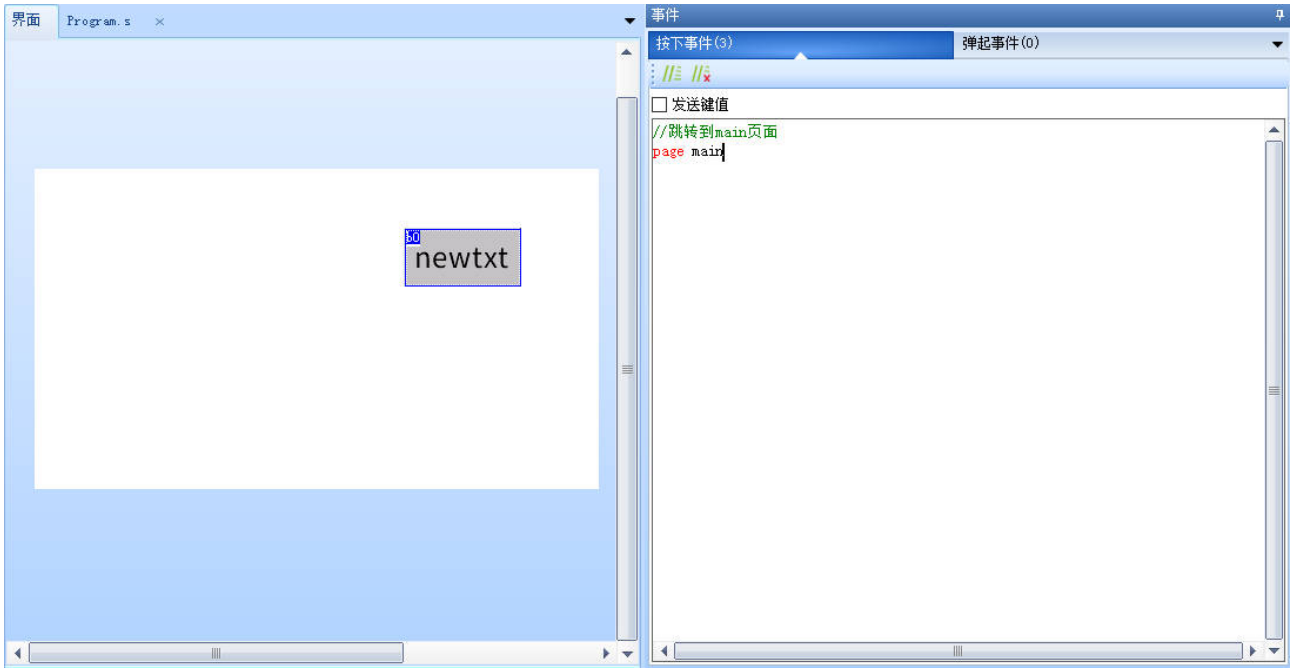
备注： page 指令后面的代码将不会被执行，所以千万不要把代码写在 page 指令后面

小技巧： page 指令后面的代码将不会被执行，所以千万不要把代码写在 page 指令后面

警告： page 指令后面的代码将不会被执行，所以千万不要把代码写在 page 指令后面

4.1.1.1 page-示例 1

```
//跳转到main页面
page main
```



4.1.1.2 page-示例 2 返回上一个页面

dp-示例 4: 返回上一页 (page 744)

4.1.1.3 page-应用实例

通过按钮控件跳转页面 (page 431)

4.1.1.4 page-c 语言示例

单片机通过串口让串口屏跳转页面

```
//跳转到页面ID为1的页面
printf("page 1\xff\xff\xff");
```

```
//跳转到页面名称为main的页面,推荐使用page + 页面名称 的跳转方法
printf("page main\xff\xff\xff");
```

4.1.1.5 page 指令-相关链接

页面和页面控件的区别 (page 1326)

4.1.1.6 page 指令-样例工程下载

演示工程下载链接:

《page 指令》演示工程下载

《page 指令-返回上一个页面》演示工程下载

4.1.2 prints-从串口打印一个变量/常量

将变量/常量从串口发送出去

如果要发送固定的 16 进制数据请使用 printh，请参考*printh-从串口打印 16 进制* (page 586)

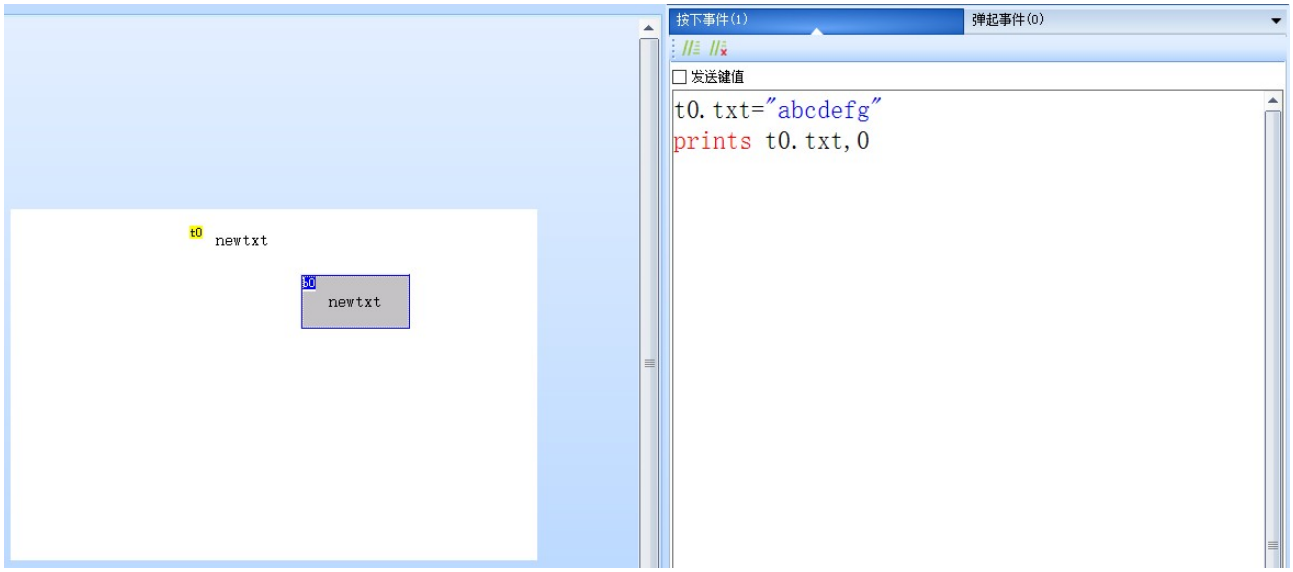
```
prints att,lenth

att:变量名称

lenth:发送长度(0为自动长度)
```

> **注意：** 当 att 为数值类型时，length 最大为 4，length 为 0 和 length 为 4 的效果是完全相同的。
>
> 当 att 为字符串类型时，length 为 0 会发出整个完整的字符串，当 length 小于字符串长度时，发出指定的字节数（不是字符数），在 99% 的情况下，当 att 为字符串时，length 都是为 0。

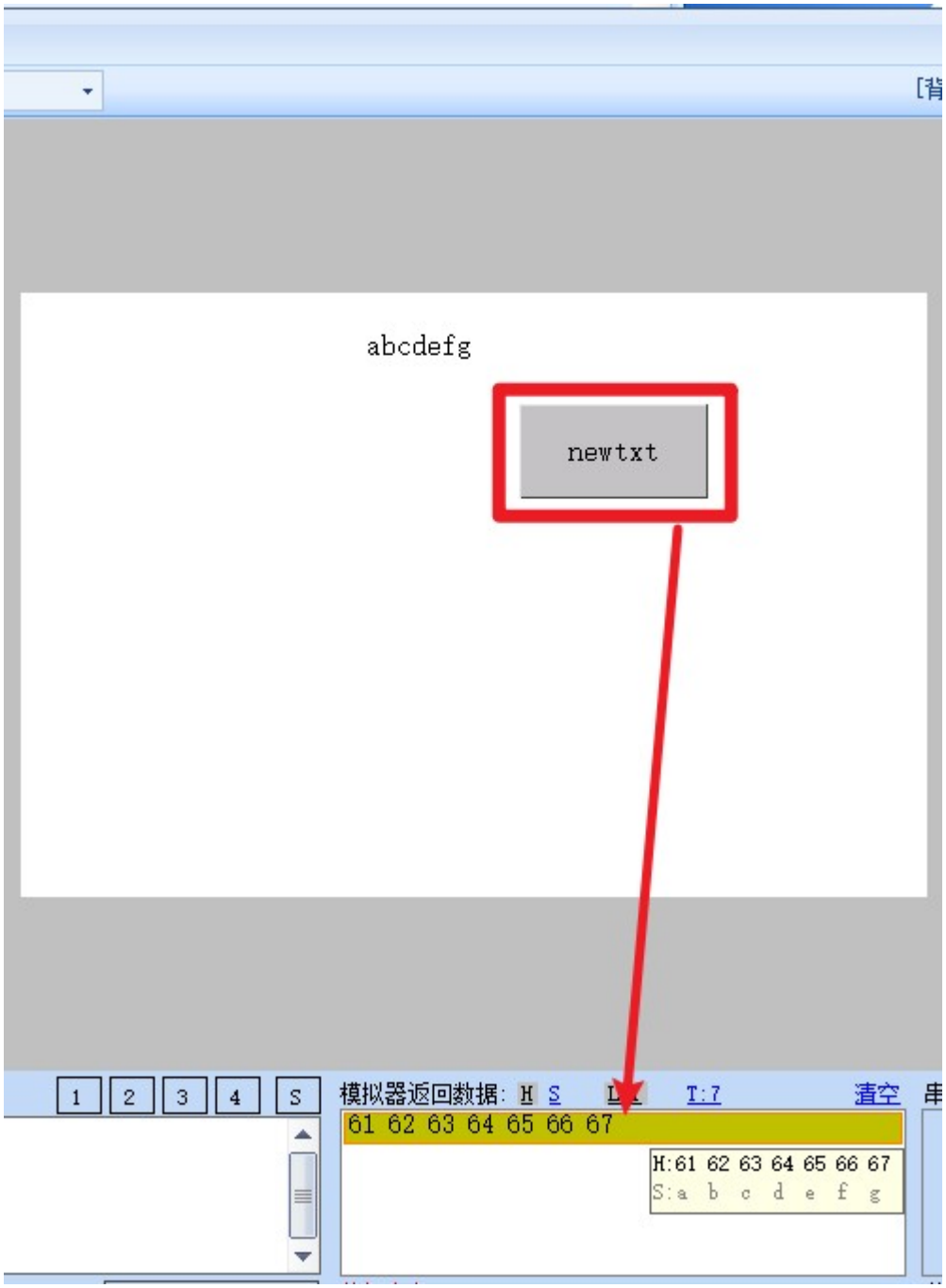
4.1.2.1 prints-示例 1

在按钮的按下事件（或弹起事件）中编写以下代码

```
//发送控件t0的txt属性值,长度为文本实际长度，也就是发出整个文本。
prints t0.txt,0
```



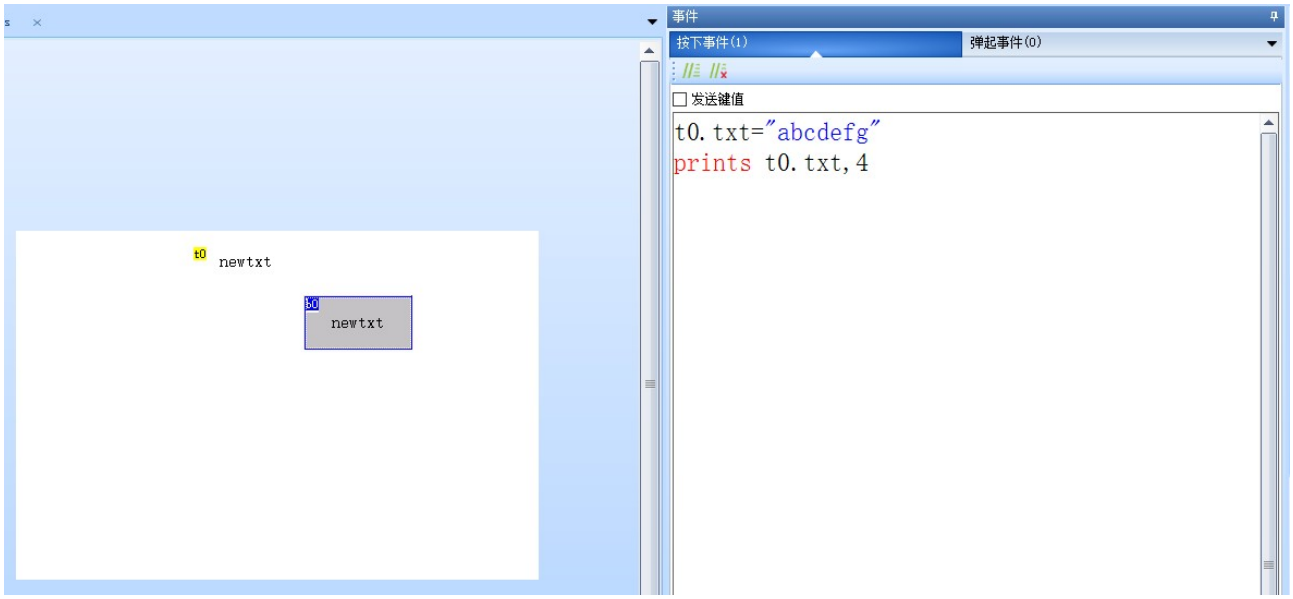
点击调试后，点击按钮，即可发出对应的数据



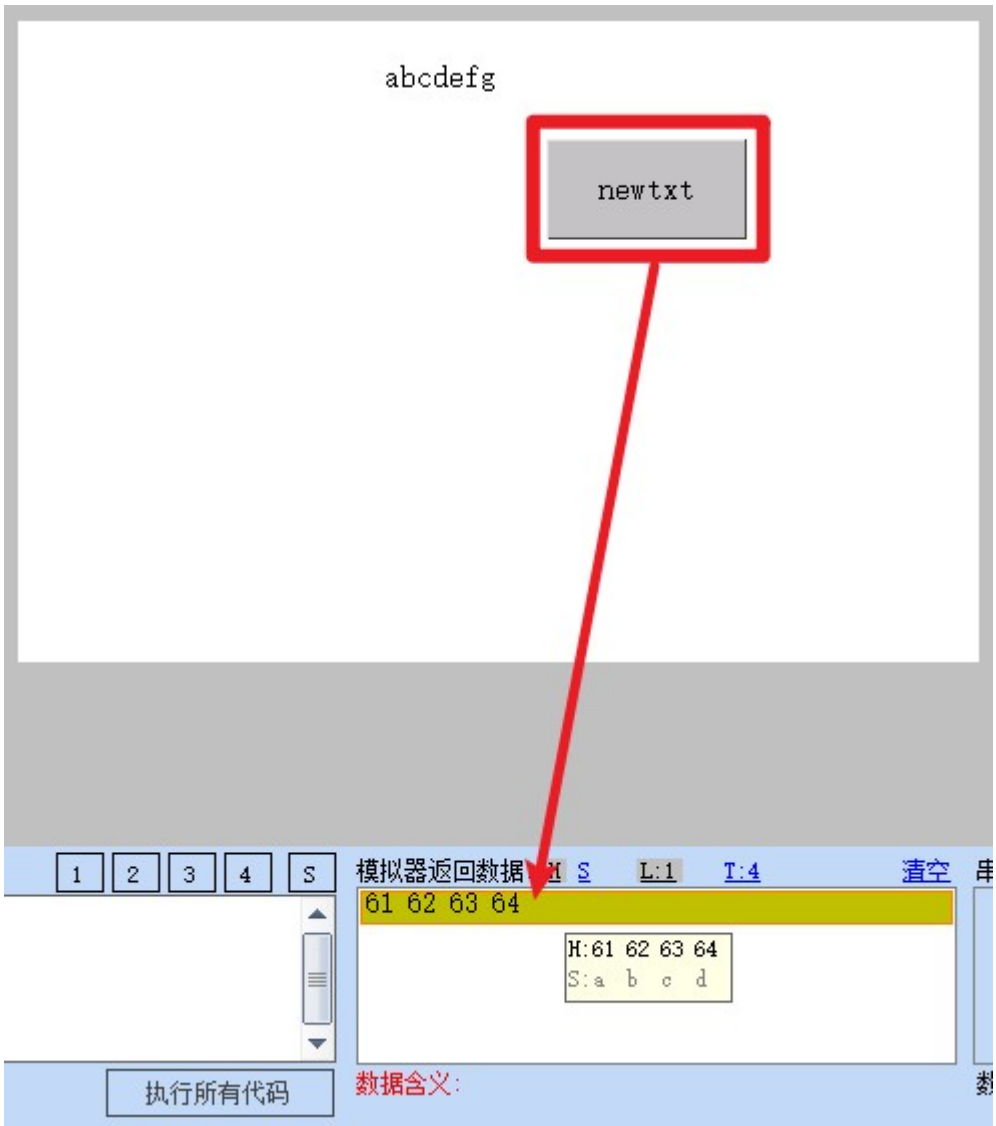
4.1.2.2 prints-示例 2

在按钮的按下事件（或弹起事件）中编写以下代码

```
//发送控件t0的txt属性值,长度为文本实际长度，也就是发出整个文本。
prints t0.txt,4
```



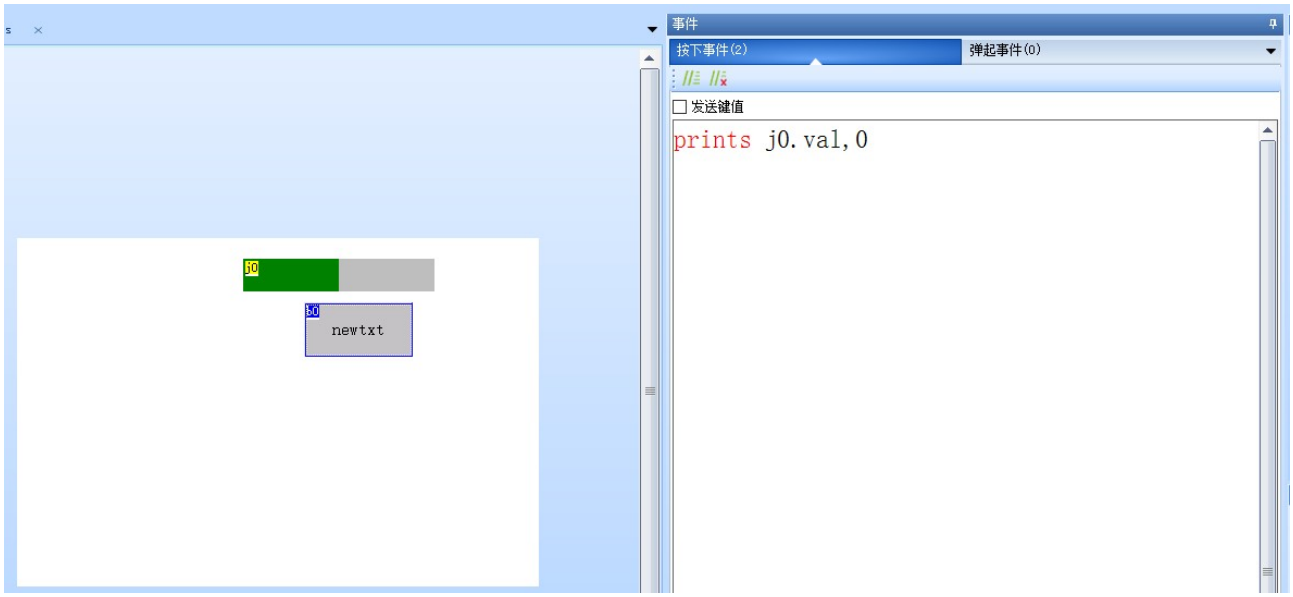
点击调试后，点击按钮，即可发出对应的数据



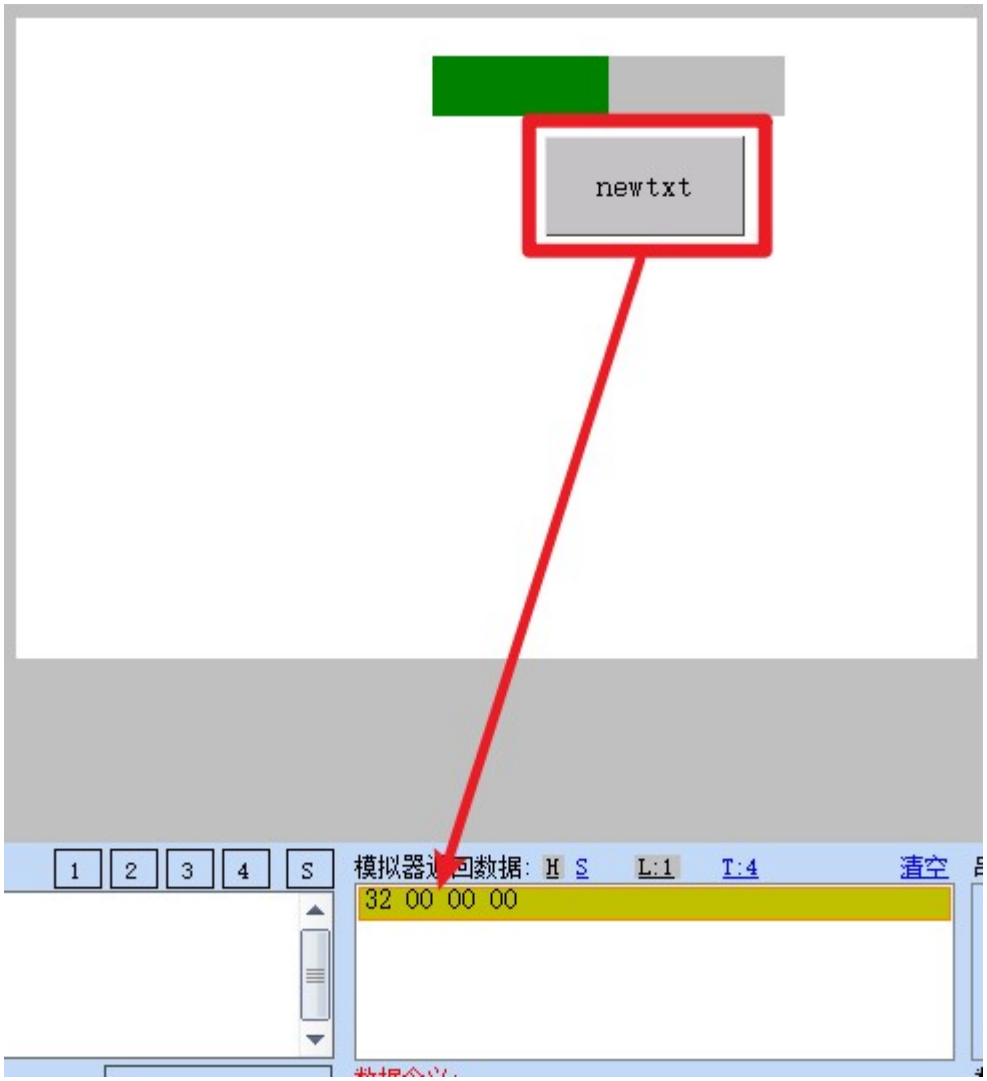
4.1.2.3 prints-示例 3

在按钮的按下事件（或弹起事件）中编写以下代码

```
//发送控件j0的val属性值,默认长度为4字节整形数据，小端模式储存
prints j0.val,0
```



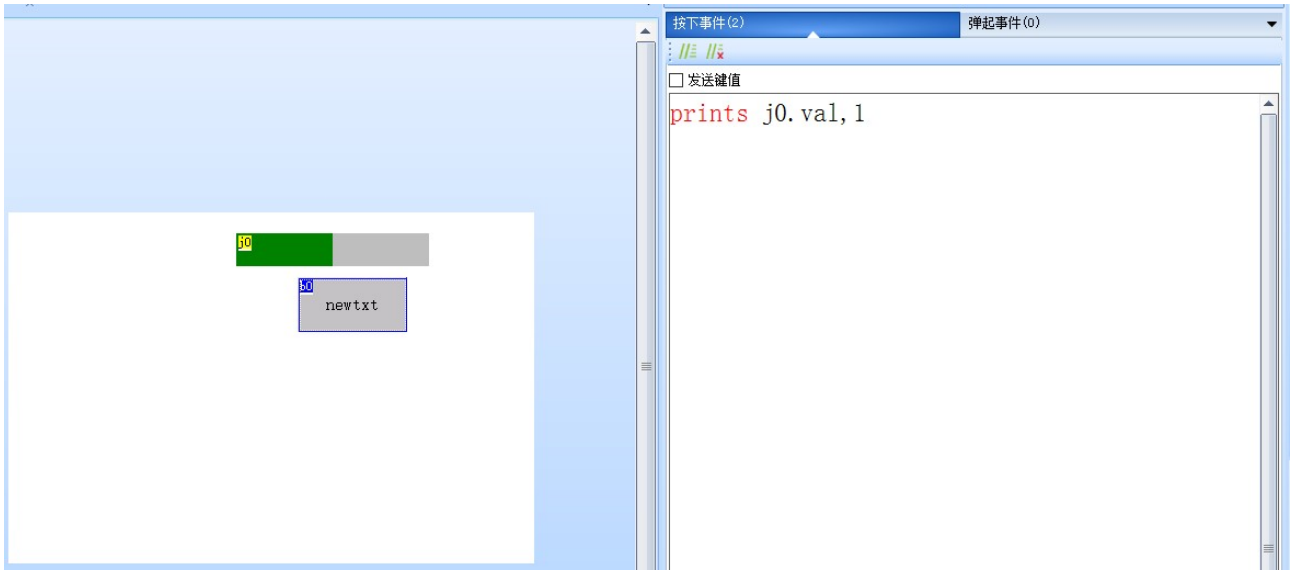
点击调试后，点击按钮，即可发出对应的数据



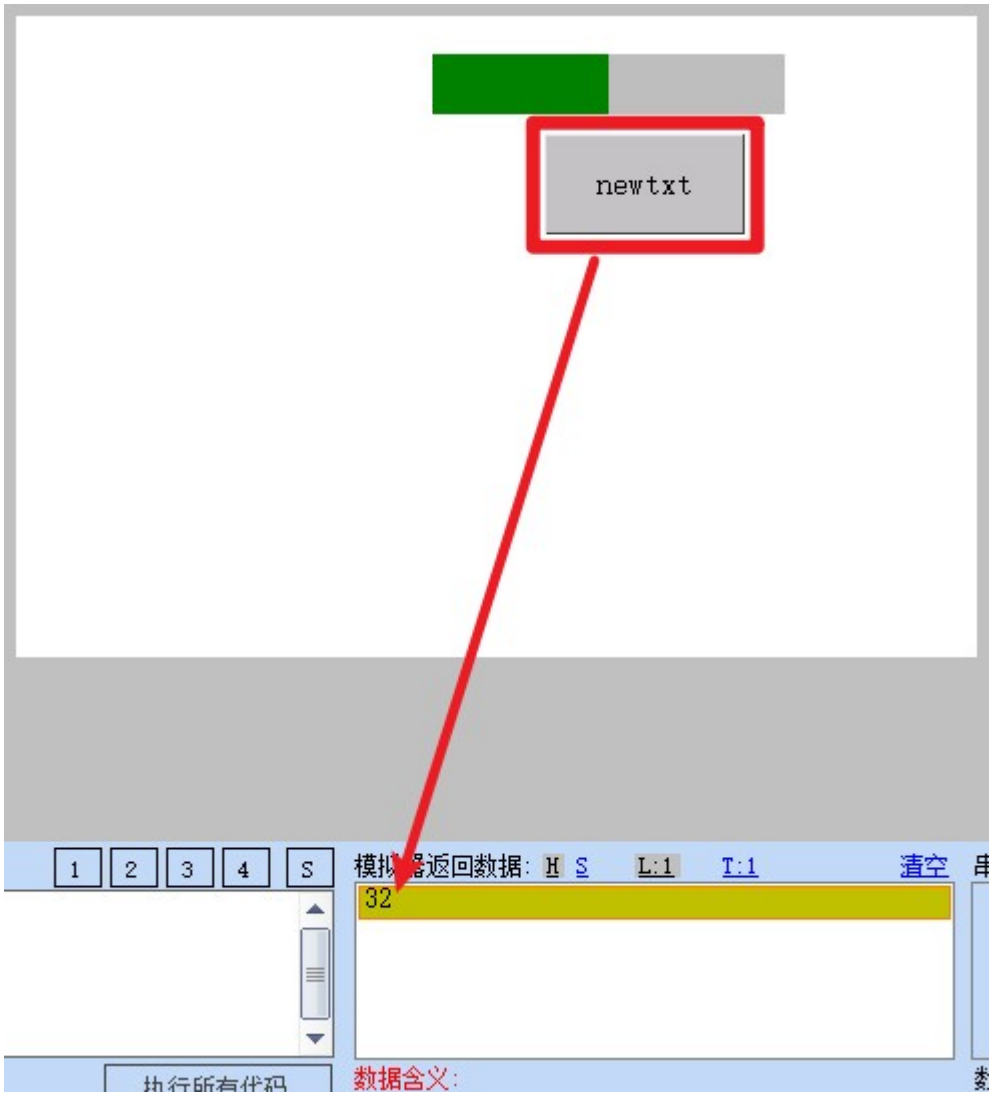
4.1.2.4 prints-示例 4

在按钮的按下事件（或弹起事件）中编写以下代码

```
//发送控件j0的val属性值,长度为1字节整形数据
prints j0.val,1
```



点击调试后，点击按钮，即可发出对应的数据



4.1.2.5 prints-示例 5

```
//发送常量字符串"123"即：0x31 0x32 0x33
prints "123",0
```

4.1.2.6 prints-示例 6

```
//发送常量数值：123 即：0x7b 0x00 0x00 0x00
prints 123,0
```

4.1.2.7 prints-示例 7

```
//发送常量数值：123的低1字节数据 即：0x7b
prints 123,1
```

4.1.2.8 prints-示例 8

```
//发送字符串"abcdef"
prints "abcdef",0
```

4.1.2.9 prints-示例 9

```
//发送中文字符串"你好哈哈"
prints "你好哈哈",0
```

4.1.2.10 prints-发送日期时间

```
//拼接字符串并发送
covx rtc0,t0.txt,0,0
covx rtc1,t1.txt,0,0
covx rtc2,t2.txt,0,0
covx rtc3,t3.txt,0,0
covx rtc4,t4.txt,0,0
covx rtc5,t5.txt,0,0
sendStr.txt=t0.txt+"年"+t1.txt+"月"+t2.txt+"日"+t3.txt+"时"+t4.txt+"分"+t5.txt+"秒"
prints sendStr.txt,0
```

注意： 使用 prints 发送的变量为字符串类型时，设备直接返回字符串内码，如果是数值类型 (如进度条的 val 属性) 设备直接返回变量的 4 字节整形数据 (Hex 数据，储存方式为小端模式，即低位在前)。

使用 prints 指令获取数据的时候，设备仅仅只发送数据内容，没有起始标示符，也没有结束符。

prints 指令可以配合 printh 指令在前面加一段自定义标示来告诉单片机此变量是属于哪个控件的。

prints 指令和 get 指令很类似, 区别是 get 发送的数据带了起始标示符（0x70 或 0x71）和结束符 (0xff 0xff 0xff), 而 prints 没有, 不过 prints 可以在后面继续用 printh 语句来加任何自定义标识符。

从串口打印一个 Hex 请使用 printh，点击查看详细资料*printh-从串口打印 16 进制* (page 586)

4.1.2.11 prints 指令-样例工程下载

演示工程下载链接：

《prints 指令》演示工程下载

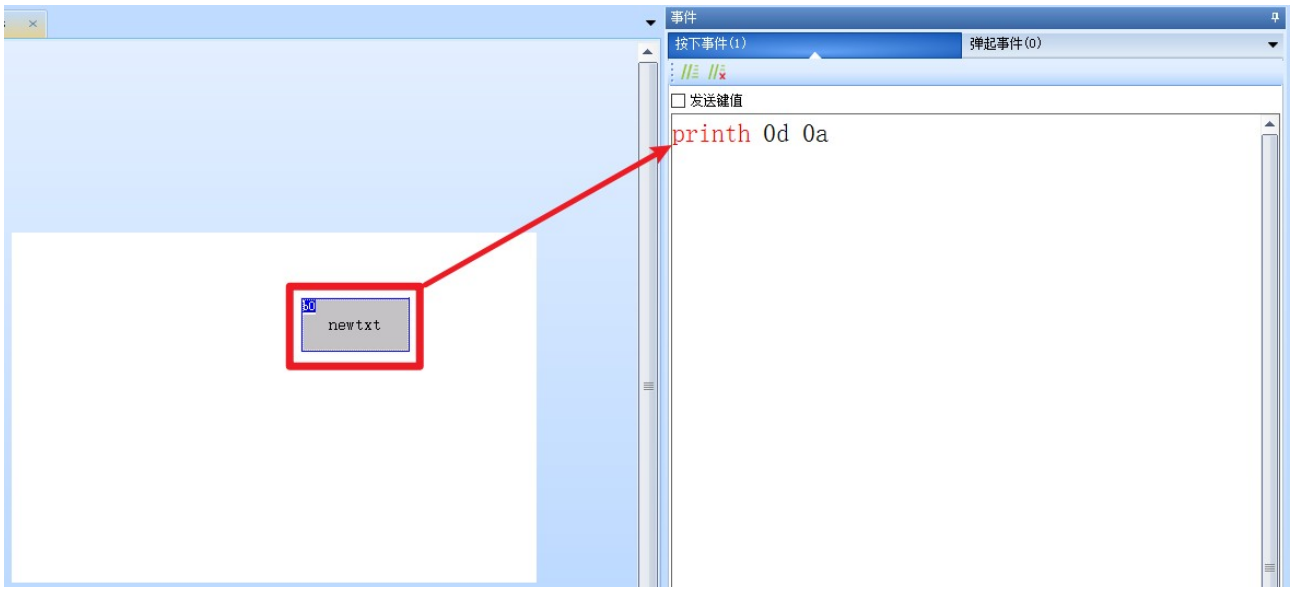
4.1.3 printh-从串口打印 16 进制

```
printh hex

hex:需要发送的固定的16进制，hex可以是多个16进制，必须写成偶数个的形式

正确：printh 0d 0a

错误：printh d a

正确：printh 05 03

错误：printh 5 3
```

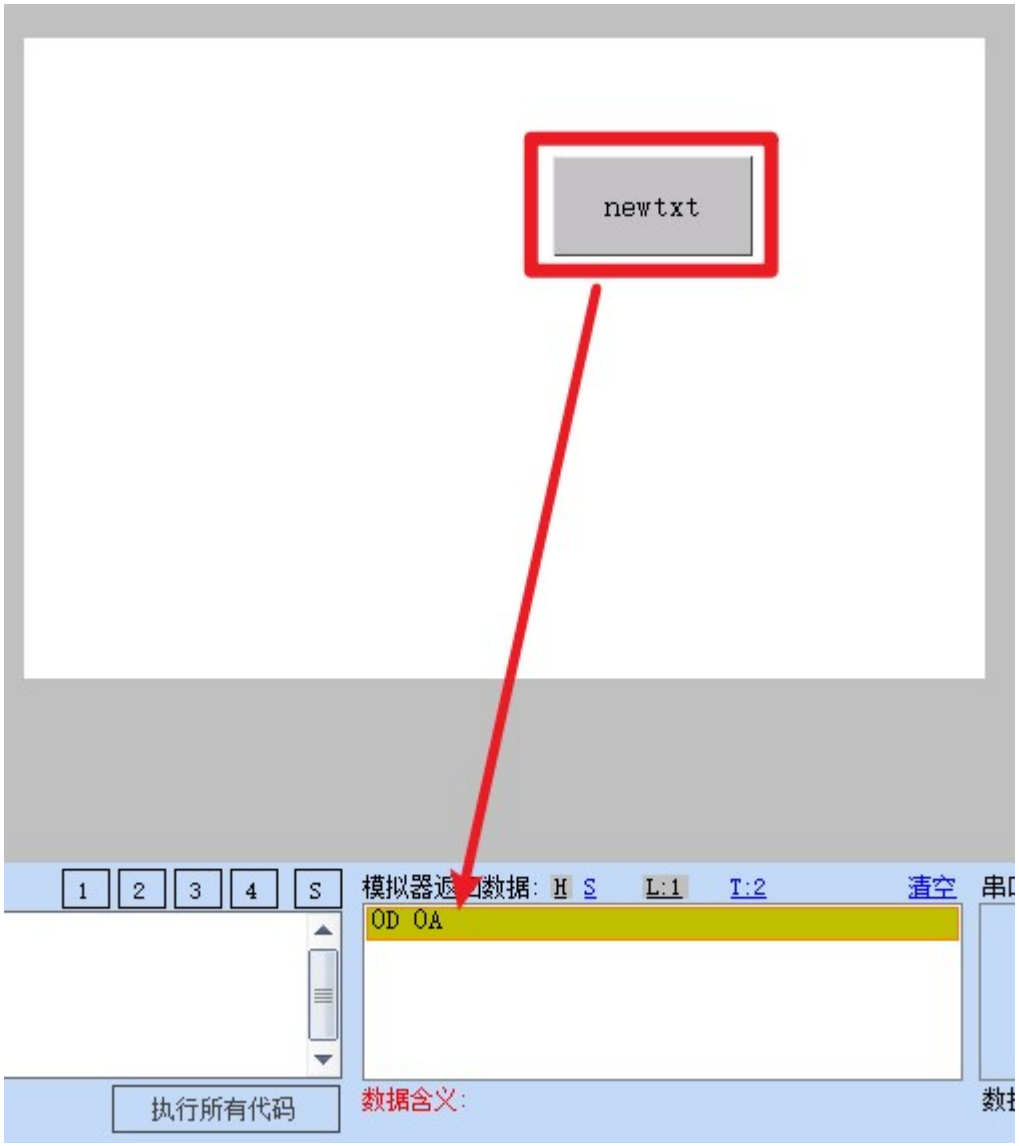
4.1.3.1 printh-示例 1

在按钮的按下事件（或弹起事件）中编写以下代码

```
//在音频通道0上播放ID为1的音频文件，循环播放
printh 0d 0a
```



点击调试后，点击按钮，即可发出对应的数据

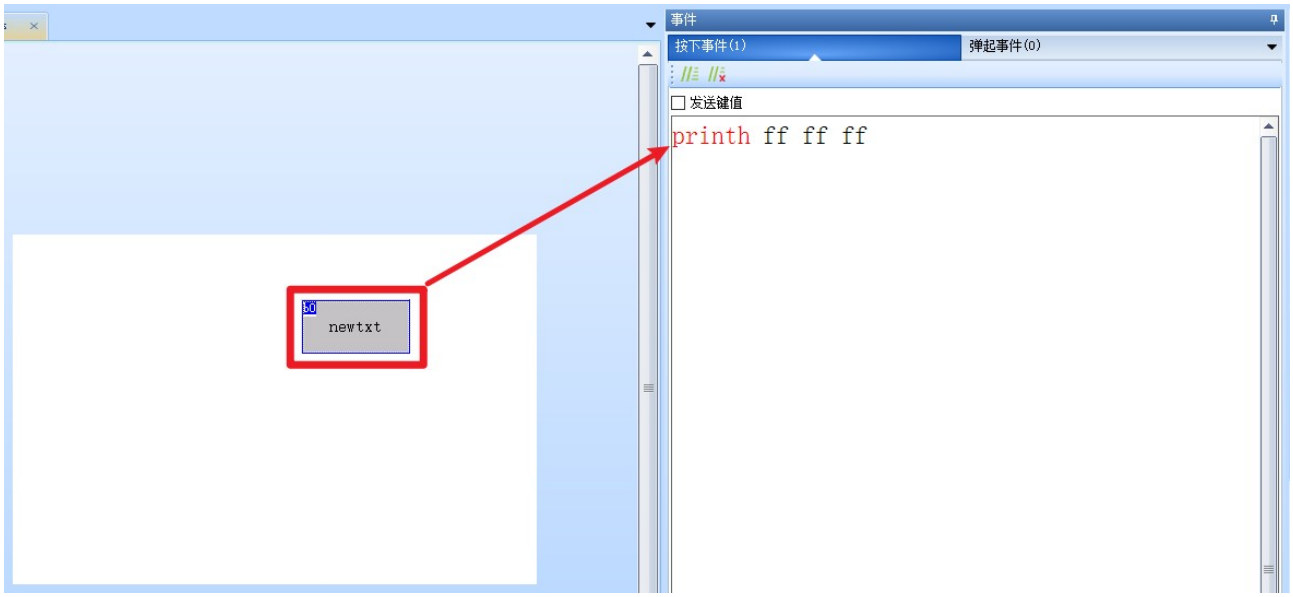


注意： 请注意不能写成 printh d a ， printh 后面的数据必须将长度补全为两位，也就是 printh 0d 0a
例如发送数值：1，不能写成 printh 1，必须写成 printh 01

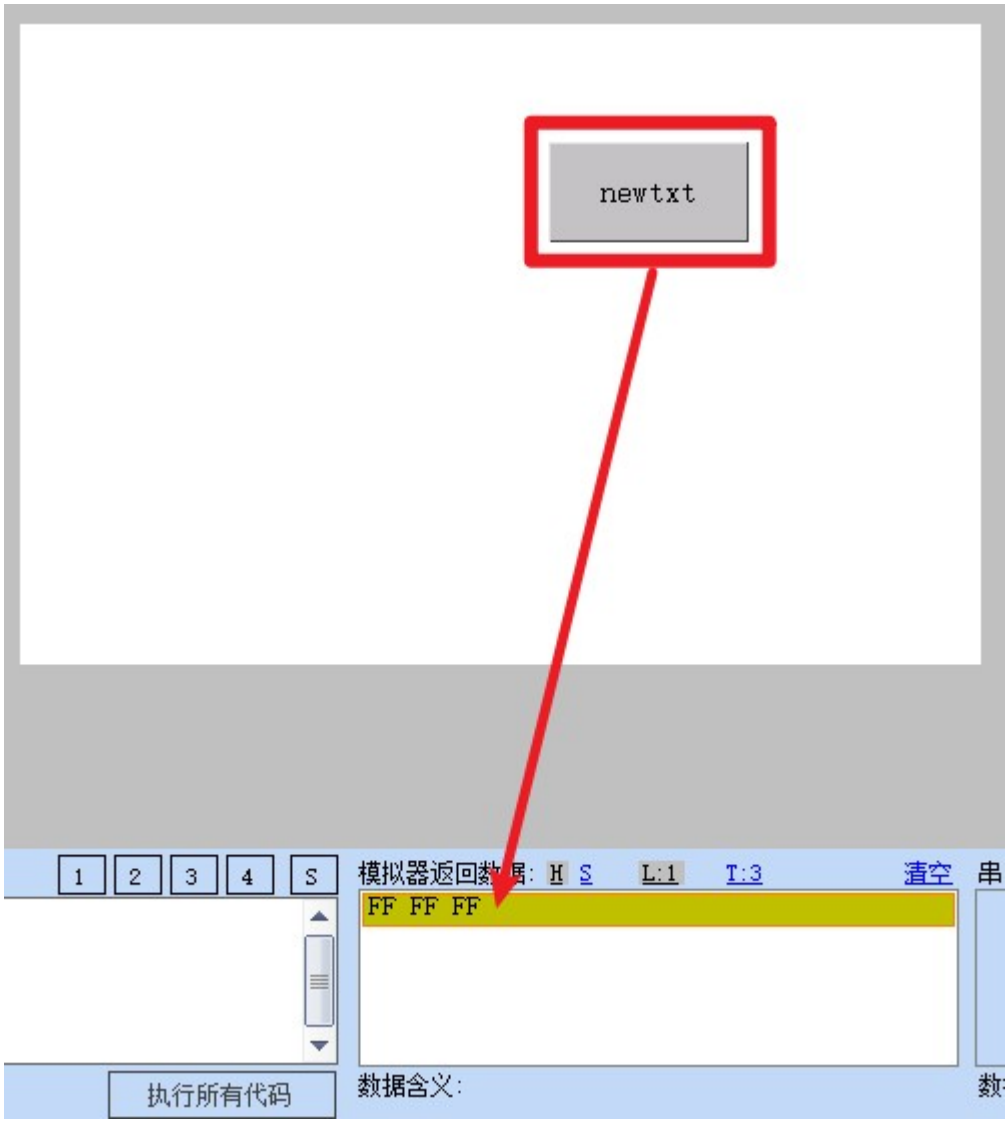
4.1.3.2 printh-示例 2: 发送 3 个 0xff

在按钮的按下事件（或弹起事件）中编写以下代码

```
//串口屏发送 0xff 0xff 0xff，也可以写成 printh FF FF FF
printh ff ff ff
```

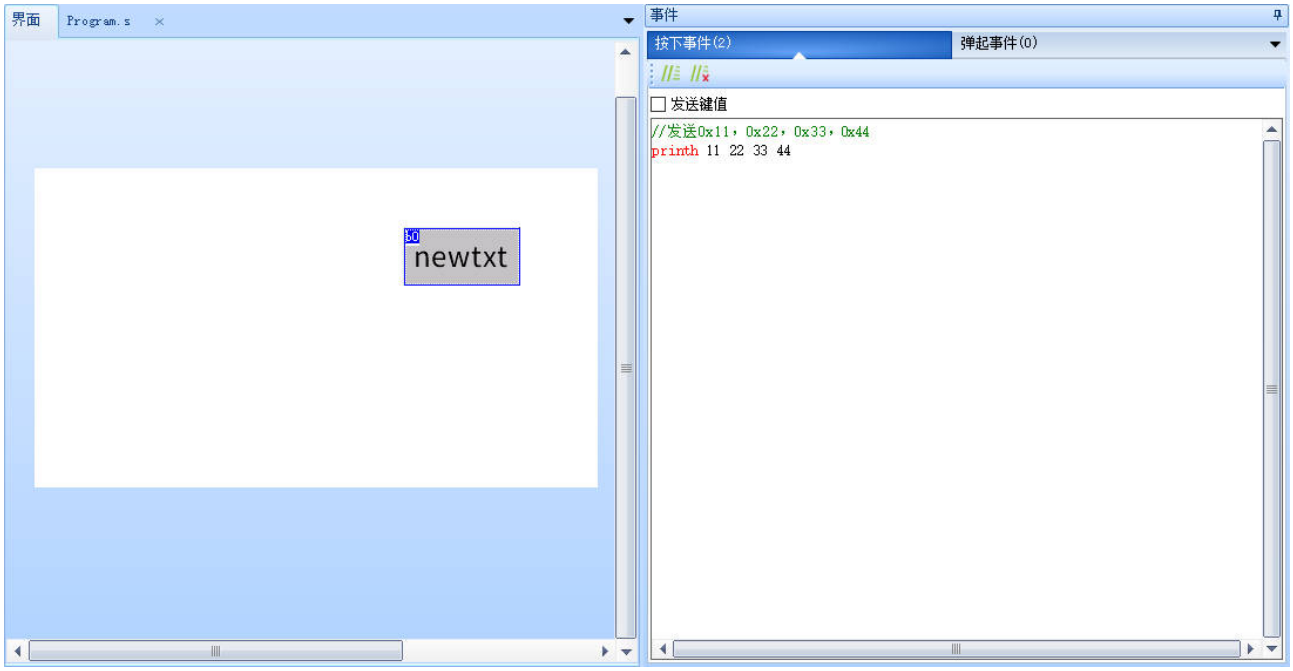


点击调试后，点击按钮，即可发出对应的数据



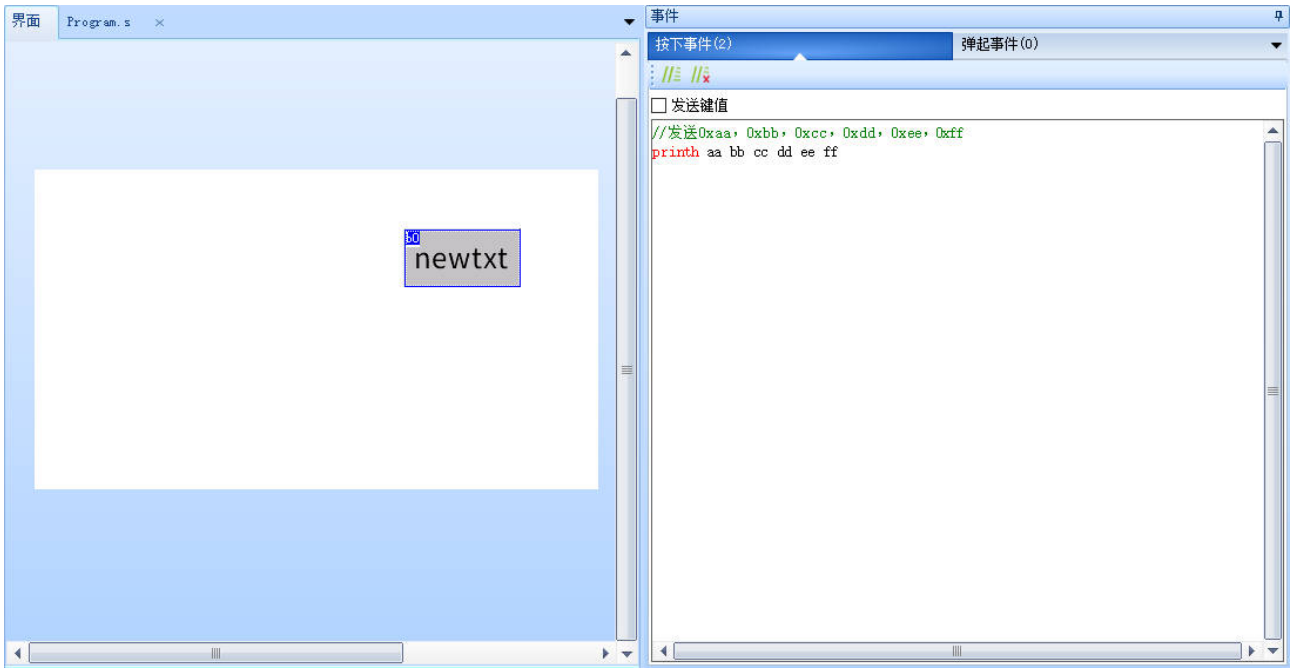
4.1.3.3 printh-示例 3

```
//发送0x11, 0x22, 0x33, 0x44
printh 11 22 33 44
```



4.1.3.4 printh-示例 4

```
//发送0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff
printh aa bb cc dd ee ff
```



注意: 需要发送多个固定的 16 进制数据时，建议用一个 printh 发送即可，不建议使用多个 printh 进行发送

使用 printh 指令发送数据的时候，设备仅仅只发送指定的字符

参数中每组字符间必须有且只能有一个空格隔开，16 进制的字符串表达式大小写均支持。

printh 只能发送固定格式的 16 进制数据, 不能发送变量, 变量需要使用 prints 指令*prints-从串口打印一个变量/常量* (page 578)

4.1.3.5 printh 指令-样例工程下载

演示工程下载链接：

《printh 指令》演示工程下载

4.1.4 click-激活控件的按下/弹起事件

激活当前页面控件的按下或弹起事件，不支持跨页面激活控件的按下或弹起事件

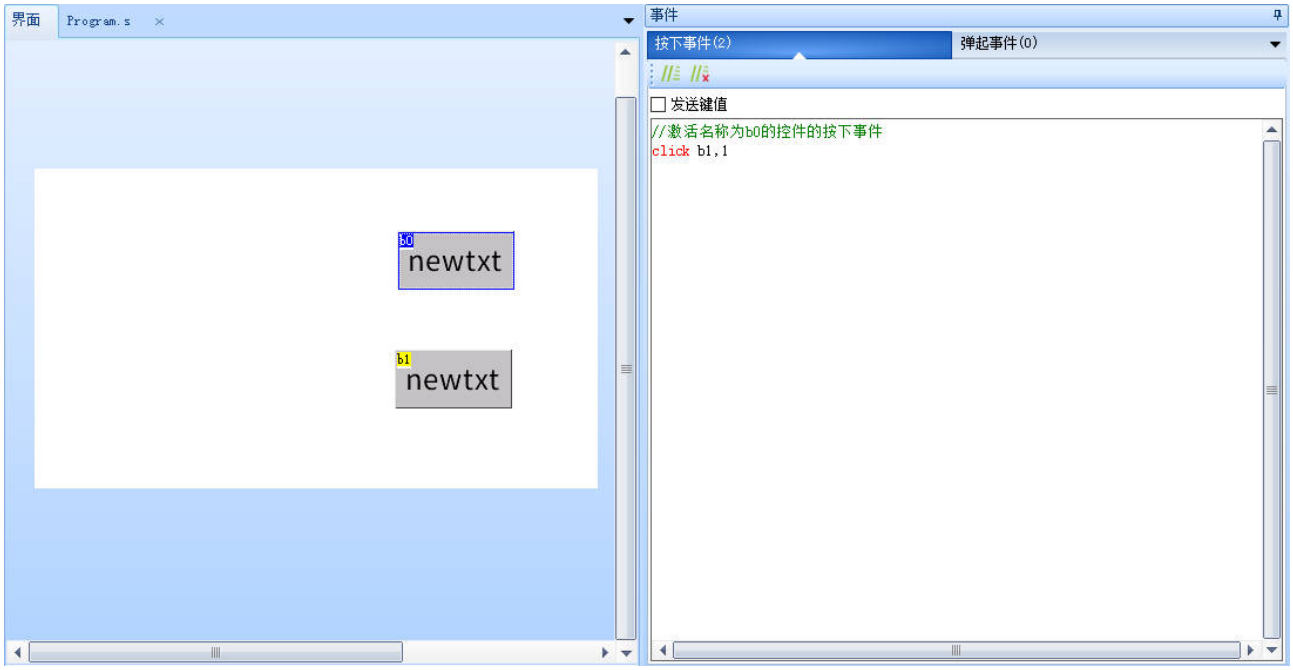
控件被隐藏后，无法使用手指去触发控件，但是仍然可以通过 click 指令触发。

请勿 click 控件自身，否则会因为递归过深导致问题。

```
click obj,event

obj:控件ID或控件名称

event:事件序号:0为弹起，1为按下
```

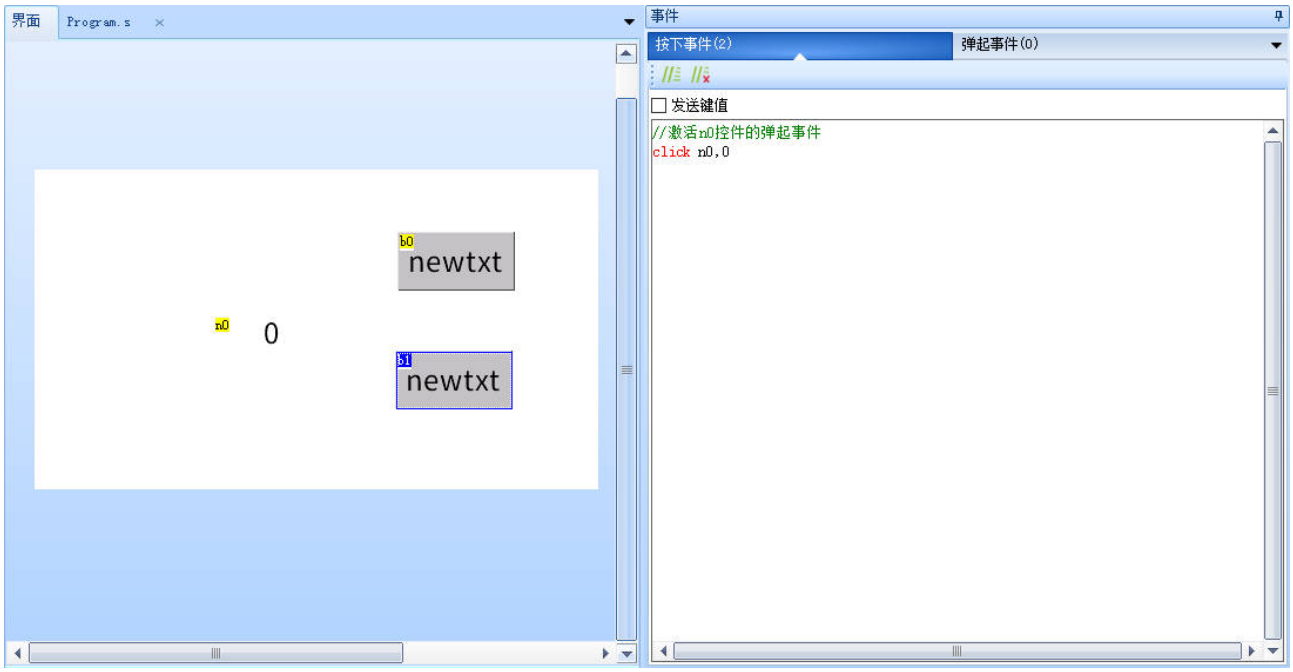
4.1.4.1 click-示例 1

```
//激活名称为b0的控件的按下事件
click b0,1
```



4.1.4.2 click-示例 2

```
//激活n0控件的弹起事件
click n0,0
```



注意： 控件的按下/弹起事件在屏幕上触摸的时候会自动激活，如果在没有触摸的情况下想要手动激活，就使用 click 指令即可。

由于串口屏上没有函数的概念，因此有大量重复的代码需要调用时，可以将代码写在触摸热区内，然后用 click 指令去触发

click 指令触发控件时，不会导致触摸捕捉控件被触发

4.1.4.3 click-c 语言示例

单片机通过串口触发按键按下并在 100 毫秒后弹起

```
//激活名称为b0的控件的按下事件,click指令不支持跨页面
printf("click b0,1\xff\xff\xff");

//单片机延时100ms，如果不延时，指令速度太快，看不到按下弹起的效果
delay_ms(100);

//激活名称为b0的控件的弹起事件,click指令不支持跨页面
printf("click b0,0\xff\xff\xff");
```

4.1.4.4 click 指令-相关链接

触摸热区控件 (page 448)

4.1.4.5 click 指令-样例工程下载

演示工程下载链接：

《click 指令》演示工程下载

4.1.5 vis-隐藏/显示控件

隐藏/显示当前页面控件，不支持跨页面显示或隐藏控件

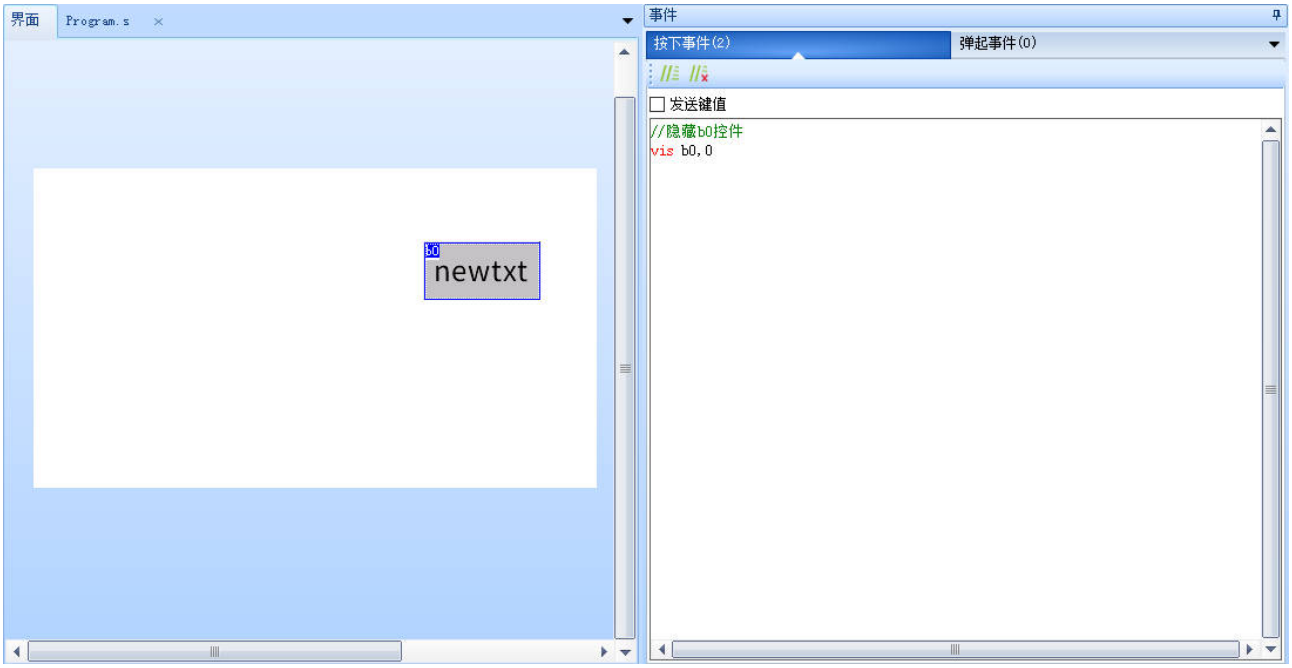
控件被隐藏后，无法使用手指去触发控件，但是仍然可以通过 click 指令触发。

被设置为全局的控件，通过 vis 设置了隐藏后，切换页面后重新切换回来，依然隐藏

```
vis obj,state

obj:控件名称或控件ID

state:状态(0或1)
```

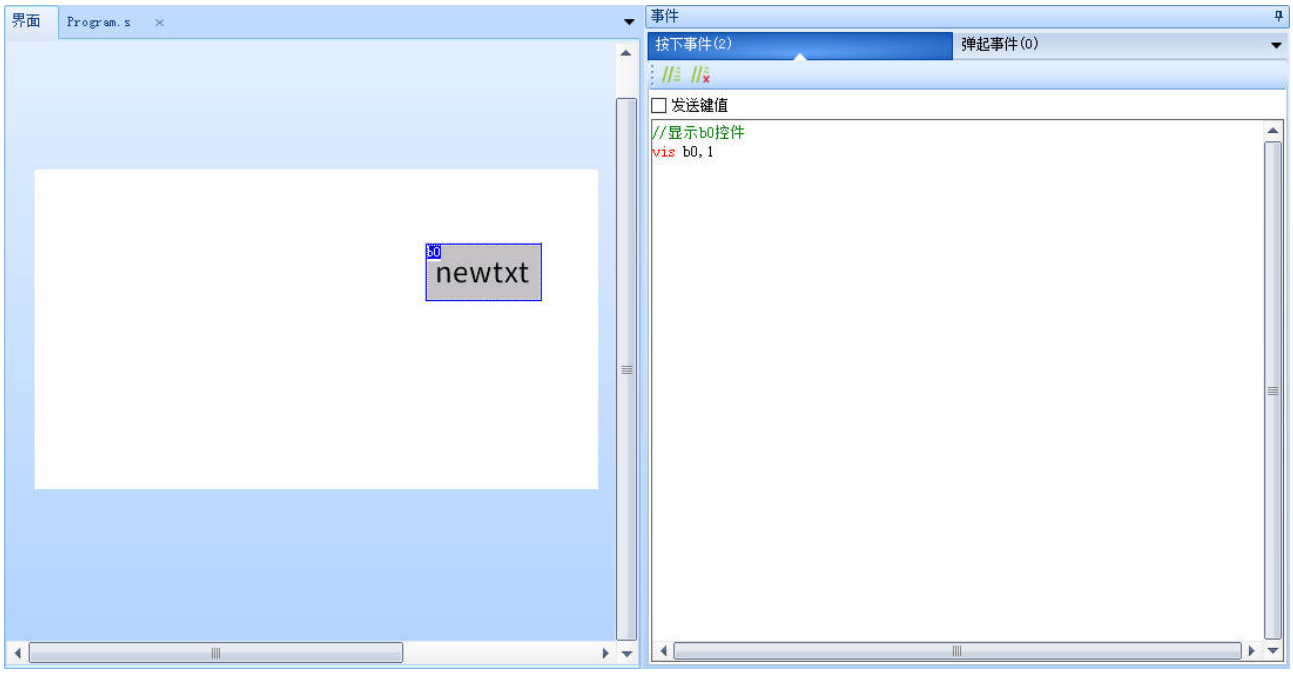
4.1.5.1 vis-示例 1

```
//隐藏b0控件
vis b0,0
```



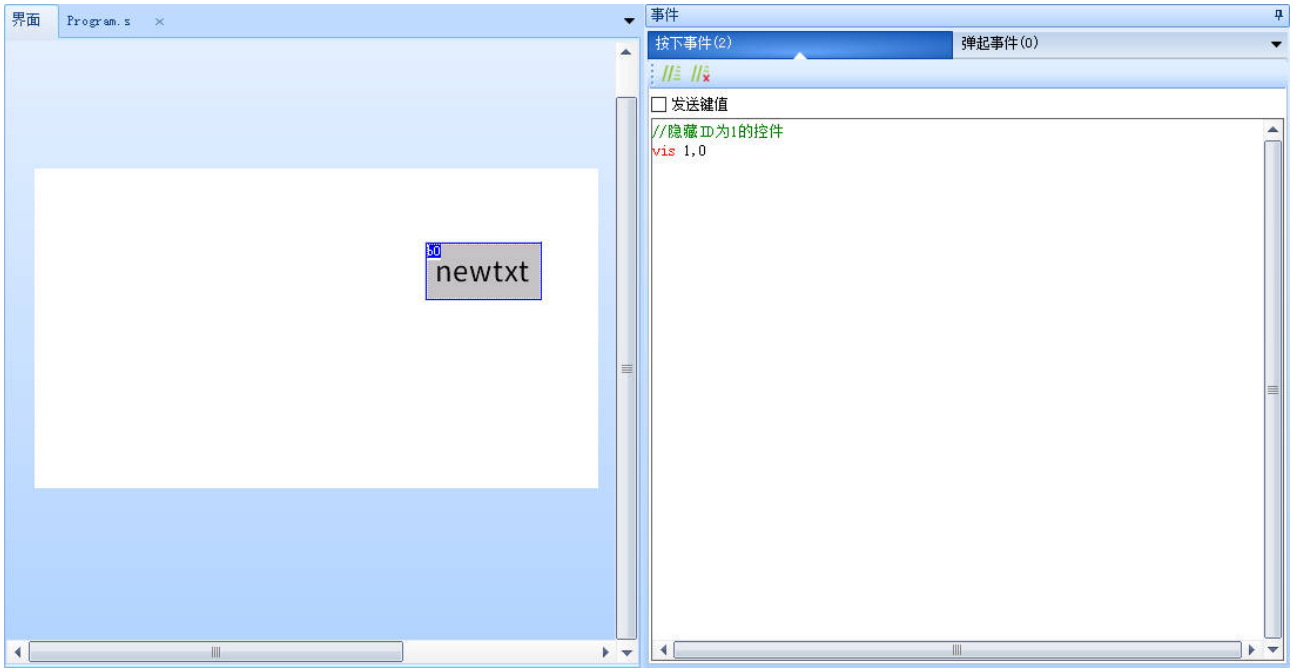
4.1.5.2 vis-示例 2

```
//显示b0控件
vis b0,1
```



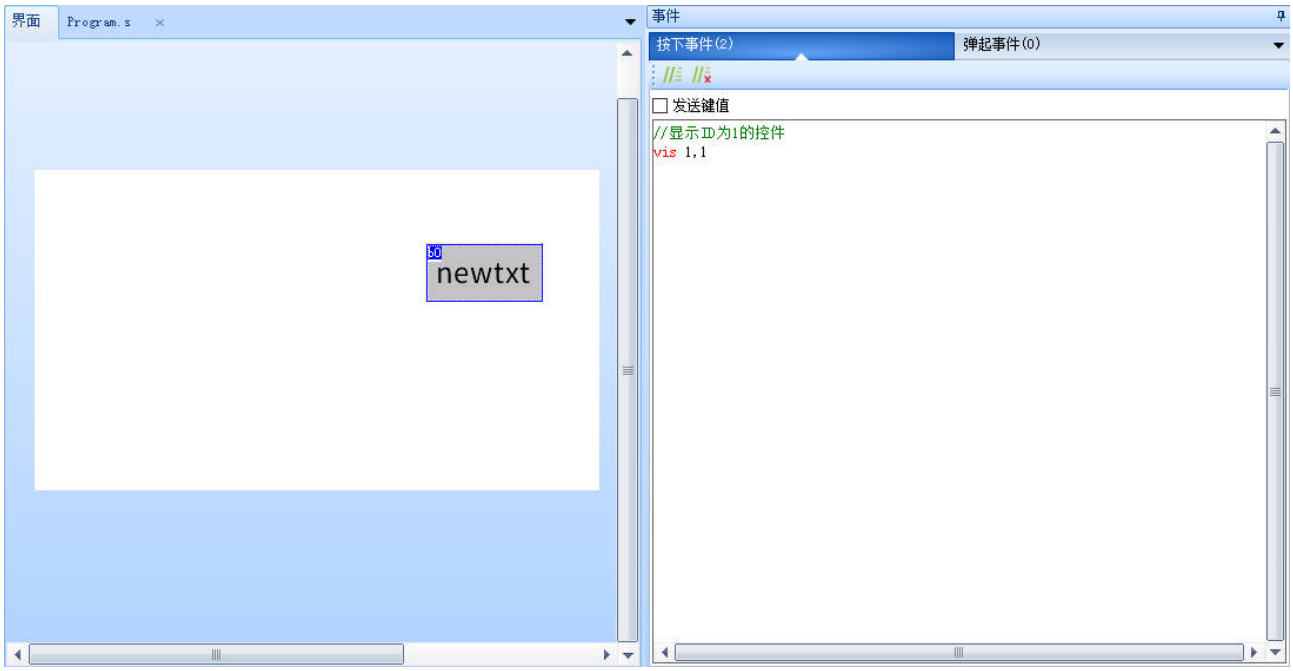
4.1.5.3 vis-示例 3

```
//隐藏ID为1的控件
 vis 1,0
```



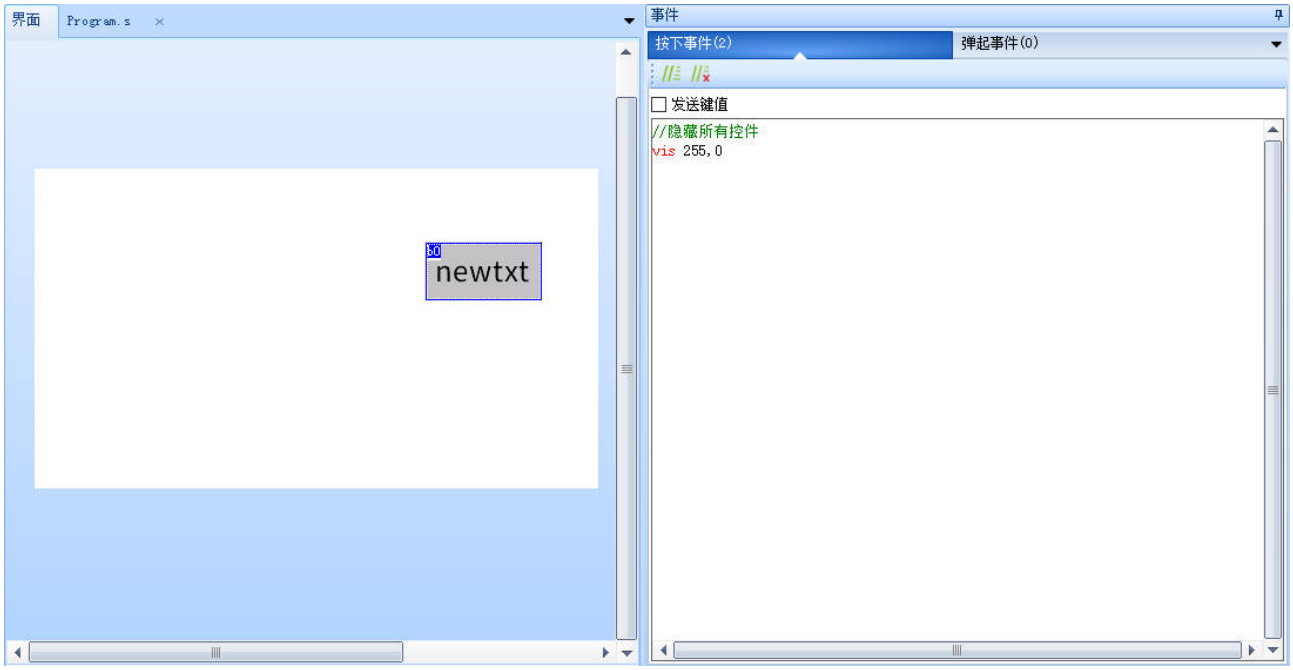
4.1.5.4 vis-示例 4

```
//显示ID为1的控件
vis 1,1
```



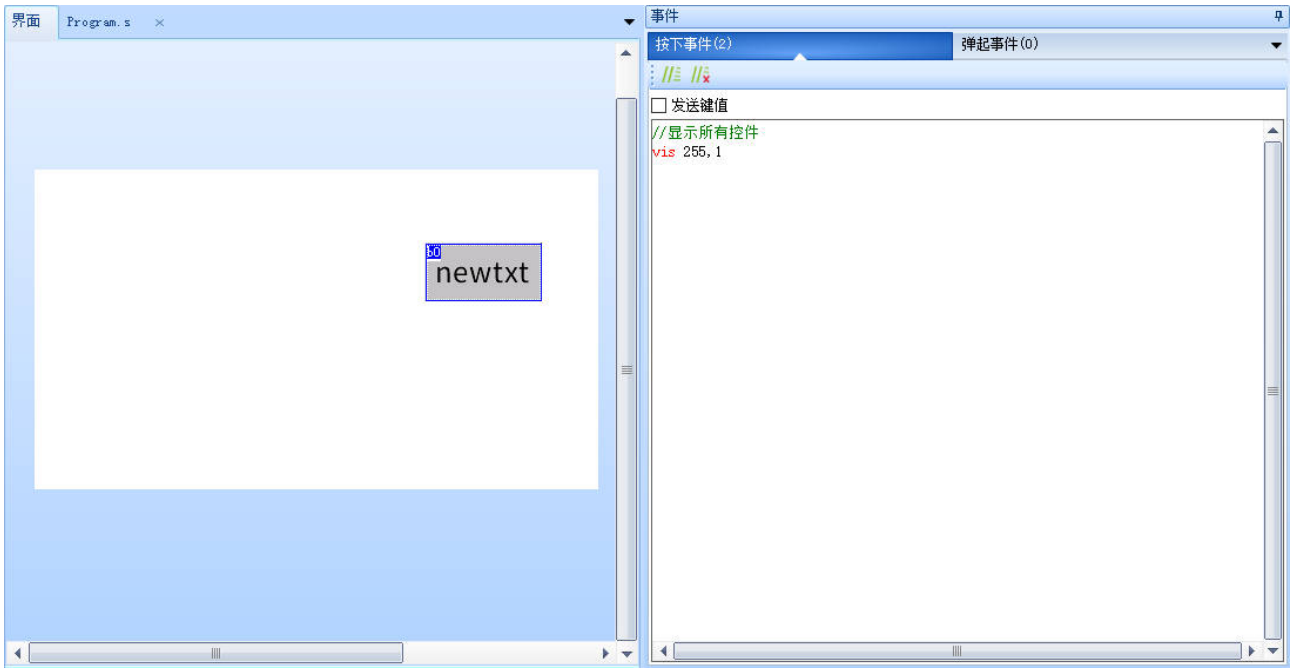
4.1.5.5 vis-示例 5

```
//隐藏所有控件
 vis 255,0
```



4.1.5.6 vis-示例 6

```
//显示所有控件
vis 255,1
```



注意： 第一个参数为 255 表示当前页面所有控件，例:vis 255,0(隐藏当前页面所有控件) vis 255,1(显示当前页面所有控件)。

使用了 vis 命令将控件隐藏了之后，控件 1 不就看不到也摸不到了，但是可以使用 click 命令触发

4.1.5.7 vis 指令-相关链接

如何实现跨页面隐藏或显示控件:

可以利用全局变量来跨页面隐藏或显示控件

即用全局变量来记录控件的隐藏状态，在前初始化事件里去显示或者隐藏控件

4.1.5.8 vis 指令-样例工程下载

演示工程下载链接:

《vis 指令》演示工程下载

4.1.6 tsw-控件触摸使能

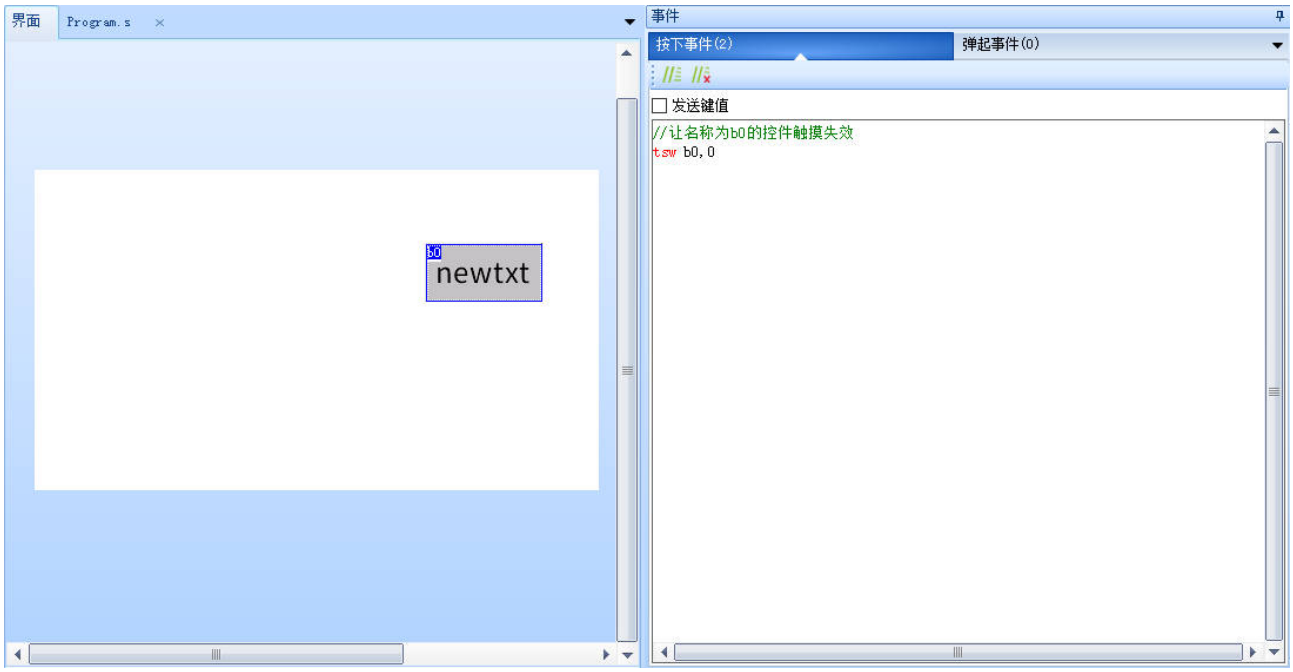
使能或失能当前页面控件，不支持跨页面使能或失能控件

被设置为全局的控件，通过 tsw 设置了触摸使能后，切换页面后重新切换回来，依然无法触摸

```
tsw obj,state

obj:控件名称或控件ID

state:状态(0或1)
```

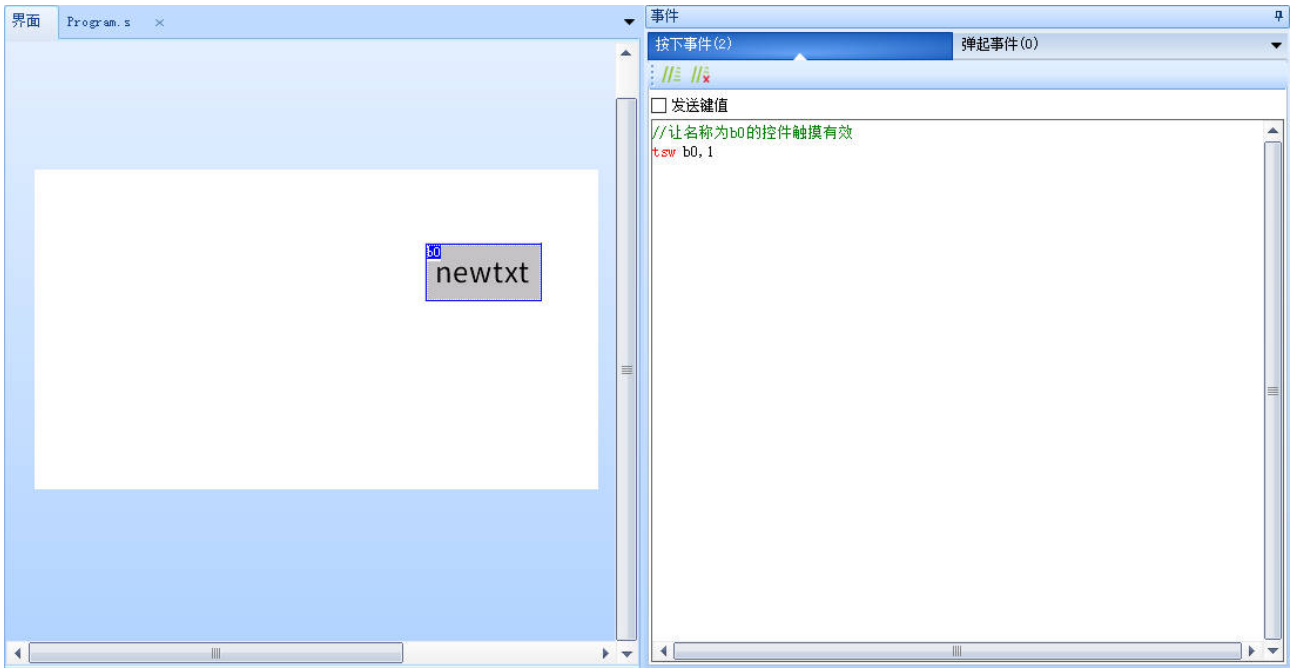
4.1.6.1 tsw-示例 1

```
//让名称为b0的控件触摸失效
tsw b0,0
```



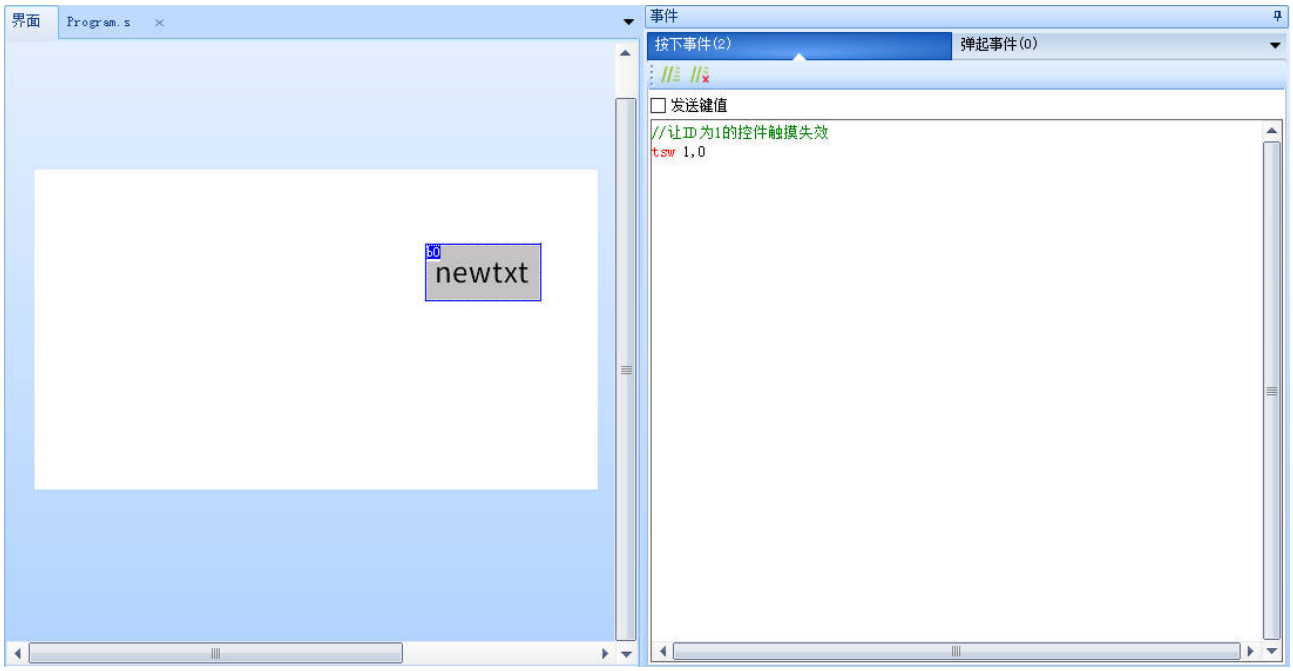
4.1.6.2 tsw-示例 2

```
//让名称为b0的控件触摸有效
tsw b0,1
```



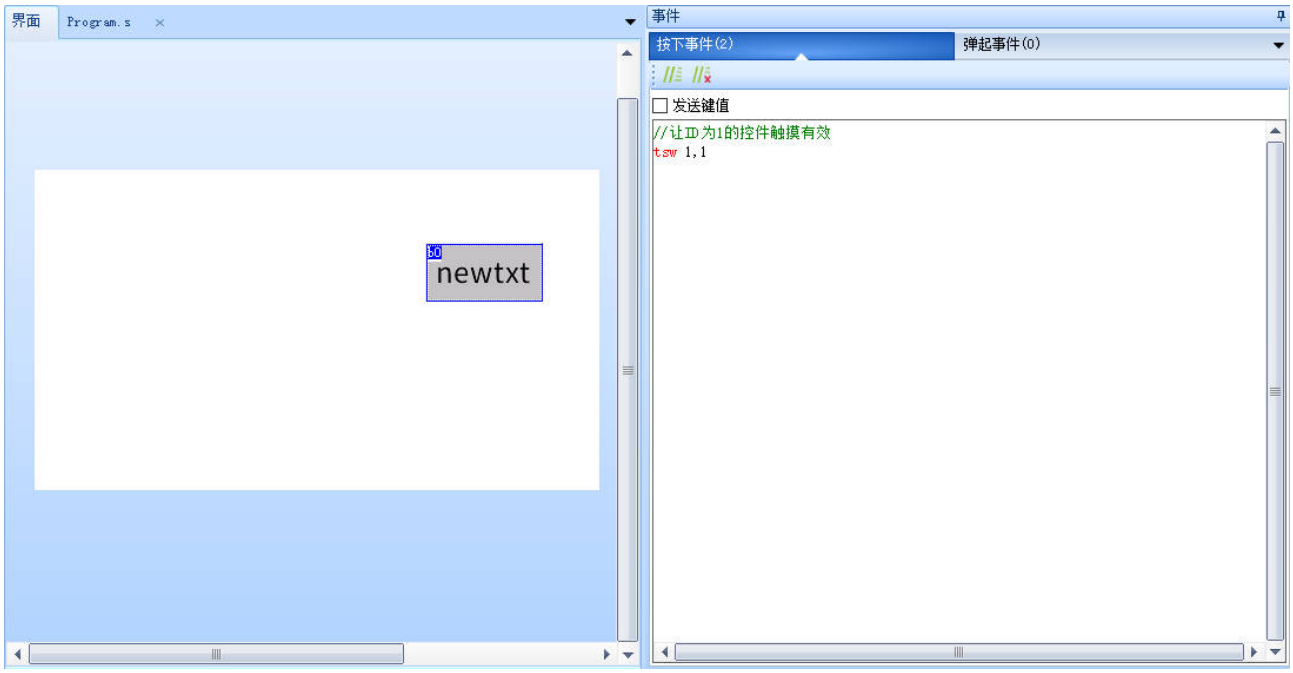
4.1.6.3 tsw-示例 3

```
//让ID为1的控件触摸失效
tsw 1,0
```



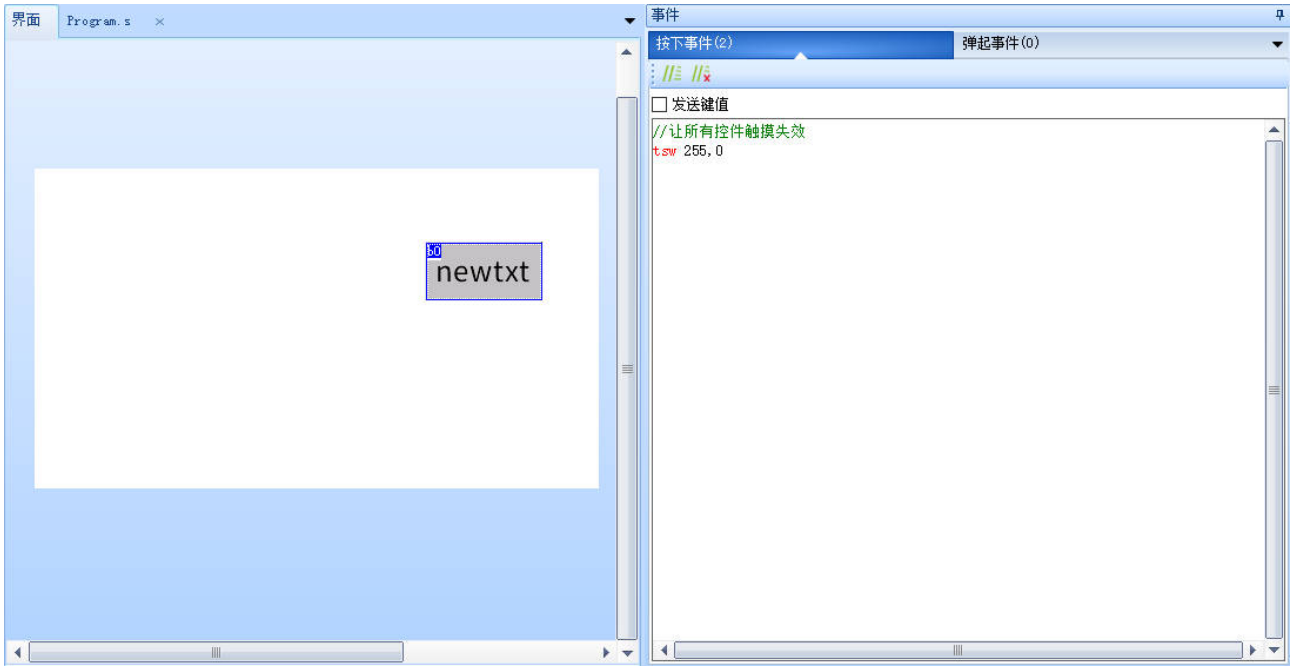
4.1.6.4 tsw-示例 4

```
//让ID为1的控件触摸有效
tsw 1,1
```



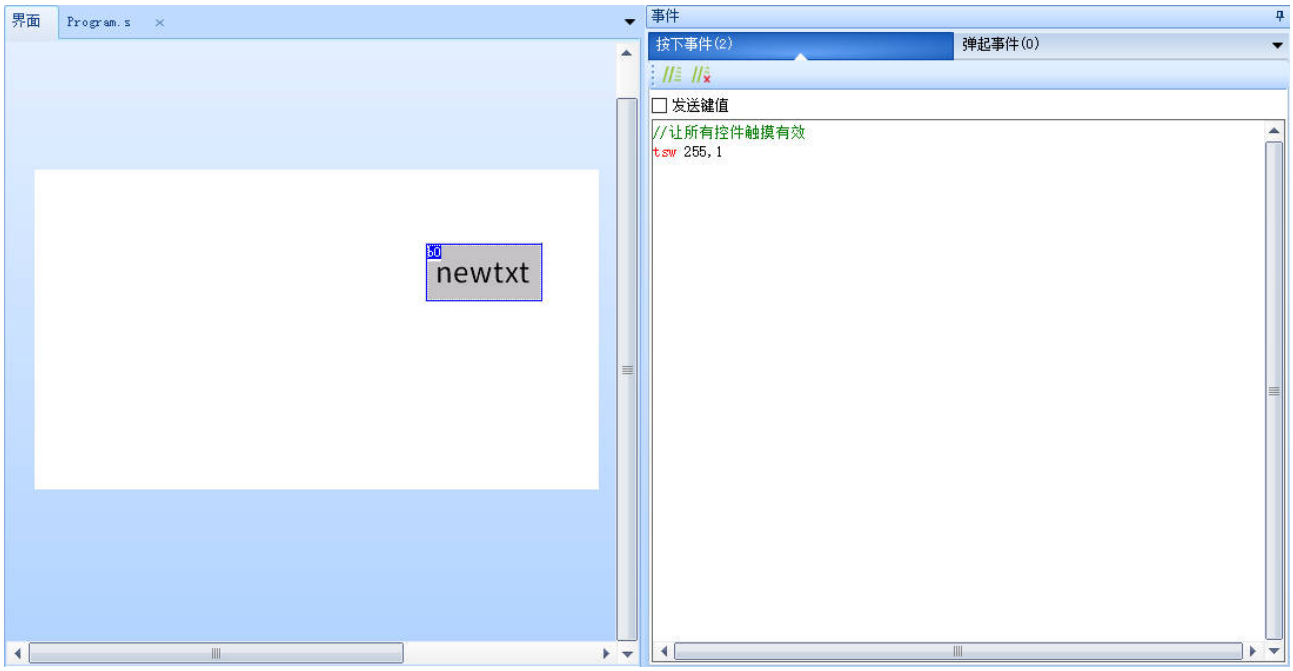
4.1.6.5 tsw-示例 5

```
//让所有控件触摸失效
tsw 255,0
```



4.1.6.6 tsw-示例 6

```
//让所有控件触摸有效
tsw 255,1
```



注意： 第一个参数为 255 表示当前页面所有控件，例:tsw 255,0(当前页面所有控件触摸失效) tsw 255,1(当前页面所有控件触摸有效，包括触摸捕捉控件)。

使用了 tsw 命令将控件失能了之后，控件就无法触摸了，但是可以使用 click 命令触发

4.1.6.7 tsw-使用详解

4.1.6.7.1 tsw-实现按键锁定功能

实现按键锁定功能 (page 1423)

4.1.6.8 如何实现跨页面使能或失能控件

可以利用全局变量来跨页面使能或失能控件
在前初始化事件中，根据全局变量的值来判断是否隐藏或者失能控件

4.1.6.9 tsw 指令-样例工程下载

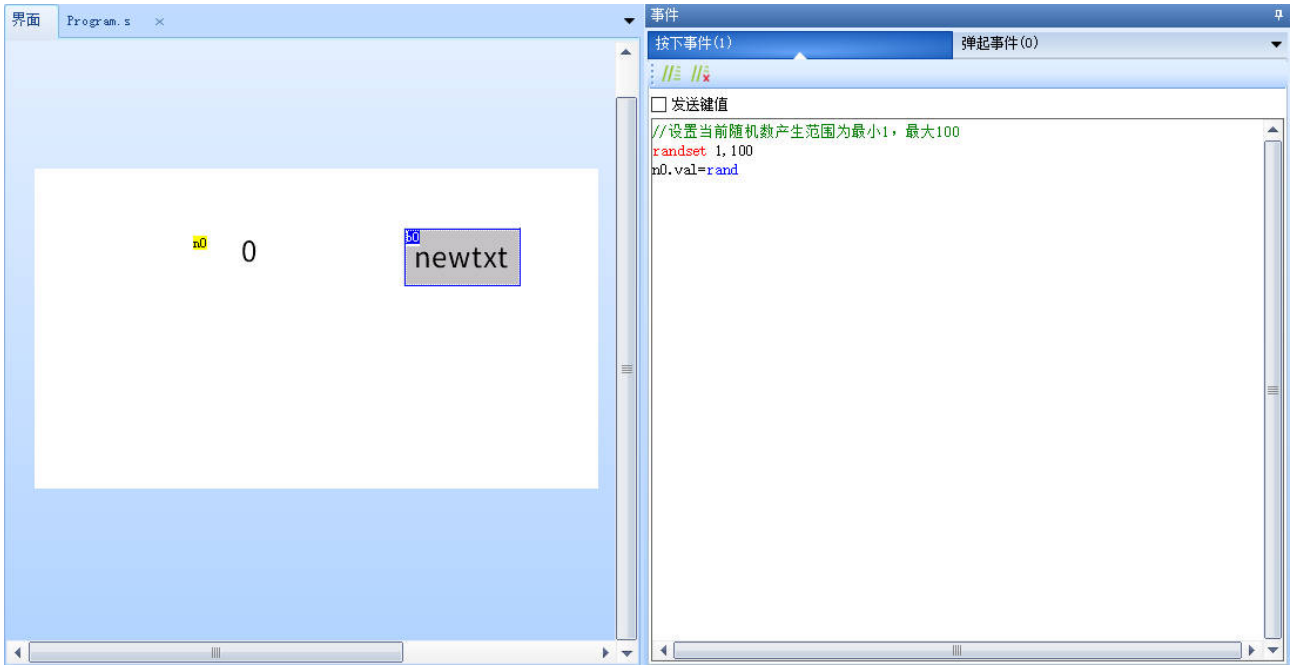
演示工程下载链接：
《tsw 指令》演示工程下载

4.1.7 randset-随机数范围设置

```
randset minval,maxval

minval:最小值

maxval:最大值
```

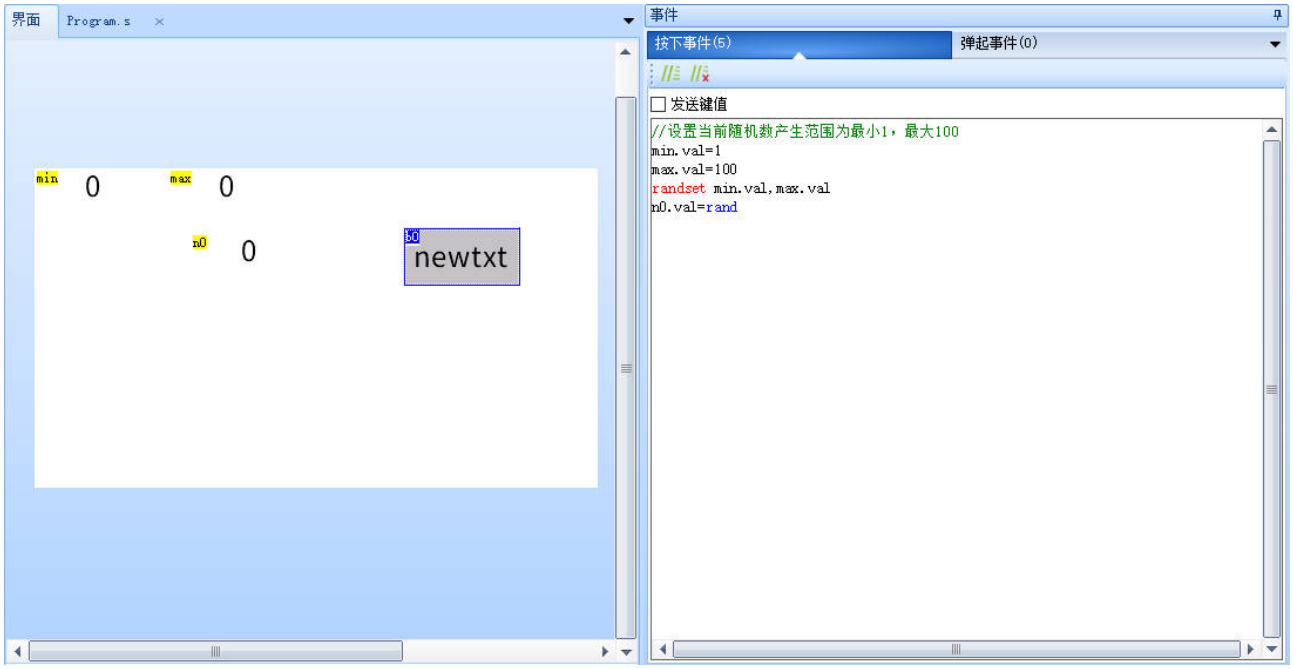
4.1.7.1 randset-示例 1

```
 //设置当前随机数产生范围为最小1，最大100
randset 1,100
n0.val=rand
```



4.1.7.2 randset-示例 2

```
//设置当前随机数产生范围为最小1，最大100
min.val=1
max.val=100
randset min.val,max.val
n0.val=rand
```



注意： 使用随机数之前需要先使用 randset 指令设定一次随机数产生范围，如果不设置，默认是最小 0，最大 2147483647。设置完范围以后，每读取一次系统变量 rand 将会得到一个随机数。

使用 randset 指令每设定一次范围，将一直有效，直到重新上电或者设备复位才会恢复默认。

随机数设定范围的数据类型为 int 类型 (即：最小-2147483648, 最大 2147483647)。

4.1.7.3 randset 指令-样例工程下载

演示工程下载链接：

《randset 指令》演示工程下载

《rand-随机数》演示工程下载

4.1.7.4 rand 指令-相关链接

rand-随机数 (page 791)

4.1.8 covx-变量类型转换

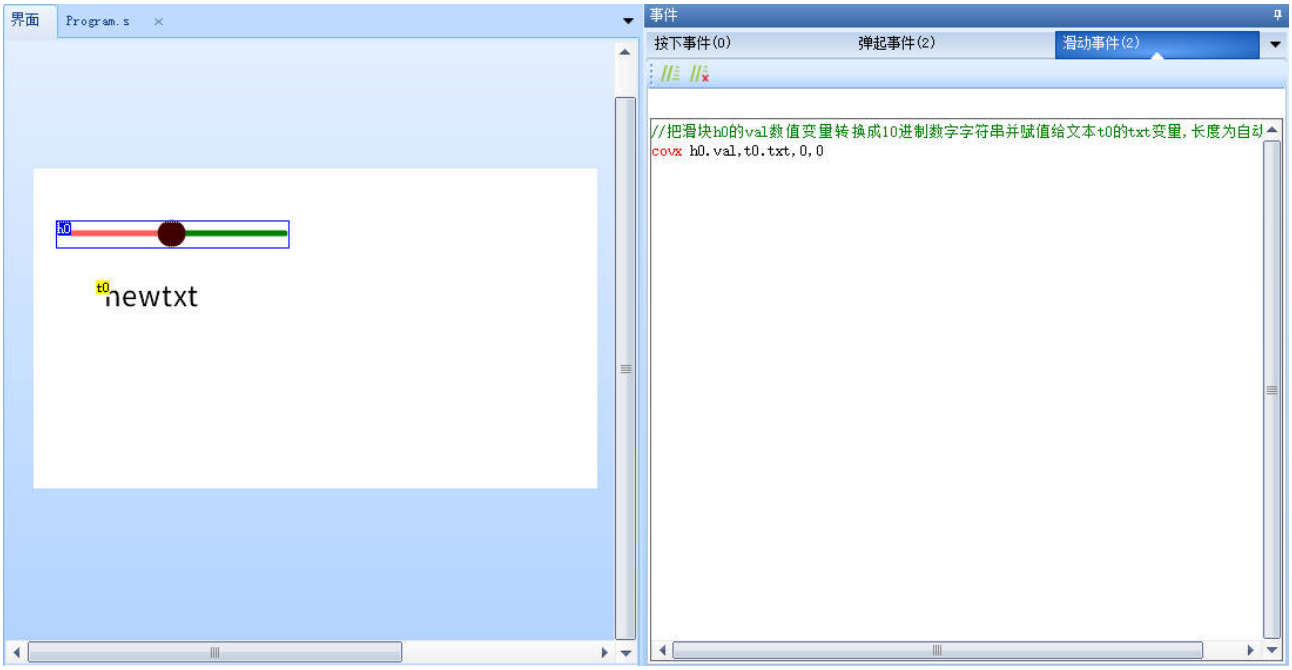
> **注意：** 串口屏上仅有两种数据类型，即数值和字符串类型，当需要将字符串类型转换为数字类型或者将数字类型转换为字符串类型时，才使用 covx 指令
>
> 目前仅有 txt、path、dir、filter 这 4 个属性属性是字符串型属性，其他属性一般都是数值型属性。

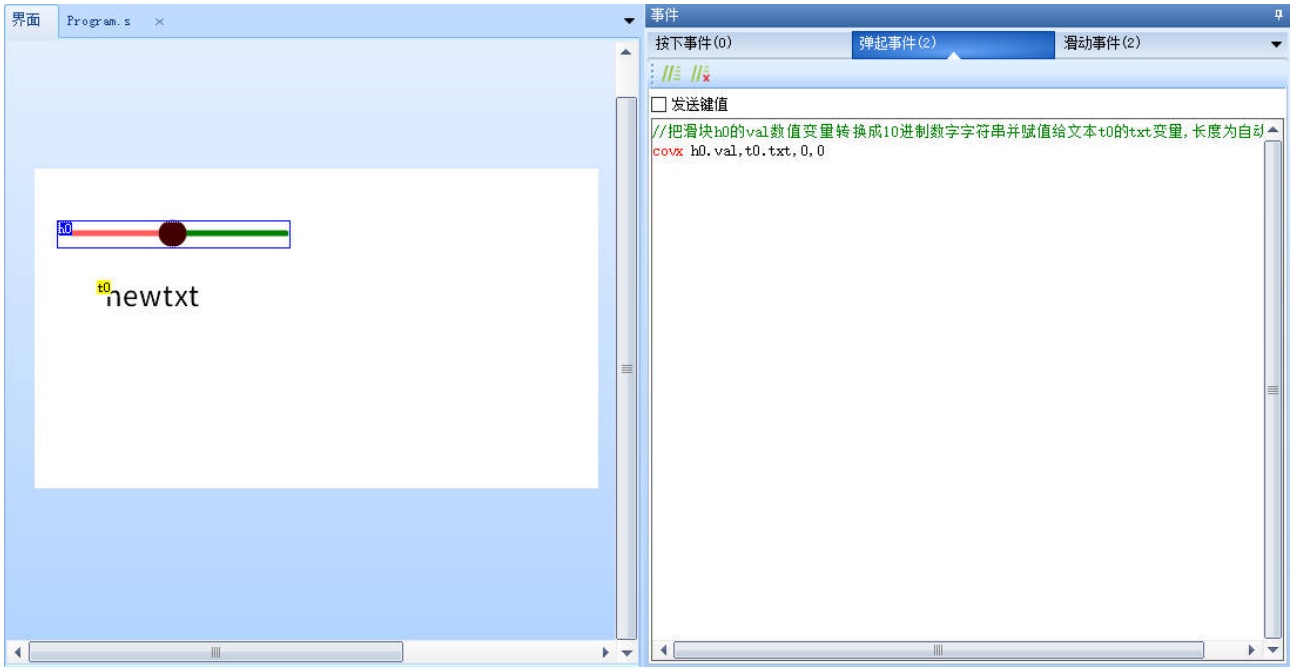
```
covx att1,att2,lenth,format

att1:源变量

att2:目标变量

lenth:字符串的长度(0为自动长度，非0为固定长度)

format:申明数值类型(0-数字;1-货币;2-Hex)
```

4.1.8.1 covx-示例 1

```
//把滑块h0的val数值变量转换成10进制数字字符串并赋值给文本t0的txt变量,长度为自动
covx h0.val,t0.txt,0,0
```

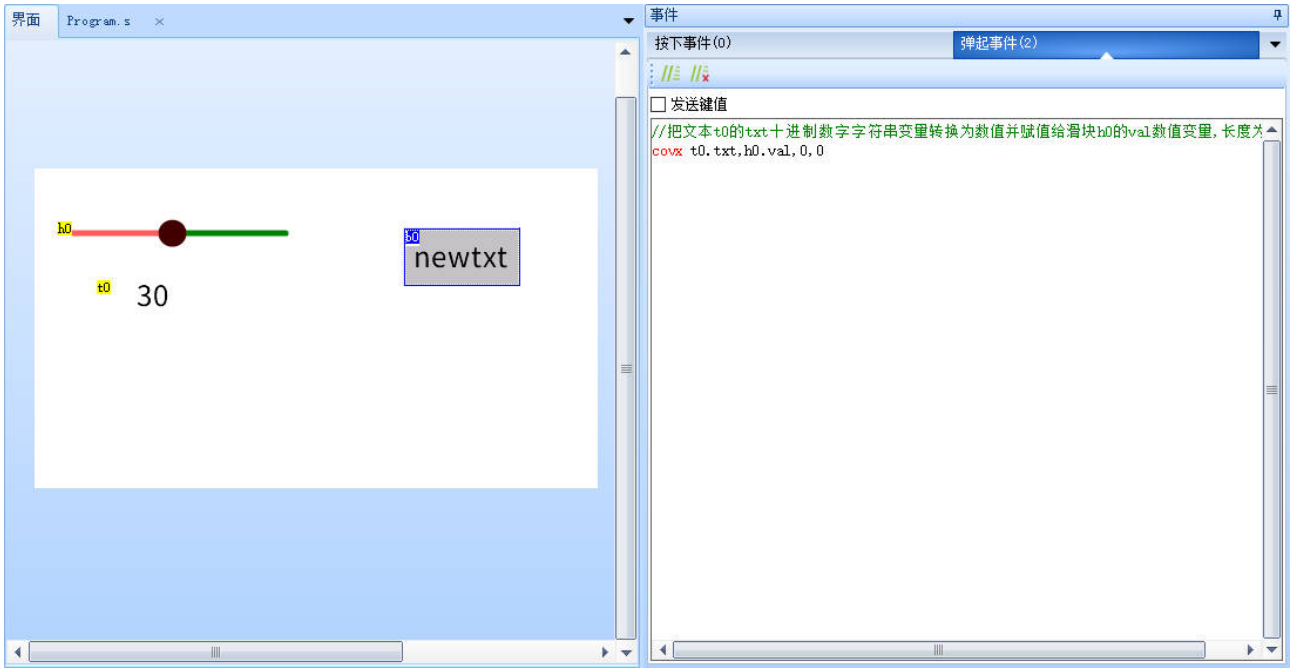
在滑块控件的滑动事件和弹起事件中都要写





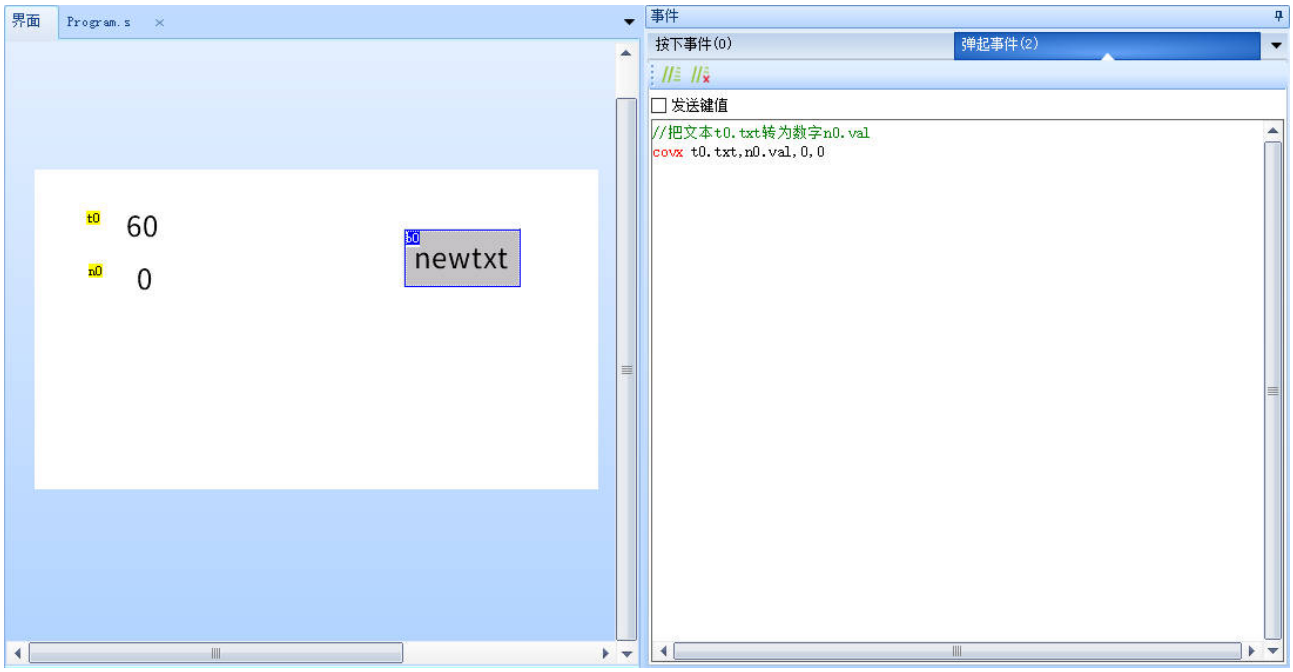
4.1.8.2 covx-示例 2

```
//把文本t0的txt十进制数字字符串变量转换为数值并赋值给滑块h0的val数值变量，长度为自动
covx t0.txt,h0.val,0,0
```



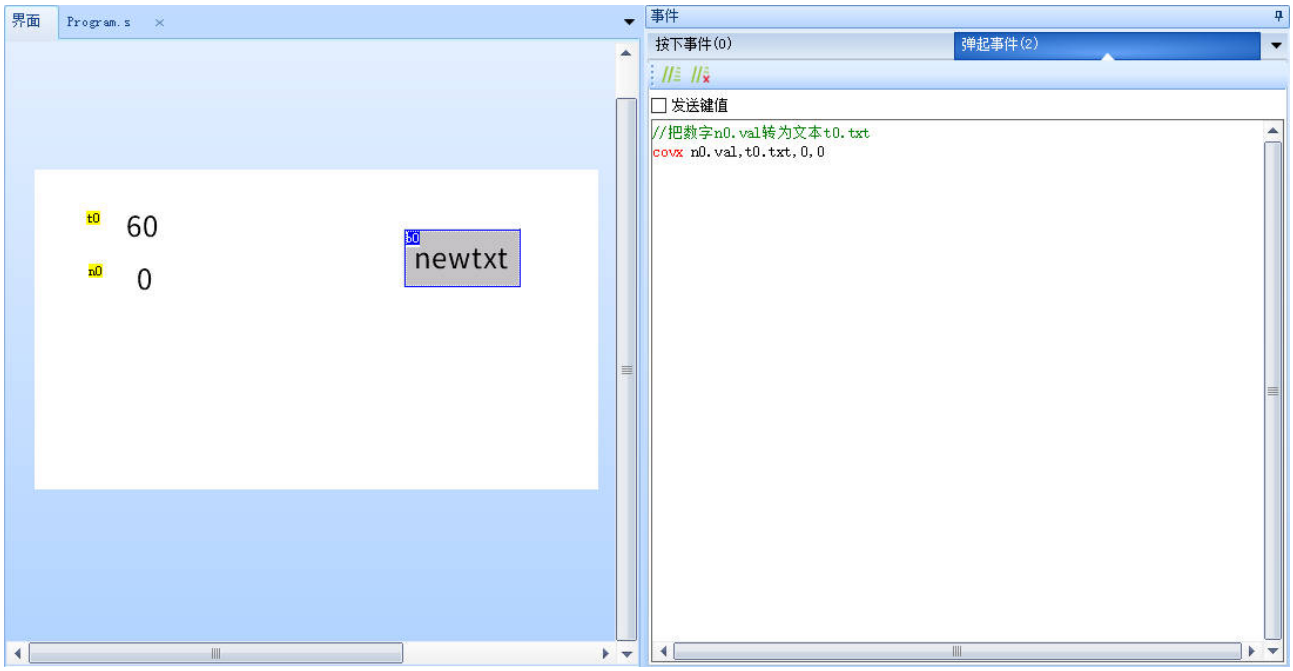
4.1.8.3 covx-示例 3

```
//把文本t0.txt转为数字n0.val
covx t0.txt,n0.val,0,0
```



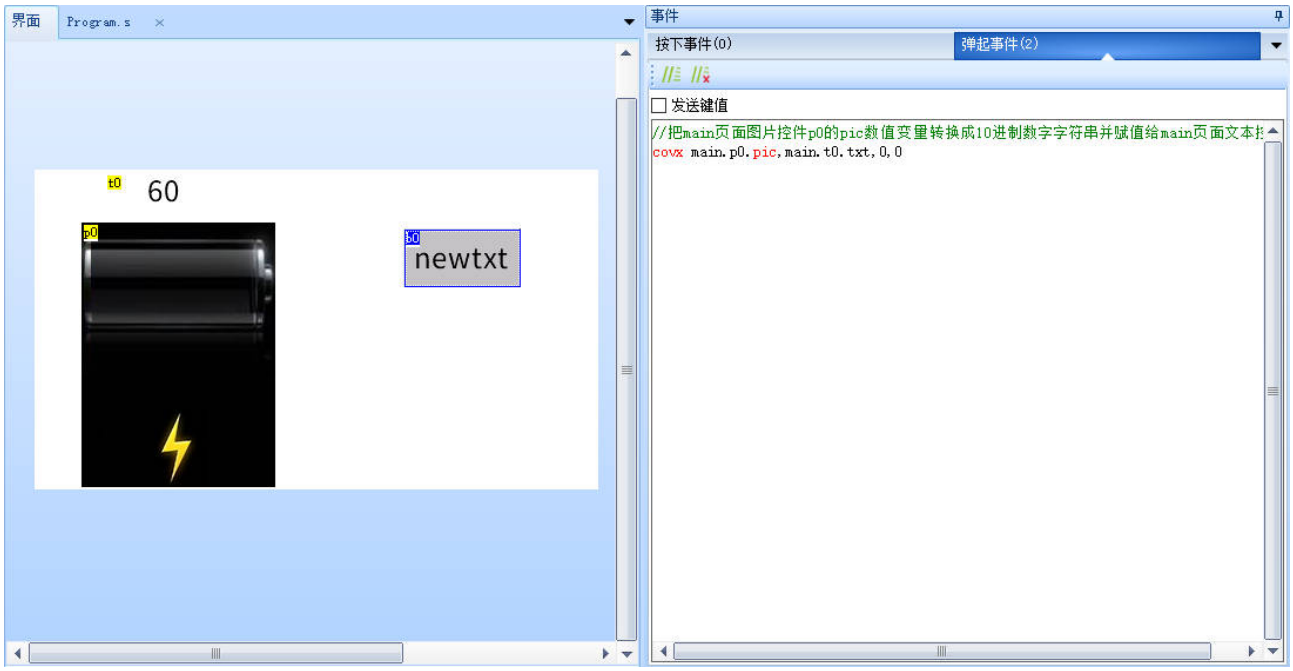
4.1.8.4 covx-示例 4

```
//把数字n0.val转为文本t0.txt
covx n0.val,t0.txt,0,0
```



4.1.8.5 covx-示例 5

```
//
↪把main页面图片控件p0的pic数值变量转换成10进制数字字符串并赋值给main页面文本控件t0的txt变量,
↪长度为自动
covx main.p0.pic,main.t0.txt,0,0
```



注意： lenth 始终表示的是字符串长度，数值转字符串的时候是目标变量的长度，字符串转数值的时候是源变量长度。

当原变量是数值属性时，目标变量必须是字符串属性，当原变量是字符串属性时，目标变量必须是数值属性。

如果目标变量和源变量类型相同，转换失败。

4.1.8.6 covx 指令-相关链接

数字控件 (page 417)

文本控件 (page 405)

txt 属性和 val 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

4.1.8.7 covx 指令-样例工程下载

演示工程下载链接：

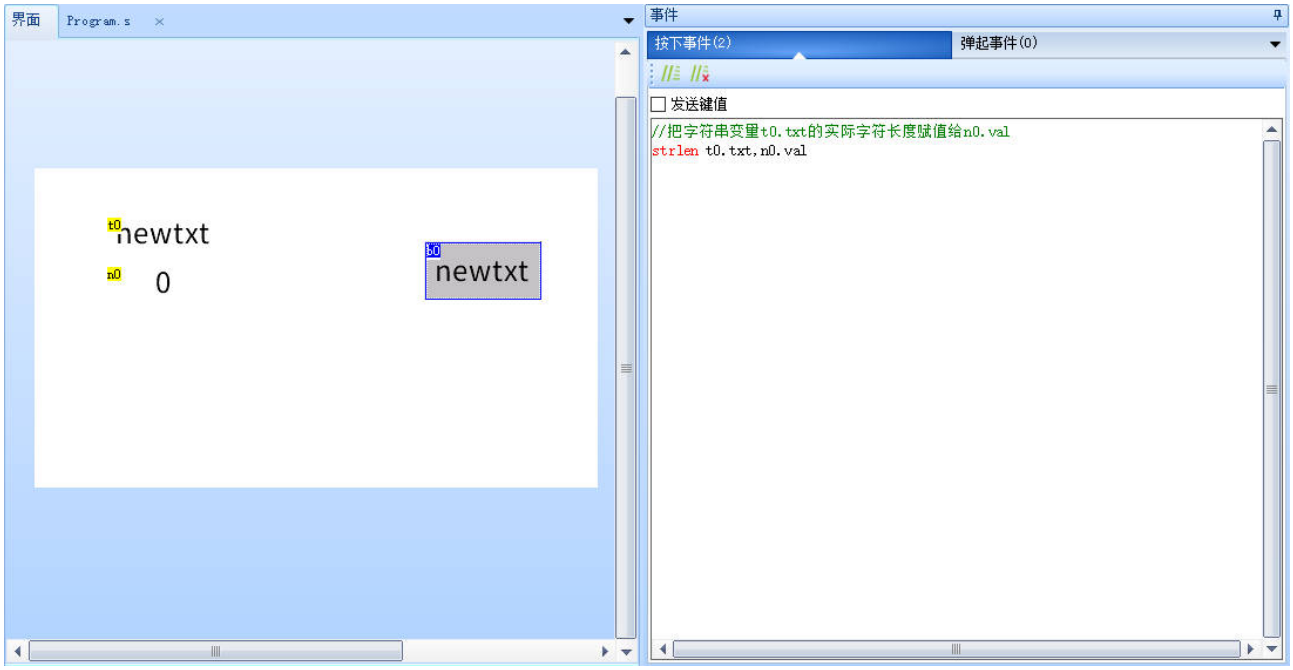
《covx 指令》演示工程下载

4.1.9 strlen-字符串变量字符长度测试

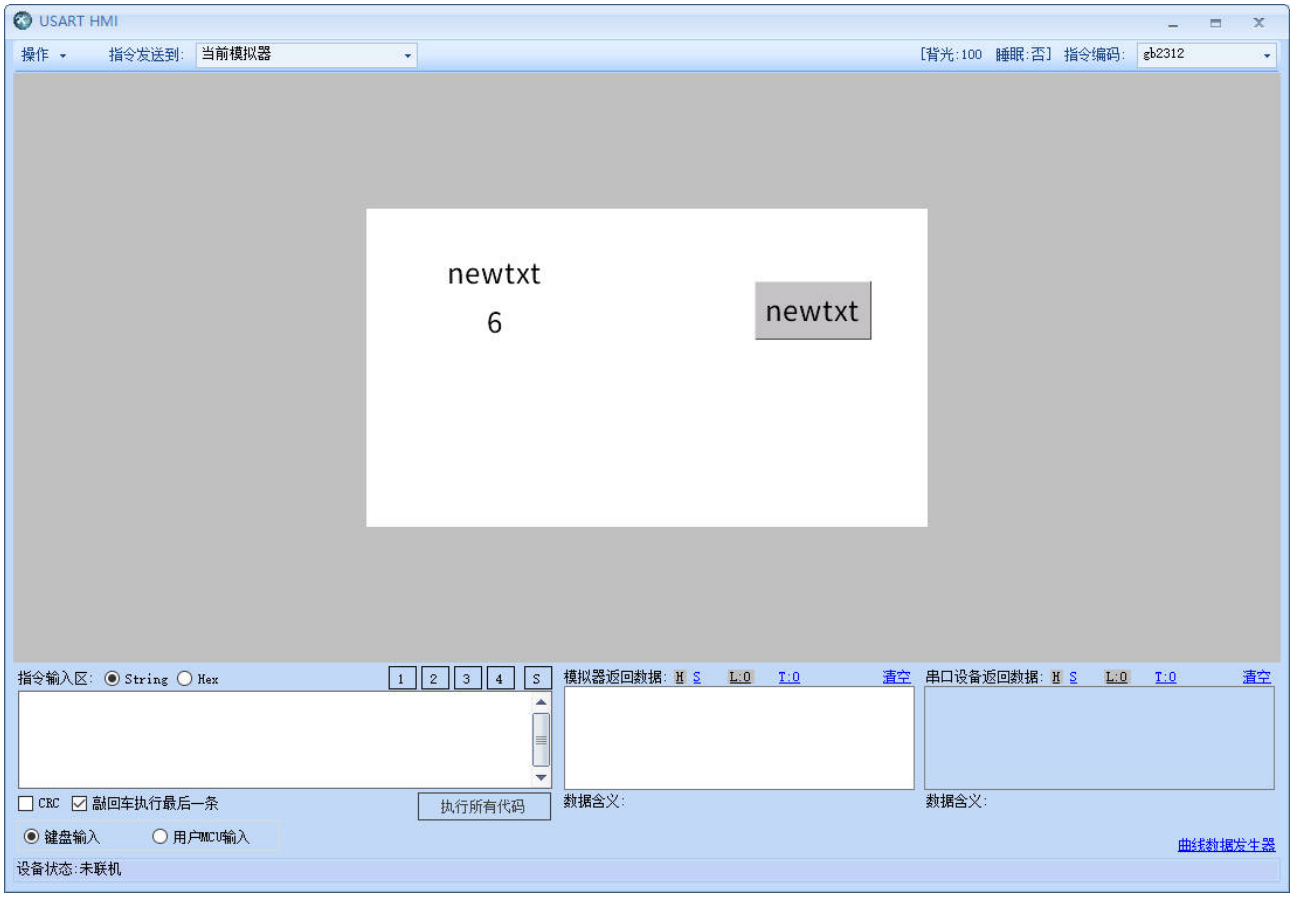
```
strlen att0,att1

att0:需要测试的字符串变量

att1:把测试结果赋值给此变量
```

4.1.9.1 strlen-示例

```
//把字符串变量t0.txt的实际字符长度赋值给n0.val
strlen t0.txt,n0.val
```



运行结果如下



注意： strlen 测试的是以字符为单位的长度，而 btlen 测试的是以字节为单位的长度，比如一个汉字用 btlen 测试出来的长度是 2 字节，用 strlen 测试出来的长度是 1 字符。

被测试的变量必须是字符串类型，写入的变量必须是数值类型，否则会报错。

4.1.9.2 strlen 指令-样例工程下载

演示工程下载链接：

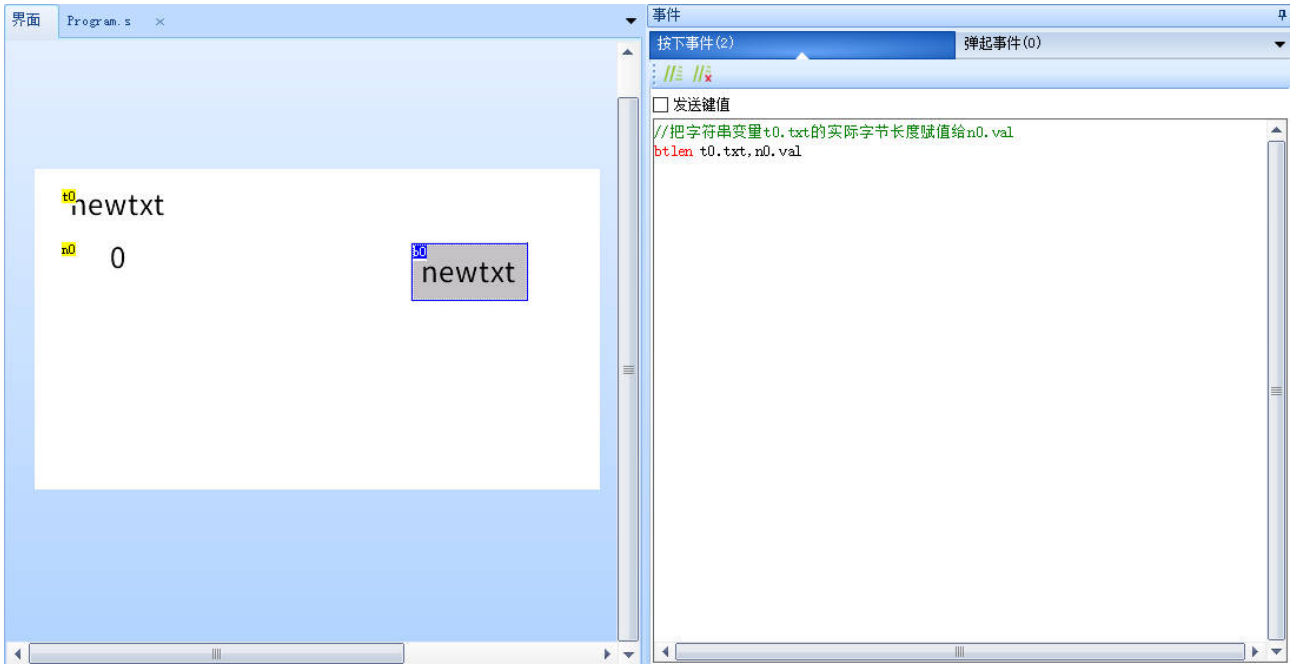
《strlen 指令》演示工程下载

4.1.10 btlen-字符串变量字节长度测试

```
btlen att0,att1

att0:需要被测试的字符串变量

att1:把测试结果赋值给此变量
```

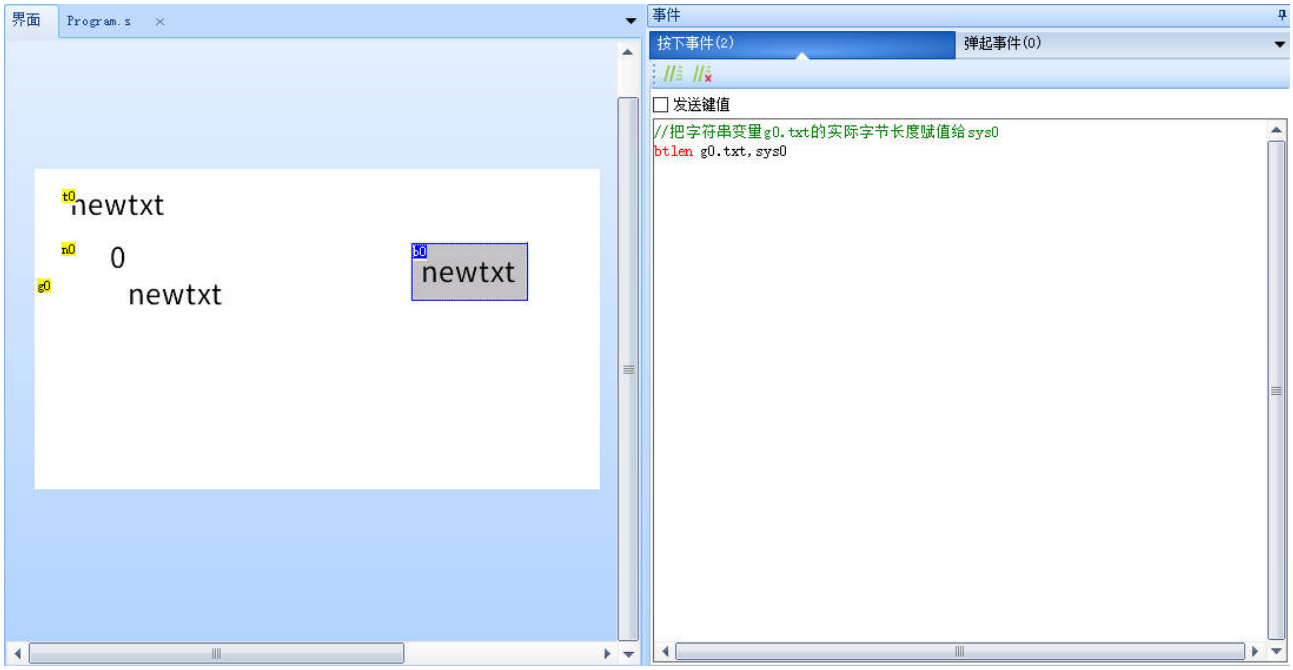
4.1.10.1 btlen-示例 1

```
//把字符串变量t0.txt的实际字节长度赋值给n0.val
btlen t0.txt,n0.val
```



4.1.10.2 btlen-示例 2

```
//把字符串变量g0.txt的实际字节长度赋值给sys0
btlen g0.txt,sys0
```



4.1.10.3 btlen-c 语言示例

单片机使用 btlen 指令把字符串变量 t0.txt 的实际字节长度赋值给 n0.val

```
//将main页面的t0的txt属性转换到main页面的n0的val属性
printf("btlen main.t0.txt,main.n0.val\xff\xff\xff");
```

注意： btlen 测试的是以字节为单位的长度，而 strlen 测试的是以字符为单位的长度，比如在 GB2312 下，一个汉字用 btlen 测试出来的长度是 2 字节，用 strlen 测试出来的长度是 1 字符。

汉字在不同编码下占用的字节数量不一样，例如在 gb2312 编码下，1 个汉字占用 2 字节，但是在 utf8 编码下，1 个汉字占用 3 字节。

被测试的变量必须是字符串类型，写入测试结果的变量必须是数值类型，否则会报错。

4.1.10.4 btlen 指令-样例工程下载

演示工程下载链接：

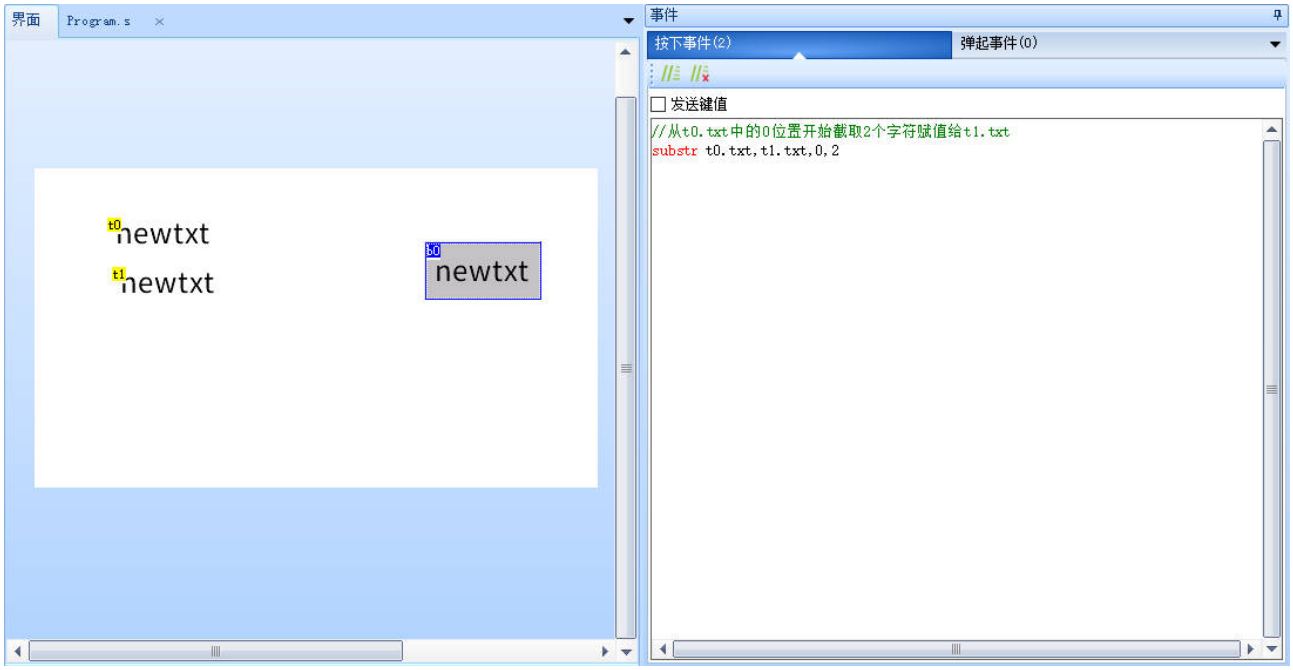
《btlen 指令》演示工程下载

4.1.11 substr-字符串截取

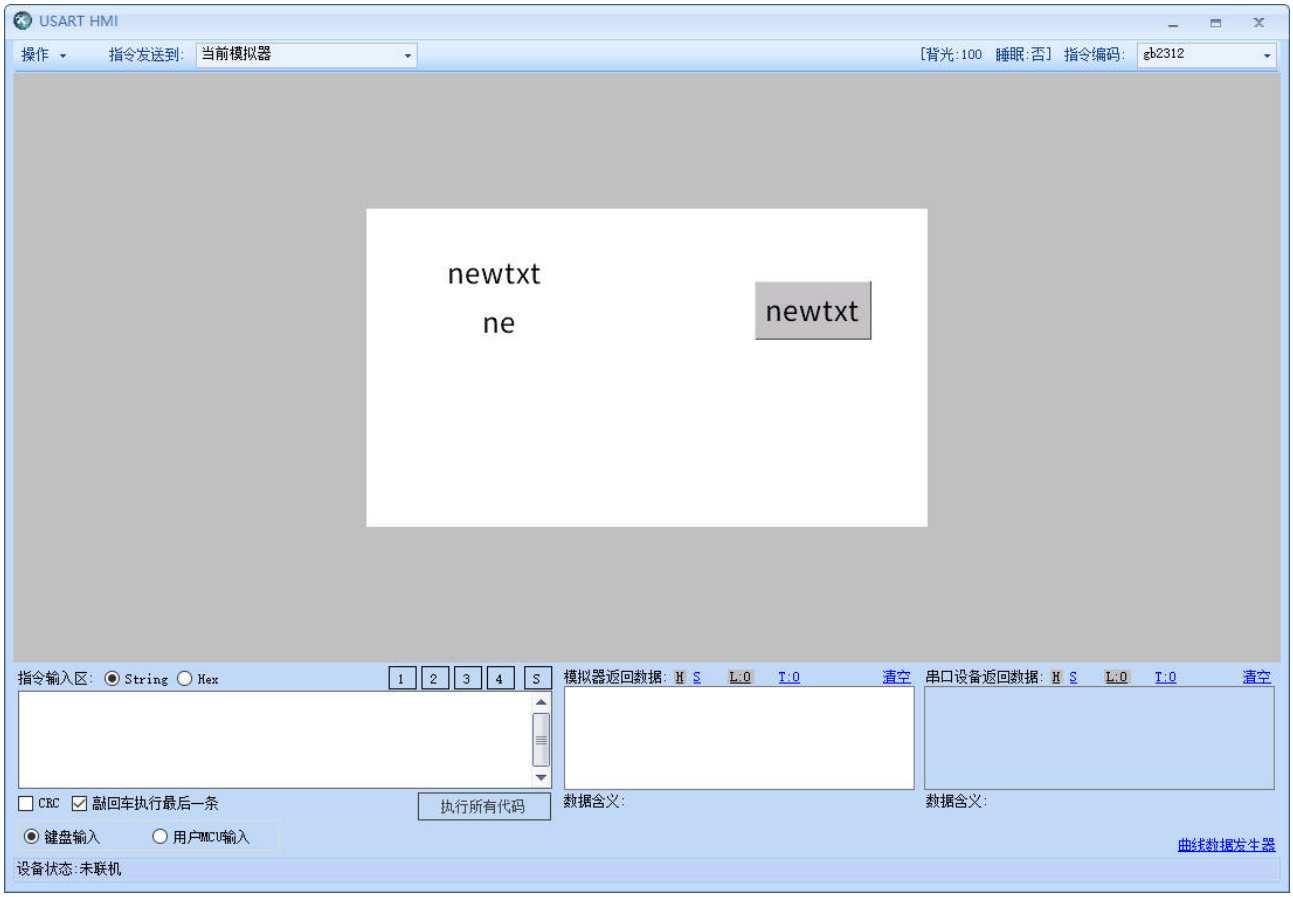
```
substr att0,att1,star,lenth

att0:源变量(必须是字符串变量)

att1:目标变量(必须是字符串变量)

star:在源变量中的字符起始位置

lenth:截取字符串长度
```

4.1.11.1 substr-示例

```
//从t0.txt中的0位置开始截取2个字符赋值给t1.txt
substr t0.txt,t1.txt,0,2
```



运行结果如下



4.1.11.2 substr 指令-样例工程下载

演示工程下载链接:

《substr 指令》演示工程下载

4.1.12 spstr-字符串分割

```
spstr src,dec,key,index

src:源变量(必须是字符串变量)

dec:目标变量(必须是字符串变量)

key:分隔符字符串(必须是字符串变量)

index:取第几份(在src字符串中用key字符串做分割后，取第index份字符内容赋值给dec变量)
```

4.1.12.1 spstr-示例 1

data0 控件为数据记录控件

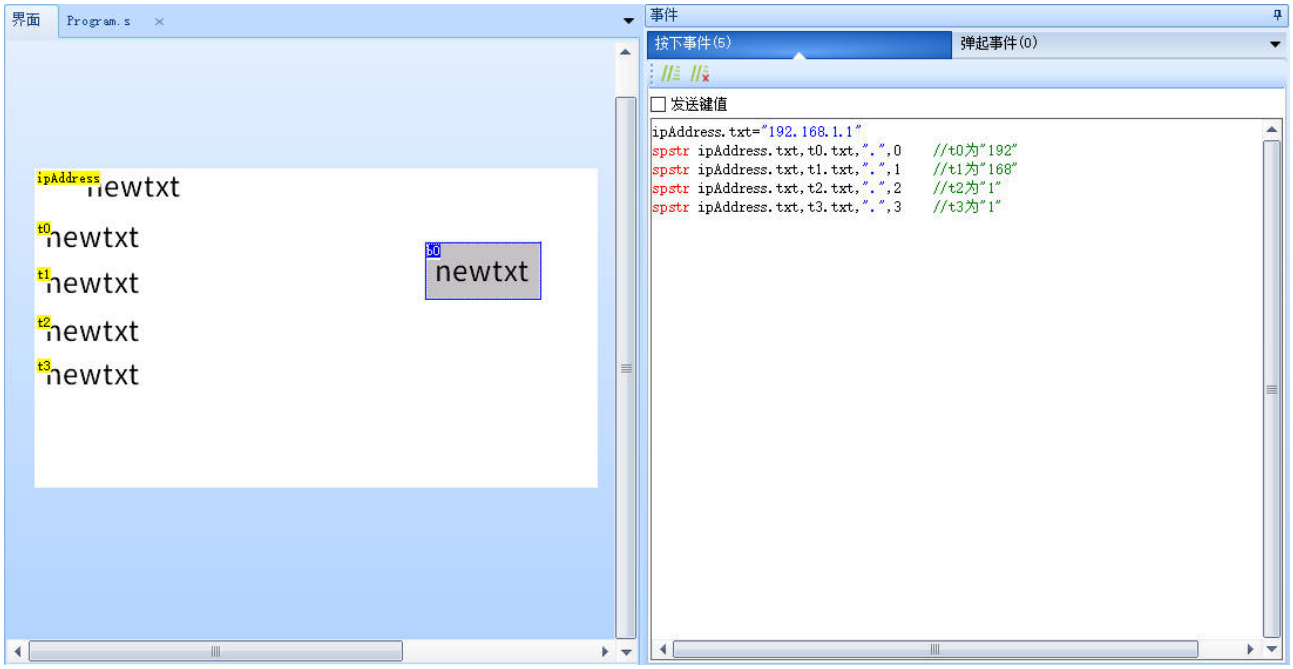
data0.txt 的字符内容为:aaaa^bbbb^cccc^dddd

```
spstr data0.txt,t0.txt,"^",0    //以^为分隔符,截取第0个,t0为"aaaa"
spstr data0.txt,t1.txt,"^",1    //以^为分隔符,截取第1个,t1为"bbbb"
spstr data0.txt,t2.txt,"^",2    //以^为分隔符,截取第2个,t2为"cccc"
spstr data0.txt,t3.txt,"^",3    //以^为分隔符,截取第3个,t3为"dddd"
```

4.1.12.2 spstr-示例 2

ipAddress.txt 的字符内容为:192.168.1.1

```
ipAddress.txt="192.168.1.1"

spstr ipAddress.txt,t0.txt,".",0    //以.为分隔符,截取第0个,t0为"192"

spstr ipAddress.txt,t1.txt,".",1    //以.为分隔符,截取第1个,t1为"168"

spstr ipAddress.txt,t2.txt,".",2    //以.为分隔符,截取第2个,t2为"1"

spstr ipAddress.txt,t3.txt,".",3    //以.为分隔符,截取第3个,t3为"1"
```



4.1.12.3 spstr-示例: 截取换行符

test.txt 的字符内容为:

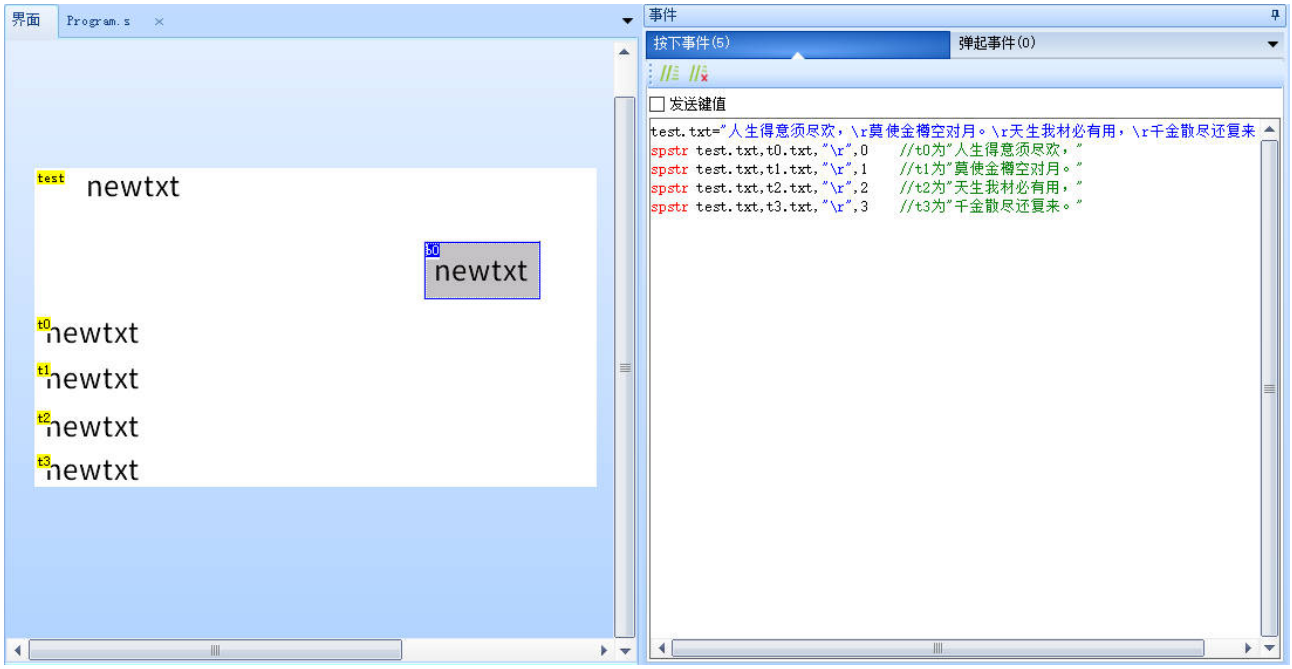
人生得意须尽欢,

莫使金樽空对月。

天生我材必有用,

千金散尽还复来。

```
// \r在串口屏中代表回车换行的意思

test.txt="人生得意须尽欢，\r莫使金樽空对月。\r天生我材必有用，\r千金散尽还复来。"

spstr test.txt,t0.txt,"\r",0    //以\r为分隔符,截取第0个,t0为"人生得意须尽欢，"

spstr test.txt,t1.txt,"\r",1    //以\r为分隔符,截取第1个,t1为"莫使金樽空对月。"

spstr test.txt,t2.txt,"\r",2    //以\r为分隔符,截取第2个,t2为"天生我材必有用，"

spstr test.txt,t3.txt,"\r",3    //以\r为分隔符,截取第3个,t3为"千金散尽还复来。"
```



4.1.12.4 spstr-示例: 获取名人名言左右两边的数据

```
test.txt="出淤泥而不染，濯清涟而不妖。——周敦颐"

spstr test.txt,t0.txt,"。——",0     //以"。——"为分隔符,截取左边的数据,t0为 ␣
↪出淤泥而不染，濯清涟而不妖
spstr test.txt,t1.txt,"。——",1    //以"。——"为分隔符,截取右边的数据,t0为   周敦颐
```

4.1.12.5 spstr-示例: 截取冒号和句号之间的数据

```
test.txt="周敦颐：出淤泥而不染，濯清涟而不妖。"

spstr test.txt,temp_str.txt,"：",1     //以冒号为分隔符,截取冒号右边的数据 ␣
↪出淤泥而不染，濯清涟而不妖。
spstr temp_str.txt,t0.txt,"。",0     //以句号为分隔符,截取句号左边的数据,t0为 ␣
↪出淤泥而不染，濯清涟而不妖
```

4.1.12.6 spstr-示例: 分割浮点数

```
spstr test.txt,t0.txt,".",0     //以.为分隔符,截取第0个,t0为整数部分
spstr test.txt,t1.txt,".",1     //以.为分隔符,截取第1个,t1为小数部分
```

4.1.12.7 spstr-示例: 解析 json 格式

```
{
    "name": "test",
    "Info": {
        "age": 16,
        "pass": false
    }
}
```

在实际传输时，json 会被压缩为以下格式（被压缩为 1 行，不带换行和缩进等）

`{"name":"test","Info":{"age":16,"pass":false}}`

如何提取到 name 属性 (字符串属性)

以 `"name":"` 为分隔符，获取到右边部分 `test","Info":{"age":16,"pass":false}}`

再以 `"` 为分隔符，获取到左边部分的属性 `test`

如何提取到 “age“属性 (数值属性)

以 `"age":` 为分隔符，获取到右边部分 `16,"pass":false}}`

再以 `,` 为分隔符，获取到左边部分的属性 `16`

如何提取到 `age` 属性 (布尔值属性)

设置一个文本控件 tBool，txt_maxl 长度为 4

以 `"pass":` 为分隔符，获取到右边部分将值放入 tBool，因为 tBool 的 txt_maxl 长度为 4，所以会截取到前 4 字节 `fals` 或者 `true`

由于 json 通常只在串口传输中使用，一般是在主动解析模式下使用，需要的请参考下面的主动解析教程链接。
接收 json 数据字符串 (page 1003)

4.1.12.8 spstr 指令-样例工程下载

演示工程下载链接：

《spstr 指令》演示工程下载

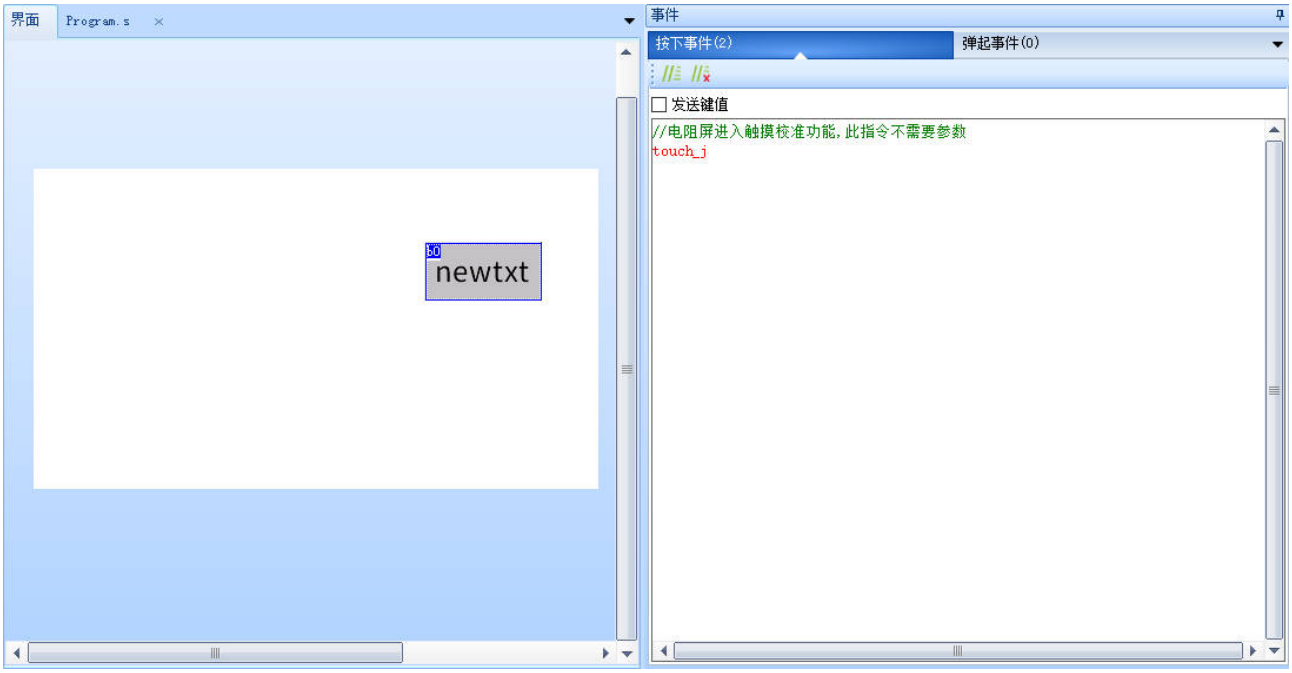
4.1.13 touch_j-触摸校准

> **注意:** 仅电阻屏实物支持该指令
> 不支持在模拟器中使用
> 不支持在电容屏和无触摸款串口屏中使用

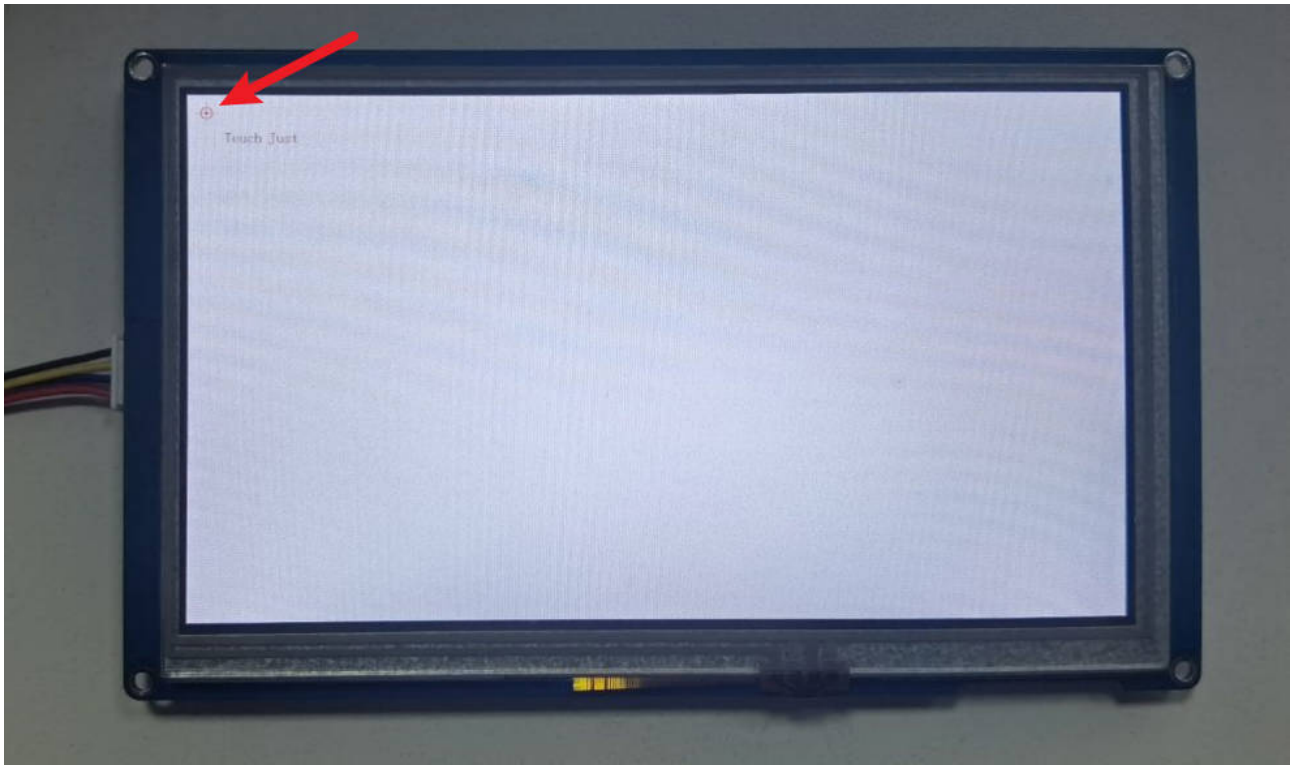
```
touch_j
```

4.1.13.1 touch_j-示例

```
//电阻屏进入触摸校准功能,此指令不需要参数
touch_j
```



进入校准后，根据要求点击屏幕四个角进行校准即可



4.1.13.2 touch_j 指令-样例工程下载

演示工程下载链接：

《touch_j 指令》演示工程下载

4.1.14 add-往曲线控件添加数据

往当前页面的曲线控件添加数据，不支持跨页面添加曲线控件数据

```
add objid,ch,val

objid:曲线控件ID序号(此处必须是ID号，不支持使用控件名称)，推荐使用s0.id这种形式

ch:曲线控件通道号（范围：0-3）

val:数据（最小0，最大255，还取决于曲线控件的实际高度，例如曲线控件实际高200像素，当你添加的值超过200的部分就显示不出来。超过255软件会自动取低8位，即对255取余）
```

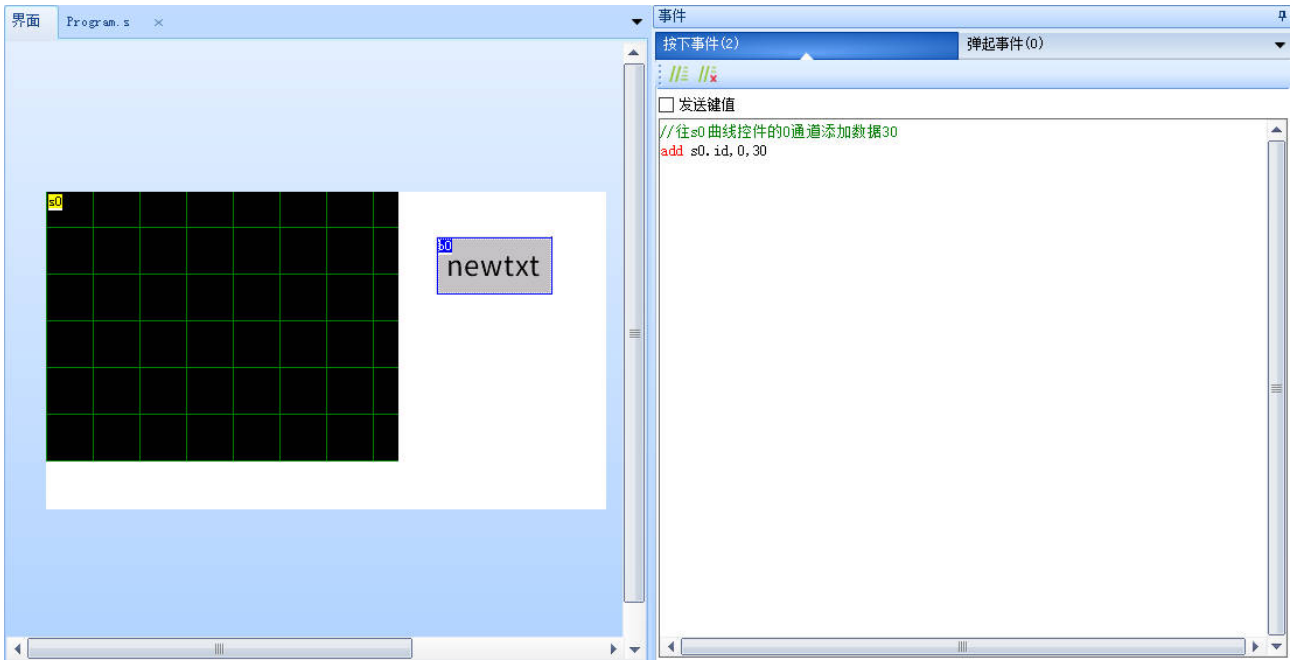
4.1.14.1 add-示例 1

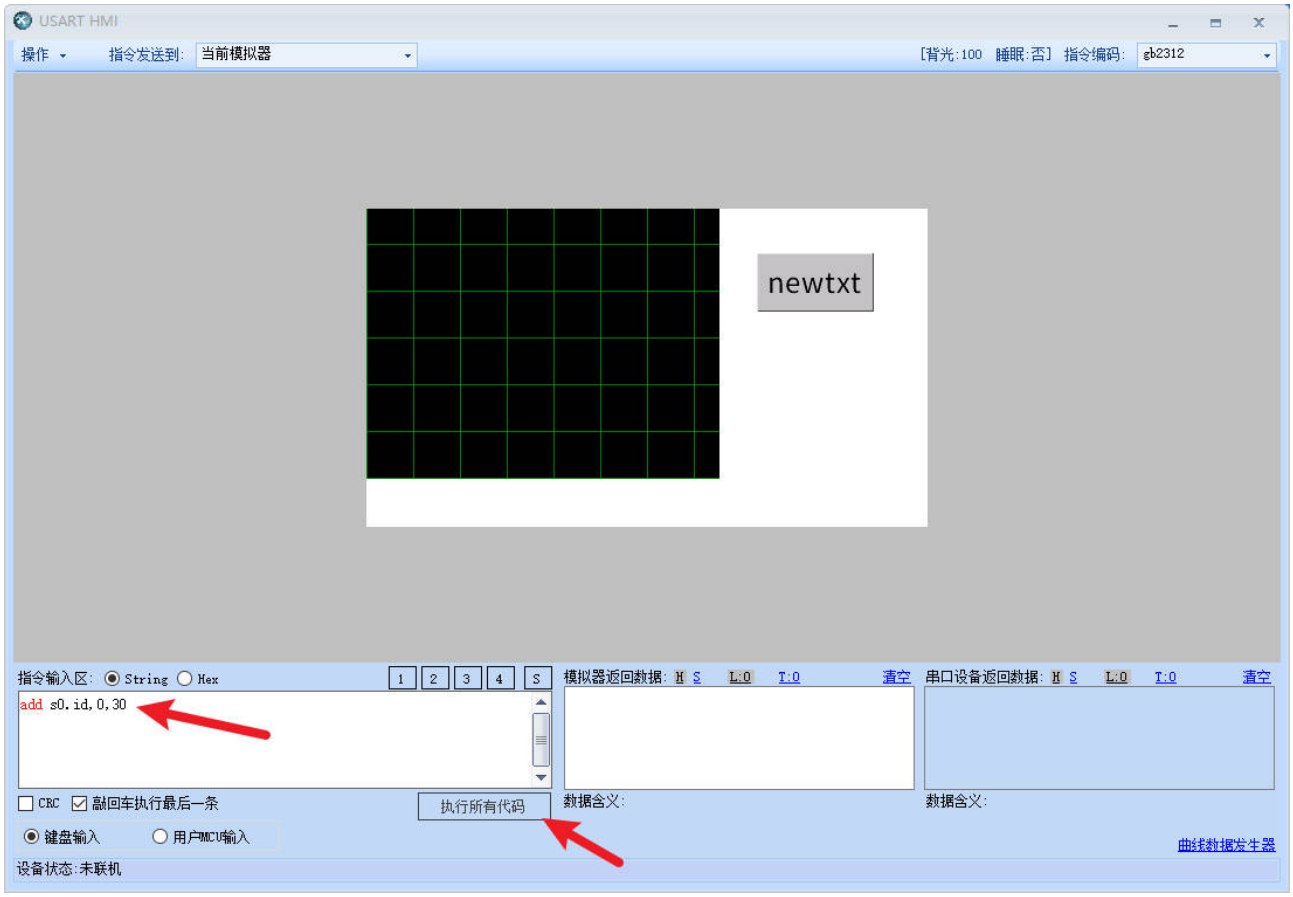
注意： 请注意, 操作的是 s0.id 而不是 s0

```
//往s0曲线控件的0通道添加数据30
add s0.id,0,30
```

使用 s0.id 的好处: 避免因为置顶/置底或者删除控件导致 id 改变从而出错

注意： 只触发一次按钮的话，无法明显观测到波形，需要点击按钮控件多次触发指令才会出现明显的波形

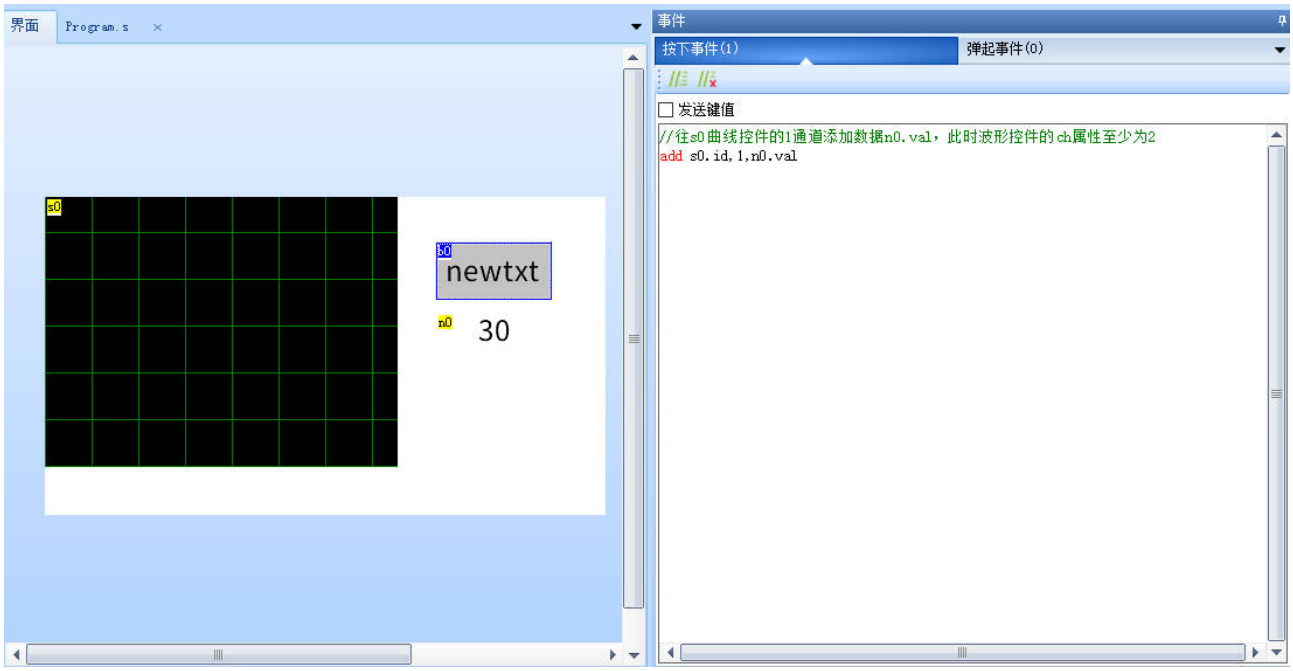


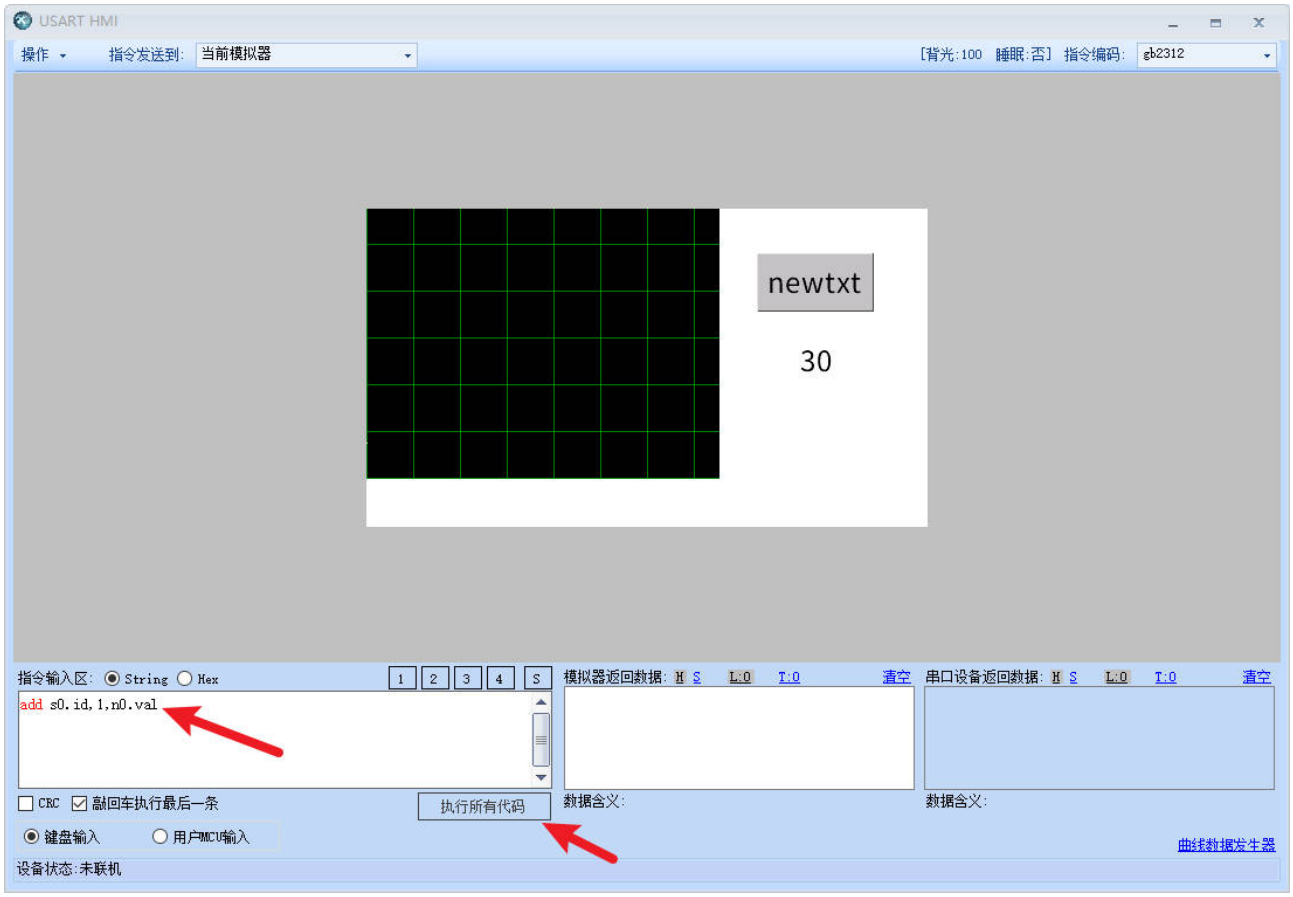


4.1.14.2 add-示例 2

```
//往s0曲线控件的1通道添加数据n0.val，此时波形控件的ch属性至少为2
add s0.id,1,n0.val
```

注意： 只触发一次按钮的话，无法明显观测到波形，需要点击按钮控件多次触发指令才会出现明显的波形

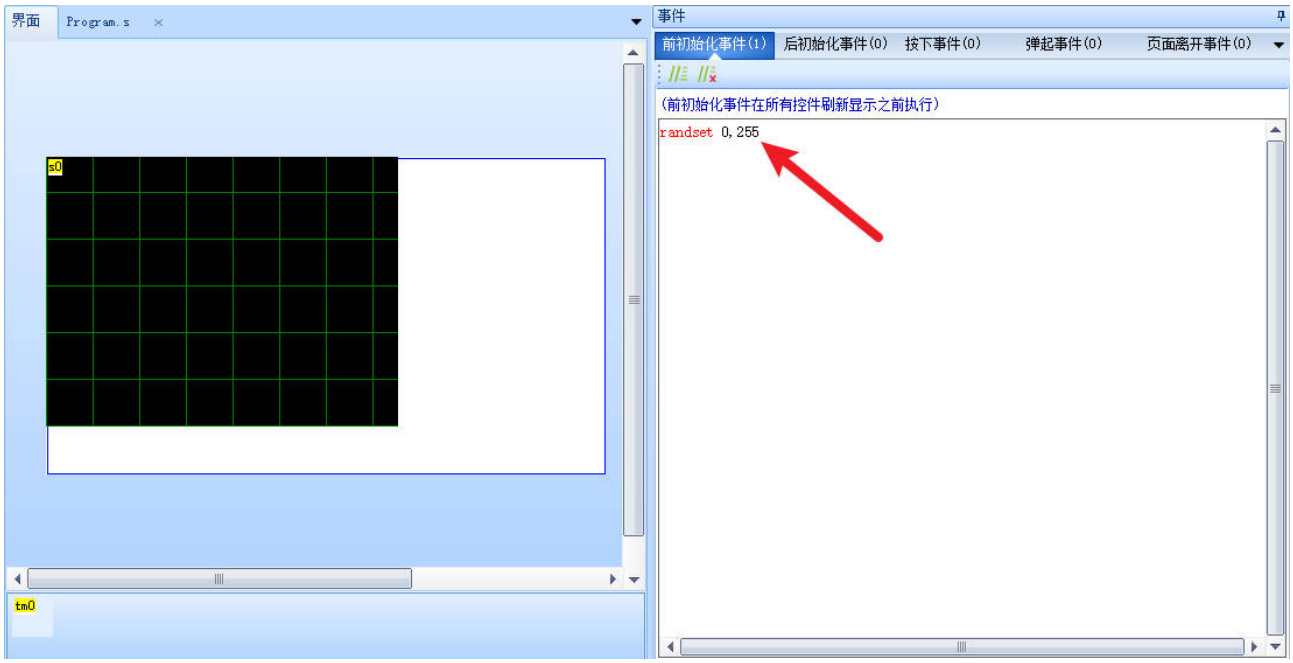

界面
Program.s
事件
按下事件(1)
弹起事件(0)
发送键值
//往s0曲线控件的1通道添加数据n0.val，此时波形控件的ch属性至少为2
add s0.id,1,n0.val
s0
b0
newtxt
n0
30


USART HMI
操作
指令发送到:
当前模拟器
[背光:100 睡眠:否] 指令编码:
gb2312
newtxt
30
指令输入区:
String
Hex
1
2
3
4
S
add s0.id,1,n0.val
模拟器返回数据:
串口设备返回数据:
清空
CRC
附回车执行最后一条
执行所有代码
数据含义:
键盘输入
用户MCU输入
曲线数据发生器
设备状态:未联机

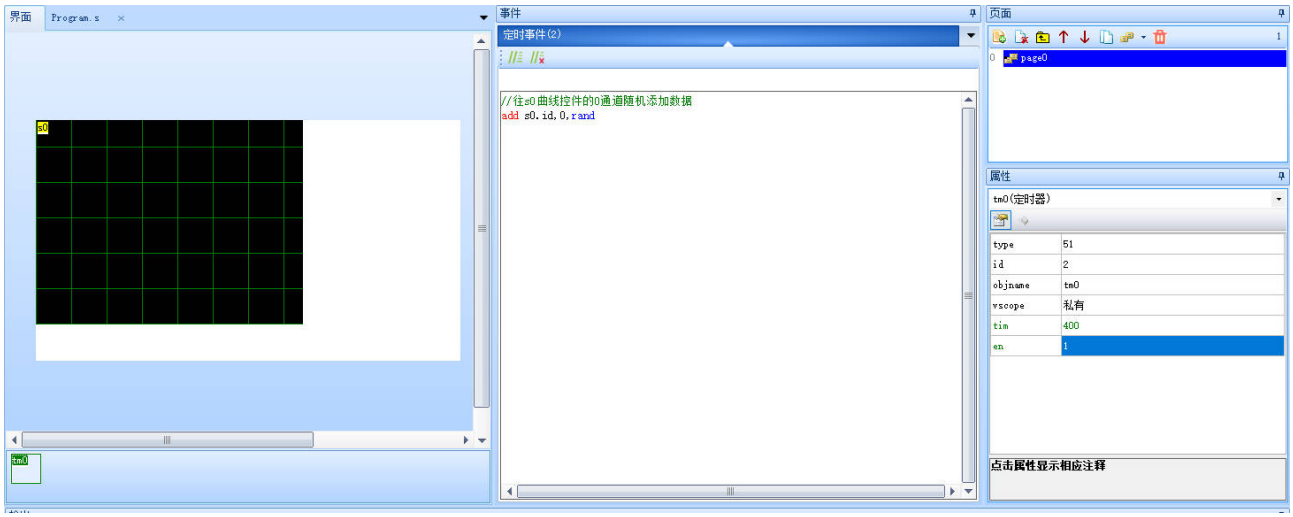
4.1.14.3 add-示例 3

```
//往s0曲线控件的0通道随机添加数据
add s0.id,0,rand
```

在页面的前初始化事件中设置随机数范围



在定时器中定时添加随机数到曲线波形控件中



注意： 曲线数据只支持 8 位数据，最小 0，最大 255。

每个 page 页面最多支持 4 个曲线控件, 每个曲线控件最多支持 4 个通道。可以连续发送数据，控件会自动平推显示数据. 在发送数据的过程中也可以随时修改控件属性，比如随时修改各个通道的前景色或背景色。

当曲线波形控件的 ch 属性为 1（通道数量为 1）时，只有一个通道 0 可以使用，而不是通道 1

4.1.14.4 add-c 语言示例

单片机使用 add 指令向串口屏添加 100 个点

```
for(int i =0;i<100;i++)
{
    //向曲线s0的通道0传输1个数据,add指令不支持跨页面
    printf("add s0.id,0,%d\xff\xff\xff",(int)(rand() % 256));
}
```

4.1.14.5 add 指令-相关链接

曲线波形控件 (page 459)

使用 *add* 指令时屏幕没有任何反应 (page 464)

4.1.14.6 add 指令-样例工程下载

演示工程下载链接：

《add 指令》演示工程下载

4.1.15 addt-曲线数据透传指令

往当前页面的曲线控件透传数据，不支持跨页面透传曲线控件数据

```
addt  objid,ch,qyt

objid: 曲线控件ID序号(此处必须是ID号，不支持使用控件名称)

ch:曲线控件中的通道号

qyt:本次透传数据的点数量
```

4.1.15.1 addt-示例 1

```
//s0曲线控件的通道0进入数据透传模式，透传点数为100点
//推荐使用这种方式，不会因为s0的id号改变而报错
addt s0.id,0,100
```

注意: 曲线数据只支持 8 位数据，最小 0，最大 255。单次透传数据量最大 1024 字节

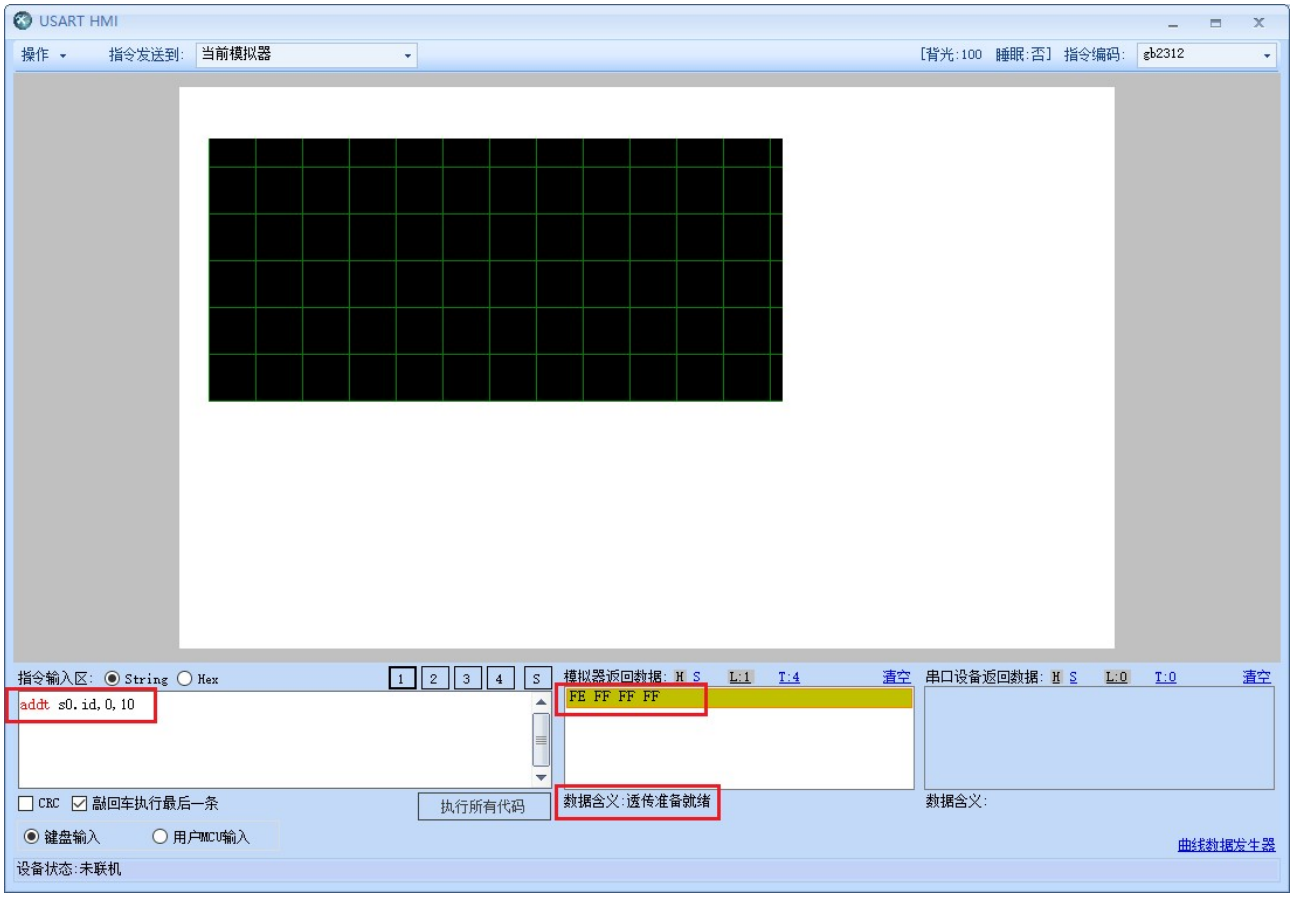
发完透传指令后，用户需要等待设备响应才能开始透传数据，设备收到透传指令后，准备透传初始化数据大概需要 5ms 左右 (如果在透传指令执行前串口缓冲区还有很多别的指令，那时间会更长, 且 t 系列和 k 系列会更久)，设备透传初始化准备好以后会发送一个透传就绪的数据给用户（0XFE+ 结束符），表示设备已经准备好，此时可以开始发送透传数据。透传数据为纯 16 进制数据，不再使用字符串，也不再需要结束符, 设备收完指定的数据量以后，才会恢复指令接收状态。否则一直处于数据透传状态，透传数据完成以后, 设备会发送结束标记给用户（0XFD+ 结束符）。

在指定的透传数量传输完成以前，曲线不会刷新，透传完毕之后会立即自动刷新。

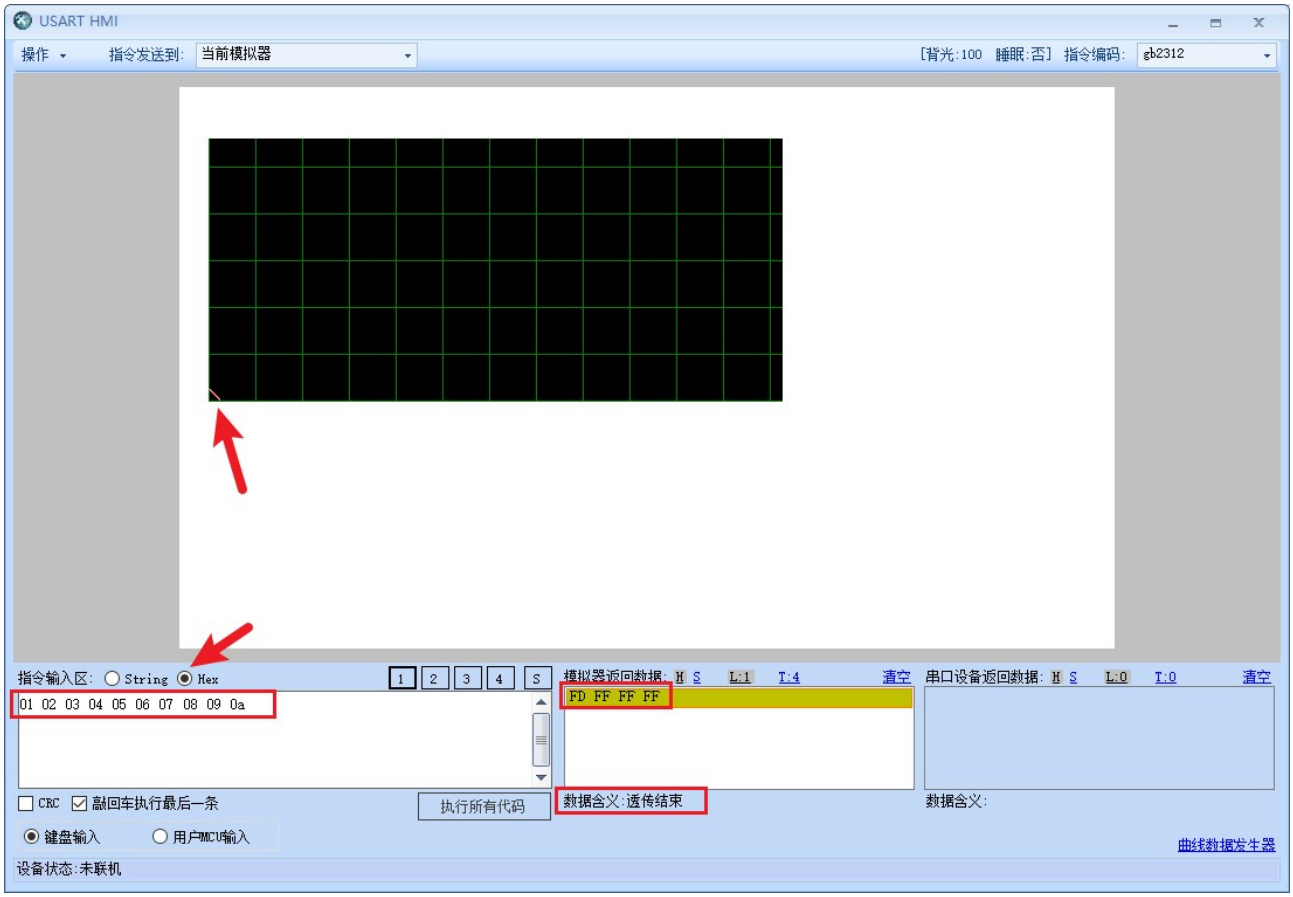
注意: 请注意当曲线波形控件的 ch 属性为 1（通道数量为 1）时，只有一个通道 0 可以使用，而不是通道 1

4.1.15.2 在调试窗口中使用 addt 指令

1. 使用 addt 向 s0 的通道 0 透传 10 个点



2. 切换到 hex 模式发送数据



4.1.15.3 addt 指令-样例工程下载

暂无

4.1.15.4 addt-c 语言示例

单片机使用 addt 指令向串口屏透传 100 个点

```
//向曲线s0的通道0透传100个数据,addt指令不支持跨页面
printf("addt s0.id,0,100\xff\xff\xff");

//等待适量时间
delay_ms(100);

for(int i =0;i<100;i++)
{
    printf("%c",(int)(rand() % 256));
}

//确保透传结束，以免影响下一条指令
printf("\x01\xff\xff\xff");
```

4.1.15.5 addt 指令-相关链接

使用 *add* 指令时屏幕没有任何反应 (page 464)

4.1.16 cle-清除曲线控件中的数据

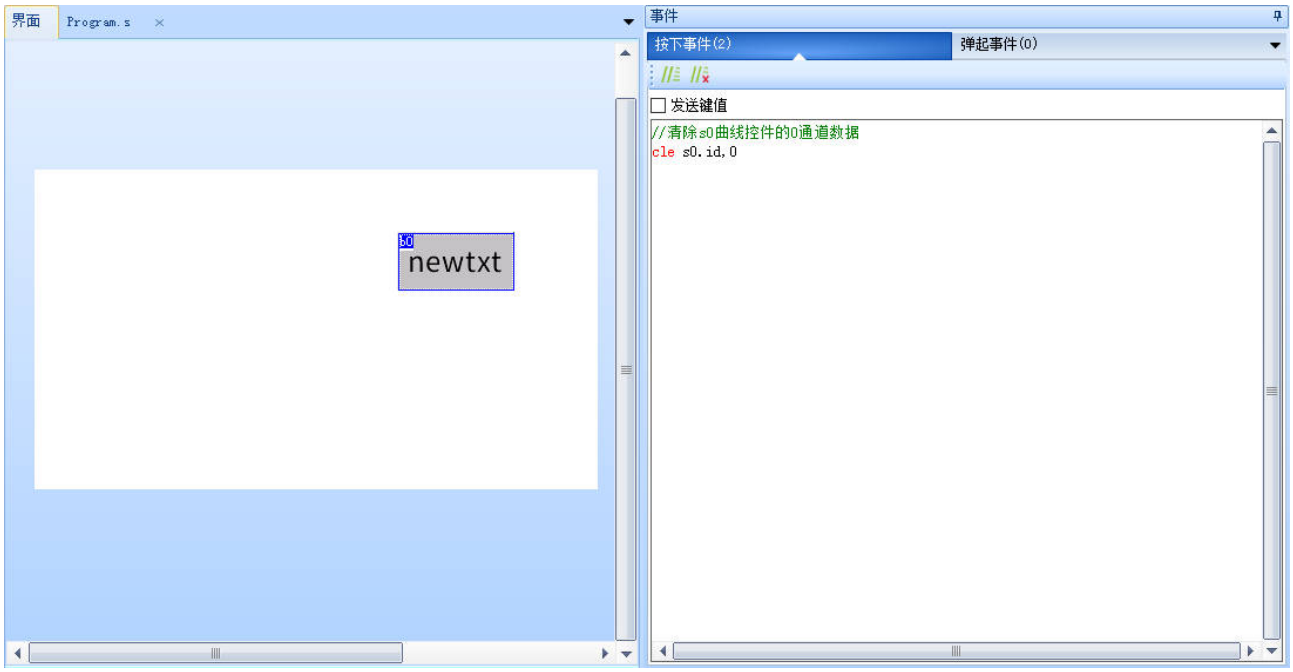
用于清除当前页面曲线控件的数据

只能清除当前页面曲线控件的数据/不支持跨页面清除曲线控件的数据/不能清除其他页面曲线控件的数据

```
cle objid,ch

objid:曲线控件ID序号(此处必须是ID号，不支持使用控件名称)

ch:曲线控件通道号(255表示所有通道)
```

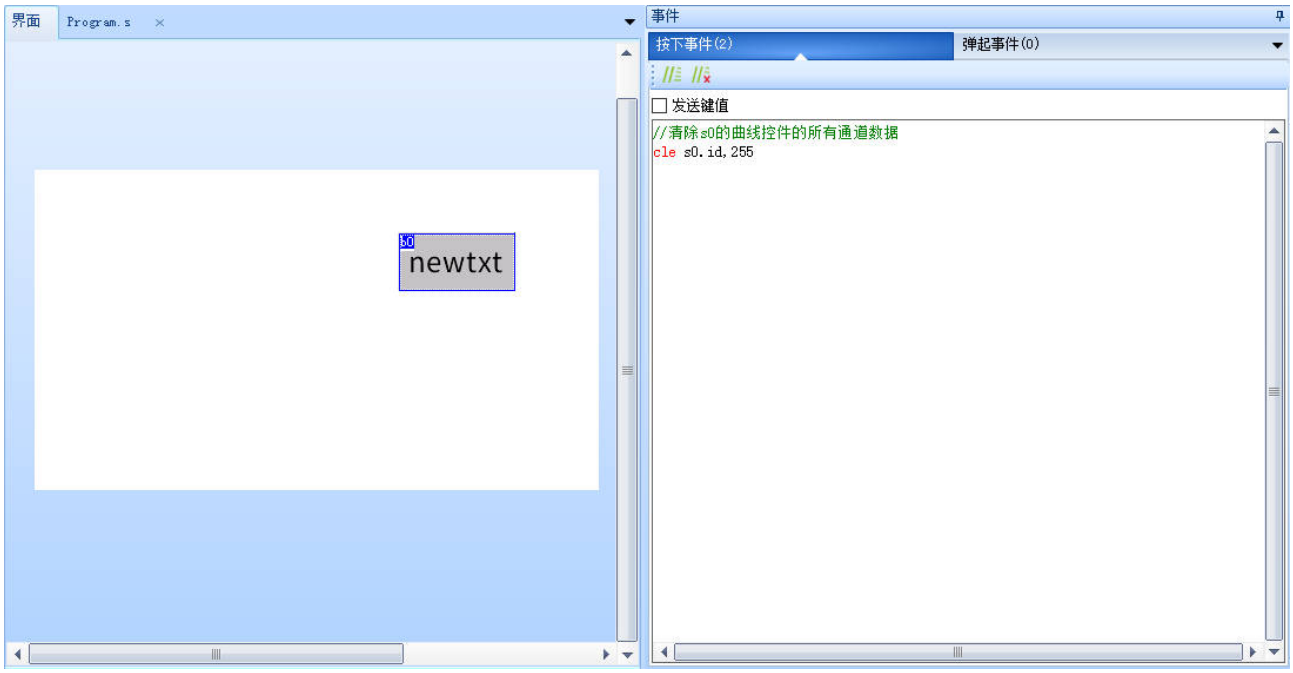
4.1.16.1 cle-示例 1

```
//清除s0曲线控件的0通道数据
cle s0.id,0
```



4.1.16.2 cle-示例 2

```
//清除s0的曲线控件的所有通道数据
cle s0.id,255
```



注意： 通道号为 255 时表示清除此曲线控件内的所有通道数据。

4.1.16.3 cle 指令-样例工程下载

演示工程下载链接：

《cle 指令》演示工程下载

4.1.16.4 cle-c 语言示例

单片机通过串口清除曲线控件中的数据

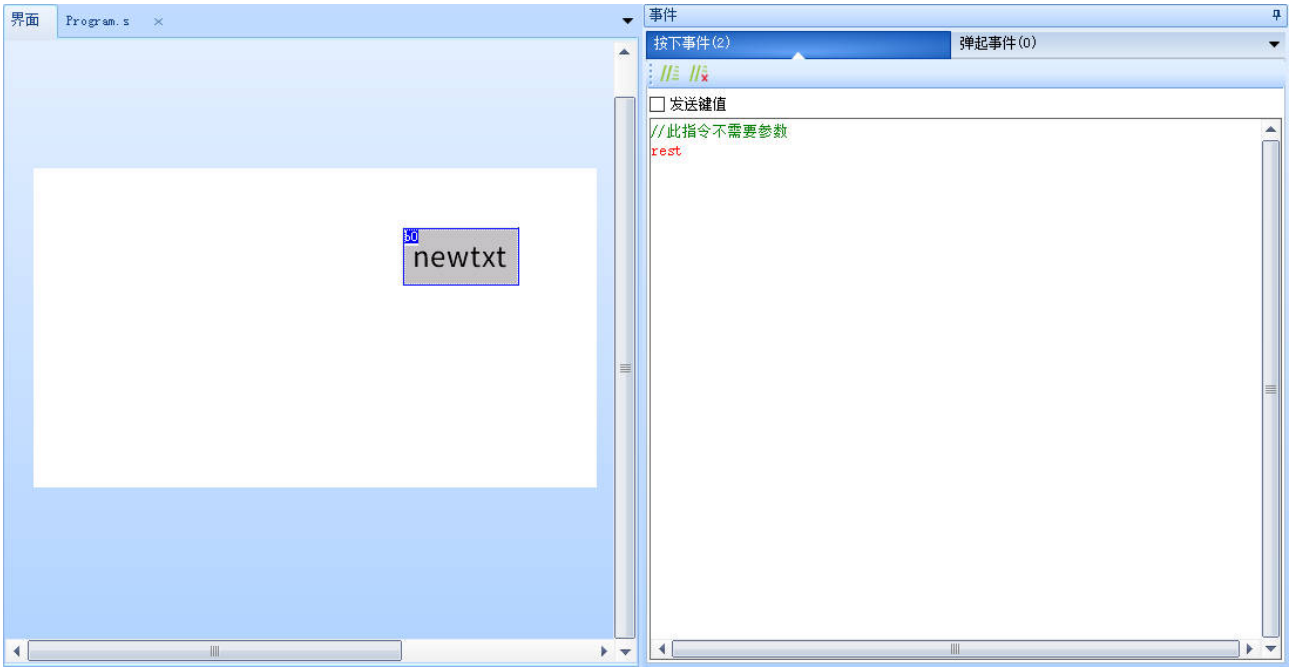
```
//清除曲线控件s0的0通道数据,cle指令不支持跨页面
printf("cle s0.id,0\xff\xff\xff");

//清除曲线控件s1的1通道数据,cle指令不支持跨页面
printf("cle s1.id,1\xff\xff\xff");

//清除曲线控件s2的所有通道数据,cle指令不支持跨页面
printf("cle s2.id,255\xff\xff\xff");
```

4.1.17 rest-复位串口屏

```
rest
```

4.1.17.1 rest-示例 1

```
//此指令不需要参数
rest
```



注意： 串口屏将直接重启

危险： 禁止在 program.s 以及 program.s 里跳转的页面初始化事件里写 rest 指令, 会导致屏幕不断重启!!!
如果你已经这么做了, 请用 sd 卡烧写一个正常的工程进入即可

4.1.17.2 rest 指令-样例工程下载

演示工程下载链接:
《rest 指令》演示工程下载

4.2 高级指令

仅部分型号的串口屏支持下列指令

4.2.1 wepo-写入数据到掉电存储空间

(仅 k0 系列/x 系列支持)

注意: 第一次使用掉电存储空间前（新屏幕），必须对掉电存储空间进行初始化如何对掉电存储空间进行初始化 (page 1320) 。

未初始化的掉电存储空间里面有什么数据是不确定的，可能会导致程序运行出错，例如会导致模拟器中的效果与串口屏实物的效果不一致。

存储空间的读写范围是 0-1023, 当读写的是 val 属性时, 最后一个读写的位置是 1020, 因为当读写 1020 时, 其读写范围是 1020-1023。

危险: 危险提示!!!

掉电存储空间写入寿命有限，请勿频繁擦写，只建议存储低频次修改的数据，例如用户名，密码等，写入消耗掉电存储空间寿命，读取不消耗掉电存储空间寿命

K0 系列是板子上有物理 eeprom，所以每个 byte 标称 100W 次写入寿命

X 系列使用 flash 来模拟 eeprom，10W 次寿命

掉电存储空间就像 1 张纸一样，读取不消耗寿命，但是写入时需要擦除，此时纸张越来越薄，直到有一天纸张破了，就无法写入了，这将会导致屏幕功能异常!!!

不建议用户使用掉电存储空间来记录开关机时间!!!

如果以 1 秒 1 次的速度向 eeprom 写入数据，一天将会写入 86400 次左右，大约 1-10 天的时间便会将掉电存储空间的寿命用尽!!!

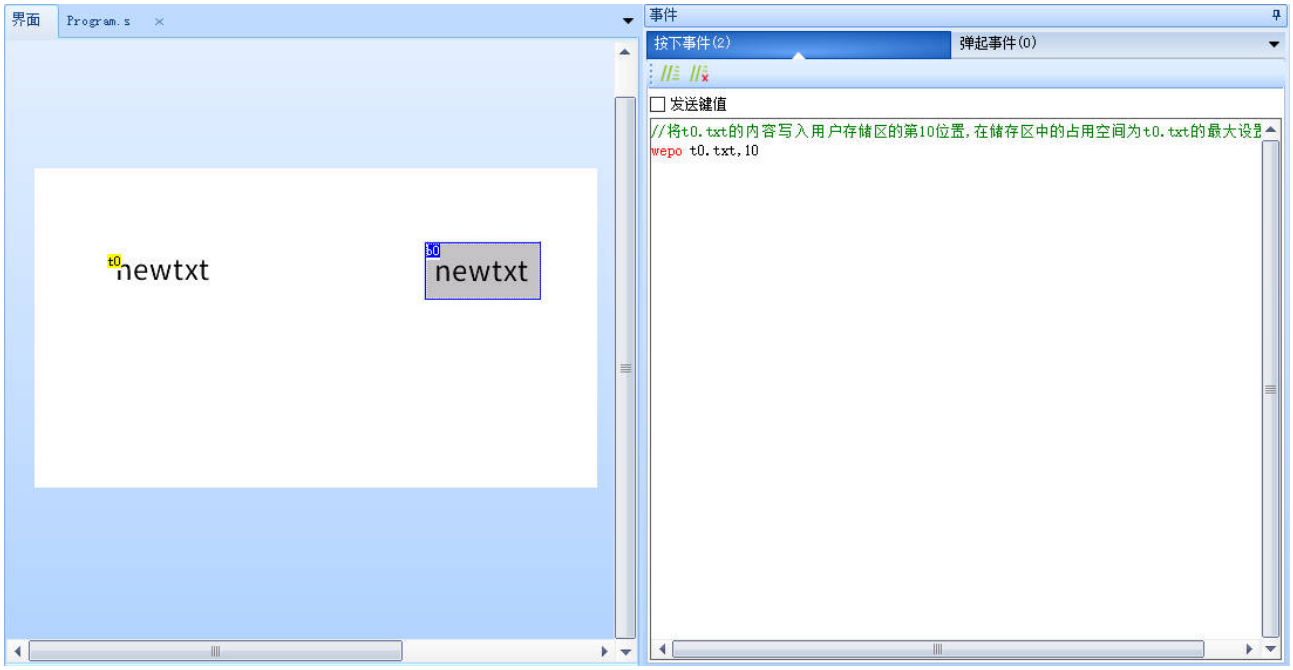
写入数据后不能马上从原有位置读取数据, 需等待几秒后再读取为什么 *wepo* 写入后不允许立刻读取或者断电 (page 1338)

写入数据后不允许立刻断电, 需等待几秒后再断电

```
wepo-att,add

att:变量/常量

add: 用户存储区位置(从0开始)
```

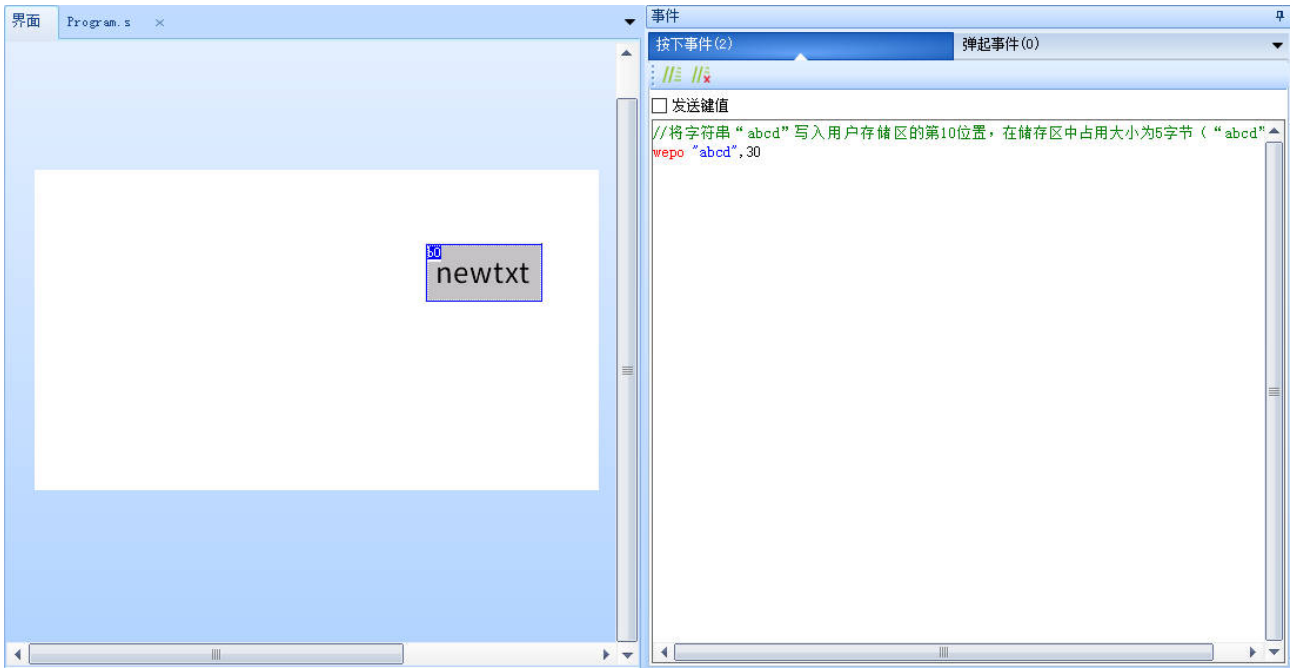
4.2.1.1 wepo-示例 1

```
//将t0.txt的内容写入用户存储区的第10位置,在储存区中的占用空间为t0.
↪txt的最大设置值+1, 即t0的txt_maxl属性的值+1
//例如控件的txt_maxl等于10, 则使用wepo写入时总共需要占用11个字节, 占用的地址为10-20
//下一个写入的地址至少为21, 否则会覆盖之前的数据导致数据丢失
wepo t0.txt,10
```



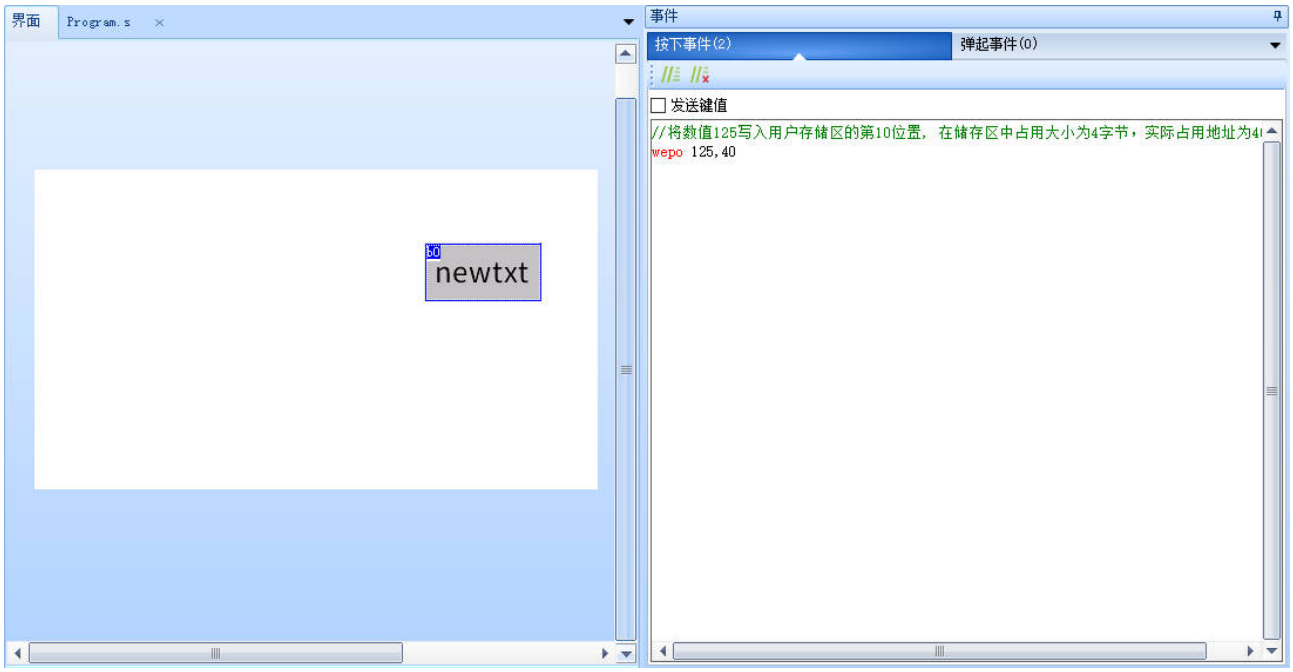
4.2.1.2 wepo-示例 2

```
//将字符串“abcd”写入用户存储区的第10位置，在储存区中占用大小为5字节
//（“abcd”共4字节加上0x00共5字节），实际占用地址为30-34
//下一个写入的地址至少为35，否则会覆盖之前的数据导致数据丢失
wepo "abcd",30
```



4.2.1.3 wepo-示例 3

```
//将数值125写入用户存储区的第10位置
//数值类型在储存区中占用大小固定为4字节，实际占用地址为40-43
//下一个写入的地址至少为44，否则会覆盖之前的数据导致数据丢失
wepo 125,40
```



备注:

1. 写入内容为变量字符串（例如 wepo t0.txt,0）的时候，在储存区中的占用空间为此控件的最大字符数 +1；

2. 写入内容为常量字符串（例如 wepo “abcd” ,0）的时候，在储存区中的占用空间为此常量字符串的实际字符数 +1, 多出来的一个字节是为了存储字符串结束符 0x00。

3. 写入内容为变量数值（例如 wepo n0.val,0）或常量数值（例如 wepo 123,0）的时候，在储存区中的占用空间统一为 4 字节。

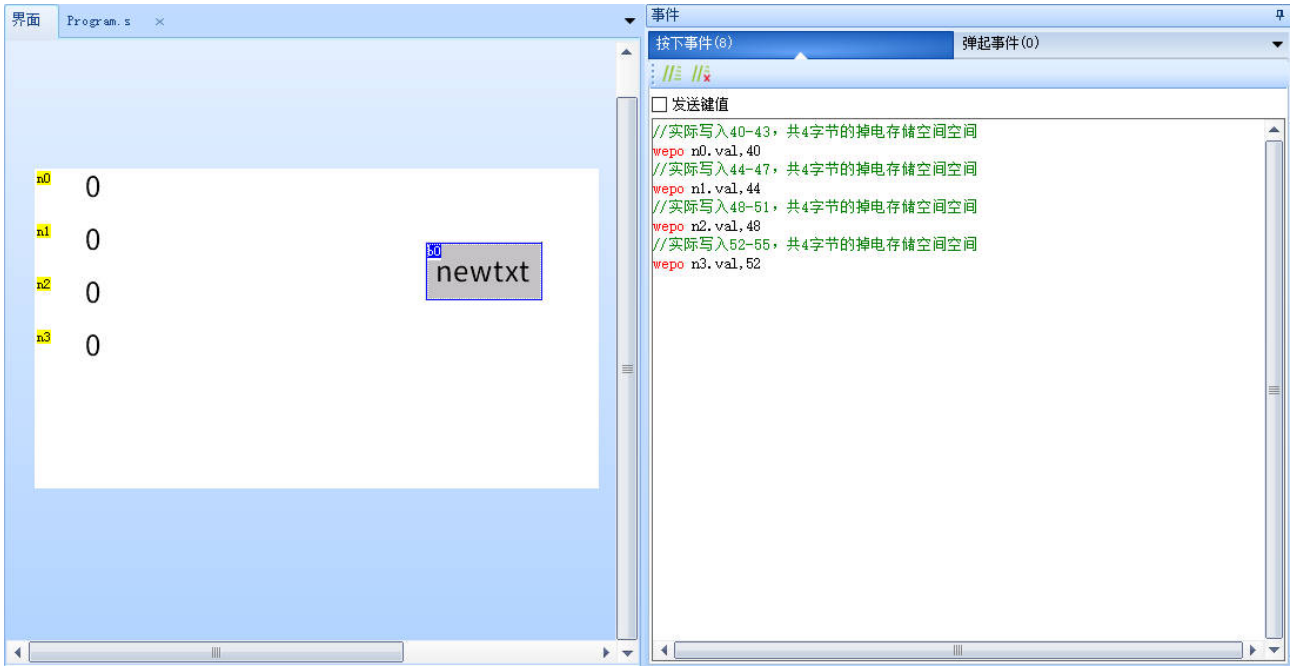
4. 使用用户存储区读写操作过程中请切记规划好数据区位置，以免位置交错引起数据覆盖错乱。

5. 用户存储区大小为 1k，位置为 0-1023

4.2.1.4 wepo-应用实例

存储多个 val 属性

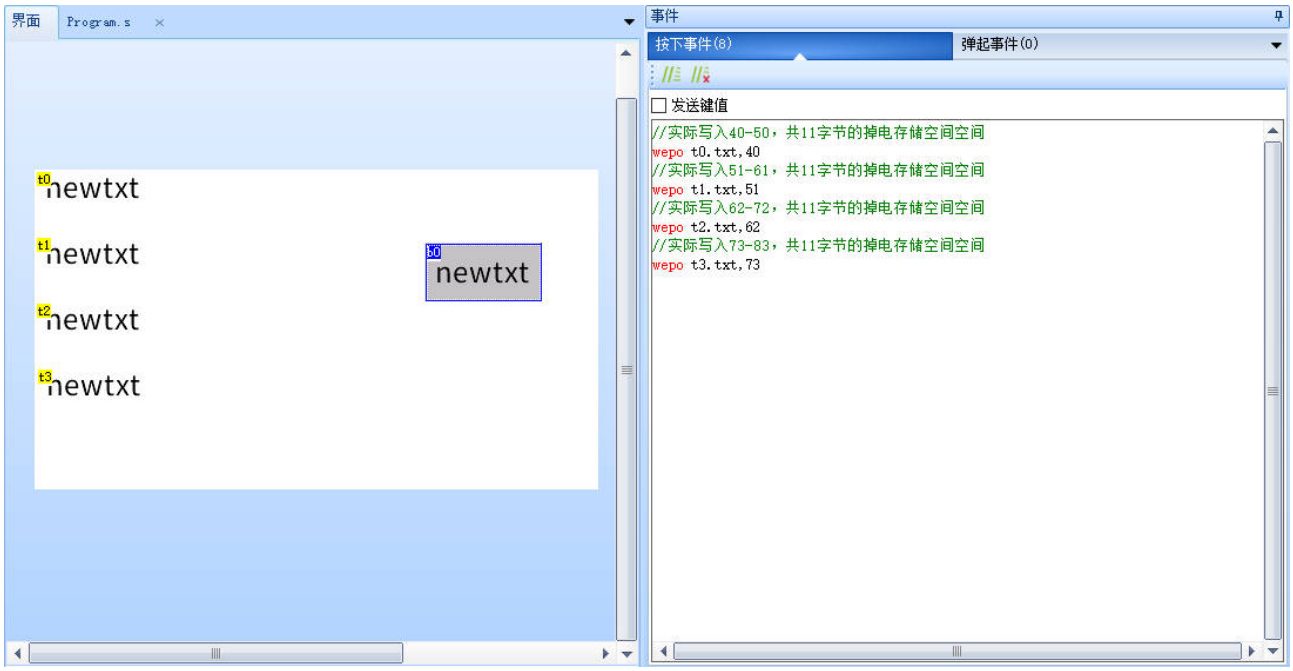
```
//实际写入40-43，共4字节的掉电存储空间空间
wepo n0.val,40

//实际写入44-47，共4字节的掉电存储空间空间
wepo n1.val,44

//实际写入48-51，共4字节的掉电存储空间空间
wepo n2.val,48

//实际写入52-55，共4字节的掉电存储空间空间
wepo n3.val,52
```



存储多个 txt 属性

假设所有的 txt_maxl 均为 10, 每次写入 txt 属性时，还需要加上字符串结束符 \0 , 因此每次会占用 11 个字节的掉电存储空间空间

```
//实际写入40-50，共11字节的掉电存储空间空间
wepo t0.txt,40

//实际写入51-61，共11字节的掉电存储空间空间
wepo t1.txt,51

//实际写入62-72，共11字节的掉电存储空间空间
wepo t2.txt,62

//实际写入73-83，共11字节的掉电存储空间空间
wepo t3.txt,73
```



使用名称组存储 n0-n9 多个连续的 val 属性, 请确保 n0-n9 的 id 号是连续的

```
eepAddr.val=40
for(sys0=n0.id;sys0<=n9.id;sys0++)
{
    wepo b[sys0].val,eepAddr.val
    eepAddr.val+=4
}
```

注意： 怎么写进去的数据就怎么读出来，以字符串方式写进去的数据，就要读取到字符串属性中，以数值类型写进去的数据，就要读取到数值类型属性中

接下来演示将 n0-n9 从 eeprom 中读取出来

```
eepAddr.val=40
for(sys0=n0.id;sys0<=n9.id;sys0++)
{
    repo b[sys0].val,eepAddr.val
    eepAddr.val+=4
}
```

4.2.1.5 wepo-问答测试

1.wepo n0.val,100 ，下一个 wepo 指令写入的位置是哪里

提示： val 变量固定占用 4 字节，因此 wepo n0.val,100 这个值占用了 100~103 的位置，下一个写入位置是 104。

2.wepo t0.txt,200 ,t0 的 txt_maxl 属性为 20，下一个 wepo 指令写入的位置是哪里

提示： 在储存区中的占用空间为 t0.txt_maxl 的值 +1，即占用了 21 字节，占用了 200~220 的位置（不足的部分会被写入为 0x00），下一个写入位置是 221。

3.wepo “tjcwiki”,300 ，下一个 wepo 指令写入的位置是哪里

提示： “tjcwiki”这个字符串的长度为 7，在储存区中的占用空间为 7+1，即占用了 8 字节，占用了 300~307 的位置，下一个写入位置是 308。

4.2.1.6 wepo 指令-样例工程下载

演示工程下载链接：

《多种控件演示》演示工程下载

《数字控件写入和读取 eeprom》演示工程下载

《文本控件写入和读取 eeprom》演示工程下载

4.2.1.7 wepo 指令-相关链接

如何对掉电存储空间进行初始化 (page 1320)
repo-从掉电存储空间读取数据 (page 640)
wept-通过串口透传数据到掉电存储空间 (page 643)
rept-从掉电存储空间读取数据并透传发送到串口 (page 646)

4.2.2 repo-从掉电存储空间读取数据

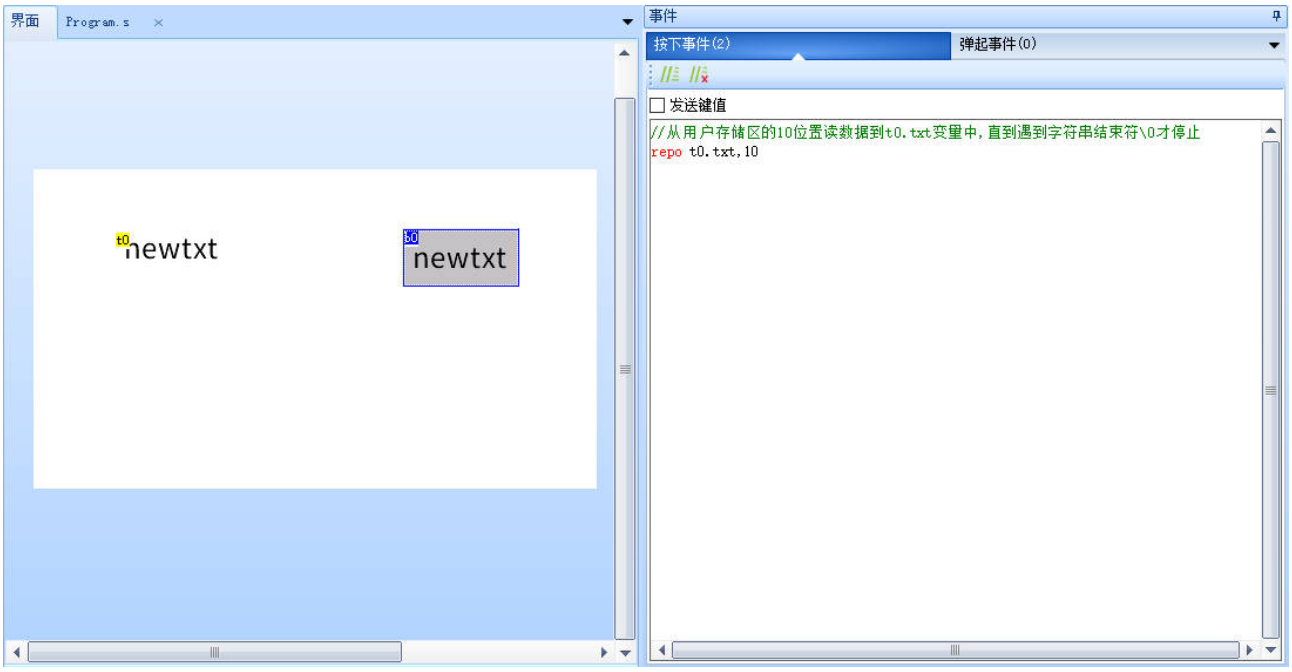
(仅 k0 系列/x 系列支持)

> **注意：** 第一次使用掉电存储空间前（新屏幕），必须对掉电存储空间进行初始化如何对掉电存储空间进行初始化 (page 1320)
>
> 未初始化的掉电存储空间里面有什么数据是不确定的，如果未初始化就直接读取，可能会导致程序运行出错，例如会导致模拟器中的效果与串口屏实物的效果不一致
>
> 读取不消耗掉电存储空间寿命
>
> 存储空间的读写范围是 0-1023, 当读写的是 val 属性时, 最后一个读写的位置是 1020, 因为当读写 1020 时, 其读写范围是 1020-1023。

```
repo att,add

att:目标变量

add: 用户存储区位置(从0开始)
```

4.2.2.1 repo-示例 1

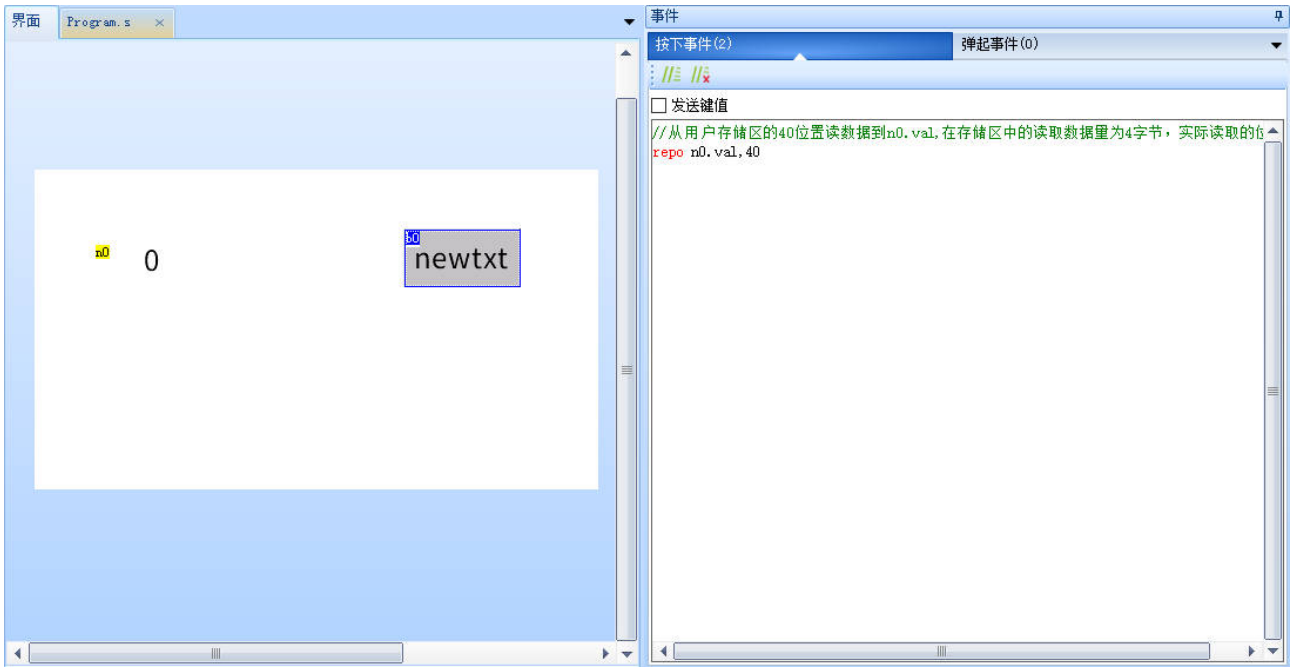
```
//从用户存储区的10位置读数据到t0.txt变量中,直到遇到字符串结束符\0才停止
repo t0.txt,10
```



> **注意：** 读取到 txt 属性时遇到字符串结束符 0 才停止，因此之前通过 wepo 或者 wept 写入字符串类型的变量时，一定要留有空间存储 0, 否则读取时就会出错

4.2.2.2 repo-示例 2

```
//从用户存储区的40位置读数据到n0.val,在存储区中的读取数据量为4字节，实际读取的位置为10-13
repo n0.val,40
```



注意： 读入内容为变量字符串的时候，在储存区中的读取数据量为此变量的最大字符数 +1。
读入内容为变量数值时候，在储存区中的读取数据量统一为 4 字节。
使用用户存储区读写操作过程中请切记规划好数据区位置，以免位置交错引起数据覆盖错乱。
用户存储区大小为 1k，位置为 0-1023

4.2.2.3 repo-应用实例

读取多个连续的 val 属性

```
//实际读取40-43
repo n0.val,40

//实际读取44-47
repo n1.val,44

//实际读取48-51
repo n2.val,48

//实际读取52-55
repo n3.val,52
```

使用名称组读取 n0-n9 多个连续的 val 属性, 请确保 n0-n9 的 id 号是连续的

```
eepAddr.val=40
for(sys0=n0.id;sys0<=n9.id;sys0++)
{
    repo b[sys0].val,eepAddr.val
```

(续下页)

(接上页)

```
    eepAddr.val+=4
}
```

读取多个 txt 属性

```
//从40地址开始读取，直到读取到 \0 ,才会停止读取
repo t0.txt,40

//从51地址开始读取，直到读取到 \0 ,才会停止读取
repo t1.txt,51

//从62地址开始读取，直到读取到 \0 ,才会停止读取
repo t2.txt,62

//从73地址开始读取，直到读取到 \0 ,才会停止读取
repo t3.txt,73
```

注意： 怎么写进去的数据就怎么读出来，以字符串方式写进去的数据，就要读取到字符串属性中，以数值类型写进去的数据，就要读取到数值类型属性中

4.2.2.4 repo 指令-样例工程下载

演示工程下载链接：

《多种控件演示》演示工程下载

《数字控件写入和读取 eeprom》演示工程下载

《文本控件写入和读取 eeprom》演示工程下载

4.2.2.5 repo 指令-相关链接

如何对掉电存储空间进行初始化 (page 1320)

wepo-写入数据到掉电存储空间 (page 634)

wept-通过串口透传数据到掉电存储空间 (page 643)

rept-从掉电存储空间读取数据并透传发送到串口 (page 646)

4.2.3 wept-通过串口透传数据到掉电存储空间

(仅 k0 系列/x 系列支持)

注意: 第一次使用掉电存储空间前（新屏幕），必须对掉电存储空间进行初始化如何对掉电存储空间进行初始化 (page 1320)

未初始化的掉电存储空间里面有什么数据是不确定的，可能会导致程序运行出错，例如会导致模拟器中的效果与串口屏实物的效果不一致

存储空间的读写范围是 0-1023。

危险: 危险提示!!!

掉电存储空间写入寿命有限，请勿频繁擦写，只建议存储低频次修改的数据，例如用户名，密码等，写入消耗掉电存储空间寿命，读取不消耗掉电存储空间寿命

掉电存储空间就像 1 张纸一样，读取不消耗寿命，但是写入时需要擦除，此时纸张越来越薄，直到有一天纸张破了，就无法写入了，这将会导致屏幕功能异常!!!

不建议用户使用掉电存储空间来记录开关机时间!!!

如果以 1 秒 1 次的速度向 eeprom 写入数据，一天将会写入 86400 次左右，大约 1-10 天的时间便会将掉电存储空间的寿命用尽!!!

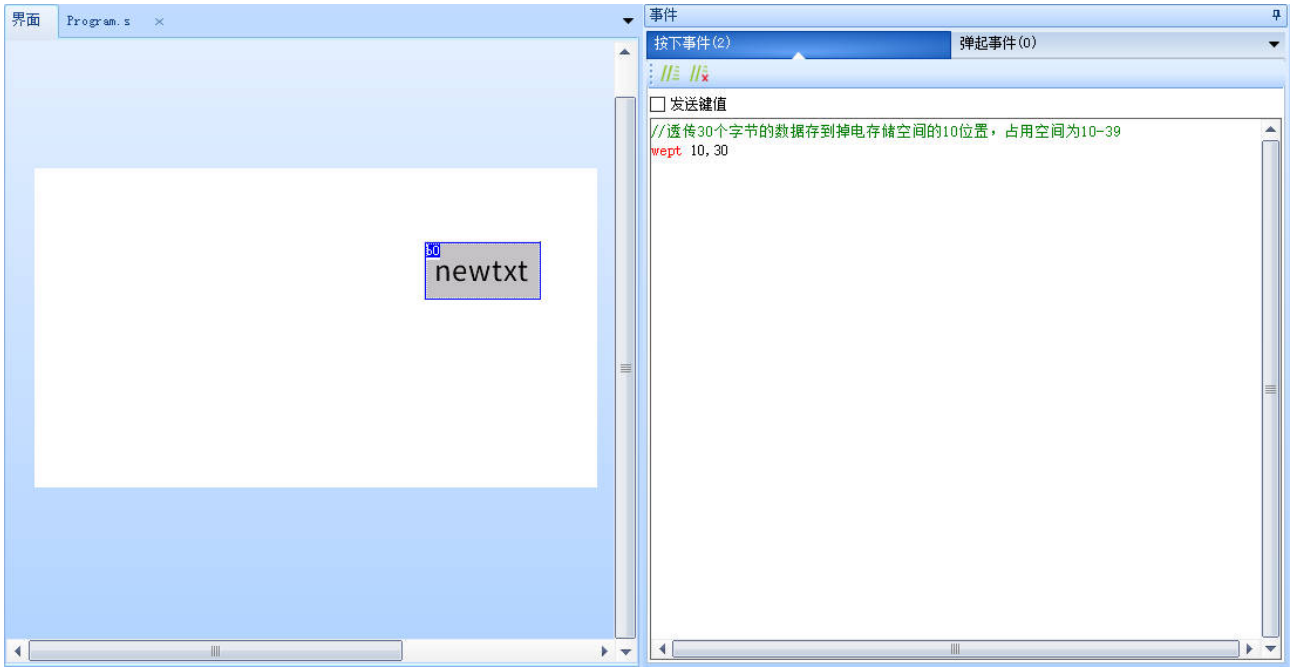
写入数据后不能马上从原有位置读取数据,需等待几秒后再读取

写入数据后不允许立刻断电,需等待几秒后再断电

```
wept add,lenth

add: 用户存储区位置(从0开始)

lenth:透传长度
```

4.2.3.1 wept-示例

```
//透传30个字节的数据存到掉电存储空间的10位置，占用空间为10-39
wept 10,30
```



备注：

1. 发完透传指令后，用户需要等待设备响应才能开始透传数据，设备收到透传指令后，准备透传初始化数据大概需要 5ms 左右 (如果在透传指令执行前串口缓冲区还有很多别的指令，那时间会更长)，设备透传初始化准备好以后会发送一个透传就绪的数据给用户（0XFE+ 结束符），表示设备已经准备好，此时可以开始发送透传数据。透传数据为纯 16 进制数据，不再使用字符串，也不再需要结束符，设备收完指定的数据量以后，才会恢复指令接收状态。否则一直处于数据透传状态，透传数据完成以后，设备会发送结束标记给用户（0XFD+ 结束符）。

2. 用户存储区大小为 1k，位置为 0-1023

4.2.3.2 wept-c 语言示例

透传 30 个字节的数据存到掉电存储空间的 10 位置，占用空间为 10-39

```
//透传30个字节的数据存到掉电存储空间的10位置，占用空间为10-39
printf("wept 10,30\xff\xff\xff");

//等待适量时间
delay_ms(100);

for(int i =0;i<30;i++)
{
    //发送16进制数据
    printf("%c",(int)(rand() % 256));
}

//确保透传结束，以免影响下一条指令
printf("\x01\xff\xff\xff");
```

4.2.3.3 wept 指令-样例工程下载

演示工程下载链接：

《wepo 指令和 repo 指令》演示工程下载

4.2.3.4 wept 指令-相关链接

如何对掉电存储空间进行初始化 (page 1320)

wepo-写入数据到掉电存储空间 (page 634)

repo-从掉电存储空间读取数据 (page 640)

rept-从掉电存储空间读取数据并透传发送到串口 (page 646)

4.2.4 rept-从掉电存储空间读取数据并透传发送到串口

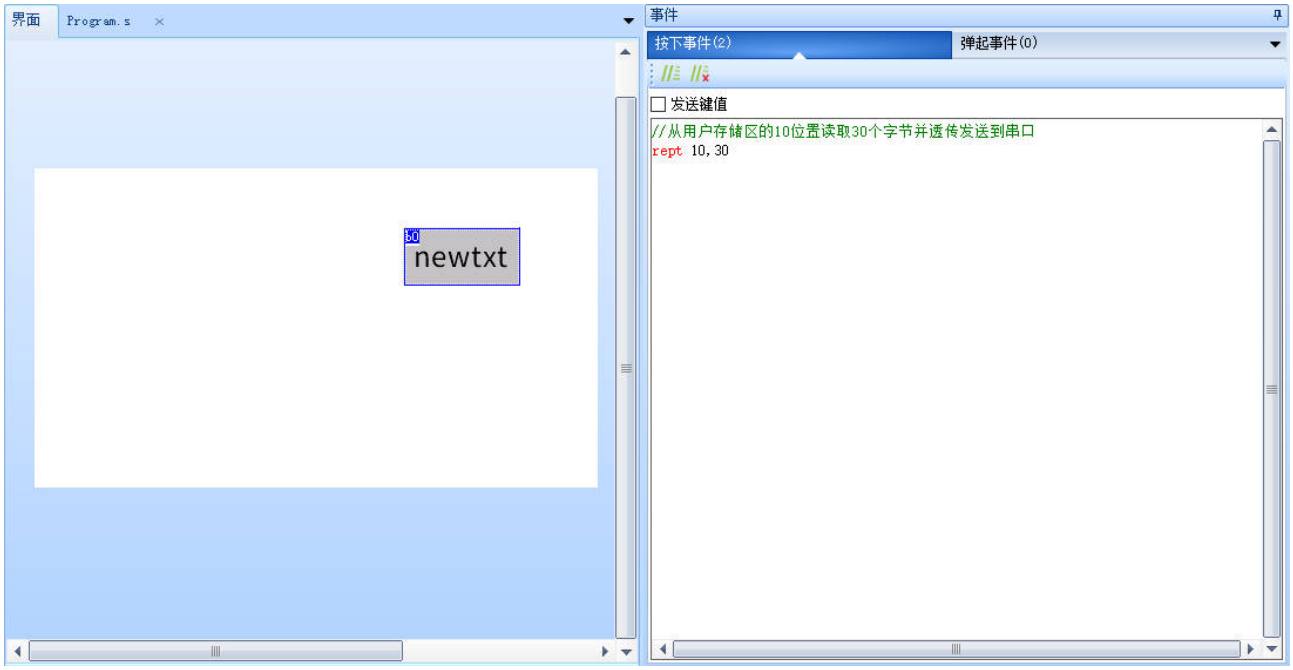
(仅 k0 系列/x 系列支持)

> **注意:** 第一次使用掉电存储空间前（新屏幕），必须对掉电存储空间进行初始化如何对掉电存储空间进行初始化 (page 1320)
>
> 未初始化的掉电存储空间里面有什么数据是不确定的，可能会导致程序运行出错，例如会导致模拟器中的效果与串口屏实物的效果不一致
>
> 读取不消耗掉电存储空间寿命
>
> 存储空间的读写范围是 0-1023。

```
rept add,lenth

add: 用户存储区位置(从0开始)

lenth:读取并透传发送的长度
```

4.2.4.1 rept-示例 1

```
//从用户存储区的10位置读取30个字节并透传发送到串口
rept 10,30
```



> **注意:** 不管存储区中的数据是字符串还是数值，设备都按 16 进制来读取和发送指定的字节数量到串口，并且不会发结束符。
>
> 用户存储区大小为 1k，位置为 0-1023

4.2.4.2 rept 指令-样例工程下载

演示工程下载链接：

《wepo 指令和 repo 指令》演示工程下载

4.2.4.3 rept 指令-相关链接

如何对掉电存储空间进行初始化 (page 1320)

wepo-写入数据到掉电存储空间 (page 634)

repo-从掉电存储空间读取数据 (page 640)

wept-通过串口透传数据到掉电存储空间 (page 643)

4.2.5 beep-蜂鸣器鸣叫

蜂鸣器只能在串口屏实物上使用，需要支持蜂鸣器的型号才能使用。

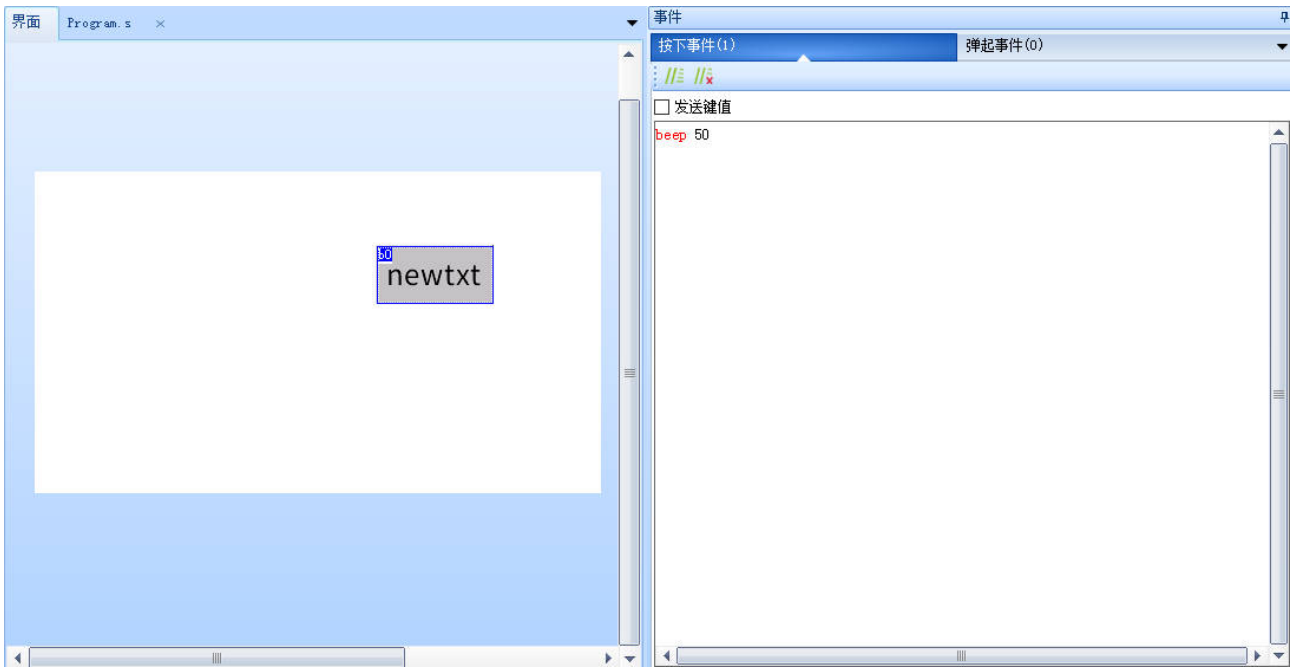
支持蜂鸣器的屏幕，屏幕背面会有黑色圆形或者黑色方形的蜂鸣器。请参考：如何判断屏幕是否支持蜂鸣器 (page 1263)

蜂鸣器的音量和频率无法调节，是固定的

```
beep time

time:时间，单位毫秒
```

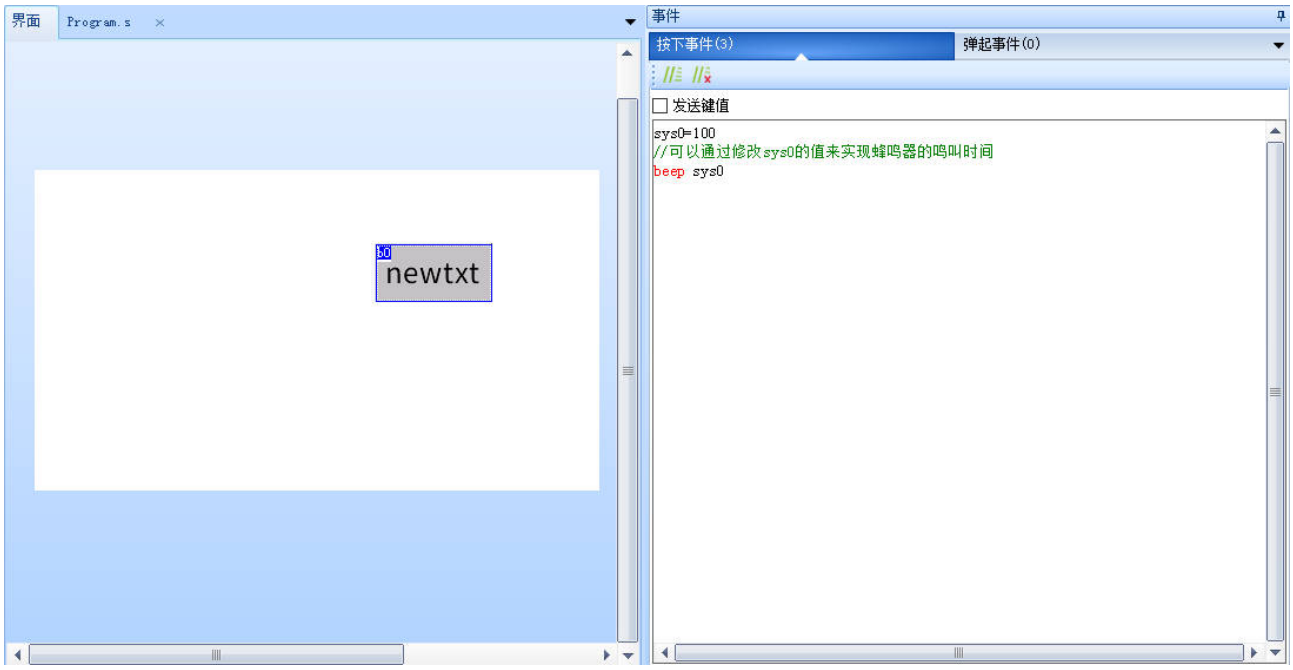
4.2.5.1 beep-示例 1

```
beep 50      //蜂鸣器鸣叫50ms
```



4.2.5.2 beep-示例 2

```
sys0=100

//可以通过修改sys0的值来实现蜂鸣器的鸣叫时间
beep sys0
```



4.2.5.3 beep-c 语言示例

单片机使用 beep 指令让串口屏蜂鸣器鸣叫 100ms

```
//串口屏蜂鸣器鸣叫100ms
printf("beep 100\xff\xff\xff");
```

提示： 仅带有蜂鸣器的型号支持, 可通过查看串口屏背面电路板是否有蜂鸣器来判断

带喇叭的型号可以导入蜂鸣器的音效素材来实现类似蜂鸣器的功能

4.2.5.4 beep 指令-样例工程下载

演示工程下载链接：

《beep 指令》演示工程下载

4.2.6 cfgpio-扩展 IO 模式配置

具体的使用方法请参考:ref: *拓展 IO 资料*

支持的型号:

x5 系列支持 8 路 IO

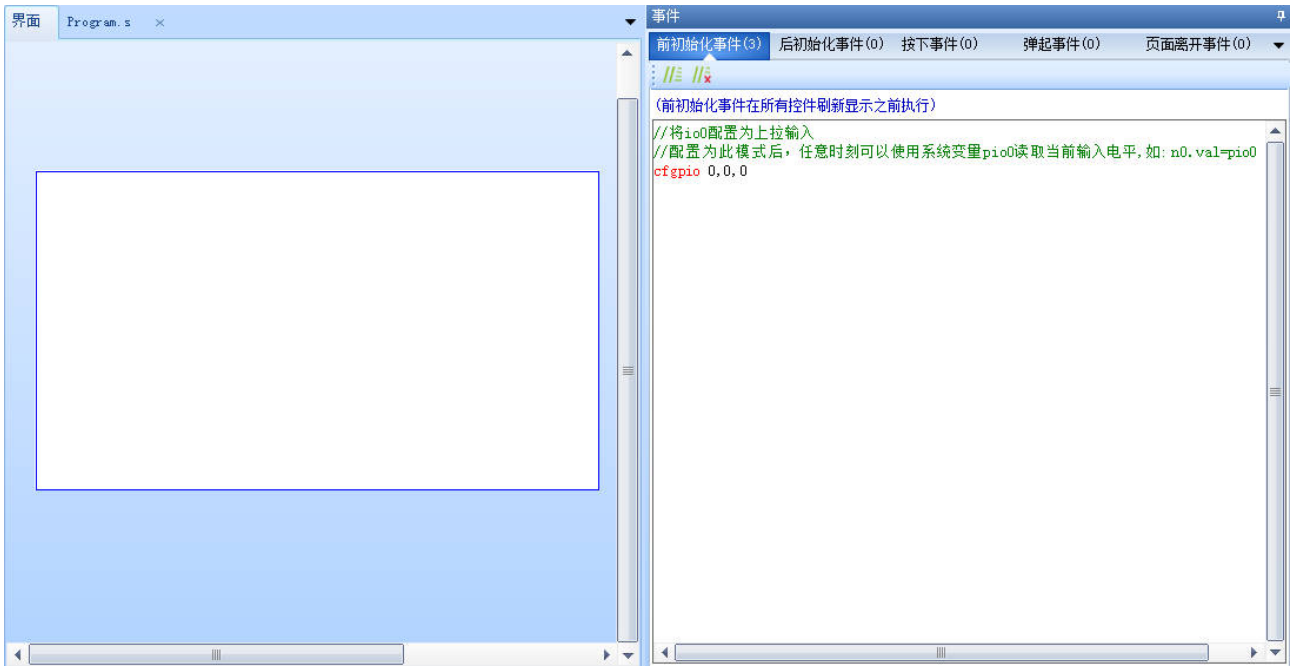
k0 系列支持 8 路 IO

X2 系列部分型号支持 8 路 IO

```
cfgpio id,state,obj

id:扩展IO的序号

state:配置模式(0-上拉输入模式,1-控件事件邦定输入模式,2-推挽输出模式,3-PWM输出模式,4-
↪开漏模式)

obj:绑定控件名称或ID(此参数仅在配置为控件事件邦定输入模式下有效，其他模式下无效)
```

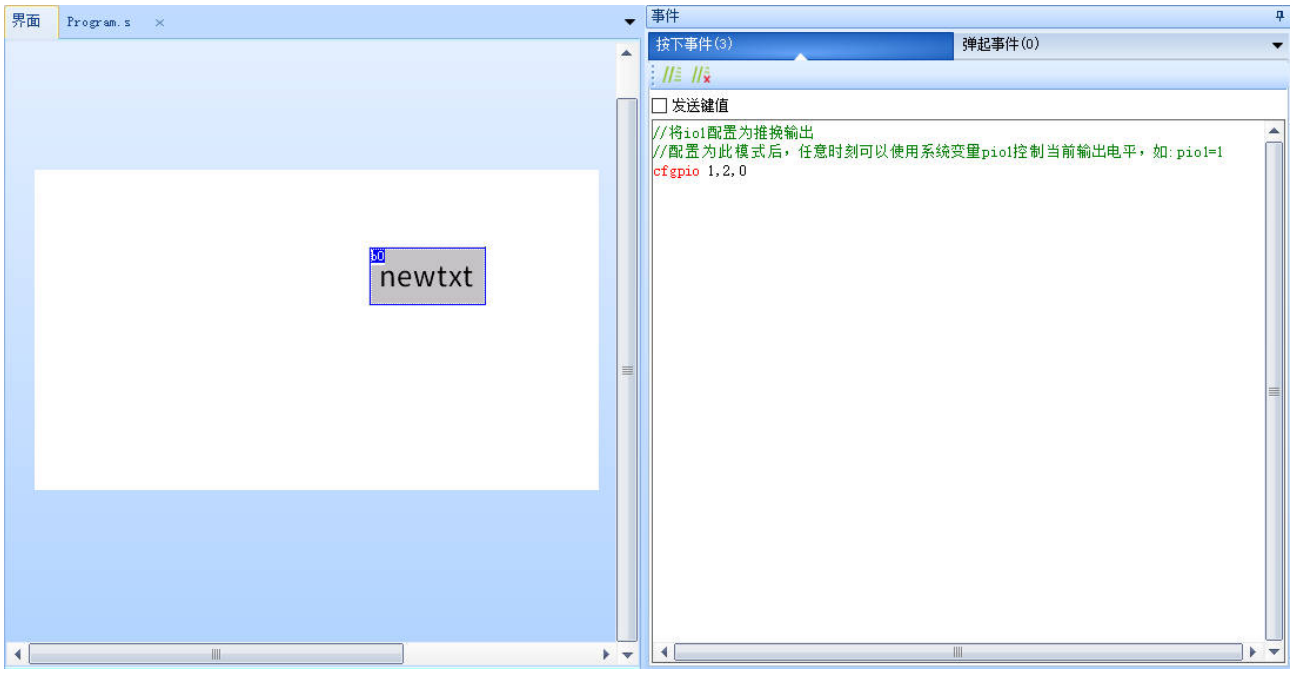
4.2.6.1 cfgpio-示例 1:

```
//将io0配置为上拉输入
//配置为此模式后，任意时刻可以使用系统变量pio0读取当前输入电平,如:n0.val=pio0
cfgpio 0,0,0
```



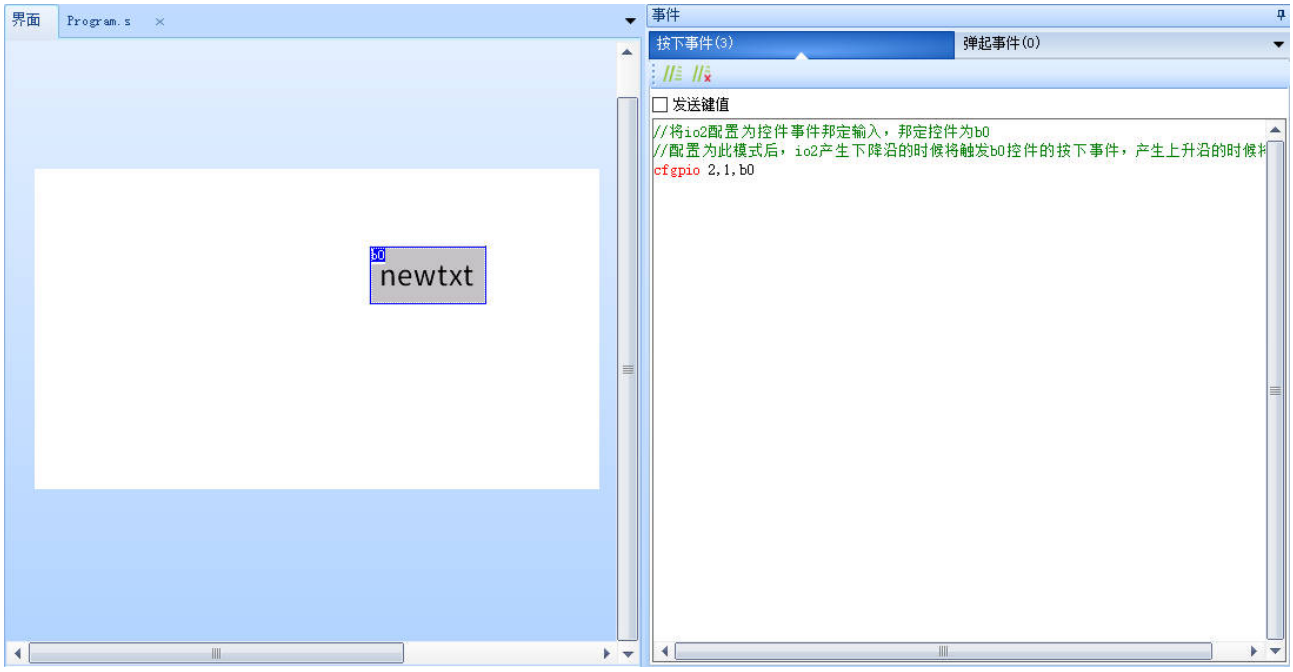
4.2.6.2 cfgpio-示例 2:

```
//将io1配置为推挽输出
//配置为此模式后，任意时刻可以使用系统变量pio1控制当前输出电平，如:pio1=1
cfgpio 1,2,0
```



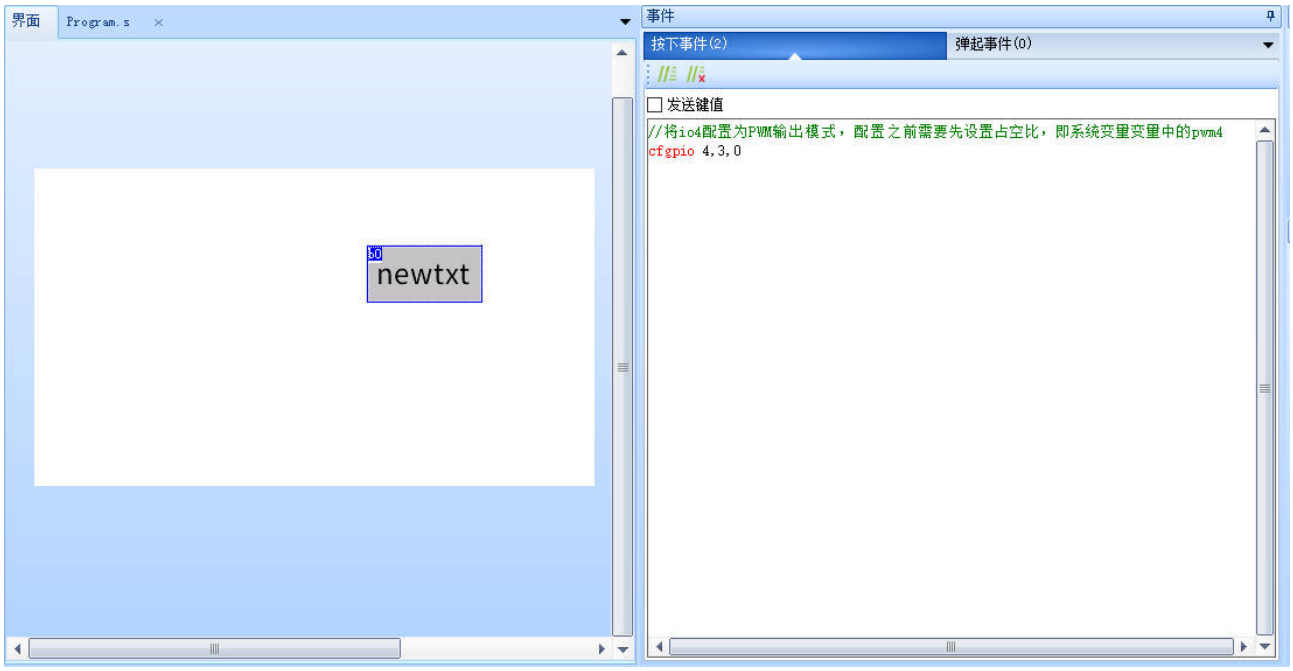
4.2.6.3 cfgpio-示例 3:

```
//将io2配置为控件事件邦定输入，邦定控件为b0
//
↪配置为此模式后，io2产生下降沿的时候将触发b0控件的按下事件，产生上升沿的时候将触发b0控件的弹起事件
cfgpio 2,1,b0
```



4.2.6.4 cfgpio-示例 4:

```
//将io4配置为PWM输出模式，配置之前需要先设置占空比，即系统变量变量中的pwm4
cfgpio 4,3,0
```



注意： K0/X2 系列只有 io4-io7 才支持 PWM 输出，X5 系列只有 io7 才支持 PWM 输出其他 IO 不支持。配置其他 IO 为 PWM 模式会报错。

使用控件事件邦定输入模式时，必须是在当前配置时刻的当前页面的控件才能邦定，不可以邦定其他页面的控件（即使是全局内存占用的控件也不可以），邦定当前页面控件以后，当重新刷新页面或者切换到别的页面后，邦定事件将不会继续触发，因此每次刷新页面需要重新邦定，建议将邦定代码写在页面的前初始化事件中最为合适。

每个需要用到外部 IO 的页面，都必须在前初始化事件中重新配置一次需要用到的相关 IO 口

所有系列拓展 IO 口电平都是 3.3v

4.2.6.5 cfgpio 指令-样例工程下载

演示工程下载链接：

《拓展 IO 例程》演示工程下载

《IOtest》演示工程下载

4.2.6.6 cfgpio 指令-相关链接

拓展 IO 资料 (page 1030)
拓展 IO 相关问题 (page 1258)
pio0~pio7-扩展 IO 端口 (page 802)
pwm4~pwm7-扩展 IO 占空比 (page 804)
pwmf-PWM 输出的频率 (page 806)
sleep 睡眠模式与拓展 IO (page 1041)

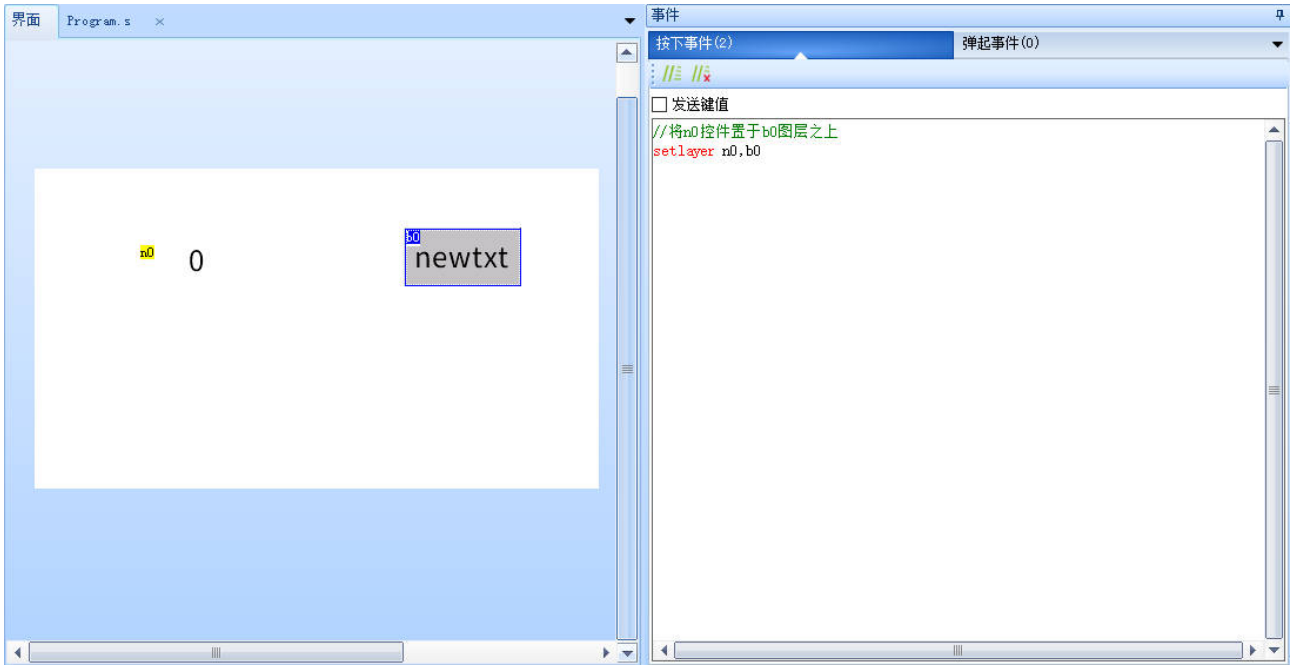
4.2.7 setlayer-运行中改变控件图层顺序

(仅 X2、X3、X5 系列支持), 不支持跨页面改变控件图层顺序，只会改变图层顺序，不会改变实际 id 值

```
setlayer obj0,obj1

obj0:需要改变图层的控件ID或控件名称

obj1:控件ID或控件名称(将obj0控件置于此控件之上,
↪此参数为0表示将obj0控件放置到所有控件的下面
,此参数为255表示将obj0控件放置到所有控件的上面)
```

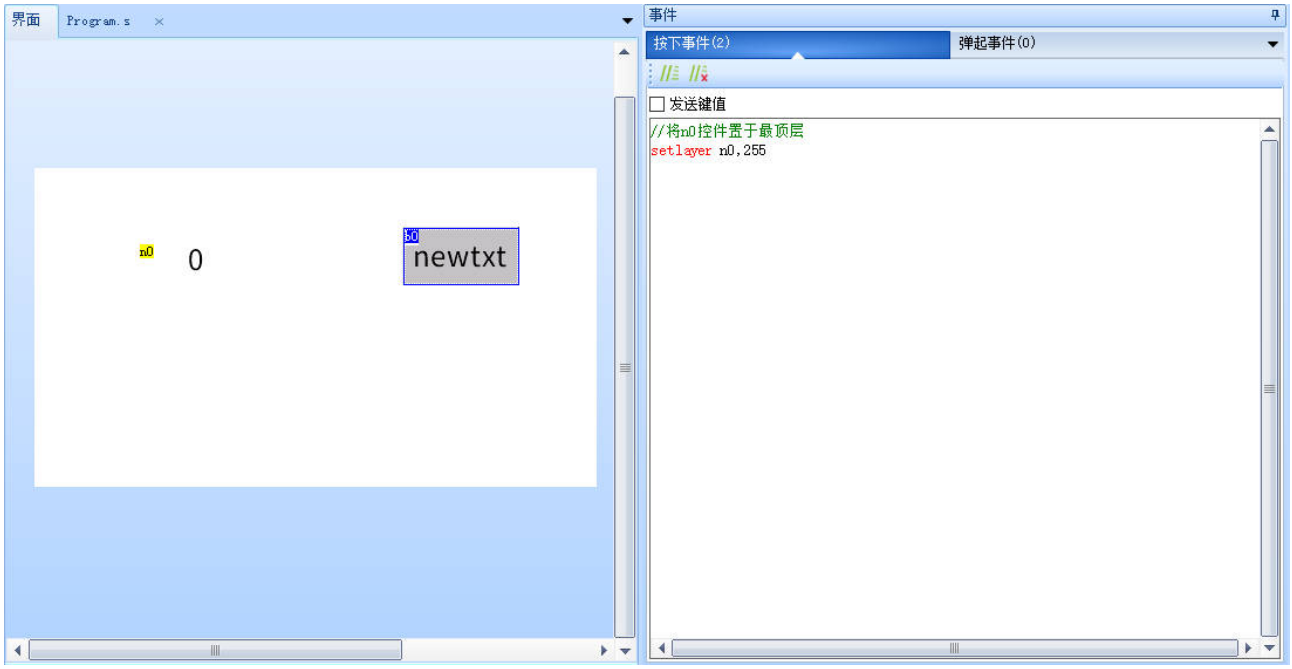
4.2.7.1 setlayer-示例 1

```
//将n0控件置于b0图层之上
setlayer n0,b0
```



4.2.7.2 setlayer-示例 2

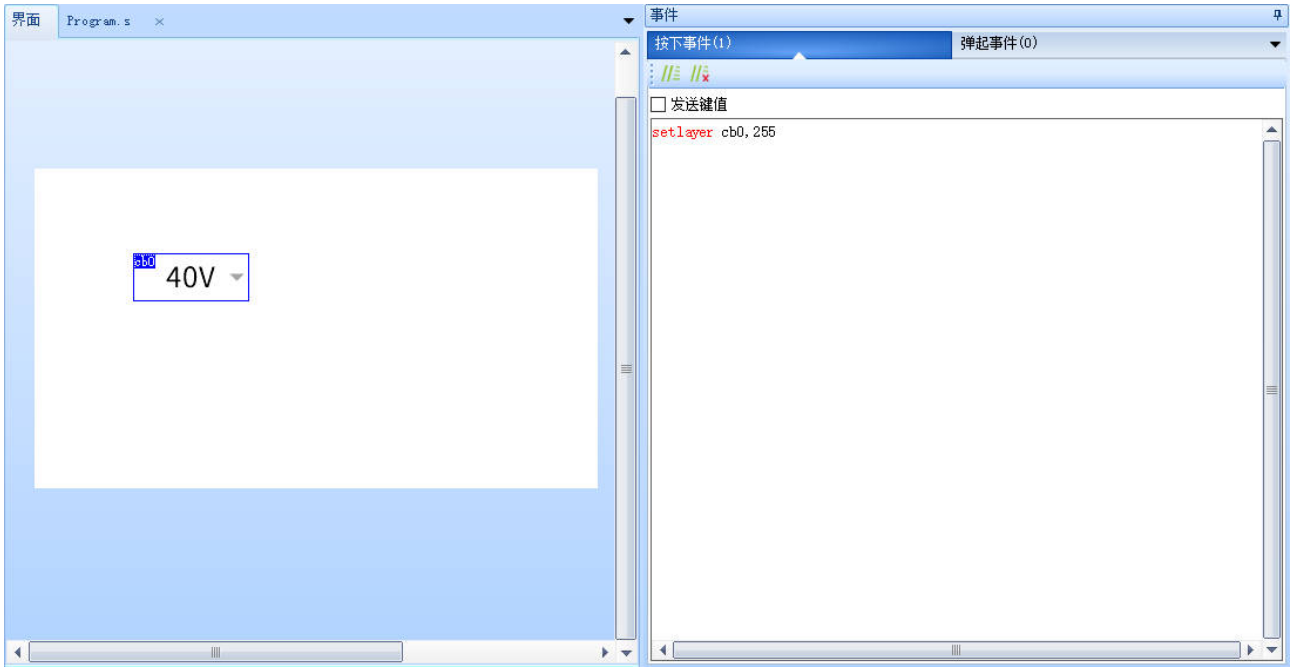
```
//将n0控件置于最顶层
setlayer n0,255
```



4.2.7.3 setlayer-示例 3

注意： 最常见的用法是将下拉框指定，避免下拉框展开时被其他空间遮挡住选项

```
//将n0控件置于最顶层
setlayer cb0,255
```



4.2.7.4 setlayer 指令-样例工程下载

演示工程下载链接：

《setlayer 指令》演示工程下载

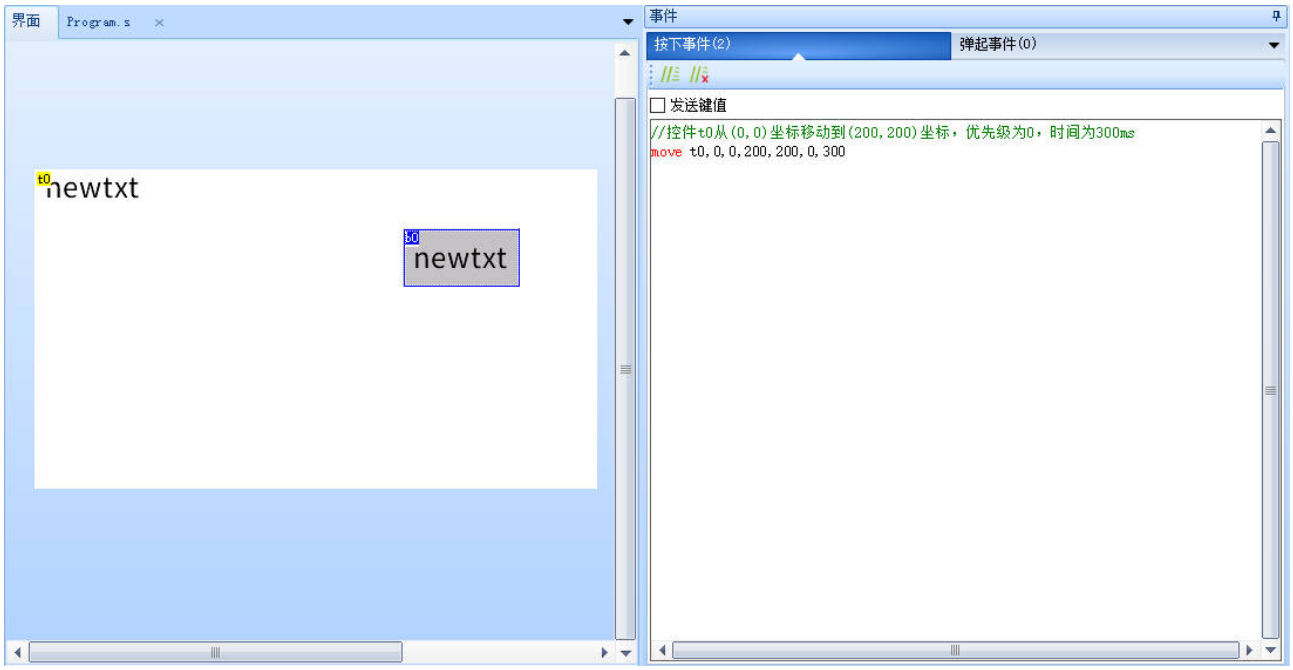
4.2.8 move-控件移动

仅 X2、X3、X5 系列支持

```
move obj,starx,stary,endx,endy,first,time

obj:控件名称或控件ID

starx:起始坐标X

stary:起始坐标Y

endx:结束坐标X

endy:结束坐标Y

first:优先级(0-100,数字越大优先级越大)

time:移动时间(单位ms)
```

4.2.8.1 move-示例

```
//控件t0从(0,0)坐标移动到(200,200)坐标，优先级为0，时间为300ms
move t0,0,0,200,200,0,300
```



注意： 可以同时写入多条不同优先级的 move 指令，系统会按照优先级顺序来移动。优先级高的会先移动，移动完成以后，才会启动下一个优先级的指令。

4.2.8.2 move 指令-样例工程下载

演示工程下载链接：

《move 指令》演示工程下载

4.2.9 play-音频播放

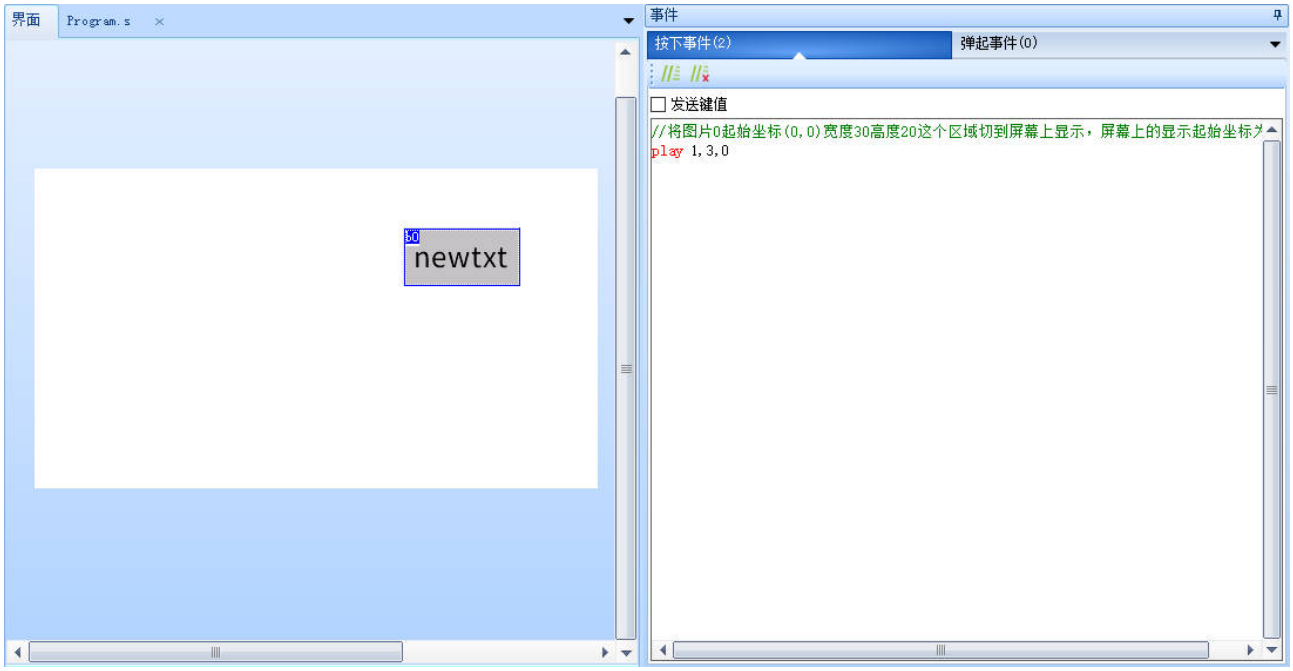
仅 X3,X5 系列支持

```
play ch,audio,loop

ch:音频通道序号

audio:音频ID

loop: 是否循环
```

> **注意:** 导入音频请参考: 如何导入音频 (page 1467)

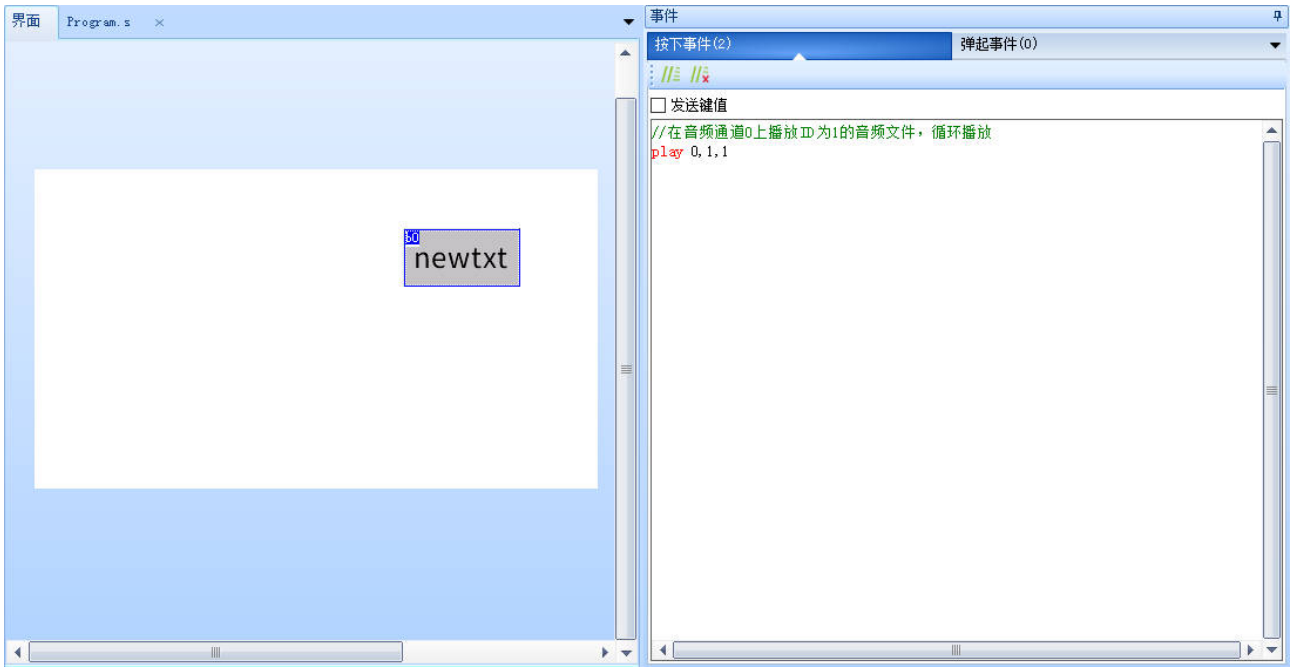
4.2.9.1 play-示例 1

```
//在音频通道1上播放ID为3的音频文件，只播放1次
play 1,3,0
```



4.2.9.2 play-示例 2

```
//在音频通道0上播放ID为1的音频文件，循环播放
play 0,1,1
```



4.2.9.3 play-c 语言示例

单片机通过串口在音频通道 1 上播放 ID 为 3 的音频文件，不循环播放

```
int ch=1, audio=3, loop=0;
printf("play %d,%d,%d\xff\xff\xff", ch, audio, loop);
```

注意： 如果使用了“play”指令后屏幕黑屏、重启，说明供电不足，请更换更大功率的电源

play 指令仅用于配置和启动音频播放，暂停和停止操作请参考：*audio0~audio1-音频通道控制* (page 808) 。

play 指令控制的是独立于视频之外的音频通道，与视频中使用的音频通道没有关系，也不会产生冲突。

音频播放功能是全局的，不属于某个页面，因此 play 指令启动播放后，即便是跳转页面，音频依然会继续播放，如果希望离开页面后停止播放，可以在页面的离开事件中使用 audio0/audio1 系统变量来关闭或暂停指定通道的音频播放状态。

4.2.9.4 play 指令-样例工程下载

演示工程下载链接：

《play 指令》演示工程下载

4.2.9.5 play 指令-相关链接

audio0~audio1-音频通道控制 (page 808)
volume-系统音量 (page 749)

4.3 文件操作指令

仅 X 系列串口屏支持下列指令

4.3.1 twfile-单片机发送文件给串口屏

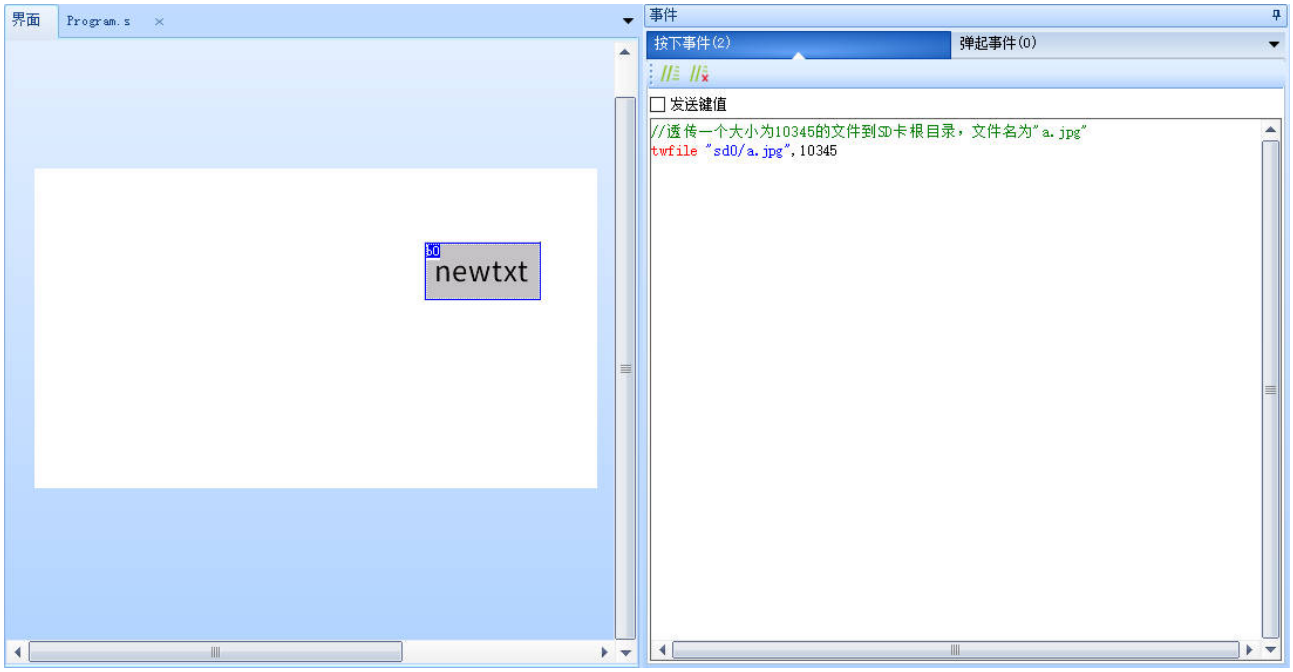
仅 X2、X3、X5 系列支持

> **注意：** twfile 有特殊的传输格式要求，详细的文件透传逻辑请参考: 运行中串口传输文件到内存或 SD 卡 (page 1022)

```
twfile filepath,filesize

filepath:目标文件路径(例:"ram/0.jpg"或"sd0/1.jpg")

filesize:文件大小
```

4.3.1.1 twfile-示例 1

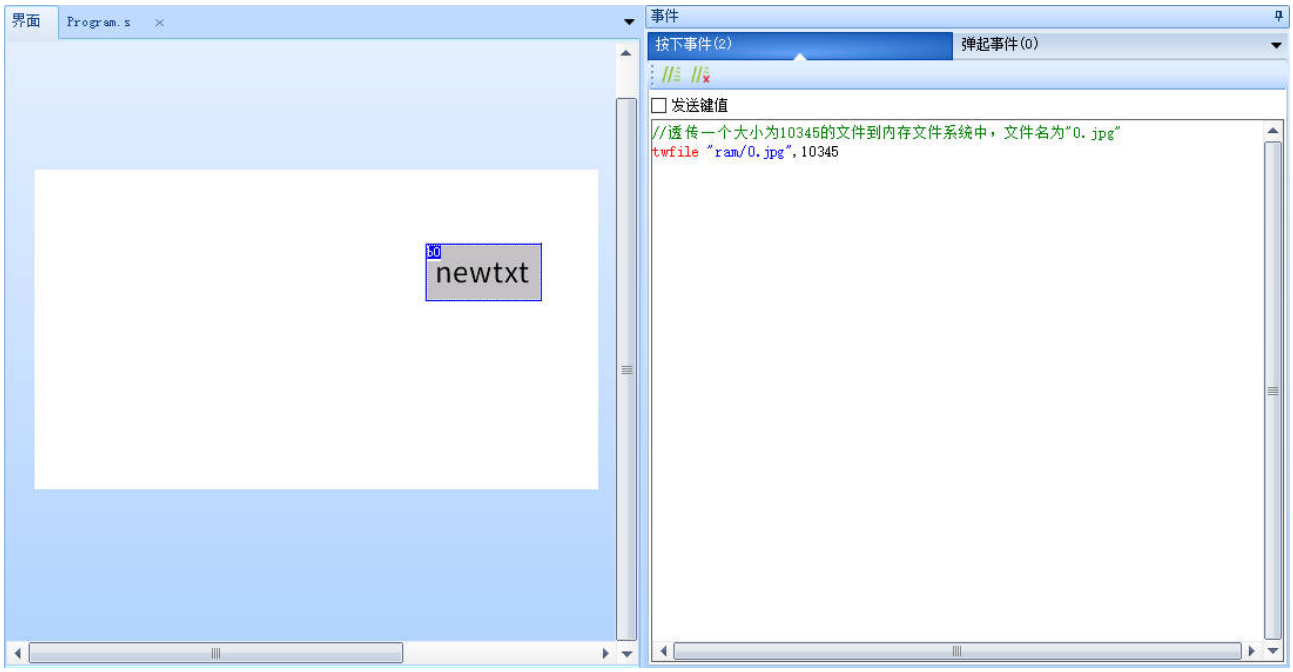
```
//透传一个大小为10345的文件到SD卡根目录，文件名为"a.jpg"
twfile "sd0/a.jpg",10345
```



> **注意：** twfile 有特殊的传输格式要求，详细的文件透传逻辑请参考: 运行中串口传输文件到内存或 SD 卡 (page 1022)

4.3.1.2 twfile-示例 2

```
//透传一个大小为10345的文件到内存文件系统中，文件名为"0.jpg"
twfile "ram/0.jpg",10345
```



注意： twfile 有特殊的传输格式要求，详细的文件透传逻辑请参考: 运行中串口传输文件到内存或 SD 卡 (page 1022)

要使用内存文件系统必须先在工程配置选项中配置内存文件系统的大小，新建工程默认内存文件系统大小为 0，即不可能使用。

4.3.2 rdfile-串口屏发送文件给单片机

仅 X2、X3、X5 系列支持

注意： 1、使用 rdfile 发送文件时，请确保文件是存在的, 建议用 findfile 指令先检查文件是否存在

2、透传文件是手动将文件一部分一部分传出来，不是一次性传出来，建议每次传输几十个字节即可，如果要发送一个完整的文件，在定时器中循环发送整个文件即可

```
rdfile filepath,addr,size,crc

filepath:文件路径

addr:件数据起始地址

size:透传读数据大小 (如果为0,则返回小端模式的4字节整形数据表示文件大小)

crc:数据尾部添加CRC检验码设置(0-无crc;1-crc16;)
```

4.3.2.1 rdfile-示例 1

```
//从SD卡根目录中的"a.jpg"文件的数据0位置开始，读取10个字节透传到串口，
↪并在数据尾部加入crc16校验码。
//合计是10+4=14字节
rdfile "sd0/a.jpg",0,10,1
```

注意： CRC16 校验算法为 MODBUS CRC16，请参考*CRC16 校验算法* (page 1016) 。

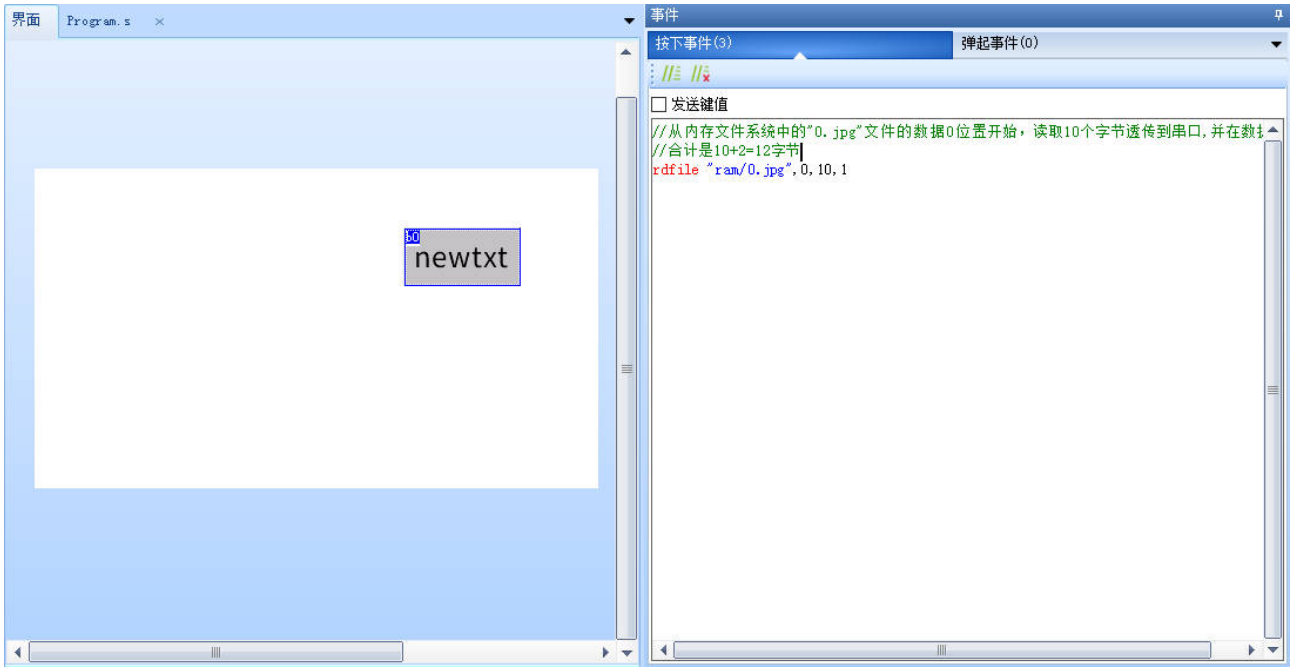
4.3.2.2 rdfile-示例 2

```
//从SD卡根目录中的"a.jpg"文件的数据0位置开始，读取10个字节透传到串口，
↪并在数据尾部加入crc16校验码。
//合计是10+4=14字节
t0.txt="sd0/a.jpg"
rdfile t0.txt,0,10,1
```

注意： CRC16 校验算法为 MODBUS CRC16，请参考*CRC16 校验算法* (page 1016) 。

4.3.2.3 rdfile-示例 3

```
//从内存文件系统中的"0.jpg"文件的数据0位置开始，读取10个字节透传到串口，
↪并在数据尾部加入crc16校验码。
//合计是10+2=12字节
rdfile "ram/0.jpg",0,10,1
```



注意： 要使用内存文件系统必须先在工程配置选项中配置内存文件系统的大小，新建工程默认内存文件系统大小为 0，即不可能使用。

CRC16 校验算法为 MODBUS CRC16，请参考*CRC16 校验算法* (page 1016) 。

4.3.2.4 rdfile 指令-样例工程下载

演示工程下载链接：

《rdfile 指令》演示工程下载

4.3.3 newfile-创建文件

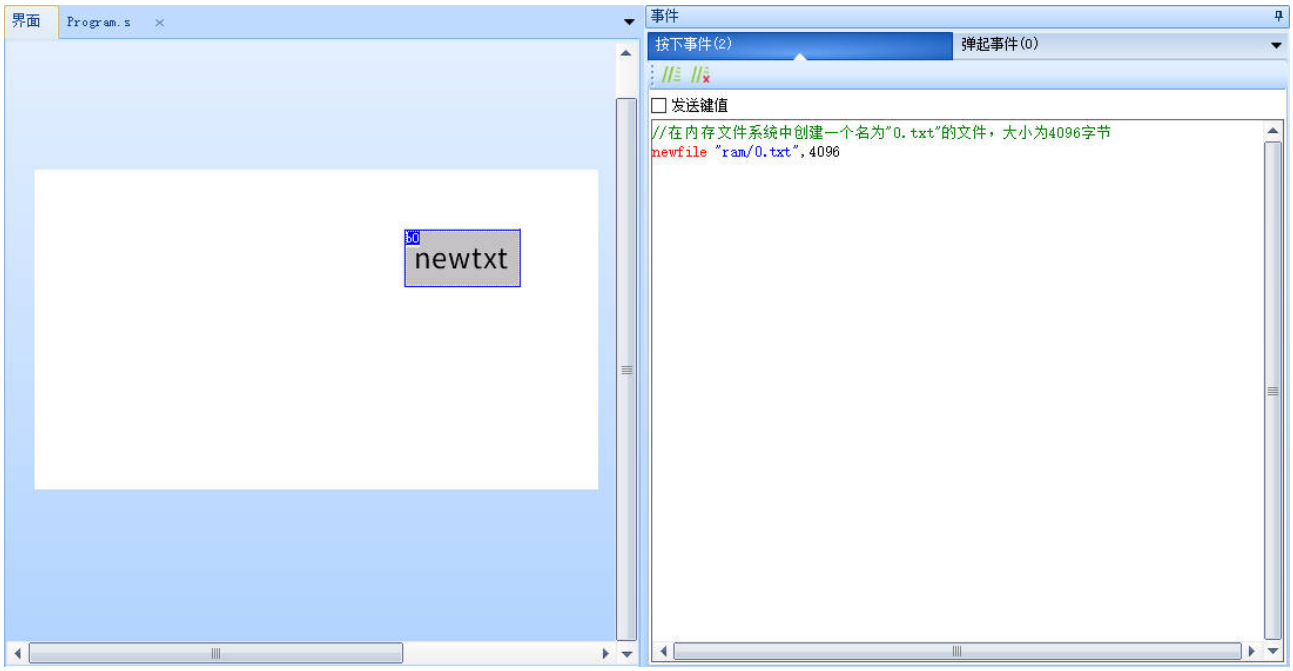
仅 X2、X3、X5 系列支持

> **注意：** 创建文件时文件名不允许包含以下字符 \ / : * ? " < > |

```
newfile filepath,size

filepath:文件路径(如:"sd0/1.txt")

size:文件大小(以字节为单位,最大2147483647)
```

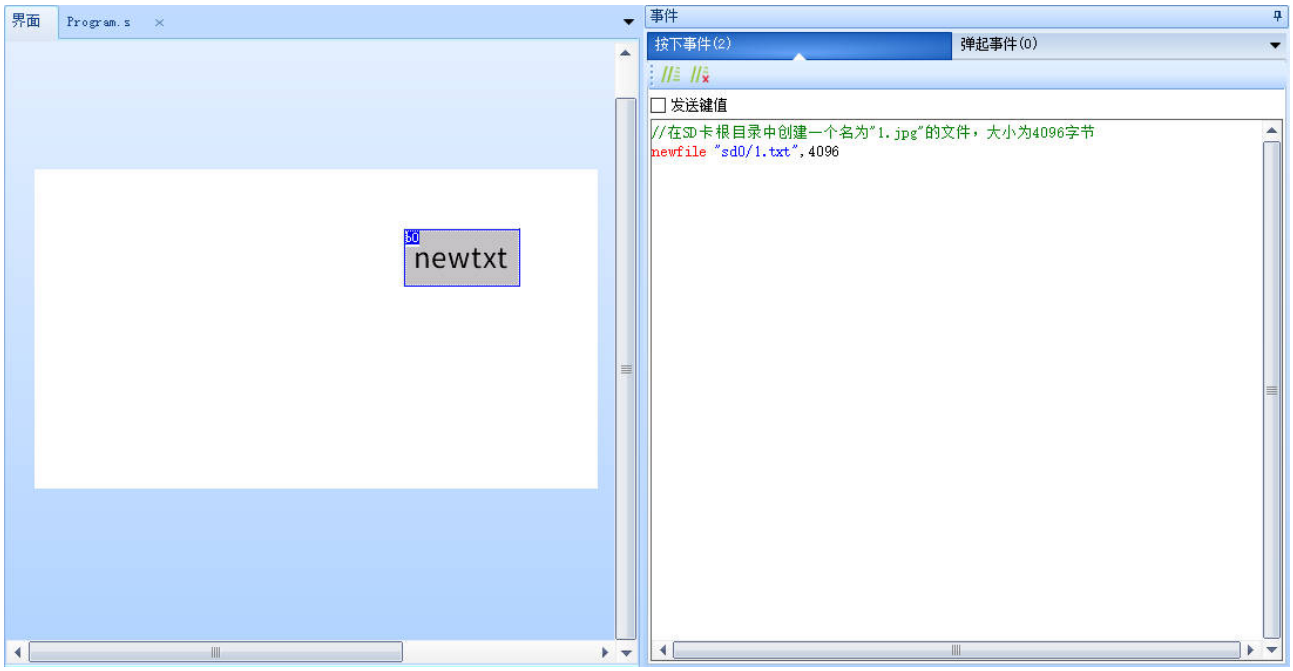
4.3.3.1 newfile-示例 1

```
//在内存文件系统中创建一个名为"0.txt"的文件，大小为4096字节
newfile "ram/0.txt",4096
```



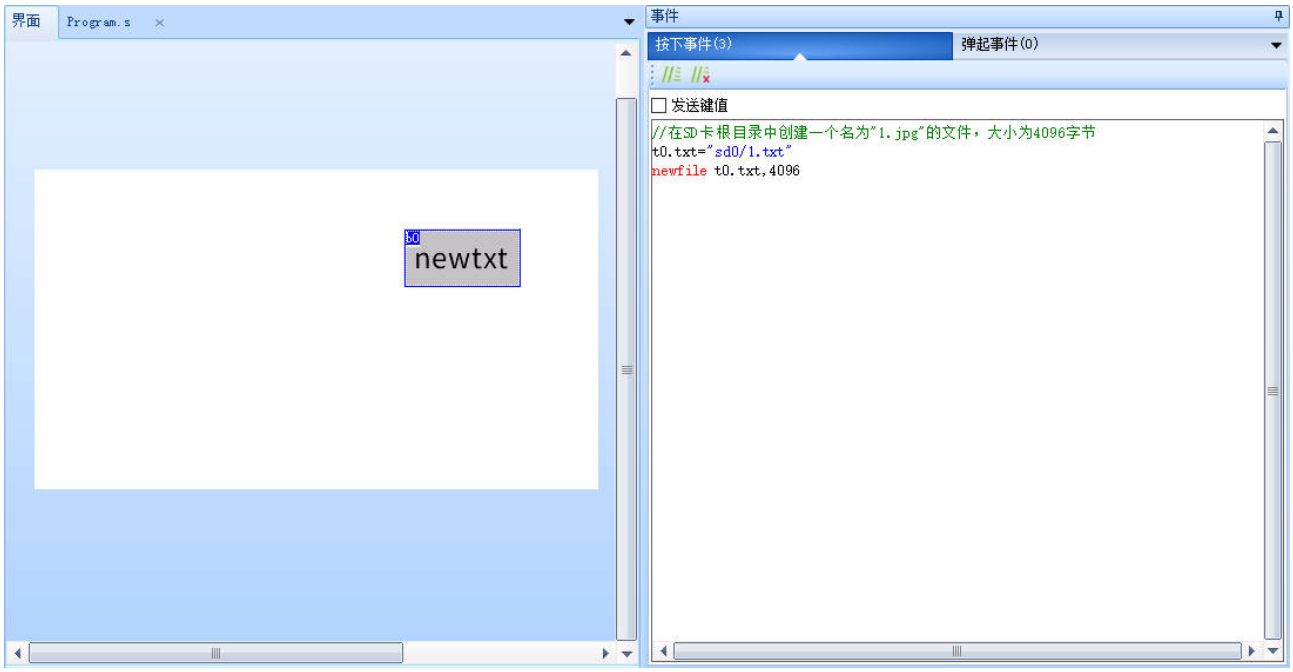
4.3.3.2 newfile-示例 2

```
//在SD卡根目录中创建一个名为"1.jpg"的文件，大小为4096字节
newfile "sd0/1.txt",4096
```



4.3.3.3 newfile-示例 3

```
//在SD卡根目录中创建一个名为"1.jpg"的文件，大小为4096字节
t0.txt="sd0/1.txt"
newfile t0.txt,4096
```



注意： 请保证 t0 的 txt_maxl 足够大，否则会提示操作文件失败。

注意： 要使用内存文件系统必须先在工程配置选项中配置内存文件系统的大小，新建工程默认内存文件系统大小为 0，即不能使用。

4.3.3.4 newfile-应用实例

创建多个有规律的文件，例如 a1.txt~a9.txt

```
for(sys0=1;sys0<=9;sys0++)
{
    covx sys0,t0.txt,0,0  //将变量转换为文本
    t1.txt="sd0/"+"a"+t0.txt+".txt"
    findfile t1.txt,sys1
    if(sys1==0)
    {
        newfile t1.txt,4096
    }

}
```

4.3.3.5 newfile 指令-样例工程下载

演示工程下载链接：

《文件浏览器 v3.0》演示工程下载

4.3.4 delfile-删除文件

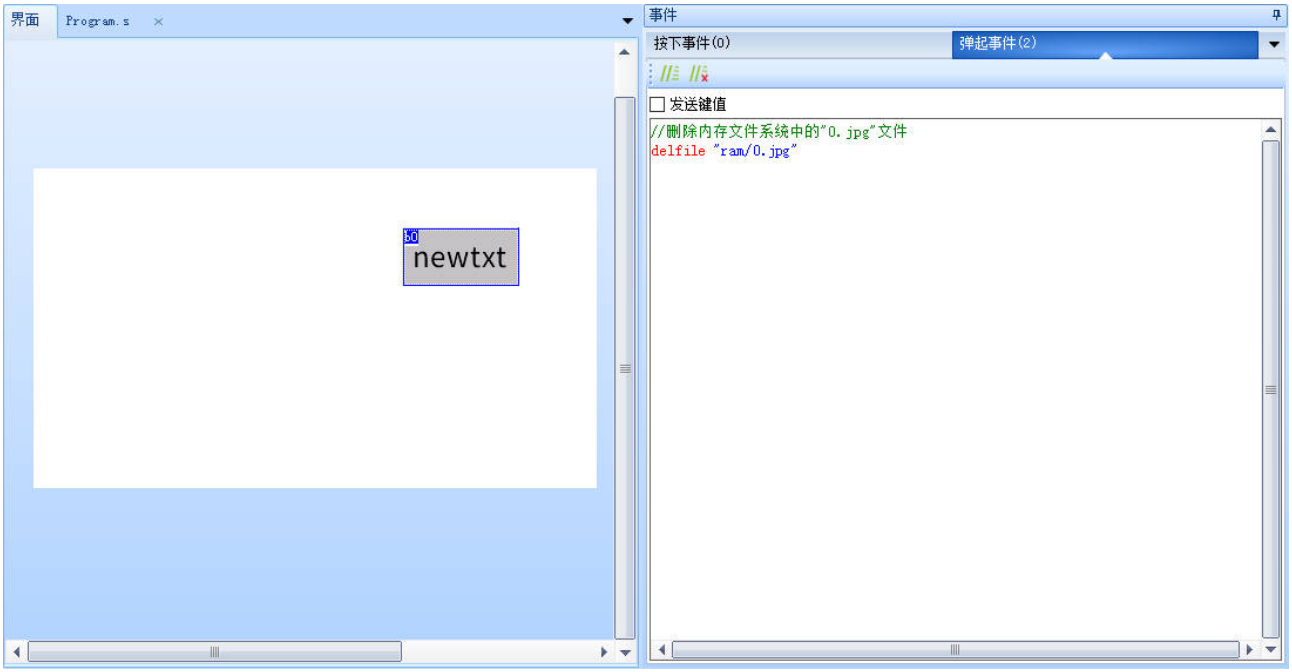
仅 X2、X3、X5 系列支持

```
delfile filepath

filepath:文件路径(例:"ram/0.jpg"或"sd0/1.jpg")
```

注意: 使用 delfile 删除文件后，如果当前页面使用了文件浏览器控件，需要重新 page 到当前页面以重新刷新文件浏览器控件

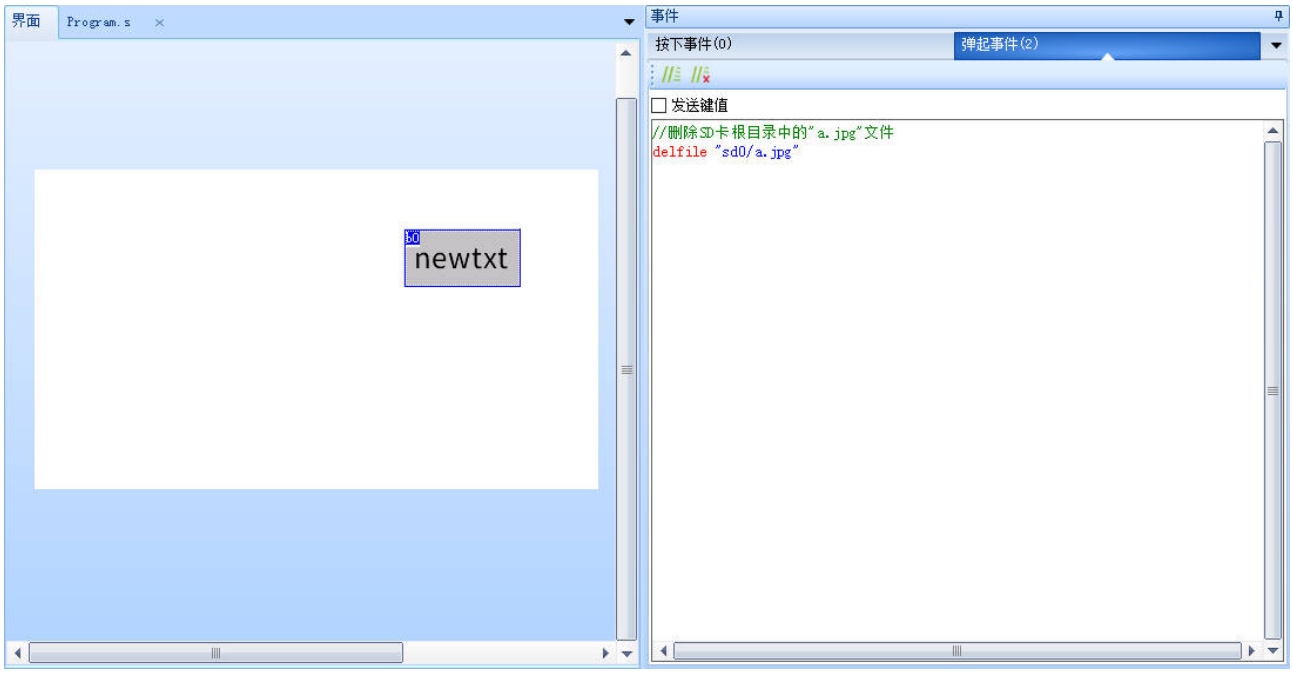
4.3.4.1 delfile-示例 1

```
//删除内存文件系统中的"0.jpg"文件
delfile "ram/0.jpg"
```



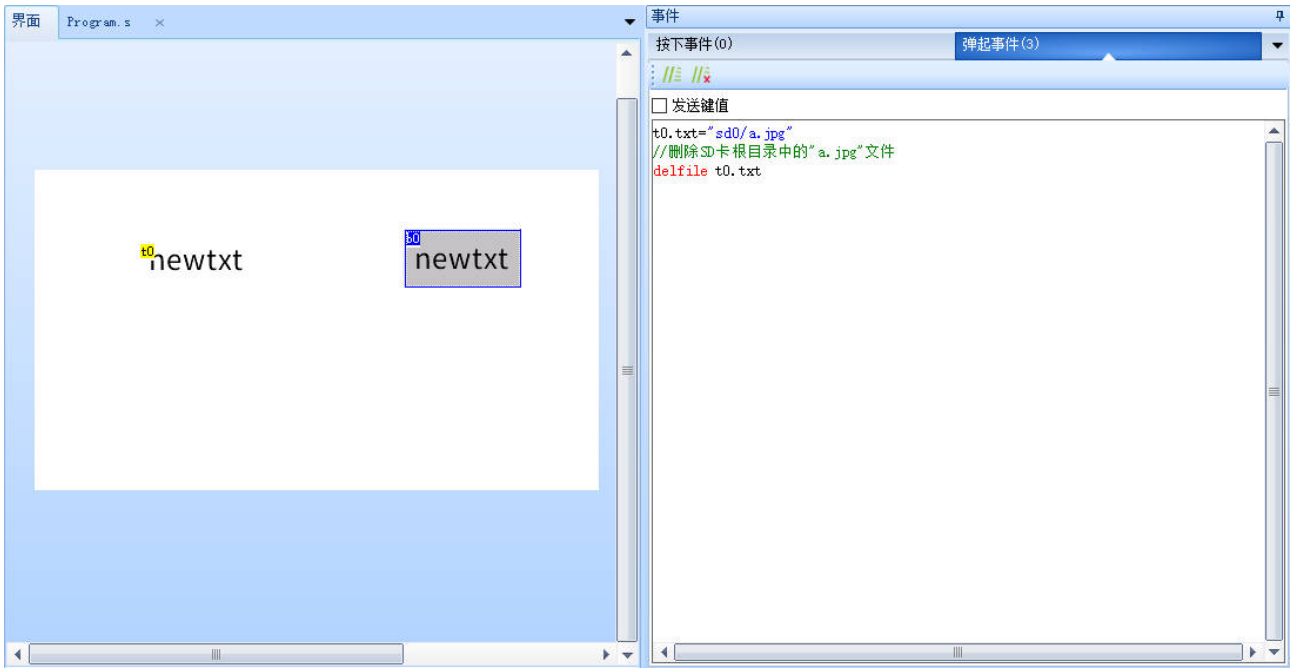
4.3.4.2 delfile-示例 2

```
//删除SD卡根目录中的"a.jpg"文件
delfile "sd0/a.jpg"
```



4.3.4.3 delfile-示例 3

```
t0.txt="sd0/a.jpg"

//删除SD卡根目录中的"a.jpg"文件
delfile t0.txt
```



注意：要使用内存文件系统必须先在工程配置选项中配置内存文件系统的大小，新建工程默认内存文件系统大小为 0，即不可能使用。

4.3.4.4 delfile 指令-样例工程下载

演示工程下载链接：

《文件浏览器 v3.0》演示工程下载

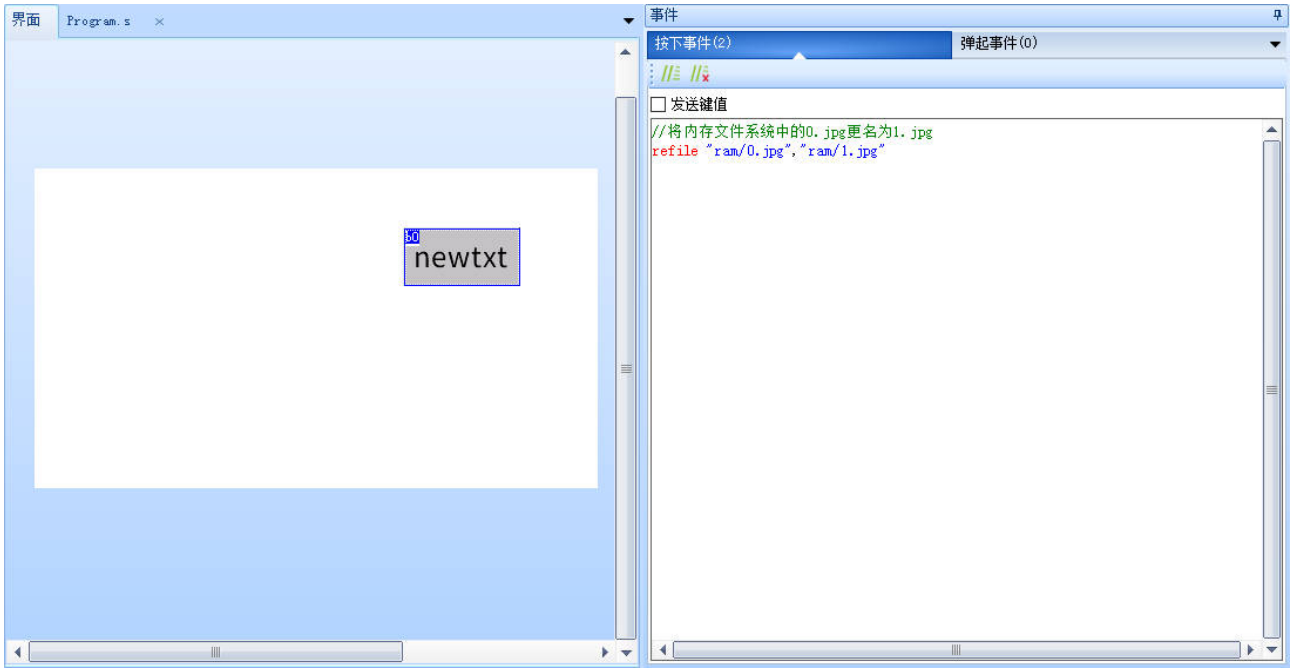
4.3.5 refile-重命名文件

仅 X2、X3、X5 系列支持

```
refile srcfilepath,decfilepath

srcfilepath:源文件路径

decfilepath:目标文件路径
```

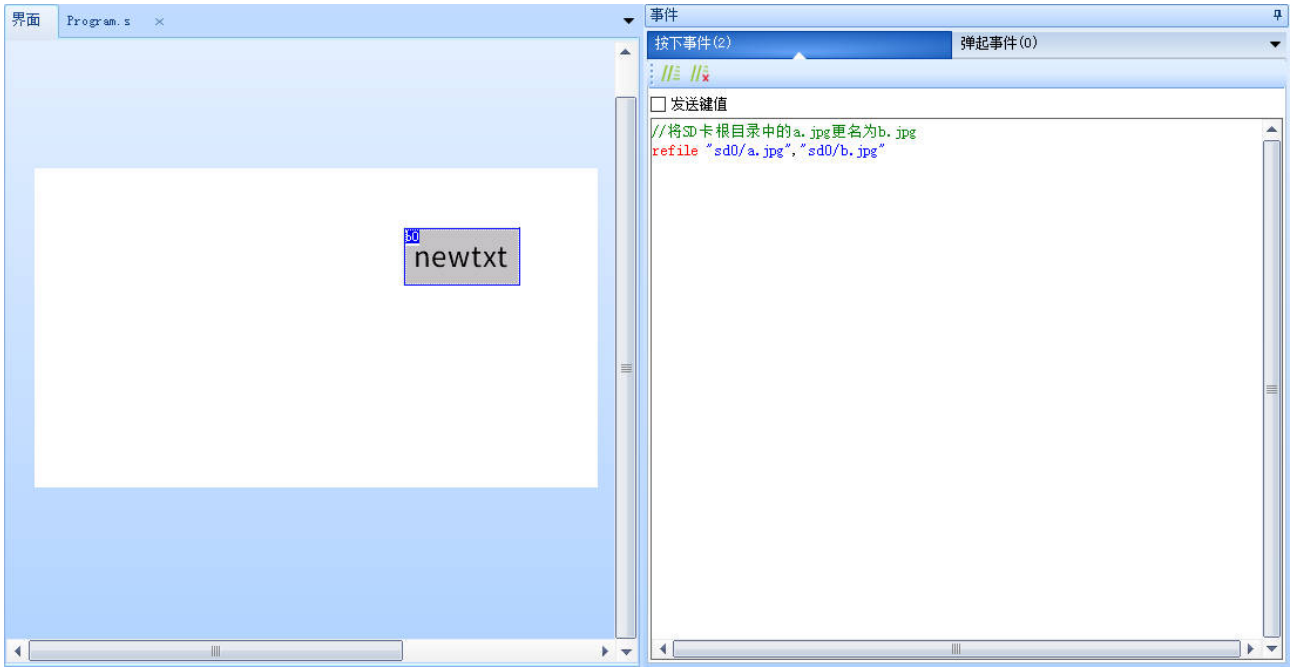
4.3.5.1 refile-示例 1

```
//将内存文件系统中的0.jpg更名为1.jpg
refile "ram/0.jpg","ram/1.jpg"
```



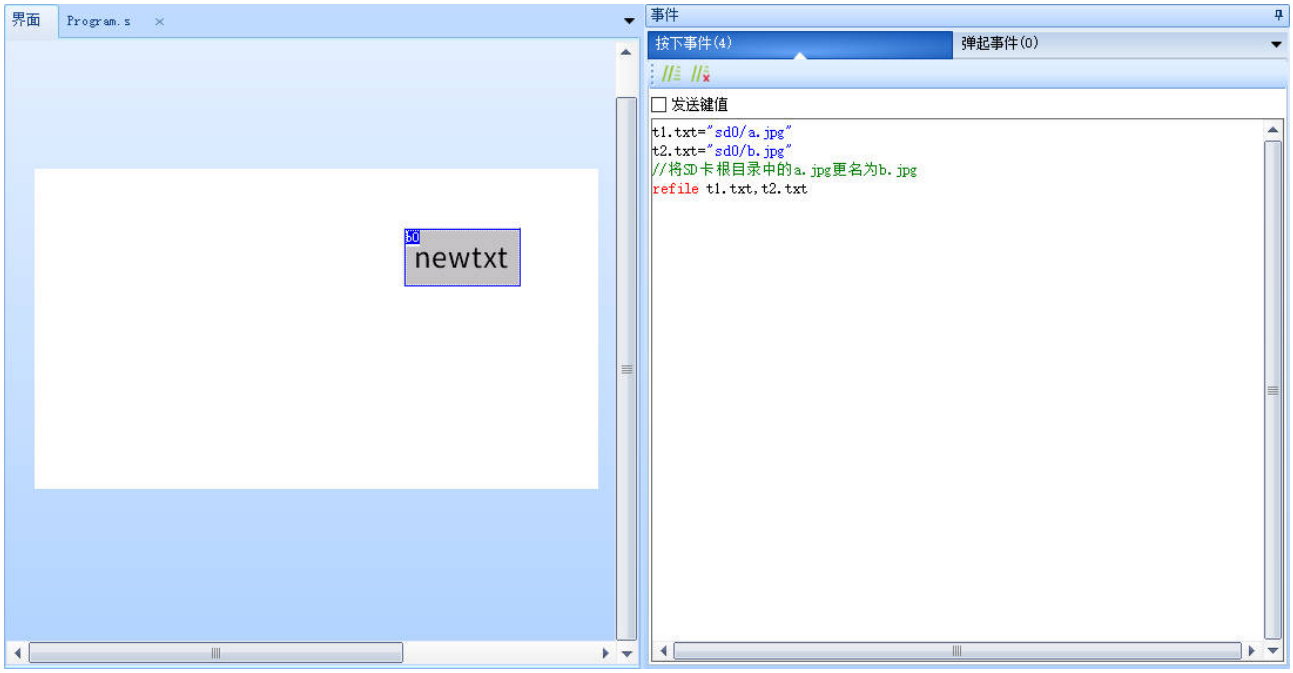
4.3.5.2 refile-示例 2

```
//将SD卡根目录中的a.jpg更名为b.jpg
refile "sd0/a.jpg","sd0/b.jpg"
```



4.3.5.3 refile-示例 3

```
t1.txt="sd0/a.jpg"
t2.txt="sd0/b.jpg"

//将SD卡根目录中的a.jpg更名为b.jpg
refile t1.txt,t2.txt
```



备注:

要使用内存文件系统必须先在工程配置选项中配置内存文件系统的大小，新建工程默认内存文件系统大小为 0，即不可能使用。

4.3.5.4 refile 指令-样例工程下载

演示工程下载链接：

《文件浏览器 v3.0》演示工程下载

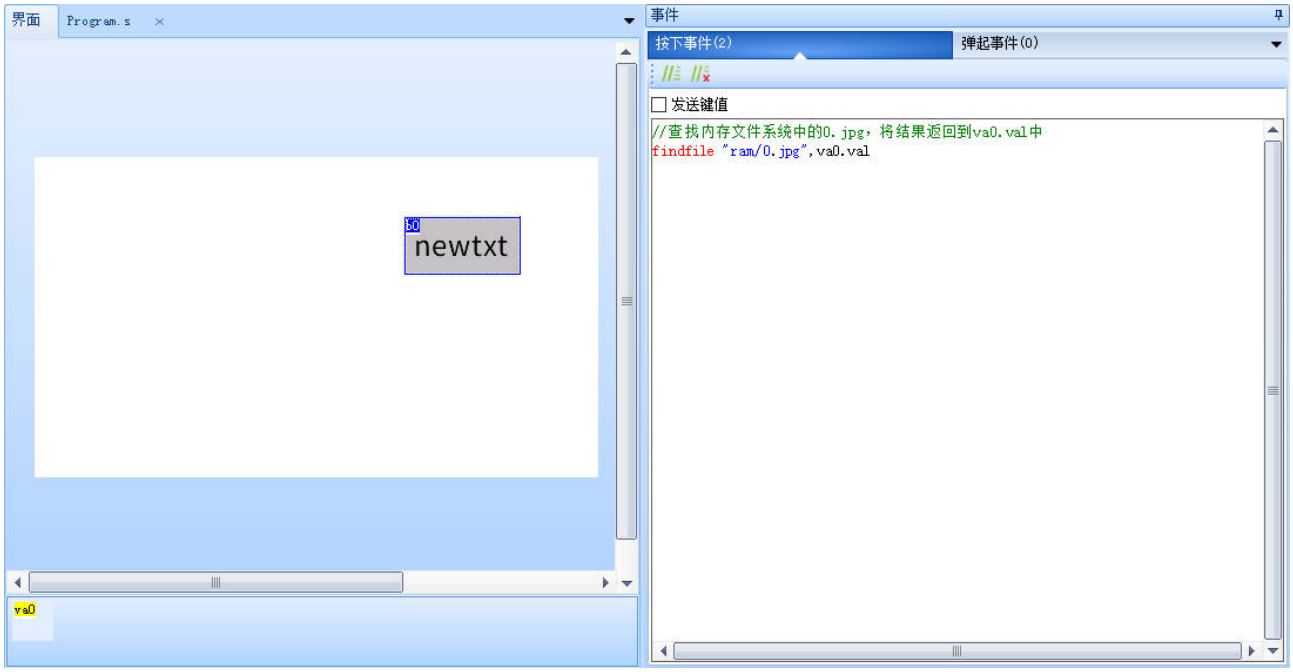
4.3.6 findfile-查找文件

仅 X2、X3、X5 系列支持

```
findfile filepath,att

filepath:文件路径

att:查找结果写入变量,必须是数值型变量(0-查找失败;1-查找成功)
```

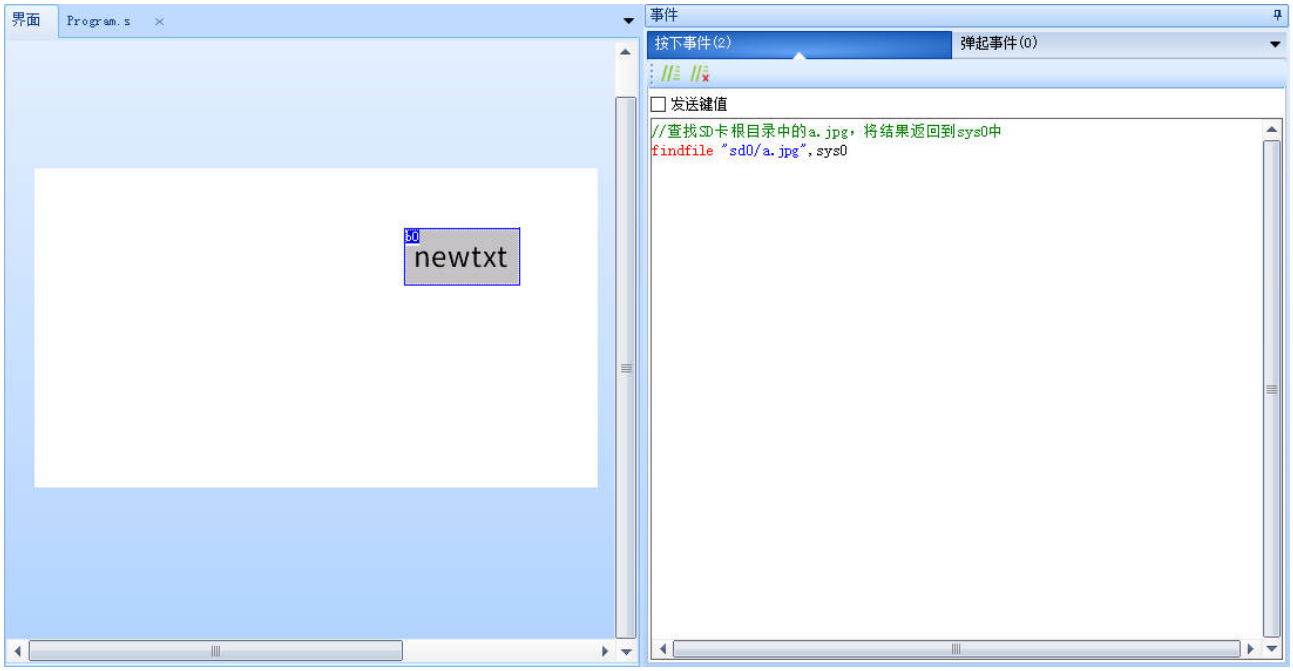
4.3.6.1 findfile-示例 1

```
//查找内存文件系统中的0.jpg, 将结果返回到va0.val中
findfile "ram/0.jpg",va0.val
```



4.3.6.2 findfile-示例 2

```
//查找SD卡根目录中的a.jpg, 将结果返回到sys0中
findfile "sd0/a.jpg",sys0
```

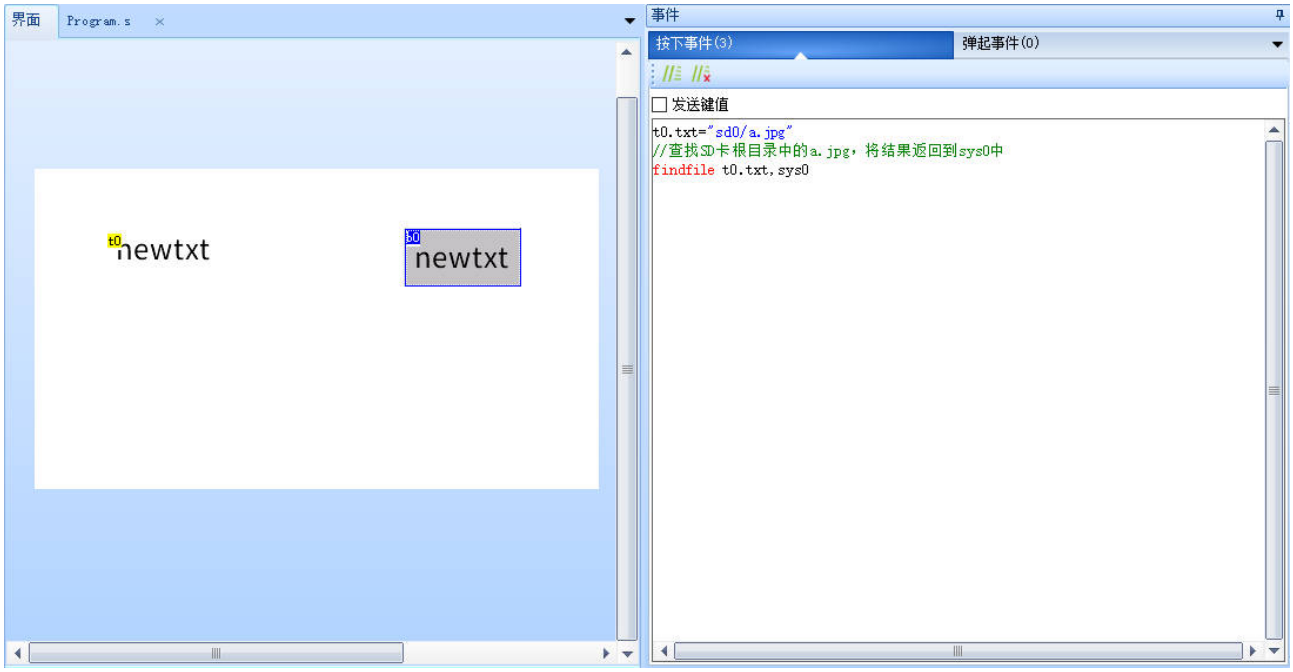


注意： 要使用内存文件系统必须先在工程配置选项中配置内存文件系统的大小，新建工程默认内存文件系统大小为 0，即不可能使用。

使用文件流打开文件前，务必先用 findfile 查找文件是否存在

4.3.6.3 findfile-示例 3

```
t0.txt="sd0/a.jpg"

//查找SD卡根目录中的a.jpg，将结果返回到sys0中
findfile t0.txt,sys0
```



注意：要使用内存文件系统必须先在工程配置选项中配置内存文件系统的大小，新建工程默认内存文件系统大小为 0，即不可能使用。

使用文件流打开文件前，务必先用 findfile 查找文件是否存在

4.3.6.4 findfile-应用实例

查找多个有规律的文件，例如 a1.txt~a9.txt，如果没找到，则创建文件

```
for(sys0=1;sys0<=9;sys0++)
{
    covx sys0,t0.txt,0,0  //将变量转换为文本
    t1.txt="sd0/"+"a"+t0.txt+".txt"
    findfile t1.txt,sys1
    if(sys1==0)
    {
        newfile t1.txt,4096
    }
}
```

4.3.6.5 findfile 指令-样例工程下载

演示工程下载链接：

《文件浏览器 v3.0》演示工程下载

4.3.7 newdir-创建文件夹

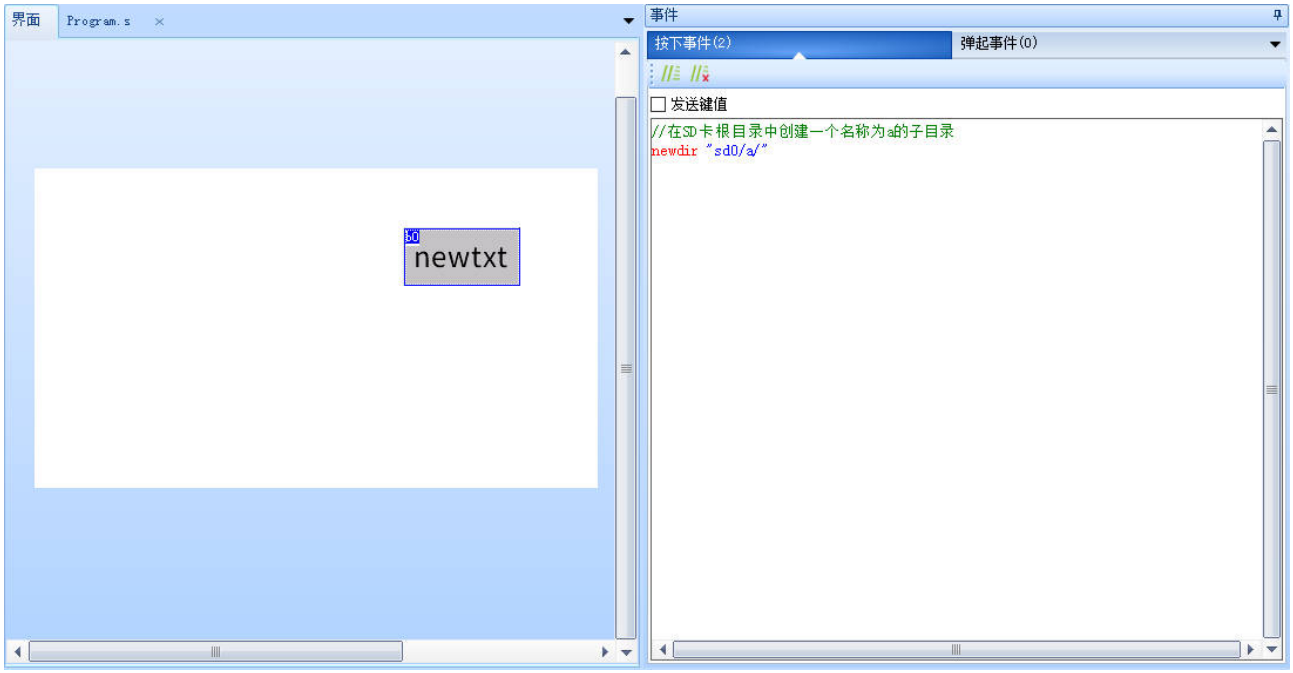
仅 X2、X3、X5 系列支持

> **注意:** 创建文件夹时文件夹名不允许包含以下字符 \ / : * ? " < > |

```
newdir dir

dir:文件夹目录(如:"sd0/newfolder/")
```

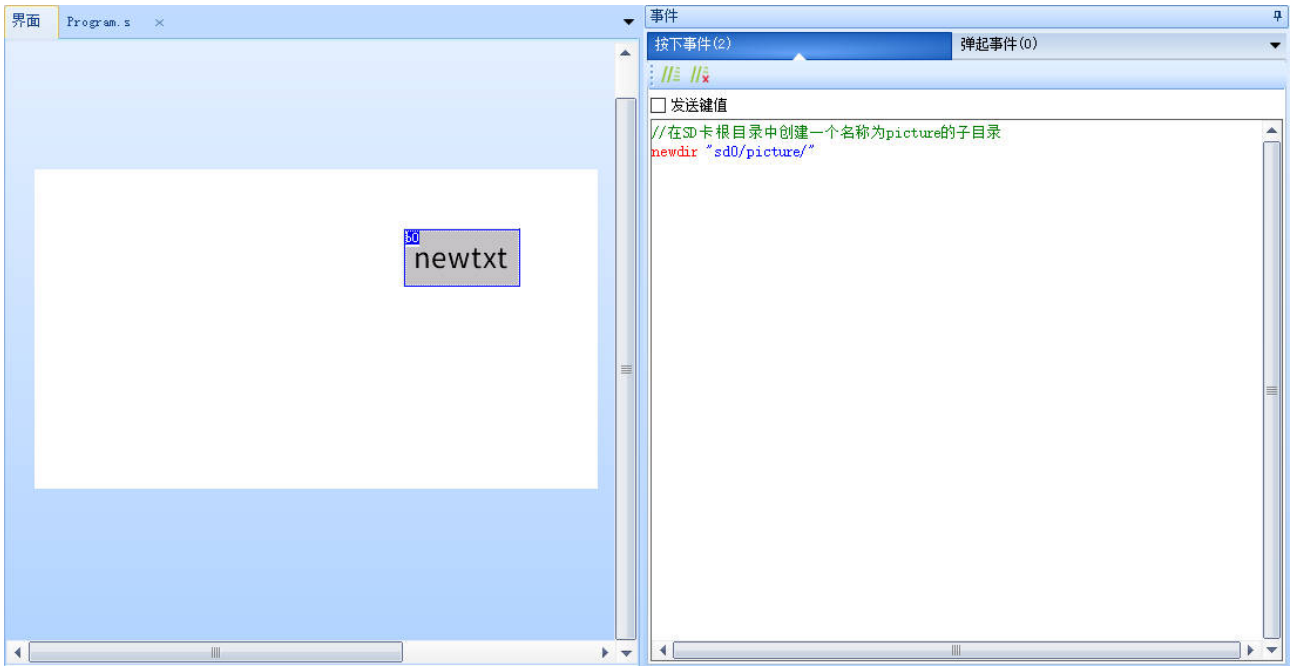
4.3.7.1 newdir-示例 1

```
//在SD卡根目录中创建一个名称为a的子目录
newdir "sd0/a/"
```



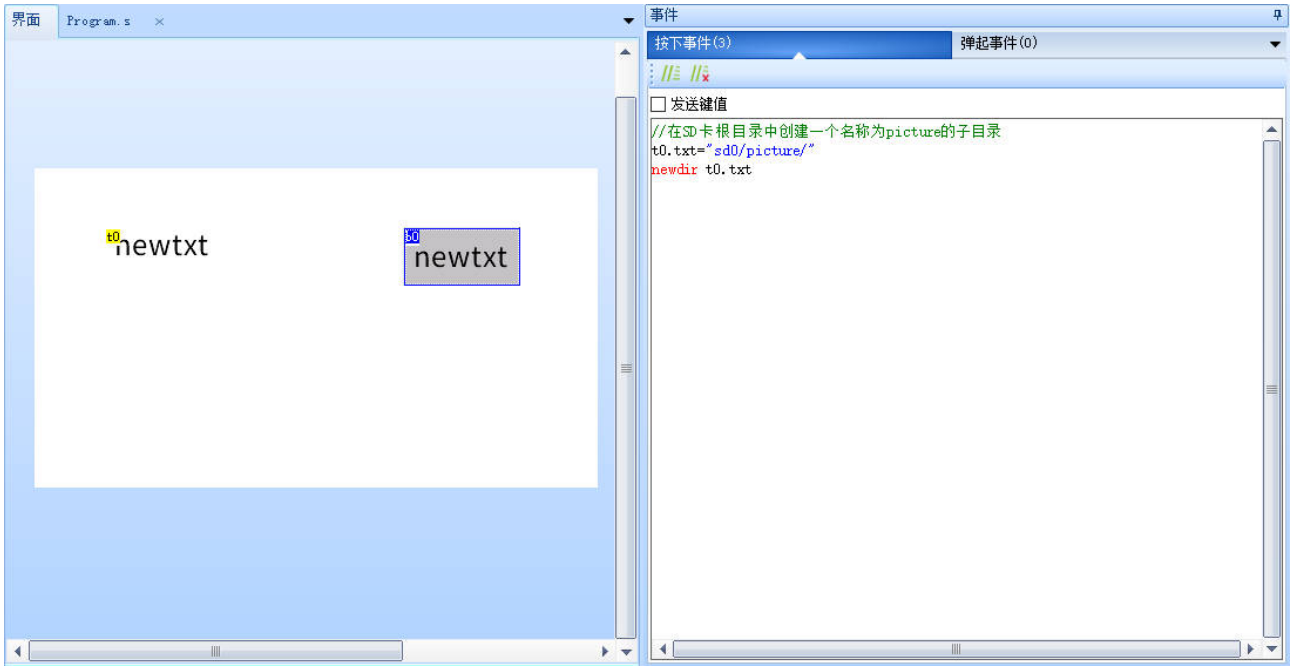
4.3.7.2 newdir-示例 2

```
//在SD卡根目录中创建一个名称为picture的子目录
newdir "sd0/picture/"
```



4.3.7.3 newdir-示例 3

```
//在SD卡根目录中创建一个名称为picture的子目录
t0.txt="sd0/picture/"
newdir t0.txt
```



请保证 t0 的 txt_maxl 足够大，否则会提示操作文件失败。

注意： 内存文件系统不支持子目录，SD 卡支持子目录，目录路径必须以”/”结尾。
在创建或文件夹前请先使用 finddir 指令检查该文件夹是否存在。
在某个文件夹下创建子文件夹，需保证父文件夹已存在。

4.3.7.4 newdir 指令-样例工程下载

演示工程下载链接：
《文件浏览器 v3.0》演示工程下载

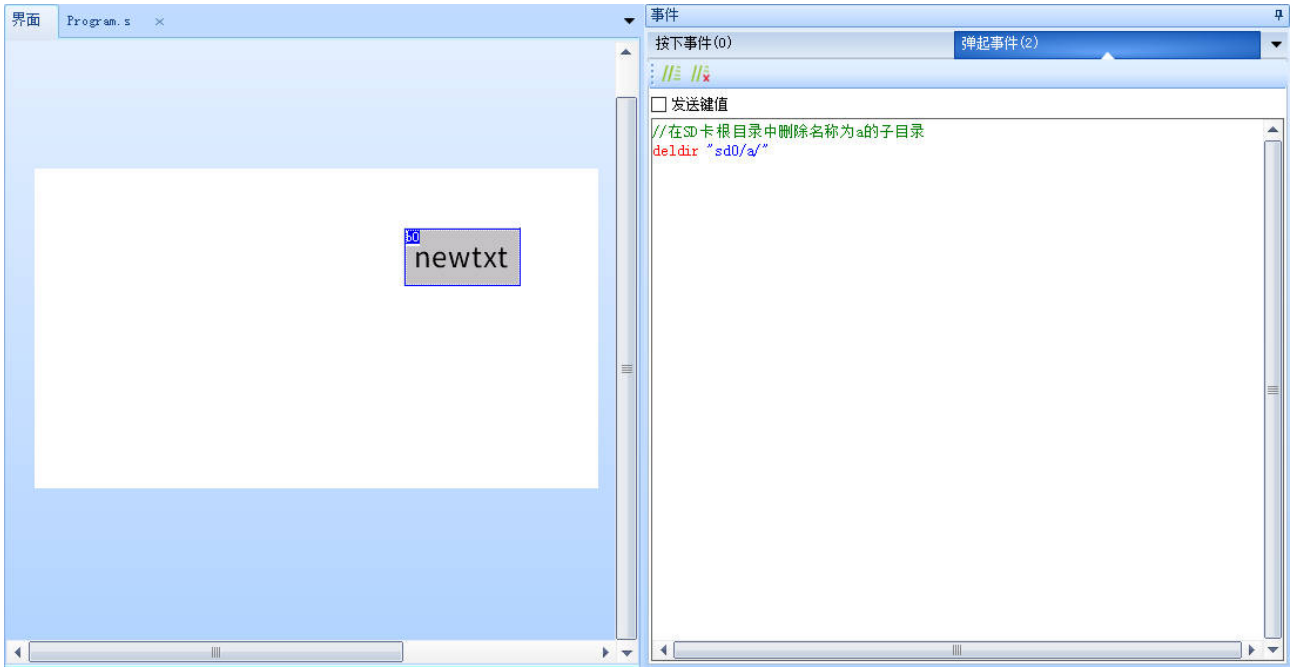
4.3.8 deldir-删除文件夹

仅 X2、X3、X5 系列支持

```
deldir dir

dir:文件夹目录(如:"sd0/newfolder/")
```

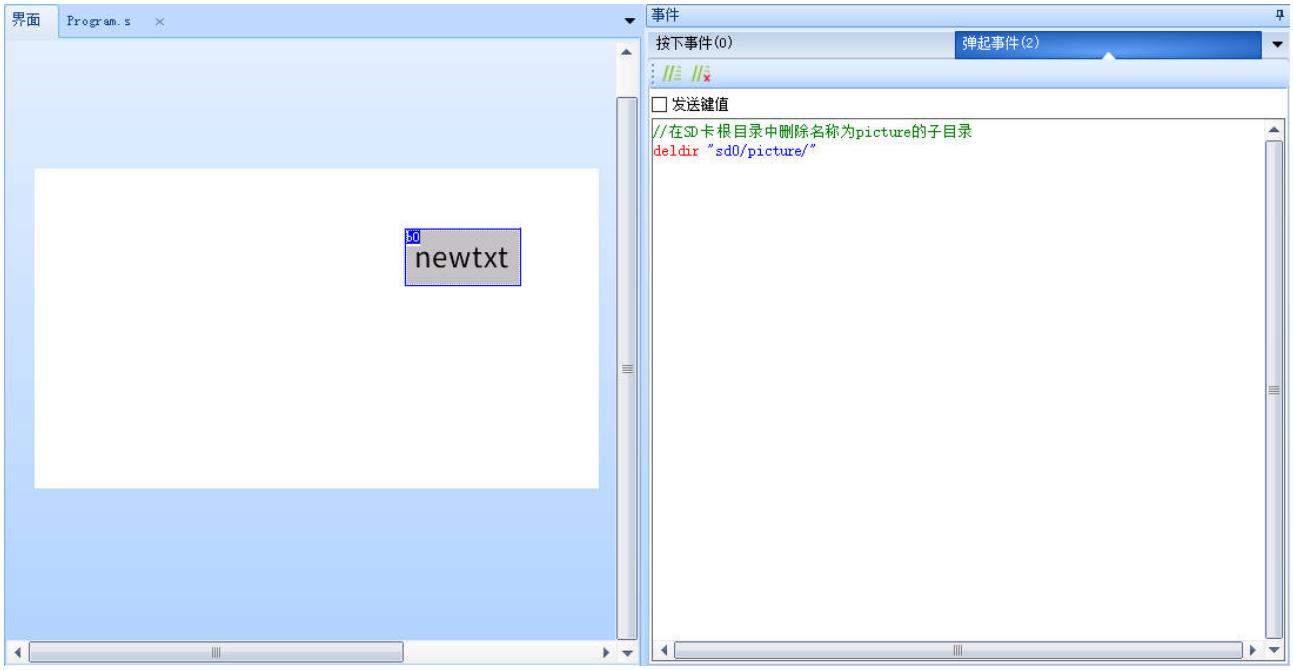
4.3.8.1 deldir-示例 1

```
//在SD卡根目录中删除名称为a的子目录
deldir "sd0/a/"
```



4.3.8.2 deldir-示例 2

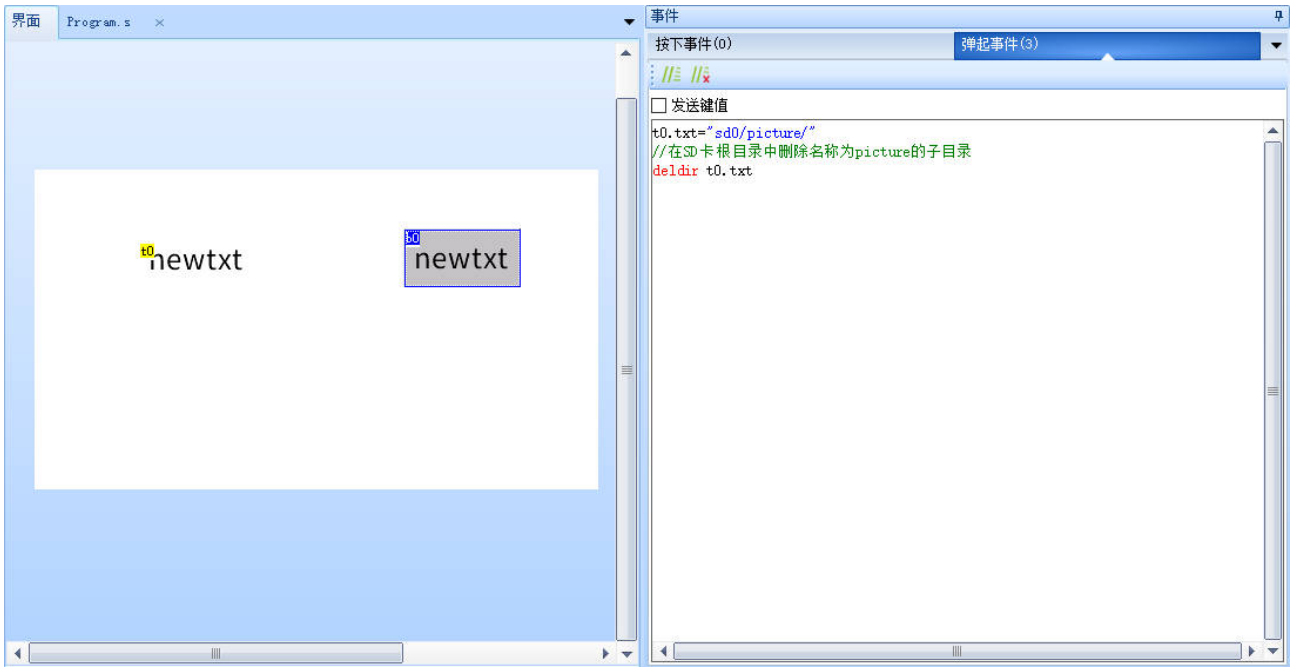
```
//在SD卡根目录中删除名称为picture的子目录
deldir "sd0/picture/"
```



4.3.8.3 deldir-示例 3

```
t0.txt="sd0/picture/"

//在SD卡根目录中删除名称为picture的子目录
deldir t0.txt
```

注意： 请保证 t0 的 txt_maxl 属性足够大



注意： 内存文件系统不支持子目录，SD 卡支持子目录，目录路径必须以”/”结尾。
在创建或文件夹前请先使用 finddir 指令检查该文件夹是否存在

4.3.8.4 deldir 指令-样例工程下载

演示工程下载链接：
《文件浏览器 v3.0》演示工程下载

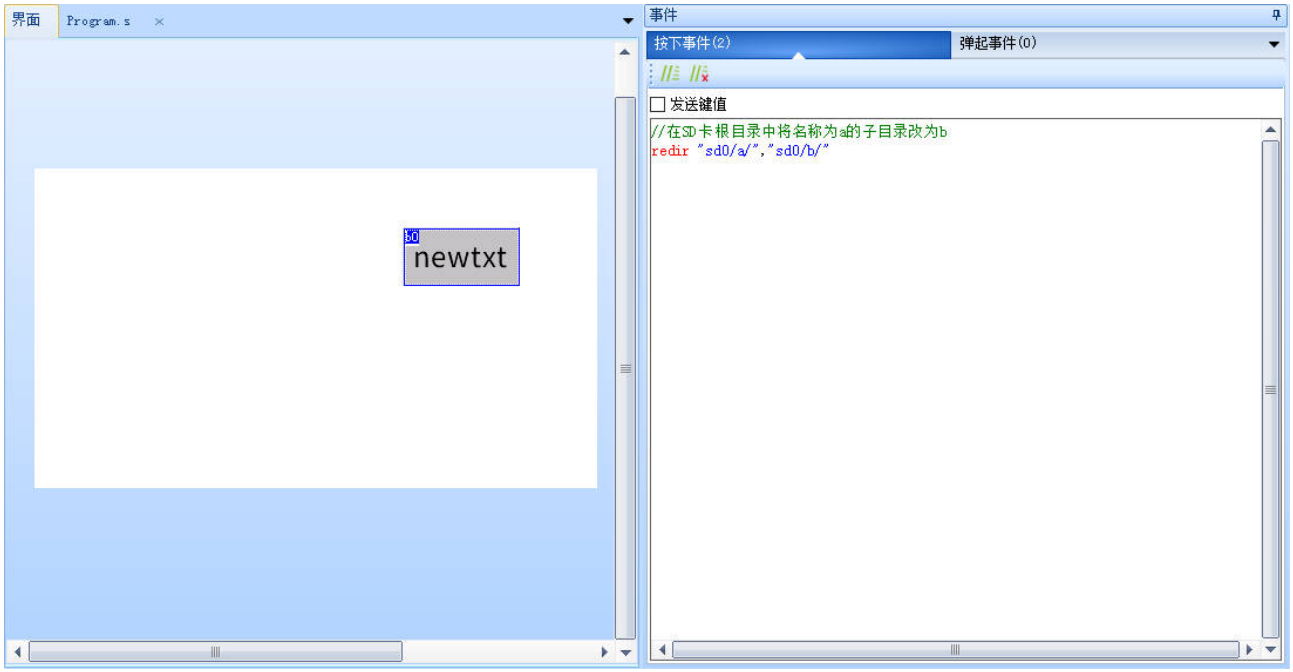
4.3.9 redir-重命名文件夹

仅 X2、X3、X5 系列支持

```
redir srcdir,decdir

srcdir:源文件夹目录(如:"sd0/newfoldersrc/")

decdir:目标文件夹目录(如:"sd0/newfolderdec/")
```

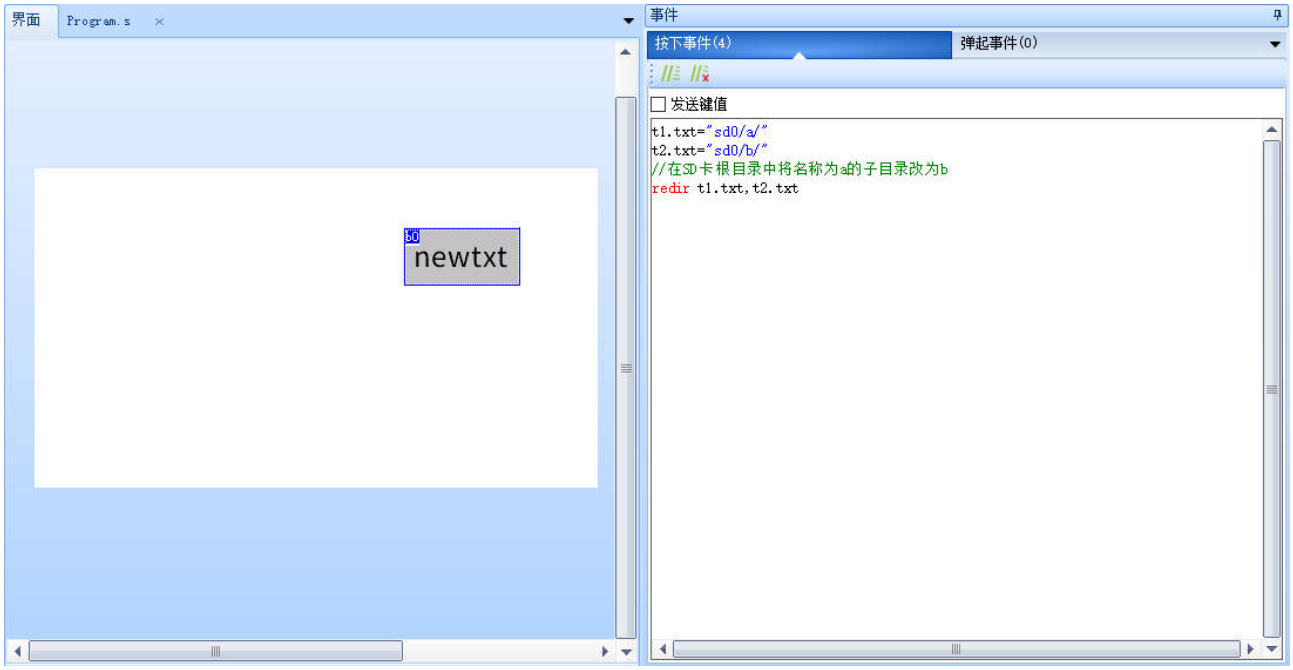
4.3.9.1 redir-示例 1

```
//在SD卡根目录中将名称为a的子目录改为b
redir "sd0/a/","sd0/b/"
```



4.3.9.2 redir-示例 2

```
t1.txt="sd0/a/"
t2.txt="sd0/b/"

//在SD卡根目录中将名称为a的子目录改为b
redir t1.txt,t2.txt
```



注意： 内存文件系统不支持子目录，SD 卡支持子目录，目录路径必须以”/”结尾。

4.3.9.3 redir 指令-样例工程下载

演示工程下载链接：

《文件浏览器 v3.0》演示工程下载

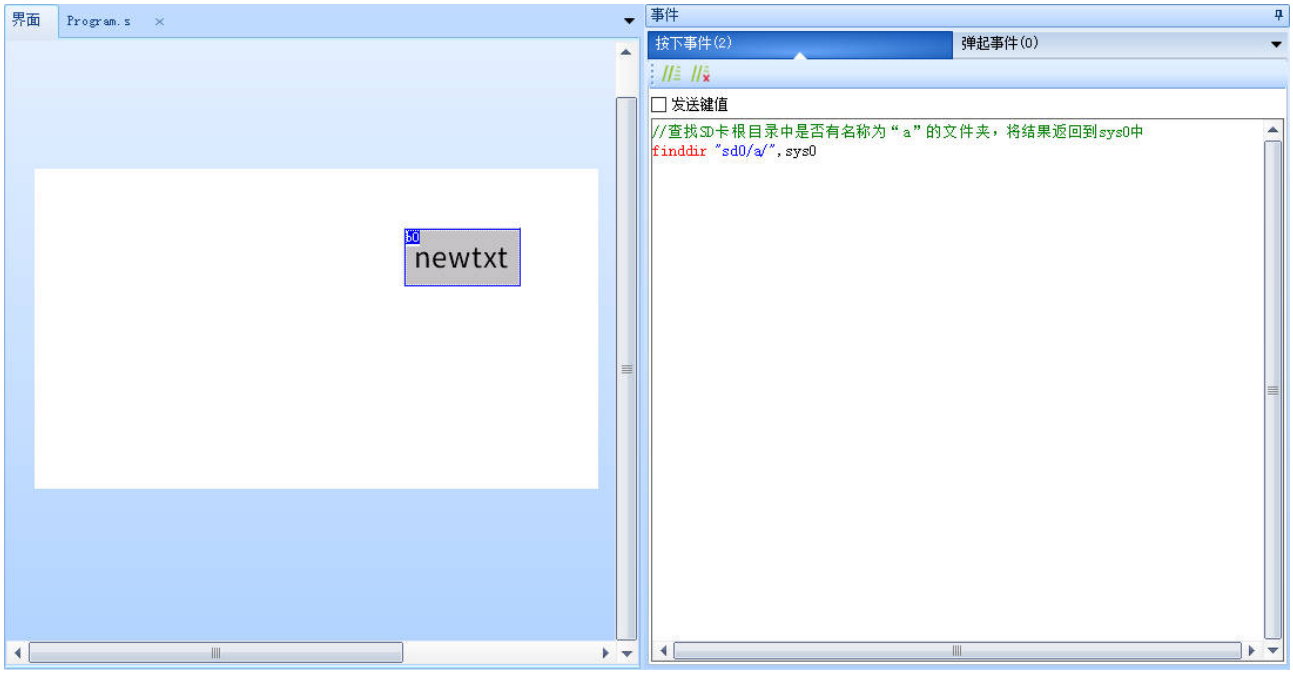
4.3.10 finddir-查找文件夹

仅 X2、X3、X5 系列支持

```
finddir dir,att

dir:文件夹目录(如:"sd0/newfolder/")

att:查找结果写入变量(0-查找失败;1-查找成功。此变量必须是数值类型变量)
```

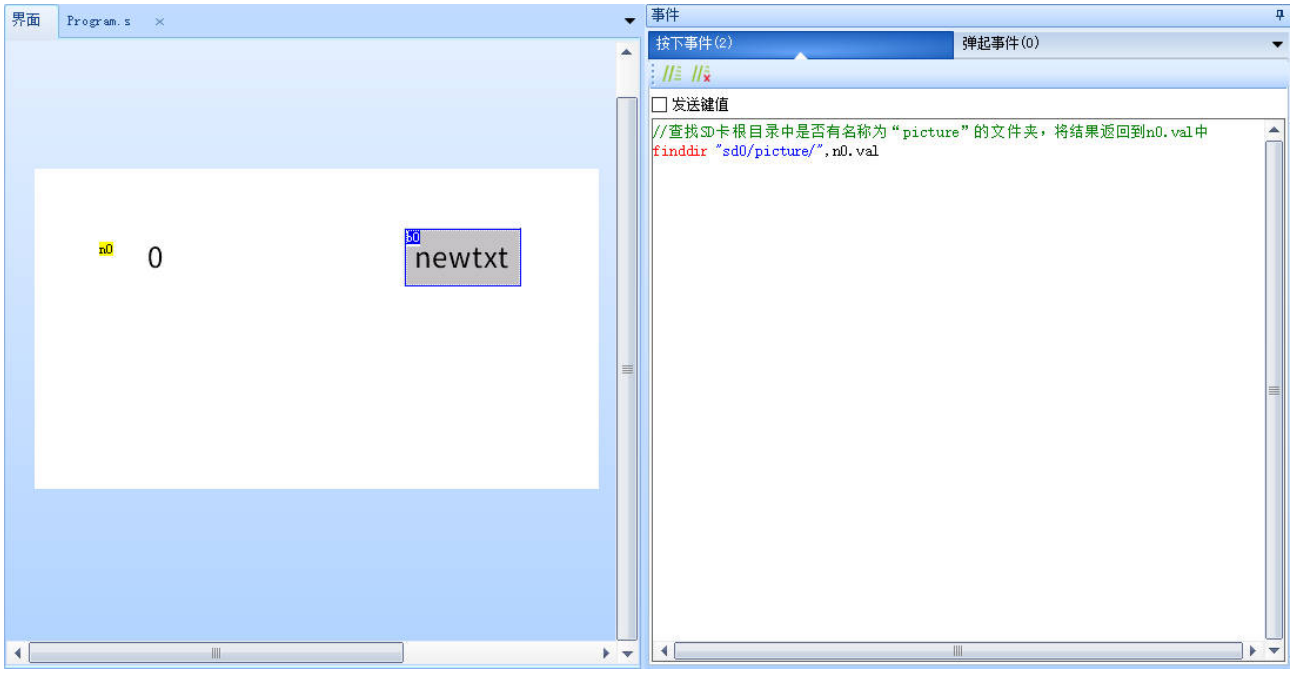
4.3.10.1 finddir-示例 1

```
//查找SD卡根目录中是否有名称为“a”的文件夹，将结果返回到sys0中
finddir "sd0/a/",sys0
```



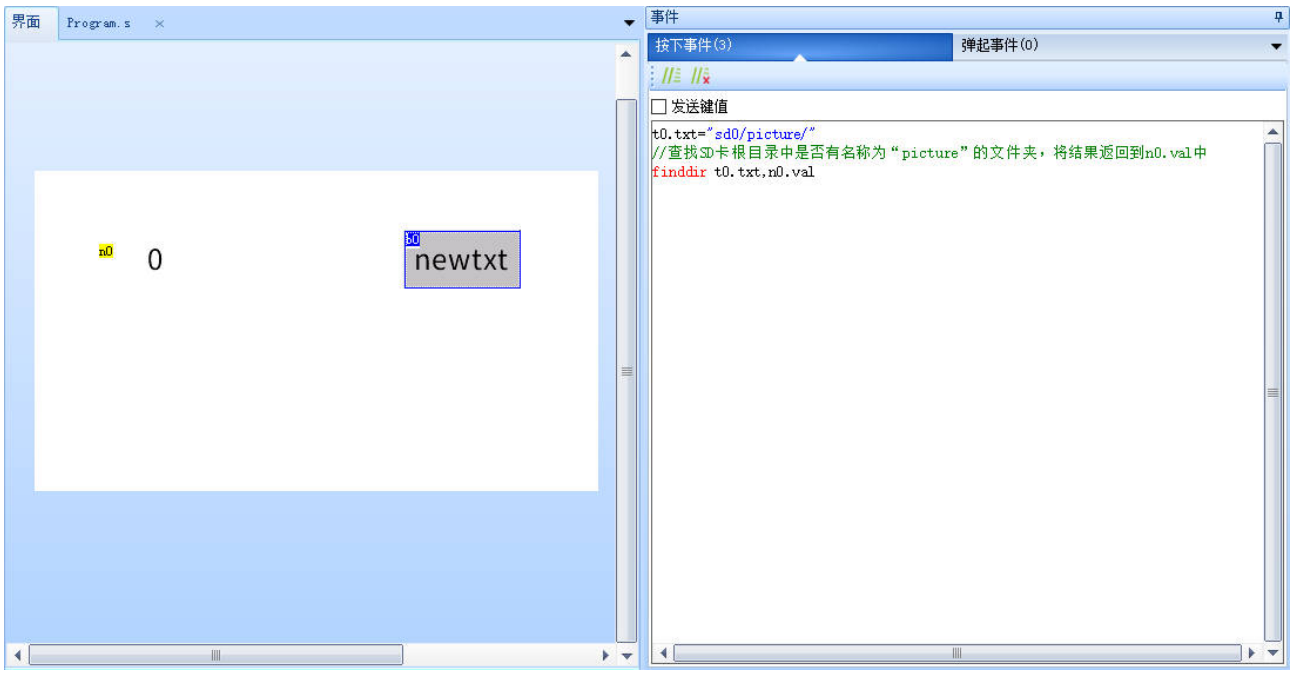
4.3.10.2 finddir-示例 2

```
//查找SD卡根目录中是否有名称为“picture”的文件夹，将结果返回到n0.val中
finddir "sd0/picture/",n0.val
```



4.3.10.3 finddir-示例 3

```
t0.txt="sd0/picture/"

//查找SD卡根目录中是否有名称为“picture”的文件夹，将结果返回到n0.val中
finddir t0.txt,n0.val
```



注意： 内存文件系统不支持子目录，SD 卡支持子目录，目录路径必须以”/”结尾。
在创建或删除文件夹前请先检查该文件夹是否存在

4.3.10.4 finddir 指令-样例工程下载

演示工程下载链接：

《文件浏览器 v3.0》演示工程下载

4.4 不常用指令

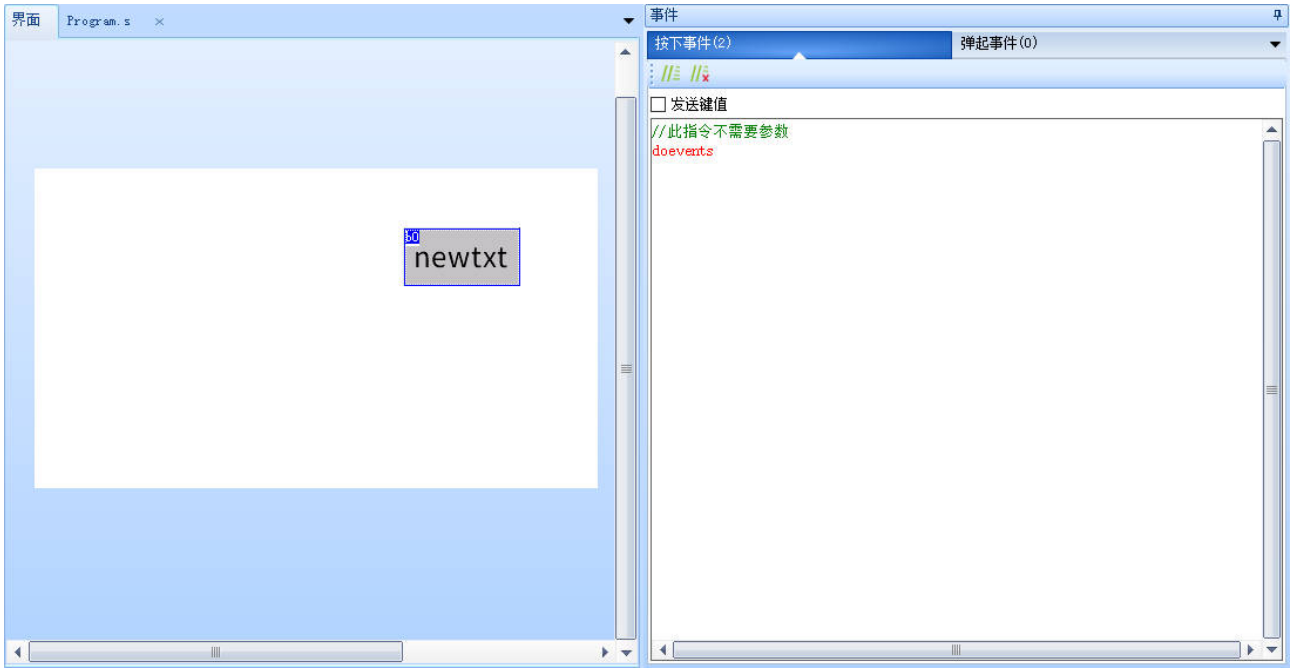
所有型号的串口屏均支持的指令，但是较少使用

4.4.1 doevents-强制刷新屏幕

```
doevents
```

4.4.1.1 doevents-示例

```
//此指令不需要参数
doevents
```



注意： 该指令极少使用，建议用户忽略此指令

在一个较多指令的过程执行中，或者在一个较长时间的循环语句中，系统所有控制权被此过程全部占用，在过程结束之前，尽管相应的内存数据可以任意正常读写，但是屏幕不会刷新显示，加入 doevents 后可以转让控制权给屏幕刷新，执行 doevents 之后，屏幕会刷新所有被改变过的控件，刷新完之后，控制权交回当前过程继续执行。防止屏幕呈现假死的显示状态。

4.4.1.2 doevents 指令-相关链接

HMI 逻辑语句 (page 854)

4.4.1.3 doevents 指令-样例工程下载

演示工程下载链接：

《doevents 指令》演示工程下载

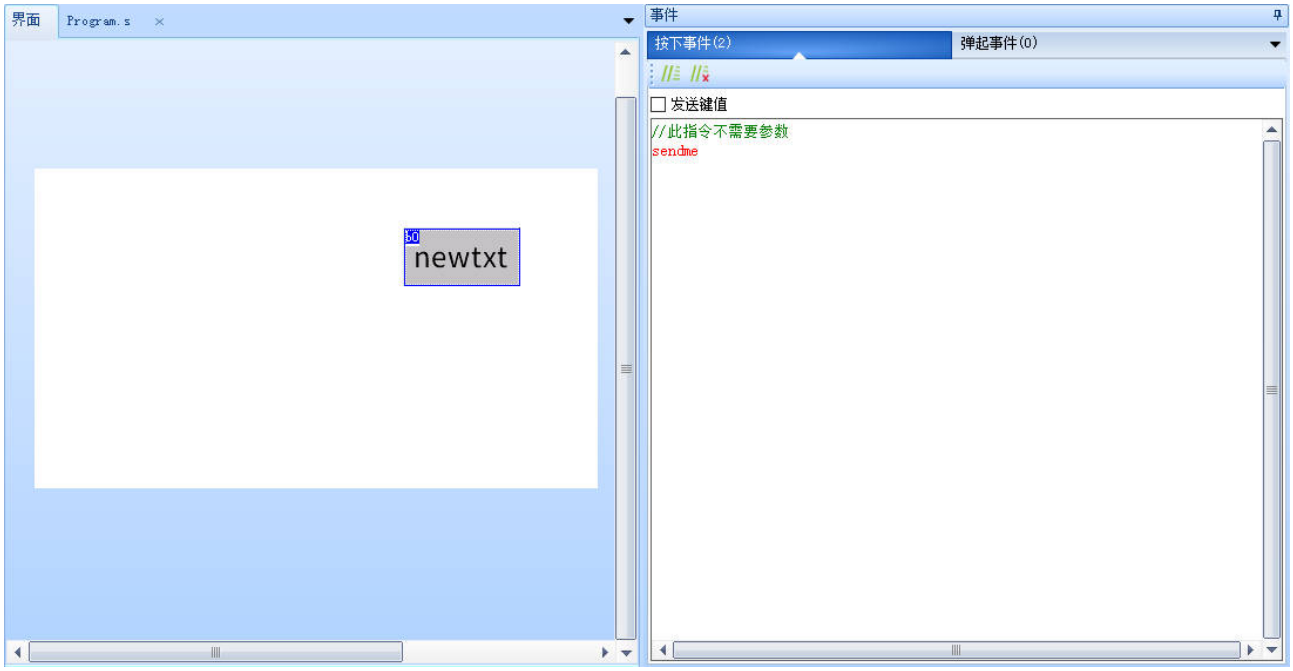
4.4.2 sendme-发送当前页面 ID 号到串口

注意： 设备收到此指令会立刻把当前页面的 ID 号发送到串口，如果想要每次刷新页面自动发送页面 ID, 请在页面的初始化事件里写上 sendme 语句即可。发送格式请参考*0X66 当前页面的 ID 号返回* (page 898)

不建议使用 sendme，因为 sendme 指令发出的数据格式是固定的，可能和你原有的格式不符

4.4.2.1 sendme-示例

```
//此指令不需要参数
sendme
```



sendme 指令可以使用 prints 和 printh 组合进行代替，例如以下的代码可以代替 sendme 的功能，并且能够完全自定义帧头帧尾

```
printh 66

//发生当前页面号，长度1字节
prints dp,1

printh ff ff ff
```

4.4.2.2 sendme 指令-样例工程下载

演示工程下载链接：

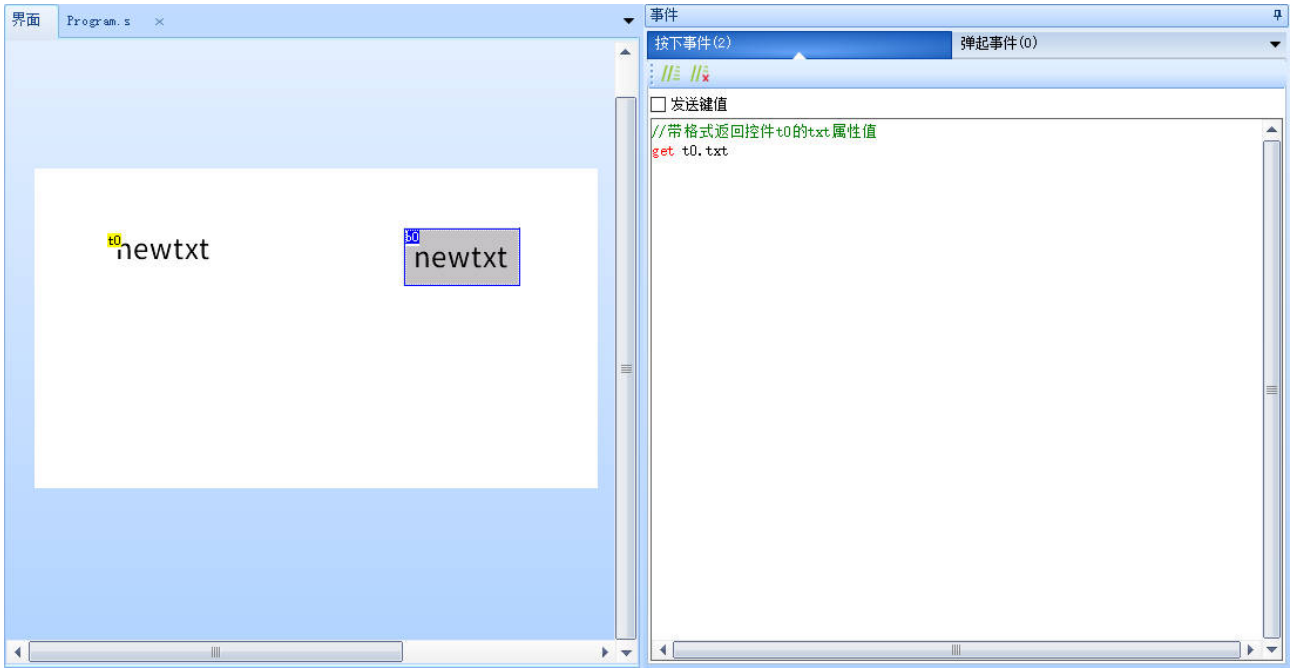
《sendme 指令》演示工程下载

4.4.3 get-带格式获取变量值/常量值

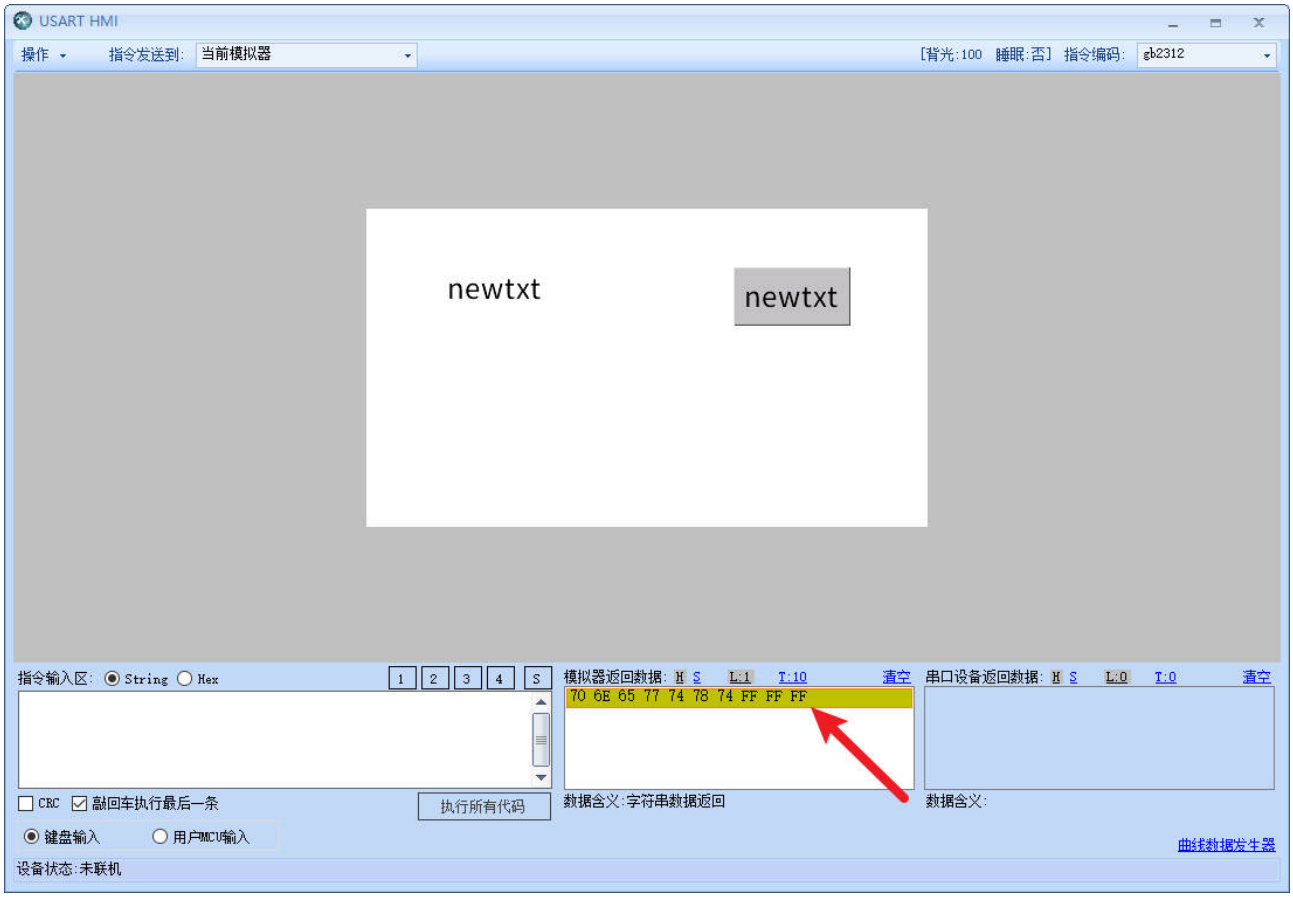
```
get att

att:变量名称
```

4.4.3.1 get-示例 1

```
//带格式返回控件t0的txt属性值
get t0.txt
```

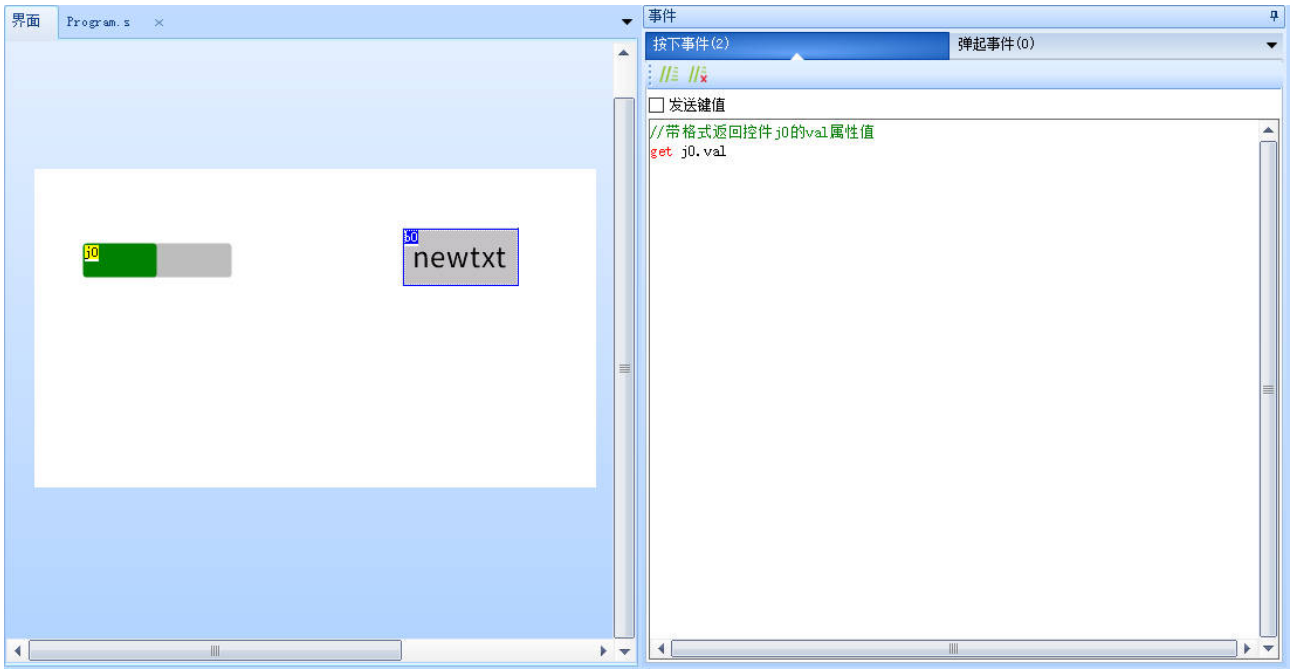


返回的数据如下

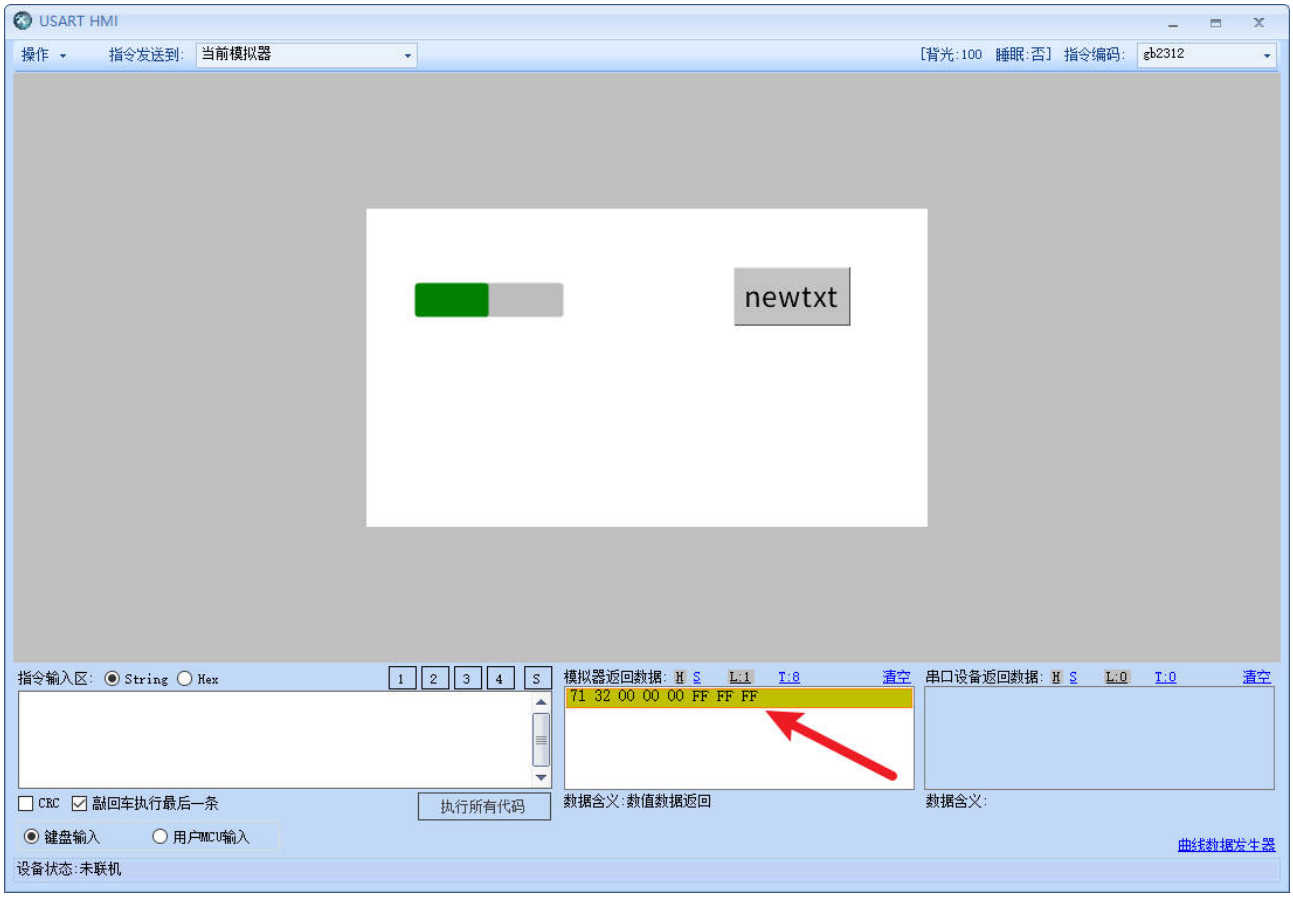


4.4.3.2 get-示例 2

```
//带格式返回控件j0的val属性值
get j0.val
```

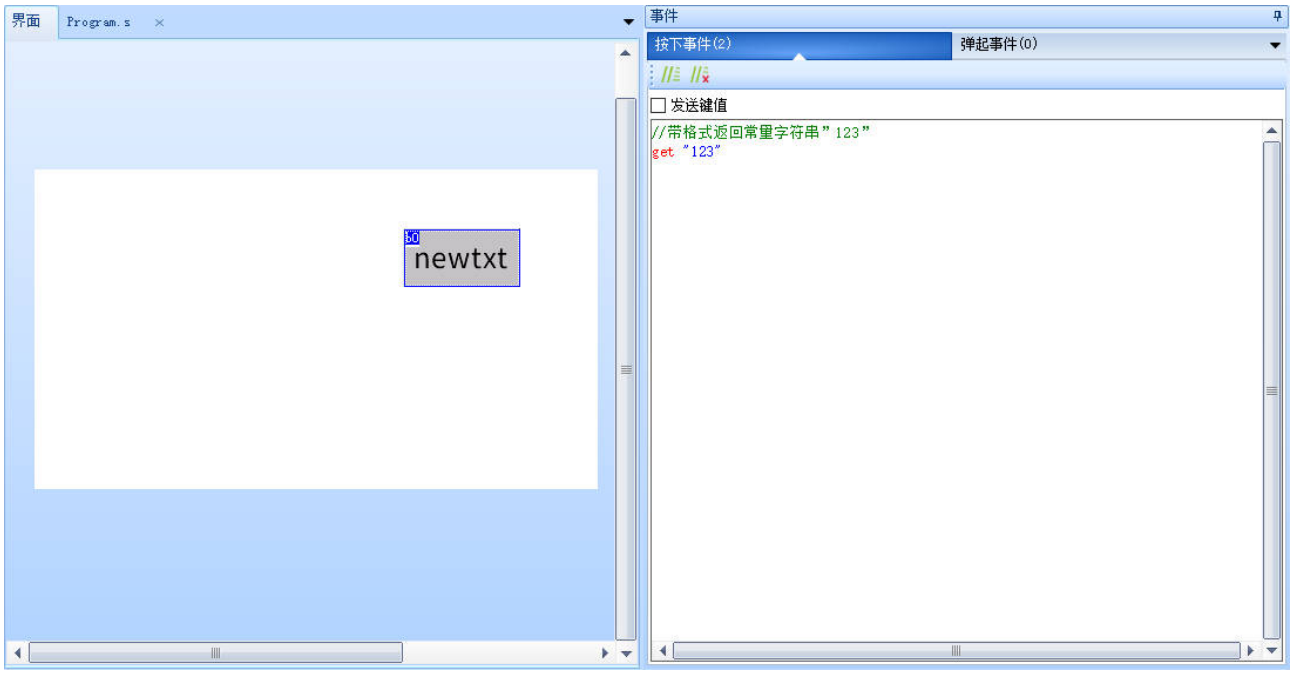


返回的数据如下（32 是 16 进制的 50）

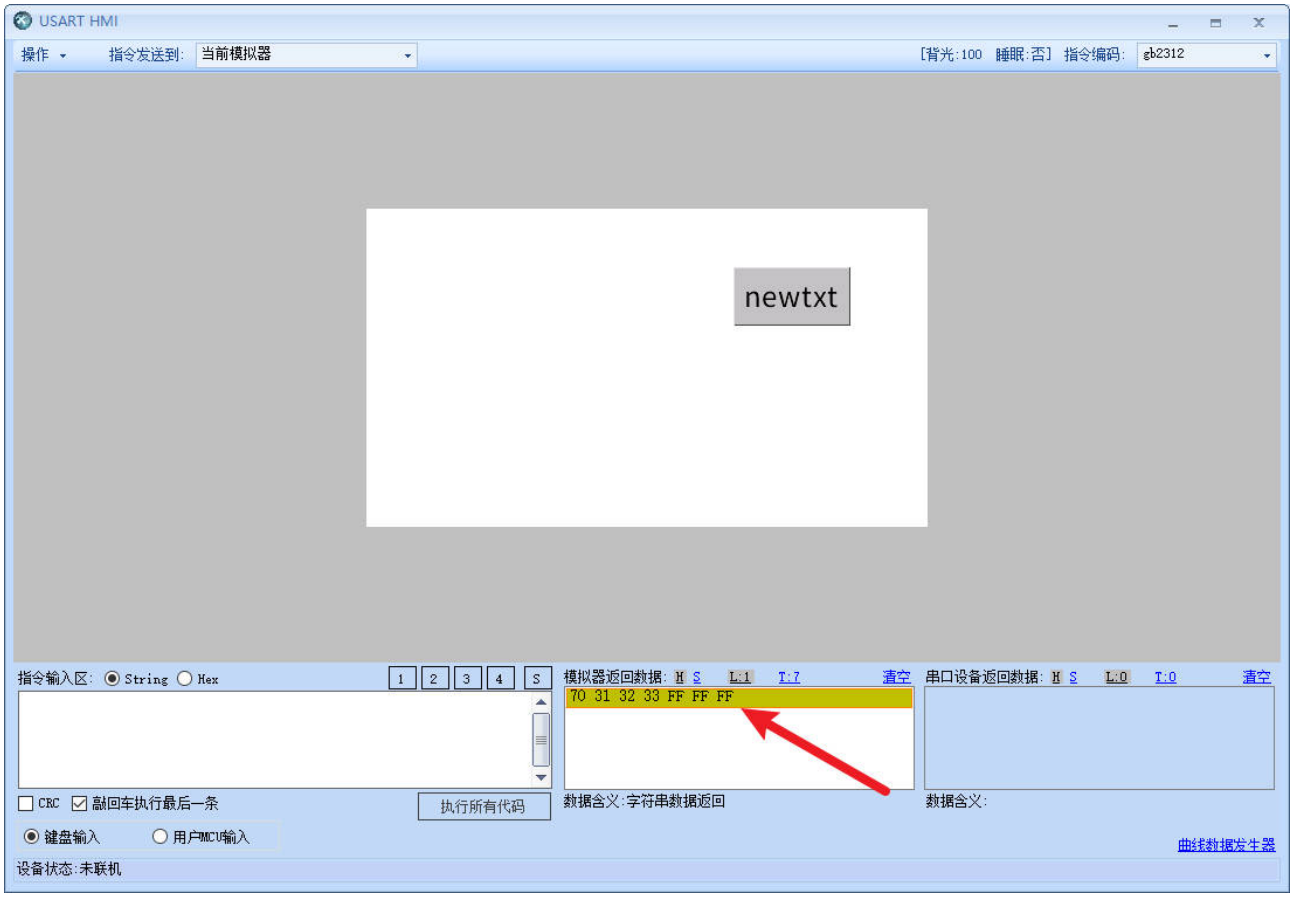


4.4.3.3 get-示例 3

```
//带格式返回常量字符串 “123”
get "123"
```

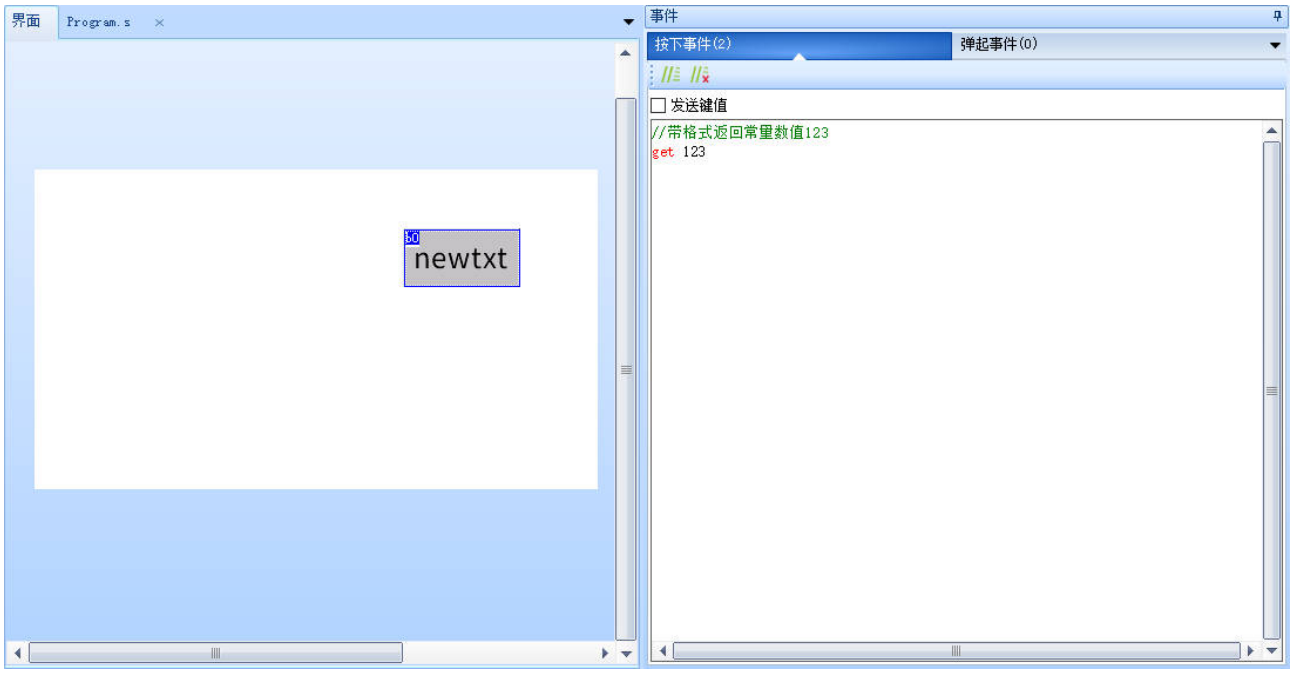


返回的数据如下

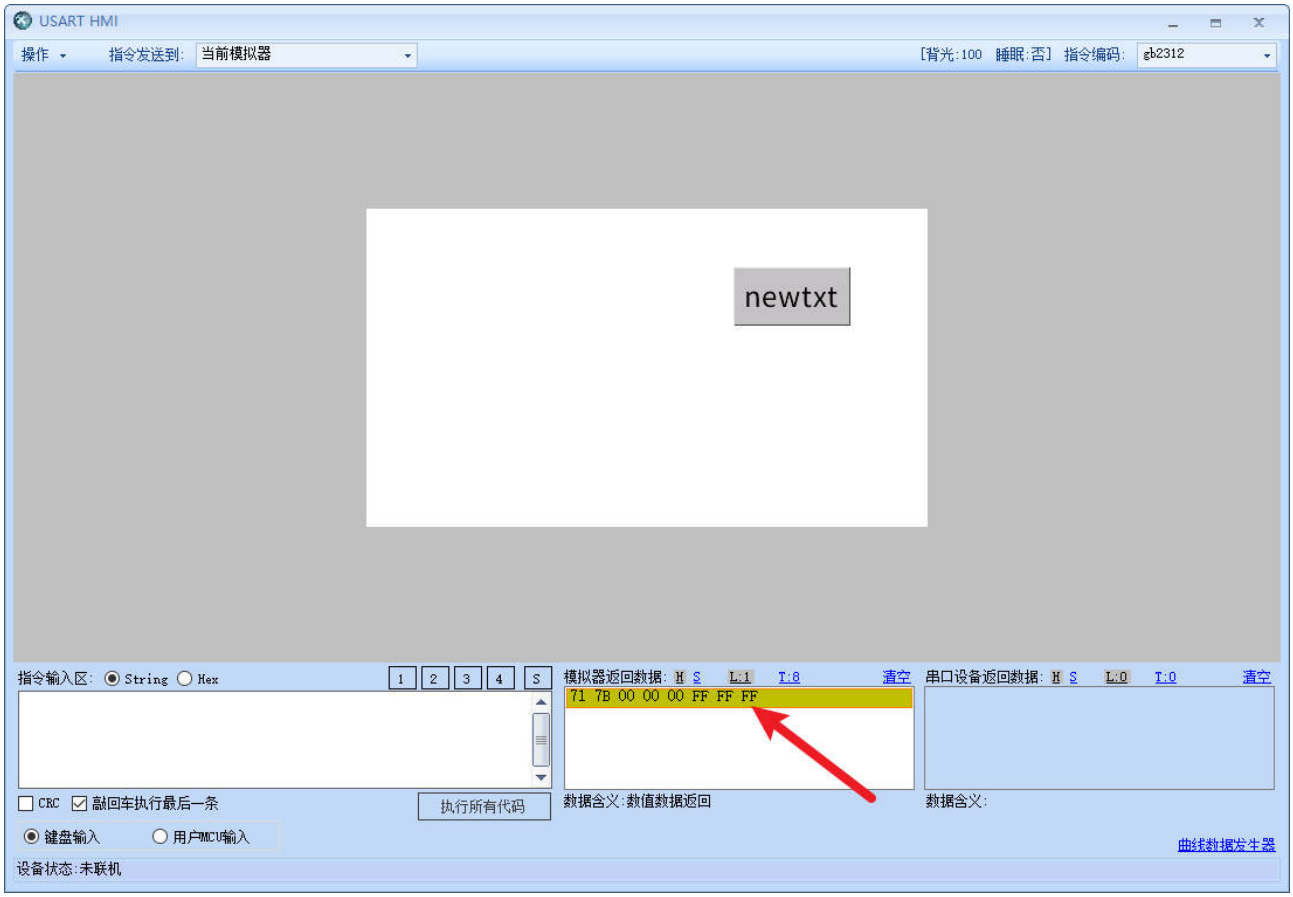


4.4.3.4 get-示例 4

```
//带格式返回常量数值123
get 123
```



返回的数据如下



4.4.3.5 get-c 语言示例

单片机通过串口获得 n0.val 和 t0.txt 的值

```
printf("get n0.val\xff\xff\xff");
printf("get t0.txt\xff\xff\xff");
```

注意： 使用 get 指令获取的变量为字符串类型时，返回的数据为 0X70+ 字符串内码 + 结束符，如果是数值类型 (如进度条的 val 属性) 设备返回 0X71+ 变量的 4 字节十六进制数据 (int 类型)+ 结束符。数值的存放模式为小端模式（即低位在前，高位在后）。

get 指令可以由串口发送，也可以在上位软件编辑界写进用户代码中实现屏幕主动发送变量 (主动发送的时候可以配合 printh 指令在前面加一段自定义标示来告诉单片机此变量是属于哪个控件的)。

get 指令和 prints 指令很类似, 唯一的区别是 get 返回的数据带了起始标示符（0x70 或 0x71）和结束符 (0xff 0xff 0xff)，而 prints 没有。

数据具体返回格式请查看本表格后面的”串口 HMI 设备返回数据格式”。

4.4.3.6 get 指令-样例工程下载

演示工程下载链接：

《get 指令》演示工程下载

4.4.4 ref-重绘控件

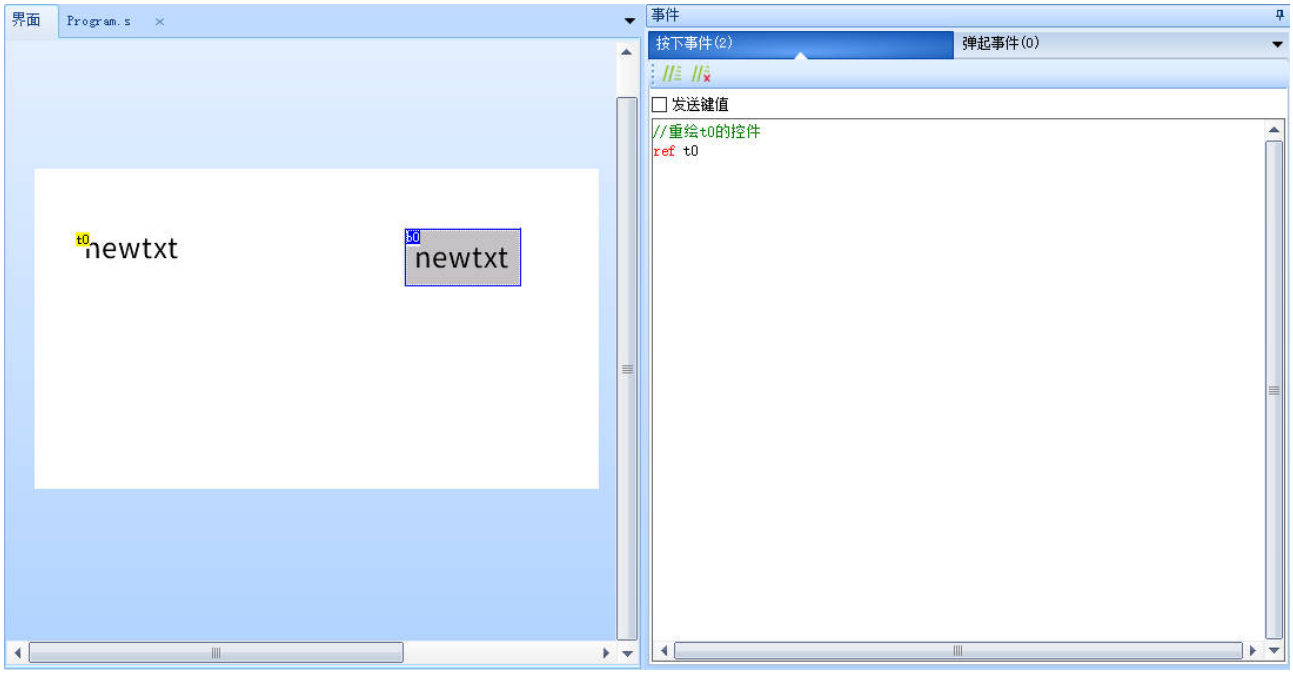
```
ref obj

obj:控件ID或控件名称
```

4.4.4.1 ref-示例 1

```
//重绘ID为1的控件
ref 1
```

4.4.4.2 ref-示例 2

```
//重绘t0的控件
ref t0
```



注意: 如果一个控件被 GUI 指令画出来的内容遮挡或者被另外的控件遮挡之后需要再显示出来，就使用 ref 来重绘。

只能重绘当前页面的控件

不支持跨页面重绘控件

不能重绘其他页面的控件

4.4.4.3 ref 指令-样例工程下载

演示工程下载链接：

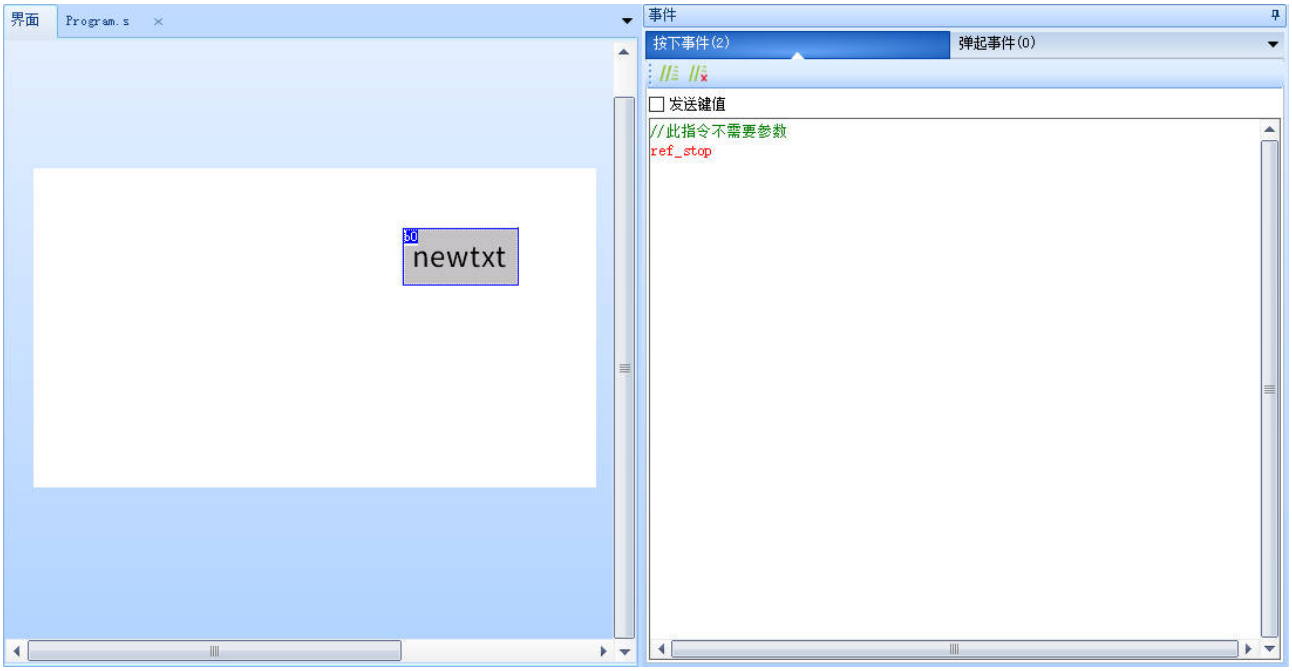
《ref 指令》演示工程下载

4.4.5 ref_stop-暂停屏幕刷新

```
ref_stop
```

4.4.5.1 ref_stop-示例 1

```
//此指令不需要参数
ref_stop
```



注意： 暂停屏幕刷新之后，所有语句会继续解析并执行，相应的属性赋值操作也会正常运行，但是屏幕上的控件不会刷新，修改任何控件的任何属性都不会自动刷新显示（但是属性已经被正常修改了）。直到设备收到恢复刷新指令 (ref_star) 后，被修改过的控件将会立刻刷新显示。

暂停刷新之后，即便使用 ref 指令也不会立刻刷新，直到执行 ref_star 指令的时候统才会统一刷新，但是所有的 gui 绘图指令 (比如画点，划线，画圆等) 是不受影响的，会立即显示。

4.4.5.2 ref_stop 指令-样例工程下载

演示工程下载链接：

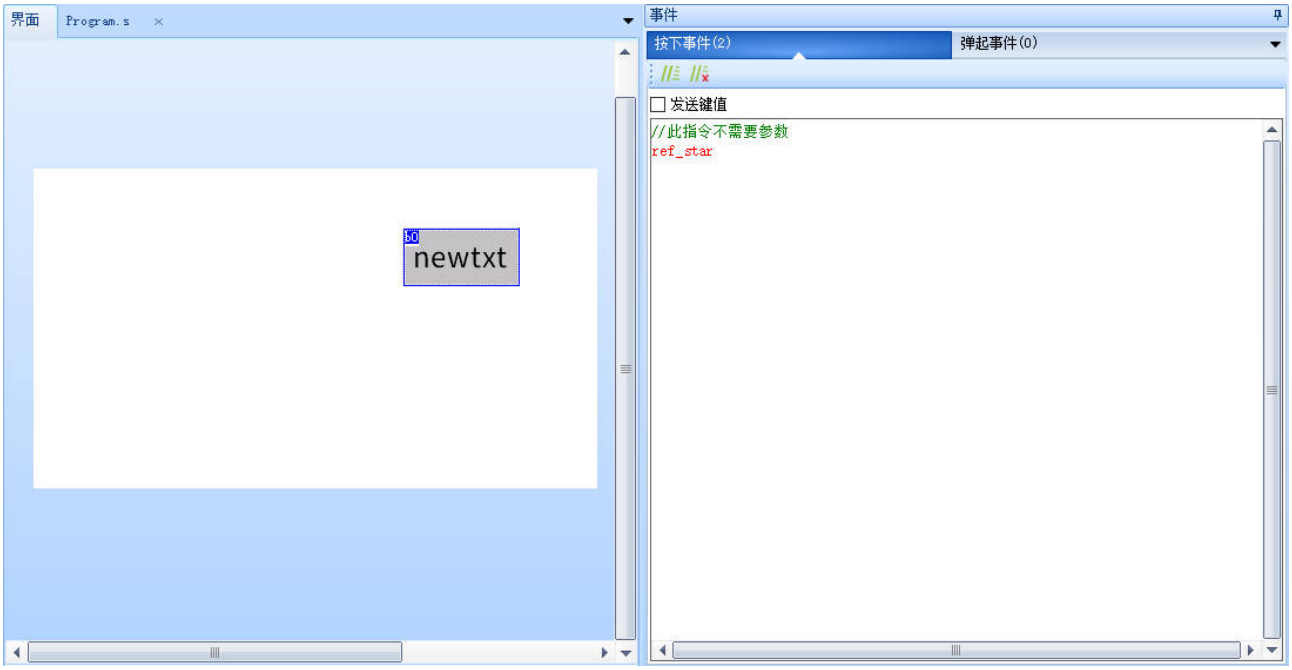
《ref_stop 指令和 ref_star 指令》演示工程下载

4.4.6 ref_star-恢复屏幕刷新

```
ref_star
```

4.4.6.1 ref_star-示例 1

```
//此指令不需要参数
ref_star
```



> **注意：** 此指令和 ref_stop 配合使用。

4.4.6.2 ref_star 指令-样例工程下载

演示工程下载链接：

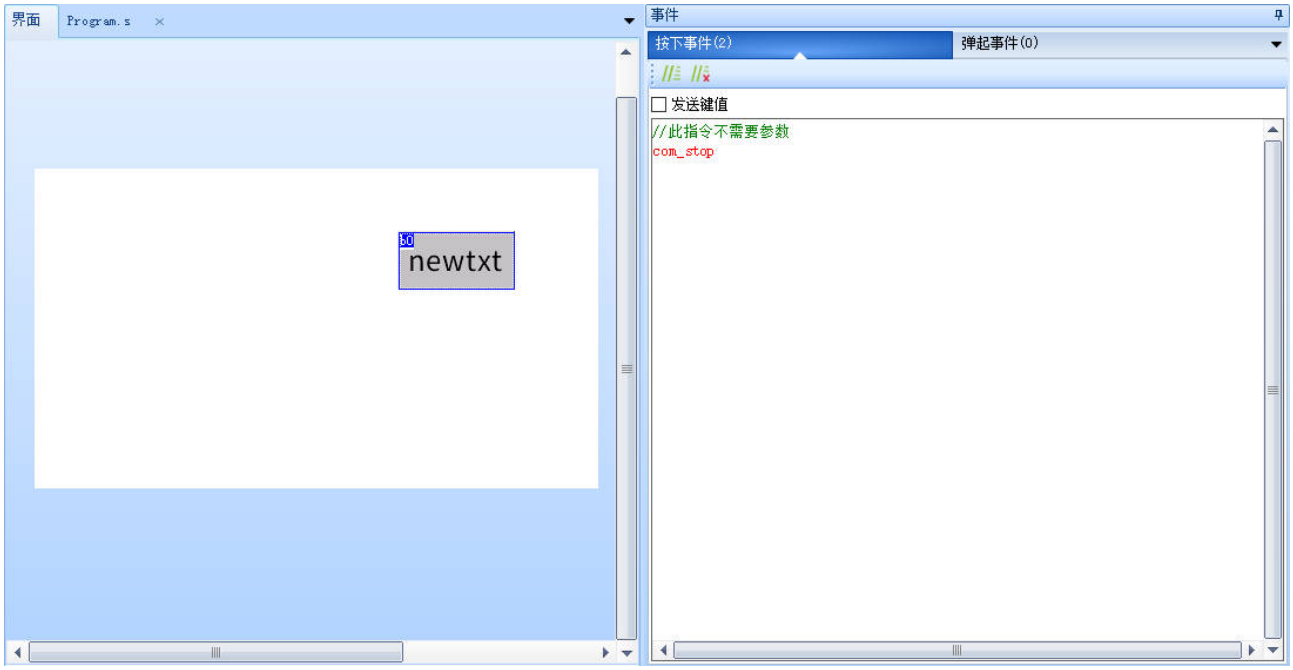
《ref_stop 指令和 ref_star 指令》演示工程下载

4.4.7 com_stop-暂停串口指令执行

```
com_stop
```

4.4.7.1 com_stop-示例

```
//此指令不需要参数
com_stop
```



4.4.7.2 com_stop-c 语言示例

不建议通过单片机发送 com_stop 指令，应在屏幕的脚本程序中判断是否执行 com_stop

注意： 暂停串口指令执行之后设备会继续接受指令，但是都不会执行，全部放在指令缓存区，直到执行“com_star”指令后，设备会从暂停时的指令开始到当前为止的所有指令全部执行。

使用指令暂停与恢复功能的时候，请评估您的设备的串口缓存区大小和指令缓存队列的最大数量是否足够支持你需要缓存的指令数目。这两项参数在你购买的设备规格书中的参数表中可以查询到。

x 系列的串口缓冲区为 4KB，其他系列的串口缓冲区为 1KB。

4.4.7.3 com_stop 与 ref_stop 的区别

com_stop: 指令实际还未执行, 在 com_stop 期间可以用 code_c 指令来清除串口中的所有指令

ref_stop: 指令已经执行完毕, 但是效果还没有刷新到屏幕上

4.4.7.4 com_stop 指令-相关链接

com_star-恢复串口指令执行 (page 704)

4.4.7.5 com_stop 指令-样例工程下载

演示工程下载链接：

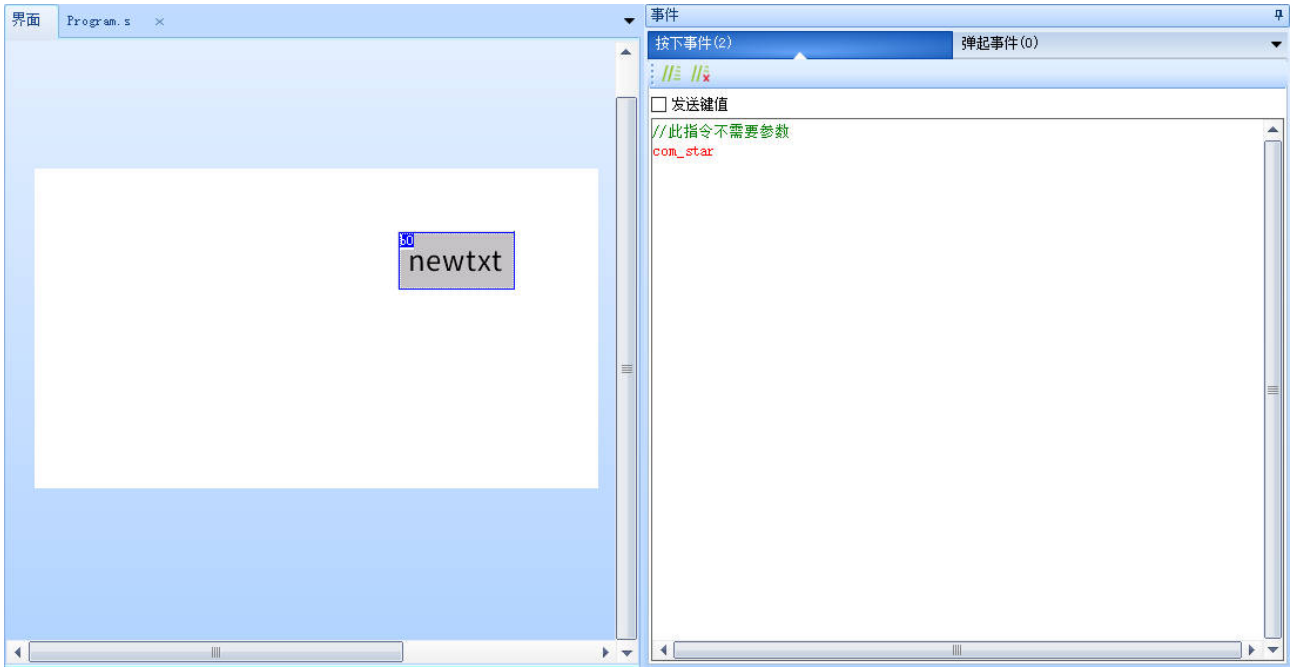
《com_stop 指令和 com_star 指令》演示工程下载

4.4.8 com_star-恢复串口指令执行

```
com_star
```

4.4.8.1 com_star-示例

```
//此指令不需要参数
com_star
```



4.4.8.2 com_star-c 语言示例

不建议通过单片机发送 com_star 指令，应在屏幕的脚本程序中判断是否执行 com_star

> **注意:** 设备执行此指令之后，将从暂停时的指令开始到当前为止的所有指令全部执行。
>
> 使用指令暂停与恢复功能的时候，请评估您的设备的串口缓存区大小和指令缓存队列的最大数量是否足够支持你需要缓存的指令数目。这两项参数在你购买的设备规格书中的参数表中可以查询到。

4.4.8.3 com_star 指令-相关链接

com_stop-暂停串口指令执行 (page 702)

4.4.8.4 com_star 指令-样例工程下载

演示工程下载链接：

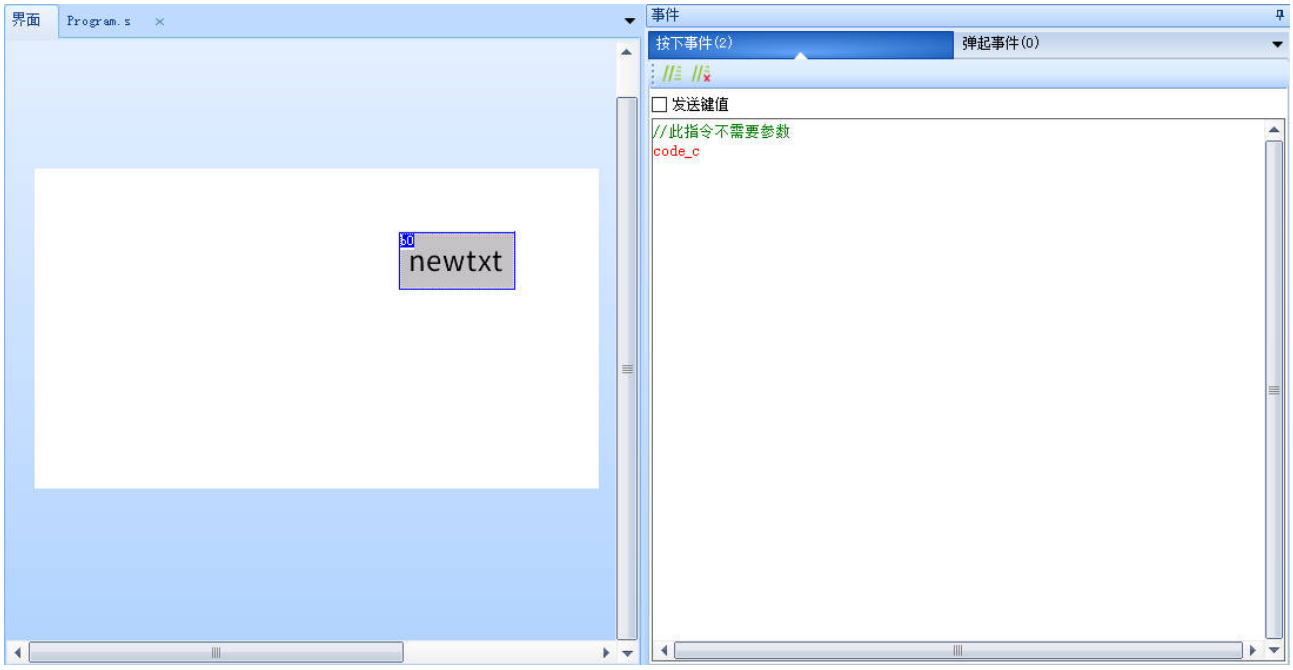
《com_stop 指令和 com_star 指令》演示工程下载

4.4.9 code_c-清空串口指令缓冲区

```
code_c
```

4.4.9.1 code_c-示例

```
//此指令不需要参数
code_c
```



4.4.9.2 code_c 指令-样例工程下载

无

4.4.9.3 code_c-c 语言示例

不建议通过串口去 code_c 串口缓冲区，没有实际意义，code_c 主要用于主动解析中

提示： code_c 会立即清空串口指令缓冲区还没有执行的所有指令，无论是否开启主动解析。

与 code_c 类似的 udelete 指令，仅在 recmod=1 下有效

code_c 与 udelete 指令通常都只在主动解析模式下使用，因为 code_c 会清空所有指令，因此不建议使用 code_c 指令

4.5 CRC 校验指令

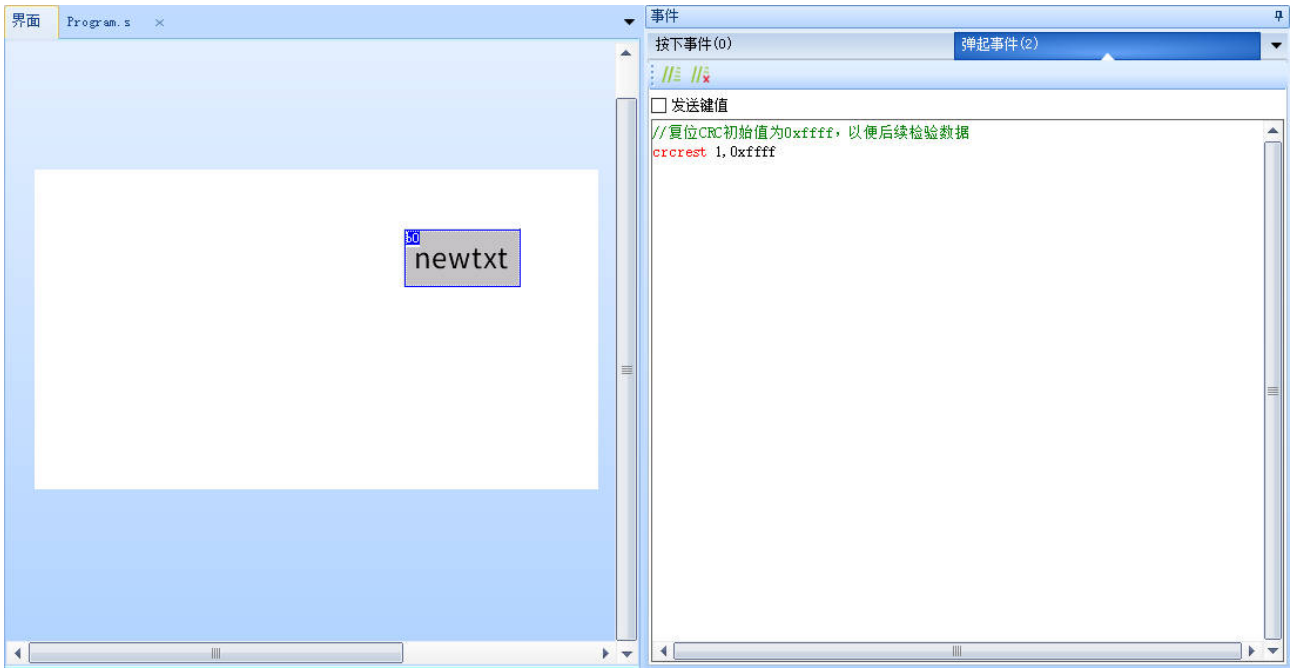
用于校验 crc_modbus16 的指令

4.5.1 crcrest-复位 crc 初始值

```
crcrest crctype,initval

crctype:crc校验类型(必须为1,Modbus crc16校验方式)

initval:初始值(一般为0xffff)
```

4.5.1.1 crcrest-示例

```
//复位CRC初始值为0xffff，以便后续检验数据
crcrest 1,0xffff
```



注意: 复位之后，可使用 crcputs 或 crcputh 或 crcputu 校验指定数据，检验完毕读取系统变量 crcval 获得校验结果。

完整的 CRC 校验实例代码请参考: 程序中使用 *CRC* 校验数据 (page 1018)

4.5.1.2 crcrest 指令-相关链接

crcputh-crc 校验一组 Hex (page 713)

crcputs-crc 校验一个变量/常量 (page 709)

crcputu-crc 校验一段串口缓冲区数据 (page 716)

crcval-crc 校验结果 (page 793)

4.5.1.3 crcrest 指令-样例工程下载

演示工程下载链接：

《CRC 校验》演示工程下载

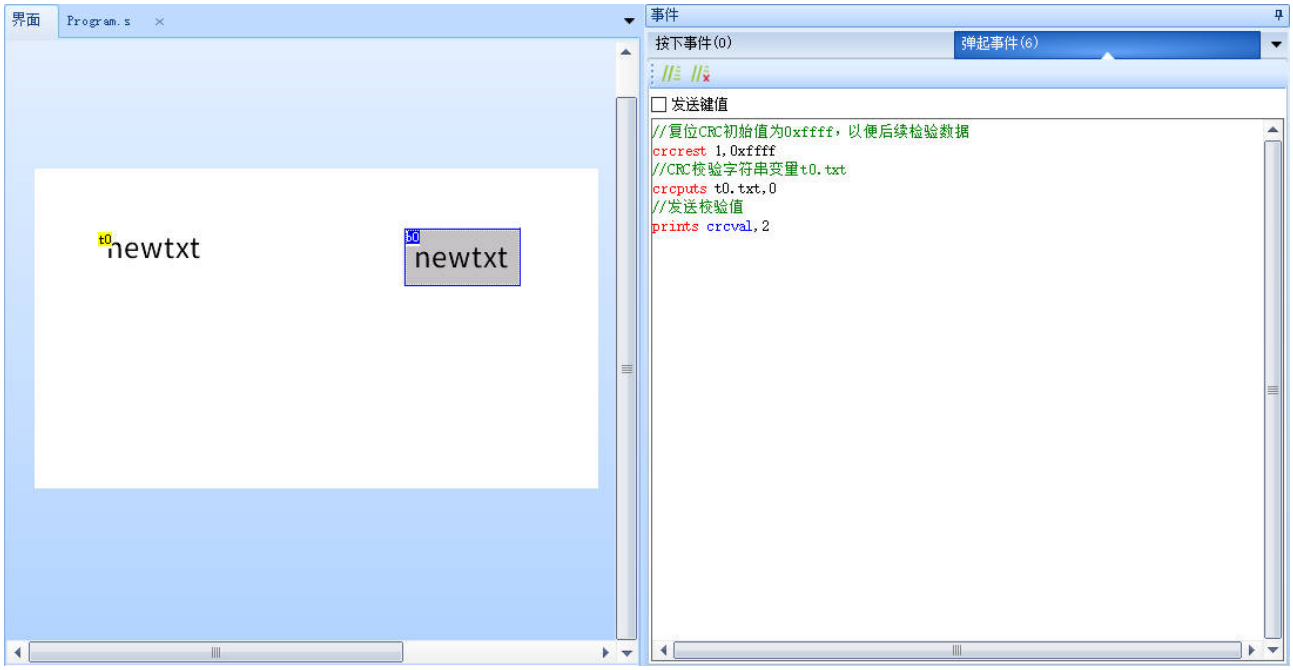
《crc 校验例程一对》演示工程下载

4.5.2 crcputs-crc 校验一个变量/常量

```
crcputs att,length

att:变量名称

length:需要校验的数据长度(0为自动长度)
```

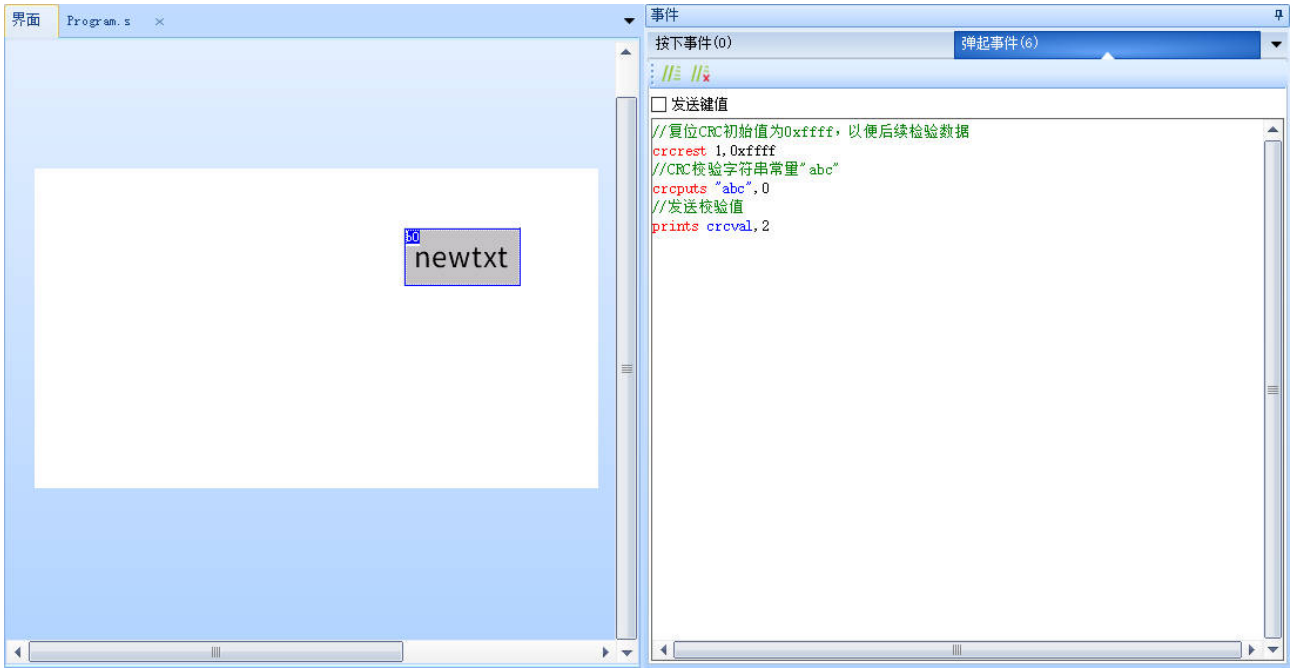
4.5.2.1 crcputs-示例 1

```
//复位CRC初始值为0xffff, 以便后续检验数据
crcrest 1,0xffff
//CRC校验字符串变量t0.txt
crcputs t0.txt,0
//发送校验值
prints crcval,2
```



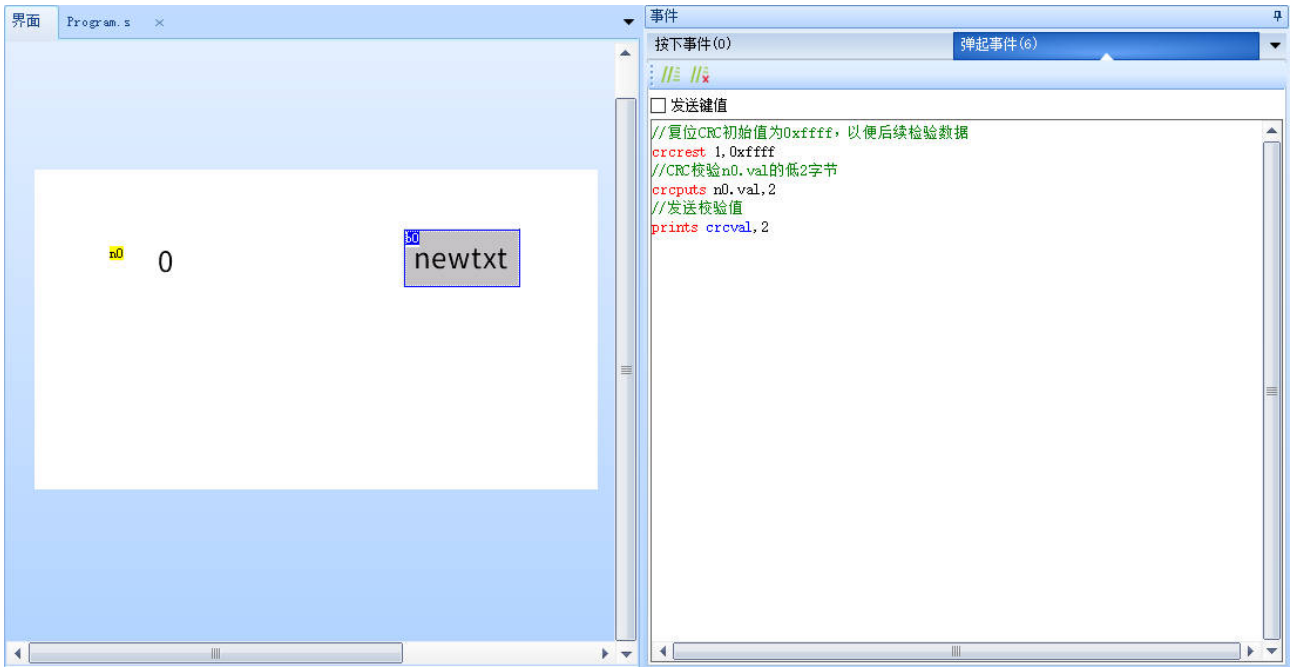
4.5.2.2 crcputs-示例 2

```
//复位CRC初始值为0xffff, 以便后续检验数据
crcrest 1,0xffff
//CRC校验字符串常量"abc"
crcputs "abc",0
//发送校验值
prints crcval,2
```



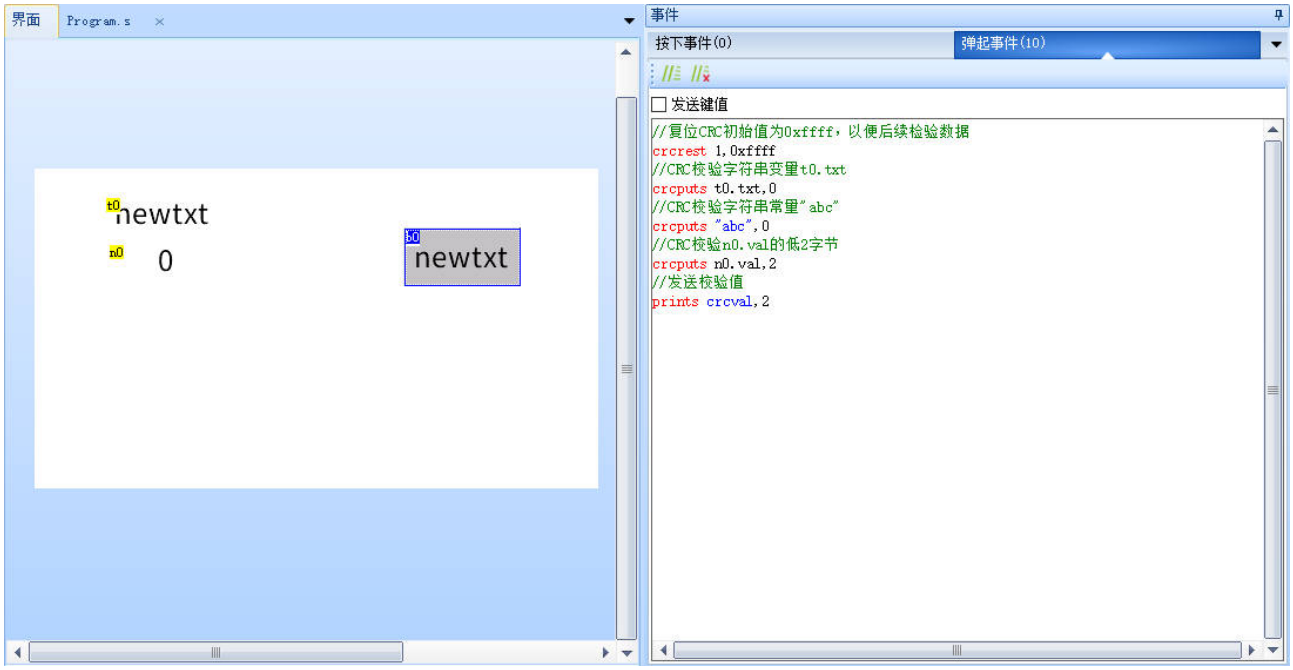
4.5.2.3 crcputs-示例 3

```
//复位CRC初始值为0xffff，以便后续检验数据
crcrest 1,0xffff
//CRC校验n0.val的低2字节
crcputs n0.val,2
//发送校验值
prints crcval,2
```



4.5.2.4 crcputs-示例 4

```
//复位CRC初始值为0xffff，以便后续检验数据
crcrest 1,0xffff
//CRC校验字符串变量t0.txt
crcputs t0.txt,0
//CRC校验字符串常量"abc"
crcputs "abc",0
//CRC校验n0.val的低2字节
crcputs n0.val,2
//发送校验值
prints crcval,2
```



注意: 开始校验前必须使用 crcrest 初始化 crc 校验功能。

使用 crcputs 或 crcputh 或 crcputu 校验指定数据, 检验完毕读取系统变量 crcval 获得校验结果。

完整的 CRC 校验实例代码请参考: *程序中使用 CRC 校验数据* (page 1018)

4.5.2.5 crcputs 指令-相关链接

crcputh-crc 校验一组 Hex (page 713)

crcputu-crc 校验一段串口缓冲区数据 (page 716)

crcrest-复位 crc 初始值 (page 707)

crcval-crc 校验结果 (page 793)

4.5.2.6 crcputs 指令-样例工程下载

演示工程下载链接：

《CRC 校验》演示工程下载

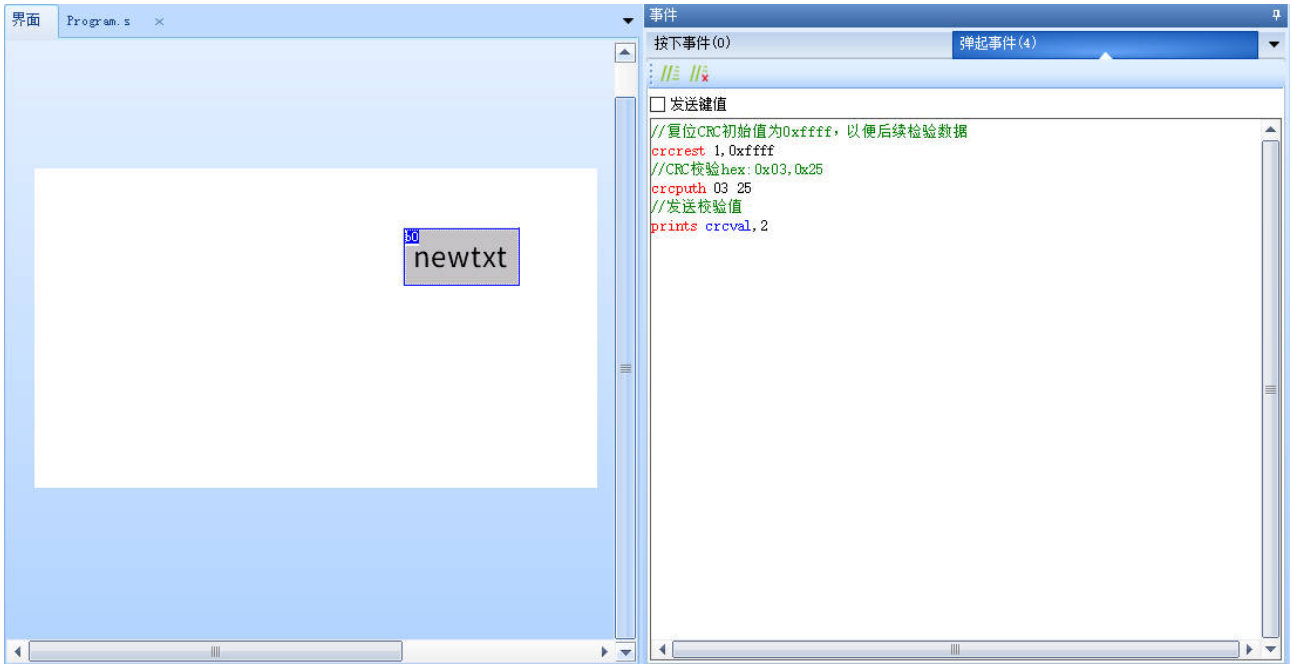
《crc 校验例程一对》演示工程下载

4.5.3 crcputh-crc 校验一组 Hex

```
crcputh Hex

Hex:需要校验的数据的16进制字符串表达式(每个字节之间用空格隔开)
```

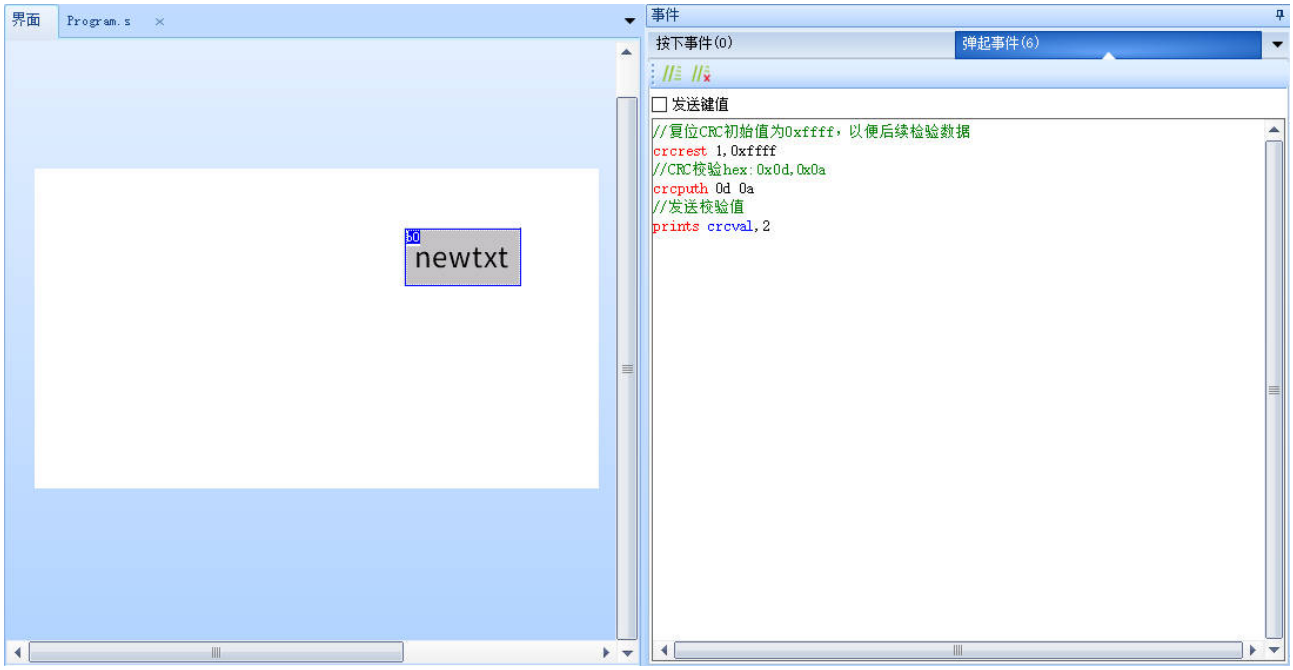
4.5.3.1 crcputh-示例 1

```
//复位CRC初始值为0xffff, 以便后续检验数据
crcrest 1,0xffff
//CRC校验hex:0x03,0x25
crcputh 03 25
//发送校验值
prints crcval,2
```



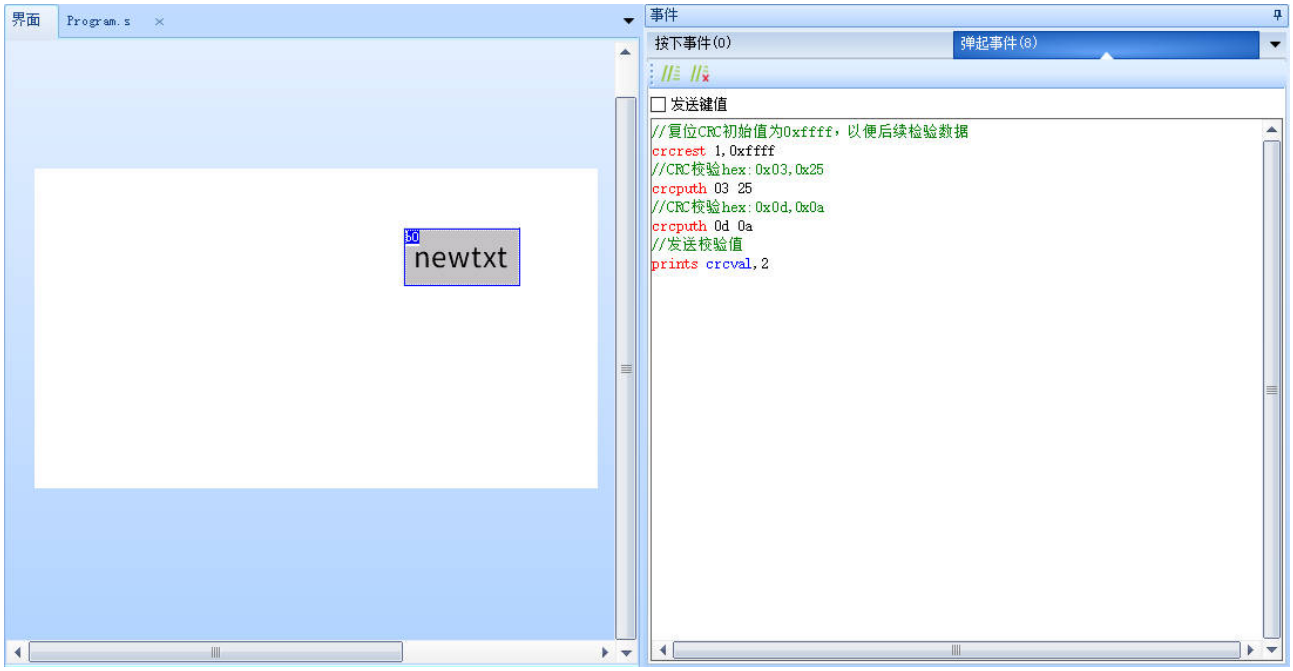
4.5.3.2 crcputh-示例 2

```
//复位CRC初始值为0xffff, 以便后续检验数据
crcrest 1,0xffff
//CRC校验hex:0x0d,0x0a
crcputh 0d 0a
//发送校验值
prints crcval,2
```



4.5.3.3 crcputh-示例 3

```
//复位CRC初始值为0xffff，以便后续检验数据
crcrest 1,0xffff
//CRC校验hex:0x03,0x25
crcputh 03 25
//CRC校验hex:0x0d,0x0a
crcputh 0d 0a
//发送校验值
prints crcval,2
```



注意： 开始校验前必须使用 crcrest 初始化 crc 校验功能。

使用 crcputs 或 crcputh 或 crcputu 校验指定数据, 检验完毕读取系统变量 crcval 获得校验结果。
完整的 CRC 校验实例代码请参考: 程序中使用 *CRC* 校验数据 (page 1018)

4.5.3.4 crcputh 指令-相关链接

crcputs-crc 校验一个变量/常量 (page 709)
crcputu-crc 校验一段串口缓冲区数据 (page 716)
*crcrest-*复位 *crc* 初始值 (page 707)
crcval-crc 校验结果 (page 793)

4.5.3.5 crcputh 指令-样例工程下载

演示工程下载链接:
《CRC 校验》演示工程下载
《crc 校验例程一对》演示工程下载

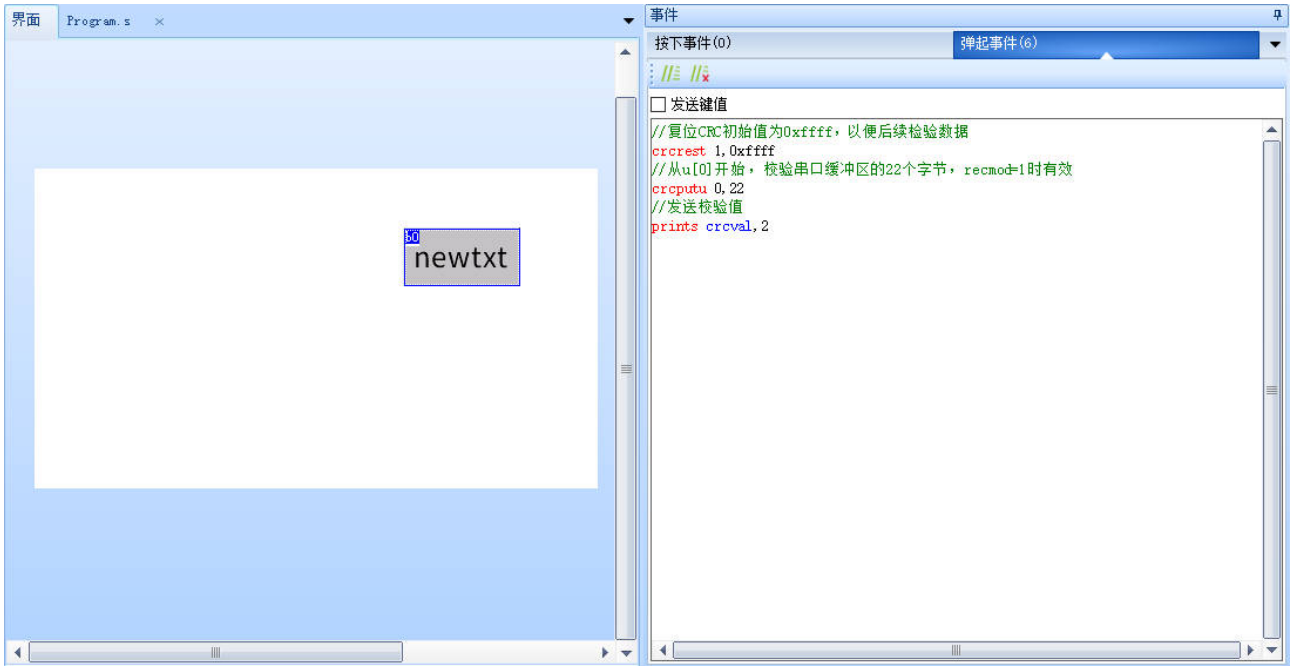
4.5.4 crcputu-crc 校验一段串口缓冲区数据

(recmod=1 时有效)

```
crcputu star,length

star:串口缓冲区数据起始位

length:需要校验的数据长度
```

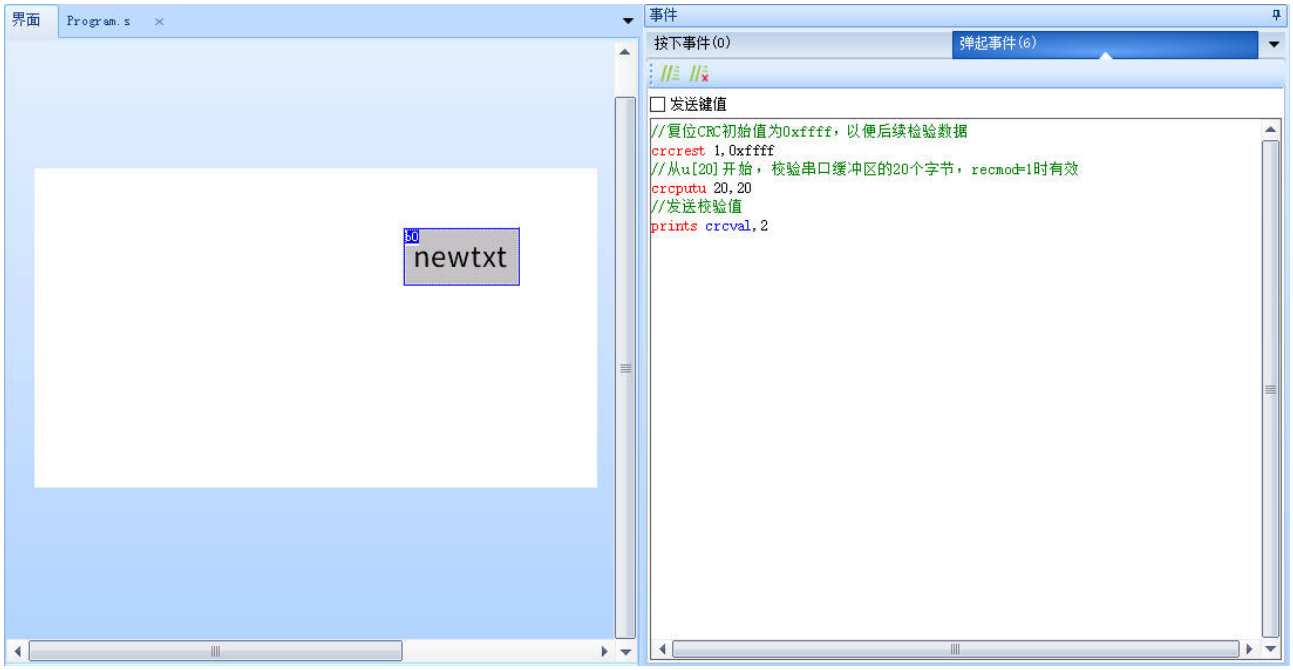
4.5.4.1 crcputu-示例 1

```
//从u[0]开始，校验串口缓冲区的22个字节，recmod=1时有效
crcputu 0,22
```



4.5.4.2 crcputu-示例 2

```
//从u[20]开始，校验串口缓冲区的20个字节，recmod=1时有效
crcputu 20,20
```



注意： 开始校验前必须使用 crcrest 初始化 crc 校验功能。

使用 crcputs 或 crcputh 或 crcputu 校验指定数据，检验完毕读取系统变量 crcval 获得校验结果。

完整的 CRC 校验实例代码请参考: *程序中使用 CRC 校验数据* (page 1018)

4.5.4.3 crcputu 指令-相关链接

crcputh-crc 校验一组 Hex (page 713)

crcputs-crc 校验一个变量/常量 (page 709)

crcrest-复位 crc 初始值 (page 707)

crcval-crc 校验结果 (page 793)

4.5.4.4 crcputu 指令-样例工程下载

演示工程下载链接:

《CRC 校验》演示工程下载

《crc 校验例程一对》演示工程下载

4.6 GUI 绘图指令

不推荐使用 GUI 绘图指令, 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖。

绘图指令在跳转页面后会丢失。

绘图指令和控件有重叠时，当控件刷新时，和控件重叠的部分会丢失。

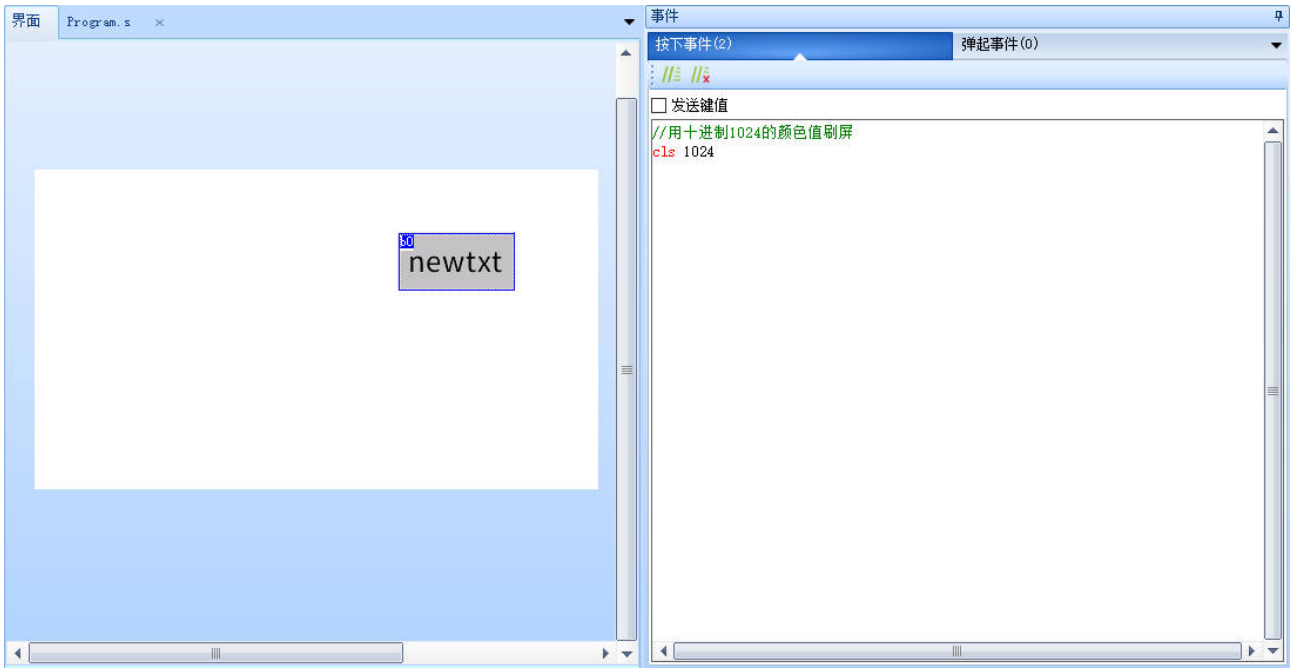
4.6.1 cls-清屏指令

在当前页面将整个屏幕刷为统一的颜色，不支持跨页面将整个屏幕刷为统一的颜色，不能写在页面的前初始化事件中

```
cls color

color:十进制颜色值或颜色代号
```

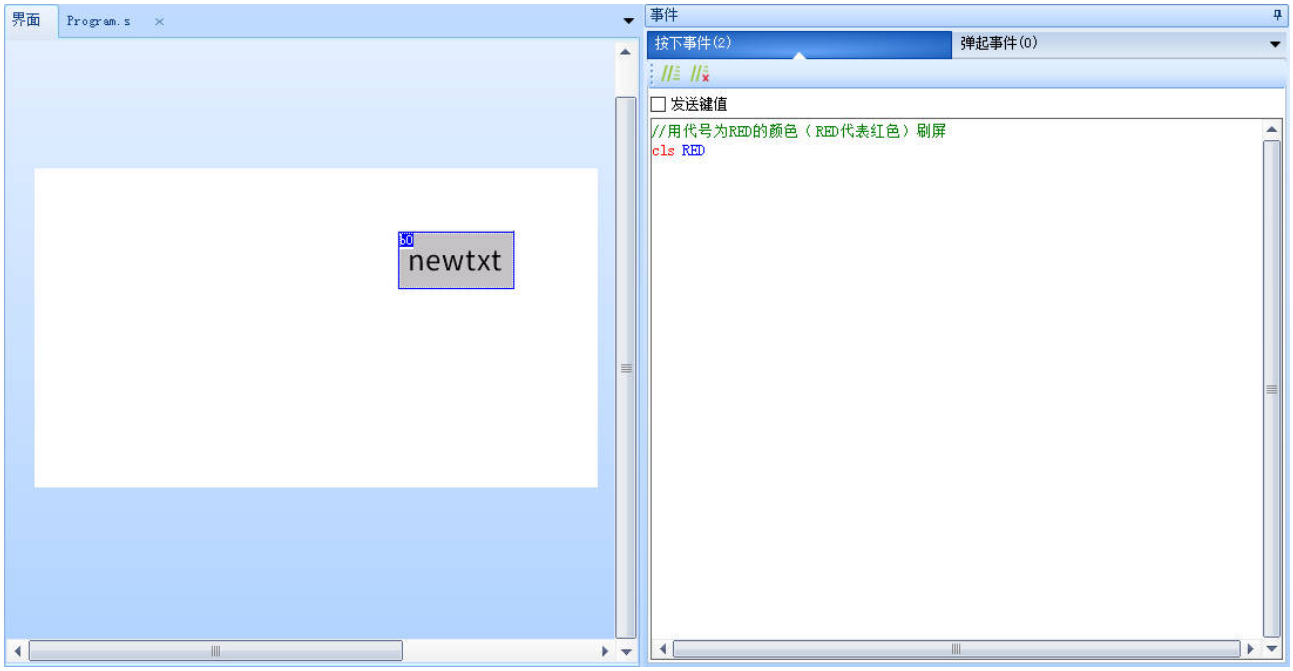
4.6.1.1 cls-示例 1

```
//用十进制1024的颜色值刷屏
cls 1024
```



4.6.1.2 cls-示例 2

```
//用代号为RED的颜色（RED代表红色）刷屏
cls RED
```



4.6.1.3 cls-c 语言示例

单片机通过串口代号为 BLACK 的颜色（BLACK 代表黑色）刷屏

```
printf("cls BLACK\xff\xff\xff");
```

注意： 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖

4.6.1.4 cls 指令-相关链接

HMI 颜色代号表 (page 740)

4.6.1.5 cls 指令-样例工程下载

演示工程下载链接：

《绘图指令》演示工程下载

《绘图指令演示工程 2》下载

4.6.2 pic-刷图指令

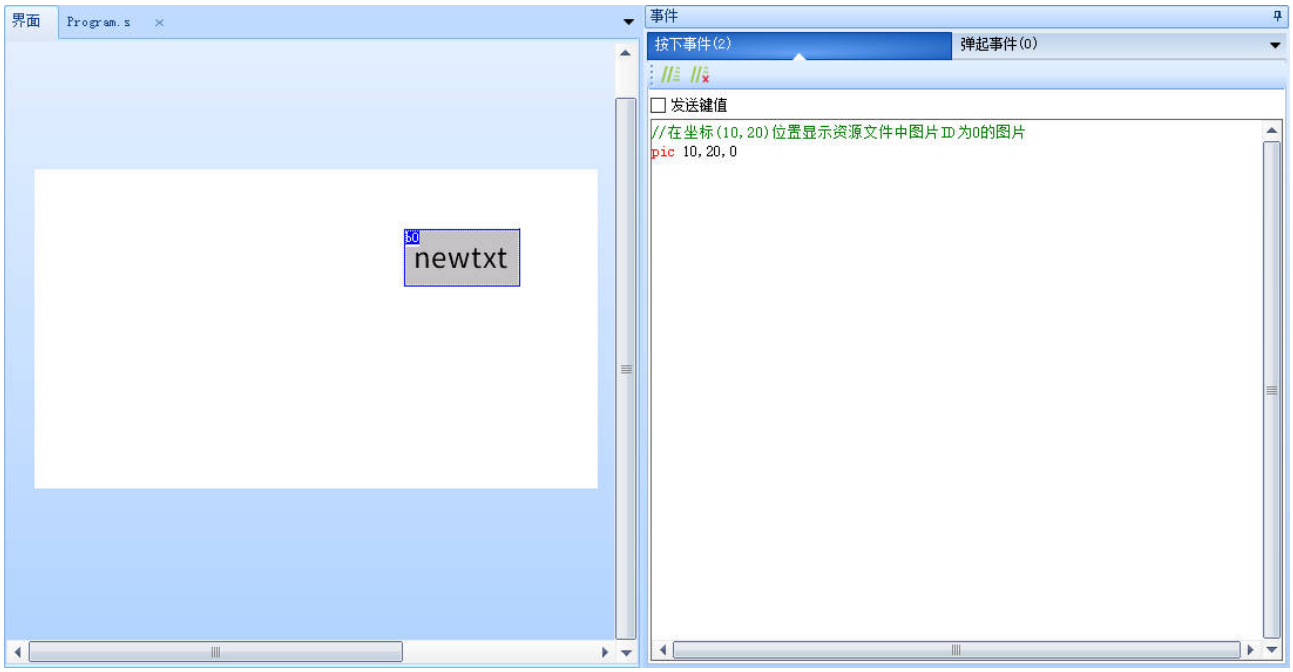
在当前页面进行刷图，不支持跨页面刷图，不能写在页面的前初始化事件中

注意： 不推荐使用 GUI 绘图指令, 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖。绘图指令在跳转页面后会丢失。绘图指令和控件有重叠时，当控件刷新时，和控件重叠的部分会丢失。

```
pic x,y,picid

x:起始点x坐标;

y:起始点y坐标;

picid:图片ID;
```

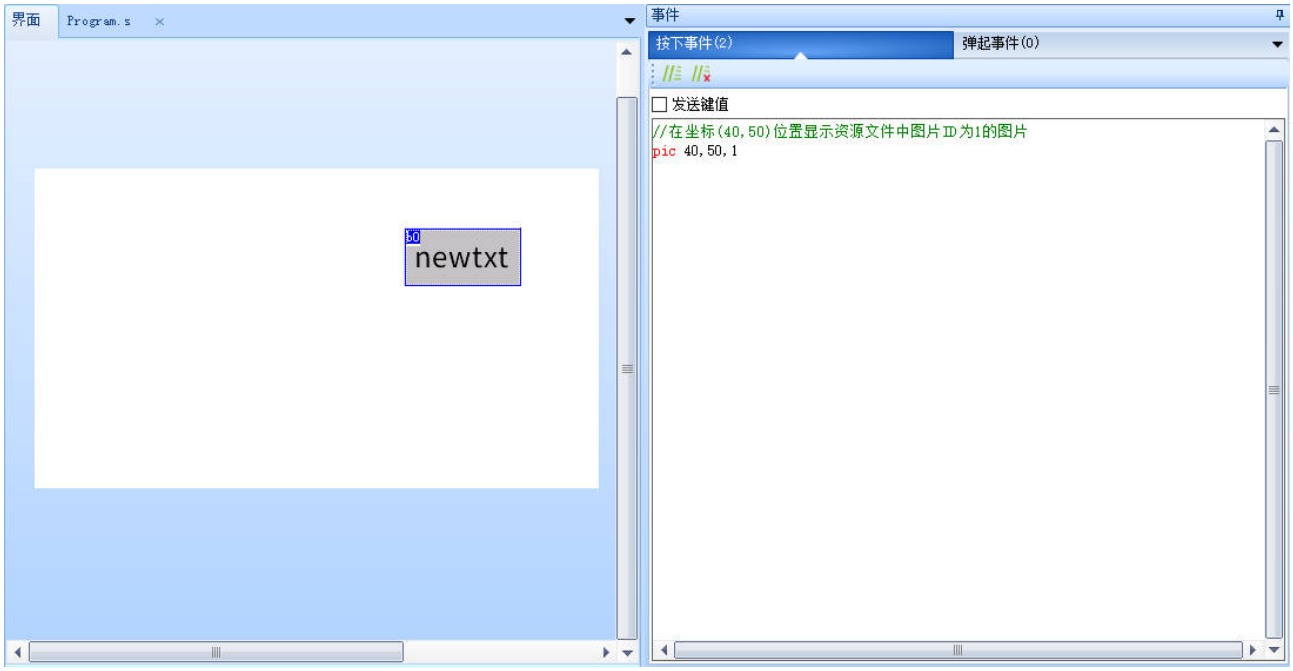
4.6.2.1 pic-示例 1

```
//在坐标(10,20)位置显示资源文件中图片ID为0的图片
pic 10,20,0
```



4.6.2.2 pic-示例 2

```
//在坐标(40,50)位置显示资源文件中图片ID为1的图片
pic 40,50,1
```



4.6.2.3 pic-c 语言示例

单片机通过串口在坐标 (40,50) 位置显示资源文件中图片 ID 为 1 的图片

```
int posX=40, posY=50, pic=1;
printf("pic %d,%d,%d\xff\xff\xff", posX, posY, pic);
```

> **注意:** 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖

4.6.2.4 pic 指令-样例工程下载

演示工程下载链接：

《绘图指令》演示工程下载

《绘图指令演示工程 2》下载

4.6.3 picq-切图指令

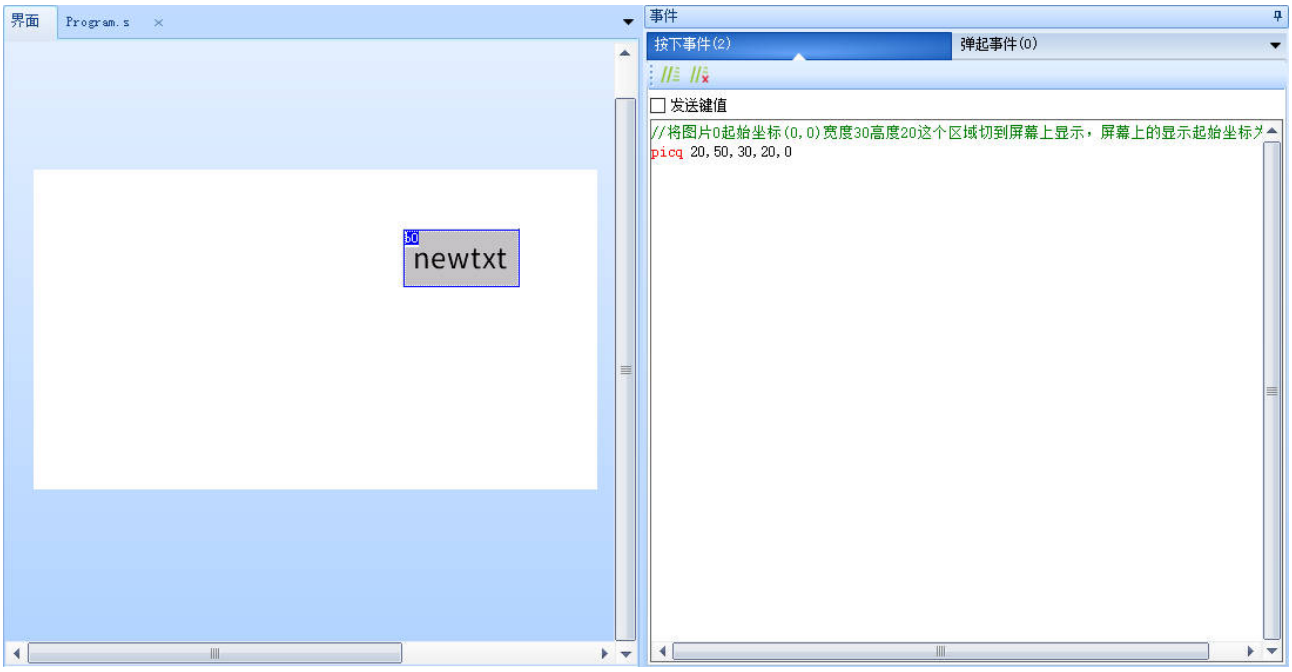
在当前页面进行切图，不支持跨页面切图，不能写在页面的前初始化事件中

> **注意：** 不推荐使用 GUI 绘图指令, 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖。绘图指令在跳转页面后会丢失。绘图指令和控件有重叠时，当控件刷新时，和控件重叠的部分会丢失。

```
picq x,y,w,h,picid

x:屏幕起始点x坐标;

y:屏幕起始点y坐标;

w:区域宽度 ;

h:区域高度;

picid:图片ID;
```

4.6.3.1 picq-示例 1

```
//将图片0起始坐标(0,0)宽度30高度20这个区域切到屏幕上显示，屏幕上的显示起始坐标为(20,50)
picq 20,50,30,20,0
```



注意： 此指令要求图片必须是全屏图片，否则切出来的图像不是你想要的。

4.6.3.2 picq-c 语言示例

单片机通过串口将图片 0 起始坐标 (0,0) 宽度 30 高度 20 这个区域切到屏幕上显示，屏幕上的显示起始坐标为 (20,50)

```
int posX=40, posY=50, width=30, height=20, pic=1;
printf("picq %d,%d,%d,%d,%d\xff\xff\xff", posX, posY, width, height, pic);
```

注意： 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖

4.6.3.3 picq 指令-样例工程下载

演示工程下载链接：

《绘图指令》演示工程下载

《绘图指令演示工程 2》下载

4.6.4 xpic-高级切图指令

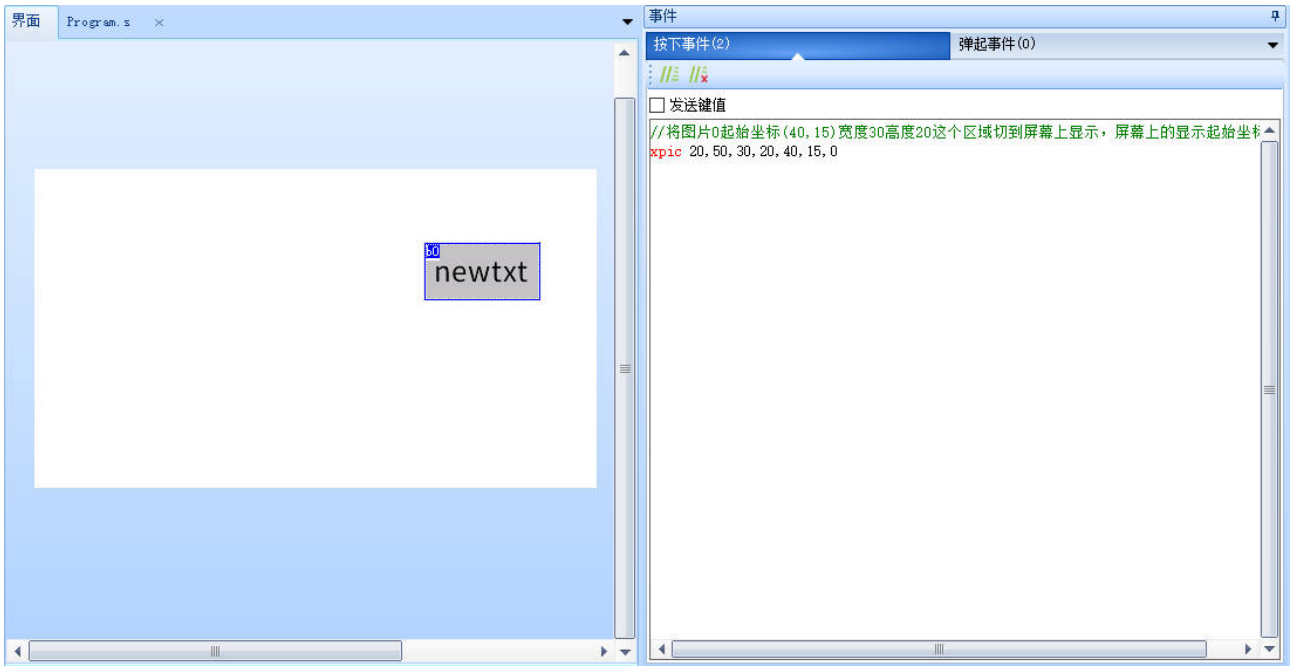
在当前页面进行高级切图，不支持跨页面高级切图，不能写在页面的前初始化事件中

> **注意：** 不推荐使用 GUI 绘图指令, 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖。绘图指令在跳转页面后会丢失。绘图指令和控件有重叠时，当控件刷新时，和控件重叠的部分会丢失。

```
xpic x,y,w,h,x0,y0,picid

x:屏幕起始点x坐标;

y:屏幕起始点y坐标;

w:区域宽度 ;

h:区域高度;

x0:图片起始点x坐标;

y0:图片起始点y坐标;

picid:图片ID;
```

4.6.4.1 xpic-示例

```
//将图片0起始坐标(40,15)宽度30高度20这个区域切到屏幕上显示，屏幕上的显示起始坐标为(20,50)
xpic 20,50,30,20,40,15,0
```



4.6.4.2 xpic-c 语言示例

单片机通过串口将图片 0 起始坐标 (40,15) 宽度 30 高度 20 这个区域切到屏幕上显示，屏幕上的显示起始坐标为 (20,50)

```
int posX=20, posY=50, width=30, height=20, posX1=40, posY1=15, pic=0;
printf("xpic %d,%d,%d,%d,%d,%d,%d\xff\xff\xff", posX, posY, width, height, posX1, posY1,␣
↪pic);
```

注意： 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖

4.6.4.3 xpic 指令-样例工程下载

演示工程下载链接：

《绘图指令》演示工程下载

《绘图指令演示工程 2》下载

4.6.5 xstr-写字指令

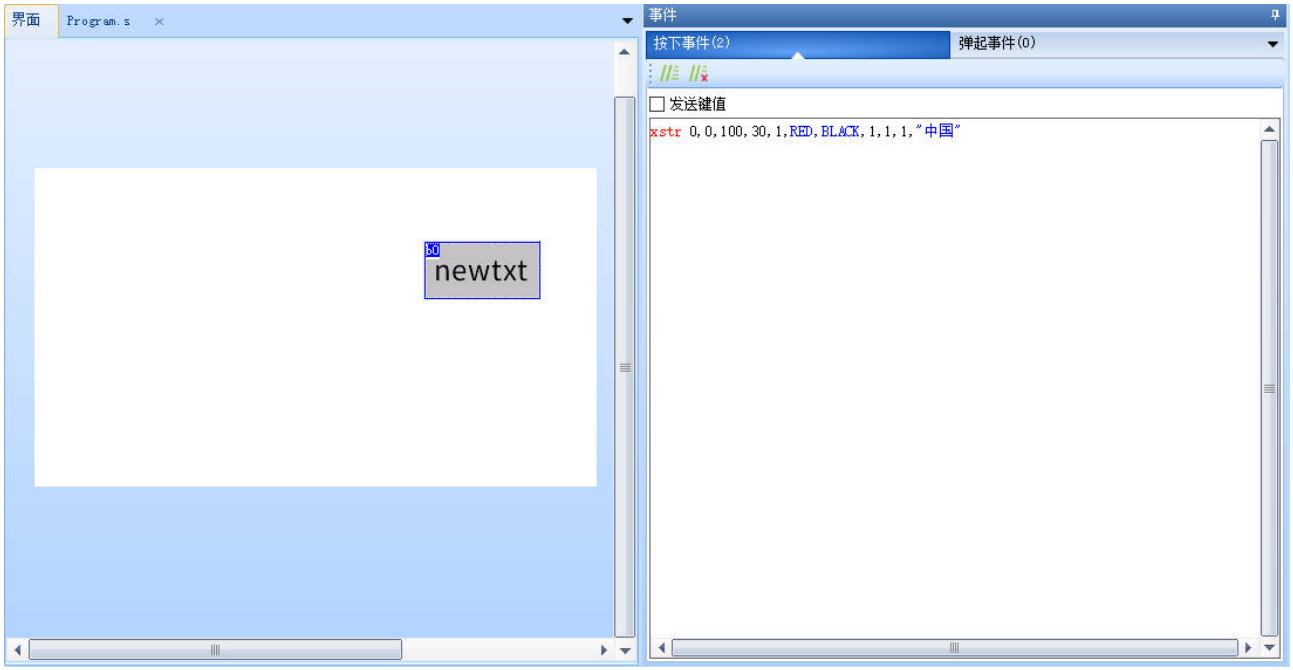
在当前页面写字，不支持跨页面写字，不能写在页面的前初始化事件中

注意: 不推荐使用 GUI 绘图指令, 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖。绘图指令在跳转页面后会丢失。绘图指令和控件有重叠时，当控件刷新时，和控件重叠的部分会丢失。

```
xstr x,y,w,h,fontid,pointcolor,backcolor,xcenter,ycenter,sta,string

x:起始点坐标x;

y:起始点坐标y;

w:区域宽度;

h:区域高度;

fontid:字库ID;

pointcolor:字体颜色;

backcolor:背景色(sta设置为切图或图片时，backcolor表示图片ID);

xcenter:水平对齐方式(0为左对齐，1为居中，2为右对齐);

ycenter: 垂直对齐方式(0为上对齐，1为居中，2为下对齐);

sta:背景填充方式(0为切图，1为单色，2为图片，3为无背景，
↪sta设置为切图或图片时，backcolor表示图片ID)

string:字符内容;
```

4.6.5.1 xstr-示例

```
xstr 0,0,100,30,1,RED,BLACK,1,1,1,"中国"
```



实例解释：使用字库 1 在起始坐标 (0,0)，宽度 100, 高度 30 这个区域写出”中国”，字体色为 RED，背景色为 BLACK(如果不想写背景色 (即无背景) 可以设置 sta 参数为 3), 水平对齐方式为居中，垂直对齐方式也为居中。

4.6.5.2 xstr-c 语言示例

使用字库 1 在起始坐标 (0,0)，宽度 100, 高度 30 这个区域写出”中国”，字体色为 RED，背景色为 BLACK(如果不想写背景色 (即无背景) 可以设置 sta 参数为 3), 水平对齐方式为居中，垂直对齐方式也为居中。

```
int posX=0, posY=0, width=100, height=30, fontid=40, pointcolor=15, backcolor=0,␣
↪xcenter=1, ycenter=1, sta=1;
printf("xstr %d,%d,%d,%d,%d,%d,%d,%d,%d,%d,%s\xff\xff\xff", posX, posY, width, height,␣
↪fontid, pointcolor, backcolor, xcenter, ycenter, sta, "中国");
```

> **注意：** 字符写到超过设定的 w（宽度）以后将自动换行，如果换行到 h（高度）之后还有剩下的字符没写完，将会被忽略。
>
> 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖

4.6.5.3 xstr 指令-样例工程下载

演示工程下载链接：

《绘图指令》演示工程下载

《绘图指令演示工程 2》下载

4.6.5.4 xstr 指令-相关链接

HMI 颜色代号表 (page 740)

4.6.6 fill-区域填充指令

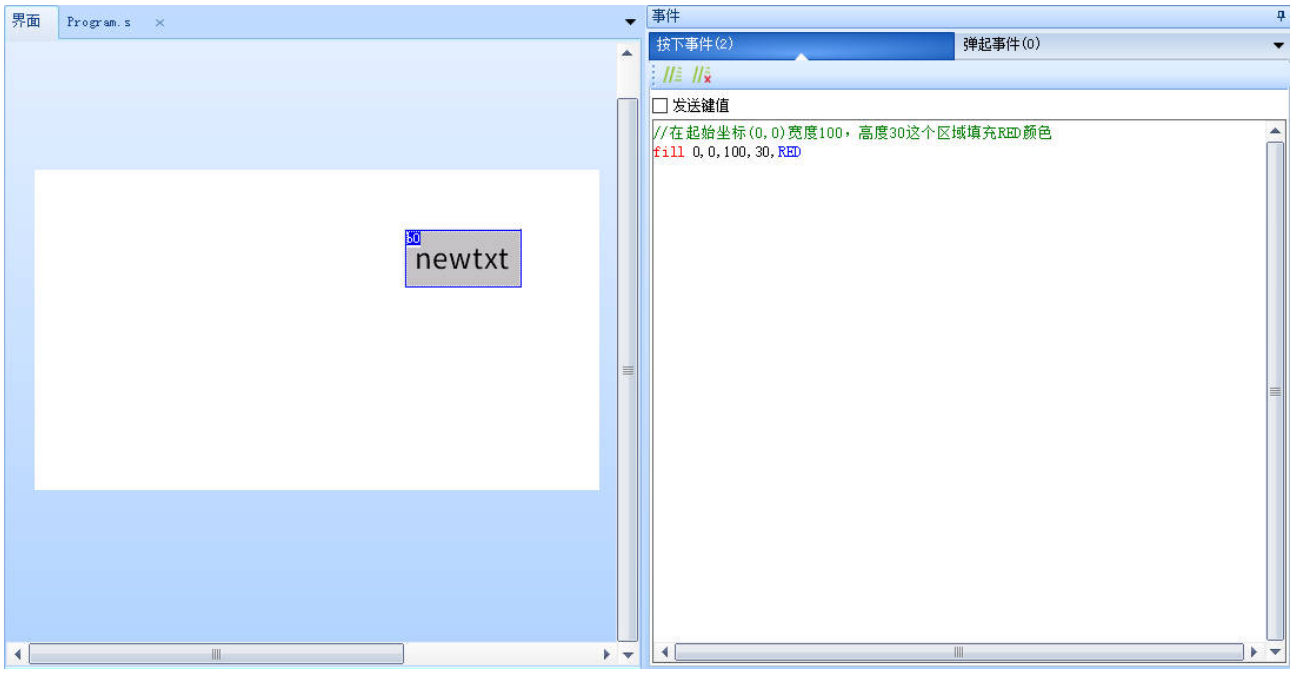
在当前页面进行画一个实心矩形，不支持跨页面画实心矩形，不能写在页面的前初始化事件中

> **注意：** 不推荐使用 GUI 绘图指令, 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖。绘图指令在跳转页面后会丢失。绘图指令和控件有重叠时，当控件刷新时，和控件重叠的部分会丢失。

```
fill x,y,w,h,color

x:起始点坐标x;

y:起始点坐标y;

w:区域宽度;

h:区域高度;

color:填充颜色;
```

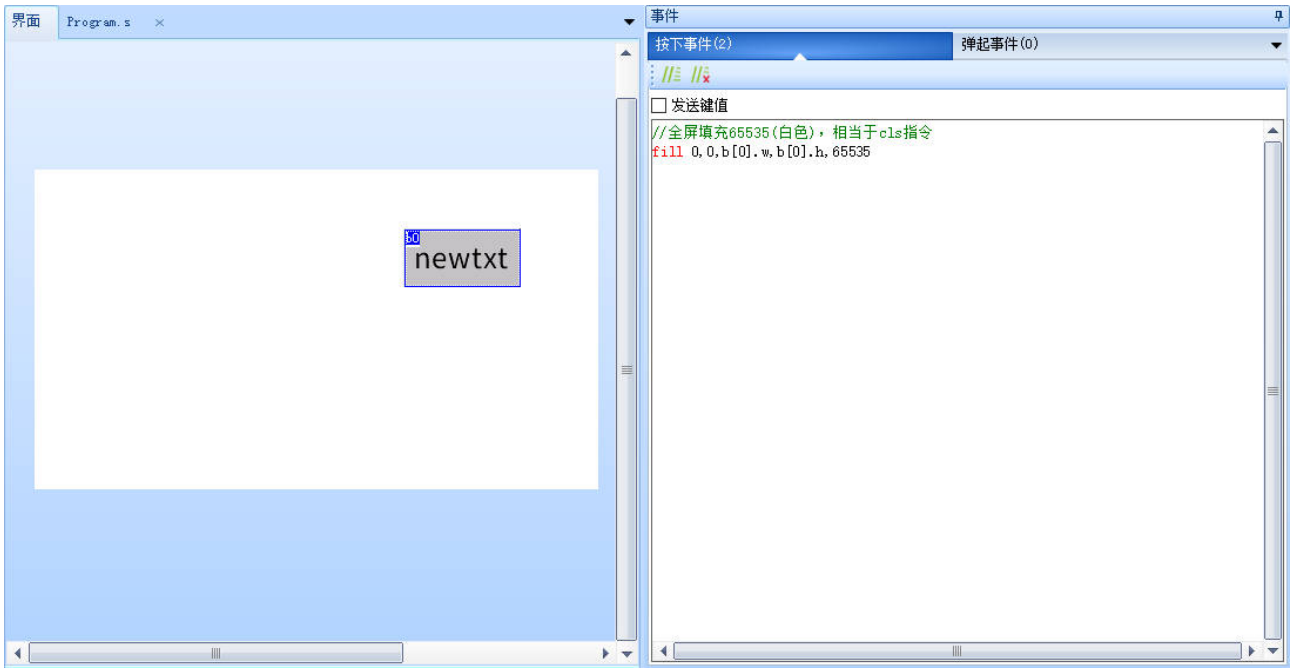
4.6.6.1 fill-示例 1

```
//在起始坐标(0,0)宽度100，高度30这个区域填充RED颜色
fill 0,0,100,30,RED
```



4.6.6.2 fill-示例 2

```
//全屏填充65535(白色)，相当于cls指令
fill 0,0,b[0].w,b[0].h,65535
```



4.6.6.3 fill-c 语言示例

单片机通过串口以起始坐标 (0,0) 宽度 100, 高度 100，填充 RGB565 颜色值为 1024

```
int posX=0, posY=0, width=100, height=100, color=1024;
printf("fill %d,%d,%d,%d,%d\xff\xff\xff", posX, posY, width, height, color);
```

注意： 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖

4.6.6.4 fill 指令-相关链接

HMI 颜色代号表 (page 740)

4.6.6.5 fill 指令-样例工程下载

演示工程下载链接：

《绘图指令》演示工程下载

《绘图指令演示工程 2》下载

4.6.7 line-画线指令

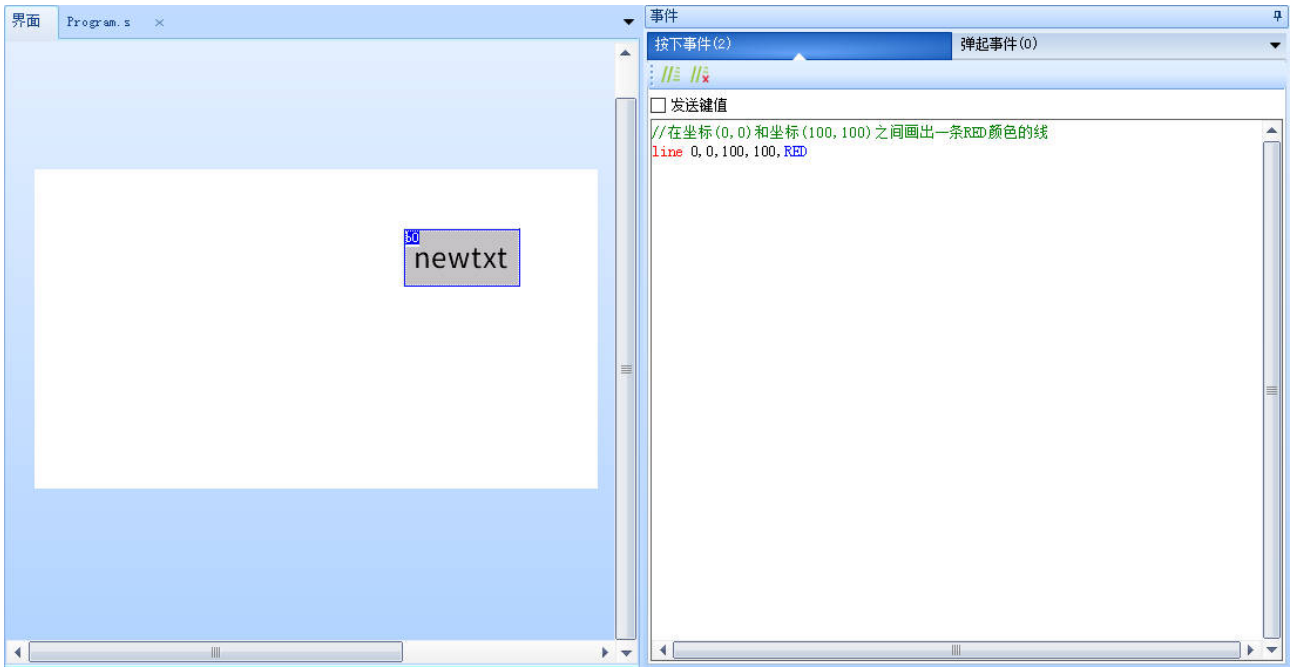
在当前页面进行画线，不能写在页面的前初始化事件中

> **注意：** 不推荐使用 GUI 绘图指令, 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖。绘图指令在跳转页面后会丢失。绘图指令和控件有重叠时，当控件刷新时，和控件重叠的部分会丢失。

```
line x,y,x2,y2,color

x:起始点坐标x;

y:起始点坐标y;

x2:结束点坐标x;

y2:结束点坐标y;

color:画线颜色;
```

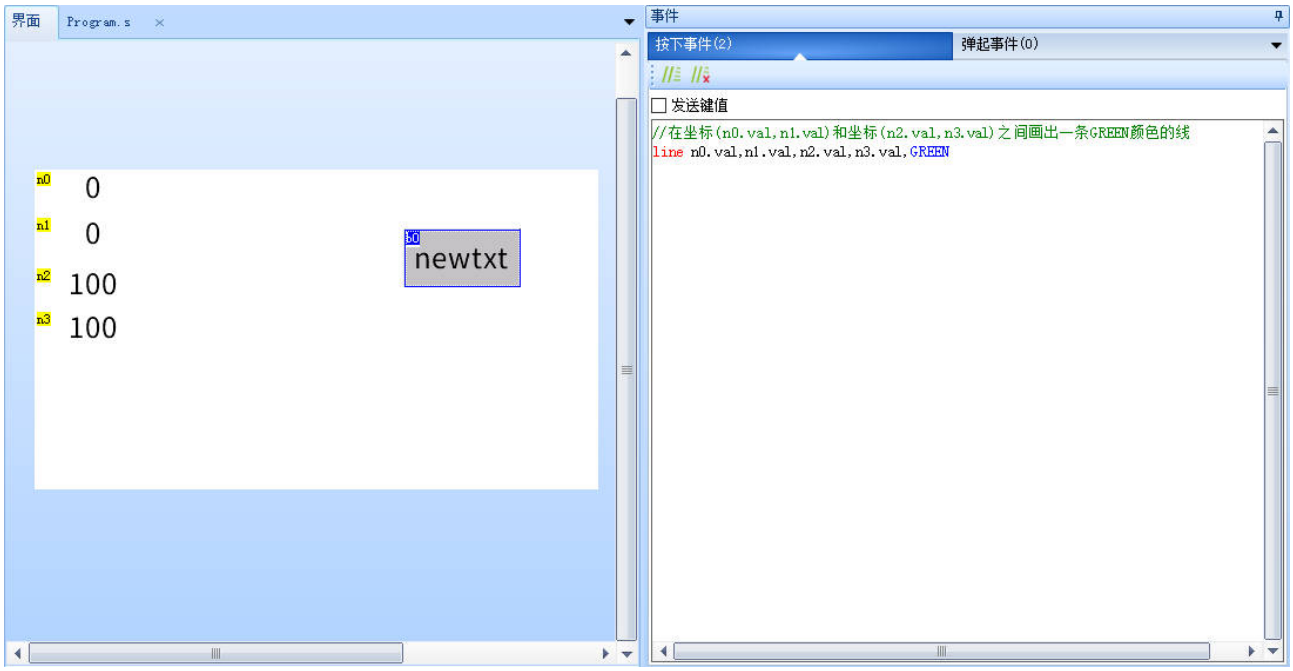
4.6.7.1 line-示例 1

```
//在坐标(0,0)和坐标(100,100)之间画出一条RED颜色的线
line 0,0,100,100,RED
```



4.6.7.2 line-示例 2

```
//在坐标(n0.val,n1.val)和坐标(n2.val,n3.val)之间画出一条GREEN颜色的线
line n0.val,n1.val,n2.val,n3.val,GREEN
```



4.6.7.3 line-c 语言示例

单片机通过串口在坐标 (0,0) 和坐标 (100,100) 之间画出一条 RGB565 颜色为 1024 的线

```
int posX1=0, posY1=0, posX2=100, posY2=100, color=1024;
printf("line %d,%d,%d,%d,%d\xff\xff\xff", posX1, posY1, posX2, posY2, color);
```

注意： 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖

4.6.7.4 line 指令-样例工程下载

演示工程下载链接：

《绘图指令》演示工程下载

《绘图指令演示工程 2》下载

4.6.7.5 line 指令-相关链接

HMI 颜色代号表 (page 740)

4.6.8 draw-画空心矩形

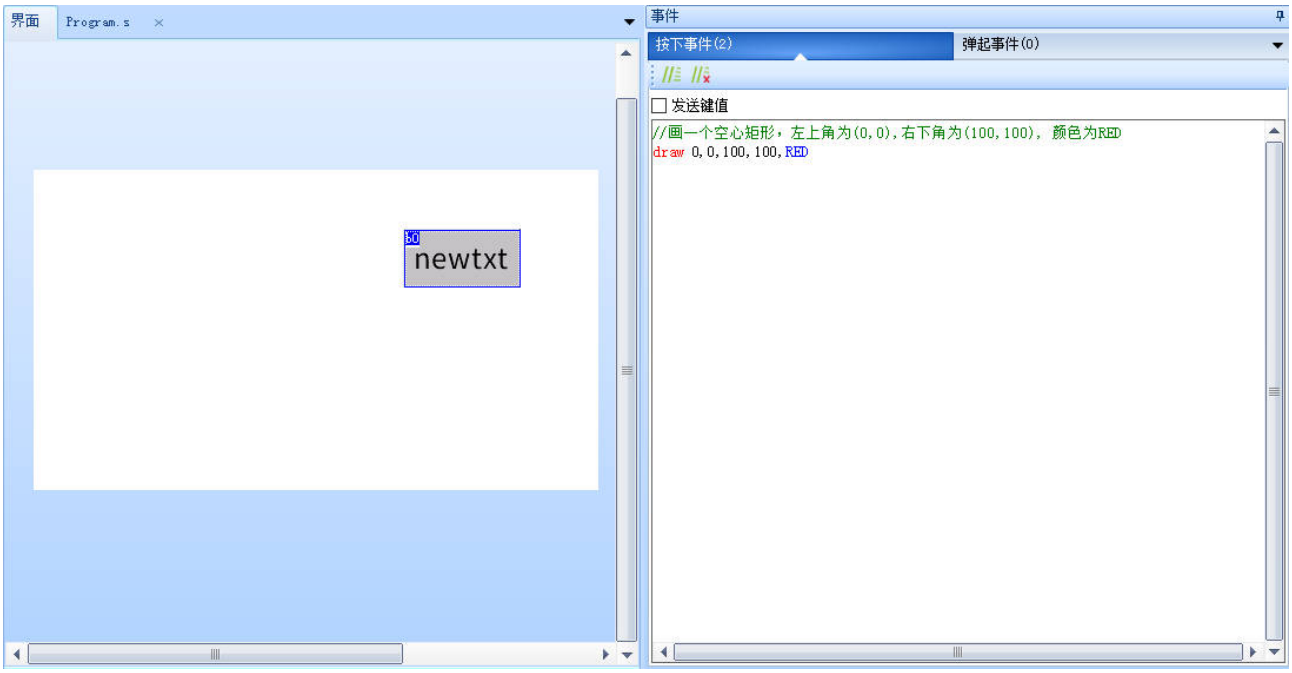
在当前页面进行画一个空心矩形，不支持跨页面画空心矩形，不能写在页面的前初始化事件中

> **注意：** 不推荐使用 GUI 绘图指令, 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖。绘图指令在跳转页面后会丢失。绘图指令和控件有重叠时，当控件刷新时，和控件重叠的部分会丢失。

```
draw x,y,x2,y2,color

x:起始点坐标x ;

y:起始点坐标y;

x2:结束点坐标x ;

y2:结束点坐标y;

color:画线颜色;
```

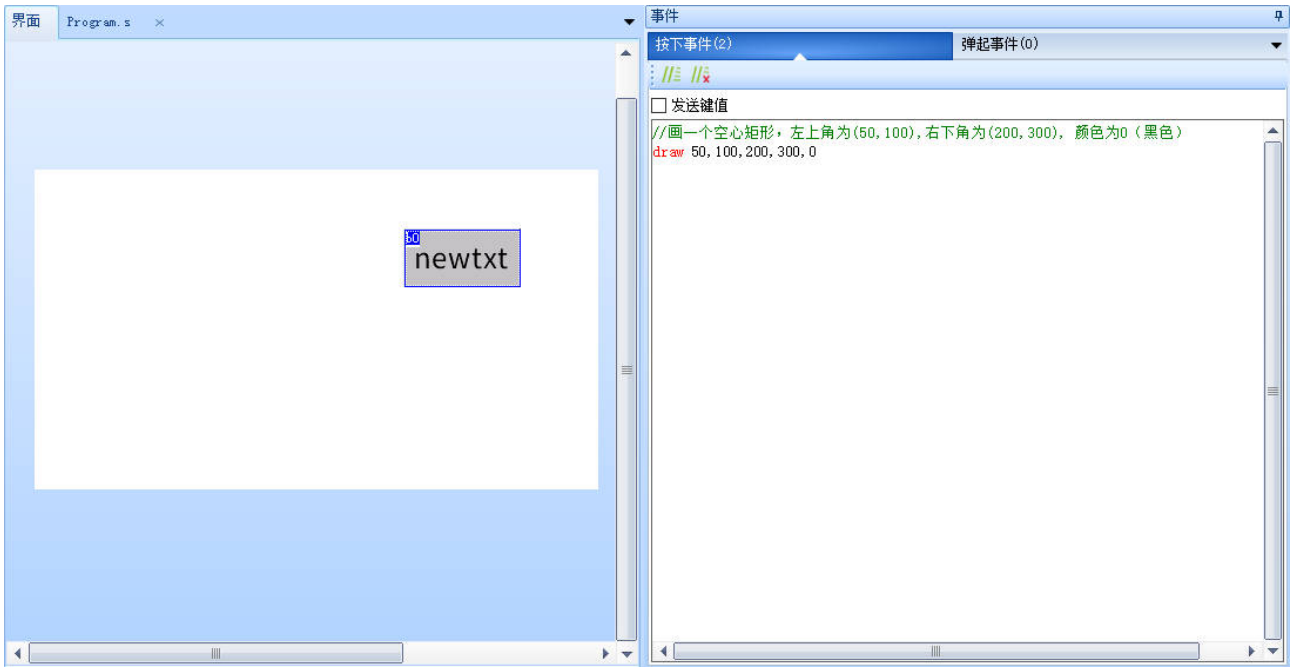
4.6.8.1 draw-示例 1

```
//画一个空心矩形，左上角为(0,0),右下角为(100,100)，颜色为RED
draw 0,0,100,100,RED
```



4.6.8.2 draw-示例 2

```
//画一个空心矩形，左上角为(50,100),右下角为(200,300)，颜色为0（黑色）
draw 50,100,200,300,0
```



4.6.8.3 draw-c 语言示例

单片机通过串口画一个空心矩形，左上角为 (0,0), 右下角为 (100,100),RGB565 颜色值为 1024

```
int posX1=0, posY1=0, posX2=100, posY2=100, color=1024;
printf("draw %d,%d,%d,%d,%d\xff\xff\xff", posX1, posY1, posX2, posY2, color);
```

注意： draw 画出来的是空心矩形，需要填充实心矩形的话请直接使用 fill 区域填充指令。

绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖

4.6.8.4 draw 指令-相关链接

HMI 颜色代号表 (page 740)

4.6.8.5 draw 指令-样例工程下载

演示工程下载链接：

《绘图指令》演示工程下载

《绘图指令演示工程 2》下载

4.6.9 cir-画空心圆

在当前页面画一个空心圆，不支持跨页面画空心圆，不能写在页面的前初始化事件中

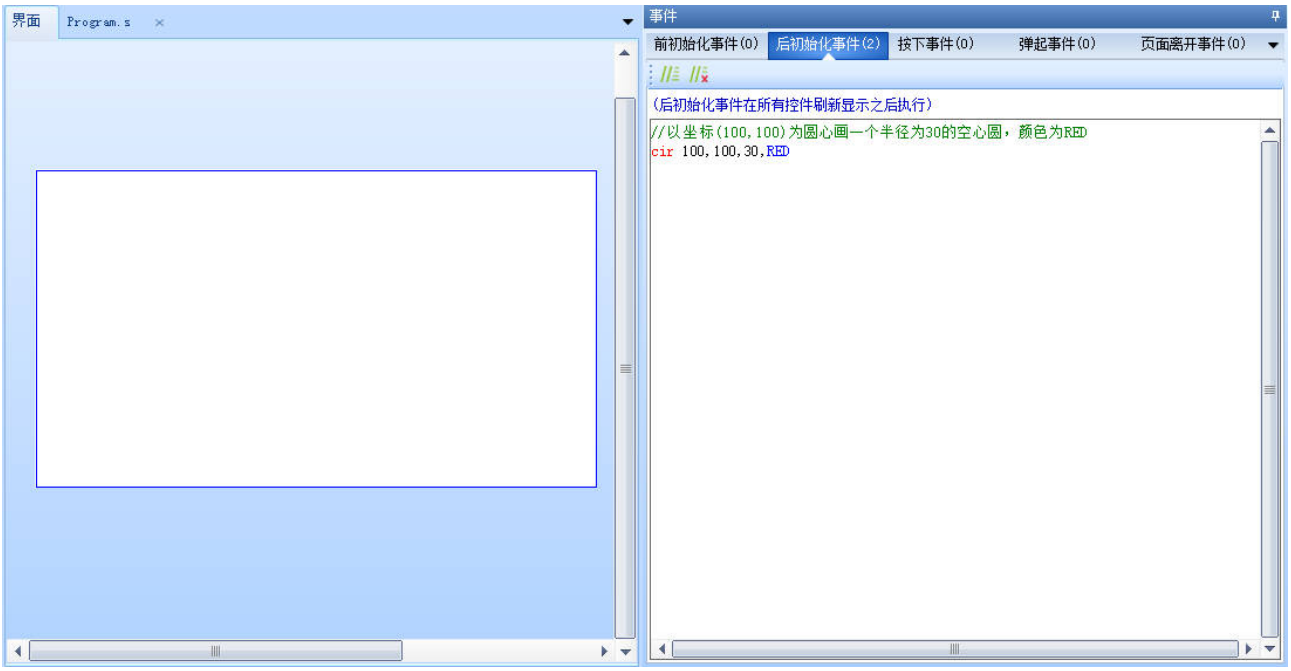
注意： 不推荐使用 GUI 绘图指令, 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖。绘图指令在跳转页面后会丢失。绘图指令和控件有重叠时，当控件刷新时，和控件重叠的部分会丢失。

注意： 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖

```
cir x,y,r,color

x:圆心坐标x

y:圆心坐标y

r:半径

color:画线颜色;
```

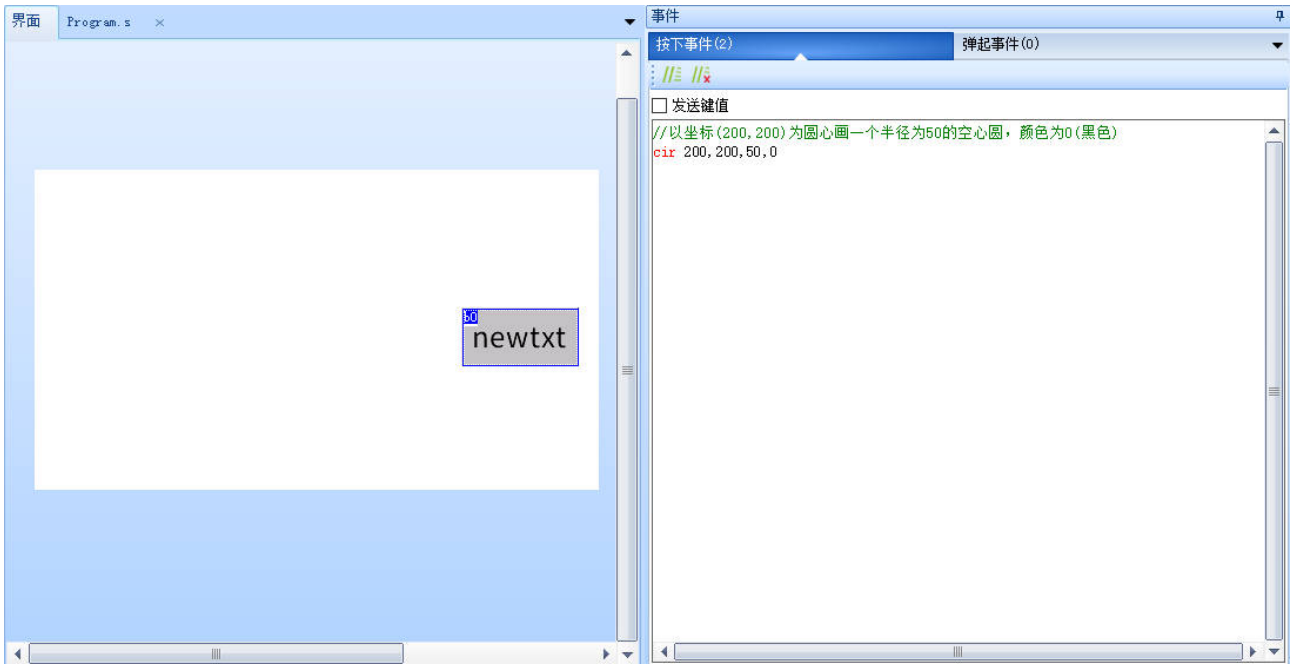
4.6.9.1 cir-示例 1

```
//以坐标(100,100)为圆心画一个半径为30的空心圆，颜色为RED
cir 100,100,30,RED
```



4.6.9.2 cir-示例 2

```
//以坐标(200,200)为圆心画一个半径为50的空心圆，颜色为0(黑色)
cir 200,200,50,0
```



4.6.9.3 cir-c 语言示例

单片机通过串口以坐标 (100,100) 为圆心画一个半径为 30 的空心圆，RGB565 颜色值为 1024

```
int posX=100,posY=100,radius=30,color=1024;
printf("cir %d,%d,%d,%d\xff\xff\xff",posX,posY,radius,color);
```

注意： 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖

4.6.9.4 cir 指令-相关链接

HMI 颜色代号表 (page 740)

4.6.9.5 cir-演示工程下载链接：

《绘图指令》演示工程下载

《绘图指令演示工程 2》下载

4.6.10 cirs-画实心圆

在当前页面画一个实心圆，不支持跨页面画实心圆，不能写在页面的前初始化事件中

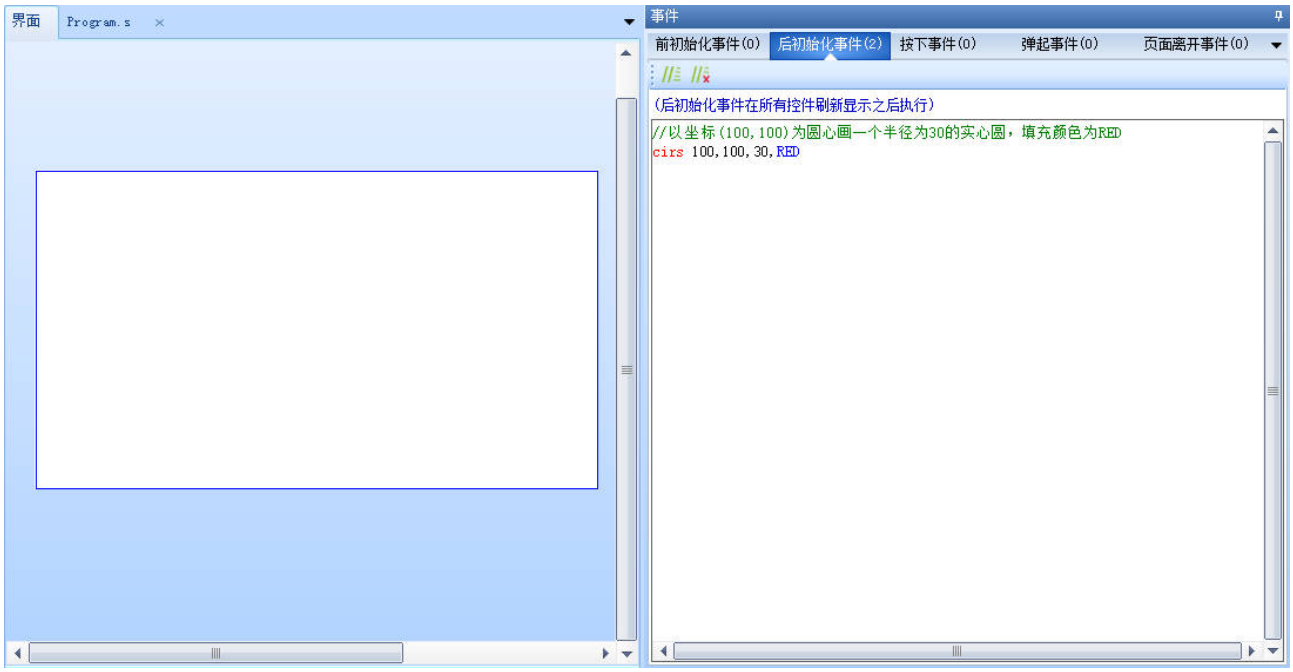
> **注意：** 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖

> **注意：** 不推荐使用 GUI 绘图指令，绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖。绘图指令在跳转页面后会丢失。绘图指令和控件有重叠时，当控件刷新时，和控件重叠的部分会丢失。

```
cirs x,y,r,color

x:圆心坐标x

y:圆心坐标y

r:半径

color:填充颜色;
```

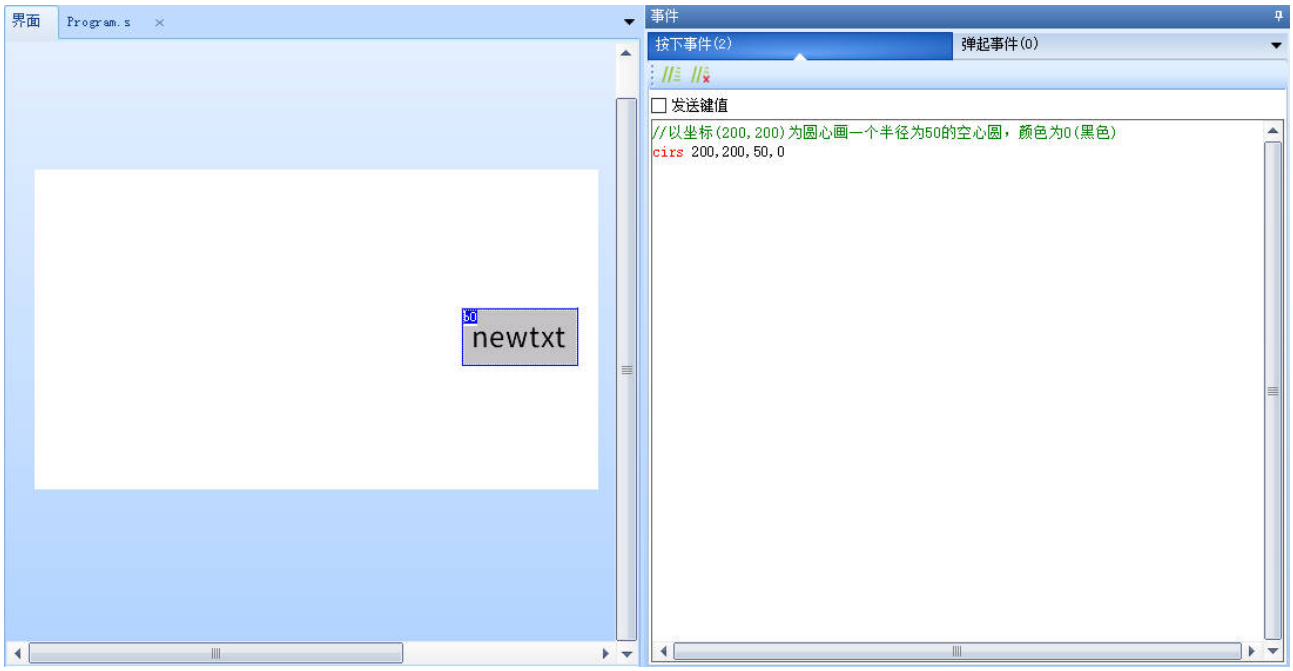
4.6.10.1 cirs-示例 1

```
//以坐标(100,100)为圆心画一个半径为30的实心圆，填充颜色为RED
cirs 100,100,30,RED
```



4.6.10.2 cirs-示例 2

```
//以坐标(200,200)为圆心画一个半径为50的实心圆，颜色为0(黑色)
cir 200,200,50,0
```



4.6.10.3 cirs-c 语言示例

单片机通过串口以坐标 (100,100) 为圆心画一个半径为 30 的实心圆，RGB565 颜色值为 1024

```
int posX=100, posY=100, radius=30, color=1024;
printf("cirs %d,%d,%d,%d\xff\xff\xff", posX, posY, radius, color);
```

注意： 绘图指令不要写在页面的前初始化事件中，否则在页面渲染完成后，将会被页面控件（每个页面 ID 为 0 的控件是与页面名称相同的页面控件）完全覆盖

4.6.10.4 cirs 指令-相关链接

HMI 颜色代号表 (page 740)

4.6.10.5 cirs 指令-样例工程下载

4.6.10.6 cirs 指令-演示工程下载链接：

《绘图指令》演示工程下载

《绘图指令演示工程 2》下载

注意： GUI 绘图指令主要应用在如下场合：当上位界面编辑软件无法实现您的某些特殊显示要求的时候，使用 GUI 指令自己绘图来实现自己想要的显示效果。大多数情况下其实是不需要使用这些绘图指令的，大多数的应用都可以通过界面编辑软件的控件操作来实现。

4.7 HMI 颜色代号表

表 4: 所有代号的书写均为大写

代号	10 进制	所表示的颜色
RED	63488	红色
BLUE	31	蓝色
GRAY	33840	灰色
BLACK	0	黑色
WHITE	65535	白色
GREEN	2016	绿色
BROWN	48192	橙色
YELLOW	65504	黄色

4.8 关于废弃指令的说明

部分指令已经被废弃, 例如 print 指令和 cov 指令, 但是目前依然可以使用, 在上位机中输入时, 废弃指令不会进行智能提示补全

print 指令请用 prints 指令来代替

cov 指令请用 covx 指令来代替

5 系统变量

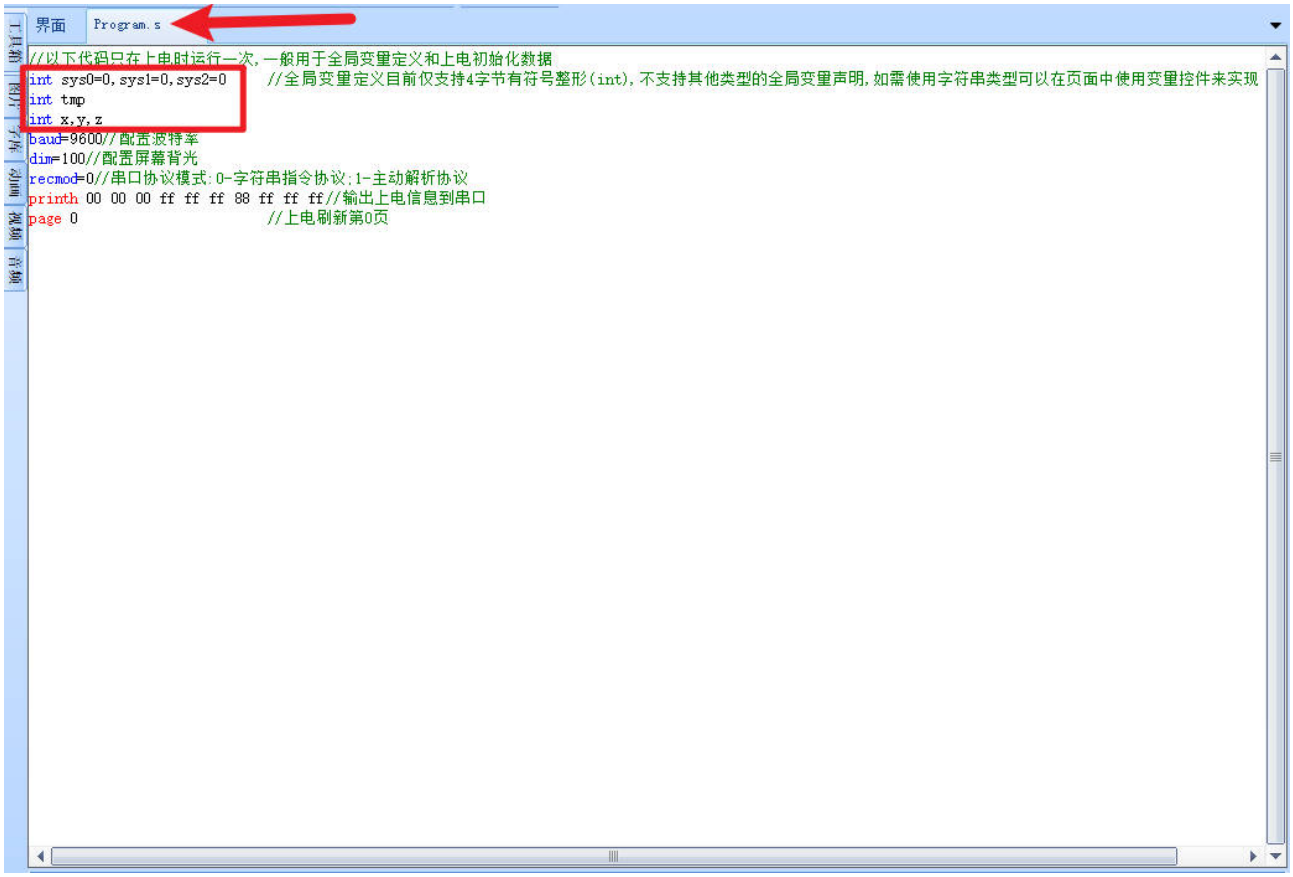
> **注意：** 所有变量名称使用小写字符

5.1 常用变量

5.1.1 sys0-sys2 默认变量

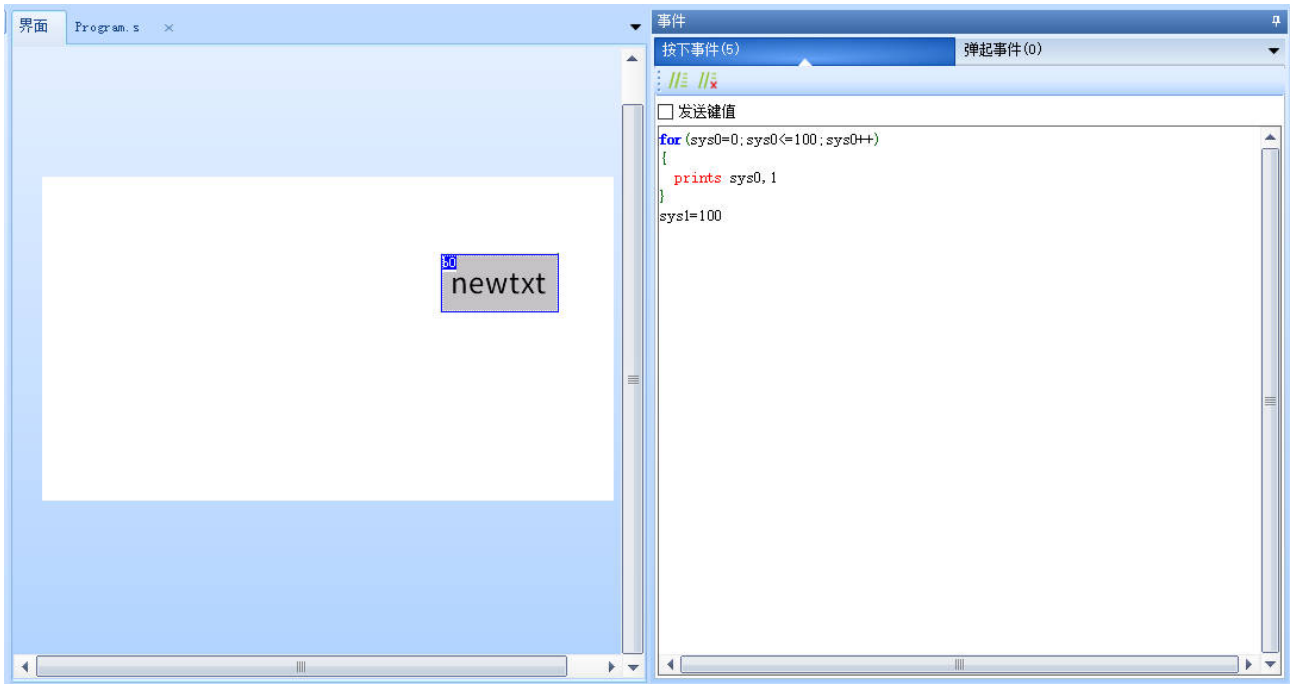
sys0-sys2 是生成工程时默认可以使用变量，您可以模仿 int 定义的写法来创建自己的变量，program.s 中创建的变量均为全局变量。

> **注意：** 在 program.s 中只能使用 int 来定义全局变量，无法定义 char，float，double 等类型，使用 int 定义的语句必须写在 program.s 的最前面



使用示例

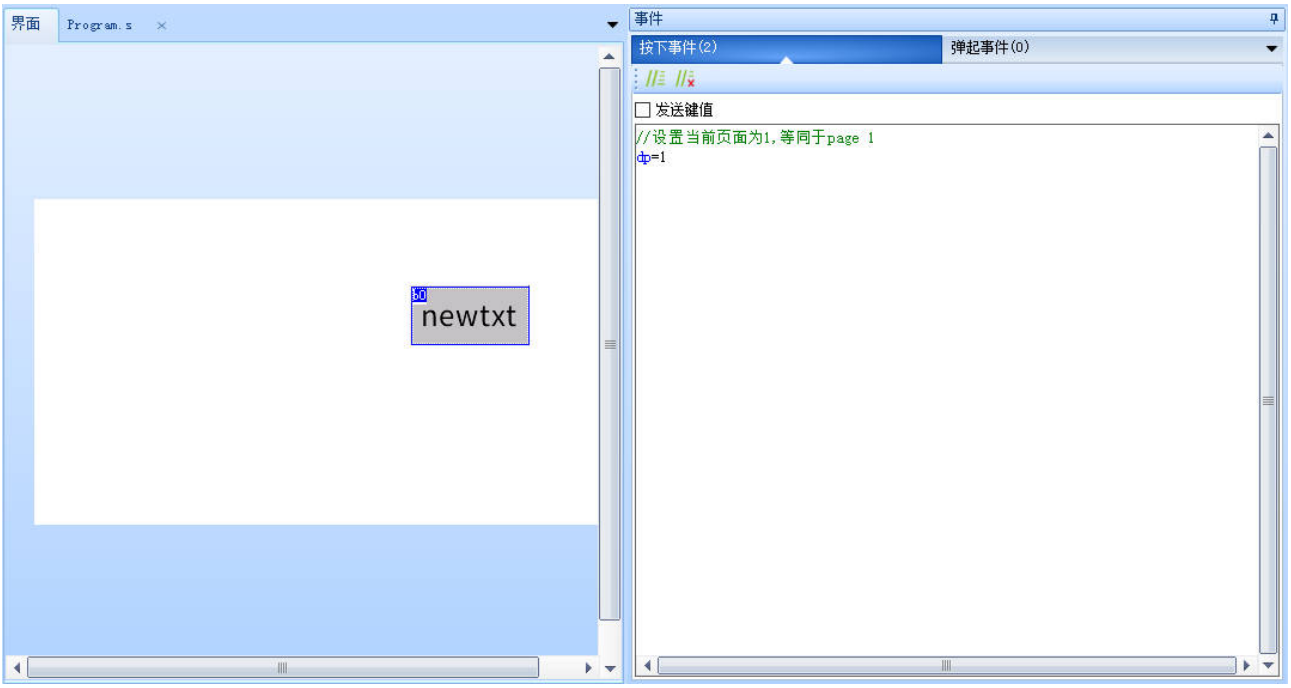
```
for(sys0=0;sys0<=100;sys0++)
{
    prints sys0,1
}

sys1=100
```


界面
Program.s
b0
newtxt
事件
按下事件(5)
弹起事件(0)
发送键值
for (sys0=0;sys0<=100;sys0++)
{
prints sys0,1
}
sys1=100

5.1.2 dp-当前页面 ID

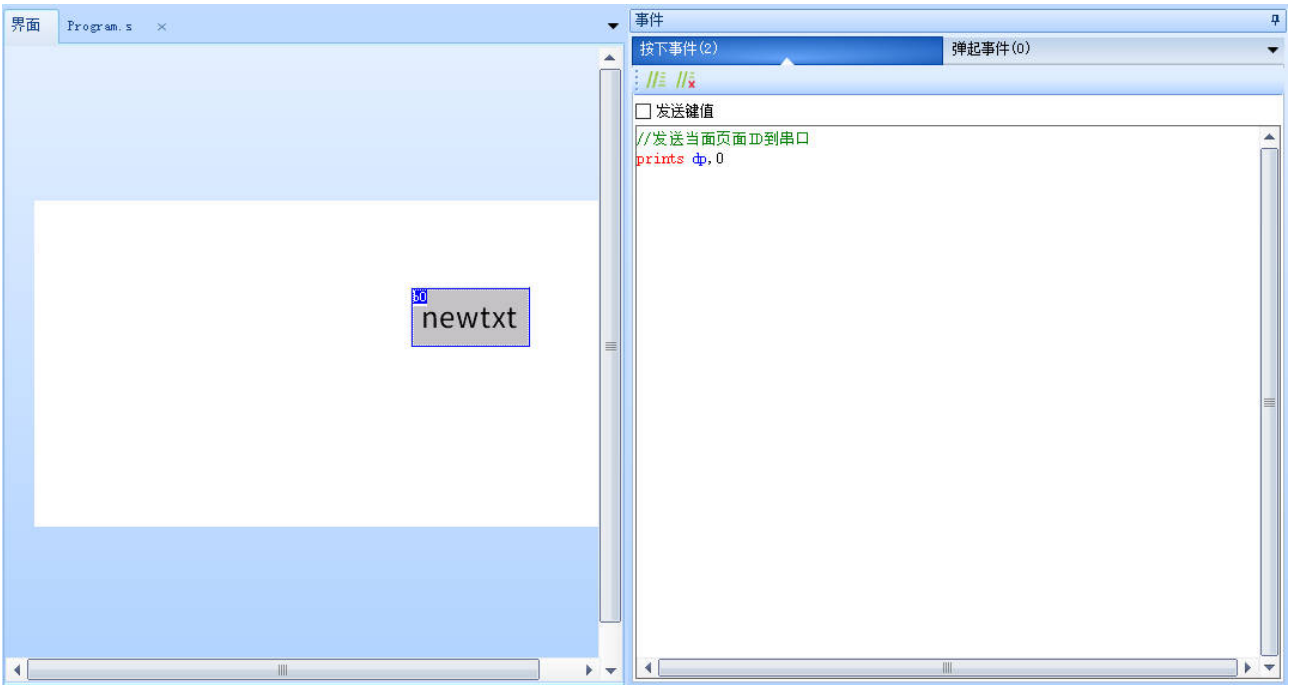
5.1.2.1 dp-示例 1，跳转页面

```
//设置当前页面为1,等同于page 1
dp=1
```



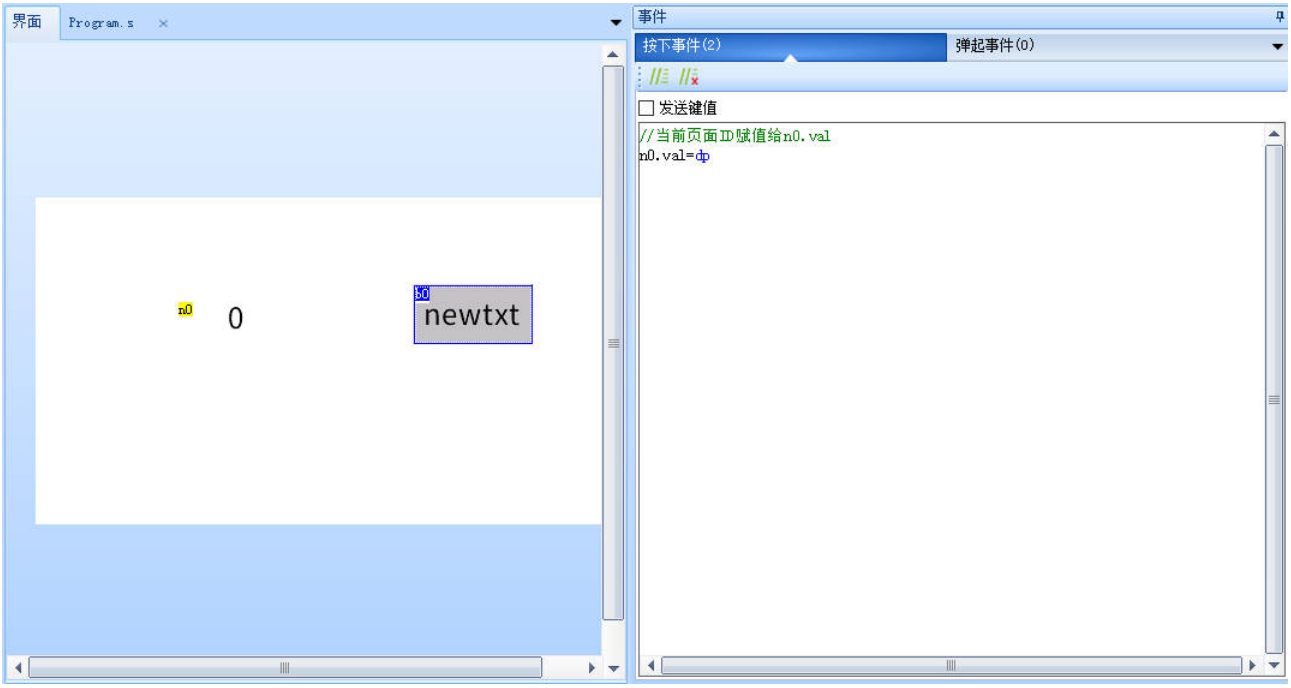
5.1.2.2 dp-示例 2，发送当面页面编号到串口

```
//发送当面页面ID到串口
prints dp,0
```



5.1.2.3 dp-示例 3，获取当前页面编号

```
//当前页面ID赋值给n0.val
n0.val=dp
```

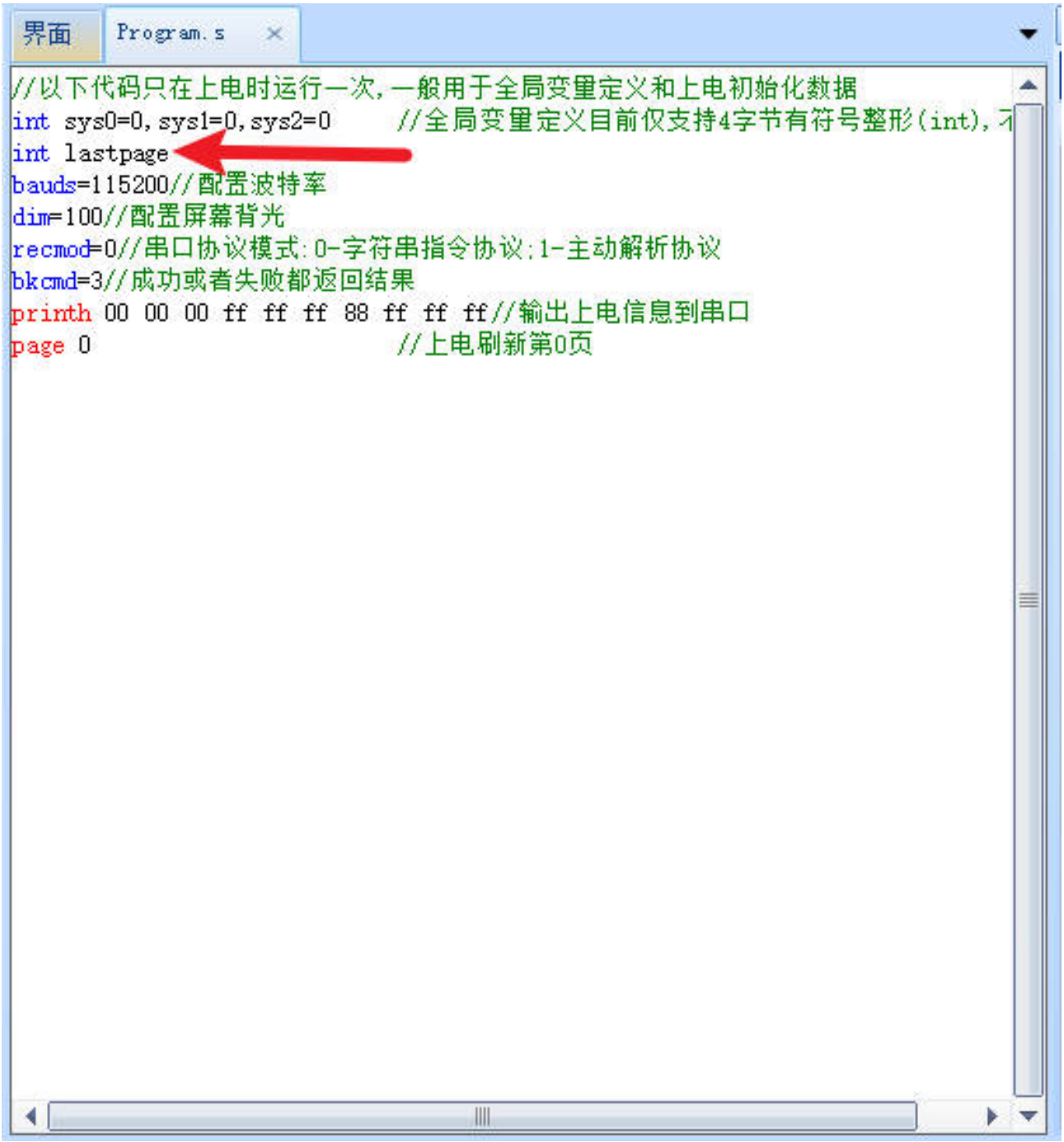


5.1.2.4 dp-示例 4: 返回上一页

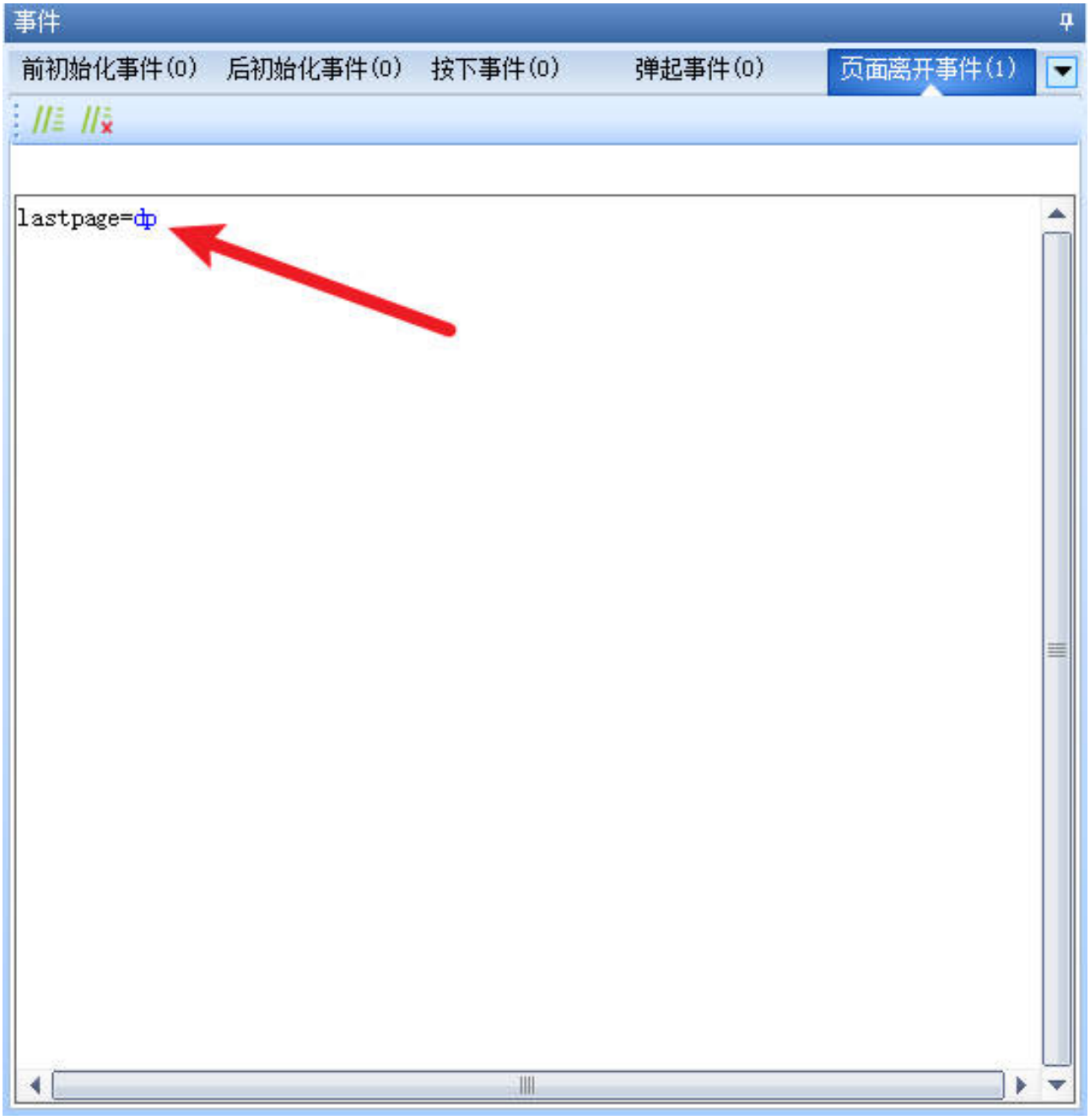
某些情况下, 可能从多个不同的页面跳转到同一个页面, 这时如果在”返回”按钮中直接跳转到某一个页面, 就会造成逻辑混乱

此时可以使用以下方法解决

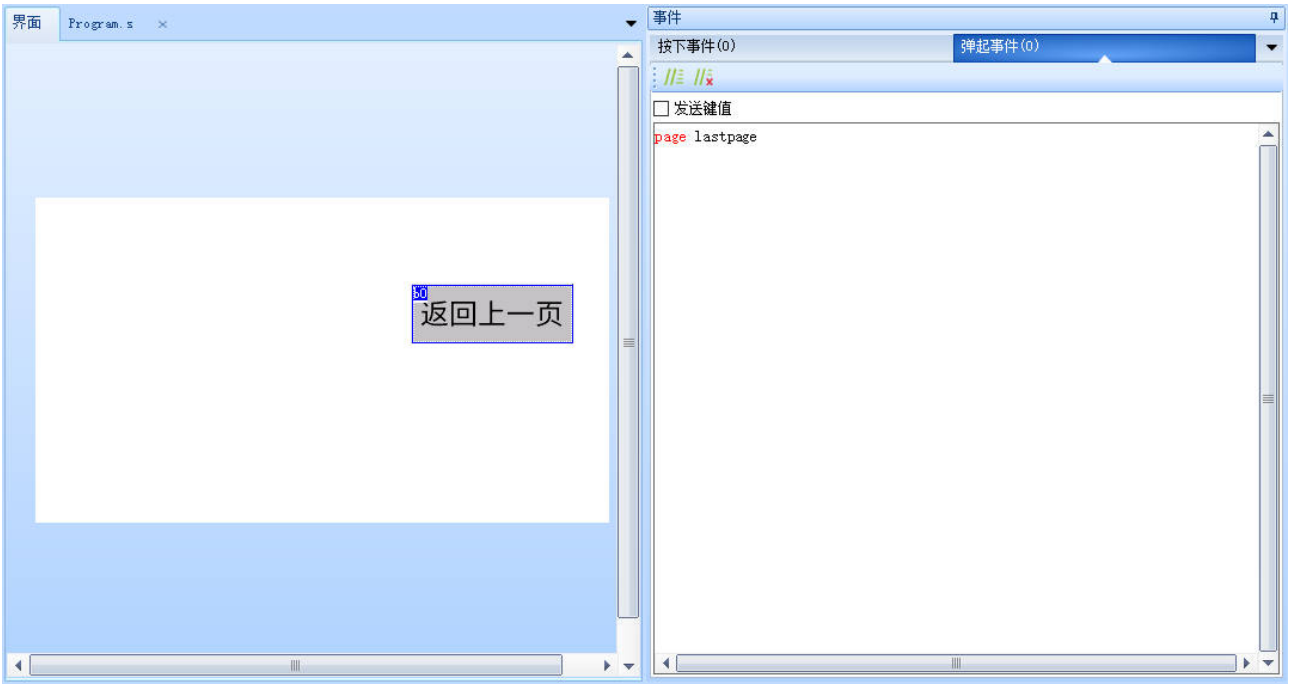
在 program.s 中新建一个 lastpage 变量



在每一个页面的离开事件中添加以下代码，将 dp 的值赋值给 lastpage 变量



添加一个“返回上一页”按钮，在按钮中跳转到 lastpage 所保存的值



5.1.2.5 dp-示例 5: 刷新当前页面

```
//重新进入到当前页面
page dp
```

5.1.2.6 dp-c 语言示例

单片机通过串口跳转到 ID 为 1 的页面

```
//设置当前页面为1,等同于page 1
printf("dp=1\xff\xff\xff");
```

单片机通过串口获取当前页面号

```
//单片机通过串口获取当前页面号
printf("prints dp,0\xff\xff\xff");
```

5.1.2.7 dp-相关链接

page-页面跳转指令 (page 575)

sendme-发送当前页面 ID 号到串口 (page 691)

5.1.2.8 dp-样例工程下载

演示工程下载链接：

《dp-当前页面 id》演示工程下载

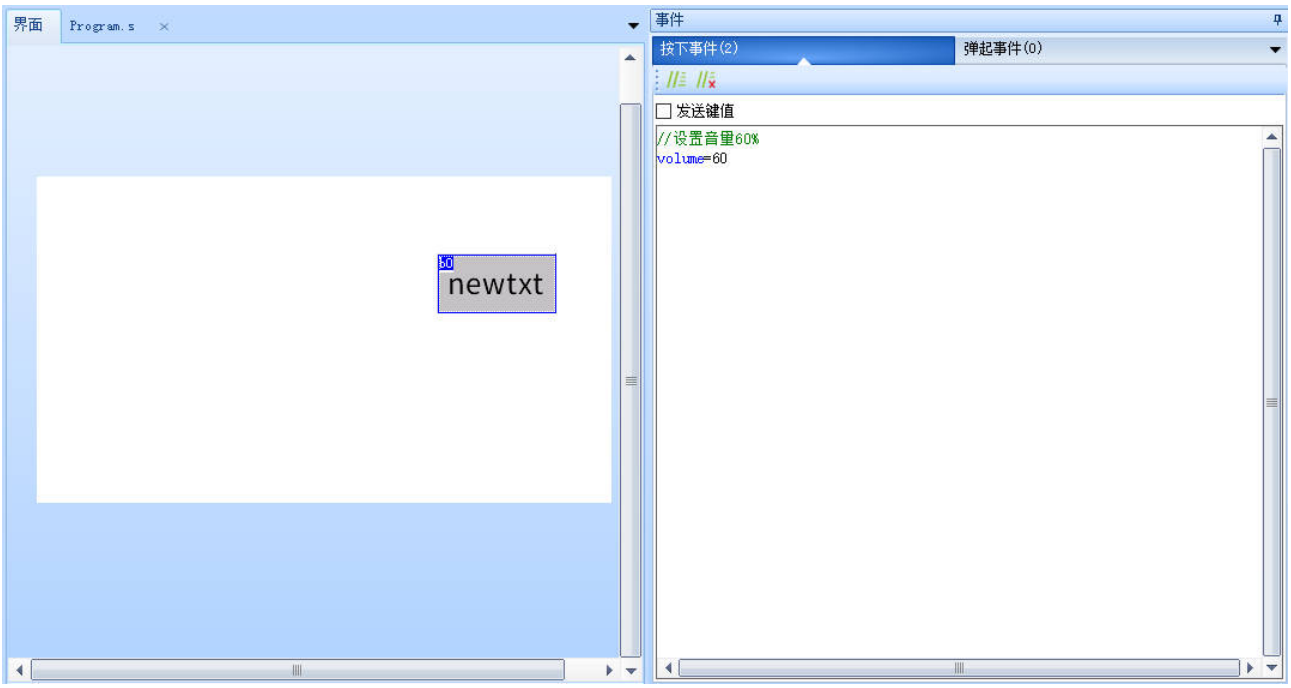
5.1.3 volume-系统音量

仅带喇叭的型号（例如 X3、X5 系列）支持

> **注意：** 设置 volume 时，会判断当前值与 flash 中保存的值是否一致，如果一致则不写入，如果不一致，则会写入 flash 中保存，因此修改 volume 值会消耗 flash 寿命

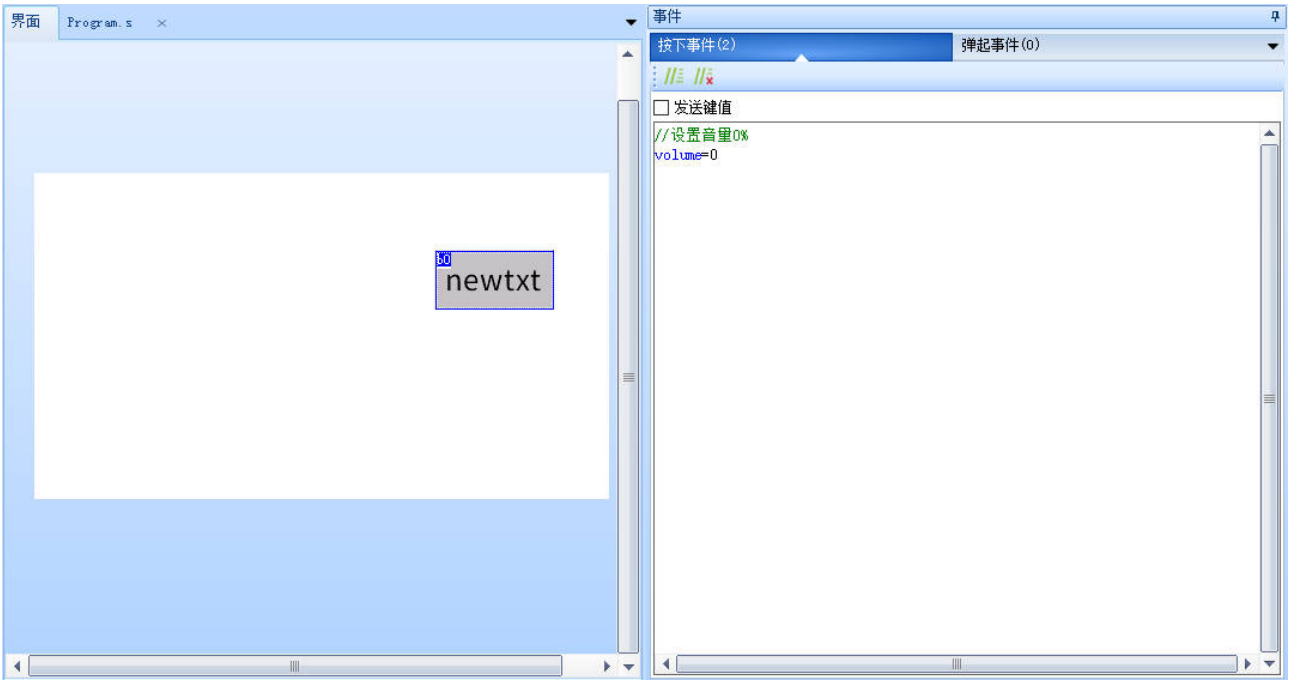
5.1.3.1 volume-示例 1

```
//设置音量60%
volume=60
```



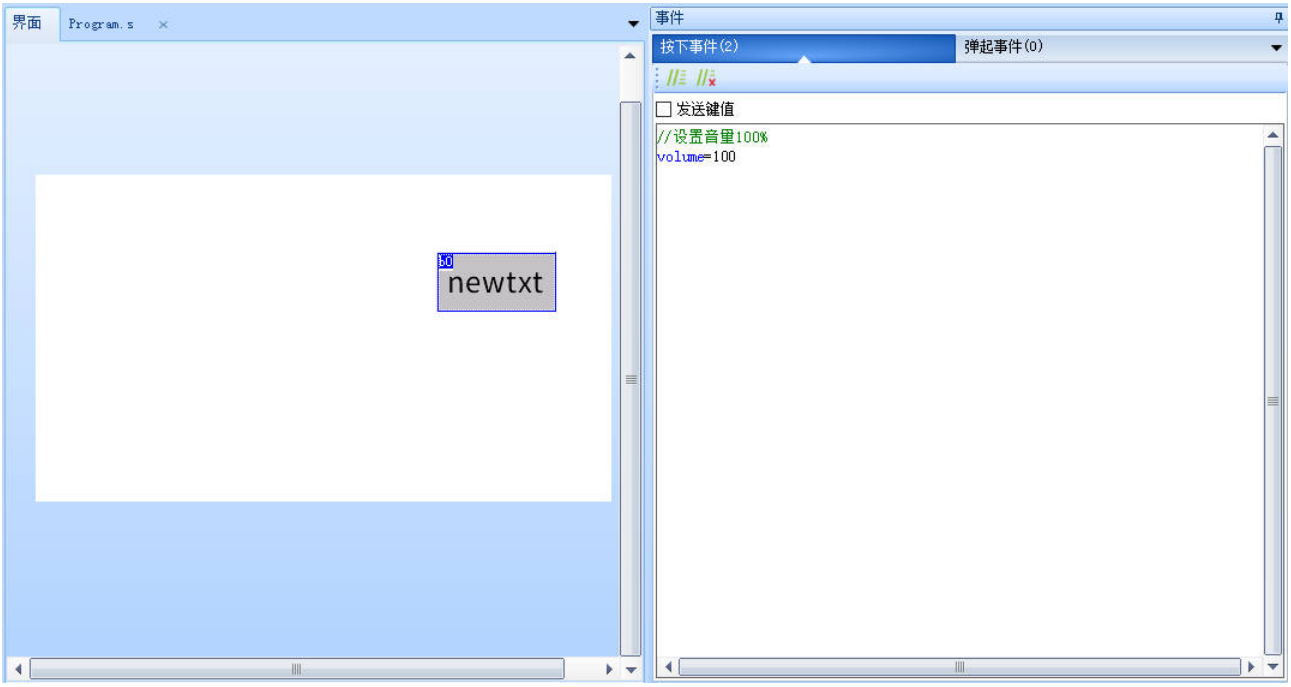
5.1.3.2 volume-示例 2

```
//设置音量0%
volume=0
```



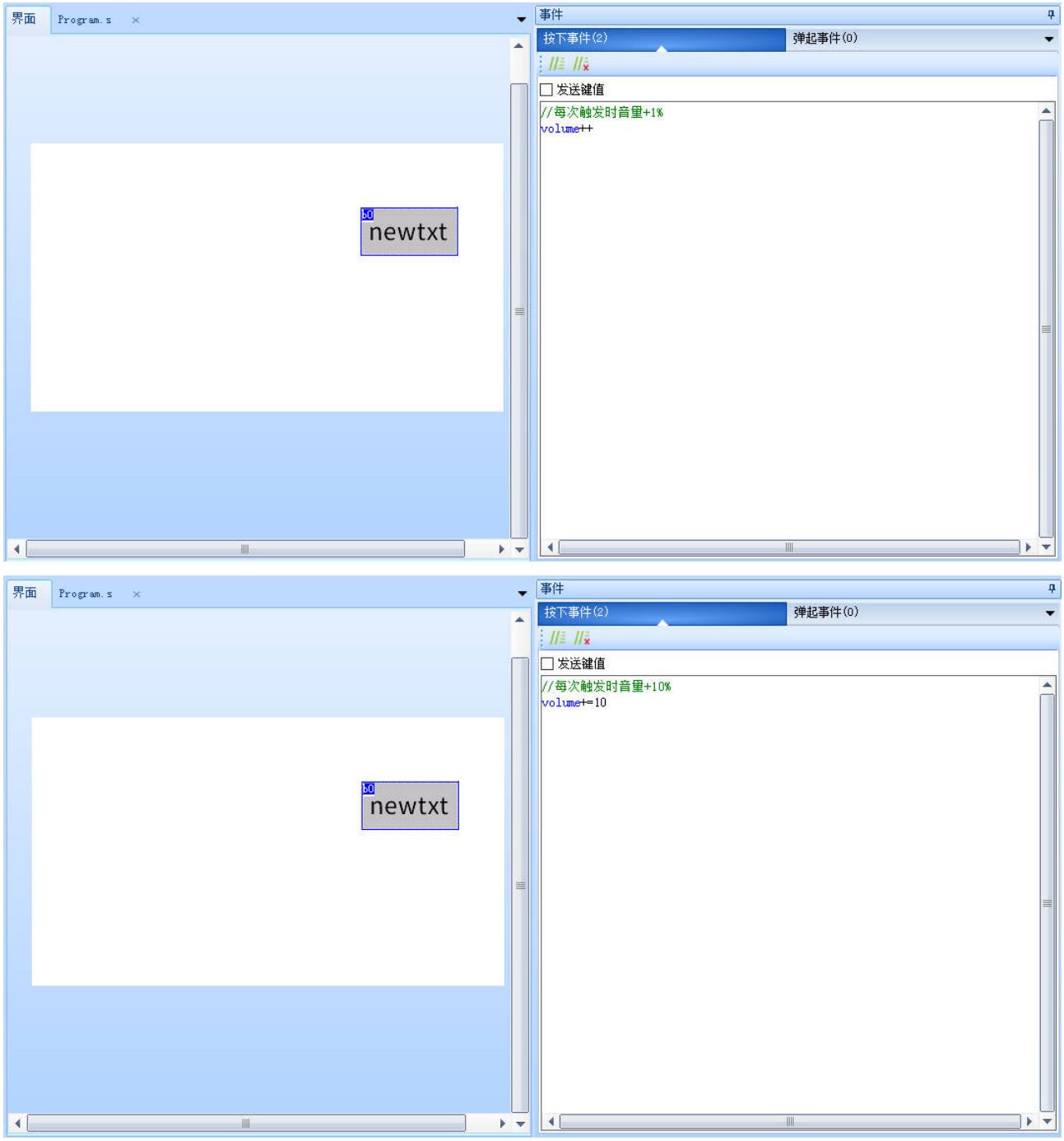
5.1.3.3 volume-示例 3

```
//设置音量100%
volume=100
```



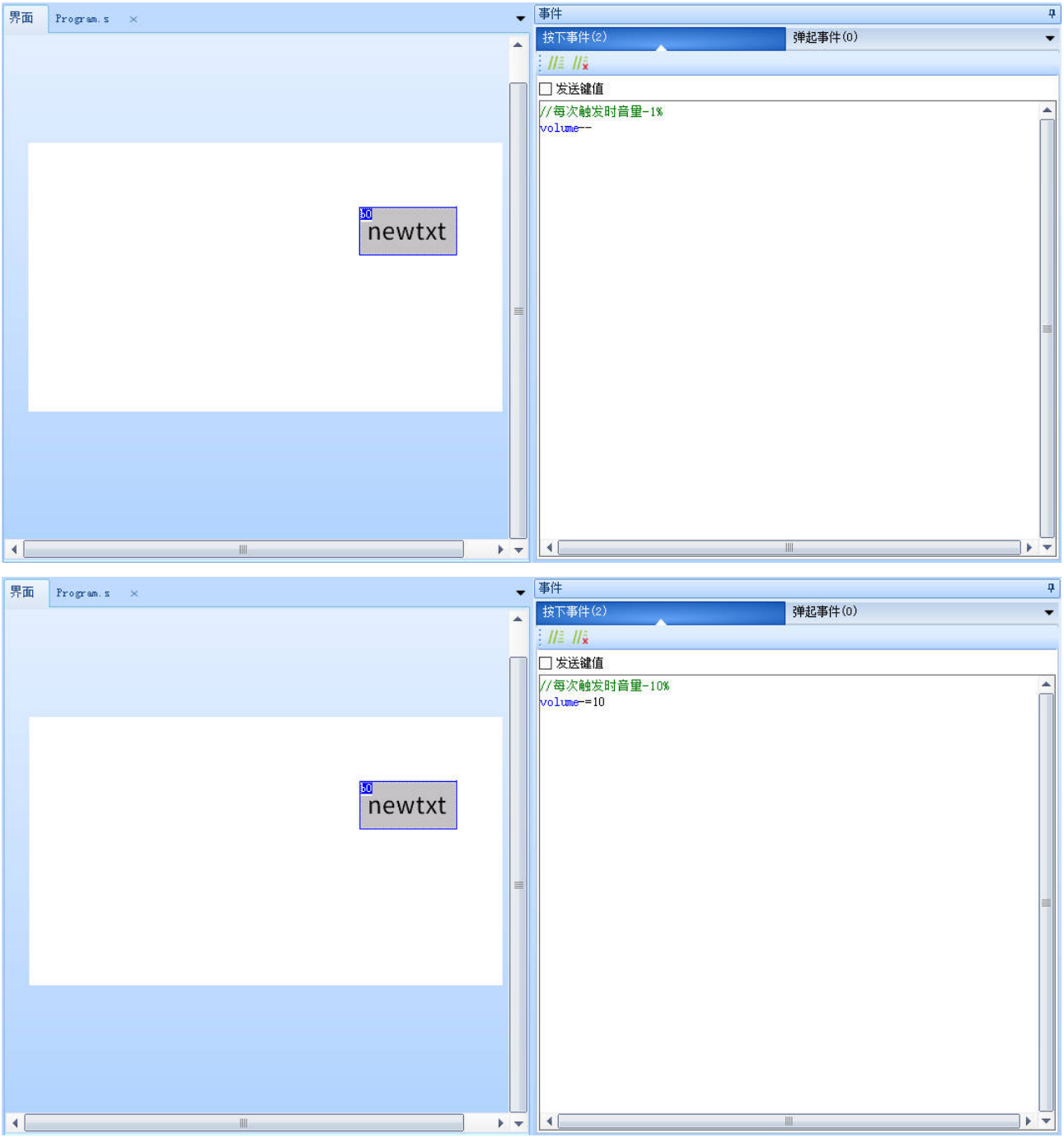
5.1.3.4 volume-示例 4

```
//每次触发时音量+1%
volume++

//每次触发时音量+10%
volume+=10
```



5.1.3.5 volume-示例 5

```
//每次触发时音量-1%
volume--

//每次触发时音量-10%
volume-=10
```



注意： 仅 x5 x3 系列才支持，音量设置范围为 0-100，每次设置会自动保存，断电后再开机依然有效。
音量设置过大导致供电不足可能会导致黑屏、重启等问题，此时应增大供电电流

参考

串口屏开机时死机/不断的闪烁/不断重启 (page 1366)

外部供电接法 (page 1262)

c 语言示例：单片机通过串口控制串口屏音量

```
//音量设置为100%
printf("volume=100\xff\xff\xff");
```

5.1.3.6 volume-相关链接

play-音频播放 (page 659)

5.1.3.7 volume-样例工程下载

演示工程下载链接：

《volume-系统音量》演示工程下载

5.1.4 dims-上电默认背光亮度值

dims 范围：0-100

dims 掉电后保存，下次上电后继续有效

> **注意：** 通常配置在 program.s 文件中，请写在 page 指令的前面，page 指令后面的代码是不会执行的

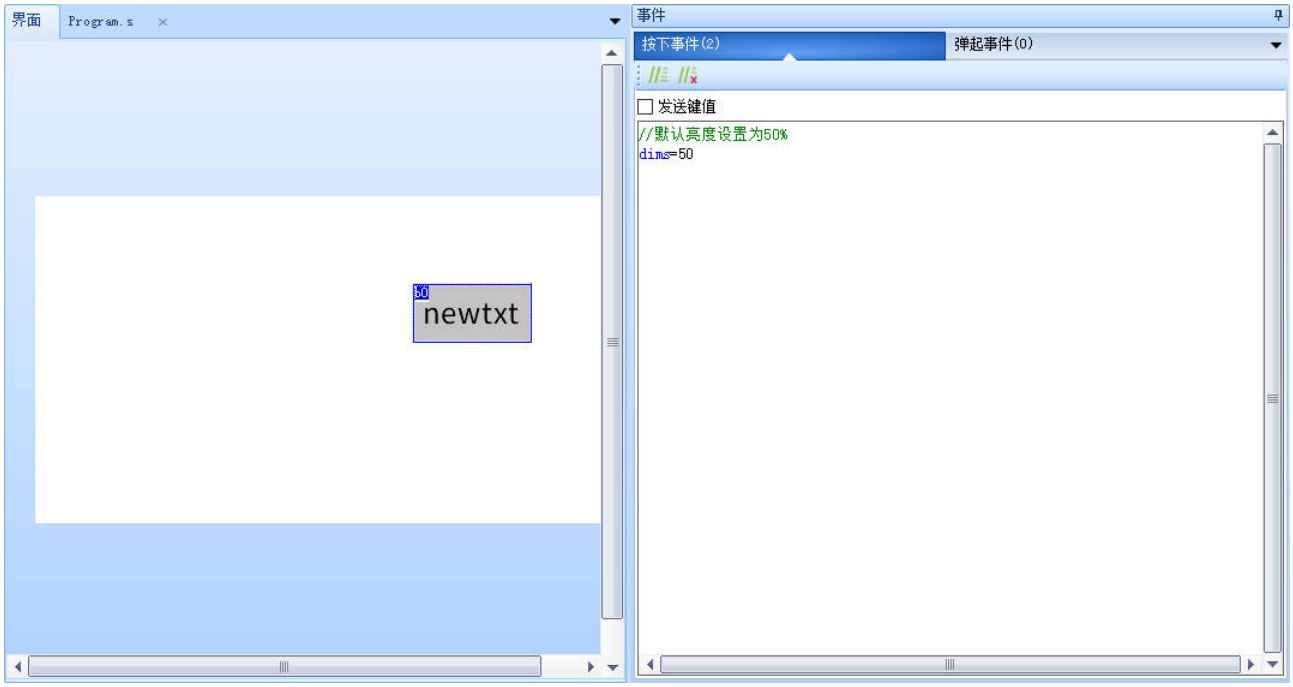
> **注意：** 设置 dims 时，会判断当前值与 flash 中保存的值是否一致，如果一致则不写入，如果不一致，则会写入 flash 中保存，因此修改 dims 值会消耗 flash 寿命

> **注意：** dim 和 dims 区别
>
> dim：不会自动关机保存
>
> dims：会自动保存，下次开机时还是此亮度值

> **注意：** 不建议新手操作 dims 时将 dims 设置为 0，因为 dims 属性是关机保存的，可能存在将 dims 设置为 0 后，结果重启后发现屏幕不亮，以为屏幕坏了，售后返修发现是亮度设置问题

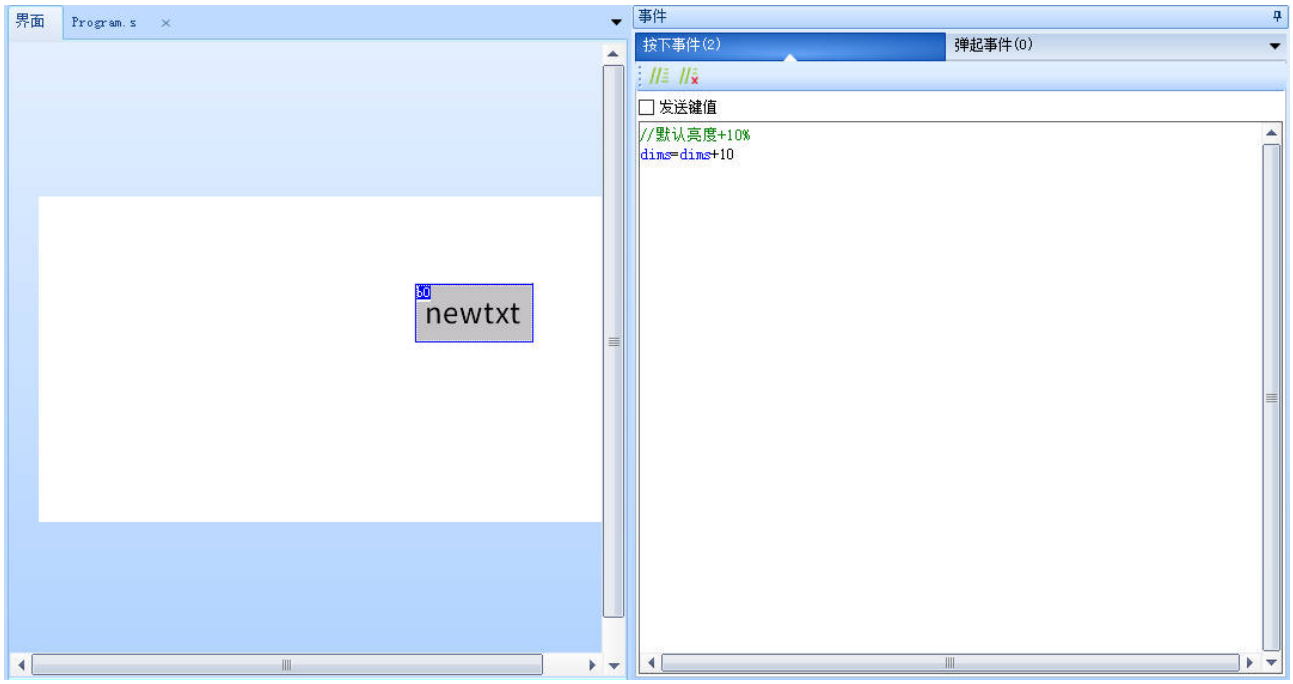
5.1.4.1 dims-示例 1

```
//默认亮度设置为50%
dims=50
```



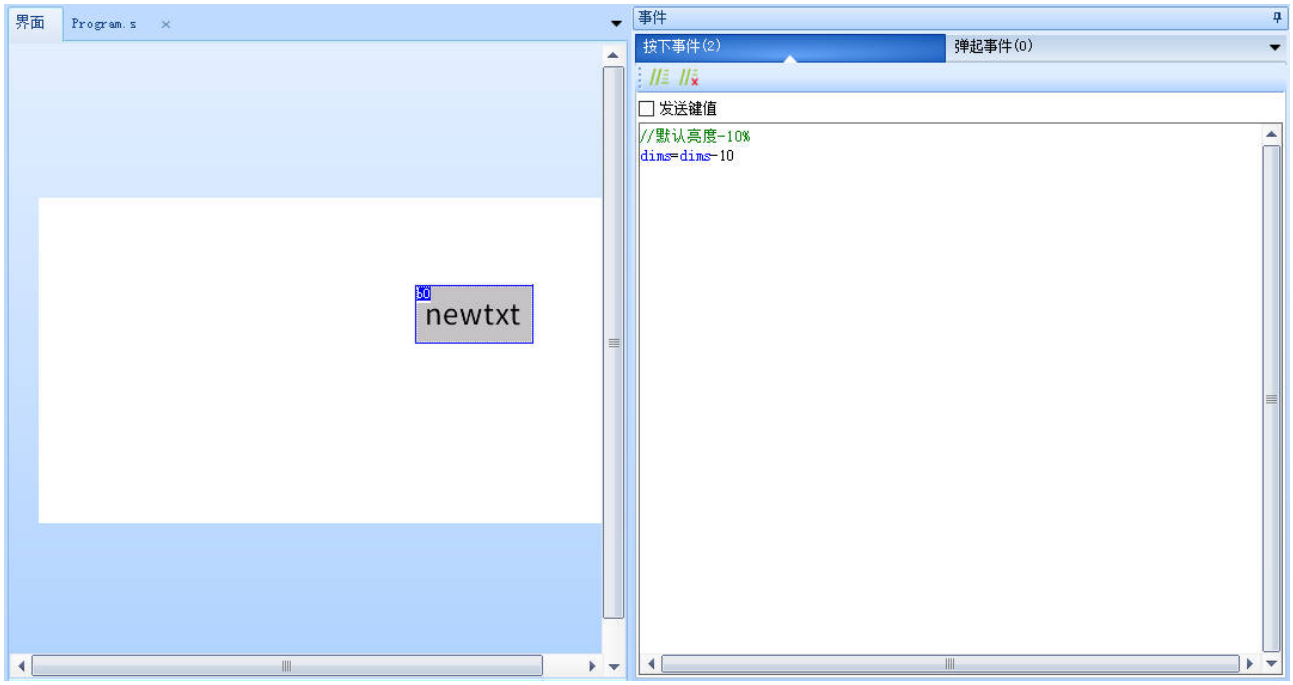
5.1.4.2 dims-示例 2

```
//默认亮度+10%
dims=dims+10
```



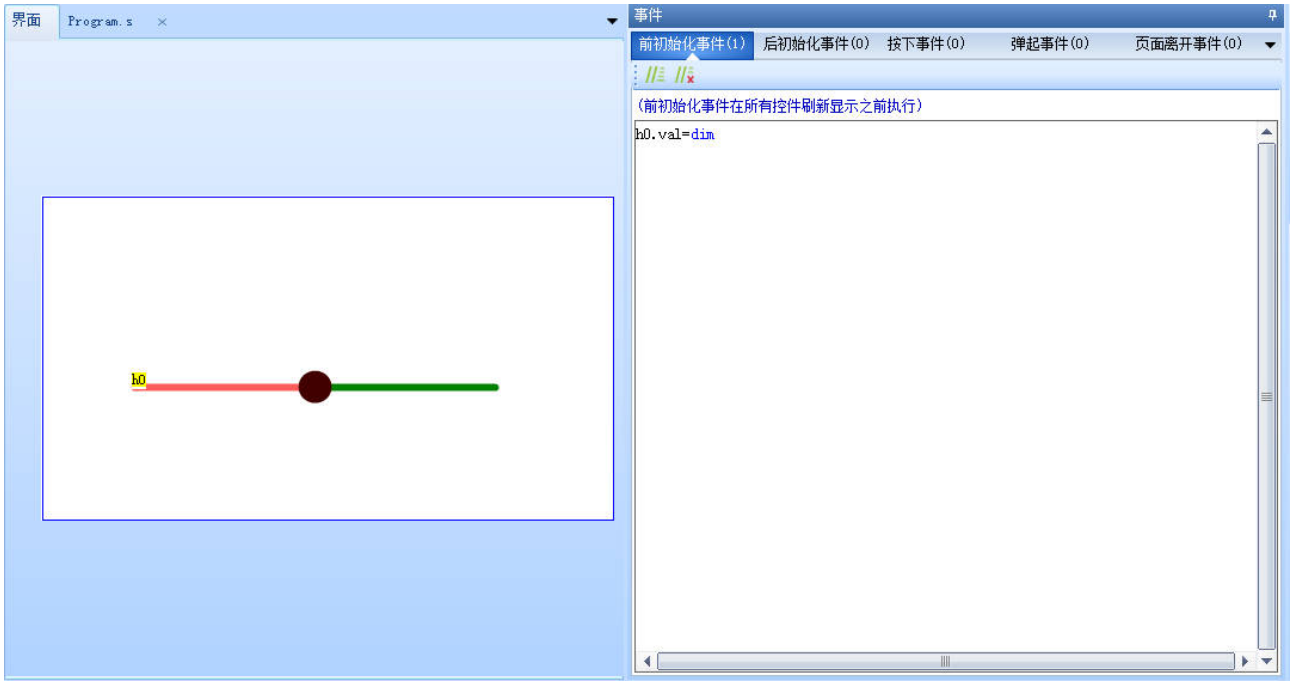
5.1.4.3 dims-示例 3

```
//默认亮度-10%
dims=dims-10
```

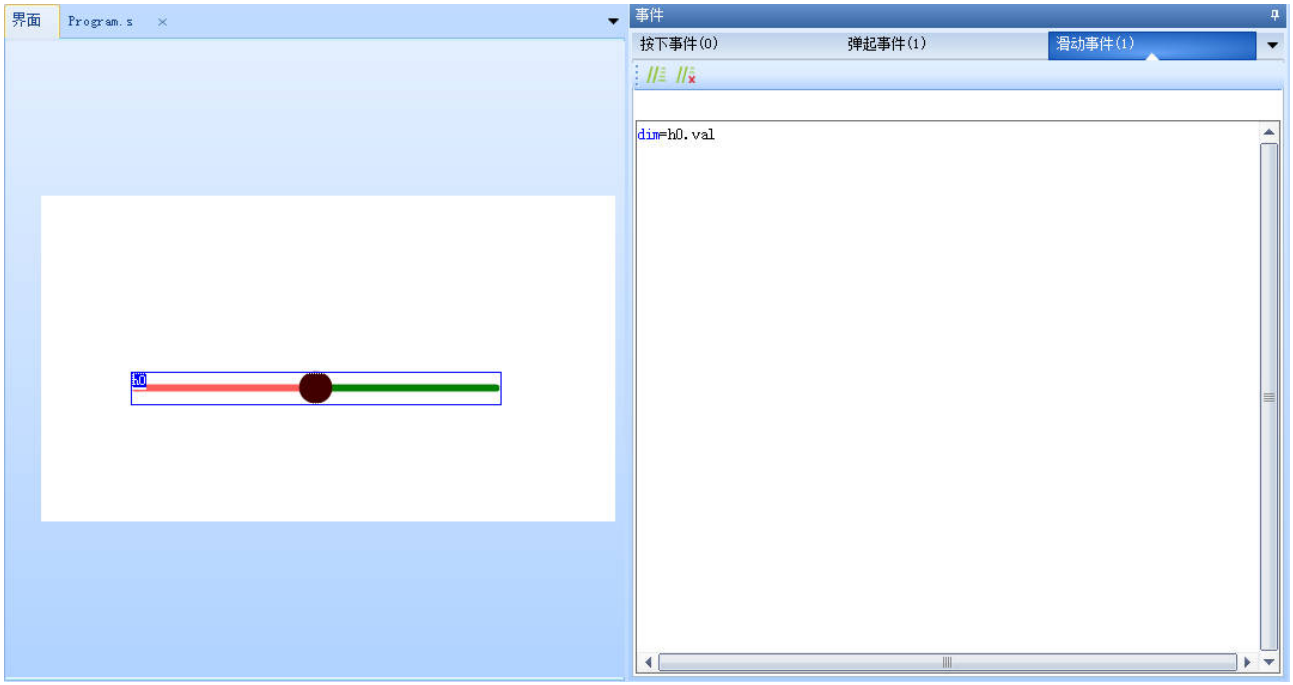


5.1.4.4 dims-示例：通过滑块设置亮度

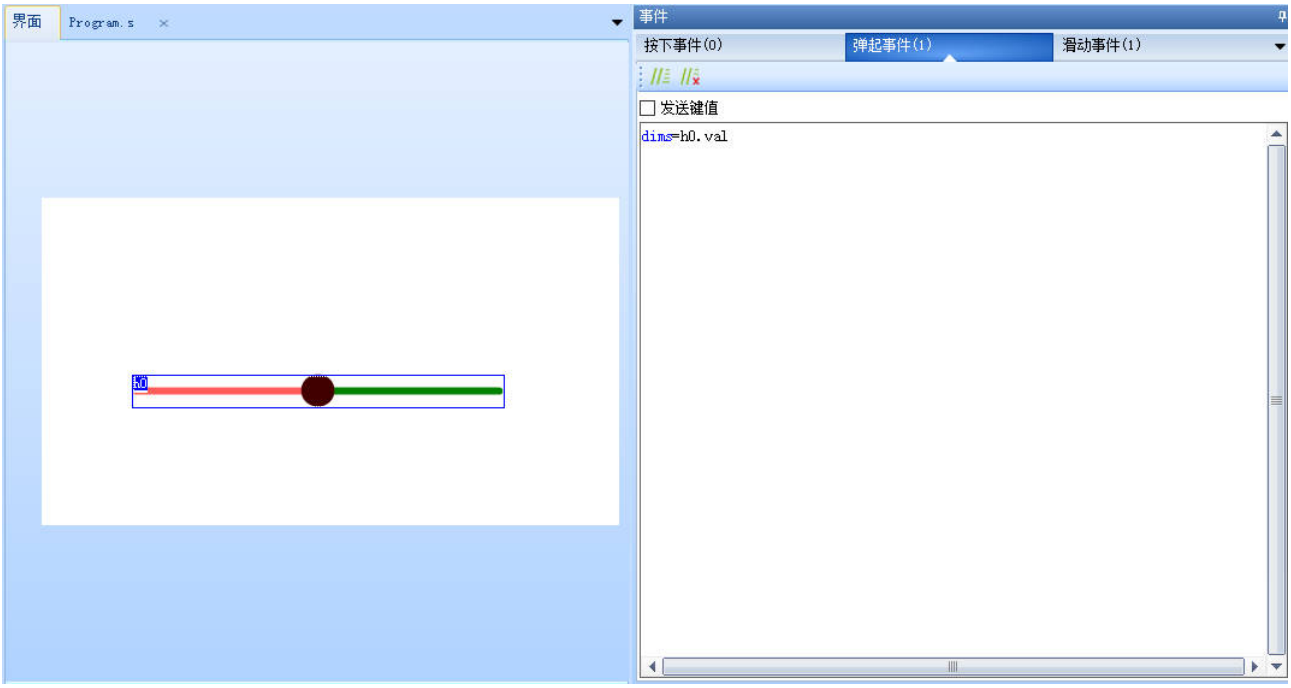
在前初始化事件中将 dim 赋值给进度条



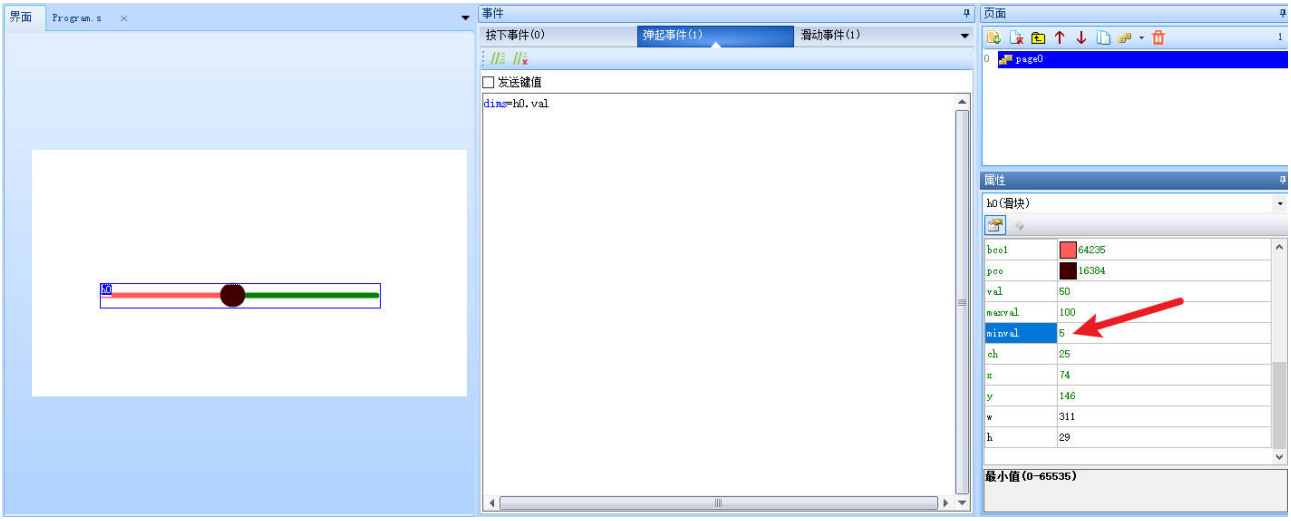
在滑动事件中将进度值赋值给 dim



在弹起事件中将进度值赋值给 dims



进度值的最小值建议设置为 5，避免完全黑屏后没有办法调回来



5.1.4.5 dims-c 语言示例

单片机通过串口控制串口屏上电默认背光亮度值

```
//默认亮度设置为100%
printf("dims=100\xff\xff\xff");
```

5.1.4.6 dims-相关链接

如何配置亮度, 主动解析, 波特率 (page 1306)
滑块控件修改亮度 (page 466)
dim 和 *dims* 的区别 (page 1330)

5.1.4.7 dims-样例工程下载

演示工程下载链接:
《dim 和 dims-串口屏亮度》演示工程下载

5.1.5 dim-当前背光亮度值

亮度范围：0-100(本次修改，重启后丢失)

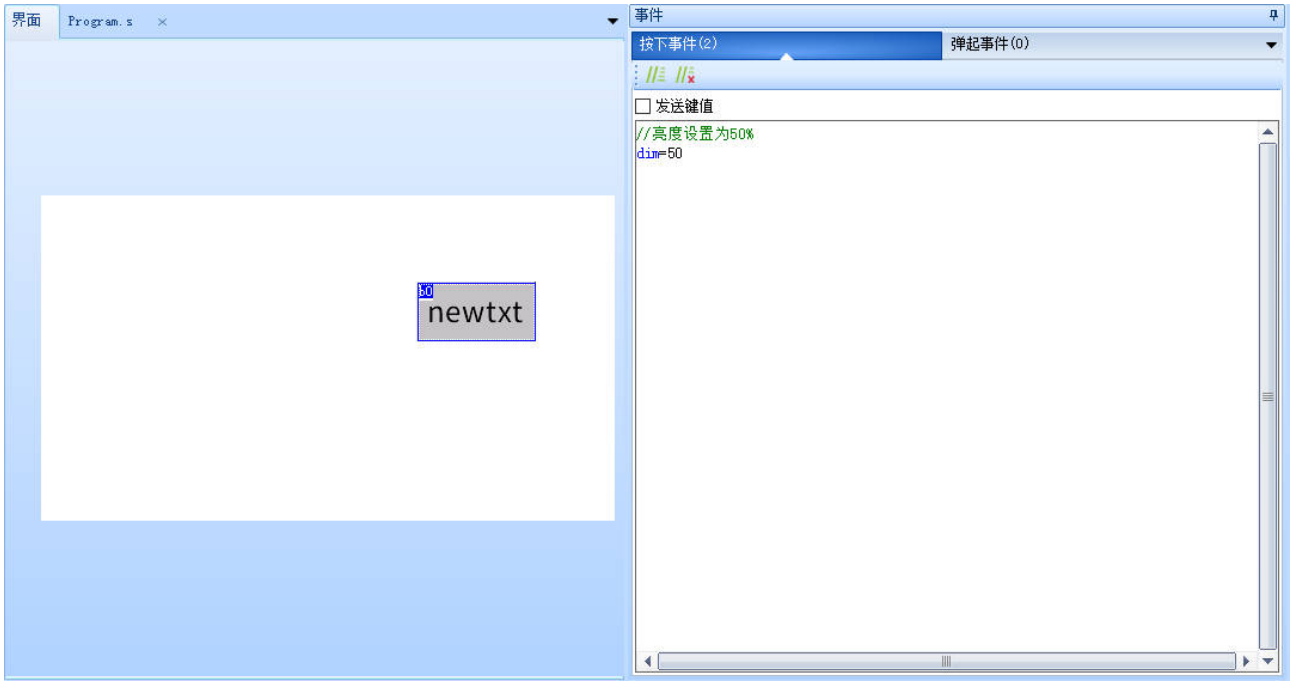
注意： 通常配置在 program.s 文件中，请写在 page 指令的前面，page 指令后面的代码是不会执行的

注意： dim 不会写入 flash 中保存，因此修改 dim 值不会消耗 flash 寿命

注意： dim 和 dims 区别 dim：不会自动关机保存 dims：会自动保存，下次开机时还是此亮度值, 每次修改为不同的值会消耗 flash 寿命

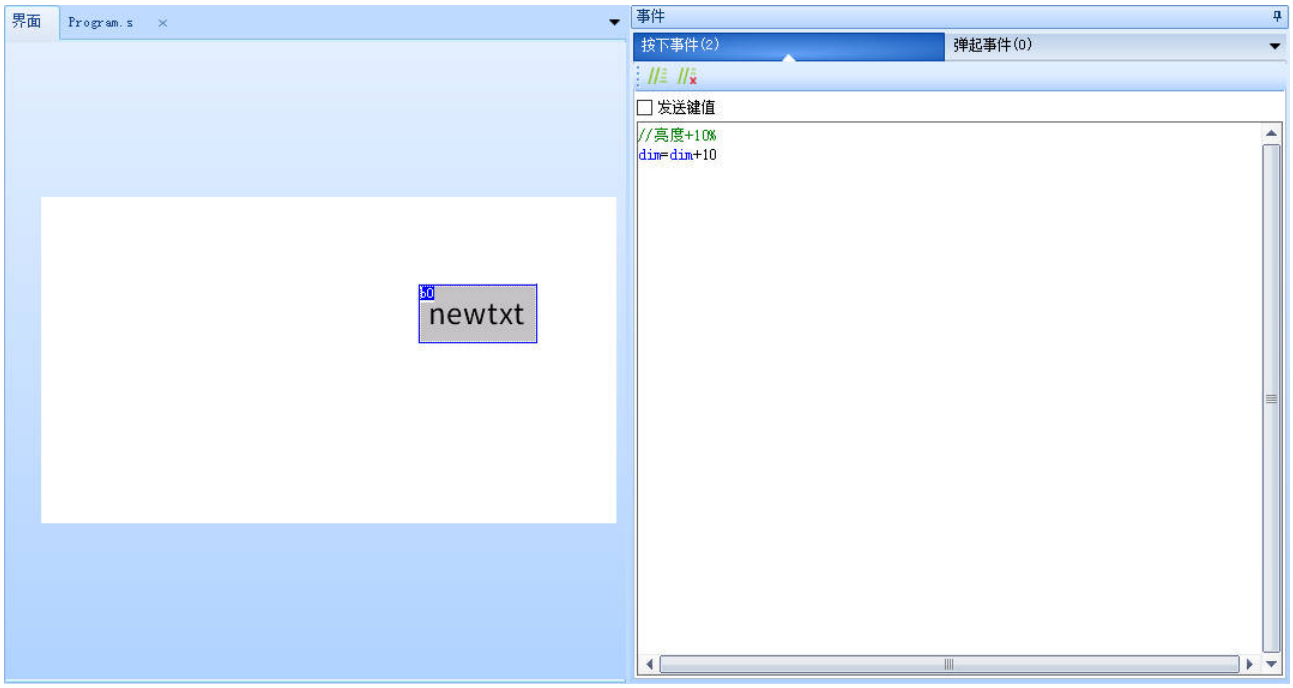
5.1.5.1 dim-示例 1

```
//亮度设置为50%
dim=50
```



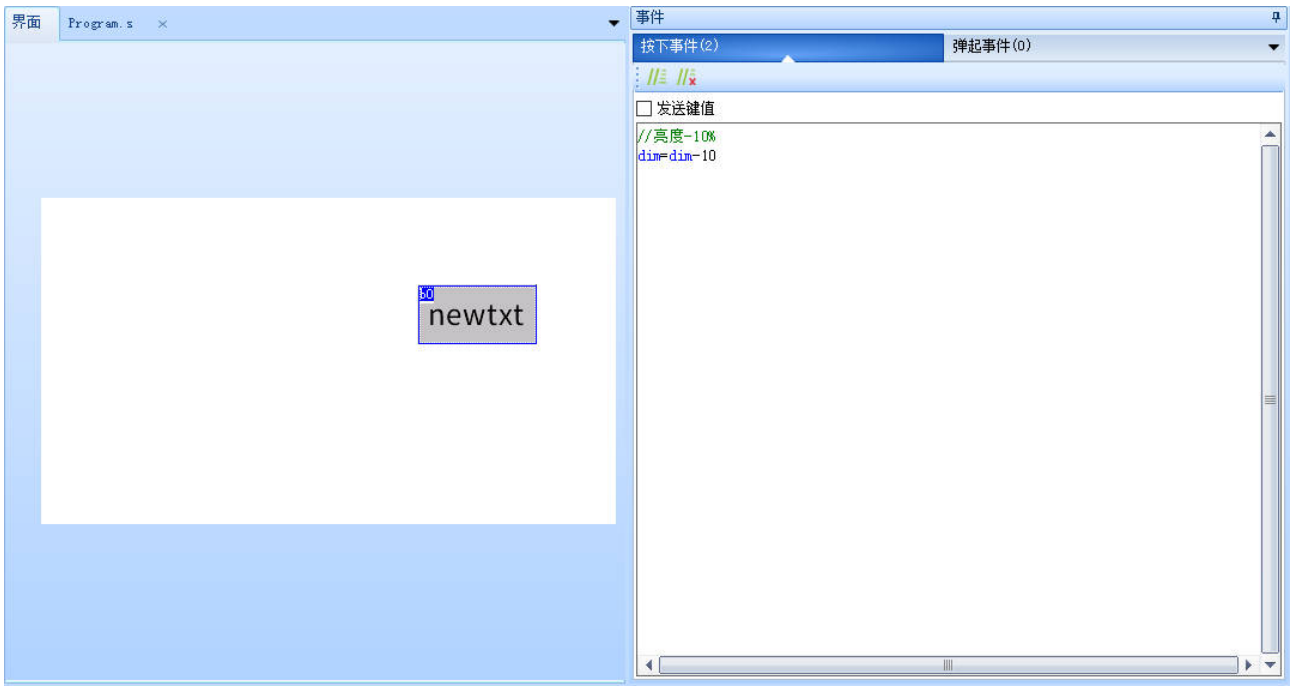
5.1.5.2 dim-示例 2

```
//亮度+10%
dim=dim+10
```



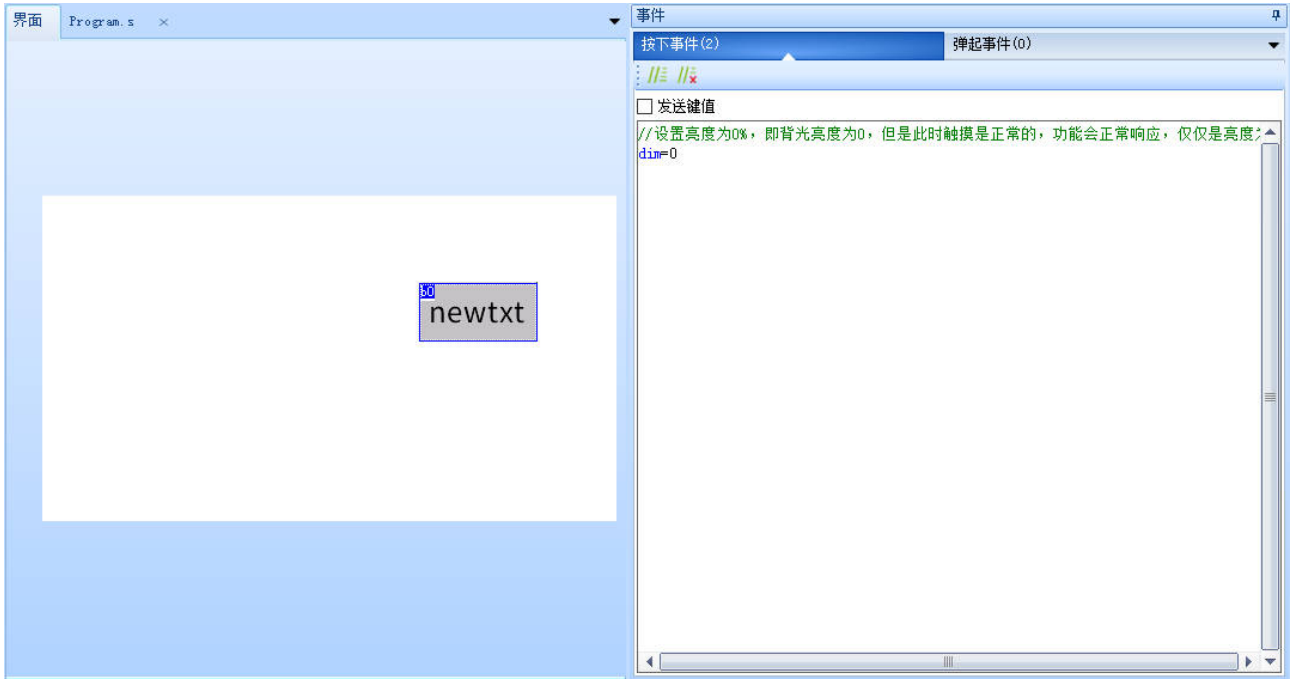
5.1.5.3 dim-示例 3

```
//亮度-10%
dim=dim-10
```



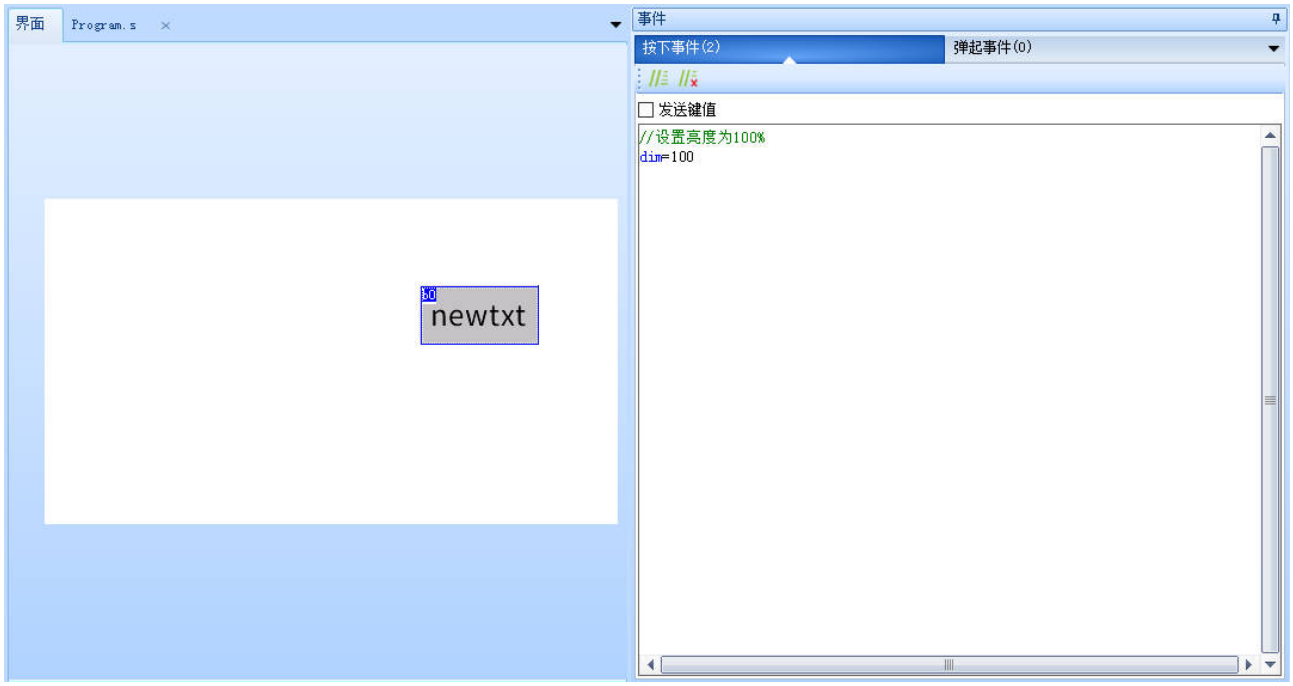
5.1.5.4 dim-示例：息屏，灭屏，设置

```
//设置亮度为0%，即背光亮度为0，但是此时触摸是正常的，功能会正常响应，仅仅是亮度为0
dim=0
```



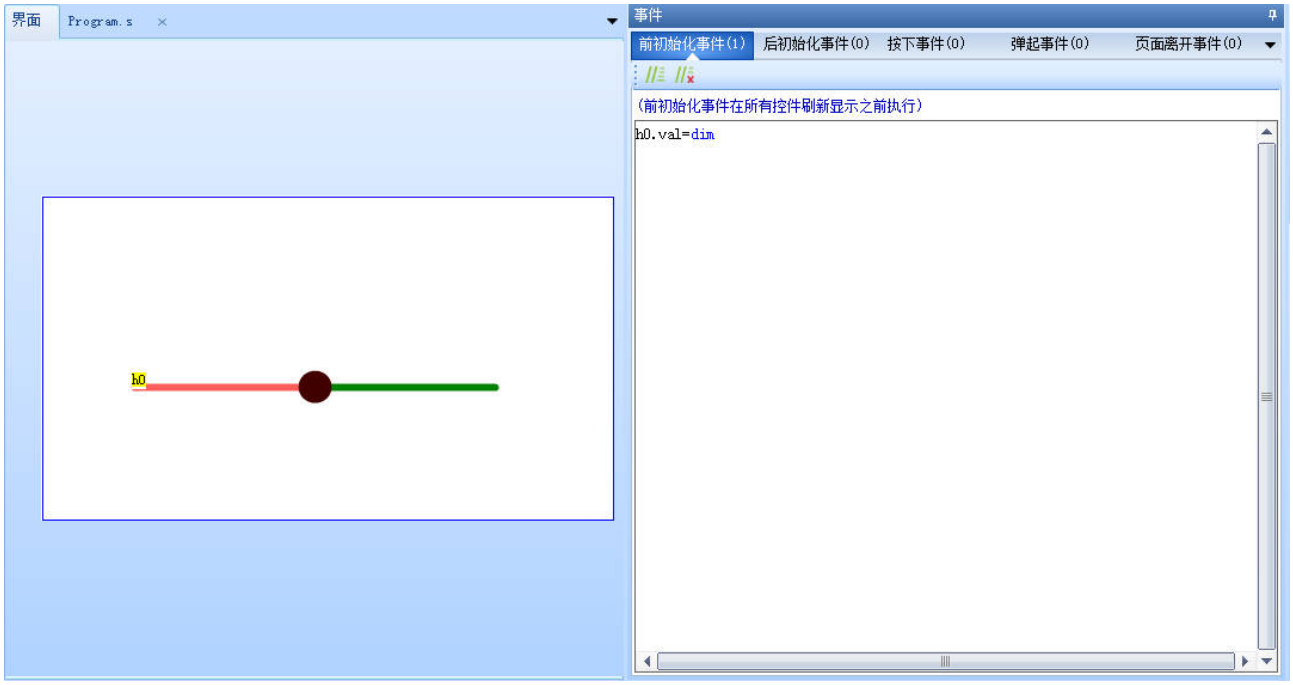
5.1.5.5 dim-示例：设置最高亮度

```
//设置亮度为100%
dim=100
```

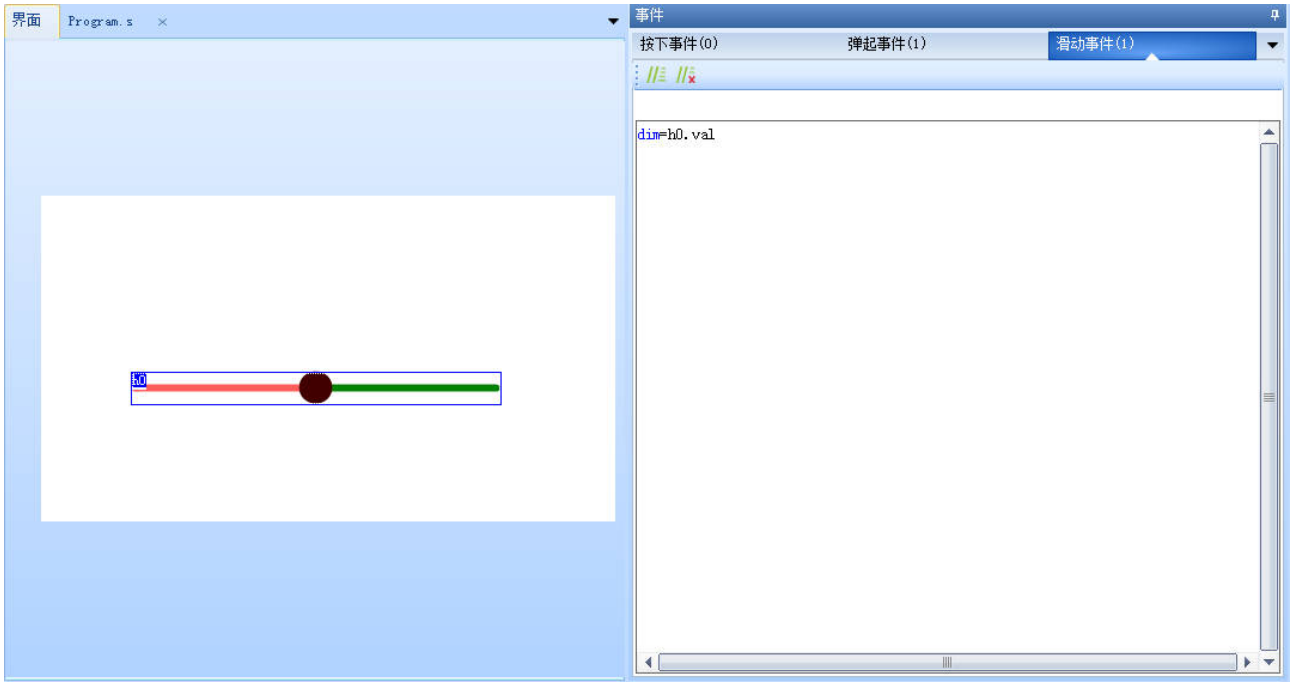


5.1.5.6 dim-示例：通过滑块设置亮度

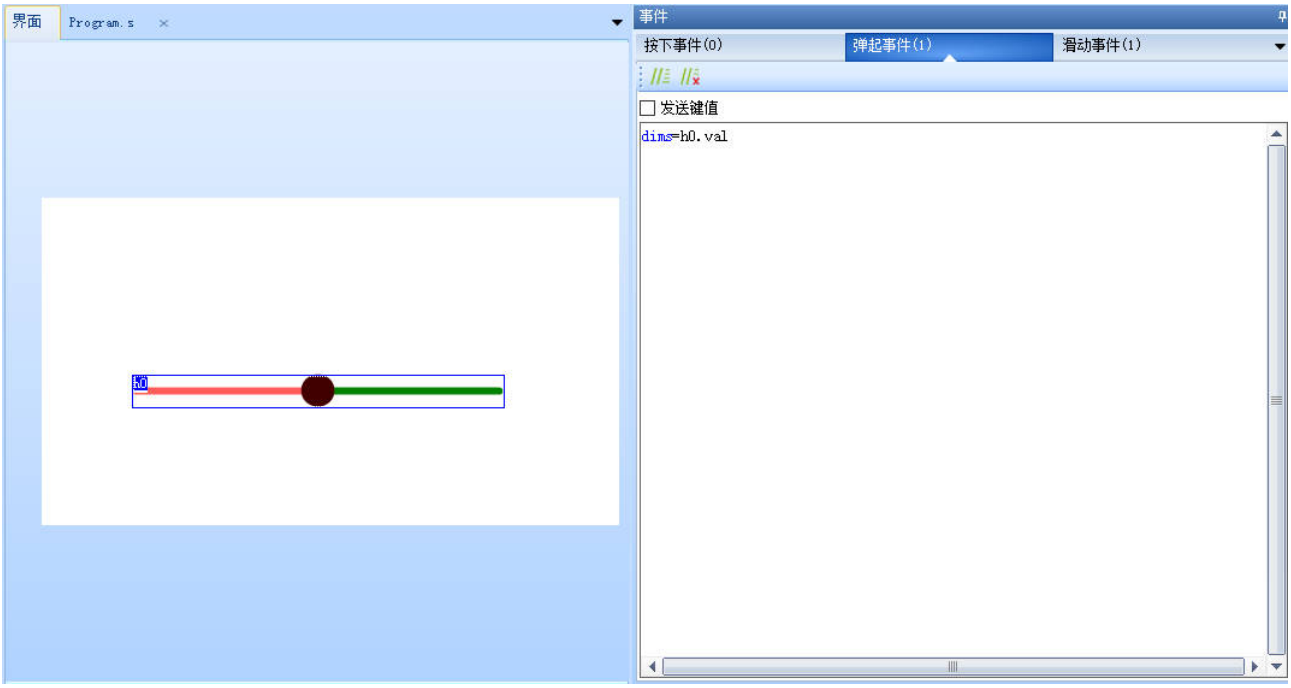
在前初始化事件中将 dim 赋值给进度条



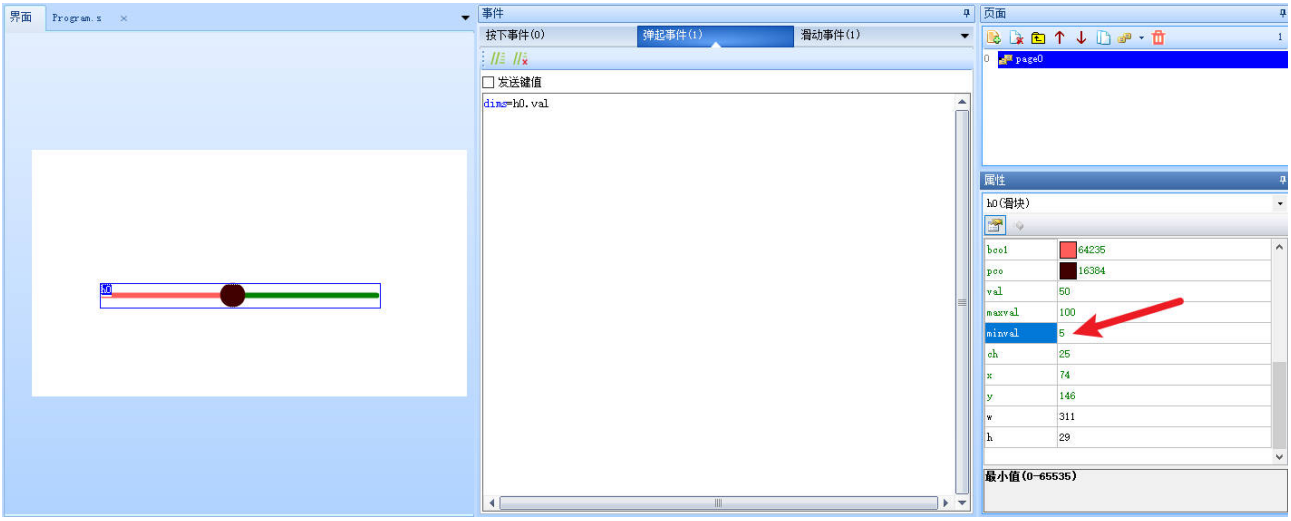
在滑动事件中将进度值赋值给 dim



在弹起事件中将进度值赋值给 dims



进度值的最小值建议设置为 5，避免完全黑屏后没有办法调回来



5.1.5.7 dim-c 语言示例

单片机通过串口控制串口屏当前背光亮度

```
//设置亮度为100%
printf("dim=100\xff\xff\xff");
```

5.1.5.8 dim-相关链接

如何配置亮度, 主动解析, 波特率 (page 1306)
滑块控件修改亮度 (page 466)
dim 和 *dims* 的区别 (page 1330)

5.1.5.9 dim-样例工程下载

演示工程下载链接:
《dim 和 dims-串口屏亮度》演示工程下载

5.1.6 bauds-上电默认波特率值

（掉电后保存，重启后继续有效）

注意： 通常配置在 program.s 文件中，请写在 page 指令的前面，page 指令后面的代码是不会执行的

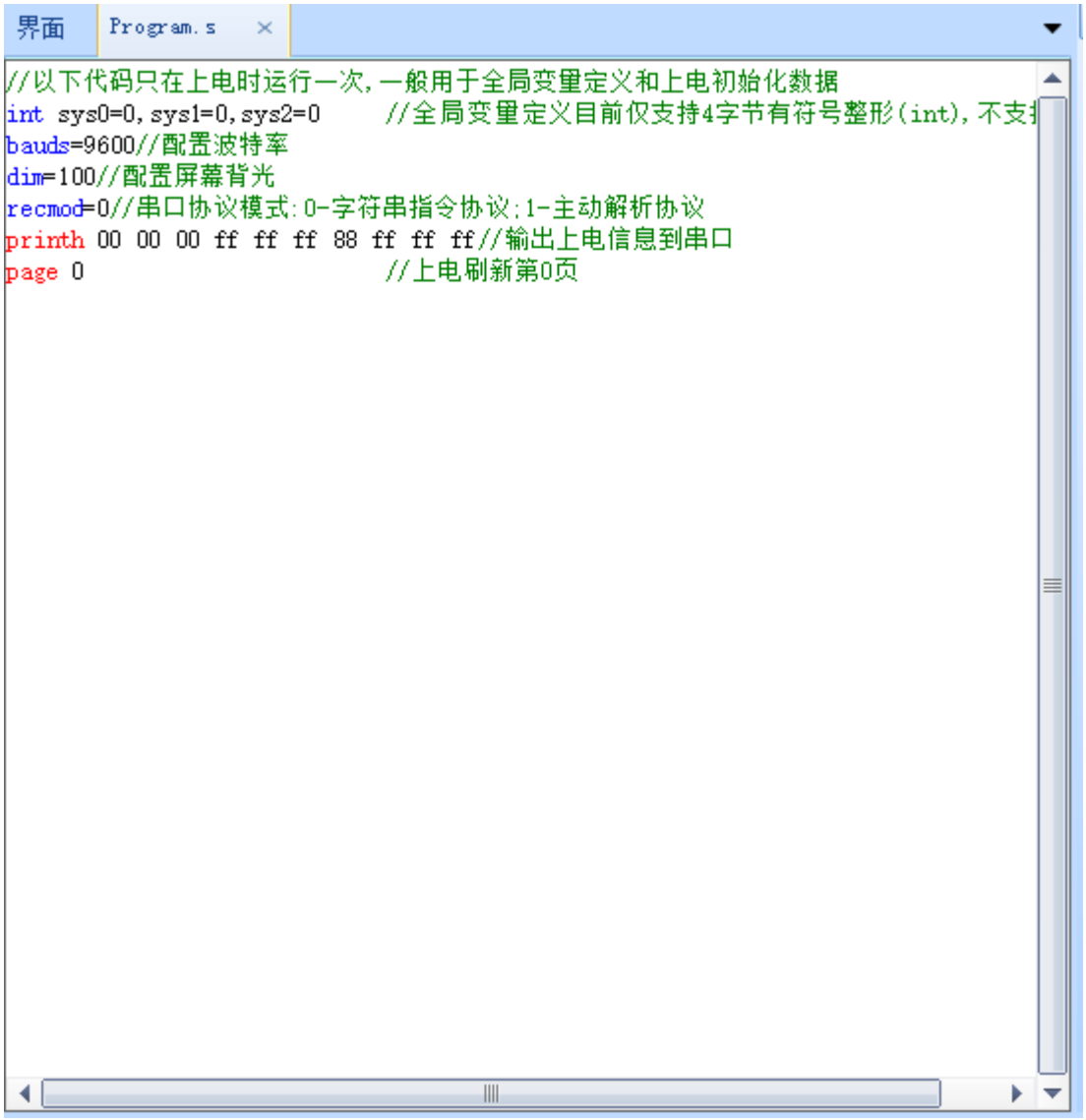
注意： 设置 bauds 时，会判断当前值与 flash 中保存的值是否一致，如果一致则不写入，如果不一致，则会写入 flash 中保存，因此修改 bauds 值会消耗 flash 寿命

注意： 设备支持的波特率有:2400 4800 9600 19200 38400 57600 115200 230400 256000 512000 921600

注意： bauds 可以在任意时刻读写，但是在上位机模拟器调试时，不管 bauds 设置的是多少，读取 bauds 始终都会是 9600，但是在串口屏实物上是正常的，你设置的多少读出来就是多少

5.1.6.1 bauds-示例 1

```
//当前波特率设置为9600
bauds=9600
```



5.1.6.2 bauds-示例 2

```
//默认波特率设置为115200
bauds=115200
```

界面 Program.s

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
int sys0=0,sys1=0,sys2=0      //全局变量定义目前仅支持4字节有符号整形(int),不支
bauds=115200//配置波特率
dim=100//配置屏幕背光
recmod=0//串口协议模式:0-字符串指令协议;1-主动解析协议
printh 00 00 00 ff ff ff 88 ff ff ff//输出上电信息到串口
page 0                        //上电刷新第0页
```

5.1.6.3 bauds-相关链接

如何配置亮度,主动解析,波特率 (page 1306)

baud 和 *bauds* 的区别 (page 1331)

使用串口下载工程速度慢怎么办 (page 1391)

5.1.7 baud-当前波特率值

(本次修改，重启后丢失)

> **注意：** 通常配置在 program.s 文件中，请写在 page 指令的前面，page 指令后面的代码是不会执行的

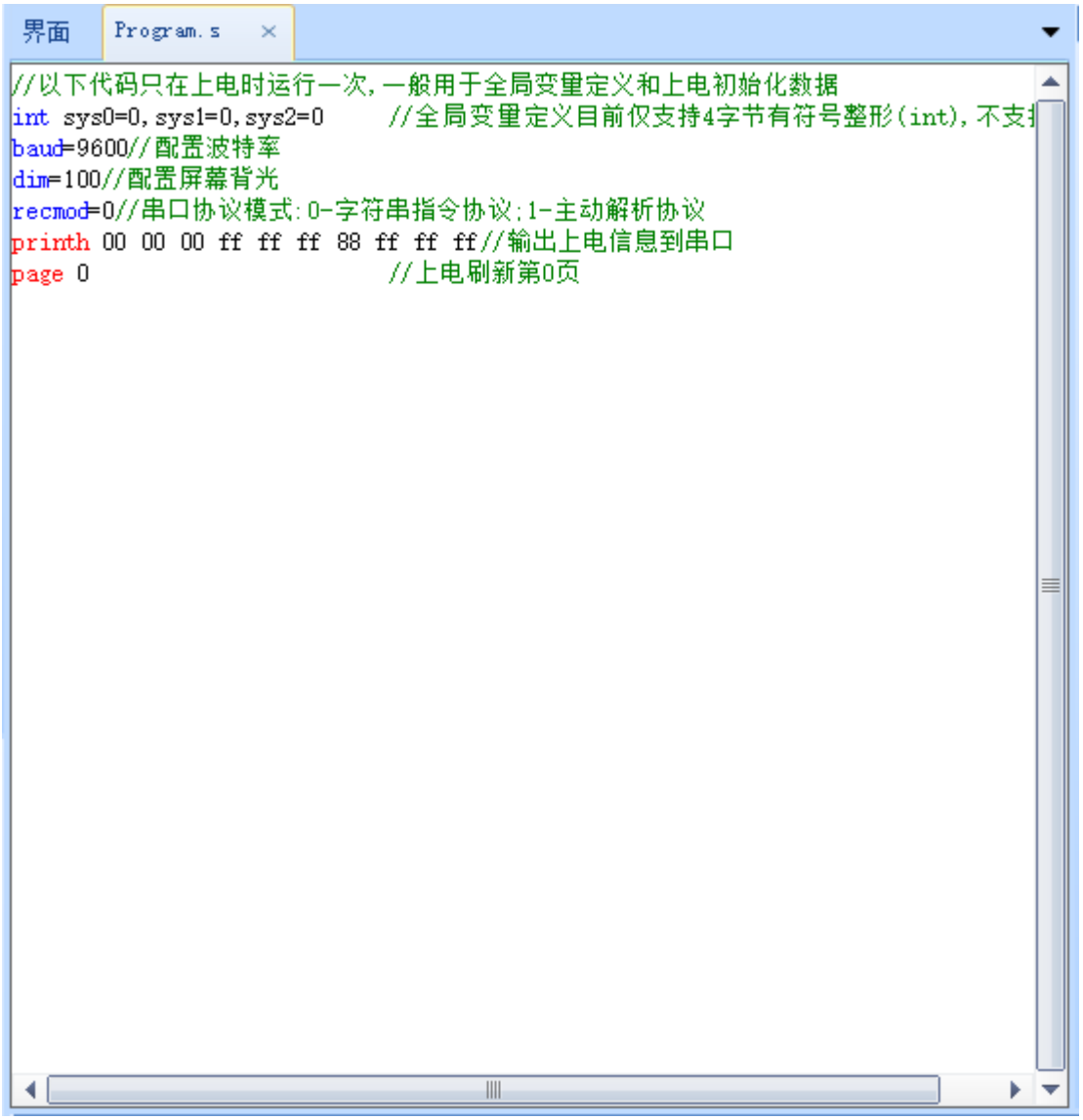
> **注意：** baud 不会写入 flash 中保存，因此修改 baud 值不会消耗 flash 寿命

> **注意：** 设备支持的波特率有:2400 4800 9600 19200 38400 57600 115200 230400 256000 512000 921600

> **注意：** baud 可以在任意时刻读写，但是在上位机模拟器调试时，不管 baud 设置的是多少，读取 baud 始终都会是 9600，但是在串口屏实物上是正常的，你设置的多少读出来就是多少
>
> 因此，如果你使用的是其他非 9600 的波特率，可以在程序中判断当前波特率是否为 9600 来判断当前是否运行在上位机模拟器中（即是否为调试状态）
>
> 如果是非调试状态，则可以将一些调试的数据隐藏起来

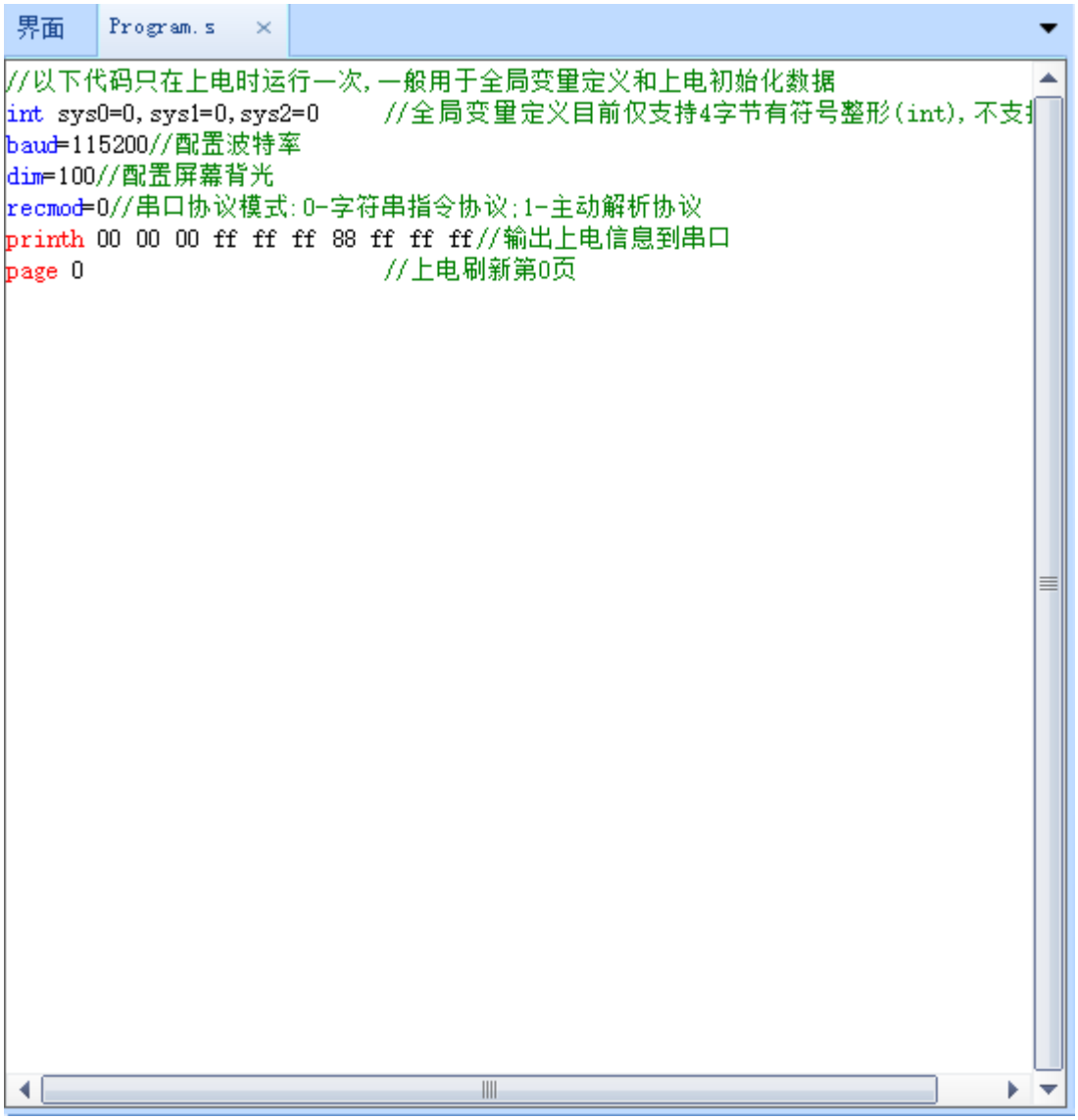
5.1.7.1 baud-示例 1

```
//当前波特率设置为9600
baud=9600
```



5.1.7.2 baud-示例 2

```
//当前波特率设置为115200
baud=115200
```



5.1.7.3 baud-相关链接

如何配置亮度,主动解析,波特率 (page 1306)

baud 和 *bauds* 的区别 (page 1331)

使用串口下载工程速度慢怎么办 (page 1391)

5.1.8 ussp-无串口数据自动睡眠时间

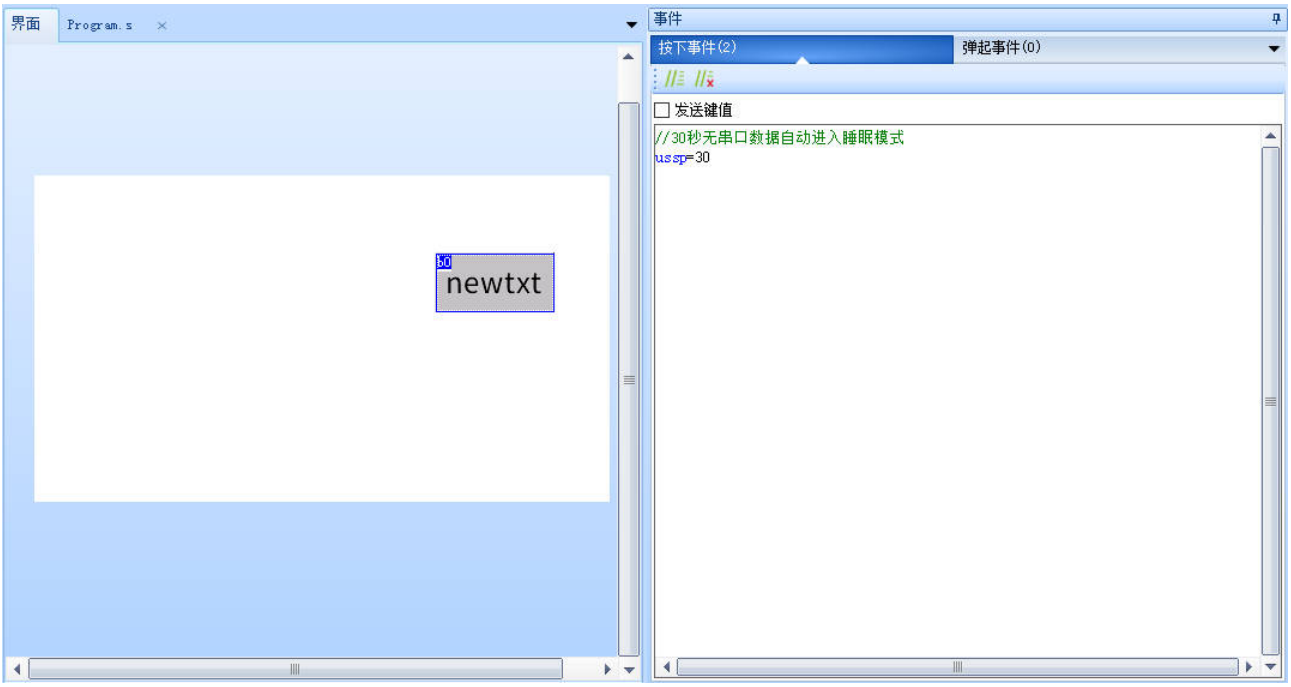
注意: 此指令不建议在主动解析下使用，因为主动解析模式下原有淘晶驰字符串指令不再生效，一旦睡眠，无法通过串口唤醒

在主动解析模式下需要实现类似休眠的功能请参考自定义屏幕保护样例工程/低功耗/睡眠休眠替代 (page 1130)

当 ussp 被赋值时, 休眠时间会清零重新计算。

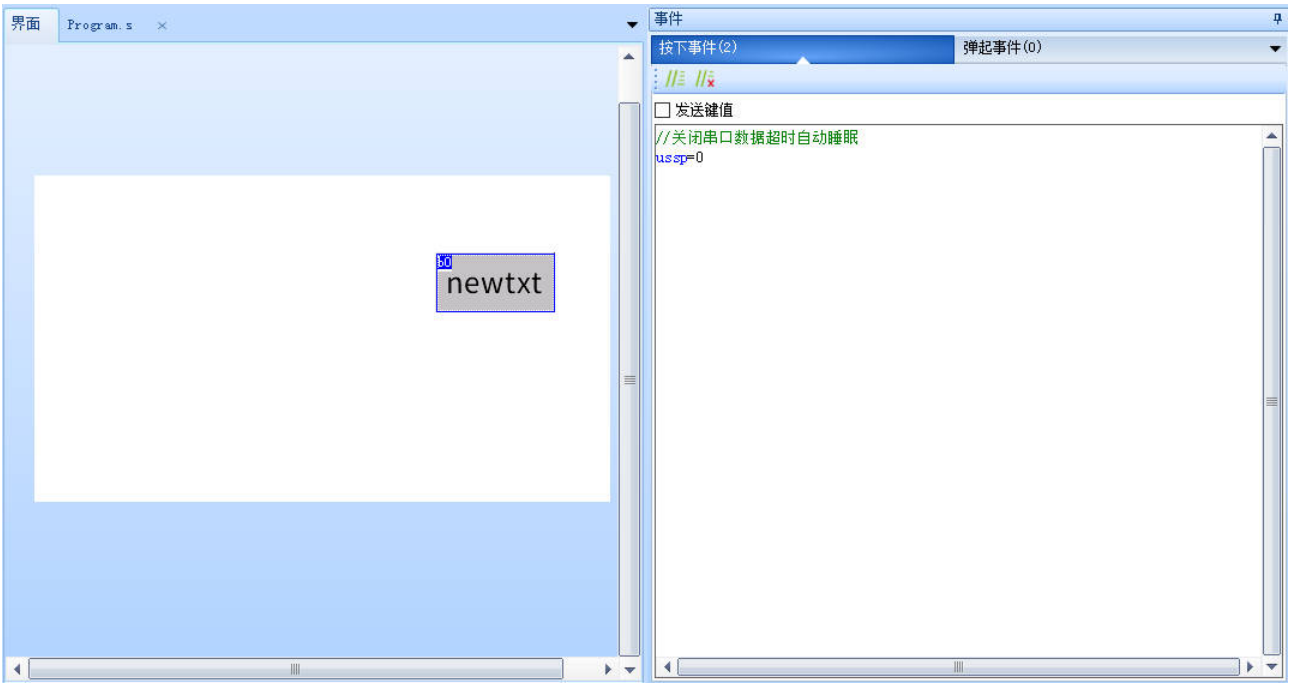
5.1.8.1 ussp-示例 1

```
//30秒无串口数据自动进入睡眠模式
ussp=30
```



5.1.8.2 ussp-示例 2

```
//关闭串口数据超时自动睡眠
ussp=0
```



注意： 单位: 秒, 最小 3，最大 65535，上电默认 0(0 为关闭串口数据超时自动睡眠)

5.1.8.3 ussp-c 语言示例

单片机通过串口控制串口屏退出睡眠

```
//退 出 睡 眠
printf("sleep=0\xff\xff\xff");
```

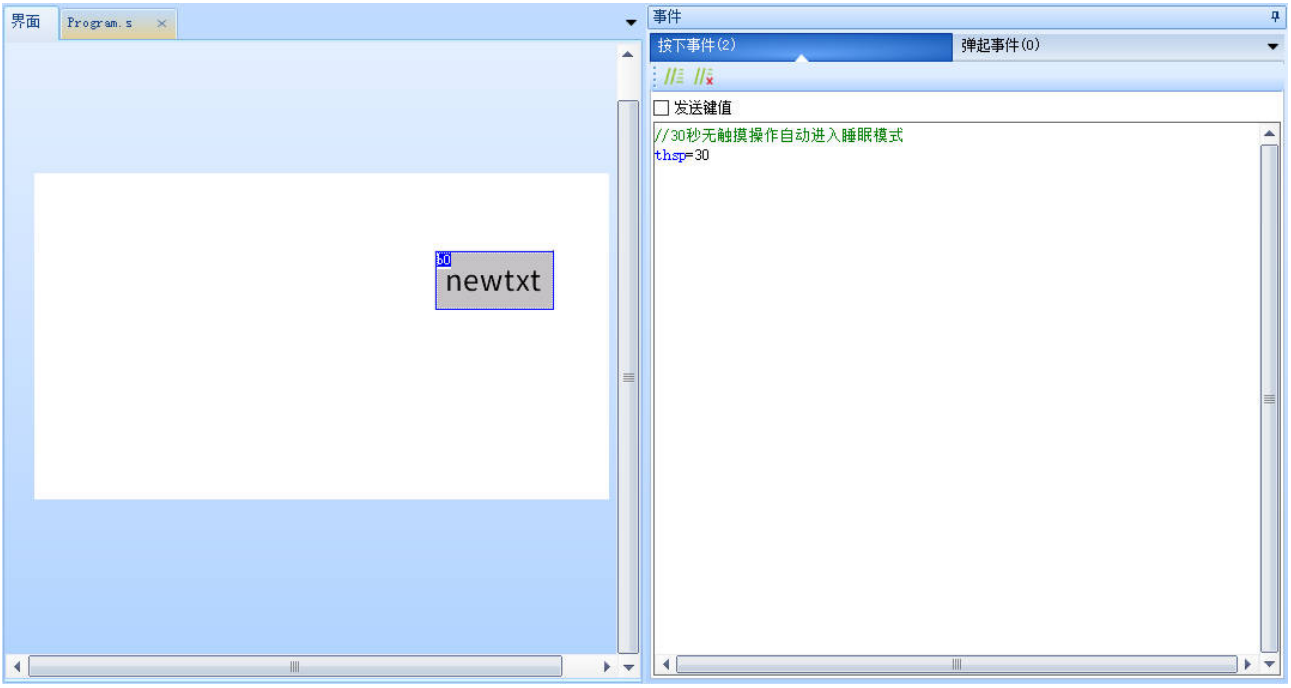
5.1.9 thsp-无触摸操作自动睡眠时间

单位：秒, 最小 3，最大 65535，上电默认为 0（0 为关闭触摸超时自动睡眠）

注意： 当 thsp 被赋值时, 休眠时间会清零重新计算

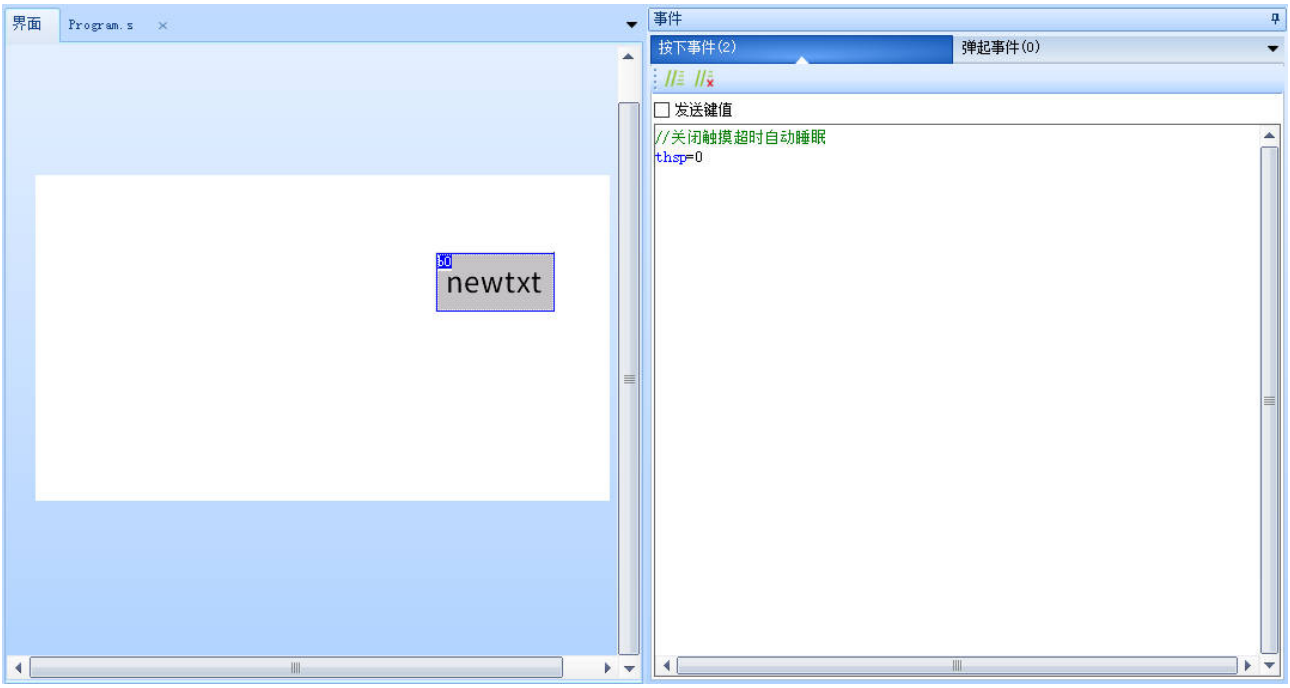
5.1.9.1 thsp-示例 1

```
//30秒无触摸操作自动进入睡眠模式
thsp=30
```



5.1.9.2 thsp-示例 2

```
//关闭触摸超时自动睡眠
thsp=0
```



注意： 单位: 秒, 最小 3，最大 65535，上电默认 0(0 为关闭触摸超时自动睡眠)

5.1.9.3 thsp-c 语言示例

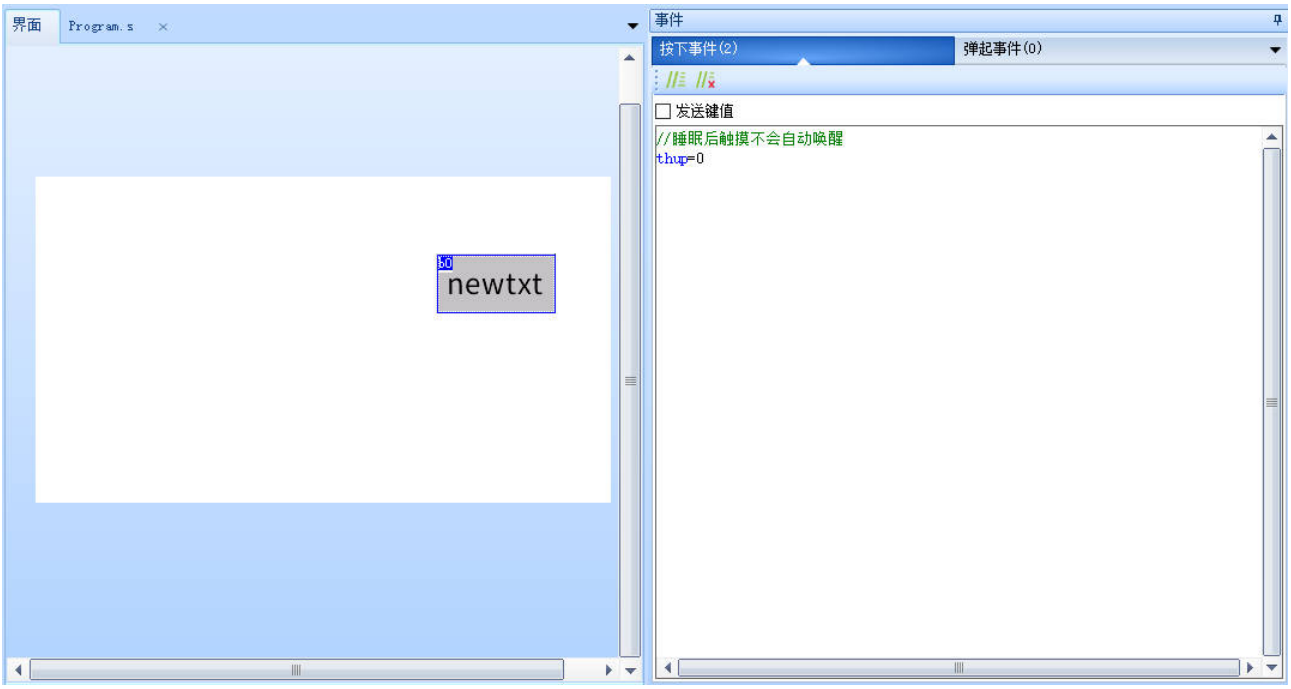
单片机通过串口控制串口屏退出睡眠

```
//退 出 睡 眠
printf("sleep=0\xff\xff\xff");
```

5.1.10 thup-睡眠模式下触摸自动唤醒开关

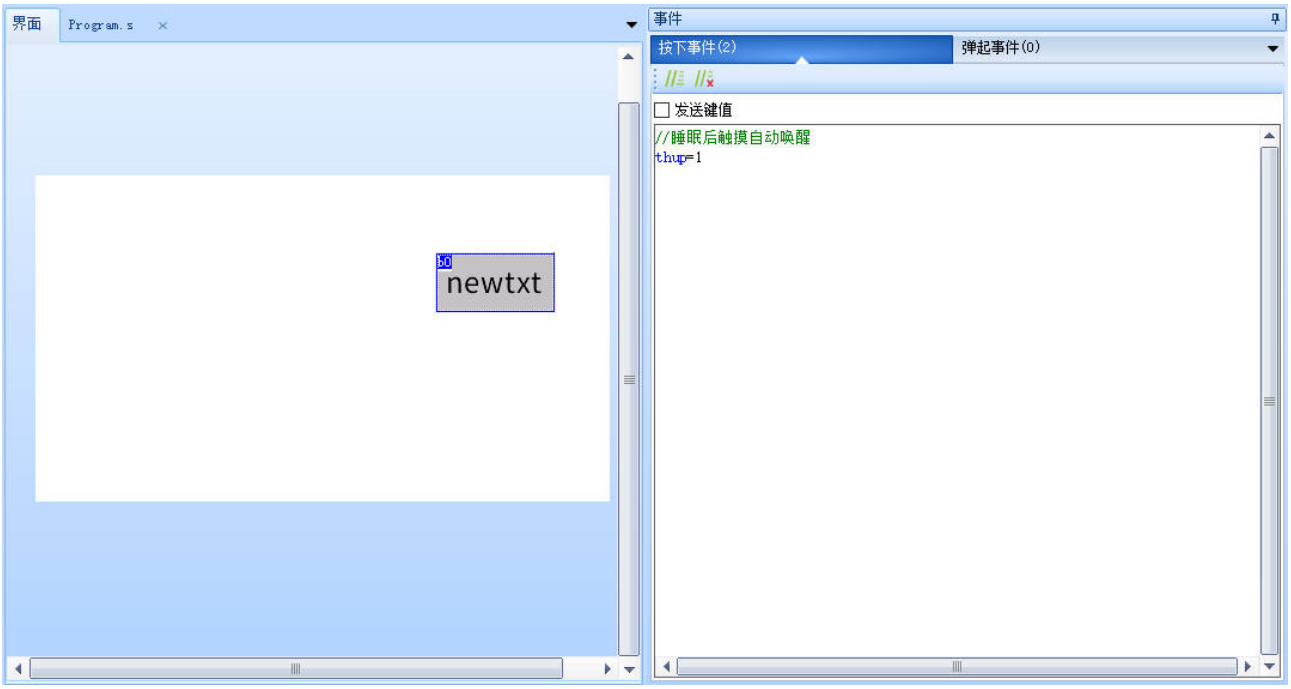
5.1.10.1 thup-示例 1

```
//睡眠后触摸不会自动唤醒
thup=0
```



5.1.10.2 thup-示例 2

```
//睡眠后触摸自动唤醒
thup=1
```



注意： 上电默认 0，不管 thup 为 0 还是 1，睡眠模式下有触摸操作的时候设备均会发送触摸坐标到串口。

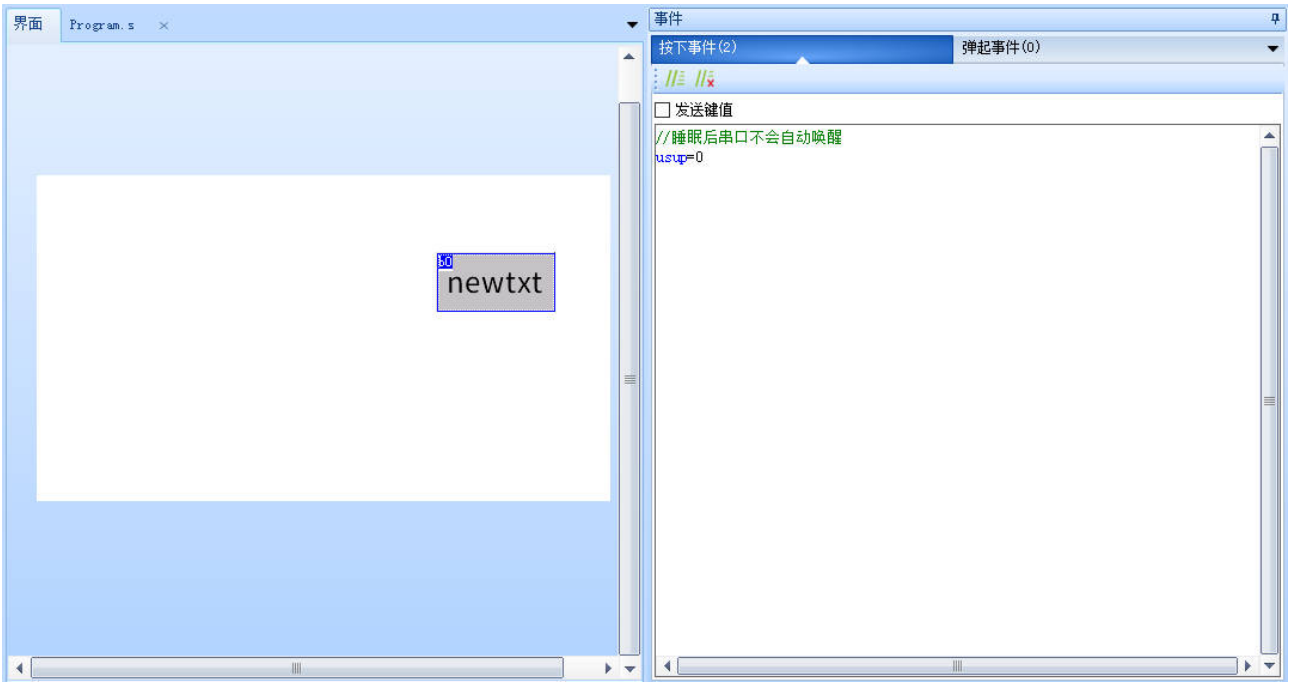
参考:*0X68 睡眠模式触摸事件* (page 900)

5.1.11 usup-睡眠模式下串口数据自动唤醒开关

注意： 此指令仅在淘晶字符串指令模式下起作用，在主动解析下串口屏无法被唤醒

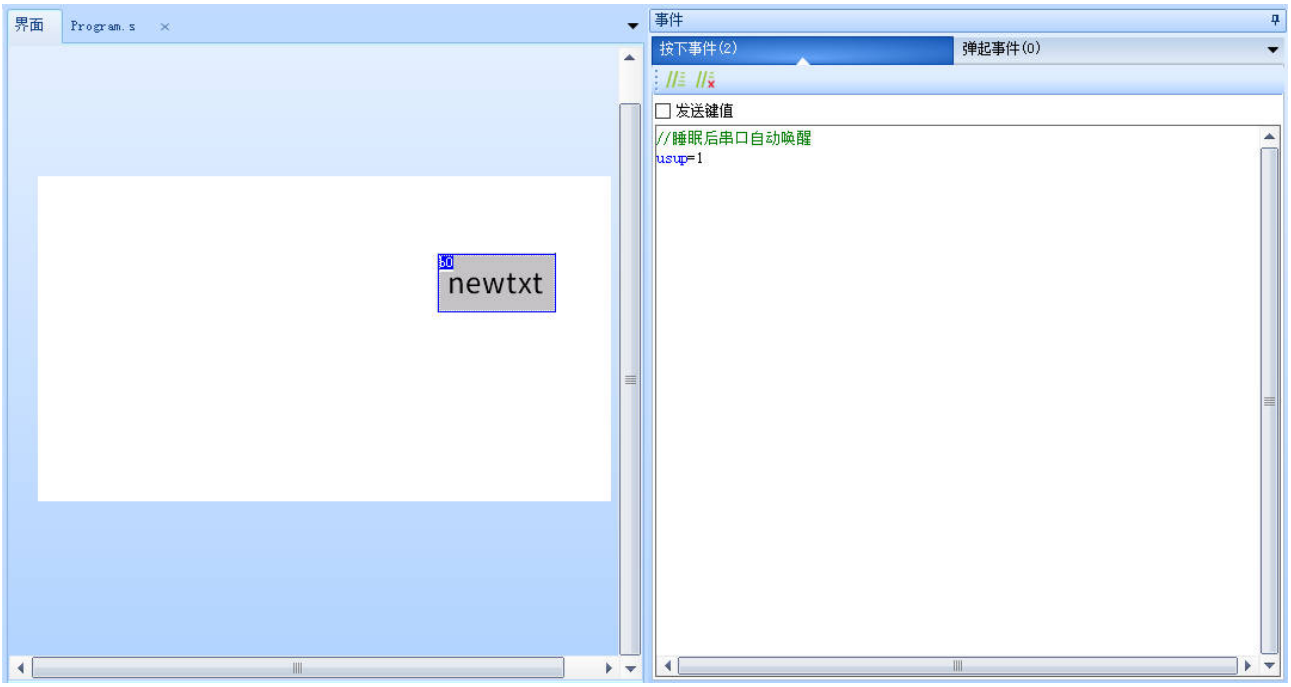
5.1.11.1 usup-示例 1

```
//睡眠后串口不会自动唤醒
usup=0
```



5.1.11.2 usup-示例 2

```
//睡眠后串口自动唤醒
usup=1
```



注意： 此指令仅在淘晶字符串指令模式下起作用，在主动解析下串口屏无法被唤醒

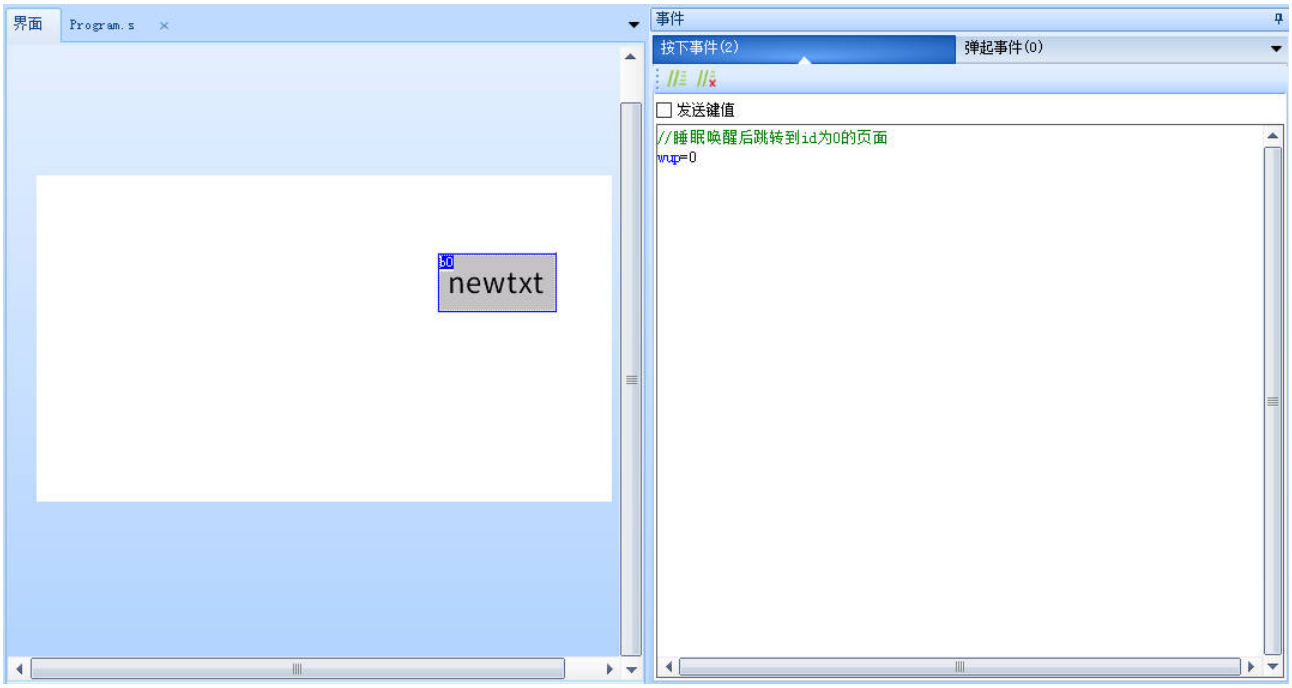
上电默认为 0，不会自定唤醒，需要发送 sleep=0 才能唤醒屏幕。

如果设置为 1，串口收到任何数据（需要加上结束符 16 进制 3 个 ff）都会立刻自动唤醒。

5.1.12 wup-睡眠唤醒后刷新页面设置

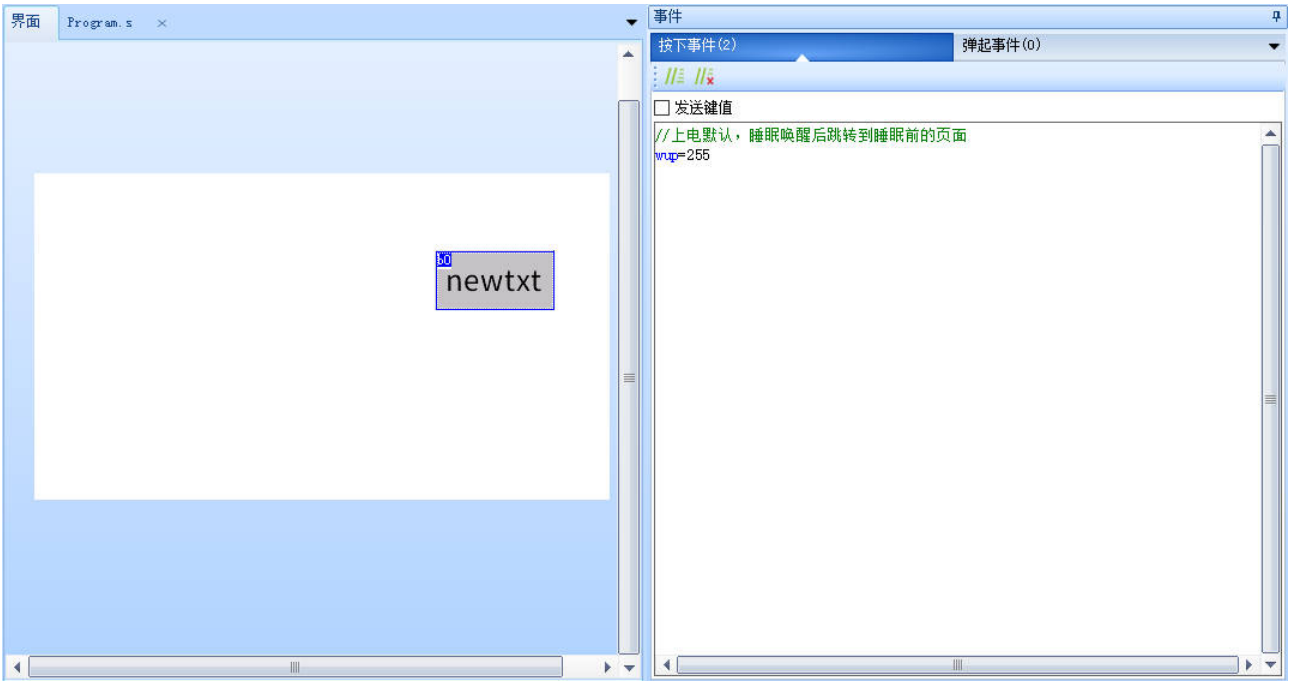
5.1.12.1 wup-示例 1

```
//睡眠唤醒后跳转到id为0的页面
wup=0
```



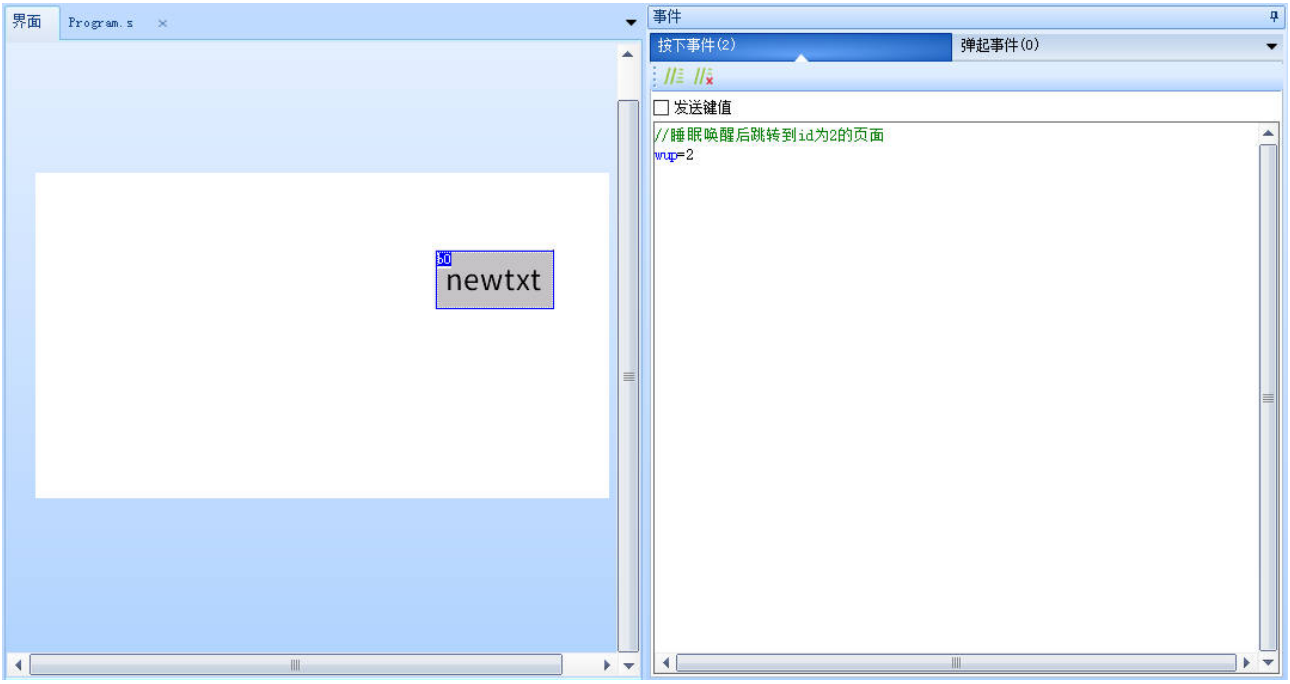
5.1.12.2 wup-示例 2

```
//上电默认，睡眠唤醒后跳转到睡眠前的页面
wup=255
```



5.1.12.3 wup-示例 3

```
//睡眠唤醒后跳转到id为2的页面
wup=2
```



注意： 设备已经在睡眠状态下，也可以执行串口传过来的 wup=X 赋值。

5.1.13 sleep-睡眠

用于配置串口屏睡眠、休眠

> **注意：** 进入睡眠状态下，亮度为 0，可以执行如下指令：get,prints, printh。也可以执行 sleep=0，wup=X 的赋值语句，但对其他系统变量以及对控件的赋值会失效，并且支持上位软件联机，其他指令不会执行。
>
> 进入睡眠状态下，所有定时器控件停止运行。
>
> 支持通过串口或者触摸唤醒屏幕，但是不支持通过 IO 口唤醒屏幕。

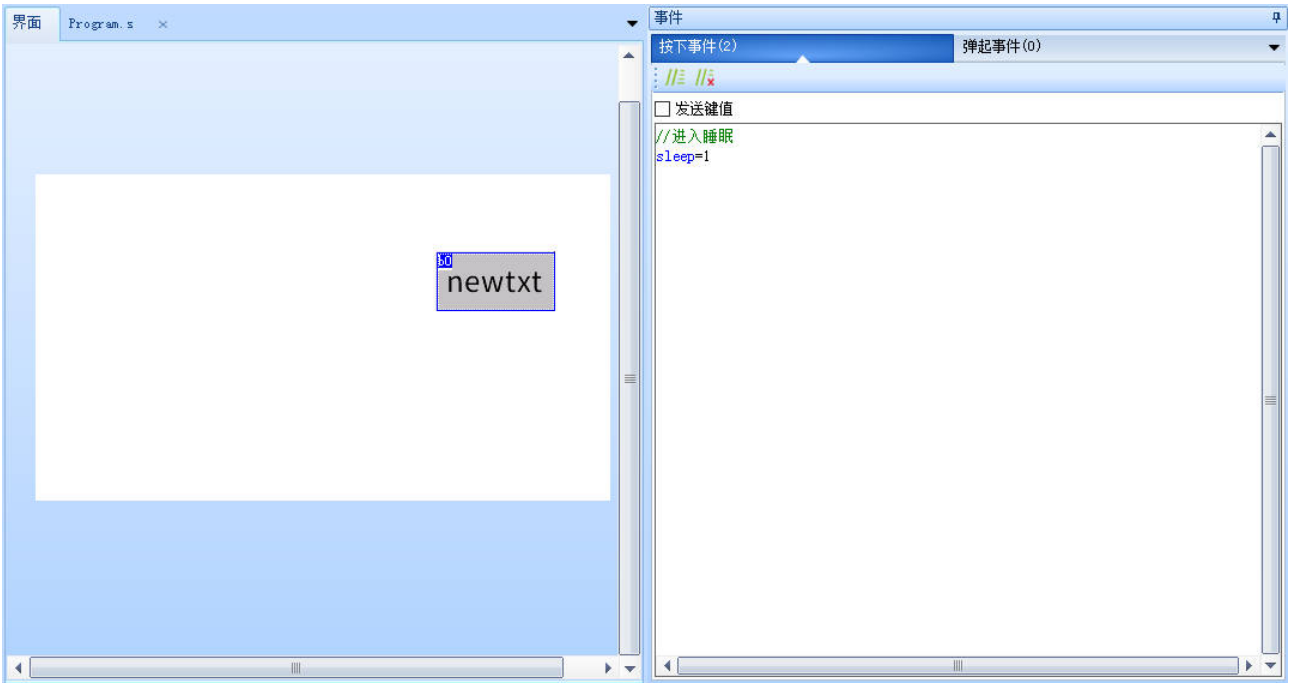
5.1.13.1 sleep 模式与关闭亮度的功耗区别

x 系列两者区别不大，t 系列的睡眠模式下功耗会低很多

> **注意：** 自动休眠相关配置请参考:
>
> *thsp-无触摸操作自动睡眠时间* (page 773)
>
> *ussp-无串口数据自动睡眠时间* (page 771)

5.1.13.2 sleep-示例 1

```
//进入睡眠
sleep=1
```



> **注意：** 进入睡眠模式, 也就是设置了 sleep=1, 后面的代码不会被执行, 如果有代码, 请写在 sleep=1 指令的前面

5.1.13.3 sleep-c 语言示例

单片机通过串口控制串口屏进入睡眠

```
//进入睡眠
printf("sleep=1\xff\xff\xff");
```

单片机通过串口控制串口屏退出睡眠

```
//退出睡眠
printf("sleep=0\xff\xff\xff");
```

5.1.13.4 sleep-相关链接

休眠替代方法 (page 1328)

sleep 睡眠模式与拓展 *IO* (page 1041)

5.1.13.5 sleep-样例工程下载

演示工程下载链接：

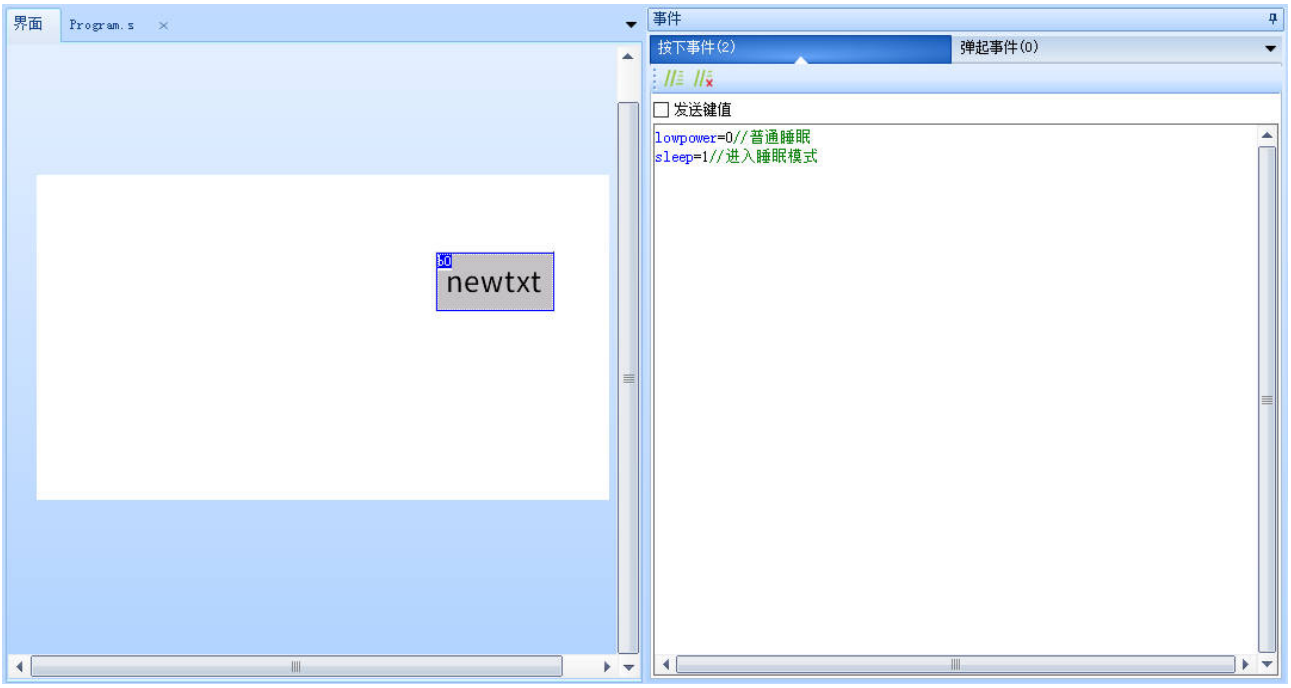
《sleep 休眠》演示工程下载

5.1.14 lowpower-睡眠模式设定

注意： 仅 t1 系列支持，上电默认为 0(关闭)

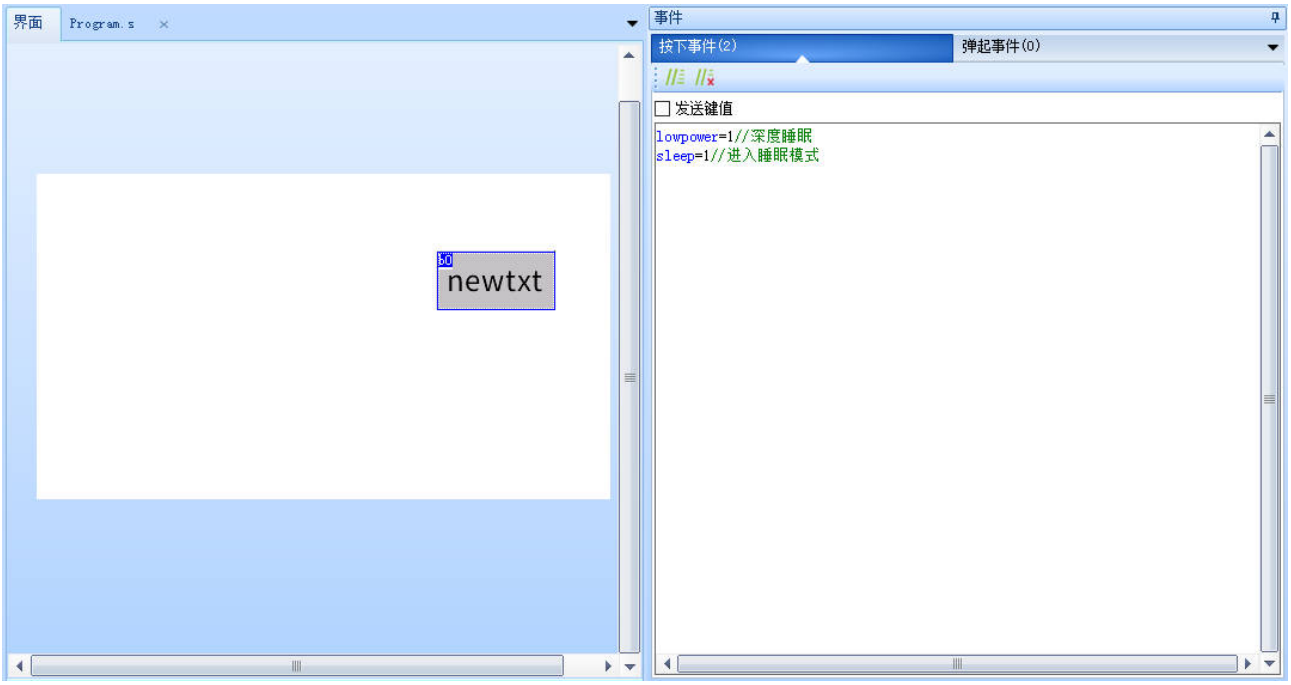
5.1.14.1 lowpower-示例 1

```
lowpower=0//普通睡眠
sleep=1//进入睡眠模式
```



5.1.14.2 lowpower-示例 2

```
lowpower=1//深度睡眠
sleep=1//进入睡眠模式
```



注意： 普通睡眠 (电流较大，但不丢串口数据)；深度睡眠 (电流较小，睡眠之后如果有串口数据过来会丢失前面 50ms 左右的串口数据)，设置 lowpower 后，还要手动设置 sleep 或者配置 thsp/ussp

5.1.15 bkcmd-串口指令执行状态数据返回

(上电默认为 2)

> **注意：** 通常配置在 program.s 文件中，请写在 page 指令的前面，page 指令后面的代码是不会执行的。
>
> bkcmd 只会设置串口数据的返回状态，如果是串口屏上本身的代码有问题，例如操作名称组范围错误或者打开文件失败等，串口屏一定会返回错误结果，不受 bkcmd 设置的影响。
>
> bkcmd=0 不返回结果；bkcmd=1 只返回成功的结果；bkcmd=2 只返回失败的结果；bkcmd=3 成功或者失败都返回结果。

5.1.15.1 bkcmd-示例 1

```
//不返回结果
bkcmd=0
```

界面 Program.s

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
int sys0=0,sys1=0,sys2=0      //全局变量定义目前仅支持4字节有符号整形(int),不
bauds=115200//配置波特率
dim=100//配置屏幕背光
recmod=0//串口协议模式:0-字符串指令协议;1-主动解析协议
bkcmd=0//不返回结果
printh 00 00 00 ff ff ff 88 ff ff ff//输出上电信息到串口
page 0                        //上电刷新第0页
```

5.1.15.2 bkcmd-示例 2

```
//只返回成功的结果
bkcmd=1
```

界面 Program.s

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
int sys0=0,sys1=0,sys2=0      //全局变量定义目前仅支持4字节有符号整形(int),不
bauds=115200//配置波特率
dim=100//配置屏幕背光
recmod=0//串口协议模式:0-字符串指令协议;1-主动解析协议
bkcmd=1//只返回成功的结果
printh 00 00 00 ff ff ff 88 ff ff ff//输出上电信息到串口
page 0                        //上电刷新第0页
```

5.1.15.3 bkcmd-示例 3

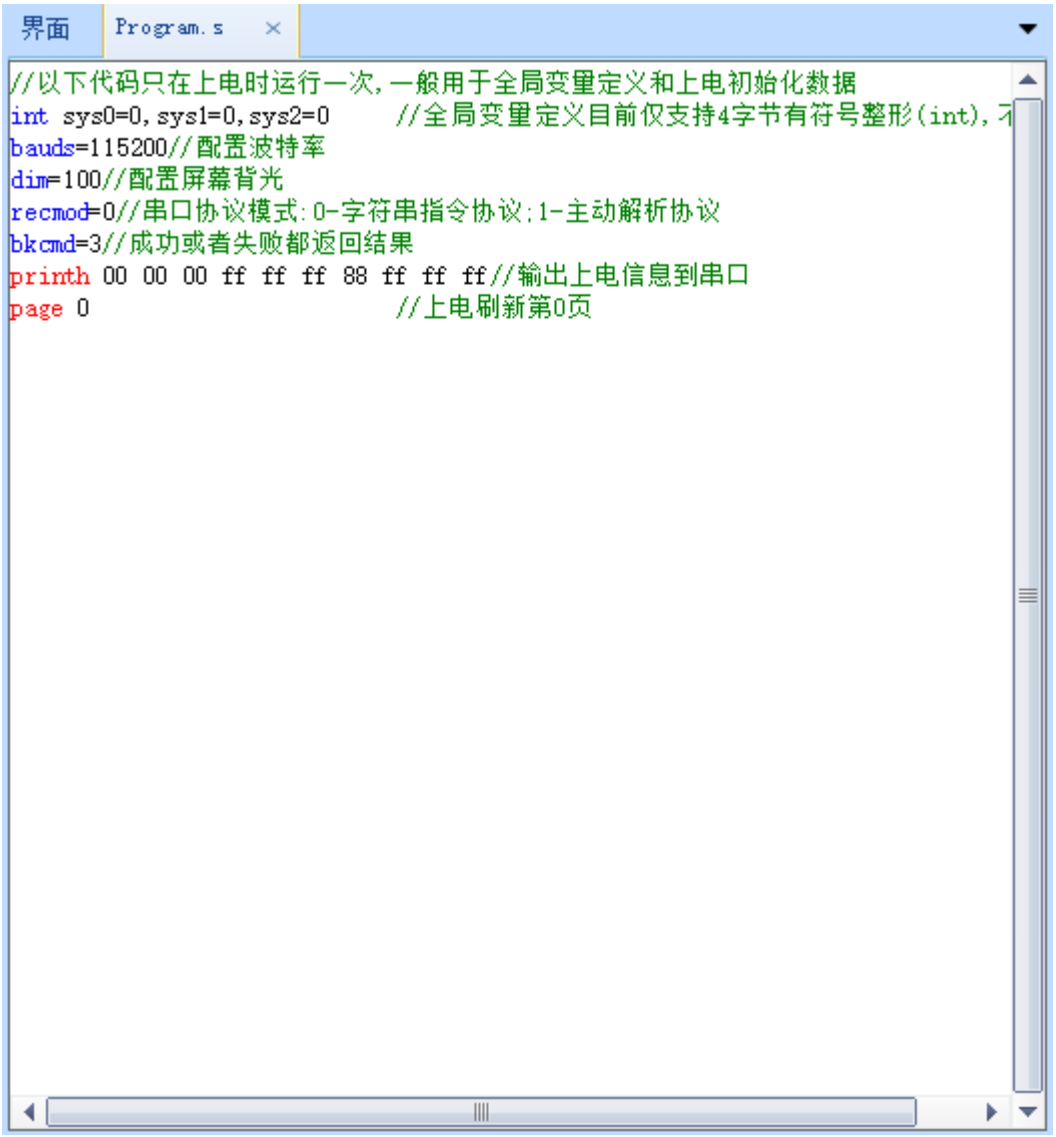
```
//只返回失败的结果
bkcmd=2
```

界面 Program.s

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
int sys0=0,sys1=0,sys2=0      //全局变量定义目前仅支持4字节有符号整形(int),不
bauds=115200//配置波特率
dim=100//配置屏幕背光
recmod=0//串口协议模式:0-字符串指令协议;1-主动解析协议
bkcmd=2//只返回失败的结果
printh 00 00 00 ff ff ff 88 ff ff ff//输出上电信息到串口
page 0                        //上电刷新第0页
```

5.1.15.4 bkcmd-示例 4

```
//成功或者失败都返回结果
bkcmd=3
```



注意： 此设置只影响串口指令执行成功或者失败的结果返回，上位软件编辑界面时写入的指令执行错误的时候一定会返回错误结果，成功的时候一定不会返回执行结果。此设置也不会影响获取设备控件数据时的数据返回。

5.1.15.5 bkcmd-相关链接

如何配置亮度，主动解析，波特率 (page 1306)

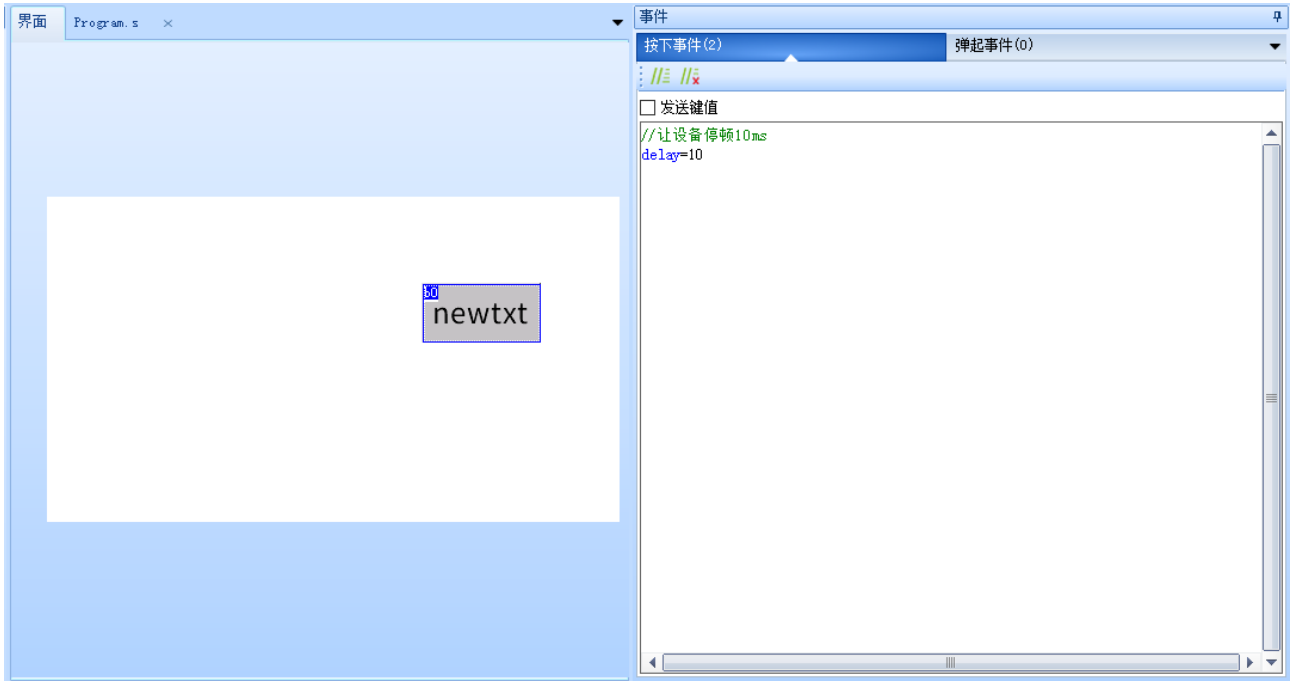
串口屏返回的数据格式请参考：串口屏返回数据格式 (page 874)

5.1.16 delay-延时

> **注意：** delay 的最小值为 1，也就是 1 毫秒
>
> 串口屏是单线程的, 调用 delay 会直接死等, 时间过长就会造成很明显的卡顿现象, 延时超过 50ms 时不建议使用 delay，超过 50ms 时建议用定时器控件来延时
>
> 执行延时指令后，设备 CPU 不会执行任何指令，但是会继续接受串口指令保存到串口指令缓存区。
>
> 50 毫秒以下的延时可以用 delay 进行延时, 超过 50 毫秒以上建议使用定时器进行延时

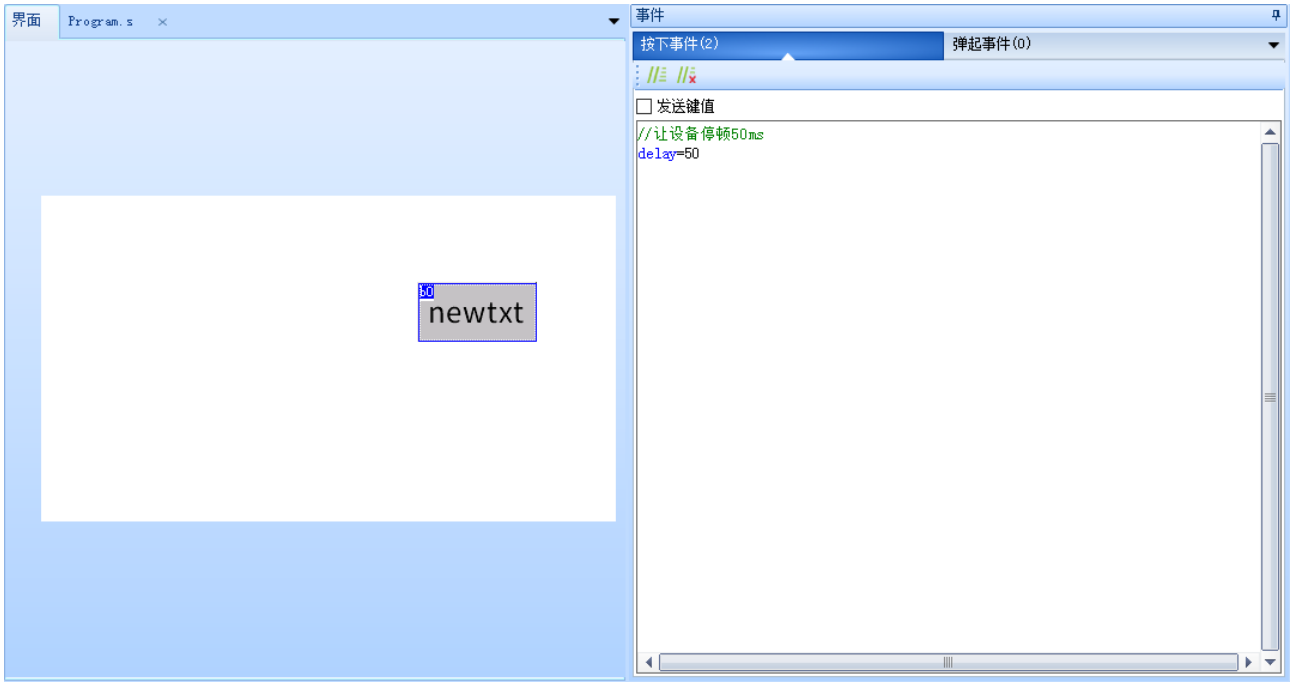
5.1.16.1 delay-示例 1

```
//让设备停顿10ms
delay=10
```



5.1.16.2 delay-示例 2

```
//让设备停顿50ms
delay=50
```



5.1.16.3 delay-样例工程下载

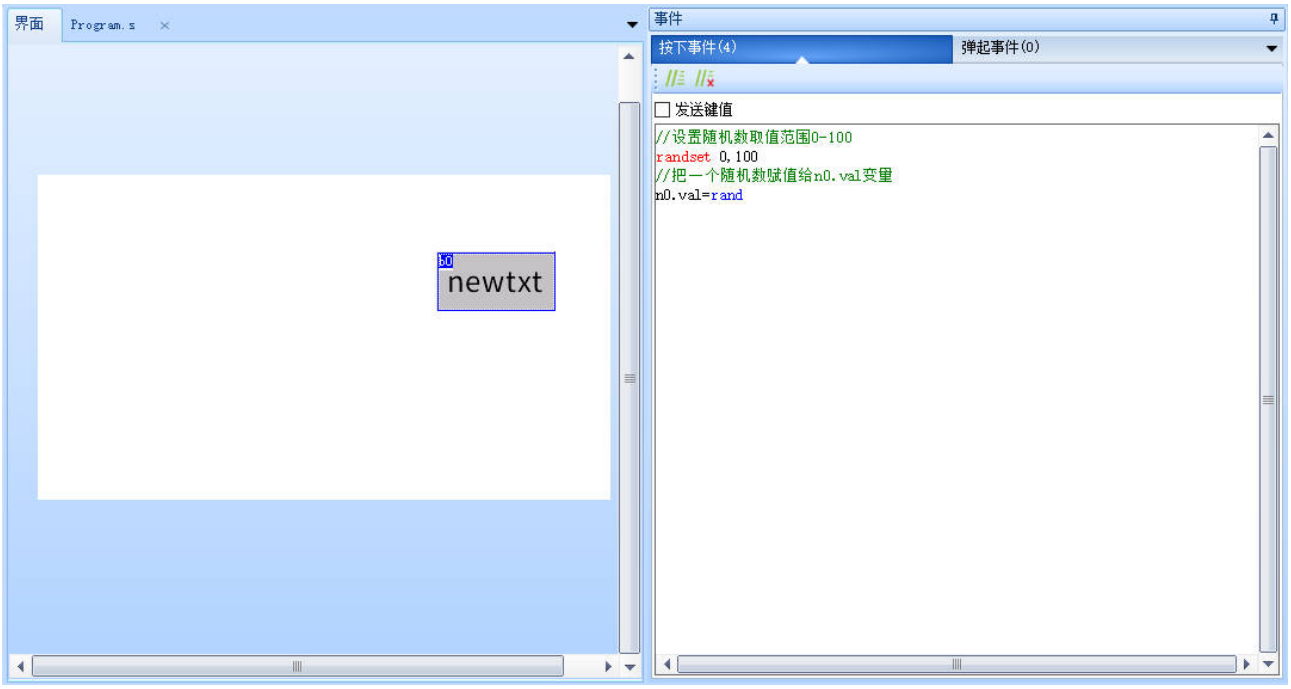
演示工程下载链接：

《delay》演示工程下载

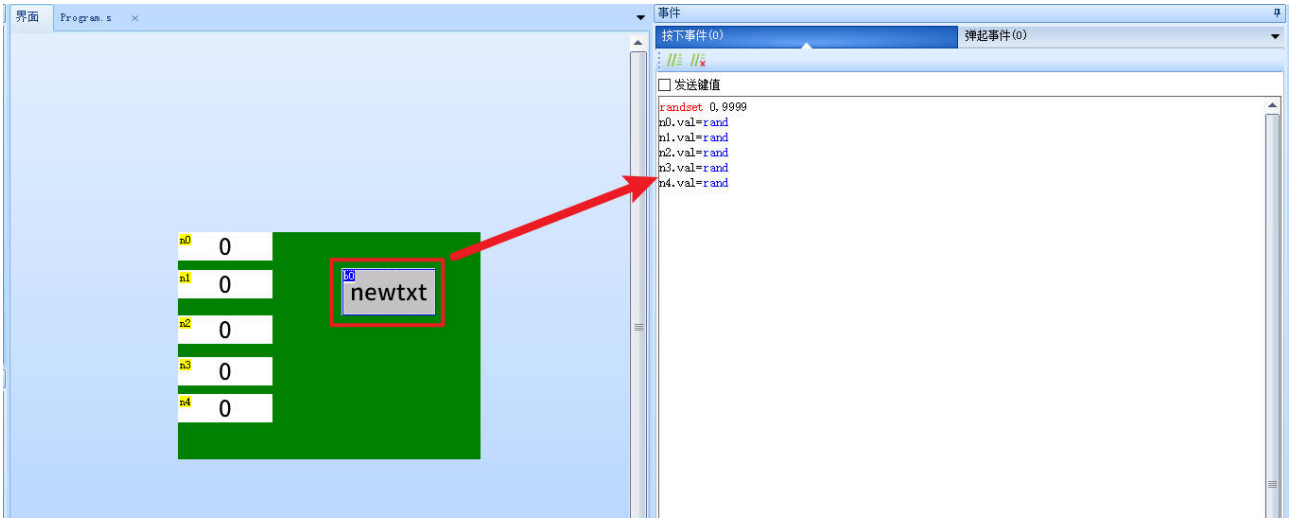
5.1.17 rand-随机数

5.1.17.1 rand-示例

```
//设置随机数取值范围0-100
randset 0,100

//把一个随机数赋值给n0.val变量
n0.val=rand
```




```
//设置随机数取值范围0-9999
randset 0,9999
n0.val=rand
n1.val=rand
n2.val=rand
n3.val=rand
n4.val=rand
//n0,n1,n2,n3,n4大概率是5个毫不相干的值,当然由于概率的原因可能有重复的
```



注意： 1. 使用随机数之前需要先使用 randset 指令设定一次随机数产生范围，如果不设置，默认是最小 0，最大 2147483647。设置完范围以后，每读取一次系统变量 rand 将会得到一个随机数。

2. 使用 randset 指令每设定一次范围，将一直有效，直到重新上电或者设备复位才会恢复默认。

5.1.17.2 rand-相关链接

randset-随机数范围设置 (page 604)

5.1.17.3 rand-样例工程下载

演示工程下载链接：

《randset 指令》演示工程下载

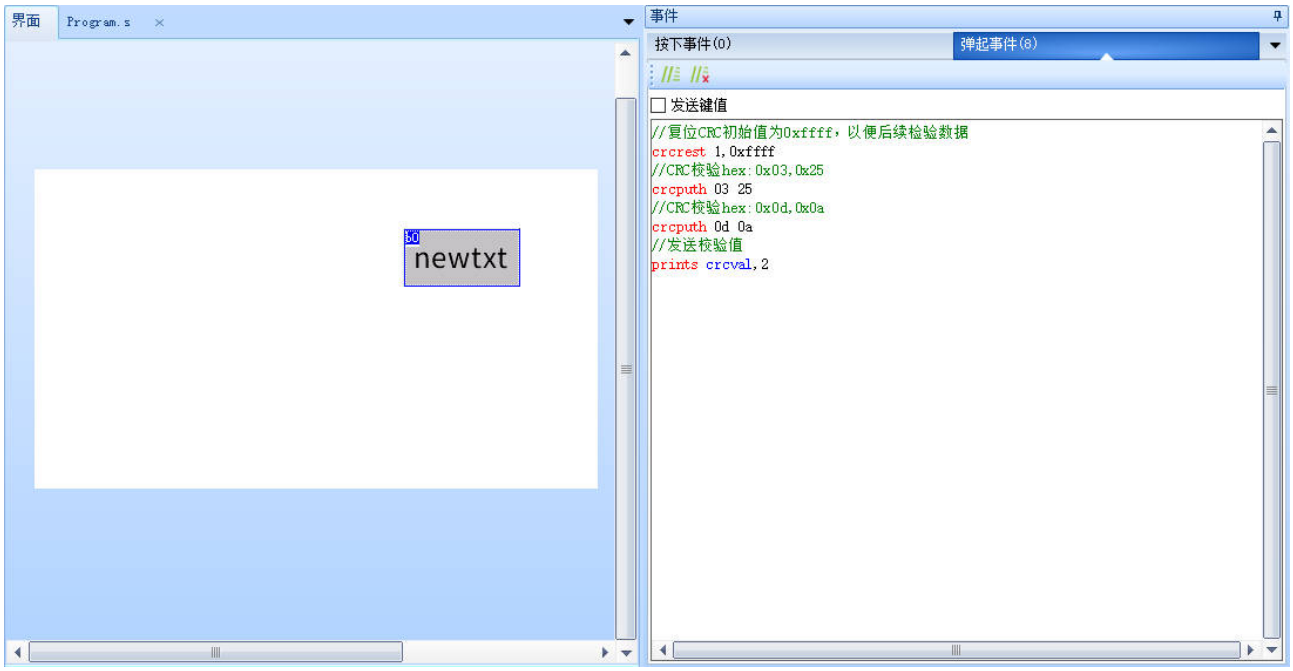
《rand-随机数》演示工程下载

5.1.18 crcval-crc 校验结果

(只可读取不可设置, 使用前请先用 crcrest 指令复位初始值)

5.1.18.1 crcval-示例 1

```
//复位CRC初始值为0xffff, 以便后续检验数据
crcrest 1,0xffff
//CRC校验hex:0x03,0x25
crcputh 03 25
//CRC校验hex:0x0d,0x0a
crcputh 0d 0a
//发送校验值
prints crcval,2
```



注意: 1. 先使用 crcrest 复位 CRC 值，复位之后，可使用 crcputs 或 crcputh 或 crcputu 校验指定数据, 检验完毕读取系统变量 crcval 获得校验结果)

2. 完整的 CRC 校验实例代码请参考: *程序中使用 CRC 校验数据* (page 1018)

5.1.18.2 crcval-相关链接

crcputh-crc 校验一组 Hex (page 713)

crcputs-crc 校验一个变量/常量 (page 709)

crcputu-crc 校验一段串口缓冲区数据 (page 716)

crcrest-复位 crc 初始值 (page 707)

5.1.18.3 crcval-样例工程下载

演示工程下载链接：

《CRC 校验》演示工程下载

《crc 校验例程一对》演示工程下载

5.1.19 rtc0~rtc6-RTC 时钟变量

需要支持 RTC 的型号才能使用。

支持 RTC 的屏幕, 屏幕背面会有电池座用于放 RTC 电池, 确保断电的情况下时钟也能正常工作，默认不带 RTC 电池，需要自己装入 RTC 电池才能在断电时正常走时（型号：CR1220）如何判断屏幕是否支持 *RTC* 实时时钟 (page 1264) 。

需配合文本控件/数字控件/指针控件将 RTC 时间显示到屏幕上

rtc0-rtc6 分别代表年、月、日、时、分、秒、星期

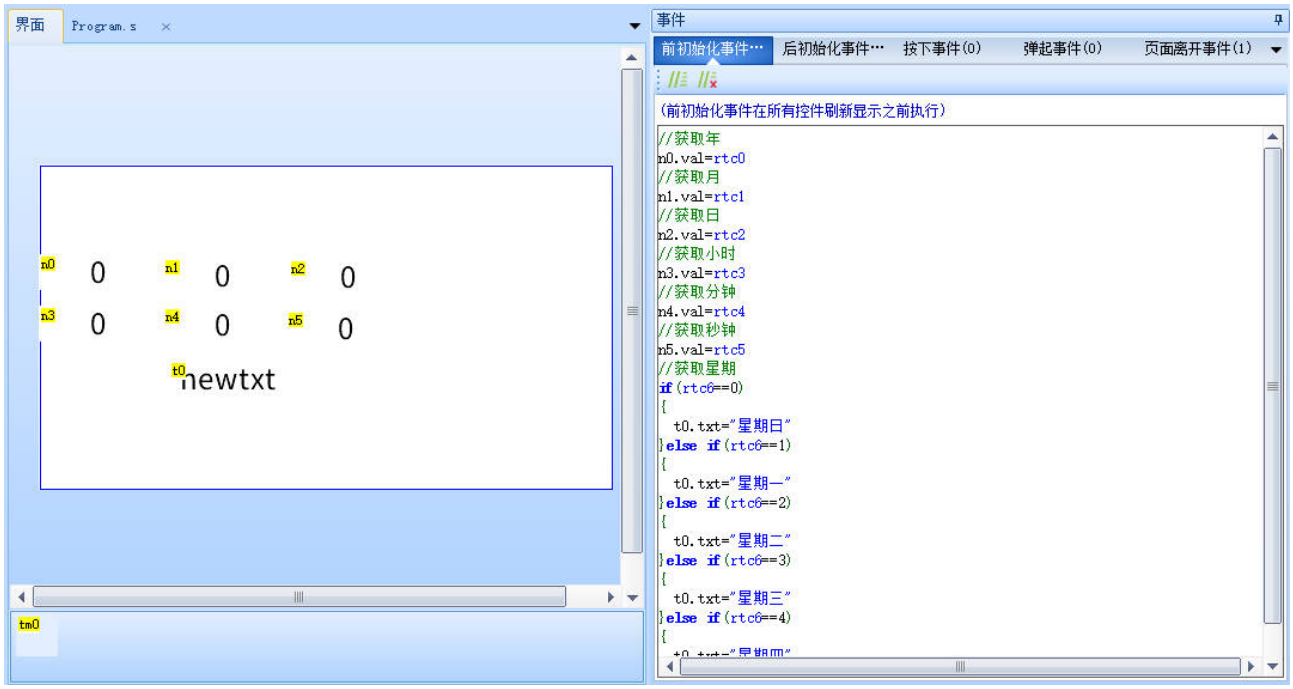
rtc6 中，1-6 分别表示星期一到星期六，0 表示星期天

5.1.19.1 rtc-示例 1

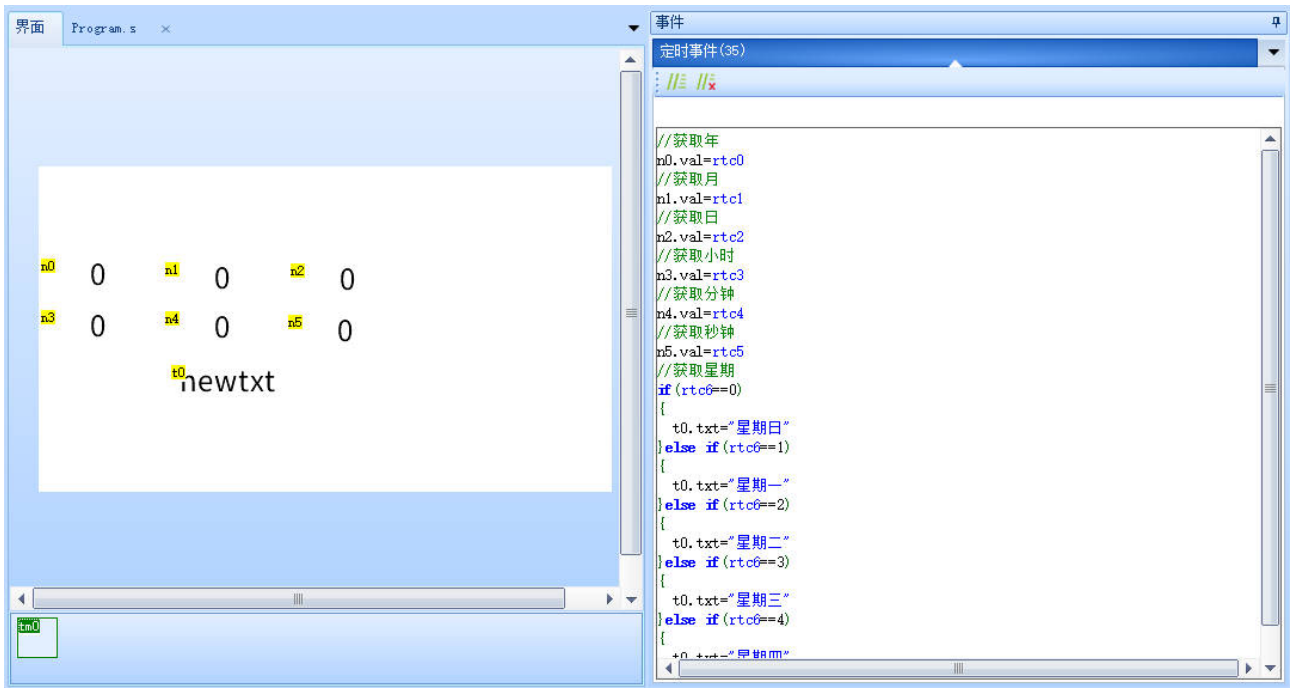
获取 rtc 时间

```
//获取年
n0.val=rtc0

//获取月
n1.val=rtc1

//获取日
n2.val=rtc2

//获取小时
n3.val=rtc3

//获取分钟
n4.val=rtc4

//获取秒钟
n5.val=rtc5
//获取星期
if(rtc6==0)
{
  t0.txt="星期日"
}else if(rtc6==1)
{
  t0.txt="星期一"
}else if(rtc6==2)
{
  t0.txt="星期二"
}else if(rtc6==3)
{
  t0.txt="星期三"
}else if(rtc6==4)
{
  t0.txt="星期四"
}else if(rtc6==5)
{
  t0.txt="星期五"
}else if(rtc6==6)
{
  t0.txt="星期六"
}
```

在页面的前初始化事件中写入以下代码



在页面的定时器事件中写入以下代码



注意： 获取 RTC 时，请将上述代码写在页面前初始化事件和定时器中，定时器时间建议为 300ms（300 比较均匀，太快浪费性能，太慢觉得卡顿）

注意： 如果跳转页面时，显示 RTC 时间的控件会闪一下才变成正确的时间，是因为没有把刷新时间的代码写在页面前初始化事件中，当跳转页面时，需要等到定时器事件到了，才会刷新时间

5.1.19.2 rtc-示例 2

设置 rtc 时间示例：

```
//设置年
rtc0=n0.val

//设置月
rtc1=n1.val

//设置月
rtc2=n2.val

//设置时
rtc3=n3.val

//设置分
rtc4=n4.val

//设置秒
rtc5=n5.val
```

> **注意：** 仅 x5/k0 系列支持 RTC。
>
> rtc0-rtc6 分别表示年，月，日，时，分，秒，星期。rtc6(星期) 为只读，根据当前的年月日自动计算生成。
>
> rtc 晶振的精度为 20ppm

5.1.19.3 rtc-示例 3

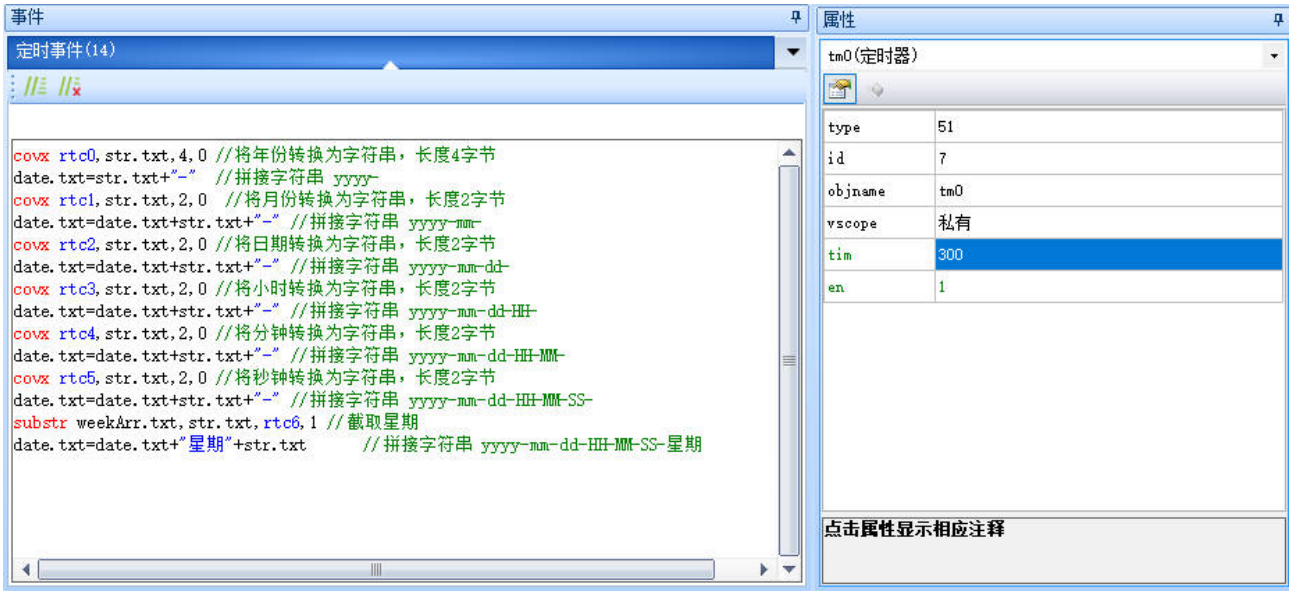
使用文本控件获取 rtc 时间

1、新建一个文本控件 date，date 控件的 txt_maxl 设置为 100。

2、新建两个变量控件，均设置为字符串格式，txt_maxl 设置为 20，两个控件分别改名为 str 和 weekArr，str 的 txt 设置为空，weekArr 的 txt 设置为“日一二三四五六”。

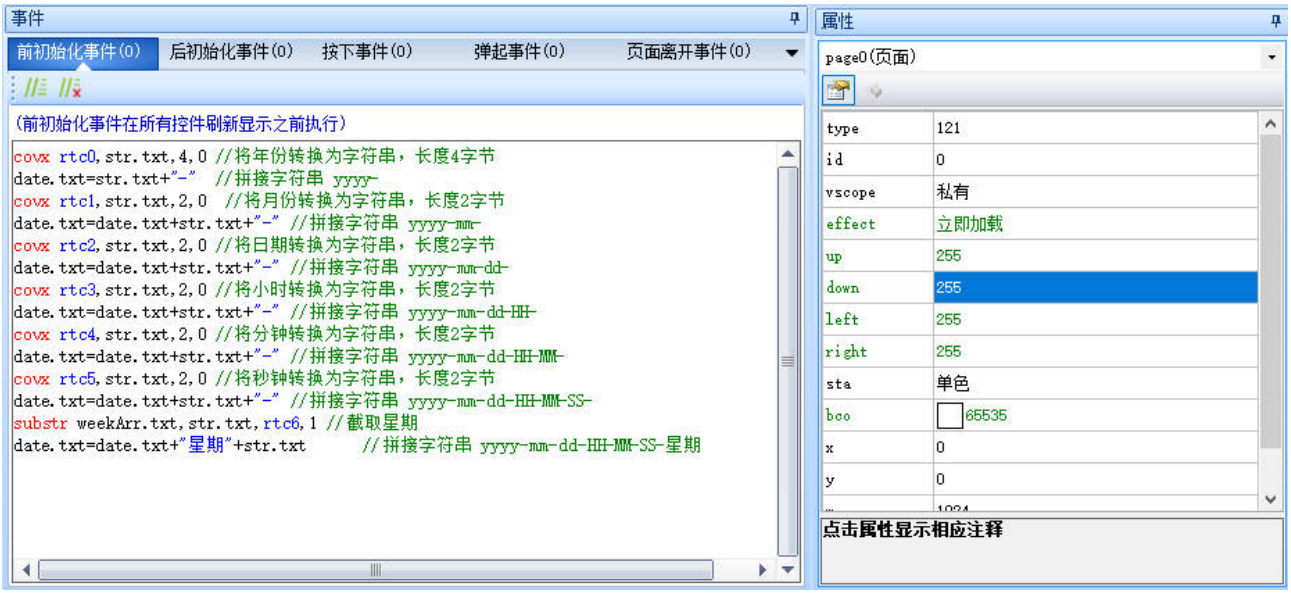
属性
str(变量)
type
52
id
7
objname
str
vscope
私有
sta
字符串
txt
txt_maxl
20
点击属性显示相应注释

3、新建一个定时器，定时器 tim 设置为 300，在定时器内编写以下代码。



```
covx rtc0,str.txt,4,0 //将年份转换为字符串，长度4字节
date.txt=str.txt+"-"  //拼接字符串 yyyy-
covx rtc1,str.txt,2,0  //将月份转换为字符串，长度2字节
date.txt=date.txt+str.txt+"-" //拼接字符串 yyyy-mm-
covx rtc2,str.txt,2,0 //将日期转换为字符串，长度2字节
date.txt=date.txt+str.txt+"-" //拼接字符串 yyyy-mm-dd-
covx rtc3,str.txt,2,0 //将小时转换为字符串，长度2字节
date.txt=date.txt+str.txt+"-" //拼接字符串 yyyy-mm-dd-HH-
covx rtc4,str.txt,2,0 //将分钟转换为字符串，长度2字节
date.txt=date.txt+str.txt+"-" //拼接字符串 yyyy-mm-dd-HH-MM-
covx rtc5,str.txt,2,0 //将秒钟转换为字符串，长度2字节
date.txt=date.txt+str.txt+"-" //拼接字符串 yyyy-mm-dd-HH-MM-SS-
substr weekArr.txt,str.txt,rtc6,1 //截取星期
date.txt=date.txt+"星期"+str.txt       //拼接字符串 yyyy-mm-dd-HH-MM-SS-星期
```

4、把代码复制到“前初始化事件”中，这样一跳转页面才会及时刷新



5.1.19.4 rtc-c 语言示例

单片机通过串口更新串口屏的 RTC 时间

```
printf("rtc0=%d\xff\xff\xff",year);

printf("rtc1=%d\xff\xff\xff",month);

printf("rtc2=%d\xff\xff\xff",day);

printf("rtc3=%d\xff\xff\xff",hour);

printf("rtc4=%d\xff\xff\xff",minute);

printf("rtc5=%d\xff\xff\xff",second);
```

针对没有 RTC 的屏幕，也可以直接给对应的数字变量赋值即可

```
printf("date.year.val=%d\xff\xff\xff",year);       //给date页面的year控件的val属性赋值
printf("date.month.val=%d\xff\xff\xff",month);     //给date页面的month控件的val属性赋值
printf("date.day.val=%d\xff\xff\xff",day);         //给date页面的day控件的val属性赋值
printf("date.hour.val=%d\xff\xff\xff",hour);       //给date页面的hour控件的val属性赋值
```

(续下页)

(接上页)

```
printf("date.minute.val=%d\xff\xff\xff",minute);   //给date页面的minute控件的val属性赋值
printf("date.second.val=%d\xff\xff\xff",second);   //给date页面的second控件的val属性赋值
```

也可以直接给文本控件赋值

```
printf("date.time.txt=\"%d-%d-%d %d:%d:%d\"\xff\xff\xff",year,month,day,hour,minute,
↪second);     //给date页面的time控件的txt属性赋值
```

rtc-相关链接

校准 *RTC* 时钟 (page 161)

RTC 电池型号 (page 162)

如何判断屏幕是否支持 *RTC* 实时时钟 (page 1264)

5.1.19.5 rtc-样例工程下载

演示工程下载链接：

《RTC 时钟》演示工程下载

5.1.20 pio0~pio7-扩展 IO 端口

注意： 具体的使用方法请参考:ref: *拓展 IO 资料*

支持的型号：

x5 系列支持 8 路 IO

k0 系列支持 8 路 IO

X2 系列部分型号支持 8 路 IO

使用 pio 端口之前一定要先使用*cfgpio-扩展 IO 模式配置* (page 650) 配置好 IO 模式，没有使用 cfgpio 指令配置前读写 IO 是不准确的。

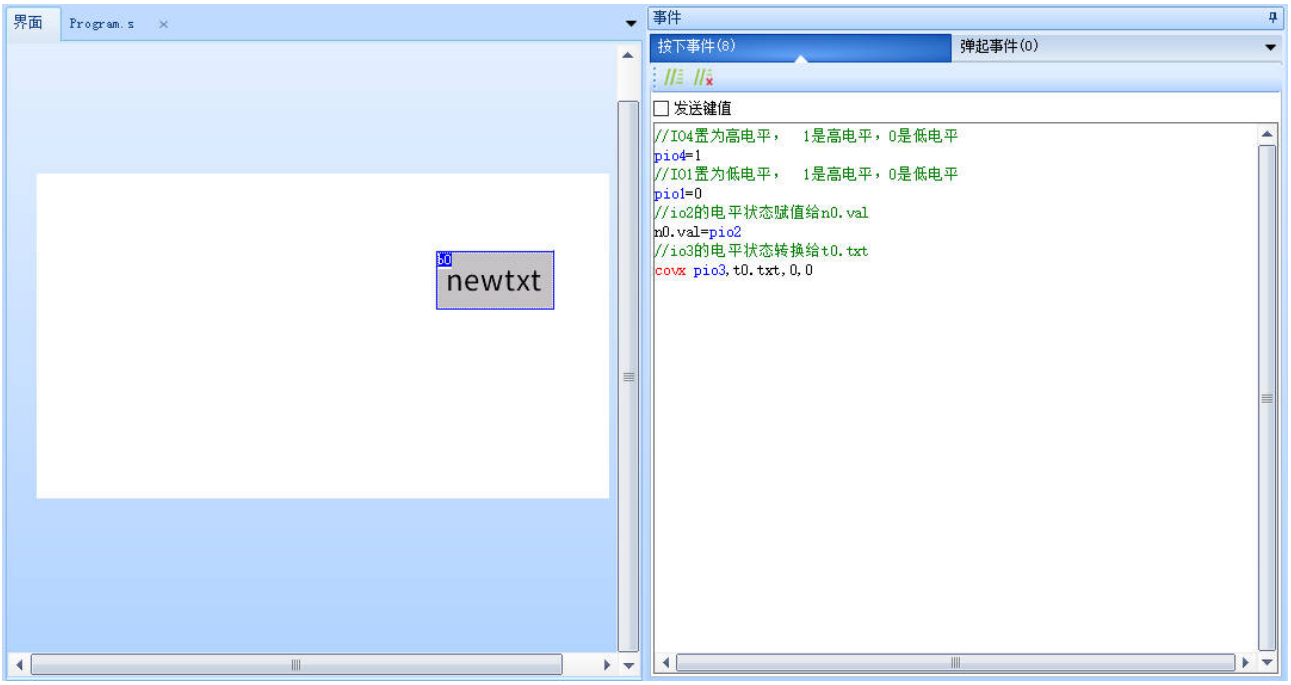
上电默认所有扩展 IO 模式为上拉输入（内部上拉电阻为 50K）。

模拟器上没有外部 IO，该指令只能在真机上测试，不保证在模拟器上的效果。

每个页面都需要重新用 cfgpio 绑定对应的引脚。

5.1.20.1 pio-示例

```
//IO4置为高电平，  1是高电平，0是低电平
pio4=1

//IO1置为低电平，  1是高电平，0是低电平
pio1=0

//io2的电平状态赋值给n0.val
n0.val=pio2

//io3的电平状态转换给t0.txt
covx pio3,t0.txt,0,0
```



5.1.20.2 pio-相关链接

拓展 IO 资料 (page 1030)
拓展 IO 相关问题 (page 1258)
cfgpio-扩展 IO 模式配置 (page 650)
pwm4~pwm7-扩展 IO 占空比 (page 804)
pwmf-PWM 输出的频率 (page 806)
sleep 睡眠模式与拓展 IO (page 1041)

5.1.20.3 pio-样例工程下载

演示工程下载链接：

《拓展 IO 例程》演示工程下载

《IOtest》演示工程下载

5.1.21 pwm4~pwm7-扩展 IO 占空比

注意： 具体的使用方法请参考:ref: *拓展 IO 资料*

支持的型号：

x5 系列支持 8 路 IO

k0 系列支持 8 路 IO

X2 系列部分型号支持 8 路 IO

K0/X2 系列仅 io4-io7 支持 PWM，X5 系列仅 io7 支持 PWM

占空比最小值 0 最大值 100, 上电默认 50。

pwm4-pwm7 依次对应扩展 IO 中的 io4-io7,

设置好 PWM 占空比以后, 需要使用 cfgpio 指令配置此 IO 的模式为 PWM 输出模式，相应 IO 才会开始输出 PWM, 配置完 PWM 模式后在 PWM 输出的过程中可以随时修改占空比，不用重新配置。

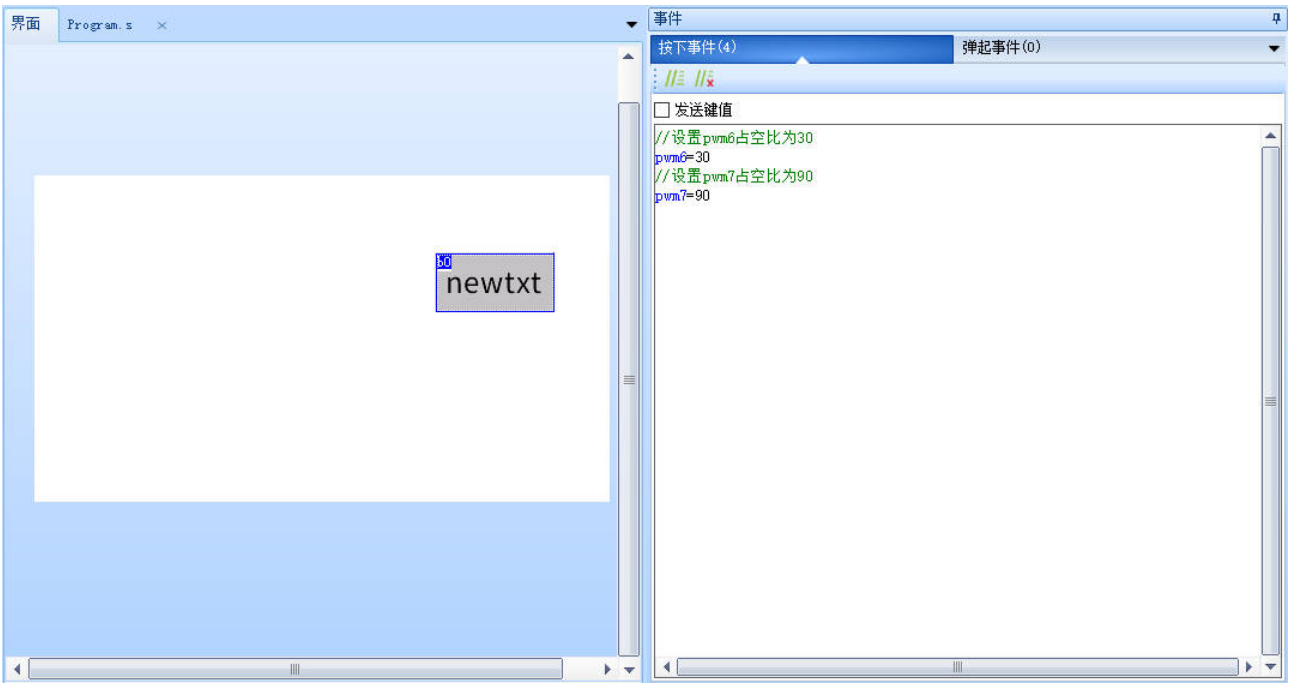
上电默认所有扩展 IO 模式为上拉输入（内部上拉电阻为 50K）。

模拟器上没有外部 IO，该指令只能在真机上测试，不保证在模拟器上的效果。

每个页面都需要重新用 cfgpio 绑定对应的引脚。

5.1.21.1 pwm-示例

```
//设置pwm6占空比为30
pwm6=30

//设置pwm7占空比为90
pwm7=90
```



5.1.21.2 pwm-相关链接

拓展 IO 相关问题 (page 1258)
拓展 IO 资料 (page 1030)
cfgpio-扩展 IO 模式配置 (page 650)
pio0~pio7-扩展 IO 端口 (page 802)
pwmf-PWM 输出的频率 (page 806)
sleep 睡眠模式与拓展 IO (page 1041)

5.1.21.3 pwm-样例工程下载

演示工程下载链接：
《拓展 IO 例程》演示工程下载
《IOtest》演示工程下载

5.1.22 pwmf-PWM 输出的频率

注意:

支持的型号:

x5 系列支持 8 路 IO

k0 系列支持 8 路 IO

X2 系列部分型号支持 8 路 IO

频率单位为:HZ, 范围为最小 1，最大 65535HZ，上电默认 1000HZ。

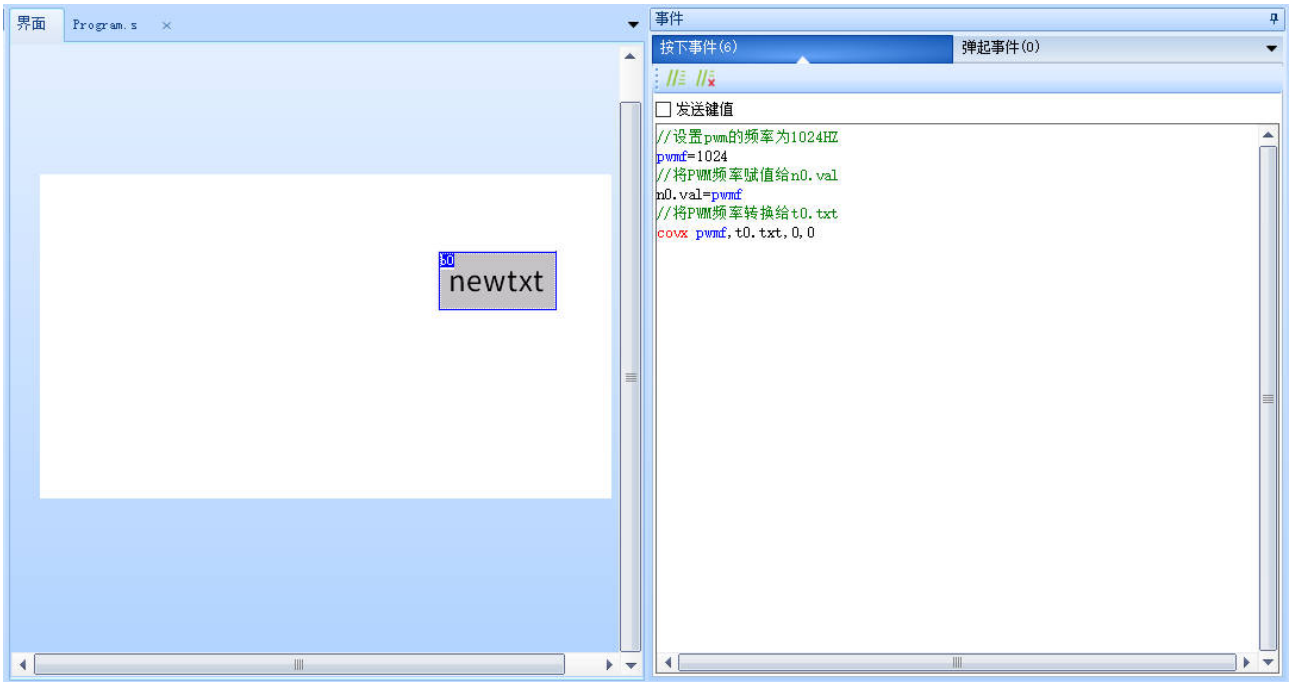
所有 PWM 输出统一为一个频率，不可单独设置。

模拟器上没有外部 IO，该指令只能在真机上测试，不保证在模拟器上的效果

具体的使用方法请参考*拓展 IO 资料* (page 1030)

5.1.22.1 pwf-示例

```
//设置pwm的频率为1024HZ
pwmf=1024

//将PWM频率赋值给n0.val
n0.val=pwmf

//将PWM频率转换给t0.txt
covx pwmf,t0.txt,0,0
```



5.1.22.2 pwmf-相关链接

拓展 IO 相关问题 (page 1258)
拓展 IO 资料 (page 1030)
cfgpio-扩展 IO 模式配置 (page 650)
pio0~pio7-扩展 IO 端口 (page 802)
pwm4~pwm7-扩展 IO 占空比 (page 804)
sleep 睡眠模式与拓展 IO (page 1041)

5.1.22.3 pwmf-样例工程下载

演示工程下载链接：
《拓展 IO 例程》演示工程下载
《IOtest》演示工程下载

5.1.23 audio0~audio1-音频通道控制

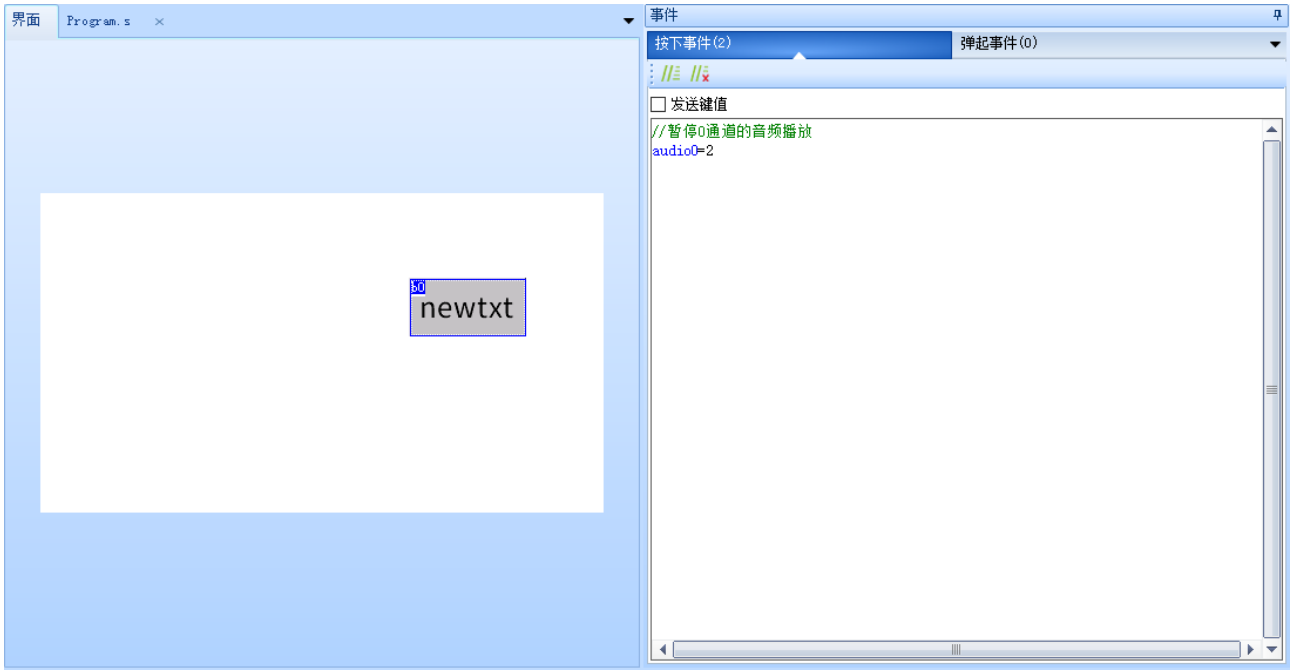
音频通道控制

仅带喇叭的型号（例如 X3、X5 系列）支持

0-停止;1-播放;2-暂停

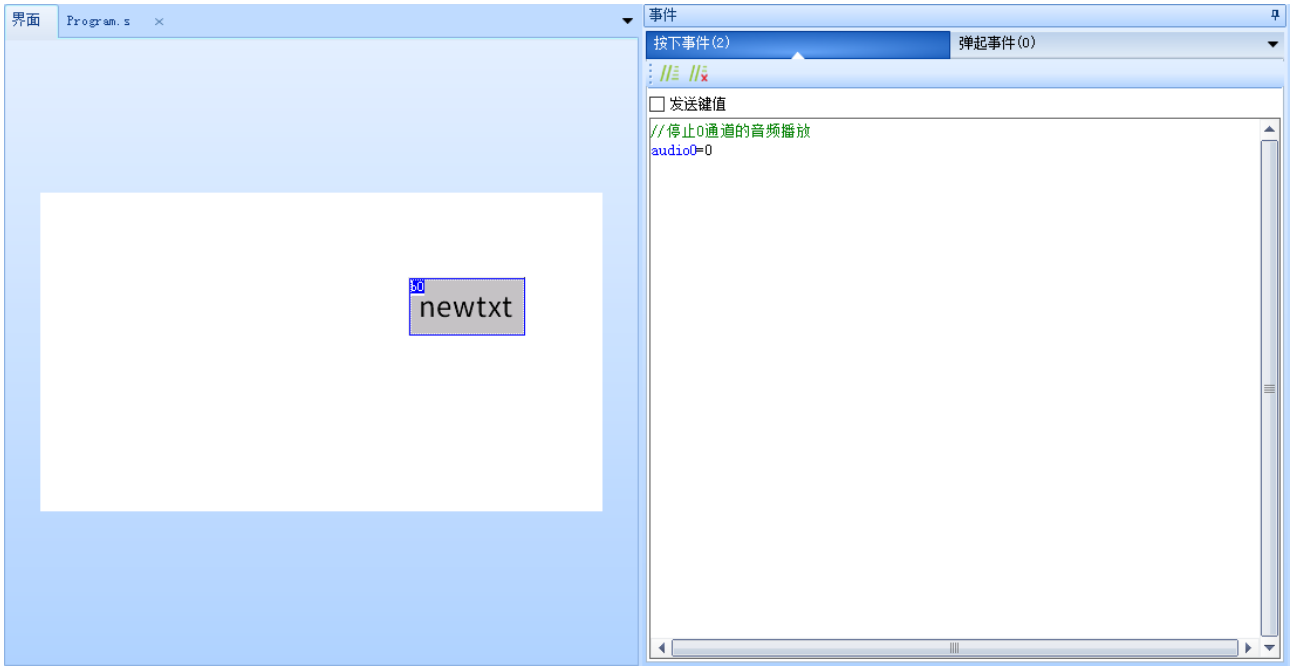
5.1.23.1 audio-示例 1

```
//暂停0通道的音频播放
audio0=2
```



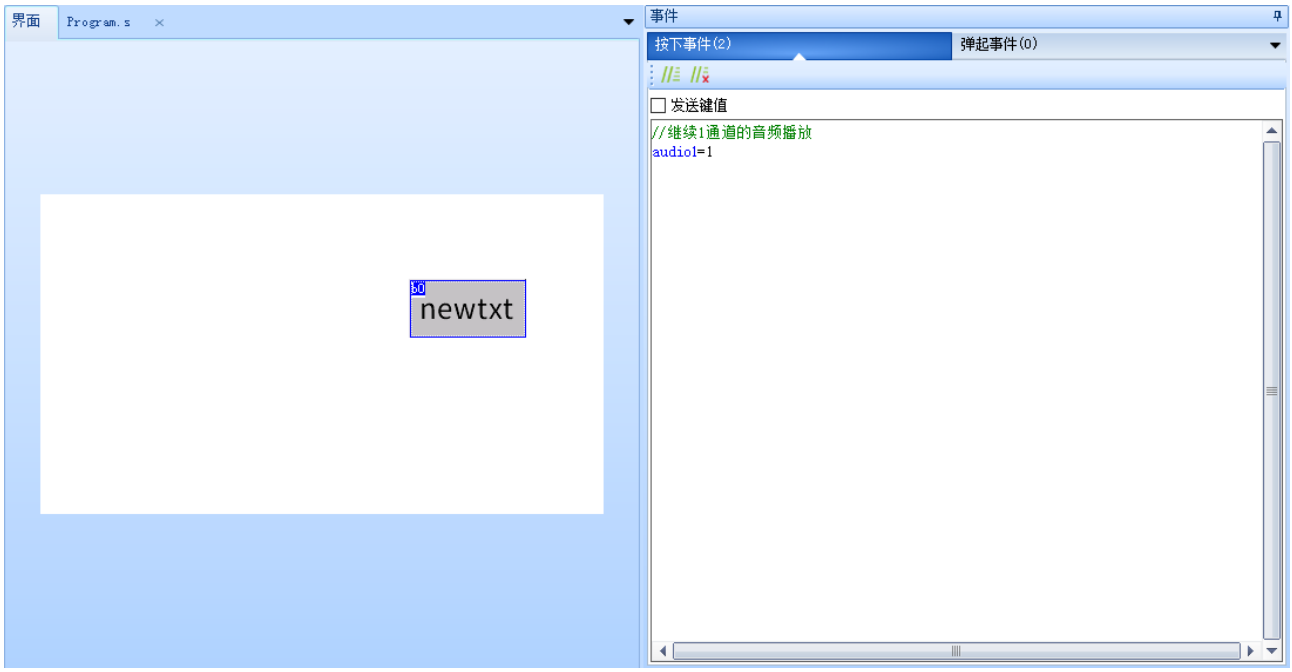
5.1.23.2 audio-示例 2

```
//停止0通道的音频播放
audio0=0
```



5.1.23.3 audio-示例 3

```
//继续1通道的音频播放
audio1=1
```



注意： 仅 x5 x3 系列支持

play 指令用于配置和启动音频播放, 系统变量 audio0,audio1 用于控制通道状态。

只有当通道状态在暂停的时候，才能配置为继续播放。如果通道状态为停止，将不能配置为继续播放，需要使用 play 指令来配置并启动播放。

音频播放功能是全局的，不属于某个页面，因此 play 指令启动播放后，即便是跳转页面，音频依然会继续播

放，如果希望离开页面后停止播放，可以在页面的离开事件中使用 audio0/audio1 系统变量来关闭或暂停指定通道的音频播放状态。

5.1.23.4 audio-相关链接

play-音频播放 (page 659)

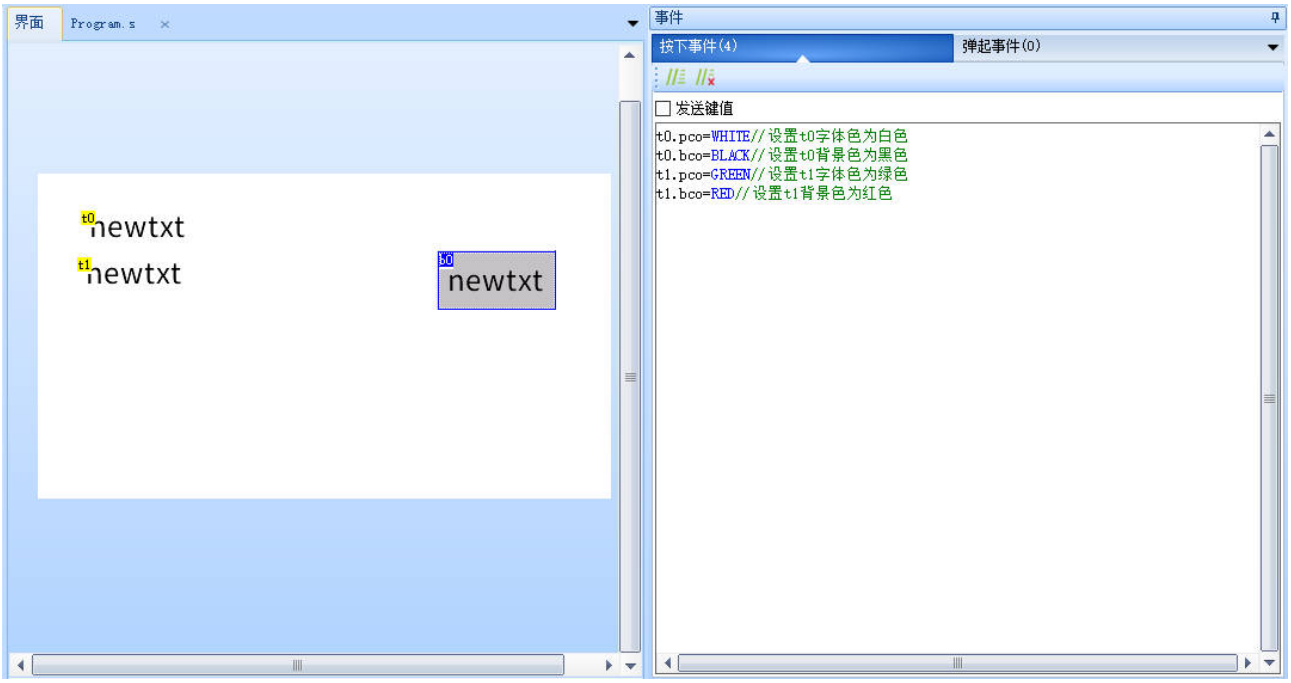
5.1.24 串口 HMI 颜色代号表

表 5: 所有代号的书写均为大写

代号	10 进制	所表示的颜色
RED	63488	红色
BLUE	31	蓝色
GRAY	33840	灰色
BLACK	0	黑色
WHITE	65535	白色
GREEN	2016	绿色
BROWN	48192	橙色
YELLOW	65504	黄色

5.1.24.1 hmi_color-示例 1

```
t0.pco=WHITE//设置t0字体色为白色
t0.bco=BLACK//设置t0背景色为黑色
t1.pco=GREEN//设置t1字体色为绿色
t1.bco=RED//设置t1背景色为红色
```



5.2 不常用变量

5.2.1 scache-串口发送缓冲区使能

仅 X2、X3、X5 系列支持

本指令较少使用，常规情况下请勿配置

串口发送缓冲区使能, 默认情况下为关, 如需开启建议配置在 program.s 中

scache=0 的时候，显示屏是阻塞每一条发送命令（prints，printh 等命令），数据发送完成才执行下一条命令。

scache=1 的话，启用缓冲。执行发送命令会把数据写入缓冲区，后台 DMA 发送数据。

> **注意：** page 指令后面的代码将不会被执行，所以千万不要把代码写在 page 指令后面

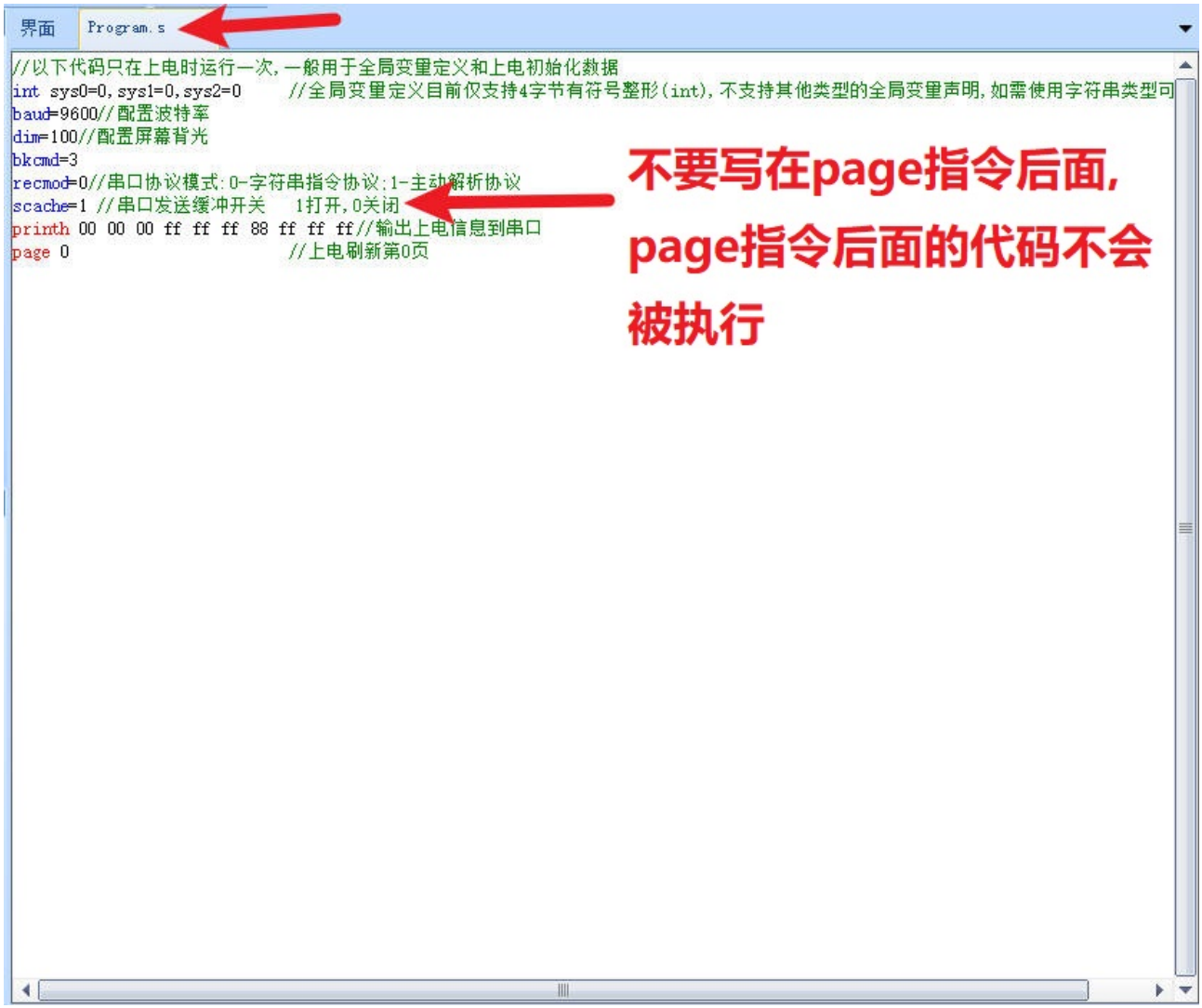
> **小心：** page 指令后面的代码将不会被执行，所以千万不要把代码写在 page 指令后面

提示： page 指令后面的代码将不会被执行，所以千万不要把代码写在 page 指令后面

重要： page 指令后面的代码将不会被执行，所以千万不要把代码写在 page 指令后面

5.2.1.1 scache-打开串口发送缓冲区使能示例

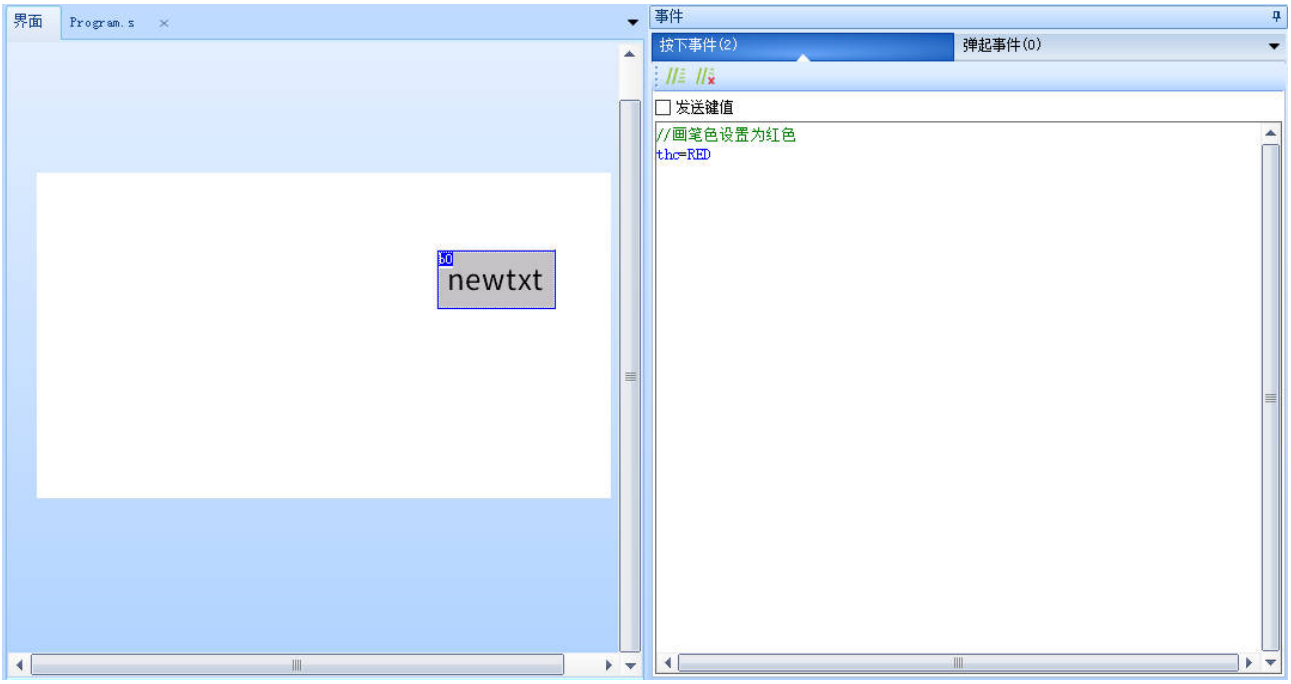
```
scache=1     //打开串口发送缓冲区使能
```


界面
Program.s
//以下代码只在上电时运行一次，一般用于全局变量定义和上电初始化数据
int sys0=0,sys1=0,sys2=0 //全局变量定义目前仅支持4字节有符号整形(int)，不支持其他类型的全局变量声明，如需使用字符串类型可
baud=9600//配置波特率
dim=100//配置屏幕背光
bkcmd=3
recmod=0//串口协议模式：0-字符串指令协议；1-主动解析协议
scache=1 //串口发送缓冲开关 1打开，0关闭
printh 00 00 00 ff ff ff 88 ff ff ff//输出上电信息到串口
page 0 //上电刷新第0页
不要写在page指令后面，
page指令后面的代码不会
被执行

5.2.2 thc-触摸绘图时的画笔色

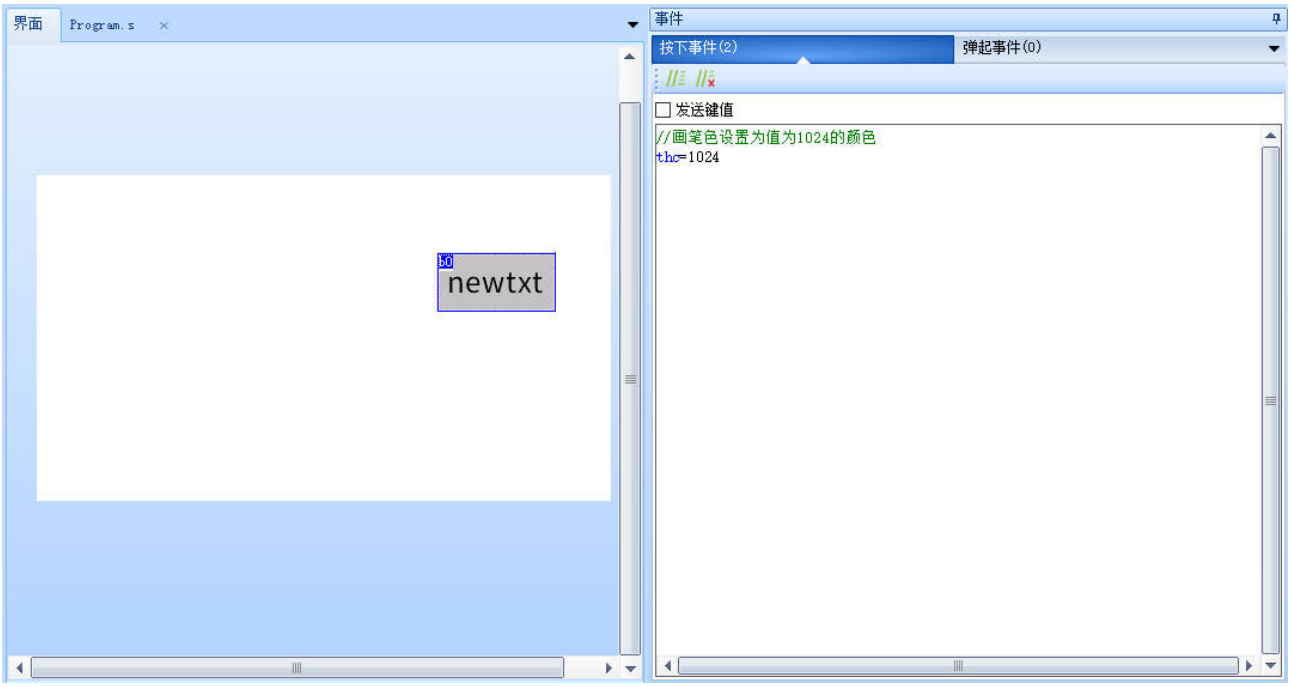
5.2.2.1 thc-示例 1

```
//画笔色设置为红色
thc=RED
```



5.2.2.2 thc-示例 2

```
//画笔色设置为值为1024的颜色
thc=1024
```

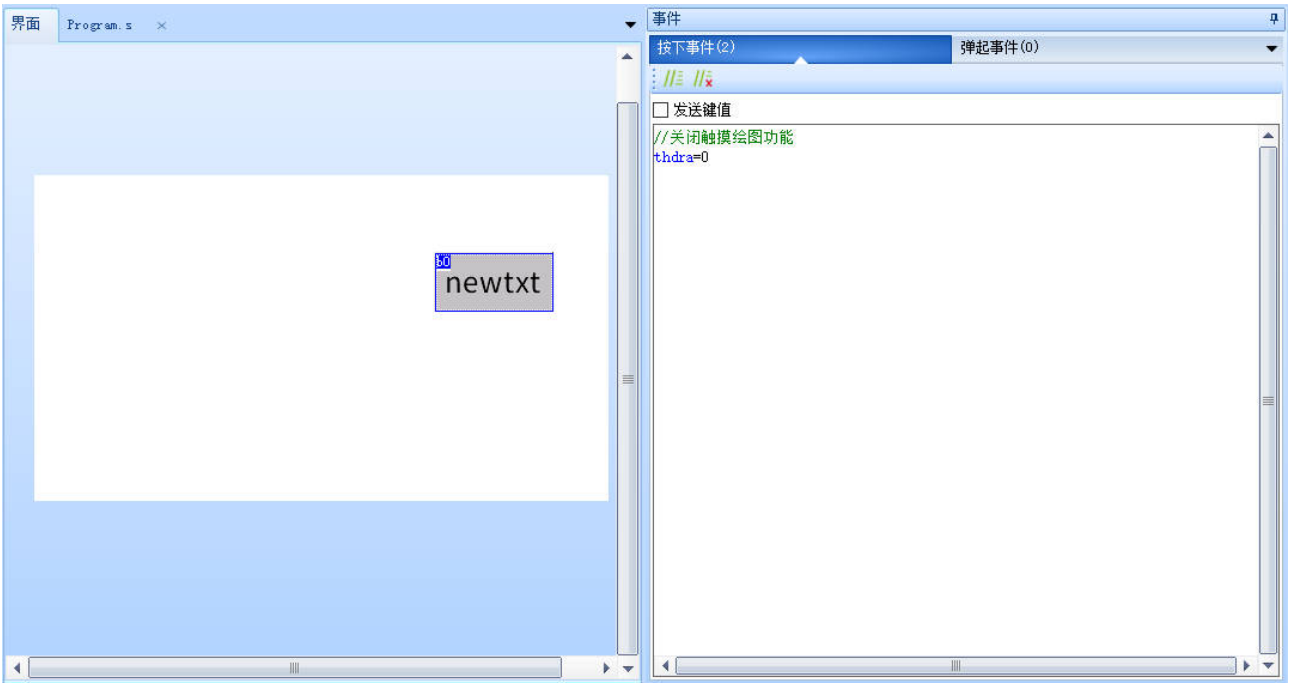


5.2.3 thdra-触摸绘图功能开关

提示： 可以用此指令来测试触摸屏幕屏是否工作正常以及触摸屏（尤其是电阻屏）是否存在触摸偏移
thdra 不受 tsw 指令控制，哪怕使用 tsw 关闭所有触摸，thdra 开启的情况下依然可以画点

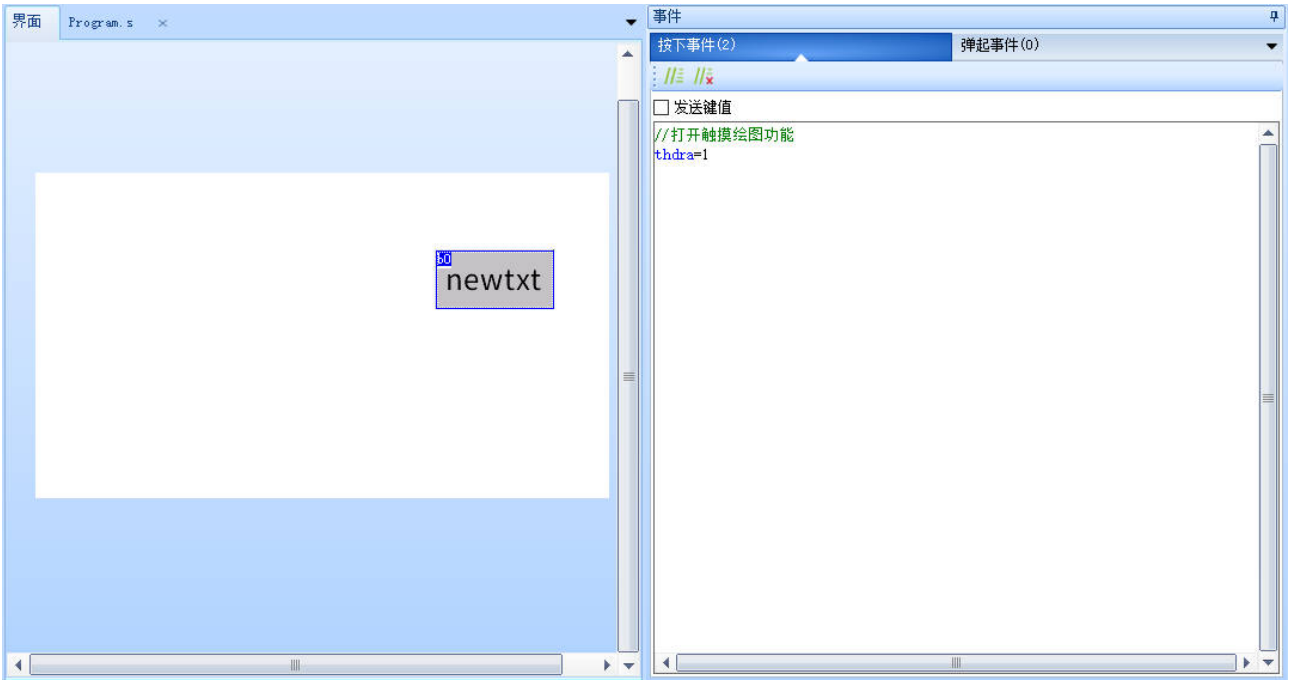
5.2.3.1 thdra-示例 1

```
//关闭触摸绘图功能
thdra=0
```



5.2.3.2 thdra-示例 2

```
//打开触摸绘图功能
thdra=1
```


界面
Program.s
50
newtxt
事件
按下事件(2)
弹起事件(0)
发送键值
//打开触摸绘图功能
thdra=1

5.2.4 tch0~tch3-实时触摸坐标

tch0: 当前触摸坐标 X

tch1: 当前触摸坐标 Y

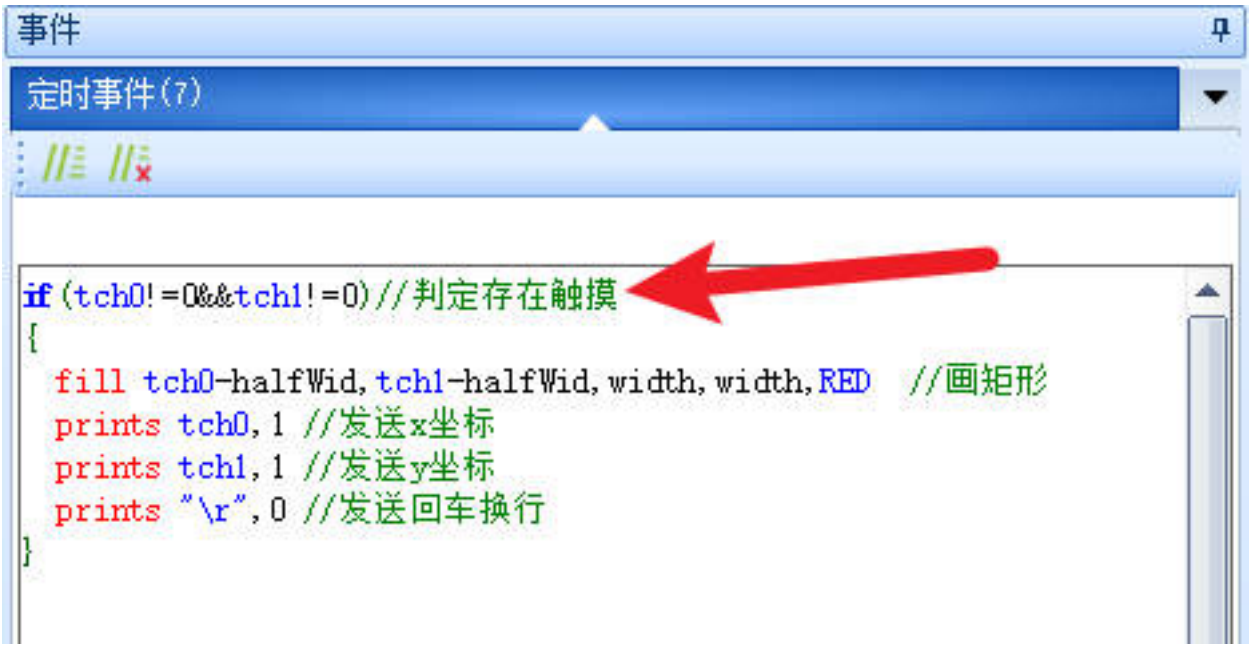
tch2: 上一次按下时的坐标 X

tch3: 上一次按下时的坐标 y

注意： 触摸坐标只能读取，不能赋值，没有按下时，实时坐标数据为 0。

注意： 通常在定时器或者控件的按下事件中读取 (弹起事件中不会更新 tch0 和 tch1, 但会更新 tch2 和 tch3)

注意： 一般情况下只判断 tch0 和 tch1, 建议使用 tch0 和 tch1 前请先判断 tch0 和 tch1 不为 0



注意： halfwid 为正方形边长的一半,width 为正方形的边长

5.2.4.1 tch0~tch3-样例工程下载

演示工程下载链接：

《T1 系列滑动翻页》演示工程下载

《tch0~tch3-实时触摸坐标》演示工程下载

5.2.5 addr-设备地址

给串口屏配置一个地址，可以通过地址来将数据发给不同的串口屏，该指令仅对串口屏实物有效，对模拟器无效，模拟器的地址始终为 0

注意: 如果你只接了 1 个屏，不建议配置设备地址。

对于 k 系列和 T 系列，断电重启依然有效。对于 X 系列（X2，X3，X5），addr 不支持掉电保存，需要在 program.s 中进行配置

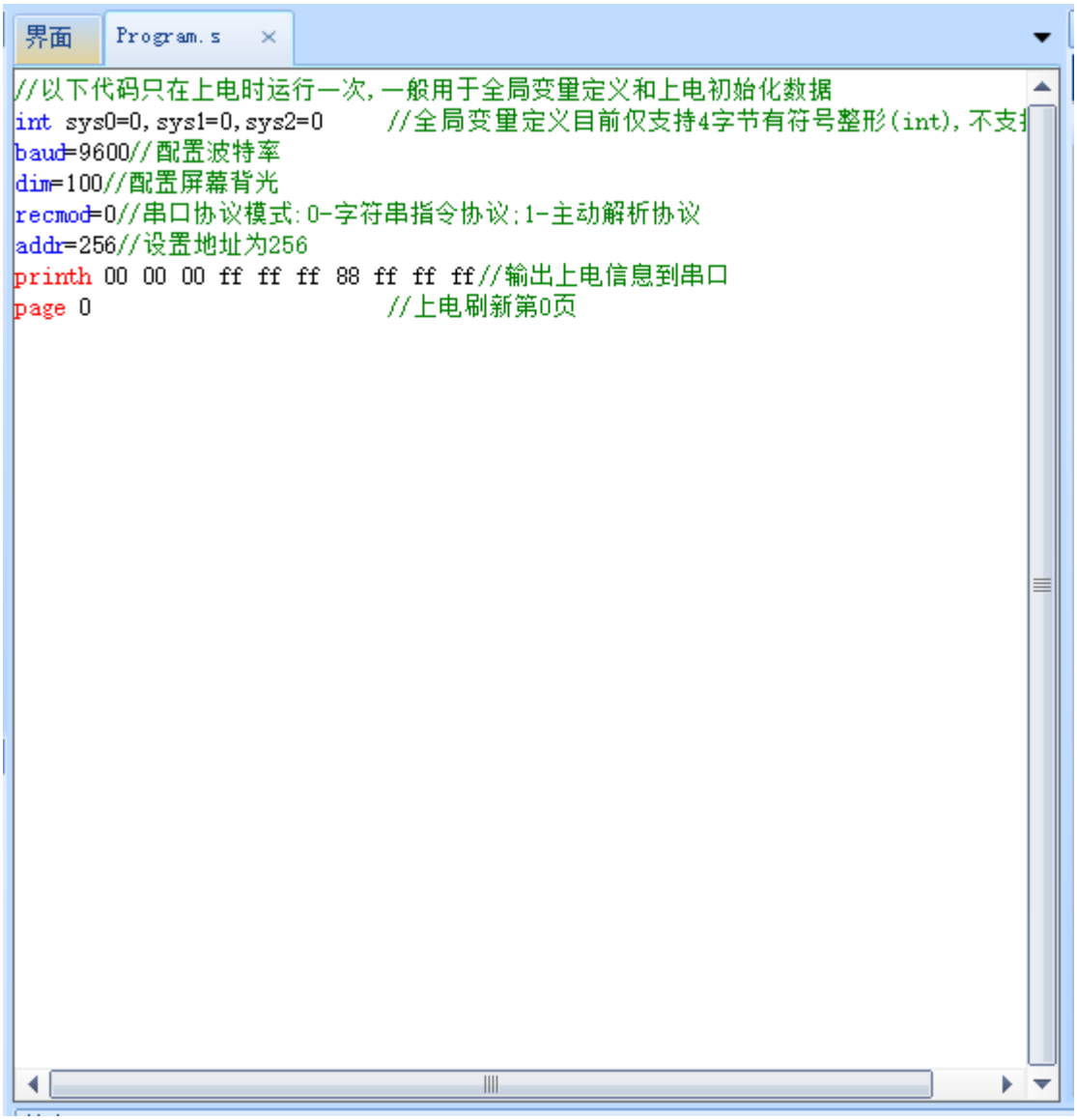
有效地址范围为 256-2815，(即 0x0100-0x0aff),0 为无地址,65535 为广播地址，广播地址只能用于广播数据，不能配置某个设备为广播地址，出厂默认地址为 0, 即没有地址。

向一个有地址的设备发送指令时，需要在指令前加上 2 字节的地址数据，以 hex 方式发送,2 字节小端模式, 比如设备配置的地址为 addr=256, 那么发送给他指令时需要在指令前面增加两个字节:0x00 0x01(注意，配置的时候是 0x0100, 发送指令的时候是低位在前，所以是 0x00 0x01 跟配置的写法是相反的)。

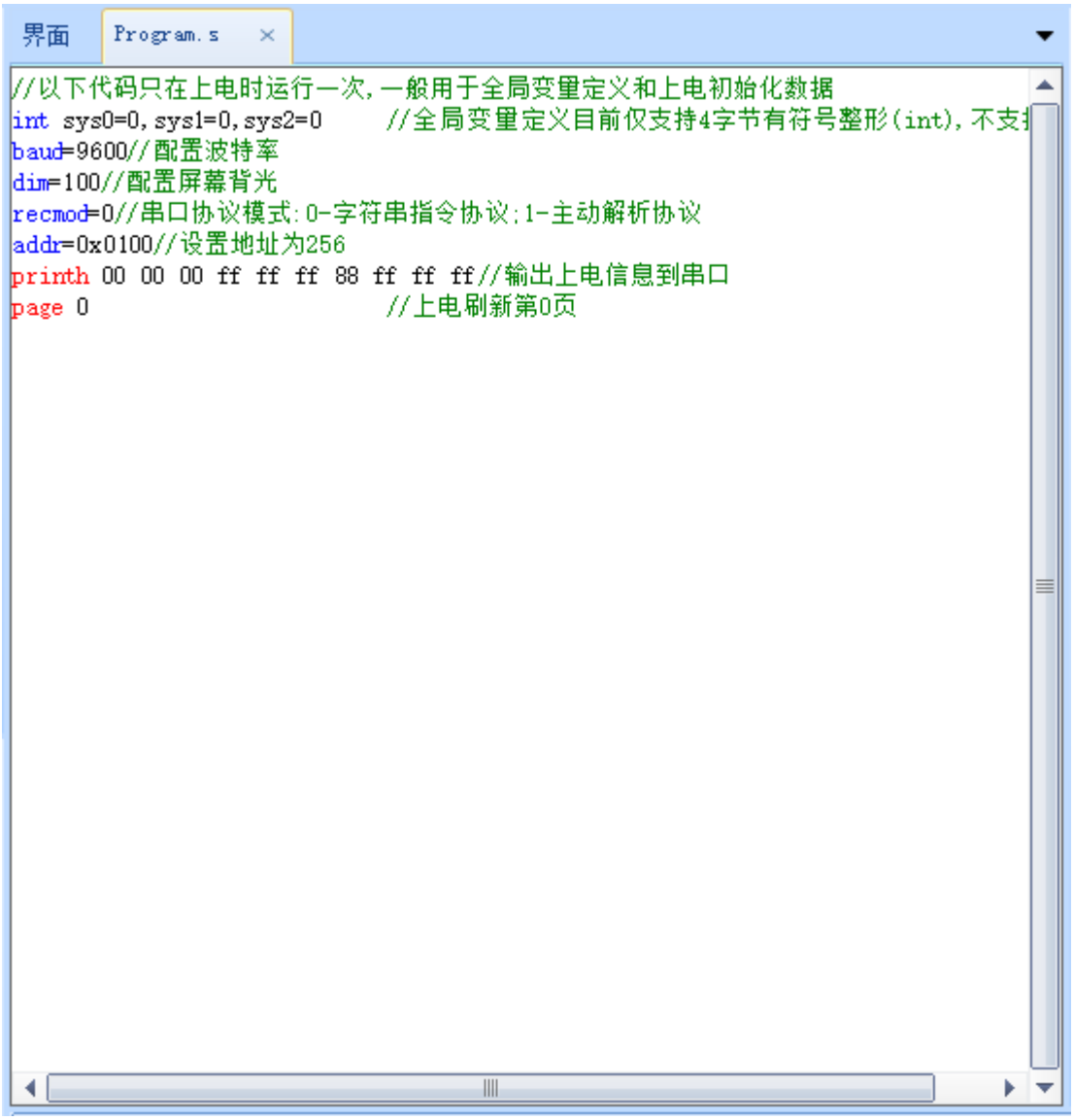
该配置只能在实物上有效，模拟器是无法测试的。

5.2.5.1 addr-示例

```
//十进制写法
addr=256
```




```
//十六进制写法
addr=0x0100
```

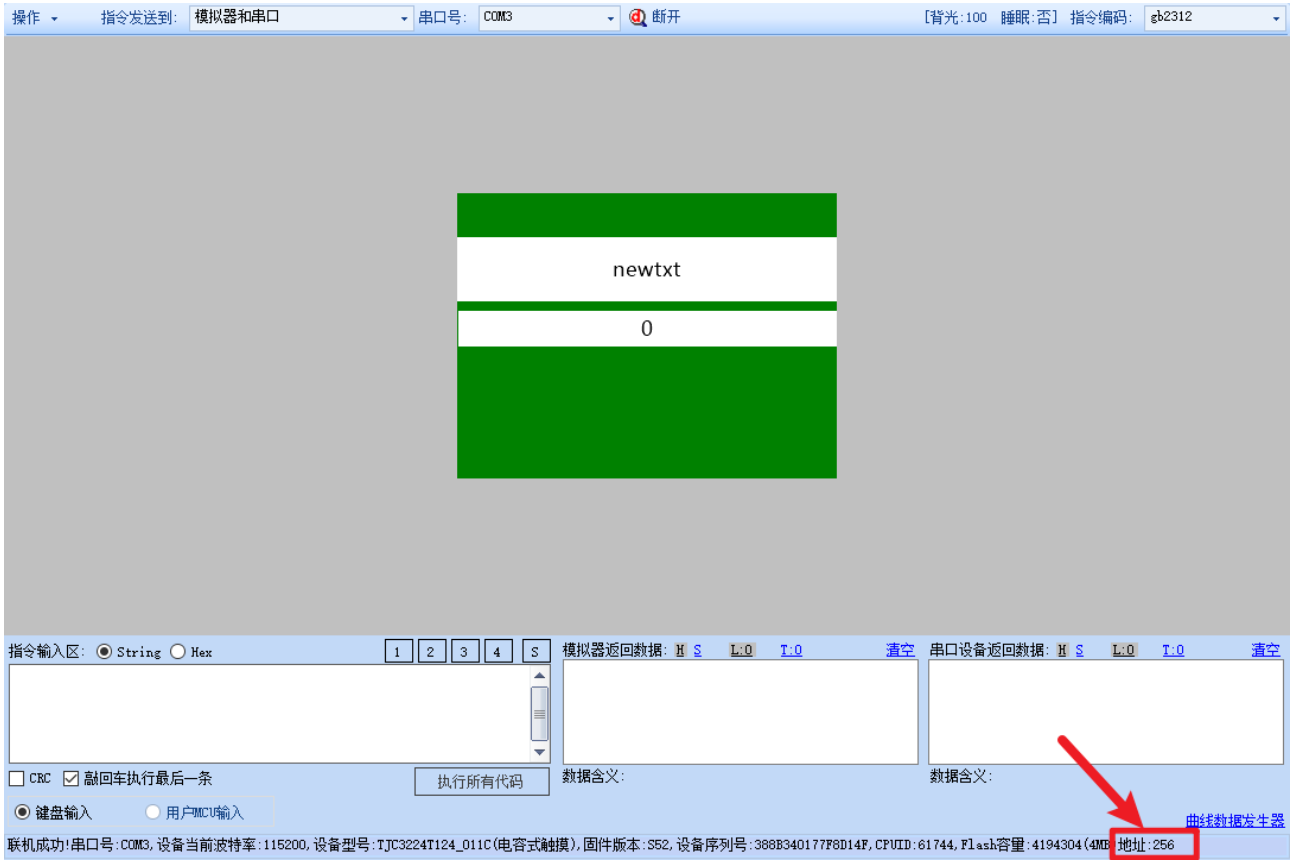


以上两条写法是同一个意思，配置的是同一个地址，配置之后有断电保存功能。

当用户想一个串口同时控制多个串口屏独立工作的时候，可以给每个屏设置不同的地址。(注意，在 TTL/RS232 通信时不能直接将多个触摸屏的 TX 连接到一起，这样会导致短路。但是可以将多个触摸屏的 RX 连接到一起，接用户设备串口的 TX。)

或者用户使用 485 总线与屏幕通信，且 485 总线上存在多个设备的时候，可以给屏幕设置地址。

配置了地址的屏幕在联机时会显示地址



例如，现在将两个触摸屏的 RX 并联到一起，然后与电脑串口（TTL 电平）的 TX 连接。设置串口屏 A 的地址 0x100(addr=256)，设置串口屏 B 的地址为 0x200（addr=512)。

单片机 c 语言操作串口发送命令详解：(\x 开头为十六进制数据)

发送给地址 0x0100 的代码如下：

```
//控制串口屏A页面上的t0控件显示内容为：addr is 256
printf("\x00\x01t0.txt=\"addr is 256\"\xFF\xFF\xFF")
```

单片机上发出的数据为：

```
H:00 01 74 30 2E 74 78 74 3D 22 61 64 64 72 20 69 73 20 32 35 36 22 FF FF FF
S:    t  0  .  t  x  t  =  "  a  d  d  r     i  s     2  5  6  "
```

发送给地址 0x0200 的代码如下：

```
//控制串口屏B页面上的t0控件显示内容为：addr is 512
printf("\x00\x02t0.txt=\"addr is 512\"\xFF\xFF\xFF")
```

单片机上发出的数据为：

```
H:00 02 74 30 2E 74 78 74 3D 22 61 64 64 72 20 69 73 20 35 31 32 22 FF FF FF
S:    t  0  .  t  x  t  =  "  a  d  d  r     i  s     5  1  2  "
```

广播发送（所有串口屏都能收到, 地址为:FFFF）的代码如下：

```
//控制所有串口屏页面上的t0控件显示内容为：broadcast addr
printf("\xFF\xFFt0.txt=\"broadcast addr\"\xFF\xFF\xFF")
```

单片机上发出的数据为：

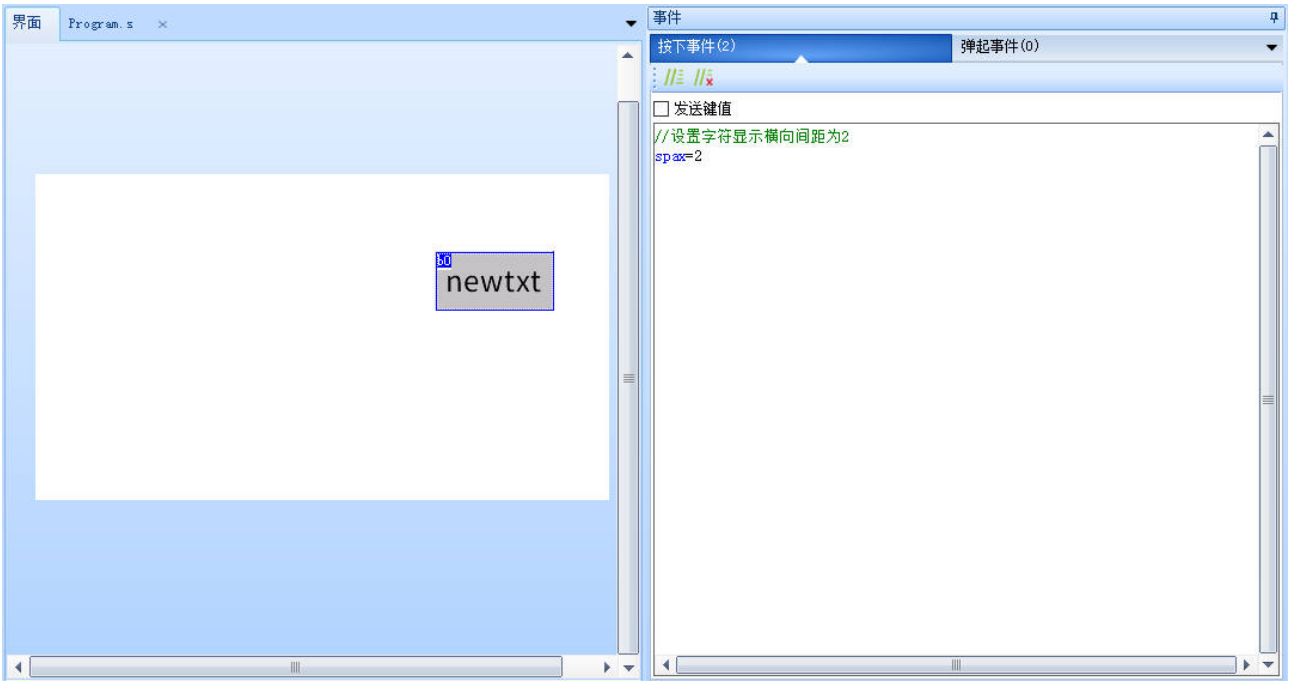
```
H:FF FF 74 30 2E 74 78 74 3D 22 62 72 6F 61 64 63 61 73 74 20 61 64 64 72 22 FF FF FF
S:    t 0 .  t x t = ″ b r o a d c a s t   a d d r ″
```

5.2.6 spax-字符显示横向间距

5.2.6.1 spax-示例

```
//设置字符显示横向间距为2
spax=2
```

注意： 上电默认为 0，仅对 xstr 指令写出来的字符有效，控件带的字符显示间距由控件内部的属性决定。

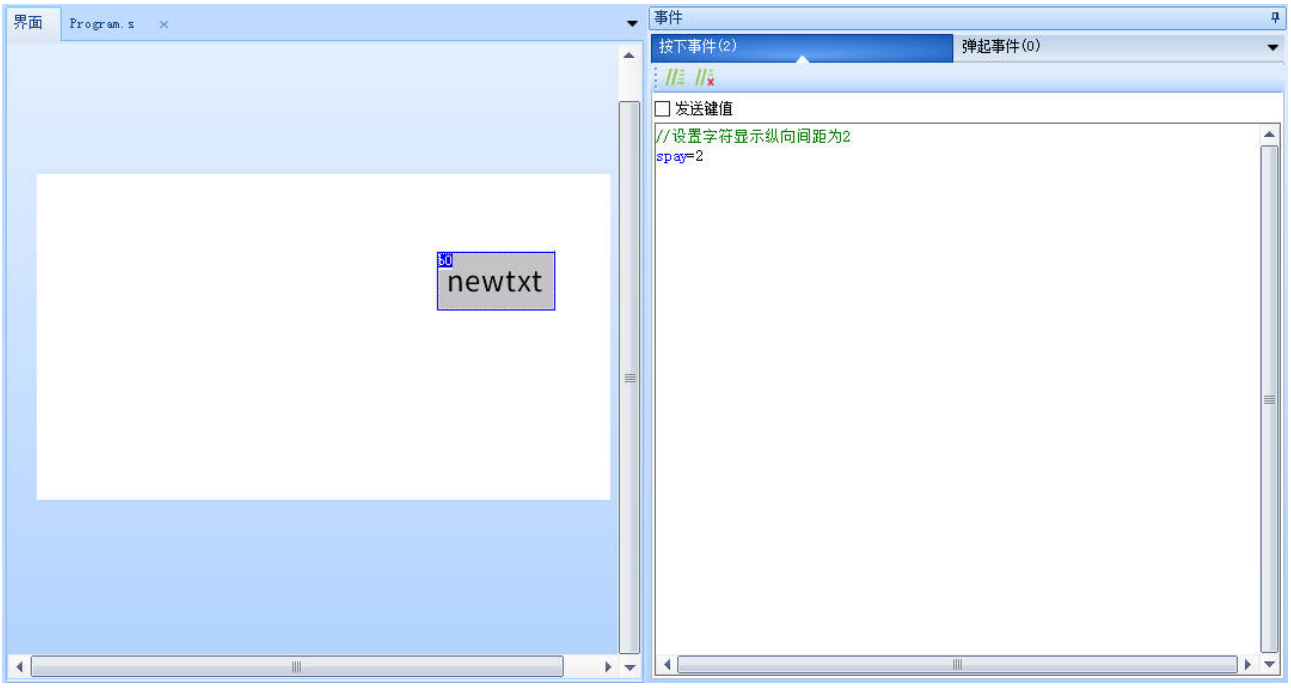


5.2.7 spay-字符显示纵向间距

5.2.7.1 spay-示例

```
//设置字符显示纵向间距为2
spay=2
```

注意： 上电默认为 0，仅对 xstr 指令写出来的字符有效，控件带的字符显示间距由控件内部的属性决定。



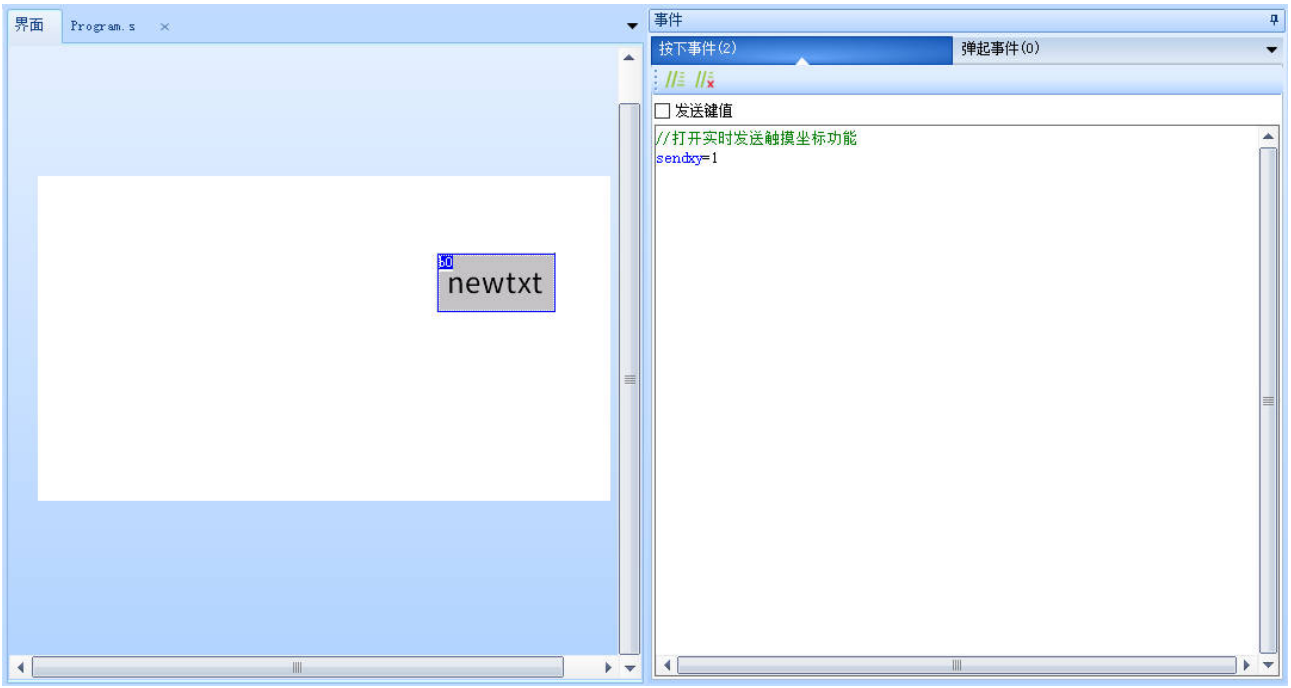
5.2.8 sendxy-实时发送触摸坐标功能开关

5.2.8.1 sendxy-示例 1

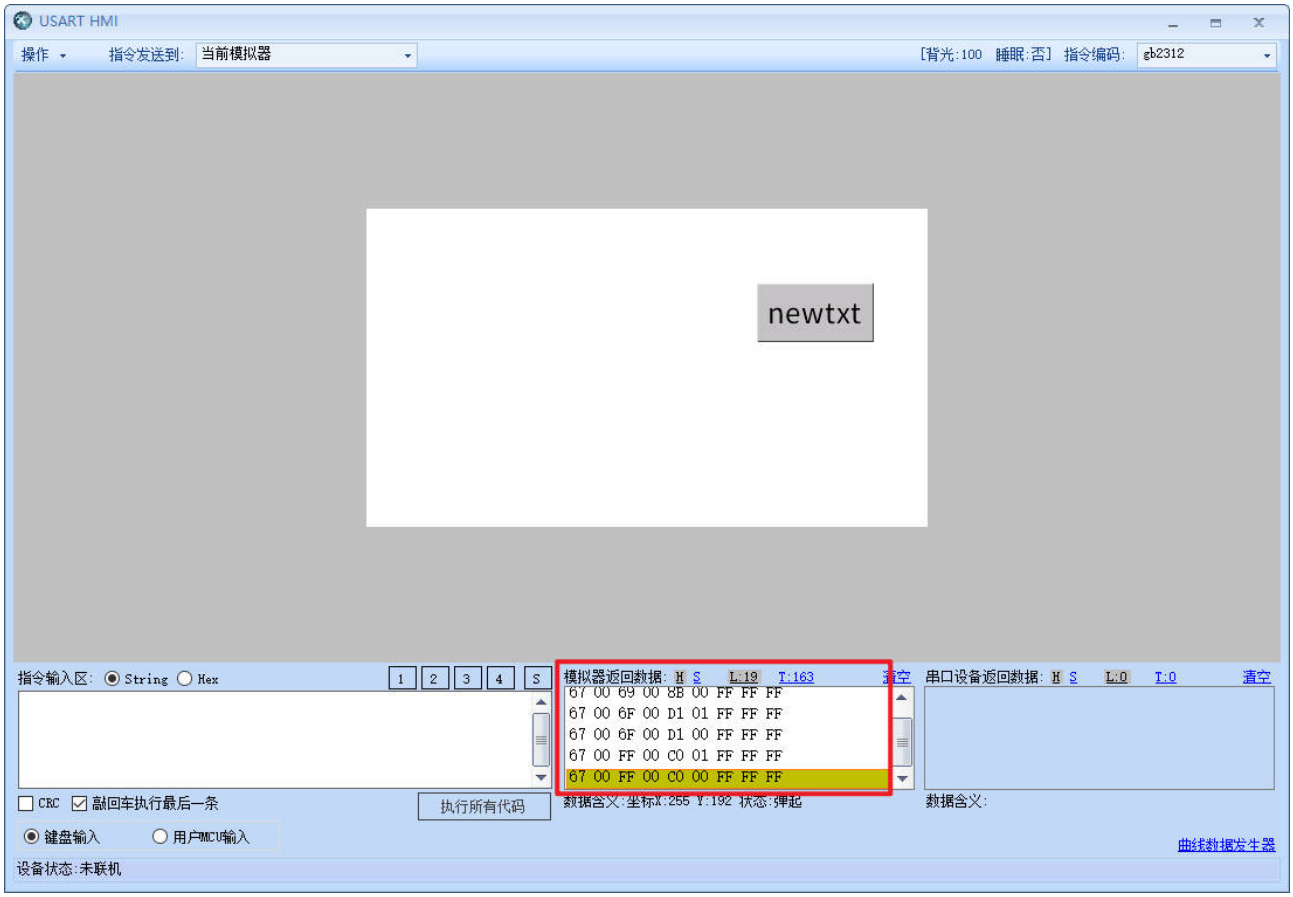
```
//关闭实时发送触摸坐标功能
sendxy=0
```

5.2.8.2 sendxy-示例 2

```
//打开实时发送触摸坐标功能
sendxy=1
```



打开实时发送触摸坐标功能后，每次点击屏幕都会自动发送点击的坐标到串口



注意: 打开发送功能以后，有触摸按下的时候设备会通过串口发送触摸坐标。

发送格式请参考*0X67 触摸坐标数据返回* (page 899)

6 书写语法

【目录】

6.1 赋值操作

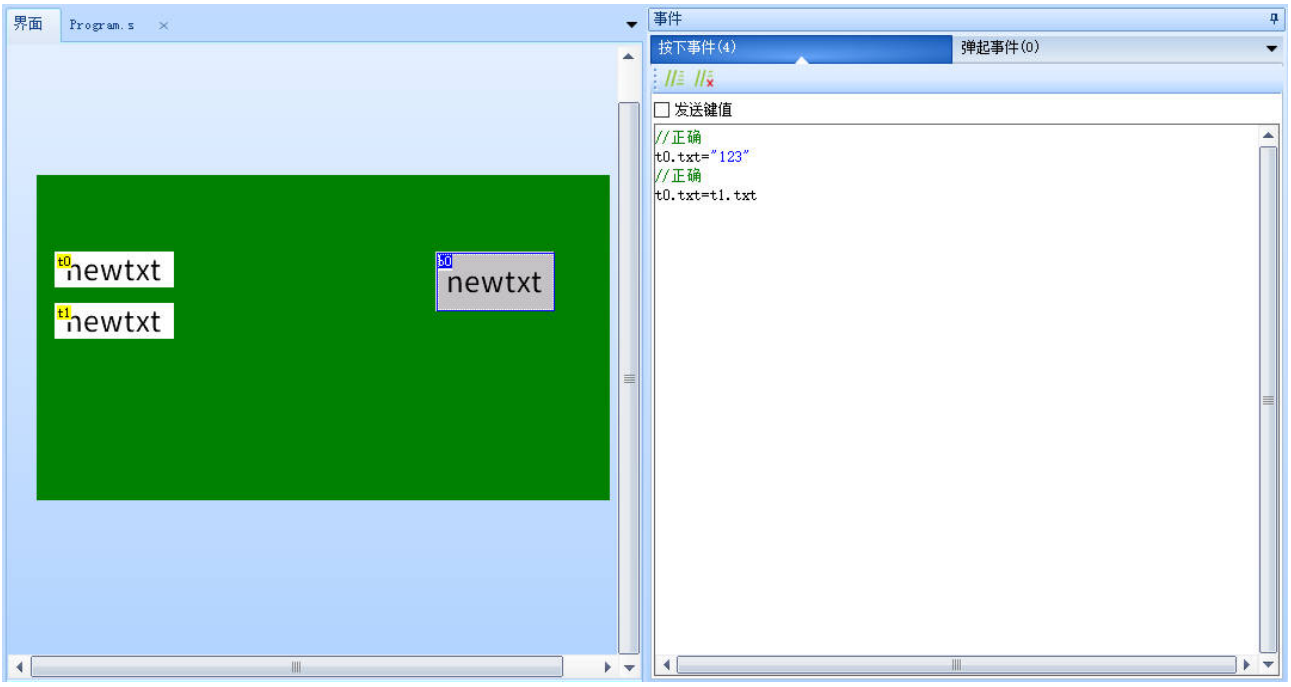
所有的赋值操作可以在上位编辑状态下写入控件事件中，也可以串口传输过来 (串口传输记得加三个 0xff 的结束符)

> **警告：** 所有的赋值操作都不支持多余空格，添加进任何空格，编译都会报错
>
> txt 属性为字符串类型，字符串类型的属性赋值常量必须加双引号

6.1.1 字符串属性赋值

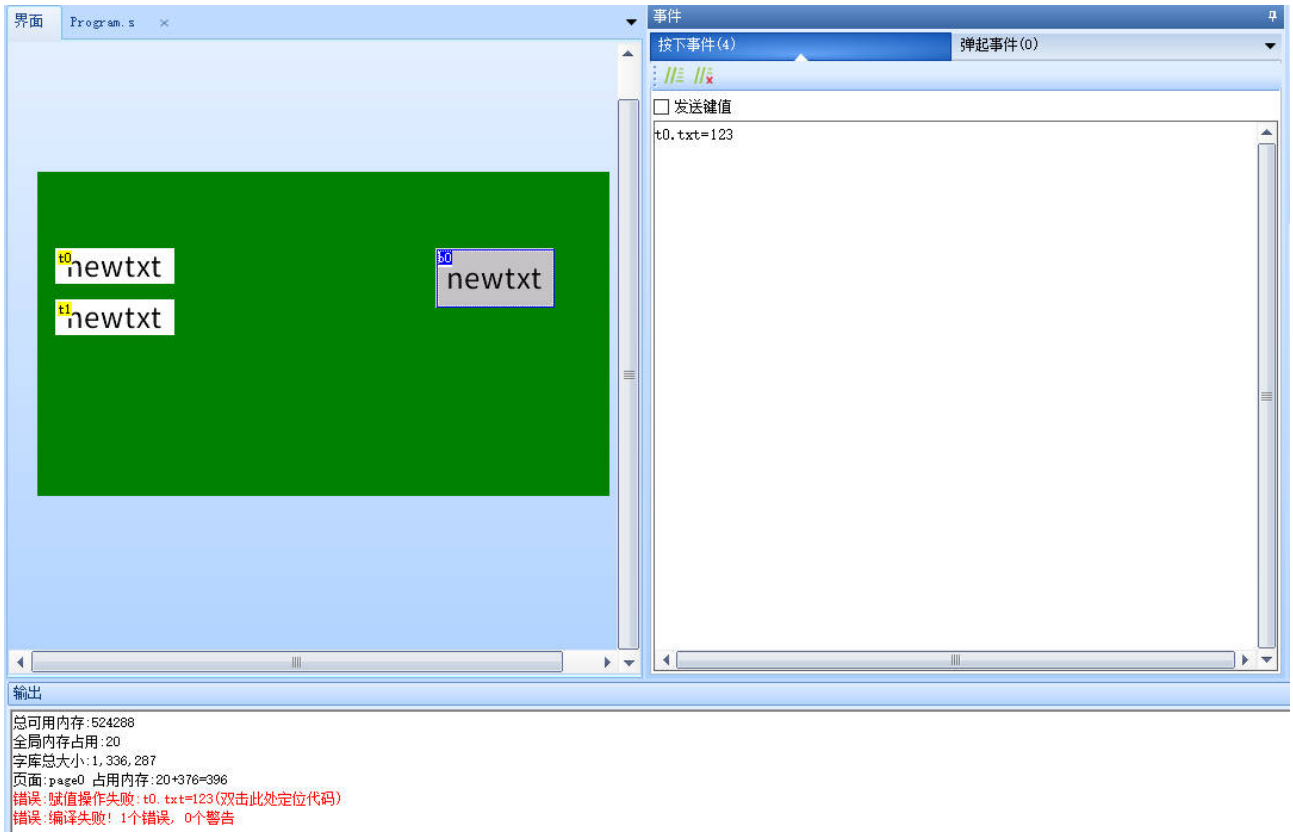
列表 1: 字符串赋值正确写法

```
//正确
t0.txt="123"

//正确
t0.txt=t1.txt
```



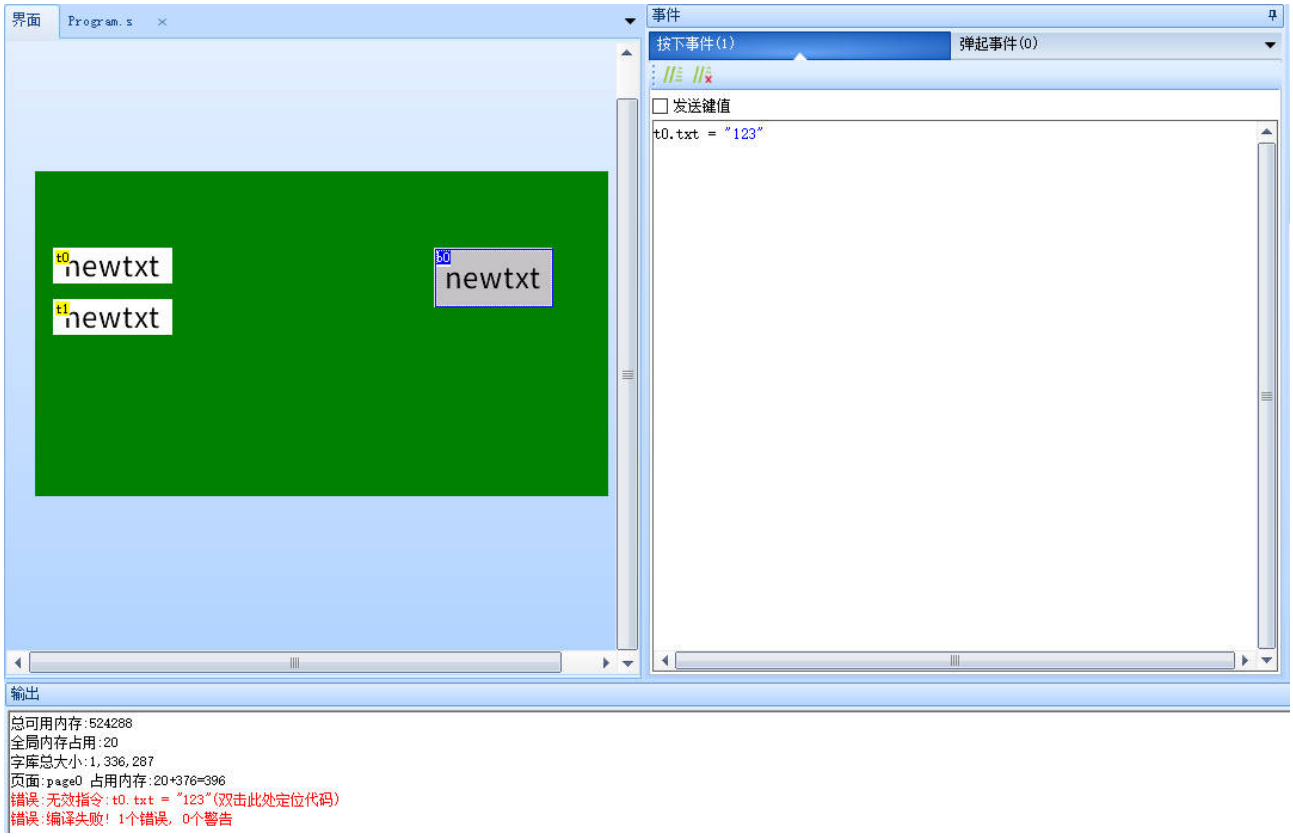
列表 2: 字符串赋值错误写法 1

```
//错误，字符串属性要用引号括起来
t0.txt=123
```



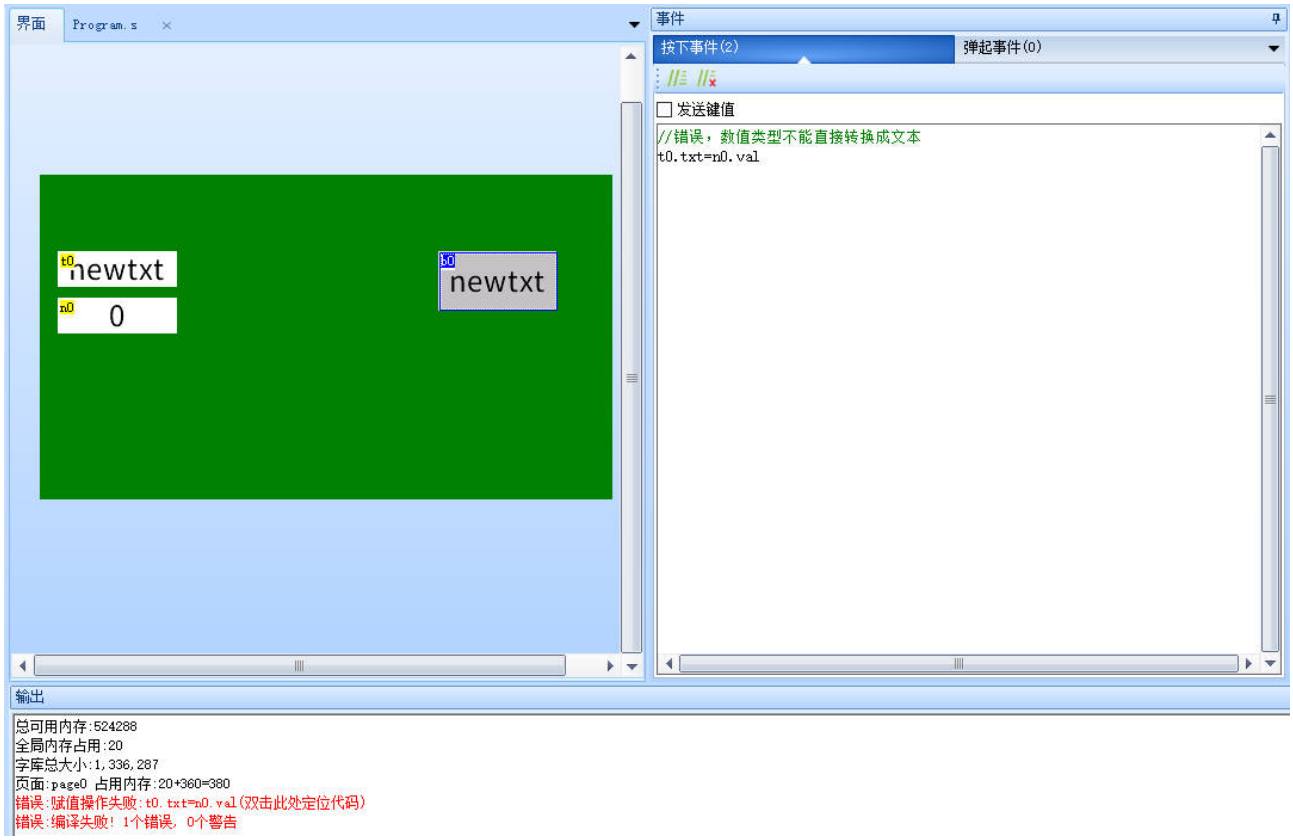
列表 3: 字符串赋值错误写法 2

```
//错误，等号两边不要加空格
t0.txt = "123"
```



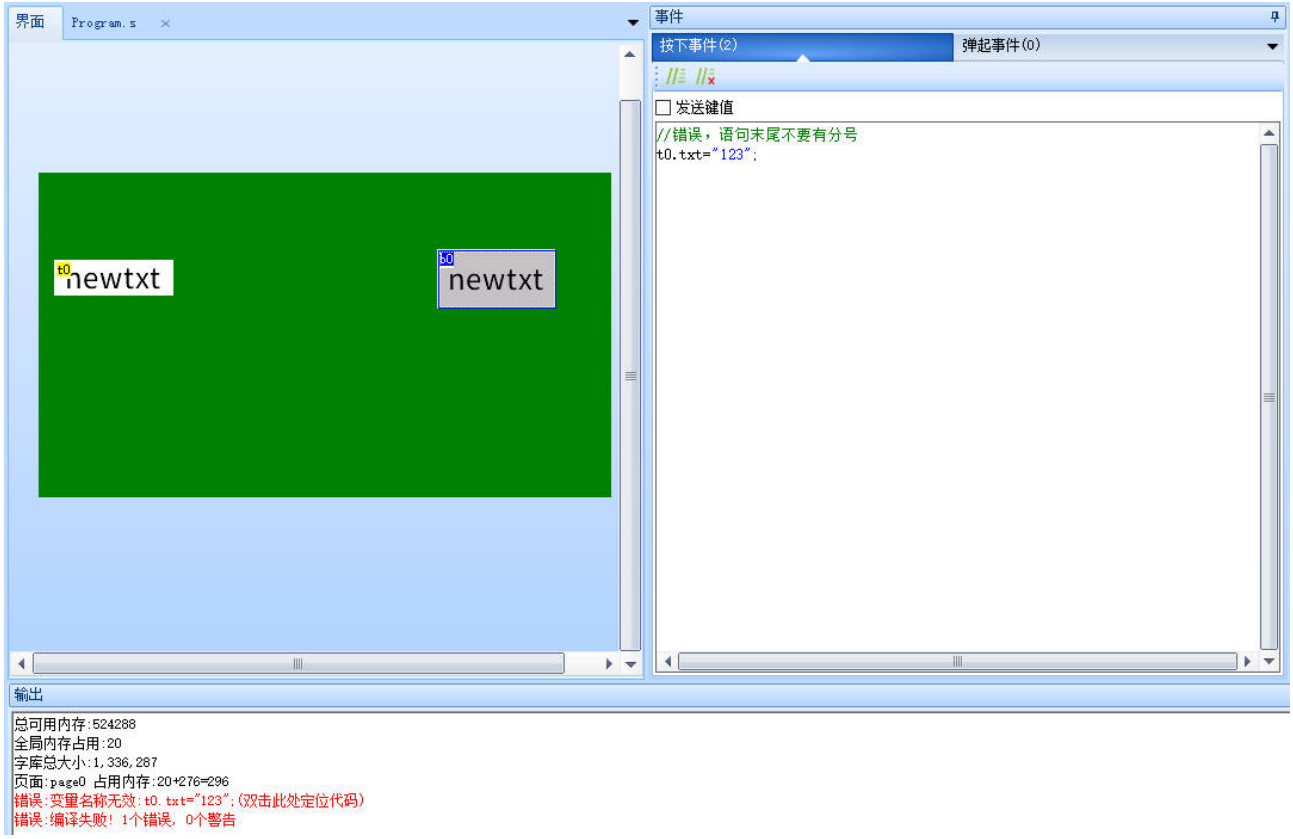
列表 4: 字符串赋值错误写法 3

```
//错误，数值类型不能直接转换成文本
t0.txt=n0.val
```



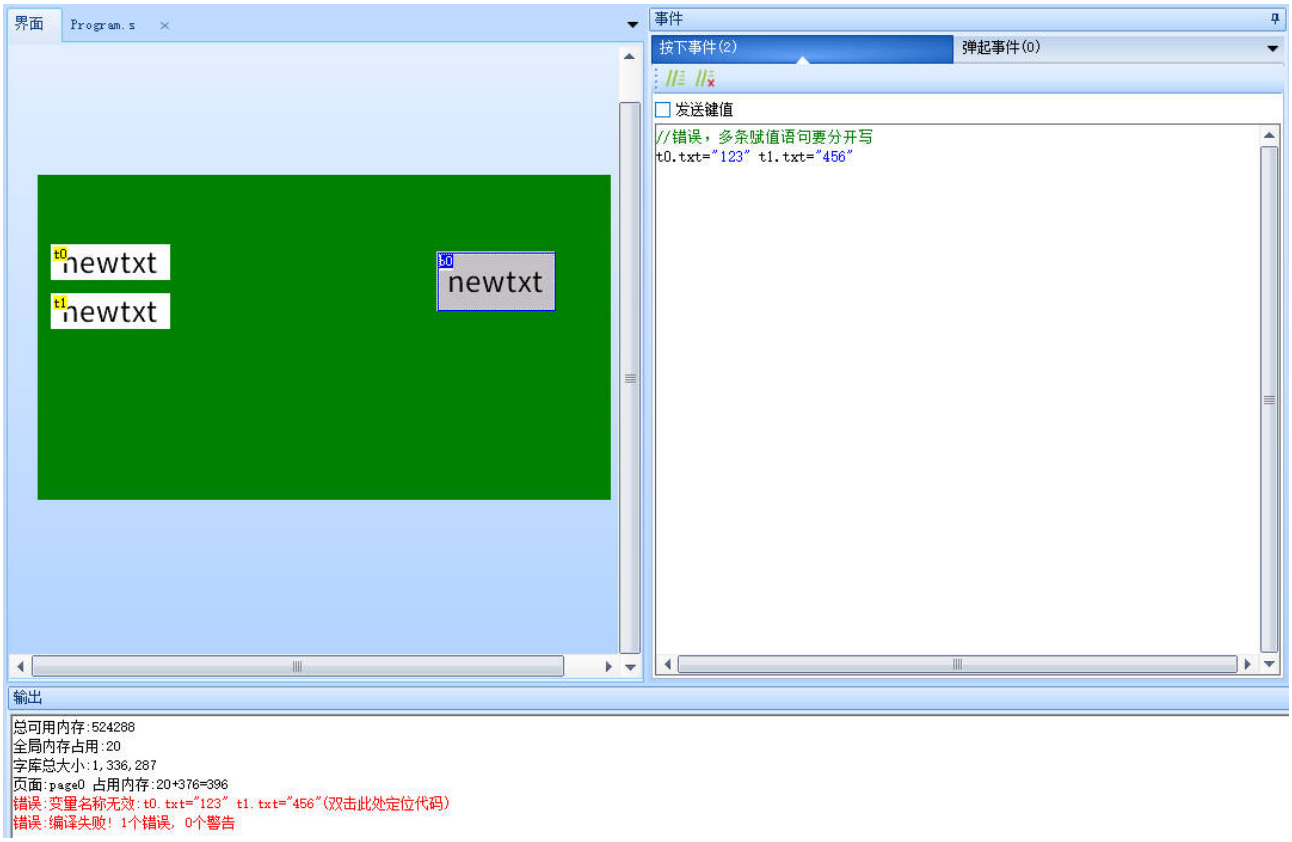
列表 5: 字符串赋值错误写法 4

```
//错误，语句末尾不要有分号
t0.txt="123";
```



列表 6: 字符串赋值错误写法 5

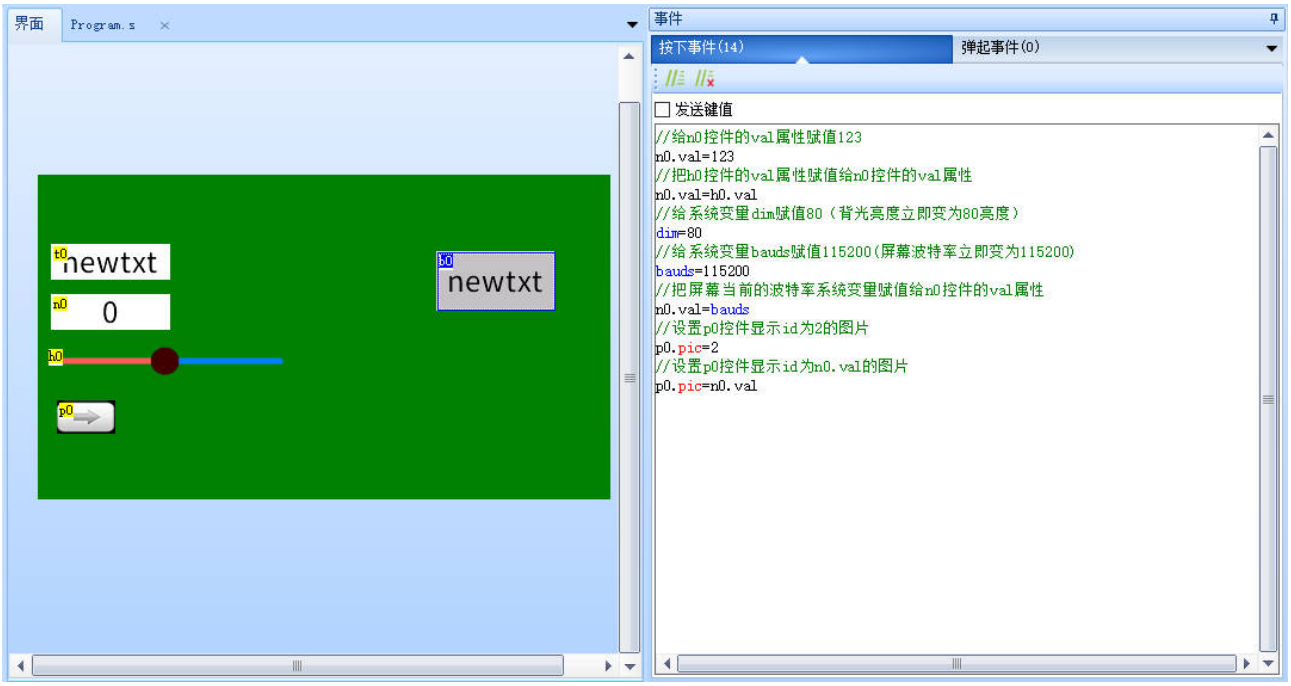
```
//错误，多条赋值语句要分开写
t0.txt="123" t1.txt="456"
```



6.1.2 数值属性赋值

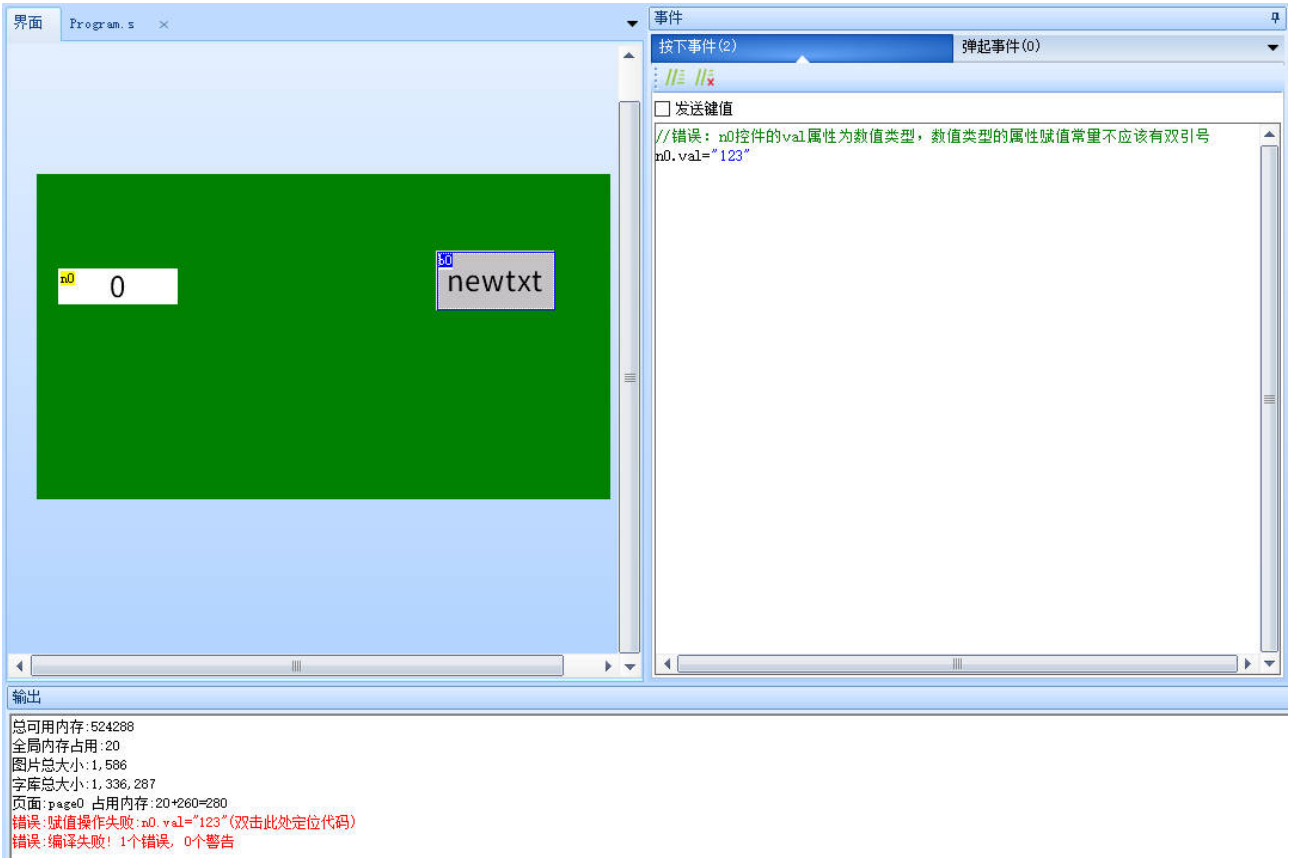
列表 7: 数值属性赋值

```
//给n0控件的val属性赋值123
n0.val=123

//把h0控件的val属性赋值给n0控件的val属性
n0.val=h0.val

//给系统变量dim赋值80（背光亮度立即变为80亮度）
dim=80

//给系统变量bauds赋值115200(屏幕波特率立即变为115200)
bauds=115200

//把屏幕当前的波特率系统变量赋值给n0控件的val属性
n0.val=bauds

//设置p0控件显示id为2的图片
p0.pic=2

//设置p0控件显示id为n0.val的图片
p0.pic=n0.val
```



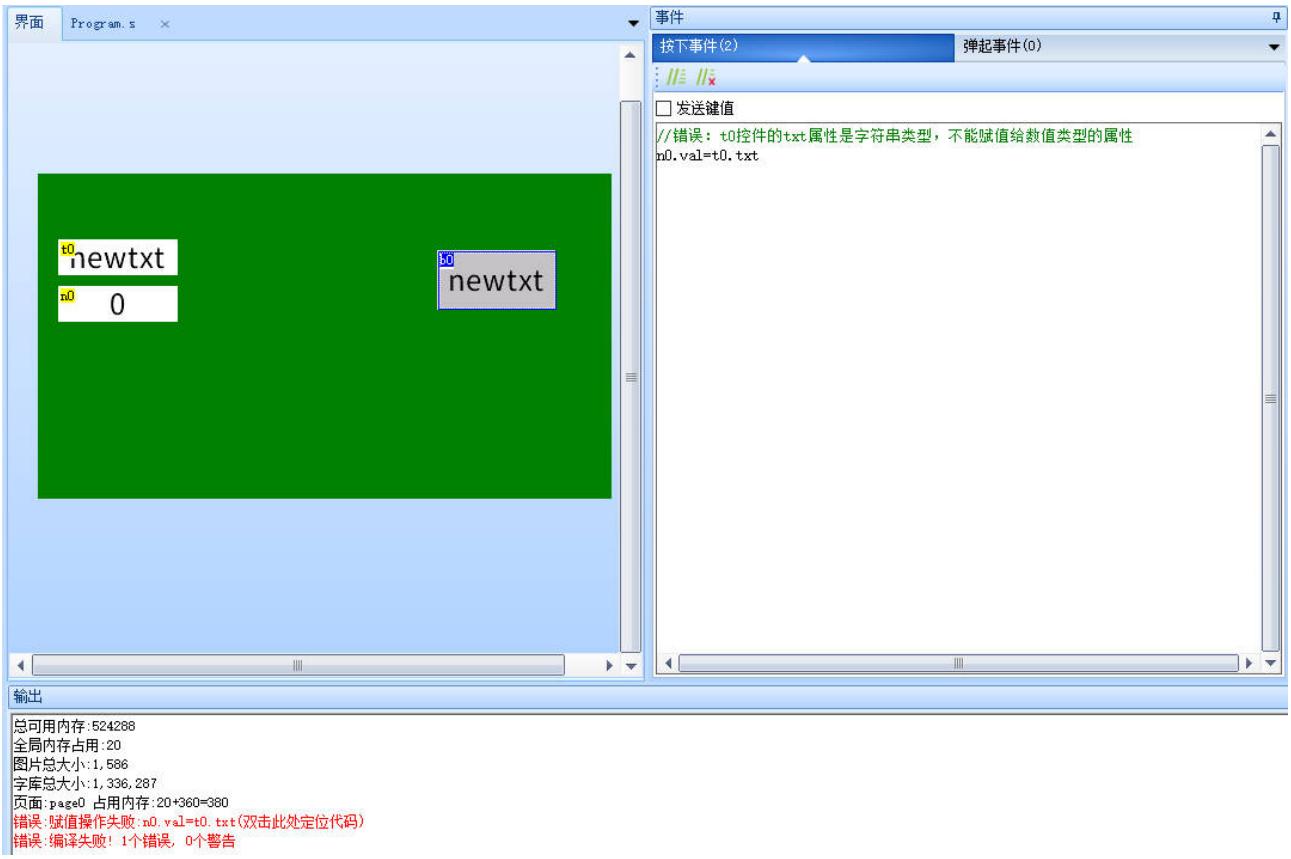
列表 8: 数值属性赋值错误写法 1

```
//错误：n0控件的val属性为数值类型，数值类型的属性赋值常量不应该有双引号
n0.val="123"
```



列表 9: 数值属性赋值错误写法 2

```
//错误：t0控件的txt属性是字符串类型，不能赋值给数值类型的属性
n0.val=t0.txt
```

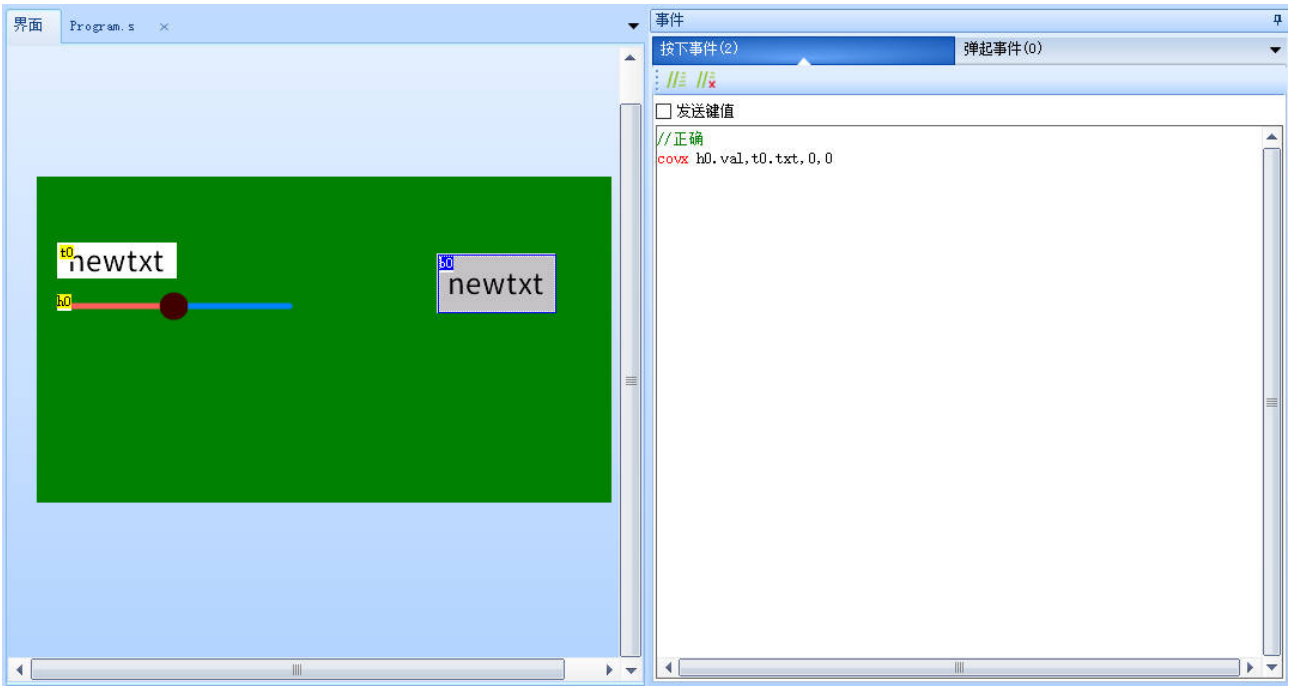


小技巧: 字符串类型和数值类型可以通过 covx 指令来实现相互转换赋值，具体请参考*covx-变量类型转换* (page 606)

6.1.3 字符串属性和数值属性相互转换

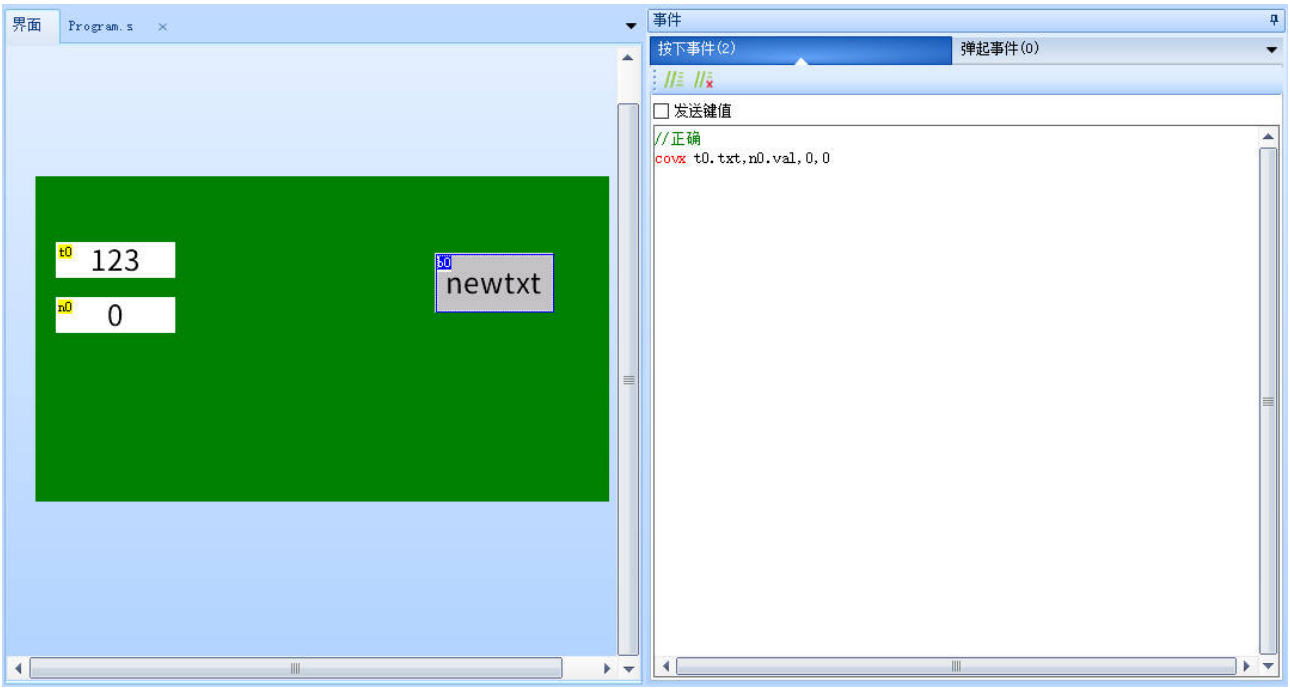
列表 10: val 和 txt 属性相互转换 1

```
//错误，数值类型不能直接转换成文本
t0.txt=h0.val

//正确
covx h0.val,t0.txt,0,0
```



列表 11: val 和 txt 属性相互转换 2

```
//错误，文本类型不能直接转换成数值
n0.val=t0.txt

//正确
covx t0.txt,n0.val,0,0
```



6.1.4 赋值操作-相关链接

哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？ (page 1295)
txt 属性和 *val* 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)
跨页面赋值，全局变量操作 (page 847)

6.1.5 赋值操作-样例工程下载

演示工程下载链接：
《赋值操作》演示工程下载

6.2 运算操作

注意: 所有运算不支持乘除法优先，也不支持括号优先级，统一从左到右的顺序，请特别注意。

所有的运算操作可以在上位编辑状态下写入控件事件中，也可以串口传输过来 (串口传输记得加三个 0xff 的结束符)

所有的运算操作都不支持多余空格，添加进任何空格，编译都会报错

6.2.1 数值类型变量运算操作

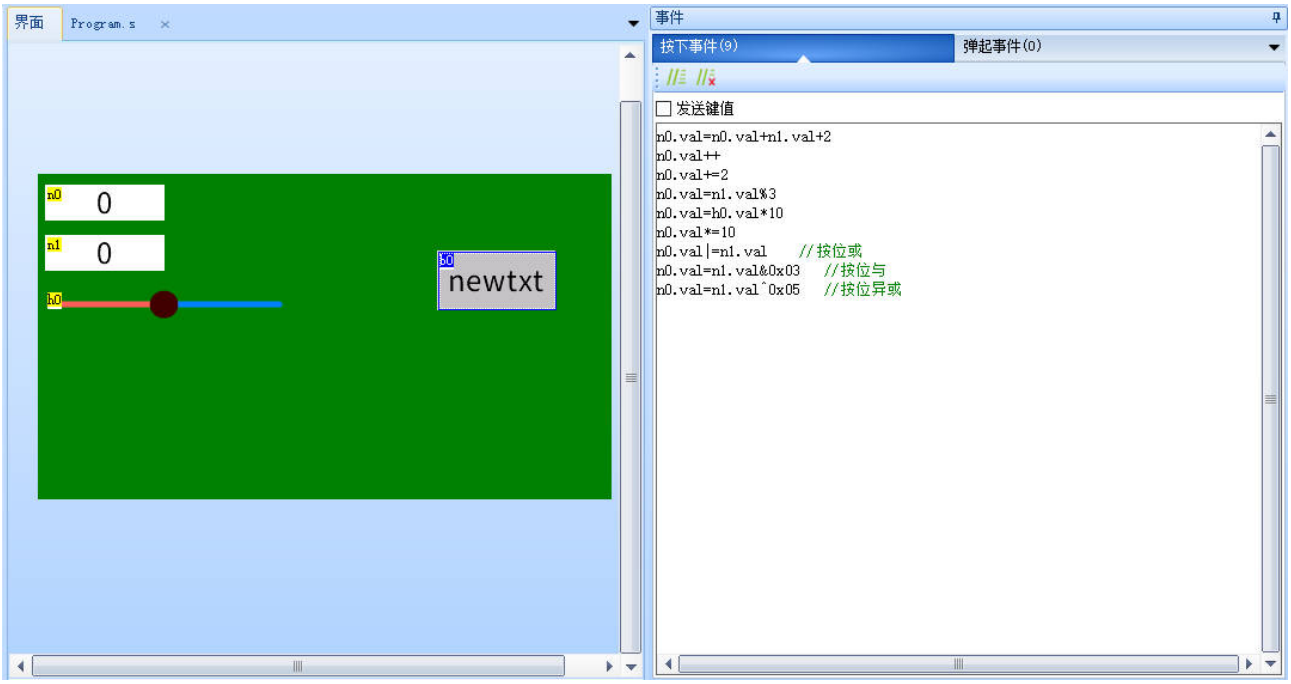
支持的运算符:

表 6: 支持的运算符

符号	作用
+	加法运算或者字符串拼接
-	减法运算或者字符串删除末尾若干字符
*	乘法运算
/	整除运算
%	取余运算
&	按位与
|	按位或
^	按位异或
«	按位左移
»	按位右移
&&	逻辑与
||	逻辑或

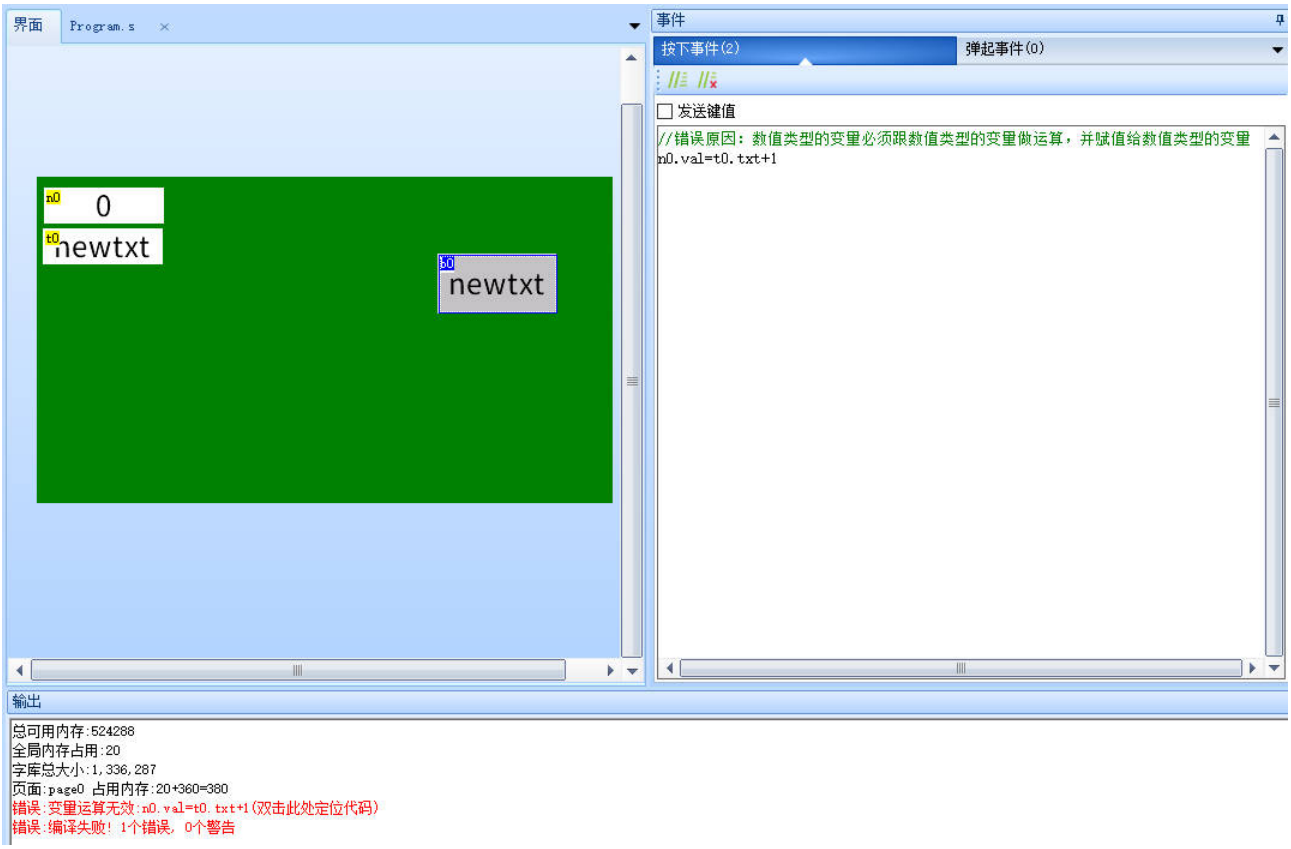
列表 12: 正确的运算操作

```
n0.val=n0.val+n1.val+2
n0.val++
n0.val+=2
n0.val=n1.val%3
n0.val=h0.val*10
n0.val*=10
n0.val|=n1.val     //按位或
n0.val=n1.val&0x03    //按位与
n0.val=n1.val^0x05    //按位异或
```



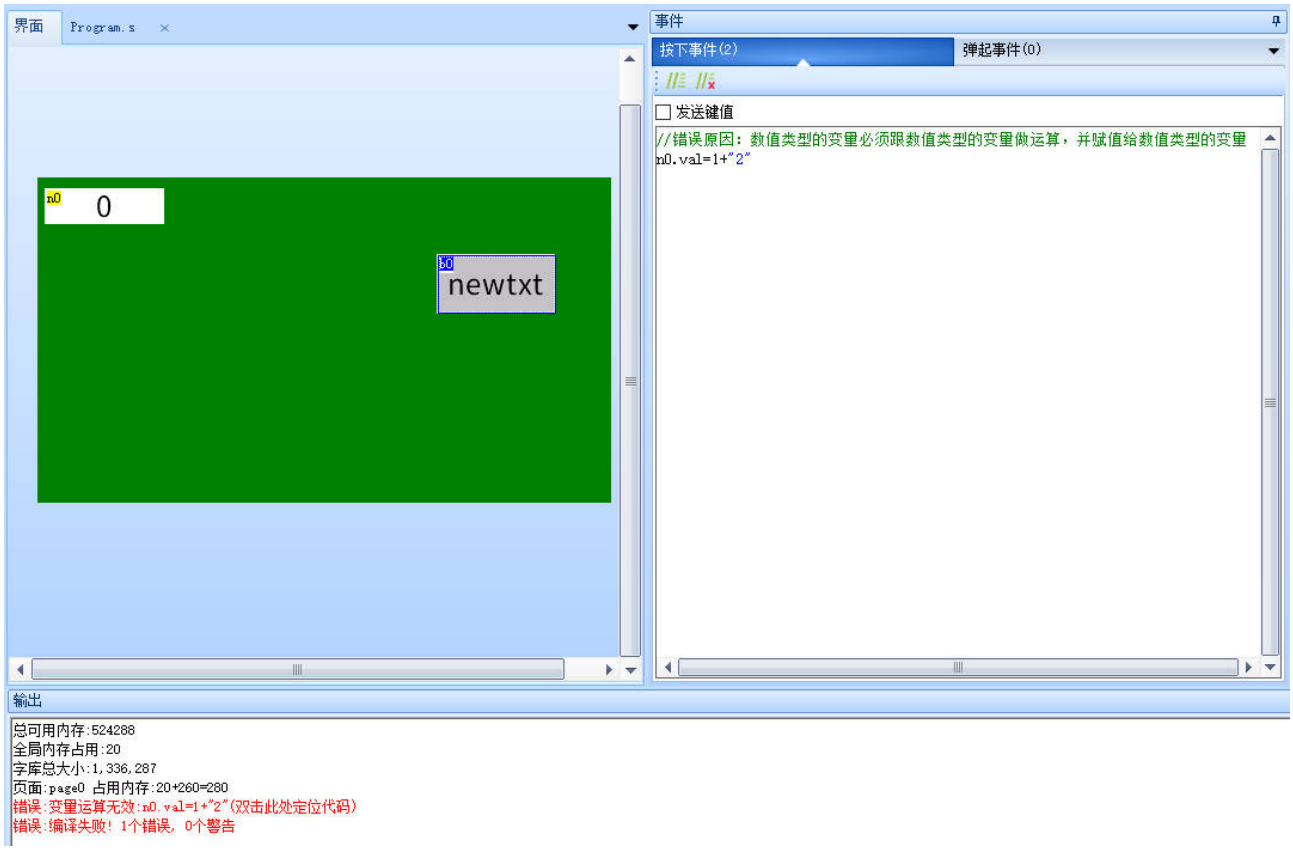
列表 13: 错误的运算写法 1

```
//错误原因：数值类型的变量必须跟数值类型的变量做运算，并赋值给数值类型的变量
n0.val=t0.txt+1
```



列表 14: 错误的运算写法 2

```
//错误原因：数值类型的变量必须跟数值类型的变量做运算，并赋值给数值类型的变量
n0.val=1+"2"
```



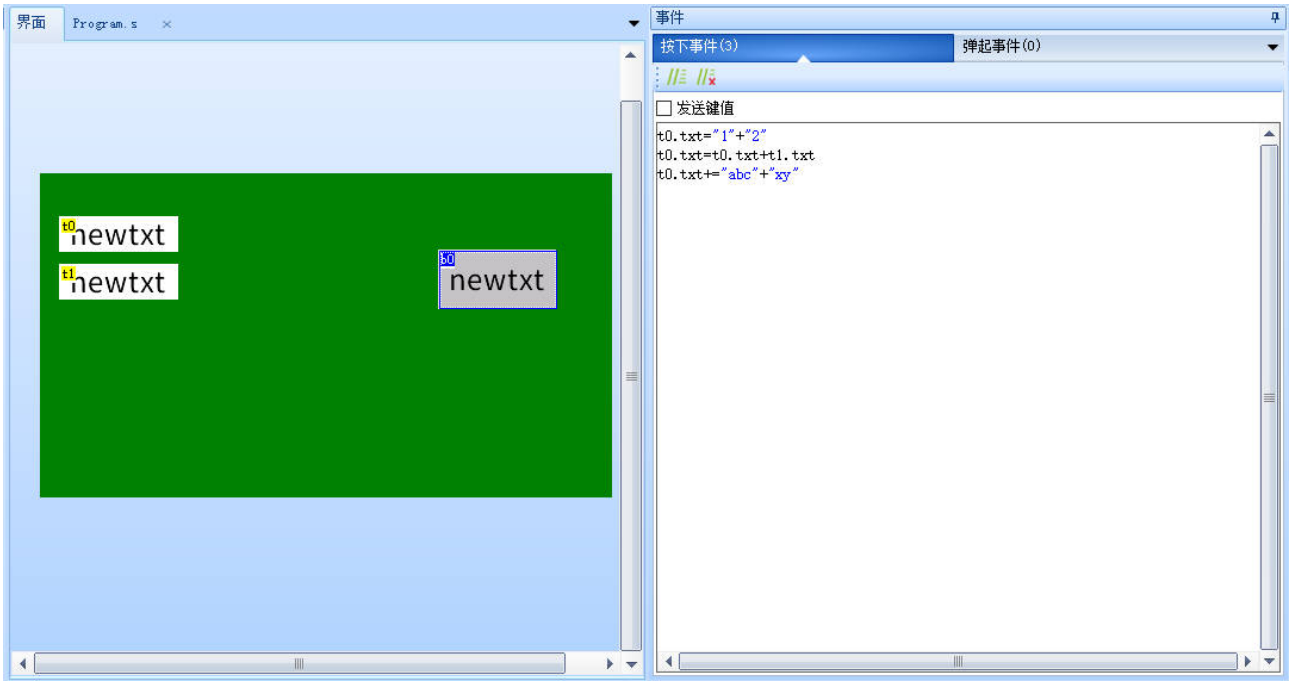
注意: 当数字控件的最高位为 1 时 (即负数), 进行右移操作时, 最高位将补 1, 例如 0x80000000»31 后变为 0xFFFFFFFF 而不是 0x01

6.2.2 字符串类型变量运算操作

运算符“+”

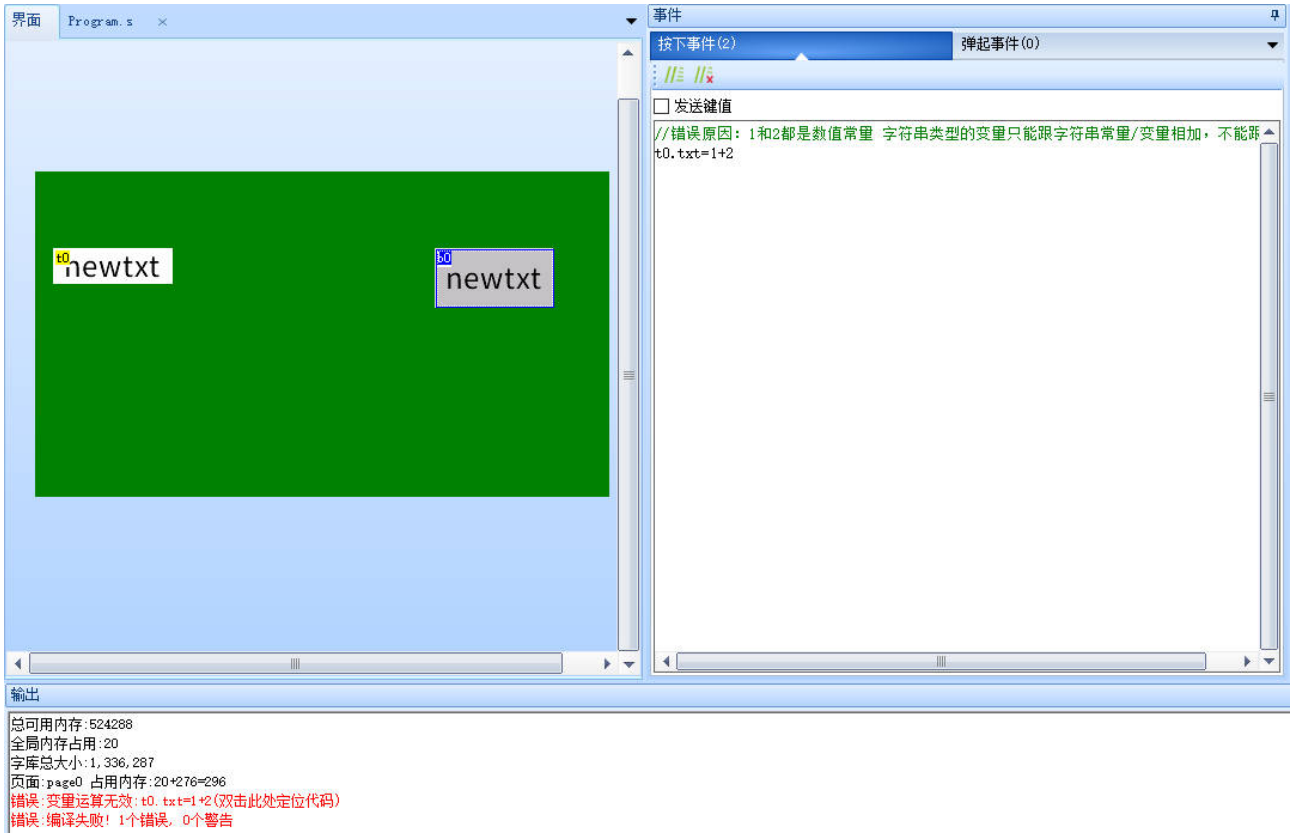
列表 15: 字符串拼接

```
t0.txt="1"+"2"
t0.txt=t0.txt+t1.txt
t0.txt+="abc"+"xy"
```



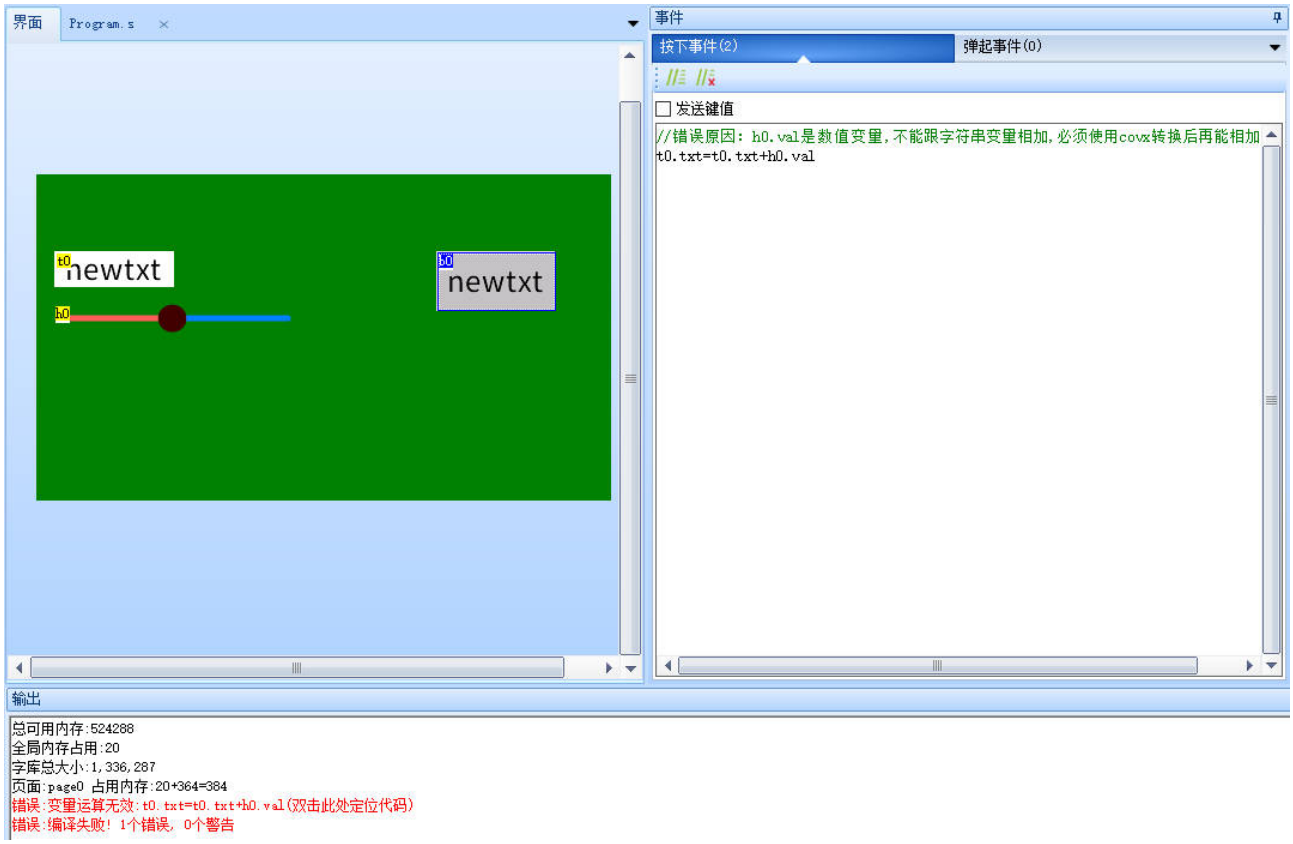
列表 16: 以下为错误的运算写法

```
//错误原因：1和2都是数值常量 字符串类型的变量只能跟字符串常量/
↪变量相加，不能跟一个数值常量/变量相加
t0.txt=1+2
```



列表 17: 以下为错误的运算写法

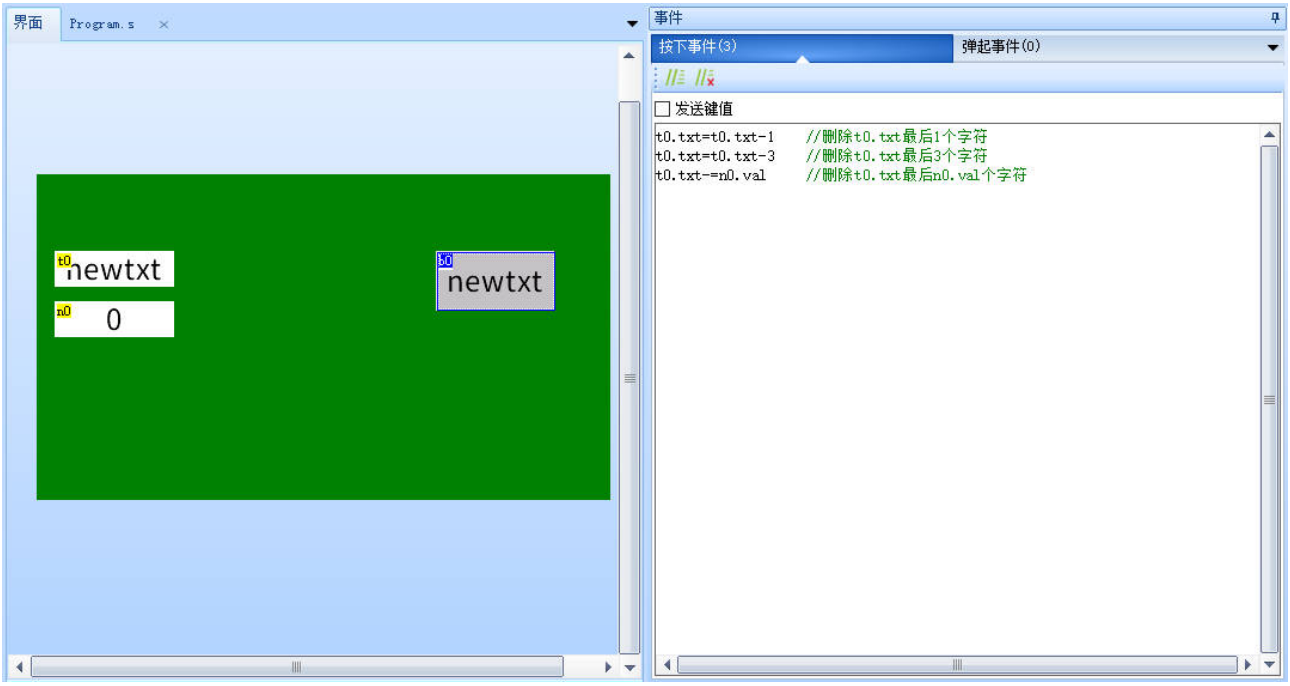
```
//错误原因：h0.val是数值变量,不能跟字符串变量相加,必须使用covx转换后再能相加
t0.txt=t0.txt+h0.val
```



运算符“-”

列表 18: 字符串删除

```
t0.txt=t0.txt-1     //删除t0.txt最后1个字符
t0.txt=t0.txt-3     //删除t0.txt最后3个字符
t0.txt-=n0.val      //删除t0.txt最后n0.val个字符
```



在字符串变量运算中,”-“代表删除的意思,所以用”-“的时候，字符串变量必须”-“一个数值常量/变量来表示删除多少个字符，这里跟用”+”是不一样的。用”+”的时候必须是字符串 + 字符串；用”-“的时候必须是字符串-数值。

6.2.3 位运算

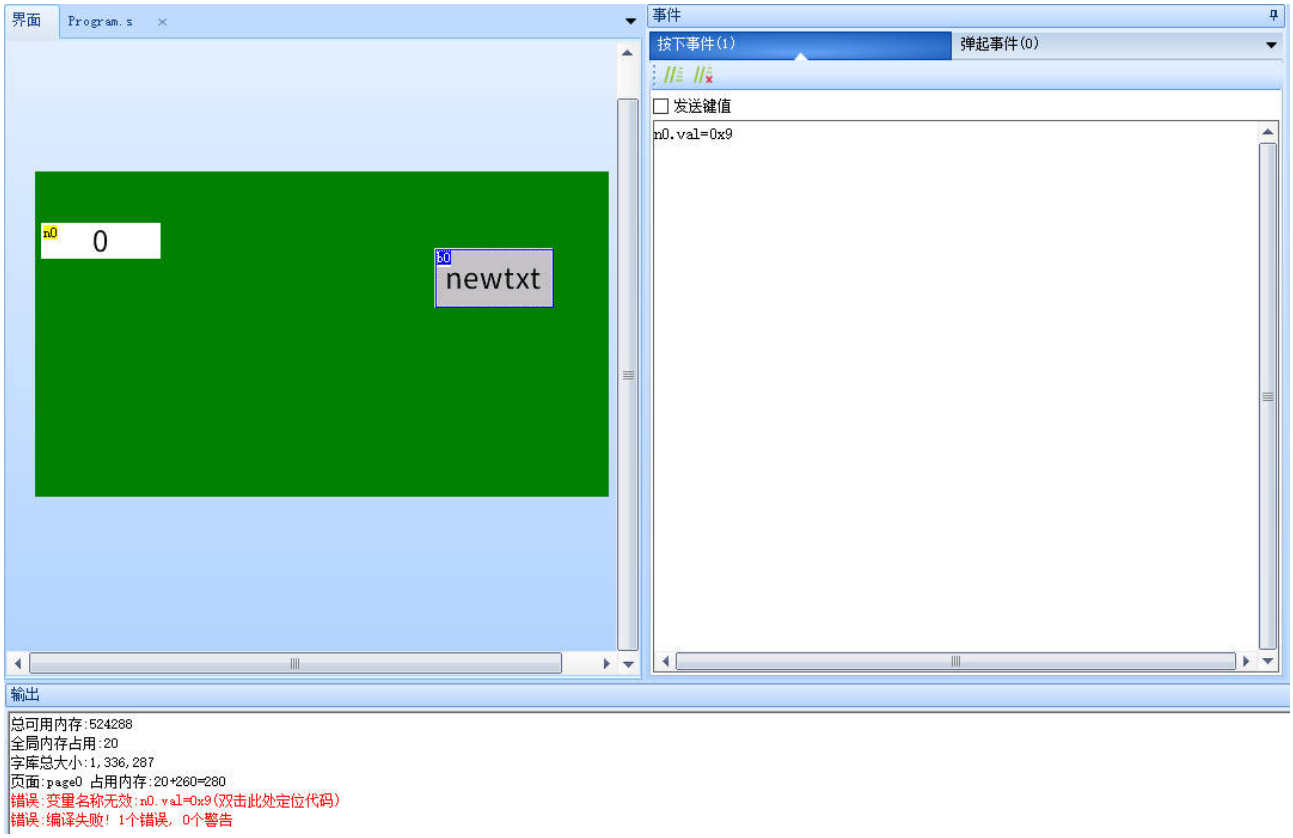
注意: 在进行位运算时，写 16 进制时，必须补全为偶数位长度。

例如:

0x9，必须补全成 0x09

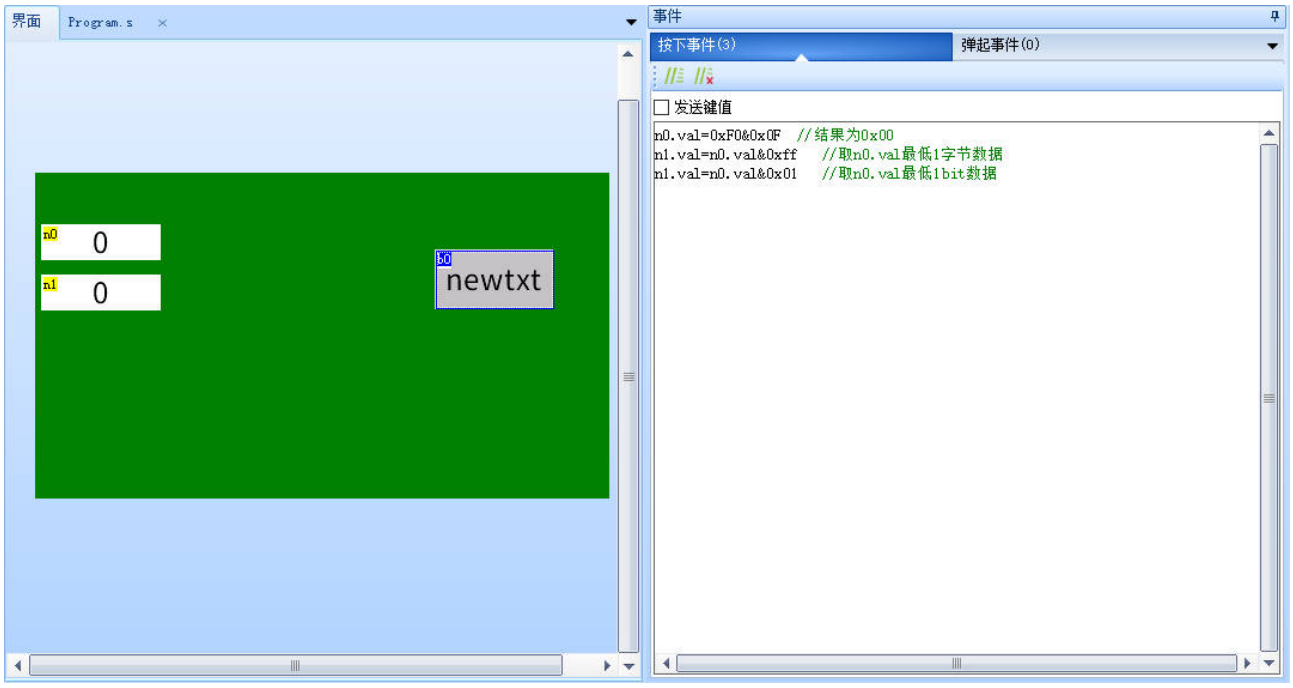
0x100，必须补全成 0x0100

0x12345，必须补全成 0x012345



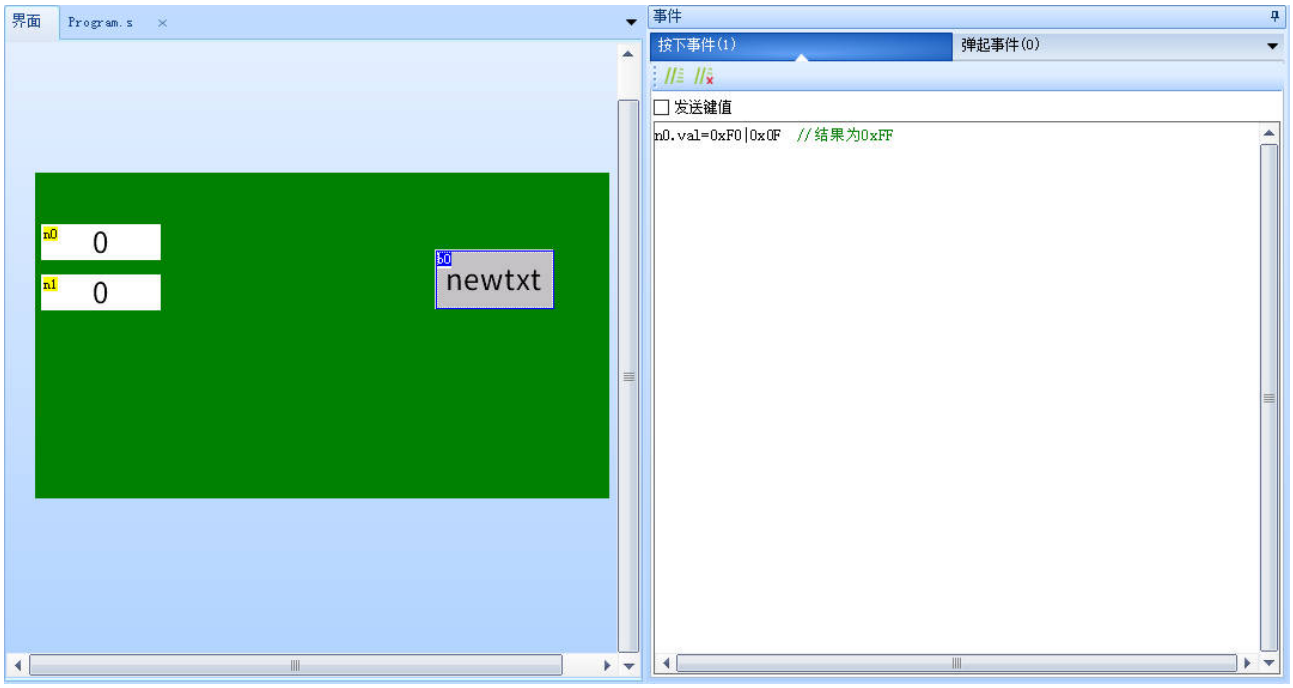
6.2.3.1 按位与

```
n0.val=0xF0&0x0F   //结果为0x00

n1.val=n0.val&0xff    //取n0.val最低1字节数据

n1.val=n0.val&0x01    //取n0.val最低1bit数据
```



6.2.3.2 按位或

```
n0.val=0xF0|0x0F   //结果为0xFF
```

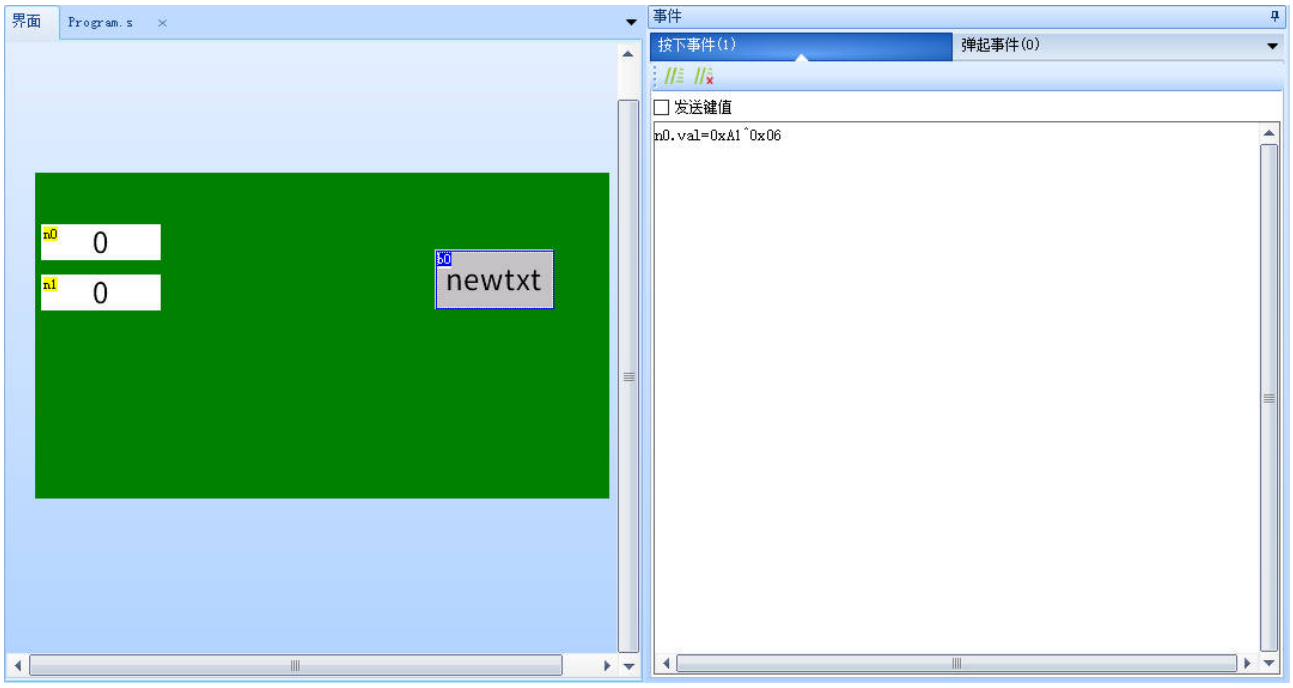


6.2.3.3 按位异或

使某些特定的位翻转

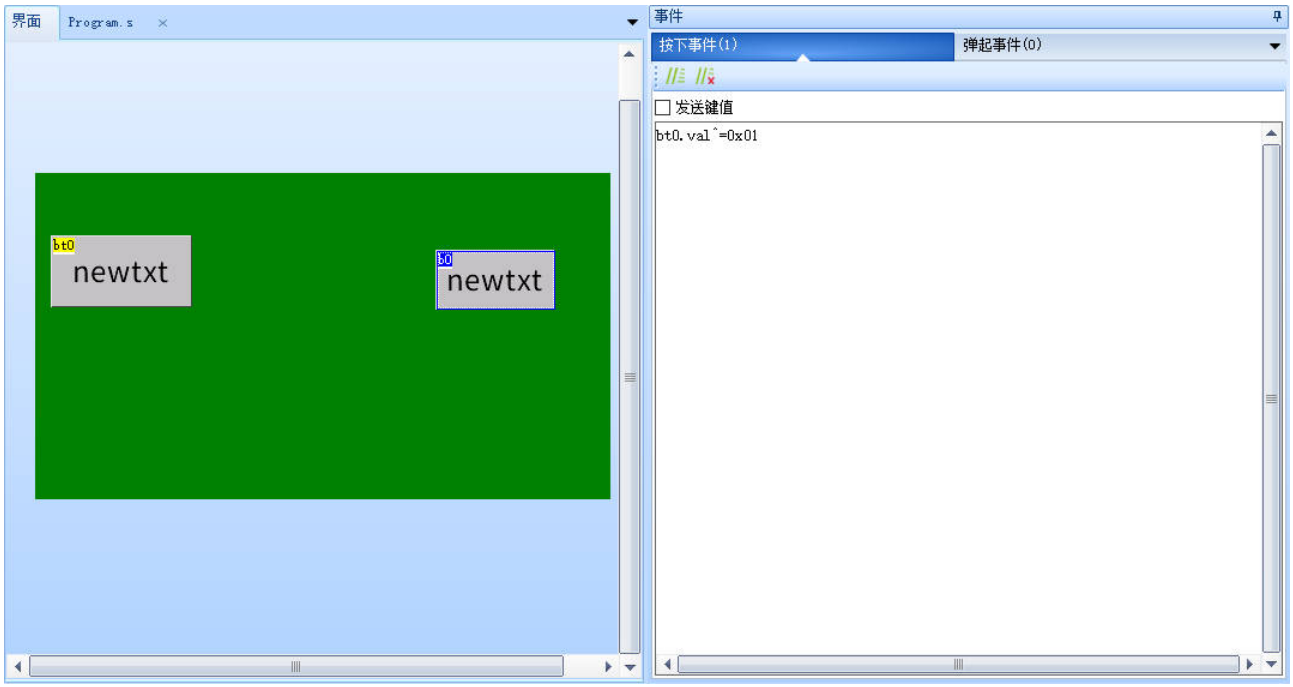
例如对二进制形式 10100001(0xA1) 的第 2 位和第 3 位翻转，则可以将该数与 00000110(0x06) 进行按位异或运算。

```
n0.val=0xA1^0x06
```



翻转按钮状态

```
bt0.val^=0x01
```



6.2.3.4 按位取反

参考按位异或，将所有的位翻转就是按位取反

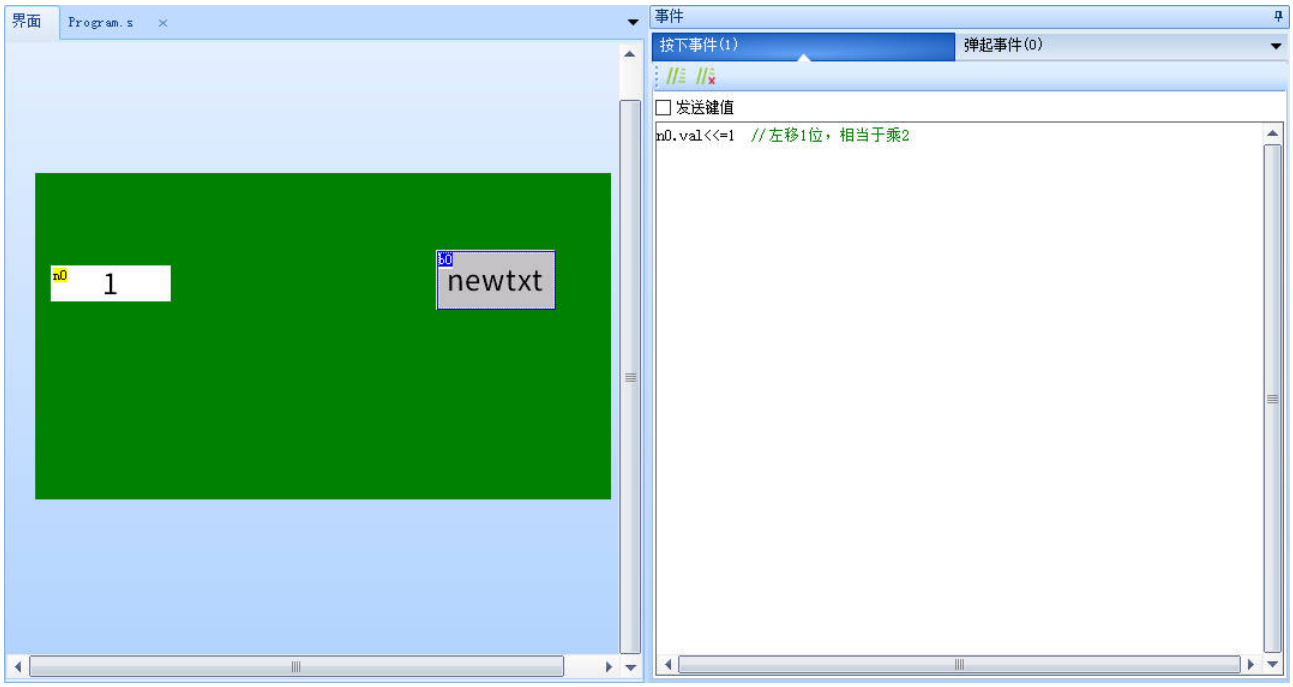
如何实现按位取反 (page 1324)

6.2.3.5 按位同或

如何实现按位同或运算 (page 1325)

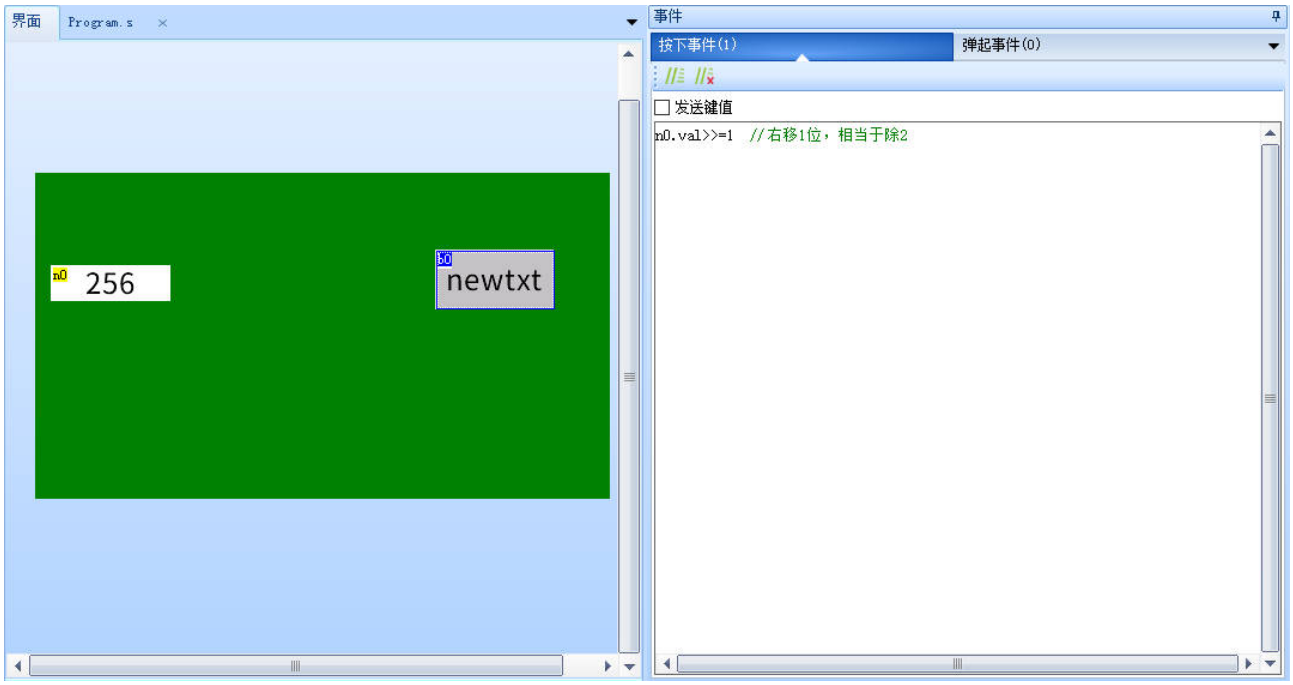
6.2.3.6 按位左移

```
n0.val<<=1   //左移1位，相当于乘2
```



6.2.3.7 按位右移

```
n0.val>>=1   //右移1位，相当于除2
```



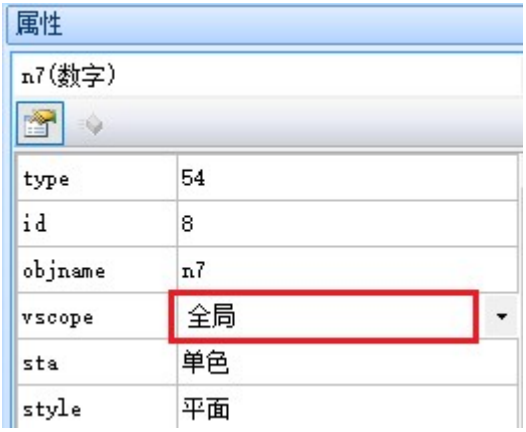
6.2.4 运算操作-相关链接

暂无

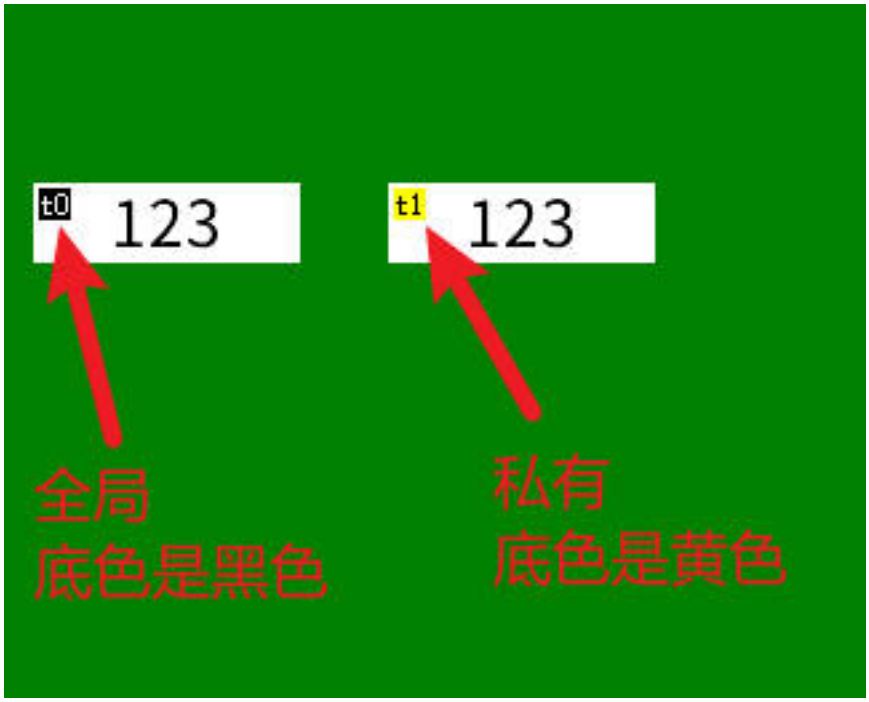
6.3 跨页面赋值，全局变量操作

6.3.1 全局变量-常规使用

多数情况下，我们都是在操作当面页面的控件属性，如果需要操作其他页面的控件属性需要提前将对应的控件设置为全局



修改了 vscope 属性后，控件的名称会由黄色变为黑色



请按如下书写方式:

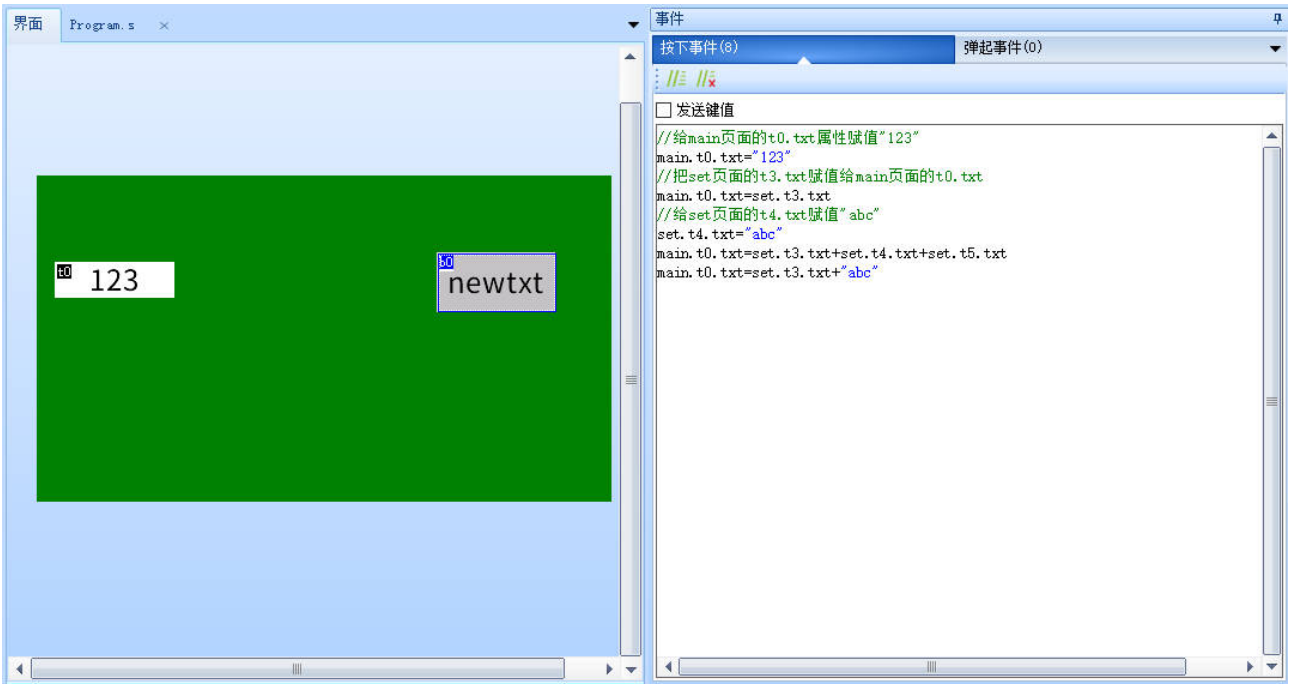
[页面].[控件].[属性]=XXX

```
//给page0页面的n0.val属性赋值123
page0.n0.val=123

//给page1页面的n0.val属性赋值456
page1.n0.val=456

//给main页面的t0.txt属性赋值"abc"
main.t0.txt="abc"

//给main页面的t0.txt属性赋值"123"
```

(续下页)

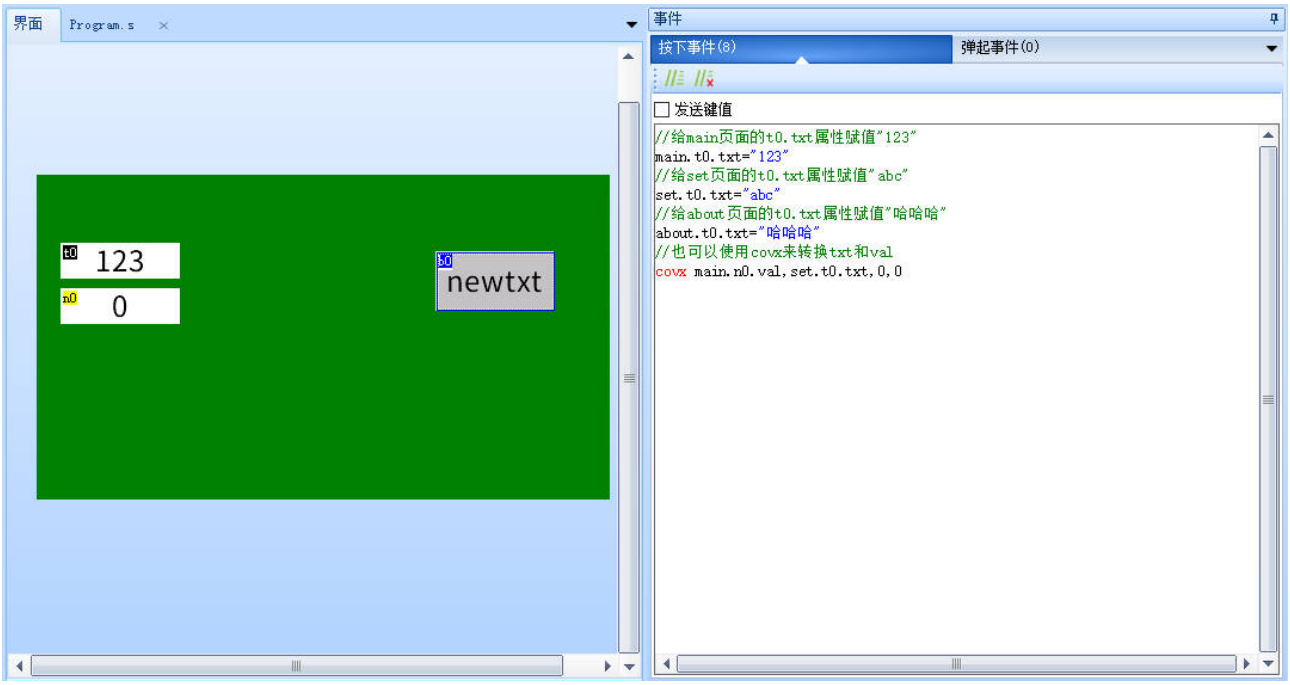
(接上页)

```
main.t0.txt="123"

//把set页面的t3.txt赋值给main页面的t0.txt
main.t0.txt=set.t3.txt

//给set页面的t4.txt赋值"abc"
set.t4.txt="abc"

//将set的页面t3.txt、t4.txt、t5.txt的文本内容拼接后赋值给main页面的t0.txt
main.t0.txt=set.t3.txt+set.t4.txt+set.t5.txt

//将set的页面t3.txt、拼接字符串"abc"后赋值给main页面的t0.txt
main.t0.txt=set.t3.txt+"abc"
```



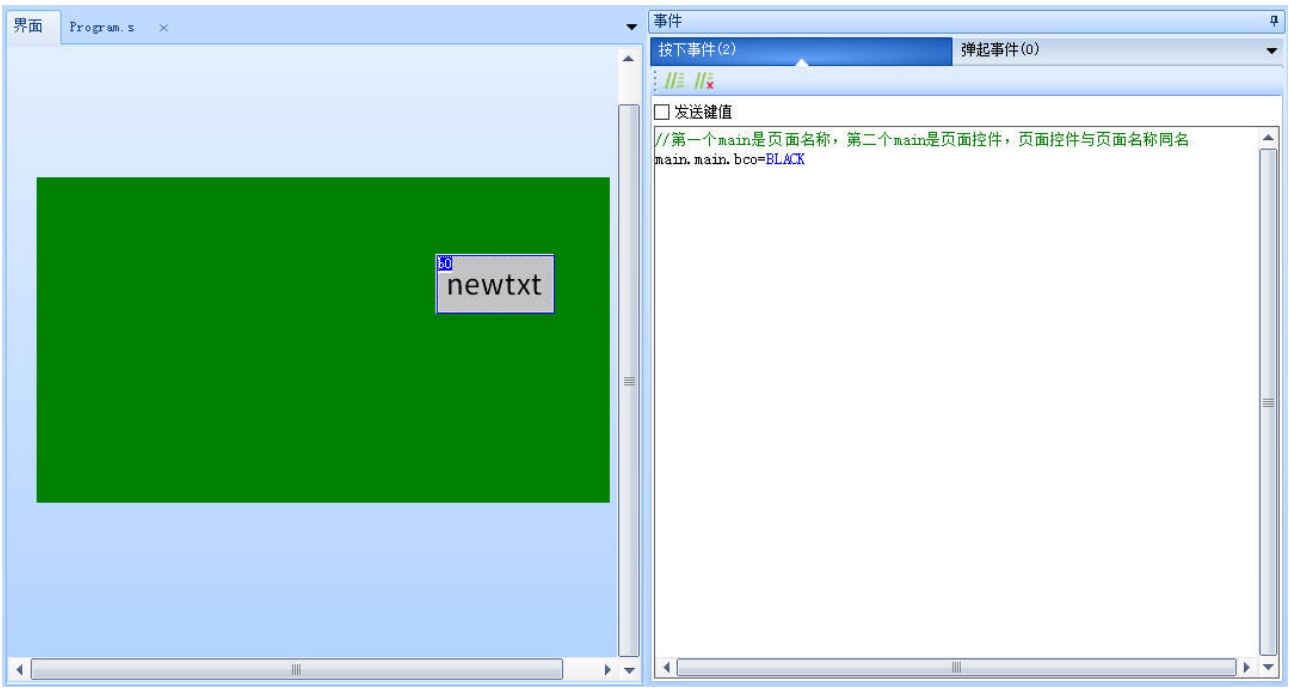
如果不同页面有名称相同的控件需要赋值不同的数据，可以将他们设置为全局

```
//给main页面的t0.txt属性赋值"123"
main.t0.txt="123"

//给set页面的t0.txt属性赋值"abc"
set.t0.txt="abc"

//给about页面的t0.txt属性赋值"哈哈哈"
about.t0.txt="哈哈哈"

//也可以使用covx来转换txt和val
covx main.n0.val,set.t0.txt,0,0
```



如果需要跨页面修改页面的背景颜色，需要将页面控件设置为全局

```
//第一个main是页面名称，第二个main是页面控件，页面控件与页面名称同名
main.main.bco=BLACK
```

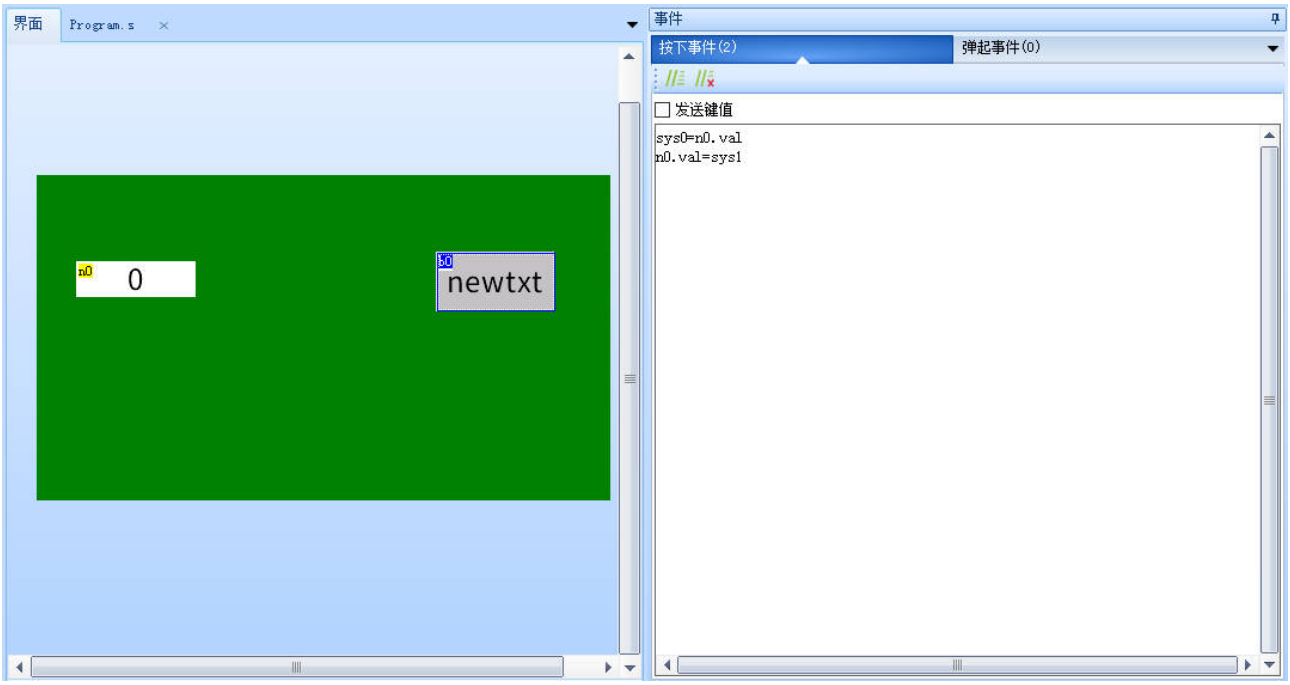


注意： 1. 跨页面操作控件属性的时候，不管是读取还是赋值，被操作控件的 vscope 属性必须设置为全局（默认是私有），否则操作会失败。

2.click vis tsw 指令无法跨页面触发控件、定时器控件只能在当前页面运行、曲线波形控件只能在当前页面添加数据点。

在 program.s 中定义的变量也是全局变量，且只能是数字类型的变量，可以在任意页面进行使用

```
sys0=n0.val

n0.val=sys1
```



6.3.2 全局变量-特殊控件

1、定时器控件

定时器控件虽然可以设置为全局，但是定时器无法跨页面运行，定时器只能在定时器所在的页面运行，一旦切换页面，定时器就会停止运行

2、触摸捕捉控件

触摸捕捉控件设置为全局是无意义的

3、数据记录控件

数据记录控件只能是全局的

4、文件流控件

文件流控件只能是私有的

6.3.3 不同页面的相同名称的控件是什么关系

比如 page0 页面的 n0 控件和 page1 页面的 n0 控件有什么关系

这是两个独立的控件

打个比方，某户人姓张，隔壁老王姓王，老张的儿子叫小明，老王的儿子也叫小明

在老张家时，别人只喊“小明”两个字，那就是在喊“张. 小明”

在老王家时，别人只喊“小明”两个字，那就是在喊“王. 小明”

在老张家想喊老王家的小明，那就得加上具体的姓氏，就是“王. 小明”

同理，page0.n0 就是在说明是 page0 页面的 n0，page1.n0 就是在说明是 page1 页面的 n0

6.3.4 全局变量-c 语言示例

单片机通过串口给全局变量赋值

给文本控件赋值时如果显示的文本不全，请检查控件的 txt_maxl 属性

```
printf("main.t0.txt=\"abc\"\xff\xff\xff");          //给main页面的t0文本控件赋值

printf("page0.t2.txt=\"%d\"\xff\xff\xff",a);        //给page0页面的t2文本控件赋值

printf("set.t3.txt=\"abc\r\n123\"\xff\xff\xff");    //给set页面的t3文本控件赋值

printf("set.n0.val=12345\xff\xff\xff");             //给set页面的n0数字控件赋值

printf("sys0=1\xff\xff\xff");                       //
↪给全局变量sys0赋值，sys0定义在program.s中

printf("dim=100\xff\xff\xff");                      //设置亮度，dim是系统变量

printf("volume=100\xff\xff\xff");                   //设置音量，volume是系统变量
```

6.3.5 全局变量-sscom 串口工具发送示例

```
main.t0.txt="abc"\xff\xff\xff

page0.t2.txt="30"\xff\xff\xff

set.t3.txt="abc\\r123"\xff\xff\xff

set.n0.val=12345\xff\xff\xff

sys0=1\xff\xff\xff

dim=100\xff\xff\xff

volume=100\xff\xff\xff
```

多条字符串发送 | stm32/GD32 ISP | STC/IAP15 ISP

拖动加宽 | 循环发送 | 多条帮助 | 导入ini

HEX	字符串(双击注释)	点击发送 + -	顺序	延时 ms
	t0.txt="淘晶驰电子"\xff\xff\xff	发送字符串	1	1000
	n0.val=2333\xff\xff\xff	发送数字	3	1000
	x0.val=314\xff\xff\xff	设置虚拟浮点数	2	1000
	b0.txt="开始"\xff\xff\xff	设置b0文本为开始	0	1000
	b0.txt="停止"\xff\xff\xff	设置b0文本为停止	0	1000
	click b0,1\xff\xff\xff	按下b0	0	1000
	click b0,0\xff\xff\xff	弹起b0	0	1000
	j0.val=0\xff\xff\xff	设置进度条为0	0	1000
	j0.val=25\xff\xff\xff	设置进度条为25	0	1000
	j0.val=50\xff\xff\xff	设置进度条为50	0	1000
	j0.val=75\xff\xff\xff	设置进度条为75	0	1000
	j0.val=100\xff\xff\xff	设置进度条为100	0	1000
	p0.pic=0\xff\xff\xff	设置为id为0的图片	0	1000
	p0.pic=1\xff\xff\xff	设置为id为1的图片	0	1000
	p0.pic=2\xff\xff\xff	设置为id为2的图片	0	1000
	page0.bco=WHITE\xff\xff\xff	设置背景为白色	0	1000
	page0.bco=BLACK\xff\xff\xff	设置背景为黑色	0	1000
	page0.bco=GREEN\xff\xff\xff	设置背景为绿色	0	1000
		19无注释	0	1000
	main.t0.txt="淘晶驰"\xff\xff\xff	20无注释	0	1000
	setting.n0.val=100\xff\xff\xff	21无注释	0	1000
		22无注释	0	1000
		23无注释	0	1000
		24无注释	0	1000
	loginStatus=10\xff\xff\xff	25无注释	0	1000
	bat1.va3.val=40\xff\xff\xff	26无注释	0	1000
	bat1.va2.val=1\xff\xff\xff	27无注释	0	1000
	openDoorIndex=0\xff\xff\xff	28无注释	0	1000
	bat1.va0.val=1\xff\xff\xff		0	1000
	bat1.va1.val=2\xff\xff\xff	电池状态	0	1000
	bat1.va0.val=0\xff\xff\xff	31无注释	0	1000
	bat1.va1.val=1\xff\xff\xff	32无注释	0	1000
	page main\xff\xff\xff	33无注释	0	1000
		34无注释	0	1000

6.3.6 全局变量-样例工程下载

演示工程下载链接：

《全局变量》演示工程下载

6.4 HMI 逻辑语句

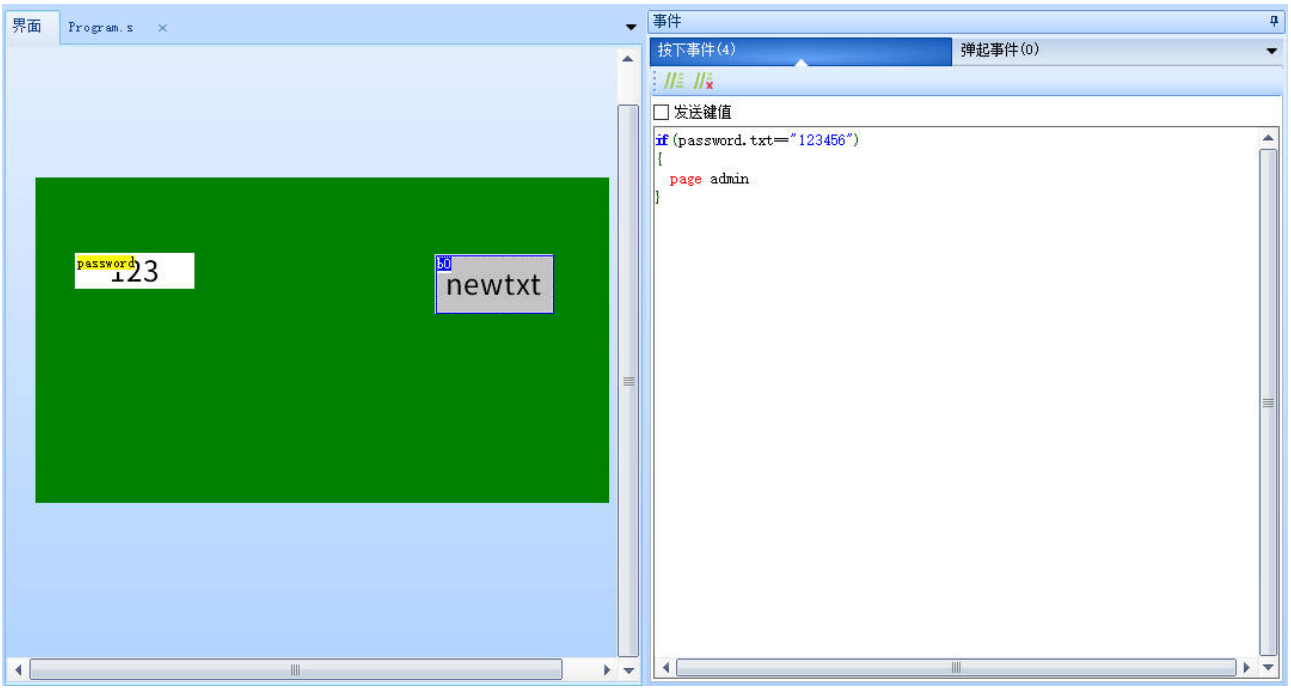
> **注意：** 所有的逻辑语句只能在上位编辑状态下写入控件的事件中，不支持串口传输逻辑语句。
> 所有的判断语句都不支持多余空格，添加进任何空格，编译都会报错

6.4.1 if 语句

6.4.1.1 if-示例 1:

如果 password.txt 等于”123456”那么就切换到 admin 页面

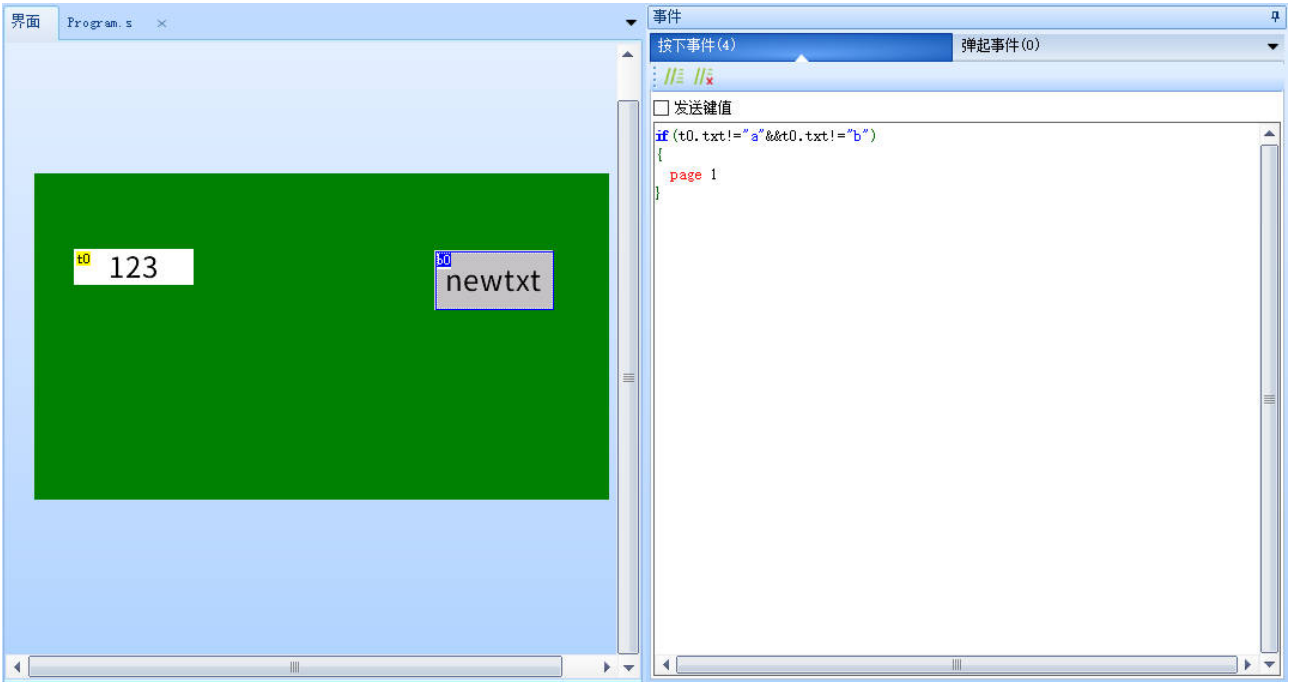
```
if(password.txt=="123456")
{
   page admin
}
```



6.4.1.2 if-示例 2:

如果 t0.txt 不等于”a”也不等于 b”那么就切换到页面 1

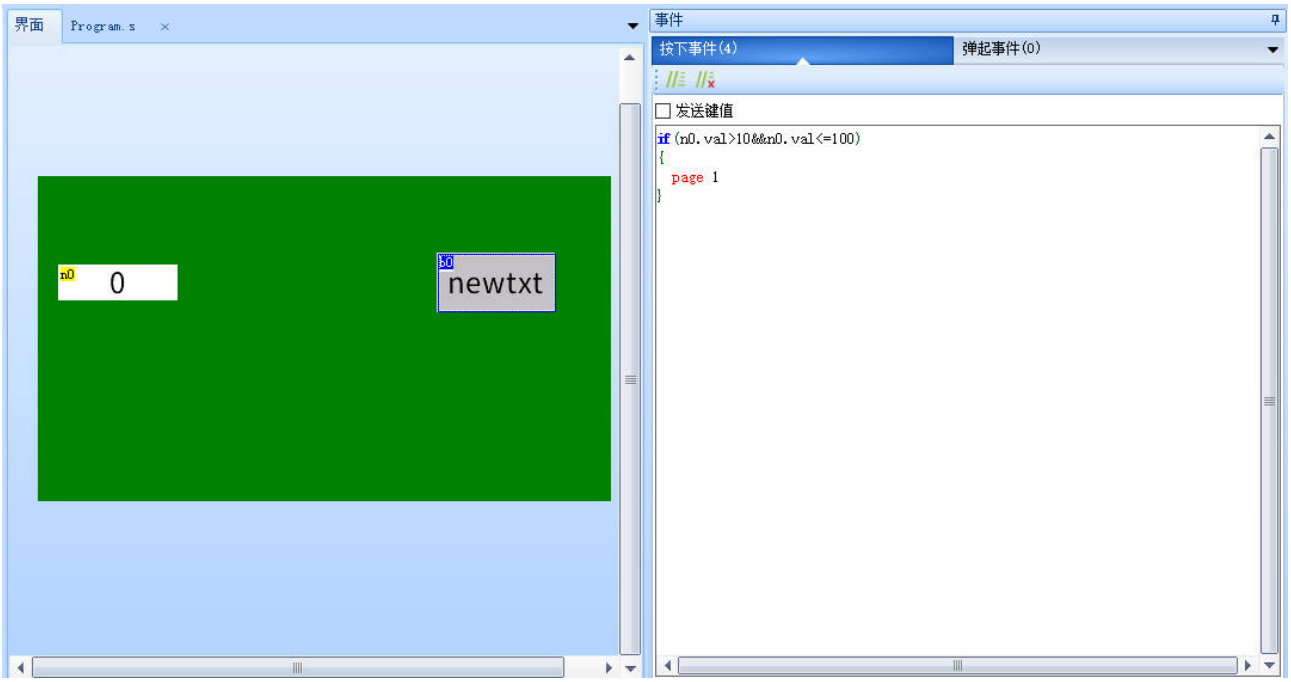
```
if(t0.txt!="a"&&t0.txt!="b")
{
    page 1
}
```



6.4.1.3 if-示例 3:

如果 10 < n0.val ≤ 100 那么就切换到页面 1

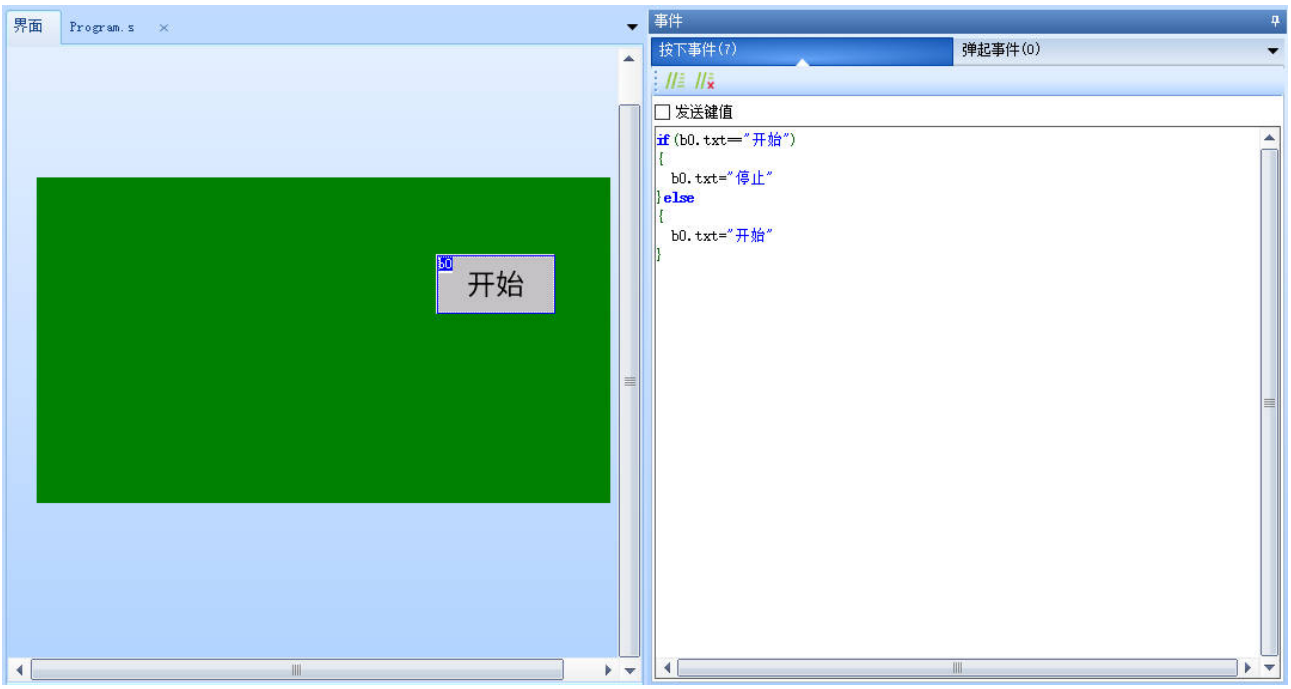
```
if(n0.val>10&&n0.val<=100)
{
    page 1
}
```



6.4.1.4 if-示例 4:

以下语句写在 b0 按钮的按下事件中将实现 b0 的 txt 内容在开始和停止之间来回切换

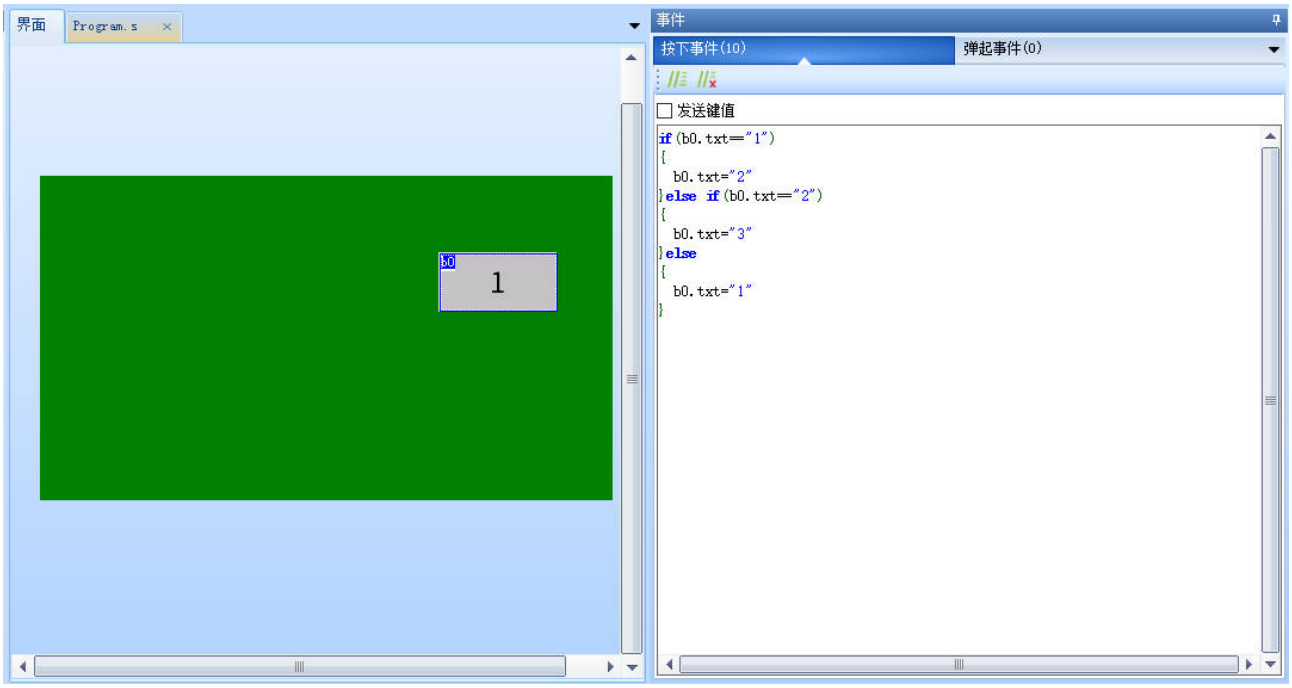
```
if(b0.txt=="开始")
{
   b0.txt="停止"
}else
{
   b0.txt="开始"
}
```



6.4.1.5 if-示例 5:

以下语句写在 b0 按钮的按下事件中将实现 b0 的 txt 内容在 1,2,3 之间来回切换

```
if(b0.txt=="1")
{
   b0.txt="2"
}else if(b0.txt=="2")
{
   b0.txt="3"
}else
{
   b0.txt="1"
}
```



注意: 1. 数值类型变量支持:

大于 >

小于 <

等于 ==

不等于!=

大于等于 >=

小于等于 <=

2. 字符串类型仅支持:

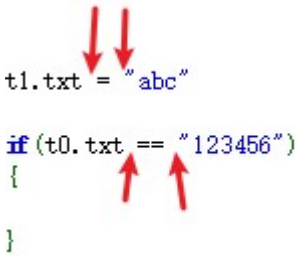
等于 ==

不等于!=

3.if 判断的时候不支持括号优先级，比如:if((t0.txt==”a”||t0.txt==”b”)&&t1.txt==”1”) 这样是不支持的。

4.if 判断的时候不允许出现运算，比如:if(n0.val+2==3) 这样是不支持的，请提前用中间变量计算好，再在 if 中判断

5. 不要出现多余的空格, 会导致编译时报错

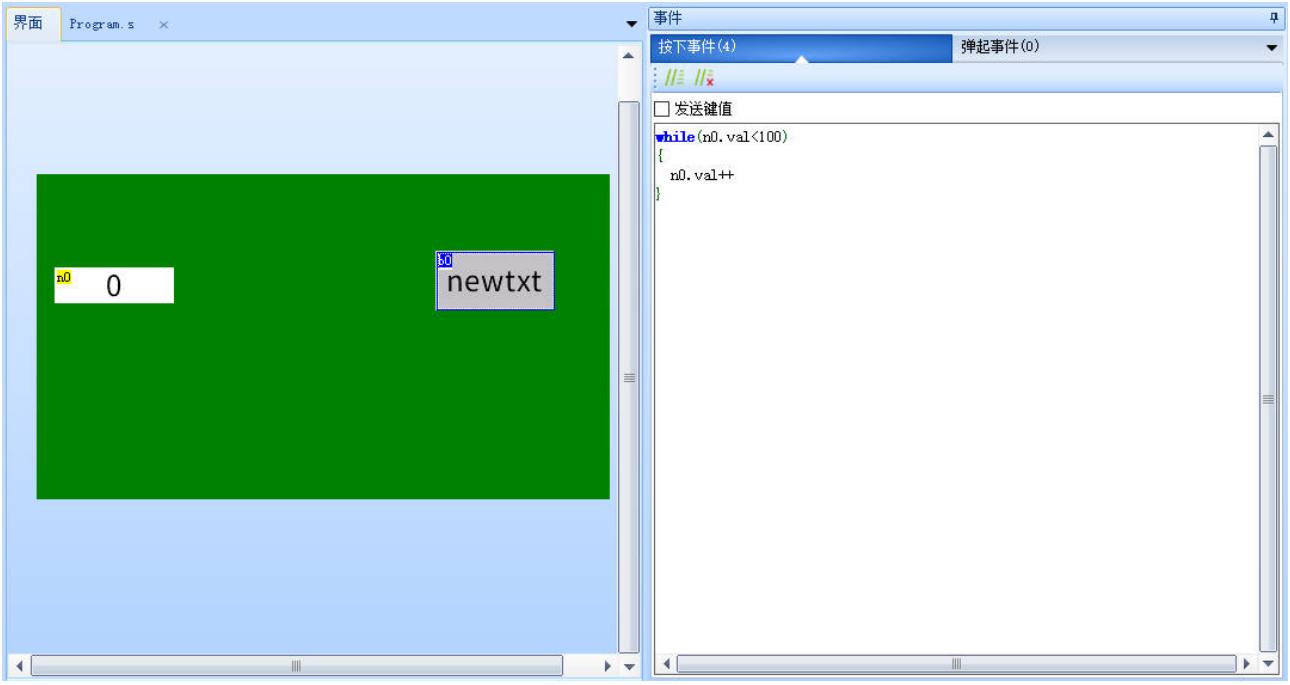

t1.txt = "abc"
if (t0.txt == "123456")
{
}

6.4.2 while 语句

6.4.2.1 while-示例 1:

n0.val 一直自加到 100 为止，在自加过程中屏幕不会刷新显示，直到整个过程所有语句结束

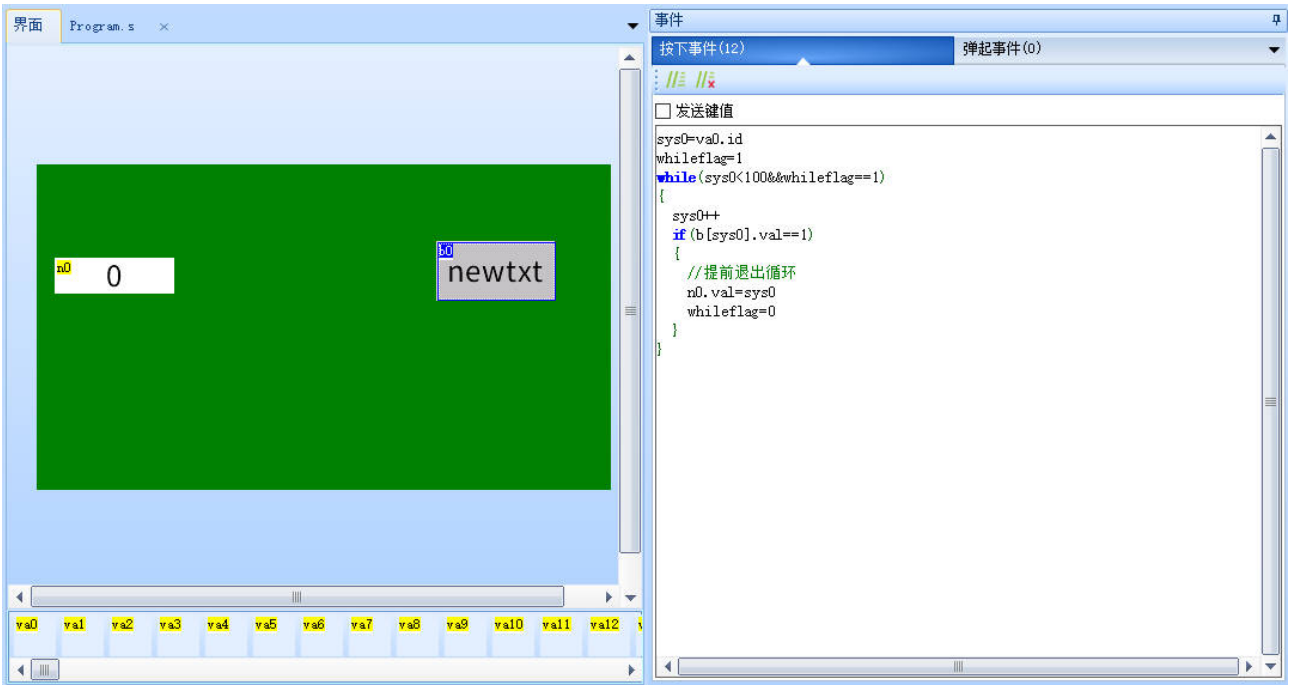
```
while(n0.val<100)
{
   n0.val++
}
```



6.4.2.2 while-示例 2:

while 提前退出循环

```
sys0=va0.id
whileflag=1
while(sys0<100&&whileflag==1)
{
   sys0++
   if(b[sys0].val==1)
   {
      //提前退出循环
      n0.val=sys0
      whileflag=0
   }
}
```



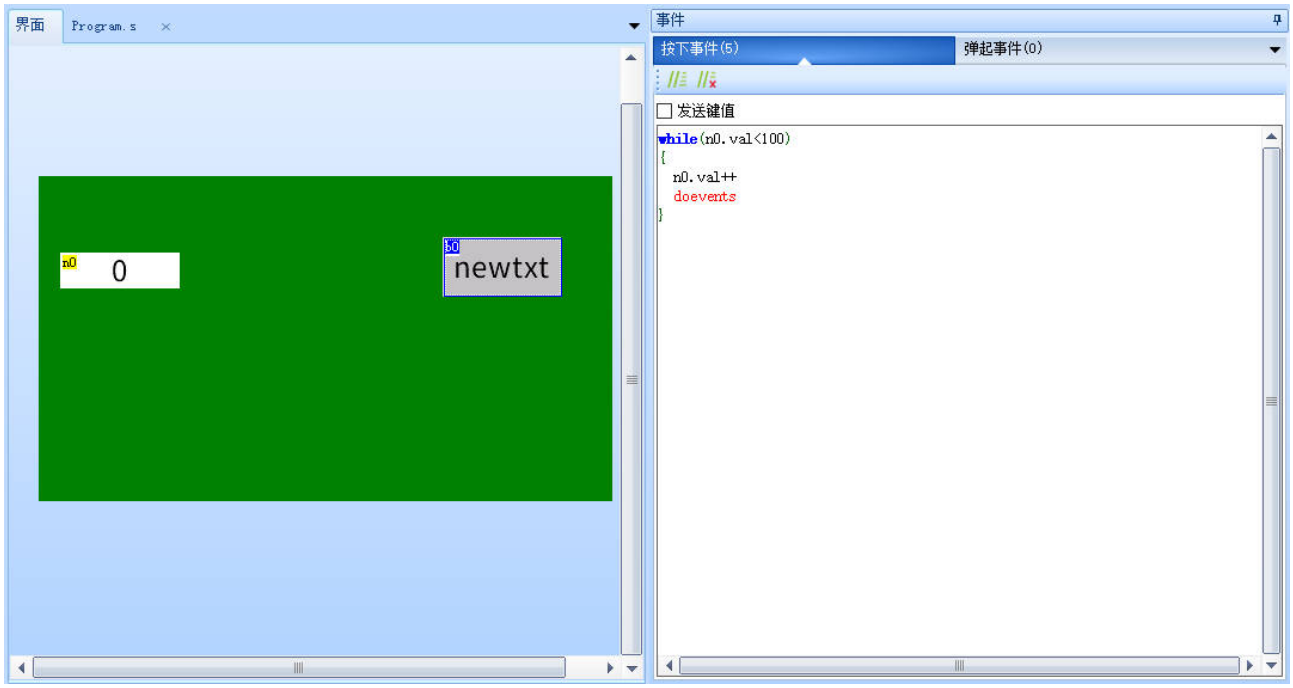
6.4.2.3 while-示例 3:

n0.val 一直自加到 100 为止，在自加过程中屏幕会一直不断的刷新 n0 控件的显示

注意： 1. 在一个较多指令的过程执行中，或者在一个较长时间的循环语句中，系统所有控制权被此过程全部占用，在过程结束之前，尽管相应的内存数据可以任意正常读写，但是屏幕不会刷新显示，加入 doevents 后可以转让控制权给屏幕刷新，执行 doevents 之后，屏幕会刷新所有被改变过的控件，刷新完之后，控制权交回当前过程继续执行。防止屏幕呈现假死的显示状态。

2.while 语句循环过程中，设备不会响应触摸事件，串口指令会接收到缓冲区，但不会执行，直到当前过程所有语句执行完毕为止，请慎重使用，以防进入死循环。

```
while(n0.val<100)
{
   n0.val++
   doevents
}
```

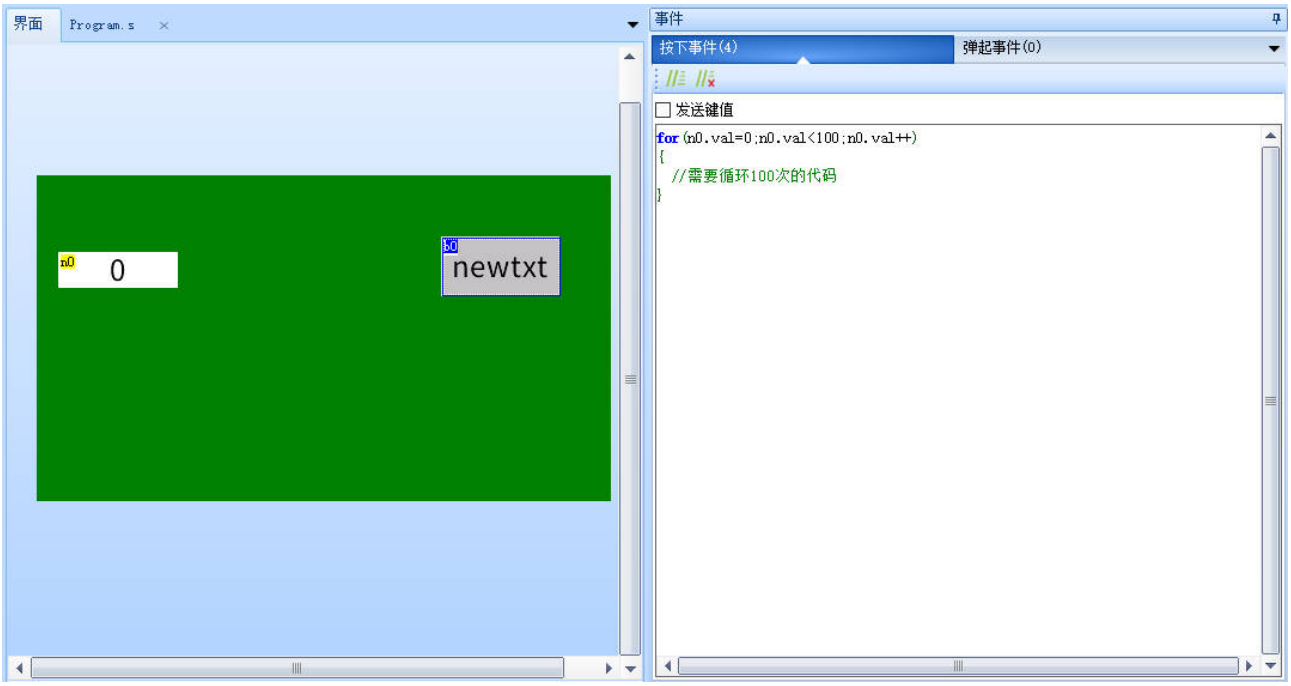

界面
Program.s
n0
0
b0
newtxt
事件
按下事件(5)
弹起事件(0)
发送键值
while(n0.val<100)
{
n0.val++
doevents
}

6.4.3 for 语句

6.4.3.1 for-示例 1:

n0.val 一直自加到 100 为止，在自加过程中屏幕不会刷新显示，直到整个过程所有语句结束

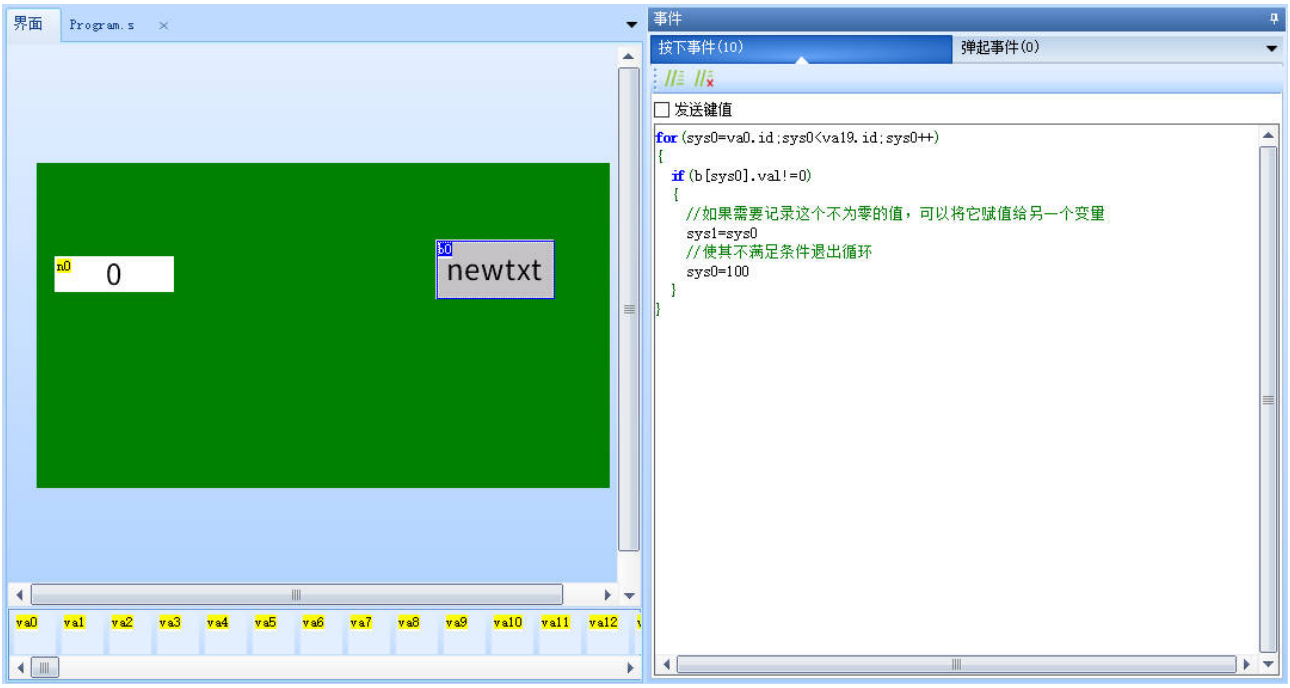
```
for(n0.val=0;n0.val<100;n0.val++)
{
    //需要循环100次的代码
}
```



6.4.3.2 for-示例 2:

满足条件后退出 for 循环

```
for(sys0=n0.id;sys0<n19.id;sys0++)
{
  if(b[sys0].val!=0)
  {
     //如果需要记录这个不为零的值，可以将它赋值给另一个变量
     sys1=sys0

     //使其不满足条件退出循环
     sys0=100
  }
}
```



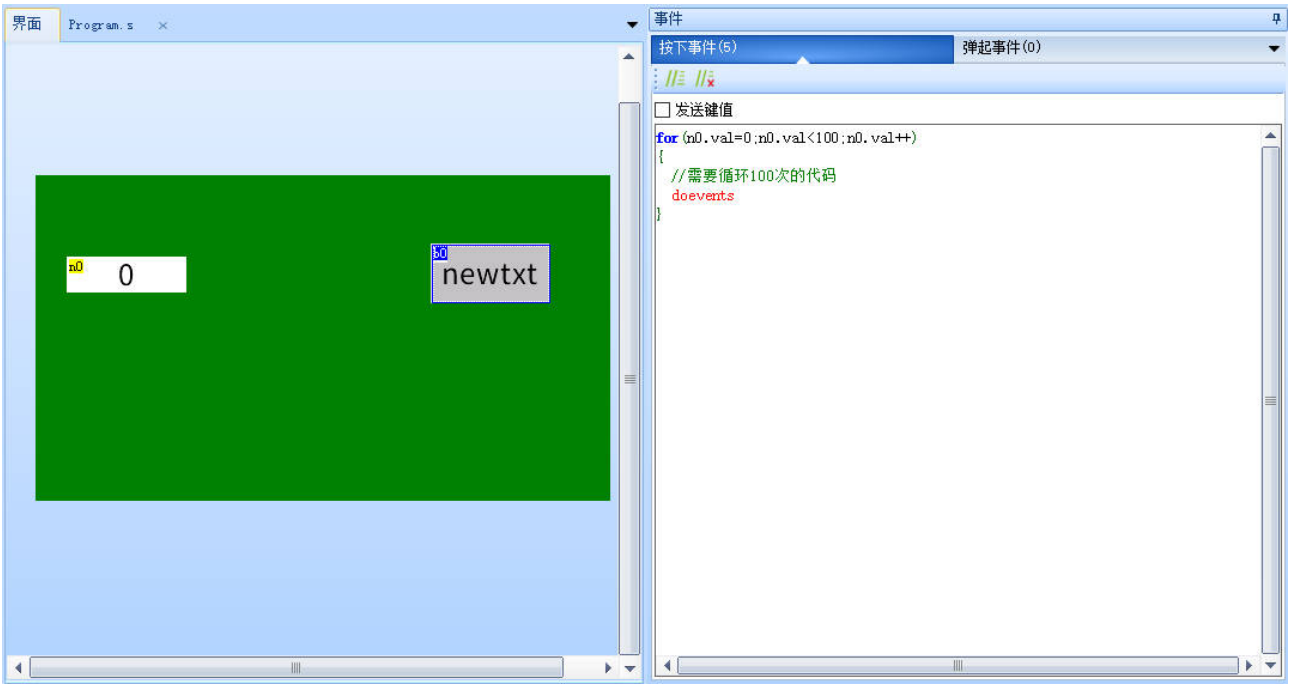
6.4.3.3 for-示例 3:

n0.val 一直自加到 100 为止，在自加过程中屏幕会一直不断的刷新 n0 控件的显示

注意： 1. 在一个较多指令的过程执行中，或者在一个较长时间的循环语句中，系统所有控制权被此过程全部占用，在过程结束之前，尽管相应的内存数据可以任意正常读写，但是屏幕不会刷新显示，加入 doevents 后可以转让控制权给屏幕刷新，执行 doevents 之后，屏幕会刷新所有被改变过的控件，刷新完之后，控制权交回当前过程继续执行。防止屏幕呈现假死的显示状态。

2.for 语句循环过程中，设备不会响应触摸事件，串口指令会接收到缓冲区，但不会执行，直到当前过程所有语句执行完毕为止，请慎重使用，以防进入死循环。

```
for(n0.val=0;n0.val<100;n0.val++)
{
   //需要循环100次的代码
   doevents
}
```


界面
Program.s
n0
0
b0
newtxt
事件
按下事件(5)
弹起事件(0)
发送键值
for (n0.val=0;n0.val<100;n0.val++)
{
//需要循环100次的代码
doevents
}

6.5 数组/名称组使用说明

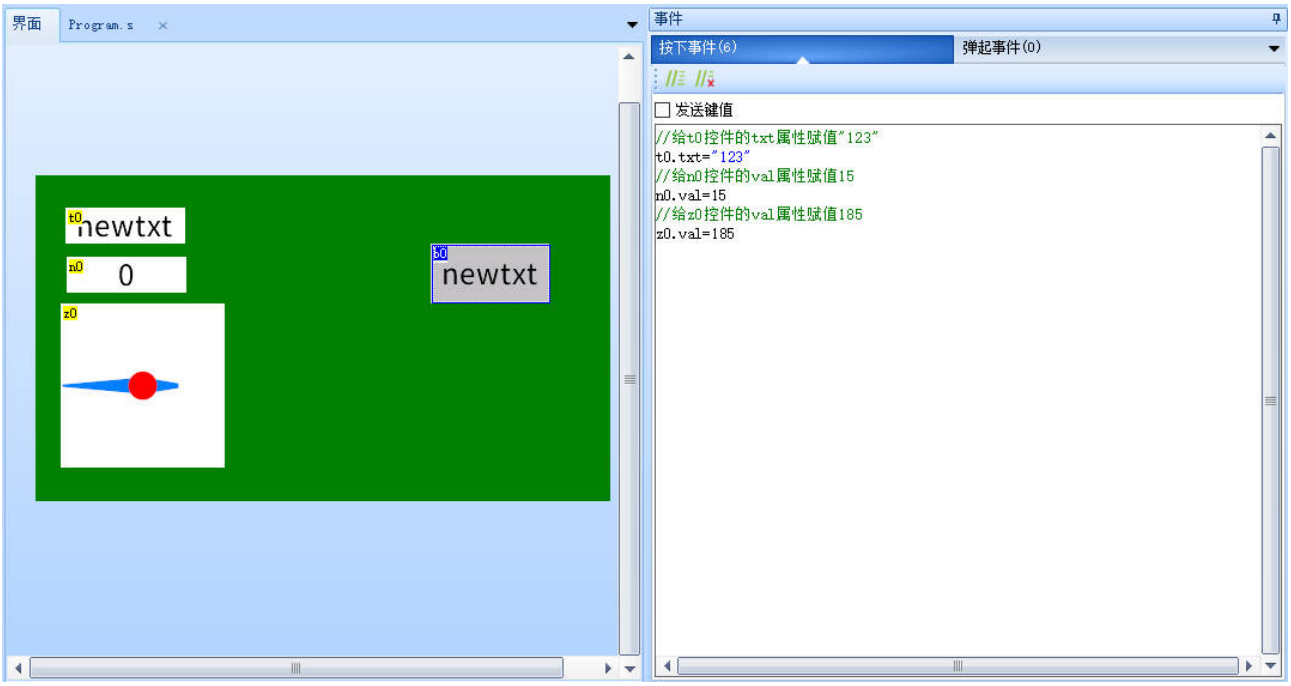
6.5.1 什么是名称组, 名称组和数组什么关系

名称组功能即创建一系列连续的控件来达到数组的效果

6.5.2 名称组的使用

大多数情况下，我们操作控件属性是这样的 (这也是我们推荐的控件操作方式):

```
//给t0控件的txt属性赋值"123"
t0.txt="123"

//给n0控件的val属性赋值15
n0.val=15

//给z0控件的val属性赋值185
z0.val=185
```



假如我们不知道控件名字，只知道控件 ID 怎么操作呢？这就需要名称组来实现

控件名称组格式: p[页面 DP 号].b[控件 ID]. 属性

注意: 这里的 p 是 page（页面）的缩写，b 是 object（对象）的缩写，而不是指 button（按钮），名称组访问控件，只有控件 id 的概念，没有控件类型的概念

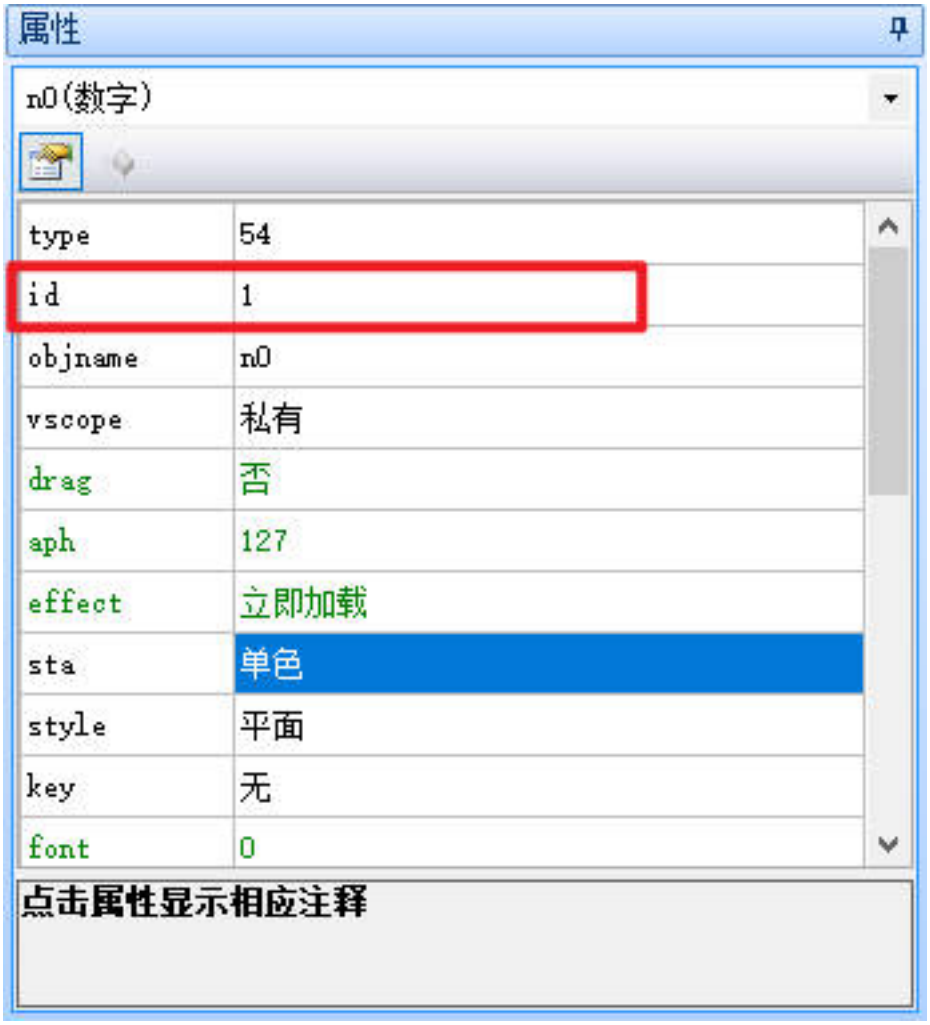
注意: 这里的 p 是 page（页面）的缩写，b 是 object（对象）的缩写，而不是指 button（按钮），名称组访问控件，只有控件 id 的概念，没有控件类型的概念

注意: 这里的 p 是 page（页面）的缩写，b 是 object（对象）的缩写，而不是指 button（按钮），名称组访问控件，只有控件 id 的概念，没有控件类型的概念

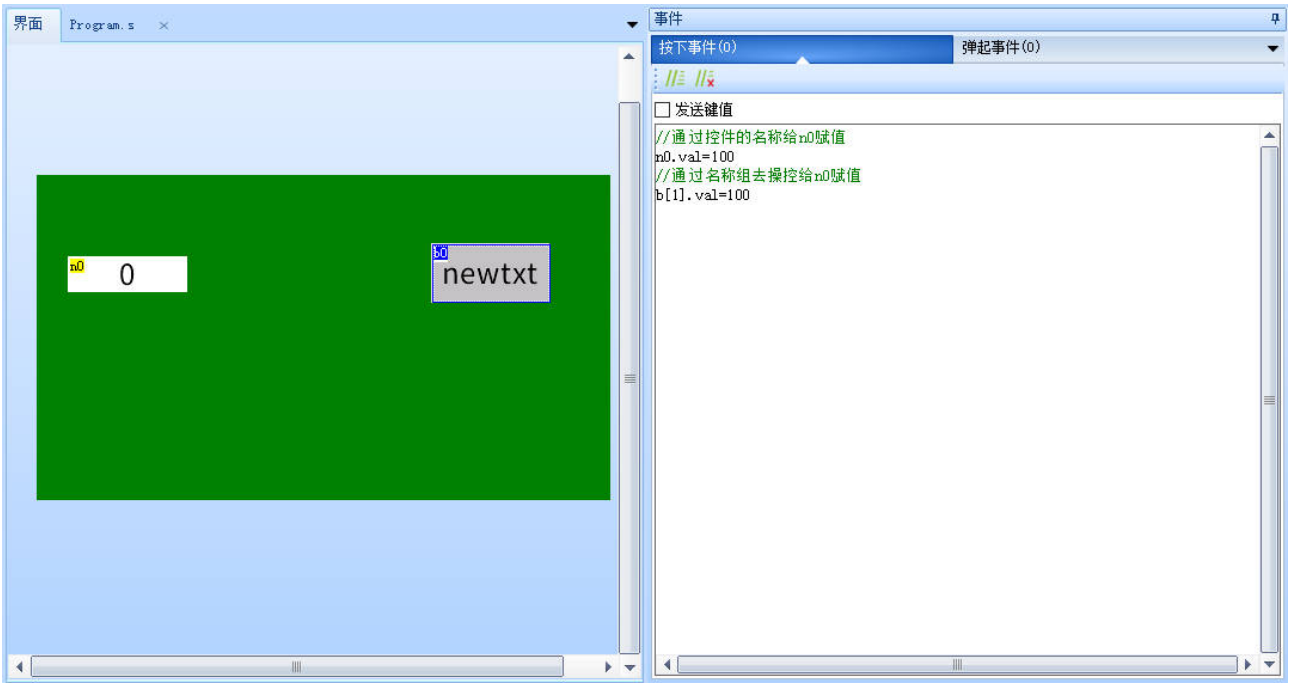
注意： 这里的 p 是 page（页面）的缩写，b 是 object（对象）的缩写，而不是指 button（按钮），名称组访问控件，只有控件 id 的概念，没有控件类型的概念

注意： 这里的 p 是 page（页面）的缩写，b 是 object（对象）的缩写，而不是指 button（按钮），名称组访问控件，只有控件 id 的概念，没有控件类型的概念

例如我们知道了 n0 的 id 号是 1

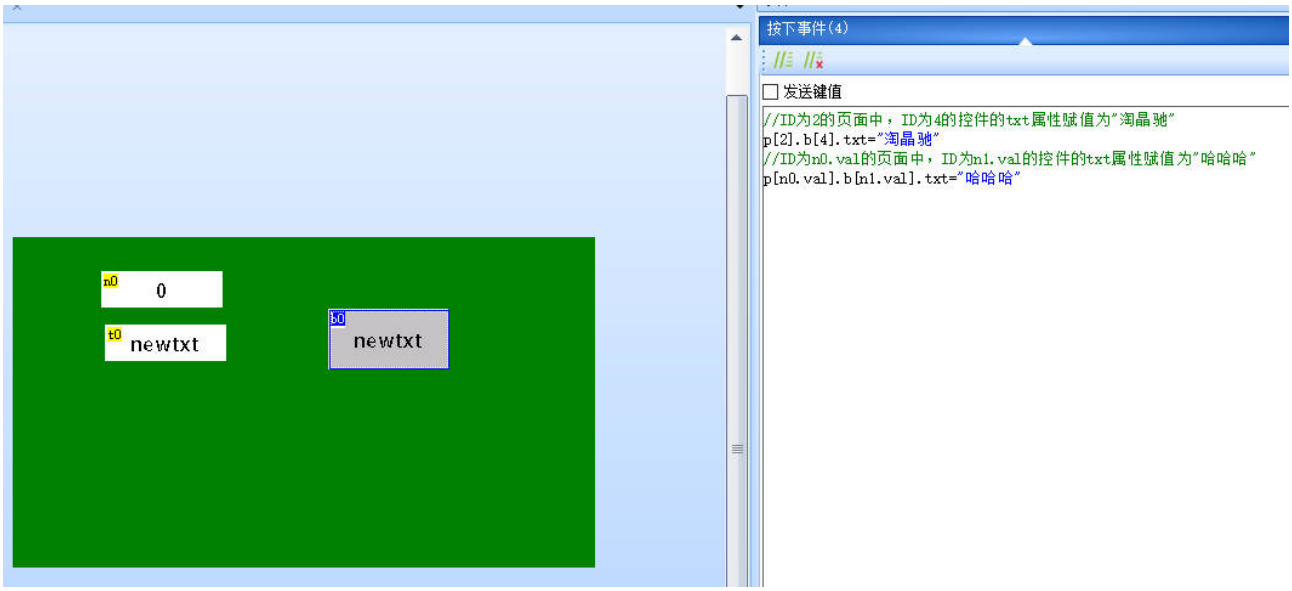


那么我们就可以使用以下两种方式来操作 n0



页面名称组格式:p[页面 DP 号].b[控件 ID 号]. 属性

```
//ID为2的页面中，ID为4的控件的txt属性赋值为"淘晶驰"
p[2].b[4].txt="淘晶驰"
//ID为n0.val的页面中，ID为n1.val的控件的txt属性赋值为"哈哈哈"
p[n0.val].b[n1.val].txt="哈哈哈"
```

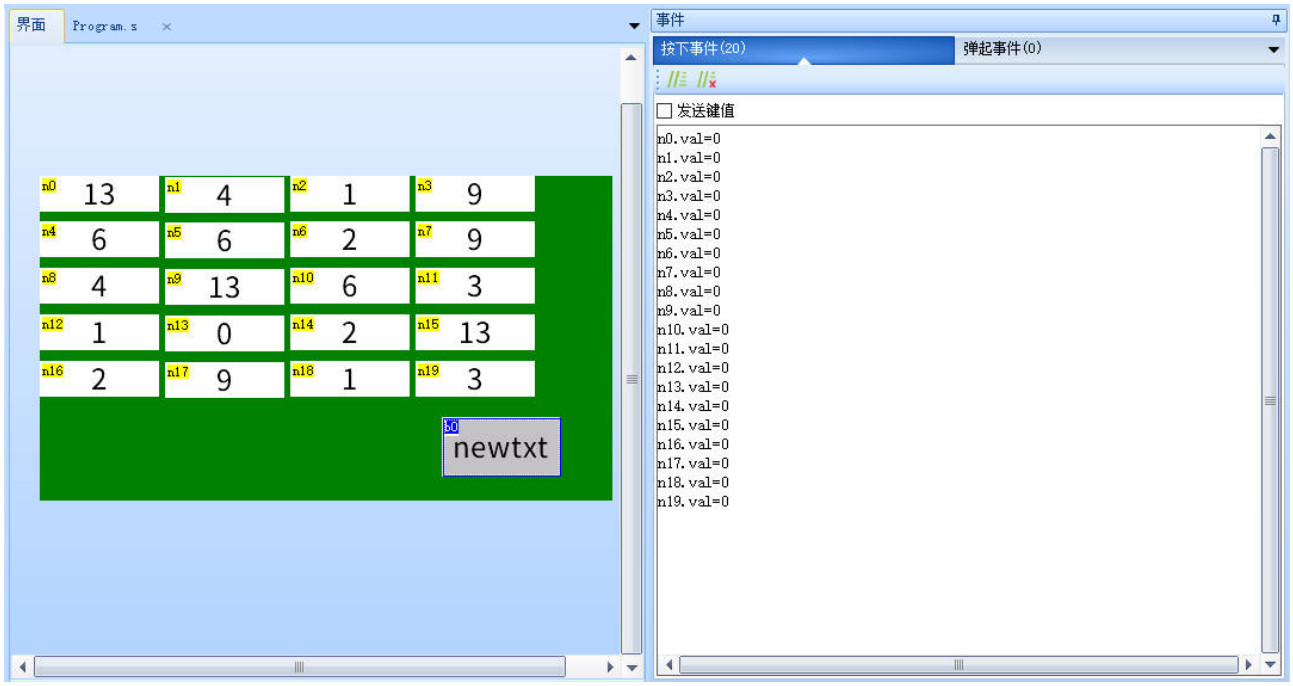


注意: 必须确保此页面中的此控件具有对应的属性, 否则赋值会失败, 例如你不能给数值控件赋值 txt 属性, 虽然能编译通过, 但是实际赋值时会返回错误信息

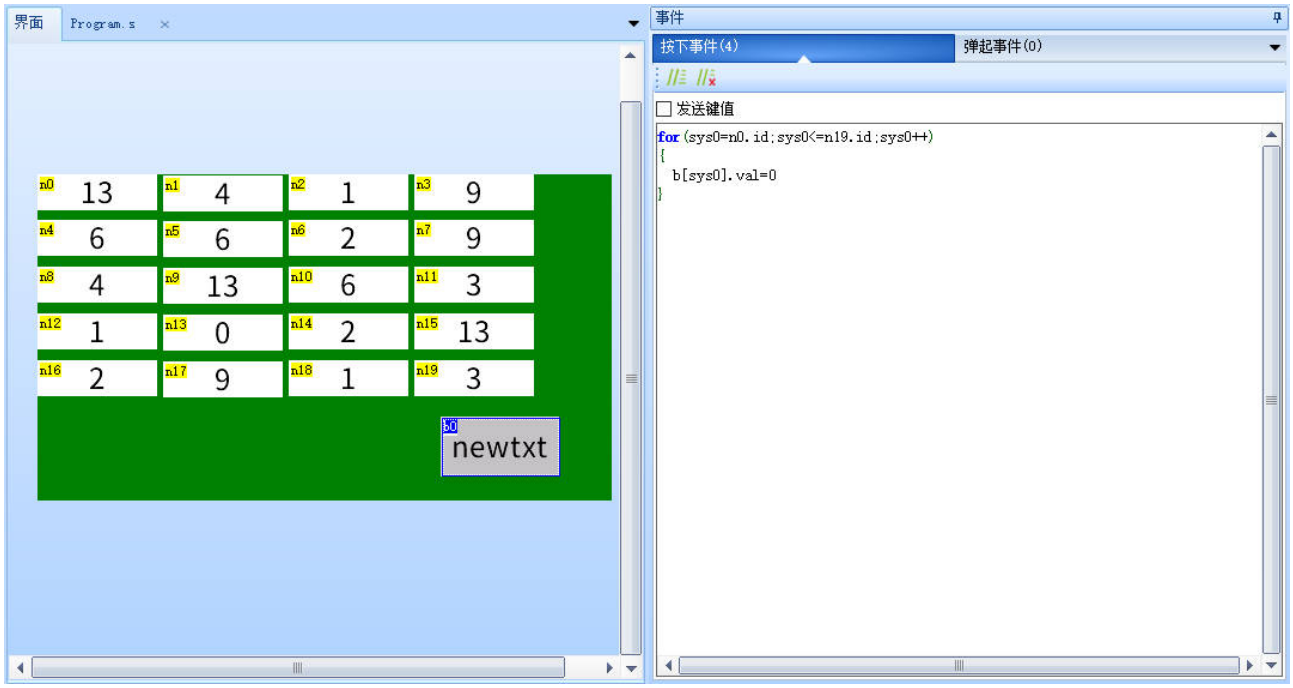
6.5.3 怎么使用名称组

例如我们要创建一个长度为 20 的名称组, 于是我们添加了 20 个数字控件,

有 n0-n19 共 20 个控件，如果要把他们全部初始化为 0，第一种方法是逐个赋值为 0。

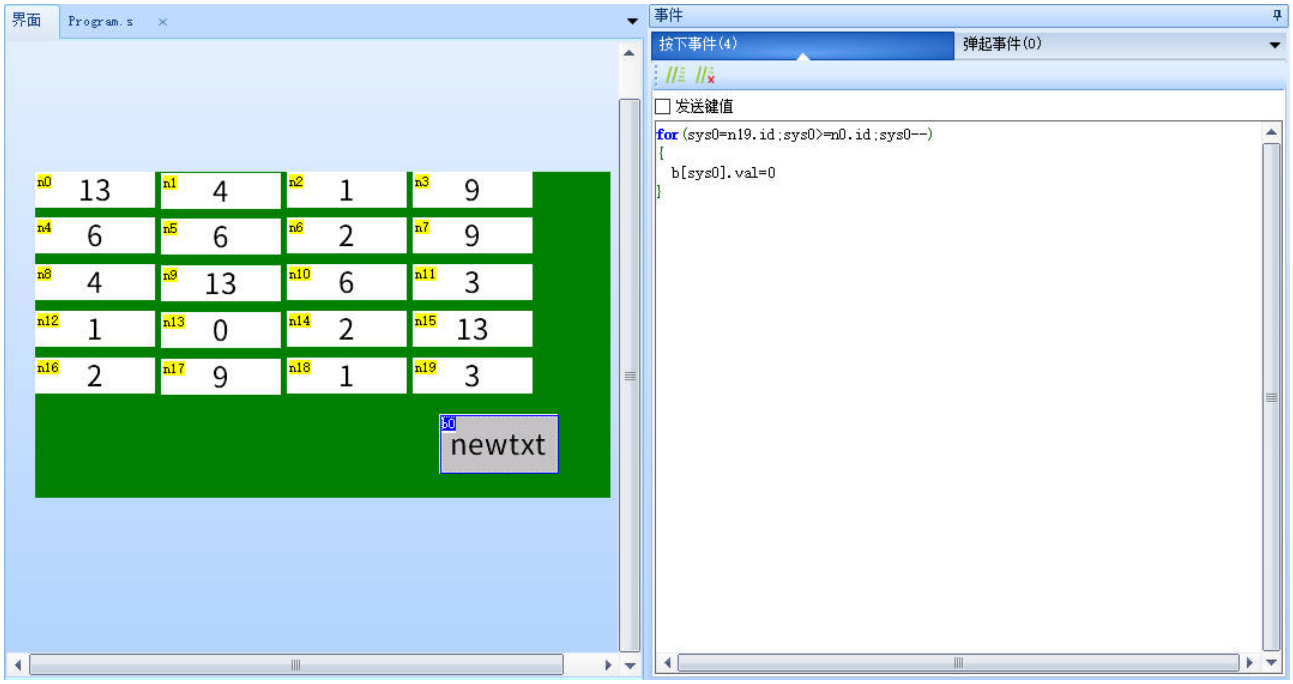


第二种方法，使用名称组进行操作，代码如下

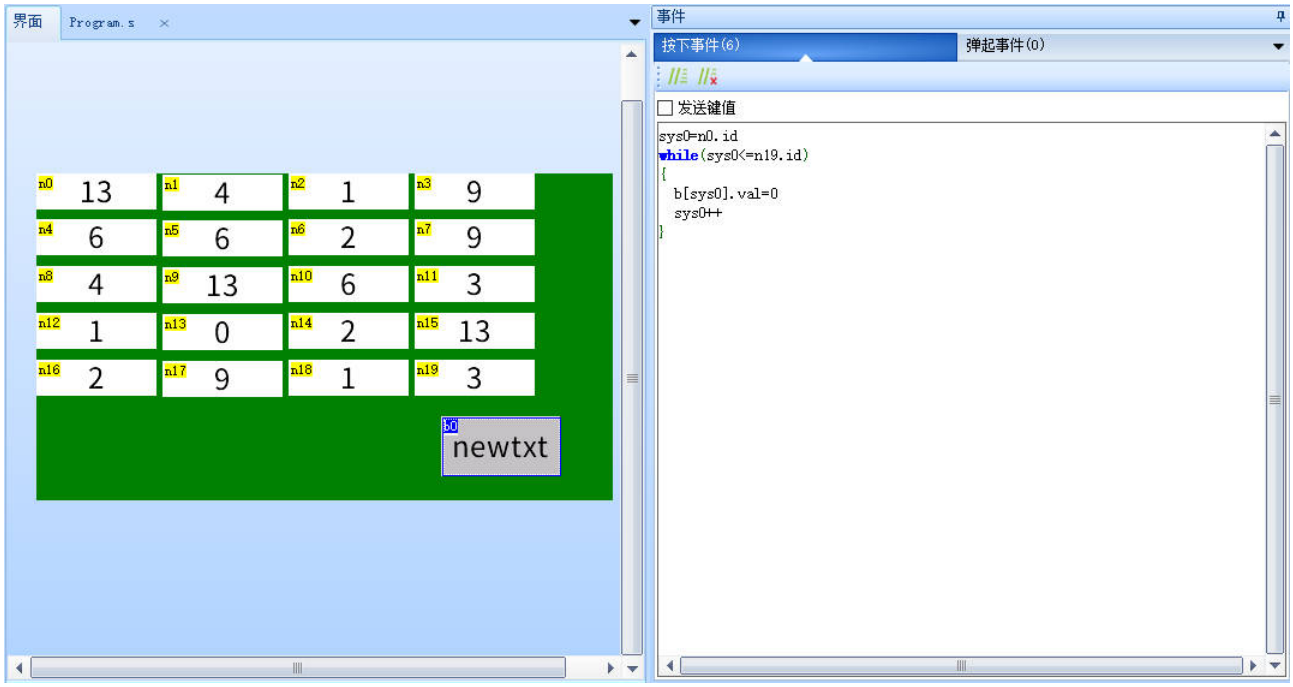


以下展示几种等价的写法:

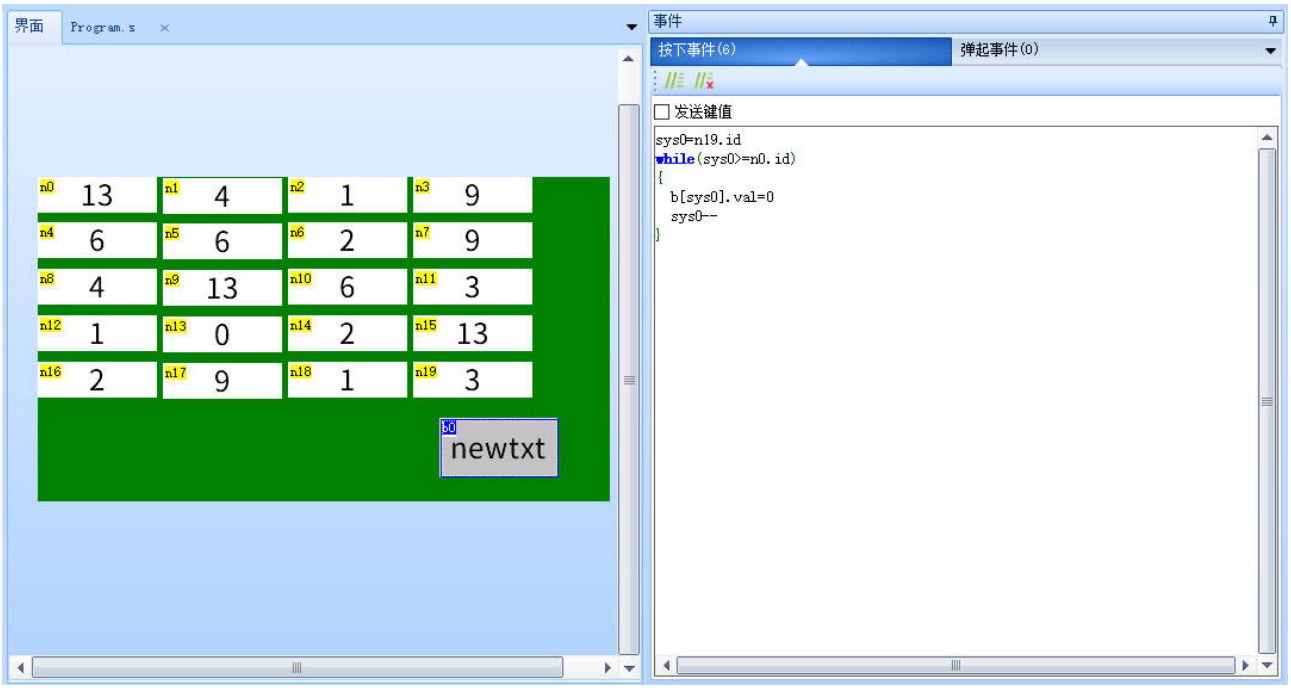
等价写法一:


界面
Program.s
事件
按下事件(4)
弹起事件(0)
发送键值
for(sys0=n19.id;sys0>=n0.id;sys0--)
{
b[sys0].val=0
}
n0 13
n1 4
n2 1
n3 9
n4 6
n5 6
n6 2
n7 9
n8 4
n9 13
n10 6
n11 3
n12 1
n13 0
n14 2
n15 13
n16 2
n17 9
n18 1
n19 3
b0
newtxt

等价写法二:


界面
Program.s
事件
按下事件(6)
弹起事件(0)
发送键值
sys0=n0.id
while(sys0<=n19.id)
{
b[sys0].val=0
sys0++
}
n0 13
n1 4
n2 1
n3 9
n4 6
n5 6
n6 2
n7 9
n8 4
n9 13
n10 6
n11 3
n12 1
n13 0
n14 2
n15 13
n16 2
n17 9
n18 1
n19 3
b0
newtxt

等价写法三:

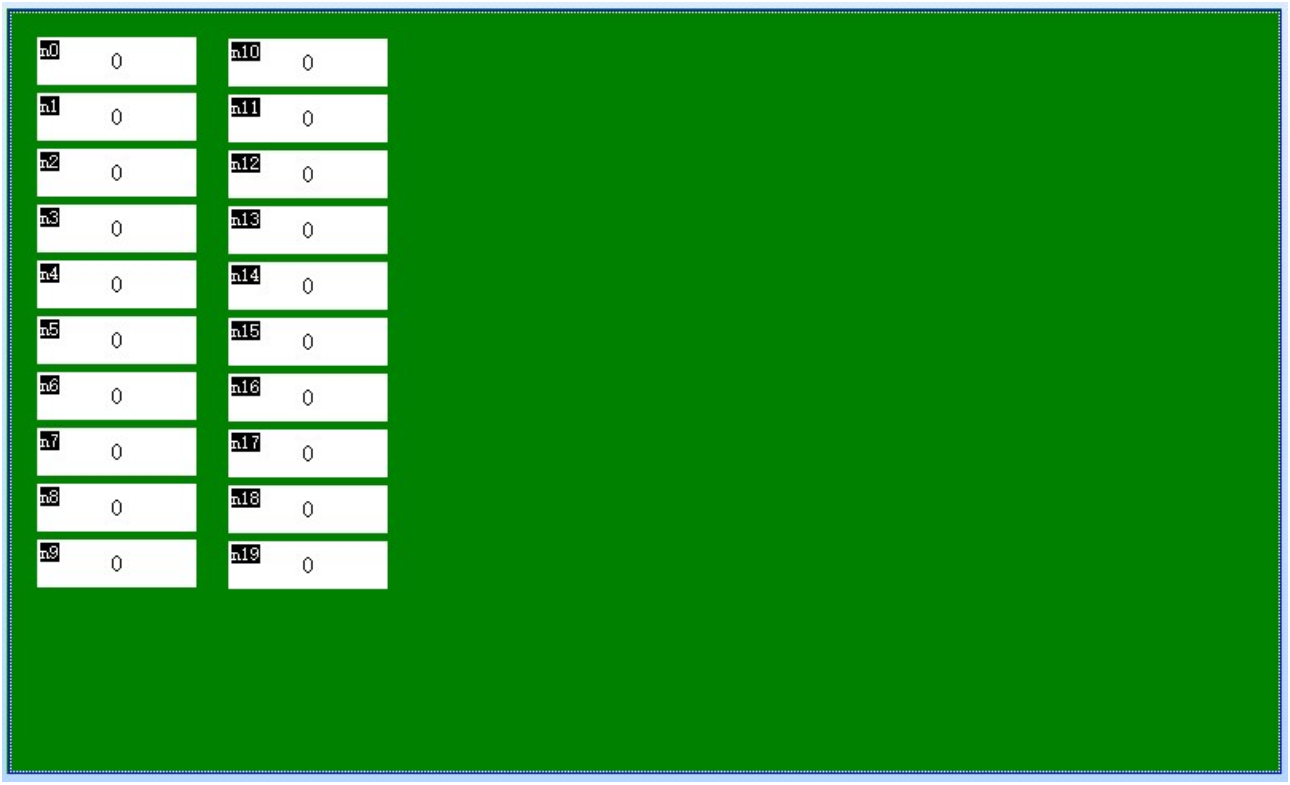


注意： 当前演示的工程中,n0-n19 的 id 号是连续且递增的，n0 的 id 号最小，n19 的 id 号最大

6.5.4 名称组应用-实现数组

示例：创建一个长度为 20 的数组

在 page0 页面新建 20 个数字控件，将所有变量控件设置为全局，这样才能跨页面访问



n0 的 id 是 1，n1 的 id 是 2，以此类推，n19 的 id 是 20

此时就可以通过名称组的方式来实现数组

代码示例：配合 for 语句批量将整个数组赋值为 100

```
for(sys0=n0.id;sys0<=n19.id;sys0++)
{
   page0.b[sys0].val=100
}
```

6.5.5 名称组常见操作方式

6.5.5.1 批量读取掉电存储空间数据

将掉电存储空间地址 100 开始连续的值加载到数值控件 va0 ~ va19，请确保 va0 ~ va19 的 id 号是连续的

```
//设置掉电存储空间读取位置
sys1=100

//for循环读取掉电存储空间并赋值给数值控件
for(sys0=n0.id;sys0<=n19.id;sys0++)
{
   //读取指定位置的掉电存储空间数据
   repo b[sys0].val,sys1

   //掉电存储空间读取地址加4，因为一个数值类型占用4byte空间。
   sys1+=4
}
```

注意： 必须确保所操纵的控件都具有对应的属性, 否则可能会报错

6.5.5.2 批量隐藏控件 n0 ~ n99

请确保 n0 ~ n99 的 id 号是连续的

```
//for循环读取掉电存储空间并赋值给数值控件
for(sys0=n0.id;sys0<=n99.id;sys0++)
{
   vis b[sys0],0
}
```

6.5.5.3 批量清空文本控件 t0 ~ t5

请确保 t0 ~ t5 的 id 号是连续的

```
for(sys0=t0.id;sys0<=t5.id;sys0++)
{
   b[sys0].txt=""
}
```

6.5.5.4 批量将文本控件 t0 ~ t5 赋值为“淘晶驰”

请确保 t0 ~ t5 的 id 号是连续的

```
for(sys0=t0.id;sys0<=t5.id;sys0++)
{
   b[sys0].txt="淘晶驰"
}
```

6.5.5.5 批量将数字控件 n0 ~ n5 赋值为 0

请确保 n0 ~ n5 的 id 号是连续的

```
for(sys0=n0.id;sys0<=n5.id;sys0++)
{
   b[sys0].val=0
}
```

所有控件的 ID 号软件自动分配，不可手动设置，用户在编辑 UI 界面时，按顺序依次放置的控件，软件将连续分配 ID。

使用快捷栏的“置顶”、“置底”功能会使控件 ID 发生变化，因为图层的前后关系是跟控件 ID 关联的，ID 最小的在最底层 (所以页面 ID 是 0),ID 最大的在最上层，每个控件都有自己的图层，全部是通过 ID 来区别前后关系的。

注意： 带有 for 的逻辑语句只能在上位编辑状态下写入控件的事件中，不支持串口传输逻辑语句。

6.5.6 名称组使用-样例工程下载

演示工程下载链接：

《名称组使用》演示工程下载

《名称组使用 2》演示工程下载

《通过按键移动选择控件》演示工程下载

7 串口屏返回数据格式

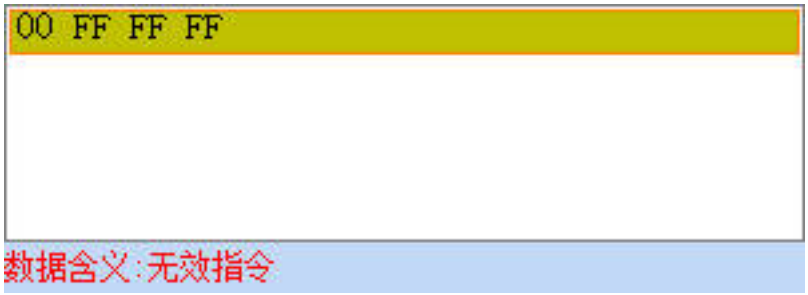
7.1 bkcmd 非 0 时的通知格式

1. 只有当系统变量 bkcmd 为非 0 的时候才会返回指令执行成功或者失败数据，每次上电后 bkcmd 默认为 2，即只返回指令执行出错的结果。

2. 上位软件编辑时写进资源文件的代码不受 bkcmd 影响，执行有错误时一定会返回错误数据，成功时不返回数据。

7.1.1 00 FF FF FF 无效指令

0X00+ 结束符



十进制数据为：0，255，255，255

当收到用户发来的无效指令时返回此数据

有以下几个原因导致：

单片机是否发送了错误的数据到串口屏

单片机是否发送了无关的数据（例如调试信息到串口屏）

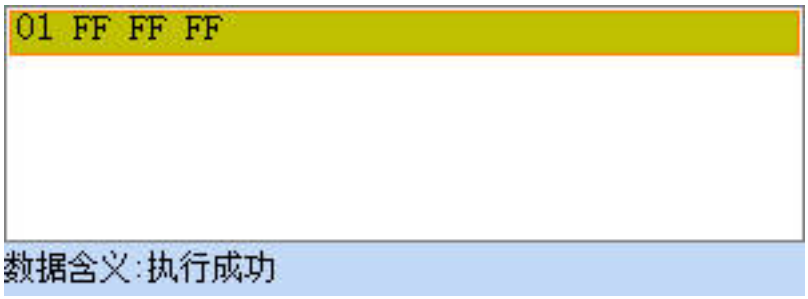
当前页面是否包含某个控件（跳转页面导致当前页面不包含此控件）

等等

建议单片机仅向屏幕的全局变量赋值，避免向页面私有变量赋值，参考跨页面赋值，全局变量操作 (page 847)

7.1.2 01 FF FF FF 指令成功执行

0X01+ 结束符



十进制数据为：1，255，255，255

用户发来的指令被成功执行完毕时返回此数据

7.1.3 02 FF FF FF 控件 ID 无效

0X02+ 结束符

十进制数据为：2，255，255，255

用户发来的指令中包含无效控件 ID 或者无效控件名称时返回此数据

7.1.4 03 FF FF FF 页面 ID 无效

0X03+ 结束符

十进制数据为：3，255，255，255

用户发来的指令中包含无效页面 ID 或者无效页面名称时返回此数据

当跳转到一个不存在的页面时

```
page page999
```

7.1.5 04 FF FF FF 图片 ID 无效

0X04+ 结束符

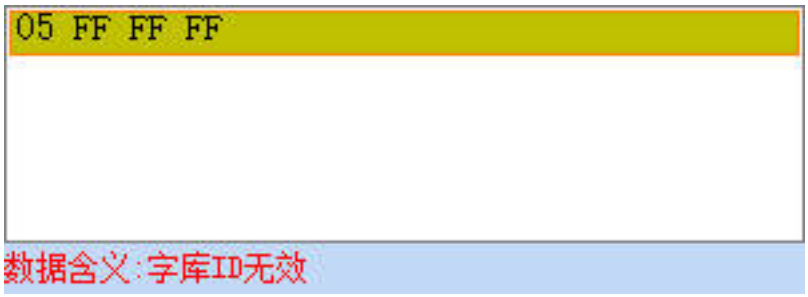
十进制数据为：4，255，255，255

用户发来的指令中包含无效图片 ID 时返回此数据，也有可能返回 1C FF FF FF （赋值操作失败）

在 1.65 版本中, 如果控件的 pic/pic2/picc/picc2 等需要调用到图片的属性为空时, 当该控件被初始化或者触发时, 也会返回 04 FF FF FF

7.1.6 05 FF FF FF 字库 ID 无效

0X05+ 结束符

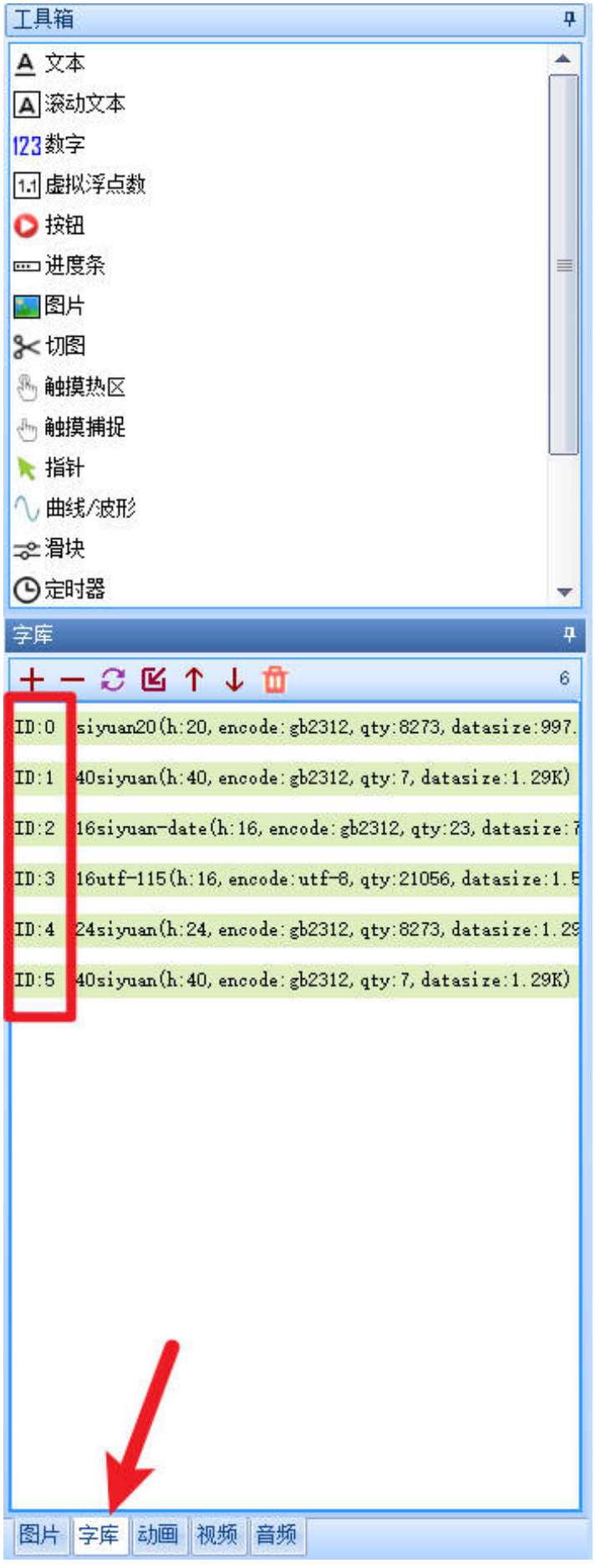


十进制数据为：5，255，255，255

用户发来的指令中包含无效字库 ID 时返回此数据

也有可能返回 1C FF FF FF （赋值操作失败）

配置字库时, 应保证 font 属性设置的值所对应的字库是存在的


工具箱
文本
滚动文本
数字
虚拟浮点数
按钮
进度条
图片
切图
触摸热区
触摸捕捉
指针
曲线/波形
滑块
定时器
字库
6
ID:0 siyuan20(h:20, encode:gb2312, qty:8273, datasize:997.
ID:1 40siyuan(h:40, encode:gb2312, qty:7, datasize:1.29K)
ID:2 16siyuan-date(h:16, encode:gb2312, qty:23, datasize:7
ID:3 16utf-115(h:16, encode:utf-8, qty:21056, datasize:1.5
ID:4 24siyuan(h:24, encode:gb2312, qty:8273, datasize:1.29
ID:5 40siyuan(h:40, encode:gb2312, qty:7, datasize:1.29K)
图片
字库
动画
视频
音频

参考:

创建字库和导入字库 (page 125)
指定字库 (page 1284)
在做字库的时候有部分字体无法选择 (page 1360)
编译报错：*XXX* 初始值无效 (page 1342)
utf8 字库下如何只选择汉字 (page 1403)

7.1.7 06 FF FF FF 文件操作失败

0X06+ 结束符

十进制数据为：6，255，255，255

操作文件失败的时候返回

拼接文件路径时请注意使用的文本控件的 txt_maxl 属性是否足够

请注意文件是否存在

是否多个文件流打开了同一个文件

检查文件的大小是否为 0

使用视频控件和音频文件，如果是调用 sd 卡的文件时，请检查电脑是否隐藏了文件的扩展名

例如文件全名为“1.video”，当隐藏了拓展名时，文件名只会显示“1”，客户如果填写了路径为“sd0/1”，那就会错误

如果客户手动添加了“.video”拓展名，真实文件名又会变成 1.video.video。仍然会报错

7.1.8 09 FF FF FF CRC 校验失败

0X09+ 结束符

十进制数据为：9，255，255，255

带校验的指令通不过的时候返回此数据，参考串口指令增加 *CRC* 校验 (page 1016)

7.1.9 11 FF FF FF 波特率设置无效

0X11+ 结束符

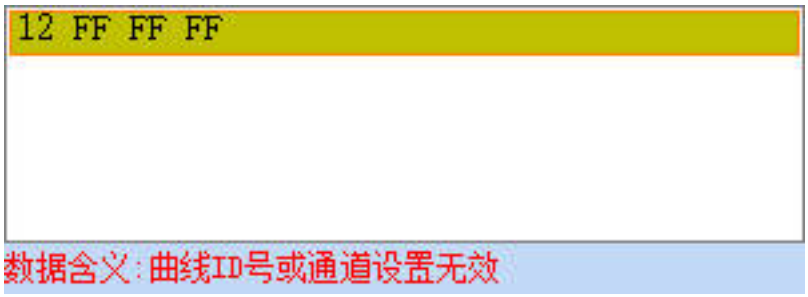
十进制数据为：17，255，255，255

用户发来的波特率配置指令中包含无效波特率参数

设备支持的波特率有:2400 4800 9600 19200 38400 57600 115200 230400 256000 512000 921600

7.1.10 12 FF FF FF 曲线控件 ID 号或通道号无效

0X12+ 结束符



十进制数据为：18，255，255，255

用户使用 add 指令往曲线控件添加数据的时候，曲线控件 ID 号或通道号无效时返回此数据

请注意检查控件的 id 号以及通道号（通道号从 0 开始，不是从 1 开始）

7.1.11 1A FF FF FF 变量名称无效

0X1A+ 结束符

十进制数据为：26，255，255，255

当串口收到的变量名称为无效名称时返回此数据

注：控件属性也称为变量，比如您设置一个控件的属性的时候，输入的是一个它没有的属性名称，也会返回此数据。

7.1.12 1B FF FF FF 变量运算无效

0X1B+ 结束符

十进制数据为：27，255，255，255

比如文本控件 t0 的 txt 属性赋值时应该写成 t0.txt=”abc”

如果你写成 t0.txt=abc 就出错了，再比如进度条 j0 的 val 属性应该是数值，所以要写成 j0.val=50，如果写成 j0.val=”50”或者 j0.val=abc 也会出错

7.1.13 1C FF FF FF 赋值操作失败

0X1C+ 结束符

十进制数据为：28，255，255，255

属性赋值失败的时候返回此数据

例如给进度条赋值为负数时

7.1.14 1D FF FF FF 掉电存储空间操作失败

0X1D+ 结束符

十进制数据为：29，255，255，255

操作掉电存储空间失败时返回此数据, 通常的会导致这个问题的情况有掉电存储空间超出范围 (page 889) 和赋值异常 (page 889) 。

7.1.14.1 掉电存储空间超出范围

掉电存储空间的地址是 0-1023

如果你写入的是 1 个 4 字节的整形，那么最多写入到 1020，此时占用的空间是 1020-1023 共 4 个字节

例如:

```
wepo n0.val,1020    //正确,写入范围为1020-1023共4个字节
wepo n0.val,1021    //错误,写入范围为1021-1024共4个字节,但掉电存储空间的地址是0-1023,
↪1024超过了范围
wepo n0.val,1022    //错误,写入范围为1022-1025共4个字节,但掉电存储空间的地址是0-1023,
↪1024-1025超过了范围
wepo n0.val,1023    //错误,写入范围为1023-1026共4个字节,但掉电存储空间的地址是0-1023,
↪1024-1026超过了范围
wepo n0.val,1024    //错误,写入范围为1024-1027共4个字节,但掉电存储空间的地址是0-1023,
↪1024-1027超过了范围
```

7.1.14.2 赋值异常

例如地址 0-3 中保存的值是 0x12345678, 你将这个值读取放入双态按钮中就会报错, 双态按钮状态只能是 0 或 1, 其他值不在范围内

repo bt0.val,0 //错误, 按钮状态只能是 0 或 1, 其他值不在范围内

7.1.15 1E FF FF FF 参数数量无效

0X1E+ 结束符

十进制数据为：30，255，255，255

用户输入的指令中参数数量错误的时候返回此数据

例如：

```
prints n0.val  //少写了一个参数

prints n0.val,1,1 //多写了一个参数
```

7.1.16 1F FF FF FF IO 操作失败

0X1F+ 结束符

十进制数据为：31，255，255，255

操作 IO 失败时返回此数据

7.1.17 20 FF FF FF 转义字符使用错误

0X20+ 结束符

十进制数据为：32，255，255，255

转义字符使用错误时返回此数据

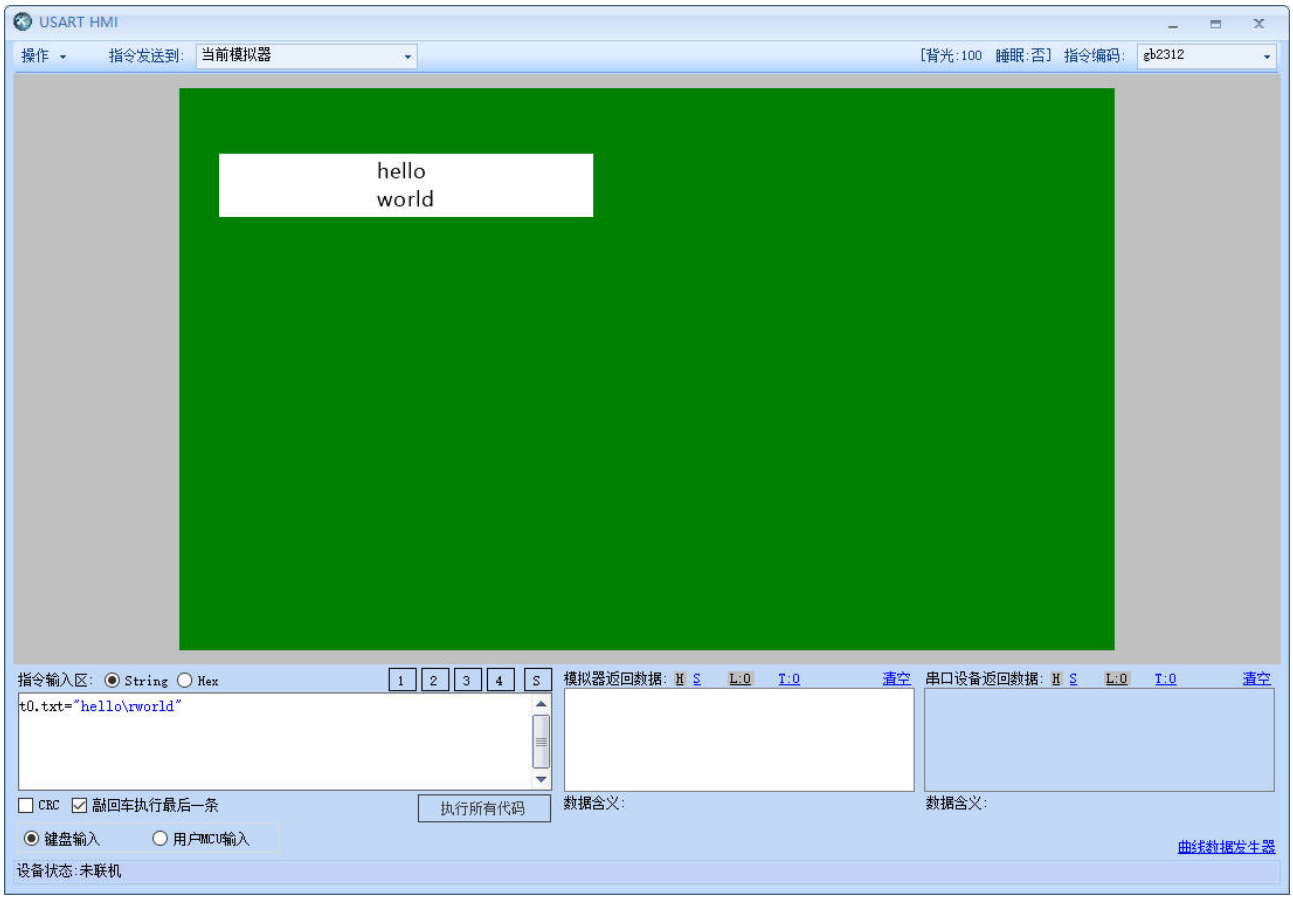
转义字符有以下几种：

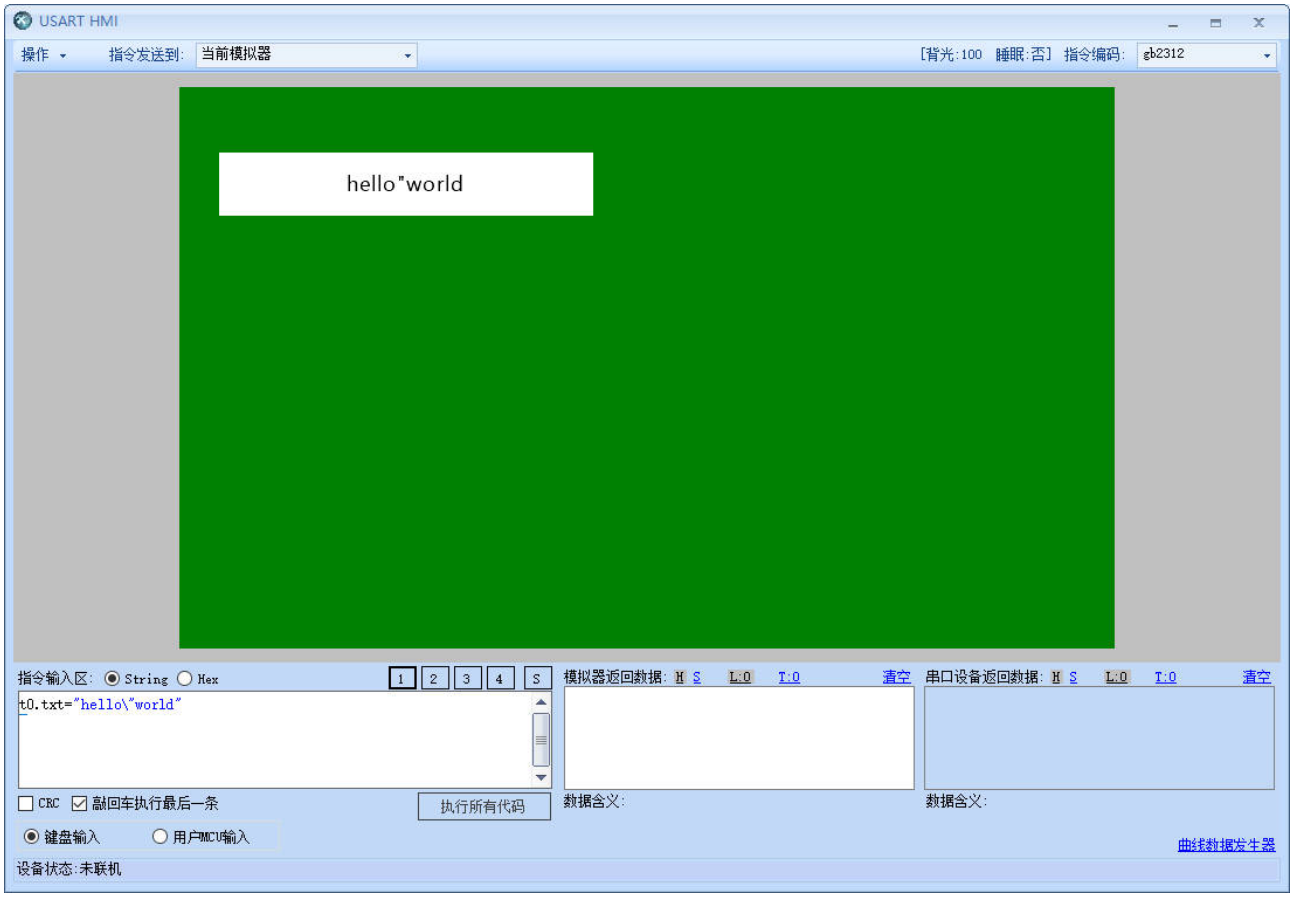
回车换行：\r

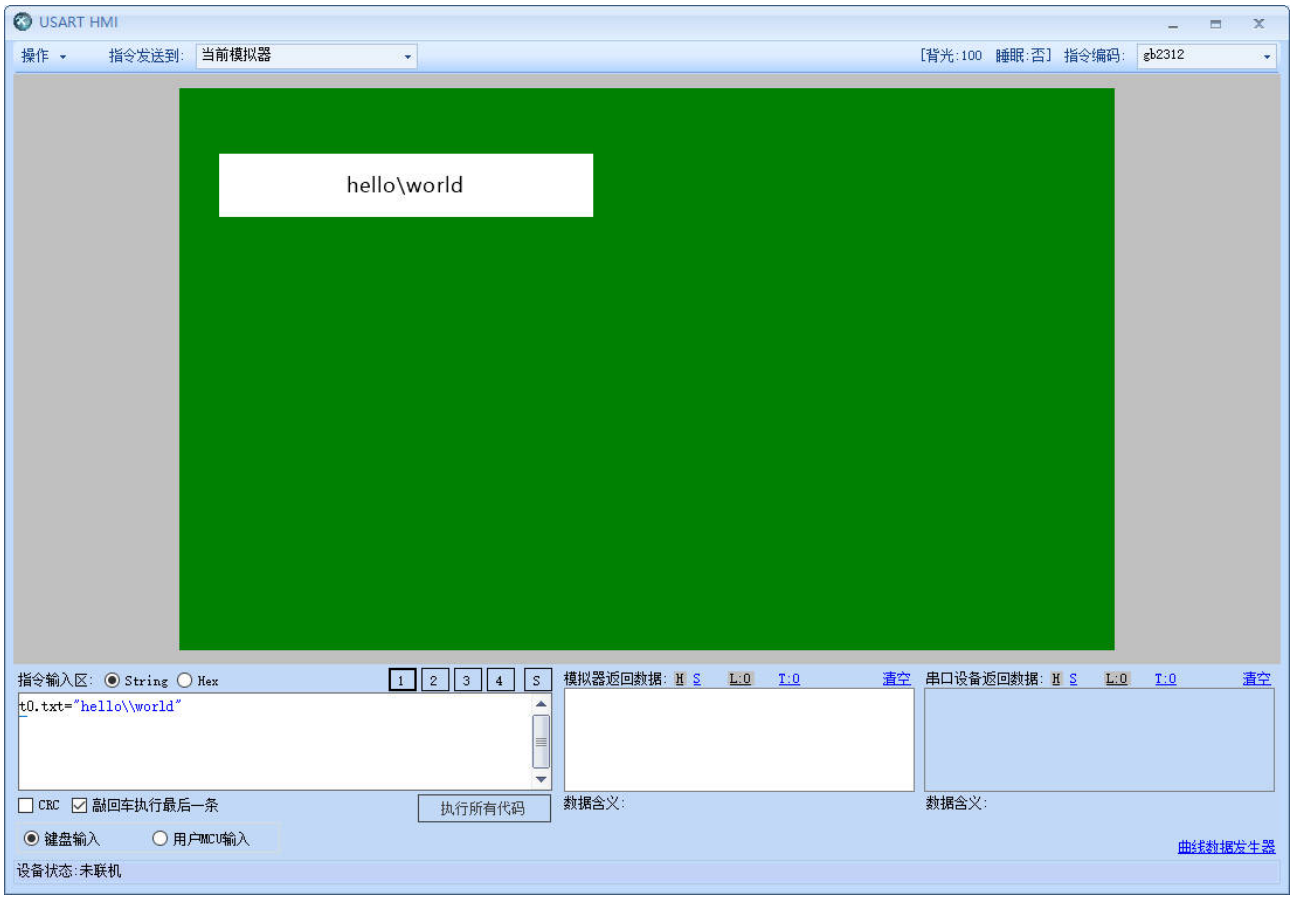
引号：\”

反斜杠：\\

```
t0.txt="hello\rworld"

t0.txt="hello\"world"

t0.txt="hello\\world"
```







在单片机上发送时，还需要对\再次转义

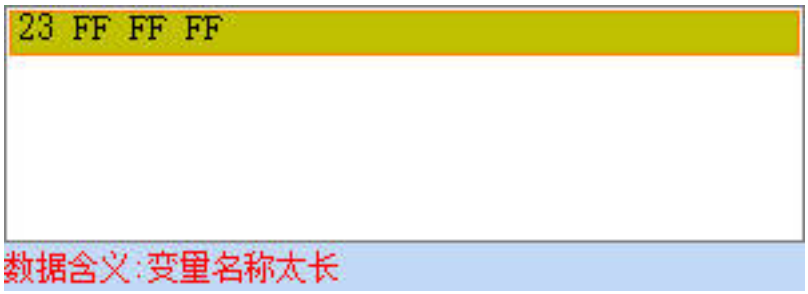
```
printf("t0.txt=\"hello\\\rworld\"")

printf("t0.txt=\"hello\\\"rworld\"")

printf("t0.txt=\"hello\\\\rworld\"")
```

相关链接

转义字符相关 (page 1321)

7.1.18 23 FF FF FF 变量名称太长

0X23+ 结束符



十进制数据为：35，255，255，255

变量名称长度最大 29 个字符，超出就会返回此数据

7.2 其他数据返回格式

1. 设备返回数据的结束符为“0XFF 0XFF 0XFF”三个字节。

2. 以下数据的返回不受 bkcmd 影响。

7.2.1 24 FF FF FF 串口缓冲区溢出

0X24+ 结束符

十进制数据为：36，255，255，255

当串口缓冲区被占满以后会返回此数据

缓冲区溢出以后，缓冲队列里的指令执行完成后会为缓冲区腾出空间以继续接收指令，在此之前，串口收到的数据将会丢弃

常见原因:

1. 发送的速度太快了，建议每条指令间隔 100 毫秒

2. 发送了太多无关的指令，串口屏无法进行解析，导致串口缓冲区被占满。

注意： 该指令无法被 bkcmd 变量关闭，在主动解析模式下当串口缓冲区满了时也会返回 24 ff ff ff

7.2.2 0X65 控件点击事件返回

0X65+ 页面 ID+ 按键 ID+ 触摸事件 + 结束符

(用户创建的控件被按下或弹起时返回此数据，前提是您勾选了控件的”发送键值”选框)

(触摸事件的定义：按下事件 0x01 弹起事件 0X00)

举例:0X65 0X00 0X02 0X01 0XFF 0XFF 0XFF

含义: 页面 0 按钮 2 按下

注意: 此返回不受 bkcmd 影响

7.2.3 0X66 当前页面的 ID 号返回

0X66+ 页面 ID+ 结束符

(设备收到”sendme”指令时会返回此数据)

举例：0X66 0X02 0XFF 0XFF 0XFF

含义：当前页面 ID 为 2

注意： 此返回不受 bkcmd 影响

7.2.4 0X67 触摸坐标数据返回

0X67++ 坐标 X 高位 + 坐标 X 低位 + 坐标 Y 高位 + 坐标 Y 低位 + 触摸事件状态 + 结束符

(当系统变量“sendxy”为 1 之后，有触摸事件时返回此数据)

(触摸事件的定义：按下事件 0x01 弹起事件 0X00)

举例:0X67 0X00 0X7A 0X00 0X1E 0X01 0XFF 0XFF 0XFF

含义: 坐标 (122,30) 事件: 按下

注意： 此返回不受 bkcmd 影响

7.2.5 0X68 睡眠模式触摸事件

0X68++ 坐标 X 高位 + 坐标 X 低位 + 坐标 Y 高位 + 坐标 Y 低位 + 触摸事件状态 + 结束符

(当设备进入睡眠模式后，有触摸事件时返回此数据)

(触摸事件的定义：按下事件 0x01 弹起事件 0X00)

举例:0X68 0X00 0X7A 0X00 0X1E 0X01 0XFF 0XFF 0XFF

含义: 坐标 (122,30) 事件: 按下

注意: 此返回不受 bkcmd 影响

7.2.6 0X70 字符串变量数据返回

0X70+ 变量内容 ASCII 码 + 结束符

使用 get 指令获取的变量为字符串类型时，返回此数据.

举例:0X70 0X61 0X62 0X63 0XFF 0XFF 0XFF

含义：返回字符串数据: “abc”

注意: 此返回不受 bkcmd 影响

7.2.7 0X71 数值变量数据返回

0X71+ 变量二进制数据（4 字节小端模式, 低位在前）+ 结束符

使用 get 指令获取的变量为数值时，返回此数据.

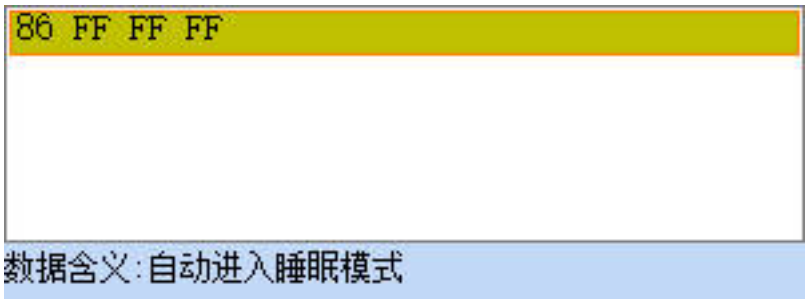
举例:0X71 0X66 0X00 0X00 0X00 0XFF 0XFF 0XFF

含义：返回数值数据:102

> **注意:** 此返回不受 bkcmd 影响

7.2.8 0X86 设备自动进入睡眠模式

0x86+ 结束符



只有设备自动进入睡眠模式的时候会返回此数据，如果是执行串口指令 sleep=1 进入的睡眠不会返回此数据

注意： 此返回不受 bkcmd 影响

7.2.9 0X87 设备自动唤醒

0x87+ 结束符

只有设备自动唤醒的时候会返回此数据，如果是执行串口指令 sleep=0 唤醒的睡眠不会返回此数据

注意： 此返回不受 bkcmd 影响

7.2.10 0X88 系统启动成功

0x88+ 结束符

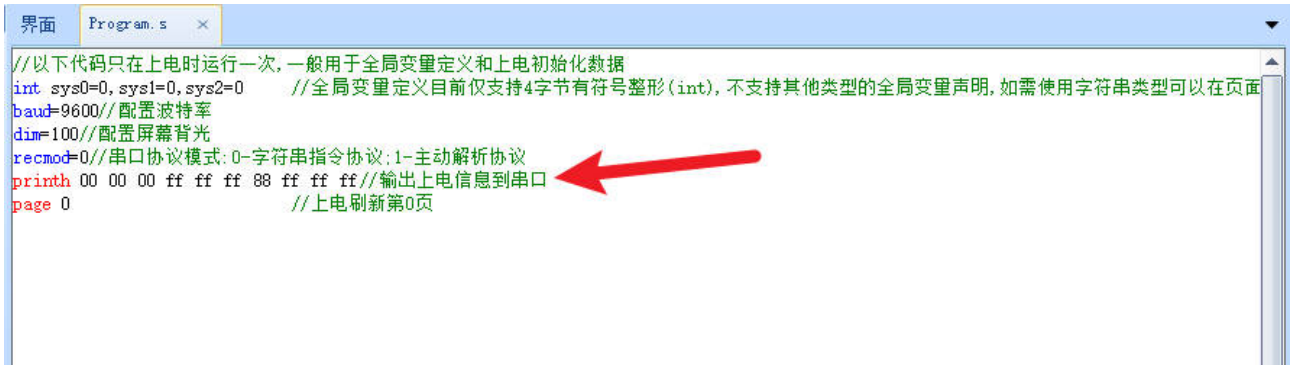
设备上电初始化成功之后发送此数据

7.2.10.1 1.65.0 版本以前

> **注意:** 此返回不受 bkcmd 影响

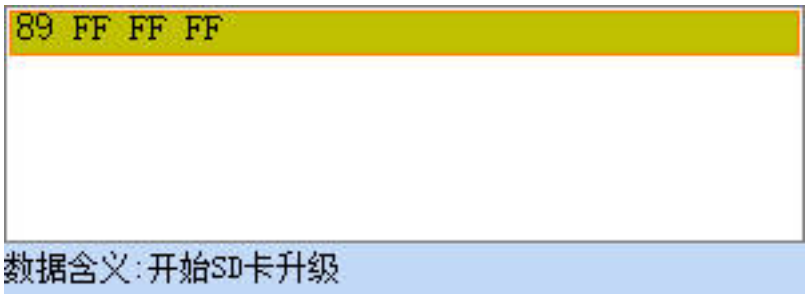
7.2.10.2 1.65.0 版本以后

取消设备开机固定发送开机信息，转为 Program.s 中由代码发送开机信息（用户可自行注释或者删除）



7.2.11 0X89 开始 SD 卡升级

0x89+ 结束符

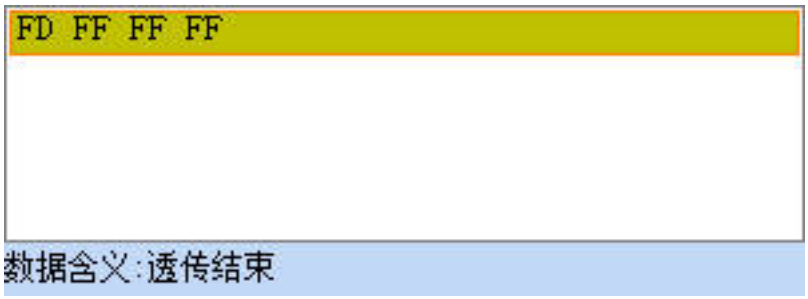


设备上电检测到 SD 卡之后将发送此数据, 然后进入升级界面

注意: 此返回不受 bkcmd 影响

7.2.12 0XFD 透传数据完成

0xFD+ 结束符



透传数据结束并处理数据完成后发送此数据

注意: 此返回不受 bkcmd 影响

7.2.13 0XFE 数据透传就绪

0xFE+ 结束符

设备收到数据透传指令后，进入透传数据初始化，初始化完成以后发送此数据，表示此时已经进入数据透传模式，可以开始数据透传

注意： 此返回不受 bkcmd 影响

8 串口屏高级应用详解

提示： 本文档主要讲述一些高级应用，实现一些特殊功能，大多数项目是用不上这些功能的，如果您的项目用不上，可以忽略此文档。

【章节索引】

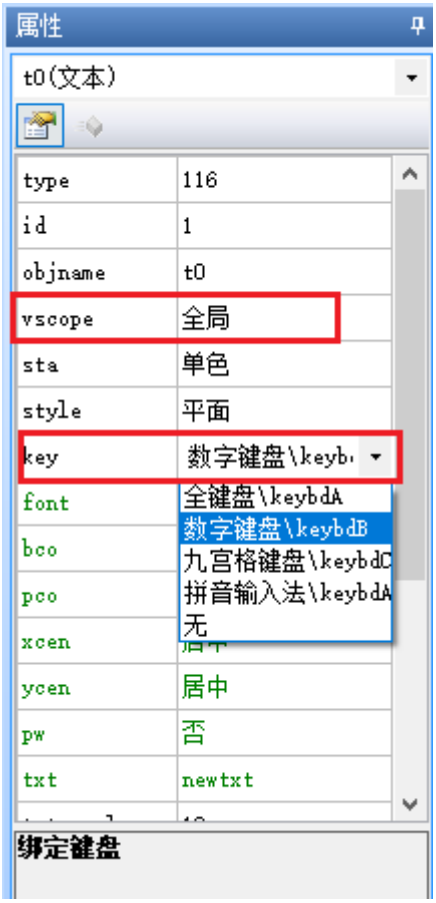
8.1 系统键盘

8.1.1 键盘基础知识

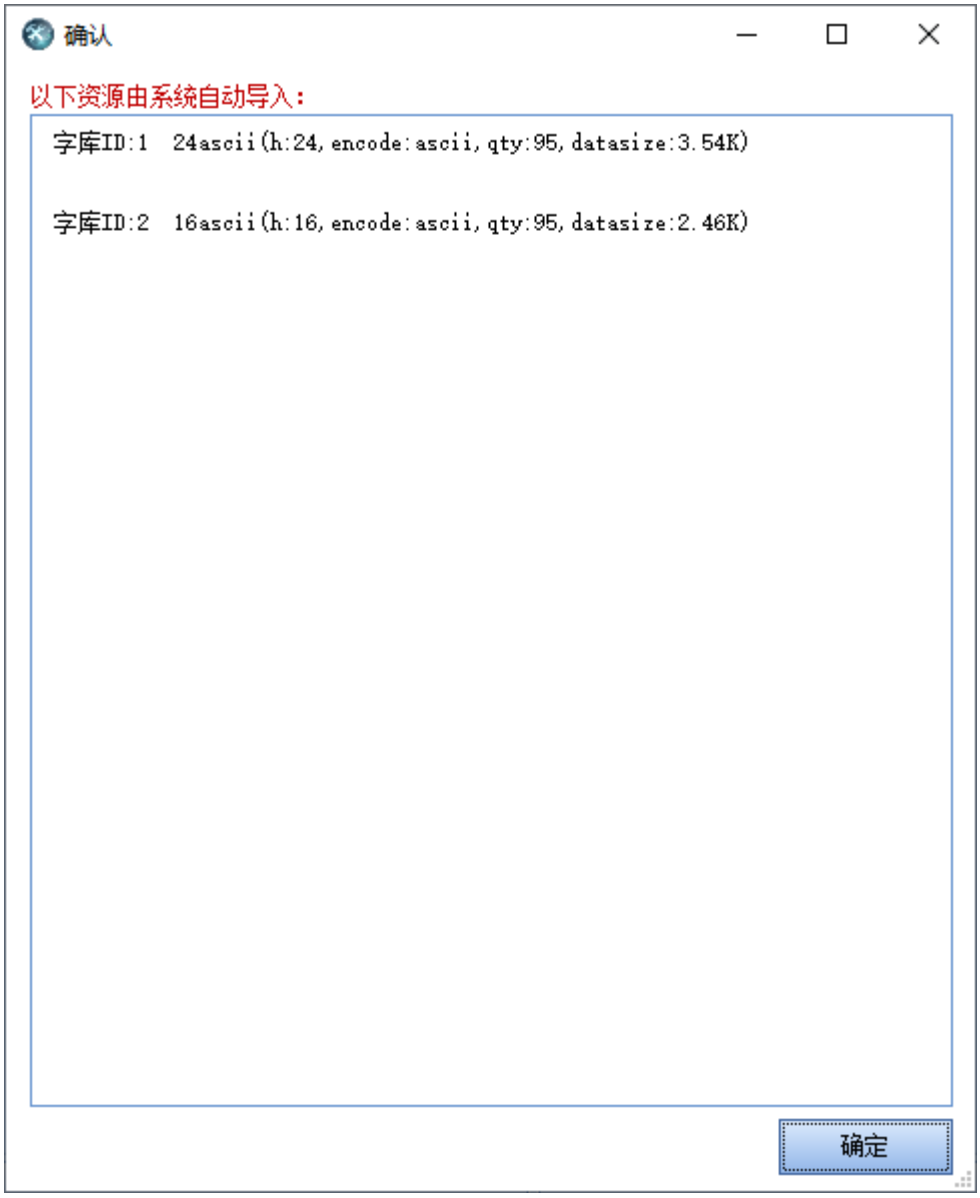
8.1.1.1 系统键盘的调用方式

支持系统键盘的控件: 文本控件、数字控件、虚拟浮点数控件、滚动文本控件、滑动文本控件。

以下图为例，首先我们将控件的 vscope 属性设置为全局，然后配置 key 属性，选择你需要的键盘即可，会自动导入键盘需要的资源



调用键盘后默认导入相关字库资源文件



注意： 某个控件绑定了键盘时，当点击了该控件时，会先触发该控件的按下事件，然后触发该控件的弹起事件，最后触发键盘跳转事件。

键盘并不是一个单纯的控件，键盘是一个独立的页面，调用键盘时，会触发当前页面的 < 页面离开事件 > 以及键盘页面的 < 前初始化事件 > 和 < 后初始化事件 >

点击键盘的 OK 按键时，会触发键盘页面的 < 页面离开事件 > 以及控件调用页面的 < 前初始化事件 > 和 < 后初始化事件 >

当触发了键盘控件跳转了页面时，当前页面私有的控件会被回收，从键盘页面返回时，私有的控件初始化并变回初始状态

8.1.1.2 键盘解锁和重置

添加键盘时，会在页面窗口最下面页面生成相应键盘界面。

默认键盘是上锁的，如果对键盘界面修改或者键盘里面功能进行修改，可以点击页面键盘右键解锁它并做相关的修改。

如果不小心对键盘做了不必要的修改，可以右键重置键盘，键盘将会恢复默认键盘最初始的设置状态。

注意： 如果您没有修改键盘功能的需求不需要再往下看了，可以查看其它章节

注意： 如果您没有修改键盘功能的需求不需要再往下看了，可以查看其它章节

8.1.2 键盘的实现原理

8.1.2.1 系统键盘-常用控件和变量说明

① 变量 loadpageid.val：调用页的页面 ID。

② 变量 loadcmpid.val：调用页的控件 ID。

③ 定时器 tm0：让输入框有个光标不断闪烁，如果不需要，控件属性 en=0 即可。

④ 变量 inputlenth：获取正在使用系统键盘控件字符最大长度。

⑤ 变量 input：输入的数据。

⑥ 文本 show：键盘显示的数据, 主要目的是为了让输入框有个闪烁的效果。

8.1.2.2 其他跳转到键盘页面的方法

> **注意：** 正常情况下是不需要配下面这三条代码的
>
> 是为了让其他原本不支持键盘的控件控件也能调用键盘
>
> 或者是从其他工程中导入的键盘, 因为不是用正常方法添加的键盘, 没法通过 key 属性进行配置, 需要使用这个方法

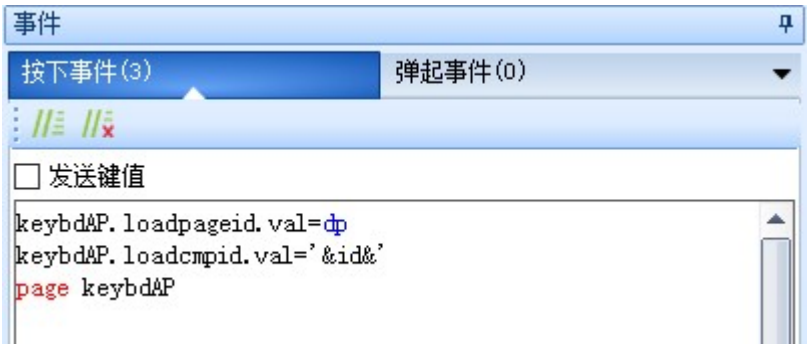
page 到键盘页之前，先对键盘页的 loadgageid.val 和 loadcmpid.val 赋值就可以了

一般在文本控件或数字控件的按下事件中进行赋值，键盘名.loadpageid.val=dp，键盘名.loadcmpid.val= 当前控件的 id

其他的逻辑会自动实现,loadpageid.val 表示调用页的页面 ID,loadcmpid.val 表示调用页的控件 ID, 然后调用 page 指令跳转到键盘页面即可

这样就可以在键盘页面判断跳转过来的控件类型（type 属性），然后根据不同类型调用不同代码转换到键盘上进行显示

```
keybdAP.loadpageid.val=dp        // dp:当前页面
keybdAP.loadcmpid.val='&id&'     // '&id&'当前被触发控件的id
page keybdAP
```



配置键盘的过程其实就是上位机帮我们自动添加了这三条代码。

演示工程下载链接：

《其他跳转到键盘页面的方法》演示工程下载

8.1.2.3 键盘如何获取到原始控件的值

下面的代码为键盘页面的前初始化事件

```
//调用此页之前，先对此页的loadpageid.val和loadcmpid.
↪val赋值就可以了，其他的逻辑本页会自动实现
//loadpageid.val表示调用页的页面ID,loadcmpid.val表示调用页的控件ID
if(p[loadpageid.val].b[loadcmpid.val].type==54)
{
  covx p[loadpageid.val].b[loadcmpid.val].val,input.txt,0,0
  inputlenth.val=24
}else if(p[loadpageid.val].b[loadcmpid.val].type==59)
{
  inputlenth.val=p[loadpageid.val].b[loadcmpid.val].val
  if(inputlenth.val<0)
  {
    inputlenth.val*=-1
    input.txt="-"
  }else
  {
    input.txt=""
  }
  temp.val=1
  for(temp2.val=0;temp2.val<p[loadpageid.val].b[loadcmpid.val].vvs1;temp2.val++)
  {
    temp.val*=10
  }
  temp2.val=inputlenth.val/temp.val//得到整数位
  cov temp2.val,tempstr.txt,0
  input.txt+=tempstr.txt+"."
  temp2.val=temp2.val*temp.val-inputlenth.val//得到小数位
  if(temp2.val<0)
  {
    temp2.val*=-1
  }
  covx temp2.val,tempstr.txt,p[loadpageid.val].b[loadcmpid.val].vvs1,0
  input.txt+=tempstr.txt
  inputlenth.val=24
}else
{
  input.txt=p[loadpageid.val].b[loadcmpid.val].txt
  inputlenth.val=p[loadpageid.val].b[loadcmpid.val].txt_maxl
  if(p[loadpageid.val].b[loadcmpid.val].type==116)
  {
    show.pw=p[loadpageid.val].b[loadcmpid.val].pw
  }
}
show.txt=input.txt
```

可以看到获取到原始控件的值分为 3 个步骤

1. 通过 p[loadpageid.val].b[loadcmpid.val].type 判断原始控件的类型是数字控件、虚拟浮点数或者其他，loadpageid.val 和 loadcmpid.val 是如何传进来可以参考其他跳转到键盘页面的方法 (page 913)
2. 根据不同的控件类型，将数据赋值到 input.txt 中
3. 将 input.txt 赋值给 show.txt

8.1.2.4 输入完成后如何赋值给原始控件

下面的代码为键盘页面的 OK 按键的弹起事件

```
//调用此页之前，先对此页的loadpageid.val和loadcmpid.
↪val赋值就可以了，其他的逻辑本页会自动实现
//loadpageid.val表示调用页的页面ID,loadcmpid.val表示调用页的控件ID
if(p[loadpageid.val].b[loadcmpid.val].type==54)
{
  covx input.txt,p[loadpageid.val].b[loadcmpid.val].val,0,0
}else if(p[loadpageid.val].b[loadcmpid.val].type==59)
{
  covx input.txt,temp.val,0,0
  if(temp.val<0)
  {
    temp.val*=-1
  }
  for(temp2.val=0;temp2.val<p[loadpageid.val].b[loadcmpid.val].vvs1;temp2.val++)
  {
    temp.val*=10
  }
  p[loadpageid.val].b[loadcmpid.val].val=temp.val
  strlen input.txt,temp.val
  temp.val--
  while(temp.val>=0)
  {
    substr input.txt,tempstr.txt,temp.val,1
    if(tempstr.txt==".")
    {
      substr input.txt,tempstr.txt,temp.val+1,p[loadpageid.val].b[loadcmpid.val].vvs1
      covx tempstr.txt,temp2.val,0,0
      strlen tempstr.txt,temp.val
      while(temp.val<p[loadpageid.val].b[loadcmpid.val].vvs1)
      {
        temp2.val*=10
        temp.val++
      }
      p[loadpageid.val].b[loadcmpid.val].val+=temp2.val
      temp.val=-1
    }
    temp.val--
  }
  substr input.txt,tempstr.txt,0,1
  if(tempstr.txt=="-")
  {
    p[loadpageid.val].b[loadcmpid.val].val*=-1
  }
}else
{
  p[loadpageid.val].b[loadcmpid.val].txt=input.txt
}
page loadpageid.val
```

可以看到输入完成后赋值给原始控件分为 3 个步骤

1. 通过 p[loadpageid.val].b[loadcmpid.val].type 判断触发控件的类型是数字控件、虚拟浮点数或者其他，loadpageid.val 和 loadcmpid.val 是如何传进来可以参考其他跳转到键盘页面的方法 (page 913)
2. 根据不同的控件类型，将 input.txt 赋值给原始控件
3. 返回 loadpageid.val 所记录的页面 id

8.1.3 自定义键盘

自定义键盘前需解锁键盘，请参考键盘解锁和重置 (page 911)

8.1.3.1 进入键盘时自动将输入框清空

提示： 其实就是在键盘的后初始化事件中将 input.txt 和 show.txt 赋值为空，请注意两个引号之间不要有空格，否则会导致转换失败的

完整的代码如下

```
input.txt=""
show.txt=""
```

8.1.3.1.1 进入键盘时自动将输入框清空-样例工程下载

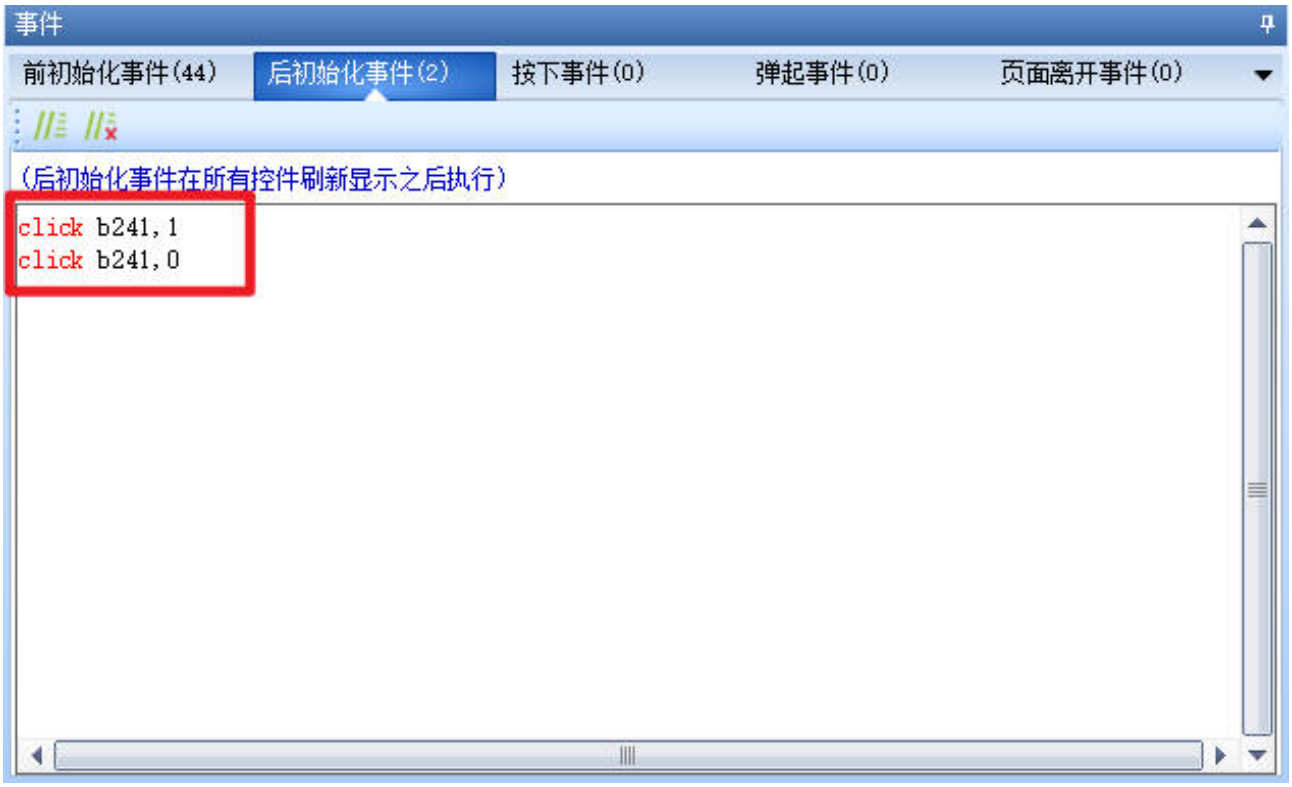
演示工程下载链接：

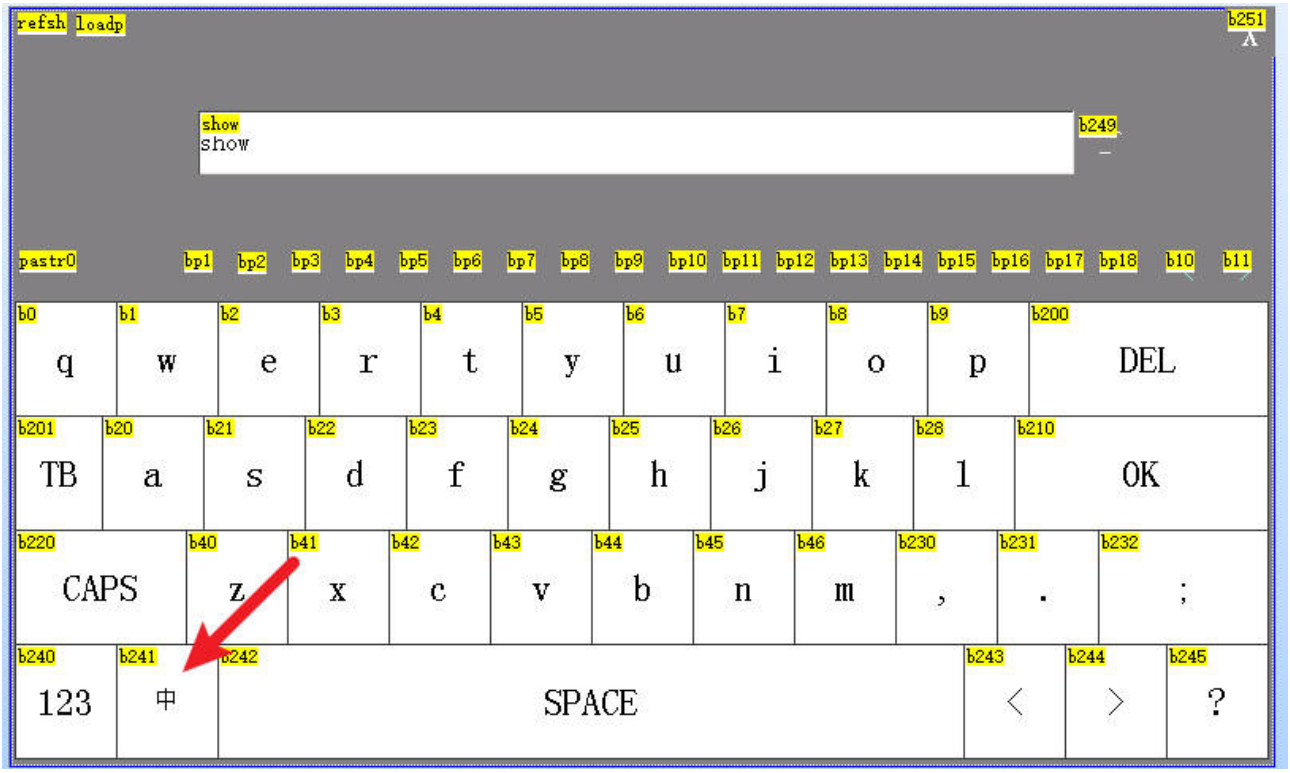
《进入键盘时自动将输入框清空》演示工程下载

8.1.3.2 键盘页面默认为拼音输入

当前工程的编码必须是 GB2312，且当前配置的键盘必须为输入法键盘。

在页面的后初始化页面中去 click 中英切换按键（模拟人手去自动切换输入法）





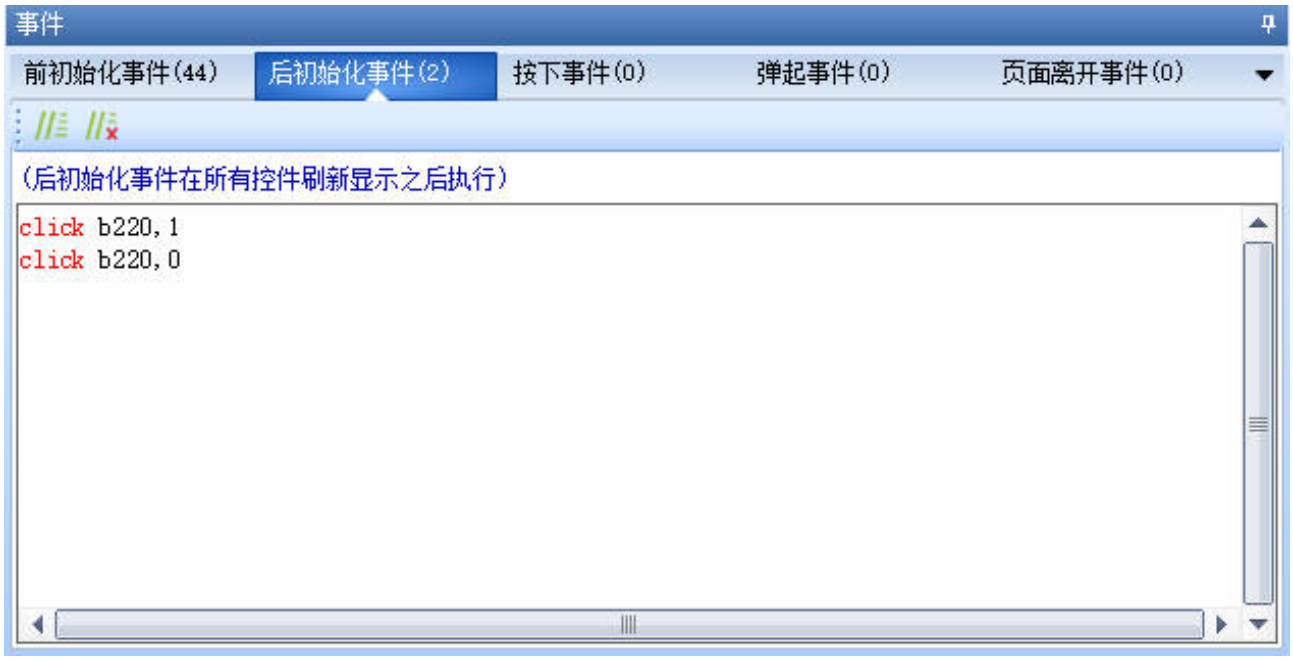
8.1.3.2.1 键盘页面默认为拼音输入-样例工程下载

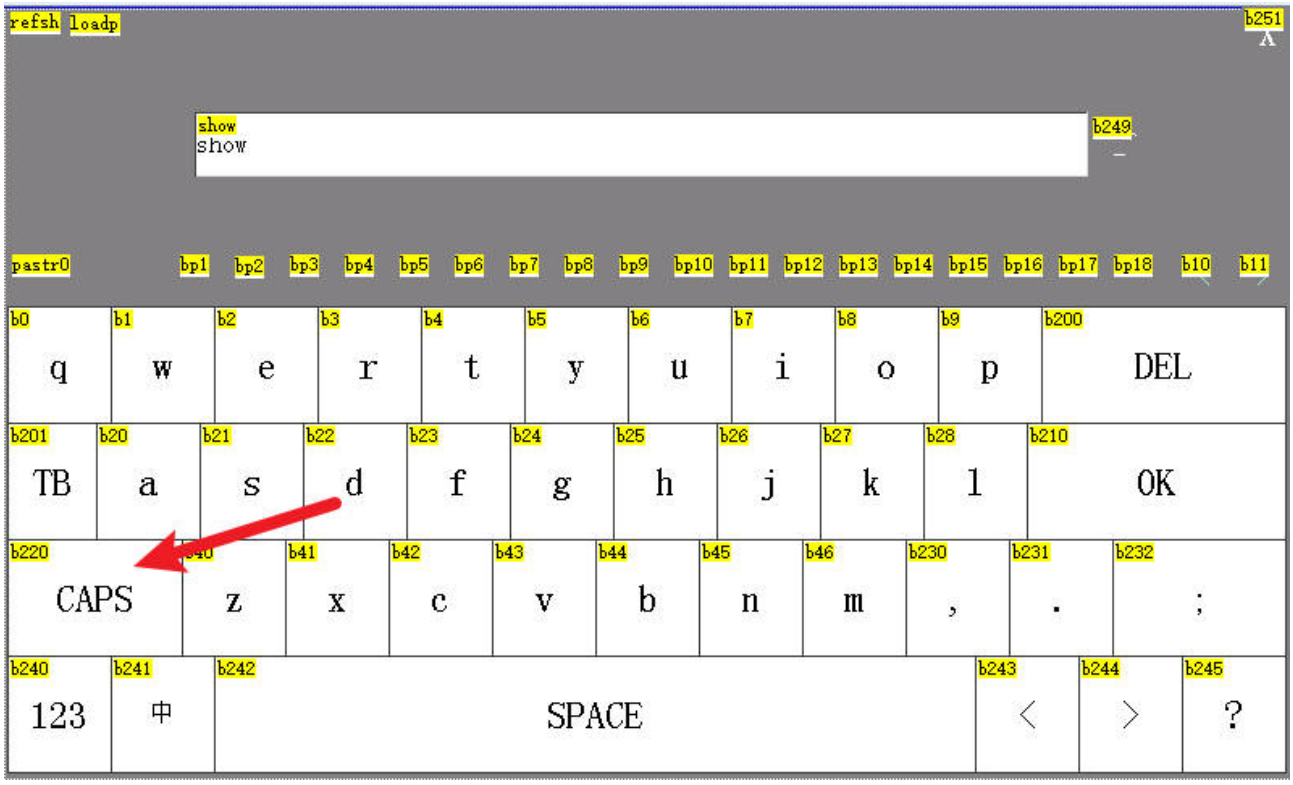
演示工程下载链接：

《键盘页面默认为拼音输入》演示工程下载

8.1.3.3 键盘页面默认为英文大写

在页面的后初始化页面中去 click caps 按键（模拟人手去自动切换输入法）





8.1.3.3.1 键盘页面默认为英文大写-样例工程下载

演示工程下载链接：

《键盘页面默认为英文大写》演示工程下载

8.1.3.4 如何让键盘的光标不闪烁

提示： 关闭键盘页面的 tm0 控件使能即可

也可以在页面后初始化事件中判断跳转过来的控件的 pw 属性（需要文本控件，数字控件不支持 pw 属性）

完整的代码如下

```
if(p[loadpageid.val].b[loadcmpid.val].type==116)
{
   if(p[loadpageid.val].b[loadcmpid.val].pw==1)
   {
      tm0.en=0
   }else
   {
      tm0.en=1
   }
}
```

8.1.3.4.1 让键盘的光标不闪烁-样例工程下载

演示工程下载链接：

《让键盘的光标不闪烁》演示工程下载

8.1.3.5 键盘页面限制输入数据的值

提示： 在 ok 键的按下事件中转换 input.txt 的数据到 sys0 中，然后对 sys0 的值进行判断

在 ok 键的按下事件中编写以下代码

```
covx input.txt,sys0,0,0
//区分不同页面的不同变量，实现对不同变量的不同限制
if(loadpageid.val==0&&loadcmpid.val==page0.n0.id)
{
    if(sys0<0)
    {
    input.txt="0"
    }else if(sys0>100)
    {
    input.txt="100"
    }
}else if(loadpageid.val==0&&loadcmpid.val==page0.n1.id)
{
    if(sys0<0)
    {
    input.txt="0"
    }else if(sys0>50)
    {
    input.txt="50"
    }
}else if(loadpageid.val==0&&loadcmpid.val==page0.n2.id)
{
    if(sys0<0)
    {
    input.txt="0"
    }else if(sys0>70)
    {
    input.txt="70"
    }
}
```

8.1.3.5.1 键盘页面限制输入数据的值-样例工程下载

演示工程下载链接：

《键盘页面限制输入数据的值》演示工程下载

8.1.3.6 在键盘页面按下 OK 键时将参数通过串口发送出去

1、以文本类型发送

提示： 编辑 ok 按钮的弹起事件, 在 page 指令前添加以下代码

```
//区分不同页面的不同变量
if(loadpageid.val==0&&loadcmpid.val==page0.n0.id)
{
   covx input.txt,sys0,0,0
   //如果是n0，写在10的位置
   wepo sys0,10
}else if(loadpageid.val==0&&loadcmpid.val==page0.n1.id)
{
   covx input.txt,sys0,0,0
   //如果是n1，写在14的位置
   wepo sys0,14
}else if(loadpageid.val==0&&loadcmpid.val==page0.n2.id)
{
   covx input.txt,sys0,0,0
   //如果是n2，写在18的位置
   wepo sys0,18
}
```

注意： page 指令之后的指令是不会被执行的，因为已经跳转到别的页面了。

提示： 这里作为帧头的 55 并不固定，是可以修改为你自己定义的其他帧头，你可以定义多个字节作为帧头 00 01 02 是作为标记来区分 n0.val、n1.val、n2.val，可以根据自己的需求进行更改

完整的代码如下

```
//调用此页之前，先对此页的loadpageid.val和loadcmpid.
↪val赋值就可以了，其他的逻辑本页会自动实现
//loadpageid.val表示调用页的页面ID,loadcmpid.val表示调用页的控件ID
if(p[loadpageid.val].b[loadcmpid.val].type==54)
{
   covx input.txt,p[loadpageid.val].b[loadcmpid.val].val,0,0
}else if(p[loadpageid.val].b[loadcmpid.val].type==59)
{
   covx input.txt,temp.val,0,0
if(temp.val<0)
{
   temp.val*=-1
}
for(temp2.val=0;temp2.val<p[loadpageid.val].b[loadcmpid.val].vvs1;temp2.val++)
{
   temp.val*=10
}
p[loadpageid.val].b[loadcmpid.val].val=temp.val
strlen input.txt,temp.val
temp.val--
while(temp.val>=0)
{
```

(续下页)

(接上页)

```
   substr input.txt,tempstr.txt,temp.val,1
   if(tempstr.txt==".")
   {
      substr input.txt,tempstr.txt,temp.val+1,p[loadpageid.val].b[loadcmpid.val].vvs1
      covx tempstr.txt,temp2.val,0,0
      strlen tempstr.txt,temp.val
      while(temp.val<p[loadpageid.val].b[loadcmpid.val].vvs1)
      {
         temp2.val*=10
         temp.val++
      }
      p[loadpageid.val].b[loadcmpid.val].val+=temp2.val
      temp.val=-1
   }
   temp.val--
}
substr input.txt,tempstr.txt,0,1
if(tempstr.txt=="-")
{
   p[loadpageid.val].b[loadcmpid.val].val*=-1
}
}else
{
   p[loadpageid.val].b[loadcmpid.val].txt=input.txt
}
//区分不同页面的不同变量
if(loadpageid.val==0&&loadcmpid.val==page0.n0.id)
{
   printh 55 00  //先发送一个帧头，用来告诉下位机这个是n0
   prints input.txt,0  //发送输入框内的数据
   printh 0d 0a  //发送帧尾
}else if(loadpageid.val==0&&loadcmpid.val==page0.n1.id)
{
   printh 55 01  //先发送一个帧头，用来告诉下位机这个是n1
   prints input.txt,0  //发送输入框内的数据
   printh 0d 0a  //发送帧尾
}else if(loadpageid.val==0&&loadcmpid.val==page0.n2.id)
{
   printh 55 02  //先发送一个帧头，用来告诉下位机这个是n2
   prints input.txt,0  //发送输入框内的数据
   printh 0d 0a  //发送帧尾
}
page loadpageid.val
```

2、以数值类型发送

提示： 编辑 ok 按钮的弹起事件, 在 page 指令前添加以下代码

```
if(loadpageid.val==0&&loadcmpid.val==page0.n0.id)
{
  printh 55 00  //先发送一个帧头，用来告诉下位机这个是n0
  covx input.txt,sys0,0,0
  prints sys0,0  //发送输入框内的数据
  printh 0d 0a  //发送帧尾
}else if(loadpageid.val==0&&loadcmpid.val==page0.n1.id)
{
  printh 55 01  //先发送一个帧头，用来告诉下位机这个是n1
  covx input.txt,sys0,0,0
  prints sys0,0  //发送输入框内的数据
  printh 0d 0a  //发送帧尾
}else if(loadpageid.val==0&&loadcmpid.val==page0.n2.id)
{
  printh 55 02  //先发送一个帧头，用来告诉下位机这个是n2
  covx input.txt,sys0,0,0
  prints sys0,0  //发送输入框内的数据
  printh 0d 0a  //发送帧尾
}
page loadpageid.val
```

注意： page 指令之后的指令是不会被执行的，因为已经跳转到别的页面了。

提示： 这里作为帧头的 55 并不固定，是可以修改为你自己定义的其他帧头，你可以定义多个字节作为帧头 00 01 02 是作为标记来区分 n0.val、n1.val、n2.val，可以根据自己的需求进行更改

完整的代码如下

```
//调用此页之前，先对此页的loadpageid.val和loadcmpid.
↪val赋值就可以了，其他的逻辑本页会自动实现
//loadpageid.val表示调用页的页面ID,loadcmpid.val表示调用页的控件ID
if(p[loadpageid.val].b[loadcmpid.val].type==54)
{
    covx input.txt,p[loadpageid.val].b[loadcmpid.val].val,0,0
}else if(p[loadpageid.val].b[loadcmpid.val].type==59)
{
    covx input.txt,temp.val,0,0
    if(temp.val<0)
    {
    temp.val*=-1
    }
    for(temp2.val=0;temp2.val<p[loadpageid.val].b[loadcmpid.val].vvs1;temp2.val++)
    {
    temp.val*=10
    }
    p[loadpageid.val].b[loadcmpid.val].val=temp.val
    strlen input.txt,temp.val
    temp.val--
    while(temp.val>=0)
    {
```

(续下页)

(接上页)

```
    substr input.txt,tempstr.txt,temp.val,1
    if(tempstr.txt==".")
    {
        substr input.txt,tempstr.txt,temp.val+1,p[loadpageid.val].b[loadcmpid.val].vvs1
        covx tempstr.txt,temp2.val,0,0
        strlen tempstr.txt,temp.val
        while(temp.val<p[loadpageid.val].b[loadcmpid.val].vvs1)
        {
        temp2.val*=10
        temp.val++
        }
        p[loadpageid.val].b[loadcmpid.val].val+=temp2.val
        temp.val=-1
    }
    temp.val--
    }
    substr input.txt,tempstr.txt,0,1
    if(tempstr.txt=="-")
    {
    p[loadpageid.val].b[loadcmpid.val].val*=-1
    }
}else
{
    p[loadpageid.val].b[loadcmpid.val].txt=input.txt
}
//区分不同页面的不同变量
if(loadpageid.val==0&&loadcmpid.val==page0.n0.id)
{
    //先发送一个帧头，用来告诉下位机这个是n0
    printh 55 00

    //将文本转换为数值
    covx input.txt,sys0,0,0

    //发送输入框内的数据
    prints sys0,0

    //发送帧尾
    printh 0d 0a

}else if(loadpageid.val==0&&loadcmpid.val==page0.n1.id)
{
    //先发送一个帧头，用来告诉下位机这个是n1
    printh 55 01

    //将文本转换为数值
    covx input.txt,sys0,0,0

    //发送输入框内的数据
    prints sys0,0

    //发送帧尾
    printh 0d 0a
}else if(loadpageid.val==0&&loadcmpid.val==page0.n2.id)
{
    //先发送一个帧头，用来告诉下位机这个是n2
    printh 55 02

    //将文本转换为数值
    covx input.txt,sys0,0,0
```

(续下页)

(接上页)

```
    //发送输入框内的数据
    prints sys0,0

    //发送帧尾
    printh 0d 0a
}
page loadpageid.val
```

8.1.3.6.1 在键盘页面按下 OK 键时将参数通过串口发送出去-样例工程下载

演示工程下载链接：

《在键盘页面按下 OK 键时将参数通过串口发送出去-以数值类型发送》演示工程下载

《在键盘页面按下 OK 键时将参数通过串口发送出去-以文本类型发送》演示工程下载

8.1.3.7 键盘只能输入 32 个字符需要改成能输入更多的字符

将键盘页面的 show.txt_maxl 和 input.txt_maxl 的值改为你需要的值即可

8.1.3.7.1 键盘只能输入 32 个字符需要改成能输入更多的字符-样例工程下载

演示工程下载链接：

《键盘只能输入 32 个字符需要改成能输入更多的字符》演示工程下载

8.1.3.8 不同控件在系统键盘设置好参数存储到用户存储区

提示： 编辑 ok 按钮的弹起事件, 在 page 指令前添加以下代码

```
//区分不同页面的不同变量
if(loadpageid.val==0&&loadcmpid.val==page0.n0.id)
{
  covx input.txt,sys0,0,0
  //如果是n0，写在10的位置
  wepo sys0,10
}else if(loadpageid.val==0&&loadcmpid.val==page0.n1.id)
{
  covx input.txt,sys0,0,0
  //如果是n1，写在14的位置
  wepo sys0,14
}else if(loadpageid.val==0&&loadcmpid.val==page0.n2.id)
{
  covx input.txt,sys0,0,0
  //如果是n2，写在18的位置
  wepo sys0,18
}
```

完整代码如下

```
//调用此页之前，先对此页的loadpageid.val和loadcmpid.
↪val赋值就可以了，其他的逻辑本页会自动实现
//loadpageid.val表示调用页的页面ID,loadcmpid.val表示调用页的控件ID
if(p[loadpageid.val].b[loadcmpid.val].type==54)
{
   covx input.txt,p[loadpageid.val].b[loadcmpid.val].val,0,0
}else if(p[loadpageid.val].b[loadcmpid.val].type==59)
{
   covx input.txt,temp.val,0,0
if(temp.val<0)
{
   temp.val*=-1
}
for(temp2.val=0;temp2.val<p[loadpageid.val].b[loadcmpid.val].vvs1;temp2.val++)
{
   temp.val*=10
}
   p[loadpageid.val].b[loadcmpid.val].val=temp.val
   strlen input.txt,temp.val
   temp.val--
   while(temp.val>=0)
{
```

(续下页)

(接上页)

```
   substr input.txt,tempstr.txt,temp.val,1
   if(tempstr.txt==".")
   {
      substr input.txt,tempstr.txt,temp.val+1,p[loadpageid.val].b[loadcmpid.val].vvs1
      covx tempstr.txt,temp2.val,0,0
      strlen tempstr.txt,temp.val
      while(temp.val<p[loadpageid.val].b[loadcmpid.val].vvs1)
      {
         temp2.val*=10
         temp.val++
      }
      p[loadpageid.val].b[loadcmpid.val].val+=temp2.val
      temp.val=-1
   }
   temp.val--
}
substr input.txt,tempstr.txt,0,1
if(tempstr.txt=="-")
{
   p[loadpageid.val].b[loadcmpid.val].val*=-1
}
}else
{
   p[loadpageid.val].b[loadcmpid.val].txt=input.txt
}
//区分不同页面的不同变量
if(loadpageid.val==0&&loadcmpid.val==page0.n0.id)
{
   covx input.txt,sys0,0,0
   //如果是n0，写在10的位置
   wepo sys0,10
}else if(loadpageid.val==0&&loadcmpid.val==page0.n1.id)
{
   covx input.txt,sys0,0,0
   //如果是n1，写在14的位置
   wepo sys0,14
}else if(loadpageid.val==0&&loadcmpid.val==page0.n2.id)
{
   covx input.txt,sys0,0,0
   //如果是n2，写在18的位置
   wepo sys0,18
}
page loadpageid.val
```

注意： page 指令之后的指令是不会被执行的，因为已经跳转到别的页面了。

存储变量类型为 txt

```
if(loadpageid.val==0&&loadcmpid.val==page0.t0.id)
{
  wepo input.txt,100  //如果是t0，写在100的位置,注意分配好存储所需的eeprom空间
}else if(loadpageid.val==0&&loadcmpid.val==page0.t1.id)
{
  wepo input.txt,120  //如果是t1，写在120的位置,注意分配好存储所需的eeprom空间
}else if(loadpageid.val==0&&loadcmpid.val==page0.t2.id)
{
  wepo input.txt,140  //如果是t2，写在140的位置,注意分配好存储所需的eeprom空间
}
page loadpageid.val
```

完整的代码如下

```
//调用此页之前，先对此页的loadpageid.val和loadcmpid.
↪val赋值就可以了，其他的逻辑本页会自动实现
//loadpageid.val表示调用页的页面ID,loadcmpid.val表示调用页的控件ID
if(p[loadpageid.val].b[loadcmpid.val].type==54)
{
   covx input.txt,p[loadpageid.val].b[loadcmpid.val].val,0,0
}else if(p[loadpageid.val].b[loadcmpid.val].type==59)
{
   covx input.txt,temp.val,0,0
if(temp.val<0)
{
   temp.val*=-1
}
for(temp2.val=0;temp2.val<p[loadpageid.val].b[loadcmpid.val].vvs1;temp2.val++)
{
   temp.val*=10
}
p[loadpageid.val].b[loadcmpid.val].val=temp.val
strlen input.txt,temp.val
temp.val--
while(temp.val>=0)
{
   substr input.txt,tempstr.txt,temp.val,1
   if(tempstr.txt==".")
   {
      substr input.txt,tempstr.txt,temp.val+1,p[loadpageid.val].b[loadcmpid.val].vvs1
      covx tempstr.txt,temp2.val,0,0
      strlen tempstr.txt,temp.val
      while(temp.val<p[loadpageid.val].b[loadcmpid.val].vvs1)
      {
         temp2.val*=10
         temp.val++
      }
      p[loadpageid.val].b[loadcmpid.val].val+=temp2.val
      temp.val=-1
   }
   temp.val--
}
substr input.txt,tempstr.txt,0,1
if(tempstr.txt=="-")
{
   p[loadpageid.val].b[loadcmpid.val].val*=-1
}
}else
{
```

(续下页)

(接上页)

```
   p[loadpageid.val].b[loadcmpid.val].txt=input.txt
}
//区分不同页面的不同变量
if(loadpageid.val==0&&loadcmpid.val==page0.t0.id)
{
   //如果是t0，写在100的位置,注意分配好存储所需的掉电存储空间空间
   //此处仅为演示，默认的input.txt_max1为32，会写入32+1=33个字节，占用100-132
   wepo input.txt,100
}else if(loadpageid.val==0&&loadcmpid.val==page0.t1.id)
{
   //如果是t1，写在120的位置,注意分配好存储所需的掉电存储空间空间
   //此处仅为演示，默认的input.txt_max1为32，会写入32+1=33个字节，占用120-152
   wepo input.txt,120
}else if(loadpageid.val==0&&loadcmpid.val==page0.t2.id)
{
   //如果是t2，写在140的位置,注意分配好存储所需的掉电存储空间空间
   //此处仅为演示，默认的input.txt_max1为32，会写入32+1=33个字节，占用140-172
   wepo input.txt,140
}
page loadpageid.val
```

注意： page 指令之后的指令是不会被执行的，因为已经跳转到别的页面了。

8.1.3.8.1 在系统键盘设置好参数存储到用户存储区-样例工程下载

演示工程下载链接：

《在系统键盘设置好参数存储到用户存储区-以数值类型存储》演示工程下载

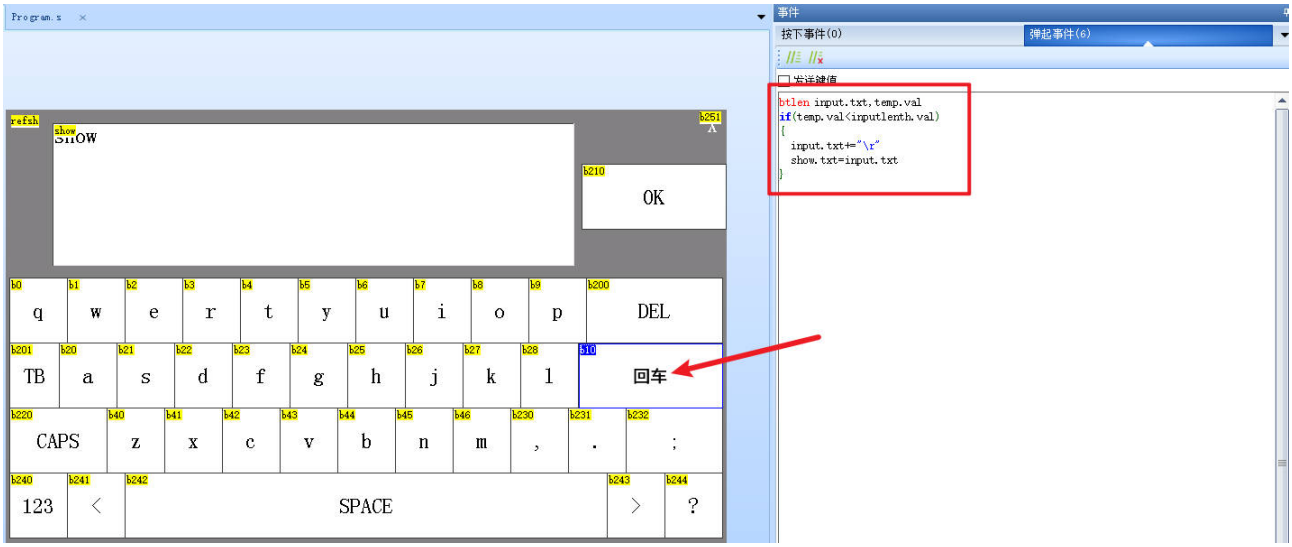
《在系统键盘设置好参数存储到用户存储区-以文本类型存储》演示工程下载

8.1.3.9 键盘输入回车

新建了一个按钮用于输入回车

按钮内的代码如下

```
btlen input.txt,temp.val
if(temp.val<inputlenth.val)
{
    input.txt+="\r"
    show.txt=input.txt
}
```



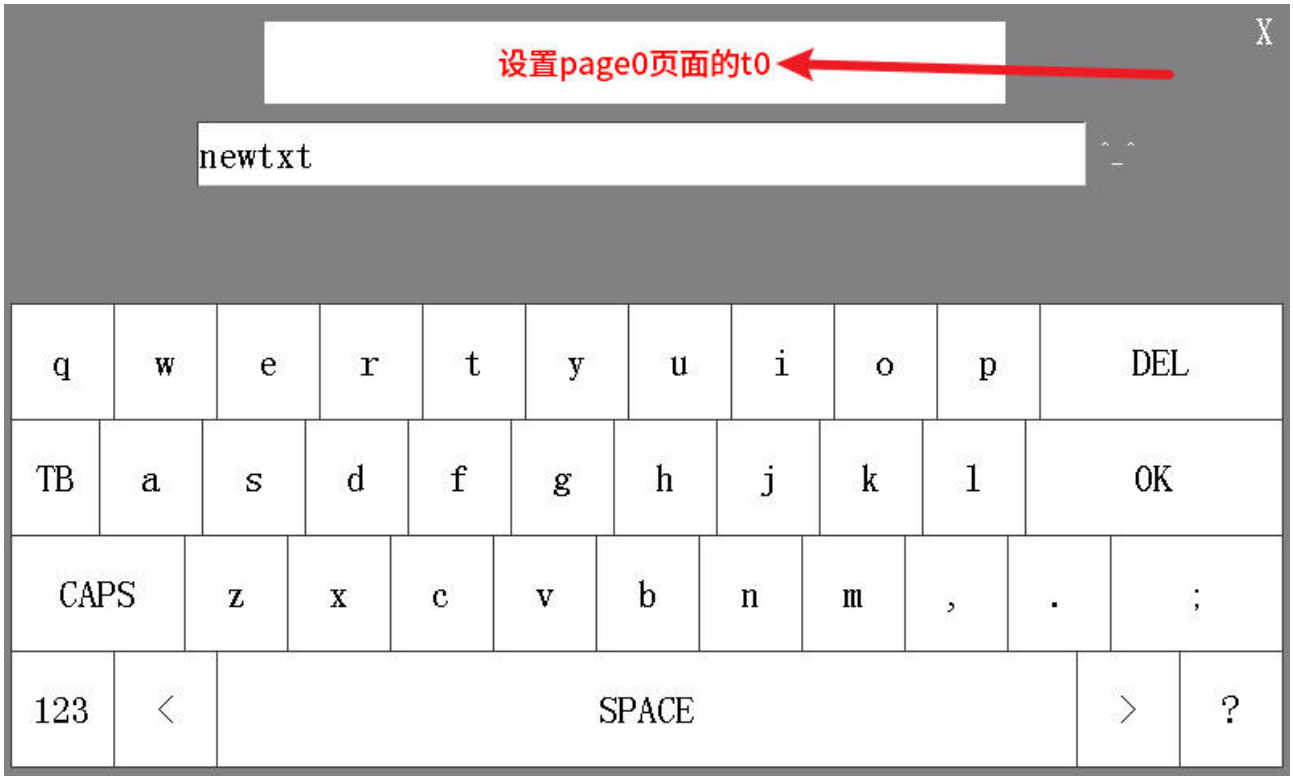
8.1.3.9.1 键盘输入回车-样例工程下载

演示工程下载链接:

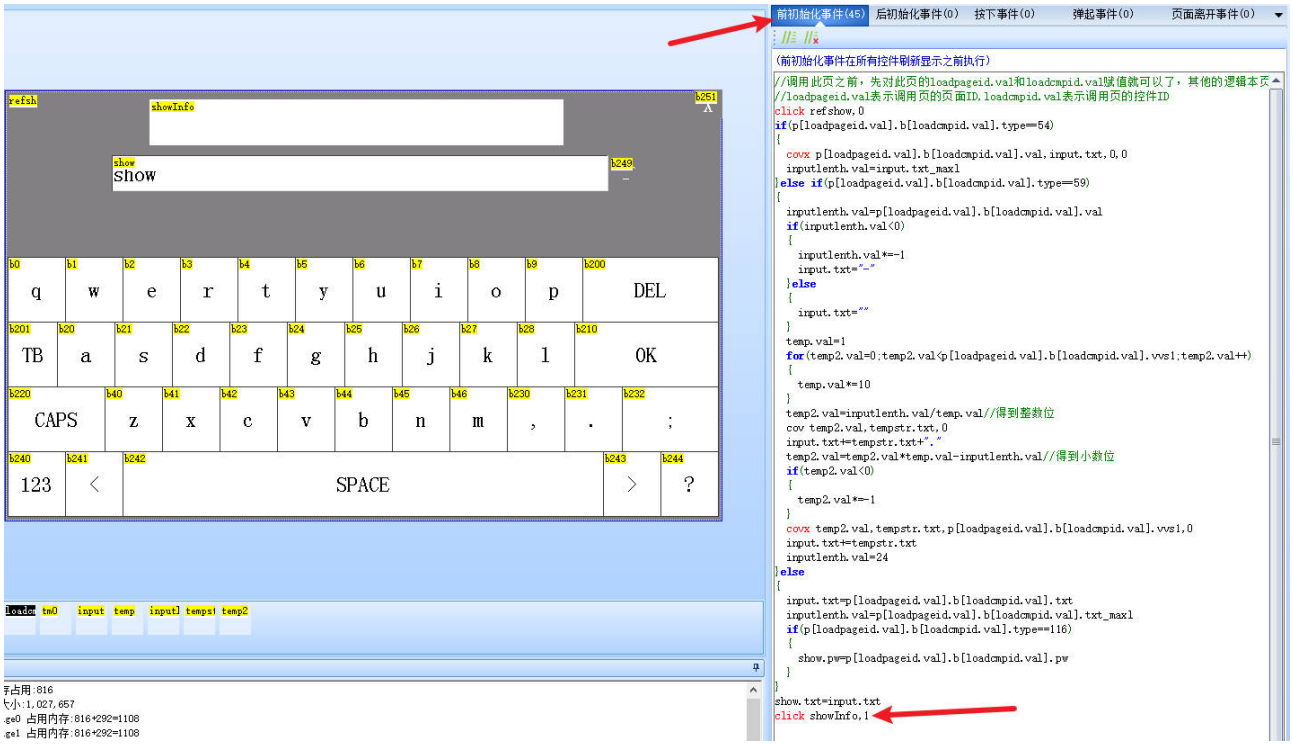
《键盘输入回车》演示工程下载

8.1.3.10 进入键盘时显示不同的备注信息

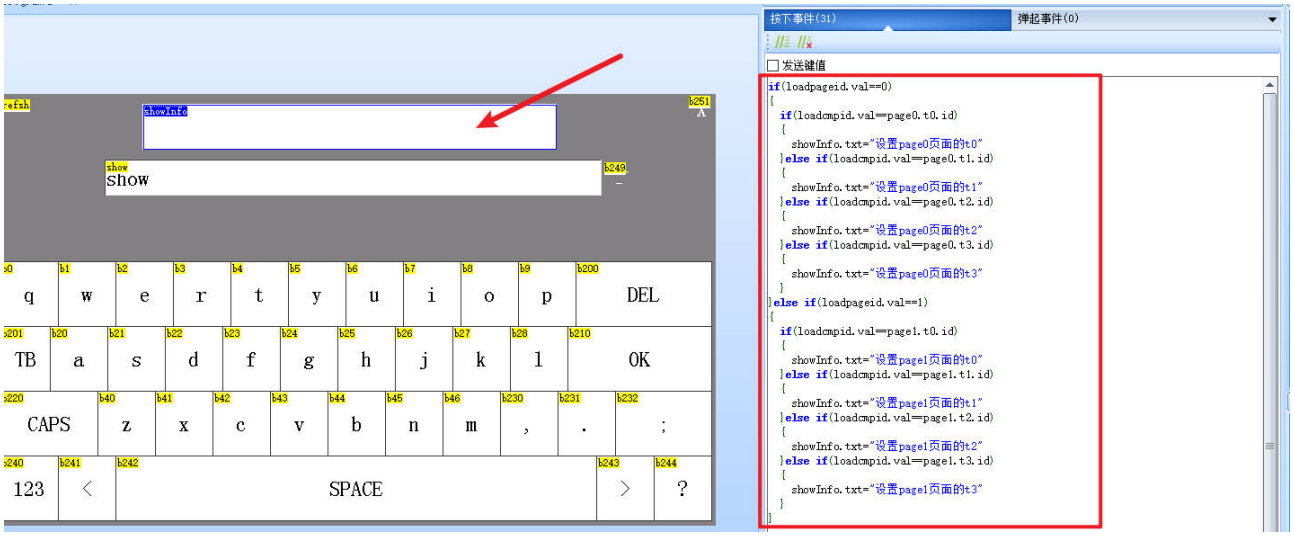
用于提示当前用户编辑的具体是哪个控件的参数



在前初始化事件里去触发控件



判断 loadpageid.val 和 loadcmpid.val 就能知道来自哪个页面和控件的点击



代码如下

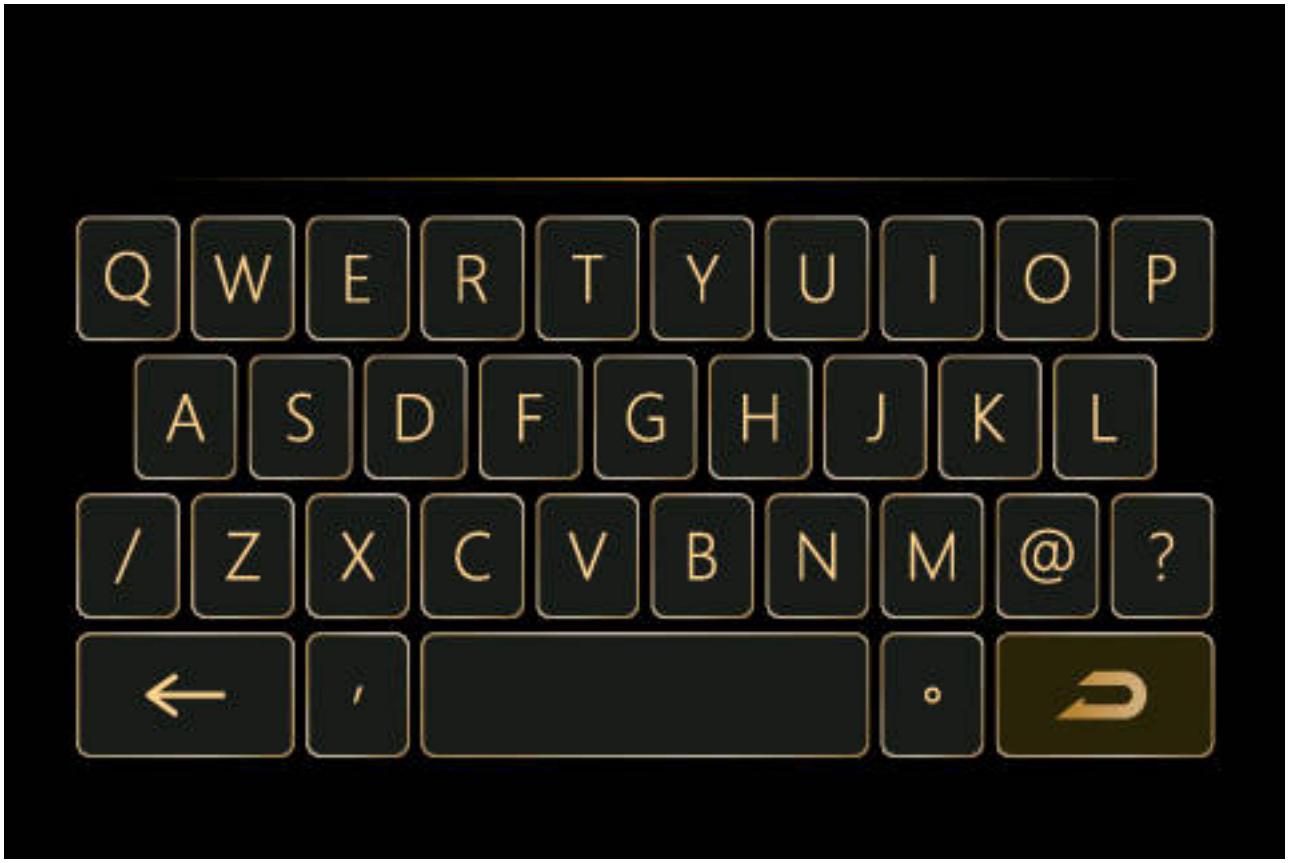
```
if(loadpageid.val==0)
{
  if(loadcmpid.val==page0.t0.id)
  {
    showInfo.txt="设置page0页面的t0"
  }else if(loadcmpid.val==page0.t1.id)
  {
    showInfo.txt="设置page0页面的t1"
  }else if(loadcmpid.val==page0.t2.id)
  {
    showInfo.txt="设置page0页面的t2"
  }else if(loadcmpid.val==page0.t3.id)
  {
    showInfo.txt="设置page0页面的t3"
  }
}else if(loadpageid.val==1)
{
  if(loadcmpid.val==page1.t0.id)
  {
    showInfo.txt="设置page1页面的t0"
  }else if(loadcmpid.val==page1.t1.id)
  {
    showInfo.txt="设置page1页面的t1"
  }else if(loadcmpid.val==page1.t2.id)
  {
    showInfo.txt="设置page1页面的t2"
  }else if(loadcmpid.val==page1.t3.id)
  {
    showInfo.txt="设置page1页面的t3"
  }
}
```

8.1.3.10.1 进入键盘时显示不同的备注信息-样例工程下载

演示工程下载链接：

《进入键盘时显示不同的备注信息》演示工程下载

8.1.3.11 自定义好看的键盘



8.1.3.11.1 《自定义好看的键盘》下载

演示工程下载链接：

《自定义好看的键盘》演示工程下载

8.1.3.12 UTF8 编码下的汉字拼音输入法键盘

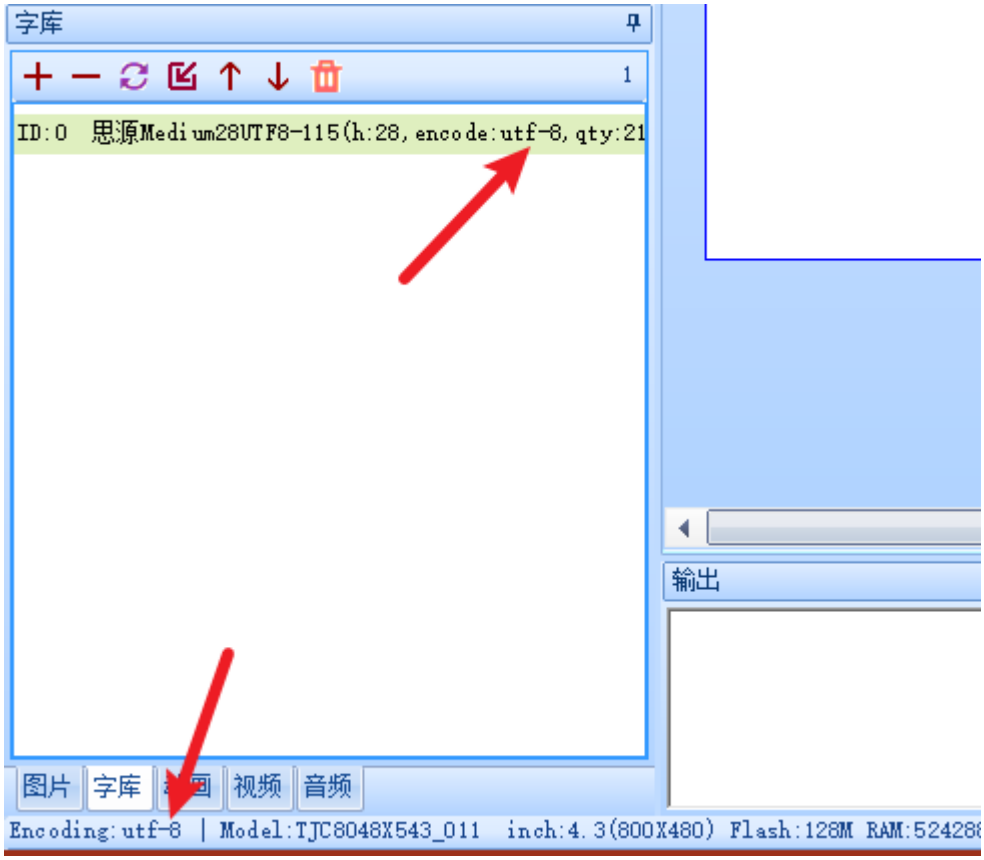
> **注意：** UTF8 键盘占用比较多的内存资源，只建议 x 系列的产品使用。

目前上位机中自带的键盘只支持 GB2312 编码下的汉字输入，如果需要支持 UTF-8 编码下的汉字输入，需要使用自定义键盘。

本文将介绍如何使用 UTF-8 编码下的汉字输入法键盘。

使用 UTF-8 汉字拼音输入法键盘的准备工作

确认工程编码和字库编码均为 UTF-8

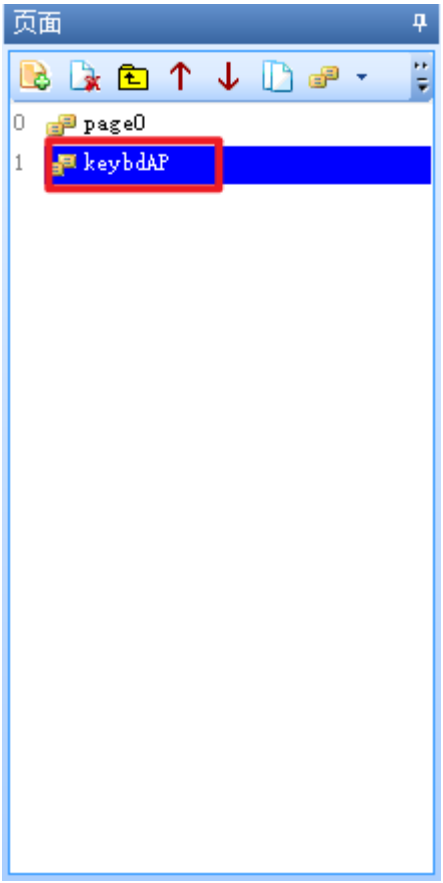


下载对应的键盘文件并解压： 《UTF8 拼音键盘》

找到横向分辨率符合你的工程的工程文件，打开，并导出键盘页面

把导出的键盘页面导入你自己的工程中

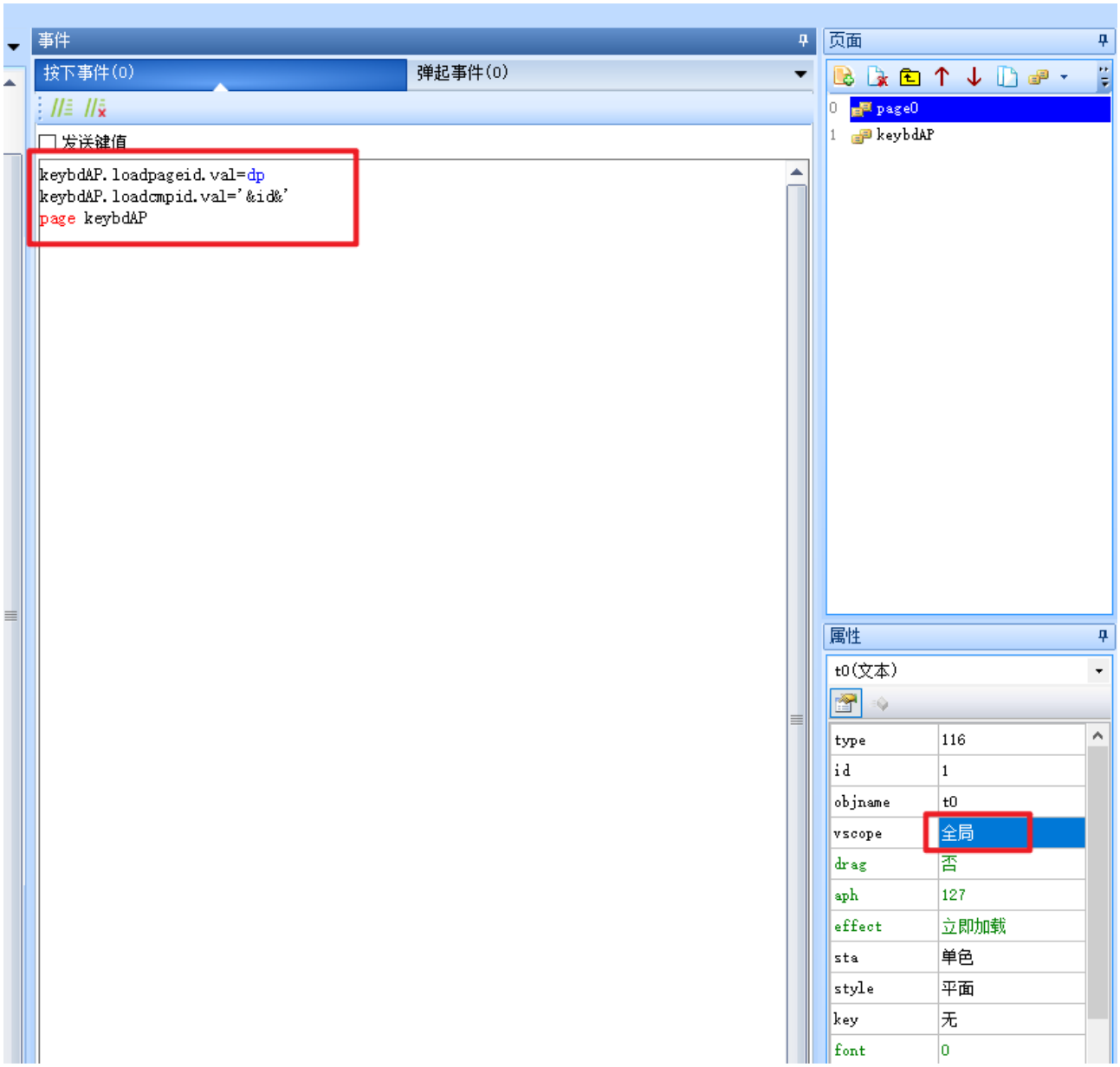
导入完毕的键盘页面如下所示



8.1.3.12.1 控件如何配置 UTF8 键盘

将控件设置为全局，不需要配置 key 属性，控件的按下事件中编写以下代码

```
keybdAP.loadpageid.val=dp          // dp:当前页面
keybdAP.loadcmpid.val='&id&'       // '&id&'当前被触发控件的id
page keybdAP
```


事件
按下事件(0)
弹起事件(0)
发送键值
keybdAP.loadpageid.val=dp
keybdAP.loadcmpid.val='&id&'
page keybdAP
页面
0 page0
1 keybdAP
属性
t0(文本)
type 116
id 1
objname t0
vscope 全局
drag 否
aph 127
effect 立即加载
sta 单色
style 平面
key 无
font 0

8.1.4 系统键盘常见问题

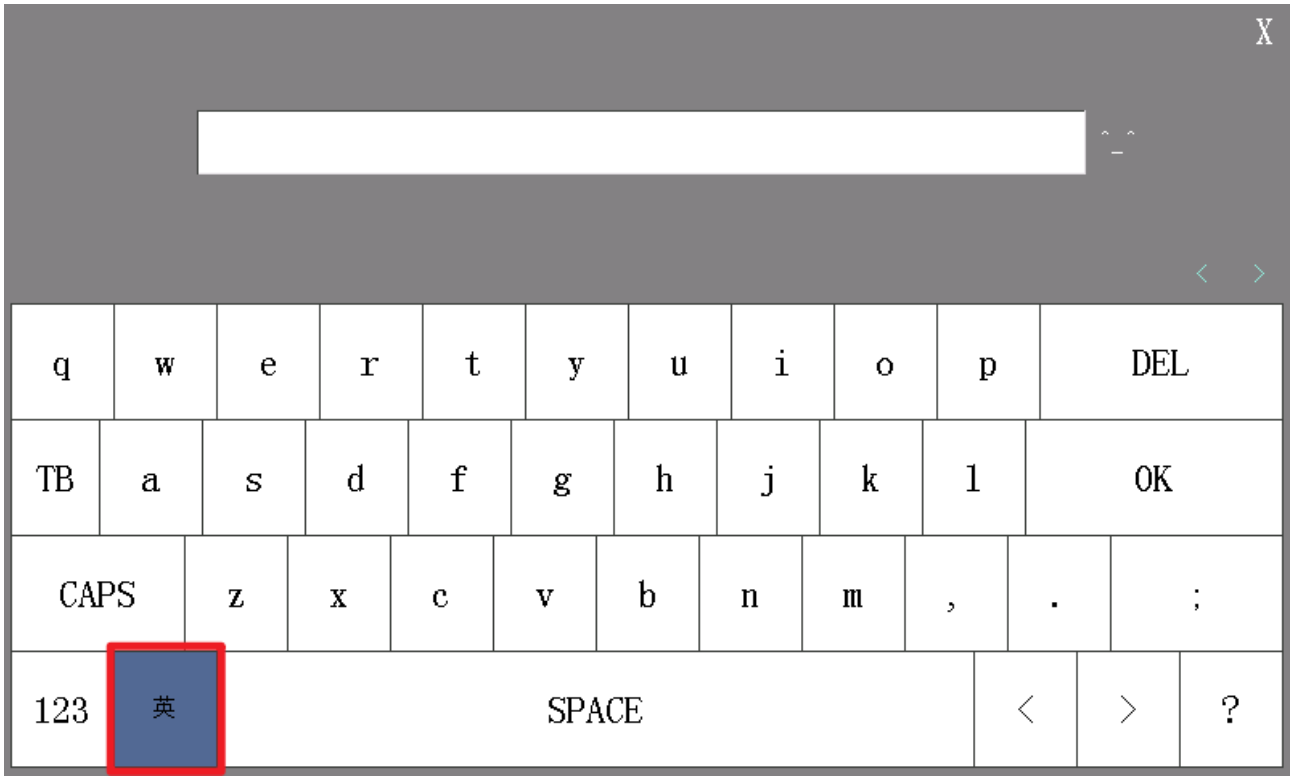
8.1.4.1 系统键盘无法用输入法打出汉字

可能原因 1：工程字符编码不是 GB2312，目前只有 GB2312 编码支持输入法打出汉字

解决办法：将工程编码修改为 GB2312 编码，参考*修改工程编码为 utf8 或者 GB2312* (page 1418)

可能原因 2：输入法键盘没有切换成中文输入法

解决办法：点击左下角的“中/英”切换按钮，只有此按钮显示为“英”时才支持中文输入



可能原因 3：输入的字符无法拼音构成汉字，例：”www”

解决办法：输入的字符为拼音可构成汉字。例：”wen”,”ni”。

8.1.4.2 系统键盘无法输入字符

可能原因 1：在调用键盘前读取了掉电存储空间中的数据，掉电存储空间并没有初始化，里面存储随机值中没有正确的结束符导致输入框被填满且无法显示。

解决办法：让设备第一次开机通过代码对掉电存储空间进行一次初始化

参考如何对掉电存储空间进行初始化 (page 1320)

可能原因 2：修改键盘部分代码，导致键盘功能失效了。

解决办法：在页面窗口，右键系统键盘页面重置系统页面，参考键盘解锁和重置 (page 911)

8.1.4.3 控件调用系统键盘输入值后，控件显示的数据没有改变

可能原因：数据输入时是通过切换页面调用系统键盘，当输入完数据会再次回到控件的页面，此时会再次执行页面初始化事件，初始化事件的赋值会对控件重新赋值，这样就会好像控件回到默认值（复位）一样，导致调用系统键盘给控件赋值无效。

解决办法：

1. 不要在此控件页面初始化事件里给控件赋初始值，可以专门做一个用来上电赋初始值的页面，做为上电默认页，在此页面的初始化事件里完成所有的初始化动作之后，再切换到工程主页，这样初始化动作就只会上电执行一次，不会重复执行。
2. 在数据输入页面建一个变量（设置全局），用来表示是否已经初始化过了，赋初始值的时候判断控件是否已经被初始化过，是的话就不要再赋初值了，这样键盘调用完之后再次回来就不会重复执行了。

8.1.4.4 控件调用系统键盘输入值后，其他控件又回到默认值

可能原因：数据输入时是通过切换页面调用系统键盘，切换页面的时候其他控件 vscope 设置为私有会释放内存，恢复为默认值。若其他控件不想恢复默认值，将它的 vscope 属性设置为全局就好了。

8.2 主动解析模式应用详解

8.2.1 主动解析基本知识

警告: 如果对串口屏不熟悉, 请谨慎使用主动解析（也叫自由协议/自由格式通讯)。

8.2.1.1 淘晶驰字符串指令和主动解析

什么是淘晶驰字符串指令

```
n0.val=100
t0.txt="abc"
covx n0.val,t0.txt,0,0
```

类似上面的这些字符串指令，可以直接写在屏幕中，或者通过串口发送给串口屏的字符串指令，称为淘晶驰字符串指令。

屏幕上电默认是淘晶驰字符串指令模式，所有串口指令需按照淘晶驰字符串指令格式来对屏幕进行操作，即单片机发送类似于 n0.val=100 的字符串指令格式来控制屏幕。

假如你需要自定义协议，不按照淘晶驰字符串指令格式来发串口数据给屏幕，而使用你自定义的格式，那么就需要把屏幕配置为主动解析模式。要使用此功能，请务必确保你有以下 2 点基础:

1. 明白什么叫 HEX, 什么叫 String, 什么叫 ASCII, 分别什么关系，怎么转换。

2. 明白单字节数值，双字节数值，四字节数值，分别有什么区别，它们在内存中是什么样的储存方式，明白什么叫小端模式，什么叫大端模式，大小端数据之间如何转换。

如果以上 2 点你都比较明白，那么请继续往下看，否则强烈建议不要再继续往下看了，因为大多数的项目是用不上这个功能的，使用默认的被动解析模式就可以了，没必要配置为下面的主动解析模式。

此篇幅涉及到以下几个内容:

主动解析原理 (page 944)

主动解析基本步骤 (page 944)

串口数据解析模式系统变量 *recmod* (page 945)

串口缓冲区数据大小系统变量 *usize* (page 946)

串口缓冲区数据组 *u[index]* (page 947)

串口缓冲区数据拷贝指令 *ucopy* (page 948)

删除缓冲区前面 *N* 字节数据 *udelete* (page 950)

如何退出主动解析 (page 951)

8.2.1.2 主动解析原理

用户在定时器控件中手动解析串口缓冲区中的数据。

如果把串口屏比喻成人，串口缓冲区是电子邮箱，定时器是闹钟。

那就是闹钟定时提醒人去查看电子邮箱中接收到了什么邮件，并且根据电子邮箱中不同的数据做出不同的操作。(通过定时器去访问串口缓冲区数据组 u[index])

为了确保邮件是发给你的，你必须确认这些邮件是以”亲爱的 XX”开头, 并以”此致敬礼”结尾的 (有帧头和帧尾)

某些情况下为了确保邮件信息是正确的, 你们还需要对暗号 (即增加校验)

电子邮箱的大小是有限的，需要根据情况将已读的数据删除或者清空邮箱。(对应 udelete 和 code_c)

8.2.1.3 主动解析基本步骤

1. 在 program.s 中配置波特率 baud 和开启主动解析 recmod=1
2. 每个页面都需要新建一个定时器控件, 在定时器中定时判断 usize 的大小，当 usize 大于等于一帧数据的长度时，进入查找帧头标志或帧尾标志流程

> **注意:** 每个页面仅建议在一个定时器中读取串口缓冲区的数据, 在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

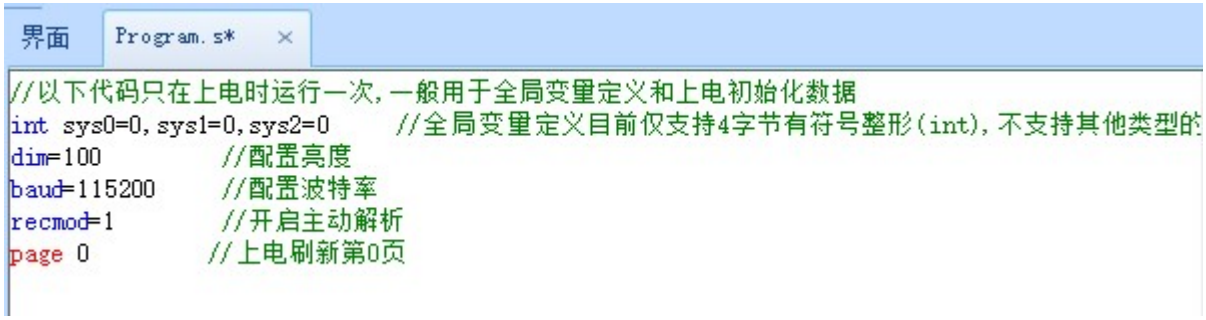
3. 查找到帧头标志或帧尾标志后，如果有校验，对数据进行校验
4. 校验不通过，丢弃该帧（删除对应长度的数据）
5. 校验通过，则提取串口缓冲区中需要的数据
6. 每次读完数据后，使用 udelete 指令删除缓冲区中已经读取的字节数，否则缓冲区溢出后就无法接收新数据。

8.2.1.4 串口数据解析模式系统变量 recmod

recmod=0 为被动解析模式，recmod=1 为主动解析模式

屏幕上电 recmod=0，即被动解析模式，在此模式下，外部设备按照标准指令集的指令格式发送串口指令给屏幕执行；例如:n0.val=100

如果你将 recmod 设置为 1，那么屏幕进入主动解析模式，然后所有的串口指令都不会被执行 (注意：是串口接收到的数据不会被执行，上位软件编辑界面时写入事件中的固件指令是不会受影响的，依然正常执行)，所有的串口数据均存放在串口缓冲区中，等待您去主动读取和处理。



具体配置代码如下，请注意，page 指令后面的代码不会被执行，请将代码写在 page 指令前面

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
int sys0=0,sys1=0,sys2=0      //全局变量定义目前仅支持4字节有符号整形(int),
↪不支持其他类型的全局变量声明,如需使用字符串类型可以在页面中使用变量控件来实现
dim=100           //配置亮度
baud=115200       //配置波特率
recmod=1          //开启主动解析
page 0           //上电刷新第0页
```

注意： 用上位机软件模拟器联机屏幕，屏幕将会强制退出主动解析模式，此时需要将屏重新上电才可以正常和模拟器联机调试屏幕。

警告： 请不要在定时器中频繁进入 recmod=1，否则将会导致工程下载失败，此时只能通过 SD 卡更新工程，请参考通过 *SD* 卡下载工程到串口屏 (page 155)

8.2.1.5 串口缓冲区数据大小系统变量 usize

usize 只能读取，不可设置

读取此变量可以知道当前串口缓冲区已经缓存多少数据。

定时器里每次进入解析之前，必须先判断 usize 的值大于 1 个数据帧的长度，才能进入正常的解析!!!

8.2.1.6 串口缓冲区数据组 u[index]

串口缓冲区数据组的写法为 u[index] (index 为序号)

> **注意:** 任何时间读取串口缓冲区数据组 u[index] 时一定要先判断 uszie 的大小!

例 1：对缓冲区中的数据进行判断（通常用于判断帧头帧尾）

```
if(usize>0&&u[0]!=0x55&&u[1]!=0xAA)
{
    udelete 1
}
```

X2/X3/X5 的串口缓冲区是 4k 字节，u 数组的索引范围是 u[0]~u[4095]

T0/T1/K0 的串口缓冲区是 1k 字节，u 数组的索引范围是 u[0]~u[1023]

例 2：将串口缓冲区的所有数据返回

```
if(usize>0)
{
   for(sys0=0;sys0<usize;sys0++)
   {
      prints u[sys0],1
   }
   code_c
}
```

8.2.1.7 串口缓冲区数据拷贝指令 ucopy

格式: ucopy,att, srcstar, lenth, decstar

说明：将串口缓冲区中的数据拷贝到变量中 (recmod=1 模式下有效)

att: 目标变量名称

srcstar: 串口缓冲区数据起始位

lenth: 拷贝长度

decstar: 目标变量数据起始位

此指令可以从串口缓冲区的指定位置连续拷贝指定数量的数据到目标变量 (目标变量可以是字符串变量，可以是数值变量)。

例 1：从缓冲区中 0 位置开始获取一个 4 字节的数值 (小端模式，低位在前)，赋值给数字控件 n0, 写法如下:

```
ucopy n0.val,0,4,0
```

注意: 每个数值变量都是有符号的 32 位类型，但是如果你使用 ucopy 从缓冲区获取小于 4 字节的数值，一定要注意剩余部分的字节数据处理，以免出现数据异常，操作方法请参考下面这个例子:

例 2: 从缓冲区中 0 位置开始获取一个 2 字节的有符号数值 (小端模式，低位在前)，赋值给数字控件 n0.val 的低 16 位, 写法如下:

```
sys0=0  //需要将sys0赋值为0,否则如果sys0的高字节有数据,
↪那么获取的数据就会和你想要的数据不符
ucopy sys0,0,2,0
if(sys0>32767)
{
  n0.val=sys0-65536
}else
{
  n0.val=sys0
}
```

例 3: 从缓冲区中 0 位置开始获取一个 1 字节的有符号数值 (小端模式，低位在前)，赋值给数字控件 n0.val 的低 8 位, 写法如下:

```
sys0=0  //需要将sys0赋值为0,否则如果sys0的高字节有数据,
↪那么获取的数据就会和你想要的数据不符
ucopy sys0,0,1,0
if(sys0>127)
{
  n0.val=sys0-256
}else
{
  n0.val=sys0
}
```

参考链接:

主动解析下如何提取负数 (page 1425)

注意: 先将 sys0 赋值为 0，目的在于确保 sys0 的 4 个字节全置 0，然后再从缓冲区拷贝 2 个字节进来，否则会因为你只拷了 2 字节而导致 sys0 原来剩下的 2 字节数据还在，然后导致 sys0 最终的数值并不是你想要的数值。

例 4: 从缓冲区中 0 位置开始获取一个 10 字节的字符串，赋值给文本控件 t0, 写法如下:

```
ucopy t0.txt,0,10,0
```

8.2.1.8 删除缓冲区前面 N 字节数据 udelete

格式: udelete nsize

说明：删除缓冲区前面 N 字节数据 (recmod=1 模式下有效)

nsize: 需要删除的字节数

例 1：删除串口缓冲区前面 8 个字节数据, 写法如下:

```
udelete 8
```

例 2：删除串口缓冲区前面多个字节数据, 写法如下:

```
n0.val=8
udelete n0.val
```

例 3：删除串口缓冲区前面多个字节数据, 写法如下:

```
frameLength=8
udelete frameLength
```

例 4：删除串口缓冲区所有数据, 写法如下:

```
udelete usize
```

使用 udelete 删除数据时, 只是修改了 usize 的值, 并移动了串口环形缓冲区的指针，并不会真正的删除数据，下次再读取数据时，会从新指向的位置开始读取。

因此, 每次解析数据前需要判断 usize 的值是否大于等于 1 个数据帧的长度, 然后再进行解析。

8.2.1.9 如何退出主动解析

方法 1：发送一串退出密码来实现退出主动解析模式, 退出密码为一串 24 字节的字符串 +3 字节的结束符。

24 字节的字符串:

DRAKJHSUYDGBNCJHGJKSHBDN (固定的字符串数据，不可修改，必须大写)

3 字节的结束符 (Hex 数据)：

0xff 0xff 0xff

合计 27 字节

其 hex 格式如下所示：44 52 41 4B 4A 48 53 55 59 44 47 42 4E 43 4A 48 47 4A 4B 53 48 42 44 4E FF FF FF

方法 2：增加一个主动解析指令，接收到对应的指令后执行 recmod=0

8.2.2 解析定长 hex 格式指令-自定义协议

8.2.2.1 例 1：帧头为 0x55, 帧尾为 3 个 0xff, 每次传输灯的序号和对应的状态

注意： 每个页面仅建议在一个定时器中读取串口缓冲区的数据, 在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

新建一个空白工程

在工程中新建一个定时器 tmDecode，tim 设置为 50，en 设置为 1，用于定时解析串口数据

新建数字控件 n1,n2,n3,n4，用于显示解析出来的数据

假设串口屏获取 4 个 led 灯的开关状态，串口屏需要解析的参数为灯的状态（开或关）

注意： 16 进制发送时，可以忽略大小写，ff 和 FF 是一样的，AA 和 aa 是一样的，但是请不要写成 Ff，Ab 这样大小写混用

```
55 01 01 ff ff ff  //含义：1号led打开

55 02 01 ff ff ff  //含义：2号led打开

55 01 00 ff ff ff  //含义：1号led关闭

55 02 00 ff ff ff  //含义：2号led关闭
```

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0

//frameLength: 每帧数据长度
//getFrameFlag: 是否找到帧头或帧尾标记
int frameLength=6,getFrameFlag

bauds=115200 //波特率115200
recmod=1     //打开主动解析
page 0                          //上电刷新第0页
```

解析定时器（tim 为 50）中的代码如下图所示

```
while(usize>=frameLength&&getFrameFlag==0)
{
  if(u[0]==0x55&&u[3]==0xFF&&u[4]==0xFF&&u[5]==0xFF)
  {
    //找到帧头,退出循环
    getFrameFlag=1
  }else
  {
    //如果帧头不对，就一直删除1个字节，直到不满足条件退出循环
    udelete 1
  }
}
if(getFrameFlag==1)
{
  if(u[1]==1)
```

(续下页)

(接上页)

```
  {
    n1.val=u[2]
  }else if(u[1]==2)
  {
    n2.val=u[2]
  }else if(u[1]==3)
  {
    n3.val=u[2]
  }else if(u[1]==4)
  {
    n4.val=u[2]
  }
  udelete frameLength //删除已经解析过的数据
  getFrameFlag=0 //清空标记变量
}
```

8.2.2.1.1 定长 hex-示例 1-样例工程下载

演示工程下载链接：

《定长 hex-示例 1》演示工程下载

8.2.2.2 例 2：帧头为 0x55，帧尾为 3 个 0xff，一次性传输 4 个灯的状态

注意： 每个页面仅建议在一个定时器中读取串口缓冲区的数据, 在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

新建一个空白工程

在工程中新建一个定时器 tmDecode，tim 设置为 50，en 设置为 1，用于定时解析串口数据

新建数字控件 n1,n2,n3,n4，用于显示解析出来的数据

假设串口屏获取 4 个 led 灯的开关状态，串口屏需要解析的参数为灯的状态（开或关）

注意： 16 进制发送时，可以忽略大小写，ff 和 FF 是一样的，AA 和 aa 是一样的，但是请不要写成 Ff，Ab 这样大小写混用

```
55 01 01 01 01 ff ff ff  //含义：4个灯状态分别为：开、开、开、开

55 00 00 00 00 ff ff ff  //含义：4个灯状态分别为：关、关、关、关

55 00 01 00 01 ff ff ff  //含义：4个灯状态分别为：关、开、关、开

55 01 00 01 00 ff ff ff  //含义：4个灯状态分别为：开、关、开、关
```

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0

//frameLength: 每帧数据长度
//getFrameFlag: 是否找到帧头或帧尾标记
int frameLength=8,getFrameFlag

bauds=115200 //波特率115200
recmod=1     //打开主动解析
page 0
```

解析定时器（tim 为 50）中的代码如下图所示

```
while(usize>=frameLength&&getFrameFlag==0)
{
  if(u[0]==0x55&&u[5]==0xFF&&u[6]==0xFF&&u[7]==0xFF)
  {
    //找到帧头,退出循环
    getFrameFlag=1
  }else
  {
    //如果帧头不对，就一直删除1个字节，直到不满足条件退出循环
    udelete 1
  }
}
if(getFrameFlag==1)
{
  n1.val=u[1]
  n2.val=u[2]
  n3.val=u[3]
```

(续下页)

(接上页)

```
  n4.val=u[4]
  udelete frameLength //删除已经解析过的数据
  getFrameFlag=0 //清空标记变量
}
```

8.2.2.2.1 定长 hex-示例 2-样例工程下载

演示工程下载链接：

《定长 hex-示例 2》演示工程下载

8.2.2.3 例 3：帧头为 0x33 ，帧尾为 0d 0a，每次传输灯的序号和对应的状态

注意： 每个页面仅建议在一个定时器中读取串口缓冲区的数据, 在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

新建一个空白工程

在工程中新建一个定时器 tmDecode，tim 设置为 50，en 设置为 1，用于定时解析串口数据

新建数字控件 n1,n2,n3,n4，用于显示解析出来的数据

假设串口屏获取 4 个 led 灯的开关状态，串口屏需要解析的参数为灯的状态（开或关）

注意： 16 进制发送时，可以忽略大小写，ff 和 FF 是一样的，AA 和 aa 是一样的，但是请不要写成 Ff，Ab 这样大小写混用

```
33 01 00 0d 0a  //含义：1号led关闭

33 04 01 0d 0a  //含义：4号led打开

33 01 01 0d 0a  //含义：1号led打开

33 04 00 0d 0a  //含义：4号led关闭
```

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0

//frameLength: 每帧数据长度
//getFrameFlag: 是否找到帧头或帧尾标记
int frameLength=5,getFrameFlag

bauds=115200 //波特率115200
recmod=1    //打开主动解析
page 0                          //上电刷新第0页
```

解析定时器（tim 为 50）中的代码如下图所示

```
while(usize>=frameLength&&getFrameFlag==0)
{
  if(u[0]==0x33&&u[3]==0x0d&&u[4]==0x0a)
  {
    //找到帧头,退出循环
    getFrameFlag=1
  }else
  {
    //如果帧头不对，就一直删除1个字节，直到不满足条件退出循环
    udelete 1
  }
}
if(getFrameFlag==1)
{
  if(u[1]==1)
  {
    n1.val=u[2]
```

(续下页)

(接上页)

```
  }else if(u[1]==2)
  {
    n2.val=u[2]
  }else if(u[1]==3)
  {
    n3.val=u[2]
  }else if(u[1]==4)
  {
    n4.val=u[2]
  }
  udelete frameLength //删除已经解析过的数据
  getFrameFlag=0 //清空标记变量
}
```

8.2.2.3.1 定长 hex-示例 3-样例工程下载

演示工程下载链接：

《定长 hex-示例 3》演示工程下载

8.2.2.4 例 4：没有帧头，帧尾为 0d 0a，每次传输灯的序号和对应的状态

注意： 每个页面仅建议在一个定时器中读取串口缓冲区的数据, 在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

新建一个空白工程

在工程中新建一个定时器 tmDecode，tim 设置为 50，en 设置为 1，用于定时解析串口数据

新建数字控件 n1,n2,n3,n4，用于显示解析出来的数据

假设串口屏获取 4 个 led 灯的开关状态，串口屏需要解析的参数为灯的状态（开或关）

注意： 16 进制发送时，可以忽略大小写，ff 和 FF 是一样的，AA 和 aa 是一样的，但是请不要写成 Ff，Ab 这样大小写混用

```
04 01 0d 0a  //含义：4号led打开

01 01 0d 0a  //含义：1号led打开

04 00 0d 0a  //含义：4号led关闭

01 00 0d 0a  //含义：1号led关闭
```

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0

//frameLength: 每帧数据长度
//getFrameFlag: 是否找到帧头或帧尾标记
int frameLength=4,getFrameFlag

bauds=115200 //波特率115200
recmod=1     //打开主动解析
page 0                          //上电刷新第0页
```

解析定时器（tim 为 50）中的代码如下图所示

```
while(usize>=frameLength&&getFrameFlag==0)
{
  if(u[2]==0x0d&&u[3]==0x0a)
  {
    //找到帧头,退出循环
    getFrameFlag=1
  }else
  {
    //如果帧头不对，就一直删除1个字节，直到不满足条件退出循环
    udelete 1
  }
}
if(getFrameFlag==1)
{
  if(u[0]==1)
  {
    n1.val=u[1]
```

(续下页)

(接上页)

```
  }else if(u[0]==2)
  {
    n2.val=u[1]
  }else if(u[0]==3)
  {
    n3.val=u[1]
  }else if(u[0]==4)
  {
    n4.val=u[1]
  }
  udelete frameLength //删除已经解析过的数据
  getFrameFlag=0 //清空标记变量
}
```

8.2.2.4.1 定长 hex-示例 4-样例工程下载

演示工程下载链接：

《定长 hex-示例 4》演示工程下载

8.2.2.5 例 5：帧头为 0x33，没有帧尾，每次传输灯的序号和对应的状态

注意： 每个页面仅建议在一个定时器中读取串口缓冲区的数据, 在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

新建一个空白工程

在工程中新建一个定时器 tmDecode，tim 设置为 50，en 设置为 1，用于定时解析串口数据

新建数字控件 n1,n2,n3,n4，用于显示解析出来的数据

假设串口屏获取 4 个 led 灯的开关状态，串口屏需要解析的参数为灯的状态（开或关）

注意： 16 进制发送时，可以忽略大小写，ff 和 FF 是一样的，AA 和 aa 是一样的，但是请不要写成 Ff，Ab 这样大小写混用

```
33 04 01  //含义：4号led打开

33 01 01  //含义：1号led打开

33 04 00  //含义：4号led关闭

33 01 00  //含义：1号led关闭
```

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0

//frameLength: 每帧数据长度
//getFrameFlag: 是否找到帧头或帧尾标记
int frameLength=3,getFrameFlag

bauds=115200 //波特率115200
recmod=1    //打开主动解析
page 0                          //上电刷新第0页
```

解析定时器（tim 为 50）中的代码如下图所示

```
while(usize>=frameLength&&getFrameFlag==0)
{
  if(u[0]==0x33)
  {
    //找到帧头,退出循环
    getFrameFlag=1
  }else
  {
    //如果帧头不对，就一直删除1个字节，直到不满足条件退出循环
    udelete 1
  }
}
if(getFrameFlag==1)
{
  if(u[1]==1)
  {
    n1.val=u[2]
```

(续下页)

(接上页)

```
  }else if(u[1]==2)
  {
    n2.val=u[2]
  }else if(u[1]==3)
  {
    n3.val=u[2]
  }else if(u[1]==4)
  {
    n4.val=u[2]
  }
  udelete frameLength //删除已经解析过的数据
  getFrameFlag=0 //清空标记变量
}
```

8.2.2.5.1 定长 hex-示例 5-样例工程下载

演示工程下载链接：

《定长 hex-示例 5》演示工程下载

8.2.2.6 例 6：没有帧头，帧尾为 3 个 0xff，每次传输 4 个灯的状态

注意： 每个页面仅建议在一个定时器中读取串口缓冲区的数据, 在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

新建一个空白工程

在工程中新建一个定时器 tmDecode，tim 设置为 50，en 设置为 1，用于定时解析串口数据

新建数字控件 n1,n2,n3,n4，用于显示解析出来的数据

假设串口屏获取 4 个 led 灯的开关状态，串口屏需要解析的参数为灯的状态（开或关）

注意： 16 进制发送时，可以忽略大小写，ff 和 FF 是一样的，AA 和 aa 是一样的，但是请不要写成 Ff，Ab 这样大小写混用

```
01 01 01 01 ff ff ff  //含义：4个灯状态分别为：开、开、开、开

00 00 00 00 ff ff ff  //含义：4个灯状态分别为：关、关、关、关

00 01 00 01 ff ff ff  //含义：4个灯状态分别为：关、开、关、开

01 00 01 00 ff ff ff  //含义：4个灯状态分别为：开、关、开、关
```

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0

//frameLength: 每帧数据长度
//getFrameFlag: 是否找到帧头或帧尾标记
int frameLength=7,getFrameFlag

bauds=115200 //波特率115200
recmod=1     //打开主动解析
page 0
```

解析定时器（tim 为 50）中的代码如下图所示

```
while(usize>=frameLength&&getFrameFlag==0)
{
  if(u[4]==0xFF&&u[5]==0xFF&&u[6]==0xFF)
  {
    //找到帧头,退出循环
    getFrameFlag=1
  }else
  {
    //如果帧头不对，就一直删除1个字节，直到不满足条件退出循环
    udelete 1
  }
}
if(getFrameFlag==1)
{
  n1.val=u[0]
  n2.val=u[1]
  n3.val=u[2]
```

(续下页)

(接上页)

```
  n4.val=u[3]
  udelete frameLength //删除已经解析过的数据
  getFrameFlag=0 //清空标记变量
}
```

8.2.2.6.1 定长 hex-示例 6-样例工程下载

演示工程下载链接：

《定长 hex-示例 6》演示工程下载

8.2.2.7 例 7：帧头为 0x55，没有帧尾，每次传输 4 个灯的状态

注意： 每个页面仅建议在一个定时器中读取串口缓冲区的数据, 在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

新建一个空白工程

在工程中新建一个定时器 tmDecode，tim 设置为 50，en 设置为 1，用于定时解析串口数据

新建数字控件 n1,n2,n3,n4，用于显示解析出来的数据

假设串口屏获取 4 个 led 灯的开关状态，串口屏需要解析的参数为灯的状态（开或关）

注意： 16 进制发送时，可以忽略大小写，ff 和 FF 是一样的，AA 和 aa 是一样的，但是请不要写成 Ff，Ab 这样大小写混用

```
55 01 01 01 01  //含义：4个灯状态分别为：开、开、开、开

55 00 00 00 00  //含义：4个灯状态分别为：关、关、关、关

55 00 01 00 01  //含义：4个灯状态分别为：关、开、关、开

55 01 00 01 00  //含义：4个灯状态分别为：开、关、开、关
```

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0

//frameLength: 每帧数据长度
//getFrameFlag: 是否找到帧头或帧尾标记
int frameLength=5,getFrameFlag

bauds=115200 //波特率115200
recmod=1     //打开主动解析
page 0
```

解析定时器（tim 为 50）中的代码如下图所示

```
while(usize>=frameLength&&getFrameFlag==0)
{
  if(u[0]==0x55)
  {
    //找到帧头,退出循环
    getFrameFlag=1
  }else
  {
    //如果帧头不对，就一直删除1个字节，直到不满足条件退出循环
    udelete 1
  }
}
if(getFrameFlag==1)
{
  n1.val=u[1]
  n2.val=u[2]
  n3.val=u[3]
```

(续下页)

(接上页)

```
  n4.val=u[4]
  udelete frameLength //删除已经解析过的数据
  getFrameFlag=0 //清空标记变量
}
```

8.2.2.7.1 定长 hex-示例 7-样例工程下载

演示工程下载链接：

《定长 hex-示例 7》演示工程下载

8.2.2.8 例 8：没有帧头也没有帧尾，每次传输 4 个灯的状态

注意： 每个页面仅建议在一个定时器中读取串口缓冲区的数据, 在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

新建一个空白工程

在工程中新建一个定时器 tmDecode，tim 设置为 50，en 设置为 1，用于定时解析串口数据

新建数字控件 n1,n2,n3,n4，用于显示解析出来的数据

假设串口屏获取 4 个 led 灯的开关状态，串口屏需要解析的参数为灯的状态（开或关）

注意： 16 进制发送时，可以忽略大小写，ff 和 FF 是一样的，AA 和 aa 是一样的，但是请不要写成 Ff，Ab 这样大小写混用

```
01 01 01 01  //含义：4个灯状态分别为：开、开、开、开

00 00 00 00  //含义：4个灯状态分别为：关、关、关、关

00 01 00 01  //含义：4个灯状态分别为：关、开、关、开

01 00 01 00  //含义：4个灯状态分别为：开、关、开、关
```

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0

//frameLength: 每帧数据长度
//getFrameFlag: 是否找到帧头或帧尾标记
int frameLength=4,getFrameFlag

bauds=115200 //波特率115200
recmod=1     //打开主动解析
page 0
```

解析定时器（tim 为 50）中的代码如下图所示

```
if(usize>=frameLength)
{
  n1.val=u[0]
  n2.val=u[1]
  n3.val=u[2]
  n4.val=u[3]
  code_c //清空串口数据缓冲区
}
```

8.2.2.8.1 定长 hex-示例 8-样例工程下载

演示工程下载链接：

《定长 hex-示例 8》演示工程下载

8.2.2.9 例 9：一个字节有 8 位，其实可以用 1 个字节传输 8 个灯的开关状态

> **注意：** 每个页面仅建议在一个定时器中读取串口缓冲区的数据, 在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

新建一个空白工程

在工程中新建一个定时器 tmDecode，tim 设置为 50，en 设置为 1，用于定时解析串口数据

新建数字控件 n1,n2,n3,n4，用于显示解析出来的数据

假设串口屏获取 4 个 led 灯的开关状态，串口屏需要解析的参数为灯的状态（开或关）

> **注意：** 16 进制发送时，可以忽略大小写，ff 和 FF 是一样的，AA 和 aa 是一样的，但是请不要写成 Ff，Ab 这样大小写混用

```
0F   //二进制为：00001111，含义：后4位表示4个灯状态分别为：开、开、开、开

00   //二进制为：00000000，含义：后4位表示4个灯状态分别为：关、关、关、关

05   //二进制为：00000101，含义：后4位表示4个灯状态分别为：关、开、关、开

0A   //二进制为：00001010，含义：后4位表示4个灯状态分别为：开、关、开、关
```

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0

//frameLength: 每帧数据长度
//getFrameFlag: 是否找到帧头或帧尾标记
int frameLength=1,getFrameFlag

bauds=115200 //波特率115200
recmod=1     //打开主动解析
page 0
```

解析定时器（tim 为 50）中的代码如下图所示

```
if(usize>=frameLength)
{
  //通过位运算来获取每个位的数据
  n1.val=u[0]&0x01
  n2.val=u[0]>>1&0x01
  n3.val=u[0]>>2&0x01
  n4.val=u[0]>>3&0x01
  code_c //清空串口数据缓冲区
}
```

8.2.2.9.1 定长 hex-示例 9-样例工程下载

演示工程下载链接：

《定长 hex-示例 9》演示工程下载

8.2.2.10 例 10：帧头为 0x33，无帧尾，使用 modbus_crc16 校验，一次性传输 4 个灯的状态

注意： 每个页面仅建议在一个定时器中读取串口缓冲区的数据, 在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

新建一个空白工程

在工程中新建一个定时器 tmDecode，tim 设置为 50，en 设置为 1，用于定时解析串口数据

新建数字控件 n1,n2,n3,n4，用于显示解析出来的数据

假设串口屏获取 4 个 led 灯的开关状态，串口屏需要解析的参数为灯的状态（开或关）

注意： 16 进制发送时，可以忽略大小写，ff 和 FF 是一样的，AA 和 aa 是一样的，但是请不要写成 Ff，Ab 这样大小写混用

```
//含义：4个灯状态分别为：开、开、开、开，modbus_crc16校验值为 B0 68
33 01 01 01 01 B0 68

//含义：4个灯状态分别为：关、关、关、关，modbus_crc16校验值为 20 04
33 00 00 00 00 20 04

//含义：4个灯状态分别为：关、开、关、开，modbus_crc16校验值为 B0 04
33 00 01 00 01 B0 04

//含义：4个灯状态分别为：开、关、开、关，modbus_crc16校验值为 20 68
33 01 00 01 00 20 68
```

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0

//frameLength: 每帧数据长度
//getFrameFlag: 是否找到帧头或帧尾标记
int frameLength=7,getFrameFlag,crcval2

bauds=115200 //波特率115200
recmod=1    //打开主动解析
page 0                          //上电刷新第0页
```

解析定时器（tim 为 50）中的代码如下图所示

```
while(usize>=frameLength&&getFrameFlag==0)
{
  crcval2=0
  ucopy crcval2,5,2,0
  prints crcval2,2
  crcrest 1,0xffff
  crcputu 0,5
  prints crcval,2
  if(u[0]==0x33&&crcval==crcval2)
  {
    //找到帧头,退出循环
    getFrameFlag=1
  }else
```

(续下页)

(接上页)

```
  {
    //如果帧头不对，就一直删除1个字节，直到不满足条件退出循环
    udelete 1
  }
}
if(getFrameFlag==1)
{
  n1.val=u[1]
  n2.val=u[2]
  n3.val=u[3]
  n4.val=u[4]
  udelete frameLength //删除已经解析过的数据
  getFrameFlag=0 //清空标记变量
}
```

8.2.2.10.1 定长 hex-示例 10-样例工程下载

演示工程下载链接：

《定长 hex-示例 10》演示工程下载

8.2.2.11 例 11：有两种帧头为 0x47 和 0x4E，帧尾为 0x0d，0x0a，每帧长度 22 字节

注意： 每个页面仅建议在一个定时器中读取串口缓冲区的数据，在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

新建一个空白工程

在工程中新建一个定时器 tmDecode，tim 设置为 50，en 设置为 1，用于定时解析串口数据

新建文本字控件 msg 和 t0，用于显示解析出来的数据，变量控件 str0 设置为字符串类型，用于临时存储解析出来的数据

注意： 16 进制发送时，可以忽略大小写，ff 和 FF 是一样的，AA 和 aa 是一样的，但是请不要写成 Ff，Ab 这样大小写混用

```
//4E开头的指令
4E 20 20 20 20 20 20 20 20 20 30 2E 30 30 30 30 20 67 20 20 0D 0A

//47开头的指令
47 20 20 20 20 20 20 20 20 20 30 2E 30 30 30 30 20 67 20 20 0D 0A
```

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0
//frameLength: 每帧数据长度
//getFrameFlag: 是否找到帧头或帧尾标记
int frameLength=22,getFrameFlag
bauds=115200 //波特率115200
recmod=1     //打开主动解析
page 0                          //上电刷新第0页
```

解析定时器（tim 为 50）中的代码如下图所示

```
while(usize>=frameLength&&getFrameFlag==0)
{
  if(u[0]==0x47&&u[20]==0x0d&&u[21]==0x0a)
  {
    //找到帧头0x47,退出循环
    getFrameFlag=1
  }else if(u[0]==0x4e&&u[20]==0x0d&&u[21]==0x0a)
  {
    //找到帧头0x4e,退出循环
    getFrameFlag=2
  }else
  {
    //如果帧头不对，就一直删除1个字节，直到不满足条件退出循环
    udelete 1
  }
}
if(getFrameFlag==1)
{
  t0.txt=""
  msg.txt="收到的是47开头的"
  for(sys0=0;sys0<=21;sys0++)
  {
```

(续下页)

(接上页)

```
    covx u[sys0],str0.txt,0,2
    t0.txt+=str0.txt
    t0.txt+=" "
  }
  udelete frameLength //删除已经解析过的数据
  getFrameFlag=0 //清空标记变量
}else if(getFrameFlag==2)
{
  t0.txt=""
  msg.txt="收到的是4E开头的"
  for(sys0=0;sys0<=21;sys0++)
  {
    covx u[sys0],str0.txt,0,2
    t0.txt+=str0.txt
    t0.txt+=" "
  }
  udelete frameLength //删除已经解析过的数据
  getFrameFlag=0 //清空标记变量
}
```

8.2.2.11.1 定长 hex-示例 11-样例工程下载

演示工程下载链接：

《定长 hex-示例 11》演示工程下载

8.2.2.12 例 101：帧头为 55 aa，定长 hex-点灯 1

注意： 每个页面仅建议在一个定时器中读取串口缓冲区的数据，在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

新建一个空白工程

在工程中新建一个定时器 tmDecode，tim 设置为 50，en 设置为 1，用于定时解析串口数据

图片控件 p0 和 p1 用来显示两个灯的状态

注意： 16 进制发送时，可以忽略大小写，ff 和 FF 是一样的，AA 和 aa 是一样的，但是请不要写成 Ff，Ab 这样大小写混用

```
55 aa 01 01 0d 0a  //左边图片变红（id为1的控件使用id为1的图片）

55 aa 02 00 0d 0a  //右边图片变黑（id为2的控件使用id为0的图片）

55 aa 01 02 0d 0a  //左边图片变黄（id为1的控件使用id为2的图片）

55 aa 02 03 0d 0a  //右边图片变蓝（id为2的控件使用id为3的图片）
```

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0
//frameLength: 每帧数据长度
//getFrameFlag: 是否找到帧头或帧尾标记
int frameLength=6,getFrameFlag
bauds=115200 //波特率115200
recmod=1    //打开主动解析
page 0                          //上电刷新第0页
```

解析定时器（tim 为 50）中的代码如下图所示

```
while(usize>=frameLength&&getFrameFlag==0)
{
  if(u[0]==0x55&&u[1]==0xaa&&u[4]==0x0d&&u[5]==0x0a)//包头为0x55aa，并且以0d0a结尾
  {
    //找到帧头,退出循环
    getFrameFlag=1
  }else
  {
    //如果帧头不对，就一直删除1个字节，直到不满足条件退出循环
    udelete 1
  }
}
if(getFrameFlag==1)
{
  b[u[2]].pic=u[3]
  //下面将串口缓冲区的各个数据写到屏幕的控件中方便观看
  sys2=0
  for(sys1=n0.id;sys1<=n5.id;sys1++)
  {
    b[sys1].val=u[sys2]
    sys2++
```

(续下页)

(接上页)

```
  }
  udelete frameLength //删除已经解析过的数据
  getFrameFlag=0 //清空标记变量
}
```

8.2.2.12.1 定长 hex-点灯 1-样例工程下载

演示工程下载链接：

《定长 hex-点灯 1》演示工程下载

8.2.2.13 例 102：帧头为 55 aa，定长 hex-点灯 2

> **注意：** 每个页面仅建议在一个定时器中读取串口缓冲区的数据, 在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

新建一个空白工程

在工程中新建一个定时器 tmDecode，tim 设置为 50，en 设置为 1，用于定时解析串口数据

图片控件 p0 和 p1 用来显示两个灯的状态

> **注意：** 16 进制发送时，可以忽略大小写，ff 和 FF 是一样的，AA 和 aa 是一样的，但是请不要写成 Ff，Ab 这样大小写混用

```
55 aa 01 01 02 00  //
↪左边图片变红，右边图片变黑（id为1的控件使用id为1的图片，id为2的控件使用id为0的图片）

55 aa 01 02 02 03  //
↪左边图片变黄，右边图片变蓝（id为1的控件使用id为2的图片，id为2的控件使用id为3的图片）

55 aa 01 01 02 01  //左边图片变红右边图片变红

55 aa 01 00 02 04  //左边图片变黑右边图片变绿
```

program.s 中的配置如图所示

解析定时器（tim 为 50）中的代码如下图所示

```
while(usize>=frameLength&&getFrameFlag==0)
{
  if(u[0]==0x55&&u[1]==0xaa)//包头为0x55aa
  {
    //找到帧头,退出循环
    getFrameFlag=1
  }else
  {
    //如果帧头不对，就一直删除1个字节，直到不满足条件退出循环
    udelete 1
  }
}
if(getFrameFlag==1)
{
  b[u[2]].pic=u[3]
  b[u[4]].pic=u[5]
  //下面将串口缓冲区的各个数据写到屏幕的控件中方便观看
  sys2=0
  for(sys1=n0.id;sys1<=n5.id;sys1++)
  {
    b[sys1].val=u[sys2]
    sys2++
  }
  udelete frameLength //删除已经解析过的数据
  getFrameFlag=0 //清空标记变量
}
```

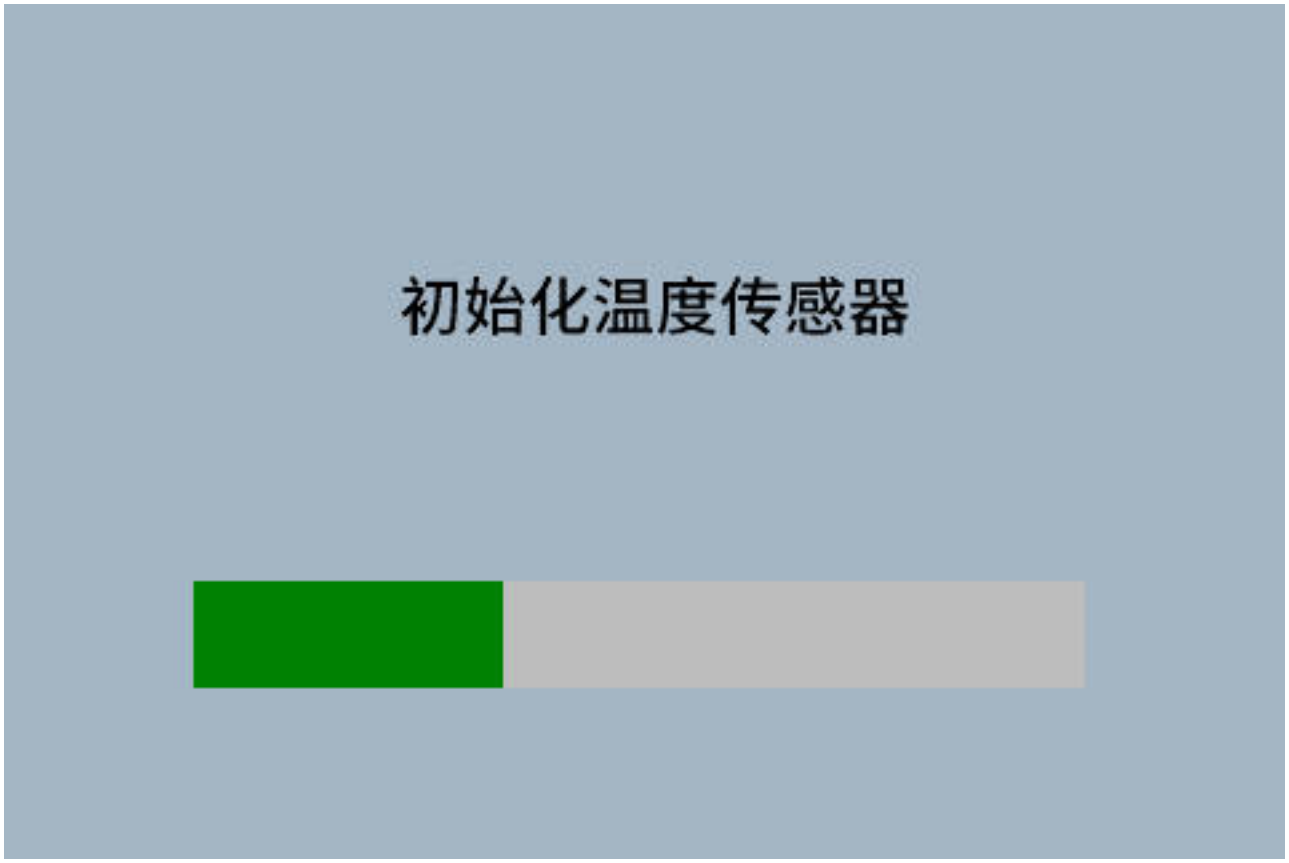
8.2.2.13.1 定长 hex-点灯 2-样例工程下载

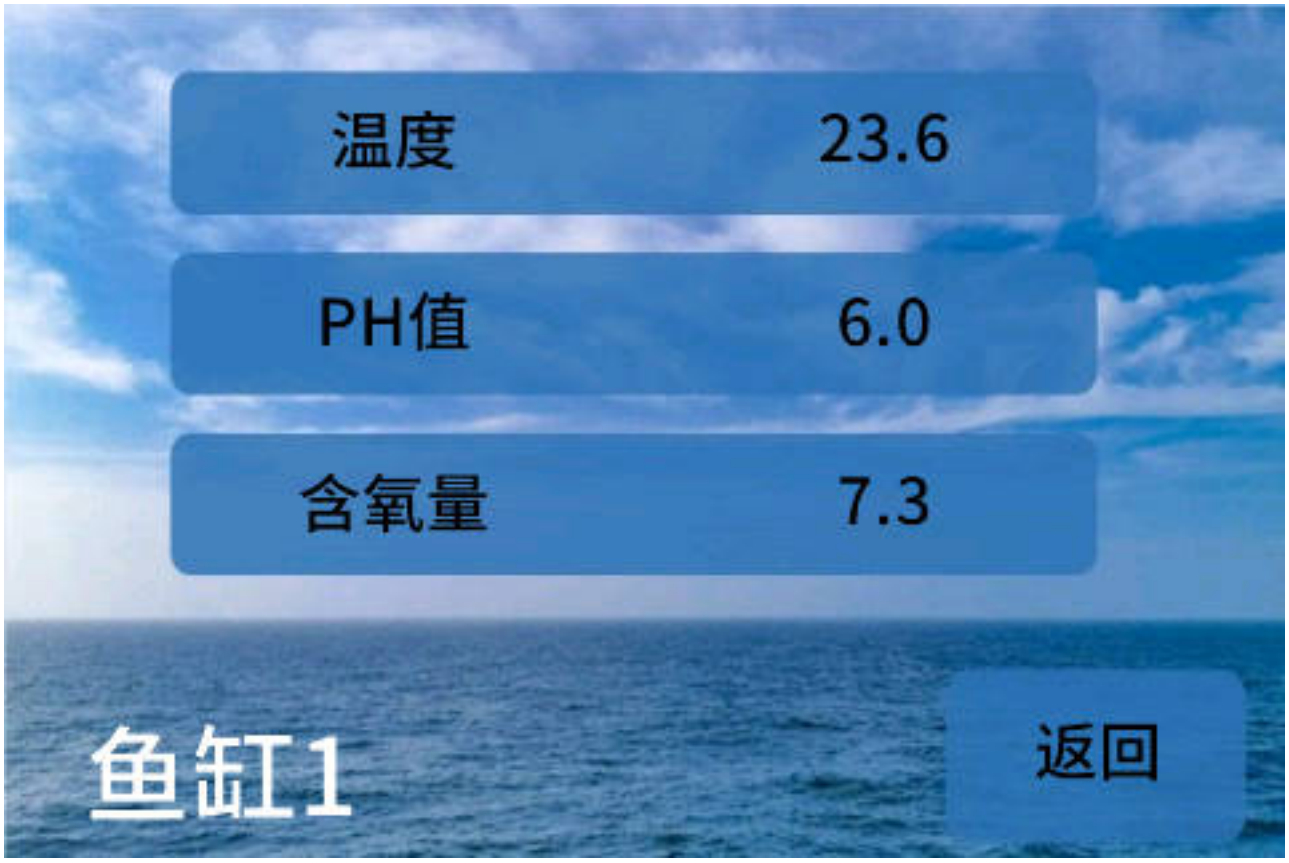
演示工程下载链接：

《定长 hex-点灯 2》演示工程下载

8.2.2.14 例 103：定长 hex-主动解析样例工程

本项目有两个工程，其中一个工程用于模拟单片机发出数据，另一个工程实现在多个页面中解析数据





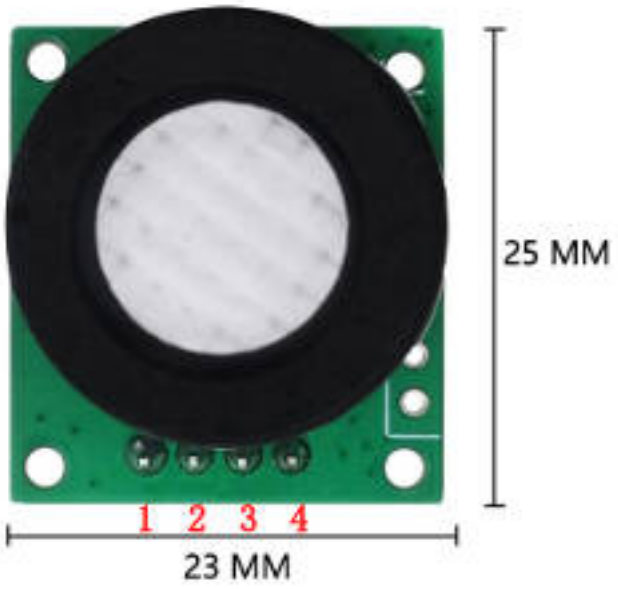
8.2.2.14.1 定长 hex-主动解析样例工程-样例工程下载

演示工程下载链接：

《定长 hex-主动解析样例工程》演示工程下载

8.2.2.15 例 104：电化学氧气模组 SC-03-O2-主动解析样例工程

模组尺寸



（板厚1.2MM 孔半径：1MM 连接器间距：2.54MM）

管脚定义

表2

管脚名称	管脚定义
Pin1	Vin（电压输入 4.2V～5.5V）
Pin2	UART（TXD） 0～3.3V 数据输出
Pin3/G排针	UART（RXD） 0～3.3V 数据输入
Pin4/V排针	GND

通讯协议

1 通用设置

表3

波特率	9600
数据位	8 位
停止位	1 位
校验位	无

2 通讯命令

发送频率，1秒1组数据，自动发送，格式如下

表4

B0	B1	B2	B3	B4	B5	B6	B7	B8
数据头	数据头	数据高位	数据低位	NC	NC	NC	NC	校验值
FF	86	00	D0	00	06	02	A0	03

气体浓度值=(气体高位*256+气体低位)*分辨率

校验值计算方式

校验值 (Byte8) = ~(Byte0+Byte1+Byte2+Byte3+Byte4+Byte5+Byte6+Byte7) + 1

8.2.2.15.1 电化学氧气模组 SC-03-O2-样例工程下载

演示工程下载链接：

《电化学氧气模组 SC-03-O2》演示工程下载

8.2.2.16 例 1000：某 bms 串口协议 1

表 7: 某 bms 串口协议 1

字节	说明	备注
Data0	0xaa	固定帧头
Data1	0x55	固定帧头
Data2	0xaa	固定帧头
Data3	0xff	固定帧头
Data4	总压高字节	0.1V/bit
Data5	总压低字节	
Data6	单体电压 0 高字节	1mV/bit
Data7	单体电压 0 低字节	
Data8	单体电压 1 高字节	1mV/bit
Data9	单体电压 1 低字节	
Data10	单体电压 2 高字节	1mV/bit
Data11	单体电压 2 低字节	
Data12	单体电压 3 高字节	1mV/bit
Data13	单体电压 3 低字节	
Data14	单体电压 4 高字节	1mV/bit
Data15	单体电压 4 低字节	
Data16	单体电压 5 高字节	1mV/bit
Data17	单体电压 5 低字节	
Data18	单体电压 6 高字节	1mV/bit
Data19	单体电压 6 低字节	
Data20	单体电压 7 高字节	1mV/bit
Data21	单体电压 7 低字节	
Data22	单体电压 8 高字节	1mV/bit
Data23	单体电压 8 低字节	
Data24	单体电压 9 高字节	1mV/bit
Data25	单体电压 9 低字节	
Data26	单体电压 10 高字节	1mV/bit
Data27	单体电压 10 低字节	
Data28	单体电压 11 高字节	1mV/bit
Data29	单体电压 11 低字节	
Data30	单体电压 12 高字节	1mV/bit
Data31	单体电压 12 低字节	
Data32	单体电压 13 高字节	1mV/bit
Data33	单体电压 13 低字节	
Data34	单体电压 14 高字节	1mV/bit
Data35	单体电压 14 低字节	
Data36	单体电压 15 高字节	1mV/bit
Data37	单体电压 15 低字节	
Data38	单体电压 16 高字节	1mV/bit
Data39	单体电压 16 低字节	
Data40	单体电压 17 高字节	1mV/bit
Data41	单体电压 17 低字节	
Data42	单体电压 18 高字节	1mV/bit
Data43	单体电压 18 低字节	
Data44	单体电压 19 高字节	1mV/bit
Data45	单体电压 19 低字节	
Data46	单体电压 20 高字节	1mV/bit
Data47	单体电压 20 低字节	
Data48	单体电压 21 高字节	1mV/bit
Data49	单体电压 21 低字节	

续下页

表 7 – 接上页

字节	说明	备注
Data50	单体电压 22 高字节	1mV/bit
Data51	单体电压 22 低字节	
Data52	单体电压 23 高字节	1mV/bit
Data53	单体电压 23 低字节	
Data54	单体电压 24 高字节	1mV/bit
Data55	单体电压 24 低字节	
Data56	单体电压 25 高字节	1mV/bit
Data57	单体电压 25 低字节	
Data58	单体电压 26 高字节	1mV/bit
Data59	单体电压 26 低字节	
Data60	单体电压 27 高字节	1mV/bit
Data61	单体电压 27 低字节	
Data62	单体电压 28 高字节	1mV/bit
Data63	单体电压 28 低字节	
Data64	单体电压 29 高字节	1mV/bit
Data65	单体电压 29 低字节	
Data66	单体电压 30 高字节	1mV/bit
Data67	单体电压 30 低字节	
Data68	单体电压 31 高字节	1mV/bit
Data69	单体电压 31 低字节	
Data70	reserved	保留参数
Data71	reserved	保留参数
Data72	电流高字节	0.1A/bit integer
Data73	电流低字节	
Data74	SOC	1%/bit
Data75	byte3	物理容量备注 5
Data76	byte2	
Data77	byte1	
Data78	byte0	
Data79	byte3	剩余容量备注 5
Data80	byte2	
Data81	byte1	
Data82	byte0	
Data83	byte3	循环容量备注 5
Data84	byte2	
Data85	byte1	
Data86	byte0	
Data87	byte3	系统时间 1s/bit
Data88	byte2	
Data89	byte1	
Data90	byte0	
Data91	MOS 温度高字节	1 ℃/bit integer
Data92	MOS 温度低字节	
Data93	均衡温度高字节	1 ℃/bit integer
Data94	均衡温度低字节	
Data95	外部温度 0 高字节	1 ℃/bit integer
Data96	外部温度 0 低字节	
Data97	外部温度 1 高字节	1 ℃/bit integer
Data98	外部温度 1 低字节	
Data99	外部温度 2 高字节	1 ℃/bit integer
Data100	外部温度 2 低字节	
Data101	外部温度 3 高字节	1 ℃/bit integer
Data102	外部温度 3 低字节	

续下页

表 7 – 接上页

字节	说明	备注
Data103	充电 MOS 状态	备注 1
Data104	放电 MOS 状态	备注 2
Data105	均衡状态	备注 3
Data106	reserved	保留参数
Data107	reserved	保留参数
Data108	reserved	保留参数
Data109	reserved	保留参数
Data110	reserved	保留参数
Data111	reserved	保留参数
Data112	reserved	保留参数
Data113	reserved	保留参数
Data114	reserved	保留参数
Data115	最高单体串数	
Data116	最高单体电压高字节	1mV/bit
Data117	最高单体电压低字节	
Data118	最低单体串数	
Data119	最低单体电压高字节	1mV/bit
Data120	最低单体电压低字节	
Data121	平均单体电压高字节	1mV/bit
Data122	平均单体电压低字节	
Data123	实际串数	
Data124	reserved	保留参数
Data125	reserved	保留参数
Data126	reserved	保留参数
Data127	reserved	保留参数
Data128	reserved	保留参数
Data129	reserved	保留参数
Data130	reserved	保留参数
Data131	reserved	保留参数
Data132	reserved	保留参数
Data133	reserved	保留参数
Data134	reserved	保留参数
Data135	reserved	保留参数
Data136	系统日志高字节	备注 6
Data137	系统日志低字节	
Data138	校验和高字节	备注 4
Data139	校验和低字节	

表 8: 备注 1-充电管状态,

数值	说明
0	关闭
1	打开
2	单体过压
3	过流
4	
5	总压过压
6	电池过温
7	功率过温
8	电流异常
9	均衡线掉串
10	主板过温
11	

续下页

表 8 – 接上页

数值	说明
12	开启失败
13	充电管异常
14	等待
15	手动关闭
16	二级过压
17	低温保护
18	压差保护
19	
20	
21	
22	总压单体异常

表 9: 备注 2-放电管状态

数值	说明
0	关闭
1	打开
2	单体欠压
3	过流
4	二级过流
5	总压欠压
6	电池过温
7	功率过温
8	电流异常
9	均衡线掉串
10	主板过温
11	
12	短路保护
13	放电管异常
14	开启失败
15	手动关闭
16	耳机欠压
17	低温保护
18	压差保护
19	
20	
21	
22	总压单体异常

表 10: 备注 3

数值	说明
0	关闭
1	均衡极限
2	压差均衡
3	均衡过温
4	自动均衡
10	主板过温

备注 4:

和校验计算方法:

定义一个 16bit 变量

u16Result=0

u16Result += Data[I], I 从 4 到 137. 如果 u16Result = Data[138]*256+Data[139]. 那么数据是完整的，否则数据是错误的。

备注 5:

对于 4 字节变量结果 = byte3 « 24 + byte2 « 16 + byte1 « 8 + byte0

容量的分辨率为 0.000001AH

表 11: 备注 6

数值	说明
bit0 - bit4	充电放电管状态
bit5 - bit9 位	电池编号
bit10 - bit14 位	先后顺序
bit15	充电放电，1 放电，0 充电

注意: 每个页面仅建议在一个定时器中读取串口缓冲区的数据, 在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

新建一个空白工程

在工程中新建一个定时器 tmDecode，tim 设置为 50，en 设置为 1，用于定时解析串口数据

注意: 16 进制发送时，可以忽略大小写，ff 和 FF 是一样的，AA 和 aa 是一样的，但是请不要写成 Ff，Ab 这样大小写混用

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0
//frameLength: 每帧数据长度
//getFrameFlag: 是否找到帧头或帧尾标记
int frameLength=140,getFrameFlag
int sumCheck=0,sumCheck2=0
bauds=19200 //波特率19200
recmod=1    //打开主动解析
page 0                          //上电刷新第0页
```

解析定时器（tim 为 50）中的代码如下图所示

```
while(usize>=frameLength&&getFrameFlag==0)
{
  //计算校验和
  sumCheck=0
  for(sys0=4;sys0<=137;sys0++)
  {
    sumCheck+=u[sys0]
  }
  sumCheck&=0xffff
  sumCheck2=u[138]<<8+u[139]
  //检查帧头和校验和
  if(u[0]==0xAA&&u[1]==0x55&&u[2]==0xAA&&u[3]==0xFF&&sumCheck==sumCheck2)
  {
    //找到帧头,退出循环
    getFrameFlag=1
  }else
```

(续下页)

(接上页)

```
  {
    //如果帧头不对，就一直删除1个字节，直到不满足条件退出循环
    udelete 1
  }
}
if(getFrameFlag==1)
{
  global.data4_5.val=u[4]<<8+u[5]
  global.data6_7.val=u[6]<<8+u[7]
  global.data8_9.val=u[8]<<8+u[9]
  global.data10_11.val=u[10]<<8+u[11]
  global.data12_13.val=u[12]<<8+u[13]
  global.data14_15.val=u[14]<<8+u[15]
  global.data16_17.val=u[16]<<8+u[17]
  global.data18_19.val=u[18]<<8+u[19]
  global.data20_21.val=u[20]<<8+u[21]
  global.data22_23.val=u[22]<<8+u[23]
  global.data24_25.val=u[24]<<8+u[25]
  global.data26_27.val=u[26]<<8+u[27]
  global.data28_29.val=u[28]<<8+u[29]
  global.data30_31.val=u[30]<<8+u[31]
  global.data32_33.val=u[32]<<8+u[33]
  global.data34_35.val=u[34]<<8+u[35]
  global.data36_37.val=u[36]<<8+u[37]
  global.data38_39.val=u[38]<<8+u[39]
  global.data40_41.val=u[40]<<8+u[41]
  global.data42_43.val=u[42]<<8+u[43]
  global.data44_45.val=u[44]<<8+u[45]
  global.data46_47.val=u[46]<<8+u[47]
  global.data48_49.val=u[48]<<8+u[49]
  global.data50_51.val=u[50]<<8+u[51]
  global.data52_53.val=u[52]<<8+u[53]
  global.data54_55.val=u[54]<<8+u[55]
  global.data56_57.val=u[56]<<8+u[57]
  global.data58_59.val=u[58]<<8+u[59]
  global.data60_61.val=u[60]<<8+u[61]
  global.data62_63.val=u[62]<<8+u[63]
  global.data64_65.val=u[64]<<8+u[65]
  global.data66_67.val=u[66]<<8+u[67]
  global.data68_69.val=u[68]<<8+u[69]
  global.data72_73.val=u[72]<<8+u[73]
  global.data74.val=u[74]
  global.data75_78.val=u[75]<<8+u[76]<<8+u[77]<<8+u[78]
  global.data79_82.val=u[79]<<8+u[80]<<8+u[81]<<8+u[82]
  global.data83_86.val=u[83]<<8+u[84]<<8+u[85]<<8+u[86]
  global.data87_90.val=u[87]<<8+u[88]<<8+u[89]<<8+u[90]
  global.data91_92.val=u[91]<<8+u[92]
  global.data93_94.val=u[93]<<8+u[94]
  global.data95_96.val=u[95]<<8+u[96]
  global.data97_98.val=u[97]<<8+u[98]
  global.data99_100.val=u[99]<<8+u[100]
  global.data101_102.val=u[101]<<8+u[102]
  global.data103.val=u[103]
  global.data104.val=u[104]
  global.data105.val=u[105]
  global.data115.val=u[115]
  global.data116_117.val=u[116]<<8+u[117]
  global.data118.val=u[118]
  global.data119_120.val=u[119]<<8+u[120]
  global.data121_122.val=u[121]<<8+u[122]
  global.data123.val=u[123]
```

(续下页)

(接上页)

```
  global.data136_137.val=u[136]<<8+u[137]
  udelete frameLength //删除已经解析过的数据
  getFrameFlag=0 //清空标记变量
}
```

8.2.2.16.1 定长 hex-某 bms 串口协议 1-样例工程下载

演示工程下载链接：

《定长 hex-某 bms 串口协议》演示工程下载

8.2.3 解析不定长 hex 格式指令-自定义协议

8.2.3.1 不定长-帧头为 0x55，帧尾为 3 个 0xff

注意： 每个页面仅建议在一个定时器中读取串口缓冲区的数据，在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

假设串口屏获取 4 个 led 灯的开关状态，串口屏需要解析的参数为灯的状态（开或关）

并且还要解析一个值为-32768 到 32767 范围的温度值

u[1] 为 AA 时代表每次传输灯的状态，每帧 9 字节

u[2] 为 AB 时代表传输当前温度值，每帧 7 字节

```
//含义：当前传输的是灯的数据，4个灯状态分别为：开、开、开、开
55 AA 01 01 01 01 ff ff ff

//含义：当前传输的是灯的数据，4个灯状态分别为：关、关、关、关
55 AA 00 00 00 00 ff ff ff

//含义：当前传输的是灯的数据，4个灯状态分别为：关、开、关、开
55 AA 00 01 00 01 ff ff ff

//含义：当前传输的是灯的数据，4个灯状态分别为：开、关、开、关
55 AA 01 00 01 00 ff ff ff

//含义：当前传输的是温度的数据(小端模式)，温度值是03E8，转换为10进制是1000
55 AB E8 03 FF FF FF

//含义：当前传输的是温度的数据(小端模式)，温度值是FFB0，转换为10进制是-80
55 AB B0 FF FF FF FF
```

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0

//最短一帧的数据
int shortestLength=7

int frameLength,getFrameFlag
bauds=115200 //波特率256000
recmod=1     //打开主动解析
page 0                          //上电刷新第0页
```

解析定时器（tim 为 50）中的代码如下图所示

```
while(usize>=shortestLength&&getFrameFlag==0)
{
  if(usize>=9&&u[0]==0x55&&u[1]==0xAA&&u[6]==0xFF&&u[7]==0xFF&&u[8]==0xFF)
  {
    //找到帧头,退出循环
    getFrameFlag=1
    frameLength=9
  }else if(usize>=7&&u[0]==0x55&&u[1]==0xAB&&u[4]==0xFF&&u[5]==0xFF&&u[6]==0xFF)
  {
    //找到帧头,退出循环
```

(续下页)

(接上页)

```
    getFrameFlag=2
    frameLength=7
  }else
  {
    //如果帧头不对，就一直删除1个字节，直到不满足条件退出循环
    udelete 1
  }
}
if(getFrameFlag!=0)
{
  if(getFrameFlag==1)
  {
    n1.val=u[2]
    n2.val=u[3]
    n3.val=u[4]
    n4.val=u[5]
  }else if(getFrameFlag==2)
  {
    sys0=0  //必须清零，否则否则高字节有数据时将会错误
    ucopy sys0,2,2,0
    //判断温度值的正负，大于32767为负数，通过负数补码获取原码
    if(sys0>32767)
    {
      sys0-=65536
    }
    x0.val=sys0 //x0是一个虚拟浮点数控件
  }
  udelete frameLength //删除已经解析过的数据
  getFrameFlag=0 //清空标记变量
}
```

8.2.3.1.1 不定长 hex-示例 1-样例工程下载

演示工程下载链接：

《不定长 hex-示例 1》演示工程下载

8.2.3.2 帧头为 0xFF ，帧尾为 0d 0a

注意: 每个页面仅建议在一个定时器中读取串口缓冲区的数据, 在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

u[1] 为 AA 时代表每次传输灯的状态，每帧 9 字节

u[1] 为 AB 时代表传输当前温度值，每帧 7 字节

u[2] 代表这帧的长度

```
//含义：当前传输的是灯的数据，9字节长度，4个灯状态分别为：开、开、开、开
FF AA 09 01 01 01 01 0D 0A

//含义：当前传输的是灯的数据，9字节长度，4个灯状态分别为：关、关、关、关
FF AA 09 00 00 00 00 0D 0A

//含义：当前传输的是灯的数据，9字节长度，4个灯状态分别为：关、开、关、开
FF AA 09 00 01 00 01 0D 0A

//含义：当前传输的是灯的数据，9字节长度，4个灯状态分别为：开、关、开、关
FF AA 09 01 00 01 00 0D 0A

//含义：当前传输的是温度的数据(小端模式)，7字节长度，温度值是03E8，转换为10进制是1000
FF AB 07 E8 03 0D 0A

//含义：当前传输的是温度的数据(小端模式)，7字节长度，温度值是FFB0，转换为10进制是-80
FF AB 07 B0 FF 0D 0A
```

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0

//最短一帧的数据
int shortestLength=7

int frameLength,getFrameFlag
bauds=115200 //波特率256000
recmod=1    //打开主动解析
page 0                          //上电刷新第0页
```

解析定时器（tim 为 50）中的代码如下图所示

```
while(usize>=shortestLength&&getFrameFlag==0)
{
  frameLength=u[2]
  if(usize>=frameLength&&u[0]==0xff&&u[frameLength-2]==0x0d&&u[frameLength-1]==0x0a)
  {
    //找到帧头,退出循环
    getFrameFlag=1
    printh ae
  }else
  {
    //如果帧头不对，就一直删除1个字节，直到不满足条件退出循环
    udelete 1
    printh af
  }
```

(续下页)

(接上页)

```
}
if(getFrameFlag!=0)
{
  if(u[1]==0xAA)
  {
    n1.val=u[3]
    n2.val=u[4]
    n3.val=u[5]
    n4.val=u[6]
  }else if(u[1]==0xAB)
  {
    sys0=0  //必须清零，否则否则高字节有数据时将会错误
    ucopy sys0,3,2,0
    //判断温度值的正负，大于32767为负数，通过负数补码获取原码
    if(sys0>32767)
    {
      sys0-=65536
    }
    x0.val=sys0
  }
  udelete frameLength //删除已经解析过的数据
  getFrameFlag=0 //清空标记变量
}
```

8.2.3.2.1 不定长 hex-示例 2-样例工程下载

演示工程下载链接：

《不定长 hex-示例 2》演示工程下载

8.2.3.3 传输不定长的字符串

注意: 每个页面仅建议在一个定时器中读取串口缓冲区的数据, 在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

帧头为 0xFF

u[1] 为 00 时代表传输给 t0，u[1] 为 01 时代表传输给 t1

u[2] 代表字符串的长度

```
//含义：给t0文本赋值为“淘晶驰电子”，文本长度10，1汉字占2字节
FF 00 0A CC D4 BE A7 B3 DB B5 E7 D7 D3

//含义：给t1文本赋值为“TJC”，文本长度3
FF 01 03 54 4A 43
```

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0

//frameLength: 每帧数据长度
//getFrameFlag: 是否找到帧头或帧尾标记
int frameLength,getFrameFlag,strLength

//当字符串长度为0时，最短的帧长度为3
int shortestLength=3

bauds=115200 //波特率115200
recmod=1     //打开主动解析
page 0                          //上电刷新第0页
```

解析定时器（tim 为 50）中的代码如下图所示

```
while(usize>=shortestLength&&getFrameFlag==0)
{
  strLength=u[2]
  frameLength=strLength+3
  if(u[0]==0xFF&&usize>=frameLength)
  {
    //找到帧头,退出循环
    getFrameFlag=1
  }else
  {
    //如果帧头不对，就一直删除1个字节，直到不满足条件退出循环
    udelete 1
  }
}
if(getFrameFlag==1)
{
  if(u[1]==0x00)
  {
    ucopy t0.txt,3,strLength,0
  }else if(u[1]==0x01)
  {
    ucopy t1.txt,3,strLength,0
  }
```

(续下页)

(接上页)

```
  udelete frameLength //删除已经解析过的数据
  getFrameFlag=0 //清空标记变量
}
```

8.2.3.3.1 不定长 hex-示例 3-样例工程下载

演示工程下载链接：

《不定长 hex-示例 3》演示工程下载

8.2.3.4 解析不定长 hex 协议-海陵科 LD-2410 生命存在感应模组

注意: 每个页面仅建议在一个定时器中读取串口缓冲区的数据, 在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

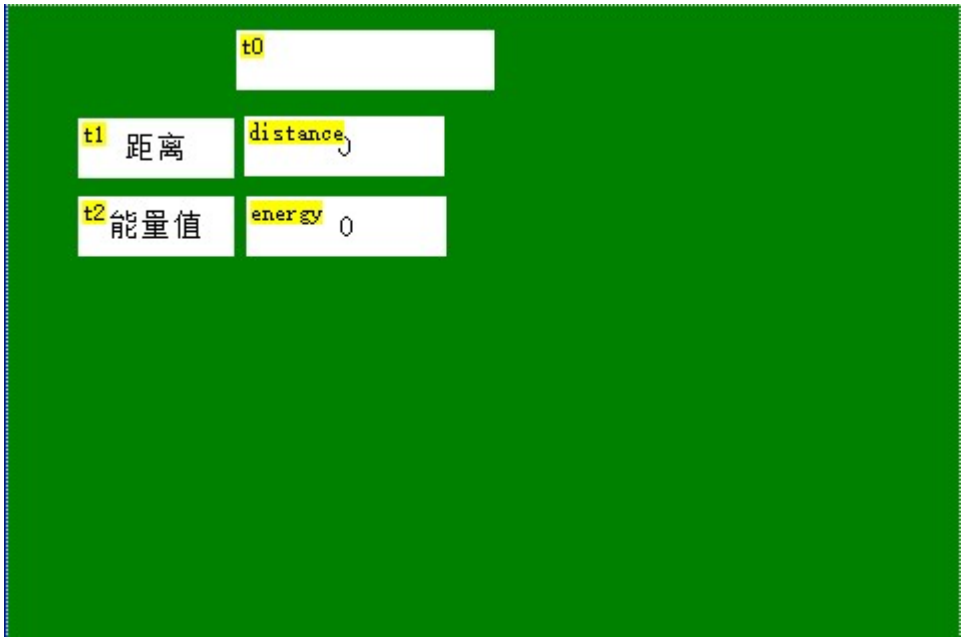
海陵科 LD-2410 生命存在感应模组的协议如下所示

帧头部	帧内数据长度	帧内数据	帧尾部
F4 F3 F2 F1	0D 00	02 AA 02 51 00 00 00 00 3B 00 00 55 00	F8 F7 F6 F5

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0

int length,totalLength,getFrameHead

//波特率256000
bauds=256000

//打开主动解析
recmod=1
page 0                          //上电刷新第0页
```

界面布局如下图所示



解析定时器（tim 为 50）中的代码如下图所示

```
while(usize>=10&&getFrameHead==0)
{
  if(u[0]==0xF4&&u[1]==0xF3&&u[2]==0xF2&&u[3]==0xF1)
  {
    //找到帧头,退出循环
    getFrameHead=1
    length=0
    ucopy length,4,2,0
    totalLength=length+10
  }else
  {
    //如果帧头不对，就一直删除1个字节，直到不满足条件退出循环
    udelete 1
  }
}
if(getFrameHead==1)
{
  if(usize>=totalLength)
  {
    if(u[totalLength-4]==0xF8&&u[totalLength-3]==0xF7&&u[totalLength-2]==0xF6&&
↪u[totalLength-1]==0xF5)
    {
      if(u[6]==0x02&&u[7]==0xAA&&u[17]==0x55&&u[18]==0x00)
      {
        if(u[8]==0x00)
        {
          t0.txt="无目标"
        }else if(u[8]==0x01)
        {
          t0.txt="运动目标"
          ucopy distance.val,9,2,0
          ucopy energy.val,11,1,0
        }else if(u[8]==0x02)
        {
          t0.txt="静止目标"
          ucopy distance.val,12,2,0
          ucopy energy.val,14,1,0
        }else if(u[8]==0x03)
        {
          t0.txt="运动&静止目标"
          ucopy distance.val,15,2,0
        }
      }
      udelete totalLength
    }else
    {
      udelete 4
    }
    getFrameHead=0
  }
}
```

下面对代码进行逐行讲解

program.s 中定义的变量

length：获取帧内数据长度

totalLength：通过 length 计算出一整帧的数据长度

getFrameHead：是否获取到帧头的标志位，1 为获取到帧头，0 为未获取到帧头

第 1 行为什么是判断 usize>=10

帧头部	帧内数据长度	帧内数据	帧尾部
F4 F3 F2 F1	~~0D 00~~ 00 00	~~02 AA 02 51 00 00 00 00 3B 00 00 55 00~~	F8 F7 F6 F5

当帧内数据长度为 00 00 时，理论上这是长度是最短的一帧, 此时数据变成了

F4 F3 F2 F1 00 00 F8 F7 F6 F5

此时共有 10 个字节，因此要求进入解析时应该最少大于等于最短的一帧数据

第 3-14 行是判断帧头是否符合协议中的要求，如果不是，则删除串口缓冲区最前面 1 字节，直到找到帧头或者 usize 小于 10

如何退出循环：当 getFrameHead!=0 或者 usize<10, 不满足 while 的条件，就会退出循环

第 16 行，判断是否获取到帧头，获取到了才会进入解析

第 18 行，判断当前串口缓冲区的数据是否大于完整 1 帧的长度

第 20 行，判断帧尾是否符合

第 22 行，判断 02、0A、55、00 这四个标志位是否符合

数据类型	头部	目标数据	尾部	校验
1字节(见表10)	0xAA	见表11，表13	0x55	0x00

第 24 行，通过第 8 位判断目标状态，随后将数据一一提取出来放入对应的控件中

目标状态	运动目标距离（厘米）	运动目标能量值	静止目标距离（厘米）	静止目标能量值	探测距离（厘米）
1字节（见表12）	2字节	1字节	2字节	1字节	2字节

表 12 目标状态值说明

目标状态值	说明
0x00	无目标
0x01	运动目标
0x02	静止目标
0x03	运动&静止目标

第 43 行，解析完毕，删除 1 帧的数据长度

第 46 行，帧头符合但是其他数据不符合，删除串口缓冲区前 4 字节（之前已经确认过是帧头），等待进入下一轮判断

第 48 行，解析完毕或者帧头已经被删除，将获取帧头标志清除

8.2.3.4.1 海陵科生命存在感应模组 hex 协议解析下载

演示工程下载链接：

《海陵科生命存在感应模组 hex 协议解析》演示工程下载

8.2.4 解析字符串格式指令

注意： 每个页面仅建议在一个定时器中读取串口缓冲区的数据，在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

8.2.4.1 解析 AT 指令

注意： 每个页面仅建议在一个定时器中读取串口缓冲区的数据，在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

首先我们自定义了几个指令给屏幕赋值

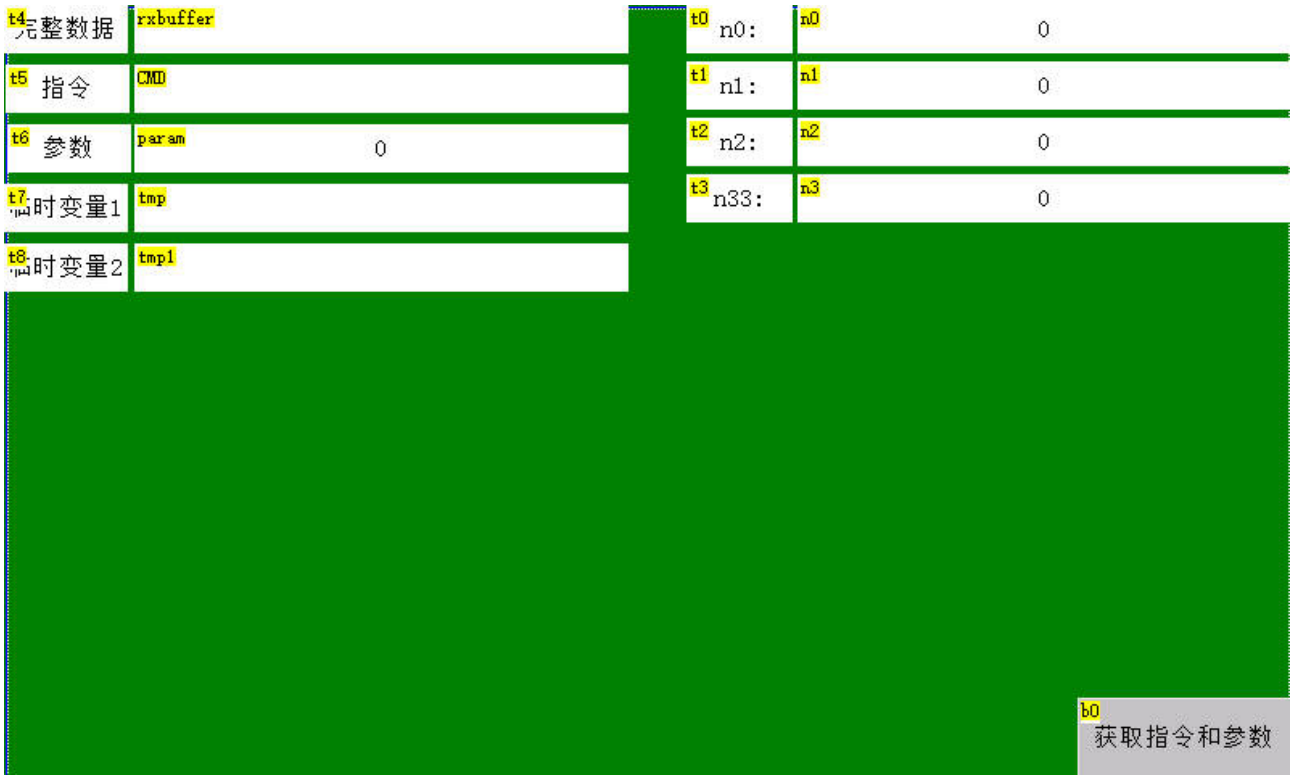
```
AT+n0=100\r\n    //给n0赋值100
AT+n1=50\r\n    //给n1赋值50
AT+n2=-30\r\n    //给n2赋值-30
AT+n3333=9999\r\n    //给n3333赋值9999
```

控件是固定的 n0，n1，n2，n3333，但是给他们赋值的参数是不固定的

program.s 中的配置如图所示

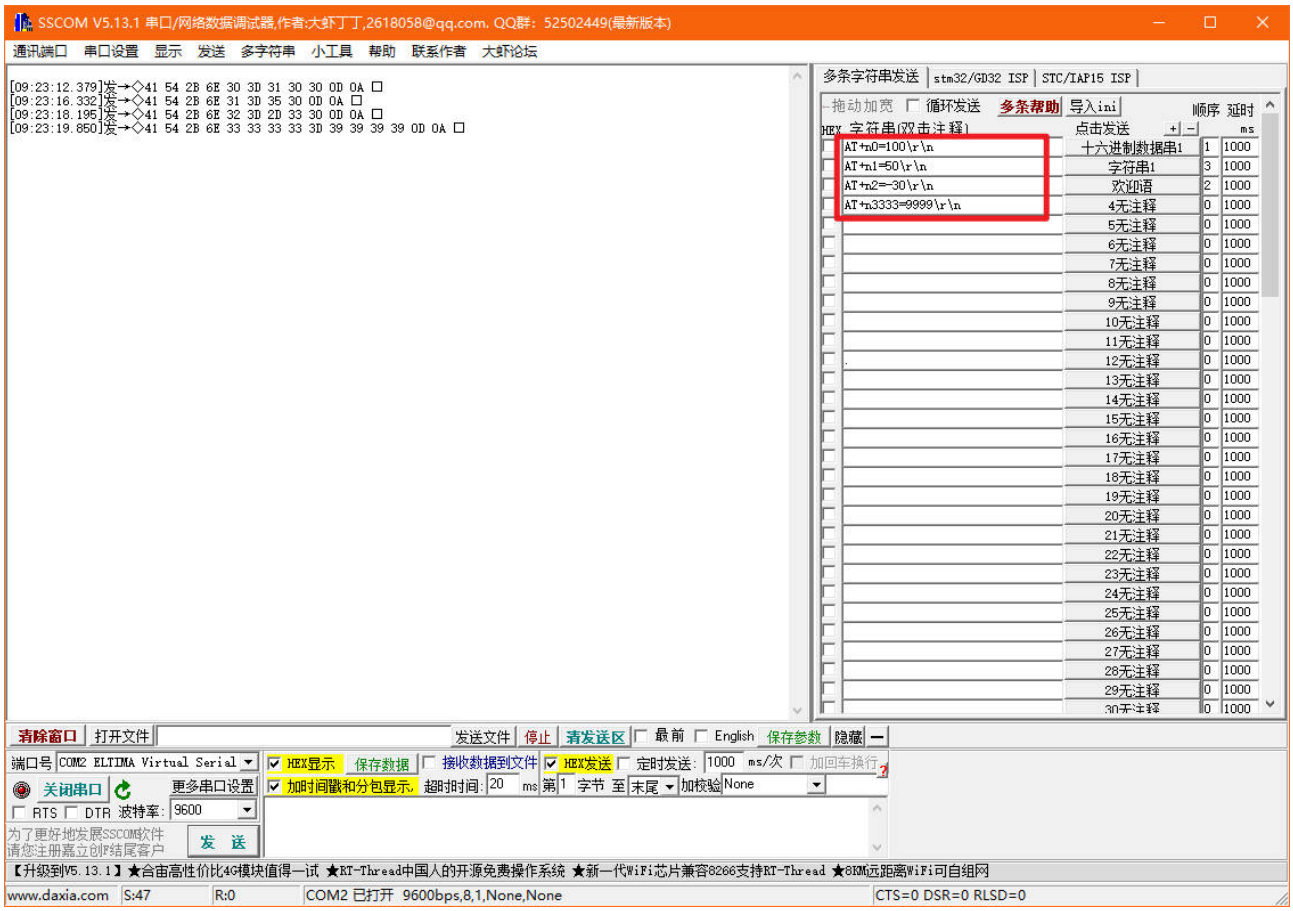
```
//以下代码只在上电时运行一次，一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int)，不支持其他类型的全局变量声明，
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0
int length,totalLength,getFrameHead,getFrameEnd
bauds=115200 //波特率115200
recmod=1     //打开主动解析
page 0                          //上电刷新第0页
```

界面布局如下图所示



注意： 注意文本控件的 txt_maxl 属性，目前我都是设置为 30

需要搭配 sscom5.13.1 使用



解析定时器（tim 为 50）中的代码如下图所示

```
while(usize>=2&&getFrameHead==0)
{
  if(u[0]==0x41&&u[1]==0x54)
  {
    //找到帧头"AT",退出循环
    getFrameHead=1
    getFrameEnd=0
  }else
  {
    //如果帧头不对，就一直删除1个字节，直到不满足条件退出循环
    udelete 1
  }
}
//查找帧尾\r\n
if(getFrameHead==1&&getFrameEnd==0)
{
  for(sys0=2;sys0<=usize;sys0++)
  {
    if(u[sys0]==0x0d&&u[sys0+1]==0x0a)
    {
      getFrameEnd=1
      length=sys0
      totalLength=sys0+2
      sys0=usize+1
    }
  }
}
//AT开头，\r\n结尾
if(getFrameHead==1&&getFrameEnd==1)
{
  getFrameHead=0
  getFrameEnd=0
  ucopy rxbuffer.txt,0,length,0
  udelete totalLength
  click b0,1
  if(CMD.txt=="n0")
  {
    n0.val=param.val
  }else if(CMD.txt=="n1")
  {
    n1.val=param.val
  }else if(CMD.txt=="n2")
  {
    n2.val=param.val
  }else if(CMD.txt=="n3333")
  {
    n3.val=param.val
  }
}
```

下面对代码进行逐行讲解

program.s 中定义的变量

length：获取帧内数据长度

totalLength：通过 length+ 结束符\r\n 的长度

getFrameHead：是否获取到帧头的标志位，1 为获取到帧头，0 为未获取到帧头

getFrameEnd：是否获取到帧尾的标志位，1 为获取到帧尾，0 为未获取到帧尾

第 1 行为什么是判断 usize>=2

代码中取“AT”作为帧头的判断标志，只要大于 2 字节，就有可能是 AT 这两个字母，当然也可以取“AT+”作为帧头的判断标志，那么第 1 行就是是判断 usize>=3

第 3-12 行是判断帧头是否符合协议中的要求，如果不是，则删除串口缓冲区最前面 1 字节，直到找到帧头或者 usize 小于 2

如何退出循环：当 getFrameHead!=0 或者 usize<2, 不满足 while 的条件，就会退出循环

第 2 行 0x41 是“A”对应的 ASCII 码表的 16 进制，0x54 是“T”对应的 ASCII 码表的 16 进制

第 17 行，从 u[2] 开始查找帧尾\r\n，\r 对应的 ASCII 码表的 16 进制是 0x0d，\n 对应的 ASCII 码表的 16 进制是 0x0a

第 22 行，当查找到\r\n 时，此时的 sys0 刚好就是除去\r\n 的数据长度，

第 23 行，sys0+2 则是加上\r\n 的总的数据的长度

第 24 行，让 sys0>usize, 才能退出循环

第 29 行，已经找到了帧头和帧尾，可以开始解析了

第 33 行，使用 ucopy 指令将除了\r\n 之外的数据拷贝至 rxbuffer.txt 进行解析

第 34 行，删除已经拷贝过的数据

第 35 行，在 b0 按钮中对数据进行解析，使用 click 去触发 b0 按钮，相当于调用了函数，具体如何解析请查看 b0 按钮

8.2.4.1.1 解析 AT 指令-样例工程下载

演示工程下载链接：

《AT 指令解析》演示工程下载

8.2.4.2 接收 json 数据字符串

注意: 每个页面仅建议在一个定时器中读取串口缓冲区的数据, 在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

以下面的 json 为例

```
{
    "name": "test",
    "info": {
        "age": 16,
        "pass": false
    }
}
```

json 数据中，左括号和右括号是成对出现的，因此我们可以通过这个特性，将第一个左括号当做帧头，最后一个右括号当做帧尾

新建一个空白工程

在工程中新建一个定时器 tmDecode，tim 设置为 50，en 设置为 1，用于定时解析串口数据

新建数字控件 n1,n2,n3,n4，用于显示解析出来的数据

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0
int jsonIndex=1
//frameLength: 每帧数据长度
//getFullJson: 是否找到完整的json
//getFrameFlag: 帧头的数量
//jsonLength: json数据实际长度
int frameLength=2,getFullJson=0,getFrameFlag,jsonLength
//breakFlag: 防止解析卡死在while中
int breakFlag
//键和值是否存在
int isKeyExist,isValueExist
bauds=115200 //波特率115200
recmod=1    //打开主动解析
page 0                          //上电刷新第0页
```

解析定时器（tim 为 50）中的代码如下图所示

```
breakFlag=0
while(usize>=frameLength&&getFullJson==0&&breakFlag==0)
{
  if(u[0]!=0x7B)
  {
    //如果帧头不对，就一直删除1个字节，直到不满足条件退出循环
    //0x7B是左括号{对应的ascii码值
    udelete 1
  }else
  {
    getFrameFlag=0
    for(sys0=0;sys0<usize;sys0++)
    {
      if(u[sys0]==0x7B)
      {
```

(续下页)

(接上页)

```
        //找到左括号，帧头数量自增
        getFrameFlag++
      }else if(u[sys0]==0x7D)
      {
        //找到右括号，帧头数量自减
        getFrameFlag--
      }
      //当getFrameFlag==0时，说明左右括号数量相等，接收到了完整数据
      if(getFrameFlag==0)
      {
        getFullJson=1
        jsonLength=sys0
        //让for循环不满足条件退出
        sys0=usize+1
      }
    }
    //已经解析过整个数据，退出while循环
    breakFlag=1
  }
}
if(getFullJson==1)
{
  //拷贝指定长度到myJson.txt
  ucopy myJson.txt,0,jsonLength,0
  //删除指定长度的串口缓冲区
  udelete jsonLength
  getFullJson=0
}
```

以上代码仅演示如何接收 json。代码本身的健壮性存在问题，请勿直接使用

建议参考以下方法发送和接收 json 字符串，可以降低接收 json 字符串的难度传输不定长的字符串 (page 993)

8.2.4.2.1 json 解析-样例工程下载

演示工程下载链接：

《json 解析》演示工程下载

8.2.4.3 解析不定长字符串 (以回车换行结尾)

注意: 每个页面仅建议在一个定时器中读取串口缓冲区的数据, 在多个定时器中读取串口缓冲区的数据容易照成逻辑混乱。

注意: 字符串需要以 0d 0a 结尾

新建一个空白工程

在工程中新建一个定时器 tmDecode，tim 设置为 50，en 设置为 1，用于定时解析串口数据

新建数字控件 t0，txt_maxl 设置为 200，用于显示解析出来的数据

program.s 中的配置如图所示

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
//全局变量定义目前仅支持4字节有符号整形(int),不支持其他类型的全局变量声明,
↪如需使用字符串类型可以在页面中使用变量控件来实现
int sys0=0,sys1=0,sys2=0
int usize2,length,totalLength
//波特率115200
bauds=115200
//亮度100
dim=100
//打开主动解析
recmod=1
page 0                          //上电刷新第0页
```

解析定时器（tim 为 50）中的代码如下图所示

```
if(usize>=2)
{
  usize2=usize-1
  for(length=0;length<usize2;length++)
  {
    if(u[length]==0x0d&&u[length+1]==0x0a)
    {
      totalLength=length+2
      ucopy t0.txt,0,length,0
      udelete totalLength
    }
  }
}
```

8.2.4.3.1 解析不定长字符串 (以回车换行结尾)-样例工程下载

演示工程下载链接:

《解析不定长字符串以回车换行结尾》演示工程下载

8.3 HMI 下载协议详解/OTA 升级

关键词：远程升级，通过单片机下载，OTA 升级，云端升级

注意： 使用本协议进行 HMI 下载时，在串口屏工程中请务必使用 bauds 来进行波特率配置，避免升级失败后导致波特率改变导致无法联机，参考*baud 和 bauds 的区别* (page 1331)

本文讲述的 HMI 下载协议仅适用于希望自己制作下载程序或者希望单片机去控制 HMI 下载资源文件的用户，属于高级应用范畴，不属于 HMI 界面设计的范畴，因此需要有一定基础的用户才能操作。

深圳市淘晶驰电子有限公司仅仅只对此协议做一个公布说明，不提供任何跟下载协议有关的技术支持。

如果对串口操作不熟悉的朋友建议忽略此说明，请直接使用 USART HMI 软件进行下载即可，无需对此协议有任何了解*通过串口下载工程到串口屏* (page 144) / *通过 SD 卡下载工程到串口屏* (page 155) 。

以下部分为下载协议的介绍。

下载情况分为 2 种：

8.3.1 未知串口号和波特率

在 windwos，linux，macos 等通用操作系统上自己编写下载工具，通常使用的是 USB 转 ttl 工具，需要通过【*下载步骤 1：联机操作*】 (page 1006) 先搜索 HMI 设备在哪个串口上，以及串口屏设备当前的波特率，然后通过【*下载步骤 2：开始下载*】 (page 1008) ，即可开始下载

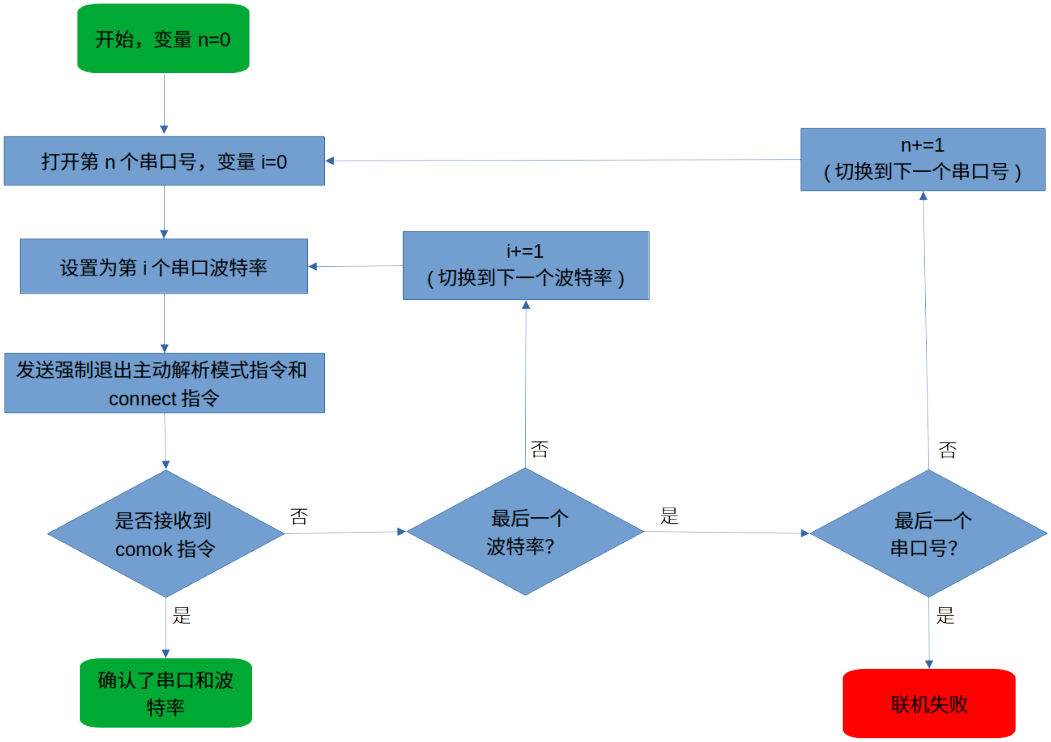
8.3.2 已知串口号和波特率

使用单片机，已经知道串口和通讯波特率的情况下，直接跳到【*下载步骤 2：开始下载*】 (page 1008) ，即可开始下载。如果当前屏幕在主动解析模式下，请先退出主动解析模式。

8.3.3 【下载步骤 1：联机操作】

注意： 此步骤主要用来搜索 HMI 设备在哪个串口上，以及设备当前的波特率。如果这两个条件是已知的，那么可以不用做这个步骤，在你的程序中直接固定串口号和设备当前使用的波特率后直接跳到步骤 2 开始下载。

8.3.3.1 【下载步骤 1：流程图】



● 搜索方法：

8.3.3.2 如果未开启主动解析模式

分别向电脑的每个串口分别用不同波特率发送一个联机指令:connect+ 结束符

其 hex 格式如下所示：00 FF FF FF 63 6F 6E 6E 65 63 74 FF FF FF

8.3.3.3 如果开启了主动解析模式

需要向串口屏发送一串数据用于强制退出主动解析模式, 紧接着发送 connect 和结束符

如何退出主动解析 (page 951)

其 hex 格式如下所示：44 52 41 4B 4A 48 53 55 59 44 47 42 4E 43 4A 48 47 4A 4B 53 48 42 44 4E FF FF FF 00 FF FF FF 63 6F 6E 6E 65 63 74 FF FF FF

8.3.3.4 联机数据说明

设备收到联机指令后会返回联机数据，如果收到正确的联机数据，说明设备联机成功，至此，得到当前设备的串口号和当前使用的波特率.

● 联机指令发送说明：

因为一直在循环发送指令，所以当屏幕在正确的波特率上收到数据时，数据的最前面肯定会有部分上一次错误的波特率下的错误数据，因此这个时候第一条指令肯定是会被当成错误指令的。所以每次发送的时候需要发两条指令，第一条发 4 个字节的 HEX 空指令（00 ff ff ff）, 第二条才是 connect+ 结束符

延时说明: 每次尝试一次联机指令后需要等待数据返回的最短时间为 (单位:ms): (1000000/尝试的波特率)+30

假如在 9600 波特率下尝试联机，需要等待返回的最短时间为:

1000000/9600+30=134ms

其他波特率以此类推

● 数据解释：

以 TJC4024T032_011R 设备为例，设备返回如下 8 组数据 (每组数据逗号隔开):

comok 1,101-0,TJC4024T032_011R,52,61488,D264B8204F0E1828,16777216\xff\xff\xff

comok: 握手回应

1: 表示带触摸 (0 是不带触摸)

101-0: 设备内部预留数据-设备地址

TJC4024T032_011R: 设备型号

52: 设备固件版本号

61488: 设备主控芯片内部编码

D264B8204F0E1828: 设备唯一序列号

16777216: 设备 FLASH 大小 (单位：字节)

\xff\xff\xff: 结束符

8.3.4 【下载步骤 2：开始下载】

此时已经知道设备在哪个串口号上，也知道设备当前的波特率了，可以发送下载指令了。

■ 第一步：发送指令 whmi-wri filesize,baud,res0\xff\xff\xff

filesize:tft 文件的大小 (单位：字节)

baud: 强制下载使用的波特率

res0: 预留数据，使用任意 ASCII 字符即可

假如需要下载的 tft 文件大小为 10000 字节, 需要使用 115200 波特率下载，那么就发送指令：

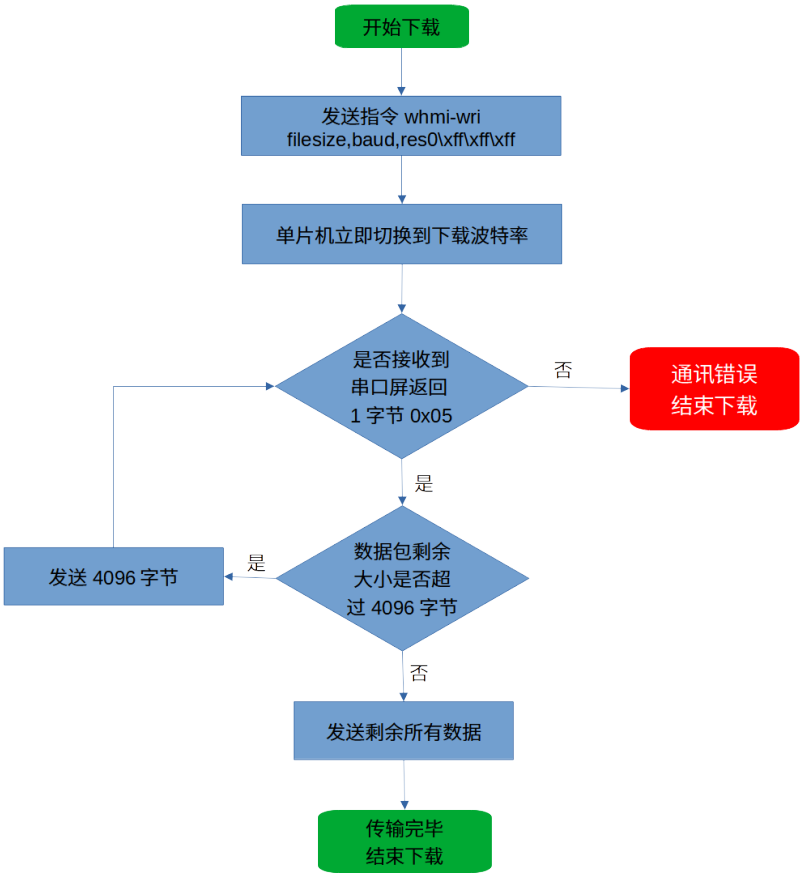
whmi-wri 10000,115200,0\xff\xff\xff

发送完此指令以后，需要立即修改电脑（或单片机）的波特率为刚才设置的强制波特率（如果当前波特率和强制下载波特率不一致的话）

■ 第二步：下发 tft 文件的二进制数据

设备收到 whmi-wri 指令后在 250ms 左右后会返回一个 0x05 的数据（仅仅是一个字节，没有 3 个 0XFF 的结束符，波特率为刚才设置的强制下载波特率），收到此数据后，可以开始下发 tft 文件的二进制数据，下发格式为每包下发 4096 字节，最后一包剩余多少就发多少，每包发送完成以后，需要等待屏幕返回响应信号，响应信号依然为一个单一字节的 0x05。

8.3.4.1 【下载步骤 2：流程图】



相关链接

TFT 文件如何下载到串口屏中 (page 1405)

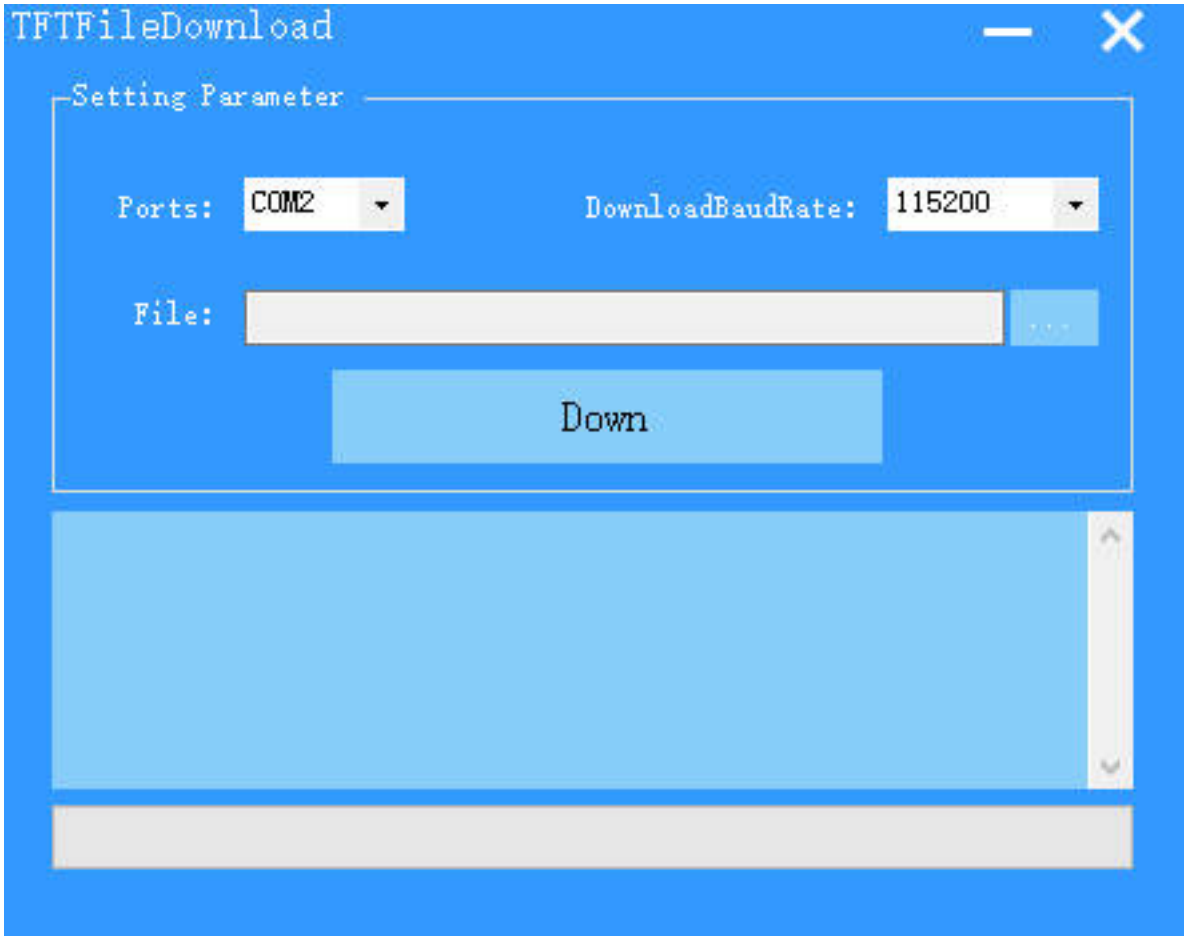
如何输出 TFT 生产文件 (page 155)

8.3.5 开源 TFT 下载工具下载

8.3.5.1 开源 TFT 下载工具-代码示例

8.3.5.1.1 c# 实现的下载工具

官方有按照本协议开发好的专用下载工具 (TFTFileDownload)，并且开放源代码 (c#，visual studio 2008)，欢迎下载使用



说明：此工具专用于 tft 文件的串口下载，没有其他功能。

使用方法：*如何使用 TFT 文件下载助手 (TFTFileDownload) 下载工程* (page 1435)

TFTFileDownload 软件下载

TFTFileDownload 工具是淘晶驰开源的下载工具，您可以下载源码进行二次开发，此工程基于 visual studio 进行开发，使用的语言为 c#，需要您熟练掌握 c# 语言，淘晶驰不对 TFTFileDownload 二次开发提供技术支持

TFTFileDownload 源代码下载

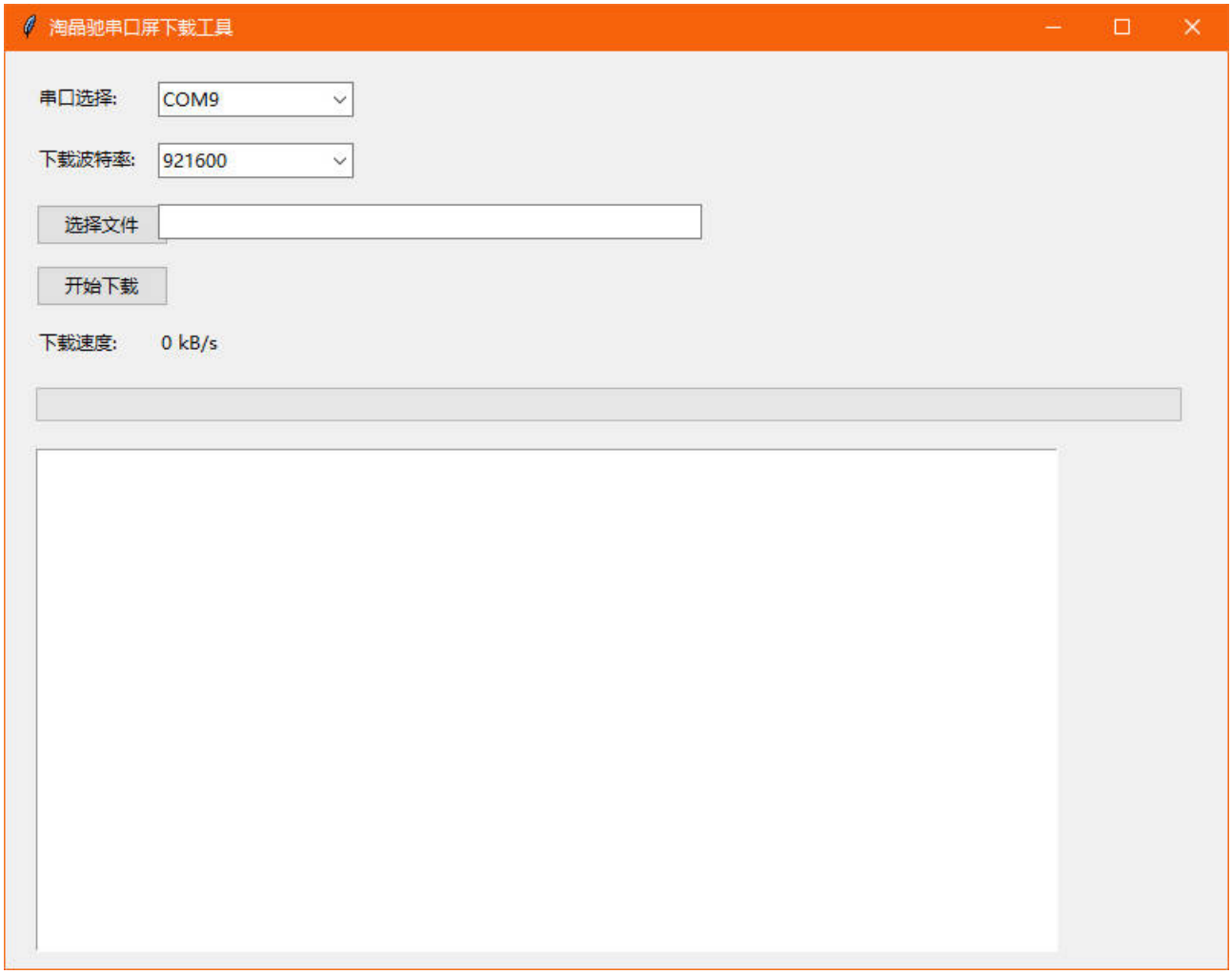
相关链接

HMI 下载协议详解/OTA 升级 (page 1006)

python 实现的下载工具 (page 1011)

0000000:ref: 串口屏实现的下载工具

8.3.5.1.2 python 实现的下载工具



TFTFileDownload python 源代码下载

```
import tkinter as tk
from tkinter import ttk, filedialog
import serial
import serial.tools.list_ports
import threading
import time
import os
from datetime import datetime

class TJCDownloadTool(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("淘晶驰串口屏下载工具")
        self.geometry("800x600")

        # 串口相关变量
        self.serial_port = None
        self.connect_baud = 0
        self.download_baud = 921600
        self.connecting = False
        self.downloading = False
        self.transferred = 0
```

(续下页)

(接上页)

```
        self.last_update = time.time()
        self.speed = 0

        # 创建界面组件
        self.create_widgets()
        self.scan_serial_ports()

    def create_widgets(self):
        # 串口选择
        ttk.Label(self, text="串口选择:").place(x=20, y=20)
        self.port_combobox = ttk.Combobox(self, width=15)
        self.port_combobox.place(x=100, y=20)

        # 波特率选择
        ttk.Label(self, text="下载波特率:").place(x=20, y=60)
        self.baud_combobox = ttk.Combobox(self, width=15, values=[
            2400, 4800, 9600, 19200, 38400, 57600,
            115200, 230400, 256000, 512000, 921600
        ])
        self.baud_combobox.set(921600)
        self.baud_combobox.place(x=100, y=60)

        # 文件选择
        self.file_btn = ttk.Button(self, text="选择文件", command=self.select_file)
        self.file_btn.place(x=20, y=100)
        self.file_path = ttk.Entry(self, width=50)
        self.file_path.place(x=100, y=100)

        # 下载按钮
        self.download_btn = ttk.Button(self, text="开始下载", command=self.start_
↪download)
        self.download_btn.place(x=20, y=140)

        # 下载速度显示
        ttk.Label(self, text="下载速度:").place(x=20, y=180)
        self.speed_label = ttk.Label(self, text="0 kB/s")
        self.speed_label.place(x=100, y=180)

        # 进度条
        self.progress = ttk.Progressbar(self, orient=tk.HORIZONTAL, length=750, mode=
↪'determinate')
        self.progress.place(x=20, y=220)

        # 日志文本框
        self.log_text = tk.Text(self, width=95, height=25)
        self.log_text.place(x=20, y=260)

    def scan_serial_ports(self):
        ports = sorted(serial.tools.list_ports.comports(), key=lambda p: p.device,␣
↪reverse=True)
        self.port_combobox['values'] = [p.device for p in ports]
        if ports:
            self.port_combobox.set(ports[0].device)

    def select_file(self):
        path = filedialog.askopenfilename(filetypes=[("TFT files", "*.tft")])
        if path:
            self.file_path.delete(0, tk.END)
            self.file_path.insert(0, path)

    def log(self, message):
```

(续下页)

(接上页)

```
        self.log_text.insert(tk.END, f"[{datetime.now().strftime('%H:%M:%S')}] {message}\
↪n")
        self.log_text.see(tk.END)

    def update_speed(self):
        if self.downloading:
            now = time.time()
            elapsed = now - self.last_update
            if elapsed > 0:
                self.speed = (self.transferred / elapsed) / 1024
                self.speed_label.config(text=f"{self.speed:.1f} kB/s")
            self.last_update = now
            self.transferred = 0
            self.after(1000, self.update_speed)

    def start_download(self):
        if not self.downloading:
            self.downloading = True
            self.transferred = 0
            self.last_update = time.time()
            self.download_btn.config(text="正在联机...")
            self.progress['value'] = 0
            self.after(1000, self.update_speed)
            threading.Thread(target=self.download_process).start()

    def download_process(self):
        port = self.port_combobox.get()
        file_path = self.file_path.get()

        if not port or not file_path:
            self.log("请先选择串口和文件")
            self.reset_btn()
            return

        try:
            # 尝试联机
            self.log("开始联机...")
            connect_bauds = [9600, 115200, 19200, 38400, 57600,
                             230400, 256000, 512000, 921600, 4800, 2400]

            for baud in connect_bauds:
                self.log(f"尝试 {baud} 波特率...")
                try:
                    with serial.Serial(port, baud, timeout=0.1) as ser:
                        connect_cmd = bytes.fromhex(
                            "44 52 41 4B 4A 48 53 55 59 44 47 42 4E 43 4A 48 47 4A 4B 53␣
↪48 42 44 4E FF FF FF 00 FF FF FF 63 6F 6E 6E 65 63 74 FF FF FF"
                        )
                        ser.write(connect_cmd)
                        time.sleep((1000000 / baud + 30) / 1000)
                        response = ser.read(1024)
                        if b'comok' in response:
                            self.connect_baud = baud
                            self.log(f"联机成功! 波特率: {baud}")
                            self.log(f"设备响应: {response.decode(errors='ignore')}")
                            break
                except Exception as e:
                    continue

            if not self.connect_baud:
                self.log("联机失败!")
```

(续下页)

(接上页)

```
                self.reset_btn()
                return

            # 开始下载
            self.download_btn.config(text="正在下载...")
            self.send_download_command(port, file_path)

        except Exception as e:
            self.log(f"发生错误: {str(e)}")
        finally:
            self.reset_btn()

    def send_download_command(self, port, file_path):
        try:
            # 获取文件大小
            file_size = os.path.getsize(file_path)
            download_baud = int(self.baud_combobox.get())
            self.progress['maximum'] = file_size

            # 发送whmi-wri指令
            with serial.Serial(port, self.connect_baud) as ser:
                cmd = f"whmi-wri {file_size},{download_baud},0".encode()
                ser.write(cmd + b'\xff\xff\xff')
                time.sleep(0.35)

                # 切换波特率
                ser.baudrate = download_baud
                response = ser.read(1)
                if response != b'\x05':
                    self.log("设备未响应准备信号")
                    return

                # 发送文件数据
                self.log("开始传输文件...")
                with open(file_path, 'rb') as f:
                    while True:
                        chunk = f.read(4096)
                        if not chunk:
                            break
                        ser.write(chunk)
                        self.transferred += len(chunk)
                        self.progress['value'] += len(chunk)
                        # 等待响应
                        while True:
                            resp = ser.read(1)
                            if resp == b'\x05':
                                break
                self.log("文件传输完成!")

        except Exception as e:
            self.log(f"下载失败: {str(e)}")

    def reset_btn(self):
        self.downloading = False
        self.connect_baud = 0
        self.download_btn.config(text="开始下载")
        self.progress['value'] = 0
        self.speed_label.config(text="0 kB/s")

if __name__ == "__main__":
    app = TJCDownloadTool()
```

(续下页)

(接上页)

```
    app.mainloop()
```

相关链接

HMI 下载协议详解/OTA 升级 (page 1006)

c# 实现的下载工具 (page 1009)

串口屏实现的下载工具 (page 1015)

8.3.5.1.3 串口屏实现的下载工具

通过 x 系列屏幕串口透传 tft 文件更新其他系列屏幕工程文件。

X 系列给其他屏幕下载工程 (page 1143)

相关链接

HMI 下载协议详解/OTA 升级 (page 1006)

c# 实现的下载工具 (page 1009)

python 实现的下载工具 (page 1011)

8.4 串口指令增加 CRC 校验

(单片机向串口屏发送的指令中增加 crc 校验，0.56 及其以上上位机版本支持)

正常情况下，指令直接发送即可，不需要校验，如果您的项目对指令传输要求很严格必须开启校验请按照下列说明发送指令

切记注意：上位软件版本 0.56 开始才支持指令 CRC，之前的版本是不支持的。

带校验或不带校验无需做任何配置，只需修改指令即可，您可以上一条指令带校验，下一条指令不带校验也是可以的。两种指令的区别如下：

8.4.1 结束符不同

普通指令结束符为\xFF\xFF\xFF

带校验的指令结束符为\xFE\xFE\xFE

8.4.2 CRC16 校验算法

指令 CRC16 校验算法使用 MODBUS 的 CRC16 校验算法。其校验结果是 2 字节（16bit）的数据

需要校验的数据为所有指令数据，如果是带地址的指令，从地址开始算，不带地址的指令，就从指令第一个字节开始计算，结束符不计算在内。

如果您的单片机需要在指令中添加 crc 校验，那么单片机里的计算函数如下，您可以参考，也可以使用自己原有的 MODBUS_CRC16 代码。

```
//单片机MODBUS_CRC16代码

static U8 InvertUint8(U8 data)
{
   int i;
   U8 newtemp8 = 0;
   for (i = 0; i < 8; i++)
   {
      if ( (data & (1 << i) ) != 0) newtemp8 |= (U8)(1 << (7 - i));
   }
   return newtemp8;
}
static U16 InvertUint16(U16 data)
{
   int i;
   U16 newtemp16 = 0;
   for (i = 0; i < 16; i++)
   {
      if ( (data & (1 << i) ) != 0) newtemp16 |= (U16)(1 << (15 - i));
   }
   return newtemp16;
}
U16 CRC16_MODBUS(U8* data, int lenth)
{
   int i;
   U16 wCRCin = 0xFFFF;
   U16 wCPoly = 0x8005;
   U16 wChar = 0;
   while (lenth > 0)
   {
      wChar = *data;
      data++;
      wChar = InvertUint8( (U8)wChar);
      wCRCin ^= (U16)(wChar << 8);
```

(续下页)

(接上页)

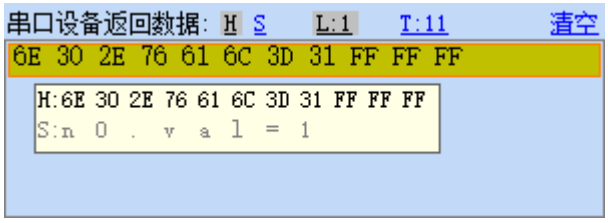
```
        for (i = 0; i < 8; i++)
        {
            if ((wCRCin & 0x8000) != 0)
            {
                wCRCin = (U16)( (wCRCin << 1) ^ wCPoly);
            }else
            {
                wCRCin = (U16)(wCRCin << 1);
            }
        }
        lenth=lenth-1;
    }
    wCRCin = InvertUint16(wCRCin);
    return (wCRCin);
}
```

8.4.3 3.CRC16 校验码写入方式

指令后面，结束符前面，加上 2 字节的 CRC16 校验码 (HEX)+1 字节的常数:0x01(HEX), 相当于在指令和结束符中间插入了 3 个字节，CRC 校验码的储存方式是小端模式, 低位在前。

常规指令示例

```
n0.val=1\xFF\xFF\xFF
```



带 crc 指令示例

```
n1.val=1\x26\x4C\x01\xFE\xFE\xFE
```

如果屏幕收到带校验的指令后发现校验失败，会返回错误：0x09 0xff 0xff 0xff

8.5 程序中使用 CRC 校验数据

(1.60.1 及其以上上位机版本支持, 目前仅支持 crc-16/modbus)

8.5.1 使用 crc 校验的步骤

8.5.1.1 初始化 crc 校验

```
//初始化crc
crcrest 1,0xffff
```

8.5.1.2 使用 crcputh/crcputs/crcputu 将需要的数据放入校验

crcputh 的用法参考 printh

crcputs 的用法参考 prints

```
//将0x03 0x25放入crc校验
crcputh 03 25

//将字符串"aaa"放入crc校验
crcputs "aaa",0

//将字符串t0.txt放入crc校验
crcputs t0.txt,0

//将字符串t1.txt的前6个字节放入crc校验
crcputs t1.txt,6

//将数值16的低2字节放入crc校验
crcputs 16,2

//将数值n0.val放入crc校验
crcputs n0.val,0

//将数值n1.val的低2字节放入crc校验
crcputs n1.val,2

//将串口缓冲区的第5个字节开始的24个字节放入crc校验
crcputu 5,24
```

8.5.1.3 获取 crc 计算结果

crcval 变量即为计算结果，可以随时获取

```
//将crcval的数据发送出去
prints crcval,2
```

8.5.2 crc 校验示例

未进行校验的代码如下所示

```
//发送55 aa 作为帧头
printh 55 aa

//串口发送数值变量n0.val的低2字节
prints n0.val,2

//串口发送字符串变量t0.txt
prints t0.txt,0

//发送0d 0a作为帧尾
printh 0d 0a
```

将上方的代码添加 crc-16 校验并发送出去

```
//重置crc校验
crcrest 1,0xffff

//发送55 aa
printh 55 aa

//将55 aa 放入crc校验
crcputh 55 aa

//串口发送数值变量n0.val的低2字节
prints n0.val,2

//将数值变量n0.val的低2字节放入crc校验
crcputs n0.val,2

//串口发送字符串变量t0.txt
prints t0.txt,0

//将字符串变量t0.txt放入crc校验
crcputs t0.txt,0

//发送0d 0a作为帧尾
printh 0d 0a

//将0d 0a放入crc校验
crcputh 0d 0a

//串口发送校验结果
prints crcval,2
```

在大部分的应用场景下，crc 校验仅仅是将 printh 改成 crcputh，prints 改为 crcputs

```
//重置crc校验
crcrest 1,0xffff

//发送55 aa
printh 55 aa

//将55 aa 放入crc校验
crcputh 55 aa

//串口发送数值变量n0.val的低2字节
prints n0.val,2

//将数值变量n0.val的低2字节放入crc校验
crcputs n0.val,2

//串口发送字符串变量t0.txt
prints t0.txt,0

//将字符串变量t0.txt放入crc校验
crcputs t0.txt,0

//发送0d 0a作为帧尾
printh 0d 0a

//将0d 0a放入crc校验
crcputh 0d 0a

//串口发送校验结果
prints crcval,2
```

注意： 注意放入 crc 校验中变量的长度

crcputs n0.val,3，crcputs n0.val,2，crcputs n0.val,1 和 crcputs n0.val,0 得出的 crc 校验值是完全不同的 4 个值！

8.5.3 串口屏计算出来的 crc 结果和单片机计算出来的结果不同如何排查

借助网上的 crc 在线校验工具，先计算出标准的 modbus-crc16 的值，然后对比一下是串口屏计算出来的 crc 有问题还是单片机计算出来的 crc 有问题，针对问题修改代码即可

8.5.4 crc 校验-样例工程下载

演示工程下载链接：

《CRC 校验》演示工程下载

《crc 校验例程一对》演示工程下载

8.6 运行中串口传输文件到内存或 SD 卡

(0.56 及其以上上位机版本支持)

如果要传输文件到内存，请先在工程设置里面配置内存文件系统空间大小，默认是 0，即没有空间用来储存内存文件。

如果是传输文件到 SD 卡，请确保 SD 卡磁盘格式为 FAT32, 最大支持 32G 容量的卡

温馨提示：官方有按照本协议开发好的专用串口透传文件工具 (SerialFileUp)，并且开放源代码，欢迎下载使用：

点击下载文件透传工具 *(SerialFileUp)* (page 1098)

以下以传输图片文件到内存为例，介绍如何使用串口传输图片文件到内存或 SD 卡，当然你也可以传输其他类型的文件，比如纯文本文件。

1. 如何使用内存中的图片文件

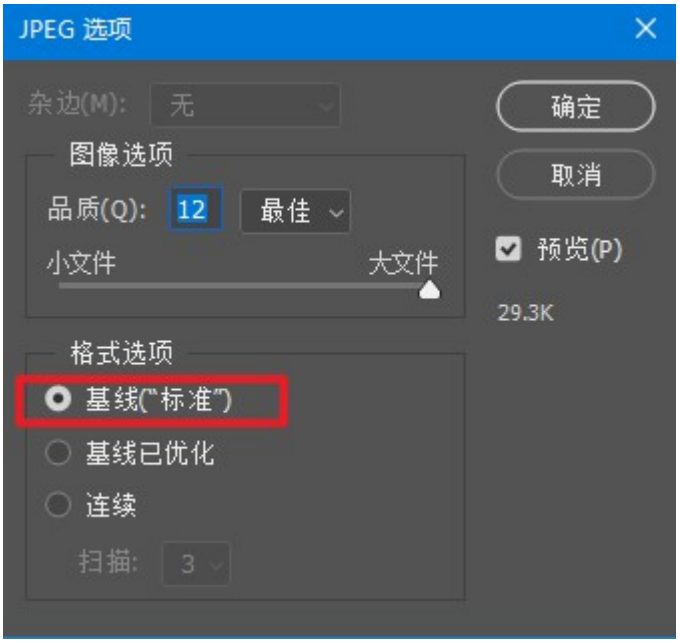
    使用“外部图片“控件，设置 path 属性为文件路径，如:ram/0.jpg

2. 如何使用 SD 卡中的图片文件

    使用“外部图片“控件，设置 path 属性为文件路径，如:sd0/0.jpg

3. 支持什么类型的图片文件

    jpg 格式: 如果使用 jpg 格式的图片，务必保证图片编码为:baseline DCT, 否则将无法显示, 使用 ps 打开 jpg 图片，将该图片另存为，保存时格式选择“基线（“标准”）”



    如果屏幕方向设置的是 0 度，直接使用 jpg 原图即可，如果屏幕方向设置的是 90 度，需要将图片顺时钟旋转 90 度再给屏幕使用，否则显示出来的图片方向是错的。其他方向 180 度，270 度以此类推，屏幕方向是多少度，就需要您提前把图片顺时钟旋转多少度。

    xi 格式: 此格式为 HMI 外部图片控件专用图片格式，支持透明背景，推荐使用这种格式，xi 格式的图片获取方式可以使用软件工具菜单中的图片转换工具:PictureBox 转换

外部图片控件目前仅支持以上两种格式的图片文件。

4. 串口传输协议

第一步: 发送串口传输文件指令:twfile filepath,filesize

filepath: 文件存放路径如:ram/a.jpg 或 sd0/a.jpg

filesize: 文件实际大小

假如要传一个文件到内存中，名字为 a.jpg, 大小为 3282 字节, 指令为：

```
twfile "ram/a.jpg",3282
```

假如要传一个文件到 SD 卡中，名字为 a.jpg, 大小为 3282 字节, 指令为:

```
twfile "sd0/a.jpg",3282
```

屏幕收到此指令后，会立即创建一个指定大小的文件在目标路径上，如果创建成功将返回:0xfe+ 结束符, 表示已经进入透传状态，可以开始分包透传数据。如果创建文件失败会返回:0x06+ 结束符, 并继续工作在指令接收状态。

第二步：分包透传文件数据

收到 0xfe+ 结束符后，可以开始分包透传数据; 一个完整的数据包由 2 部分组成：包头 + 数据

包头:3a a1 bb 44 7f ff fe + 校验类型 (1 字节整形数据)+ 包 ID(2 字节整形数据) + 数据大小 (2 字节整形数据)，合计 12 字节

校验类型:0x00 为无校验,0x01 为 CRC16(MODBUS 的 CRC16 校验算法, 指令校验篇幅中有计算函数参考),0x0A 为标准 CRC32

包 ID: 文件透传开始第一次包 ID 为 0，每成功透传一包，ID 加 1.

数据大小: 数据大小用户随意指定，最小 1 字节，最大 4096 字节 (此数据大小不包含 12 字节的包头，但是包含 CRC 校验码)

数据: 文件数据 +CRC 校验码数据 (小端模式，低位在前), 如果是无校验的包，就没有 CRC 校验码。如果是 CRC16 就是 2 字节的校验码，如果是 CRC32 就是 4 字节的校验码，切记注意校验码数据要记入包头的数据大小参数中。

CRC16 校验算法为 MODBUS CRC16, 点击此处查看参考函数代码 CRC32 校验算法为标准 CRC32。

屏幕收到一个完整的包数据后，如果处理成功会返回 0x05(单字节，没有结束符), 此时可进入下一个包的发送。

如果包 ID 不按规则累加或者包数据出错，屏幕将会返回本包处理失败的错误:0x04（单字节，无结束符）。

所有包发送完成后，屏幕会返回 0xfd+ 结束符。并自动退出透传模式转为指令模式。

如果本包透传失败（超过 500ms 没有收到屏幕回应或者收到屏幕返回本包校验失败的错误信息），重新发本包数据，重发本包数据时包 ID 无需加 1。

如果数据包发了一半后悔了，停顿 20ms 以上重新发本包数据即可。

如果透传文件中途想停止透传，请发一个包 ID 为 65535，无校验，数据大小为 0 的数据包, 即:3a a1 bb 44 7f ff fe 00 ff ff 00 00 屏幕收到这样的包数据后会立刻强制结束透传，并返回透传结束的数据：0xfd+ 结束符

如果你的文件数据中包含退出包数据是不用担心的，因为这个数据的前面满足不了 20ms 以上的停顿，屏幕只会把他当数据储存，不会当结束包来处理。

8.7 SD 卡读写文件流程

串口屏使用的卡是 microSD 卡，部分情形下使用 SD 卡来代称

提示： 仅 X2/X3/X5 系列支持通过文件流控件来读写 microSD 卡文件，其他系列仅支持用 microSD 卡来升级固件请参考: 通过 *SD* 卡下载工程到串口屏 (page 155)

注意： microSD 卡也是有写入寿命的, 具体取决于 SD 卡的质量。

注意： microSD 卡不能超过 32GB（例如：512M、1GB、2GB、4GB、8GB、16GB、32GB 都是可用的）。

8.7.1 1. 检查文件是否存在

注意： 文件路径是字符串形式，需要被引号包裹起来，文件名建议使用英文和数字，使用其他可能会显示乱码（编码不一致导致的）

```
findfile "sd0/1.txt",sys0
if(sys0==1)
{
  //文件存在
  msg.bco=GREEN
  msg.txt="文件存在"
}else
{
  //文件不存在,新建文件
  newfile "sd0/1.txt",4096
  //再次检查文件是否存在
  findfile "sd0/1.txt",sys0
  if(sys0==1)
  {
    //文件创建成功
    msg.bco=GREEN
    msg.txt="文件创建成功"
  }else
  {
    //判断为SD卡不存在
    msg.bco=RED
    msg.txt="找不到SD卡"
  }
}
```

注意： 也可以用文本控件来保存文件路径，但是请注意文本控件的 txt_maxl 属性需要足够大，建议设置为 200，否则可能因为文件路径不完整而导致打开失败

```
filePath.txt="sd0/1.txt"
findfile filePath.txt,sys0
if(sys0==1)
{
  //文件存在
  msg.bco=GREEN
  msg.txt="文件存在"
}else
{
  //文件不存在,新建文件
  newfile filePath.txt,4096
  //再次检查文件是否存在
  findfile filePath.txt,sys0
  if(sys0==1)
  {
    //文件创建成功
    msg.bco=GREEN
    msg.txt="文件创建成功"
  }else
  {
    //判断为SD卡不存在
    msg.bco=RED
    msg.txt="找不到SD卡"
  }
}
```

8.7.2 2. 打开文件

注意: 操作文件需要使用文件流控件

```
filePath.txt="sd0/1.txt"
findfile filePath.txt,sys0
if(sys0==1)
{
  //文件存在,打开文件
  sys0=fs0.open(filePath.txt)
  if(sys0==1)
  {
    //文件打开成功
    msg.bco=GREEN
    msg.txt="文件打开成功"
  }
}else
{
  //文件不存在,新建文件
  newfile filePath.txt,4096
  //再次检查文件是否存在
  findfile filePath.txt,sys0
  if(sys0==1)
  {
    //文件存在,打开文件
    sys0=fs0.open(filePath.txt)
    if(sys0==1)
    {
      //文件打开成功
      msg.bco=GREEN
      msg.txt="文件创建成功"
      for(sys0=0;sys0<1024;sys0++)
      {
        //手动初始化整个文件，循环1024次，每次写入4字节，总共写入4096个字节，将0-
↪4095初始化为0x00
        fs0.write(0,0,4)
      }
      fs0.close() //有打开就要有关闭，最好写完后立刻关闭
    }
  }else
  {
    //判断为SD卡不存在
    msg.bco=RED
    msg.txt="找不到SD卡"
  }
}
```

注意： 也可以通过文件浏览器来获取文件路径，但是请注意文本控件的 txt_maxl 属性需要足够大，建议设置为 200，否则可能因为文件路径不完整而导致各种问题

此时可以不判断文件是否存在, 只需判断 fbrowser0.txt 非空即可

fbrowser0.txt 在你选中了文件之后会自动改变

```
filePath.txt=fbrowser0.dir+fbrowser0.txt
if(fbrowser0.txt!="")
{
  sys0=fs0.open(filePath.txt)
  if(sys0==1)
  {
    //文件打开成功
    msg.bco=GREEN
    msg.txt="文件打开成功"
  }
}
```

8.7.3 3. 读、写、查找

保存数据时请注意以下几点

1. 从哪里写入就从哪里读取 (写入和读取前设置文件流的 val 属性, 即读取的位置)。

2. 写入时对应指针地址是什么属性，读取时就是什么属性，写入时是字符串属性，读取时就要读取到字符串属性中。写入时是数值属性，读取时就要读取到数值属性中。

read-从文件流读取数据 (page 532)

write-将数据写入文件流 (page 532)

find-按关键字查询并定位文件流指针 (page 533)

8.7.4 4. 关闭文件

有打开就必须有关闭

close-关闭文件流 (page 533)

8.7.5 文件流控件-注意事项

8.7.5.1 新建文件时为什么要初始化文件

电脑上在创建文件时，会自动将整个文件都初始化为 0x00, 但是在串口屏上并不会

这是因为电脑的 cpu 是多线程的且电脑的性能远远高于串口屏，而串口屏是单线程

如果串口屏也自动初始化整个文件，当用户创建比较大的文件，如 1GB 的文件时，串口屏将会因为初始化文件的原因卡住一段时间，可能几分钟到十几分钟不等，取决于 SD 卡性能

8.7.5.2 没有初始化文件会怎么样

如果没有初始化文件，那么文件里面就会有之前删除过的文件数据，直接读取时就会有问题，例如读到未初始化的地方可能显示乱码。

8.7.5.3 如果我不想在屏幕里初始化怎么办呢

你可以提前在电脑模拟器上创建好一个文件，然后把文件拷贝到 SD 卡里面

8.7.6 SD 卡读写文件流程-样例工程下载

演示工程下载链接：

《文件浏览器 v3.0》演示工程下载

《使用文件流控件记录波形数据》演示工程下载

《文件流控件简单使用 1》演示工程下载

《文件流控件简单使用 2》演示工程下载

《文件流控件简单使用 3》演示工程下载

8.8 拓展 IO 资料

8.8.1 拓展 IO 简介

提示： 电脑模拟器上没有外部 IO，拓展 IO 指令只能在真机上测试，不保证在电脑模拟器上的效果。

拓展 IO，fpc 接口规格是 fpc 1.0*10 。

IO 口上提供 5V 电源，但是所有系列拓展 IO 口电平都是 3.3V，如果外部向串口屏 IO 口输入高电压，也不建议超过 3.3V，否则可能会损坏串口屏。

扩展板属于选配开发套件，不是标配配件，如果需要请单独购买，请知晓。您也可以自行设计，请参考拓展板原理图 (page 1041)

支持的型号：

x5 系列支持 8 路 IO

k0 系列支持 8 路 IO

X2 系列部分型号支持 8 路 IO(具体是否支持请查看对应型号的规格书) 规格书下载请点击此处访问网址

拓展 IO 口不支持 AD,SPI,I2C 等功能, 仅支持速率较慢的高低电平输出, 按键检测 (非矩阵键盘)，以及部分 IO 口支持 PWM

拓展 IO 只能实现单片机一部分的 IO 功能 (如按键输入,LED,PWM)，如果需要更强大的功能（如旋转编码器），建议使用单片机实现。

K0/X2 系列只有 io4-io7 才支持 PWM 输出，X5 系列只有 io7 才支持 PWM 输出其他 IO 不支持。配置其他 IO 为 PWM 模式会报错。

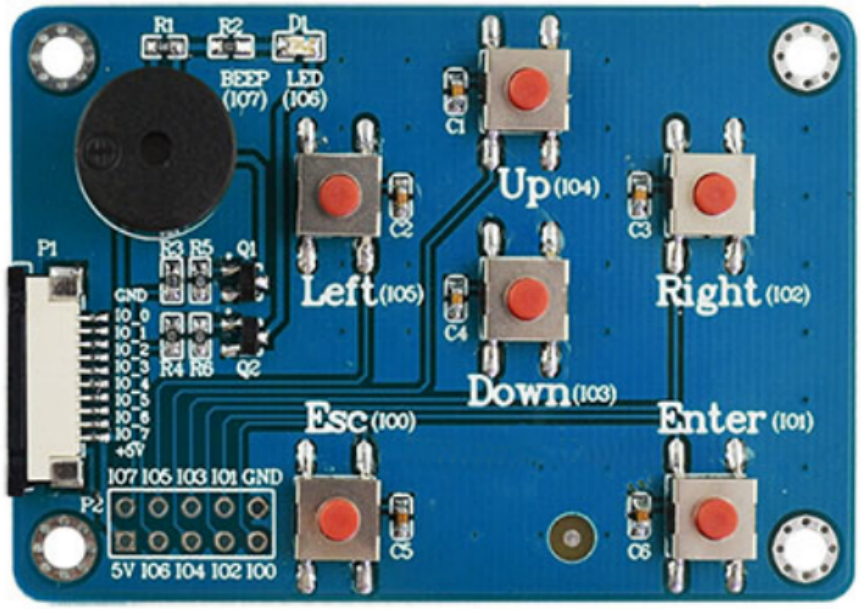
PWM 功能不支持精确控制步进电机，不支持输出精确的脉冲个数，不支持互补输出。

PWM 仅建议用于蜂鸣器，风扇调速，LED 亮度控制等简单 PWM 应用。

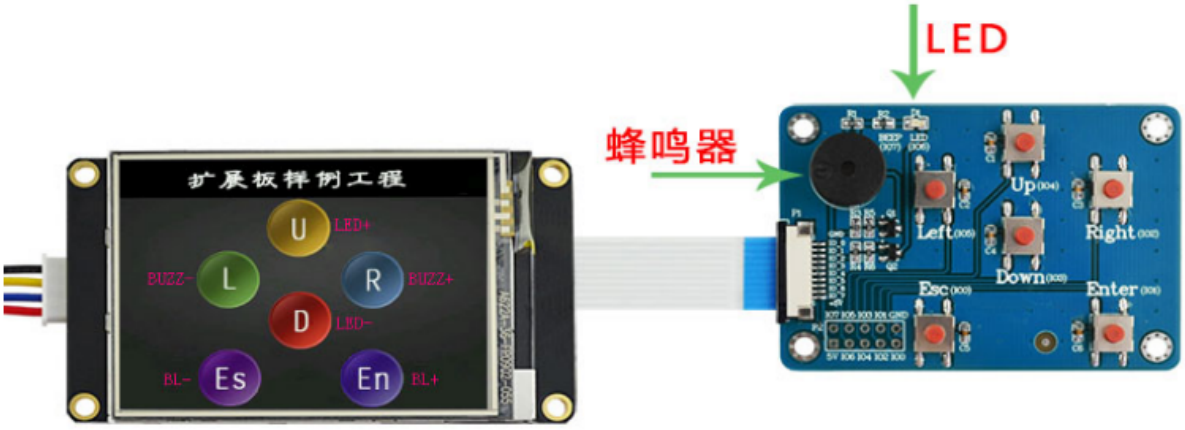
IO 口驱动继电器时，可能驱动能力不足（市面上的继电器大部分都是 5V 以上的电压驱动，串口屏的 IO 口电压只有 3.3V 左右，无法正常驱动继电器），需要自行增加外部电路以增加驱动能力。

使用拓展 IO 之前需要用 cfgpio 指令绑定 IO 口功能，每个需要使用 IO 口的页面都需要用 cfgpio 指令重新绑定，参考：*cfgpio-扩展 IO 模式配置* (page 650)

拓展 IO 作为输入时，建议输入 3.3V 的电压。如果要输入 5V，需要最好串联电阻分压一下，串联至少 1K 的电阻。

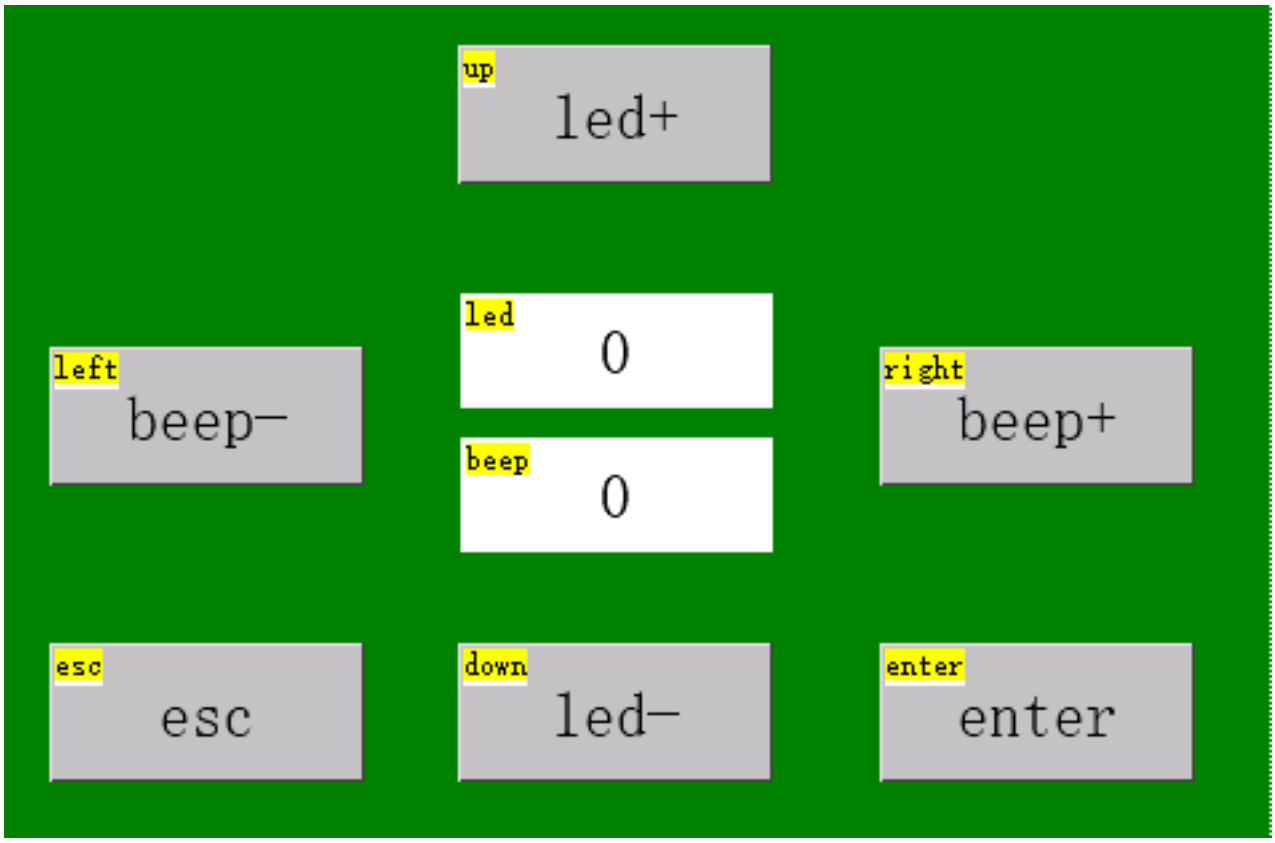

R1
R2
D1
BEEP (107)
LED (106)
C1
Up (104)
C2
C3
P1
R3 R5 Q1
Left (105)
C4
Right (102)
GND
IO_0
IO_1
IO_2
IO_3
IO_4
IO_5
IO_6
IO_7
+5V
R4 R6 Q2
Down (103)
Esc (100)
Enter (101)
107 105 103 101 GND
P2
C5
C6
5V 106 104 102 100

8.8.2 拓展 IO 简单使用

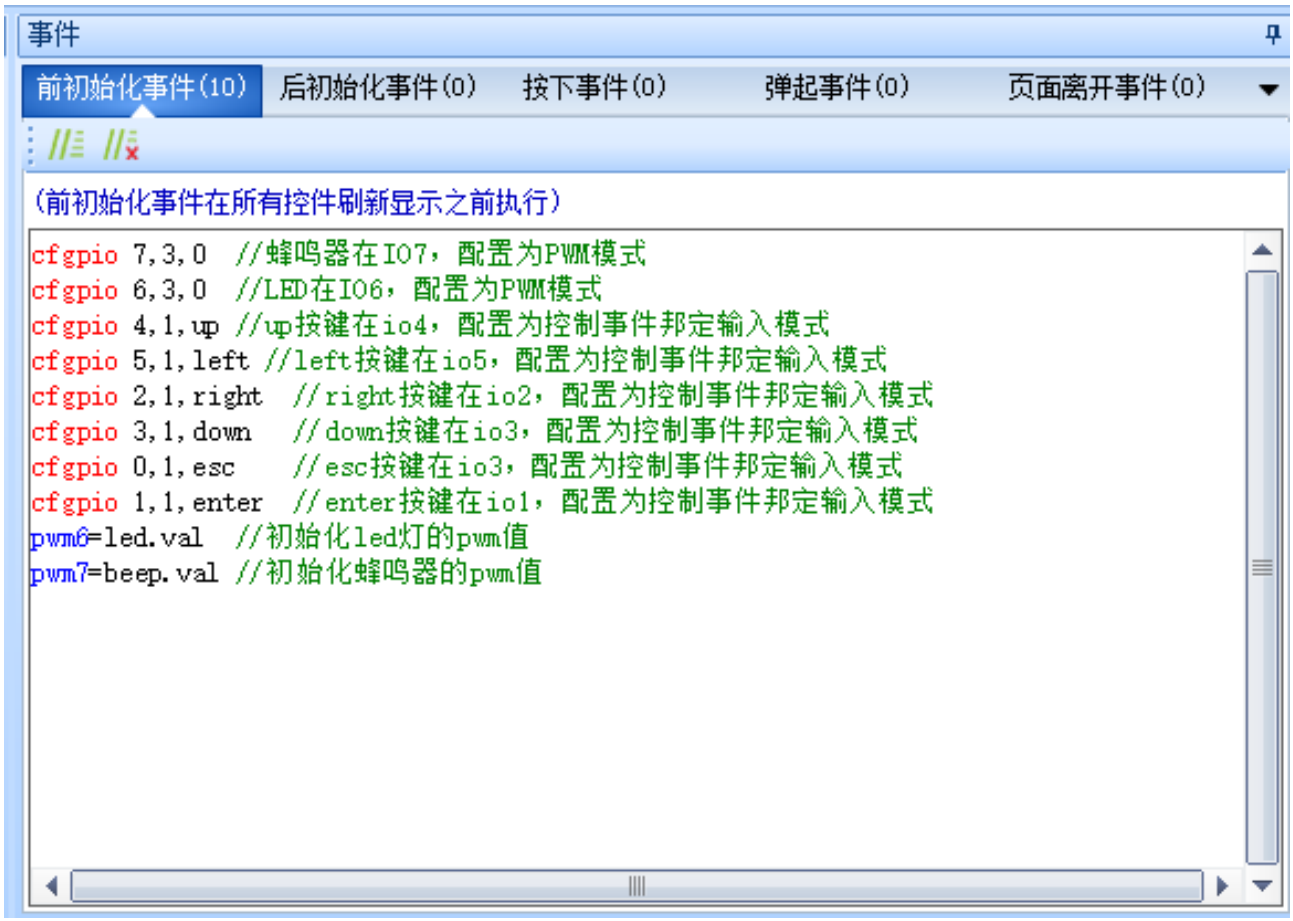


1.界面上的6个按钮分别邦定扩展板的6个按键
2.左右键控制蜂鸣器音调，上下键控制LED亮度
3.蜂鸣器和LED的控制都是通过设置PWM占空比实现
4.此实例无需单片机参与

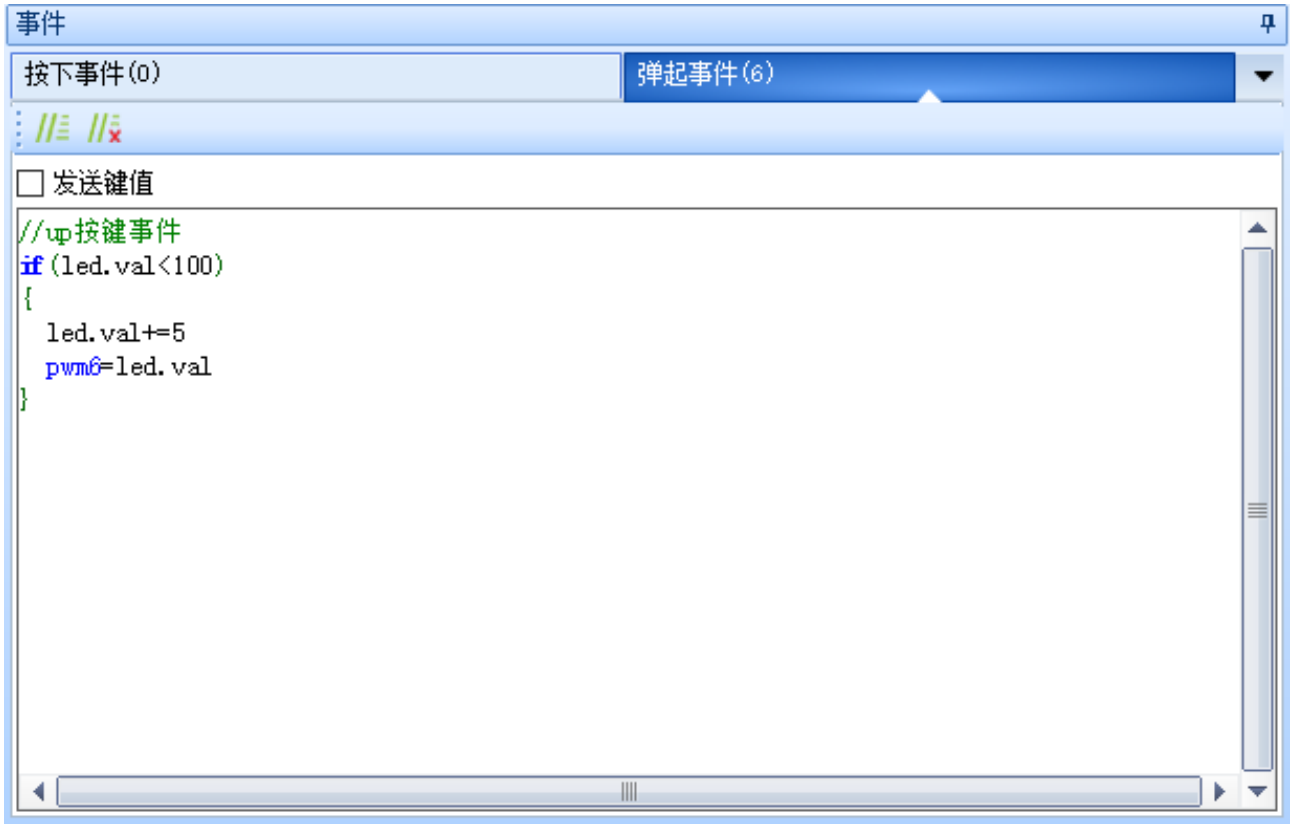
第一步，在页面中新建数个按钮和数字控件，并修改 objname 如下所示，其中 led 和 beep 是数字控件，其他是按钮控件



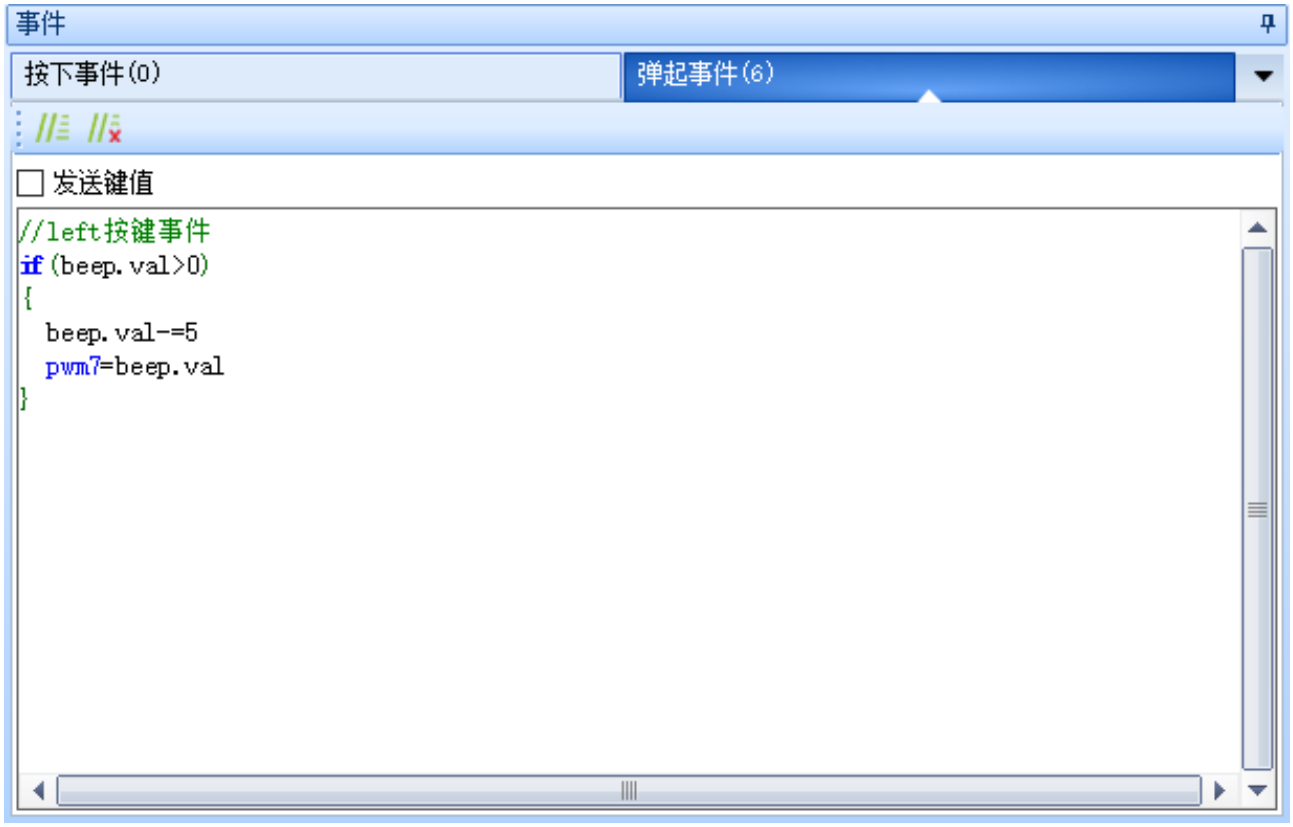
第二步，在页面前初始化事件中绑定相关按键并初始化 pwm 值



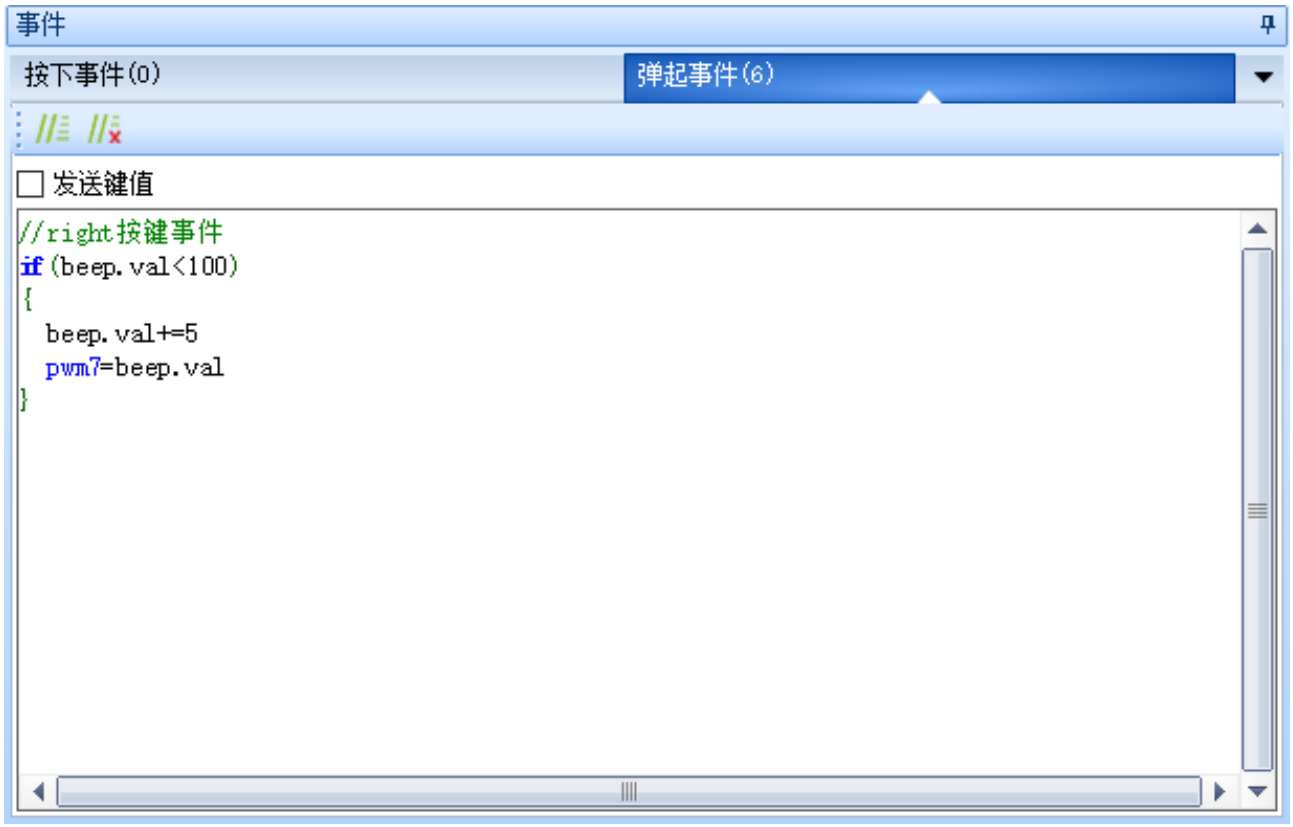
在 up 按键中编写如下代码 (建议把代码写到弹起事件中, 避免首次进入时触发, 下同)



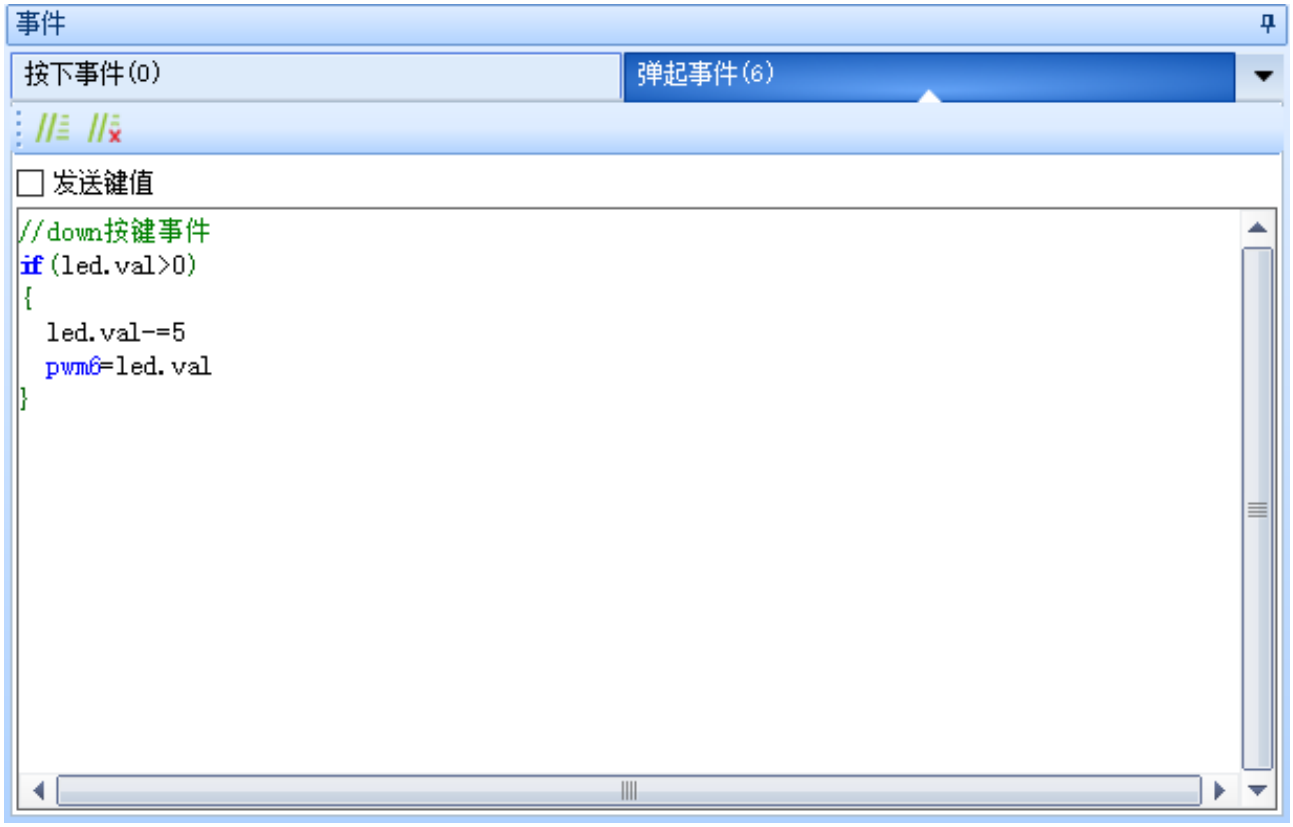
在 left 按键中编写如下代码



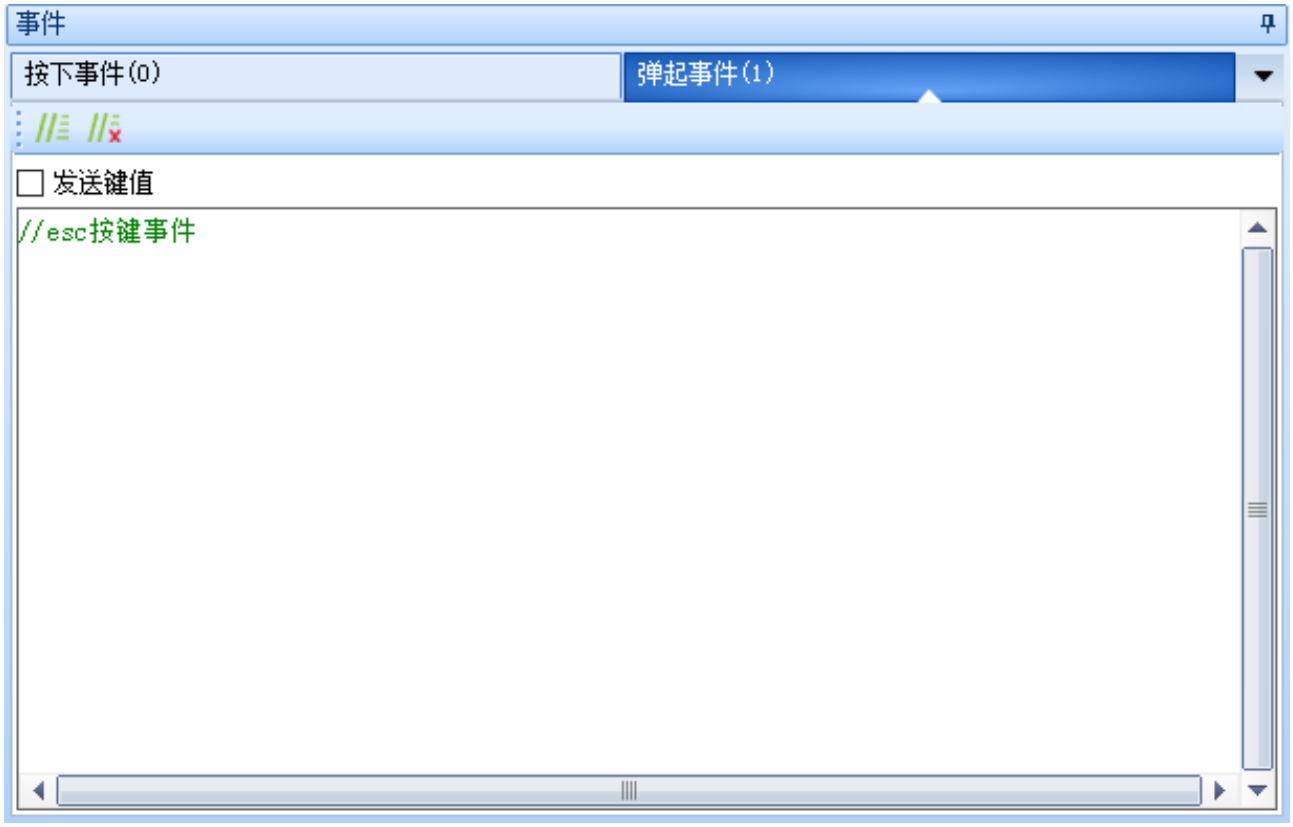
在 right 按键中编写如下代码



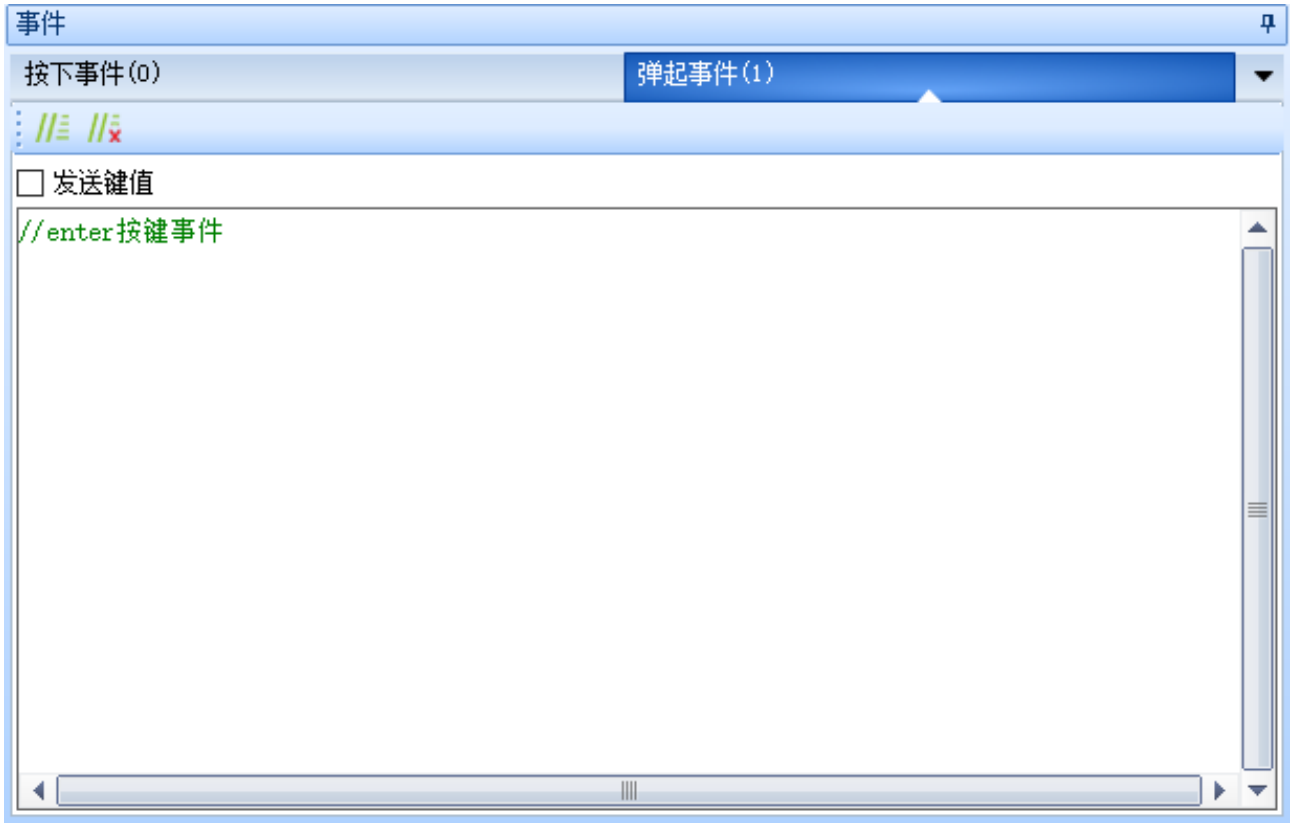
在 down 按键中编写如下代码



在 esc 按键中编写如下代码

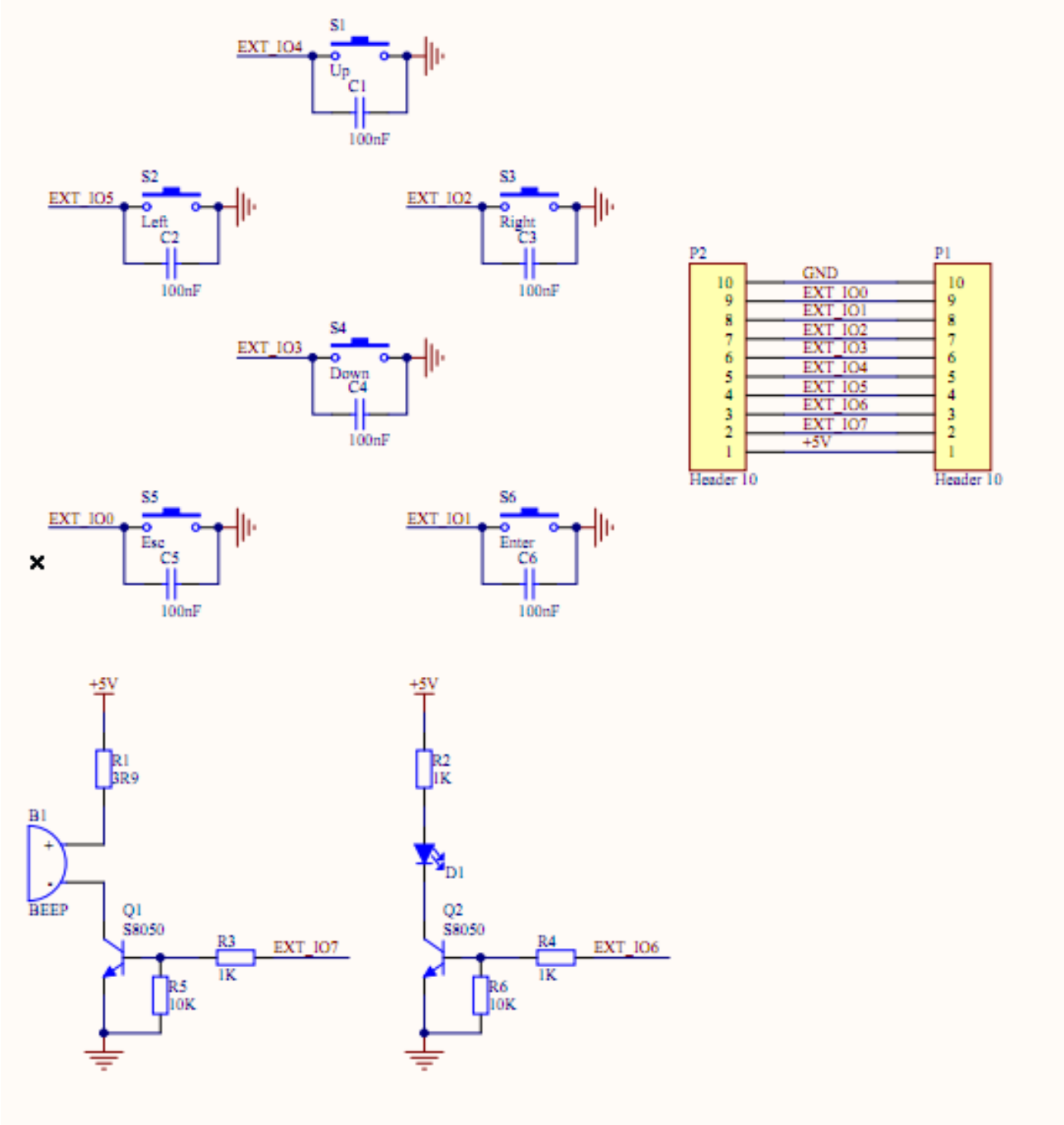


在 enter 按键中编写如下代码



下载到屏幕调试即可

8.8.3 拓展板原理图



8.8.4 sleep 睡眠模式与拓展 IO

在进入睡眠模式后, 拓展 IO 的某些功能会无法使用, 请注意

提示: 在进入睡眠模式后, 输入相关的功能如上拉输入模式、件事件邦定输入模式是无法使用的, 输出相关的功能如推挽输出模式、PWM 输出模式、开漏模式

8.8.5 外部 IO-样例工程下载

演示工程下载链接：

《拓展 IO 例程》演示工程下载

《IOtest》演示工程下载

8.8.6 拓展 IO-相关链接

cfgpio-扩展 IO 模式配置 (page 650)

拓展 IO 相关问题 (page 1258)

pio0~pio7-扩展 IO 端口 (page 802)

pwm4~pwm7-扩展 IO 占空比 (page 804)

pwmf-PWM 输出的频率 (page 806)

sleep 睡眠模式与拓展 IO (page 1041)

8.9 获取控件自身的属性

在其他语言，例如 java 中，this 是自身的一个对象，代表对象本身。

可以使用 this 来获取到对象自身的属性。

例如

```
public Date(int year, int month, int day){
            this.year = year;
            this.month = month;
            this.day = day;
    }
```

某些情况下需要在当前的控件中获取到当前控件的属性

例如获取 b0 的 txt 属性进行判断

```
if(b0.txt=="开始")
{
  b0.txt="停止"
}else
{
  b0.txt="开始"
}
```

如果有多个类似的按钮进行判断，修改起来就相对麻烦，那么就可以用下面这种方法来获取到自身的属性进行判断。

```
if('&txt&'=="开始")
{
  '&txt&'="停止"
}else
{
  '&txt&'="开始"
}
```

也可以通过’&val&’来获取自身的 val 属性或者其他属性

8.9.1 获取控件自身的属性-样例工程下载

演示工程下载链接：

《获取控件自身的属性》演示工程下载

8.10 读写用户存储区

用户存储区即 eeprom，在淘晶驰常规系列串口屏中，支持 eeprom 的产品，其 eeprom 大小通常为 1K Byte(即 1024 字节)，部分型号的用户存储区采用 flash 来模拟 eeprom。

使用用户存储区的第一步：初始化用户存储区数据，参考：*如何对掉电存储空间进行初始化* (page 1320)

使用用户存储区的第二步：新建一个按钮控件，txt 改为 “保存”，在按钮里写入用户存储区数据，参考：*wepo-写入数据到掉电存储空间* (page 634)

使用用户存储区的第三步：上电时或者跳转页面前读取用户存储区数据，参考：*repo-从掉电存储空间读取数据* (page 640)

8.10.1 读写用户存储区例程-样例工程下载

演示工程下载链接：

《多种控件演示》演示工程下载

《数字控件写入和读取 eeprom》演示工程下载

《文本控件写入和读取 eeprom》演示工程下载

8.11 获取串口屏设备唯一序列号

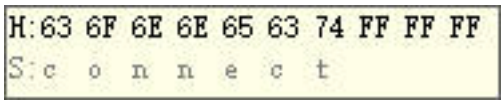
提示： 设备唯一序列号是每个串口屏出厂时由厂家烧录在芯片内的唯一标识码，类似于电脑的 CPU 序列号或者手机的 IMEI 码，可以用来区分每一台设备。

传统方法只能通过串口获取到芯片的唯一序列号, 串口屏自身无法获取到序列号, 所有系列所有型号都支持

8.11.1 通过串口获取

单片机或者电脑向串口屏发送联机指令:connect+ 结束符

发送的数据如下所示:



此时串口屏会返回一串数据

以 TJC4024T032_011R 设备为例，设备返回如下 8 组数据 (每组数据逗号隔开):

comok 1,101-0,TJC4024T032_011R,52,61488,D264B8204F0E1828,16777216

comok: 握手回应

1: 表示带触摸 (0 是不带触摸)

101-0: 设备内部预留数据-设备地址

TJC4024T032_011R: 设备型号

52: 设备固件版本号

61488: 设备主控芯片内部编码

D264B8204F0E1828: 设备唯一序列号

16777216: 设备 FLASH 大小 (单位：字节)

8.11.2 新方法-通过指令获取

注意： 新方法仅 T1 系列和 X 系列支持, 允许串口屏通过指令获取到设备唯一序列号, 需要 1.65.1 以上版本的上位机

注意： 新方法仅支持在实物屏幕上使用, 在模拟器中无法获取

1, 新建一个文本控件 t0。长度可以长一些，比如 200 字节，长度过短会读取失败，导致文本控件一片空白。

2. 写代码 “getv t0.txt”

这个命令会把串口发 connect 命令返回的所有数据装到 t0.txt 中

此时就可以通过 spstr 指令将设备唯一序列号截取出来

spstr t0.txt,t1.txt,” ,” ,5

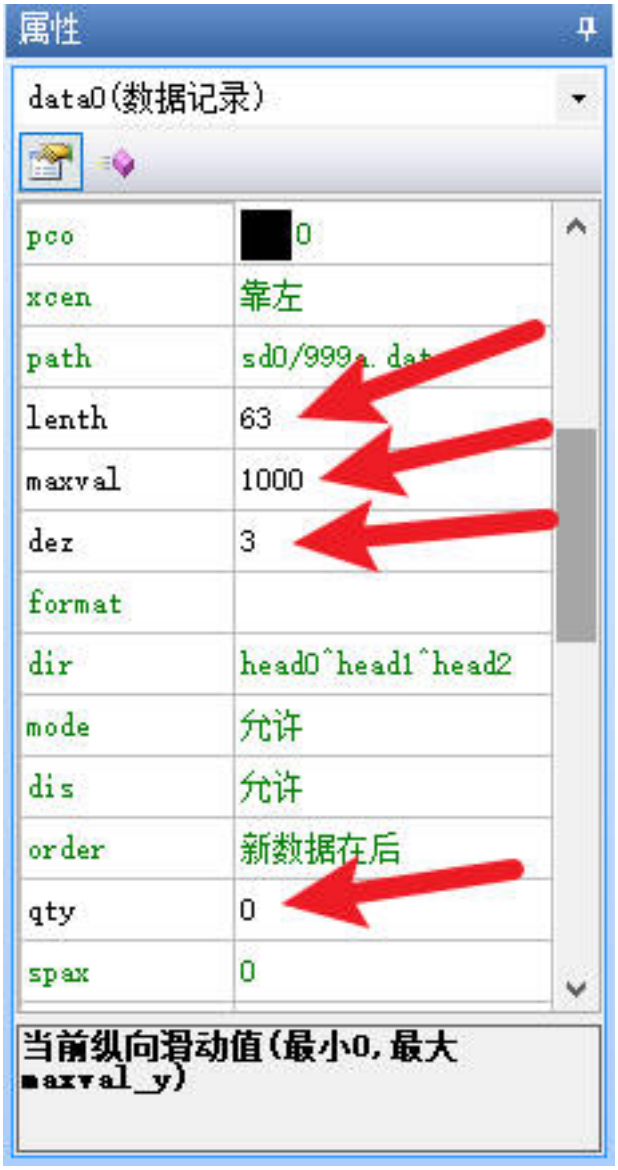
8.11.3 获取设备唯一序列号例程下载

演示工程下载链接：

《获取设备唯一序列号》演示工程下载

8.12 数据记录文件结构分析

8.12.1 数据记录文件结构分析详解



Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	CP 20936
00000000	7D	26	DA	99	5C	BD	53	00	14	00	00	00	40	00	00	00	}& \ S□□□□□@□□□
00000010	00	00	00	00	03	00	00	00	E8	03	00	00	00	00	00	00	□□□□□□□□ □□□□□□□
00000020	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000050	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000060	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000070	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000080	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000090	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000000A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000000B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000000C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000000D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000000E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000000F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000100	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000110	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000120	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000130	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000140	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000150	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000160	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000170	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000180	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000190	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000001A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000001B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000001C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000001D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000001E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000001F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000200	7D	26	DA	99	5C	BD	53	00	14	00	00	00	40	00	00	00	}& \ S□□□□□@□□□
00000210	00	00	00	00	03	00	00	00	E8	03	00	00	00	00	00	00	□□□□□□□□ □□□□□□□
00000220	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000230	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000240	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000250	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000260	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000270	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000280	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000290	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000002A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000002B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000002C0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000002D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000002E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000002F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000300	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000310	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000320	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

数据记录文件中存储的数据为小端模式

每条记录最大字节数 **length** 的地址为 0x020c, 长度 4 字节，以上图为例，0x020c 位置的值为 40 00 00 00，也就是 64（63+1，63 是当前绑定数据记录文件中每条记录最大字节数，1 是字符串结束符‘\0’）

字段数量 **dez** 的地址为 0x0214, 长度 4 字节，以上图为例，0x0214 位置的值为 03 00 00 00，也就是 3

最大行数 **maxval** 的地址为 0x0218, 长度 4 字节，以上图为例，0x0218 位置的值为 E8 03 00 00，也就是 1000

实际行数 **maxval** 的地址为 0x0208, 长度 4 字节，这个是在运行时才会获取，写入或者删除时才会改变

第一条记录的地址为 0x0400, 长度是每条记录最大字节数 length

Offset	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	CP 20936
000003F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000400	31	5E	B9	FE	B9	FE	5E	B9	FE	B9	FE	00	00	00	00	00	1^哈哈^哈哈□□□□□
00000410	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000420	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000430	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000440	32	5E	B9	FE	B9	FE	5E	B9	FE	B9	FE	00	00	00	00	00	2^哈哈^哈哈□□□□□
00000450	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000460	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000470	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000480	33	5E	B9	FE	B9	FE	5E	B9	FE	B9	FE	00	00	00	00	00	3^哈哈^哈哈□□□□□
00000490	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000004A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000004B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000004C0	34	5E	B9	FE	B9	FE	5E	B9	FE	B9	FE	00	00	00	00	00	4^哈哈^哈哈□□□□□
000004D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000004E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000004F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000500	35	5E	B9	FE	B9	FE	5E	B9	FE	B9	FE	00	00	00	00	00	5^哈哈^哈哈□□□□□
00000510	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000520	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000530	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000540	36	5E	B9	FE	B9	FE	5E	B9	FE	B9	FE	00	00	00	00	00	6^哈哈^哈哈□□□□□
00000550	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000560	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000570	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000580	37	5E	B9	FE	B9	FE	5E	B9	FE	B9	FE	00	00	00	00	00	7^哈哈^哈哈□□□□□
00000590	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000005A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000005B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000005C0	38	5E	B9	FE	B9	FE	5E	B9	FE	B9	FE	00	00	00	00	00	8^哈哈^哈哈□□□□□
000005D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000005E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000005F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000600	39	5E	B9	FE	B9	FE	5E	B9	FE	B9	FE	00	00	00	00	00	9^哈哈^哈哈□□□□□
00000610	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000620	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000630	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000640	31	30	5E	B9	FE	B9	FE	5E	B9	FE	B9	FE	00	00	00	00	10^哈哈^哈哈□□□□
00000650	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000660	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000670	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000680	31	31	5E	B9	FE	B9	FE	5E	B9	FE	B9	FE	00	00	00	00	11^哈哈^哈哈□□□□
00000690	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000006A0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000006B0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000006C0	31	32	5E	B9	FE	B9	FE	5E	B9	FE	B9	FE	00	00	00	00	12^哈哈^哈哈□□□□
000006D0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000006E0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
000006F0	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000700	31	33	5E	B9	FE	B9	FE	5E	B9	FE	B9	FE	00	00	00	00	13^哈哈^哈哈□□□□
00000710	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000720	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
00000730	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

```
//获取每条记录最大字节数 length
fs0.val=0x020c
fs0.read(length.val,0,4)
//获取字段数量dez
fs0.val=0x0214
fs0.read(dez.val,0,4)
//获取最大行数 maxval
fs0.val=0x0218
fs0.read(maxval.val,0,4)
//获取实际行数 maxval
fs0.val=0x0208
fs0.read(qty.val,0,4)
//读取第一条
fs0.val=0x0400
fs0.read(str.txt,0,length.val)
```

8.12.1.1 数据记录文件结构分析-样例工程下载

《文件浏览器 v3.0》演示工程下载

8.12.2 数据记录文件转换-python 代码

```
import tkinter as tk
from tkinter import ttk, filedialog, messagebox
import struct
import os

class DataToCsvConverter:
    def __init__(self, root):
        self.root = root
        self.root.title("Data文件转换工具")
        self.root.geometry("600x500")

        # 变量初始化
        self.file_path = tk.StringVar()
        self.output_path = tk.StringVar()
        self.encoding_var = tk.StringVar(value="GB2312")

        self.length_var = tk.StringVar(value="每条记录最大字节数: ")
        self.dez_var = tk.StringVar(value="字段数量: ")
        self.max_rows_var = tk.StringVar(value="最大行数: ")
        self.actual_rows_var = tk.StringVar(value="实际行数: ")

        self.create_widgets()

    def create_widgets(self):
        # 编码选择
        encoding_frame = ttk.LabelFrame(self.root, text="编码设置", padding=10)
        encoding_frame.pack(fill="x", padx=10, pady=5)

        ttk.Label(encoding_frame, text="选择编码:").grid(row=0, column=0, sticky="w")
        encoding_combo = ttk.Combobox(encoding_frame, textvariable=self.encoding_var,
                                      values=["GB2312", "utf-8", "GBK", "big5"], state=
↪"readonly")
        encoding_combo.grid(row=0, column=1, sticky="ew", padx=5)
        encoding_frame.columnconfigure(1, weight=1)

        # 文件选择区域
        file_frame = ttk.LabelFrame(self.root, text="文件选择", padding=10)
        file_frame.pack(fill="x", padx=10, pady=5)

        ttk.Label(file_frame, text="原始文件:").grid(row=0, column=0, sticky="w")
        ttk.Entry(file_frame, textvariable=self.file_path, state="readonly").grid(row=0,␣
↪column=1, sticky="ew", padx=5)
        ttk.Button(file_frame, text="选择文件", command=self.select_file).grid(row=0,␣
↪column=2, padx=5)

        ttk.Label(file_frame, text="输出文件:").grid(row=1, column=0, sticky="w")
        ttk.Entry(file_frame, textvariable=self.output_path, state="readonly").
↪grid(row=1, column=1, sticky="ew", padx=5)
        ttk.Button(file_frame, text="选择路径", command=self.select_output_path).
↪grid(row=1, column=2, padx=5)

        file_frame.columnconfigure(1, weight=1)

        # 文件信息显示
        info_frame = ttk.LabelFrame(self.root, text="文件信息", padding=10)
        info_frame.pack(fill="x", padx=10, pady=5)

        ttk.Label(info_frame, textvariable=self.length_var).pack(anchor="w")
        ttk.Label(info_frame, textvariable=self.dez_var).pack(anchor="w")
        ttk.Label(info_frame, textvariable=self.max_rows_var).pack(anchor="w")
```

(续下页)

(接上页)

```
        ttk.Label(info_frame, textvariable=self.actual_rows_var).pack(anchor="w")

        # 转换按钮
        button_frame = ttk.Frame(self.root)
        button_frame.pack(fill="x", padx=10, pady=10)

        ttk.Button(button_frame, text="开始转换", command=self.convert_file).pack(pady=5)

        # 日志区域
        log_frame = ttk.LabelFrame(self.root, text="日志", padding=10)
        log_frame.pack(fill="both", expand=True, padx=10, pady=5)

        self.log_text = tk.Text(log_frame, height=10, state="disabled")
        scrollbar = ttk.Scrollbar(log_frame, orient="vertical", command=self.log_text.
↪yview)
        self.log_text.configure(yscrollcommand=scrollbar.set)

        self.log_text.pack(side="left", fill="both", expand=True)
        scrollbar.pack(side="right", fill="y")

    def log_message(self, message):
        """添加日志信息"""
        self.log_text.config(state="normal")
        self.log_text.insert("end", message + "\n")
        self.log_text.see("end")
        self.log_text.config(state="disabled")
        self.root.update()

    def select_file(self):
        """选择原始文件"""
        file_path = filedialog.askopenfilename(
            title="选择.data文件",
            filetypes=[("Data files", "*.data"), ("All files", "*.*")]
        )
        if file_path:
            self.file_path.set(file_path)
            # 自动生成输出路径
            output_path = os.path.splitext(file_path)[0] + ".csv"
            self.output_path.set(output_path)
            self.read_file_info()

    def select_output_path(self):
        """选择输出文件路径"""
        if self.file_path.get():
            default_name = os.path.splitext(self.file_path.get())[0] + ".csv"
        else:
            default_name = "output.csv"

        output_path = filedialog.asksaveasfilename(
            title="保存为CSV文件",
            defaultextension=".csv",
            filetypes=[("CSV files", "*.csv"), ("All files", "*.*")],
            initialfile=default_name
        )
        if output_path:
            self.output_path.set(output_path)

    def read_file_info(self):
        """读取文件头信息"""
        if not self.file_path.get():
            return
```

(续下页)

(接上页)

```
        try:
            with open(self.file_path.get(), 'rb') as f:
                # 读取每条记录最大字节数 (0x020c)
                f.seek(0x020c)
                length_bytes = f.read(4)
                length = struct.unpack('<I', length_bytes)[0]

                # 读取字段数量 (0x0214)
                f.seek(0x0214)
                dez_bytes = f.read(4)
                dez = struct.unpack('<I', dez_bytes)[0]

                # 读取最大行数 (0x0218)
                f.seek(0x0218)
                max_rows_bytes = f.read(4)
                max_rows = struct.unpack('<I', max_rows_bytes)[0]

                # 读取实际行数 (0x0208)
                f.seek(0x0208)
                actual_rows_bytes = f.read(4)
                actual_rows = struct.unpack('<I', actual_rows_bytes)[0]

                # 更新界面显示
                self.length_var.set(f"每条记录最大字节数: {length}")
                self.dez_var.set(f"字段数量: {dez}")
                self.max_rows_var.set(f"最大行数: {max_rows}")
                self.actual_rows_var.set(f"实际行数: {actual_rows}")

                self.log_message(f"文件信息读取成功:")
                self.log_message(f"  - 记录最大字节数: {length}")
                self.log_message(f"  - 字段数量: {dez}")
                self.log_message(f"  - 最大行数: {max_rows}")
                self.log_message(f"  - 实际行数: {actual_rows}")

        except Exception as e:
            messagebox.showerror("错误", f"读取文件信息失败: {str(e)}")
            self.log_message(f"错误: 读取文件信息失败 - {str(e)}")

    def try_decode(self, data, encoding):
        """尝试用指定编码解码数据"""
        try:
            return data.decode(encoding), True
        except UnicodeDecodeError:
            return None, False

    def convert_file(self):
        """执行文件转换"""
        if not self.file_path.get():
            messagebox.showwarning("警告", "请先选择原始文件")
            return

        if not self.output_path.get():
            messagebox.showwarning("警告", "请先选择输出文件路径")
            return

        try:
            self.log_message("开始转换文件...")

            with open(self.file_path.get(), 'rb') as f:
                # 读取文件头信息
```

(续下页)

(接上页)

```
                f.seek(0x020c)
                length = struct.unpack('<I', f.read(4))[0]

                f.seek(0x0214)
                dez = struct.unpack('<I', f.read(4))[0]

                f.seek(0x0208)
                actual_rows = struct.unpack('<I', f.read(4))[0]

                self.log_message(f"转换参数: 记录长度={length}, 字段数={dez}, 实际行数=
↪{actual_rows}")

                # 定位到第一条记录 (0x0400)
                f.seek(0x0400)

                with open(self.output_path.get(), 'w', encoding=self.encoding_var.get(),␣
↪newline='') as csv_file:
                    records_processed = 0
                    encoding_issues = 0

                    for i in range(actual_rows):
                        # 读取一条记录
                        record_bytes = f.read(length)
                        if not record_bytes or len(record_bytes) < length:
                            self.log_message(f"警告: 记录 {i+1} 数据不完整或文件已结束")
                            break

                        # 找到字符串结束位置
                        try:
                            null_pos = record_bytes.index(b'\x00')
                            record_bytes = record_bytes[:null_pos]
                        except ValueError:
                            # 如果没有找到结束符，使用全部数据
                            pass

                        # 尝试使用用户选择的编码解码
                        selected_encoding = self.encoding_var.get()
                        record_text, success = self.try_decode(record_bytes, selected_
↪encoding)

                        # 如果解码失败，尝试其他编码
                        if not success:
                            encoding_issues += 1
                            self.log_message(f"记录 {i+1}: 编码 {selected_encoding}␣
↪解码失败，尝试备用编码...")

                            # 尝试其他编码
                            alternative_encodings = [enc for enc in ["GB2312", "utf-8",
↪"GBK", "big5"] if enc != selected_encoding]
                            for alt_enc in alternative_encodings:
                                record_text, success = self.try_decode(record_bytes, alt_
↪enc)
                                if success:
                                    self.log_message(f"记录 {i+1}: 使用备用编码 {alt_enc}
↪ 成功解码")
                                    break

                            # 如果所有编码都失败，使用错误处理
                            if not success:
                                try:
                                    record_text = record_bytes.decode(selected_encoding,␣
```

(续下页)

(接上页)

```
↪errors='replace')
                                    self.log_message(f"记录 {i+1}: 使用错误替换方式解码")
                                except:
                                    record_text = "解码失败"
                                    self.log_message(f"记录 {i+1}: 所有解码方式都失败")

                            # 替换^为英文逗号
                            record_text = record_text.replace('^', ',')

                            # 写入CSV文件
                            csv_file.write(record_text + '\n')
                            records_processed += 1

                            # 每处理100条记录显示进度
                            if (i + 1) % 100 == 0:
                                self.log_message(f"已处理 {i + 1} 条记录...")

                    # 转换完成统计
                    self.log_message(f"转换完成!")
                    self.log_message(f"共处理记录数: {records_processed}")
                    if encoding_issues > 0:
                        self.log_message(f"编码问题记录数: {encoding_issues}")

                    messagebox.showinfo("成功",
                                        f"文件转换完成!\n"
                                        f"输出文件: {self.output_path.get()}\n"
                                        f"处理记录数: {records_processed}\n"
                                        f"编码问题记录: {encoding_issues}")

            except Exception as e:
                error_msg = f"转换失败: {str(e)}"
                self.log_message(f"错误: {error_msg}")
                messagebox.showerror("错误", error_msg)

def main():
    root = tk.Tk()
    app = DataToCsvConverter(root)
    root.mainloop()

if __name__ == "__main__":
    main()
```

8.12.3 数据记录文件转换-js 代码

```
<!DOCTYPE html>
<html lang="zh-CN">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Data转CSV转 换 器</title>
    <style>
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
            font-family: 'Segoe UI', Arial, sans-serif;
        }
        body {
            background-color: #f0f2f5;
            padding: 2rem;
        }
        .container {
            max-width: 800px;
            margin: 0 auto;
            background: white;
            padding: 2rem;
            border-radius: 10px;
            box-shadow: 0 4px 12px rgba(0,0,0,0.1);
        }
        h1 {
            text-align: center;
            color: #1a73e8;
            margin-bottom: 1.5rem;
            font-weight: 600;
        }
        .form-group {
            margin-bottom: 1.2rem;
        }
        label {
            display: block;
            margin-bottom: 0.5rem;
            color: #5f6368;
            font-weight: 500;
        }
        .input-group {
            display: flex;
            gap: 0.5rem;
            align-items: center;
        }
        input[type="text"] {
            flex: 1;
            padding: 0.75rem;
            border: 1px solid #dadce0;
            border-radius: 4px;
            font-size: 1rem;
            transition: border 0.2s;
        }
        input[type="text"]:focus {
            outline: none;
            border-color: #1a73e8;
        }
        button {
            padding: 0.75rem 1.25rem;
            border: none;
```

(续下页)

(接上页)

```
            border-radius: 4px;
            background: #1a73e8;
            color: white;
            font-size: 1rem;
            cursor: pointer;
            transition: background 0.2s;
        }
        button:hover {
            background: #1557b0;
        }
        button#convertBtn {
            background: #34a853;
            width: 100%;
            padding: 0.9rem;
            font-size: 1.05rem;
            margin-top: 0.5rem;
        }
        button#convertBtn:hover {
            background: #2d8643;
        }
        .info-section {
            margin: 1.5rem 0;
            padding: 1rem;
            background: #f8f9fa;
            border-radius: 6px;
            border-left: 3px solid #1a73e8;
        }
        .info-title {
            font-weight: 600;
            color: #202124;
            margin-bottom: 0.8rem;
            font-size: 1.05rem;
        }
        .info-grid {
            display: grid;
            grid-template-columns: repeat(2, 1fr);
            gap: 0.8rem;
        }
        .info-item {
            display: flex;
            justify-content: space-between;
            padding: 0.4rem 0;
        }
        .info-key {
            color: #5f6368;
        }
        .info-value {
            color: #202124;
            font-weight: 500;
        }
        .status {
            margin-top: 1rem;
            text-align: center;
            padding: 0.75rem;
            border-radius: 4px;
            background: #f1f3f4;
            color: #5f6368;
        }
        .error {
            background: #fce8e6;
            color: #d93025;
```

(续下页)

(接上页)

```
        }
        .success {
            background: #e6f4ea;
            color: #137333;
        }
        select {
            width: 100%;
            padding: 0.75rem;
            border: 1px solid #dadce0;
            border-radius: 4px;
            font-size: 1rem;
            background-color: white;
            transition: border 0.2s;
        }
        select:focus {
            outline: none;
            border-color: #1a73e8;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>Data转CSV转换器</h1>

        <!-- 编码选择 -->
        <div class="form-group">
            <label for="encodingSelect">选择编码格式</label>
            <select id="encodingSelect">
                <option value="gb2312">GB2312</option>
                <option value="utf-8">UTF-8</option>
                <option value="gbk">GBK</option>
                <option value="big5">Big5</option>
            </select>
        </div>

        <!-- 输入文件选择 -->
        <div class="form-group">
            <label for="inputFilePath">输入文件 (*.data) </label>
            <div class="input-group">
                <input type="text" id="inputFilePath" placeholder="未选择文件" readonly>
                <button id="browseBtn">浏览</button>
                <input type="file" id="fileInput" accept=".data" style="display: none;">
            </div>
        </div>

        <!-- 输出文件路径 -->
        <div class="form-group">
            <label for="outputFilePath">输出文件 (*.csv) </label>
            <input type="text" id="outputFilePath" placeholder="自动生成输出路径">
        </div>

        <!-- 文件信息显示区域 -->
        <div class="info-section">
            <div class="info-title">文件结构信息</div>
            <div class="info-grid">
                <div class="info-item">
                    <span class="info-key">每条记录最大字节数: </span>
                    <span class="info-value" id="recordLength">--</span>
                </div>
                <div class="info-item">
                    <span class="info-key">字段数量: </span>
```

(续下页)

(接上页)

```
                <span class="info-value" id="fieldCount">--</span>
            </div>
            <div class="info-item">
                <span class="info-key">最大行数：</span>
                <span class="info-value" id="maxRows">--</span>
            </div>
            <div class="info-item">
                <span class="info-key">实际行数：</span>
                <span class="info-value" id="actualRows">--</span>
            </div>
        </div>
    </div>

    <!-- 转换按钮和状态 -->
    <button id="convertBtn">开始转换</button>
    <div class="status" id="status">就绪</div>
</div>

<script>
    // 全局变量存储文件数据和解析信息
    let fileData = null;
    let fileInfo = {
        recordLength: 0,
        fieldCount: 0,
        maxRows: 0,
        actualRows: 0
    };

    // DOM元素
    const browseBtn = document.getElementById('browseBtn');
    const fileInput = document.getElementById('fileInput');
    const inputFilePath = document.getElementById('inputFilePath');
    const outputFilePath = document.getElementById('outputFilePath');
    const encodingSelect = document.getElementById('encodingSelect');
    const convertBtn = document.getElementById('convertBtn');
    const statusEl = document.getElementById('status');

    // 信息显示元素
    const recordLengthEl = document.getElementById('recordLength');
    const fieldCountEl = document.getElementById('fieldCount');
    const maxRowsEl = document.getElementById('maxRows');
    const actualRowsEl = document.getElementById('actualRows');

    // 浏览文件按钮点击事件
    browseBtn.addEventListener('click', () => {
        fileInput.click();
    });

    // 文件选择事件
    fileInput.addEventListener('change', (e) => {
        const file = e.target.files[0];
        if (!file) return;

        // 显示输入文件路径
        inputFilePath.value = file.name;

        // 自动生成输出文件路径
        const outputName = file.name.replace(/\.data$/, '.csv');
        outputFilePath.value = outputName;

        try {
```

(续下页)

(接上页)

```
            // 读取文件数据
            const reader = new FileReader();
            reader.onload = (event) => {
                fileData = new Uint8Array(event.target.result);
                // 解析文件信息
                parseFileInfo();
                statusEl.textContent = "文件加载成功";
                statusEl.className = "status success";
            };
            reader.onerror = () => {
                throw new Error("文件读取失败");
            };
            reader.readAsArrayBuffer(file);
        } catch (error) {
            statusEl.textContent = `错误：${error.message}`;
            statusEl.className = "status error";
            resetFileInfo();
        }
    });

    // 解析文件信息（从指定地址读取数据）
    function parseFileInfo() {
        try {
            // 每条记录最大字节数：0x020c位置，4字节（小端）
            fileInfo.recordLength = readUint32LE(0x020c);
            // 字段数量：0x0214位置，4字节（小端）
            fileInfo.fieldCount = readUint32LE(0x0214);
            // 最大行数：0x0218位置，4字节（小端）
            fileInfo.maxRows = readUint32LE(0x0218);
            // 实际行数：0x0208位置，4字节（小端）
            fileInfo.actualRows = readUint32LE(0x0208);

            // 更新界面显示
            recordLengthEl.textContent = fileInfo.recordLength;
            fieldCountEl.textContent = fileInfo.fieldCount;
            maxRowsEl.textContent = fileInfo.maxRows;
            actualRowsEl.textContent = fileInfo.actualRows;
        } catch (error) {
            statusEl.textContent = `解析错误：${error.message}`;
            statusEl.className = "status error";
            resetFileInfo();
        }
    }

    // 从指定偏移量读取4字节小端无符号整数
    function readUint32LE(offset) {
        if (!fileData || offset + 4 > fileData.length) {
            throw new Error("文件数据不足或格式错误");
        }
        return (
            fileData[offset] +
            (fileData[offset + 1] << 8) +
            (fileData[offset + 2] << 16) +
            (fileData[offset + 3] << 24)
        );
    }

    // 重置文件信息显示
    function resetFileInfo() {
        fileInfo = {
            recordLength: 0,
```

(续下页)

(接上页)

```
            fieldCount: 0,
            maxRows: 0,
            actualRows: 0
        };
        recordLengthEl.textContent = "--";
        fieldCountEl.textContent = "--";
        maxRowsEl.textContent = "--";
        actualRowsEl.textContent = "--";
        fileData = null;
    }

    // 转换按钮点击事件
    convertBtn.addEventListener('click', async () => {
        if (!fileData) {
            statusEl.textContent = "错误：请先选择有效的.data文件";
            statusEl.className = "status error";
            return;
        }

        if (!outputFilePath.value) {
            statusEl.textContent = "错误：请指定输出文件路径";
            statusEl.className = "status error";
            return;
        }

        try {
            statusEl.textContent = "正在转换...";
            statusEl.className = "status";
            convertBtn.disabled = true;

            // 执行转换
            await convertDataToCsv();

            statusEl.textContent = "转换成功！文件已下载";
            statusEl.className = "status success";
        } catch (error) {
            statusEl.textContent = `转换错误：${error.message}`;
            statusEl.className = "status error";
        } finally {
            convertBtn.disabled = false;
        }
    });

    // 核心转换逻辑
    async function convertDataToCsv() {
        const { recordLength, fieldCount, actualRows } = fileInfo;
        const encoding = encodingSelect.value;
        const firstRecordOffset = 0x0400;
        const fieldLength = recordLength / fieldCount;

        if (!Number.isInteger(fieldLength)) {
            throw new Error("记录长度不能被字段数量整除，文件结构异常");
        }

        // 存储CSV内容
        const csvLines = [];

        // 遍历所有实际记录
        for (let row = 0; row < actualRows; row++) {
            const rowData = [];
            // 计算当前记录的起始偏移量
```

(续下页)

(接上页)

```
                const recordOffset = firstRecordOffset + row * recordLength;

                // 遍历每个字段
                for (let field = 0; field < fieldCount; field++) {
                    const fieldOffset = recordOffset + field * fieldLength;
                    // 读取字段数据（到\0结束或字段长度结束）
                    let fieldBytes = [];
                    for (let i = 0; i < fieldLength; i++) {
                        const byte = fileData[fieldOffset + i];
                        if (byte === 0x00) break; // 遇到字符串结束符'\0'停止
                        fieldBytes.push(byte);
                    }

                    // 解码字段（处理不同编码）
                    let fieldStr = await decodeBytes(fieldBytes, encoding);
                    // 替换^为英文逗号
                    fieldStr = fieldStr.replace(/\^/g, ',');
                    // 去除前后空格
                    fieldStr = fieldStr.trim();

                    rowData.push(fieldStr);
                }

                // 添加当前行到CSV（不使用双引号包裹）
                csvLines.push(rowData.join(','));
            }

            // 生成CSV字符串
            const csvContent = csvLines.join('\n');
            // 下载文件
            downloadFile(csvContent, outputFilePath.value, 'text/csv');
        }

        // 解码字节数组（支持多种编码）
        async function decodeBytes(bytes, encoding) {
            // 处理编码名称映射（TextDecoder使用的名称可能不同）
            const encodingMap = {
                'gb2312': 'gbk',  // 浏览器通常用gbk兼容gb2312
                'utf-8': 'utf-8',
                'gbk': 'gbk',
                'big5': 'big5'
            };

            const decoder = new TextDecoder(encodingMap[encoding] || encoding, { fatal:␣
↪false });
            return decoder.decode(new Uint8Array(bytes));
        }

        // 下载文件
        function downloadFile(content, fileName, mimeType) {
            const blob = new Blob([content], { type: mimeType });
            const url = URL.createObjectURL(blob);
            const a = document.createElement('a');
            a.href = url;
            a.download = fileName;
            document.body.appendChild(a);
            a.click();
            setTimeout(() => {
                document.body.removeChild(a);
                URL.revokeObjectURL(url);
            }, 100);
```

(续下页)

(接上页)

```
        }
    </script>
</body>
</html>
```

8.12.4 下载《数据记录文件解析》例程

《数据记录文件解析》下载链接：

数据记录文件解析：《数据记录文件解析》演示工程下载

文件浏览器 v2.0:《文件浏览器 v3.0》演示工程下载

9 资料下载

9.1 上位机下载

解压缩工具 7zip 下载 (page 1106)

重要： 1.67 更新说明：

1. 优化 flash 驱动。
2. 增加 T1 系列 2.0 寸产品驱动。
3. 优化 utf8 字库中部分泰语文字的显示异常。
4. 修复已知 bug。

1.67.1
第一次发布

1.67.2
优化 X 系列电容触摸滑动翻页流畅度。

1.67.3
1. 增加 2.1 寸,2.8 寸圆屏驱动。
2. 修复已知 bug。

1.67.4
1. 优化圆屏 2.1 寸和 2.8 寸驱动。

1.67.5
1. 优化串口通讯底层驱动。

1.67.6
1. 修复部分电脑输入法冲突问题。

9.1.1 旧版本上位机下载

9.1.1.1 下载 V1.67.6 USART HMI 软件（最新）

1. 修复部分电脑输入法冲突问题。

9.1.1.2 下载 V1.67.6 USART HMI 软件 (免安装版)

某些用户安装时会报错，可以使用免安装版

9.1.1.3 下载 V1.67.5 USART HMI 软件

1. 优化串口通讯底层驱动。

9.1.1.4 下载 V1.67.5 USART HMI 软件 (免安装版)

某些用户安装时会报错，可以使用免安装版

9.1.1.5 下载 V1.67.3 USART HMI 软件

1. 增加 2.1 寸,2.8 寸圆屏驱动。
2. 修复已知 bug。

9.1.1.6 下载 V1.67.3 USART HMI 软件 (免安装版)

某些用户安装时会报错，可以使用免安装版

9.1.1.7 下载 V1.67.2 USART HMI 软件

1. 优化 X 系列电容触摸滑动翻页流畅度。

9.1.1.8 下载 V1.67.2 USART HMI 软件 (免安装版)

某些用户安装时会报错，可以使用免安装版

9.1.1.9 下载 V1.67.1 USART HMI 软件

1. 优化 4.3 高分竖屏底层驱动。
2. 优化 X5 系列 NAND FLASH 底层驱动。
3. 增加 T1 系列 2.0 寸产品驱动。

9.1.1.10 下载 V1.67.1 USART HMI 软件 (免安装版)

某些用户安装时会报错，可以使用免安装版

9.1.1.11 下载 V1.66.3 USART HMI 软件

1. 优化 4.3 高分竖屏底层驱动。
2. 优化 X5 系列 NAND FLASH 底层驱动。
3. 增加 T1 系列 2.0 寸产品驱动。

9.1.1.12 下载 V1.66.3 USART HMI 软件 (免安装版)

某些用户安装时会报错，可以使用免安装版

9.1.1.13 下载 V1.66.2 USART HMI 软件

1. 优化 T1 系列产品电容触摸性能。
2. 增加 X2 系列 6.2 寸产品型号。

9.1.1.14 下载 V1.66.1 USART HMI 软件

1.66 更新说明:
1. 优化新品 T1 系列 4.3 寸屏显示效果。
2. 修复已知 bug。
1.66.1: 第一次发布

9.1.1.15 下载 V1.65 USART HMI 软件

1.65 更新说明:
1. 增加 T1 系列 4.3 寸产品型号。
2.X 系列优化图片无损压缩算法。
3. 取消设备开机固定发送开机信息，转为 Program.s 中由代码发送开机信息（用户可自行删除）。
1.65.5
增加 T1 系列 5 寸屏驱动。
1.65.4
优化 T1 系列 4.3 寸玻璃驱动。
1.65.3
优化上位机定时器的计时准确度。
修复 HMI 文件小于 7M 时部分文件受损的 bug。
1.65.2
修复部分型号触摸长按异常。
新品 T1 4.3 寸增加电容触摸驱动。
1.65.1 更新
修复大于 2G 的 HMI 文件打开异常的问题。
优化 X2 系列小图片（小于 33*33 分辨率的图片）的无损压缩效率。
1.65.0 更新

第一次发布

9.1.1.16 下载 V1.64 USART HMI 软件

1. 增加 X2 系列产品支持。

2.X 系列增加图片缓冲。

3. 增加 X2 系列的 beep 指令。

4. 优化 x 系列性能。

5. 修复 X 系列部分产品随机数异常。

6. 修复 k0 系列屏幕前初始化时间过长会导致屏闪。

7. 修复 T0/K0 在极端电源情况下触摸失效的 bug。

8. 修复定时器定时不准确。

9. 修复其他已知 bug。

1.64.1

第一次发布

9.1.1.17 下载 V1.63 USART HMI 软件

本次升级内容:

1. 增加触摸捕捉控件。

2. 优化 X 系列电阻触摸精度。

3. 增加新主控 MCU 支持。

4. 修复已知 bug。

1.63.1

第一次发布

9.1.1.18 下载 V1.61 USART HMI 软件

本次升级内容:

1. 增加字符串分割指令:spstr(用于按指定关键字分割字符串)。

2. 增加音频控件 (FLASH 内部音频和 SD 卡外部音频均可播放)。

3. 增加 1.8 寸产品线型号。

4. 优化屏幕联机时的波特率识别准确度。

5. 修复 big5 编码下部分字符和转义字符冲突问题。

6. 修复 T0 系列 3.5 寸以下在电源不稳定的情况下可能导致 Flash 数据损坏的风险。

子版本修复记录:

1.61.6

修复 X 系列电阻触摸屏长时间工作偶尔出现触摸失效的 BUG。

1.61.5

1. 滑动翻页的时候，控件按下事件和滑动事件两种操作屏幕自动二选一，不再同时执行。

2. 系统变量 bkcmd=3 的时候，get,prints,printh 指令只返回相应的变量/常量内容，不会再返回执行成功的回应，其他指令不变。

3. 修复波形控件宽度小于 50 时显示出现的 bug。

4.covx 转换 HEX 数据的时候, 手动输入转换字节数量为基数时将会报错。

5. 资源文件 ID 无效的赋值操作不再执行。

1.61.3

修复动画控件被隐藏掉 vid 属性的 bug。

1.61.2

修复 T0 系列工程下载 BUG。

1.61.1

第一次发布。

9.1.1.19 下载 V1.60 USART HMI 软件

本次升级内容:

1. 视频控件增加播放 SD 卡视频功能 (设置视频控件的 from 属性即可)

2. 增加启动文件概念: 可以在启动文件中定义任意数量的全局变量或写入各种初始化数据代码

3. 所有字库增加指定字符功能

4. 增加数据 CRC 校验功能, 此功能涉及到以下五个对象:

系统变量:crcval crc 校验结果（只可获取不可设置, 使用前请先用 crcrest 指令复位初始值）

指令：crcrest 复位 crc 初始值 (复位之后，可使用 crcputs 或 crcputh 或 crcputu 校验指定数据, 检验完毕读取系统变量 crcval 获得校验结果)

指令：crcputs crc 校验一个变量/常量

指令：crcputh crc 校验一组 Hex

指令：crcputu crc 校验一段串口缓冲区数据 (recmod=1 时有效)

5. 版本定义规则从原来的两位版本号更新为 3 位版本号, 分别表示: 主版本. 子版本. 修订号。如: 1.60.8 【主版本 1，子版本 60，修订号 8】

主版本变更：一般表示底层做了重大修改

子版本变更：一般代表有新增功能 (如新增控件或指令)

修订号变更：一般代表 bug 修复, 没有任何功能变化或底层改动

子版本修复记录:

1.60.2

修复单字节编码字库显示不全的 bug

修复 X 系列跨页面设置坐标导致触摸出现问题的 bug

1.60.1

第一次发布

9.1.1.20 下载 V0.59 USART HMI 软件

本次升级内容:

1. 增加 T1 系列支持

2. 数据记录控件增加 delete 方法 (用于删除一条或多条记录)

3. 上位软件增加图片尺寸编辑

4.X 系列提高图片压缩质量

5. 编辑界面增加控件组合功能

6. 工程下载界面增加下载波特率记忆功能

7.T1 系列增加睡眠模式设定, 支持普通睡眠和极低电流的深度睡眠 (目前仅 T1 系列支持深度睡眠, 其他系列仅支持普通睡眠)

8. 修复部分电脑兼容性问题导致退出软件时系统报错的情况

9.1.1.21 下载 V0.57 USART HMI 软件

本次升级内容:

升级 X3 系列 4.3 寸屏驱动。

优化指令 CRC 校验速度。

9.1.1.22 下载 V0.56 USART HMI 软件

本次升级内容:

优化 X3,X5 系列电阻触摸屏的触摸灵敏度。

曲线控件增加背景透明选项 (仅 X 系列支持)。

菜单栏增加工程导入选项。

属性栏文本属性编辑增加多行输入功能。

增加指令 CRC 校验。点击此处查看说明文档。

SD 卡图片播放支持 (仅 X3,X5 系列)。

新增控件”外部图片”, 用于显示内存中或 SD 卡中的图片 (仅 X3,X5 系列)。

运行中串口传输图片文件到内存或 SD 卡 (仅 X3,X5 系列)。, 点击此处查看说明文档。

新增指令:(仅 X3,X5 系列支持):

wrfile 串口透传文件

delfile 删除文件

refile 重命名文件

findfile 查找文件

rdfile 透传读文件

点击此处查看说明文档

9.1.1.23 下载 V0.55 USART HMI 软件

本次升级内容:

1. 全系列升级抗锯齿字库 (包括字体抗锯齿，非等宽字符，字符占位延伸)

2. 上位编辑界面鼠标拖拽改变控件尺寸增加 6 种方式 (上拖高度，左拖宽度，左上角拖宽 && 高，左下角拖宽 && 高，右上角拖宽 && 高, 右下角拖宽 && 高)

3. 增加控件锁定功能 (锁定后鼠标不能拖动控件位置)

4. 软件启动界面的项目打开历史记录增加右键菜单 (允许删除指定的记录或失效记录或全部历史记录)

5. 增加虚拟浮点数控件

6. 编辑界面鼠标选中控件增加防抖算法

7. 修复新产品 X 系列的一些小 BUG

9.1.1.24 下载 V0.54 USART HMI 软件

本次升级内容:

1、if/while 语句增加 && 和 || 支持啦!!!

```
if(n0.val==3||n1.val==4||n2.val==5)
{
    if(t0.txt=="abc"&&t1.txt!="dd"&&n3.val==12)
    {
        page 1
    }
}
```

注意：if 语句内部不支持括号，比如:if((t0.txt==” a” ||t0.txt==” b”)&&t1.txt==” 1”) 这样是不支持的！还有就是不要出现多余的空格。

2、串口波特率增加到 912600

3、优化 flash 读数据效率

4、新增页面编辑区缩放功能

5、页面编辑区控件允许拖出显示区域外

6、串口下载资源文件时自动跳过相同资源文件

7、曲线控件支持全局属性，实现离开页面后曲线数据不丢失

【X 系列智能串口屏为动而生】

X3,X5 系列产品新增功能

1、运行中控件图层保持（每个控件是一个图层，运行中永远保持各自的层级关系）

2、图片压缩存储（压缩质量可设置）

3、带透明信息的 png 图片支持

4、多种页面切换特效

5、滑动翻页 (上下左右 4 个方向滑动可分别设置不同目标页面)

6、控件加载特效支持 (多种飞入方式和优先级)

7、运行中可通过指令随意修改控件坐标 (允许负数坐标，允许超出显示区域的坐标)

8、增加 move 指令，指定起始坐标和结束坐标实现控件自动移动

9、增加 drag 属性实现运行中控件可以触摸拖动位置

10、所有控件支持半透明设置 (128 级，可运行中随意改变)

11、2 路音频播放通道（2 路通道可同时播放不同音乐）

12、每个页面可以最多有 6 个视频控件同时播放不同视频内容

13、每个页面最多可以有 16 个动画控件

14、增加状态开关控件

9.1.1.25 下载 V0.52 USART HMI 软件

本次升级内容:

1、修复 0.51 版本新增功能的部分 bug

9.1.1.26 下载 V0.51 USART HMI 软件

本次升级内容:

1、增加二维码控件。

2、增加实时触摸坐标读取系统变量:tch0-tch3。

3、增加屏幕地址设置功能（操作方法请参看系统变量:addr）。

4、增加数值类型的变量赋值可以使用 Hex 的写法 (比如 dim=100,dim=0x64, 两句表达的是同一个意思，前一句是十进制写法，

后一句是十六进制写法)

5、数字控件增加格式化类型属性: 可以选择数字格式，货币格式，Hex 格式，3 种显示格式。

6、增加串口输出指令 prints(用于替换原来的 print 指令，功能更强, 可以指定发送变量的数据字节数，详情请参阅指令集)。

7、增加变量类型转换指令 covx(用于替换原来的 cov 指令, 功能更强, 可以申明数字类型，详情请参阅指令集)。

8、增加全行业唯一支持的高级串口通讯功能-串口主动数据解析 (此功能可以实现自定义指令协议与屏幕通讯，也可以实现跟 PLC 的通讯，

可以实现跟任何特殊协议的串口设备通讯，具体请参看”HMI 高级应用功能与特殊指令集”文档)。

9.1.1.27 下载 V0.50 USART HMI 软件

本次升级内容:

1、修复部分老产品出现”非法型号”的 bug。

2、升级系统固件加密方式。

3、升级新款 3.2 寸屏幕驱动。

9.1.1.28 下载 V0.49 USART HMI 软件

本次升级内容:

1、增加串口数据自动唤醒功能 (详情请参看系统变量 usup)。

2、增加工程文件设置打开密码功能（打开工程后，点击“设备”，在点击“工程”即可设置)。

3、上位软件模拟器增加掉电存储空间用户存储功能。

4、修复切图失败的时候导致的花屏问题。

5、上位软件模拟器界面增加 16 进制数据输入功能。

9.1.1.29 下载 V0.47 USART HMI 软件

本次升级内容:

1、修复图片数量过多以后 (200 多张) 无法浏览所有图片的 bug。

2、修复软件联机信息中无法识别电容触摸的 bug。

3、优化电容触摸灵敏度。

4、增加串口缓冲区溢出报错数据:0x24(详情请参看数据返回格式说明文档)

9.1.1.30 下载 V0.46 USART HMI 软件

本次升级内容:

1、修复部分底层驱动 bug

9.1.1.31 下载 V0.45 USART HMI 软件

本次升级内容:

1、增加曲线控件数据添加时使用变量的功能。

2、增加繁体字库编码支持 (即 big5 编码)。

3、增加字符串截取指令 (即 substr 指令, 详情请参看最新指令集)

4、上位编辑界面增加更多对齐按钮（使水平间距相同，使垂直间距相同，增加水平间距等…）。

5、修复文本控件最后一个字符为换行符 (即:r) 时显示异常的 bug。

9.1.1.32 下载 V0.44 USART HMI 软件

本次升级内容:

1、增加电容触摸驱动。

2、优化上位软件内存占用。

9.1.1.33 下载 V0.42 USART HMI 软件

本次升级内容:

1、增加模拟调试界面的代码提示功能

2、增加控件颜色属性直接输入颜色值

3、增加属性编辑界面中颜色属性的颜色预览功能

4、增加系统内置虚拟键盘 (文本控件和数字控件可以在属性区直接设置键盘绑定)

5、增加页面导入导出功能

6、增加页面锁定功能

7、修复电脑无网络时软件占用 CPU 资源过多的 bug

8、修复颜色选择对话框不能保存自定义颜色的 bug

9、修复数值为负数时强制转换出来的字符串中多出一个”0”的 bug

10、修复软件重置窗体布局时出现的界面异常 (工具条被拖到较远处时)

9.1.1.34 下载 V0.41 USART HMI 软件

本次升级内容:

1、增加代码提示功能

2、软件设计界面窗口布局自动保存

3、增加系统变量:dp(表示当前页面 ID)

4、修复工程属性总数超过 65535 时出现的 bug

9.1.1.35 下载 V0.40 USART HMI 软件

本次升级内容:

1、修复页面初始化代码中出现 delay 时的黑屏 bug

9.1.1.36 下载 V0.39 USART HMI 软件

本次升级内容:

1、增加文本控件显示密码功能 (即显示为 *)

2、增加文本控件的风格选择（3D，平面，带边框）

3、增加按钮控件的风格选择（3D，平面，带边框）

4、增加数字控件的风格选择（3D，平面，带边框）

5、增加休眠唤醒后刷新页面的设置 (请参看系统变量 wup 含义：wup=255(上电默认)：唤醒后刷新休眠前的页面,

wup=0-254：唤醒后刷新指定页面 (wup 的值为页面序号)

6、休眠状态下可以执行如下指令：get,print,printh 除以上三条指令外，休眠状态下还可以执行 sleep=1，wup=XXX 的赋值语句，

并且支持上位软件联机

7、从此版本开始，在运行中修改控件的任何属性都将自动刷新，不再需要使用手动刷新指令。(原来的版本在修改非加粗属性时需要手动刷新)

8、优化指令解析速度，解析速度提升 20%

9、增加代码编辑区指令和系统变量的高亮显示

10、修复文本控件背景色改变时偶尔出现未知色块的 bug

11、修复控件触摸禁用后依然会响应发送键值的 bug(弹起发送键值)

9.1.1.37 下载 V0.37 USART HMI 软件

本次升级内容:

1、双态按钮增加文本显示

2、增加控件事件激活指令 (click)

3、备份老版本文件时文件名中增加当前版本号

4、优化控件隐藏/显示性能

5、优化字符显示速度

6、修复 picq 指令异常的 bug

7、修复显示方向为 270 度时文本控件背景切图的 bug

9.1.1.38 下载 V0.36 USART HMI 软件

本次升级内容:

1、优化 USART HMI 上位软件图片显示速度和页面刷新速度（上位模拟图片及页面刷新速度提升 50 倍）

2、增加滚动文本控件

3、数字控件和变量控件增加负数支持

4、增加字符串变量的字符数检测指令 strlen

5、增加 xpic 指令 (高级切图)

6、文本控件增加是否需要自动换行的设置

7、修复增强型系列不能触摸自动唤醒的 bug

8、修复部分屏幕按键在触摸边缘时触摸卡死的 bug

9、修复老版本在电源出问题时偶尔可能出现的型号非法的 bug

9.1.1.39 下载 V0.35 USART HMI 软件

本次升级内容:

1、增加增强型大屏驱动 (3.5 寸及以上尺寸)

2、优化 3.2 寸屏的显示色彩

3、增加 RTC 的”星期”系统变量:rtc6

4、修复内存使用过多的时候大字体出现乱码的 bug

5、修复上位模拟器运行不支持控件暂停刷新功能的 bug

6、ref_stop 指令修改了用法，执行 ref_stop 后，ref 指令也不会立刻刷新，直到执行 ref_star 指令的时候统一刷新

9.1.1.40 下载 V0.34 USART HMI 软件

本次升级内容:

1、优化用户代码区的代码复制粘贴操作

2、修复初始化事件中休眠后不能退出的 bug

3、修复图片不能全删的 bug

4、修复滑块不能设置 hig 属性的 bug

5、修复弹起/按下发送键值动作被交换的 bug

6、串口下载速度提示更精确

7、修复 else 内部嵌入 if 时的冲突 bug

8、调试页面增加校准设备 RTC 功能

9、模拟器支持虚拟 RTC 设置，并且不影响电脑系统时间

10、修复增强型系列用户存储数据透传的 bug

11、优化资源文件合法性检测以避开电源有问题时出现的 data error 提示

9.2 开发文档下载

离线 html 开发资料下载（适合电脑端阅读，解压后使用浏览器打开 index.html）：离线 html 文档

离线 pdf 开发资料下载（适合电脑/手机/pad/kindle 阅读）：离线 pdf 文档

mobi 格式在线电子书（仅适用于 kindle 阅读）：tjcwiki.mobi 文档

本次资料中心更新时间：2026-05-13 17:26:41 ，版本号：1.1.0-2026-05-13 17:26:41

9.3 常用工具下载

9.3.1 sscom5.13.1 串口调试助手下载

说明：sscom5.13.1，比较常用的串口调试助手，调试淘晶驰串口屏必须使用 5.13.1 这个版本

sscom 串口助手下载

sscom 串口调试助手使用说明

串口助手软件 *(sscom)* 和屏幕联调 (page 233)

串口助手软件 *(sscom)* 和串口屏模拟器联调 *1* (page 240)

串口助手软件 *(sscom)* 和串口屏模拟器联调 *2* (page 247)

9.3.2 虚拟串口工具 vspd 下载

此工具可以成对添加串口, 以下图为例, 添加了一对虚拟串口 COM1 和 COM2

此时 com1 发送的消息会被转发到 com2, com2 发送的消息会被转发到 com1

《虚拟串口工具 vspd》

9.3.3 TFT 文件下载助手 (TFTFileDownload)

说明：此工具专用于 tft 文件的串口下载，没有其他功能。

使用方法：*如何使用 TFT 文件下载助手 (TFTFileDownload) 下载工程* (page 1435)

TFTFileDownload 软件下载

TFTFileDownload 工具是淘晶驰开源的下载工具，您可以下载源码进行二次开发，此工程基于 visual studio 进行开发，使用的语言为 c#，需要您熟练掌握 c# 语言，淘晶驰不对 TFTFileDownload 二次开发提供技术支持

TFTFileDownload 源代码下载

如果需要自行开发对应的下载工具，详细的下载协议请参考*HMI 下载协议详解/OTA 升级* (page 1006)

相关链接

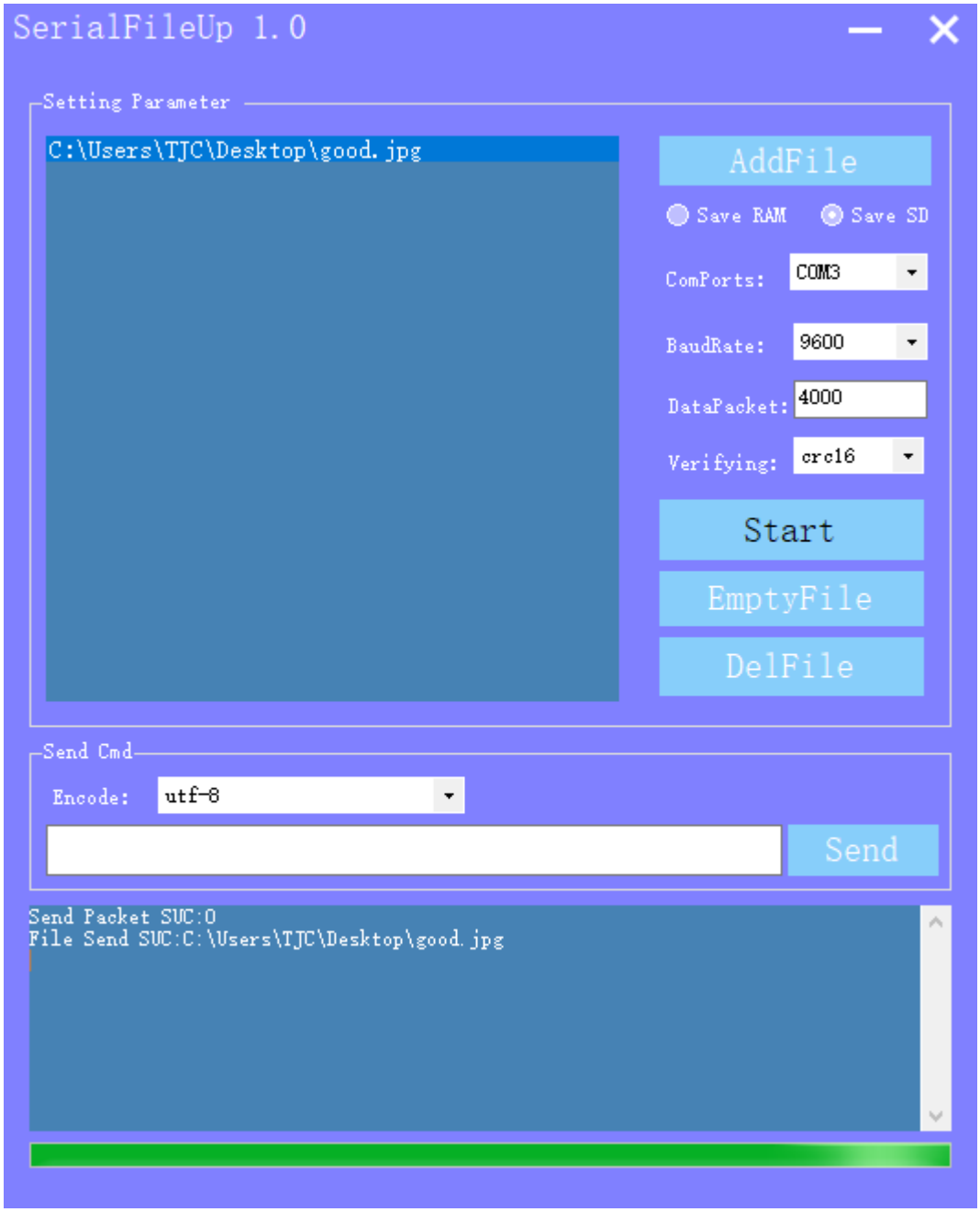
TFT 文件如何下载到串口屏中 (page 1405)

如何输出 TFT 生产文件 (page 155)

9.3.4 文件透传工具 (SerialFileUp)

说明：此工具用于电脑通过串口透传文件到屏幕的内存中或 SD 卡，根据高级应用文档说明使用 VS2008 开发，开发语言为 C#, 源代码完全开源。

以我这里测试为例，实际使用，导入图片文件 good.jpg，选中存到屏幕 SD 卡上，端口号为 3，屏幕通讯波特率 9600，每包数据包大小 4000 字节，校验方式为 crc16 校验，点击 Start 开始下载。等待下方进度条填满并且下方 cmd 输出信息框提示 File Send SUC 说明文件传输成功



SerialFileUp 软件下载

SerialFileUp 源代码下载

9.3.5 VC++ 运行库合集

说明：微软常用运行库合集是采用 microsoft visual studio 20xx 编写的软件必须使用的公用 dll 运行库，是目前为止最全最新的运行库集合，相当于程序的字典文件。某些网上和论坛的部分精简软件没有附带这些公用 dll，所以安装这些运行库是系统安装后第一件要做的事情。所有的安装文件全部来自微软官方网站，且为最新数字签名版本。

VC++ 运行库合集下载

9.3.6 Microsoft .net Framework 3.5

说明：USART HMI 界面设计软件必须要系统有 framework 3.5 才能正常运行，假如你的系统只有 framework 4.0, 没有 3.5 也是不行的，微软的 framework 4.0 并不完全兼容 framework 3.5。每个版本都各不兼容，所以必须要系统有 framework 3.5 才能正常运行。

win7 系统是自带 framework 3.5 和 framwork 4.0 两个版本都有的，可以直接运行 USART HMI，但是某些 ghost 版本可能精简掉 framework 3.5。

XP 系统系统没有自带，所以 XP 系统一定要安装 framework 3.5。

win10 和 win11 系统需要联网才能安装 Framework3.5，请不要关闭 win10 和 win11 自动升级服务，否则会导致自动下载失败，如果自动安装.NET Framework 3.5 失败，请开启所有的自动升级相关服务，然后重启电脑，重新打开 USART HMI 根据提示进行自动安装

Fmework3.5 运行环境下载 (xp、win7)

win10 离线安装.NET Framework 3.5 请参考 Win10 内网离线安装 NET3.5，绝对成功

部分客户反馈上面的连接需要付费才能查看，所以把文章贴出来，上文链接的文章内容如下

信息化运维有时候需要 net3.5 组件,cdsn 爬了太多坑，总结其实就一条，不同大版本 win10 的镜像内 net3.5 不能混刷，下面各镜像版本的 net3.5 已经提取出来了，把我提取的文件放在 C 盘根目录，在管理员模式下 powershell 或者 cmd 复制下面脚本直接刷就行了。大家去替换路径名刷入就行!

9.3.6.1 如何查看 windows 版本

cmd 输入 winver 查看 window 版本版本信息

如何安装.net Framework 3.5

```
dism.exe /online /enable-feature /featurename:netfx3 /Source:解压后的路径\sxs
```

例如下载的路径为 D:\Downloads\dotnetFramework35.7z

解压后的路径为 D:\Downloads\dotnetFramework35\sxs 打开 windows powershell(管理员)，输入以下命令安装.net Framework 3.5

```
dism.exe /online /enable-feature /featurename:netfx3 /Source:D:\Downloads\
↪dotnetFramework35\sxs
```

1809 可独装

1903-1909 可混装

2004-2009-20h2-21h2-22h1

另附微软官网的.NET Framework 3.5 SP1 Offline Installer 下载

微软官方.NET Framework 3.5 安装包

9.3.7 数据记录控件数据转 excel 文件工具 (DataBox)

说明：此工具可以将数据记录控件生成的 data 文件转换为 csv 文件。

csv 文件可以用 office 套件中的 excel 或者金山 wps 直接打开。

DataBox 软件下载

9.3.8 常见串口驱动下载

驱动来自网络，淘晶驰不保证其可靠性和安全性

常见串口驱动下载

9.3.9 powertoys 下载

PowerToys(微软小工具)，可以使用其中的“图像大小调整器”辅助调整图片分辨率

9.3.9.1 PowerToys0.97.2-x64

PowerToys0.97.2-x64 下载

9.3.9.2 PowerToys0.95.1-x64(旧版本)

PowerToys0.95.1-x64 下载

9.3.9.3 PowerToys0.90.1-x64(旧版本)

powertoys0.90.1-x64 下载

9.3.10 解压缩工具 7zip 下载

一款小巧的解压缩工具

解压缩工具 7zip 下载

9.3.11 python 安装包

python-3.6.8-amd64

python-3.9.13-amd64

python-3.10.11-amd64

python-3.12.10-amd64

9.4 标准出厂工程样例

提示： 同一系列相同分辨率的工程是一样的，可能需要修改型号之后才能正确烧录到串口屏中，请参考如何修改设备型号 (page 1259)

也可以根据自己的型号来选择对应的工程，链接–> 新版规格书和出厂工程下载 (page **??**)

X5 系列出厂样例

X3 系列出厂样例

X2 系列出厂样例

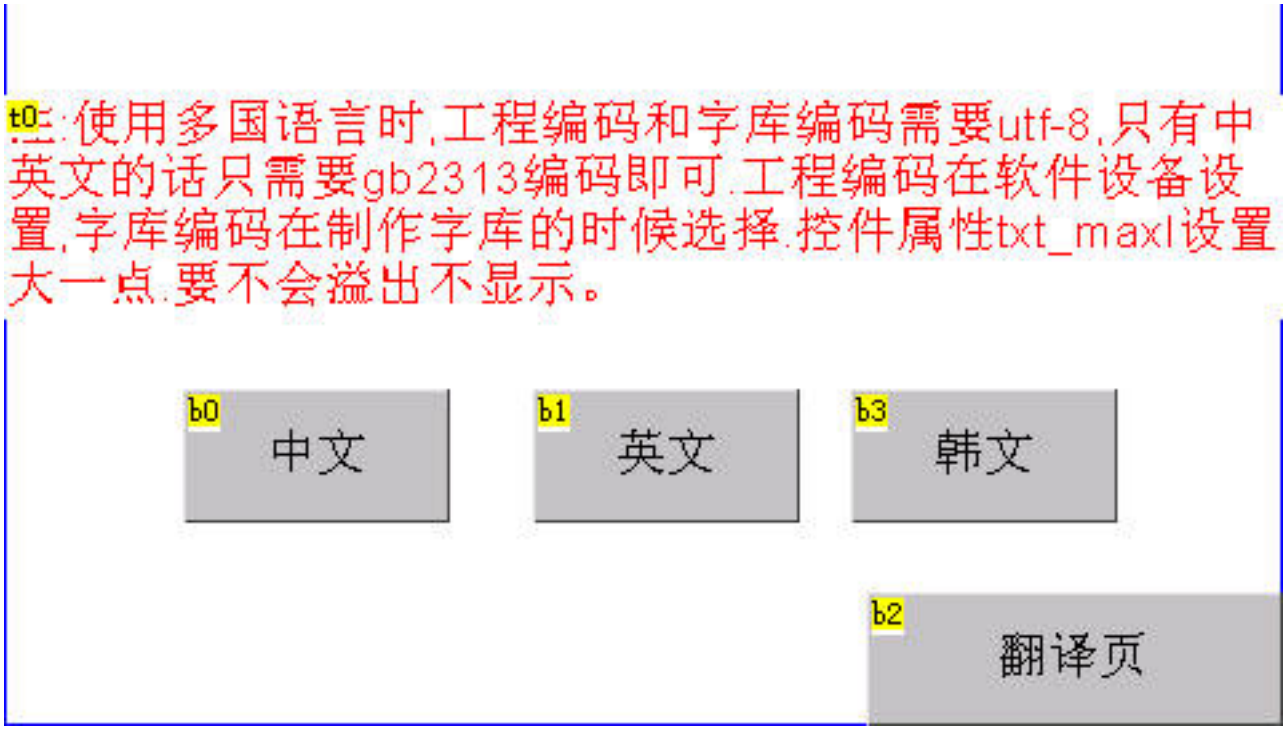
T1 系列出厂样例

K0 系列出厂样例

T0 系列出厂样例

9.5 官方样例工程

9.5.1 多语言切换样例工程



素材说明: 该例程演示了切换中文、英文、韩文

9.5.1.1 多语言切换-样例工程下载

演示工程下载链接:

《多语言切换》演示工程下载

9.5.2 串口屏弹窗样例工程

实现弹窗、提示框、确认框

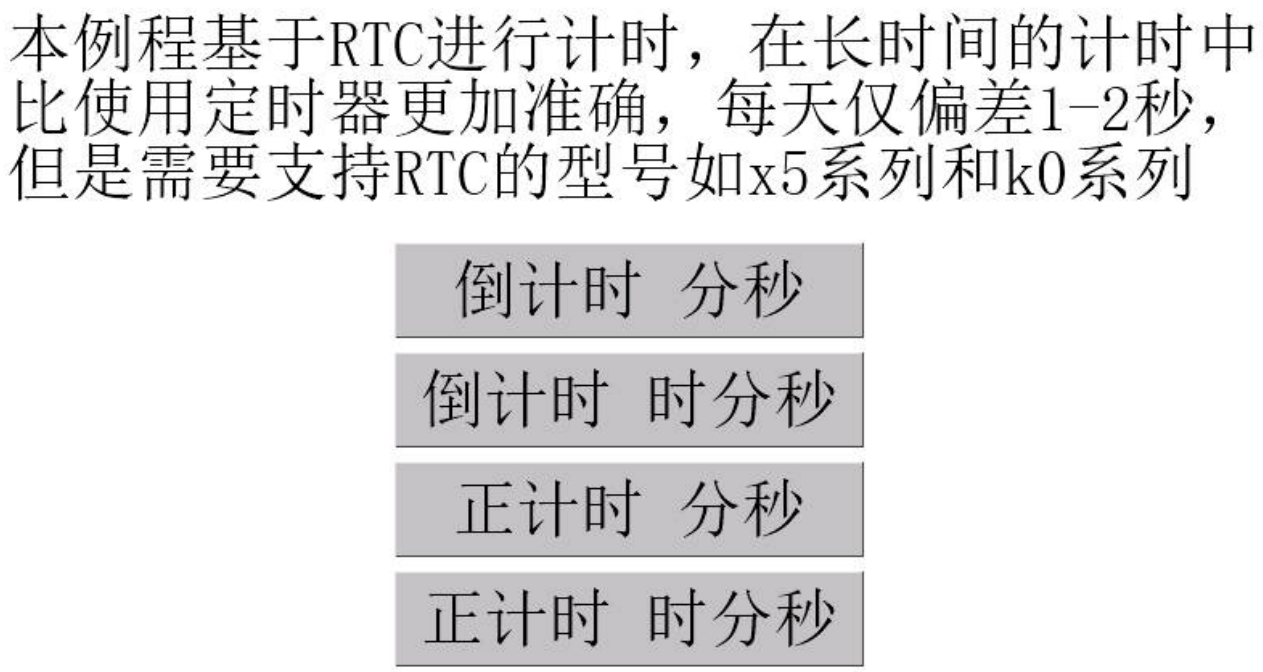
素材说明: 弹窗效果展示

9.5.2.1 串口屏弹窗-样例工程下载

演示工程下载链接:

《串口屏弹窗》演示工程下载

9.5.3 正计时和倒计时样例工程



素材说明: 总共包含 2 个例程，分别是基于 RTC 进行计时和使用定时器进行计时

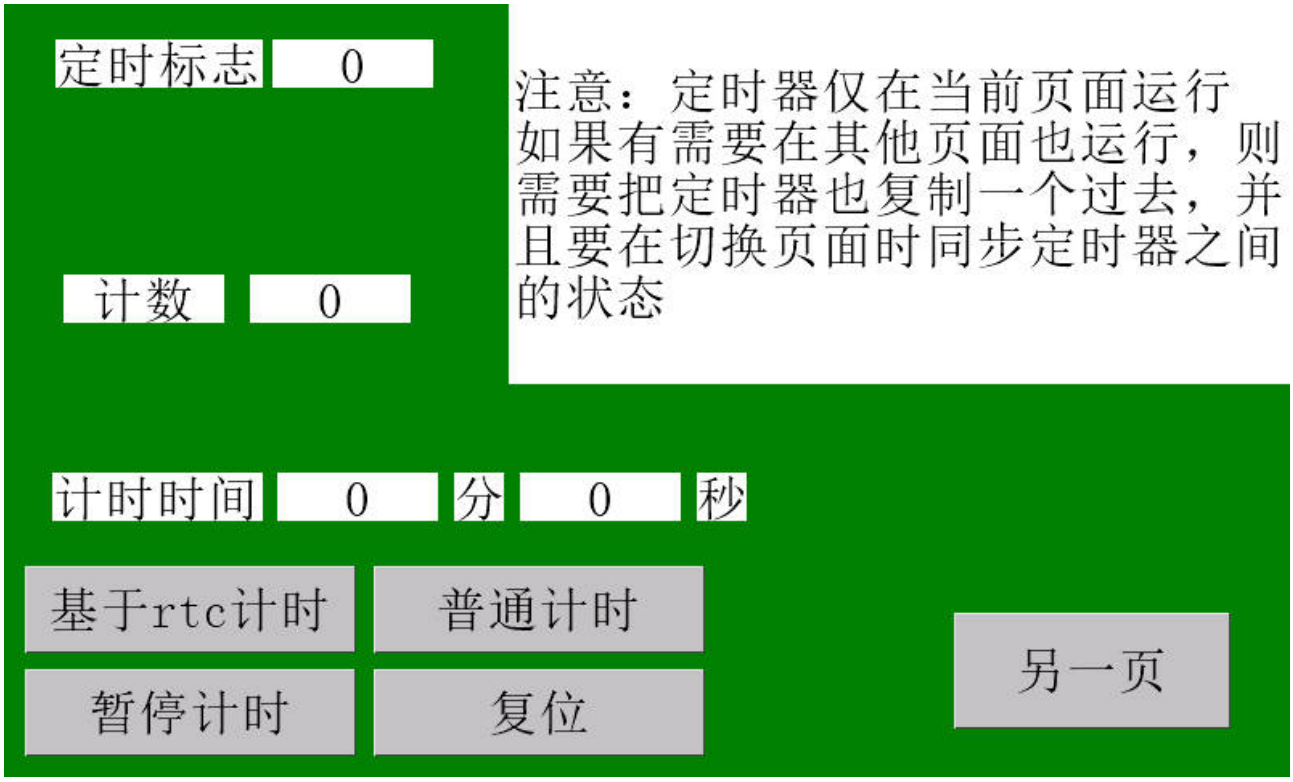
9.5.3.1 正计时和倒计时-样例工程下载

演示工程下载链接:

《正计时和倒计时》演示工程下载

《普通计时》演示工程下载

9.5.4 串口屏跨页面计时样例工程



素材说明: 串口屏上实现跨页面计时功能

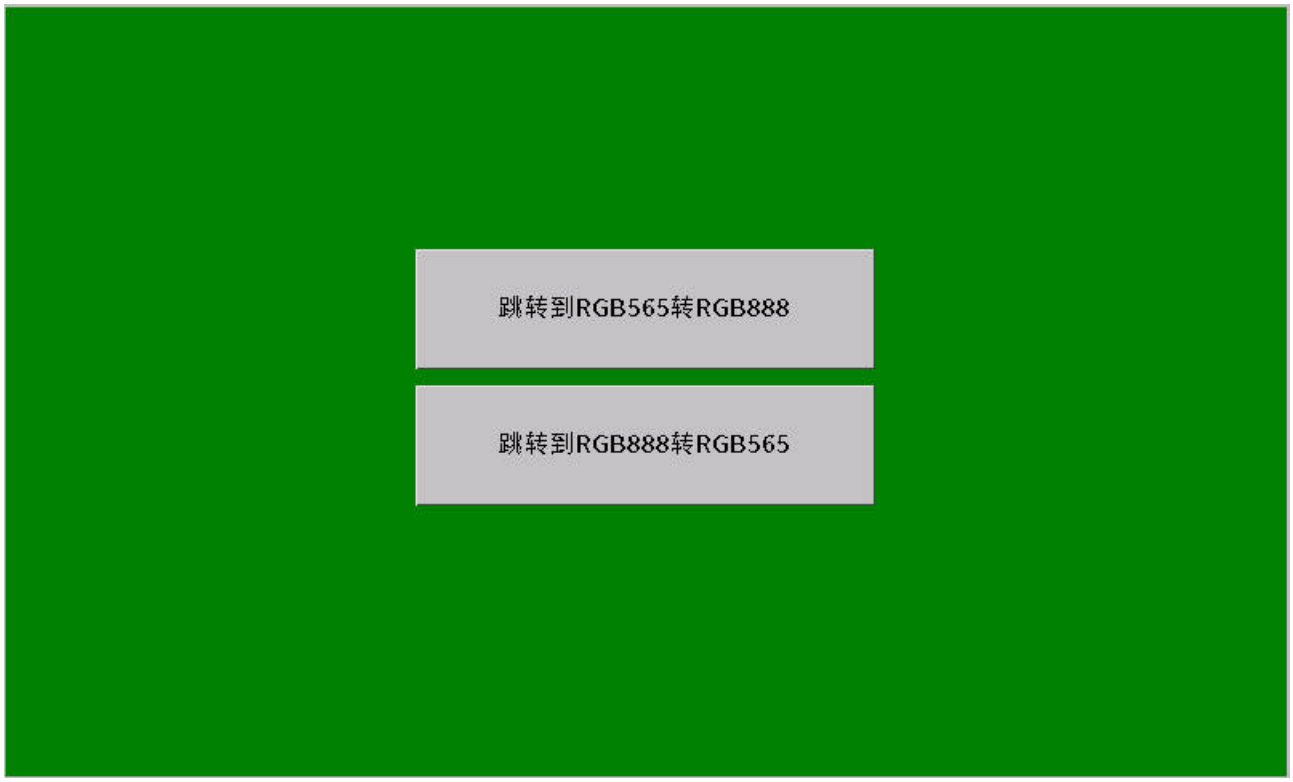
9.5.4.1 串口屏跨页面计时-样例工程下载

演示工程下载链接:

《串口屏跨页面计时》演示工程下载

9.5.5 RGB 转换样例工程

素材说明: 将 RGB888 转为 RGB565，或者将 RGB565 转为 RGB888



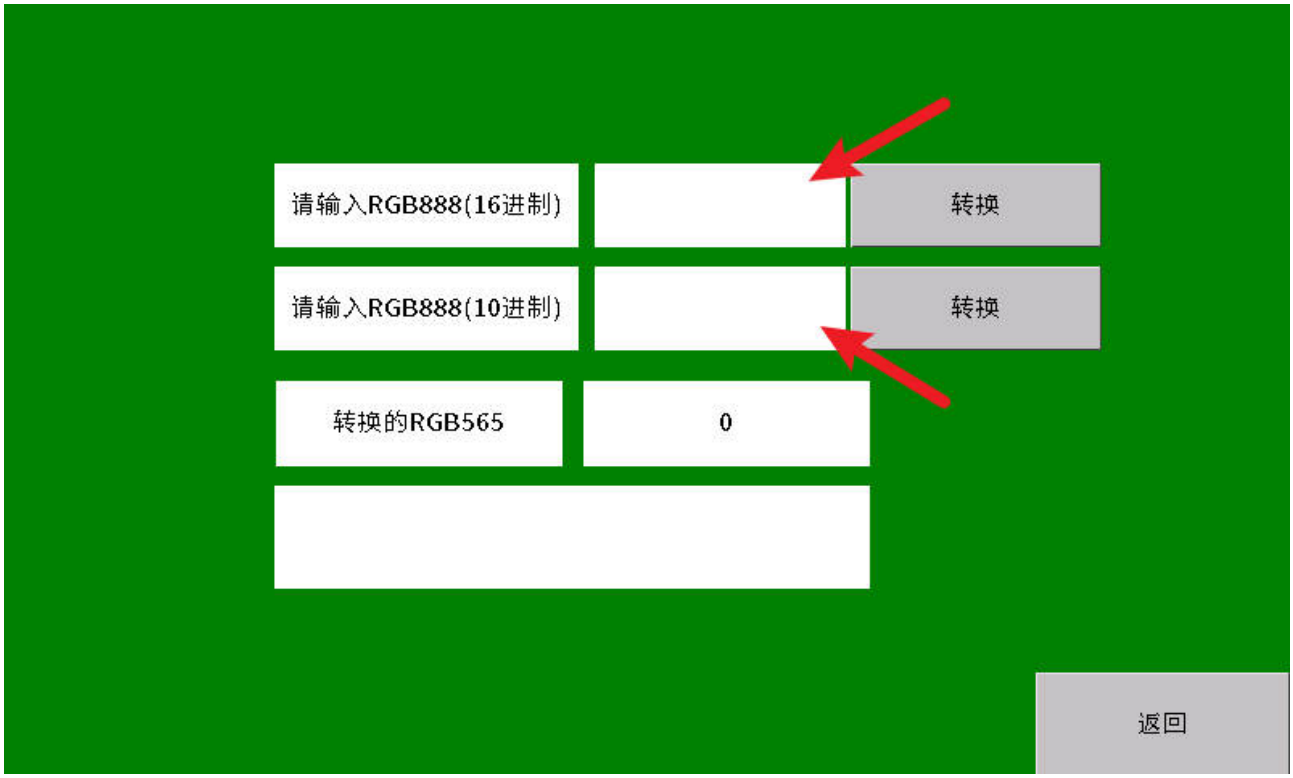
9.5.5.1 RGB565 转 RGB888

1. 点击此处输入 RGB565 的数值，然后点击转换按钮，即可得到对应的 RGB888 数值

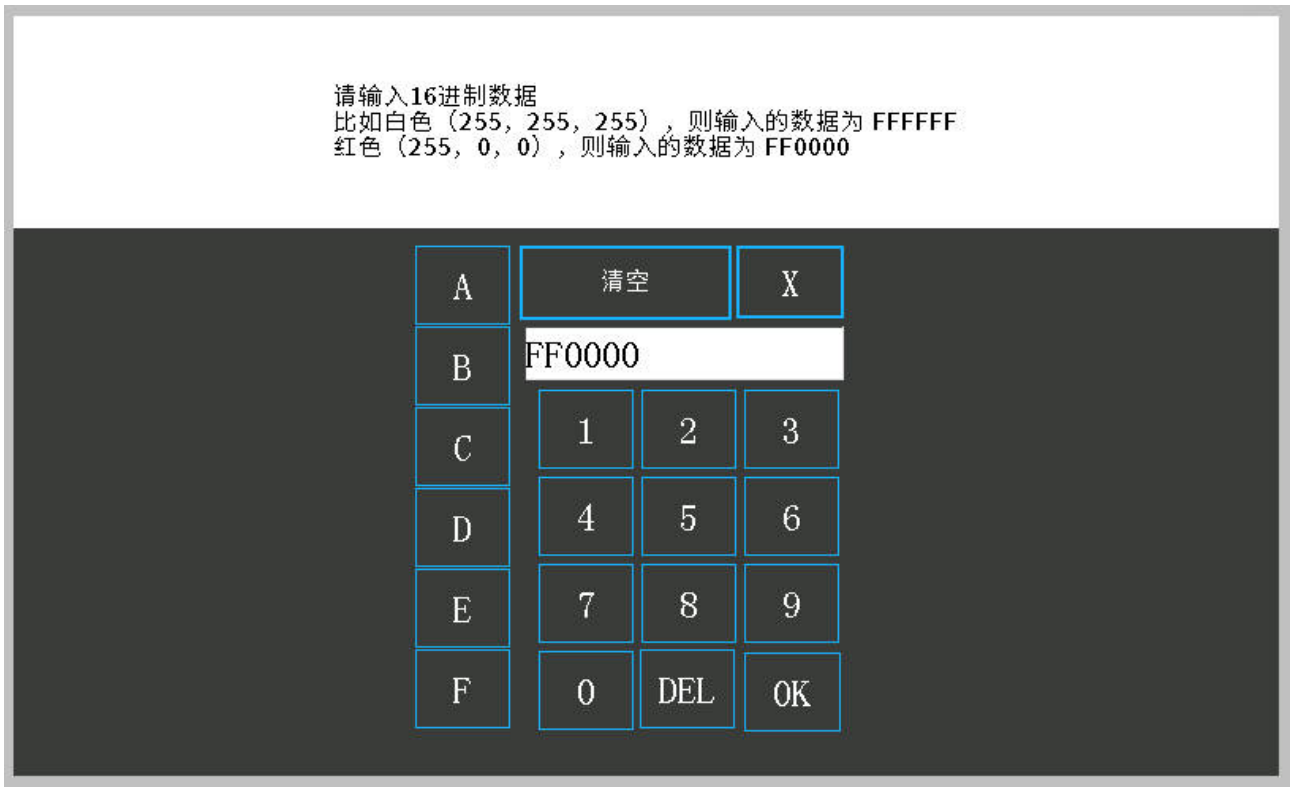
2.RGB565 转 RGB888 过程中，由于是低质量转高质量，低位会被 0 填充，因此例如纯白色 65535 转为 RGB888，本来结果应该是 0xFFFFFF，但是最后的结果是 F8FCF8(因为低位填充的是 0)，这是无法避免的

9.5.5.2 RGB888 转 RGB565

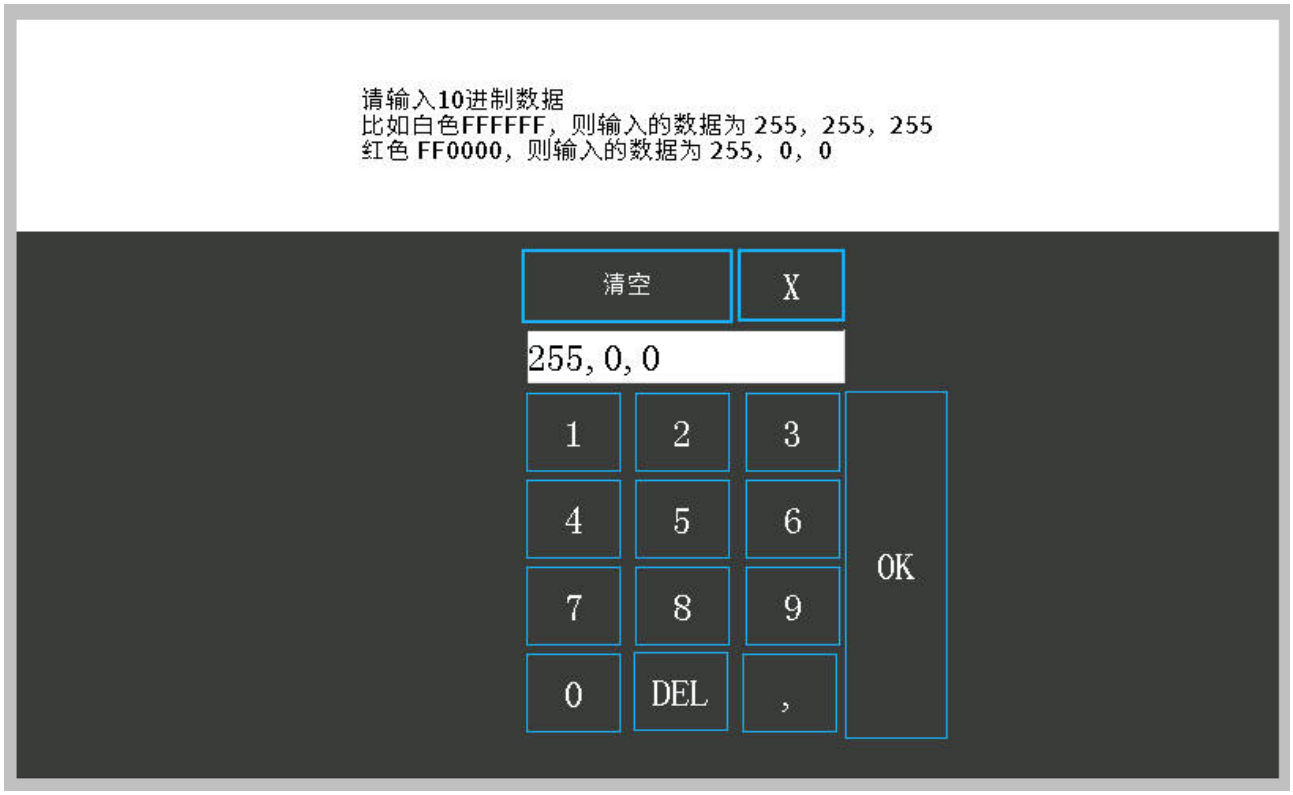
1. 输入其中的任意一个，点击转换即可。



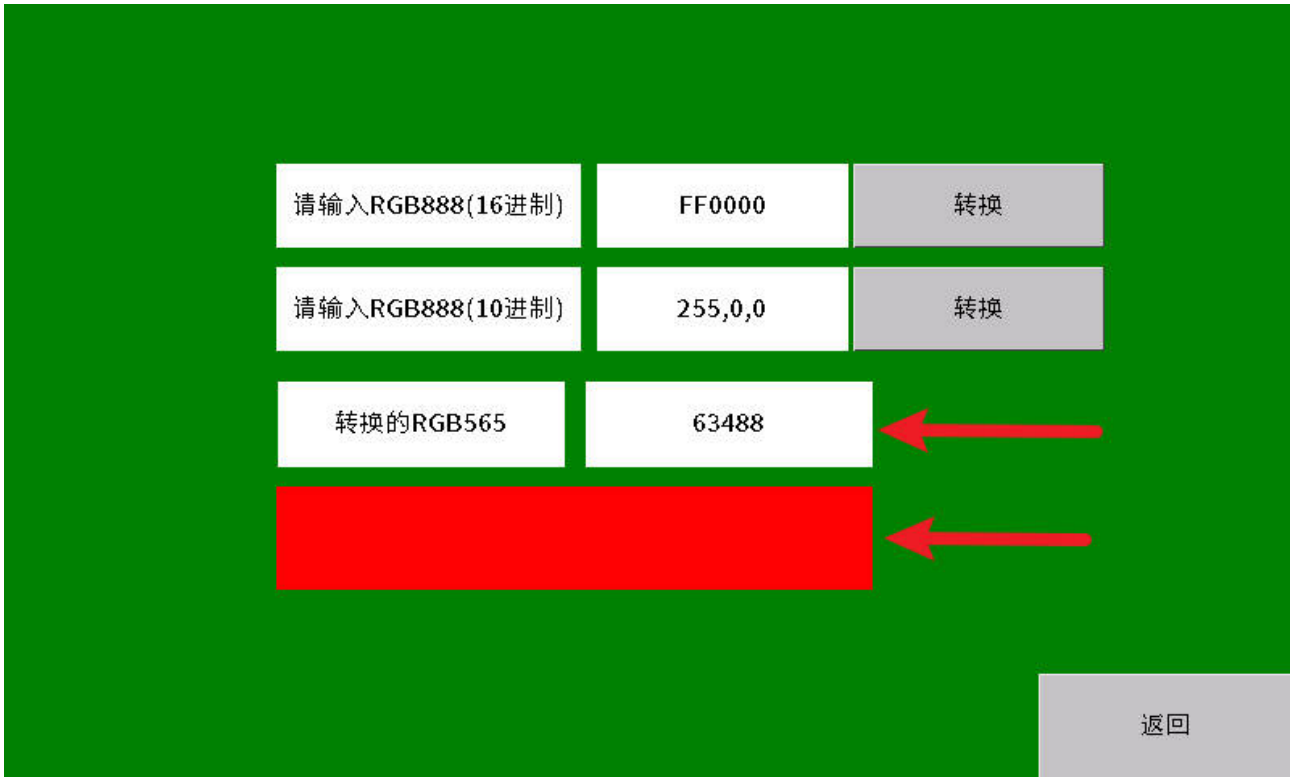
2. 如果是以 16 进制方式输入的，此处输入的数据为 FF0000（纯红色）。



3. 如果是以 10 进制方式输入的，此处输入的数据为 255,0,0（纯红色）。



4. 转换结果如下所示，不仅会显示对应的 RGB565 数值，还会显示对应的颜色

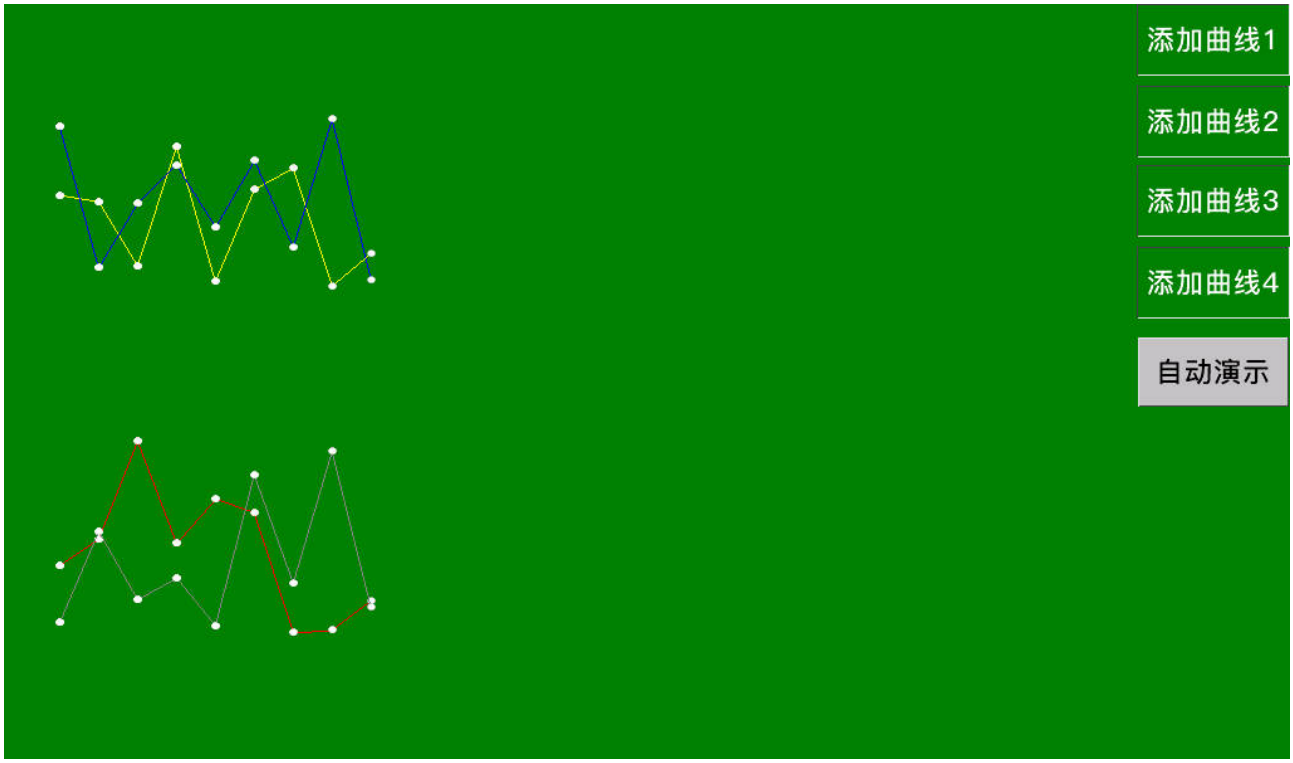


9.5.5.3 RGB 转换-样例工程下载

演示工程下载链接：

《RGB 转换》演示工程下载

9.5.6 自定义曲线样例工程



素材说明: 实现自定义的曲线

9.5.6.1 自定义曲线-样例工程下载

演示工程下载链接:

《自定义曲线》演示工程下载

9.5.7 51 单片机样例工程

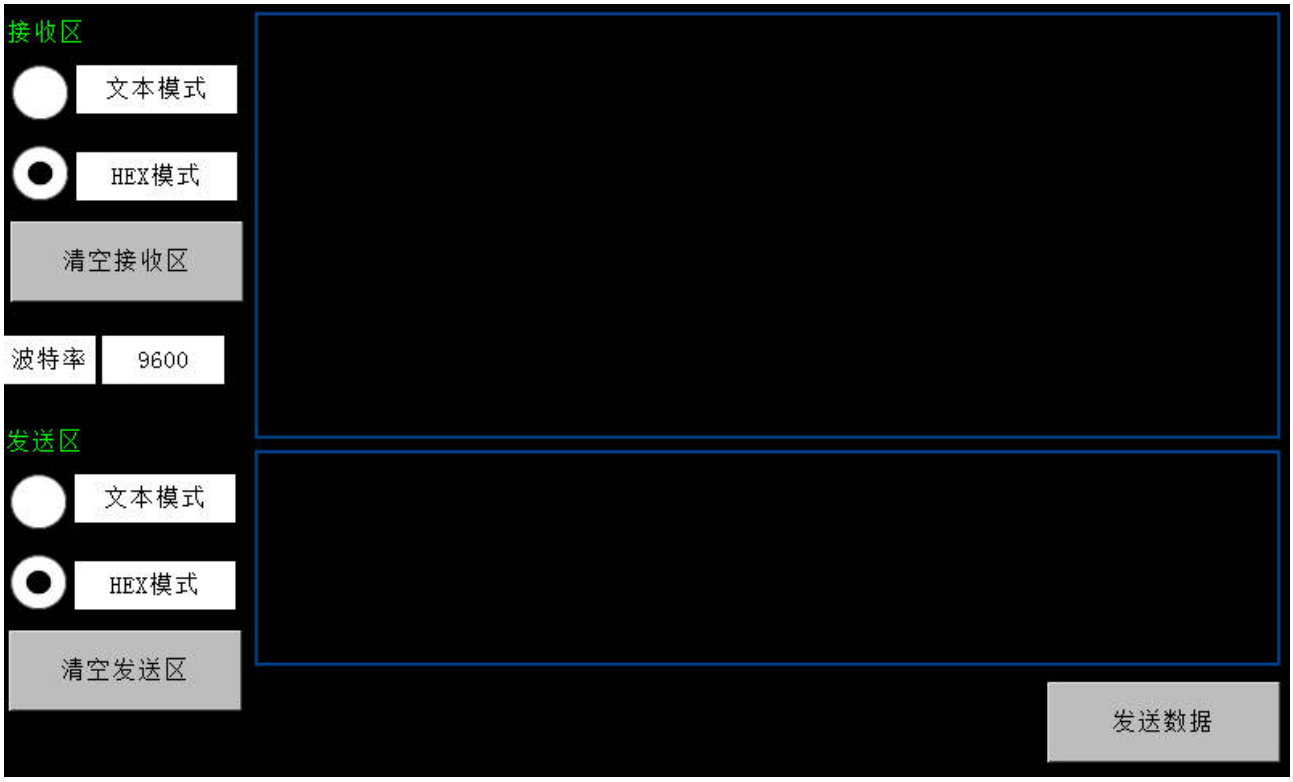
素材说明:51(STC) 单片机和 HMI 屏通信, 通过单片机对 HMI 屏刷色及改变 HMI 屏文本内容。

9.5.7.1 《51 单片机》-样例工程下载

演示工程下载链接:

《51 单片机》演示工程下载

9.5.8 简易串口助手样例工程



素材说明：（全系列支持，上位软件 0.58 及以上版本支持）该工程使用主动解析功能在屏幕上实现串口助手的功能。可以用来做一个串口助手成品设备，方便外出携带分析串口数据。

9.5.8.1 简易串口助手-样例工程下载

演示工程下载链接：

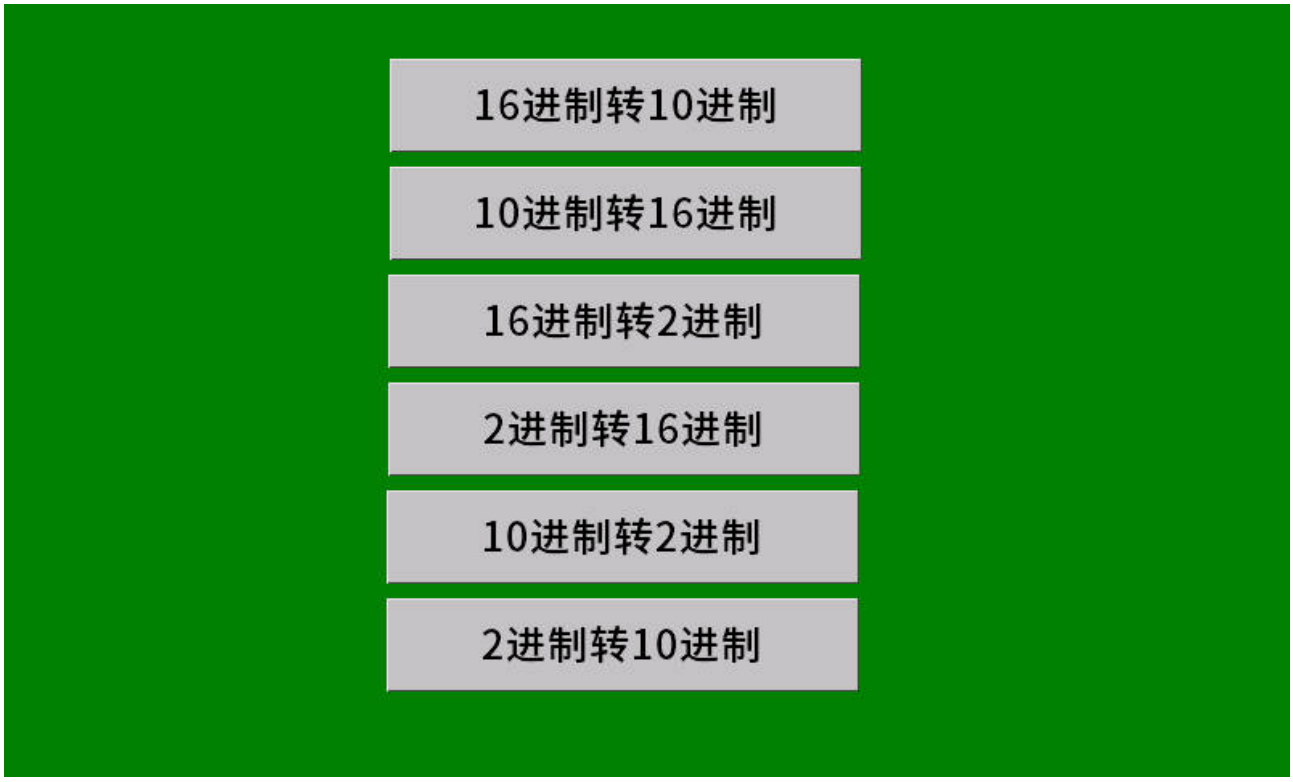
《简易串口助手》演示工程下载

《串口助手智能显示数据》演示工程下载

《小屏简易串口助手》演示工程下载

9.5.9 进制转换样例工程

素材说明: 实现常见进制之间的转换

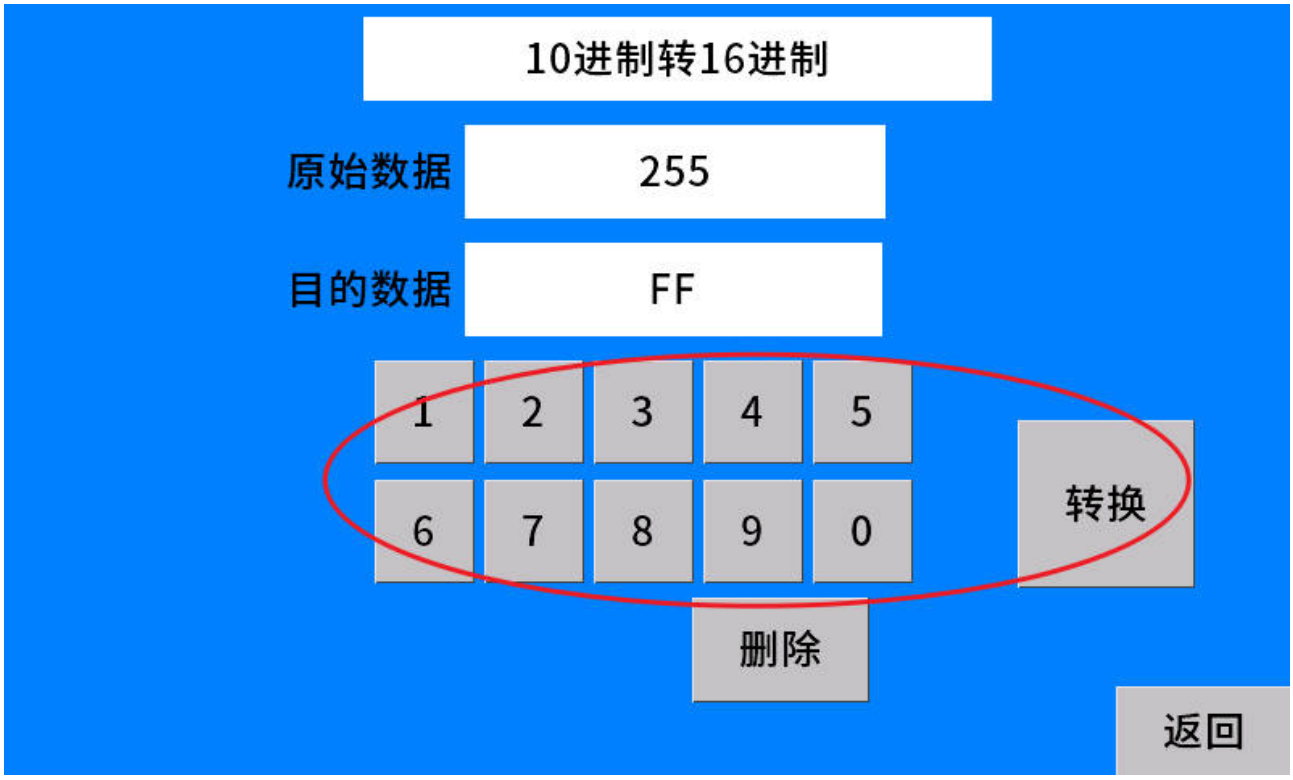


9.5.9.1 16 进制转 10 进制

通过底部的键盘输入数据后，点击转换就能转换成 10 进制

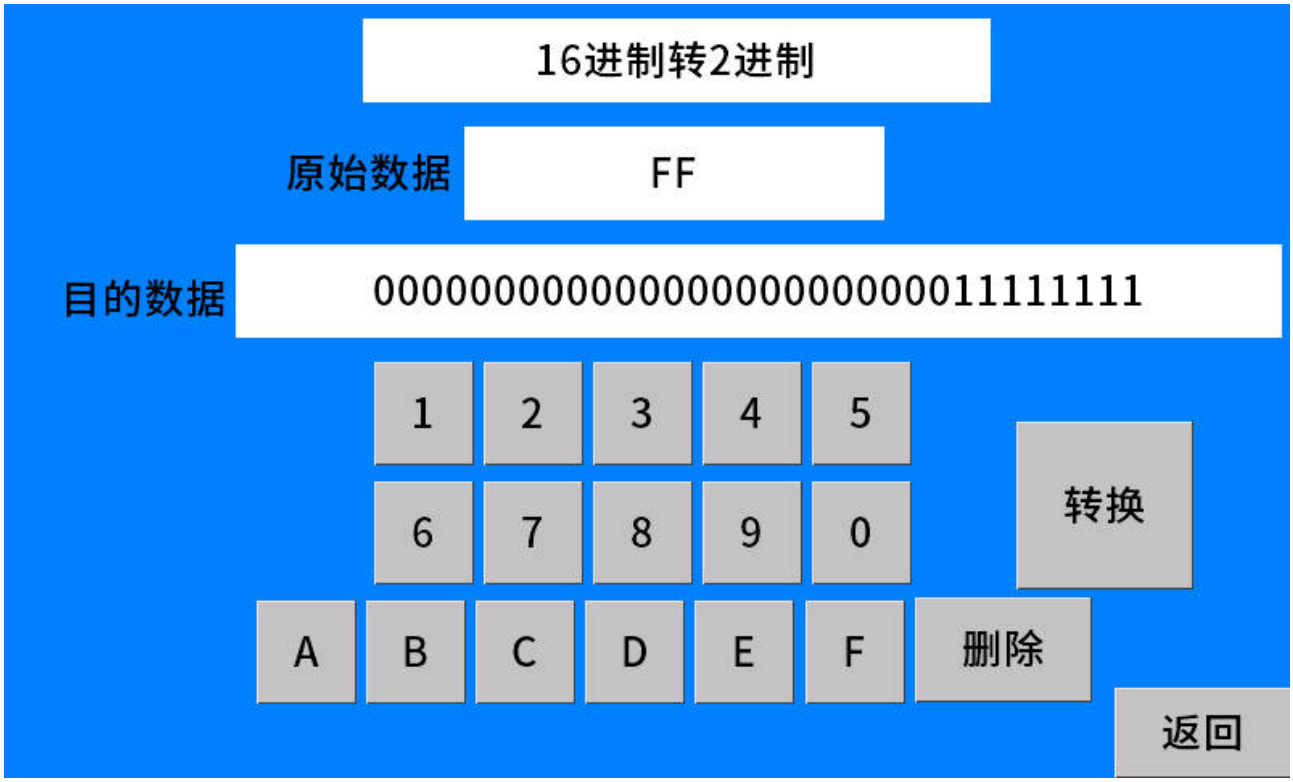
9.5.9.2 10 进制转 16 进制

通过底部的键盘输入数据后，点击转换就能转换成 16 进制



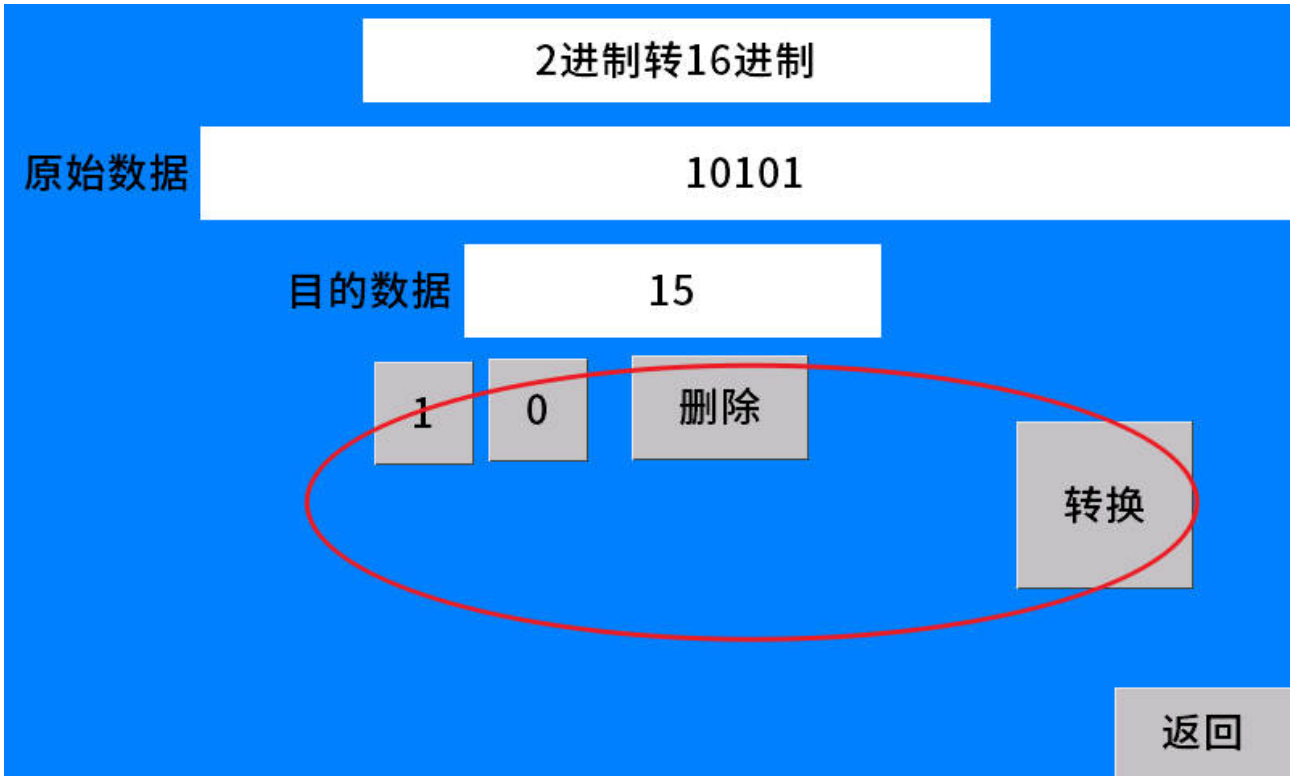
9.5.9.3 16 进制转 2 进制

通过底部的键盘输入数据后，点击转换就能转换成 2 进制



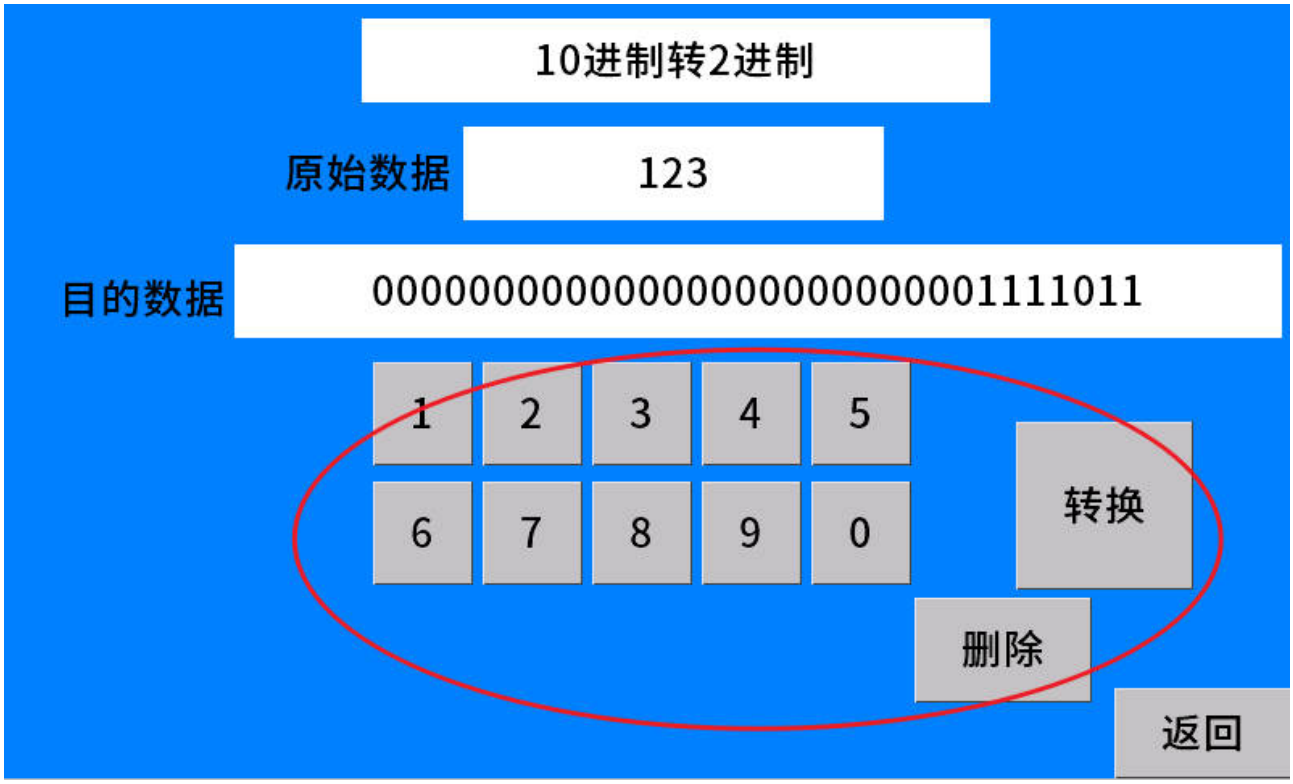
9.5.9.4 2 进制转 16 进制

通过底部的键盘输入数据后，点击转换就能转换成 16 进制



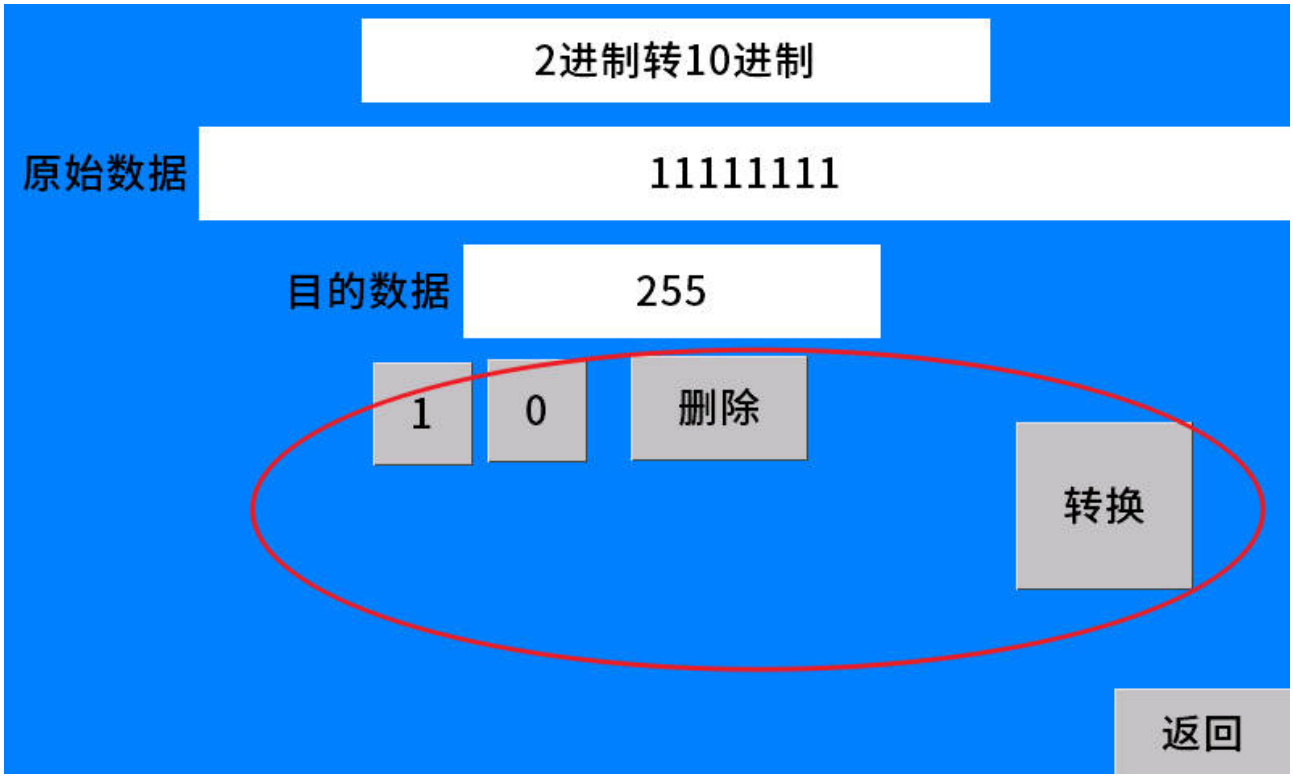
9.5.9.5 10 进制转 2 进制

通过底部的键盘输入数据后，点击转换就能转换成 2 进制



9.5.9.6 2 进制转 10 进制

通过底部的键盘输入数据后，点击转换就能转换成 16 进制



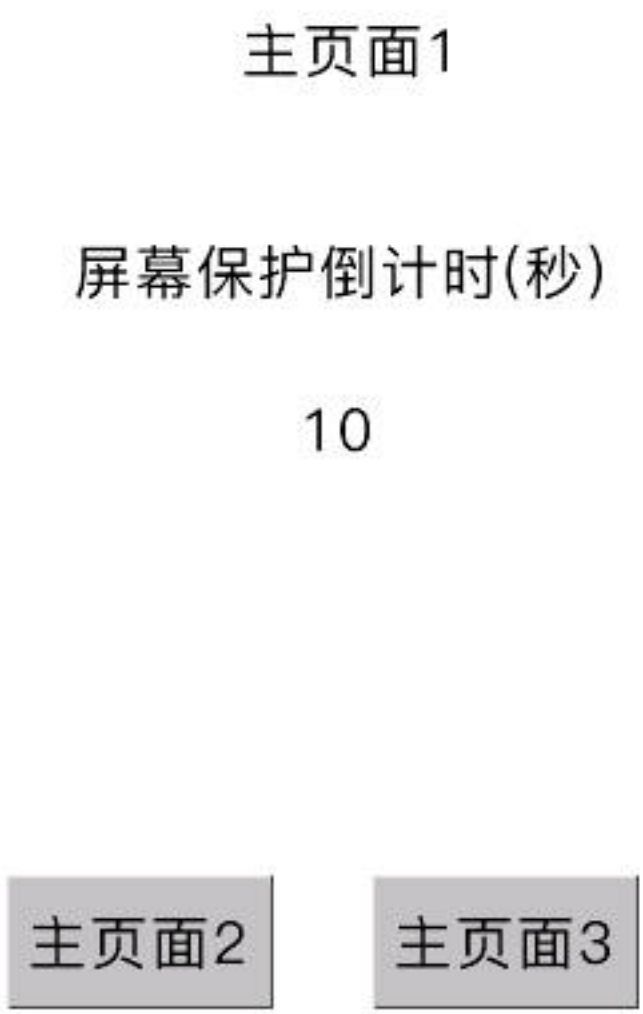
9.5.9.6.1 《进制转换》-样例工程下载

演示工程下载链接：

《进制转换》演示工程下载

《进制转换 2》演示工程下载

9.5.10 自定义屏幕保护样例工程/低功耗/睡眠休眠替代



素材说明: 实现自定义的休眠/睡眠/屏保/屏幕保护，请在 program.s 中设置数据：timeOut（倒计时时间），saverDim（进入屏保时的屏幕亮度, 如果设置为 0 则显示黑屏）

9.5.10.1 自定义屏幕保护-样例工程下载

演示工程下载链接：

《自定义屏幕保护》演示工程下载

9.5.11 虚拟浮点数与文本控件互相转换样例工程

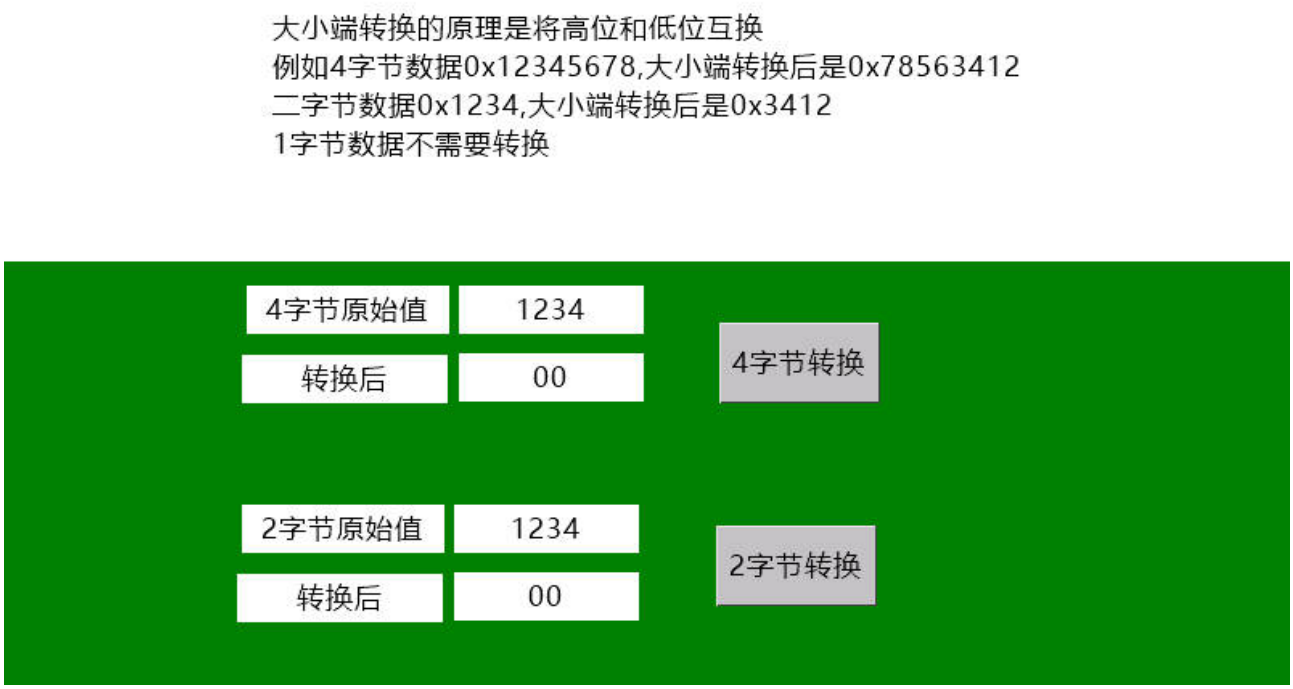
> **注意：** 以下例程仅做演示使用，演示如何进行转换虚拟浮点数和文本之间的转换，转换思路不唯一，如使用本方法进行转换，请自行测试，在生产环境中使用时请注意限定范围，避免转换失败导致其他问题

9.5.11.1 虚拟浮点数与文本控件互相转换-样例工程下载

演示工程下载链接：

《虚拟浮点数与文本控件互相转换》演示工程下载

9.5.12 大小端数据互相转换样例工程



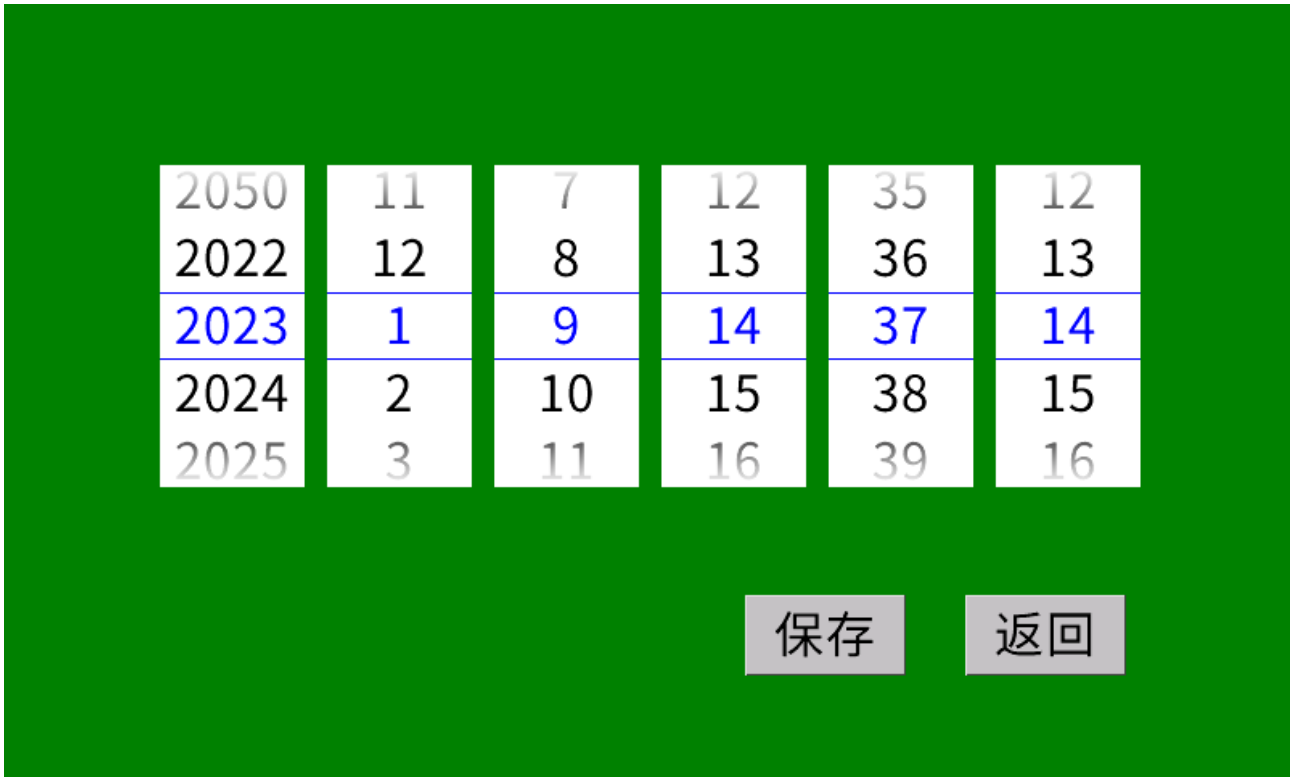
素材说明: 实现大小端互相转换的问题，大端转换为小端或者小端转换为大端

9.5.12.1 大小端数据互相转换-样例工程下载

演示工程下载链接:

《大小端数据互相转换》演示工程下载

9.5.13 显示时间和设置时间样例工程



素材说明: 多页面显示时间和使用选择文本控件设置时间

9.5.13.1 多页面显示时间和使用选择文本控件设置时间-样例工程下载

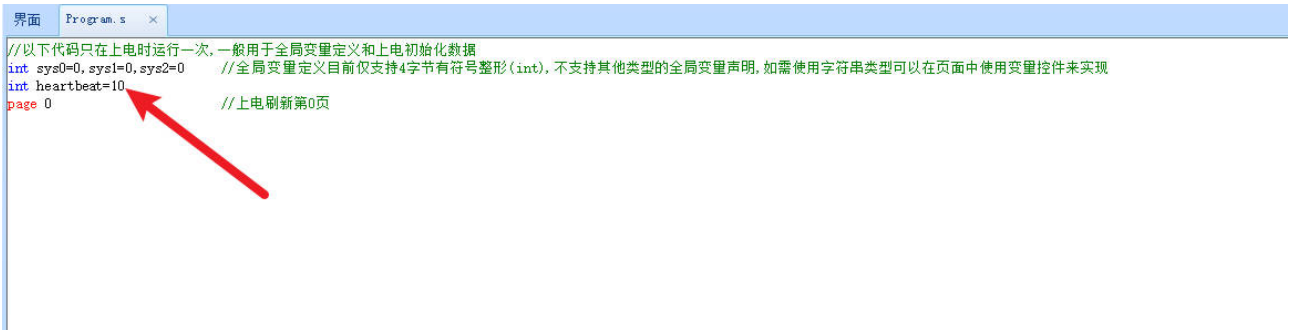
演示工程下载链接:

《显示时间和设置时间》演示工程下载

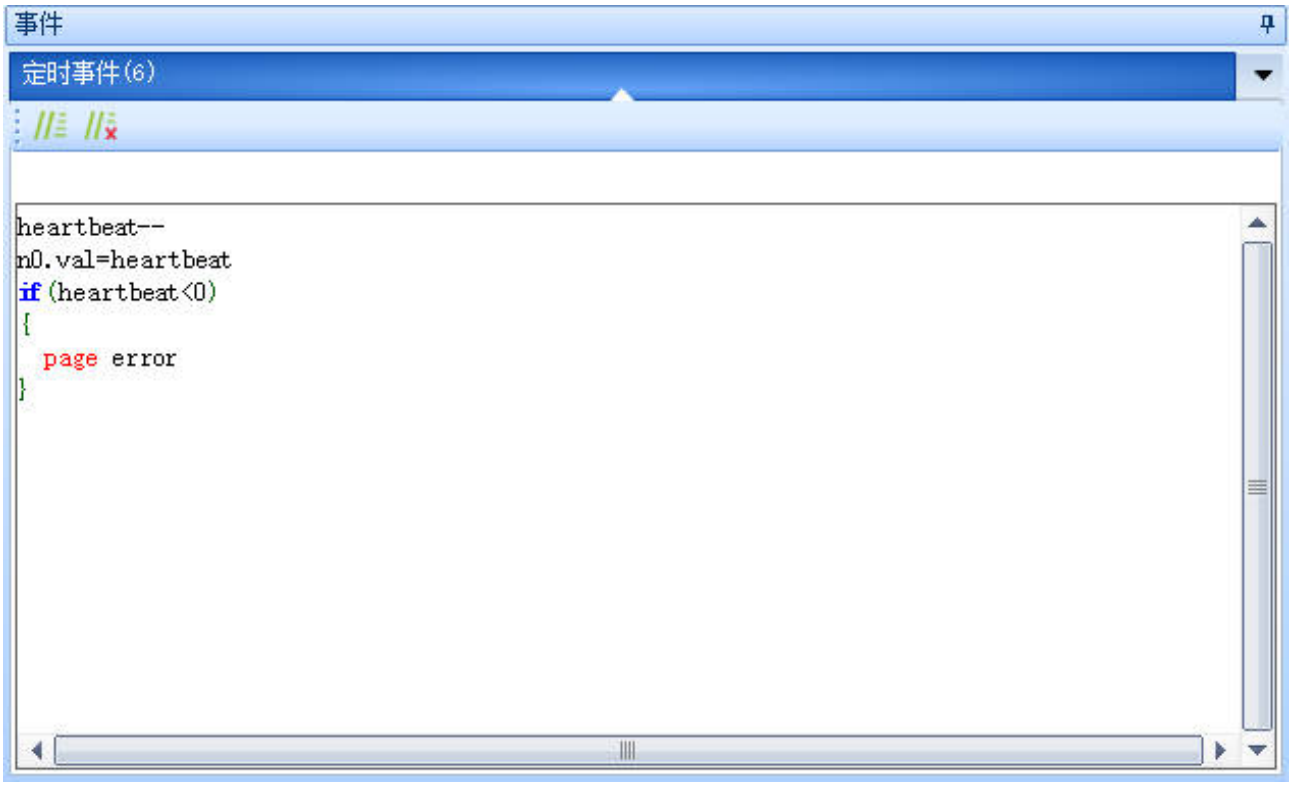
9.5.14 串口屏实现通讯心跳样例工程

用于判断单片机与串口屏是否通讯正常

定义一个 int 变量 heartbeat



在定时器中对 heartbeat 自减，当 heartbeat 小于 0 时跳转到错误页面

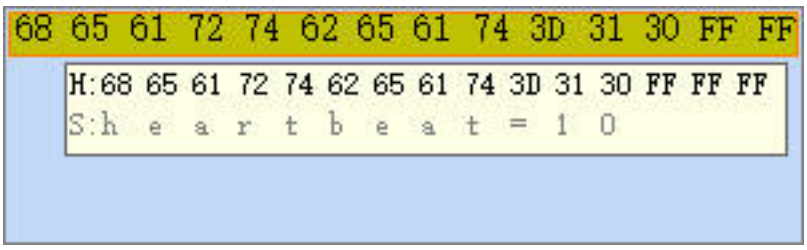


单片机需要向心脏跳动一样，定时向串口屏不断重新赋值 heartbeat 的值为 10。

单片机上的代码如下

```
printf("heartbeat=10\xff\xff\xff");
```

发送出来的数据如下所示

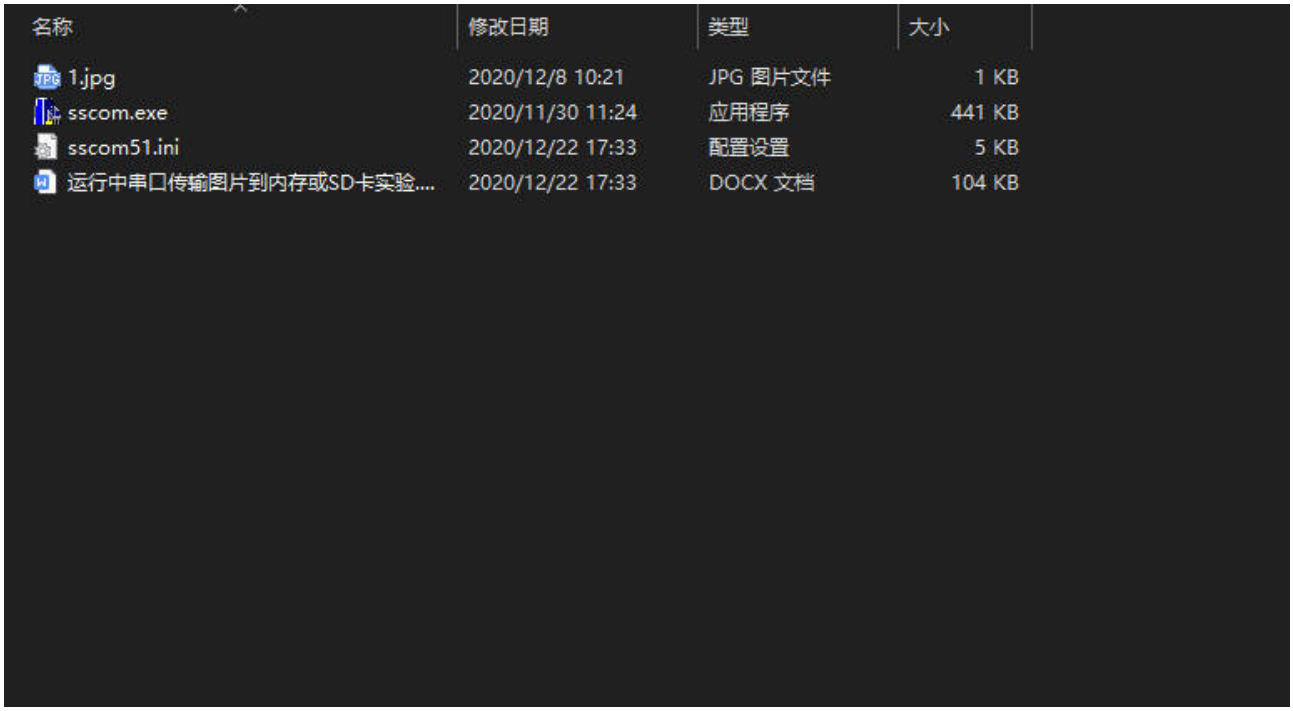


9.5.14.1 串口屏实现通讯心跳-样例工程下载

演示工程下载链接：

《串口屏实现通讯心跳》演示工程下载

9.5.15 运行中发送图片到串口屏样例工程



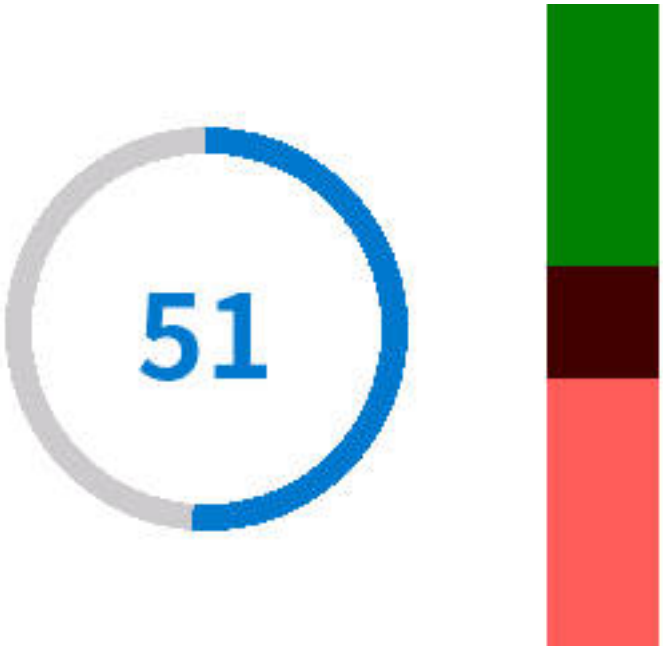
素材说明: 运行中发送图片到串口屏

9.5.15.1 运行中发送图片到串口屏-样例工程下载

演示工程下载链接:

《运行中发送图片到串口屏》演示工程下载

9.5.16 环形进度条样例工程



使用图片实现的，所有系列都可以实现

资料中提供了 50*50 100*100 150*150 200*200 250*250 300*300 350*350 400*400 450*450 500*500 550*550 600*600 分辨率的素材

如果客户需要生成其他颜色或者分辨率的素材，也可以自行通过代码来生成环形素材，python 代码已经提供在压缩包中

注意： 如果需要使用 python 来自行生成素材，请右键“素材”文件夹，以文件夹的型式打开，否则无法在对应的目录下生成图片素材

注意： 仅建议更改以下这 4 个参数：

图片的边长（size_x），图片必须是正方形的

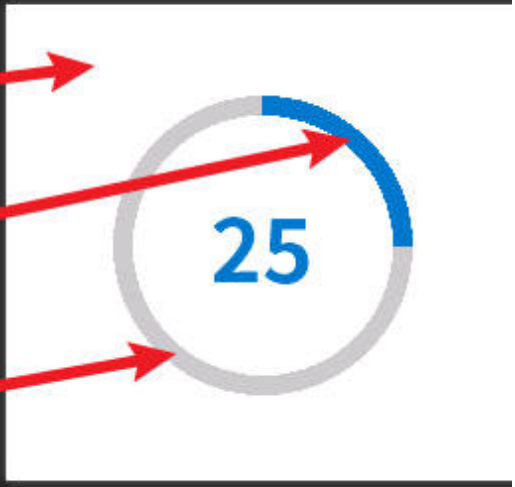
进度条步进值（progress_step）

生成图片的背景颜色（background_color）

生成图片的进度条上的颜色（progress_color）

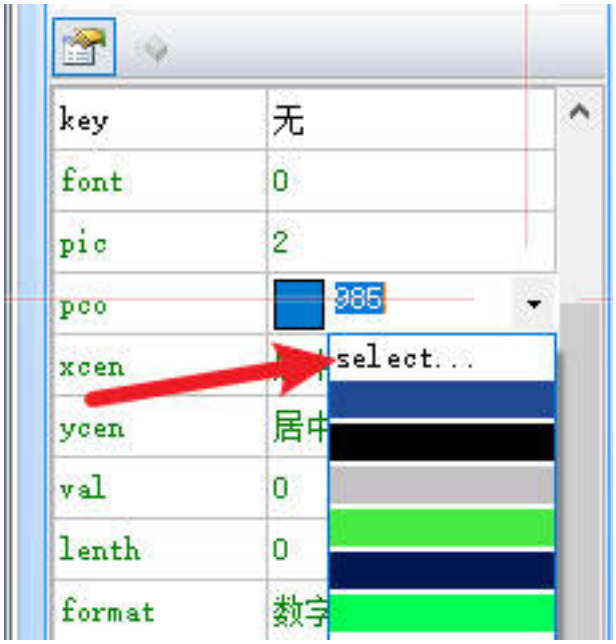
生成图片的进度条上未到达部分的颜色（empty_color）

```
############################################################
    # 设置图片的边长
    size_x = 640

    # 设置进度条步进值1,2,10,20
    # 设置成1就是每一张都生成
    # 设置成2就是隔一张生成一张
    # 设置成5就是隔五张生成一张
    # 设置成20就是隔二十张生成一张
    progress_step = 1

    # 环形进度条的颜色
    # 生成图片的背景颜色
    background_color = (255, 255, 255)

    # 生成图片的进度条上的颜色
    progress_color = (0, 122, 204)

    # 生成图片的进度条上未到达部分的颜色
    empty_color = (200, 200, 200)


############################################################
```

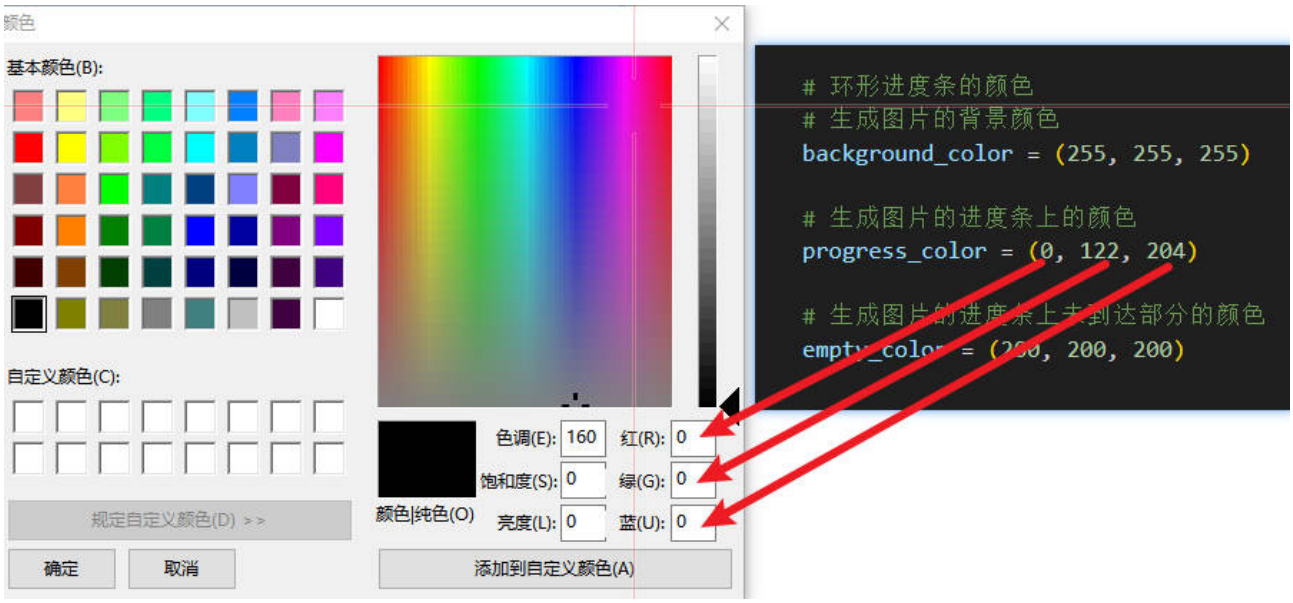


最后用数字控件或者图片控件来显示对应的图片即可

如果需要让数字控件中的数值颜色和进度条的颜色一致的话，选中数字控件的 pco 属性，然后选择 select



将对应的 RGB 数值填入即可



这是使用 python 代码实现的自动

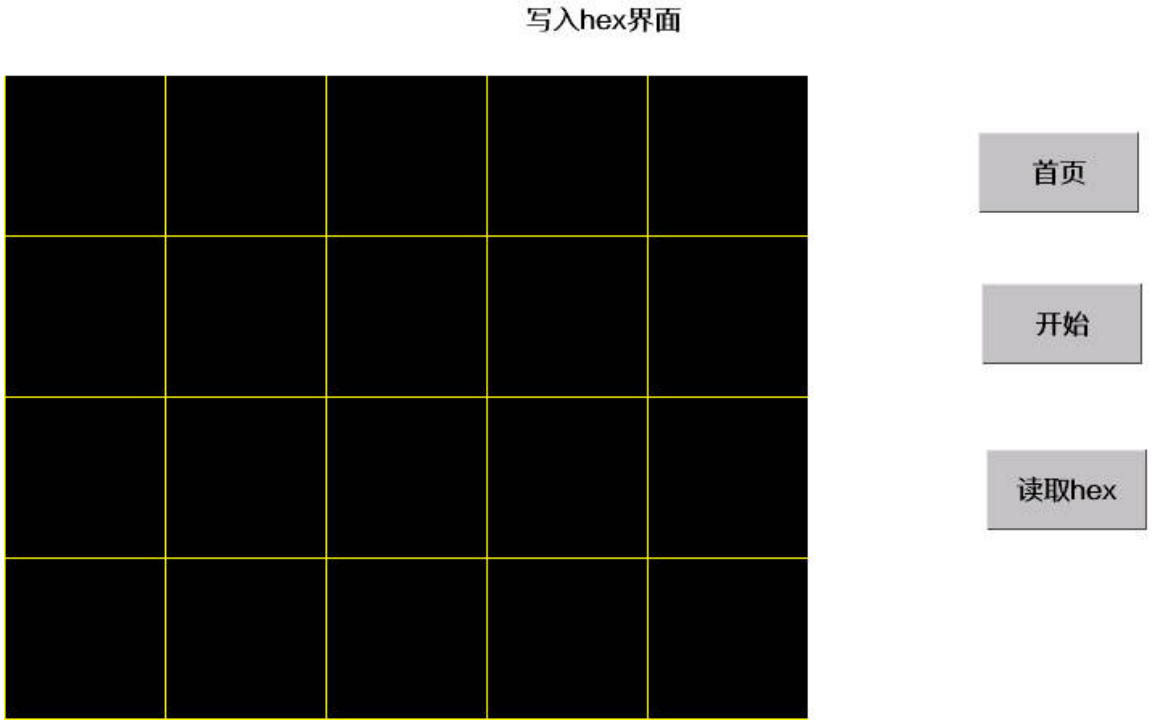
9.5.16.1 环形进度条-样例工程下载

演示工程下载链接：

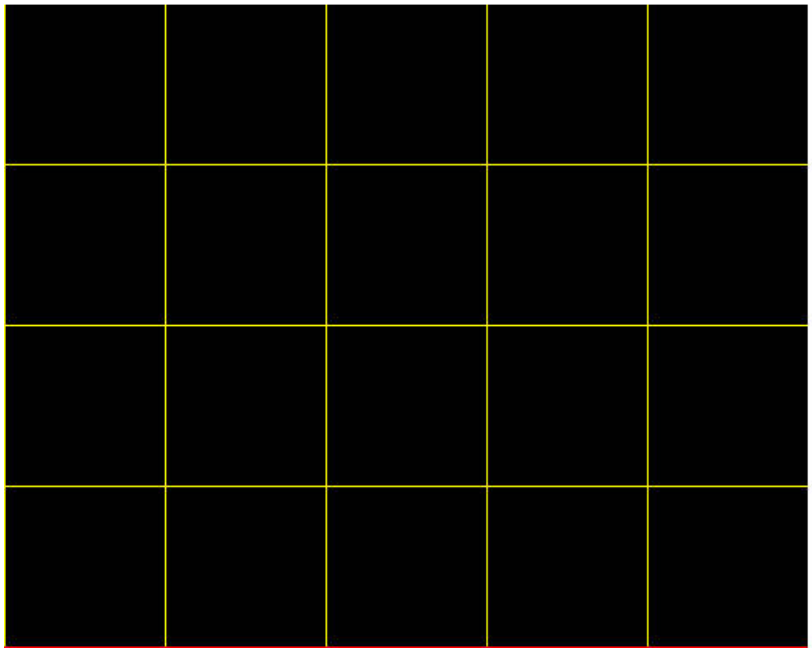
《环形进度条》演示工程下载

9.5.17 使用存储卡跨页面记录波形数据-文件流控件样例工程

使用写入hex生成的数据需要用电脑查看时，在电脑上用winhex软件打开testHex.wwd进行查看。
使用写入Str生成的数据需要用电脑查看时，在电脑上用记事本软件打开testStr.wwd进行查看。



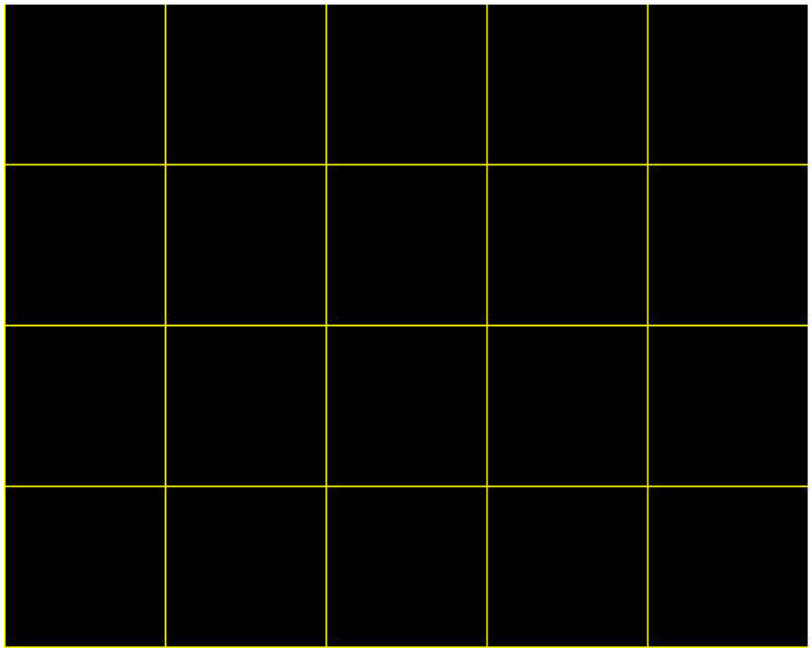
读取hex界面



首页

写入hex

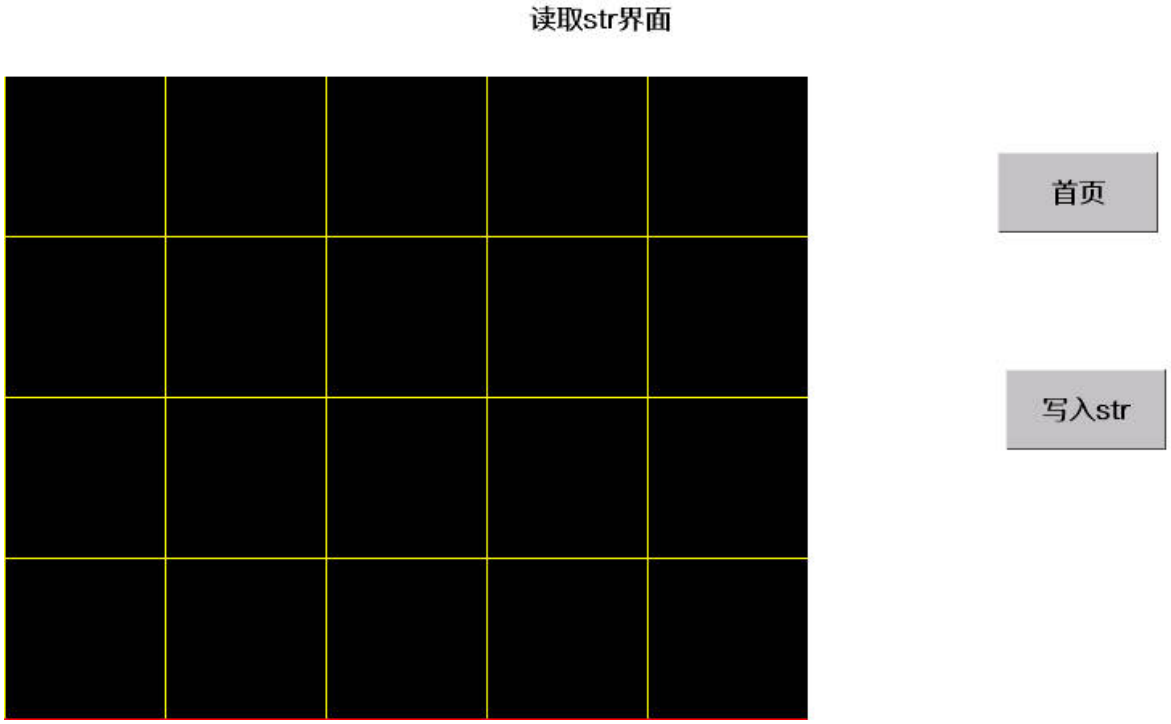
写入str界面



首页

开始

读取str



素材说明: 实现使用存储卡跨页面记录波形数据-文件流控件

9.5.17.1 使用存储卡跨页面记录波形数据-样例工程下载

演示工程下载链接:

《使用存储卡跨页面记录波形数据》演示工程下载

9.5.18 X 系列给其他屏幕下载工程

素材说明: 实现使用 X 系列给其他屏幕下载工程

准备工作 1:

准备两块串口屏，其中一块必须是 x 系列屏幕

此处我以 TJC8048X550_011（尺寸为 5 寸，以下简称 X550）和 TJC4832T135_011（尺寸为 3.5 寸，以下简称 T135）为例进行演示，其中 X550 作为下载器，给 T135 升级工程

先将此工程下载进入 X550 串口屏，如果是其他 x 系列屏幕，可以点击上位机顶部的设备按钮，修改为其他 x 系列屏幕参考如何修改设备型号 (page 1259)。

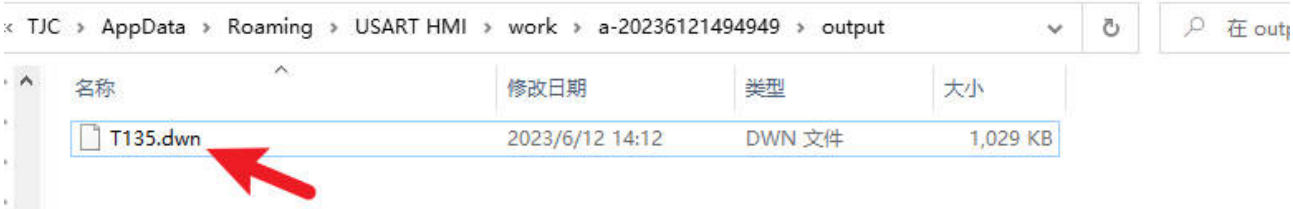
TJC8048X550_011 下载完成后屏幕如下图所示:

准备工作 2:

打开任一 TJC4832T135_011 的工程

1. 点击文件-输出生产文件

2. 将 tft 文件后缀名改为 dwn, 然后拷到 microSD 卡中



3. 将 microSD 卡插入屏幕，屏幕上电，如果中途拔插 microSD 卡更新文件，请点击“刷新 SD 卡文件”按钮

4. 将 X550 屏幕 tx 接 T135 屏幕的 rx，X550 屏幕 rx 接 T135 屏幕的 tx，给屏幕供电，共地。

> **注意:** 给 t0/t1/k0 系列下载时, 如果无法联机, 请在 x 系列的 TX 引脚和 t0/t1/k0 系列的 RX 引脚上串 1k 电阻降压

5. 联机波特率选择和 T13.5 寸屏幕一致波特率。

6. 点击联机按钮，联机成功后最上方白色文本框显示 T135 信息。

7. 选中需要下载的工程文件，然后点击下载按钮，此时 T135 屏幕开始进入下载界面，等待下载完成即可即可。

此工程需要下载到串口屏实物中使用, 请勿在模拟器上使用

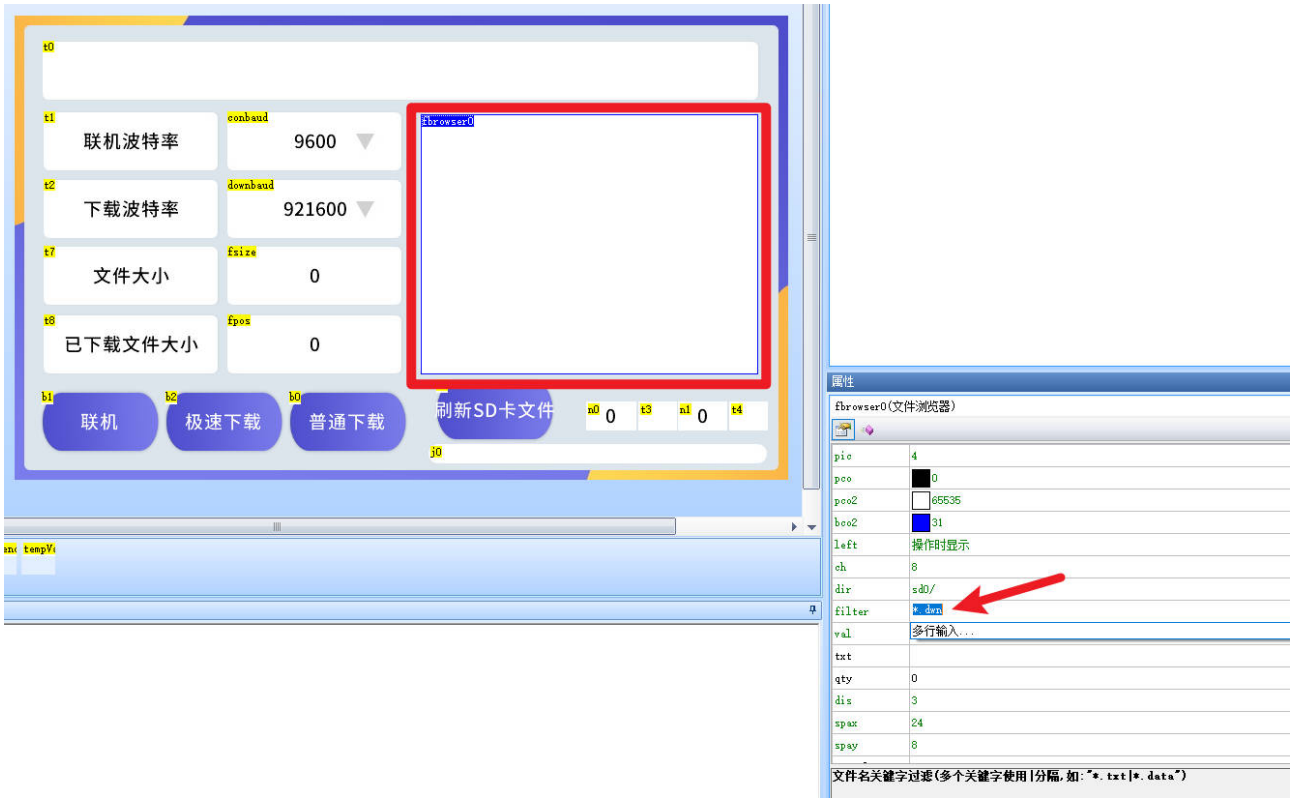
9.5.18.1 为什么要将生成的 tft 文件后缀名改为 dwn

因为我们是要给别的屏幕升级，而不是给自己升级

如果上电时串口屏检测到 microSD 卡中有 tft 格式的文件, 就会进入升级流程, 如果不是 tft 格式的文件, 就不会进入升级流程。

这个文件的后缀名你可以改成任意你喜欢的, 建议 1-4 字符之间

然后选中文件浏览器, 修改 filter 属性为你所设定的文件格式即可 (或者将其清空, 这样所有文件类型都会显示)



9.5.18.2 X 系列给其他屏幕下载工程-样例工程下载

演示工程下载链接:

《X 系列给其他屏幕下载工程》演示工程下载

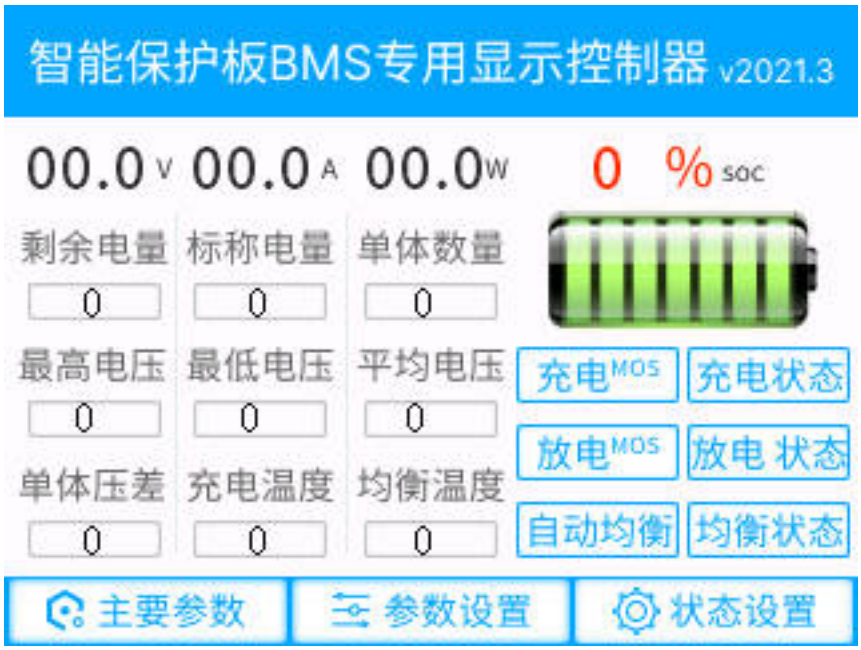
9.6 UI 样例工程

9.6.1 AGV 小车 UI(X2_800x480)

样例工程下载

《AGV 小车 UI(X2_800x480)》演示工程下载

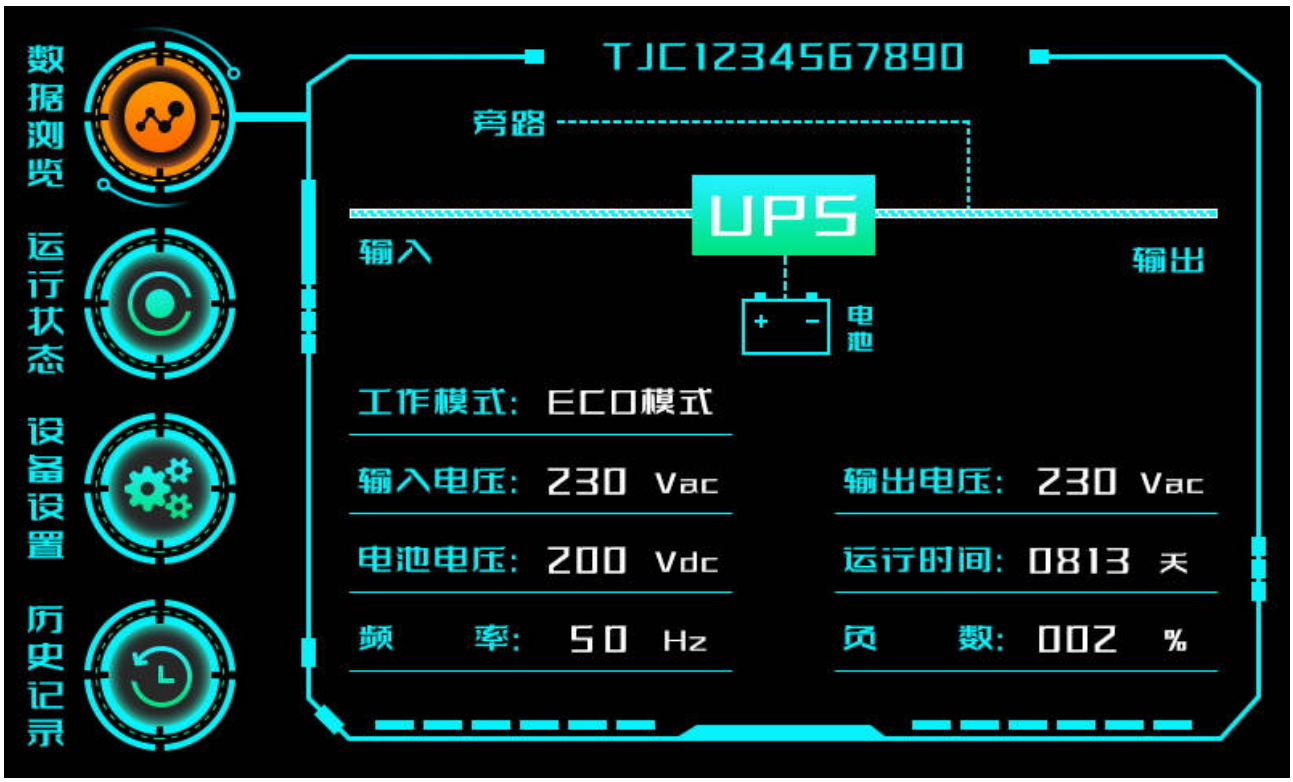
9.6.2 BMS 控制器 UI(T1_240x320)



样例工程下载

《BMS 控制器 UI(T1_240x320)》演示工程下载

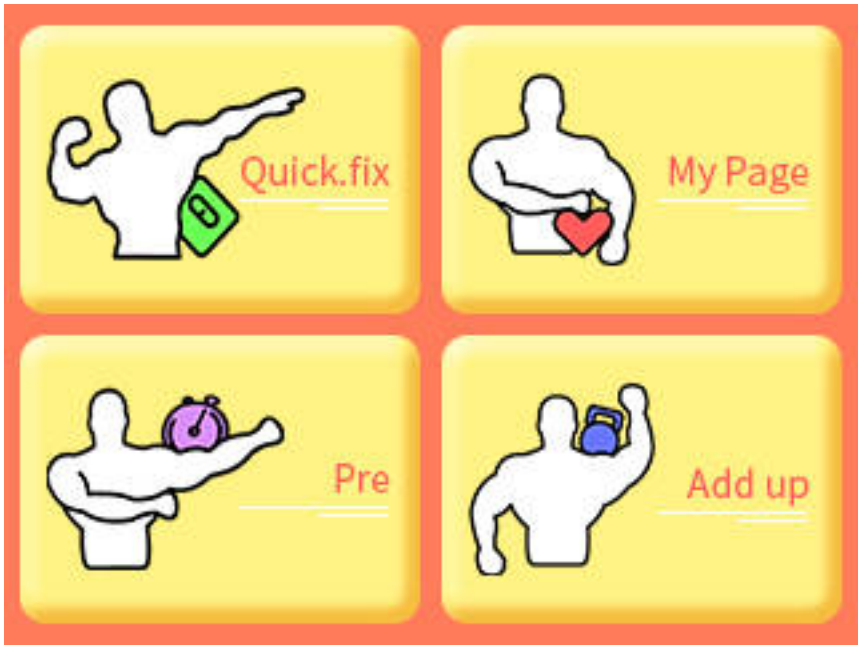
9.6.3 UPS 储能电源 UI(X2_800x480)



样例工程下载

《UPS 储能电源 UI(X2_800x480)》演示工程下载

9.6.4 按摩仪 UI(T1_320x240)



样例工程下载

《按摩仪 UI(T1_320x240)》演示工程下载

9.6.5 按摩仪蓝色 UI(T1_320x240)

样例工程下载

《按摩仪蓝色 UI(T1_320x240)》演示工程下载

9.6.6 按摩椅 UI(X3_800x480)

样例工程下载

《按摩椅 UI(X3_800x480)》演示工程下载

9.6.7 按摩椅 UI(X5_1024x600)

样例工程下载

《按摩椅 UI(X5_1024x600)》演示工程下载

9.6.8 便携式检测仪 UI(K0_320x240)

样例工程下载

《便携式检测仪 UI(K0_320x240)》演示工程下载

9.6.9 叉车充电器 UI(X2_480x272)

样例工程下载

《叉车充电器 UI(X2_480x272)》演示工程下载

9.6.10 超声波清洗机 UI(X5_800x480)

样例工程下载

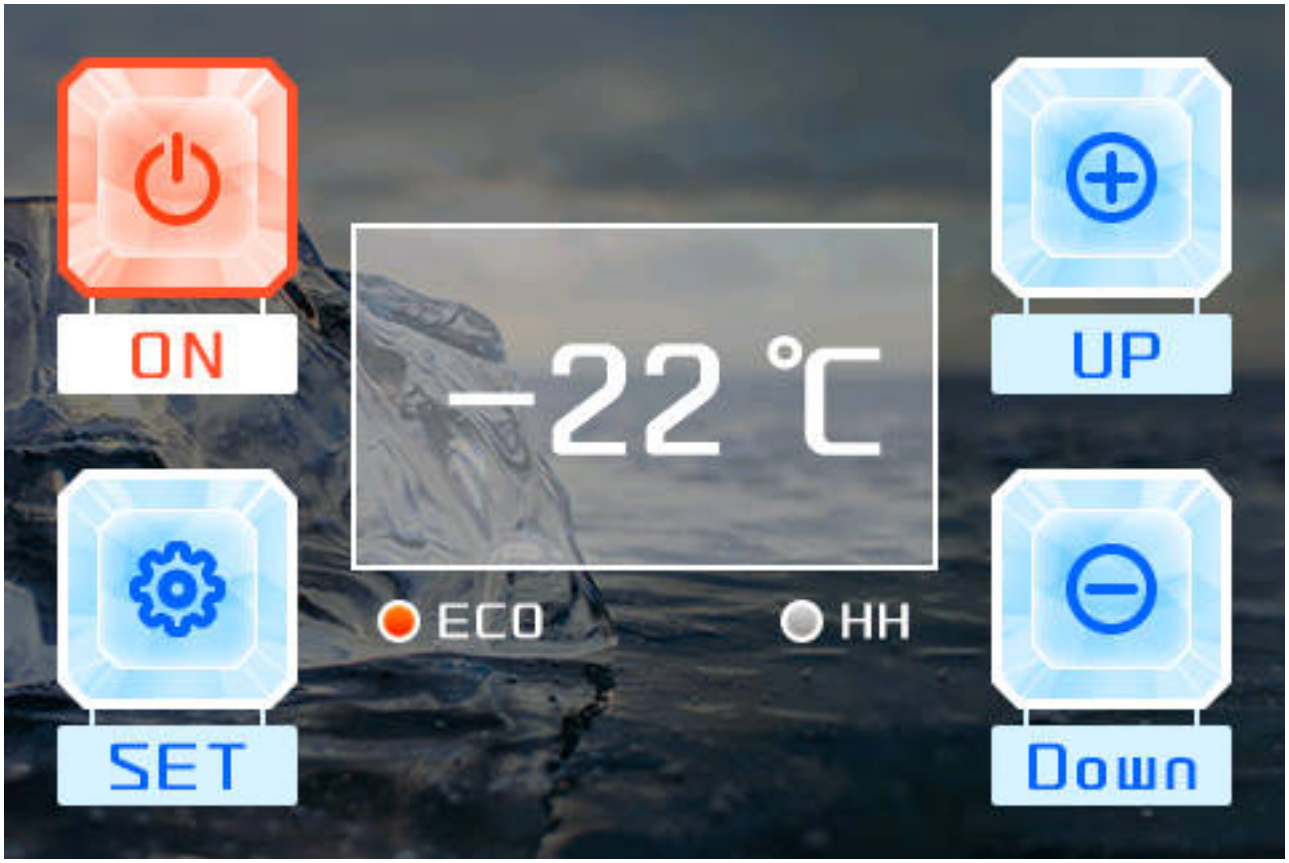
《超声波清洗机 UI(X5_800x480)》演示工程下载

9.6.11 炒菜机 UI(T1_480x272)

样例工程下载

《炒菜机 UI(T1_480x272)》演示工程下载

9.6.12 车载冰箱 UI(T1_480x320)



样例工程下载

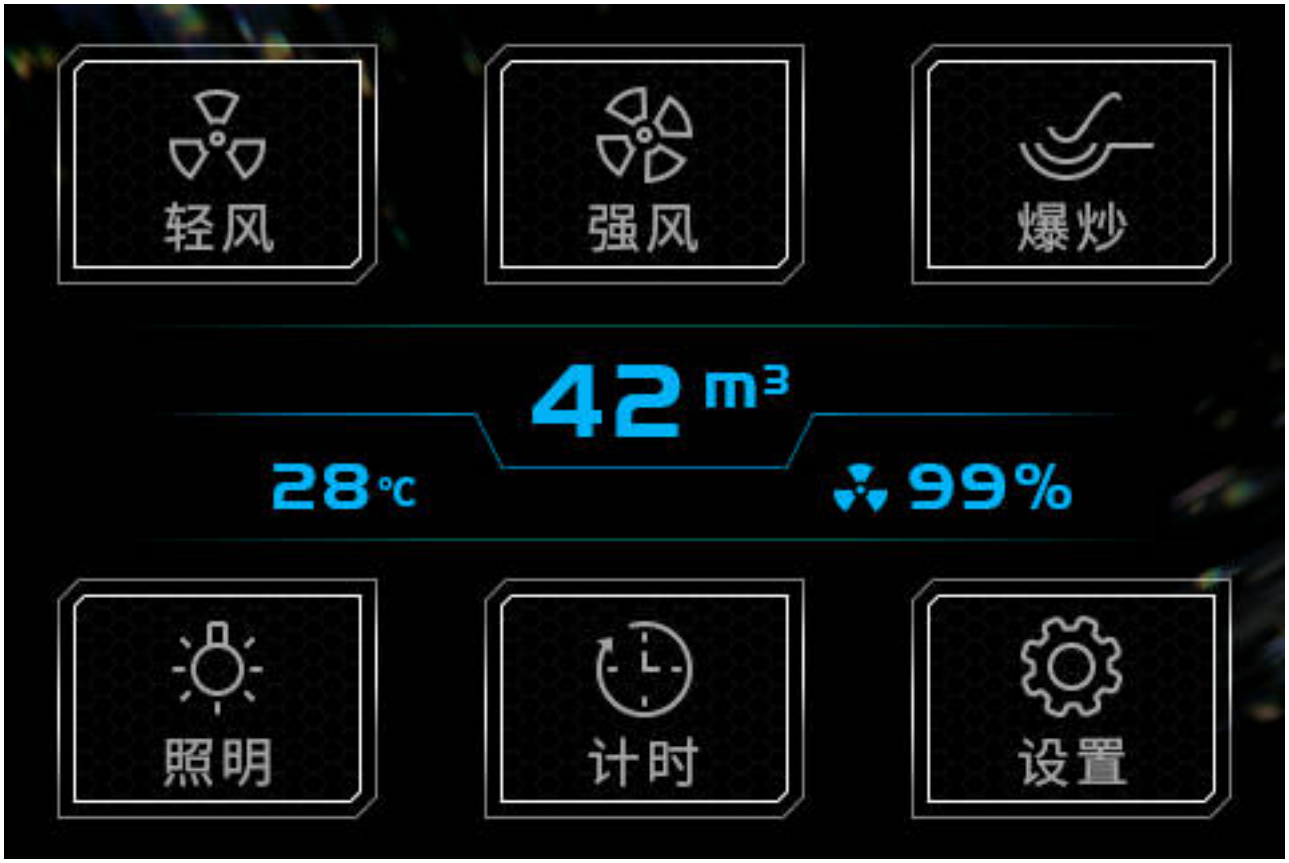
《车载冰箱 UI(T1_480x320)》演示工程下载

9.6.13 充电桩行业 UI(T1_320x240)

样例工程下载

《充电桩行业 UI(T1_320x240)》演示工程下载

9.6.14 抽油烟机 UI(T1_480x320)



样例工程下载

《抽油烟机 UI(T1_480x320)》演示工程下载

9.6.15 灯光控制器 UI(T1_480x320)

样例工程下载

《灯光控制器 UI(T1_480x320)》演示工程下载

9.6.16 电梯屏 UI(X5_1024x600)

样例工程下载

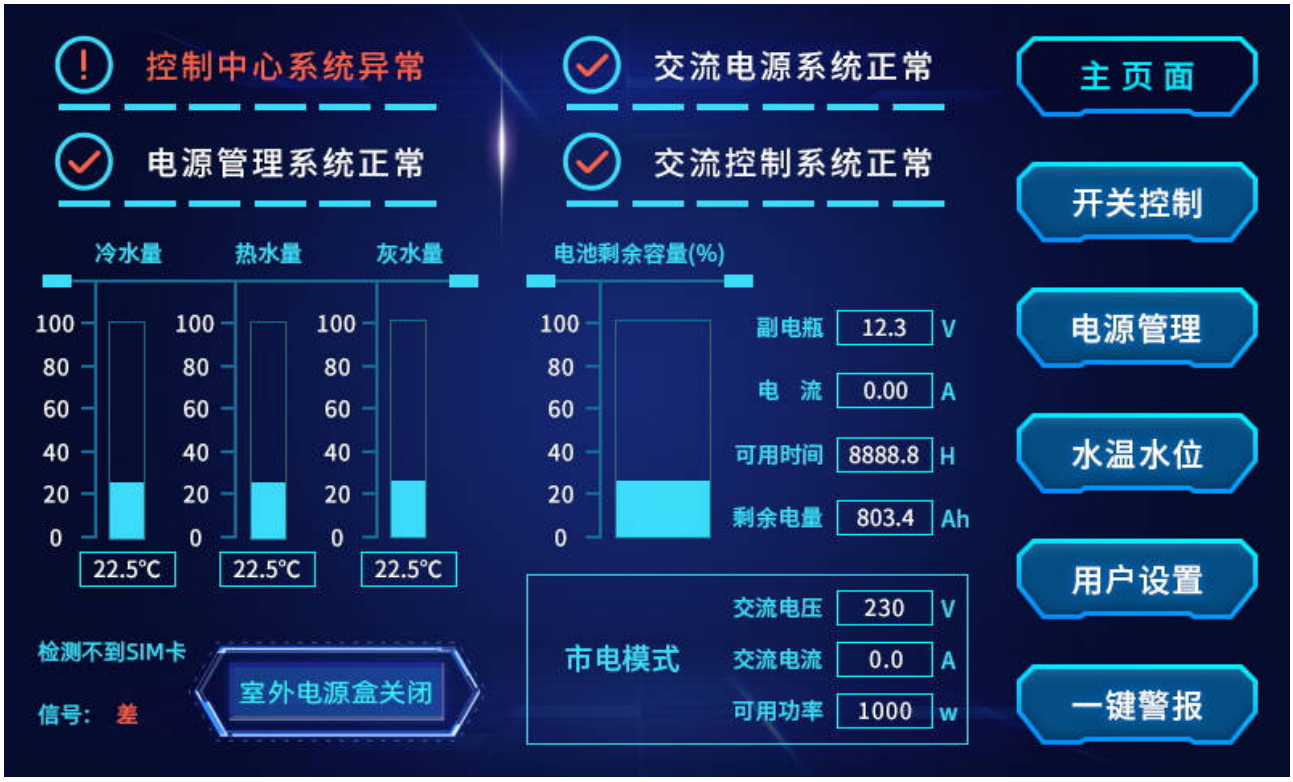
《电梯屏 UI(X5_1024x600)》演示工程下载

9.6.17 电子智能储物柜 UI(X2_800x480)

样例工程下载

《电子智能储物柜 UI(X2_800x480)》演示工程下载

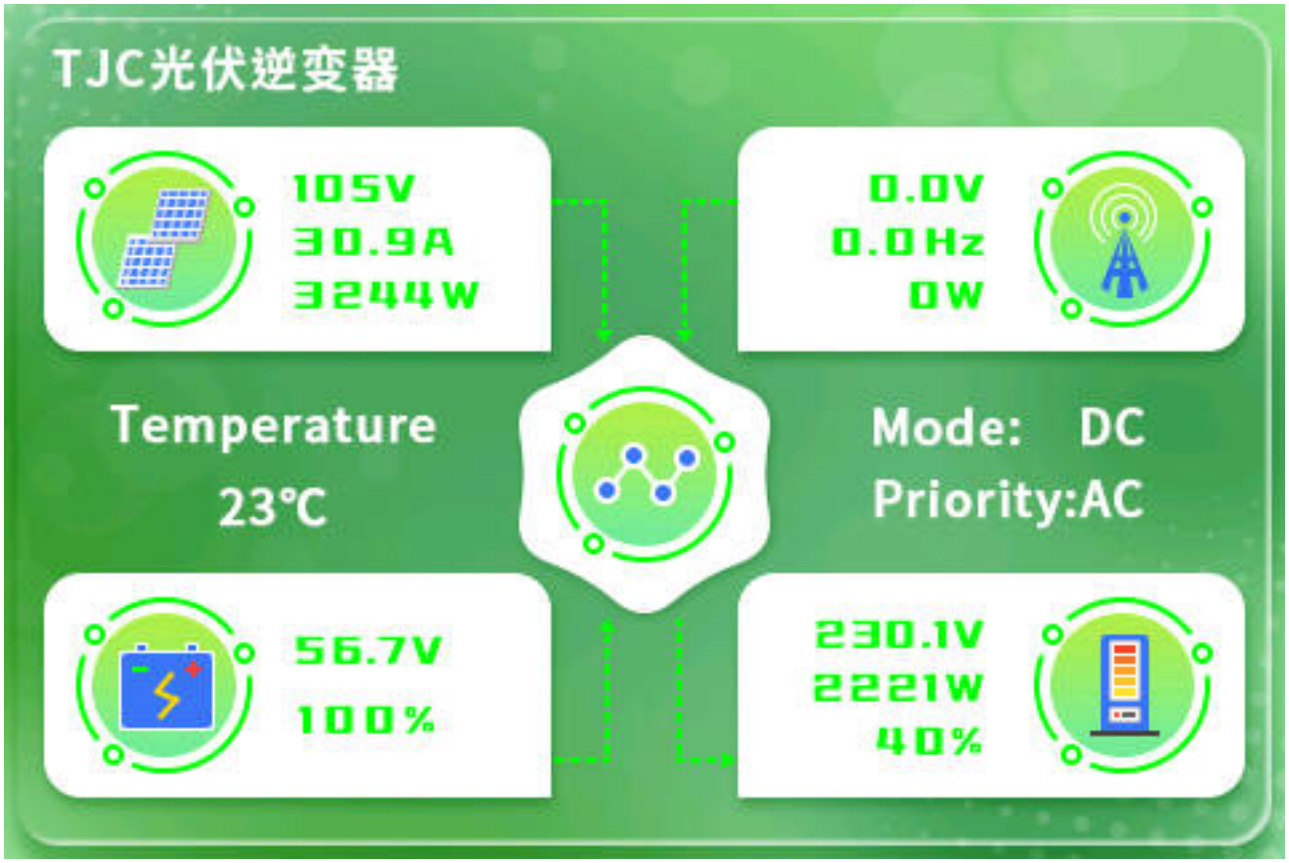
9.6.18 房车控制面板 UI(X5_800x480)



样例工程下载

《房车控制面板 UI(X5_800x480)》演示工程下载

9.6.19 光伏逆变器 UI(T1_480x320)



样例工程下载

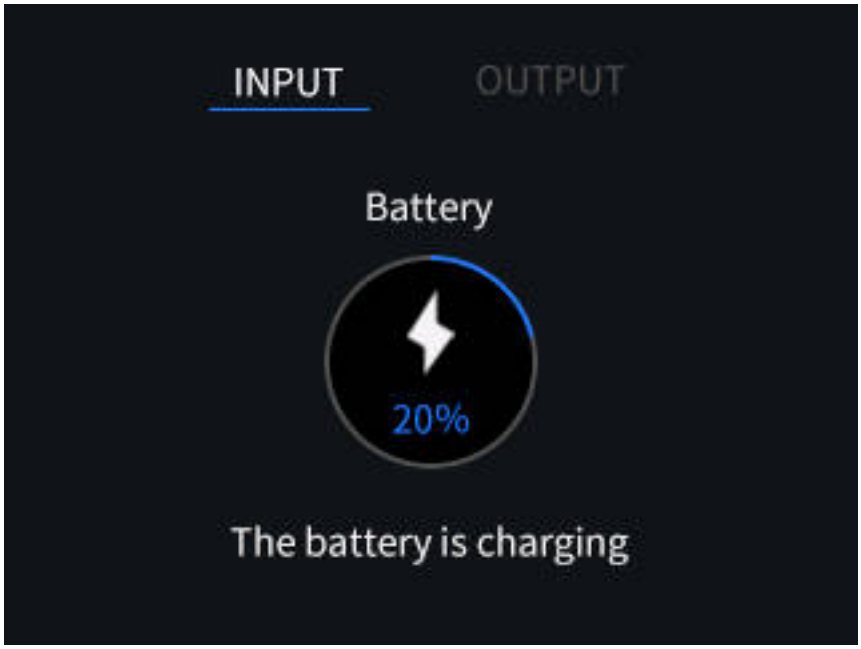
《光伏逆变器 UI(T1_480x320)》演示工程下载

9.6.20 焊接机行业 UI(X3_800x480)

样例工程下载

《焊接机行业 UI(X3_800x480)》演示工程下载

9.6.21 户外电源 UI(T1_320x240)



样例工程下载

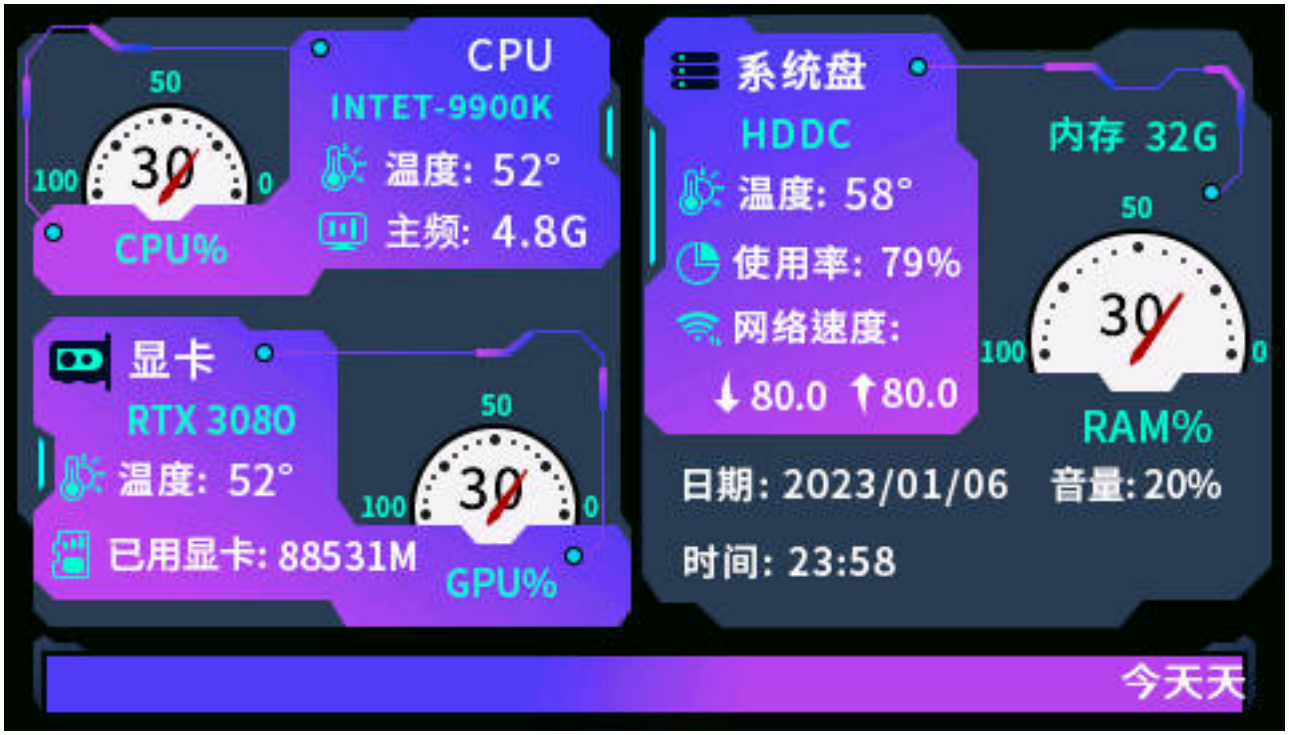
《户外电源 UI(T1_320x240)》演示工程下载

9.6.22 会议系统设备 UI(X2_800x480)

样例工程下载

《会议系统设备 UI(X2_800x480)》演示工程下载

9.6.23 机箱副屏 UI(X2_480x272)



样例工程下载

《机箱副屏 UI(X2_480x272)》演示工程下载

9.6.24 交流充电桩 UI(T1_320x240)

样例工程下载

《交流充电桩 UI(T1_320x240)》演示工程下载

9.6.25 交流充电桩 UI(T1_480x320)

样例工程下载

《交流充电桩 UI(T1_480x320)》演示工程下载

9.6.26 进度条控件 UI(X5_1024x600)

样例工程下载

《进度条控件 UI(X5_1024x600)》演示工程下载

9.6.27 咖啡机 UI1(T1_480x320)

样例工程下载

《咖啡机 UI1(T1_480x320)》演示工程下载

9.6.28 咖啡机 UI2(T1_480x320)

样例工程下载

《咖啡机 UI2(T1_480x320)》演示工程下载

9.6.29 咖啡机 UI3(T1_480x272)

样例工程下载

《咖啡机 UI3(T1_480x272)》演示工程下载

9.6.30 咖啡机 UI4(X5_800x480)

样例工程下载

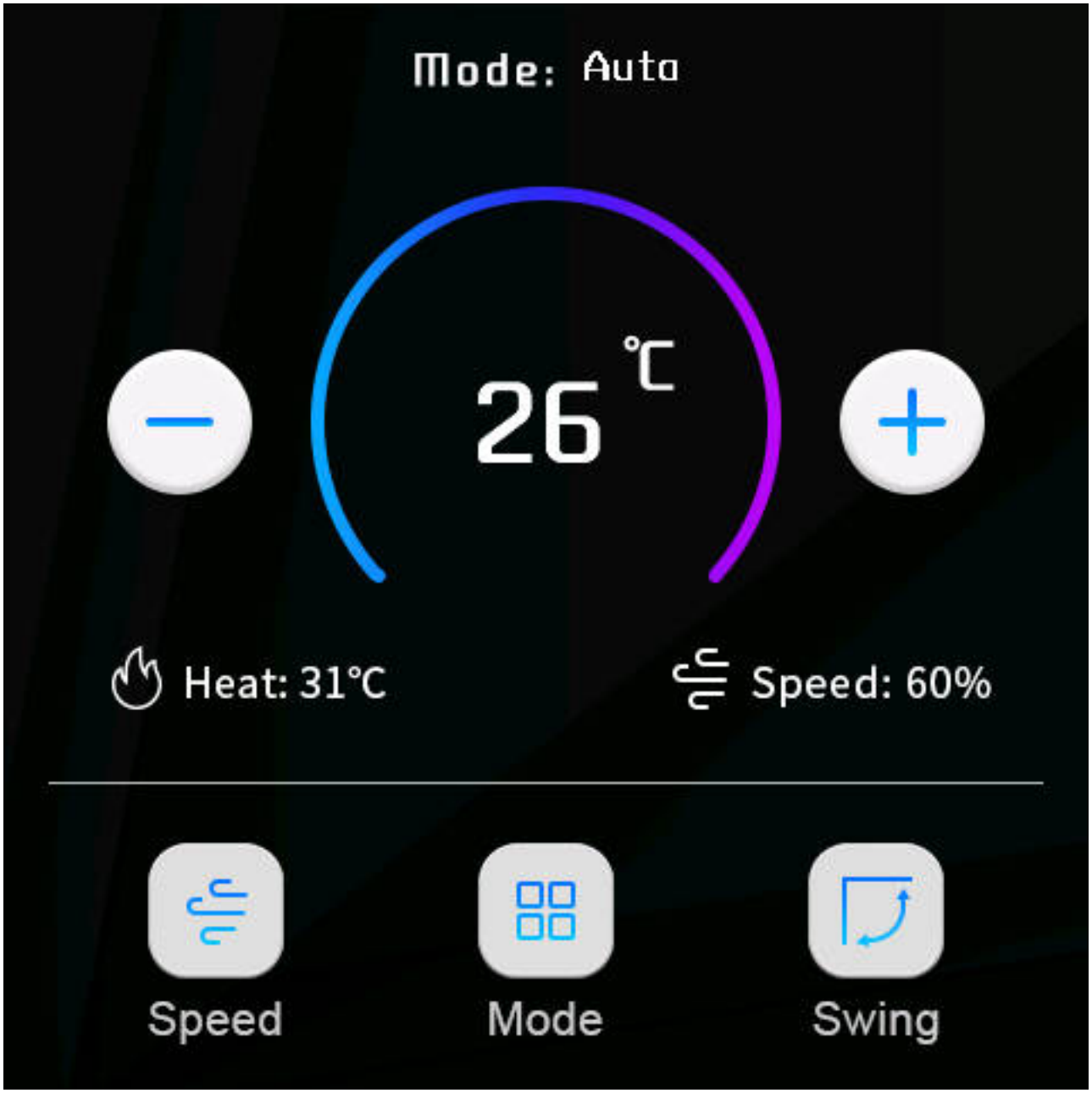
《咖啡机 UI4(X5_800x480)》演示工程下载

9.6.31 考勤机 UI(T1_320x240)

样例工程下载

《考勤机 UI(T1_320x240)》演示工程下载

9.6.32 空调线控器 UI(X5_480x480)



样例工程下载

《空调线控器 UI(X5_480x480)》演示工程下载

9.6.33 空气检测仪 UI(T1_480x320)

样例工程下载

《空气检测仪 UI(T1_480x320)》演示工程下载

9.6.34 空气炸锅 UI(T1_480x272)

样例工程下载

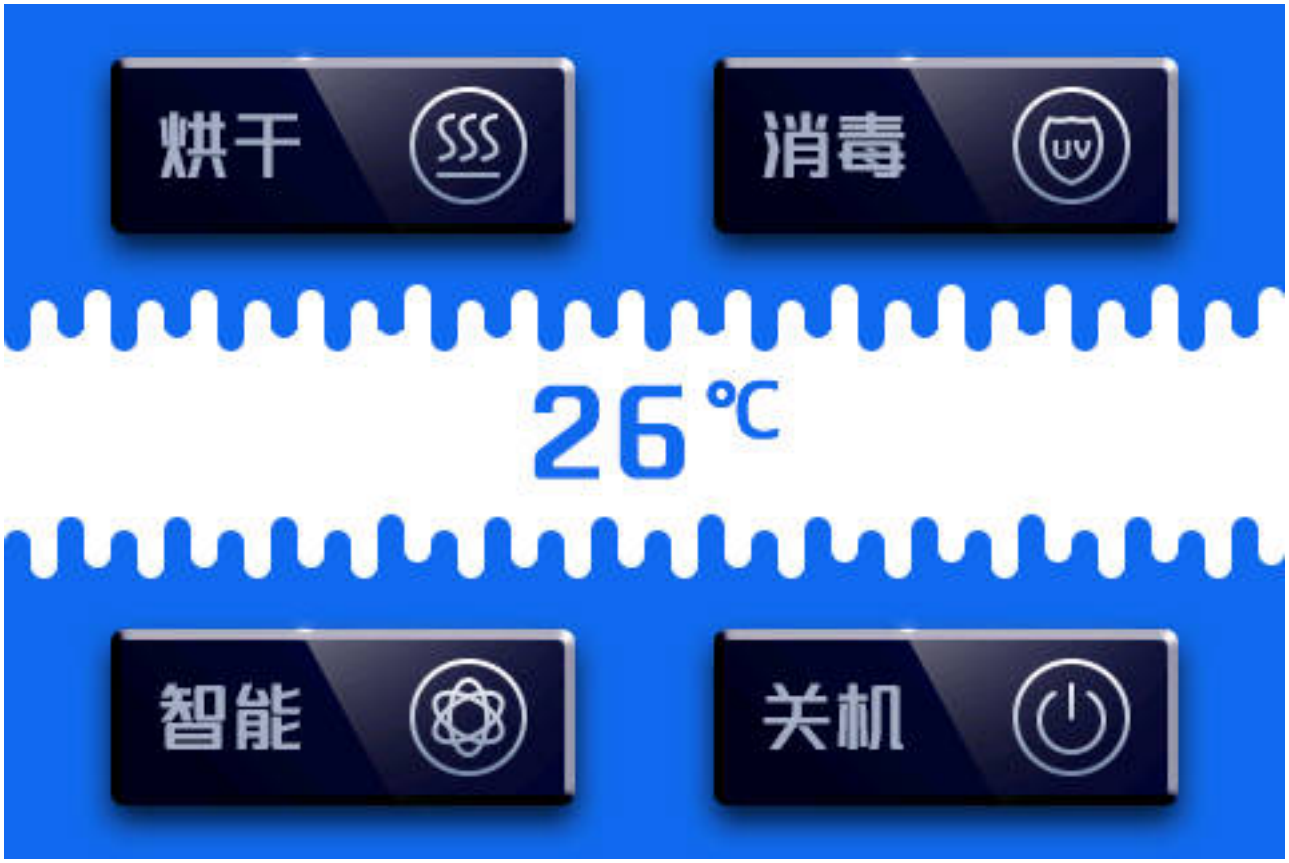
《空气炸锅 UI(T1_480x272)》演示工程下载

9.6.35 螺丝机 UI(X5_800x480)

样例工程下载

《螺丝机 UI(X5_800x480)》演示工程下载

9.6.36 毛巾烘干机 UI(T1_480x320)



样例工程下载

《毛巾烘干机 UI(T1_480x320)》演示工程下载

9.6.37 美容仪器 UI(X2_800x480)

样例工程下载

《美容仪器 UI(X2_800x480)》演示工程下载

9.6.38 美容仪器 UI(X2_800x600)

样例工程下载

《美容仪器 UI(X2_800x600)》演示工程下载

9.6.39 跑步机 UI(X5_1024x600)

样例工程下载

《跑步机 UI(X5_1024x600)》演示工程下载

9.6.40 泡茶机 UI(T1_480x320)

样例工程下载

《泡茶机 UI(T1_480x320)》演示工程下载

9.6.41 喷码机 UI(T1_480x272)

样例工程下载

《喷码机 UI(T1_480x272)》演示工程下载

9.6.42 破壁机 UI(T1_480x272)

样例工程下载

《破壁机 UI(T1_480x272)》演示工程下载

9.6.43 汽车充电桩 UI(X5_800x480)

样例工程下载

《汽车充电桩 UI(X5_800x480)》演示工程下载

9.6.44 取暖器 UI(T1_480x320)

样例工程下载

《取暖器 UI(T1_480x320)》演示工程下载

9.6.45 热水壶 UI(T1_320x240)

样例工程下载

《热水壶 UI(T1_320x240)》演示工程下载

9.6.46 热水器 UI(T1_480x272)

样例工程下载

《热水器 UI(T1_480x272)》演示工程下载

9.6.47 商用烤箱 UI(X3_800x480)

样例工程下载

《商用烤箱 UI(X3_800x480)》演示工程下载

9.6.48 摄影灯 UI(K0_320x240)

样例工程下载

《摄影灯 UI(K0_320x240)》演示工程下载

9.6.49 数字功放一体机 UI(X3_800x480)

样例工程下载

《数字功放一体机 UI(X3_800x480)》演示工程下载

9.6.50 水质检测仪 UI(X2_800x480)

样例工程下载

《水质检测仪 UI(X2_800x480)》演示工程下载

9.6.51 睡眠呼吸机 UI(T1_480x320)

样例工程下载

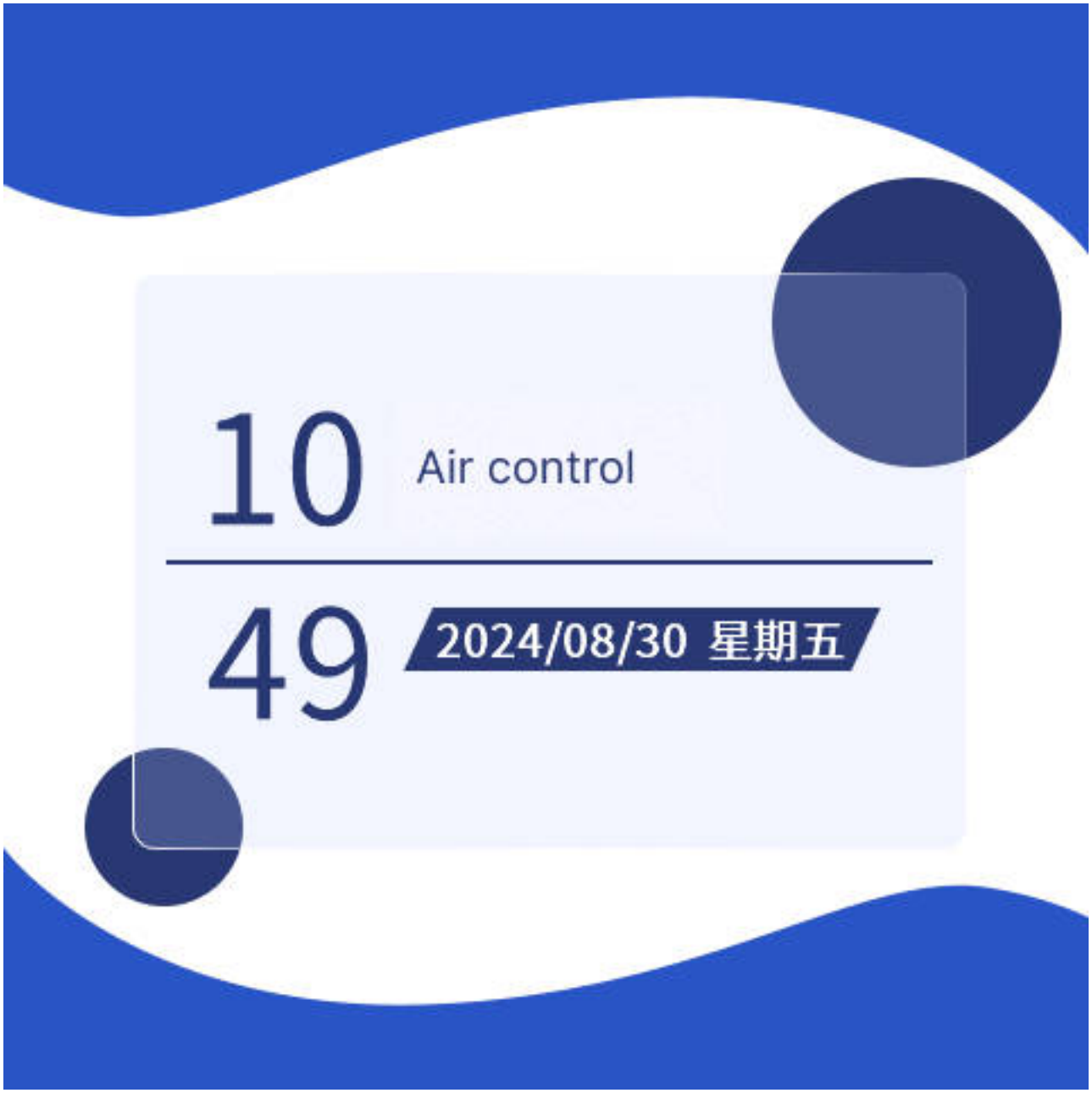
《睡眠呼吸机 UI(T1_480x320)》演示工程下载

9.6.52 微波炉 UI(T1_480x320)

样例工程下载

《微波炉 UI(T1_480x320)》演示工程下载

9.6.53 温度控制器 UI(X5_480x480)



样例工程下载

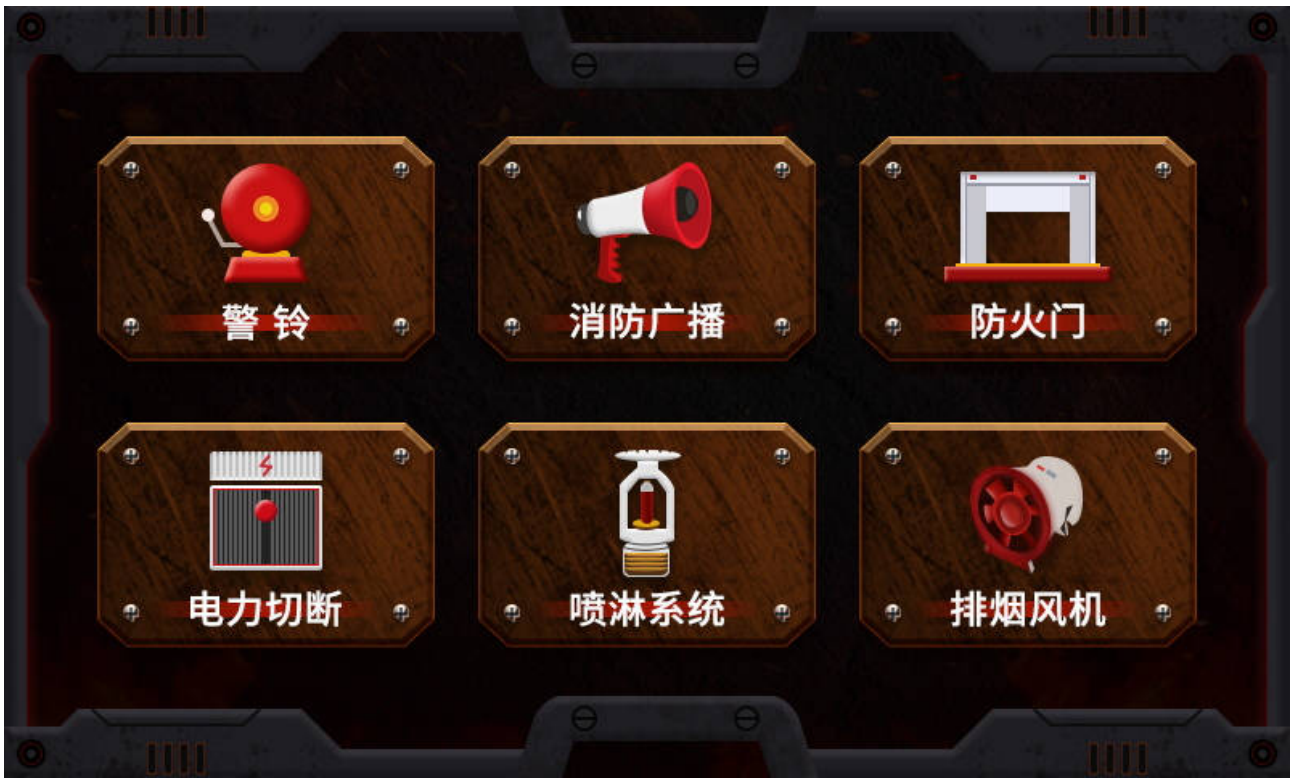
《温度控制器 UI(X5_480x480)》演示工程下载

9.6.54 物流高拍仪 UI(X5_1024x600)

样例工程下载

《物流高拍仪 UI(X5_1024x600)》演示工程下载

9.6.55 消防主机 UI(X5_800x480)



样例工程下载

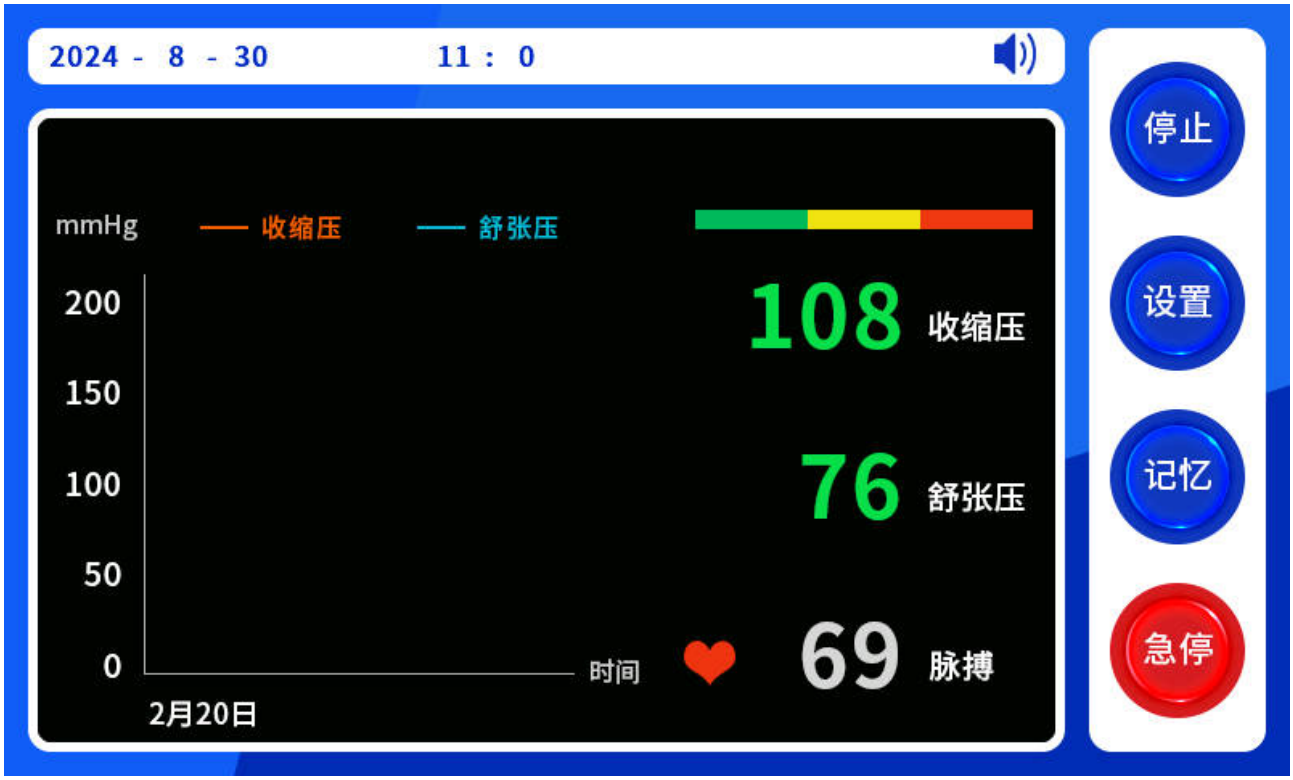
《消防主机 UI(X5_800x480)》演示工程下载

9.6.56 新风系统 UI(X5_800x480)

样例工程下载

《新风系统 UI(X5_800x480)》演示工程下载

9.6.57 血压计 UI(X2_800x480)



样例工程下载

《血压计 UI(X2_800x480)》演示工程下载

9.6.58 液体灌装机 UI(X5_800x480)

样例工程下载

《液体灌装机 UI(X5_800x480)》演示工程下载

9.6.59 医疗美容 UI(X3_800x480)

样例工程下载

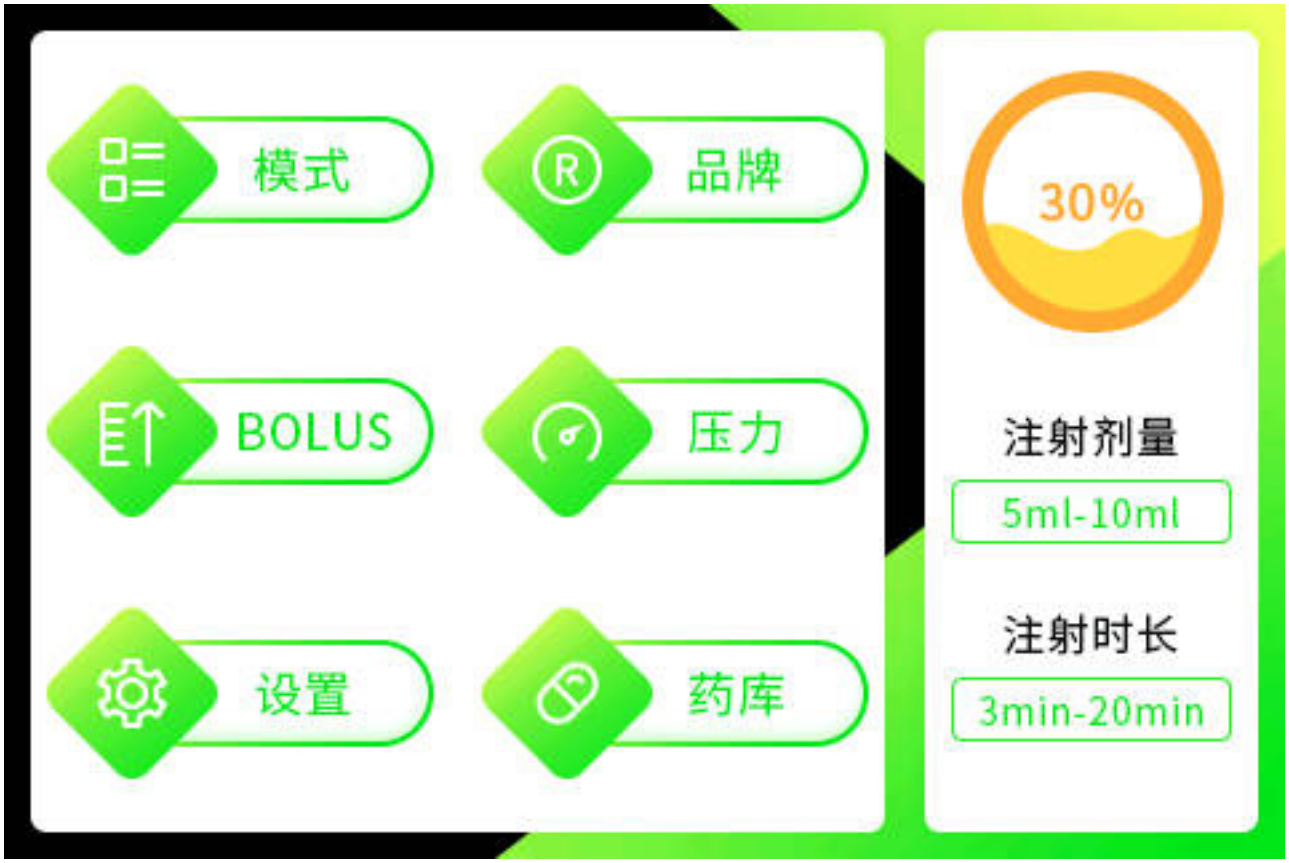
《医疗美容 UI(X3_800x480)》演示工程下载

9.6.60 医疗美容 UI(X5_480x272)

样例工程下载

《医疗美容 UI(X5_480x272)》演示工程下载

9.6.61 医疗仪器输液泵 UI(T1_480x320)



样例工程下载

《医疗仪器输液泵 UI(T1_480x320)》演示工程下载

9.6.62 照明行业 UI(X5_480x272)

样例工程下载

《照明行业 UI(X5_480x272)》演示工程下载

9.6.63 照明行业 UI(X5_800x480)

样例工程下载

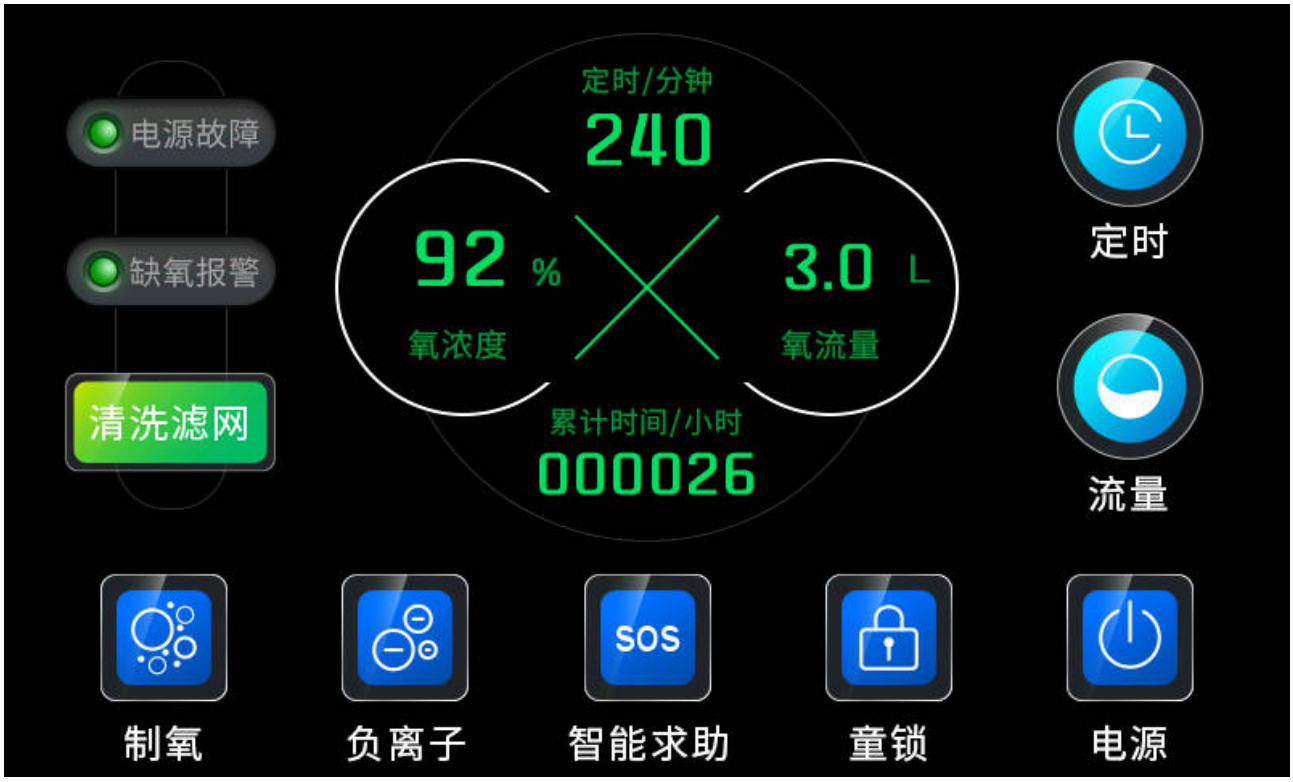
《照明行业 UI(X5_800x480)》演示工程下载

9.6.64 制冰机 UI(X3_480x272)

样例工程下载

《制冰机 UI(X3_480x272)》演示工程下载

9.6.65 制氧机 UI(X2_800x480)



样例工程下载

《制氧机 UI(X2_800x480)》演示工程下载

9.6.66 智能称重一体机 UI(X2_800x480)

样例工程下载

《智能称重一体机 UI(X2_800x480)》演示工程下载

9.6.67 智能家居 UI(X5_480x480)

样例工程下载

《智能家居 UI(X5_480x480)》演示工程下载

9.6.68 智能马桶 UI(T1_320x240)

样例工程下载

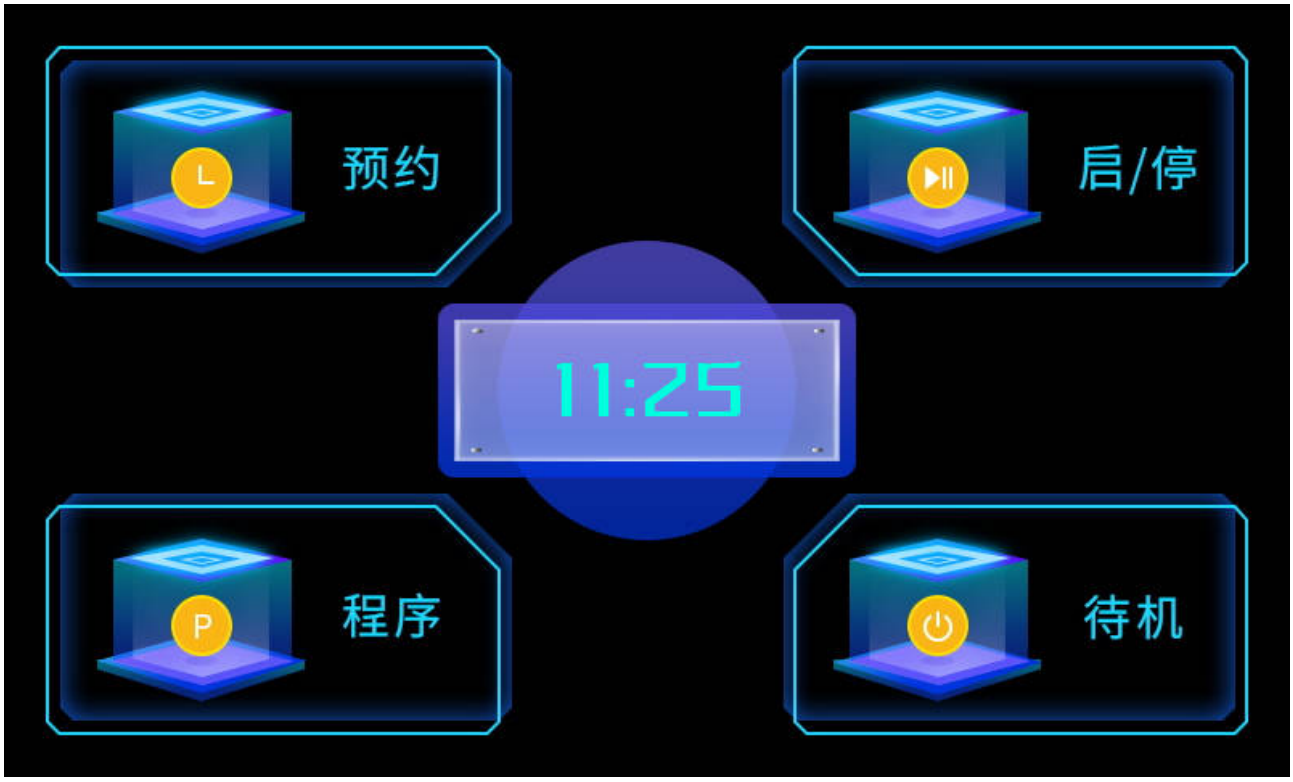
《智能马桶 UI(T1_320x240)》演示工程下载

9.6.69 智能燃气灶 UI(T1_480x272)

样例工程下载

《智能燃气灶 UI(T1_480x272)》演示工程下载

9.6.70 智能洗碗机 UI(X2_800x480)



样例工程下载

《智能洗碗机 UI(X2_800x480)》演示工程下载

9.6.71 智能洗衣机 UI(T1_480x272)

样例工程下载

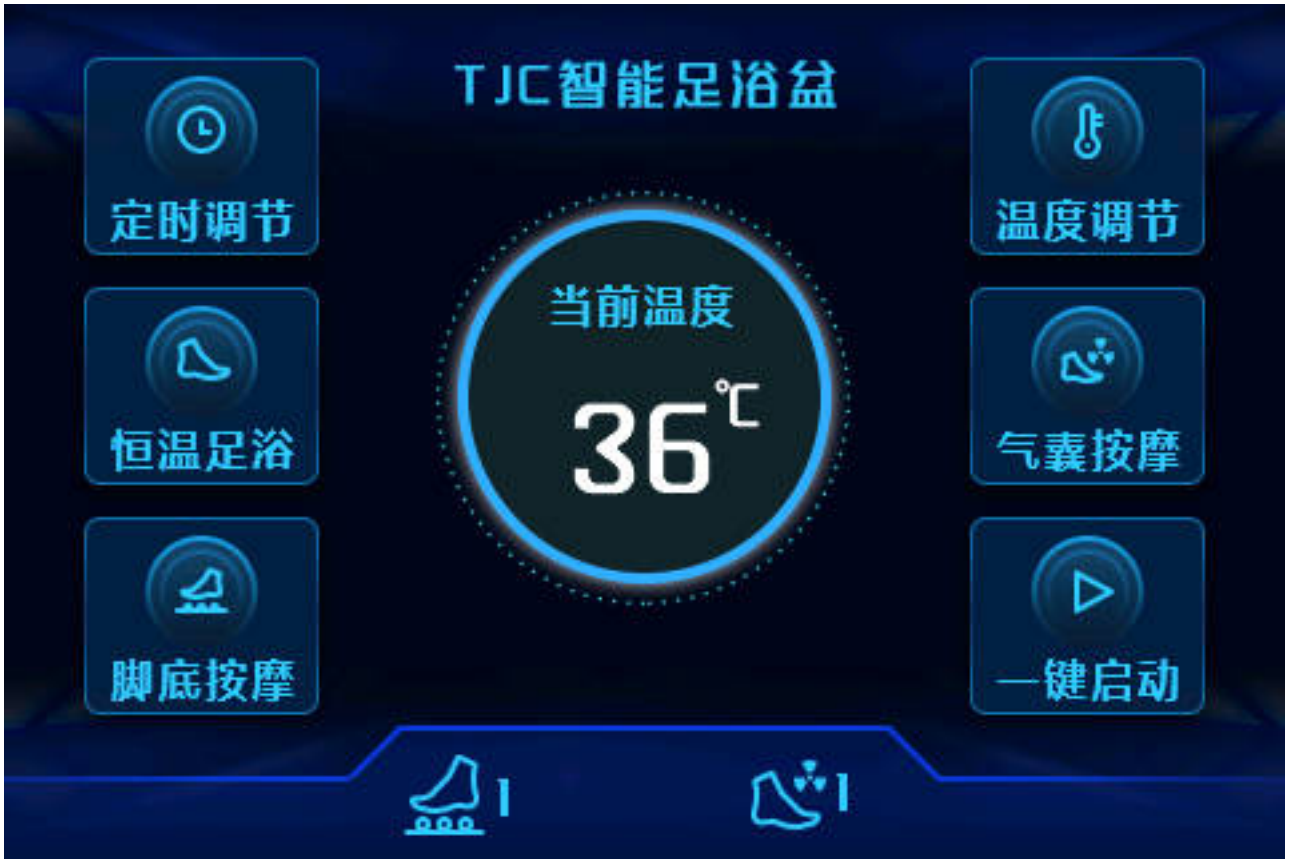
《智能洗衣机 UI(T1_480x272)》演示工程下载

9.6.72 智能消毒柜 UI(X2_800x480)

样例工程下载

《智能消毒柜 UI(X2_800x480)》演示工程下载

9.6.73 智能足浴盆 UI(X2_480x320)



样例工程下载

《智能足浴盆 UI(X2_480x320)》演示工程下载

9.6.74 中频治疗仪 UI(X5_480x272)

样例工程下载

《中频治疗仪 UI(X5_480x272)》演示工程下载

9.6.75 自动贩卖机 (X5 10 寸 1024600)

样例工程下载

《自动贩卖机 (X5 10 寸 1024600)》演示工程下载

9.7 图片资源下载

9.7.1 按钮图片资源下载

按钮图片资源下载

9.7.2 背景图片资源下载

背景图片资源下载

9.8 免费字体下载

提示: 如何安装新字体 (page 1439)

9.8.1 思源黑体下载

该款字体是由 Google 和 Adobe 合作开发的。Google 将其命名为 Noto SansCJK，作为 Google 的 Noto 字体家族的成员。Adobe 则命名为 Source Han Sans，作为 Adobe 的 Source 字体家族的一员。Adobe 拥有字体设计的版权。发布的字体文件则可以不受限制的免费使用。这些字体文件使用 Apache 许可证版本 2

9.8.1.1 win10 和 win11 建议使用 otf 字体

《思源黑体 otf》下载

9.8.1.2 win7 建议使用 ttf 字体

《思源黑体 ttf》下载

9.8.1.3 数码管字体

数码管字体安装后，在字库软件中显示的字体为 DigifaceWide

《数码管字体》下载

已经生成好的数码管字体 zi 文件，从字高 16-字高 254 ·

《数码管字体字库文件》下载

9.8.2 免费中文字体列表

免费中文字体收集计划

9.8.3 字体下载相关链接

在做字库的时候有部分字体无法选择 (page 1360)

创建字库和导入字库 (page 125)

指定字库 (page 1284)

编译报错：*XXX* 初始值无效 (page 1342)

utf8 字库下如何只选择汉字 (page 1403)

如何安装新字体 (page 1439)

9.9 声音资源下载

仅 x3/x5 系列支持播放音频，可以使用 play 指令或者音频控件来播放

《哔音效》下载

《嘀音效》下载

《点击音效》下载

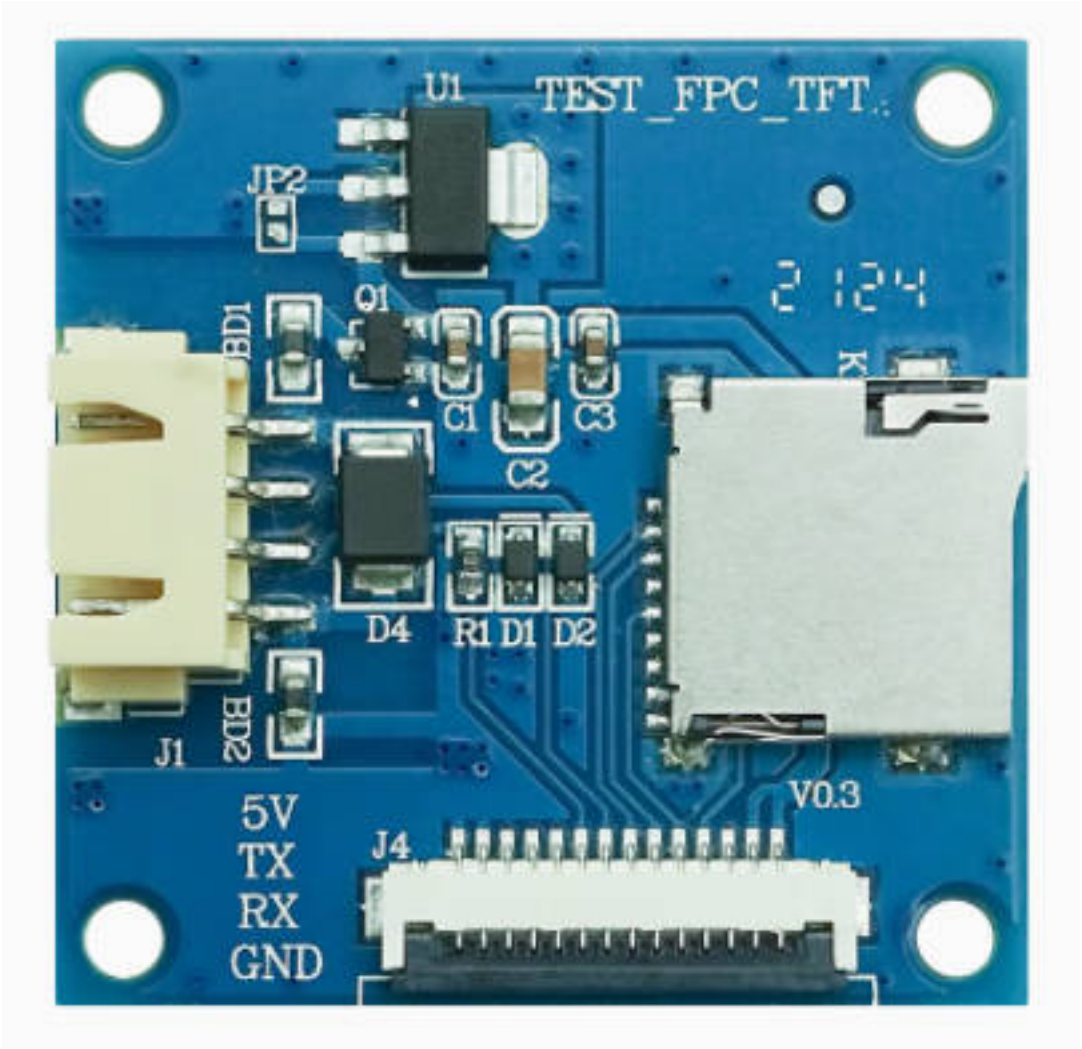
《蜂鸣器音效》下载

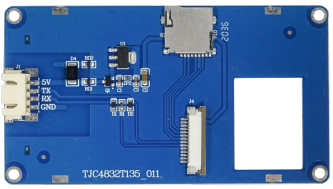
9.10 原理图下载

提示： 串口屏常用接口型号说明 (page 1268)

9.10.1 T1/X2 系列 COF 裸屏测试小板/背板原理图

下面两款 PCB 板的原理图是一样的，只是布局有所区别。





备注：

1、提供 LDO 和 DCDC 两个版本，PCB 原理图上的位号会与您现有的 PCB 板有所区别，量产前请先打样进行测试。

2、fpc1.0*14 座子立创参考料号：C19269357、C2797240。

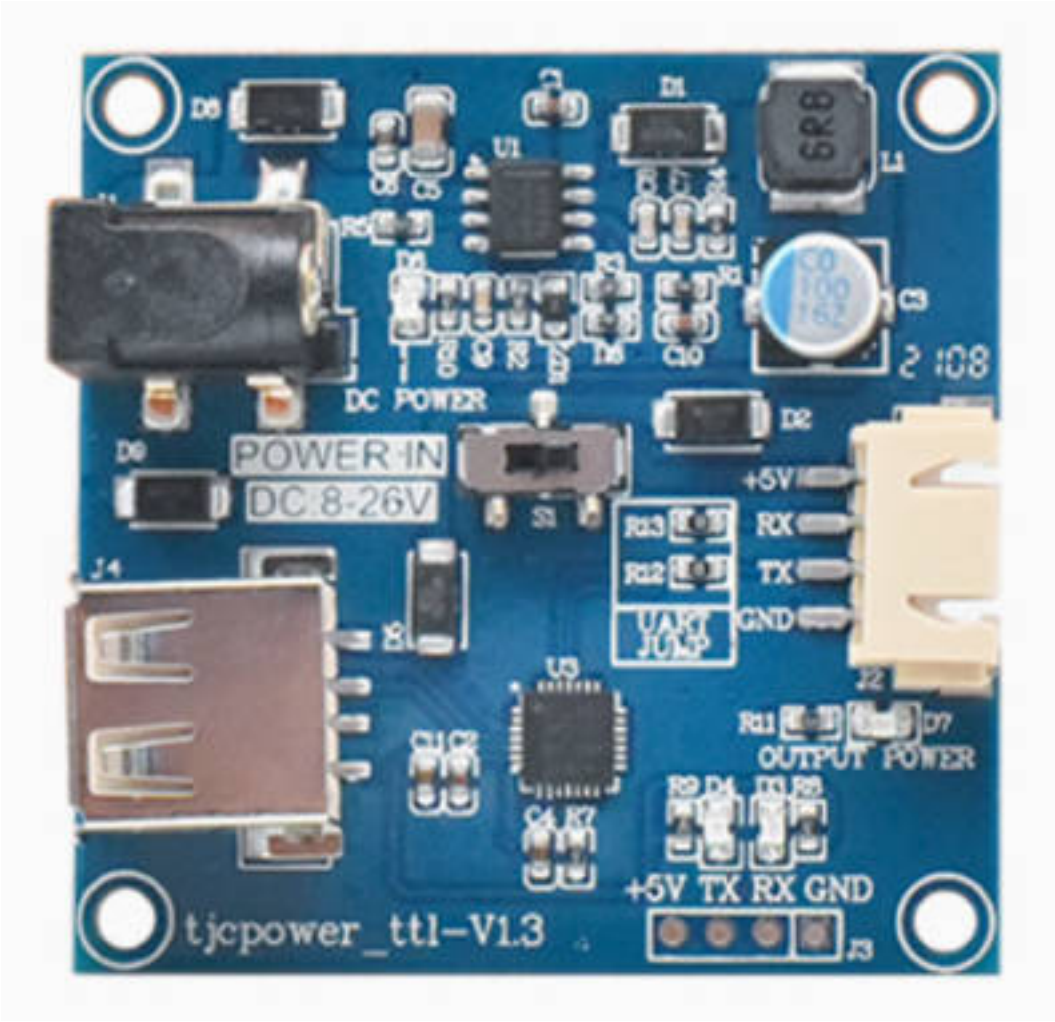
3、microSD 卡座立创参考料号：C2889260。

4、X2 系列优先推荐使用 DCDC 版本，能提供更大的电流。

下载《COF 裸屏测试小板原理图 LDO 版本》

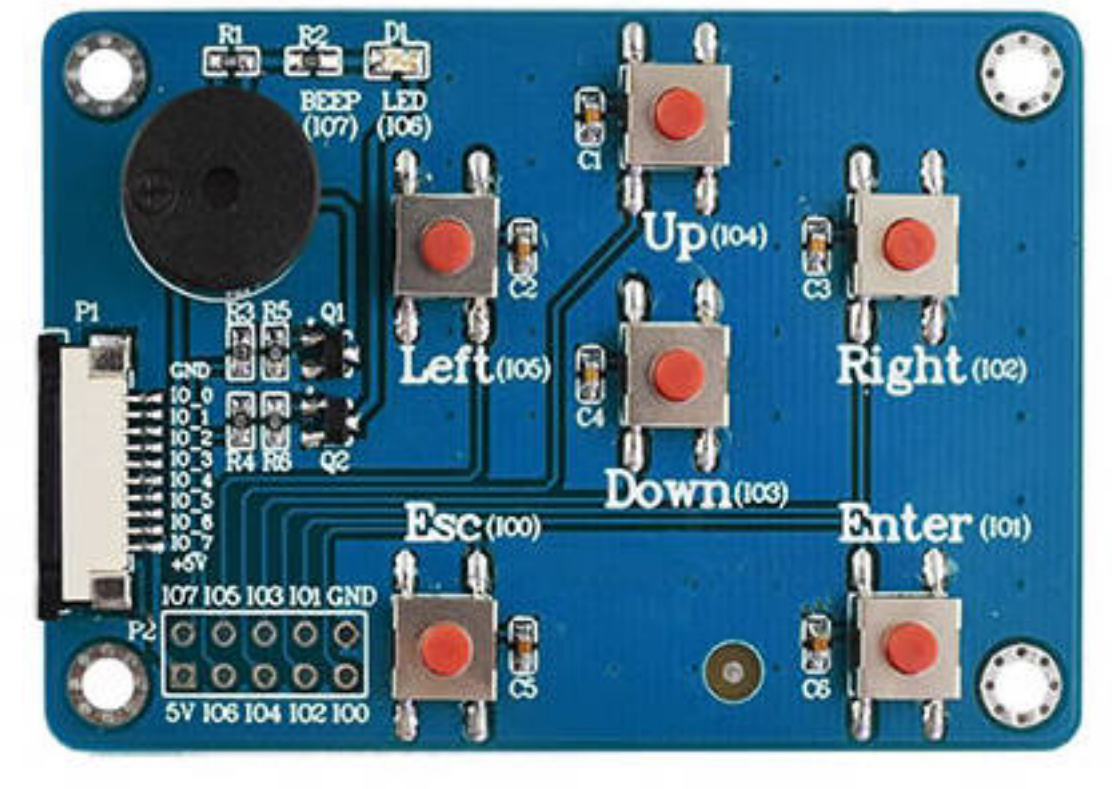
下载《COF 裸屏测试小板原理图 DCDC 版本》

9.10.2 USB 转 TTL 转接板



下载《USB 转 TTL 转接板原理图》

9.10.3 IO 拓展板



下载《IO 拓展板原理图》

FPC 的接口

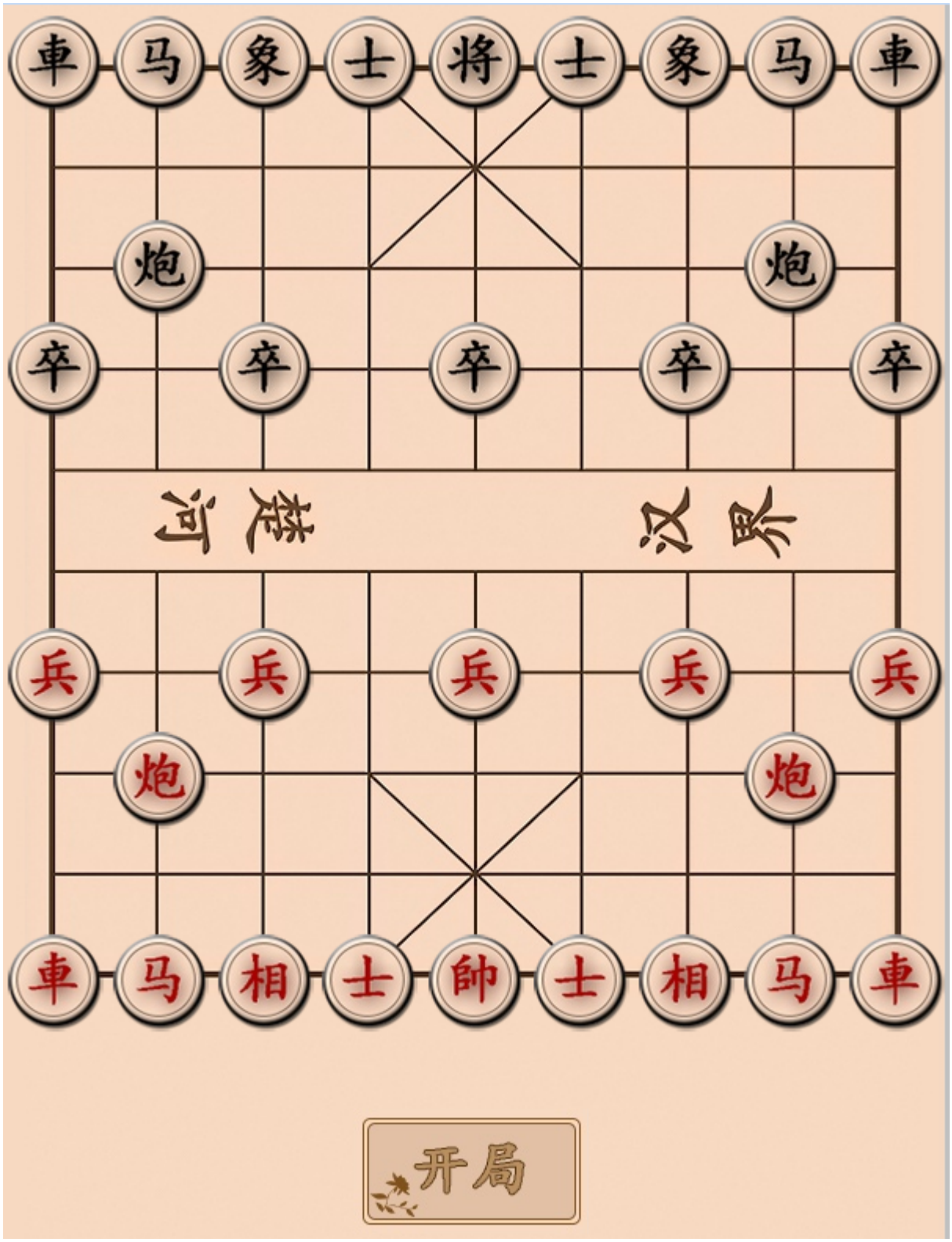
FPC 软排接口 (page 91)

9.11 网友提供应用样例

注意: 以下样例为网友提交, 我们不保证每一个例子都能完全正常使用，仅做参考!

9.11.1 中国象棋

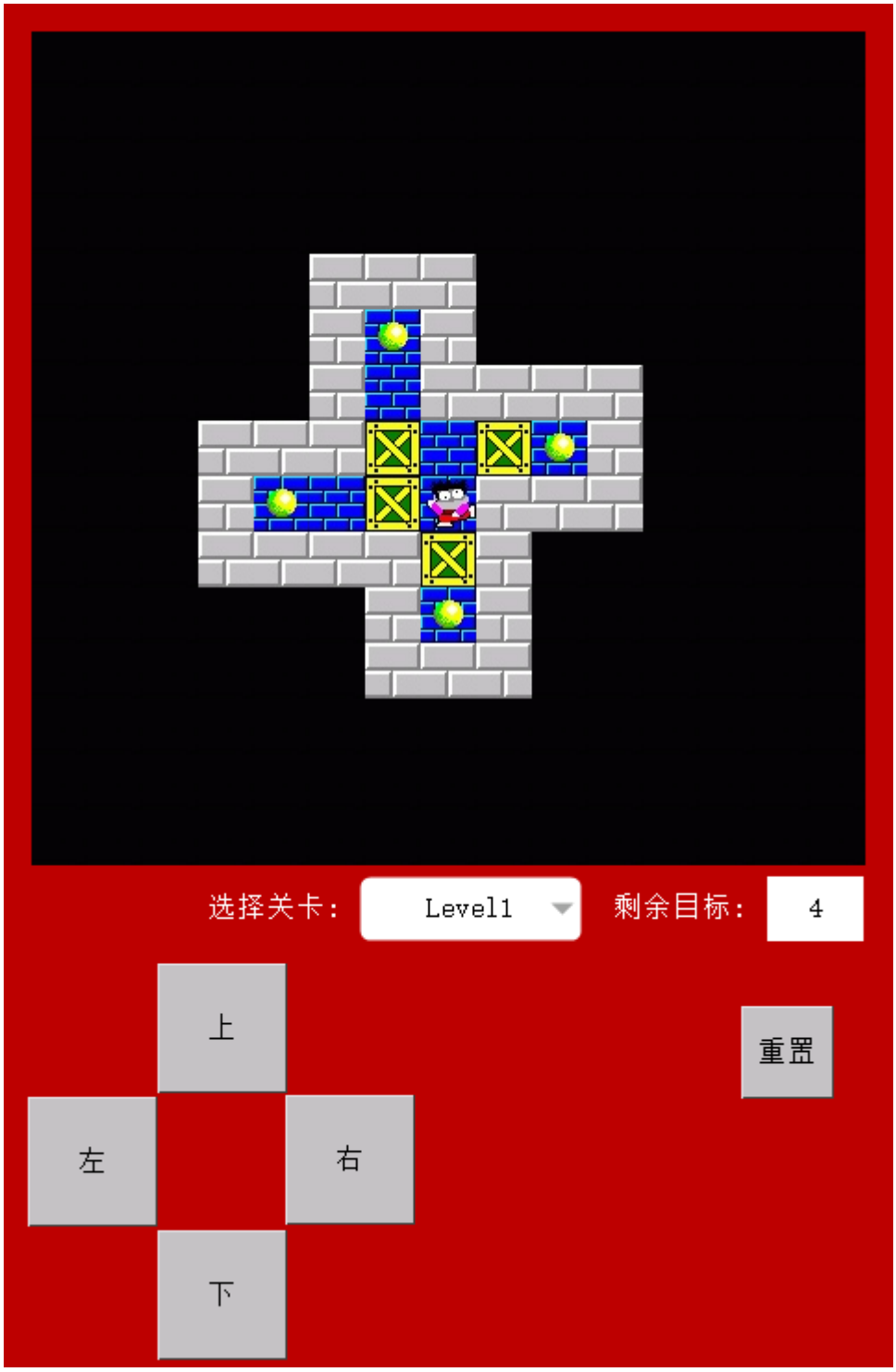
由 QQ 群友王同学提供



《中国象棋》下载

9.11.2 推箱子游戏

由 QQ 群友王同学提供



《推箱子游戏》下载

9.11.3 多功能测量仪

素材说明：界面华丽，带有示波器功能。

《多功能测量仪》下载

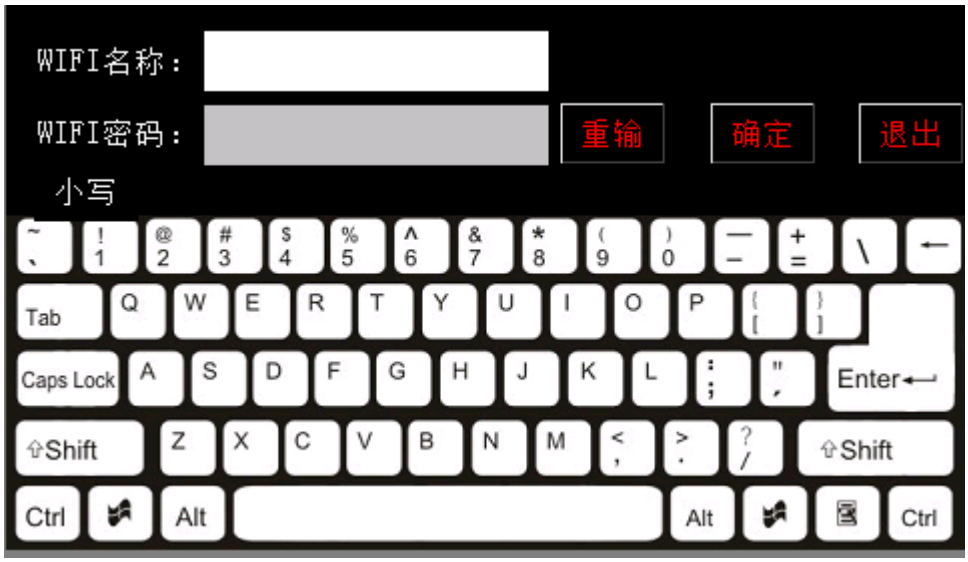
9.11.4 全键盘输入、密码登录

素材说明:

1、全键盘输入，支持大小写切换，数字，字符。

2、密码登录，利用掉电存储空间保存，修改密码。

3、进入设置页面的万能密码为 890327。



《全键盘输入、密码登录》下载

9.11.5 简易计算器

素材说明：

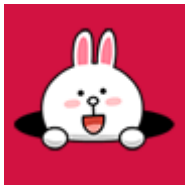
第一次接触串口屏，就随便做了个简易计算器，试试手，免 MCU 控制。不支持负数和小数点，计算过大或出现负数时会有溢出问题。

《简易计算器》下载

9.11.6 打兔子，无需单片机支持

素材说明：

根据前面的打兔子修改，无需单片机支持，直接运行，增加时间显示。



《打兔子，无需单片机支持》下载

9.11.7 电压数据采集及系统参数设置

素材说明：

该案例单片机电压显示已有很多案例，不再多言，主要是屏本身参数设定。有如下功能：1 休眠时间：通过双态按钮实现休眠的开关功能。当休眠开时，可以通过 cal 界面来设置休眠时间；当休眠关时，休眠时间设置无效。2 工作时间：当屏处于非休眠状态时，可以一直“不间断”地记录工作时间。这里不间断是指切换页面的过程，它也一直保持工作。3 背光亮度：通过改变滑块的位置改变屏背光的亮度。

《电压数据采集及系统参数设置》下载

9.11.8 串口屏控制 3 路继电器

素材说明：

这个案例主要通过 printh 指令，从屏幕发送字节到串口，单片机收到后加以判断，并对指定的 IO 输出高低电平，实现继电器开关，并返回开关状态；单片机端使用 STC89C52；晶振 11.0592；继电器控制 IOP1^0,P1^1,P1^2；波特率 9600

《串口屏控制 3 路继电器》下载

9.11.9 九宫格全英文输入（支持串口打印字符）

素材说明：

极简九宫格全英文输出（支持串口打印字符）对于小于 3.5 寸的小屏如果做全键盘会影响触摸体验，因为键太多屏幕不够大只能缩小按键尺寸，影响视觉体验和触摸体验。九宫格键盘可以解决这个问题，10 个按键布局实现全键盘的功能。数字，英文均可输入。您值得尝试一下!

《九宫格全英文输入（支持串口打印字符)》下载

9.11.10 酷炫 Loading 进度条界面

素材说明：

酷炫 Loading 进度条界面为节约 FLASH 空间本实例没有使用切换整图来实现动画，而是使用单个小图片元素通过定时器来实现超炫动画效果

《酷炫 Loading 进度条界面》下载

9.11.11 86 面板智能家居

素材说明：

特别说: 明对于单片机通讯方面，每个人的方法都不一样所以我的通讯协议大家未必适合，大家可以用自己的方法实现单片机的控制

《86 面板智能家居》下载

9.11.12 简洁圆形数字键盘（支持全局输入）

素材说明：

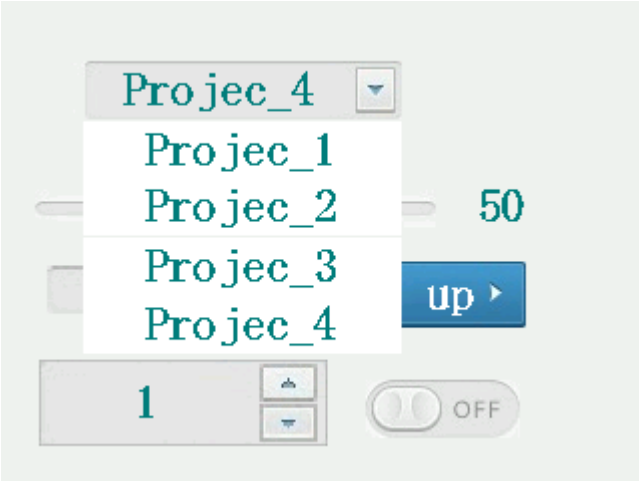
简洁圆形数字键盘（支持全局输入）

《圆形数字键盘》下载

9.11.13 多种控件效果

素材说明：

在同一个页面中实现下拉列表框效果，并支持滚动条



《多种控件效果》下载

9.12 游戏工程

9.12.1 中国象棋游戏工程

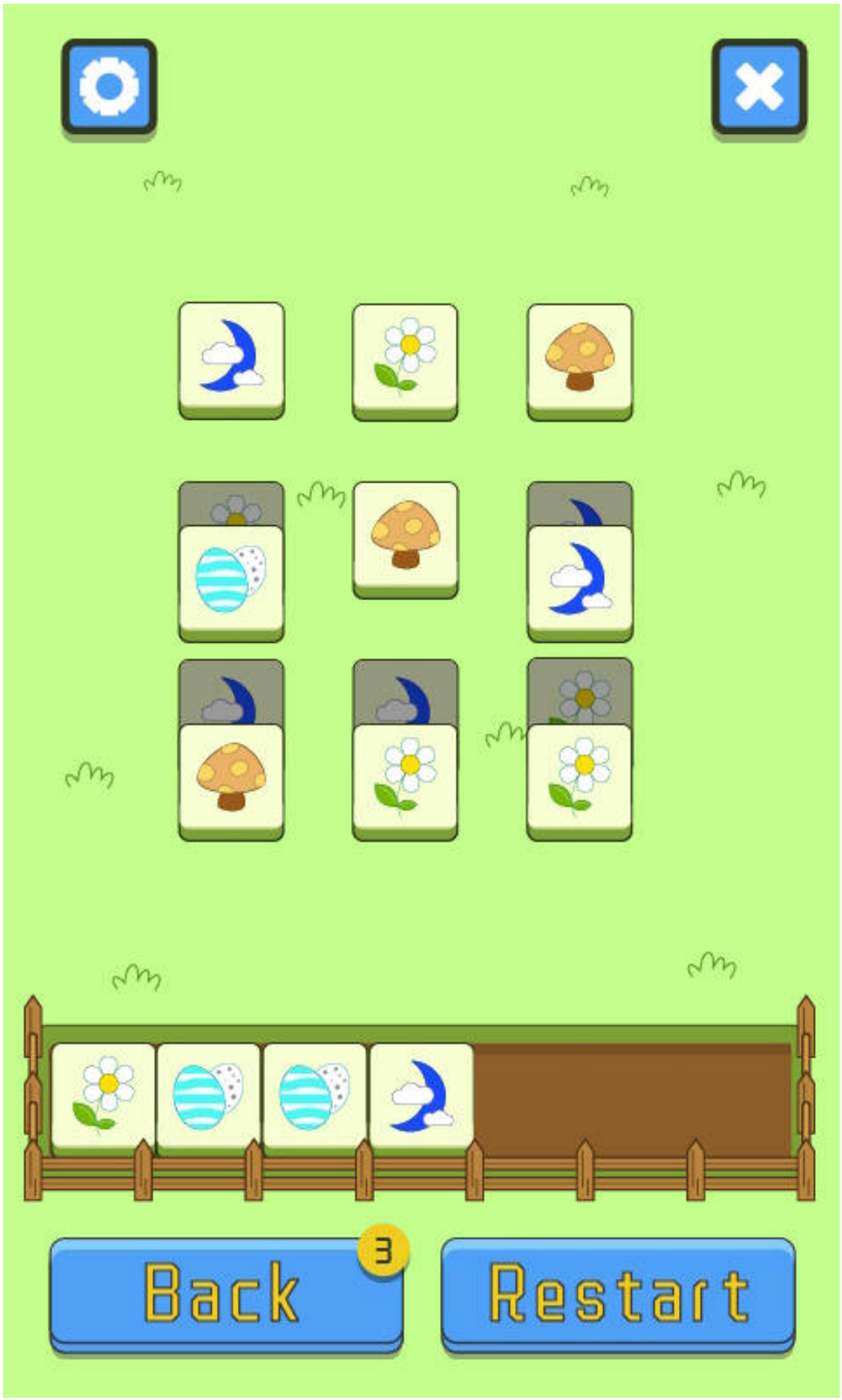
中国象棋 (page 1216)

9.12.2 推箱子游戏工程

推箱子游戏 (page 1218)

9.12.3 兔了个兔

素材说明: 兔了个兔

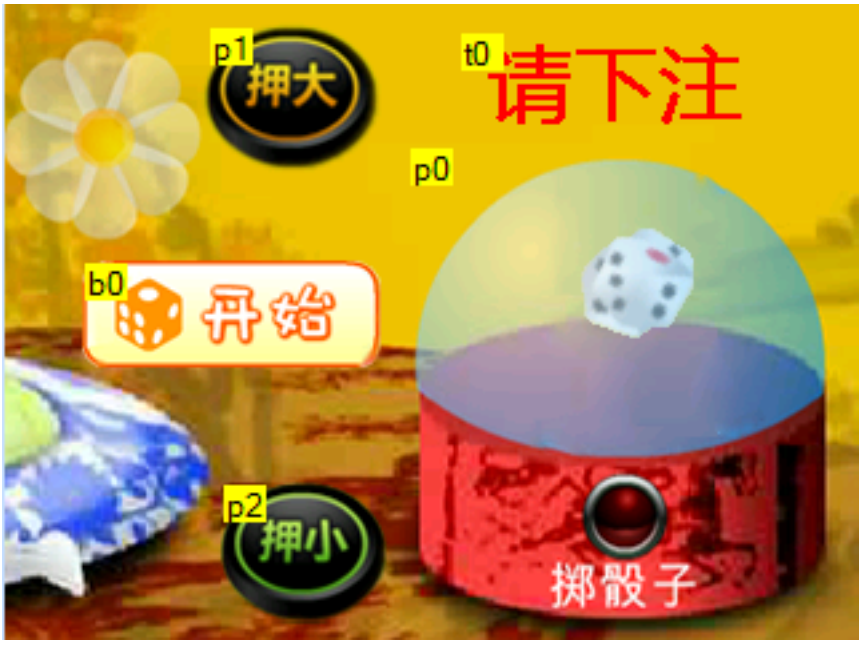

3
Back
Restart

9.12.4 打地鼠

素材说明: 打地鼠游戏

《打地鼠》下载

9.12.5 掷骰子赌大小游戏



素材说明：

1. 整个游戏过程 HMI 自动完成，无需用户单片机参与。工程中用到了全局控件,if esle, 自锁按键，定时器，随机数，系统内置数值变量 sys0,sys1, 是个不错的参考例程。

2. 此工程按 320*240 分辨率设计，别的分辨率也可以用来做测试，如果在更大的分辨率上测试，多出来的区域将会是白色。下载之前先选择您设备对应的型号。

《摇骰子比大小》下载

9.12.6 猜拳大通关

素材说明：猜拳游戏

《猜拳大通关》下载

9.12.7 flappy Bird

素材说明：

《flappy Bird》下载

9.13 淘晶驰资料合集

上位机下载 (page 1066)

9.13.1 网盘链接 1

通过百度网盘分享的文件：淘晶驰资料合集

通过网盘分享的文件：tjcwiki_resource_collection

链接: https://pan.baidu.com/s/1VmBo78IJGVM4aRSerOXHKg?pwd=tjcc

提取码: tjcc

9.13.2 网盘链接 2

通过百度网盘分享的文件：淘晶驰资料合集

通过网盘分享的文件：tjcwiki_resource_collection

链接: https://pan.baidu.com/s/1gxpUViimqxLZtGdIraMMdQ?pwd=tjcc

提取码: tjcc

---

10 视频教程

10.1 基础入门

1. 上位机软件下载与工程创建
2. 简单工程的制作
3. 串口屏调试
4. 模拟器联机调试
5.arduino 控制串口屏
6. 自定义串口协议
7.arduino 解析协议
8. 串口屏常见提示与解决方法
9. 串口屏主动解析
10. 串口屏显示电脑运行信息
11. 网络时钟制作分享
12. 串口屏名称组使用
13. 串口屏主动解析原理分析
14. 键盘高级用法
15. 如何实现掉电保存
16. 快速制作好看的界面

10.2 控件详解

认识控件
页面控件
文本控件 & 滚动文本控件
数字控件 & 虚拟浮点数控件
按钮控件 & 双态按钮控件 & 状态按钮控件
进度条控件 & 滑块控件
触摸捕捉控件、触摸热区控件和定时器控件
曲线控件 & 二维码控件
文件流控件使用教程一
文件流控件使用教程二
文件流控件使用教程三

10.3 视频对应的工程下载链接

https://pan.baidu.com/s/1yD0kG5j4HwfME-iOuIDiLw

提取码：tjcc

10.4 其他教程

串口屏显示 AIDA64 数据

自定义键盘

EEPROM 的使用

使用 python 动态获取奥运会奖牌数据

11 常见问题

11.1 硬件相关

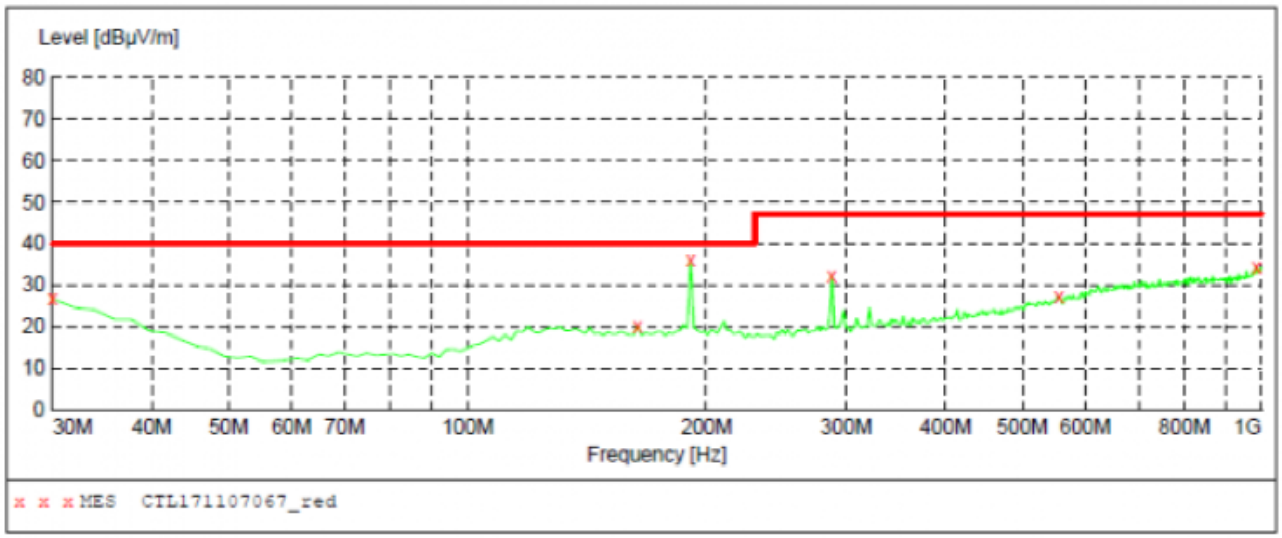
11.1.1 ESD 能过多少 V，EMI 性能如何

ESD 电压：

1. 常规等级 (淘宝按此等级出货): 接触放电 4KV, 空气放电 8KV 2. 可选等级: 接触放电 8KV, 空气放电 15KV

EMI 特性

下图是 TJC8048T070_011 的 EMI 测试图，我们认证的是 class B 级别



11.1.2 串口通讯线最长能接多少米

TTL 通讯: 如果通讯在 9600 下 TTL 传输理论值达 2 米，实际应用建议不要超过 50cm，这个与电磁环境有关系，主要受限于 TTL 器件的带负载能力和抗干扰能力的影响。

232 通讯: RS-232-C 标准规定，驱动器允许有 2500pF 的电容负载，通信距离将受此电容限制，例如，采用 150pF/m 的通信电缆时，最大通信距离为 15m；若每米电缆的电容量减小，通信距离可以增加。传输距离短的另一原因是 RS-232 属单端信号传送，存在共地噪声和不能抑制共模干扰等问题，因此一般用于 15m 以内的 [rs232（9 针）接口] 通信

为什么 232 通讯不能超过 256000 (page 1265)

11.1.3 通讯口电压多少 V，是否可以直接接单片机

TTL 通讯：

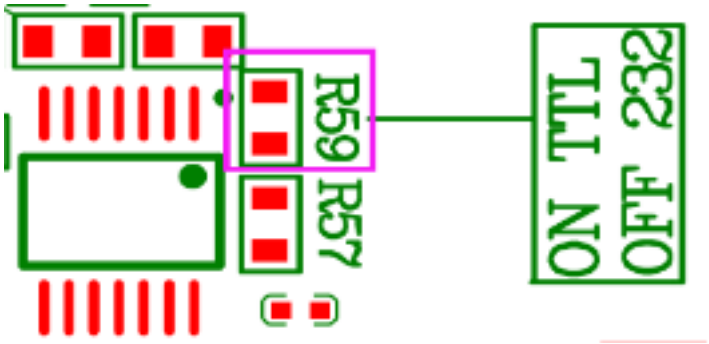
接 5V 单片机：

X3/X5 系列通讯口电压为 5V ，T0/T1/K0/X2 系列为 3.3V ，都可以直接接 5V 单片机。

接 3.3V 单片机：

T0/T1/K0/X2 系列为 3.3V ，都可以直接接 3.3V 单片机。X3/X5 系列通讯口电压为 5V，如果需要接 3.3V 单片机，请检查单片机的串口所使用的引脚是否为 5V 耐受（标有 FT 参数），如果是 5V 耐受的引脚就可以直接接 X3/X5 系列。如果接上之后，串口屏能收到单片机的数据，但是单片机无法接收串口屏数据，可以尝试在串口屏的 TX 和单片机的 RX 之间穿 1 个 1K 的电阻。

232 通讯：X3/X5 系列是兼容 TTL 和 232 的，如果要用 232 通讯，要把 R59 这个电阻焊掉，此时是不可以直接接单片机的（单片机一般只带 ttl 电平的串口，需要接一个 ttl 转 232 的转换芯片才能和去掉 R59 电阻的串口屏连接）。



此时如果使用 usb 转 232 下载器来下载工程，请降低波特率，不建议 232 模式下的波特率高于 256000，尽量使用 SD 卡进行下载。

为什么 232 通讯不能超过 256000 (page 1265)

11.1.4 屏幕通电后不断的闪烁 (供电不足导致不断重启)

这种现象一般出现在 4.3 寸及以上的屏，因为供电不足导致的，如果你是使用电脑 usb 进行供电，那请增加额外供电，因为电脑 usb 的输出能力是有限的，通常只有 500mA。

如果你使用供电芯片供电，请检查你的供电芯片带载能力，在通电情况下检测屏幕电源座子上的电压是否正常。务必亲自动手检测带载能力，不要看电源标签上的标称值。

串口屏开机时死机/不断的闪烁/不断重启 (page 1366)

外部供电接法 (page 1262)

11.1.5 喇叭相关问题

11.1.5.1 觉得喇叭声音小怎么办

有 2 个解决方法

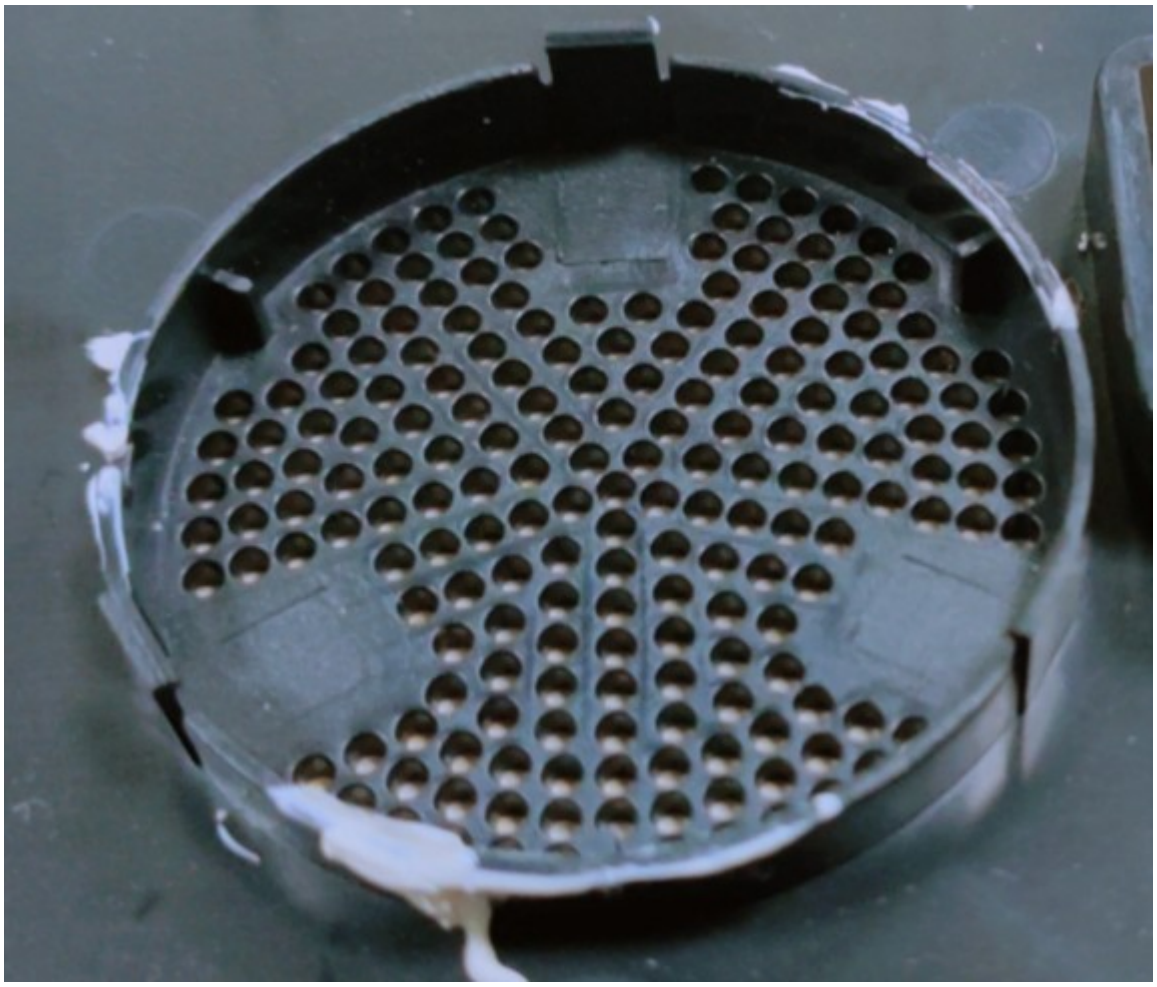
一是增加喇叭的功率，前提是你的电源要能够带得动，最大支持 3W 的喇叭，则必须增加 600mA 以上的电流冗余，如果供电不足会导致无法开机，重启，闪烁等异常。

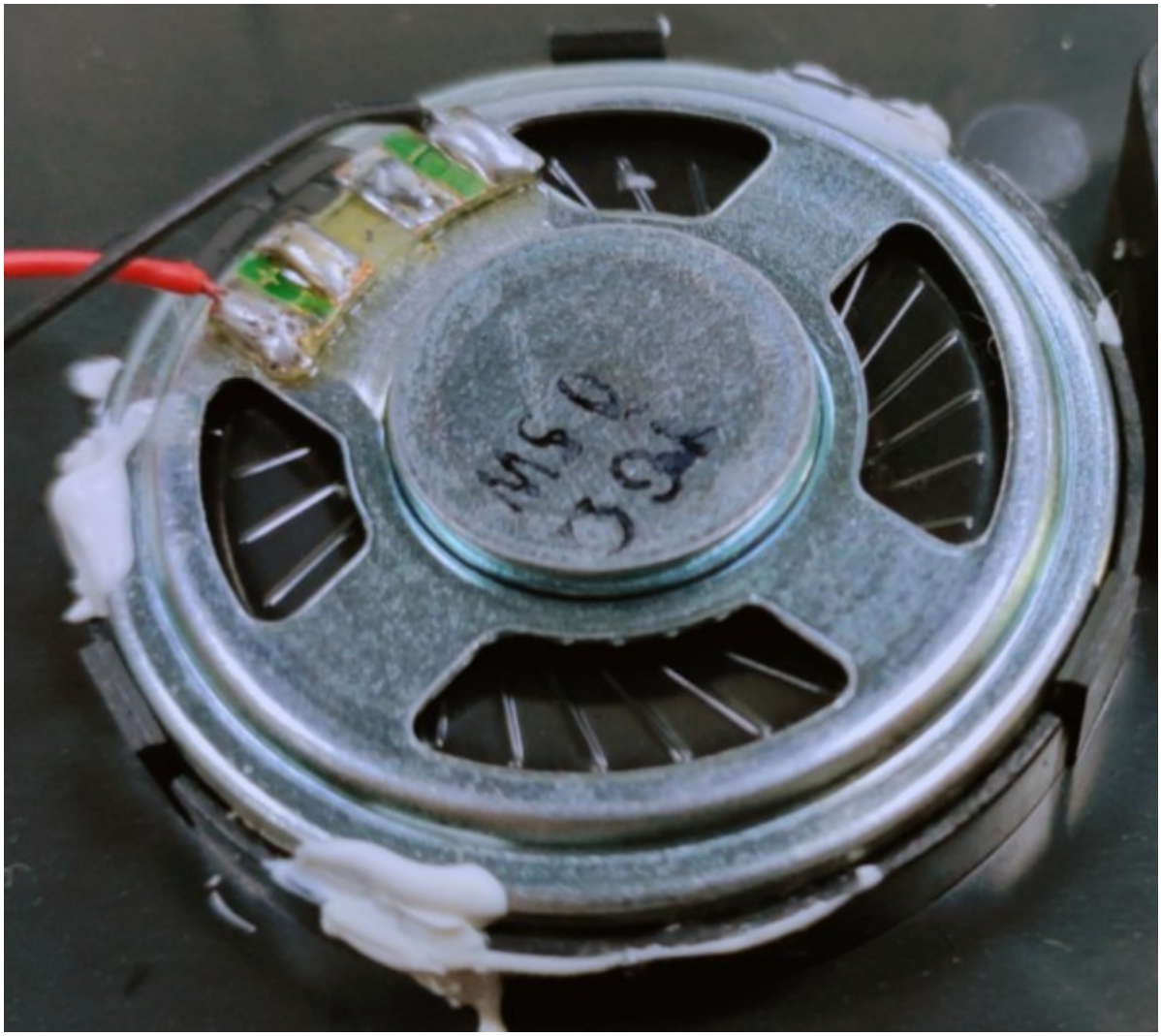
参考

串口屏开机时死机/不断的闪烁/不断重启 (page 1366)

外部供电接法 (page 1262)

二是是给喇叭做一个好的音腔 (共振腔)。因为仅仅是喇叭功率变大，不一定能很明显提升音量，加上共振腔就能明显提升音量





参考资料:

《扬声器音腔设计.pdf》下载

《有关喇叭的音腔的设计规范.pdf》下载

11.1.5.2 喇叭接口型号

PH1.25

11.1.5.3 喇叭的正负极

虽然上面标有“+/-”，但是那不是表示正负的意思，是相位。

接法决定喇叭的运动方向，在同一个音箱上面，一根信号线应和所有的喇叭的同一极连接，另一根接另一极，使所有的喇叭的运动方向一致，才会使声音加强，否则的话会造成声短路，极大地影响音质，同时会缩短喇叭上纸盆的寿命

用一节 1.5 伏电池短时通断喇叭接线端，如果纸盆向外运动，电池的正极就是喇叭正极。

分正负是因为多只喇叭同时工作是的声音相位相同，不至于不同相位的声音相互抵消。

总结：喇叭的话保证出声的时候纸盆是往外鼓的

11.1.5.4 喇叭插上有电流声

我们自己测过的，只有耳朵贴着听的时候才能听到，很细微的只能将就一下，毕竟我们不是专业做音乐设备的

11.1.6 关于串口工具 (cp2102/ft232/ch340/pl2303)

11.1.6.1 为什么不建议用 ch340

ch340 不是不能用，是假货太多了，假货延迟太大并且可能丢失数据，造成通讯失败，所以不建议用。

ch340 只是假货相对较多，如果能买到正品也是能够正常使用的，淘宝上几块钱一个模块的，容易出现无法下载的问题。

有些时候 ch340 用得好好的会突然出现无法下载的情况，下面提供一个群友的解决方案

> 找到一个小窍门，还是用 CH340 串口模块，下载成功了，试了几次全部成功。
>
> 方法是，先想好把串口屏瞬时断电的方法，但要保证串口模块始终不断电啊，在电脑上点击“联机并开始下载”按钮后，马上给串口屏断电再上电，就开始下载了。这个“马上”以及“瞬间”都是在 2 秒以内。我是受 STC 单片机收录方式启发的。你们可以一试！这说明经常下载不成功可能并不是串口模块的问题，或者不完全是，根本原因可能是信号时机的问题

11.1.6.2 为什么不建议用 pl2303

pl2303 太老了，驱动装起来很麻烦，不建议使用。

建议从正规渠道购买 cp2102 或者 ft232 芯片的下载器（如淘晶驰，立创商城）

11.1.6.3 不建议使用的其他串口工具

不建议使用多功能的下载工具，除了 USB 转 TTL 之外还有其他调试功能的下载器

例如：

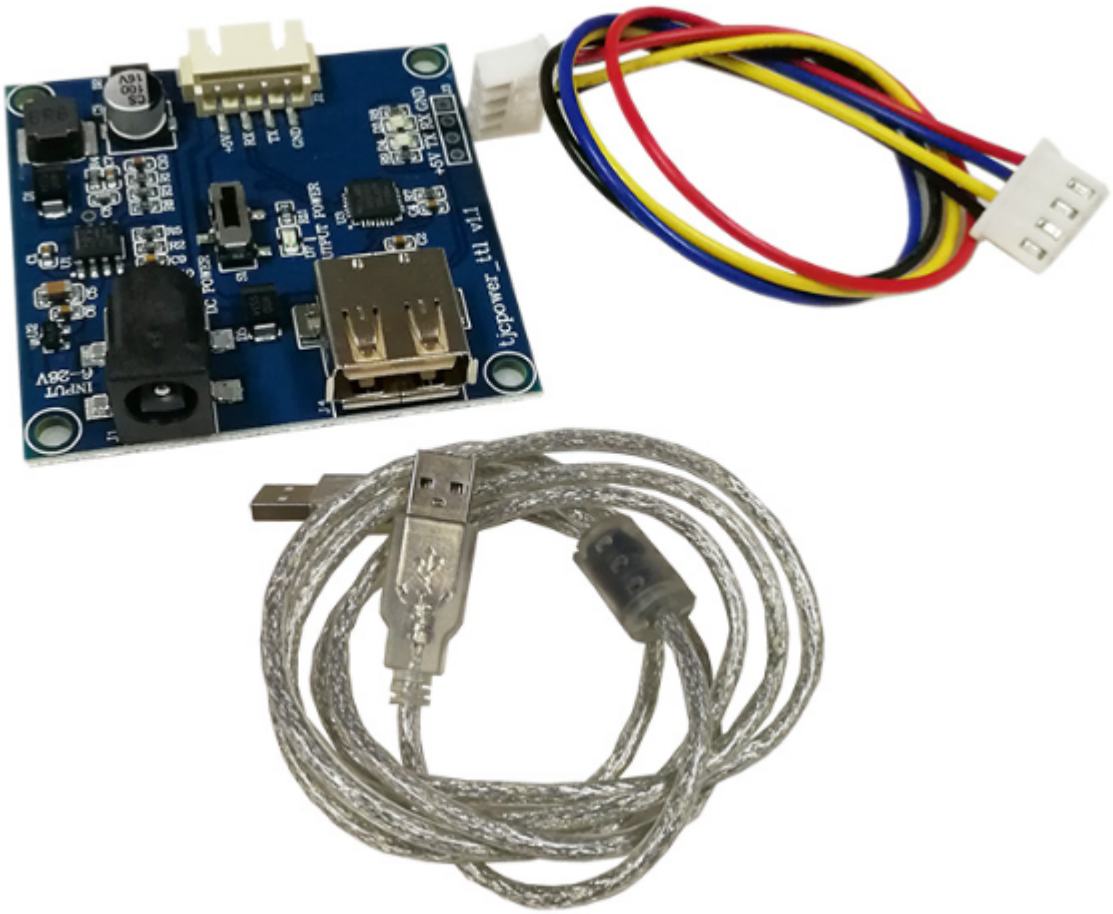
1. stc 单片机下载器
2. 使用单片机的 usb 功能实现的 usb 转 ttl 的下载器
3. 带有 jlink/stlink/daplink 调试、i2c、spi 的 usb 转 ttl 的下载器
4. 同时支持 ttl、232、485 等多种通讯功能的下载器等等
5. USB 转 ttl 再转 232 再转 ttl 等经过多重转换的下载器
6. 带隔离功能的 usb 转 ttl 工具

以上工具可以勉强用来通讯，但是不要用来下载工程，通讯和下载的数据量是有几十上百倍的差距

11.1.6.4 建议使用哪些串口工具

只推荐使用单纯的 USB 转 TTL 下载工具，即芯片为 cp2102 或者 ft232 芯片的

推荐使用以下的 USB 转 ttl 工具（在淘宝或者咨询销售购买）



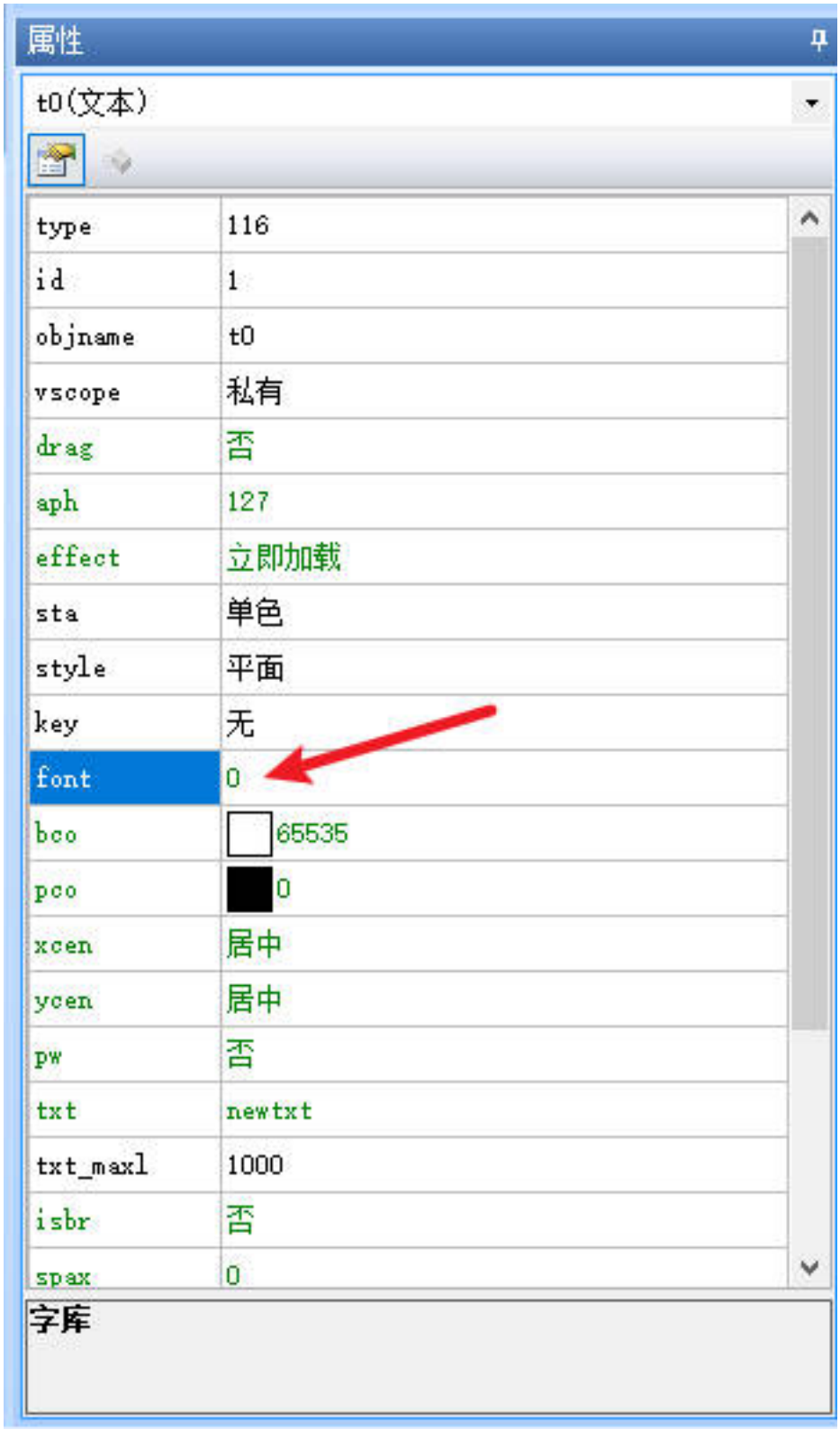
如果是 5 寸及以上（或者使用喇叭的），请购买额外供电

CAUTION
RISK OF ELECTRIC SHOCK
DRY LOCATION USE ONLY
ATTENTION
POUR UTILISATION A
L'INTERIEUR SEULEMENT
AC/DC ADAPTER
Model:DLL015122000D
Input:100-240V~50/60Hz 0.55A
Output: +12V ⎓ 2A
CAUTION
INDOOR USE ONLY
MADE IN CHINA

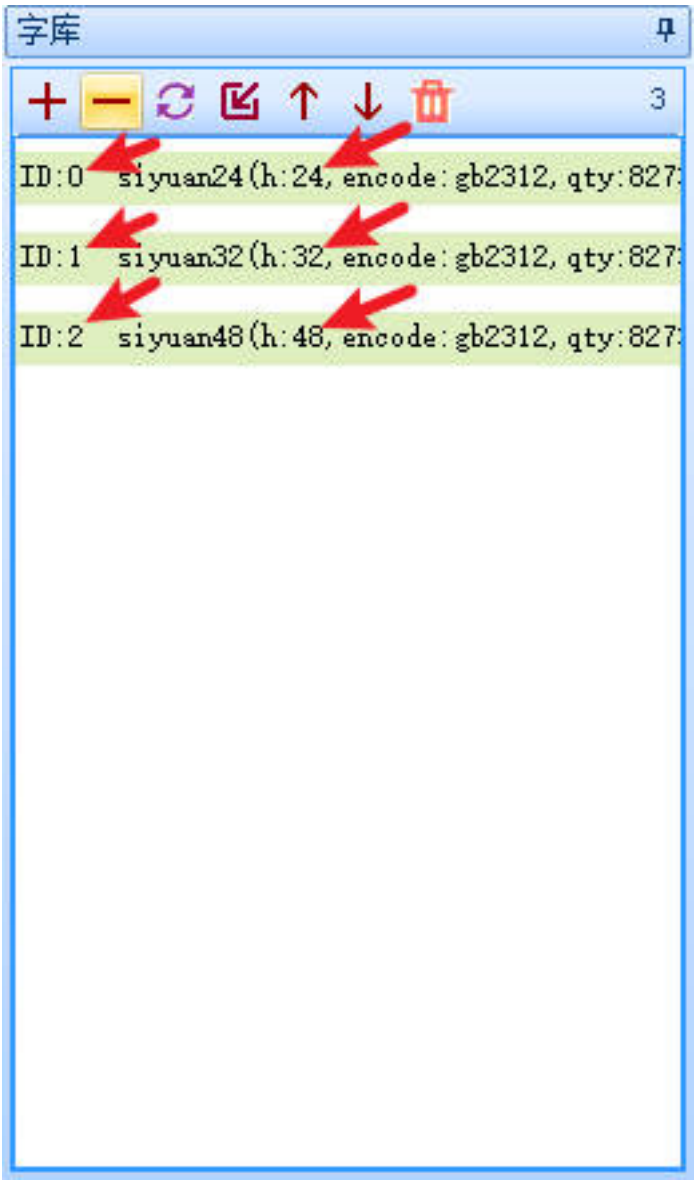
12V 2A电源

11.1.7 如何修改控件显示的字体

控件显示的字体大小取决于该控件的 font 属性所对应的字体



如果需要显示不同大小或者显示不同的字体样式，制作多个字库并导入到工程中，参考创建字库和导入字库 (page 125)



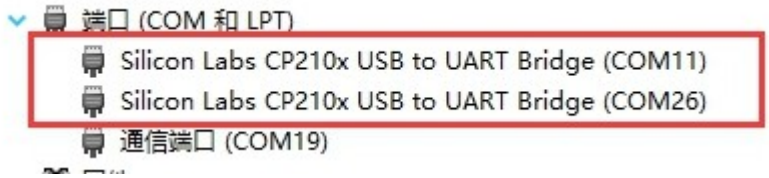
将对应控件的 font 属性修改为相应的字库即可

此时你需要用鼠标点一下对应的控件，才会刷新成新的字体样式

11.1.8 串口驱动相关问题

11.1.8.1 如何查看串口驱动是否正确安装

右键我的电脑-管理-设备管理器-端口，应有正确的端口号并且无感叹号



11.1.8.2 如何安装串口驱动

将串口转 ttl 工具插到 USB 接口上，安装驱动人生/360 驱动大师/驱动精灵软件后进行自动安装
推荐使用 CP2102、FT232 芯片的串口工具，不推荐使用 ch340 芯片或 pl2303 芯片的工具
为什么不建议用 *ch340* (page 1248)

11.1.9 串口电平状态

ttl 空闲时 rx、tx 都是高电平，tx 是推挽输出，RX 是浮空输入

11.1.10 关于一上电就关闭屏幕背光

如果需要一上电就关闭屏幕背光时，可以在 program.s 里面设置 dim=0

但是不建议这么做，如果要这么做请考虑您已经充分熟悉屏幕，有些用户自己设置了屏幕亮度为 0 后，忘记了这么一回事，联系返修，发现屏幕并没有问题，只是设置了屏幕背光为 0

11.1.11 结构兼容性

K050 和 T050 结构兼容。T050 和 X350,X550 结构不兼容。

5 寸是唯一一款 X 系列改变结构的型号

11.1.12 拓展 IO 相关问题

FPC 座是 10P 1.0MM

fpc 是 1.0x10pin

支持的型号：

x5 系列支持 8 路 IO

k0 系列支持 8 路 IO

X2 系列部分型号支持 8 路 IO

K0/X2 系列仅 io4-io7 支持 PWM，X5 系列仅 io7 支持 PWM

所有系列拓展 IO 口电平都是 3.3v

11.1.12.1 如何初始化 IO 口

使用 cfgpio 指令*cfgpio-扩展 IO 模式配置* (page 650)

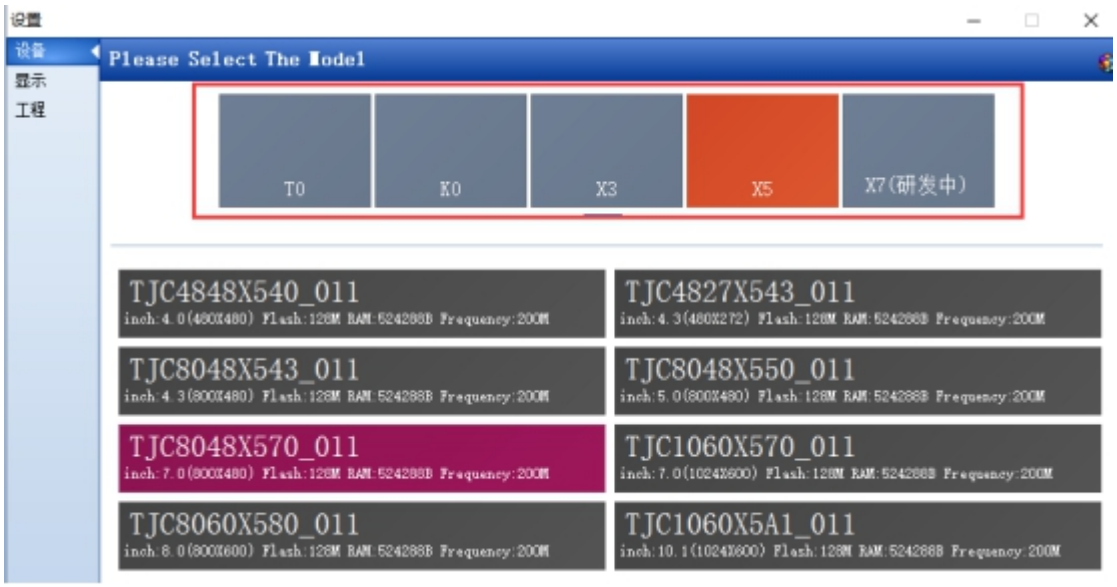
sleep 睡眠模式与拓展 IO (page 1041)

11.1.13 如何修改设备型号

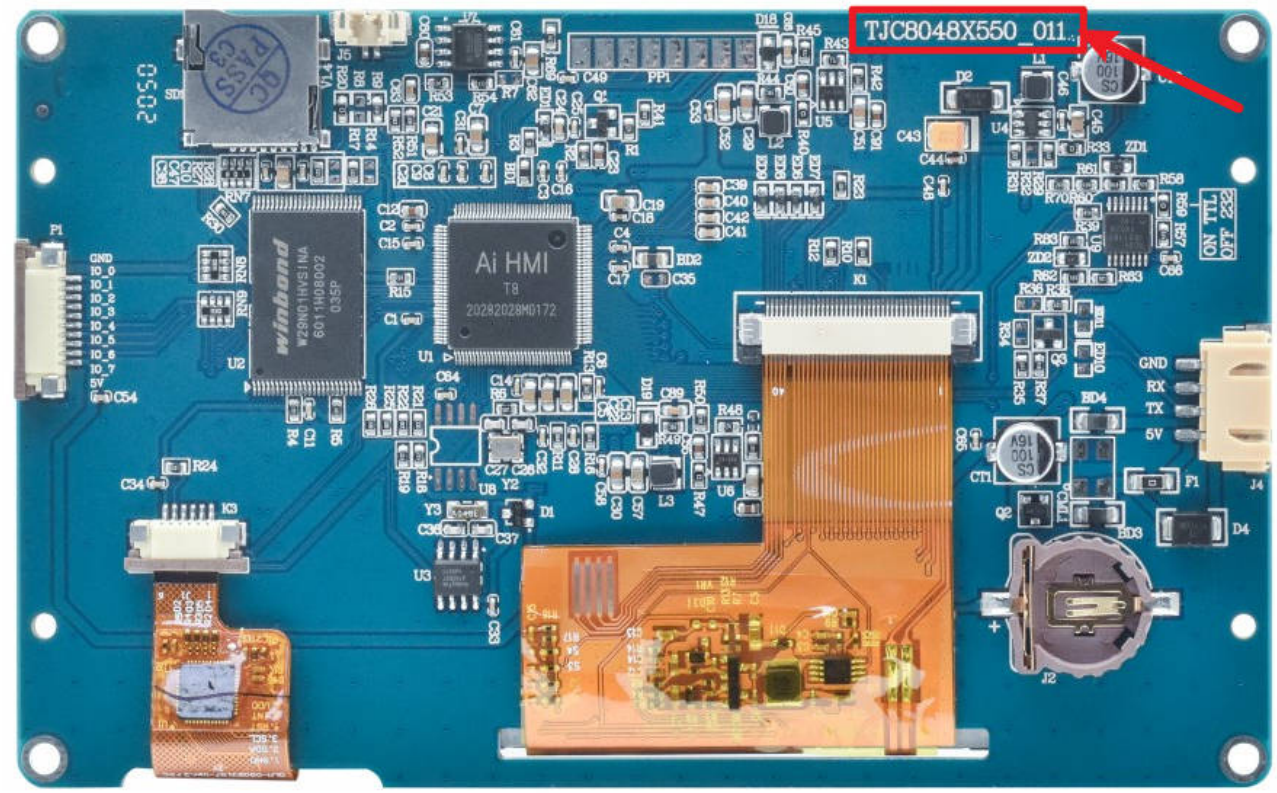
第一步

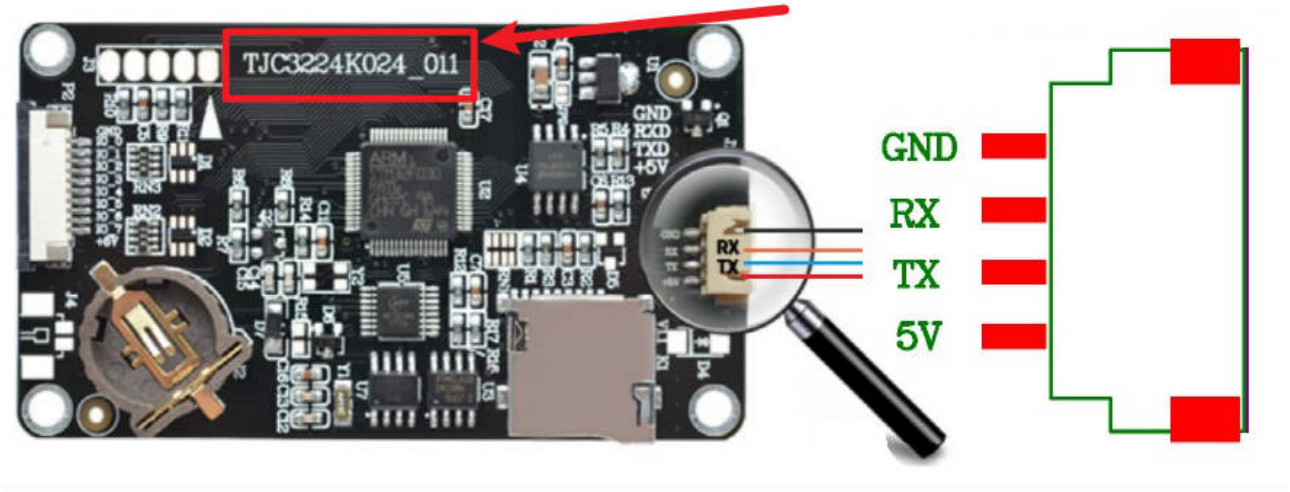
第二步

第三步



如果不知道屏幕的型号，可以从屏幕 PCB 背面得到





注意： 修改前的型号和修改后的型号分辨率应一致

修改了型号之后，某些功能如果不支持，需要把对应的控件或者代码删除才能编译通过

定制版本的型号，只要”_011”之前的型号能对的上即可

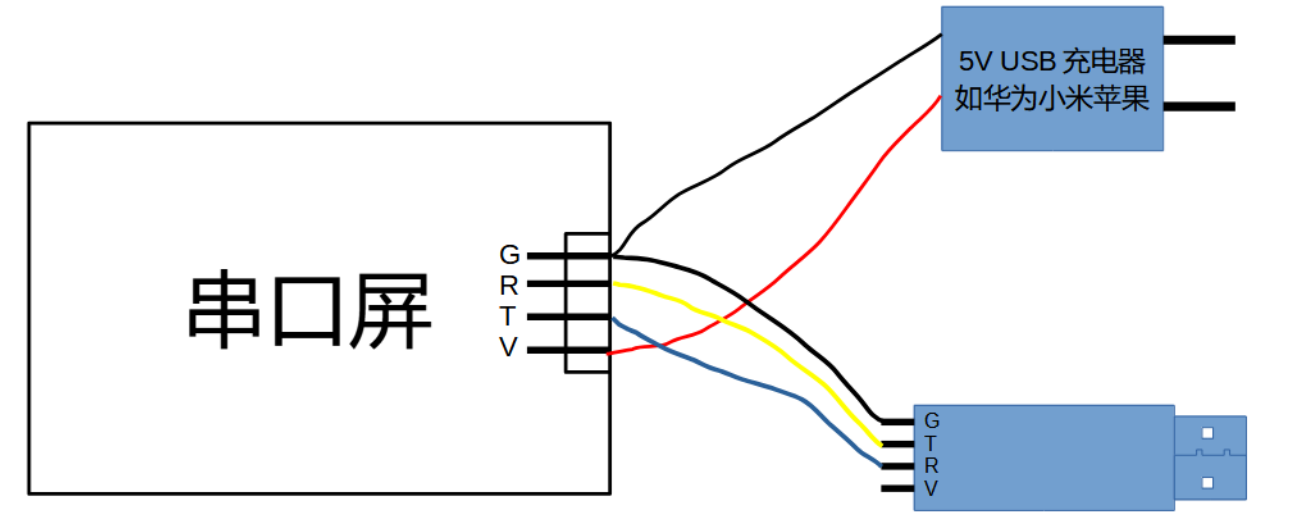
参考产品命名规则 (page 1488)

11.1.14 屏幕边框为什么不对称

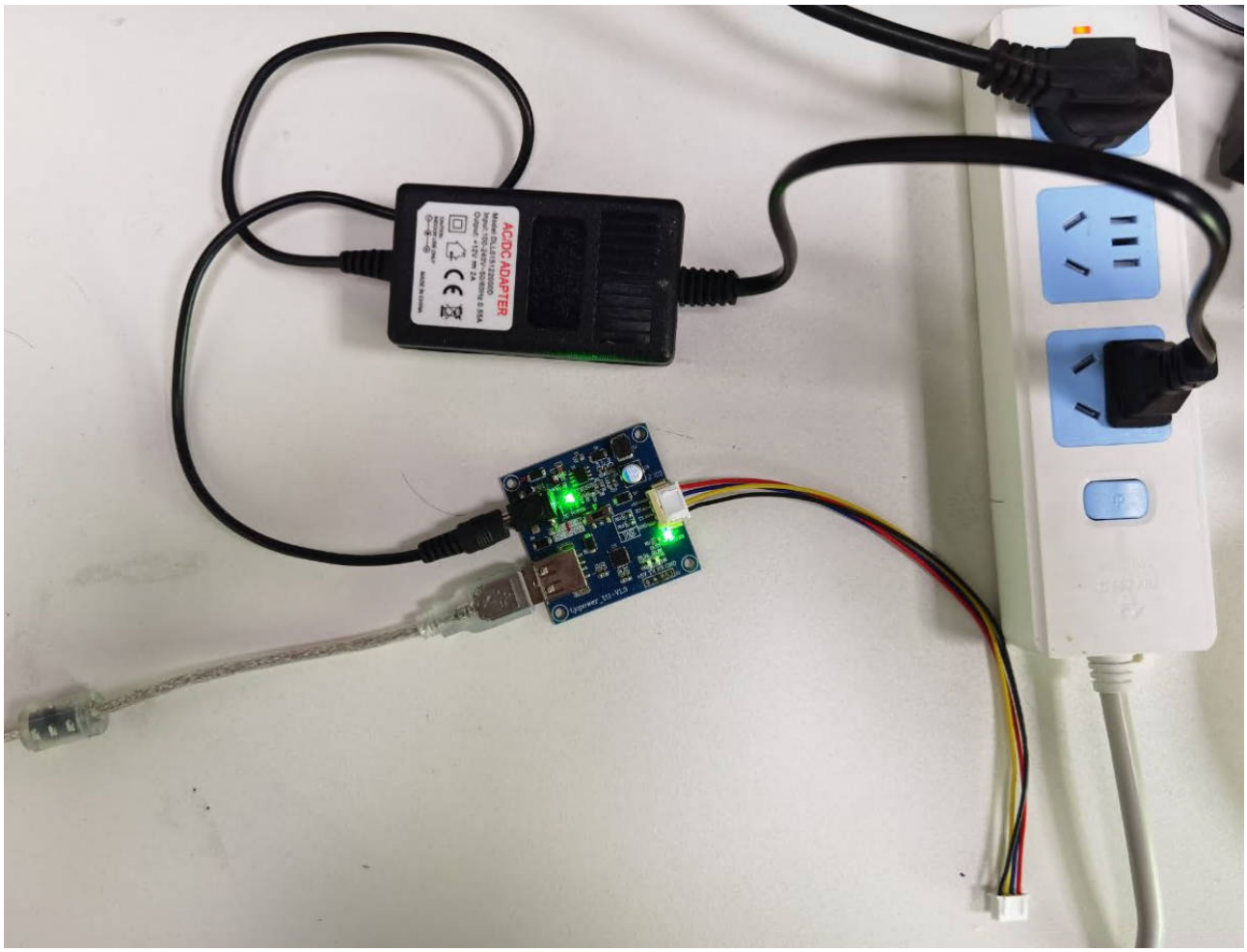
屏幕边框是因为供货商的屏幕他就这样
市面上大部分七寸屏边框都是这样
主要因为两侧要有排线放置的空间

11.1.15 外部供电接法

外接供电，请参考下面的接法



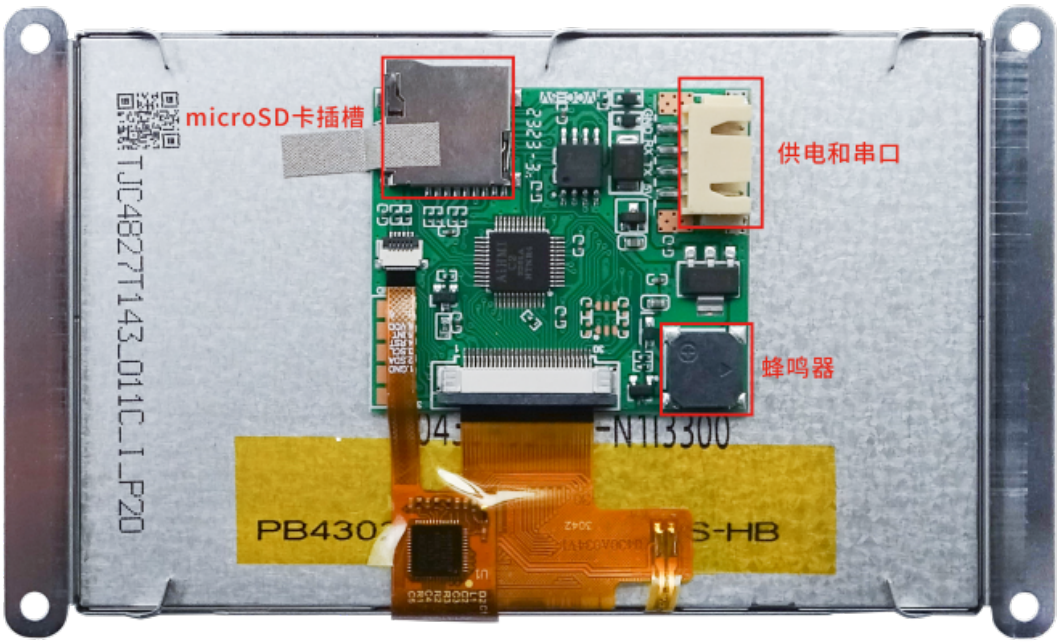
也可以购买官方的串口工具和 12V 电源

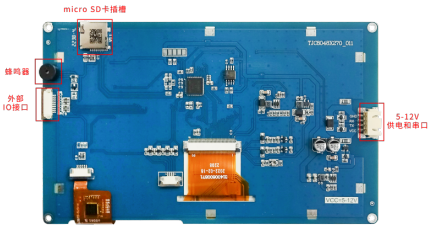


11.1.16 如何判断屏幕是否支持蜂鸣器

如何判断屏幕是否支持蜂鸣器/如何判断屏幕是否支持 beep 指令

支持蜂鸣器的屏幕, 屏幕背面会有黑色圆形或者黑色方形的蜂鸣器。



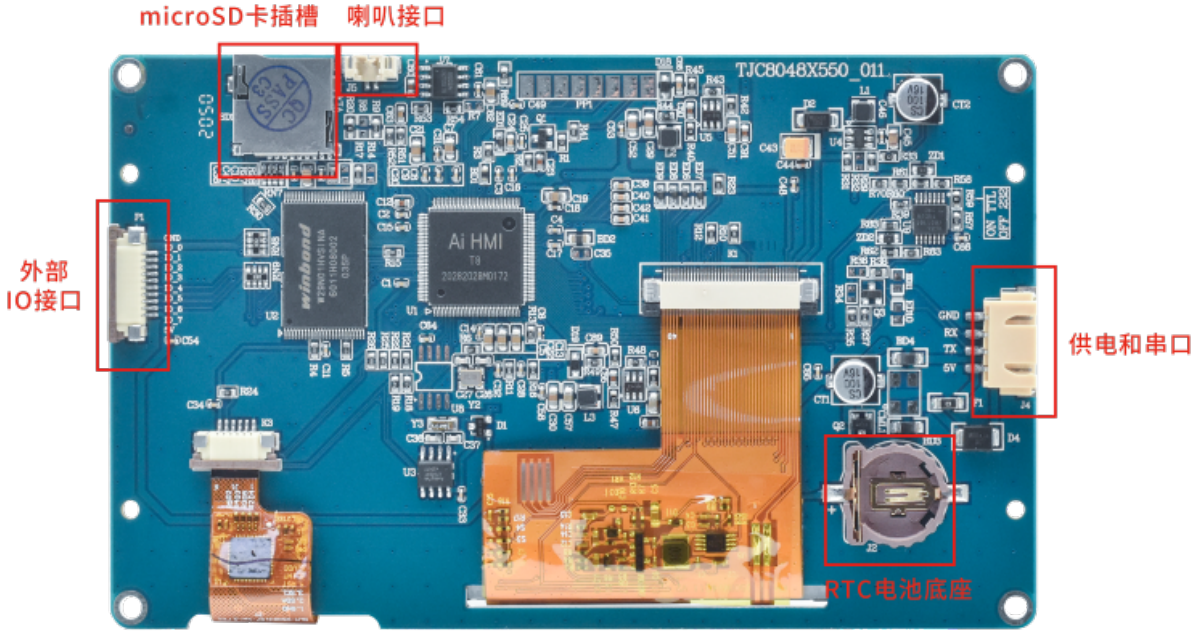


相关链接

beep-蜂鸣器鸣叫 (page 648)

11.1.17 如何判断屏幕是否支持 RTC 实时时钟

支持 RTC 实时时钟的屏幕, 屏幕背面会有 RTC 电池座。



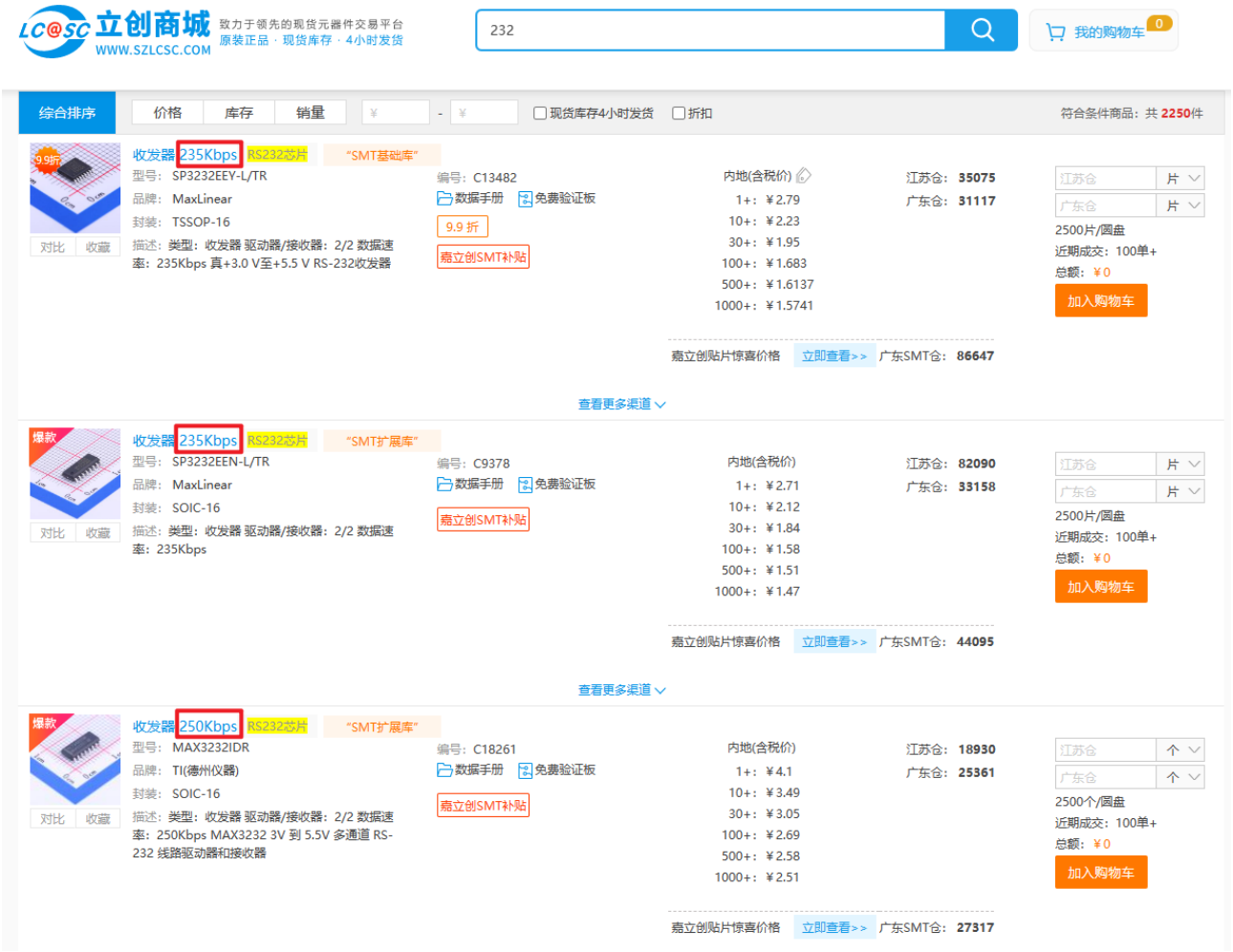
相关链接

rtc0~rtc6-RTC 时钟变量 (page 795)

校准 RTC 时钟 (page 161)

RTC 电池型号 (page 162)

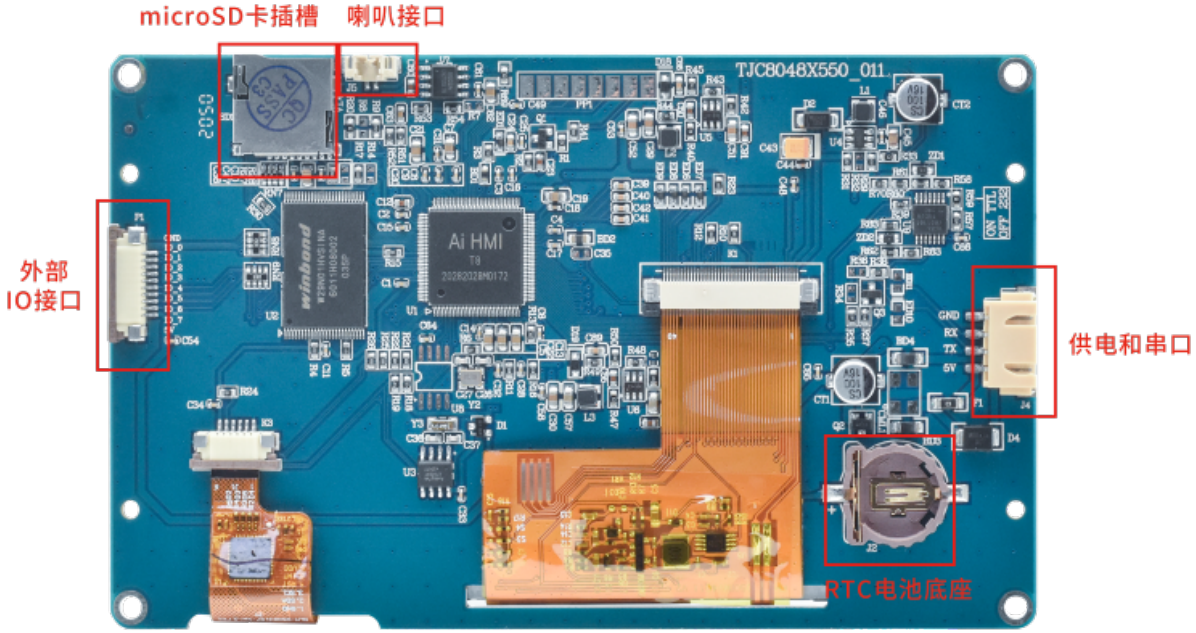
11.1.18 为什么 232 通讯不能超过 256000



以在立创商城上搜索 232 为例，出来的 ttl 转 232 芯片，大部分芯片均标明了速率，250Kbps = 250*1024 bps = 256,000 bps

11.1.19 如何判断屏幕是否支持喇叭

支持喇叭的屏幕，屏幕背面会有喇叭接口。



相关链接

喇叭相关问题 (page 1245)

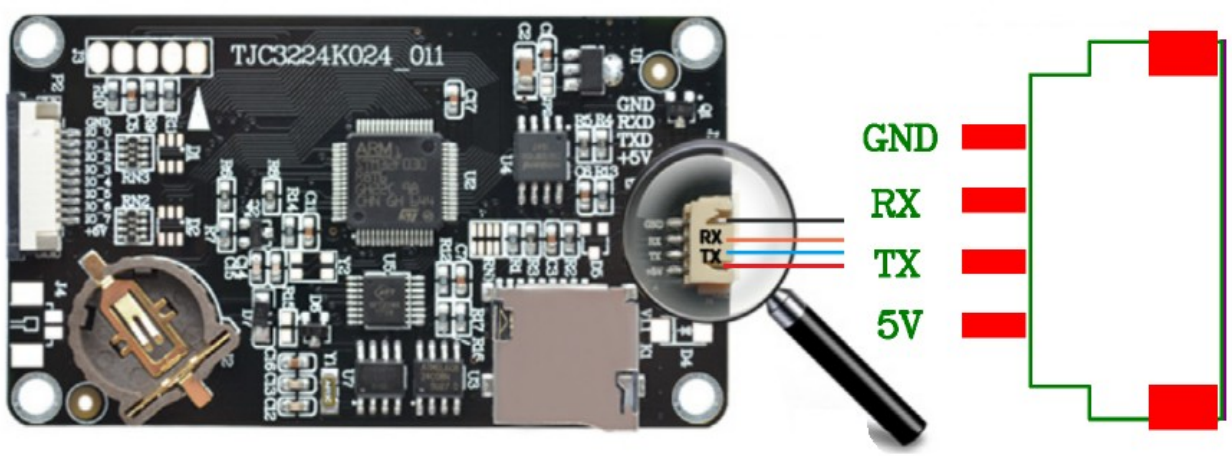
觉得喇叭声音小怎么办 (page 1245)

喇叭接口型号 (page 1246)

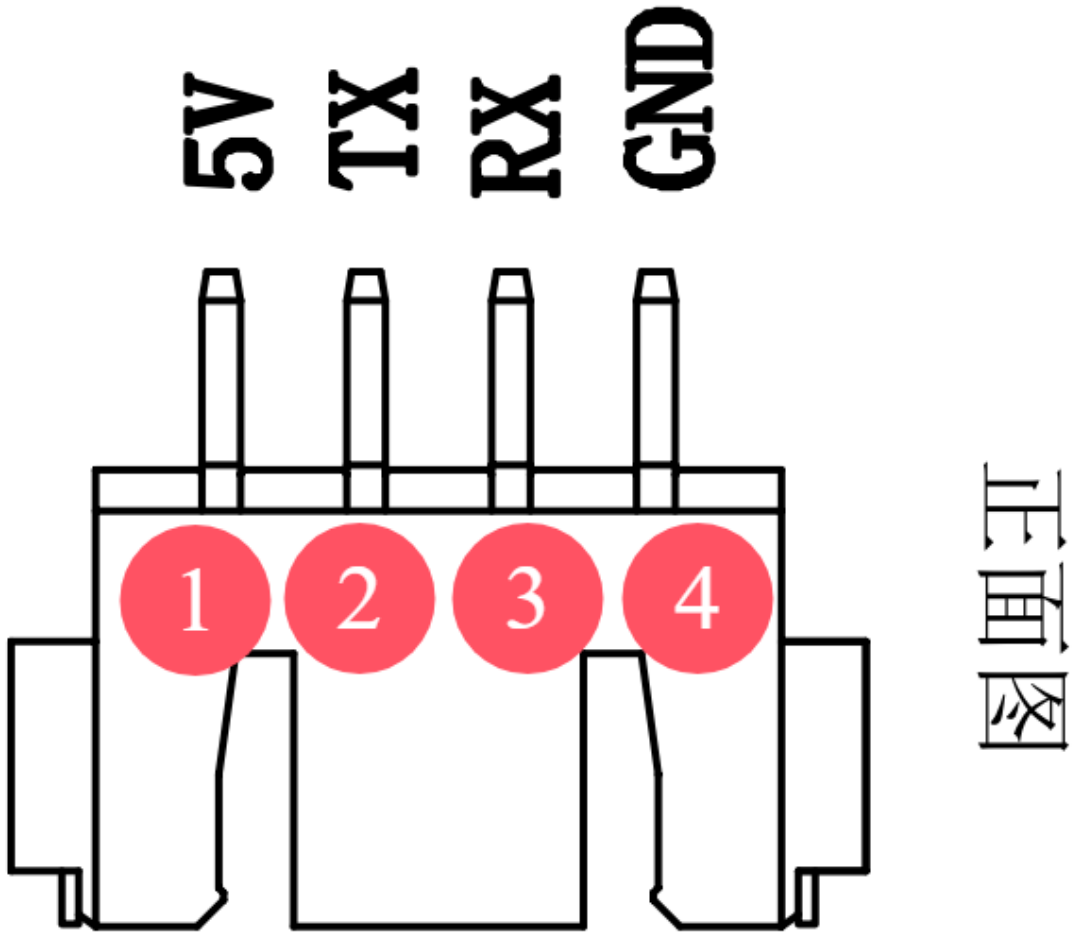
11.1.20 串口屏常用接口型号说明

11.1.20.1 XH2.54*4pin 接口说明

11.1.20.1.1 XH2.54*4pin 实物图片



11.1.20.1.2 XH2.54*4pin 连接器母座说明



以下举例的为兼容型号

JST 连接器公司的 4pin 2.54 间隔连接器母座，其产品型号为 S4B-XH-SM4-TB

HC(虹成电子) 的 4pin 2.54 间隔连接器母座，其产品型号为 HCZZ0021-4

HCTL(华灿天禄) 的 4pin 2.54 间隔连接器母座，其产品型号为 HC-XH-4AWT

下方截图仅做参考，非购买指南。

预售商品

暂无图片

S4B-XH-SM4-TB

品牌: JST

封装: SMD,P=2.5mm,卧贴

插针结构: 1x4P

间距: 2.5mm

安装方式: 卧贴

参考系列: XH

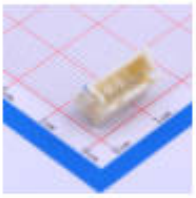
类目: 线对板针座

编号: C20561980

详细: 数据手册

领券 包邮券 嘉立创贴片惊喜价格(库存0) SMT扩展库

查看更



S4B-XH-SM4-TB(LF)(SN)

品牌: JST

封装: SMD,P=2.5mm,卧贴

插针结构: 1x4P

间距: 2.5mm

安装方式: 卧贴

参考系列: XH

类目: 线对板针座

编号: C161861

详细: 数据手册

领券 包邮券 嘉立创贴片惊喜价格(库存3502) SMT扩展库

查看更

HCZZ0021-4

品牌: HC(虹成电子)

封装: SMD,P=2.54mm,卧贴

描述: 针座XH2.54-4P 卧贴编带

插针结构: 1x4P

间距: 2.5mm

安装方式: 卧贴

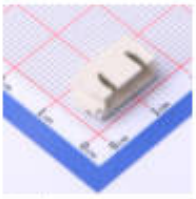
参考系列: XH

类目: 线对板针座

编号: C7433709

详细: 数据手册

领券 16-15 9.5 折 嘉立创贴片惊喜价格(库存10K+) SMT扩展库



HC-XH-4AWT

品牌: HCTL(华灿天禄)

封装: SMD,P=2.5mm,卧贴

插针结构: 1x4P

间距: 2.5mm

安装方式: 卧贴

参考系列: XH

类目: 线对板针座

编号: C2845685

详细: 数据手册

领券 最高减15 嘉立创贴片惊喜价格(库存141) SMT扩展库

TJC3-4AWB

品牌: DLL(德利)

封装: SMD,P=2.5mm,卧贴

描述: 参考系列：DL25001(XH)系...

插针结构: 1x4P

间距: 2.5mm

安装方式: 卧贴

排数: 1

类目: 线对板针座

编号: C22374301

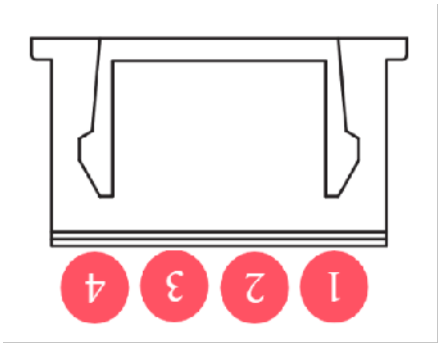
详细: 数据手册

领券 20-15 6 折 嘉立创贴片惊喜价格(库存0) SMT扩展库

11.1.20.1.3 XH2.54*4pin 连接器端子说明

胶套：XHP-4

端子：SXH-001T-P0.6



11.1.20.2 喇叭（ph1.25*2pin）接口说明

11.1.20.2.1 ph1.25*2pin 实物图片

11.1.20.2.2 ph1.25*2pin 连接器母座说明

型号 PH1.25-2p （立贴）

molex 原厂立贴编号 533980271

可以在立创商城或者淘宝直接搜索 533980271

兼容型号官网链接 https://www.molex.com/zh-cn/products/part-detail/533980271

molex 原厂卧贴编号 532610271

可以在立创商城或者淘宝直接搜索 532610271

兼容型号官网链接 https://www.molex.com/zh-cn/products/part-detail/532610271

下方截图仅做参考，非购买指南。

11.1.20.2.3 ph1.25*2pin 连接器端子说明

molex 原厂编号 510210200

兼容型号官网链接 https://www.molex.com/zh-cn/products/part-detail/510210200

可以在立创商城或者淘宝直接搜索 510210200

11.1.20.3 外部 IO（FPC1.0 * 10pin）接口说明

11.1.20.3.1 FPC1.0 * 10pin 连接器说明

类型：FPC 连接器

锁定特性：抽屉式

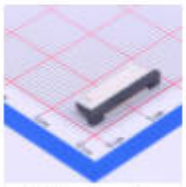
触点类型：下接

触点数量：10P

间距：1.0mm

安装方式：卧贴

下方截图仅做参考，非购买指南。



AFA07-S10FCC-00

品牌: JS(钜硕电子)

封装: SMD,P=1mm,卧贴

锁定特性: 抽屉式

触点类型: 下接

触点数量: 10P

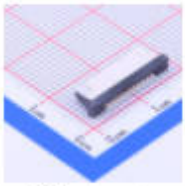
间距: 1mm

类目: FFC/FPC连接器

编号: C52162

详细: 数据手册

领券 16-15 嘉立创贴片惊喜价格(库存18K+) SMT扩展库



AFA07-S10FCA-00

品牌: JS(钜硕电子)

封装: SMD,P=1mm,卧贴

锁定特性: 抽屉式

触点类型: 下接

触点数量: 10P

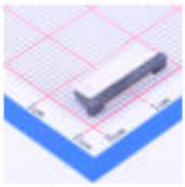
间距: 1mm

类目: FFC/FPC连接器

编号: C262716

详细: 数据手册

领券 16-15 嘉立创贴片惊喜价格(库存11K+) SMT扩展库



AFA07-S10ECA-00

品牌: JS(钜硕电子)

封装: SMD,P=1mm,卧贴

锁定特性: 抽屉式

触点类型: 上接

触点数量: 10P

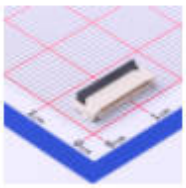
间距: 1mm

类目: FFC/FPC连接器

编号: C262737

详细: 数据手册

领券 16-15 嘉立创贴片惊喜价格(库存12K+) SMT扩展库



F1004-H-10-20G-R

品牌: XFCN(兴飞)

封装: SMD,P=1mm,卧贴

锁定特性: 翻盖式

触点类型: 双侧触点/上...

触点数量: 10P

间距: 1mm

类目: FFC/FPC连接器

编号: C668117

详细: 数据手册

领券 16-15 嘉立创贴片惊喜价格(库存7687) SMT扩展库

11.1.20.3.2 FPC1.0 * 10pin 排线说明

类型：FFC 连接线 (柔性扁平线缆)

总 PIN 数：10P

间距：1.0mm

触点：购买时需注意两端的触点方向是同向或反向 (将排线展平时, 金手指触点是否在同一面)

11.1.20.4 大尺寸高电压串口屏供电接口（HY2.0 8pin）接口说明

11.1.20.4.1 HY2.0 8pin 实物图片

11.1.20.4.2 HY2.0 8pin 连接器母座说明

插针结构：1x8P

间距：2mm

安装方式：卧贴

下方截图仅做参考，非购买指南。

ZX-HY2.0-8PWT

品牌: Megastar(兆星)
封装: SMD,P=2mm,卧贴
描述: 配套胶壳型号：ZX-HY2.0-...
插针结构: 1x8P
间距: 2mm
安装方式: 卧贴
参考系列: HY
类目: 线对板针座
编号: C7429575
详细: 数据手册

领券 16-15 | 9.5 折 | 领券 PLUS专享 | 嘉立创贴片惊喜价格(库存24K+) | SMT扩展库

WAFER-HY2.0-8PWB

品牌: XUNPU(讯普)
封装: SMD,P=2mm,卧贴
描述: 带扣
插针结构: 1x8P
间距: 2mm
安装方式: 卧贴
参考系列: HY
类目: 线对板针座
编号: C3029464
详细: 数据手册

领券 2000-188 | 9.5 折 | 嘉立创贴片惊喜价格(库存20K+) | SMT扩展库

ZX-HY2.0-8PZZ

品牌: Megastar(兆星)
封装: 插件,P=2mm
描述: 配套胶壳型号：ZX-HY2.0-...
插针结构: 1x8P
间距: 2mm
安装方式: 直插
参考系列: HY
类目: 线对板针座
编号: C7429584
详细: 数据手册

领券 16-15 | 9.5 折 | 领券 PLUS专享 | 嘉立创贴片惊喜价格(库存8133) | SMT扩展库

WAFER-HY2.0-8PZZ

品牌: XUNPU(讯普)
封装: 插件,P=2mm
描述: 配套：WAFER-HY2.0-8PJK...
插针结构: 1x8P
间距: 2mm
安装方式: 直插
参考系列: HY
类目: 线对板针座
编号: C5374801
详细: 数据手册

领券 2000-188 | 9.5 折 | 嘉立创贴片惊喜价格(库存23K+) | SMT扩展库

11.1.20.4.3 HY2.0 8pin 连接器端子说明

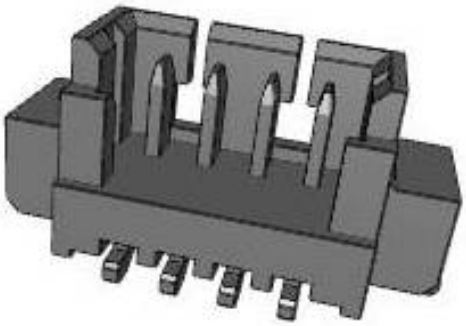
胶套：WAFER-HY2.0-8PJK

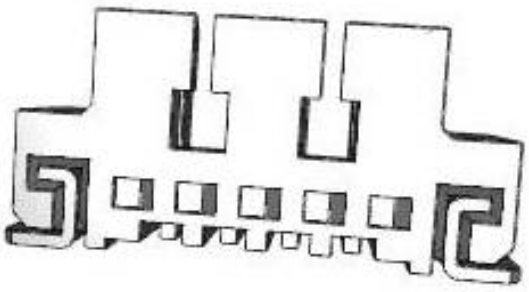
端子：ZX-HY2.0-DZ

11.1.20.4.4 ph1.25*4pin 连接器母座说明

用于小尺寸的屏幕，例如 x2 系列圆形屏

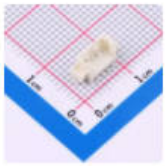
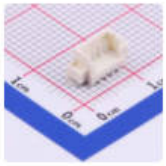
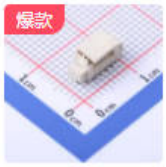
立贴兼容型号：molex 533980471



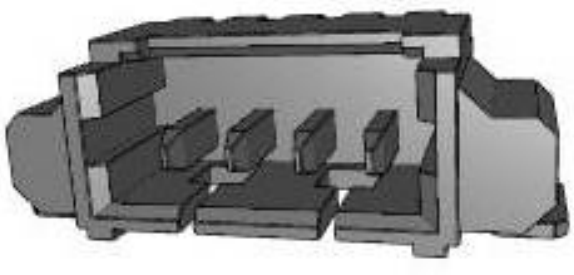


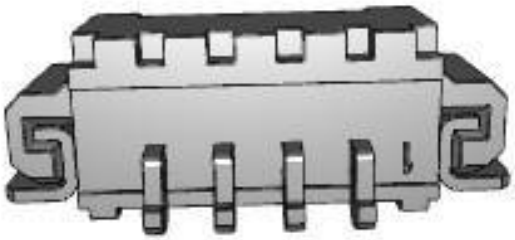
兼容型号官网链接 https://www.molex.com/zh-cn/products/part-detail/533980471

可以在立创商城或者淘宝直接搜索 533980471

型号/品牌/封装/类目	参数/描述	优惠券/标签	价格梯度(含税)
 533980471 品牌：MOLEX 封装：SMD 类目：线对板针座 编号：C17617036 收藏 对比 数据手册 复制 更多渠道	插针结构：1x4P 间距：1.25mm 安装方式：立贴 参考系列：PicoBlade(MX 1.25)	SMT 扩展库 SMT补贴 嘉立创库存28... 运 包邮券	近30天最低价 5+：￥0.8495 50+：￥0.6826 150+：￥0.5992 1400+：￥0.5366 2800+：￥0.4865 5600+：￥0.4615
 533980471 品牌：MOLEX 封装：SMD,P=1.25mm 类目：线对板针座 编号：C234195 收藏 对比 数据手册 复制 更多渠道	插针结构：1x4P 间距：1.25mm 安装方式：立贴 参考系列：PicoBlade(MX 1.25)	SMT 扩展库 SMT补贴 嘉立创库存134 运 包邮券	5+：￥0.8113 50+：￥0.6481 150+：￥0.5665
 爆款 HC-1.25-4PLT 品牌：HCTL(华灿天禄) 封装：SMD,P=1.25mm 类目：线对板针座 编号：C2845390 收藏 对比 数据手册 复制	插针结构：1x4P 间距：1.25mm 安装方式：立贴 参考系列：PicoBlade(MX 1.25)	SMT 扩展库 SMT补贴 嘉立创库存14... 商 最高减100 PLUS专享券	近30天最低价 5+：￥0.5472 50+：￥0.4506 150+：￥0.4009 1500+：￥0.3416 3000+：￥0.3273 4500+：￥0.3177
 HCZZ0015-4 品牌：HC(虹成电子) 封装：SMD,P=1.25mm 类目：线对板针座 编号：C7433470 收藏 对比 数据手册 复制	插针结构：1x4P 间距：1.25mm 安装方式：立贴 参考系列：PicoBlade(MX 1.25) 描述：针座1.25-4P 立贴闭口不...	SMT 扩展库 SMT补贴 嘉立创库存43... 商 满16-15	9.5折 近30天最低价 20+：￥0.140315 200+：￥0.114855 600+：￥0.1007 1500+：￥0.091105 10500+：￥0.083695 19500+：￥0.0798

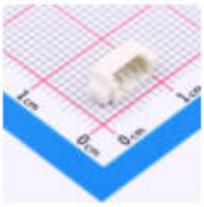
卧贴兼容型号：molex 532610471





兼容型号官网链接 https://www.molex.com/zh-cn/products/part-detail/532610471

可以在立创商城或者淘宝直接搜索 532610471

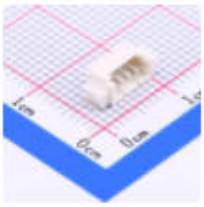


532610471
品牌：MOLEX
封装：SMD,P=1.25mm
类目：线对板针座
编号：C696092

插针结构：1x4P
间距：1.25mm
安装方式：卧贴
参考系列：PicoBlade(MX 1.25)

SMT 扩展库
SMT补贴 嘉立创库存78...
运 包邮券

收藏 对比 数据手册 复制 更多渠道

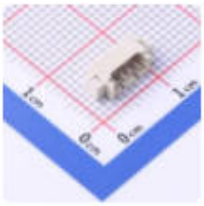


532610471
品牌：MOLEX
封装：SMD,P=1.25mm
类目：线对板针座
编号：C240847

插针结构：1x4P
间距：1.25mm
安装方式：卧贴
参考系列：PicoBlade(MX 1.25)
描述：4 位 接头 连接器 间距1.2...

SMT 扩展库
运 包邮券

收藏 对比 数据手册 复制 更多渠道

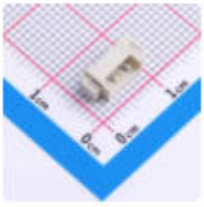


HC-1.25-4PWT
品牌：HCTL(华灿天禄)
封装：SMD,P=1.25mm,卧贴
类目：线对板针座
编号：C2845381

插针结构：1x4P
间距：1.25mm
安装方式：卧贴
参考系列：PicoBlade(MX 1.25)

SMT 扩展库
SMT补贴 嘉立创库存73...
商 最高减100
PLUS专享券

收藏 对比 数据手册 复制



HCZZ0001-4
品牌：HC(虹成电子)
封装：SMD,P=1.25mm,卧贴
类目：线对板针座
编号：C7433455

插针结构：1x4P
间距：1.25mm
安装方式：卧贴
参考系列：PicoBlade(MX 1.25)
描述：针座1.25-4P 闭口卧贴 编...

SMT 扩展库
SMT补贴 嘉立创库存52...
商 满16-15

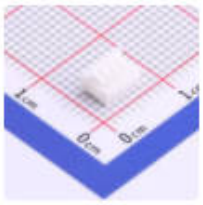
收藏 对比 数据手册 复制

11.1.20.4.5 ph1.25*4pin 连接器端子说明

立贴兼容型号：molex 510210400

兼容型号官网链接 https://www.molex.com/zh-cn/products/part-detail/510210400

可以在立创商城或者淘宝直接搜索 510210400

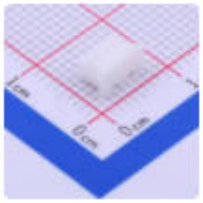


510210400
品牌：MOLEX
封装：P=1.25mm
类目：胶壳(线对板/线对线)
编号：C504988

插孔结构：1x4P
间距：1.25mm
参考系列：PicoBlade(MX 1.25)
排数：1

SMT 扩展库
SMT补贴 嘉立创库存86...
运 包邮券

收藏 对比 数据手册 复制 更多渠道

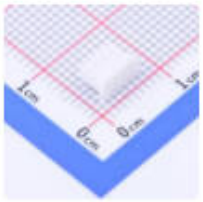


A1251H-4P
品牌：CJT(长江连接器)
封装：P=1.25mm
类目：胶壳(线对板/线对线)
编号：C338942

插孔结构：1x4P
间距：1.25mm
排数：1
每排PIN数：4

SMT 扩展库
SMT补贴 嘉立创库存11...
运 包邮券

收藏 对比 数据手册 复制



HC-1.25-4Y
品牌：HCTL(华灿天禄)
封装：P=1.25mm
类目：胶壳(线对板/线对线)
编号：C2962291

插孔结构：1x4P
间距：1.25mm
参考系列：PicoBlade(MX 1.25)
排数：1

SMT 扩展库
SMT补贴 嘉立创库存125
商 最高减100
PLUS专享券

收藏 对比 数据手册 复制

11.2 结构设计相关

11.2.1 户外使用说明

固定于户外使用时, 除了常规的防潮、防盐雾、防霉、防尘、耐高低温及绝缘等需求外, 还需注意以下几点:

1、在屏幕四周增加遮光罩，避免阳光直射，阳光会加速屏幕的老化，缩短屏幕寿命

2、避免设备过热，过热也会加快屏幕的老化

3、我司出厂的屏幕默认不喷涂三防漆，客户需要自行喷涂三防漆时，请注意避开金手指和 FPC 连接器（金手指和 FPC 连接器不要喷涂三防漆），以免造成屏幕损坏

11.3 软件相关

11.3.1 设计好的工程有哪些下载方式

11.3.1.1 通过串口下载

有 HMI 文件, 可以通过 USARTHMI 上位机来下载工程—-> 通过串口下载工程到串口屏 (page 144)
只有 TFT 文件—-> 如何使用 *TFT* 文件下载助手 *(TFTFileDownload)* 下载工程 (page 1435)

11.3.1.2 通过 SD 卡下载

SD 卡请勿超过 32GB—-> 通过 *SD* 卡下载工程到串口屏 (page 155)

11.3.2 tft 文件串口/SD 卡下载失败如何解决

1. 新建一个简易工程，单页面的，添加个背景色就可以了，编译保存，通过串口或 SD 卡重新下到屏里。
2.SD 卡重新格式化，慢格，FAT32 格式，重新下编译好的.tft 文件

相关链接

TFT 文件如何下载到串口屏中 (page 1405)

如何输出 TFT 生产文件 (page 155)

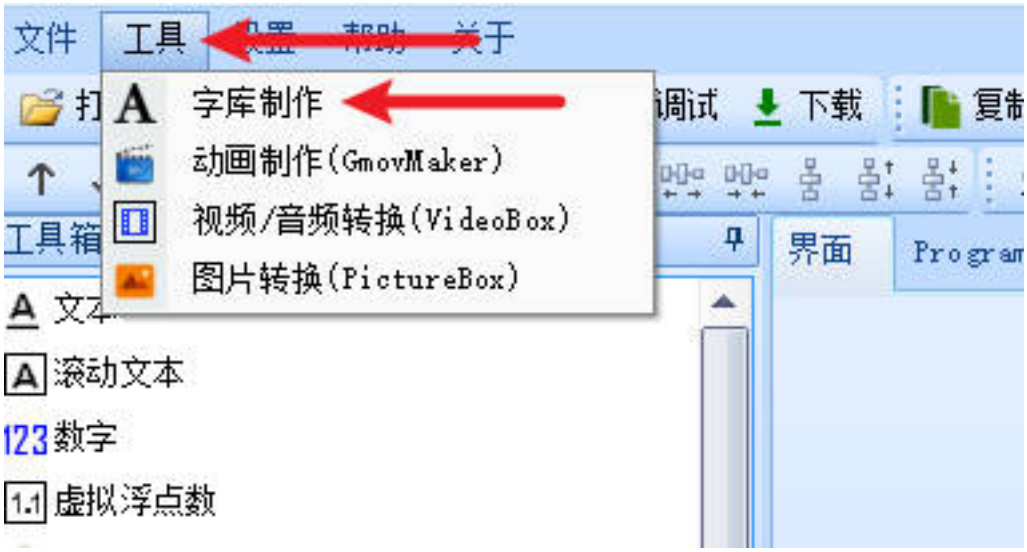
11.3.3 调试或下载时联机失败

1. 检测硬件，串口驱动是否安装正确，确认串口存在。
2. 检测连接方式是否正确，是用 TTL 电平还是用 232 电平。
3. 检测是否共地，如果不用电脑的 USB 供电的话，除了 RX,TX 连接外，还要共 GND。
4. 检查电源是否稳定，电压有没有被拉低，电源不稳定可能会造成任何异常。
5. 检查接线是不是太长了 (不建议超过 30cm)

11.3.4 指定字库

1、什么是指定字库

打开字库制作工具



在新建字库时，范围选择指定字符，并且在下方的输入框输入需要的文字，生成的就是指定字库

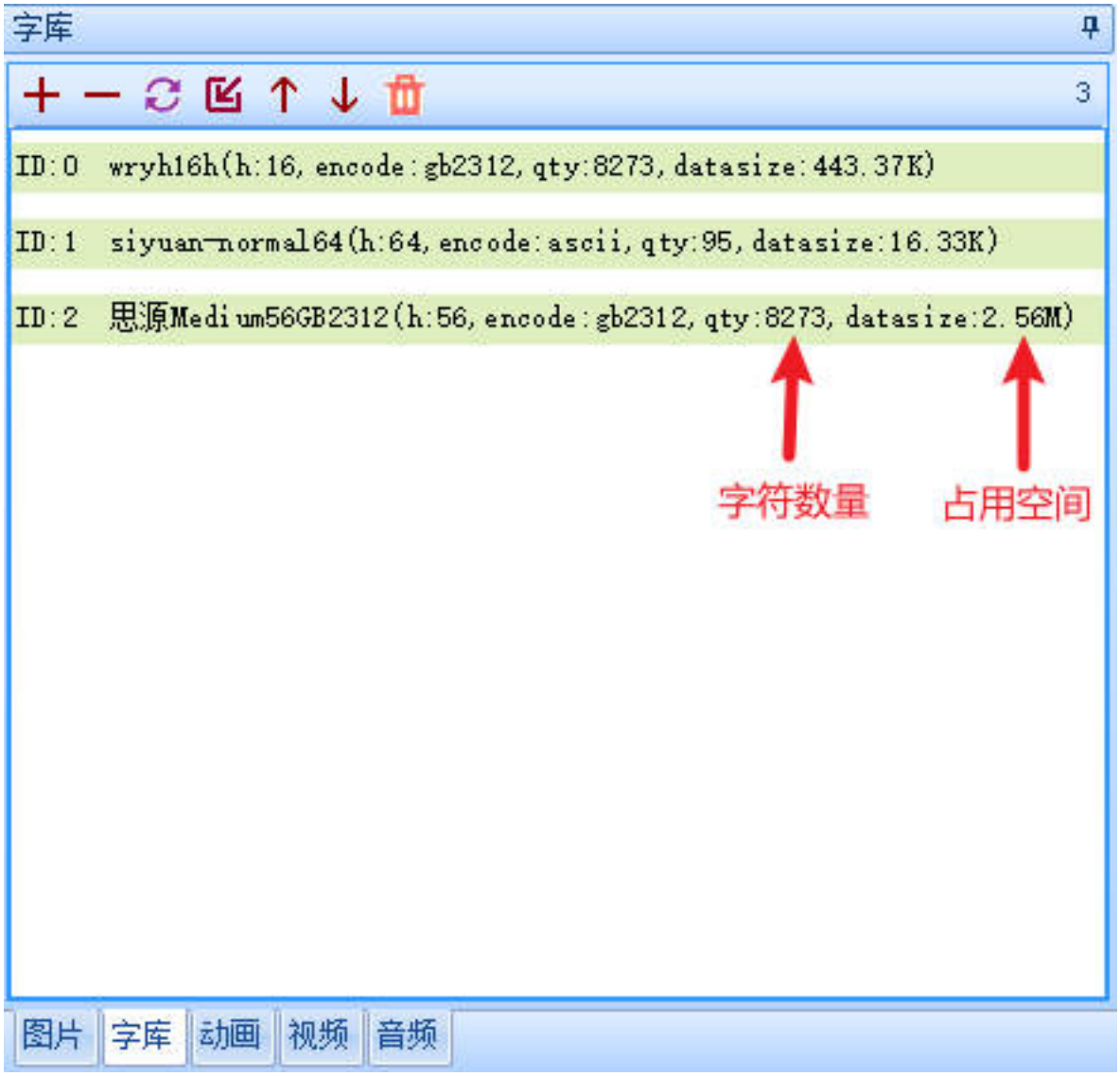
字库制作工具
字高: 56
编码: gb2312
字体抗锯齿
字体: 思源黑体 CN Medium
字体加粗
预览区
散
范围: 指定字符
字库名称:
生成字库

2、指定字库有什么作用

可以大大缩小字库占用的体积

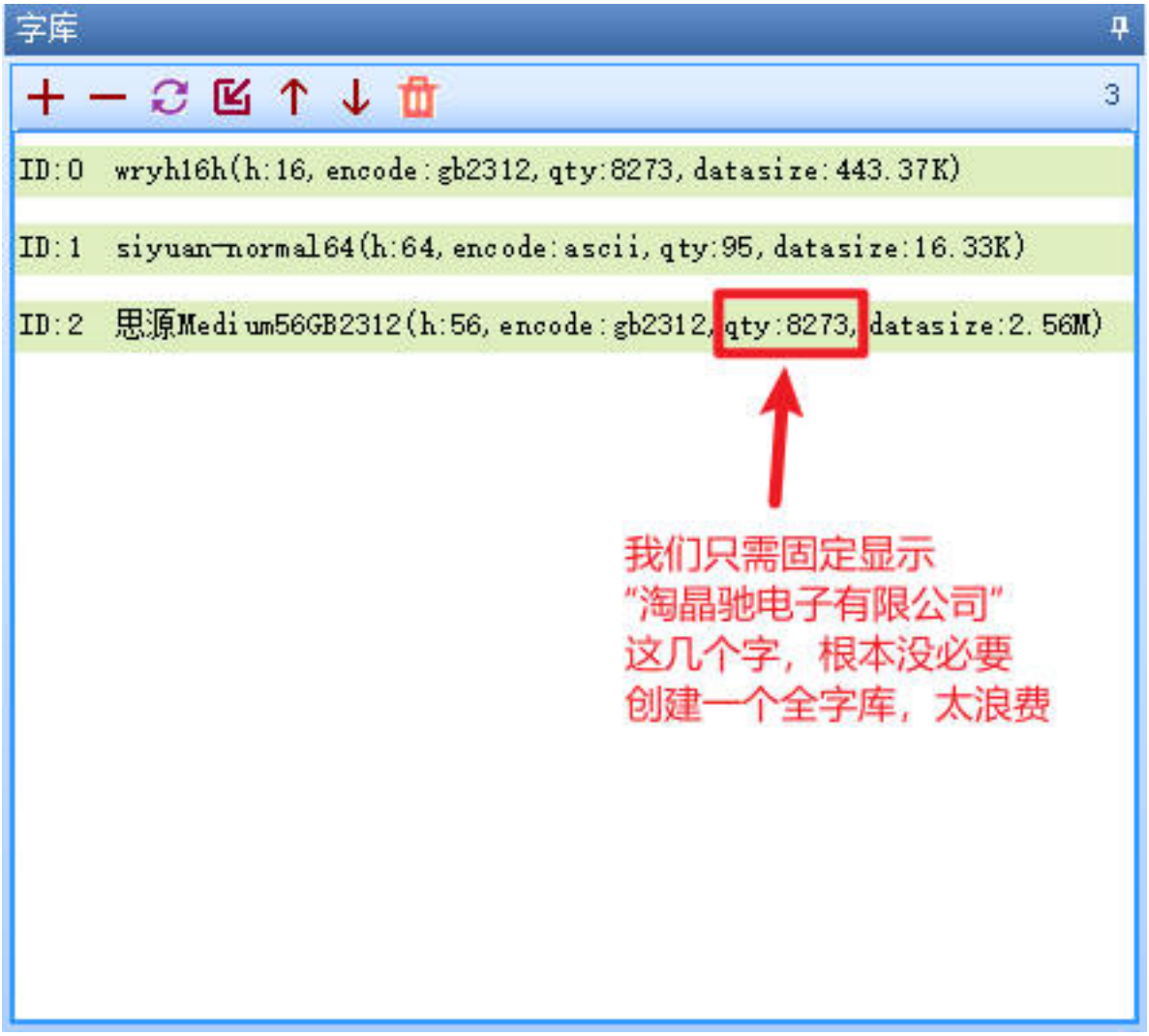
以字高 56，编码 GB2312，字体为思源，当范围选择所有字符时，生成的文件大小 2,683,293 字节

导入工程后该字库占用空间 2.56M, 占用空间非常大



字库占用空间大的原因有两个，一个是字高，一个是字符数量

如果要优化字库占用的空间，要么减小字高，要么减小字符数量



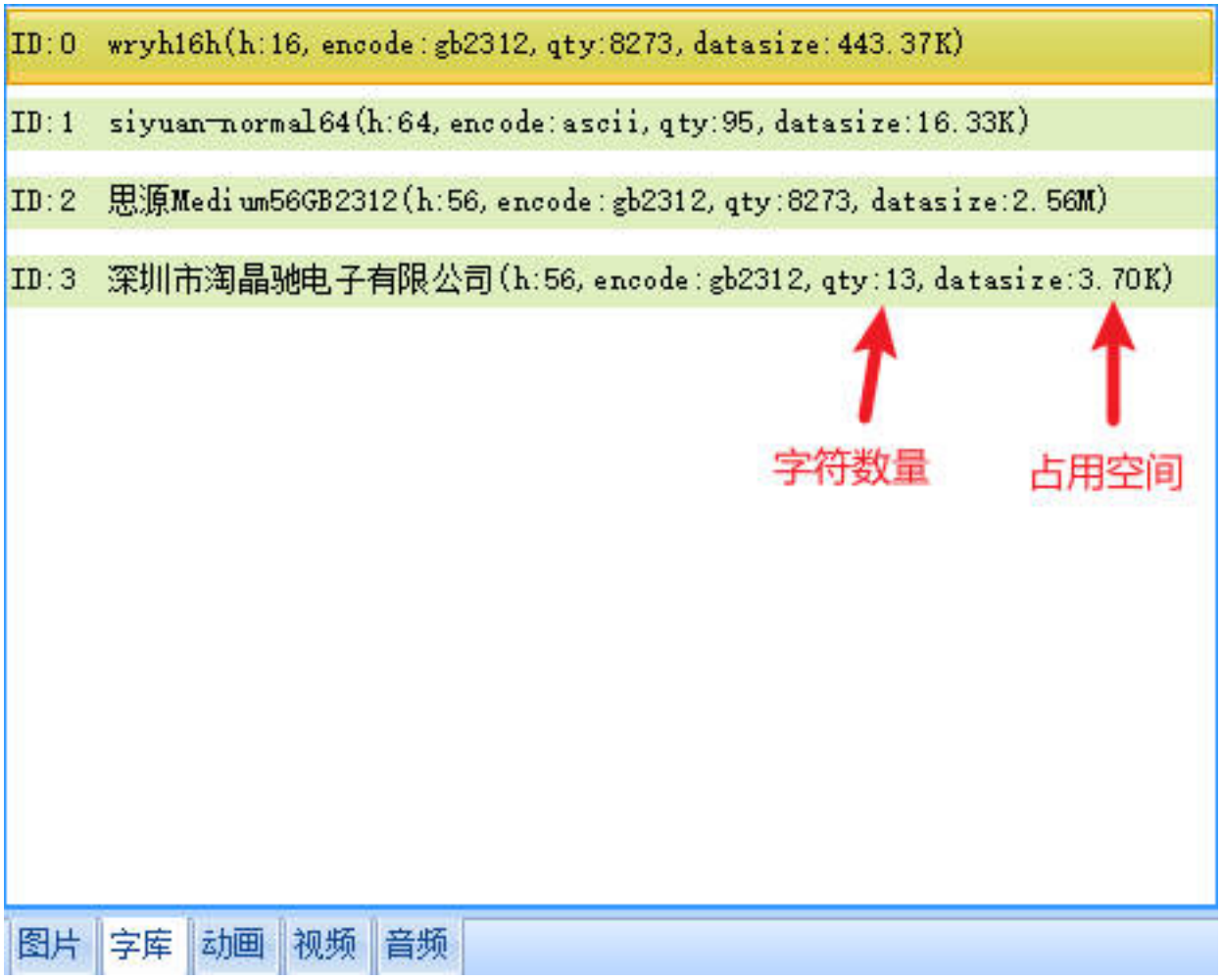
可以看到当范围选择所有字符时，整个字库有 8273 个字符，几乎包括了所有的汉字，但通常情况下，使用 56 这么大的字号，通常是作为标题使用，大部分情况下用到的字是很少的，此时我们建议使用指定字符来生成字库

3、如何制作指定字库

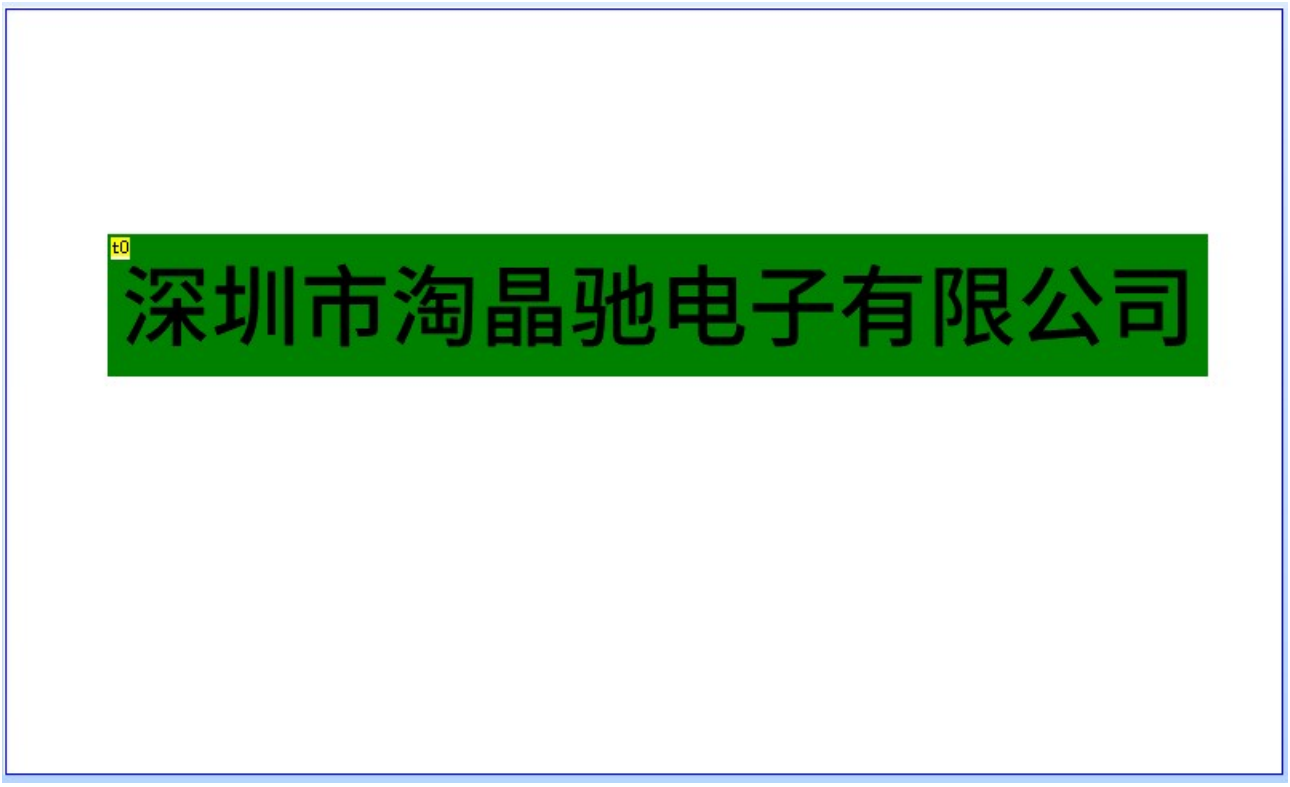
如图所示，范围选择指定字符，并且在下方的输入框中输入你需要显示的文字

生成的指定字库文件大小 3033 字节

导入工程后该字库占用空间 3.7k

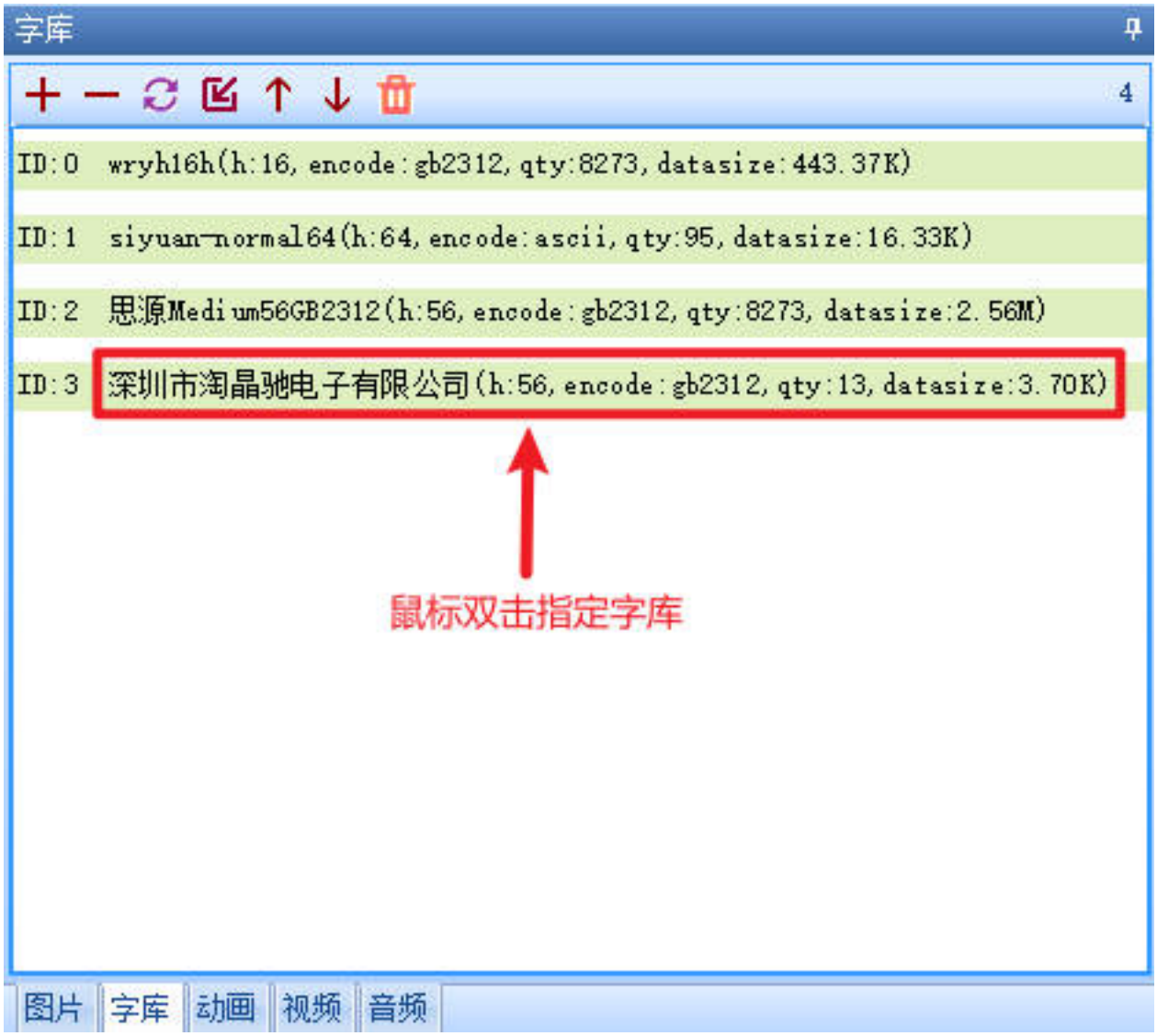


用于显示固定的标题文字

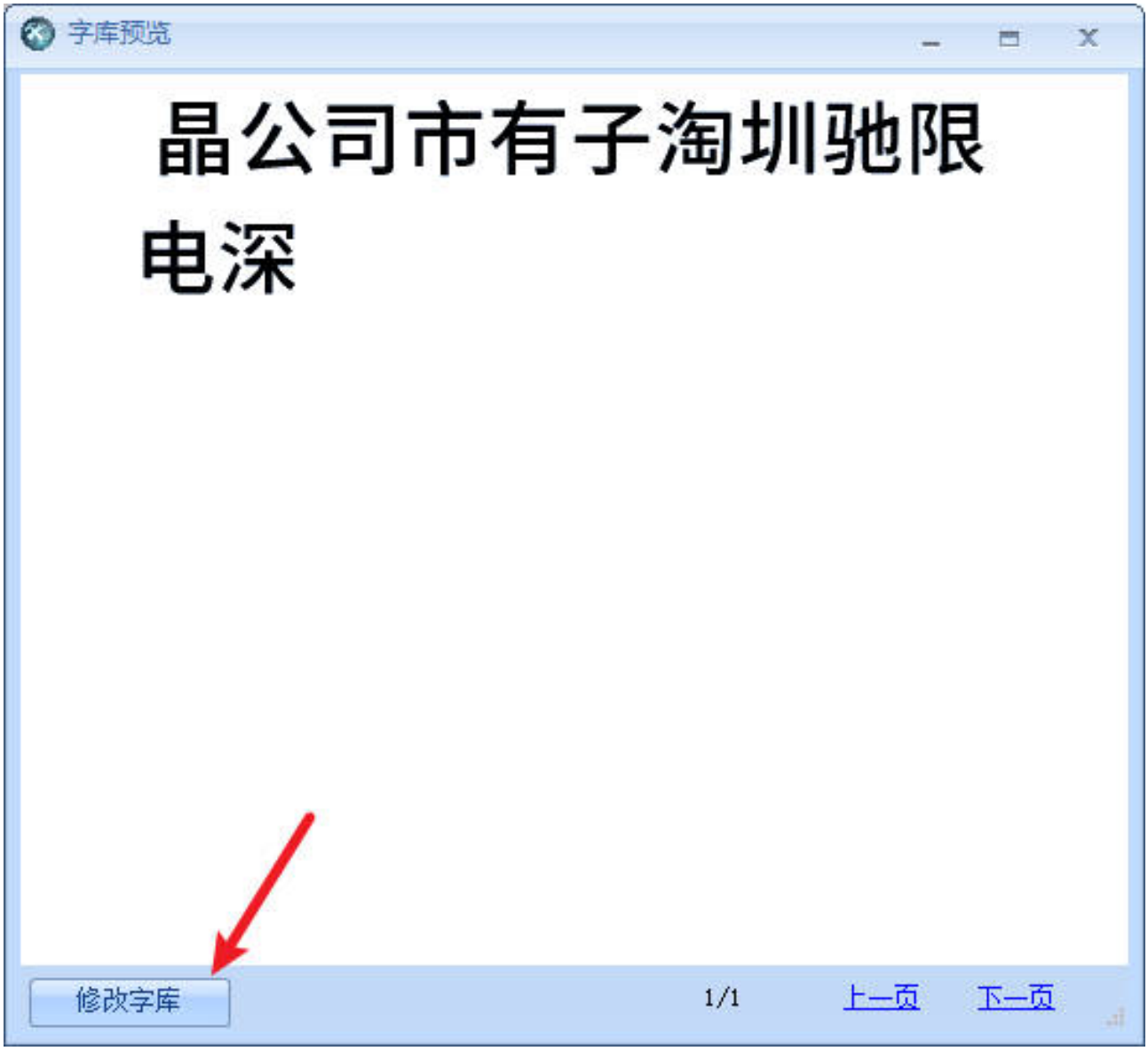

深圳市淘晶驰电子有限公司

4、修改指定字库

指定字库是可以修改的，使用鼠标双击字库



可以看到左下角出现了修改字库的按钮

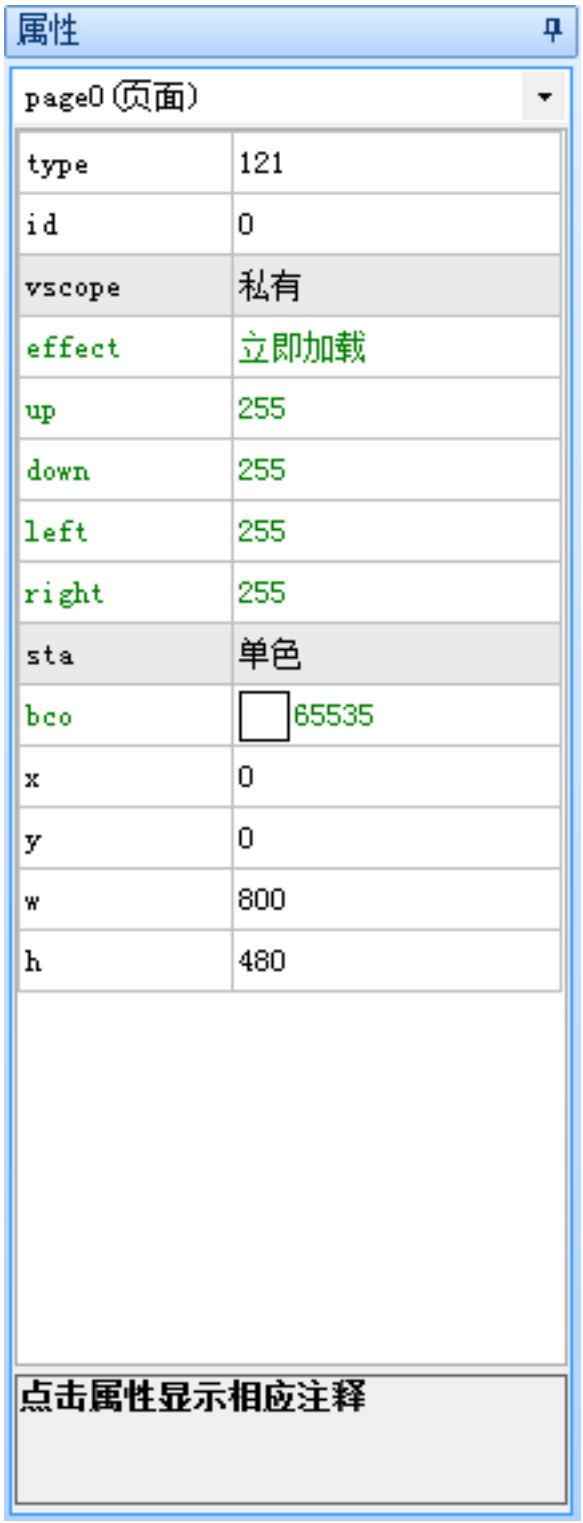


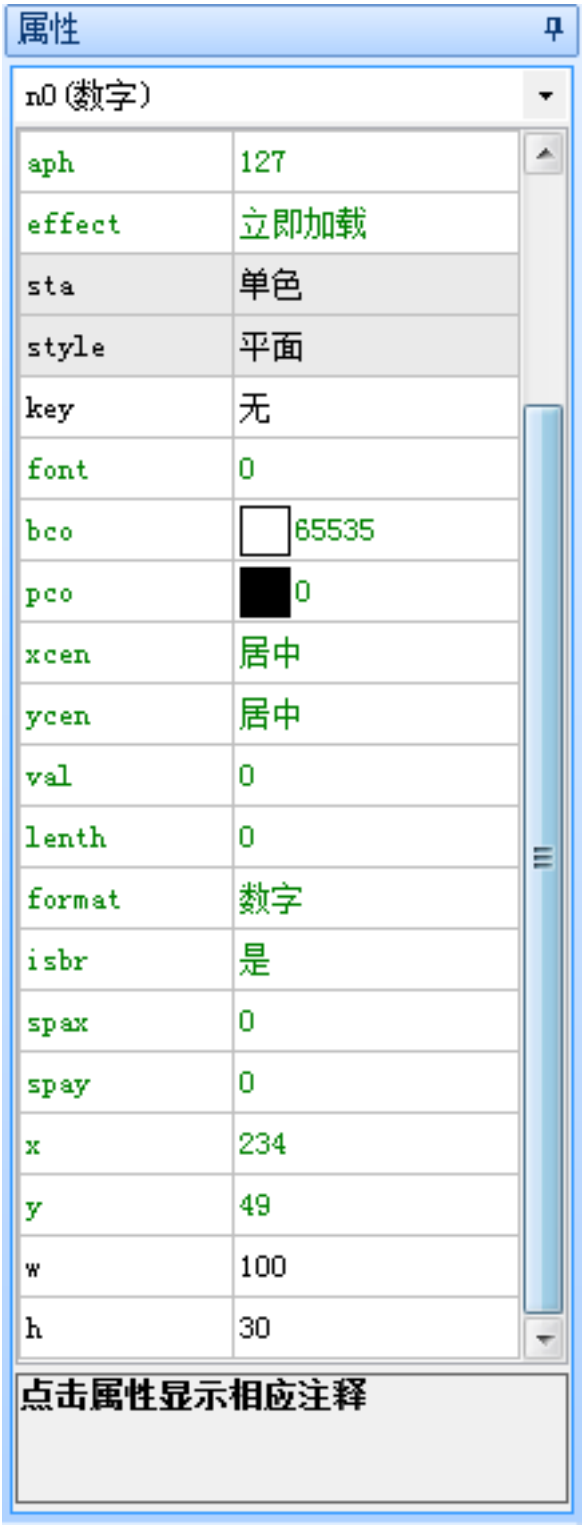
11.3.4.1 指定字库-相关链接

创建字库和导入字库 (page 125)
在做字库的时候有部分字体无法选择 (page 1360)
编译报错：*XXX* 初始值无效 (page 1342)
utf8 字库下如何只选择汉字 (page 1403)
报错:*file is too large for destination device* (page 1359)

11.3.5 哪些控件属性可以运行中修改，哪些不能运行中修改，绿色属性和黑色属性有什么区别？

如下图所示：属性栏中显示为绿色的属性都可以在运行中修改，黑色的属性只能在编辑时修改，不能在运行中修改。





注意:

1. type 属性只能读取不能修改
2. objname 属性只能在上位机修改，不允许读取

视频说明：绿色属性和黑色属性有什么区别

11.3.6 txt 属性和 val 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号

txt 属性，是字符串型属性。在给字符串做常量赋值时，需要添加双引号 (与 C 语言相同)。

例如:

```
t0.txt="淘晶驰"

t0.txt=t0.txt+"123"       t0.txt在原来字符串的尾部在追加"123"这三个ASCII字符，因为"123
↪"被引号包裹了，所以"123"是字符串
```

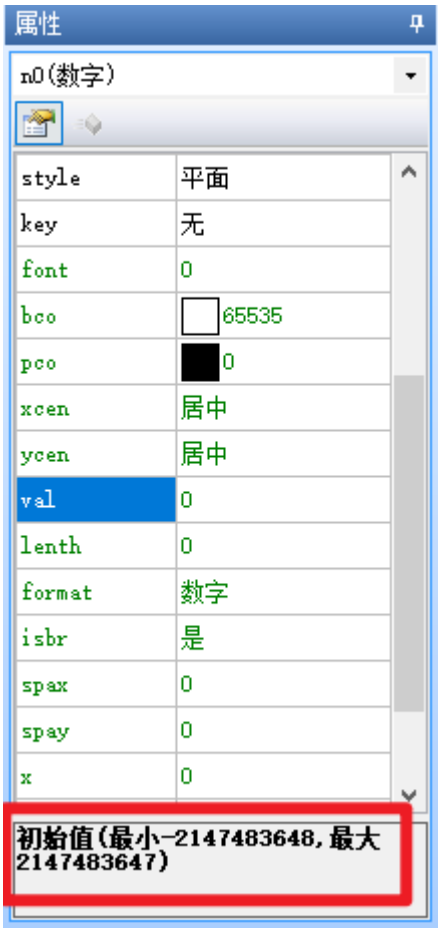
val 属性，是数值型属性。所有数值型操作时，不需要不需要使用双引号。

例如:

```
n0.val=123       //123没有被引号包裹，所以123是数值

n2.val=n0.val+666
```

字符串型属性可以存储任意字符，例如数字 0-9，字母 a-z/A-Z，以及中文等文字

但是数值型属性只能存储数值，只能存储数值，且有范围限制



那么为什么有了文本控件还要设计一个数字控件呢

这涉及到了计算的问题

```
t0.txt="123"
t1.txt="456"
t2.txt=t0.txt+t1.txt     //t2将显示123456

n0.val=123
n1.val=456
n2.val=n0.val+n1.val     //n2将显示579
```

对于“+”来说，两个字符串型属性用“+”连接，相当于字符串拼接。两个数值型属性用“+”连接，相当于数值相加。

对于“-”来说，两个字符串型属性用“-”连接，将会报错。两个数值型属性用“-”连接，相当于数值相减。

对于“*”来说，两个字符串型属性用“*”连接，将会报错。两个数值型属性用“*”连接，相当于数值相乘。

对于“/”来说，两个字符串型属性用“/”连接，将会报错。两个数值型属性用“/”连接，相当于数值相除。

还有一种特殊的用法

字符串型属性-数值型属性，将会删除字符串属性后面若干个字符。

小技巧： 目前仅有 txt、path、dir、filter 这 4 个属性属性是字符串型属性，其他属性一般都是数值型属性。

数值型属性和字符串属性互相转换需要使用 covx 指令。

objname 属性在运行时不可读不可写。

11.3.6.1 字符串属性和数值属性-相关链接

数字控件 (page 417)

文本控件 (page 405)

txt 属性和 val 属性有什么区别，为什么有些属性赋值要加双引号，有些不用加双引号 (page 1297)

11.3.7 运行中控件属性被修改后，离开页面再回来，不希望属性回到初始值怎么操作

将对应控件的 vscope 属性由默认的“私有”修改为“全局”即可。

注意:

1. 将控件设置为全局以后，只有在设备复位后会对所有全局控件做一次赋值操作。其他任何设备不会主动去修改全局控件属性值。
2. 每次进入页面时设备会执行页面的“前初始化事件”或者“后初始化事件”中的代码。如果用户在这里增加了任何操作属性的代码，则会按照用户代码操作去修改对应变量属性值。

11.3.7.1 跨页面操作-相关链接

跨页面赋值，全局变量操作 (page 847)

11.3.8 SD 卡有哪些作用

对于 K0/T0/T1 系列来说，SD 卡只能用于升级屏幕固件，请参考通过 *SD* 卡下载工程到串口屏 (page 155)

对于 X2/X3/X5 系列来说，SD 卡不仅能用于升级屏幕固件，还能读写 SD 卡中的文件，请参考*SD* 卡读写文件流程 (page 1024)

11.3.9 串口屏卡顿的原因

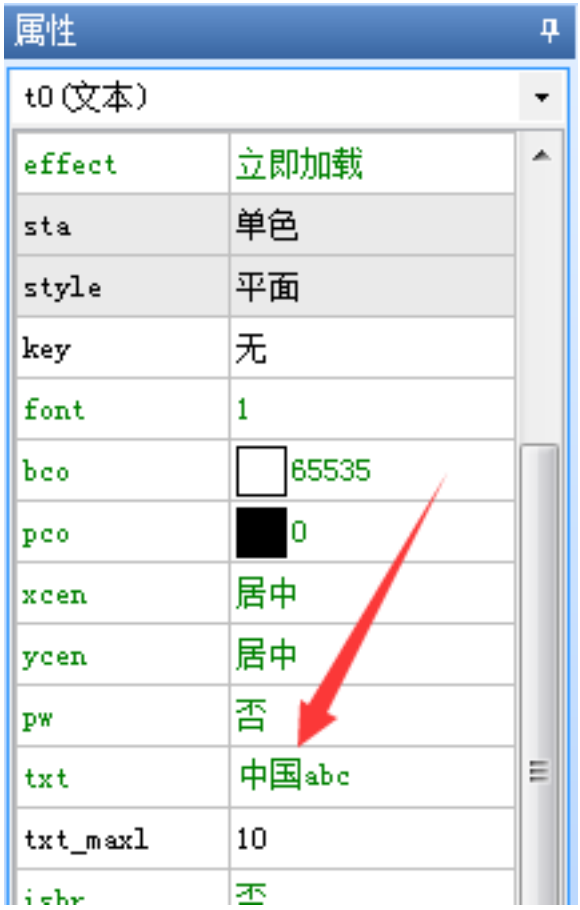
1. 设置背景图片时，使用了图片控件来显示背景，导致图片控件与其他控件重叠，正确的背景图片设置方式请参考设置页面的背景图片 (page 397)

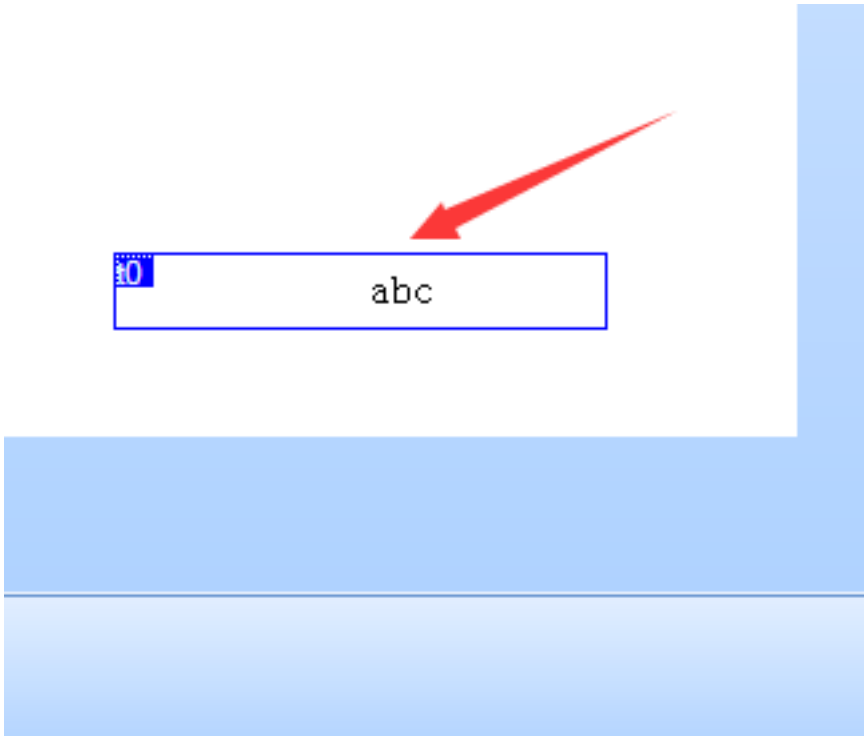
2. 定时器中有耗时的代码在不断运行

3. 控件之间重叠严重，多个控件之间互相覆盖（有接触），当刷新其中一个控件时，有接触的控件也会被一同刷新，照成系统效率低下，因此控件之间请保持一个安全的距离（互不重叠）

11.3.10 文本控件或按钮控件不显示我输入的字符内容或显示不全

如下图：输入的内容是:”中国 abc”，控件显示出来的是:”abc”

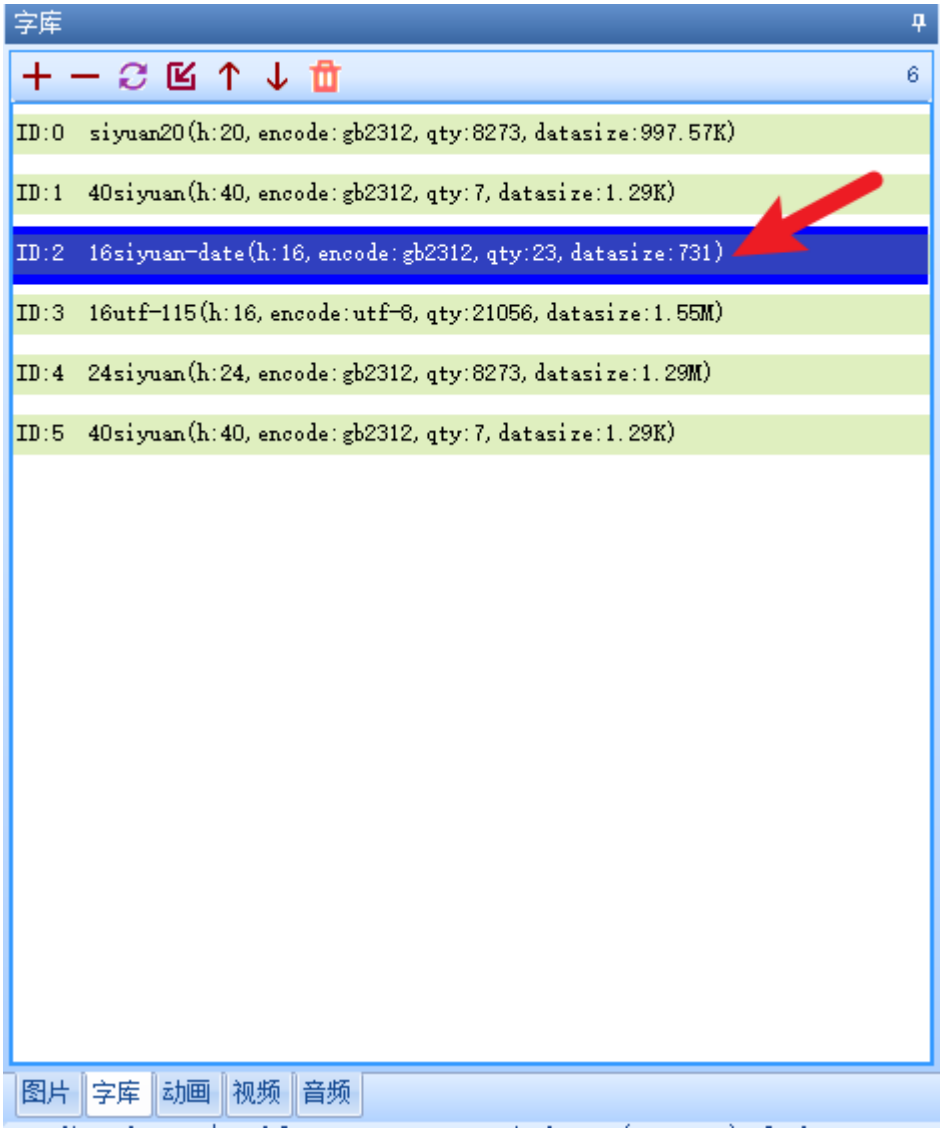




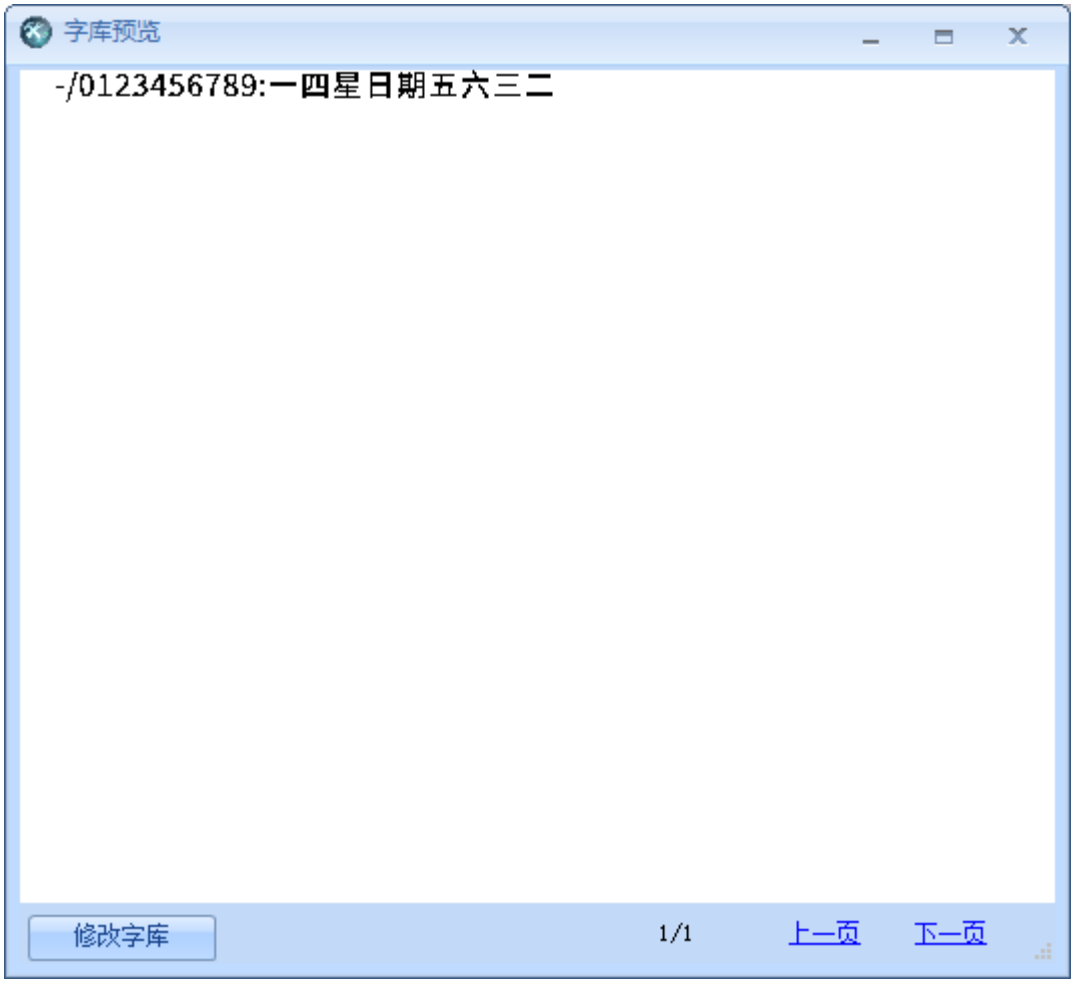
以上情况是因为 t0 控件的 font 属性设置的那个字库里面没有”中国”两个字, 所以”中国”没有显示出来。

解决办法是重新做一个全字库或指定字符字库（指定字符字库的话一定要含有”中国”两个字）。

双击某个字库, 可以查看字库中有哪些文字



双击某个字库进入字库预览, 如果控件使用了该字库, 但是调用了字库中没有的文字, 就会无法显示


字库预览
-/0123456789:一四星日期五六三二
修改字库
1/1
上一页
下一页

11.3.11 单片机发指令给屏幕没有反应

1. 确定接线线序是否有问题，正确的接线方法是：单片机的 RX 接屏的 TX，单片机的 TX 接屏的 RX。

2. 单片机的通信波特率要和屏的一致，串口屏的默认波特率是 9600，数据格式 8-0-1（1 位起始位，8 位数据位，无校验位，1 位结束位）。

3. 单片机控制屏的指令格式，一条控制指令，一条结束符，控制指令见指令集的指令，结束符是 16 进制的 3 个 FF(FF FF FF).

4. 确定以上 3 点都没问题，可以用串口助手监视单片机发过来的数据。

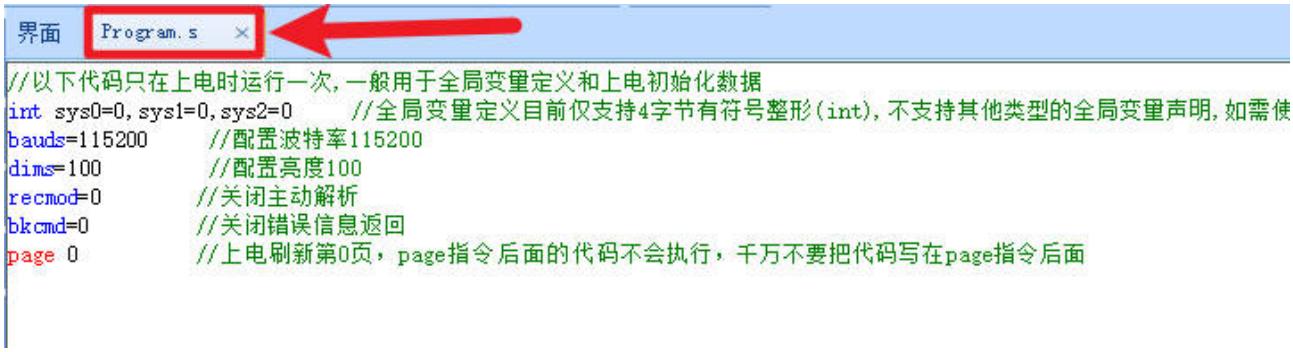
6. 要参考 STM32 控制屏的例子请参考使用 *stm32cubeide(HAL 库) 开发* (page 279)

11.3.12 如何配置亮度，主动解析，波特率

亮度范围：0-100

设备支持的波特率有：2400 4800 9600 19200 38400 57600 115200 230400 256000 512000 921600

> **注意：** page 指令后面的代码将不会被执行，所以千万不要把代码写在 page 指令后面
> page 指令后面的代码将不会被执行，所以千万不要把代码写在 page 指令后面
> page 指令后面的代码将不会被执行，所以千万不要把代码写在 page 指令后面
> page 指令后面的代码将不会被执行，所以千万不要把代码写在 page 指令后面




```
//以下代码只在上电时运行一次，一般用于全局变量定义和上电初始化数据
int sys0=0,sys1=0,sys2=0      //全局变量定义目前仅支持4字节有符号整形(int)，
↪不支持其他类型的全局变量声明，如需使用字符串类型可以在页面中使用变量控件来实现
bauds=115200      //配置波特率115200
dims=100          //配置亮度100
recmod=0         //关闭主动解析
bkcmd=0          //关闭错误信息返回
page 0           //
↪上电刷新第0页，page指令后面的代码不会执行，千万不要把代码写在page指令后面
```

11.3.12.1 亮度，主动解析，波特率-相关链接

baud 和 bauds 的区别 (page 1331)

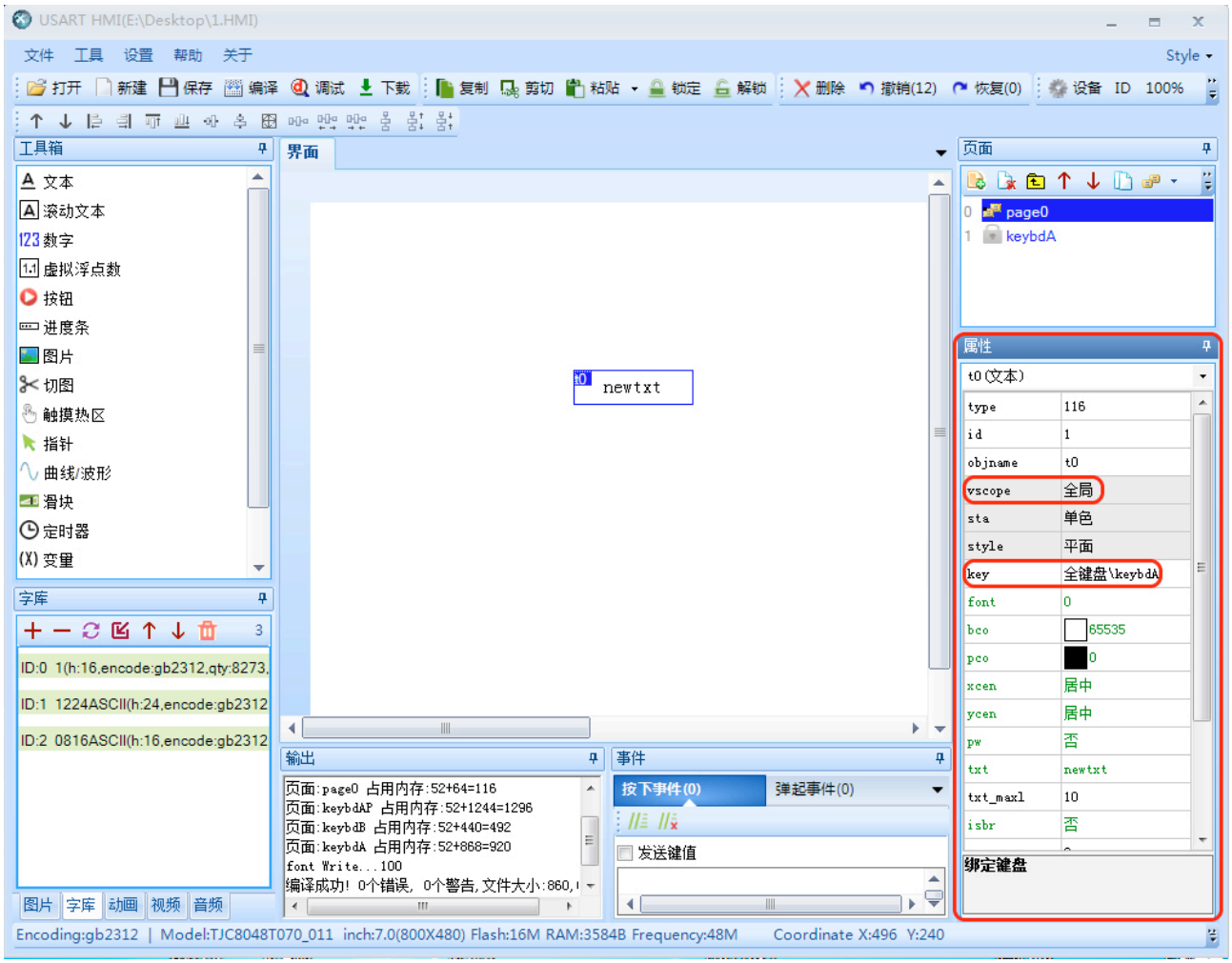
dim 和 dims 的区别 (page 1330)

bkcmd-串口指令执行状态数据返回 (page 785)

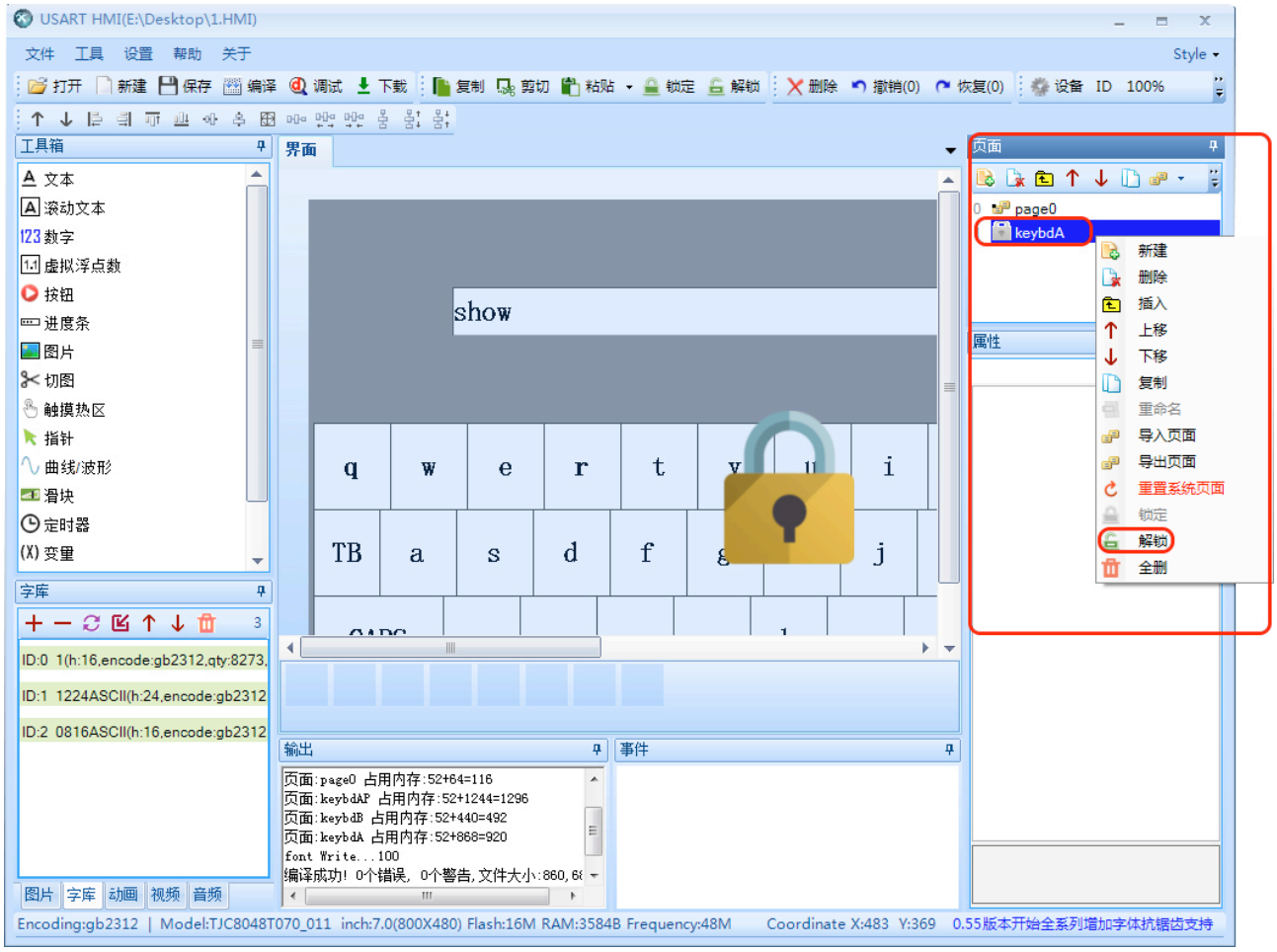
recmod 是配置主动解析，不使用主动解析可以忽略这个配置项，不需要了解

11.3.13 系统内置的触摸键盘怎么修改风格

1. 先找到要需要修改的内置键盘：新建一个文本控件，把文本（vscope）属性改为全局，在文本（key）属性选择自己所需键盘的大致风格。



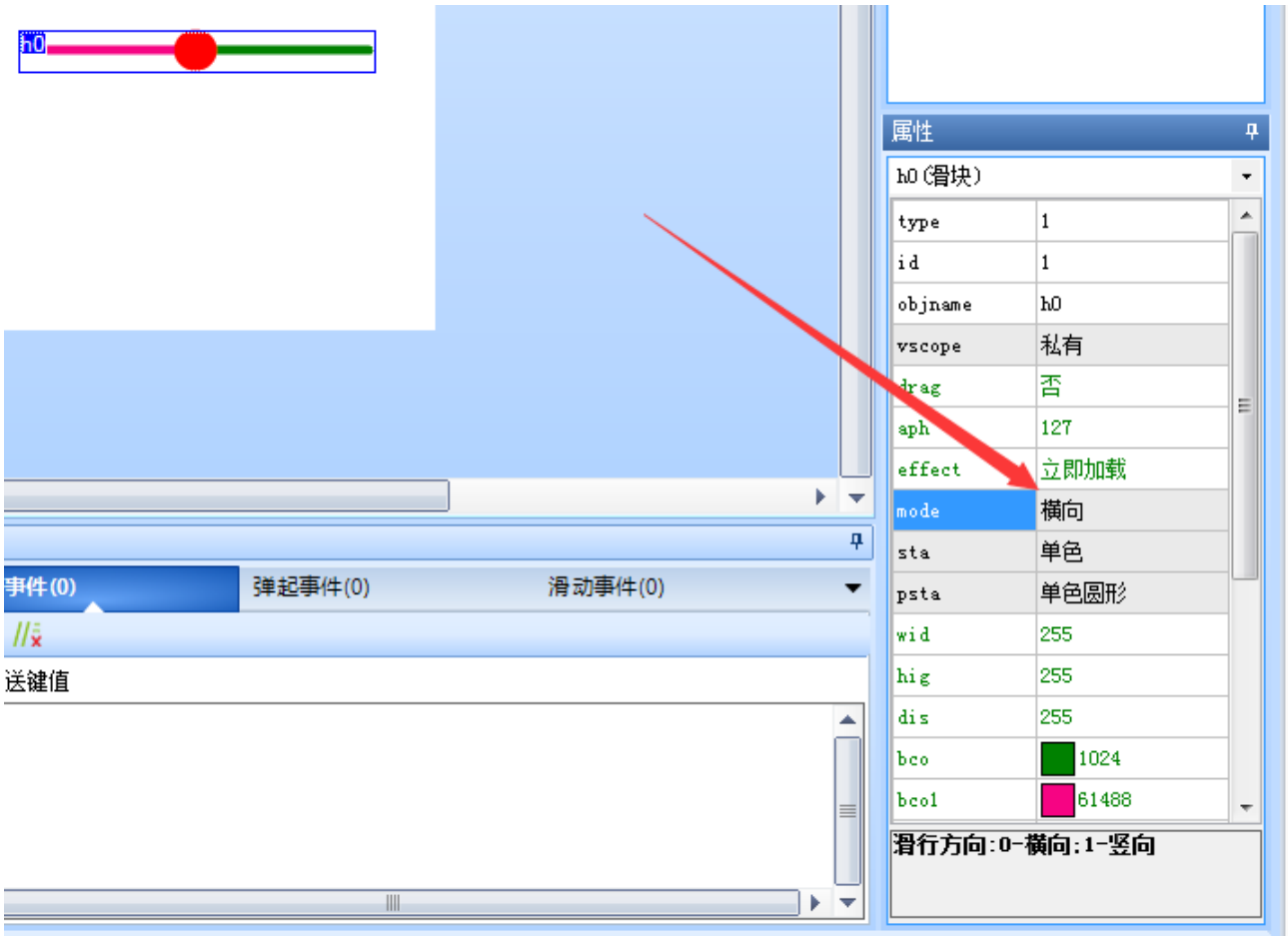
2. 选择完后页面会自动出现一个键盘页面，而且是有上锁的标志，此时选定右击选择解锁。



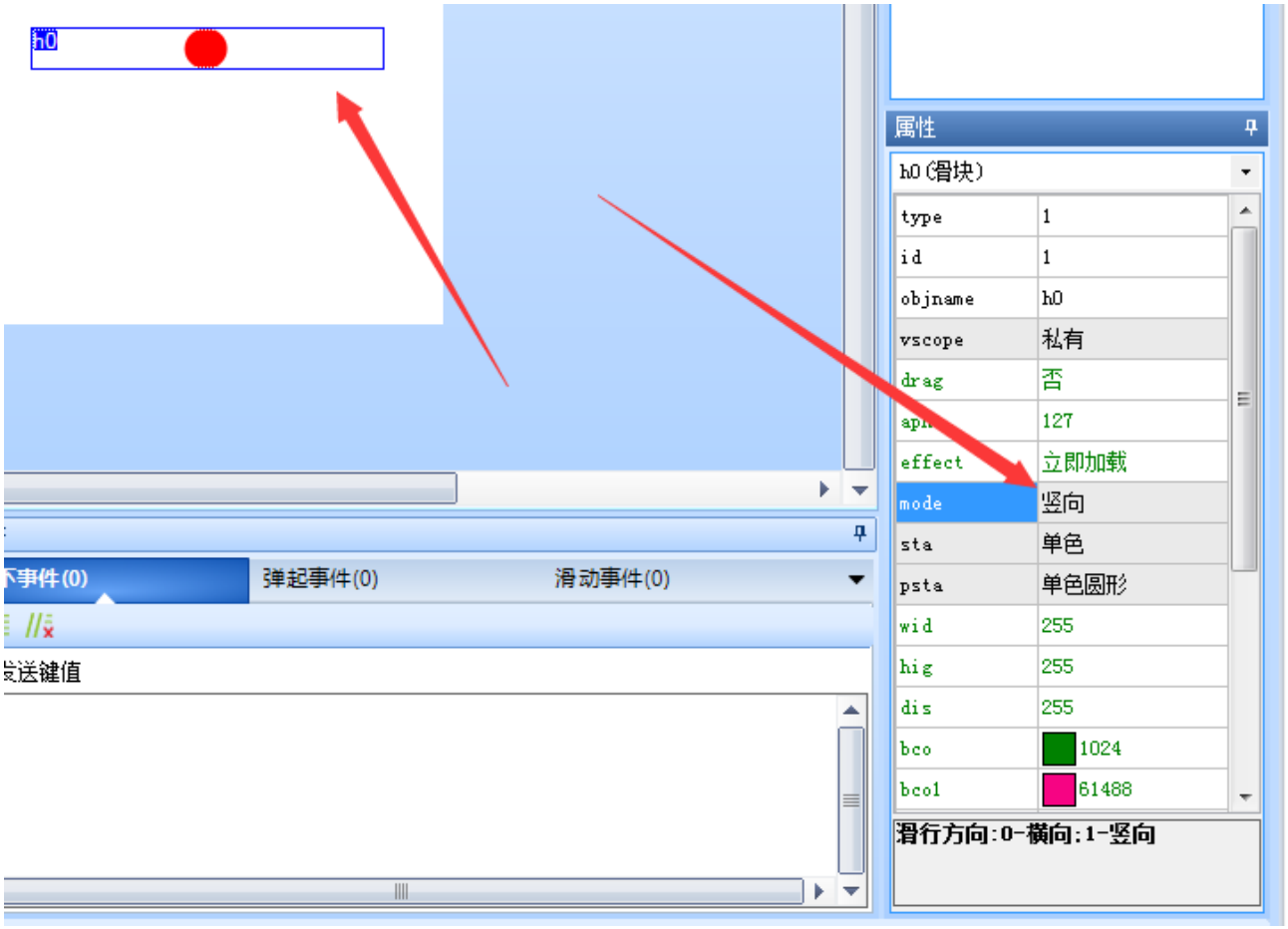
3. 解锁完以后就可以按照自己想要的风格布局及代码修改，当然修改完以后你还可以对键盘右击锁定，锁定会提示输入密码，不填默认无，如果填了，不小心忘记了密码，选定右击重置会回来系统默认的键盘状态。

11.3.14 滑块或进度条控件默认是横向，为什么我设为竖向后没有变化，还是显示横向呢

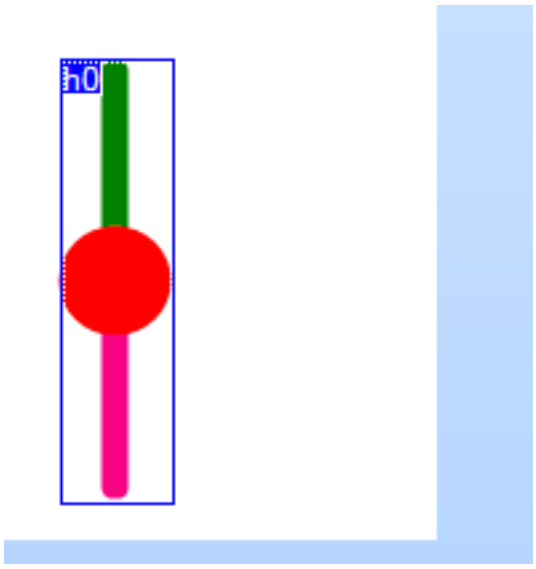
默认横向，显示是正常的，如下图：



但是修改为竖向以后，控件还是横的，如下图：



这是因为设置为竖向后，软件并不会修改控件的宽度和高度，需要你手动调整他的宽度和高度，调整之后再看显示就是正常的了，如下图：



11.3.15 单片机和串口屏通信为什么单片机上电后要延时一会再发指令，延时多久合适

屏上电到显示这个过程是有一段时间的（相对跑系统的来说可以忽略）。

如果用户主控和串口屏同时上电，用户主控有可能比串口屏先完成初始化，如果这时候马上发指令给屏，而串口屏并未完全成功启动，这样会导致用户发的数据丢失。

所以，我们建议用户 MCU 完成初始化以后延时一段时间再发数据给串口屏。

对于 T0 和 K0 系列我们建议延时 250MS, 对于 X3 和 X5 系列建议延时 1.5S。

如果用户对串口屏启动时间要求比较精准，串口屏启动成功后会发 0x88+ 结束符，这是说明串口屏已成功启动，用户单片机可以发数据给屏了。

11.3.16 单片机上电后为什么要先发一次 0x00 0xff 0xff 0xff 给屏幕

这个操作仅适用与某些单片机。

因为某些单片机在上电初始化过程中，因为某些代码原因，可能会翻转串口相关的 IO 口电平，导致串口屏收到了某些无关的数据，因此第一条指令会解析失败。

第一次发指令前先发一次 0x00 0xff 0xff 0xff, 是因为上电过程中有可能串口引脚上产生了杂波导致屏幕已经收到一个或者多个错误数据了，所以先发一次 0x00 0xff 0xff 0xff 来结束当前指令，后面就可以正常操作了。

因为这条指令带有 3 个 0xff，也就是结束符，所以这条指令会被解析，然后连同清空掉屏幕串口缓冲区里的无关数据。

当然你也可以在第一次发指令时发送两条一样的指令，则第一条指令充当了清空串口缓冲区的角色。

对应的单片机代码

```
printf("\x00\xff\xff\xff");
```

11.3.17 屏幕地址怎么设置，怎么使用

当用户想一个串口同时控制多个串口屏独立工作的时候，可以给每个屏设置不同的地址。(注意，在 TTL/RS232 通信时不能直接将多个触摸屏的 TX 连接到一起，这样会导致短路。但是可以将多个触摸屏的 RX 连接到一起，接用户串口的 TX 型号。)

或者用户使用 485 总线与屏幕通信，且 485 总线上存在多个设备的时候，可以给屏幕设置地址。

屏幕地址的具体设置，请参考系统变量 addr

例如，现在将两个触摸屏的 RX 并联到一起，然后与电脑串口（TTL 电平）的 TX 连接。设置串口屏 A 的地址 0x100(addr=256)，设置串口屏 B 的地址为 0x200（addr=512）. 通过电脑操作串口发送命令详解：(红色下划线为十六进制数据，黑色框中为 ascii 码)

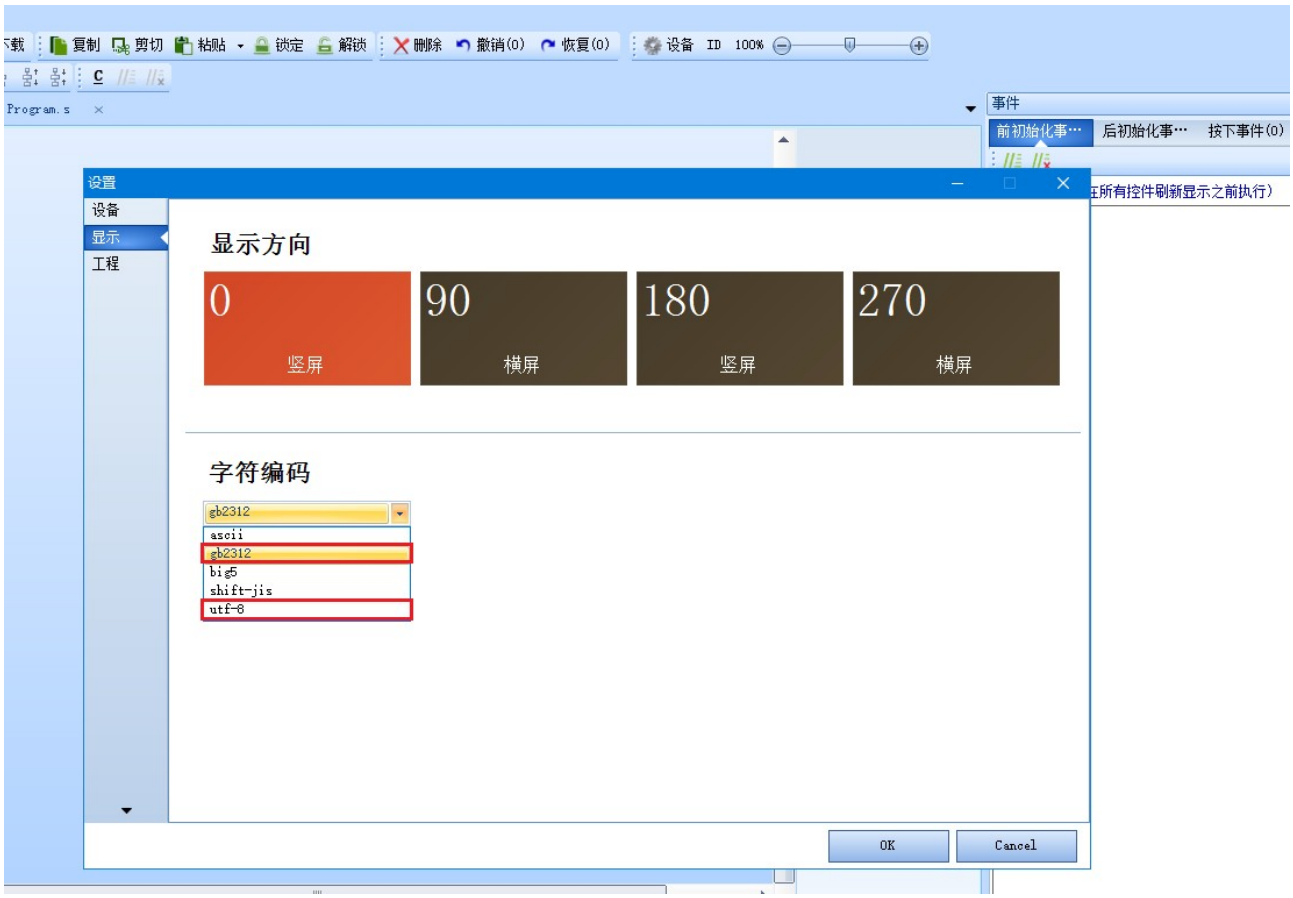
0x00 0x01 t0.txt=”addr is 256”0xFF 0xFF 0xFF 控制串口屏 A 页面上的 t0 控件显示内容为：addr is 256

0x00 0x02 t0.txt=”addr is 512”0xFF 0xFF 0xFF 控制串口屏 B 页面上的 t0 控件显示内容为：addr is 512

0xFF 0xFF t0.txt=”broadcast addr”0xFF 0xFF 0xFF 控制所有串口屏页面上的 t0 控件显示内容为：broadcast addr

11.3.18 注释乱码或者不显示

工程编码不是 GB2312 或者 UTF8, 仅这两个编码支持简体中文显示，如果使用了其他编码，输入简体中文可能会乱码



如果工程编码为 ascii，编码中只能有 ascii 编码中的字符，即英文字母大小写和常见的英文标点符号

如果工程编码为 big5，则只能显示繁体中文，所以注释也必须是繁体中文

如果工程编码为 shift_jis, 只能显示日文，所以注释也必须是日文

GB2312，UTF8，big5，shift_jis 均兼容 ascii 编码，即

工程编码和字库编码应一致，否则混用不同字库编码的控件之间相互赋值将会乱码

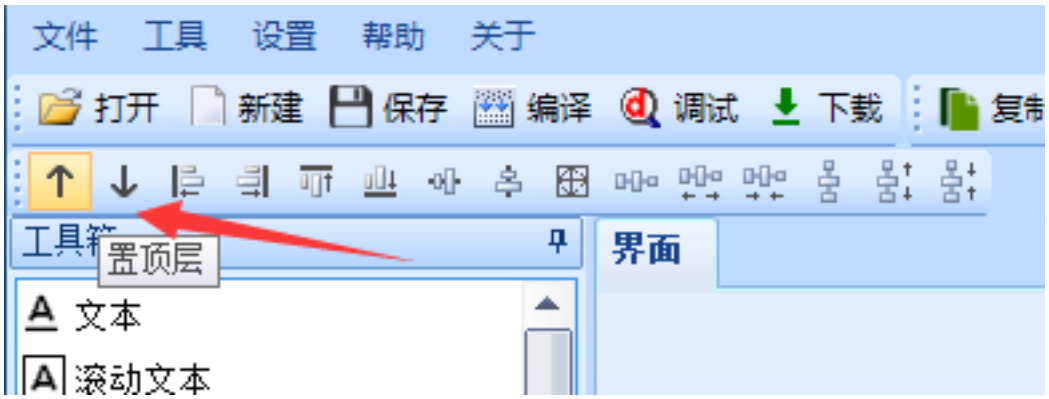
11.3.19 如何更改控件的前后图层关系

图层的前后关系跟控件 ID 是关联的，图层最底层的控件 ID 最小, 图层顶层的控件 ID 最大，每个控件占用一个图层，所有控件通过 ID 来区别前后关系。

对于 X 系列来说, 顶层的图层会挡住底层的图层。

每页 UI 最底层（空白处）的是“页面控件”所以页面控件 ID 永远是 0。

使用快捷栏的“置顶”、“置底”功能便可使控件 ID 发生变化 (图层关系也同时发生相应的变化)。如下图:



所有控件的 ID 号软件自动生成，用户在编辑 UI 界面时，按放置控件的顺序自动连续分配 ID；在属性项中不能手动修改。

控件 ID 及图层关系只在编辑界面可以修改，运行中无法修改。

11.3.20 如何设计多语言界面

界面上的控件使用 UTF-8 编码的, 然后在页面初始化里根据不同语言为控件的 txt 属性赋不同的值即可。

最好是在第一页里面选择语言，然后存到一个全局变量里，然后在每一页的初始化里刷就好了

串口屏:

新建一个变量 chooseLang，用于选择语言

在页面初始化中写:

```
if(chooseLang.val==1)
{
    t0.txt="中 文"
}else if(chooseLang.val==2)
{
    t0.txt="English"
}else if(chooseLang.val==3)
{
    t0.txt="🄵 🄵"
}else if(chooseLang.val==4)
{
    t0.txt="真 🄵 は い つ も ひ と つ"
}
```

也可以建立不同的页面来显示不同的语言

11.3.21 K 系列和 T 系列如何实现透明效果

方法一：使用整张的图片，然后将按钮控件的 sta 设置为切图

方法二：如果是单独切出小张的图片时，需要连同四个角落的颜色一起切出来

11.3.22 单片机如何接收和解析串口屏数据

1. 实现一个环形队列，此队列为串口的环形缓冲区，以下称为环形缓冲区
2. 在单片机的串口中断事件中，将串口接收到的数据存入环形缓冲区中（入队）
3. 在 main 函数的循环中，检测串口缓冲区的长度是否大于 1 帧的长度
4. 如果大于 1 帧的长度，判断帧头帧尾以及校验
5. 如果帧头帧尾符合，且校验通过，将相应的数据拷贝到数组或者结构体中并将相应的数据出队，执行第 7 步
6. 如果不符合，将队首出队，重新执行第 4 步
7. 对数据进行解析
8. 解析完毕，将已解析过的数据出队

参考以下工程

使用 stm32cubeide(HAL 库) 开发 (page 279)

11.3.23 如何对掉电存储空间进行初始化

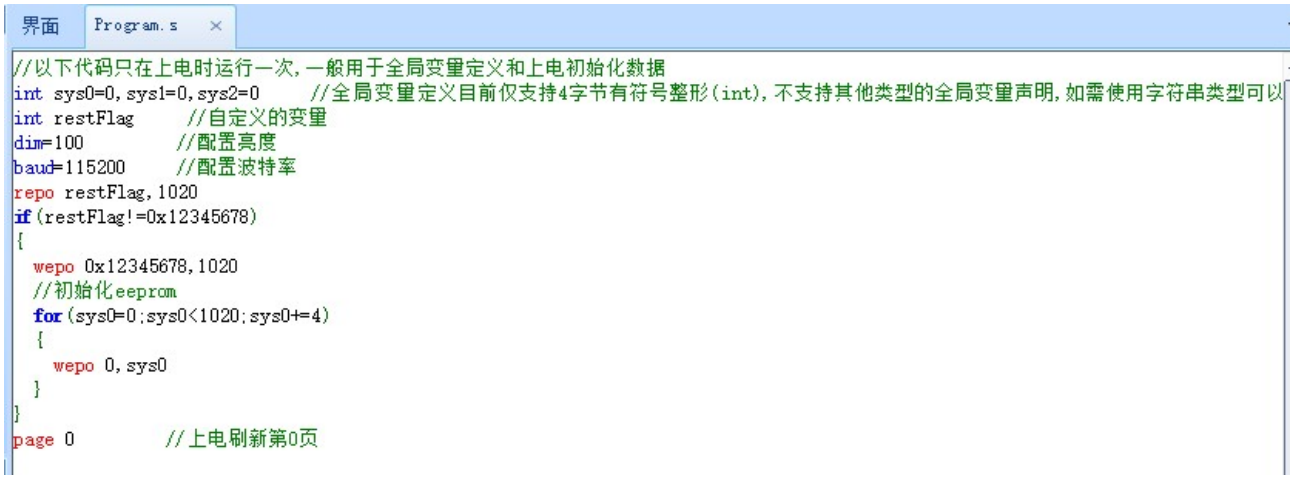
> **注意：** 使用掉电存储空间前，建议先对掉电存储空间进行初始化，因为掉电存储空间中的数据大概率是随机的。
>
> 在没有写入的情况下请不要读取掉电存储空间中的数据，因为这个数据不一定符合你程序中的条件，可能会出现莫名其妙的问题

掉电存储空间初始化原理

1. 先从 1020 的地址（或其他地址）读取一个数据出来
2. 判断这个这个数据是否符合预先设计的数据（比如 0x12345678，你也可以设置为其他数据，但不建议是 0x00 或者 0xffffffff），
3. 如果不符合，将 0-1019 的位置循环写入 0，并且在 1020 的位置写入预先设定的数据，防止下次上电时重新初始化

以下代码写在 program.s 中

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
int sys0=0,sys1=0,sys2=0      //全局变量定义目前仅支持4字节有符号整形(int),
↪不支持其他类型的全局变量声明,如需使用字符串类型可以在页面中使用变量控件来实现
int restFlag      //自定义的变量
dim=100          //配置亮度
bauds=115200      //配置波特率
//从掉电存储区最后一个位置读取数据出来
repo restFlag,1020
//如果不是指定的数据，说明掉电存储区未初始化
if(restFlag!=0x12345678)
{
    //用for循环初始化掉电存储空间
    for(sys0=0;sys0<1020;sys0+=4)
    {
        wepo 0,sys0
    }
    //写入掉电存储区的标记位置,防止下次开机时重复初始化掉电存储区
    wepo 0x12345678,1020
}
page 0          //上电刷新第0页
```



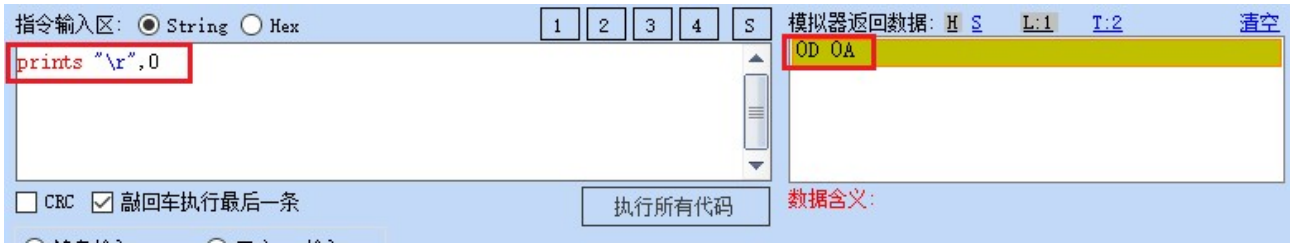
11.3.24 转义字符相关

11.3.24.1 回车换行

串口屏上的回车换行为 \r , 请不要使用\n 以及 \r\n

```
txt.txt="123\r456"
```

如果使用 prints 输出”\r”，实际上会输出 0x0d 0x0a



如果仅仅为了输出 0x0d 或者输出 0x0a，则使用 printh 进行输出

```
printh 0d        //输出0x0d

printh 0a        //输出0x0a

printh 0d 0a     //输出0d0a
```

11.3.24.2 引号

仅双引号需要进行转义，单引号不需要进行转义

双引号转义为 \”

注意: 不支持以下转义字符

```
//不支持以下转义字符

\? 在书写连续问号时，防止被解析成三字母词( ??) = ] )
\' 用于表示字符常量'
\\ 用于表示一个反斜杠
\a 警告字符，蜂鸣
\b 退格符
\f 进纸符
\n 换行符
\t 水平制表符
\v 垂直制表符
```

(续下页)

(接上页)

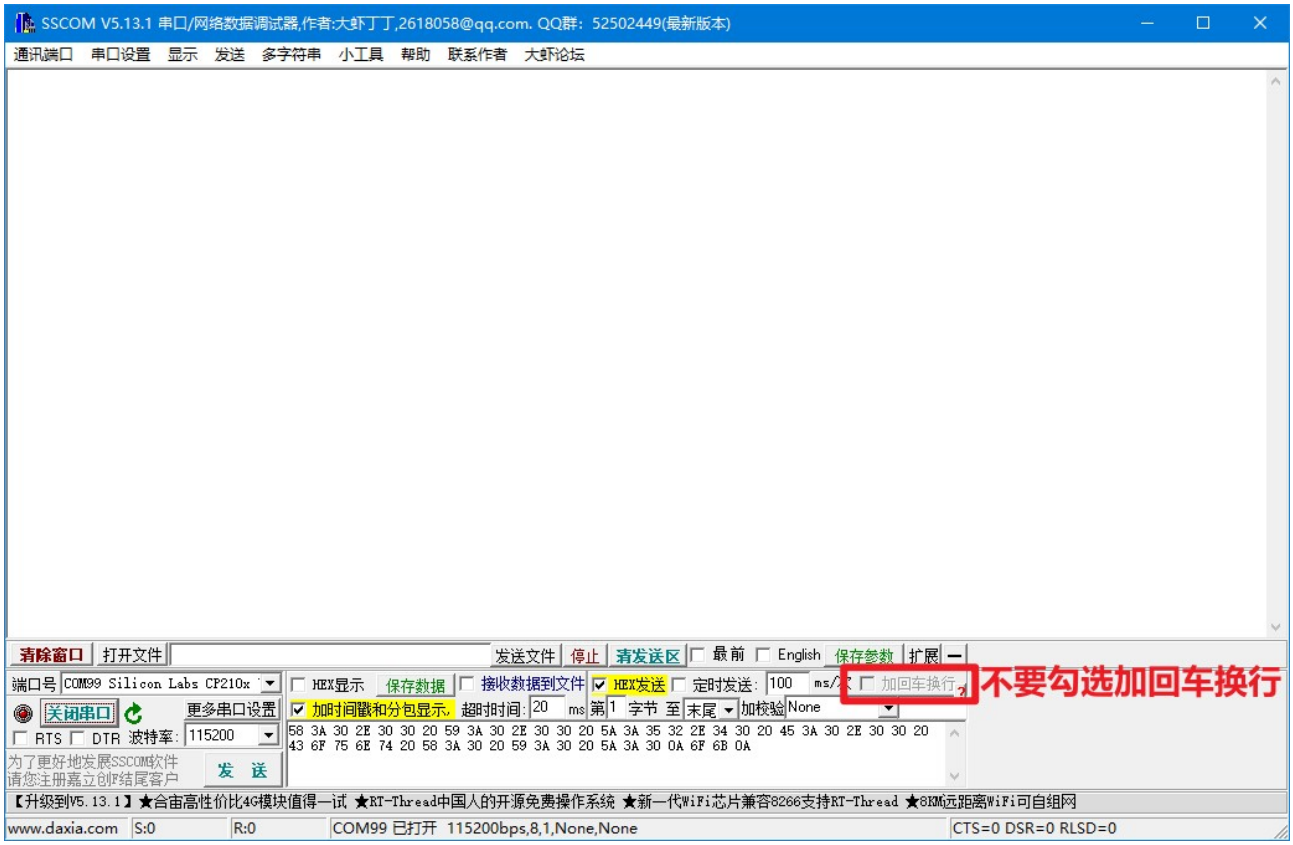
```
\ddd ddd表示3个八进制数字(0~7)
\xdd dd表示2个十六进制数字
```

相关链接

20 FF FF FF 转义字符使用错误 (page 892)

11.3.25 使用 sscom 时，sscom 可以接收到串口数据，但是 sscom 发送的数据屏幕接收不到

查看是否勾选了发送新行选项：不要勾选发送新行



11.3.26 如何实现按位取反

串口屏支持 ^（按位异或）运算，可以将数据与 0xff 进行按位异或运算得到按位取反结果

11.3.26.1 计算 1 字节按位取反

```
n0.val^=0xff
```

11.3.26.2 计算 2 字节按位取反

```
n0.val^=0xffff
```

11.3.26.3 计算 4 字节按位取反

```
n0.val^=0xffffffff
```

最大不能超过 4 字节

11.3.26.4 按位取反-相关链接

如何实现按位同或运算 (page 1325)

11.3.27 如何实现按位同或运算

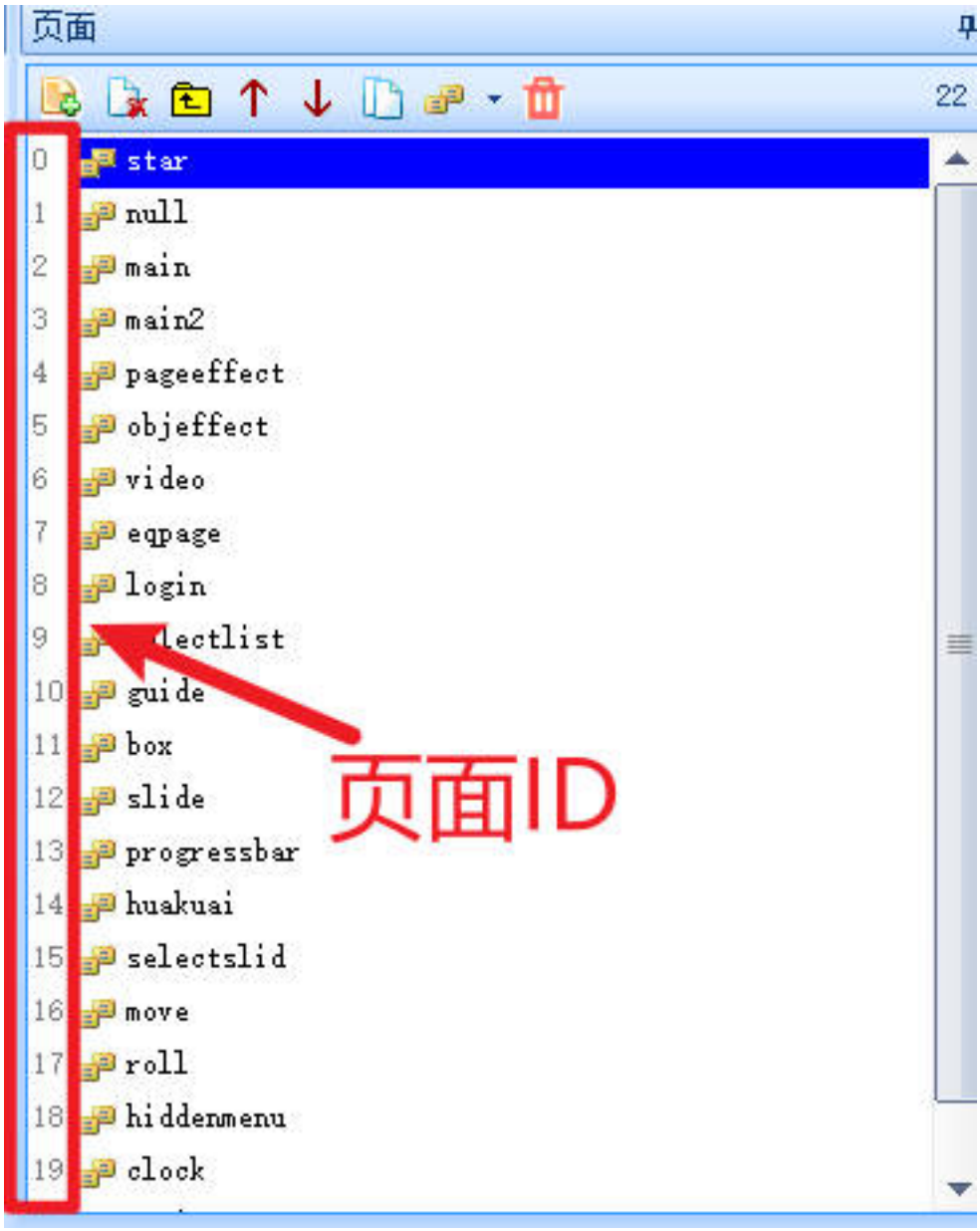
串口屏支持 ^（按位异或）运算，可以将数据进行按位异或运算再按位取反得到结果

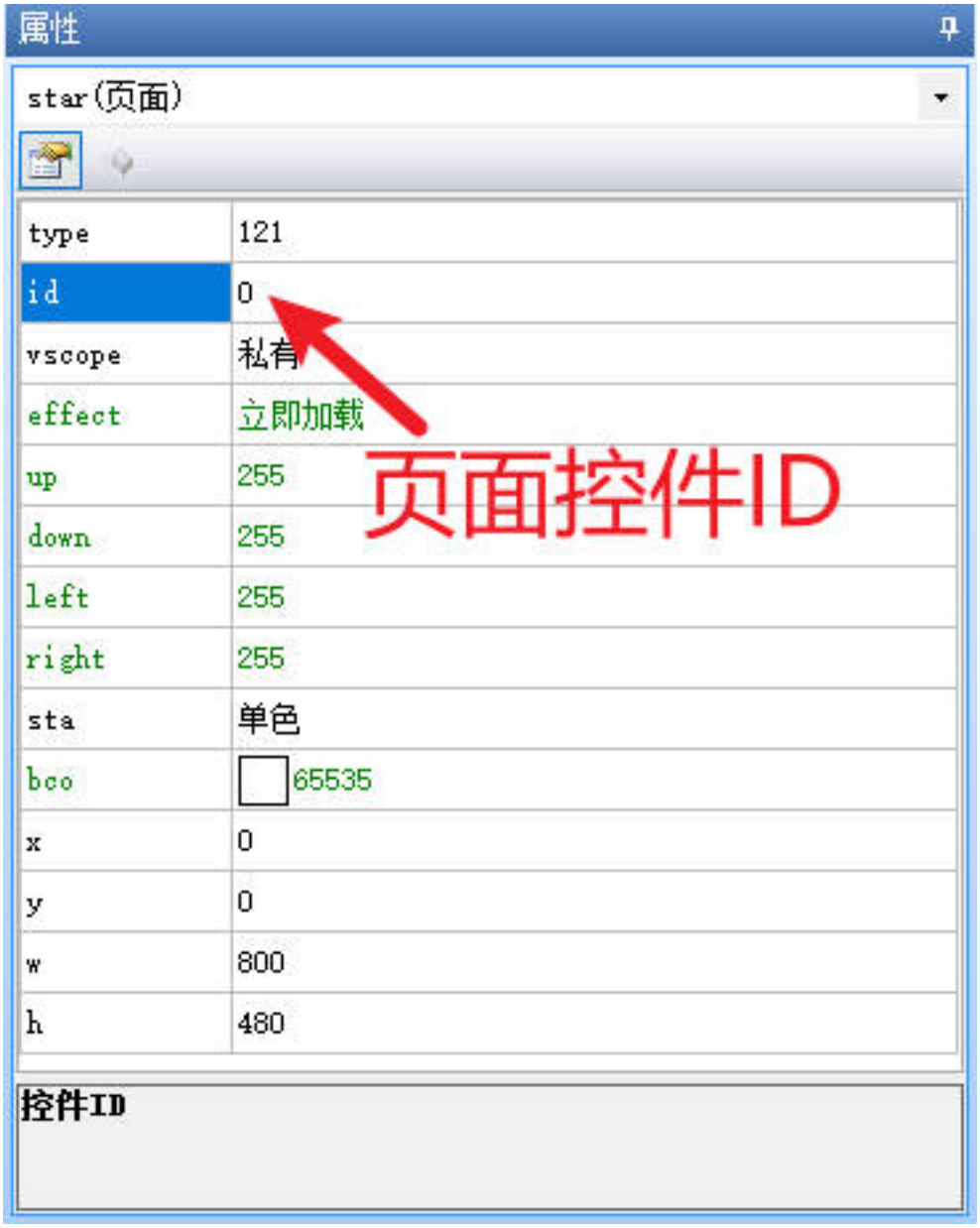
11.3.27.1 位运算-相关链接

如何实现按位取反 (page 1324)

11.3.28 页面和页面控件的区别

创建每个页面时系统会自动创建一个页面控件，页面控件只会在最底层，且 id 必定为 0





跳转页面时，使用的是页面 ID，如 main 页面的 ID 号是 2，可以使用 page main ，也可以使用 page 2 来跳转到 main 页面。

11.3.29 休眠替代方法

进入睡眠状态下，亮度为 0，可以执行如下指令：get,prints, printh。也可以执行 sleep=1，wup=X 的赋值语句，并且支持上位软件联机，其他指令不会执行。如果是带扩展 IO 的硬件,IO 配置为绑定控件事件时，睡眠模式下定时器不会运行。

如果需要能够在熄灭屏幕的状态下保证串口屏的定时器运行，请参考自定义屏幕保护样例工程/低功耗/睡眠休眠替代 (page 1130)

11.3.30 淘晶驰串口屏支持哪些通讯格式

目前淘晶驰串口屏仅支持 8N1 通讯格式，即 8 位数据位，无校验位，1 位停止位
这也是大多数串口设备默认的通讯格式
不支持 5/6/7 位数据位，不支持奇校验和偶校验，也不支持 1.5/2 位停止位

11.3.31 dim 和 dims 的区别

dim：设置当前亮度，当前亮度不会被保存，设置 dim 时 dims 不会改变。

dims：设置默认亮度，默认亮度会被保存 (写入 flash 前会判断和 flash 保存的是否一致，不一致才写入，会消耗 flash 寿命)，设置 dims 时 dim 会一起改变。

开机时如果没有亮度设置的代码，则默认使用的是 dims 的值

11.3.31.1 什么时候使用 dim，什么时候使用 dims

1. 需要频繁修改亮度的时候使用 dim, 比如通过修改亮度来实现呼吸灯效果的时候
2. 通过滑块修改亮度时，在滑动事件里使用 dim=h0.val, 在弹起事件里使用 dims=h0.val, 这是一个特殊的技巧
3. 避免通过 dims 将亮度设置为 0，这可能会导致以后的工程没有配置亮度信息时，直接显示黑屏
4. 其他情况下建议使用 dims

11.3.32 baud 和 bauds 的区别

baud：设置当前波特率，当前波特率不会被保存，设置 baud 时 bauds 不会改变

bauds：设置默认波特率，默认波特率会被保存，设置 bauds 时 baud 会一起改变，每次修改为不同的值会消耗 flash 寿命

通常只在 program.s 中配置一次波特率，请参考如何配置亮度，主动解析，波特率 (page 1306)

当配置在 program.s 中时，两者并没有太大的区别，只有当需要通过单片机对串口屏工程进行升级时，请务必使用 bauds 来配置波特率。

因为通过串口升级时因为某些原因 (例如网络, 供电, 干扰) 导致升级失败时, 是无法进入之前的工程，也就无法初始化串口波特率，导致单片机与串口屏无法通讯。

举例：

出厂时，默认 bauds 为 9600，如果在工程中没有任何配置波特率的代码，那么波特率就是 9600

如果单片机波特率是 115200，在 program.s 中也配置了 baud=115200，如果能正常进入工程，那么进入工程后，波特率变为 115200，单片机能与串口屏正常通讯。

如果单片机给串口屏通过串口升级的过程中升级失败了，此时无法进入工程，也就无法正确初始化波特率为 115200，变成了默认的 9600，照成单片机无法与串口屏正常通讯。

因此，当需要通过单片机对串口屏工程进行升级时，请务必使用 bauds 来配置波特率。

11.3.33 能不能用 printf 一次性发送多条指令

不建议

比如以下代码

```
printf("n0.val=666\xff\xff\xffb0.val=1\xff\xff\xff"); //
↪b0会被识别为HEX，导致串口屏无法正确识别
```

其中 b0.val 中的 b0 会被 printf 认为是 16 进制而不是字符串，导致串口屏无法收到正确的指令

```
printf("n0.val=666\xff\xff\xffz0.val=1\xff\xff\xff"); //
↪z0会被识别为字符串，串口屏正确识别
```

11.3.34 如何显示和计算百分比

11.3.34.1 屏幕单纯显示百分比，由单片机计算好后发给屏幕显示

使用文本控件来显示百分比

1、在上位机工程新建一个文本控件，假设为 t0，将程序下载到串口屏上，

2、串口屏串口与单片机串口连接，两者波特率应一致，单片机 RX 接串口屏 TX，单片机 TX 接串口屏 RX。

3、发送指令：单片机串口通过字符串模式发送 t0.txt=”99%”

4、发送结束符：单片机通过 HEX 模式发送 0xff 0xff 0xff

5、此时屏幕上的 t0 控件内的文字变为“99%”

使用数值控件配合文本控件来显示百分比，文本控件仅显示百分号，或者也可以将百分号做在背景图片上

1、在上位机工程新建一个数字控件，假设为 n0，将程序下载到串口屏上，

2、串口屏串口与单片机串口连接，两者波特率应一致，单片机 RX 接串口屏 TX，单片机 TX 接串口屏 RX。

3、发送指令：单片机串口通过字符串模式发送 n0.val=99

4、发送结束符：单片机通过 HEX 模式发送 0xff 0xff 0xff

5、此时屏幕上的 n0 控件内的文字变为“99”，配合文本控件可以将其显示为“99%”

11.3.34.2 屏幕计算百分比

例如总数 88，其中 87 个为合格品，合格总数为 87，则百分比为 87/88=98%。

由于串口屏不支持浮点数，因此直接用除法计算，会变成取整计算，因此需要将结果乘以 100 再除以总数，得到百分比。

1、在上位机工程新建一个数字控件，假设为 n0，则 n0.val=87*100/88=98, 配合一个文本控件来显示百分号，就能显示完整的百分比。

```
n0.val=87*100/88      //n0.val=98
```

11.3.35 内存文件存储区相关

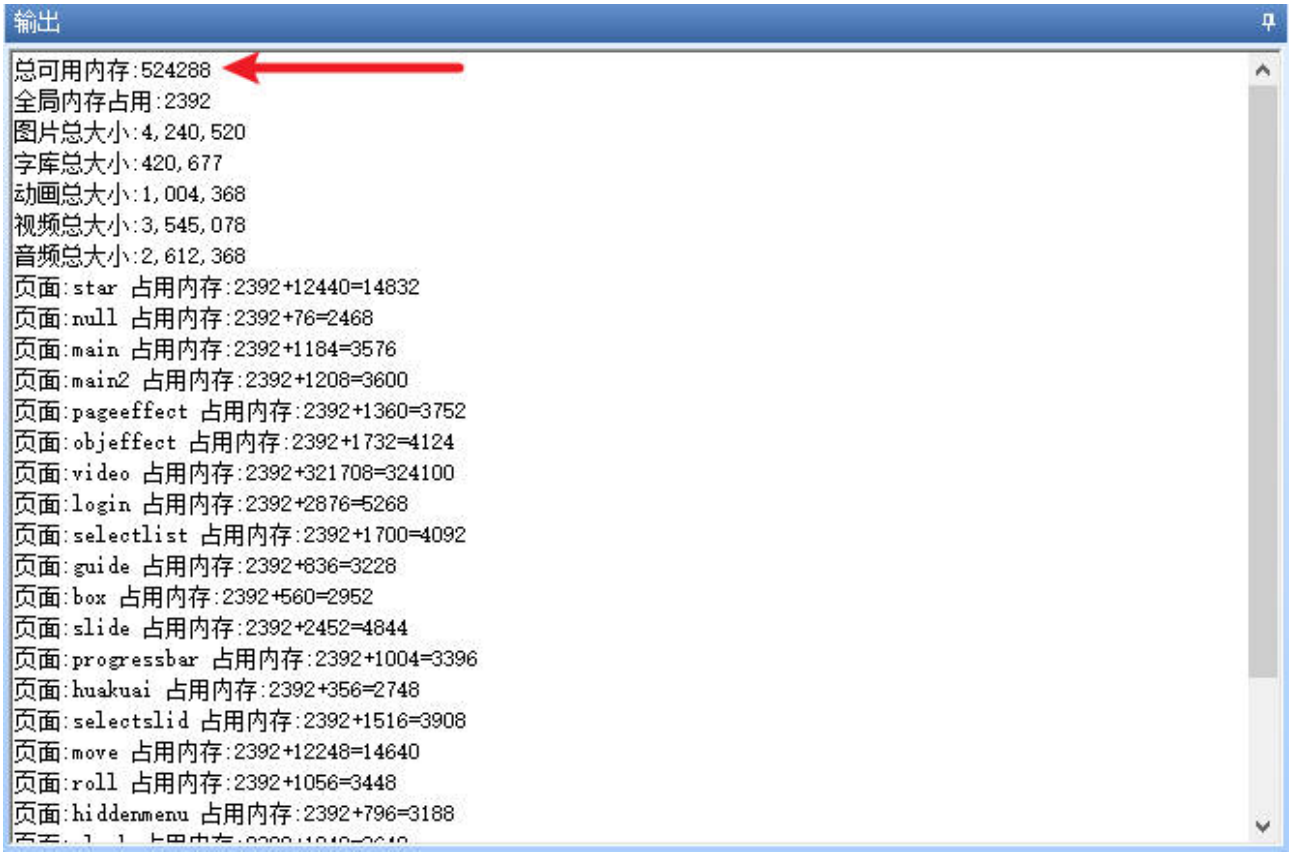
> **注意：** 内存文件存储区仅 X2、X3、X5、X7 系列产品支持，其他系列产品不支持。
>
> 内存文件存储区中不允许创建文件夹
>
> 内存文件存储区中的数据在断电后会丢失，请谨慎使用。
>
> 使用内存文件系统会占用设备的内存资源，可能会导致某些控件无法使用，请谨慎使用。

11.3.35.1 什么是内存文件存储区

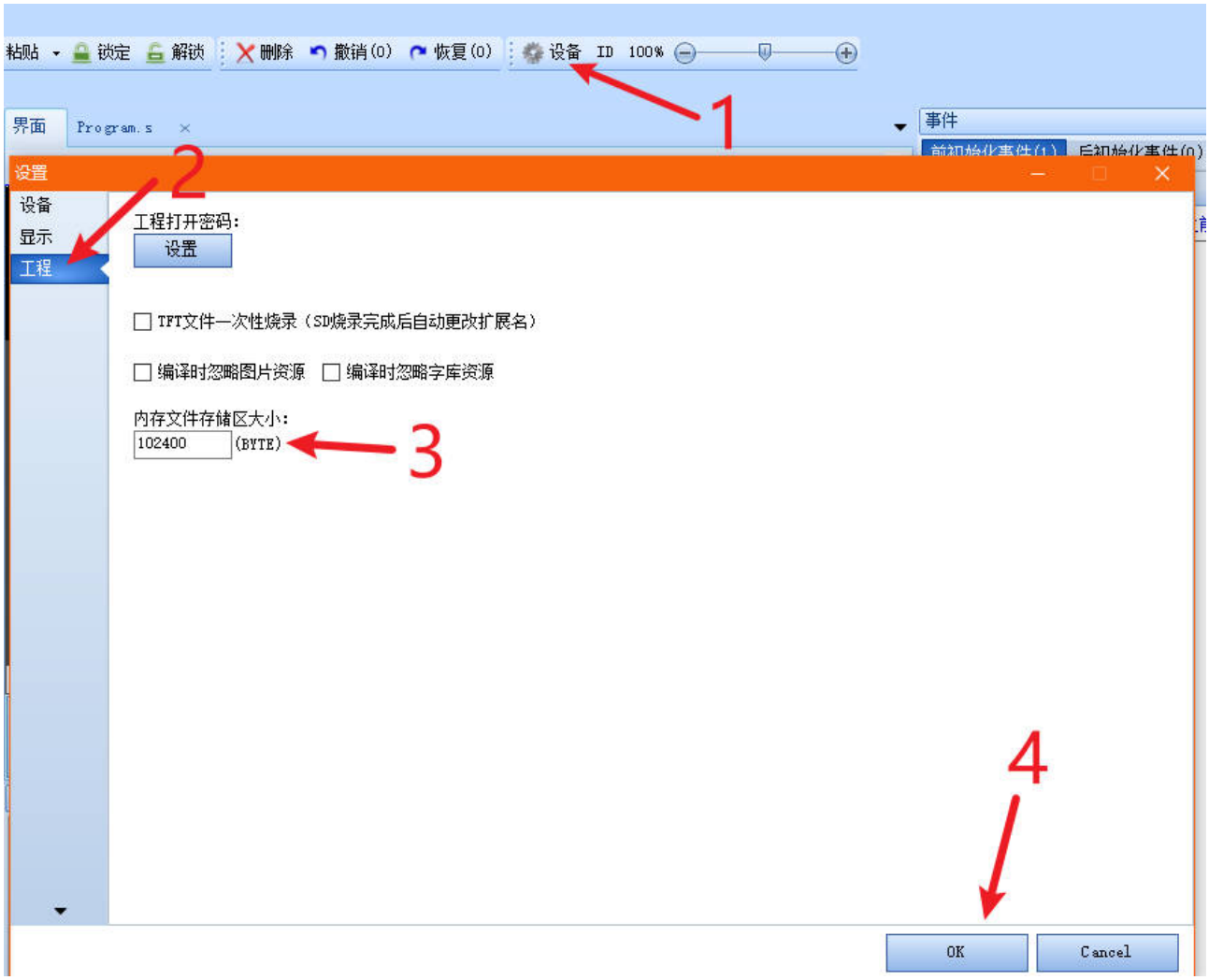
对于 X2、X3、X5、X7 系列产品，内存文件存储区是指在设备的内存中创建的一个虚拟文件系统。

它允许用户在设备的内存中存储和访问文件，而不需要使用物理存储介质（如 Micro SD 卡），在某些情况下可以节省一张 Micro SD 卡的成本。

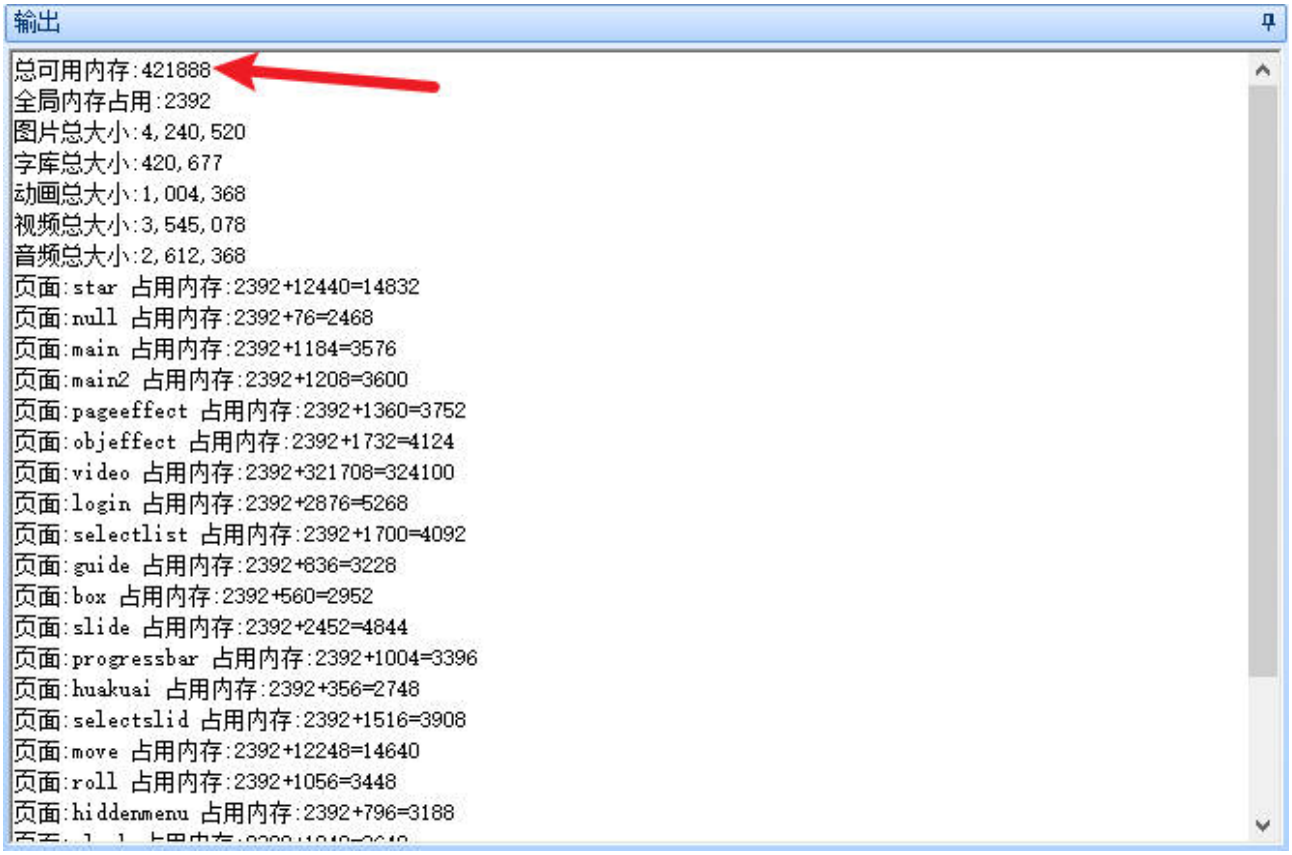
以 TJC8048X550 为例，在默认情况下，内存的大小为 524288 字节（512KB）



进入设备-工程，设置内存文件存储区大小为 102400 字节（100KB），并点击 OK，此时就划分了 100KB 的内存空间供用户使用。



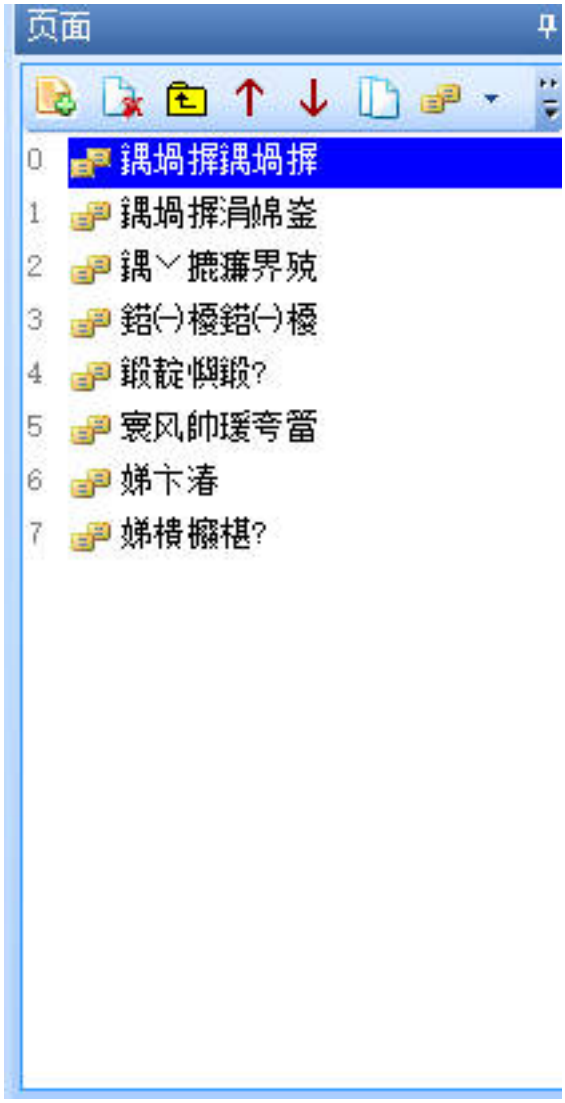
此时重新编译，则总可用内存对应减少了 102400 字节（100KB），即总可用内存的空间为 421888 字节（412KB）。



当内存文件存储区设置过大，导致控件能使用的内存空间不足时，编译时会报错

11.3.35.2 如何使用内存文件存储区

11.3.36 页面名称乱码



1、在 UTF8 模式下使用中文来命名页面名称会乱码

解决方法：不要使用中文来命名页面名称，使用英文字母来命名页面名称。

2、修改了工程的编码后，页面名称会乱码

解决方法：不要使用中文来命名页面名称，使用英文字母来命名页面名称

11.3.37 为什么 wepo 写入后不允许立刻读取或者断电

NAND Flash 的写入有限制，不能像 RAM 或 NOR Flash 那样随便按字节覆盖。

单字节写入的实现，并不是直接在原地改那一个字节，而是 **用读–改–写（Read-Modify-Write）**的方式完成的。

NAND Flash 的物理限制：

1、NAND Flash 的最小擦除单位是 Block（通常是 64KB、128KB…）。

2、最小写入单位是 Page（通常是 2KB、4KB…）。

3、写入时，只能把 1 变成 0，不能把 0 变回 1。

所以如果目标字节中任何一位需要从 $0 \rightarrow 1$，就必须先擦除整个块。

在串口屏应用中，客户可能会同时写入多条 wepo 指令，底层代码会将多条 wepo 指令的修改合并在一起写入到 flash 中，减少写入的次数，从而提高 flash 寿命

11.3.38 常用英语

英文		翻译
n	number	数字
t	text	文本
p	picture	图片
pic	picture	图片
v	video	视频
vid	video	视频
type		类型
id		编号
objname	object name	控件名称
vscope	variable scope	控件范围
drag		拖动
aph	alpha	透明度
effect		效果
sta	status	状态
style	style	风格
key	keyboard	键盘
font		字体
bco	background color	背景色
pco	primary color	字体色/前景色
xcen	X-center	x 轴方向居中方式
ycen	Y-center	y 轴方向居中方式
pw	password	密码
txt	text	文本
txt_maxl	txt max length	字符最大长度
isbr	is break row	是否强制换行
spax	Spacing-x	x 轴方向字符间距
spay	Spacing-y	y 轴方向字符间距
x		x 轴坐标
y		y 轴坐标
w	width	宽度
h	height	高度
dir	direction	方向
dis	distance	距离
tim	time	时间
en	enable	使能
val	value	值
lenth	length	长度
format		格式
dez		

11.3.39 页面跳转后, 控件怎么保持原来的状态

相似问题:

页面跳转后, 状态丢失怎么办

调用键盘后, 其他控件都变回默认怎么办

原因:

页面跳转后, 控件的状态会丢失, 包括控件的默认值、选中状态、输入内容等

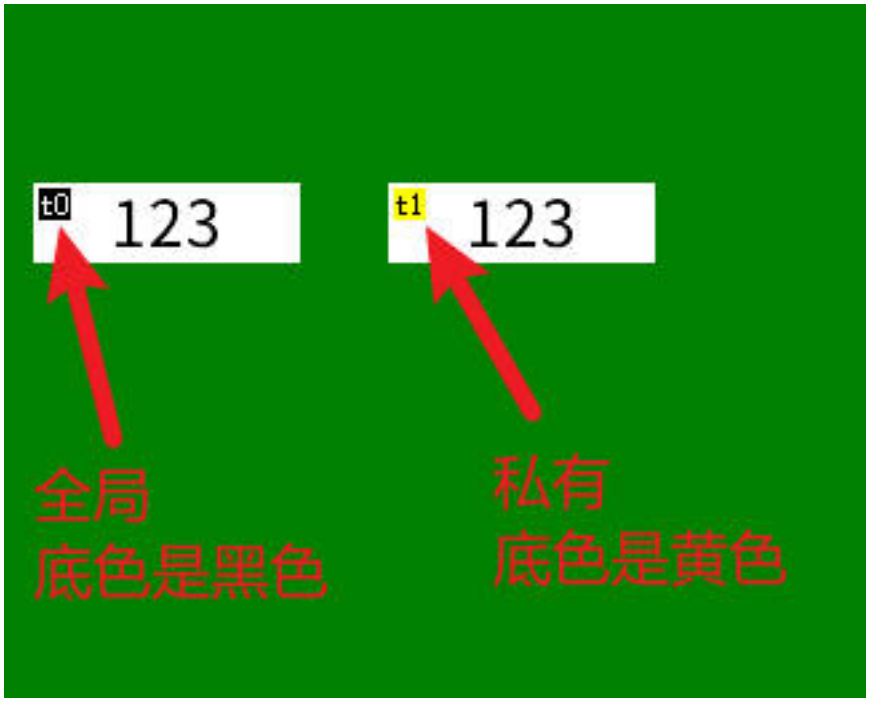
这是因为跳转页面或调用键盘 (调用键盘就是跳转到键盘页面, 本质上也是跳转页面) 后, 跳转到新的页面时, 原来页面会被销毁, 内存回收, 因此原来页面上的控件的状态也就丢失了。

解决方案:

将不想变的控件的 sta 属性设置为全局, 全局控件在切换页面时不会被回收, 可以保持原来的状态。

将对应的控件设置为全局

修改了 vscope 属性后，控件的名称会由黄色变为黑色



其他相关的全局控件操作请参考跨页面赋值，全局变量操作 (page 847)

11.3.40 串口屏重启后, 控件怎么保持原来的状态

相似问题:

串口屏重启后, 状态丢失怎么办

原因:

串口屏断电后, 运行内存 (ram) 中的所有数据都会丢失, 包括程序运行的状态信息, 因此, 当串口屏重启后, 控件的状态也会丢失.

解决方法一, 存储在串口屏中:

如果控件的状态需要保存, 可以将控件的状态信息保存到串口屏的非易失性存储器中, 比如用户存储区或 SD 卡中, 然后在串口屏重启后, 从串口屏的非易失性存储器中读取控件的状态信息, 并恢复到控件上.

需要注意的是非易失性存储器基本上都是有写入寿命的, 如果经常写入, 会将非易失性存储器的寿命耗尽, 会导致串口屏工作异常

解决方法二, 存储在单片机中:

通过串口将数据传输给单片机, 写入单片机的非易失性存储器中

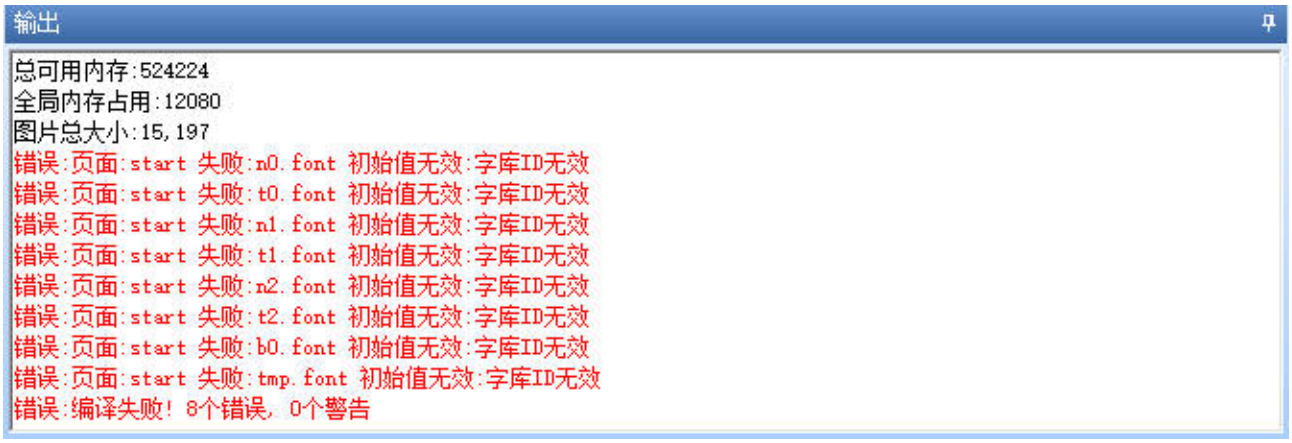
读写用户存储区 (page 1044)

SD 卡读写文件流程 (page 1024)

11.4 错误提示

11.4.1 编译报错：XXX 初始值无效

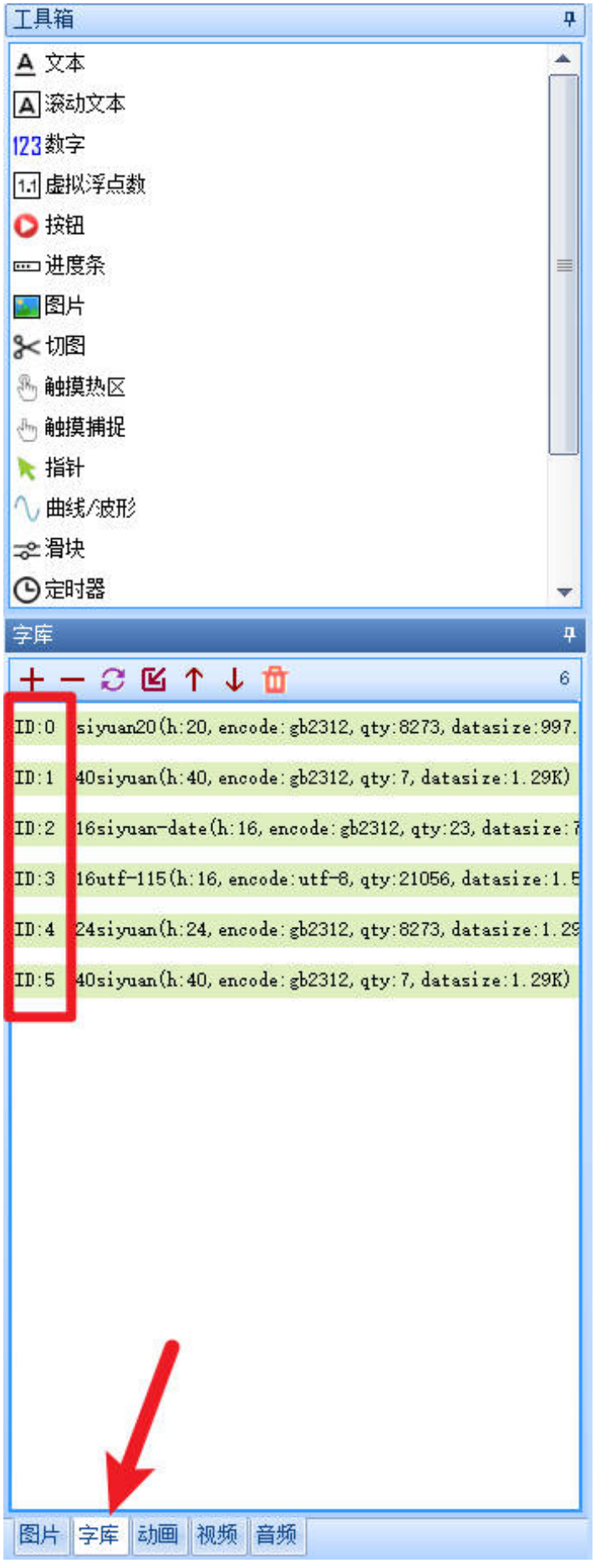
11.4.1.1 字库 ID 无效



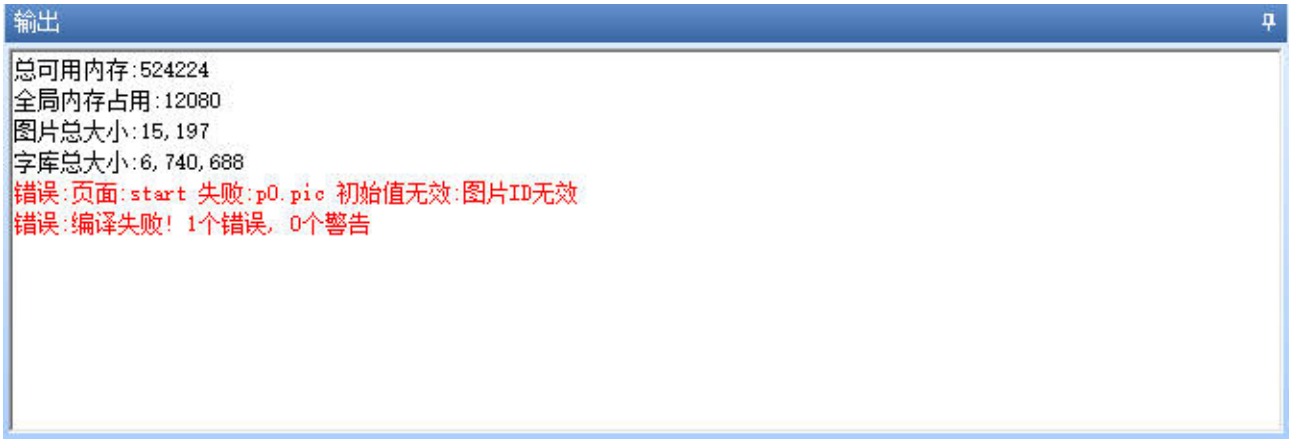
出现这类错误是因为：控件使用了字库。但是字库资源中不存在这个字库。解决办法是添加一个字库文件到字库资源中，并修改控件的字库属性对应到指定的字库 ID。

如何制作字库和添加字库，请参阅创建字库和导入字库 (page 125)

配置字库时, 应保证 font 属性设置的值所对应的字库是存在的


工具箱
文本
滚动文本
数字
虚拟浮点数
按钮
进度条
图片
切图
触摸热区
触摸捕捉
指针
曲线/波形
滑块
定时器
字库
6
ID:0 siyuan20(h:20, encode:gb2312, qty:8273, datasize:997.
ID:1 40siyuan(h:40, encode:gb2312, qty:7, datasize:1.29K)
ID:2 16siyuan-date(h:16, encode:gb2312, qty:23, datasize:7
ID:3 16utf-115(h:16, encode:utf-8, qty:21056, datasize:1.5
ID:4 24siyuan(h:24, encode:gb2312, qty:8273, datasize:1.29
ID:5 40siyuan(h:40, encode:gb2312, qty:7, datasize:1.29K)
图片
字库
动画
视频
音频

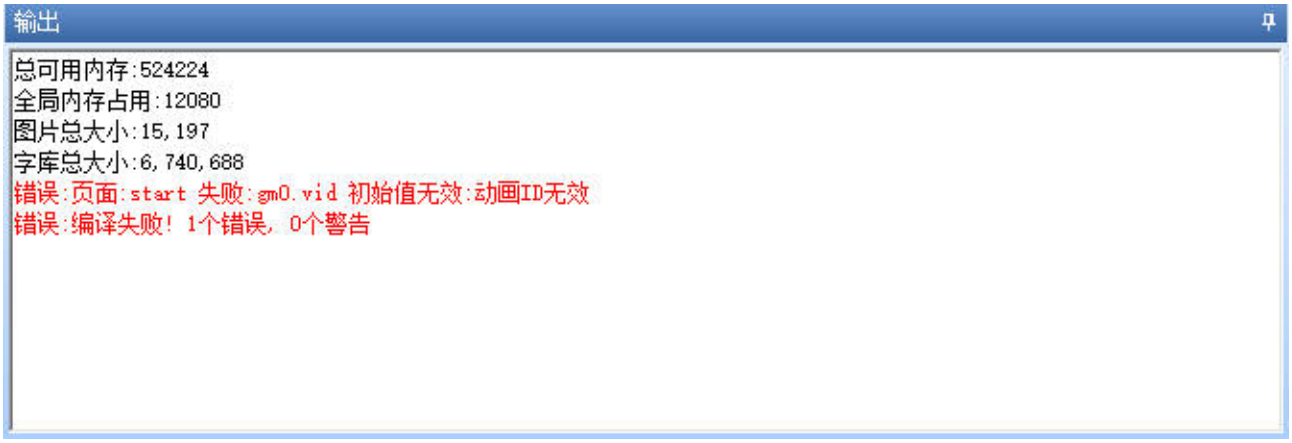
11.4.1.2 图片 ID 无效



出现这类错误是因为：选择了页面/控件的 sta 为图片，但是图片资源中又不存在这个 ID 的图片，或者你没有配置控件对应的属性。解决办法是添加对应的图片文件到图片资源中，并修改控件的图片属性对应到指定的图片 ID。

图片
1
2
Taojingchi淘晶驰
ID:0 (218X50 Q:96 14.84K) png-alpha
1
图片
字库
动画
视频
音频

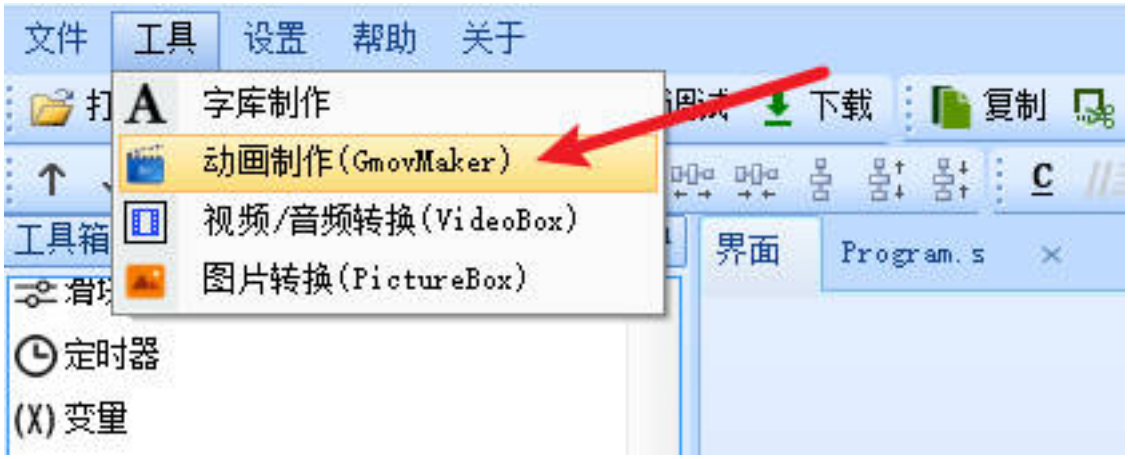
11.4.1.3 动画 ID 无效



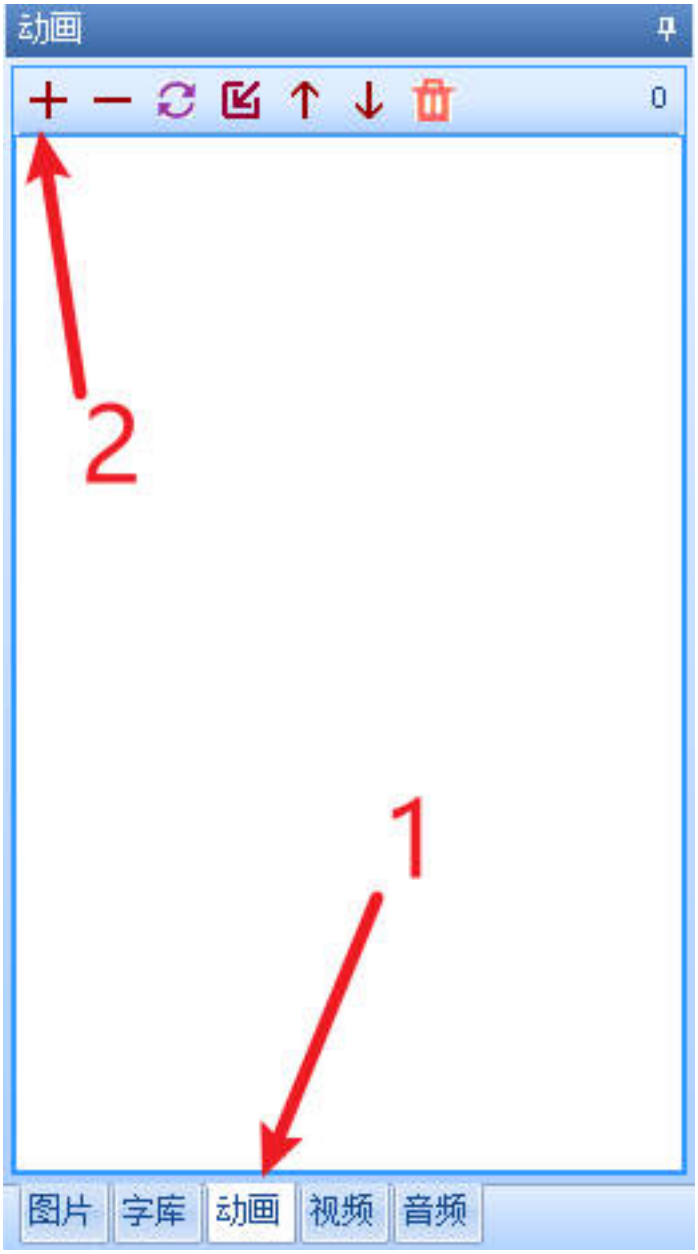
动画控件仅 x 系列支持

出现这类错误是因为：添加了动画控件，但是动画资源中又不存在这个动画，或者你没有配置动画控件的 vid 属性。解决办法是添加对应的动画文件到动画资源中，并修改动画控件的 vid 属性对应到指定的动画 ID。

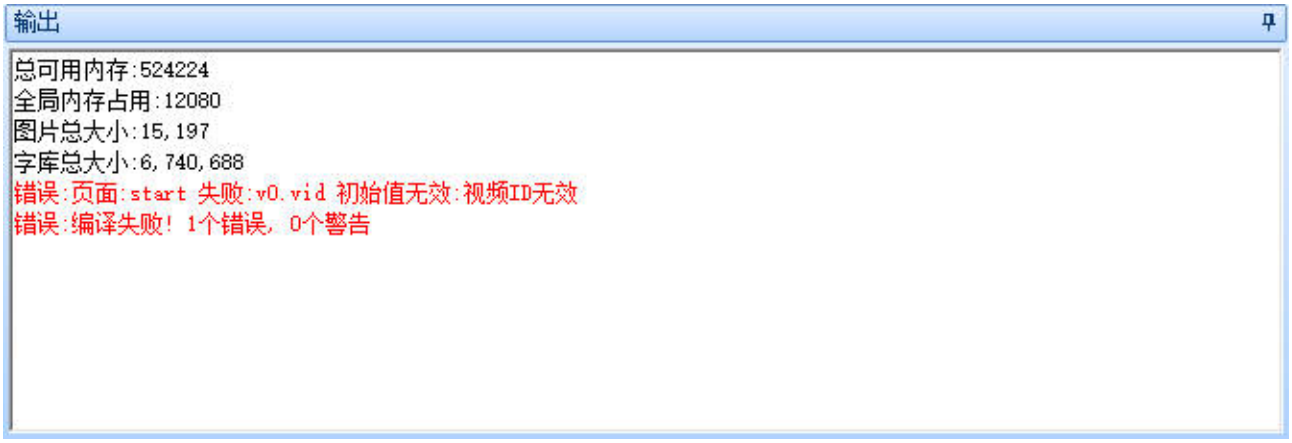
动画的转换方法



添加动画（可以添加转换后的 gmov 文件，也可以添加 gif 文件，会自动转换成动画）


动画
0
2
1
图片
字库
动画
视频
音频

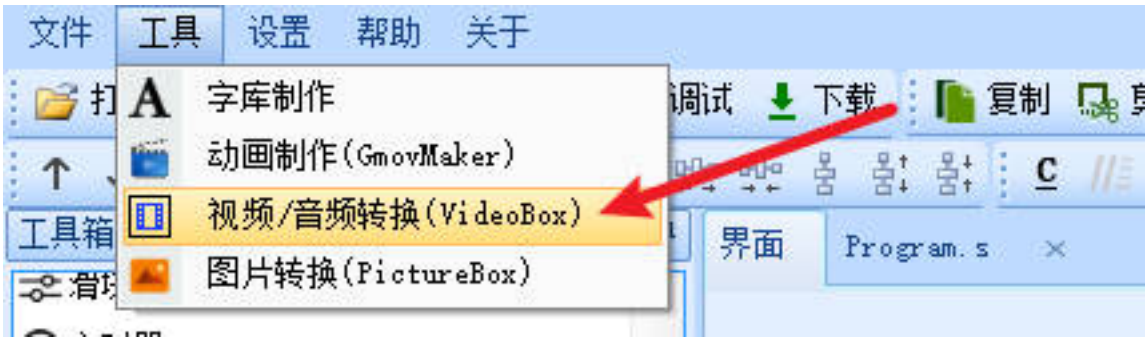
11.4.1.4 音频 ID 无效



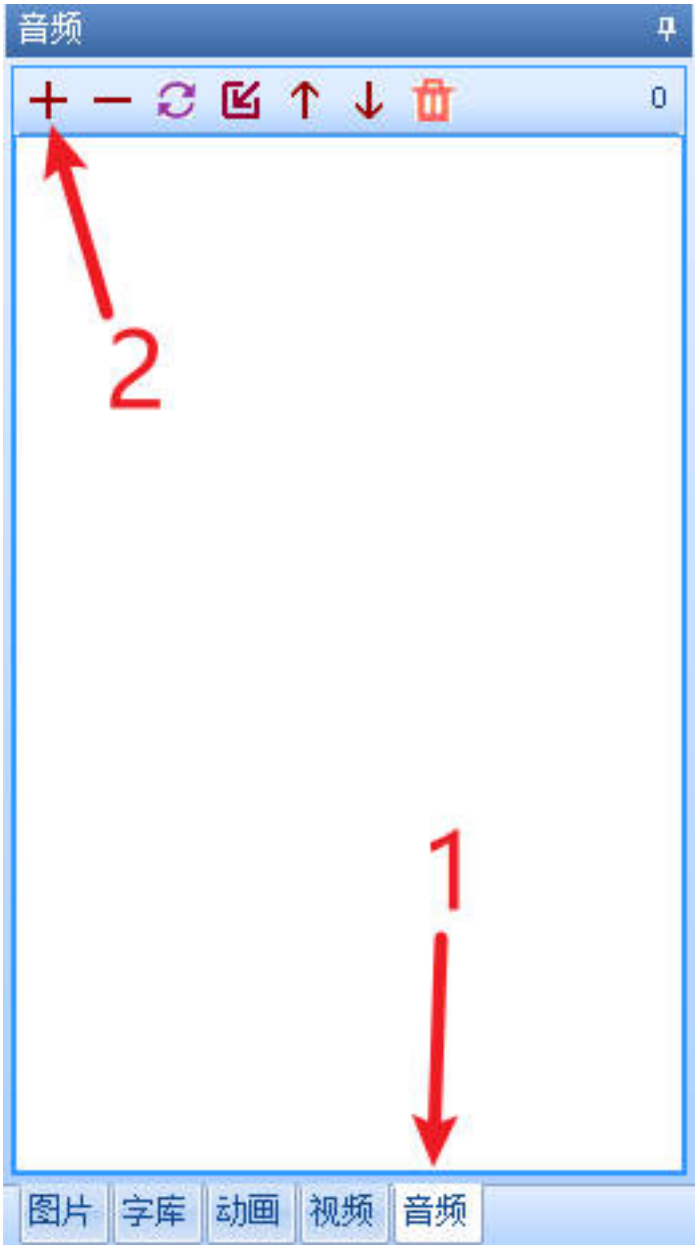
音频控件仅 x3、x5 系列支持

出现这类错误是因为：添加了音频控件，但是音频资源中又不存在这个音频，或者你没有配置音频控件的 vid 属性。解决办法是添加对应的音频文件到音频资源中，并修改音频控件的 vid 属性对应到指定的音频 ID。

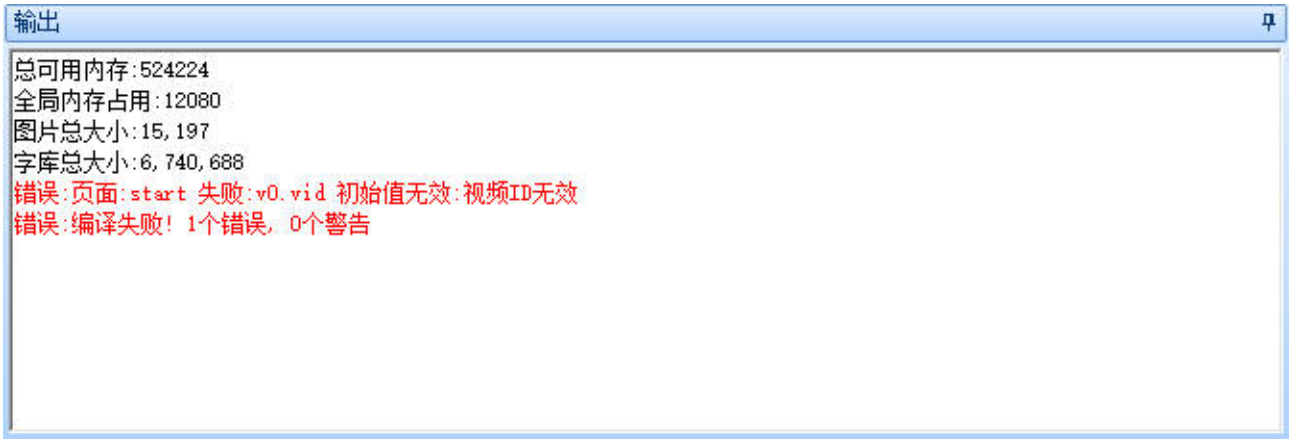
音频的转换方法



音频的添加方法


音频
0
2
1
图片
字库
动画
视频
音频

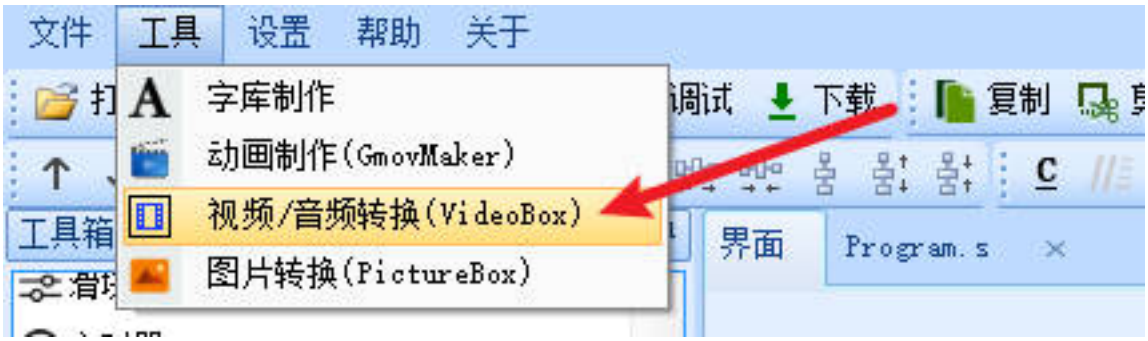
11.4.1.5 视频 ID 无效



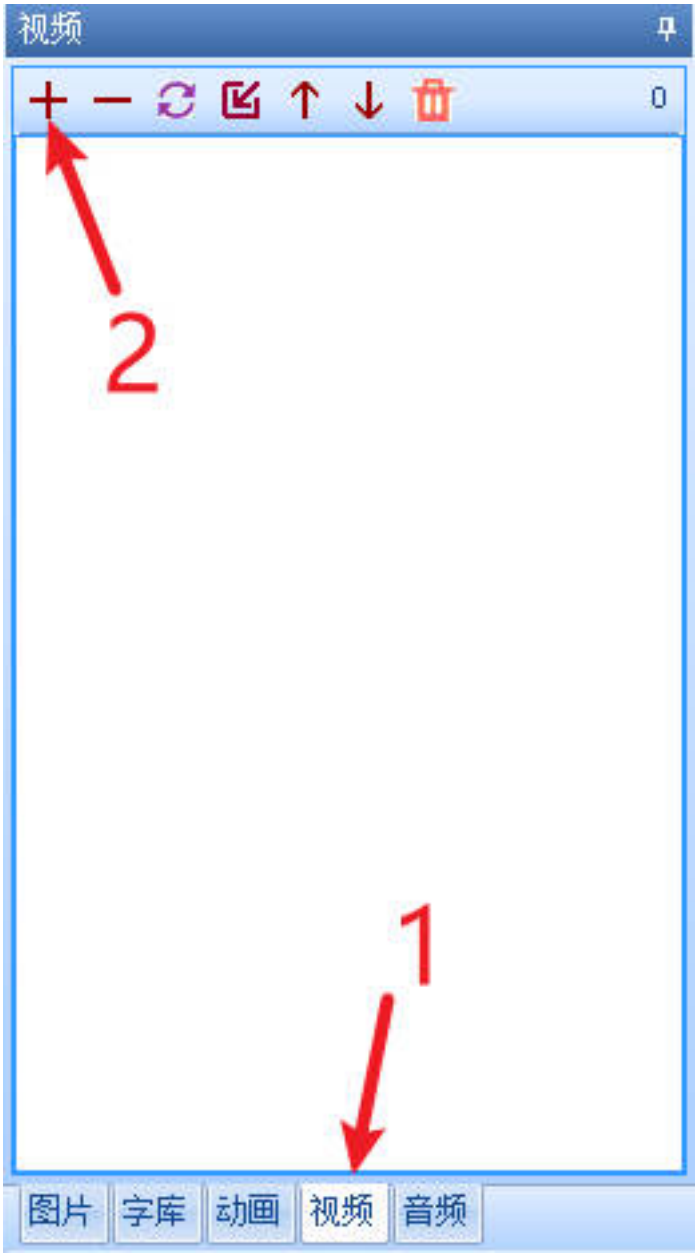
视频控件仅 x5 系列支持

出现这类错误是因为：添加了视频控件，但是视频资源中又不存在这个视频，或者你没有配置视频控件的 vid 属性。解决办法是添加对应的视频文件到视频资源中，并修改视频控件的 vid 属性对应到指定的视频 ID。

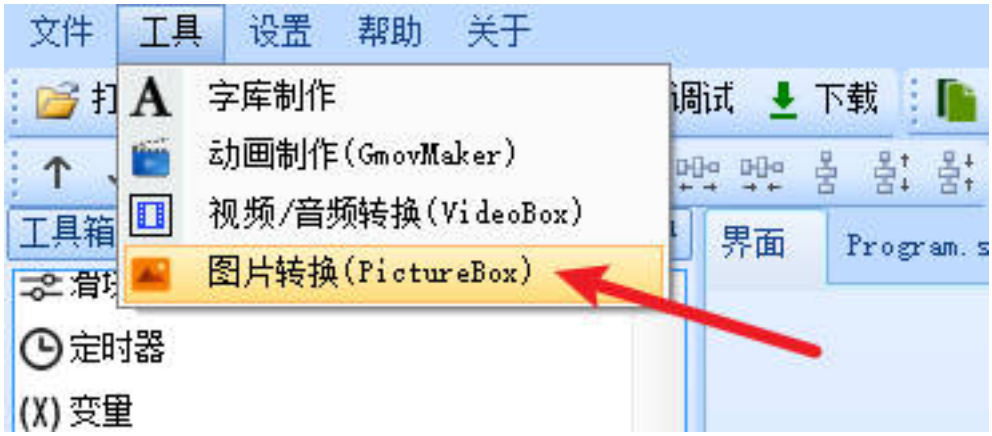
视频的转换方法



视频的添加方法


视频
0
2
1
图片
字库
动画
视频
音频

11.4.1.6 图片转换工具



图片转换工具是给外部图片控件使用的，这个功能仅 x 系列支持

图片转换工具转化出来的是 xi 格式，支持透明图层

外部图片控件支持 jpg 和 xi 格式，普通图片可以直接使用 jpg 格式，如果有透明图层，需要使用图片转换工具将 png 转换为 xi 格式

11.4.1.6.1 初始值无效-相关链接

创建字库和导入字库 (page 125)

指定字库 (page 1284)

在做字库的时候有部分字体无法选择 (page 1360)

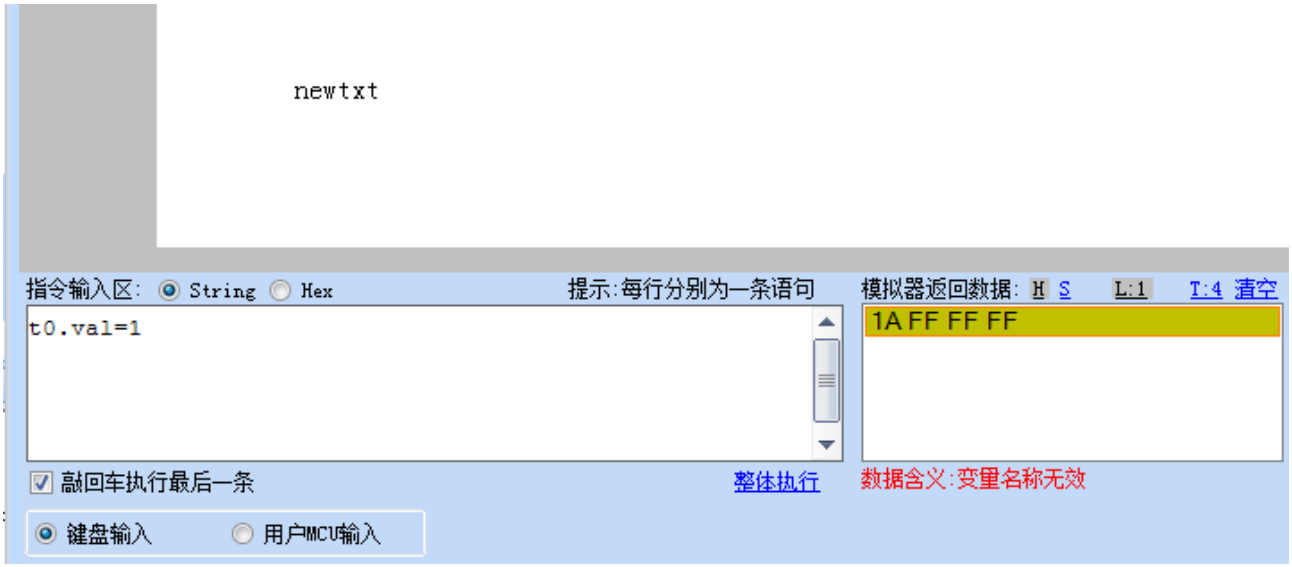
编译报错：*XXX* 初始值无效 (page 1342)

utf8 字库下如何只选择汉字 (page 1403)

11.4.2 单片机发指令给屏幕，屏幕返回 1A FF FF FF 或 1C FF FF FF 四个字节的 HEX 数据

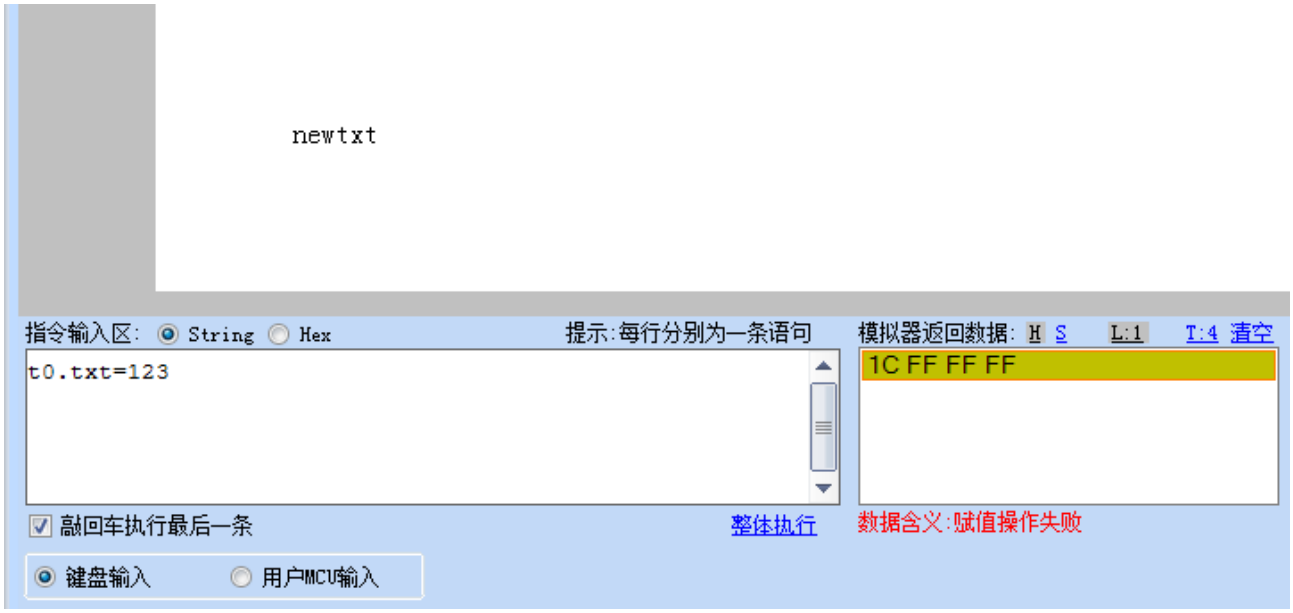
1A FF FF FF

如下图：模拟器已经帮我们翻译了，变量名称无效，因为 t0 控件并没有 val 属性，t0.val 这是个无效的变量



1C FF FF FF

如下图：模拟器已经帮我们翻译了，赋值操作失败，因为 t0.txt 是字符串类型。赋值常亮必须加引号，所以赋值失败



更多错误返回信息请参考串口屏返回数据格式 (page 874)

11.4.3 串口连接不上屏幕

屏幕是否有亮?（电压是否足够？请务必接 5V 而不是 3.3v，电流是否足够？正负极有没有插反?）
用的什么 usb 转 ttl 芯片, 如果是 ch340 请更换为 ft232 或者 cp2102。
(为什么要换 ch340？因为 ch340 有很多假货)
是否使用了 usbhub（集线器或拓展坞），如果有请去除然后直接插电脑 usb 口试试
TX、RX 是否连接正确——5v 接 5v，gnd 接 gnd，tx 接 rx，rx 接 tx
串口上是否并联其他设备
串口屏和电脑或者单片机是否共地
检查串口驱动是否正确安装
串口号是否选对/手动选择串口
尝试拔插串口工具
尝试重启电脑
尝试更换一个串口工具试试
使用串口软件调试时，不要加回车换行
尝试更换屏幕
波特率是否匹配
检测连接方式是否正确，是用 TTL 电平还是用 232 电平
检查电源是否稳定，电压有没有被拉低，电源不稳定可能会造成任何异常

11.4.4 串口屏黑屏原因

导线是否太长, 电压和电流是否足够-用万用表测量下串口屏供电端口电压是否正常

是否设置了 sleep（睡眠模式）

是否设置了 dims=0 或者 dim=0（亮度被设置为 0）

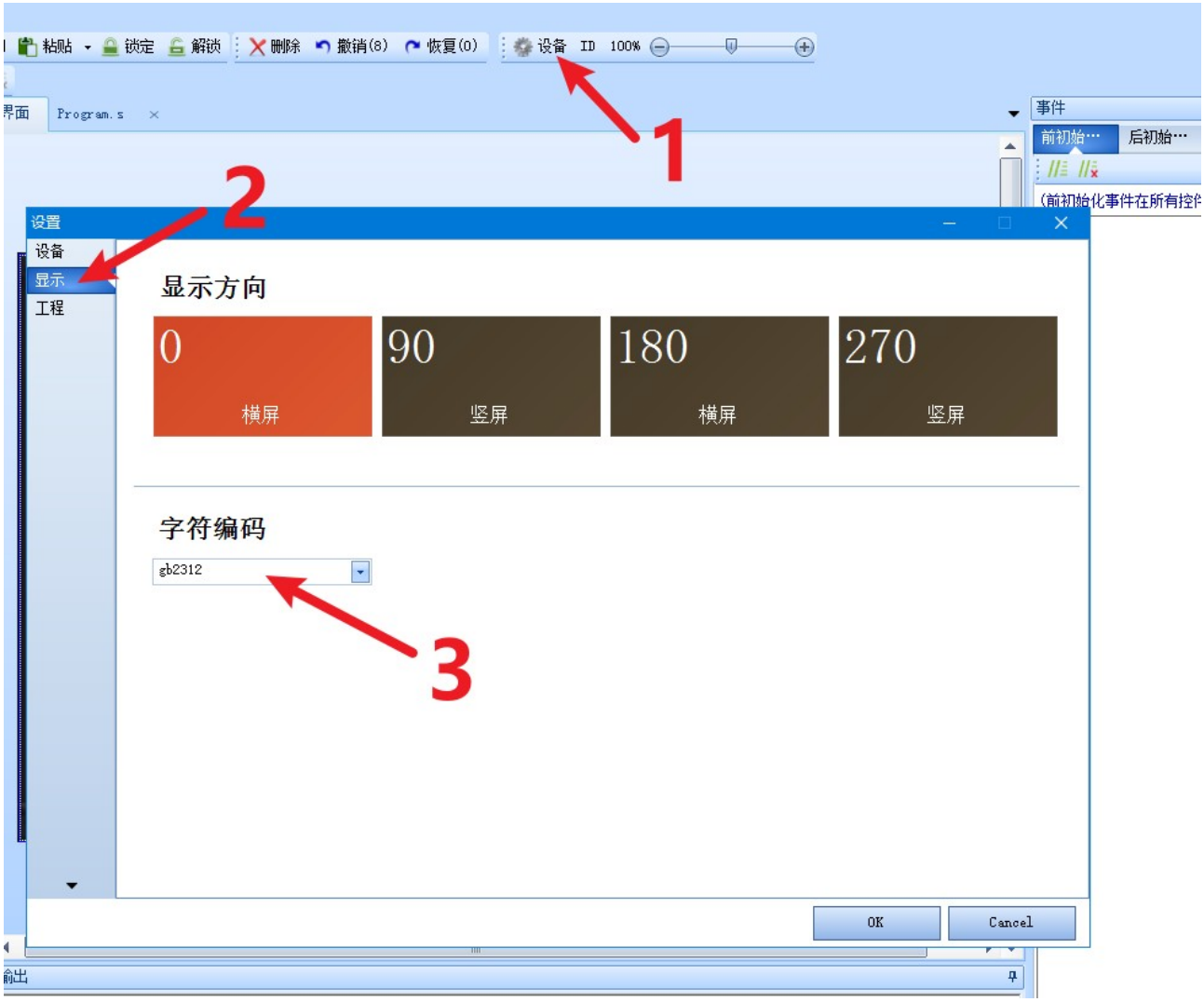
如果是供电不足，参考下面的解决方法

外部供电接法 (page 1262)

11.4.5 变量名称无效 zstr

这是由于工程编码和字库编码不一致导致的，将编码改为一致即可

工程编码的查看位置如下



字库编码的查看位置如下

工程编码和字库编码尽量一致，如果不一致的情况下，字库编码必须是工程编码的子集，例如 ascii 编码是 GB2312 的编码的子集，ascii 编码也是 UTF8 编码的子集

千万不能工程编码选用了 ascii，但是字库编码缺用了 GB2312 和 UTF8

11.4.6 file version is too low

下载最新版本的上位机上位机下载 (page 1066)
重新安装最新的上位机版本后重新编译下载
如果还是显示 file version is too low, 建议更换串口下载工具

11.4.7 报错:file is too large for destination device

提示: 翻译：文件对目标设备太大

说明：烧录文件过大，超出目标设备的容量限制，下载不进去。

需要优化一下字库和图标之类的，让生成的文件小一点

建议检查一下字库占用的大小

参考：指定字库 (page 1284)

11.4.8 在做字库的时候有部分字体无法选择

先重启一下电脑，仍然不能找到字体的时候再尝试下面的方法

Windows 常用的的字体文件类型为.ttf 的文件。但是有少部分字体文件为.otf 或者其他类型格式。

我们需要将不支持的字体转换成.ttf 的字体类型。方法如下：

1，找到 windows 的字体文件夹：C:\Windows\Fonts。将不支持的字体文件复制到电脑其他地方。然后再删除 windows 字体文件夹中的该字体。

2，使用字体转换工具将，其他类型的字体文件转换成.ttf 文件。

3，重新安装字体。将转换好的.ttf 文件重新放会 Windows 的字体文件夹：C:\Windows\Fonts。

11.4.8.1 字库创建-相关链接

创建字库和导入字库 (page 125)

指定字库 (page 1284)

编译报错：*XXX* 初始值无效 (page 1342)

utf8 字库下如何只选择汉字 (page 1403)

如何安装新字体 (page 1439)

11.4.9 如何解决调用系统键盘给控件赋值无效

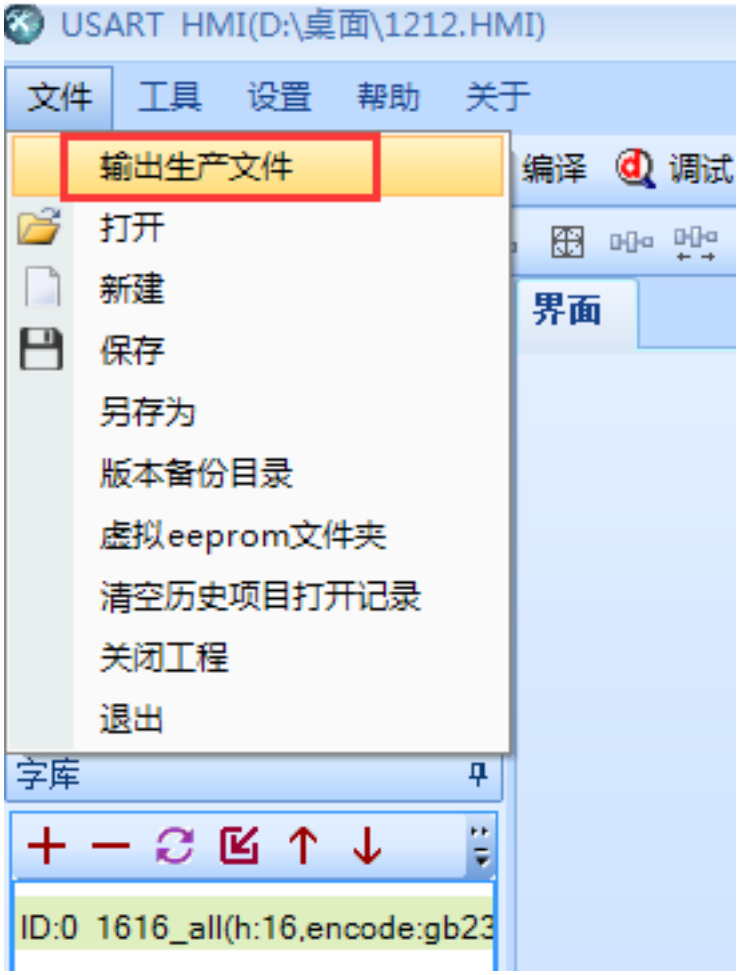
数据输入是通过切换到页面的方式来实现键盘调用的，当你点击需要输入数据的控件时，当前页面就离开了，重新刷新键盘页面，效果等同于 page 指令，因此如果你在数据页面的初始化事件里给控件赋了初始值的话，键盘输入完以后重新刷新数据页面时，数据就再次被初始化了，导致键盘输入的值不起作用了。

解决方案一: 不要在此数据页面的初始化里给控件赋初始值，比如你可以建一个专门用来做上电赋初始值的页面，做为上电默认页，在此页面的初始化事件里完成所有的初始化动作之后，切换到工程主页，这样初始化动作就只会上电执行一次，不会重复执行。

解决方案二: 在数据输入页面建一个变量 (要全局的哦), 用来表示是否已经初始化过了，赋初始值的时候判断控件是否已经被初始化过，是的话就不要再赋初值了，这样键盘调用完之后再次回来就不会重复执行了。

11.4.10 软件输出生产文件，出现提示框拒绝访问

1. 打输出文件夹，用于 sd 卡下载工程



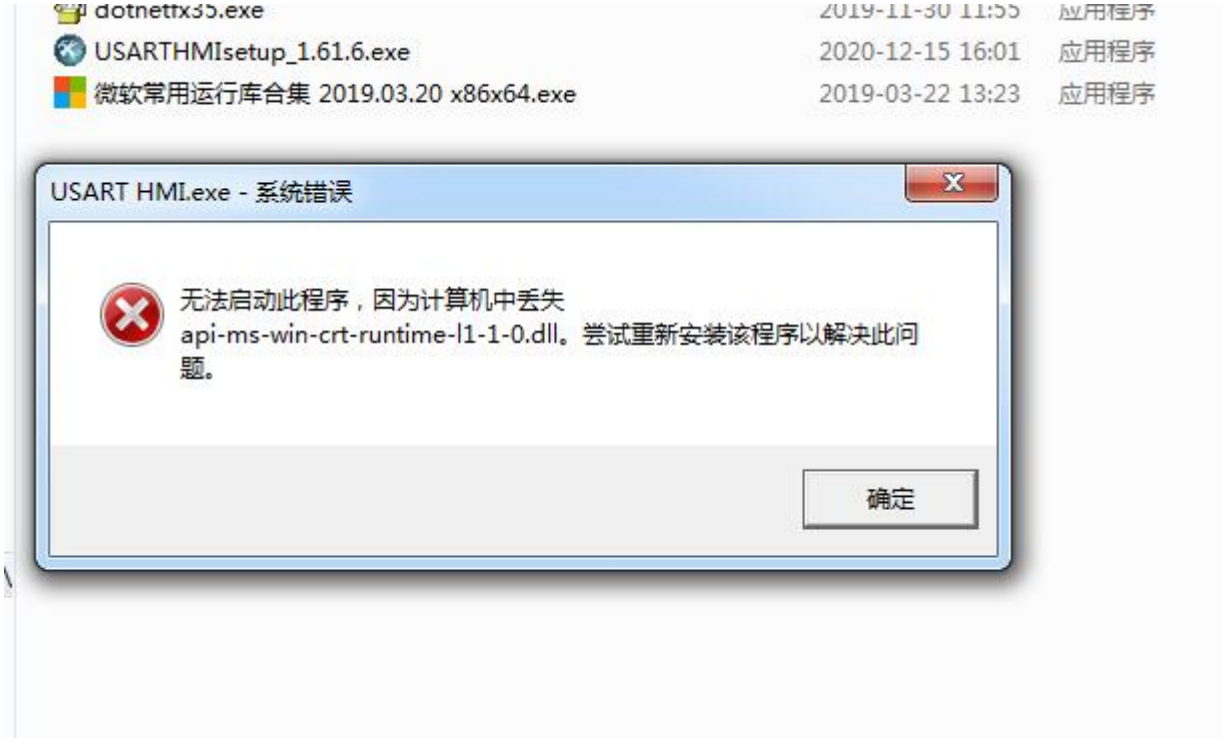
2. 软件出现提示框

3. 这是因为被电脑 360 软件拦截了。

解决办法：建议退出电脑上的 360 杀毒软件（或其他杀毒软件），再重新输出即可

11.4.11 window7 无法打开软件安装包

window7 64 位操作系统打开软件安装包提示下面的错误



可通过下面链接打上相应补丁即可

https://www.microsoft.com/zh-cn/download/confirmation.aspx?id=49093

11.4.12 用串口下载程序，成功但显示 Update failed check error

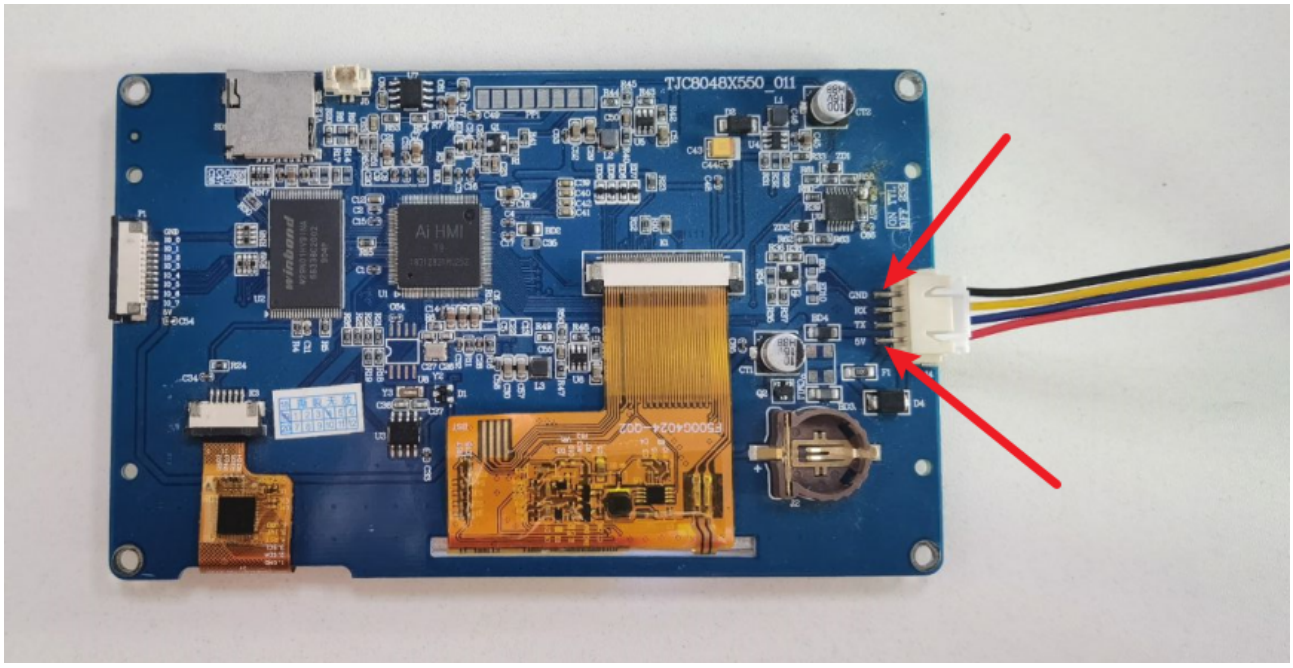
出现这个问题是因为，串口传输数据产生误码了。可以从以下方面着手解决。

1. 检查是否有使用 USB-HUB（集线器/拓展坞）之类的，如果有请先去掉，直接接电脑 USB 接口。
2. 尝试降低串口波特率。
3. 缩短串口延长线距离。
4. 换一个 USB 转串口模块。

11.4.13 串口屏开机时死机/不断的闪烁/不断重启

这种现象一般出现在 4.3 寸及以上的屏，因为供电不足导致的，请检查你的电源带载能力，在通电情况下检测屏幕电源座子上的电压是否正常。务必亲自动手检测带载能力，不要看电源标签上的标称值，多数电源的标称值都是假的。

当出现串口屏开机时死机/不断的闪烁/不断重启的现象时，请测量串口屏供电端子上的电压是否始终高于 5V



如果是 x3/x5 系列的屏, 检查开机时是否播放视频或音频, 建议先去掉喇叭, 测试是否死机

调试时需要外接供电，请参考下面的接法

外部供电接法 (page 1262)

11.4.14 调试时可以正常显示，但是下载进屏幕就无法使用

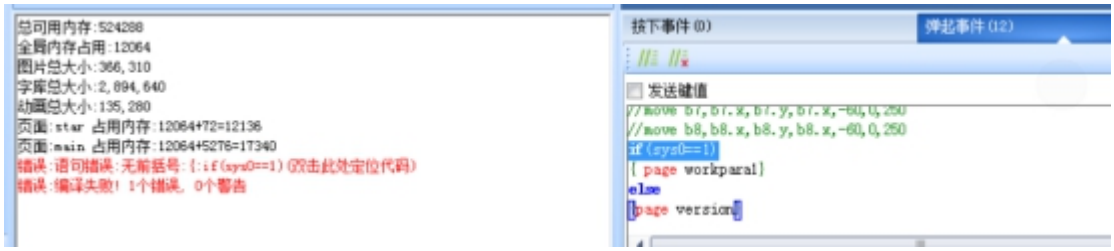
这种情况是因为没有配置波特率导致的

如何配置波特率请参考如何配置亮度，主动解析，波特率 (page 1306)

11.4.15 视频 IDO 文件方向与当前工程显示方向不一致

更改了屏的现实方向以后，需要重新用视频转换工具转换导入视频。并在导入的时候选择方向。

11.4.16 提示语句错误：无前括号



大括号要单独一行

11.4.17 提示转义字符使用错误

这种情况发生在输入“\”时，只需要换成“\\”就行了

```
t0.txt=t0.txt+"\"    //错误,将会提示转义字符使用错误
t0.txt=t0.txt+"\\"   //正确
```

如果需要回车换行，请使用 r 例如 t0.txt=”123r456”

11.4.18 编译时提示变量名称无效

检查是否存在这个变量

检查变量是否添加了.val 或者.txt 等属性

如果是跨页面调用，需要设置该变量为全局，并且调用时在前面加上页面的名称

检查运算符号两边是否存在空格，有空格请删除

是否把. 写成了，

检查是否添加了分号

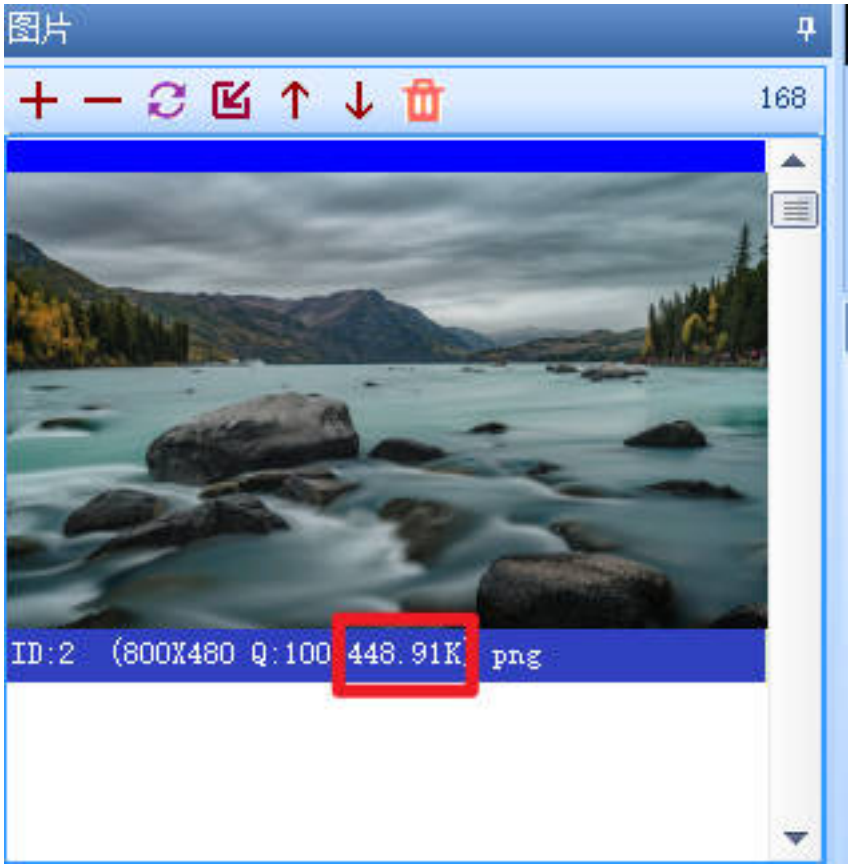
11.4.19 编译时提示无效指令

检查关键字是否拼写正确
检查运算符两边是否有多余的空格

11.4.20 提示 Show Picture Error

出现这种情况有以下几种可能：

如果是导入工程内部的图片，请检查图片体积是否超过 300K，如果超过 300K，请右键图片选择“压缩质量设置”，将图片的压缩质量降低到 200k 以下甚至更低。



另外请检查是否勾选了编译时忽略图片资源，这项被勾选时将导致所有的图片无法显示

如果设置了文件存储区大小，将会直接影响屏幕的内存容量，屏幕内存会被分割出去存储数据，可能导致内存不足从而显示失败

使用外部图片控件时出现请参考外部图片显示不正常 (page 570)

如果还是不行请

11.4.21 下载一半失败后下载不进去

这种情况通常出现在使用串口下载的情况下

有以下几种解决方式：

1. 尝试重启电脑和屏幕，再次通过串口进行下载
2. 如果使用的是 ch340 或者 pl2303，建议更换为 cp2102 或者 ft232
3. 通过 *SD* 卡下载工程到串口屏 (page 155)，下载成功后，就可以继续用串口进行下载了

11.4.22 输入中文时上位机卡死

这是因为上位机和输入法冲突导致的，现有的方案是将输入法换成 qq 输入法

11.4.23 no find cpu model

更新上位机即可

11.4.24 提示：请先选择控件

这是因为没有选中控件就复制或剪切时提示的错误，请选中控件后再复制或剪切。

11.4.25 提示：没有控件可以粘贴

这是因为没有复制或剪切控件就进行粘贴导致的问题，请选中控件后复制或剪切，在进行粘贴。

11.4.26 明明型号是对的，但是烧录的时候提示 Model does not match

请再三确认型号是正确的，如果确认型号确实和 tft 文件符合，那么请按照以下的方法解决：

下载最新的上位机 USART HMI

选择设备，切换成其他设备然后编译一次

切换回原设备，再编译一次

最后重新生成 tft 文件即可

11.4.27 MAC 电脑下拷贝文件时提示有多个 TFT 文件

使用 mac 电脑拷贝文件时，会生成一个同名的隐藏文件

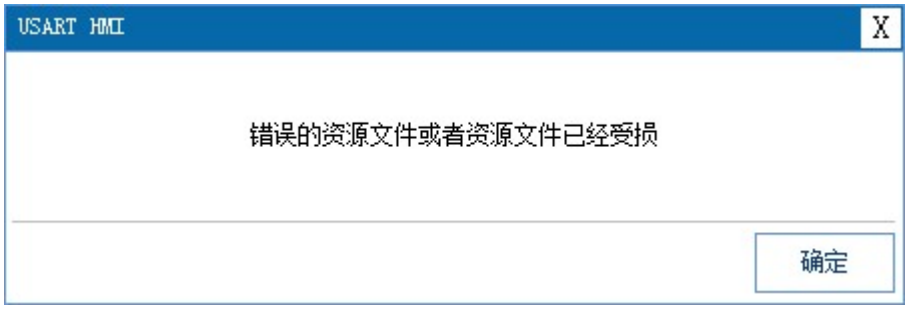
mac 电脑使用以下工具将文件拷贝到 SD 卡即可

下载链接：

mac 电脑拷贝工具下载

mac 电脑拷贝工具源码下载

11.4.28 错误的资源文件或者资源文件已经受损



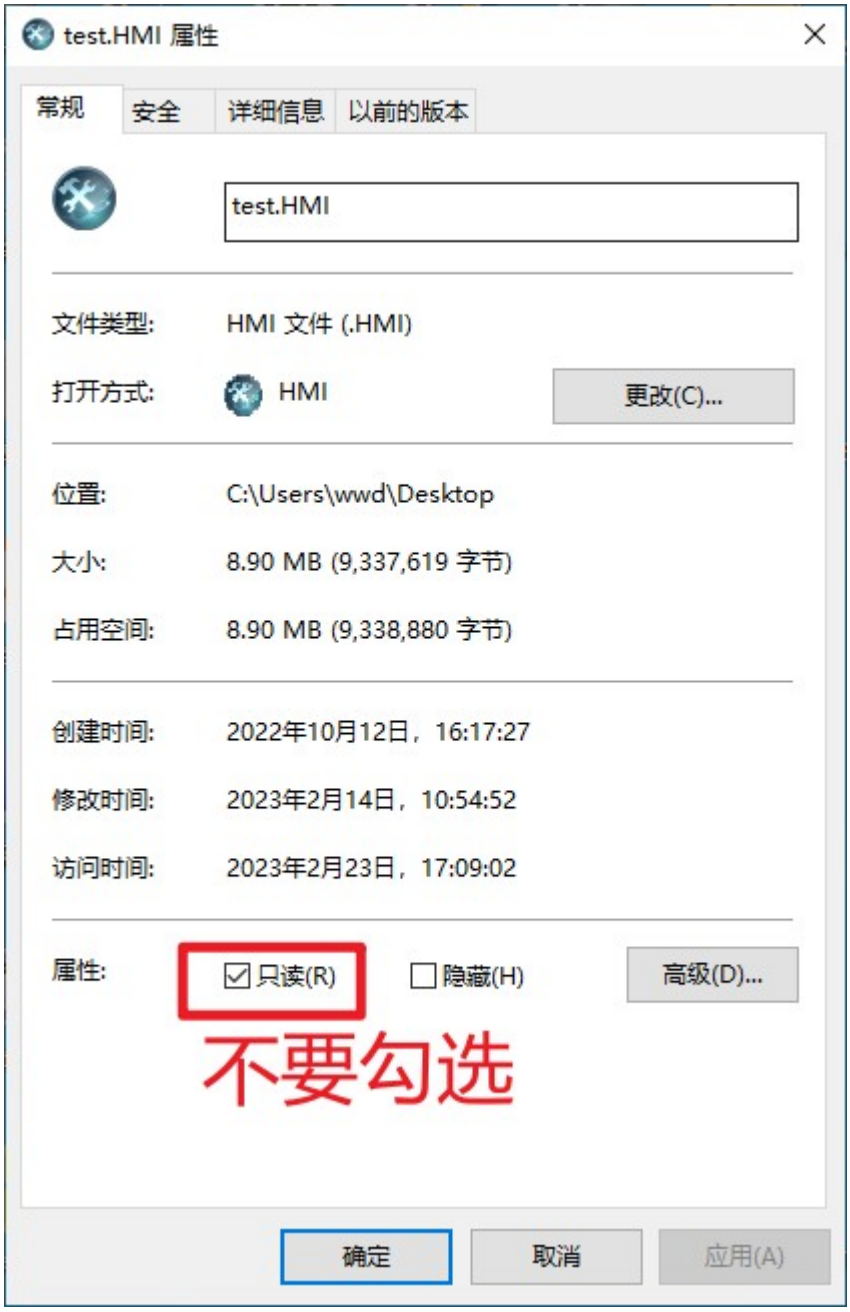
打开文件时提示：错误的资源文件或者资源文件已经受损

目前发现以下三个可能导致问题的原因：

> **注意：** 1、客户通过微信传输了 HMI 工程文件，导致文件被设置了只读，导致打开 HMI 工程文件时提示错误。
>
> 2、win10，win11 下需要解压才能打开文件，不要直接双击压缩包（此时并没有真正的解压），请先解压后再打开 HMI 工程文件。
>
> 3、客户的电脑使用了加密功能，文件被加密了，导致打开文件时校验不通过从而报错，建议客户关闭对 HMI 工程文件的加密功能后再尝试打开 HMI 工程文件。

通过微信传输了 HMI 工程文件导致的问题可以用下面的方法设置。

解决方法：右键选择文件-属性，将只读属性的勾去掉，点击确定即可。


test.HMI 属性
常规
安全
详细信息
以前的版本
test.HMI
文件类型:
HMI 文件 (.HMI)
打开方式:
HMI
更改(C)...
位置:
C:\Users\wwd\Desktop
大小:
8.90 MB (9,337,619 字节)
占用空间:
8.90 MB (9,338,880 字节)
创建时间:
2022年10月12日，16:17:27
修改时间:
2023年2月14日，10:54:52
访问时间:
2023年2月23日，17:09:02
属性:
只读(R)
隐藏(H)
高级(D)...
不要勾选
确定
取消
应用(A)

11.4.29 烧录文件提示 open file error

请勿通过微信和 QQ 直接传输 HMI 工程文件和 tft 文件，请将文件用压缩软件压缩成.zip 或者.rar 的格式再进行发送。

HMI 工程文件直接通过微信发送后会变成只读文件，需要右键文件-属性, 把只读勾选掉，点击确定，就能重新打开。

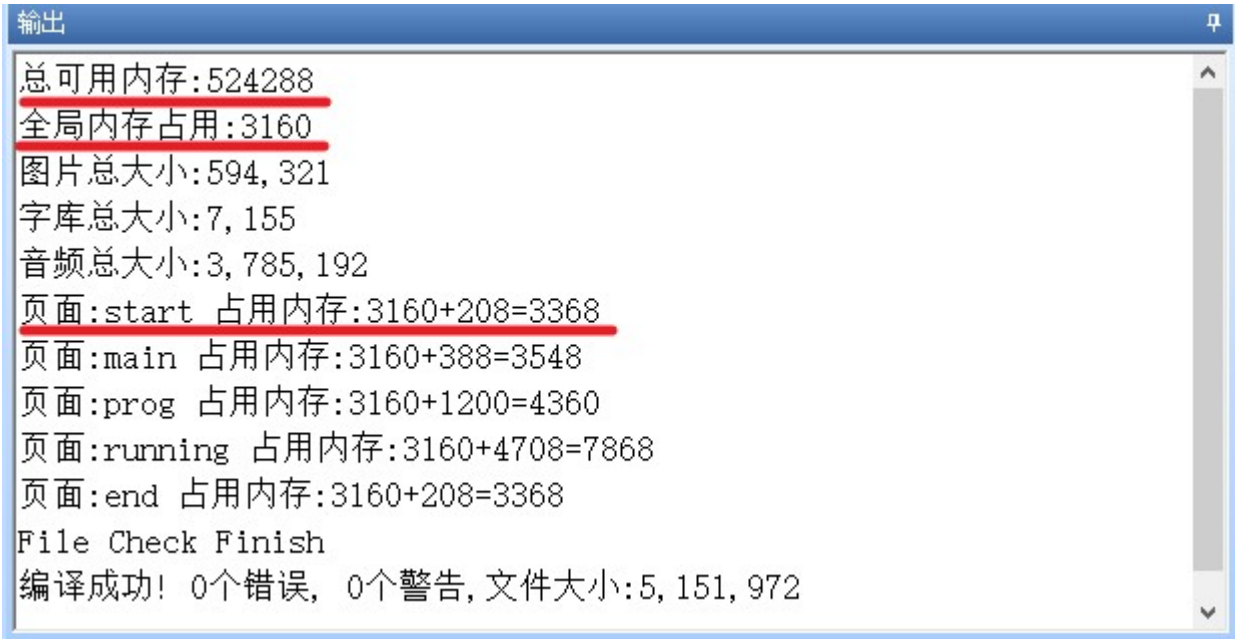
tft 文件会因为微信不知名的原因导致数据错误，必须压缩成.zip 或者.rar 的格式再进行发送。

11.5 其他

11.5.1 如何查看编译时输出的信息

11.5.1.1 运行内存占用情况

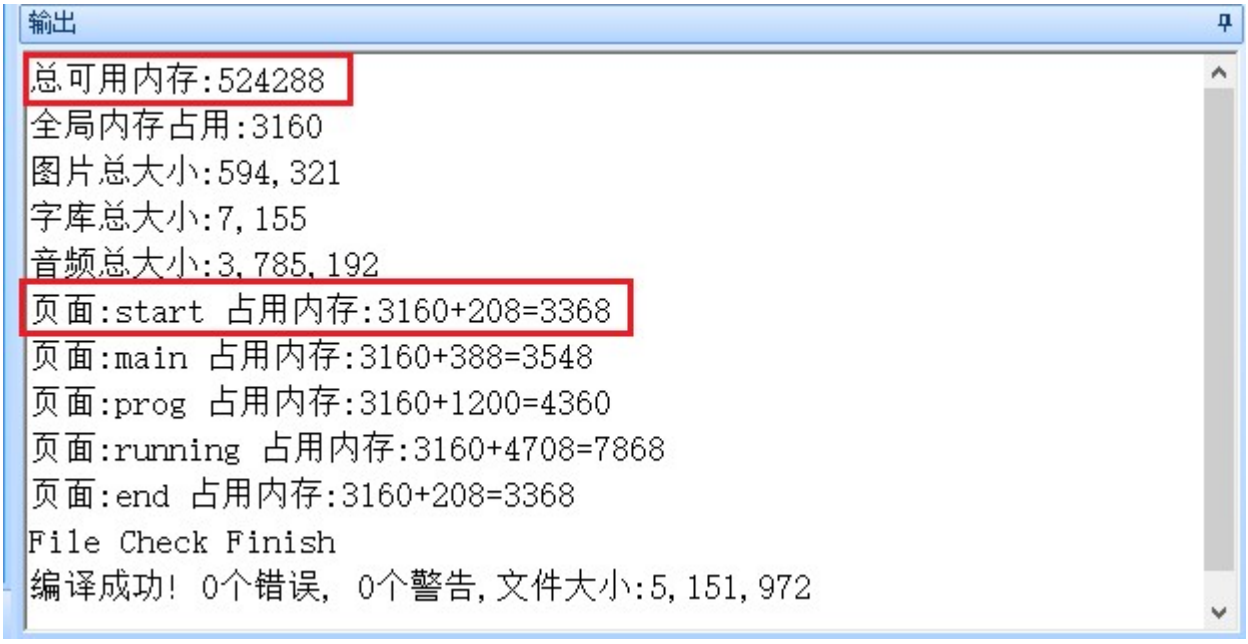
1. 在 program.s 中定义的变量以及 vscope 被设置为全局的变量会占用全局内存



2. 每个页面的内存占用 = 全局内存占用 + 私有内存占用。当切换页面时，私有变量会被释放，全局变量则不会。任意一个页面占用的内存大小不得超过总可用内存。

以 X 系列为例，总可用内存 524288 字节 =512kB，running 页面占用内存最多，但仅为 7868 字节，远远没有超过 524288 字节

正常情况下，X 系列的运行内存是完全足够的，T 系列和 K 系列因为仅有 3.5K 内存，在控件较多的情况下可能不够用



11.5.1.2 FLASH（存储空间）占用情况

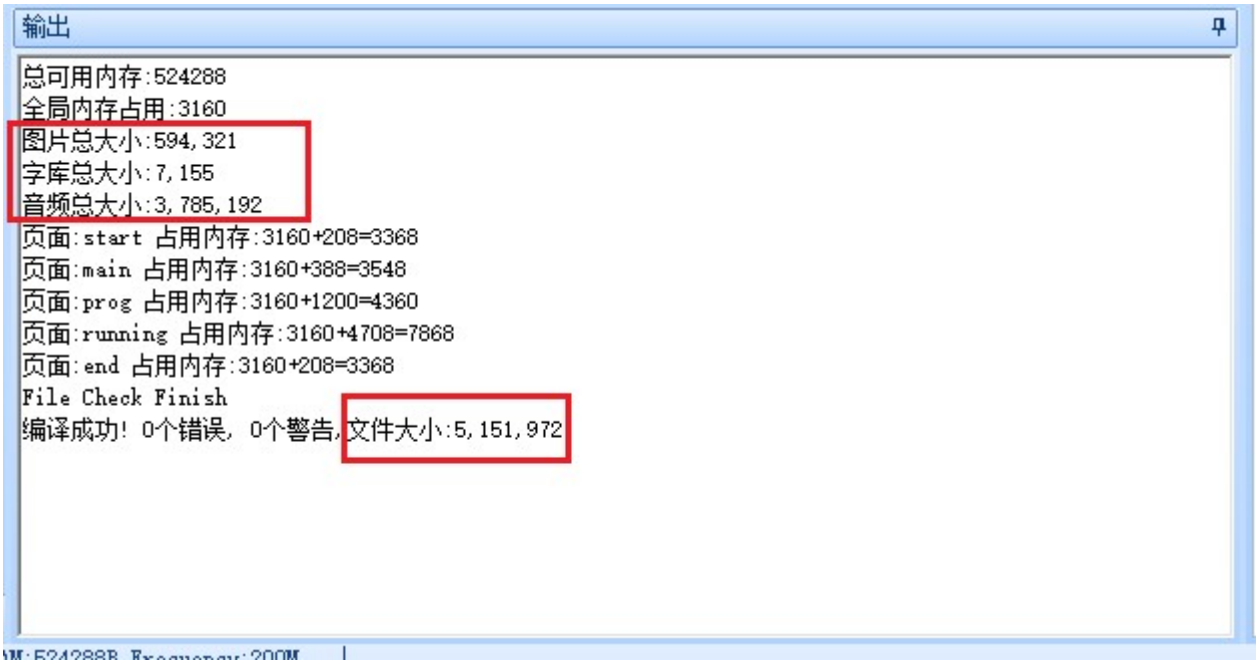
TFT 文件大小 = 图片 + 字库 + 动画 + 视频 + 音频 + 代码

文件大小为最终输出的 tft 文件的大小，应略小于屏幕的 FLASH 大小，否则会提示报错:*file is too large for destination device* (page 1359)

相关链接

TFT 文件如何下载到串口屏中 (page 1405)

如何输出 TFT 生产文件 (page 155)



11.5.2 不同页面之间的同名控件有什么联系

没有联系

page0 页面的 t0 和 page1 页面的 t0，他们之间是没有任何联系的

打个比方，张三家的小狗叫小白，李四家的小狗也叫小白，虽然他们都叫小白，但是他们是独立的个体，张三家的小白吃饱了，但是李四家家的小白可能还饿着

但是当你通过串口去呼叫小白时，你应该知道当前是处于张三家还是李四家，如果你不知道当前在哪，那保险的方法就是呼叫主人名称加小狗名称，例如“张三家的小白”，“李四家的小白”

11.5.2.1 如何同步两个不同页面的控件

比如两个页面里的 t0 控件，分别是 page0 页面的 t0 控件和 page1 页面的 t0 控件

将其中一个控件的 vscope 属性设置为全局，例如将 page0 页面的 t0 设置为全局

在 page1 页面添加一个定时器用于同步数据，定时器的时间可以根据自己的需求设置 50 或更多

在 page1 页面的前初始化事件和定时器里添加以下代码

```
t0.txt=page0.t0.txt
```

11.5.3 在定时器 0 里面加入使能定时器 1 的语句为什么系统不执行

每次使能定时器后, 时间都会重头算

比如 timer0 是 1 秒 1 次,timer1 是 10 秒一次, 每次 timer0 里赋值 timer1.en=1 时,timer1 都会从 0 开始计时, 那么 timer1 就永远执行不了, 应该在 timer0 中使能 timer1 时先判断 timer1d 的状态

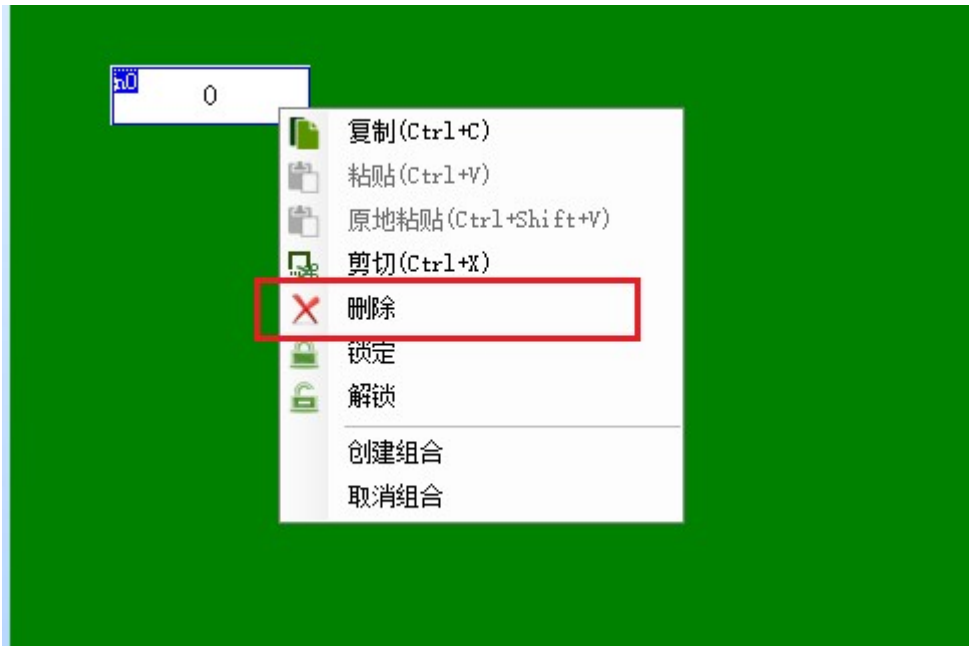
```
if(timer1.en==0)
{
    timer1.en=1
}
```

11.5.4 大小端模式

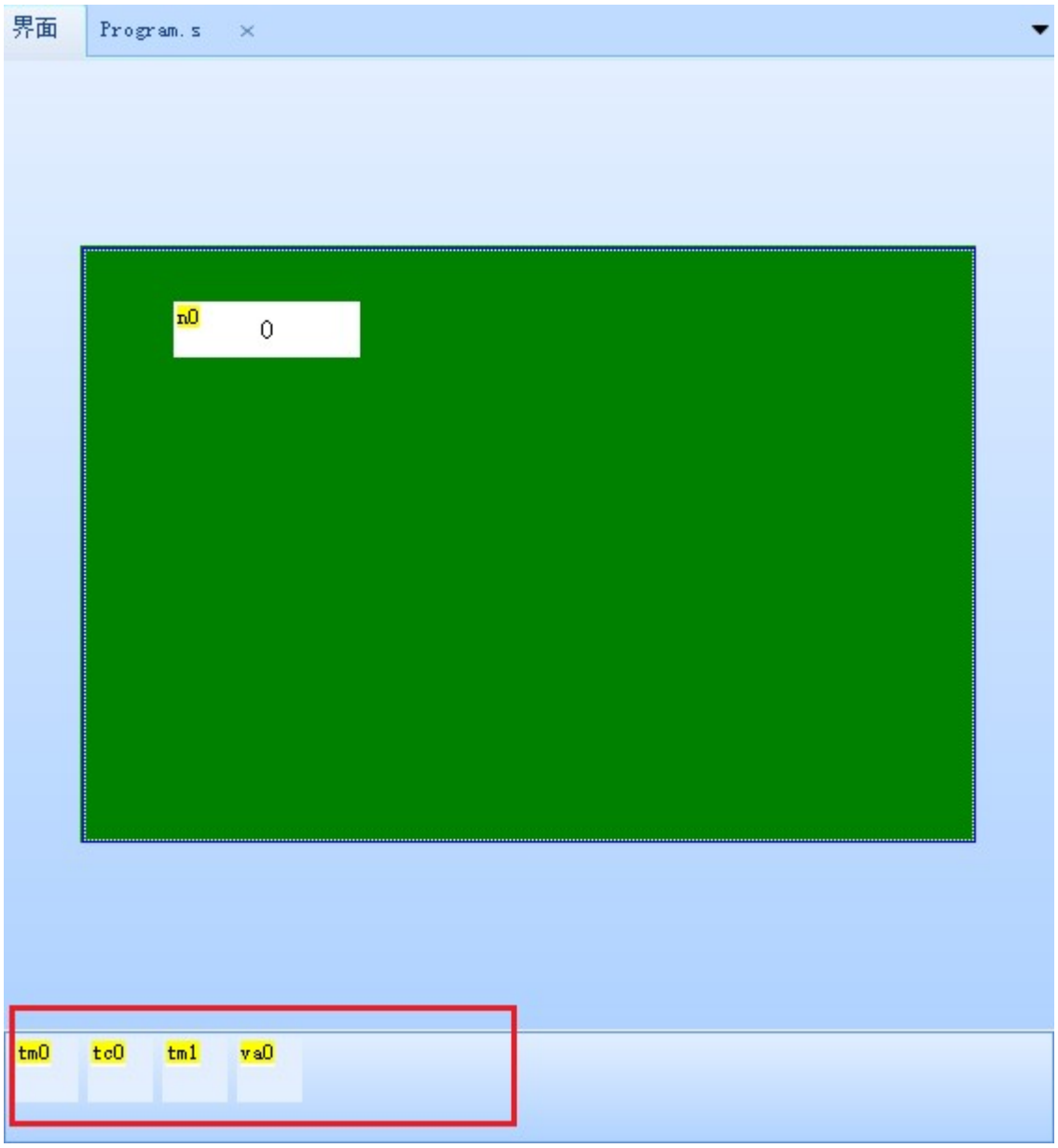
串口屏是小端模式的，因此在跟大端设备（如 PLC）通讯时，需要进行大小端转换
如何进行大小端转换请参考：大小端数据互相转换样例工程 (page 1132)

11.5.5 如何删除控件

对于可见的控件，可以鼠标右键点击控件，选择删除



位于特殊控件栏上的控件，例如变量控件，定时器控件，触摸捕捉控件，文件流控件等，可以左键选中，点击键盘上的 del 键删除

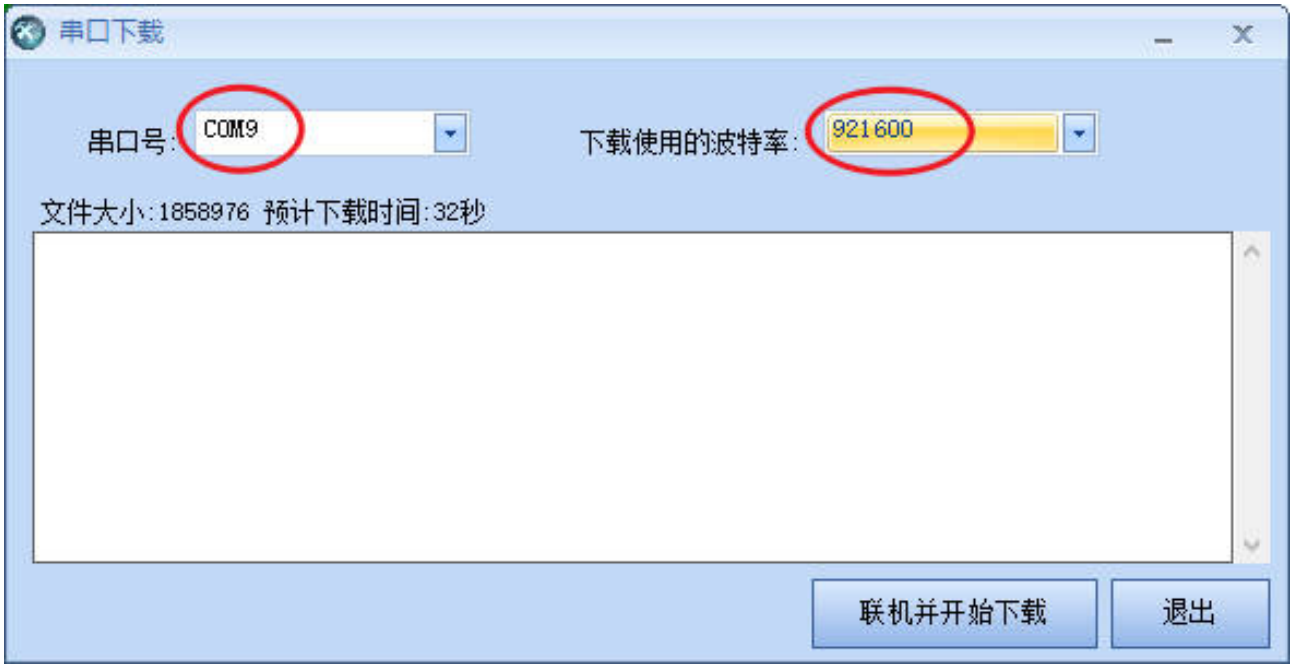


也可以点击顶部的删除按钮删除

11.5.6 使用串口下载工程速度慢怎么办

1、手动选择自己的串口，可以节省搜索串口的时间

2、下载波特率和通讯波特率是分开的, 下载波特率可以使用最快的 921600, 通讯的波特率需要单片机和屏幕匹配, 如果是 cp2102 和 ft232 的串口芯片，建议直接选择 921600 的最高波特率进行下载



3、使用 sd 卡进行下载，下载完成后断电，拔出 sd 卡重新上电即可

参考链接

使用 *TFT* 文件下载助手 *(TFTFileDownload)* 通过串口下载工程到串口屏 (page 150)

通过串口下载工程到串口屏 (page 144)

通过 *SD* 卡下载工程到串口屏 (page 155)

为什么不建议用 *ch340* (page 1248)

为什么 *232* 通讯不能超过 *256000* (page 1265)

11.5.7 printf() 重定向之后，发送命令和结束符

```
printf("n0.val=666"); //发送命令
printf("\xff\xff\xff"); //发送结束符
printf("n0.val=666\xff\xff\xff");    //一次性发完命令和结束符

printf("n0.val=%d\xff\xff\xff",MyData); //数字控件带参数一次性发完命令和结束符
printf("t0.txt=\"%d\"\xff\xff\xff",MyTxt); //文本控件带参数一次性发完命令和结束符
```

注意： 这里发送 16 进制是用 xff，不要写成 0xff, 会导致结束符无法发送

11.5.8 实现自己的 printf 函数

以下代码仅供参考，使用时需要根据不同的单片机进行匹配和修改，仅供有一定能力的人进行学习，不提供技术支持

```
void ScreenPrintf(const char *cmd, ...)
{
    uint8_t txt[30];                                    //文本数组
    uint8_t *txt_p;                                     //文本数组指针
    memset(&txt,0,sizeof(txt));
    memset(&txt_p,0,sizeof(txt_p));
    va_list args;       //定义一个va_list类型的变量，用来储存单个参数
    va_start(args,cmd); //使args指向可变参数的第一个参数
    vsprintf((char *)txt, (const char *)cmd, args);
    va_end(args);
    txt_p = txt;
    while(*txt_p!='\0')
    {
        USART_SCREEN_write(*txt_p);//这里修改成你单片机写入串口单字节的函数
        txt_p++;
    }
    USART_SCREEN_write(0xff);//这里修改成你单片机写入串口单字节的函数
    USART_SCREEN_write(0xff);//这里修改成你单片机写入串口单字节的函数
    USART_SCREEN_write(0xff);//这里修改成你单片机写入串口单字节的函数
}
//使用时ScreenPrintf("n0.val=%d",MyData); //一次性发完命令和结束符
```

11.5.9 单片机如何控制屏幕

11.5.9.1 单片机如何控制屏幕 (文本控件)

1、在上位机工程新建一个文本控件，假设为 t0，将程序下载到串口屏上

2、串口屏串口与单片机串口连接，两者波特率应一致，单片机 RX 接串口屏 TX，单片机 TX 接串口屏 RX。

3、发送指令：单片机串口通过字符串模式发送 t0.txt=”六六六”

//注意 arduino 需要注意编码，因为 arduino 的默认编码是 utf-8，串口屏默认编码是 gb2312，请保证单片机与串口屏的编码相同，否则串口屏会显示乱码, 或者不显示

4、发送结束符：单片机通过 HEX 模式发送 0xff 0xff 0xff

5、此时屏幕上的 t0 控件内的文字变为“六六六”

11.5.9.2 单片机如何控制屏幕 (数字控件)

1、在上位机工程新建一个数字控件，假设为 n0，将程序下载到串口屏上

2、串口屏串口与单片机串口连接，两者波特率应一致，单片机 RX 接串口屏 TX，单片机 TX 接串口屏 RX。

3、发送指令：单片机串口通过字符串模式发送 n0.val=666

4、发送结束符：单片机通过 HEX 模式发送 0xff 0xff 0xff

5、此时屏幕上的 n0 控件内的文字变为“666”

11.5.9.3 串口屏如何显示小数

方法 1：使用虚拟浮点数控件

1、在上位机工程新建一个虚拟浮点数控件，假设为 x0，设置虚拟浮点数控件的小数点位数 2 位，将程序下载到串口屏上，

2、串口屏串口与单片机串口连接，两者波特率应一致，单片机 RX 接串口屏 TX，单片机 TX 接串口屏 RX。

3、发送指令：单片机串口通过字符串模式发送 x0.val=314

4、发送结束符：单片机通过 HEX 模式发送 0xff 0xff 0xff

5、此时屏幕上的 x0 控件内的文字变为“3.14”

方法 2：使用文本控件

1、在上位机工程新建一个文本控件，假设为 t0，将程序下载到串口屏上

2、串口屏串口与单片机串口连接，两者波特率应一致，单片机 RX 接串口屏 TX，单片机 TX 接串口屏 RX。

3、发送指令：单片机串口通过字符串模式发送 t0.txt=”3.14”

4、发送结束符：单片机通过 HEX 模式发送 0xff 0xff 0xff

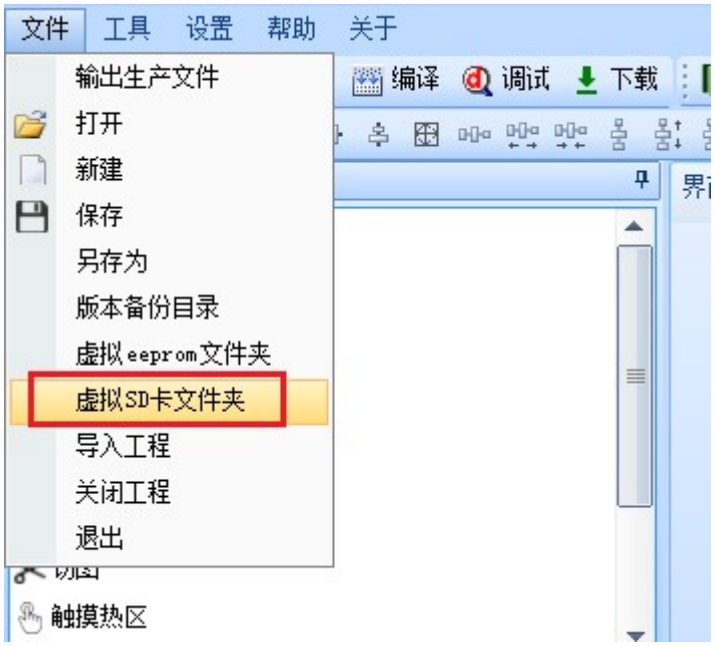
5、此时屏幕上的 t0 控件内的文字变为“3.14”

11.5.9.4 单片机如何控制屏幕-相关链接

串口屏通讯协议 (page 224)

11.5.10 虚拟 sd 卡文件夹如何打开

左上角文件-虚拟 sd 卡文件夹



11.5.11 如何实现页面滑动切换

设置页面的 up/down/left/right 属性, 仅 X 系列支持滑动翻页/划动翻页。

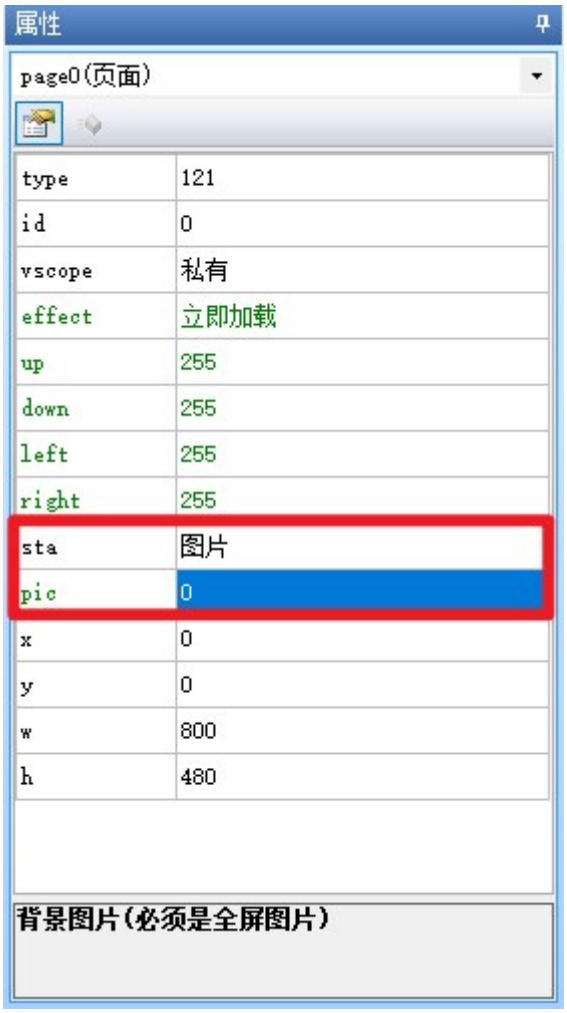
当 up/down/left/right 为 255 时, 代表不翻页。

填写对应的页面 id 号, 则触发相关滑动事件时跳转到相应的页面。

页面
6
0 page0
1 page1
2 page2
3 page3
4 page4
5 page5
属性
page5(页面)
type 121
id 0
vscope 私有
effect 立即加载
up 255
down 255
left 255
right 255
sta 单色
bco 65535
x 0
y 0
w 800
h 480
点击属性显示相应注释

11.5.12 如何设置页面背景

设置页面的 sta 属性为图片，然后设置页面的 pic 属性即可，注意，背景图片分辨率需要与当前的屏幕分辨率一致



11.5.13 HMI 文件和 TFT 文件的关系

HMI 是工程源文件，包含了源代码，可以通过上位机打开编写和修改其中的代码

TFT 文件是 HMI 经过编译后输出的文件，可以理解为单片机上的 hex 文件，是没有办法编辑的，也不能反编译成 HMI 文件

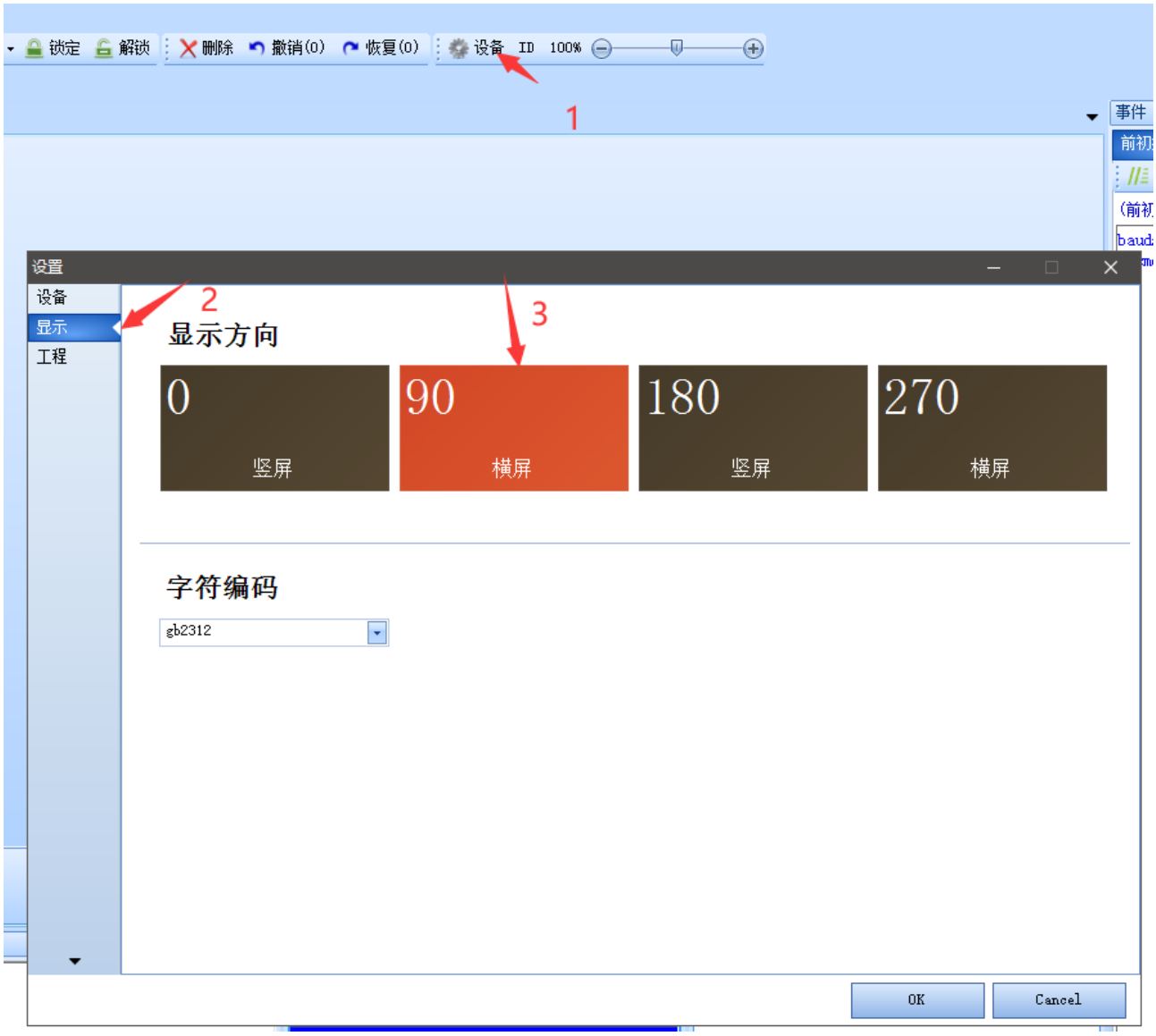
相关链接

TFT 文件如何下载到串口屏中 (page 1405)

如何输出 TFT 生产文件 (page 155)

11.5.14 界面旋转/翻转/方向

修改界面显示的方向角度



11.5.15 串口屏如何制作弹窗

11.5.15.1 弹窗实现方法 1

此方法实现的弹窗本质上是一个页面 (但是背景没有完全覆盖前一个页面), 就会形成弹窗的效果

1. 将页面设置为无背景

2. 在页面上根据自己的需求放置文本控件或者图片控件

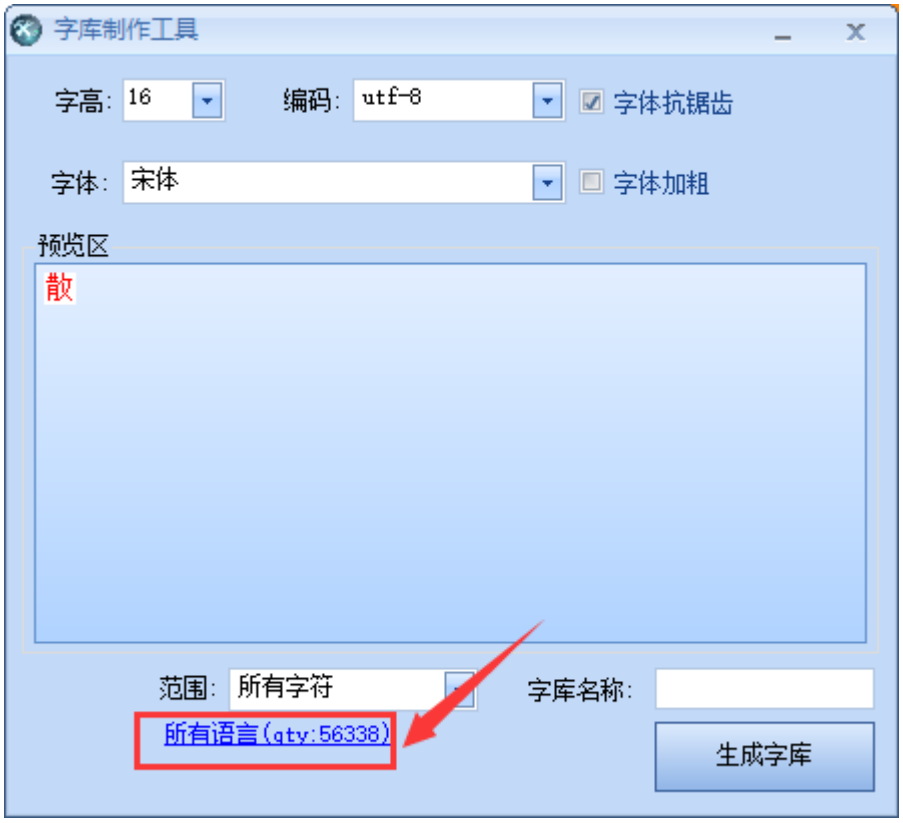
11.5.15.2 弹窗实现方法 2

提前用 vis 指令隐藏弹窗, 在需要时显示出来

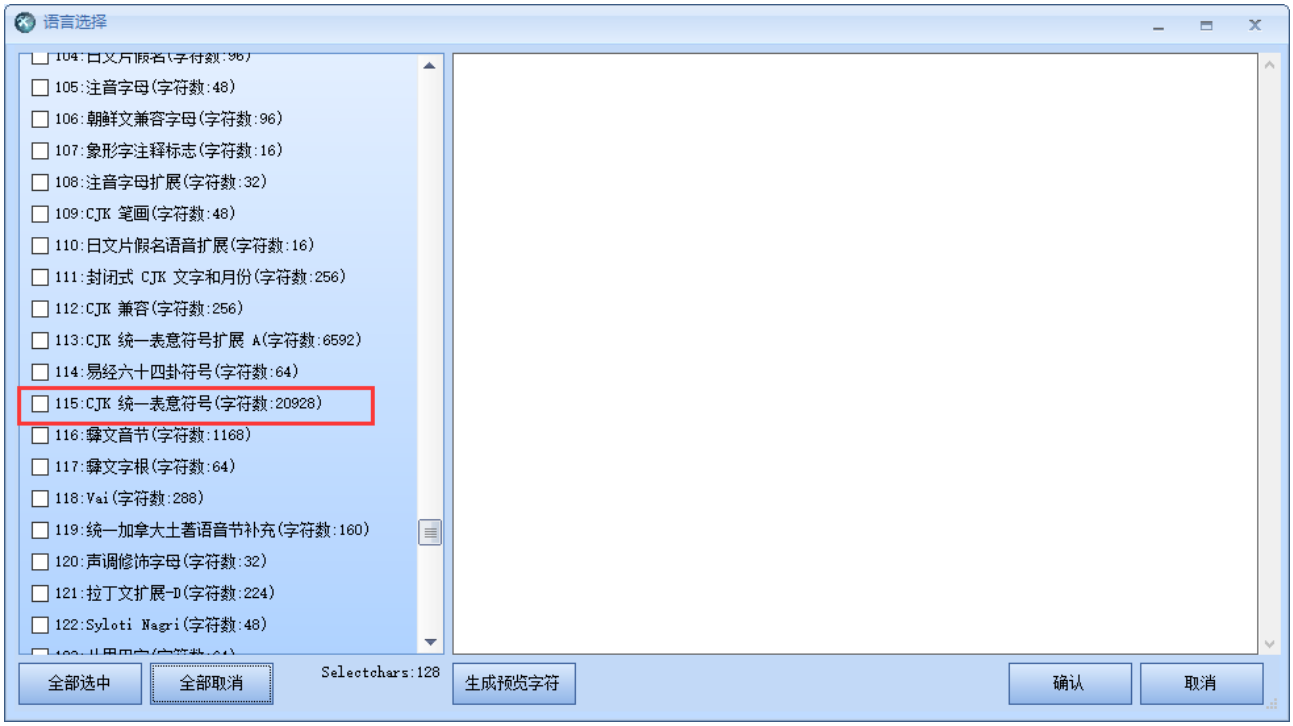
11.5.15.3 串口屏如何制作弹窗-样例工程

串口屏弹窗样例工程 (page 1112)

11.5.16 utf8 字库下如何只选择汉字



选择 115 号，cjk 统一表意符号，CJK 是中文（Chinese）、日文（Japanese）、韩文（Korean）三国文字的缩写，它能够支持这三种文字，其余 CJK 拓展可以自行选择



11.5.16.1 UTF8 字库-相关链接

创建字库和导入字库 (page 125)
指定字库 (page 1284)
在做字库的时候有部分字体无法选择 (page 1360)
编译报错：*XXX* 初始值无效 (page 1342)

11.5.17 TFT 文件如何下载到串口屏中

11.5.17.1 通过 SD 卡下载 TFT 文件到串口屏

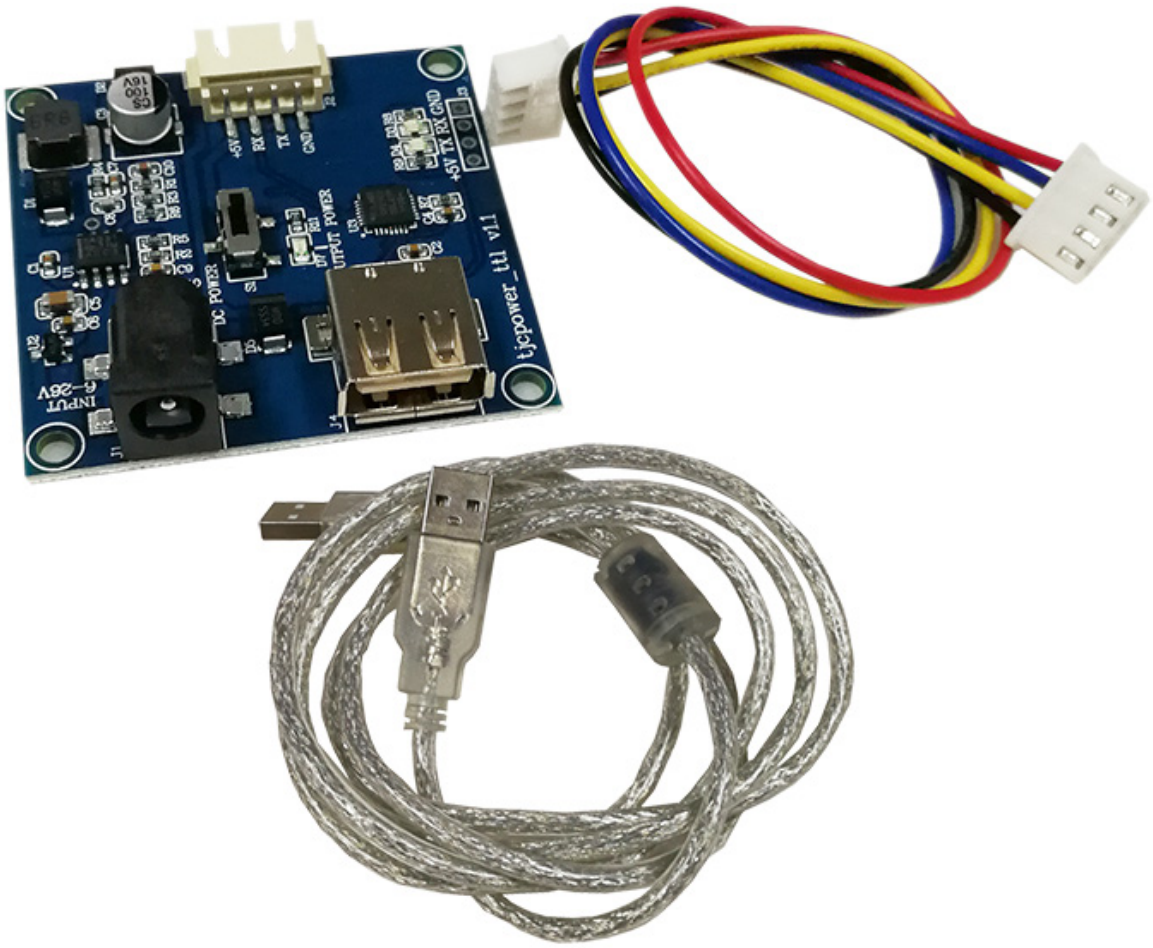
1. SD 卡下载用于在批量生产时使用，使用前请确保 SD 卡没有隐藏分区（没有做过启动盘），SD 卡容量不超过 32G，且格式化为 fat32 格式
2. 将 TFT 文件拷贝到 SD 卡根目录
3. 将串口屏断电，SD 卡插入串口屏的 SD 卡座中
4. 串口屏重新上电，等待烧录完成
5. 串口屏断电并取出 SD 卡
6. 串口屏重新上电，程序下载完成

11.5.17.2 通过串口下载 TFT 文件到串口屏

需要使用软件通过串口将 TFT 文件下载到串口屏，下载地址*TFT 文件下载助手 (TFTFileDownload)* (page 1097)

需要使用到串口工具，推荐使用 cp2102 或者 ft232，不推荐使用 ch340 或者 pl2303

推荐使用官方的串口下载套件

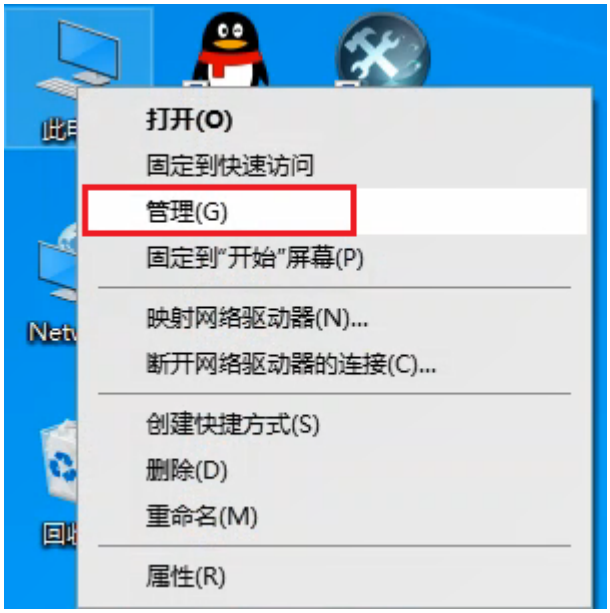


注意： 如果您购买的屏大于 5 寸或者有使用喇叭的需求，请再购买一个 12V 电源用于额外供电，否则可能因为电脑 USB 供电不足导致开机启动失败

CAUTION
RISK OF ELECTRIC SHOCK
DRY LOCATION USE ONLY
ATTENTION
POUR UTILISATION A
L'INTERIEUR SEULEMENT
AC/DC ADAPTER
Model:DLL015122000D
Input:100-240V~50/60Hz 0.55A
Output: +12V ⎓ 2A
CAUTION
INDOOR USE ONLY
MADE IN CHINA

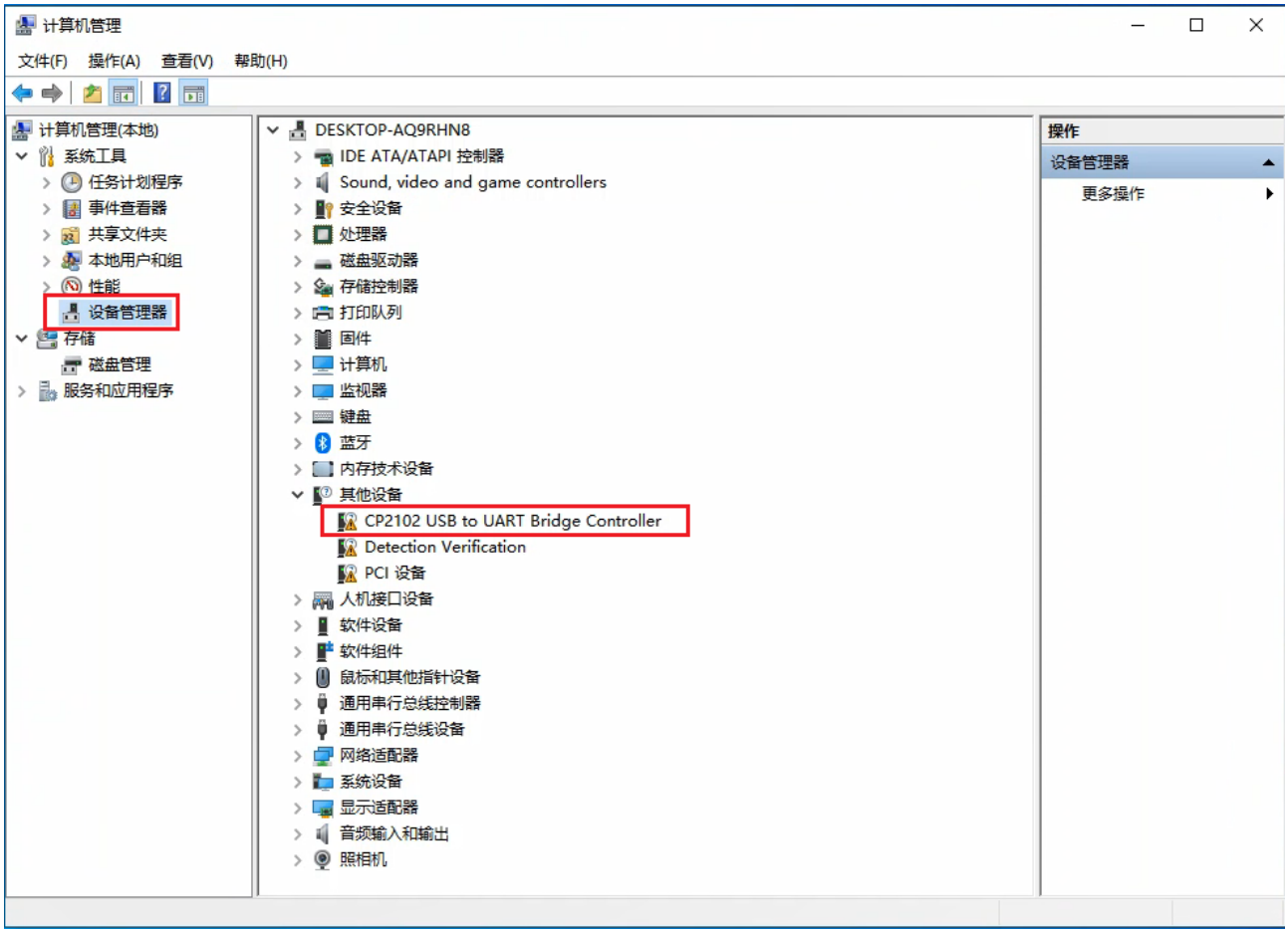
12V 2A电源

1. 右键此电脑-管理

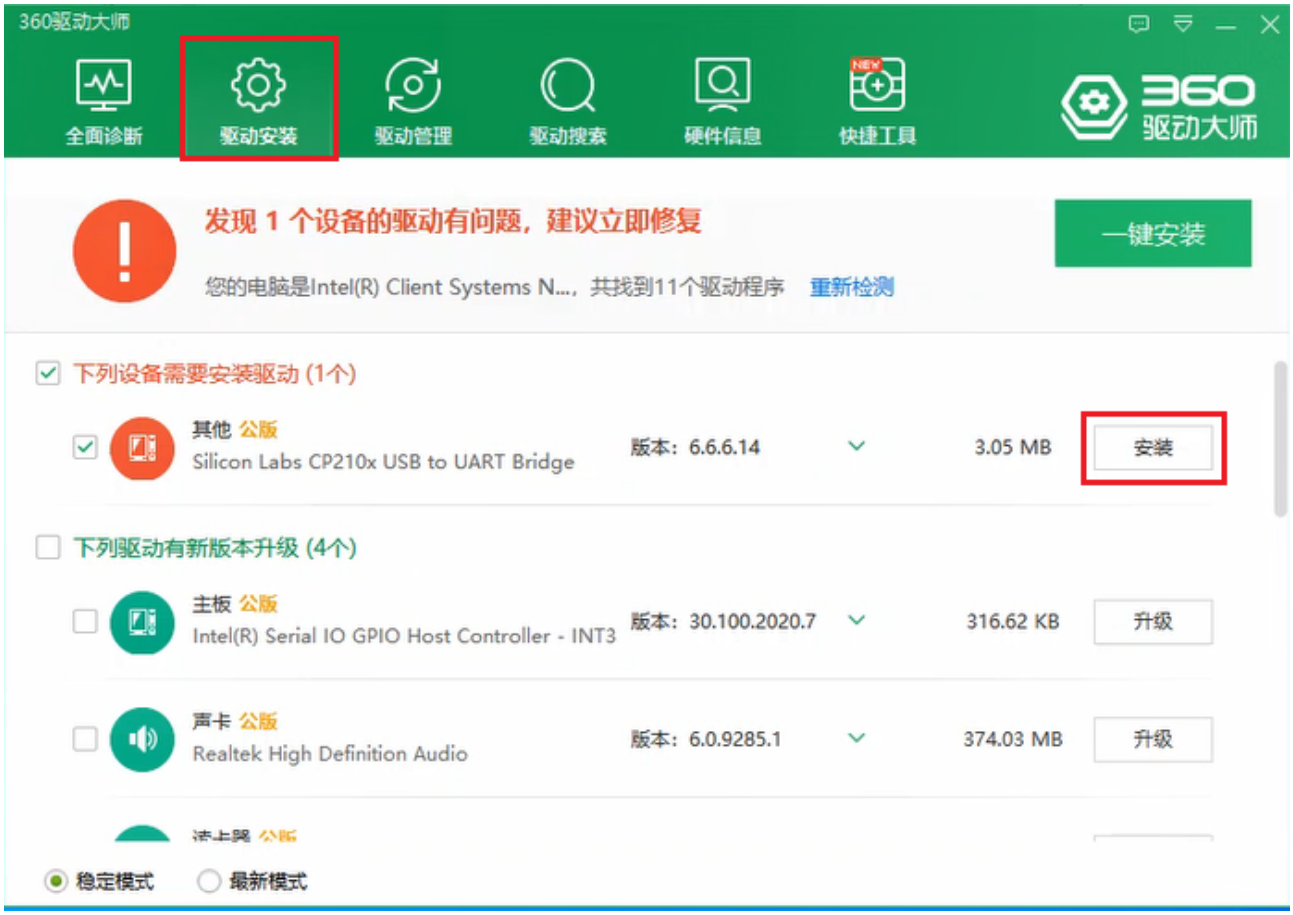

打开(O)
固定到快速访问
管理(G)
固定到"开始"屏幕(P)
映射网络驱动器(N)...
断开网络驱动器的连接(C)...
创建快捷方式(S)
删除(D)
重命名(M)
属性(R)

---

2. 可以看到其他设备下有一个 cp2102 的串口工具缺少驱动



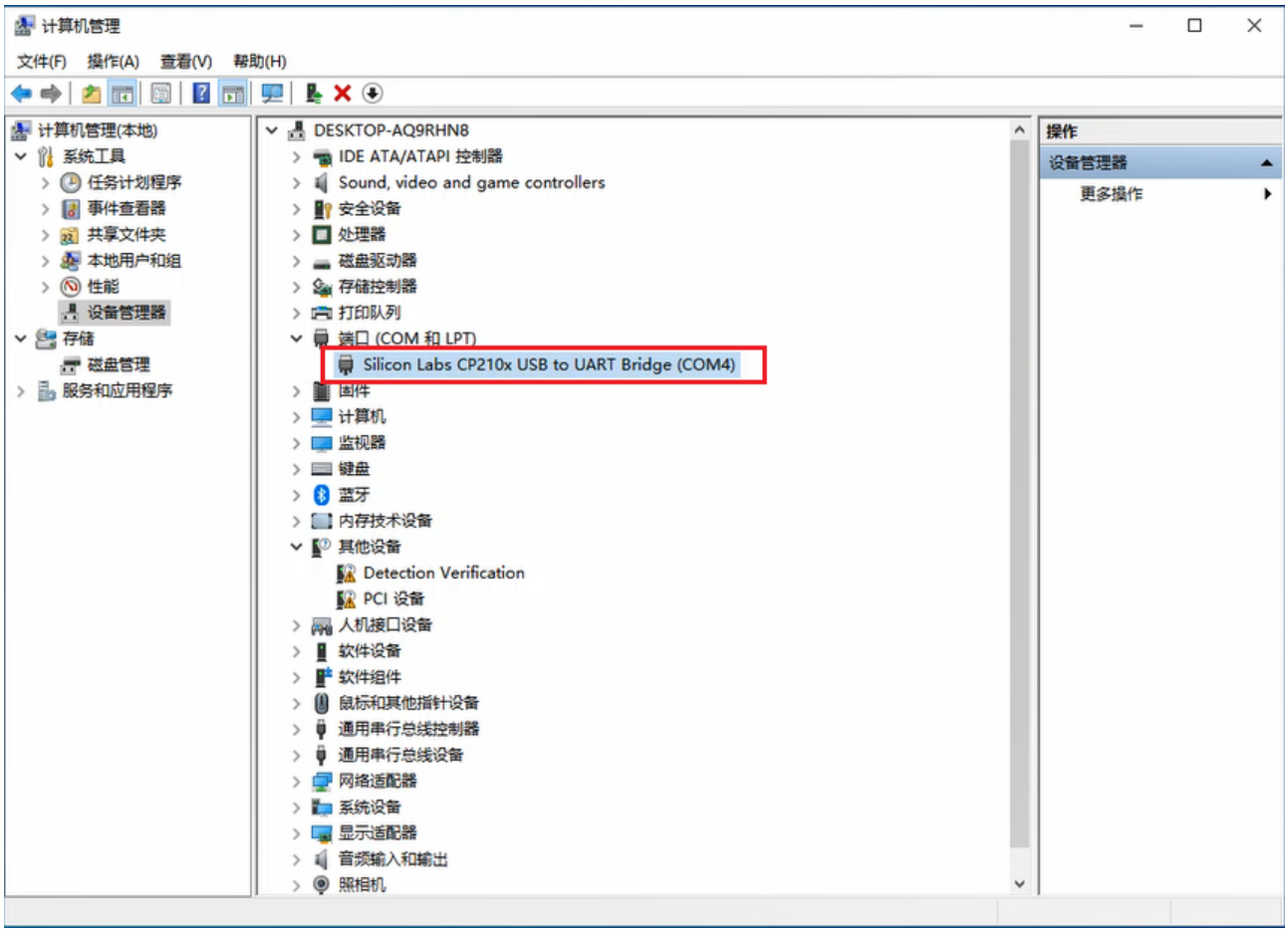
3. 推荐大家使用 360 驱动大师，选择驱动安装，会自动扫描出缺少的驱动，点击安装，即可自动完成安装



您也可以直接下载驱动 (驱动来自网络，淘晶驰不保证其可靠性和安全性)

常见串口驱动下载 (page 1104)

4. 安装完成后，可以在端口中看到 cp210x 设备，具体的端口号是 com4，请记住你自己电脑上对应的端口号



5. 连接下载器和屏幕

备注： 连接方式如下所示

5V 接串口屏 5V

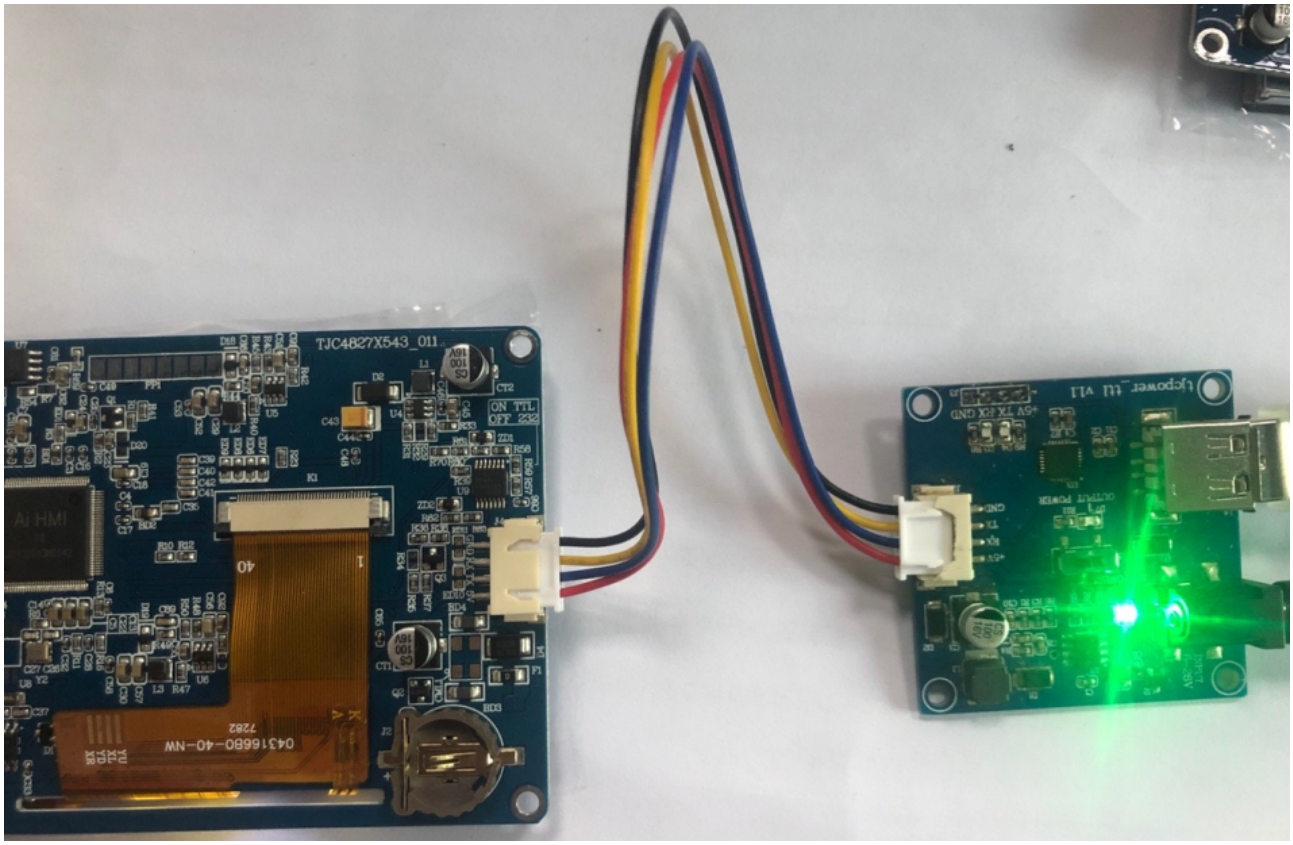
TX 接串口屏 RX

RX 接串口屏 TX

GND 接串口屏 GND

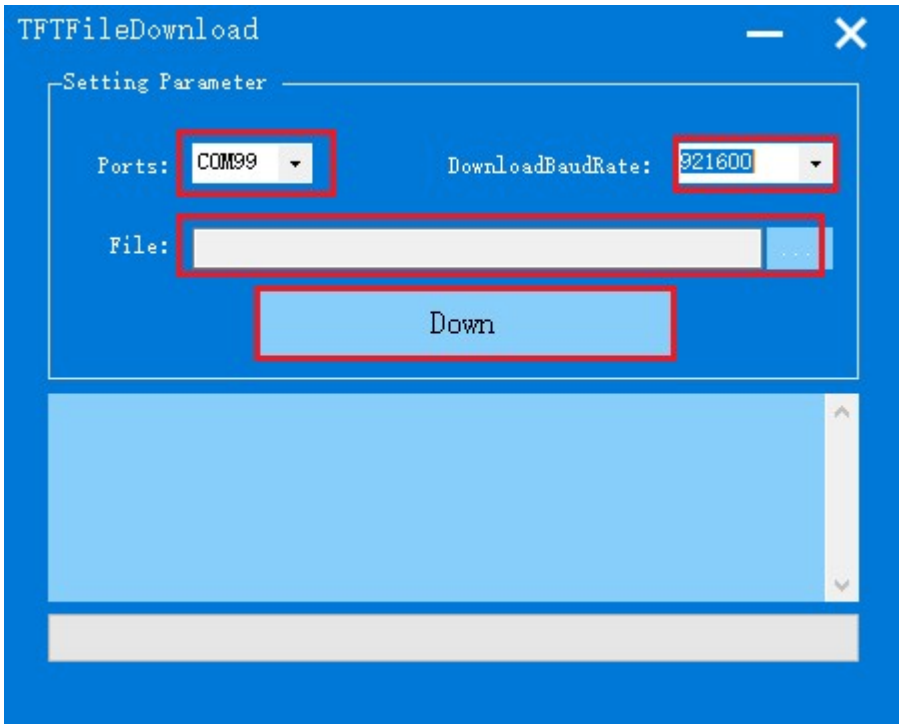
TX 是 Transmit（发送），RX 是 Receive（接收），发送对接收，接收对发送。

另外说明：有些板子和原理图标注的是 TXD 和 RXD。TX 就是 TXD，RX 就是 RXD。概念都是一样的，可以不做区分。



---

6. 打开 TFTFileDownload 软件，Ports 选择刚刚看到的串口号，DownloadBaudRate 选择最高的 921600,File 选择你需要下载的 TFT 文件，点击 Down 即可开始下载



注意： 如果下载失败，尝试给串口屏重新上电，然后降低波特率后再次下载

相关链接

TFT 文件如何下载到串口屏中 (page 1405)

如何输出 TFT 生产文件 (page 155)

11.5.18 只有屏/tft 文件, 或者源 HMI 工程丢了, 能反编译出 HMI 文件吗

不能, 这就和做菜一样, 你可以根据食谱做出一道菜, 但是如果想根据做好的菜反推出食谱, 厉害的人可以自己动手实现出来，将显示效果做得一模一样, 甚至做得比原本更好, 但是在代码上可能是完全不一样的实现方式。

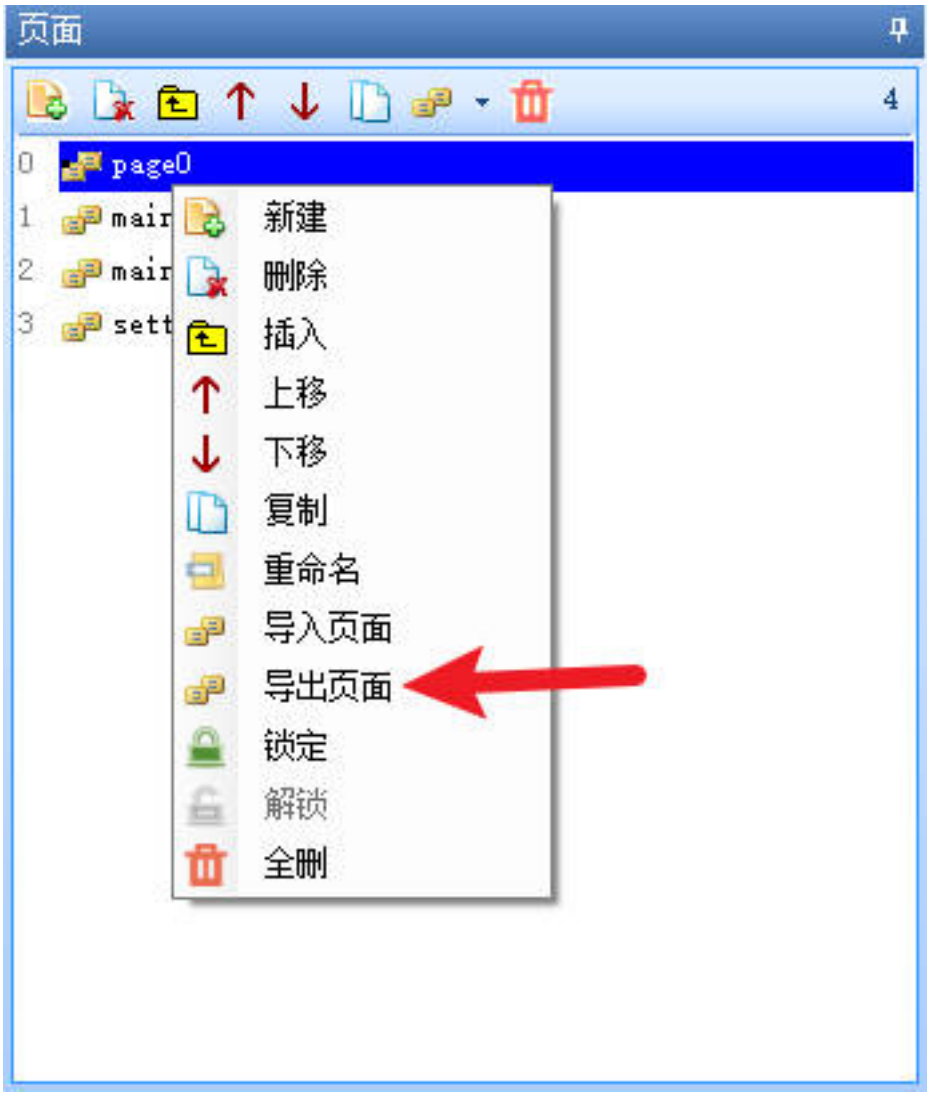
相关链接

TFT 文件如何下载到串口屏中 (page 1405)

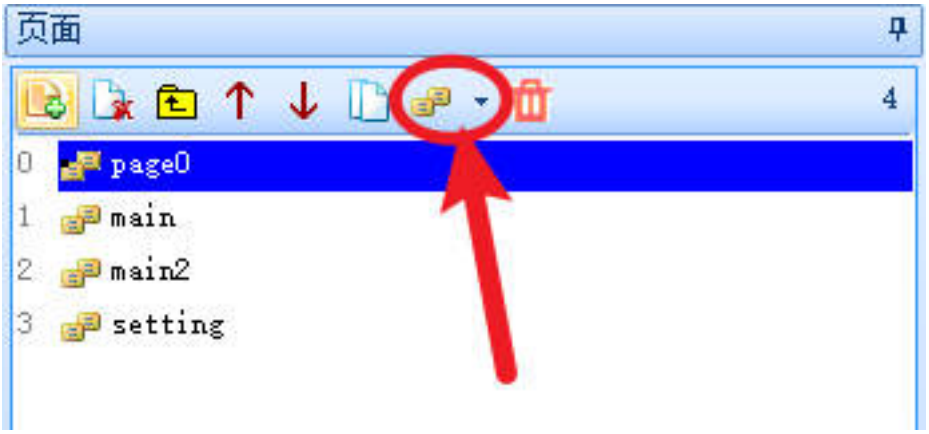
如何输出 TFT 生产文件 (page 155)

11.5.19 如何导入导出 page 文件

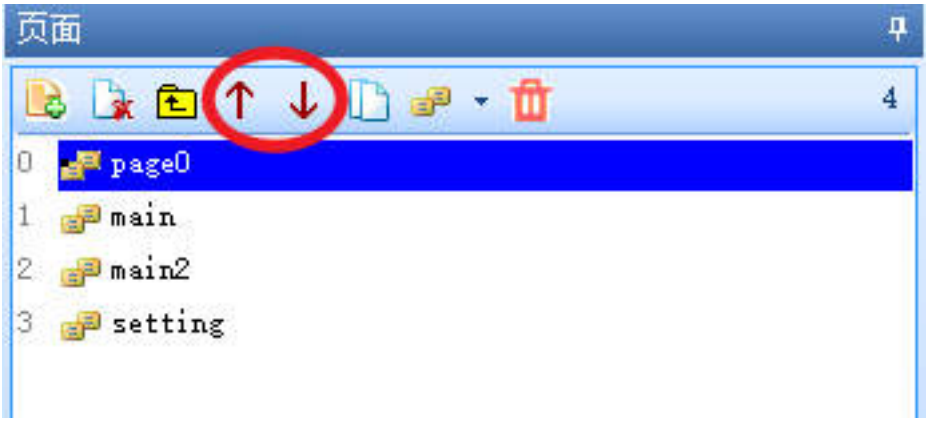
选中对应的页面，可以将对应的页面导出为 page 文件



也可以选择导入页面



导入页面后可以通过箭头修改页面顺序



11.5.20 屏幕保护制作

1，在页面上新建一个定时器

2，如果有任何按键操作，那么给这个定时器定时清零

3，如果一定时间没操作。跳转到屏保页面

4，屏保页面有操作，返回主页面

如果是在主动解析模式下, 有省电的需求, 可以在屏保页面的初始化中将 dim 设置为 0, 在退出界面中将 dim 设置为 100

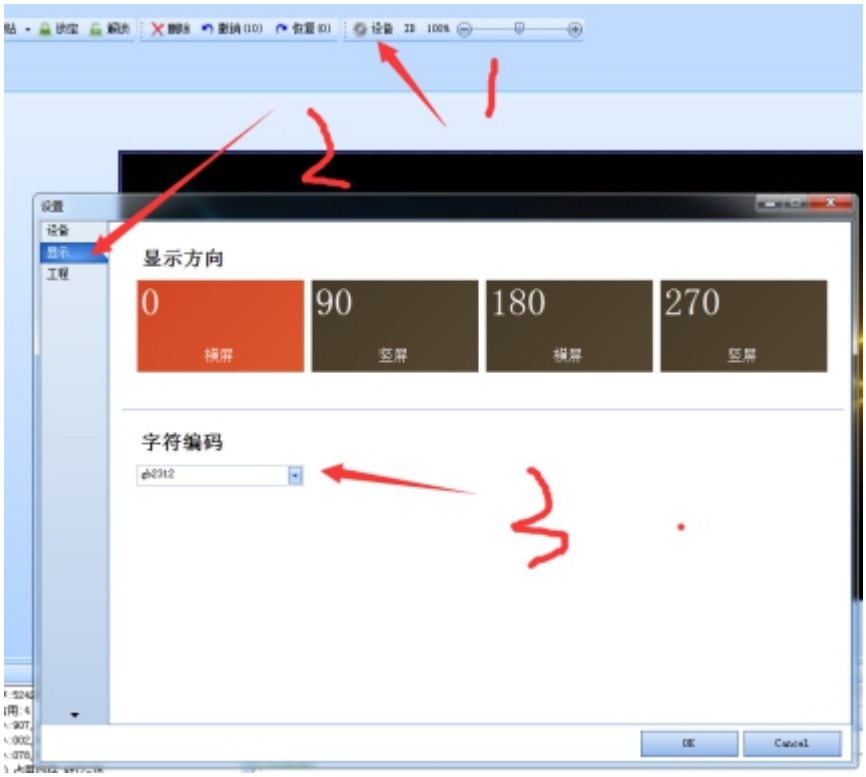
11.5.21 二维码控件是 gb2312 还是 utf8 编码

单片机发什么格式给二维码，二维码就是什么格式，二维码控件自己并不知道什么格式，他只管显示 hex 数值

11.5.22 修改工程编码为 utf8 或者 GB2312

> **注意:** 最好一开始就确定编码，不建议修改工程编码，会导致乱码等问题
>
> 如果一定要修改，修改前先备份工程，修改编码之后如果工程中有中文，中文会乱码，可以从备份的工程查找到相应的中文, 不要直接复制过去, 要在新的工程中手动输入, 否则直接复制过去的还是原来的编码, 保存后再打开工程会发现中文还是乱码

1. 修改工程的编码



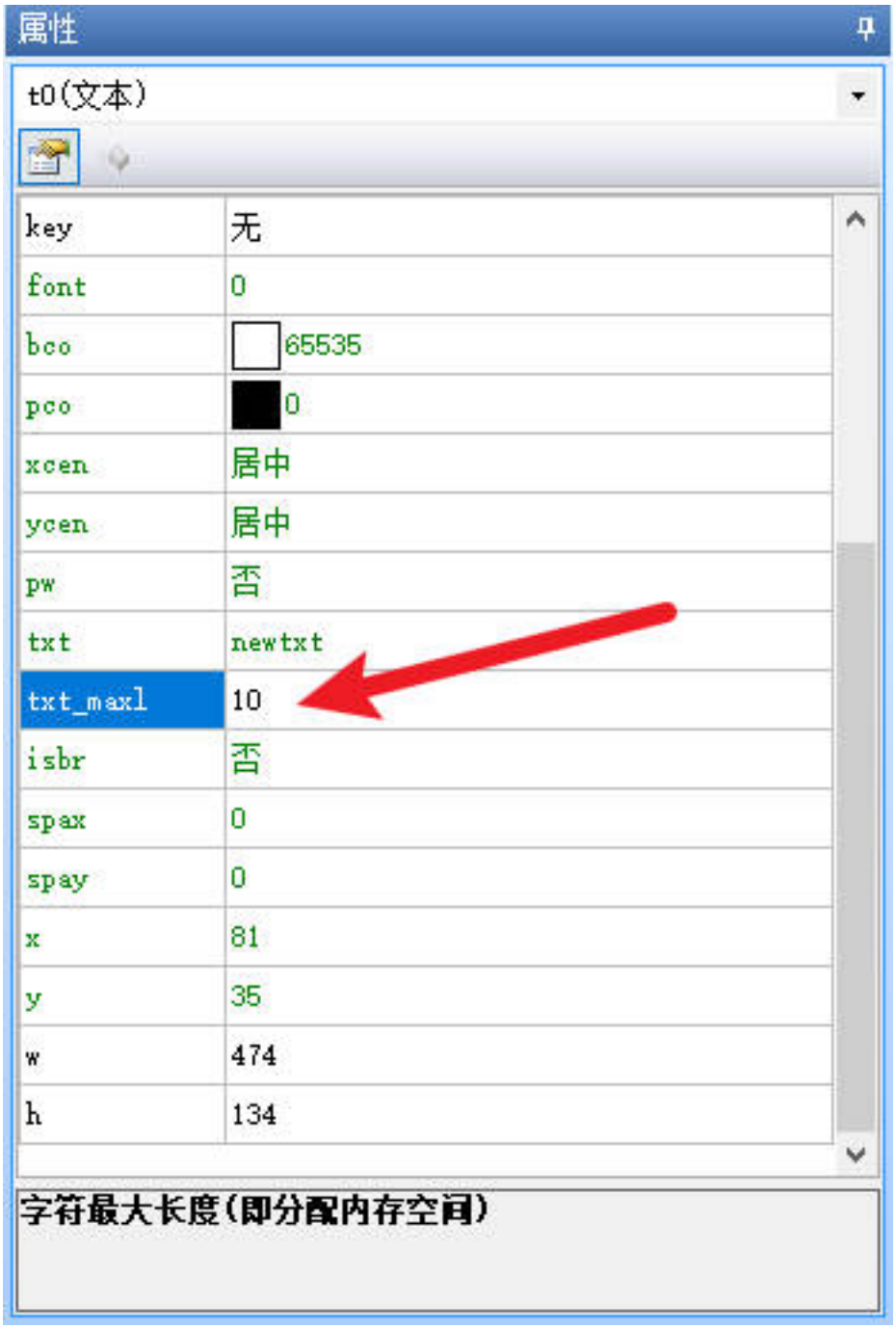
2. 删除原有的字库, 修改为 utf8 或 GB2312 字库 (需要勾选自己所需要的语言, 如果添加了所有语言, 字库会相当大)

3. 修改了编码后，很多地方（非 ASCII 编码的部分）都会乱码，需要重新输入文本

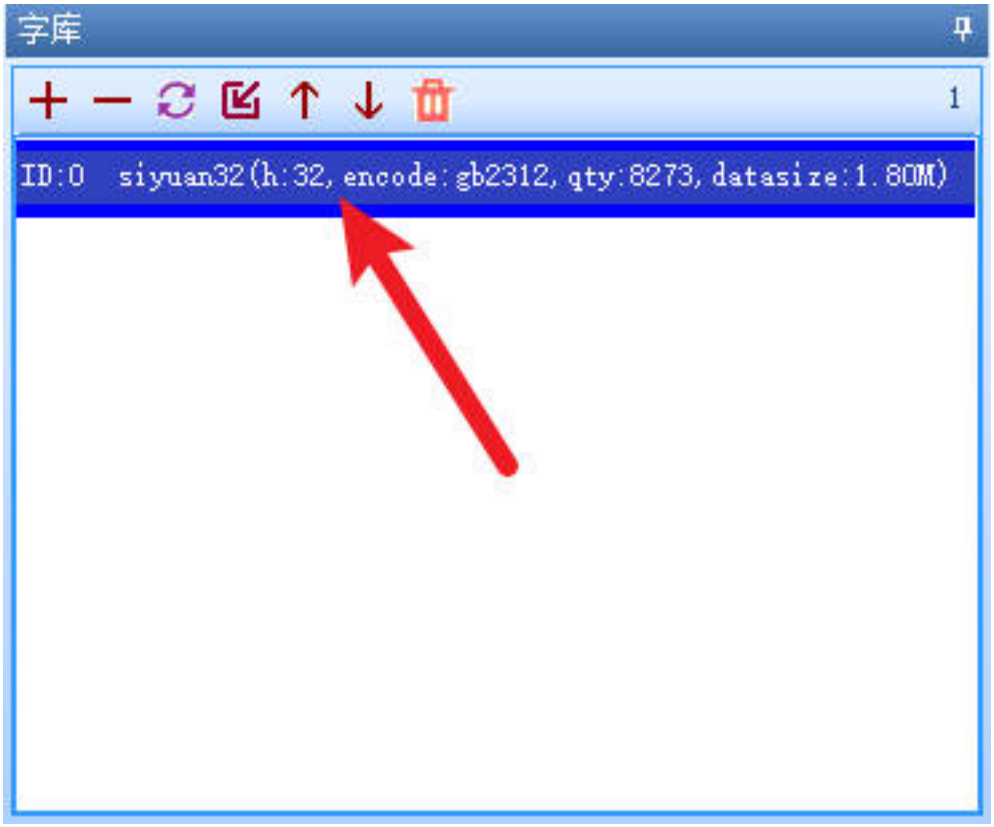
11.5.23 文本数据显示不完全

文本控件（或滚动文本控件等）的 txt_maxl 属性不够大或者字库中没有某些字

调整 txt_maxl 属性请修改这里



双击字库，可以查看字库中有哪些文字



11.5.24 键盘无法输入中文

不能简拼，必须全拼

例如输入淘晶驰三个汉字，不能直接输入 tjc

应该先输入 t a o ，选择“淘”

然后输入 j i n g , 选择“晶”

最后输入 c h i ，选择“驰”

11.5.25 单片机如何判断串口屏有没有接入

1、program.s 中使用 int 定义一个 timeout 变量

```
//以下代码只在上电时运行一次,一般用于全局变量定义和上电初始化数据
int sys0=0,sys1=0,sys2=0      //全局变量定义目前仅支持4字节有符号整形(int),
↪不支持其他类型的全局变量声明,如需使用字符串类型可以在页面中使用变量控件来实现
int timeout=0    //定义一个超时变量
bauds=115200      //配置波特率115200
dim=100          //配置亮度100
recmod=0         //关闭主动解析
bkcmd=0          //关闭错误信息返回
page 0                          //上电刷新第0页
```

2、串口屏中新建一个定时器 tm0, 将 objname 改为 tmTimeout，tim 改为 1000，en 设置为 1，在 tmTimeout 定时器中编写以下代码

```
timeout++
prints "usart hmi alive!"
if(timeout>10)
{
    page disconnected    //跳转到未连接页面
}
```

3、单片机定时向串口屏发送 timeout=0

单片机代码示例:

```
printf("timeout=0\xff\xff\xff");
```

11.5.26 实现按键锁定功能

相似功能：童锁

按下锁屏图标后，禁用所有控件的触摸事件，只有再次触摸解锁图标解锁才可以触摸其他按键

新建一个双态按钮，objname 属性改为“btn_lock”，txt 属性改为“锁定”，在 btn_lock 的按下事件编写以下代码

```
if(btn_lock.val==1)
{
    btn_lock.txt="解锁"
    tsw 255,0   //失能所有控件
    tsw btn_lock,1  //使能btn_lock
}else
{
    btn_lock.txt="锁定"
    tsw 255,1   //使能所有控件
}
```

11.5.27 串口屏的串口缓冲区大小

X2/X3/X5 的串口缓冲区是 4k 字节，主动解析模式下，u 数组的索引范围是 u[0]~u[4095]

T0/T1/K0 的串口缓冲区是 1k 字节，主动解析模式下，u 数组的索引范围是 u[0]~u[1023]

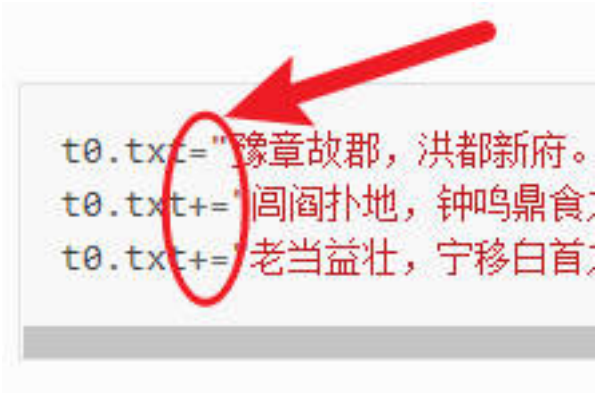
在淘晶驰被动解析模式下，一般不需要关心串口缓冲区的大小，需要注意的是以下几点

1. 发送数据的间隔，建议是每条指令间隔 100ms，避免发送过快导致屏幕解析不过来从而导致数据丢失。
2. 发送的文本数据时不要过长，例如发送以下文本（《滕王阁序》）全文

```
t0.txt="豫章故郡，洪都新府...（省略数百字）...请洒潘江，各倾陆海云尔。"\xff\xff\xff
```

x 系列有 4K 的串口缓冲区，足以放下整段数据，能够正常解析和显示。但是 t 系列或者 k 系列只有 1k 的缓冲区，接收到一半时串口缓冲区就满了，根本接收不到结束符\xff\xff\xff，导致无法显示，如果 t 系列和 k 系列想要正常显示，建议用”+=”进行拼接，方法如下

```
t0.txt="豫章故郡，洪都新府...（省略数百字）...山原旷其盈视，川泽纡其骇瞩。"\xff\xff\xff
t0.txt+="闾阎扑地，钟鸣鼎食之家...（省略数百字）...所赖君子见机，达人知命。"\xff\xff\xff
t0.txt+="老当益壮，宁移白首之心...（省略数百字）...请洒潘江，各倾陆海云尔。"\xff\xff\xff
```



11.5.28 主动解析下如何提取负数

负数在计算机中存储和传输的是补码，我们只需要通过补码求得原码即可，以下是不同数据长度的负数计算原码的方法

首先应确保数据过来的是小端模式

11.5.28.1 signed int8

一个 signed int8 的数据，如果发-1 过来，它发送的值其实是 255

要判断这个值是不是负数，只要判断这个值是不是大于 127 即可

如果这个数大于 127，就用这个数减去 256，就可以得到正确的负数

11.5.28.2 signed int16

一个 signed int16 的数据，如果发-1 过来，它发送的值其实是 65535

要判断这个值是不是负数，只要判断这个值是不是大于 32767 即可

如果这个数大于 32767，就用这个数减去 65536，就可以得到正确的负数

11.5.28.3 signed int32

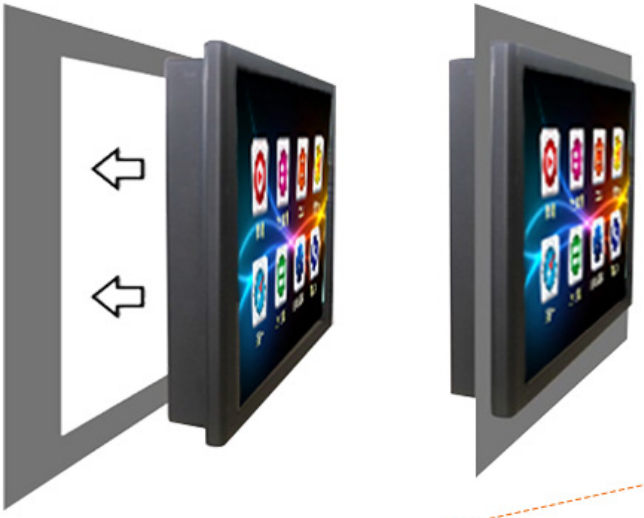
sint32 格式是最简单的，直接用 ucopy 指令即可

```
ucopy n0.val,0,4,0
```

11.5.29 带外壳产品安装示意图

01

将HMI嵌入所开的孔中



02

如图中所示 红色框中为卡扣安装孔

03

如图所示，将卡扣置入安装孔中旋紧螺丝，将HMI固定在设备上

11.5.30 显示比例不对，正方形变成长方形，圆型变成椭圆

这是因为有些型号的液晶颗粒不是方形的，是长方形的

11.5.30.1 3.9 寸屏 320*320

TAOJINGCHI

TJC3232T139_011C_F

3.9寸　　电容式触摸

串口屏基本参数

项目	参数
外形尺寸	82.80mm（L）× 82.80mm（W）× 3.92mm（H）
显示屏尺寸/铁框尺寸	71.14mm（L）× 69.82mm（W）
可视尺寸	71.94mm（L）× 70.62mm（W）
分辨率	320*320
液晶屏色彩	65K（65536）色，16 bit调色板，5R6G5B
显示方式	常黑，全透方式

以 3.9 寸屏 320*320 为例

宽度是 71.94mm, 像素 320, 单个像素宽度 0.2248mm

高度 70.62mm, 像素 320, 单个像素高度 0.2207mm

像素的宽高比为 0.2248/0.2207=1.0276

11.5.30.2 4 寸屏 480*480

TAOJINGCHI

TJC4848X540_011C_I_Y

4.0寸　　电容触摸

串口屏基本参数

项目	参数
外形尺寸	86.00mm（L）× 86.00mm（W）× 45.45mm（H）
显示屏尺寸	76.43mm（L）× 79.38mm（W）
可视尺寸	71.86mm（L）× 70.18mm（W）
分辨率	480*480
液晶屏色彩	65K（65536）色，16 bit调色板，5R6G5B
显示方式	常黑，全透方式

以 4 寸屏 480*480 为例

宽度是 71.86mm, 像素 480, 单个像素宽度 0.1497mm

高度 70.18mm, 像素 480, 单个像素高度 0.1462mm

像素的宽高比为 0.179/0.179=1.0239

11.5.30.3 4.3 寸屏 800*480

TAOJINGCHI

TJC8048X543_011N_I

4.3寸　　无触摸

串口屏基本参数

项目	参数
外形尺寸	120.00mm（L）× 74.00mm（W）×4.90mm（H）
显示屏尺寸	105.50mm（L）× 67.20mm（W）
可视尺寸	95.04mm（L）× 53.86mm（W）
分辨率	800*480
液晶屏色彩	65K（65536）色，16 bit调色板，5R6G5B
显示方式	常黑，全透方式

以 4.3 寸屏 800*480 为例

宽度是 95.04mm, 像素 800, 单个像素宽度 0.1188mm

高度 53.86mm, 像素 480, 单个像素高度 0.1122mm

像素的宽高比为 0.1188/0.1122=1.059

11.5.30.4 7 寸屏规格 800*480

TAOJINGCHI

TJC8048X570_011N

7.0寸　　无触摸

串口屏基本参数

项目	参数
外形尺寸	181.00mm（L）× 108.00mm（W）×7.7mm（H）
显示屏尺寸	164.90mm（L）× 100.00mm（W）
可视尺寸	154.08mm（L）× 85.92mm（W）
分辨率	800*480
液晶屏色彩	65K（65536）色，16 bit调色板，5R6G5B
显示方式	常黑，全透方式
可视角度（上/下/左/右）	典型值：50/70/70/70　最小值:40/60/60/60
触控次数	无触摸
背光模式	LED

以 7 寸屏 800*480 为例

宽度是 154.08mm, 像素 800, 单个像素宽度 0.1926mm

高度 85.92mm, 像素 480, 单个像素高度 0.179mm

像素的宽高比为 0.1926/0.179=1.075978

11.5.30.5 7 寸屏规格 1024*600

TAOJINGCHI

TJC1060X570_011R_I

7寸　　电阻触摸

串口屏基本参数

项目	参数
外形尺寸	181.00mm（L）× 108.00mm（W）×7.10mm（H）
显示屏尺寸	164.90mm（L）× 100.00mm（W）
可视尺寸	154.08mm（L）× 85.92mm（W）
分辨率	1024*600
液晶屏色彩	65K（65536）色，16 bit调色板，5R6G5B
显示方式	常黑，全透方式
可视角度（上/下/左/右）	典型值:80/80/80/80 最小值：70/70/70/70
触控次数	不低于100万次

以 7 寸屏 1024*600 为例

宽度是 154.08mm, 像素 1024, 单个像素宽度 0.1543mm

高度 85.92mm, 像素 600, 单个像素高度 0.1432mm

像素的宽高比为 0.1543/0.1432=1.0777

11.5.30.6 10.1 寸屏 1024*600

TAOJINGCHI

TJC1060X5A1_011C

10.1寸　　电容触摸

串口屏基本参数

项目	参数
外形尺寸	258.00mm（L）× 152.00mm（W）×11.70mm（H）
显示屏尺寸	235.00mm（L）× 143.00mm（W）
可视尺寸	222.72mm（L）× 125.28mm（W）
分辨率	1024*600
液晶屏色彩	65K（65536）色，16 bit调色板，5R6G5B
显示方式	常黑，全透方式
可视角度（上/下/左/右）	典型值：50/70/70/70 最小值:40/60/60/60

以 10.1 寸屏 1024*600 为例

宽度是 222.72mm, 像素 1024, 单个像素宽度 0.2175mm

高度 125.28mm, 像素 600, 单个像素高度 0.2088mm

像素的宽高比为 0.2175/0.2088=1.042

11.5.30.7 原因说明

在横屏的情况下，因为像素的宽度要大于像素的高度，因此显示的图形会变扁

如果要显示正常的图形比例，作图时把素材中的圆形和正方形提前略微拉高（高度大约为宽度的 1.04-1.07 倍），与像素点的比例互相抵消，此时就正常了

11.5.31 发送中文汉字相关问题

11.5.31.1 keil 编译的程序“中文 (汉字) 显示不全、串口发送中文汉字接收后乱码缺失”问题

https://blog.csdn.net/qq_34824576/article/details/79292584

今天知道一个keil的bug，就是当汉字的机内码包含0xfd时，keil就会自动丢弃fd，mark一下。

解决方法：

```
1、遇到带有fd的汉字采用机内码替代，
2.给keil打补丁。具体过程可百度：keil+汉字+FD
```

解决方法是在出现问题的字体后面加上 xFD

如：unsignedchar code s[]=”数学”；改为 unsignedchar code s[]=”数 xFD 学”；即可正常显示。

下面是从 GB2312 中挑出来的相关汉字：

褒饼昌除待谍洱俘庚过糊积箭烬君魁例笼慢谬凝琵讫驱三升数她听妄

锡淆旋妖引育札正铸佚冽邶埤荤蔟摭啐帻猃恺泯潺躯纨琮絮辇挛臊

忑睚铨稞瘕颀螨簸酏觚鳊鼾

替代方法：

比如我做一个触摸热区，按下事件里写“电机异常”，弹起事件里写“电机正常”，把这个触摸热区屏幕外（避免被误触发），然后用单片机发送 click 去触发他

11.5.31.2 arduino 发送中文时乱码

这是因为 arduino 默认是 utf8 编码，而串口屏默认 gb2312 编码

解决方法 1：用记事本打开你的代码文件 另存为页面 在保存按钮旁边有个编码 修改编码为 ANSI 即可

解决方法 2：将屏幕的编码修改为 utf8（相应的所有字库也要改为 utf8 字库），但是注意在 utf8 模式下暂时不支持中文输入法

11.5.31.3 单片机发送中文到屏幕上，但是屏幕出现乱码

大部分原因是因为编码方式不统一照成的，大部分情况时单片机默认 gb2312 编码，但是新建屏幕工程时选择了 utf8 编码，另外请注意，屏幕工程的编码与字库的编码也要一致

参考链接：*修改工程编码为 utf8 或者 GB2312* (page 1418)

11.5.32 为什么关闭小板上的电源时，RX 灯会亮起

为什么关闭小板上的电源时，RX 灯会亮起

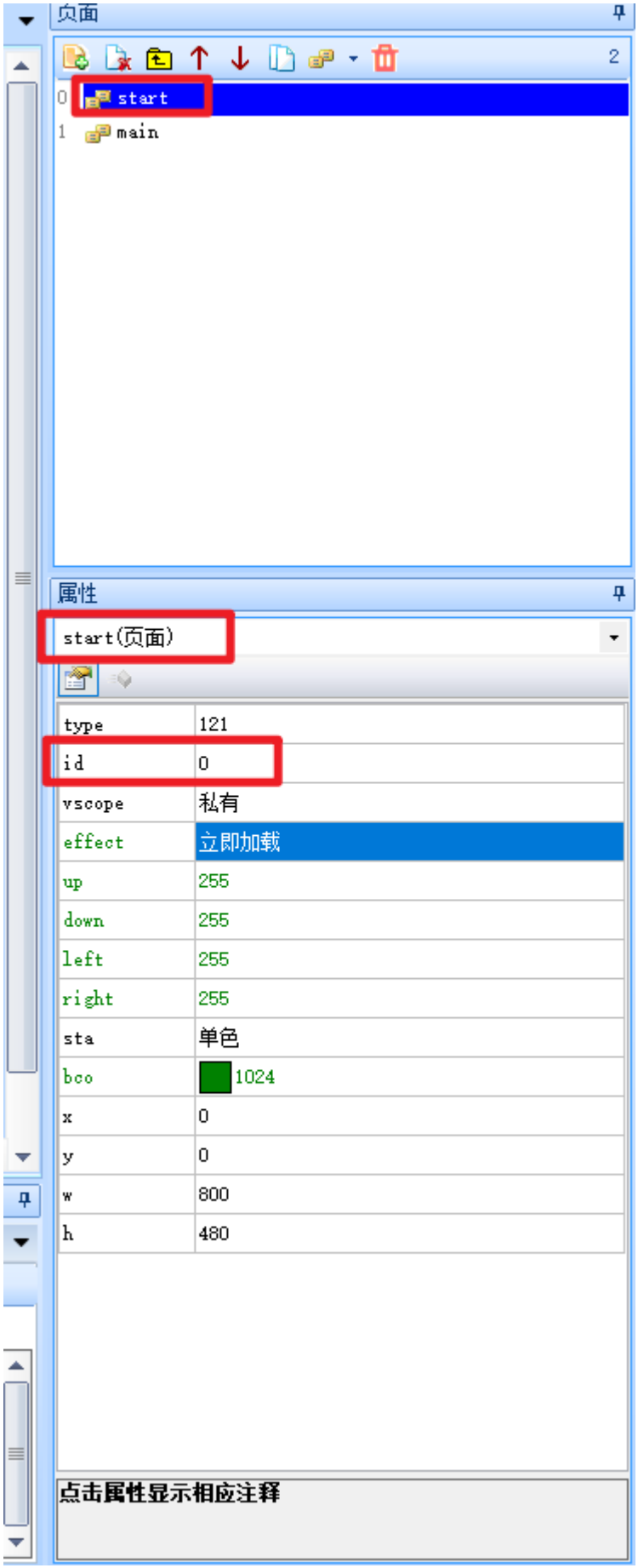
当关闭小板电源时，只是关闭了 5V 所在的电源，串口并没有断开，因此此时小板 TX 是推挽输出，串口空闲状态为高电平，RX 是浮空输入，因此 RX 容易被单片机屏拉低，电流是由电源通过限流电阻、发光二极管流入 RX，所以此时 RX 灯会亮

11.5.33 变量控件名和页面名称可以一样吗

不可以和控件所在页面名称相同（报错：名称重复），但是可以和其他页面名称相同一样。

这是因为在创建页面时，会在页面里自动创建一个同名的页面控件（id 号固定为 0，点击页面空白的地方就能看到）

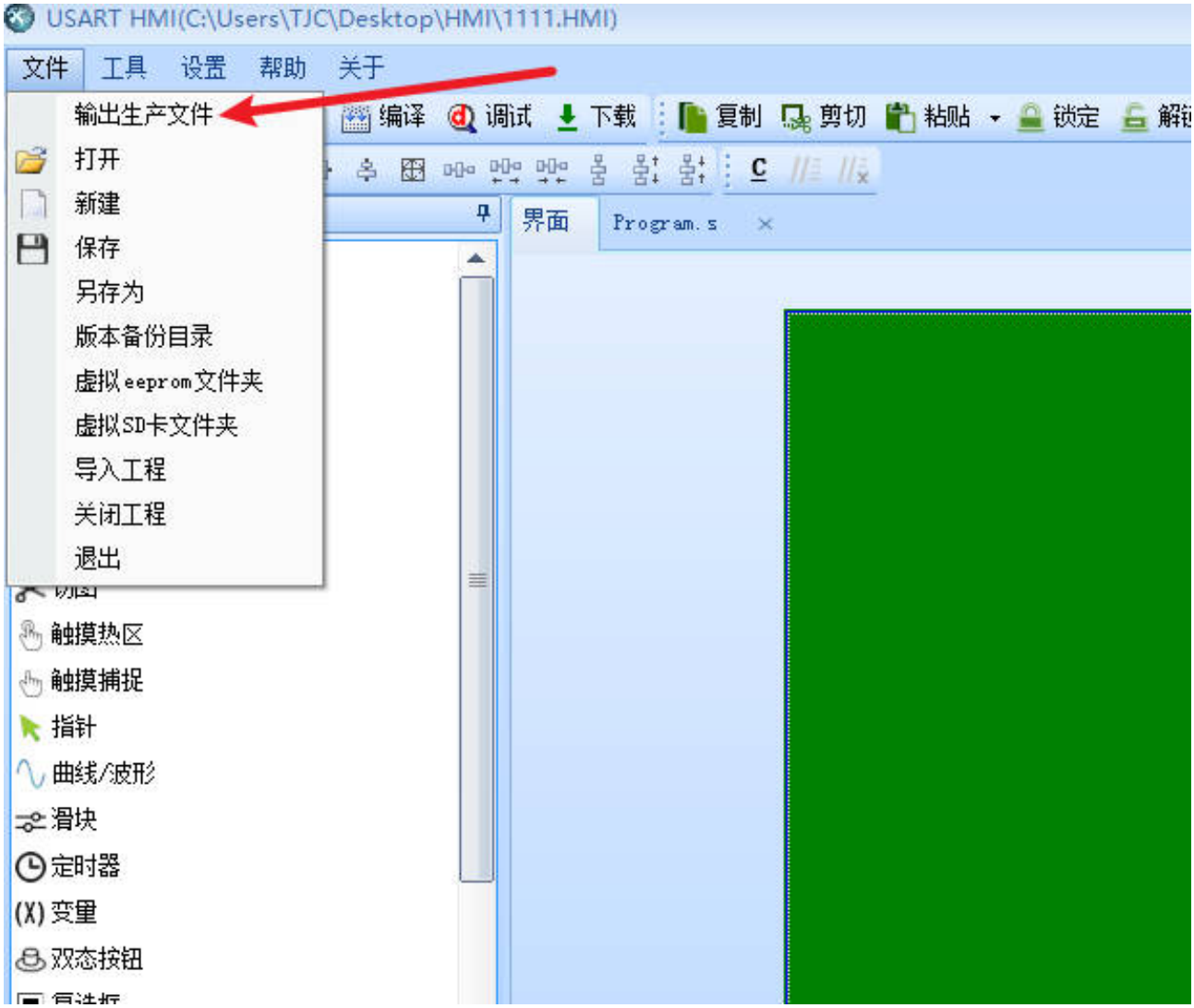
用于可以设置页面的基本属性，如页面背景，页面特效等


页面
2
0 start
1 main
属性
start(页面)
type 121
id 0
vscope 私有
effect 立即加载
up 255
down 255
left 255
right 255
sta 单色
bco 1024
x 0
y 0
w 800
h 480
点击属性显示相应注释

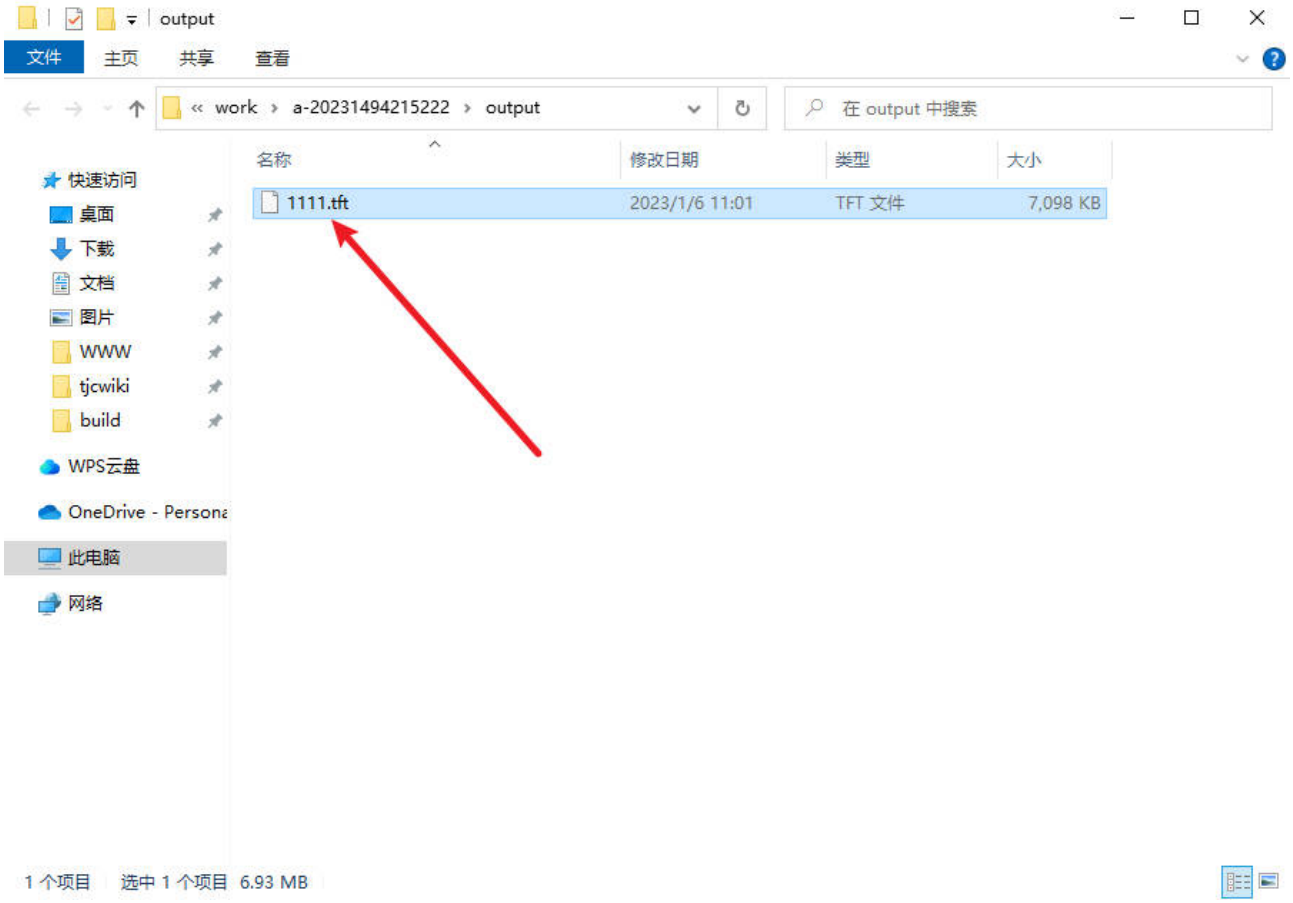
11.5.34 如何使用 TFT 文件下载助手 (TFTFileDownload) 下载工程

TFTFileDownload 工具是淘晶驰开源的下载工具，您可以下载源码进行二次开发，此工程基于 visual studio 进行开发，使用的语言为 c#，需要您熟练掌握 c# 语言，淘晶驰不对 TFTFileDownload 二次开发提供技术支持

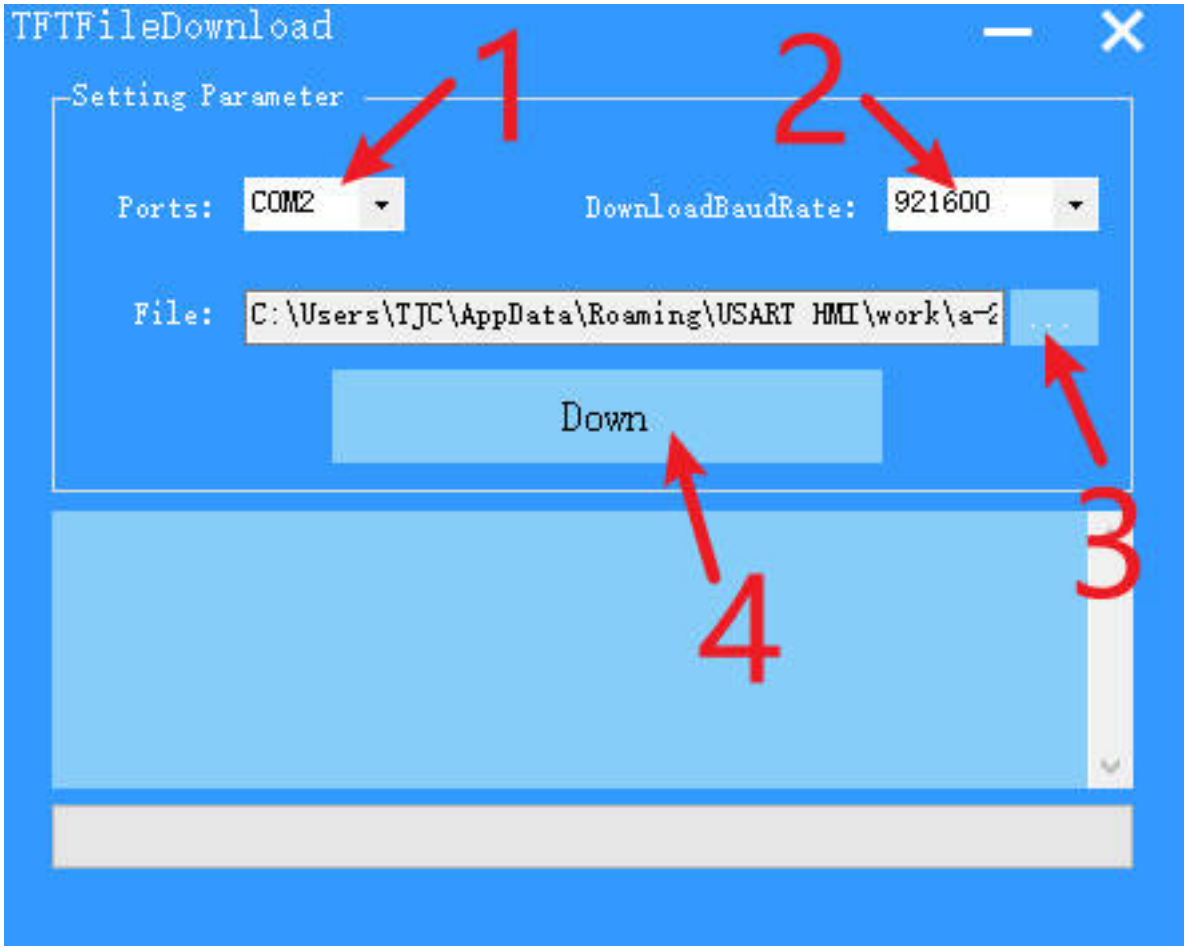
TFTFileDownload 下载地址：*TFT 文件下载助手 (TFTFileDownload)* (page 1097)



点击输出，如果此时跳出“拒绝访问”，则建议退出电脑上的 360 杀毒软件（或其他杀毒软件），再重新输出即可



这个 tft 文件就是生产文件，可以放 SD 卡的根目录进行烧写（通过 SD 卡下载工程到串口屏 (page 155)），也可以使用 TFTFileDownload 通过 usb 转 ttl 进行下载



第一步选择正确的端口号，需要您正确安装串口驱动。(参考*安装串口驱动* (page 138))

第二步选择下载波特率。(下载波特率和通讯波特率是分开的，ttl/485 下建议直接 921600,232 下载时不建议超过 256000)，*为什么 232 通讯不能超过 256000* (page 1265)

第三步选择要下载的 TFT 文件。

第 4 步点击 Dwon 按钮进行下载。

如果下载失败请检查您的 USB 转 TTL 工具的芯片是否为 ch340，(参考：*为什么不建议用 ch340* (page 1248))

11.5.35 单片机 3.3V 的串口可以直接接串口屏的串口吗

兼容 ttl 和 232 的型号（如 X3、X5 系列），串口电压为 5V

其他系列（如 X2、T0、T1、K0 系列）通常为 3.3V

如果您的单片机串口电压是 3.3V，需要连接 X3、X5 系列串口屏，可以先查看单片机的规格书，检查该串口 IO 是否是 5V 耐受的。

如果是 5V 耐受的，可以直接连接即可，如果不是 5V 耐受的，可以在单片机的 RX 和串口屏的 TX 之间串联一个 1k 电阻

如果您还有担忧，可以增加串口电平转换电路。

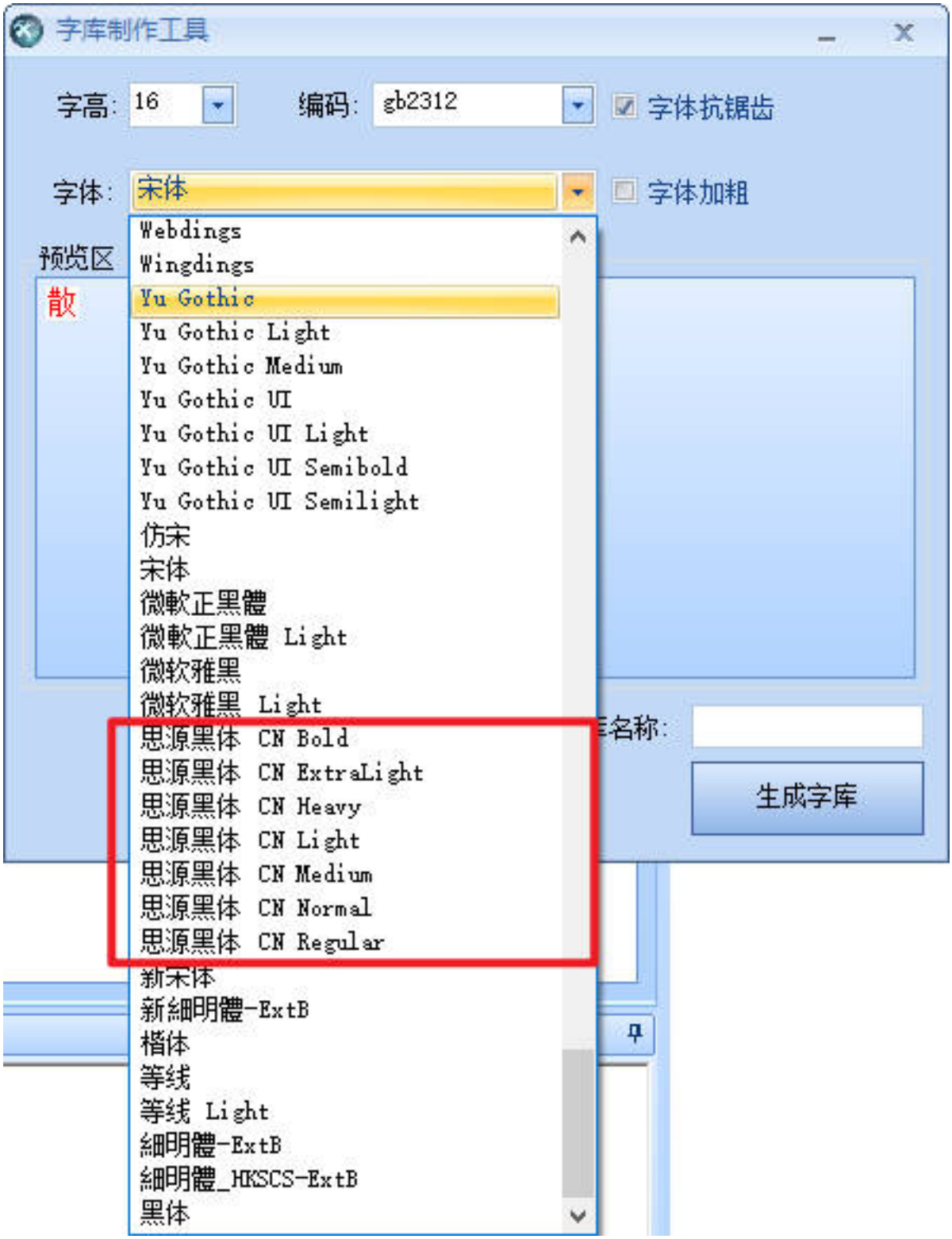
为什么 232 通讯不能超过 256000 (page 1265)

11.5.36 如何安装新字体

1、请务必使用.ttf 格式的字体

2、双击打开字体文件, 点击安装即可

3、重启上位机 USART HMI，就能在新建字库时选择已经安装的字体。

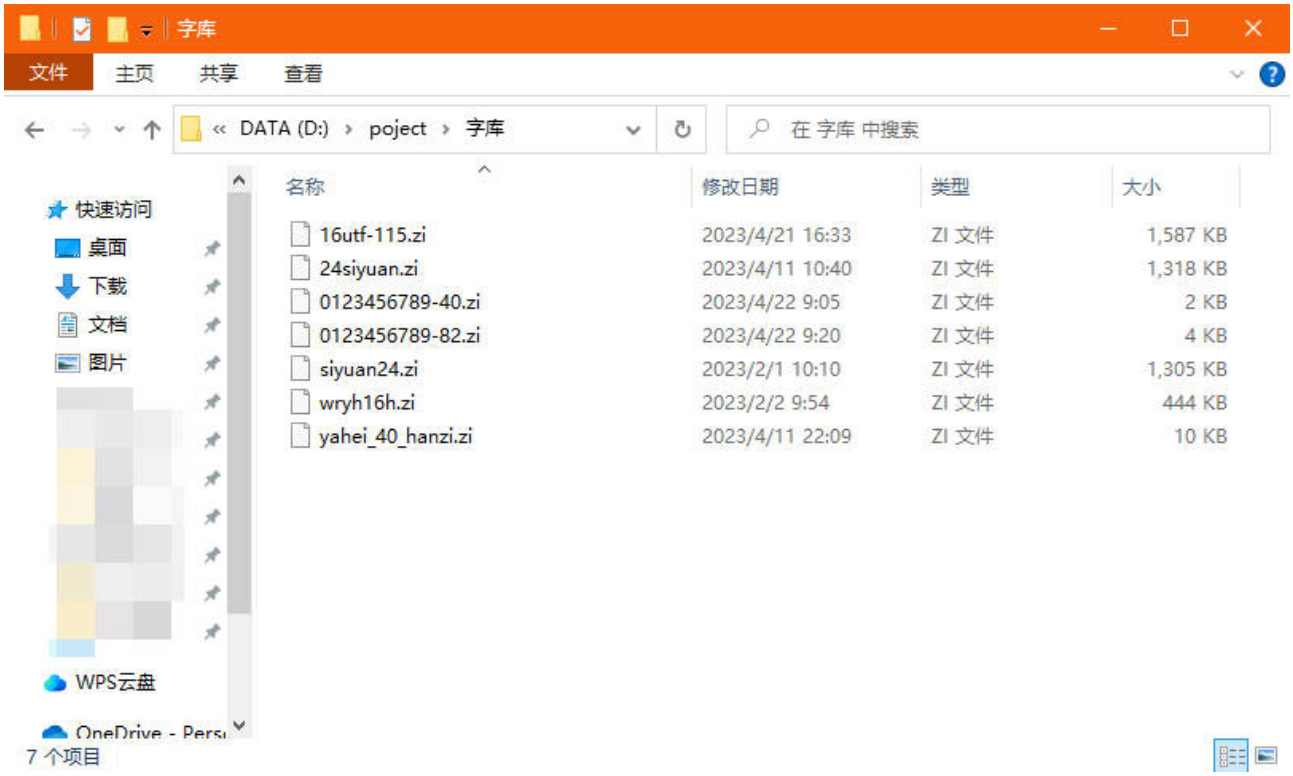

字库制作工具
字高: 16
编码: gb2312
字体抗锯齿
字体: 宋体
字体加粗
预览区
散
Webdings
Wingdings
Yu Gothic
Yu Gothic Light
Yu Gothic Medium
Yu Gothic UI
Yu Gothic UI Light
Yu Gothic UI Semibold
Yu Gothic UI Semilight
仿宋
宋体
微軟正黑體
微軟正黑體 Light
微软雅黑
微软雅黑 Light
思源黑体 CN Bold
思源黑体 CN ExtraLight
思源黑体 CN Heavy
思源黑体 CN Light
思源黑体 CN Medium
思源黑体 CN Normal
思源黑体 CN Regular
新宋体
新細明體-ExtB
楷体
等线
等线 Light
細明體-ExtB
細明體_HKSCS-ExtB
黑体
名称:
生成字库

11.5.36.1 如何安装新字体-相关链接

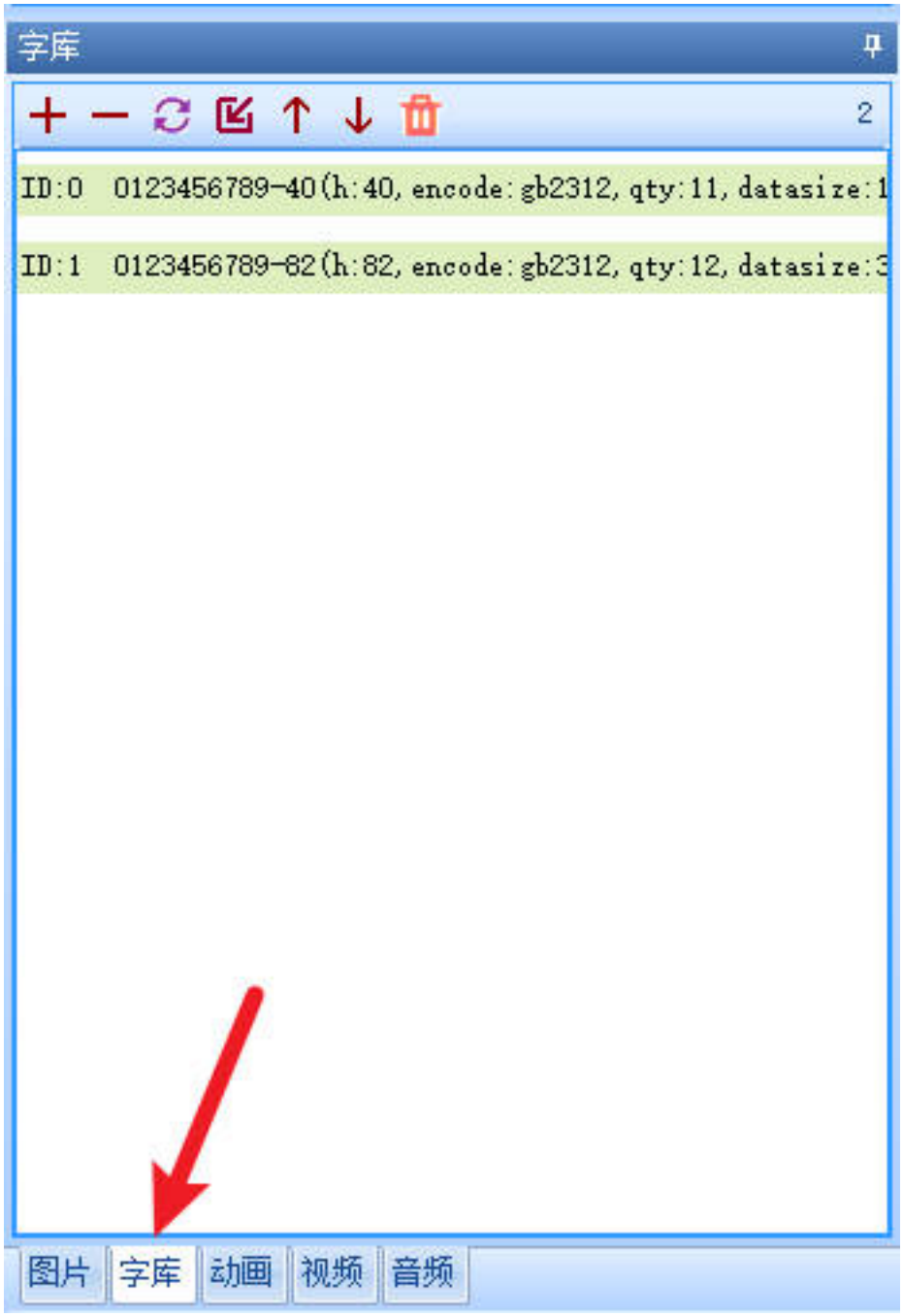
创建字库和导入字库 (page 125)
指定字库 (page 1284)
编译报错：*XXX* 初始值无效 (page 1342)
utf8 字库下如何只选择汉字 (page 1403)
在做字库的时候有部分字体无法选择 (page 1360)
思源黑体下载 (page 1211)

11.5.37 如何导入字库

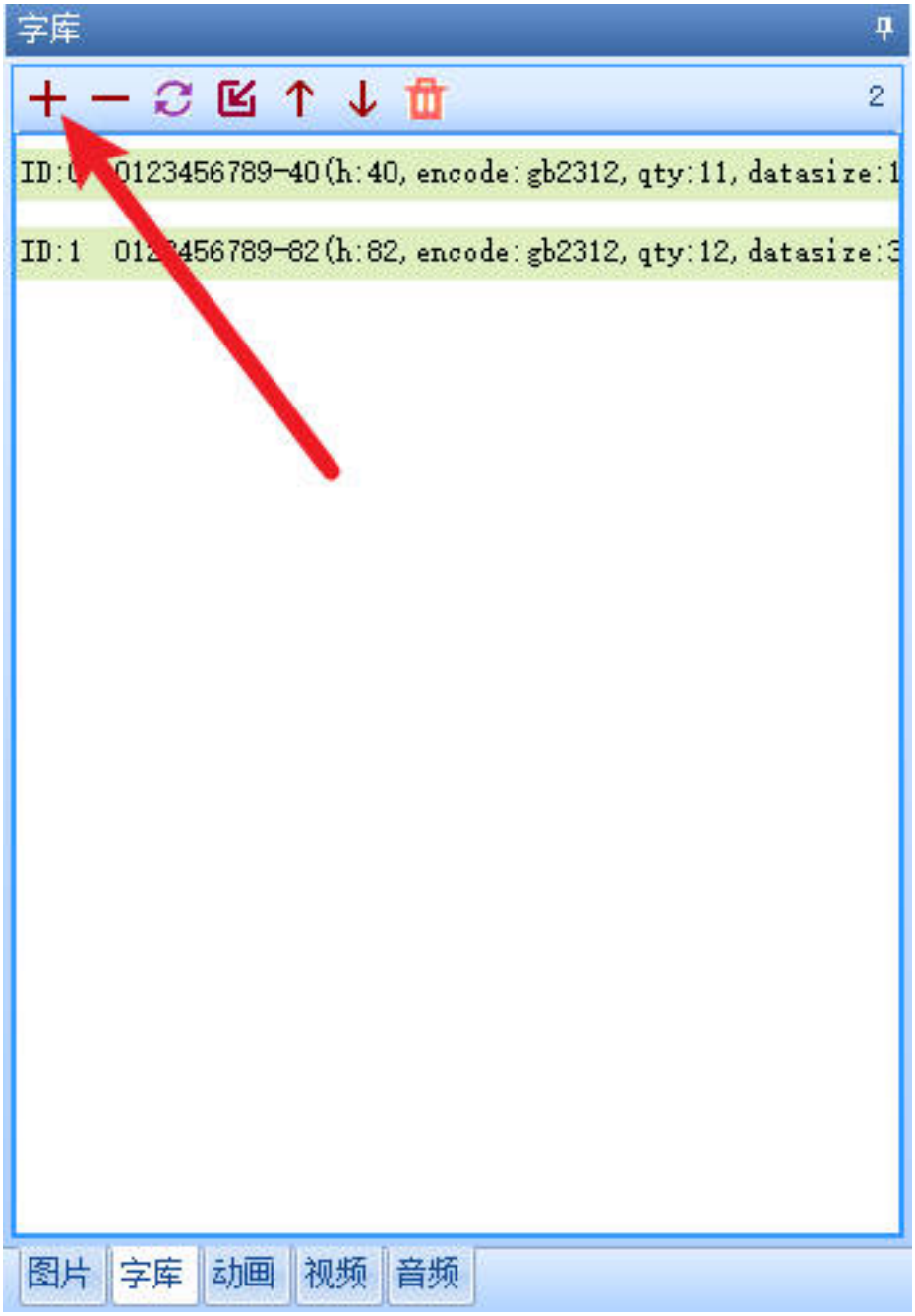
已经创建好的字库是.zi 文件，需要创建字库请参考创建字库和导入字库 (page 125) 指定字库 (page 1284)



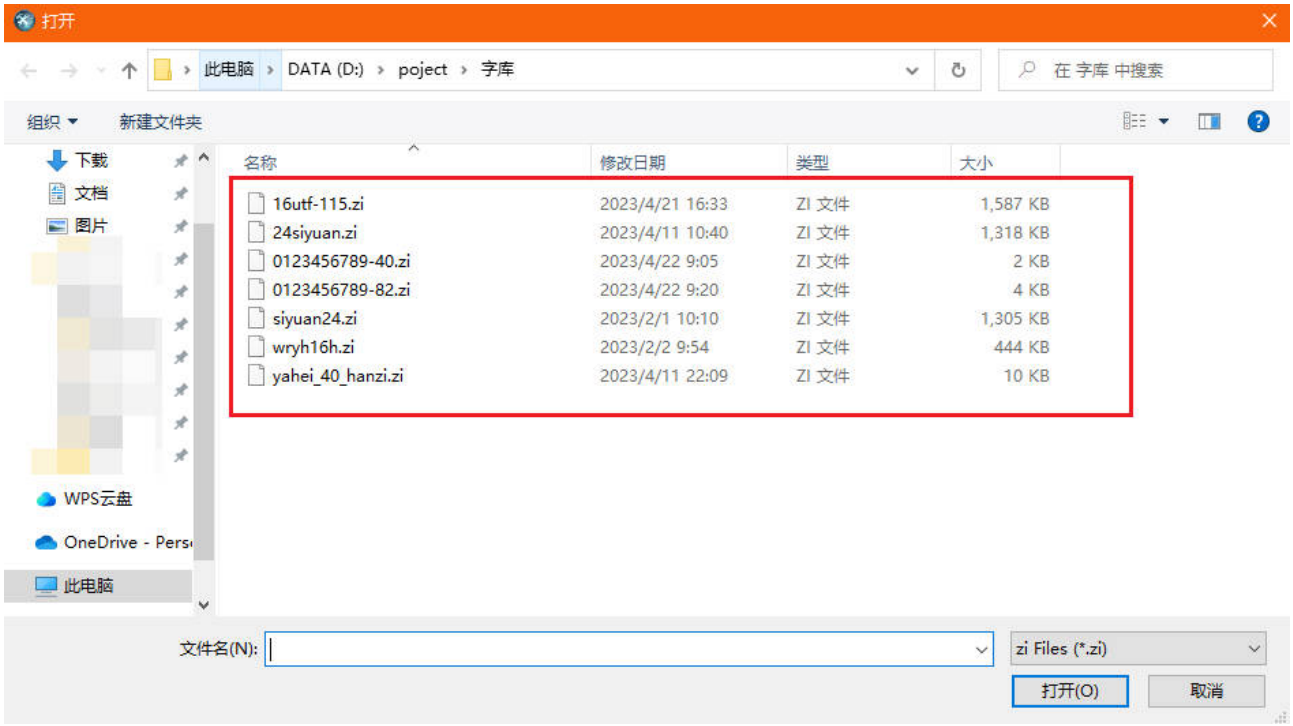
1、切换到”字库”


字库
2
ID:0 0123456789-40(h:40,encode:gb2312,qty:11,datasize:1
ID:1 0123456789-82(h:82,encode:gb2312,qty:12,datasize:3
图片
字库
动画
视频
音频

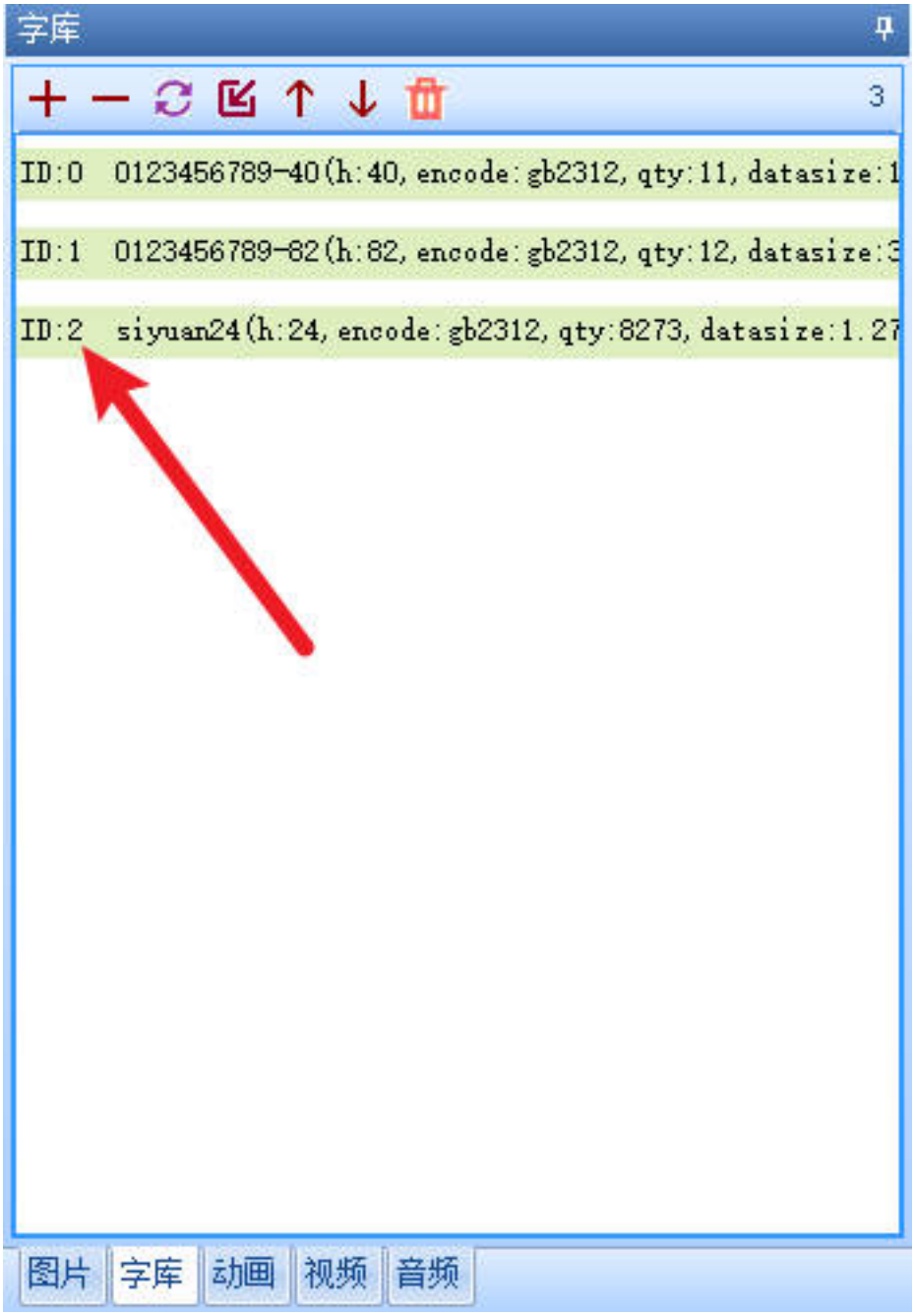
2、点击”+”号



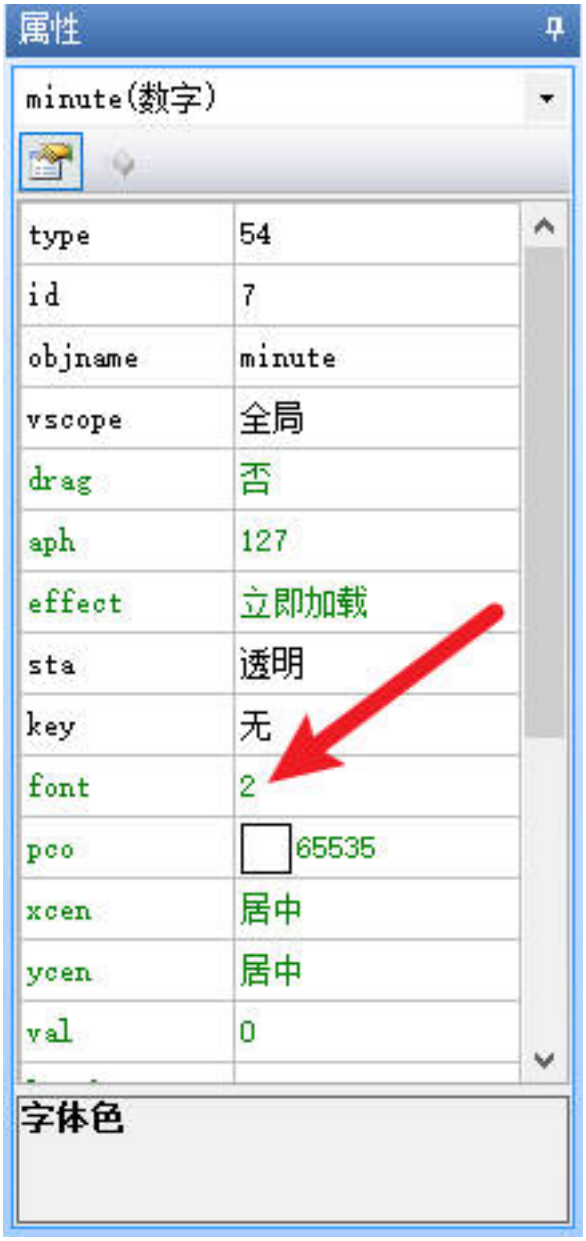
3、选择需要导入的字库, 点击打开，即可导入到工程中



4、新导入的字库, 其字库 id 是 2



5、选中需要修改的控件, 修改控件的 font 属性为对应的字库 id, 然后回车, 即可完成修改



11.5.37.1 如何导入字库-相关链接

如何导入字库 (page 1442)

如何导入图片 (page 1449)

如何导入动画 (page 1455)

如何导入视频 (page 1461)

如何导入音频 (page 1467)

如何安装新字体 (page 1439)

创建字库和导入字库 (page 125)

指定字库 (page 1284)

编译报错：*XXX* 初始值无效 (page 1342)

utf8 字库下如何只选择汉字 (page 1403)

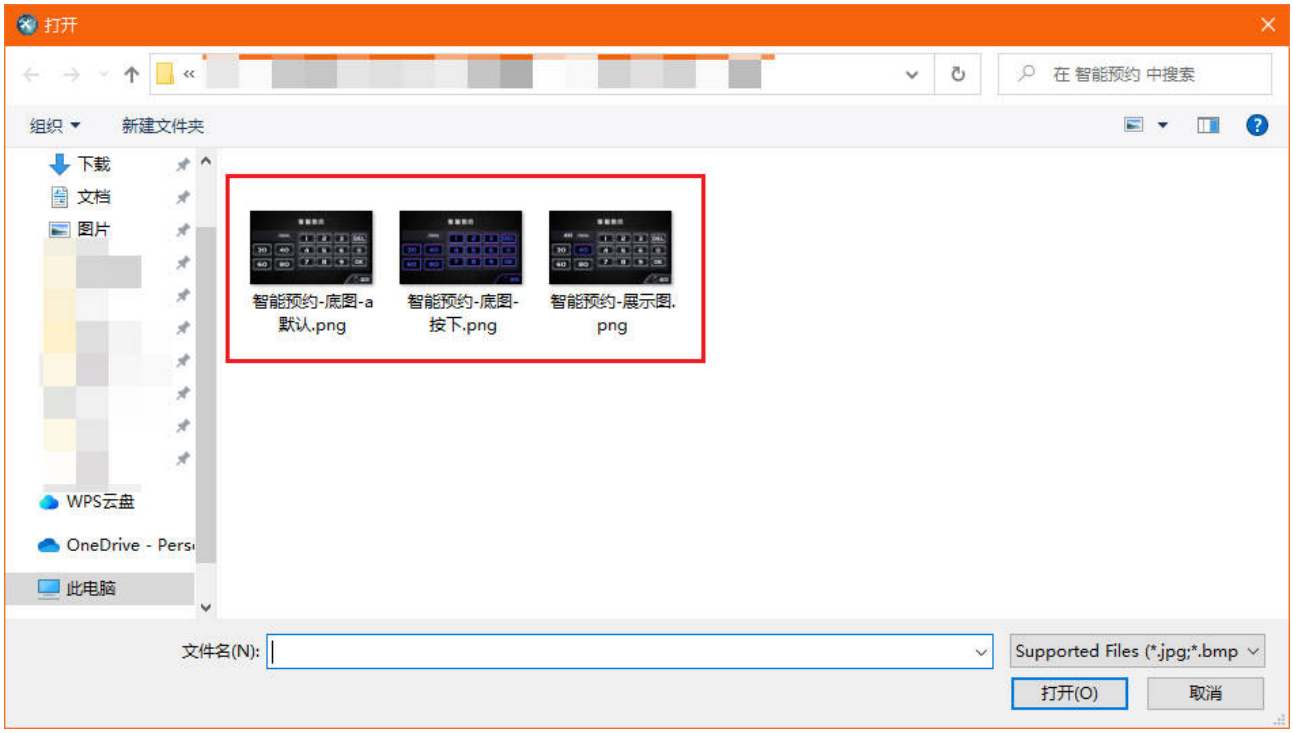
在做字库的时候有部分字体无法选择 (page 1360)
思源黑体下载 (page 1211)

11.5.38 如何导入图片

1、切换到”图片”

2、点击”+”号

3、选择需要导入的图片，点击打开，即可导入到工程中



4、每张图片都有对应的图片 id

5、选中需要修改的控件, 修改控件的 pic（或者 pic1/pic2/picc/picc1/picc2 等）属性为对应的图片 id, 然后回车, 即可完成修改

11.5.38.1 如何导入图片-相关链接

如何导入字库 (page 1442)

如何导入图片 (page 1449)

如何导入动画 (page 1455)

如何导入视频 (page 1461)

如何导入音频 (page 1467)

如何安装新字体 (page 1439)

创建字库和导入字库 (page 125)

指定字库 (page 1284)

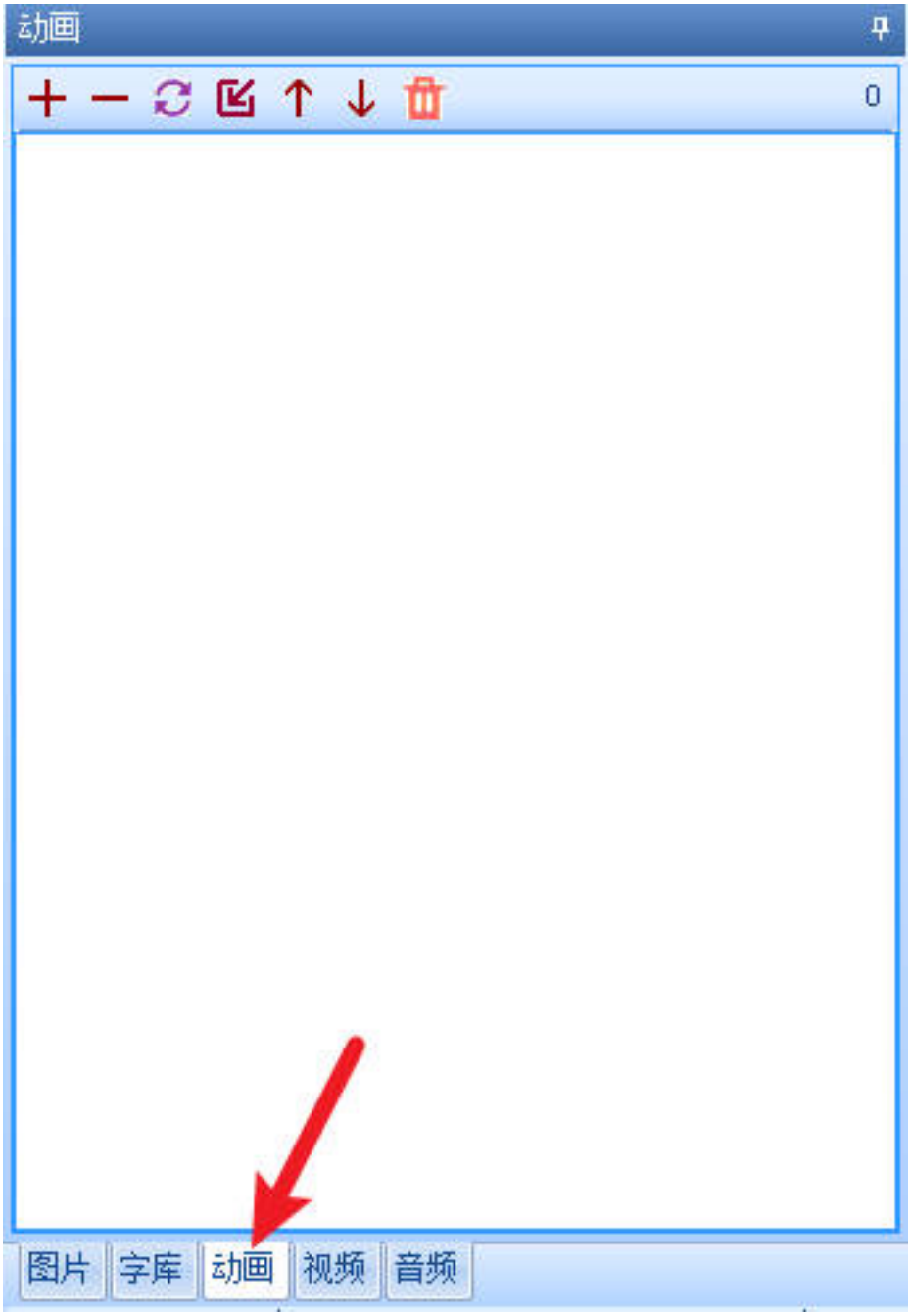
编译报错：*XXX* 初始值无效 (page 1342)

utf8 字库下如何只选择汉字 (page 1403)

在做字库的时候有部分字体无法选择 (page 1360)
思源黑体下载 (page 1211)

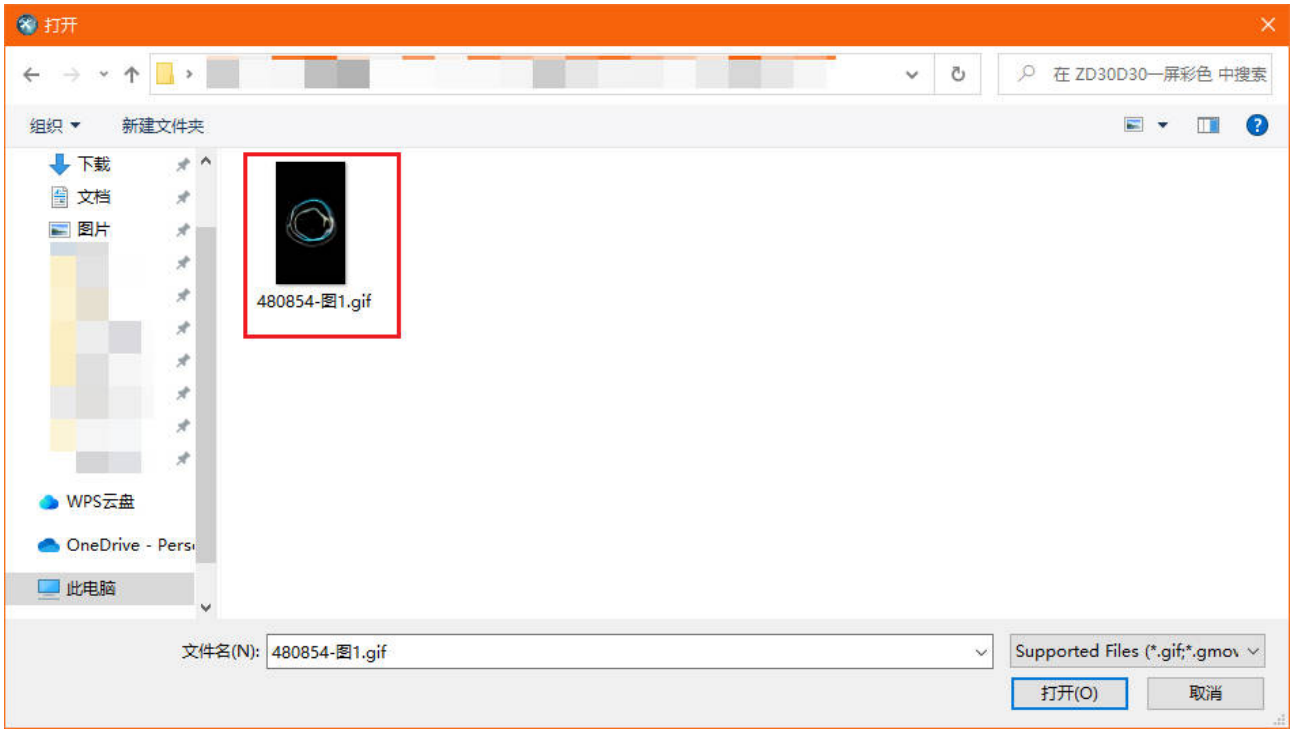
11.5.39 如何导入动画

1、切换到”动画”

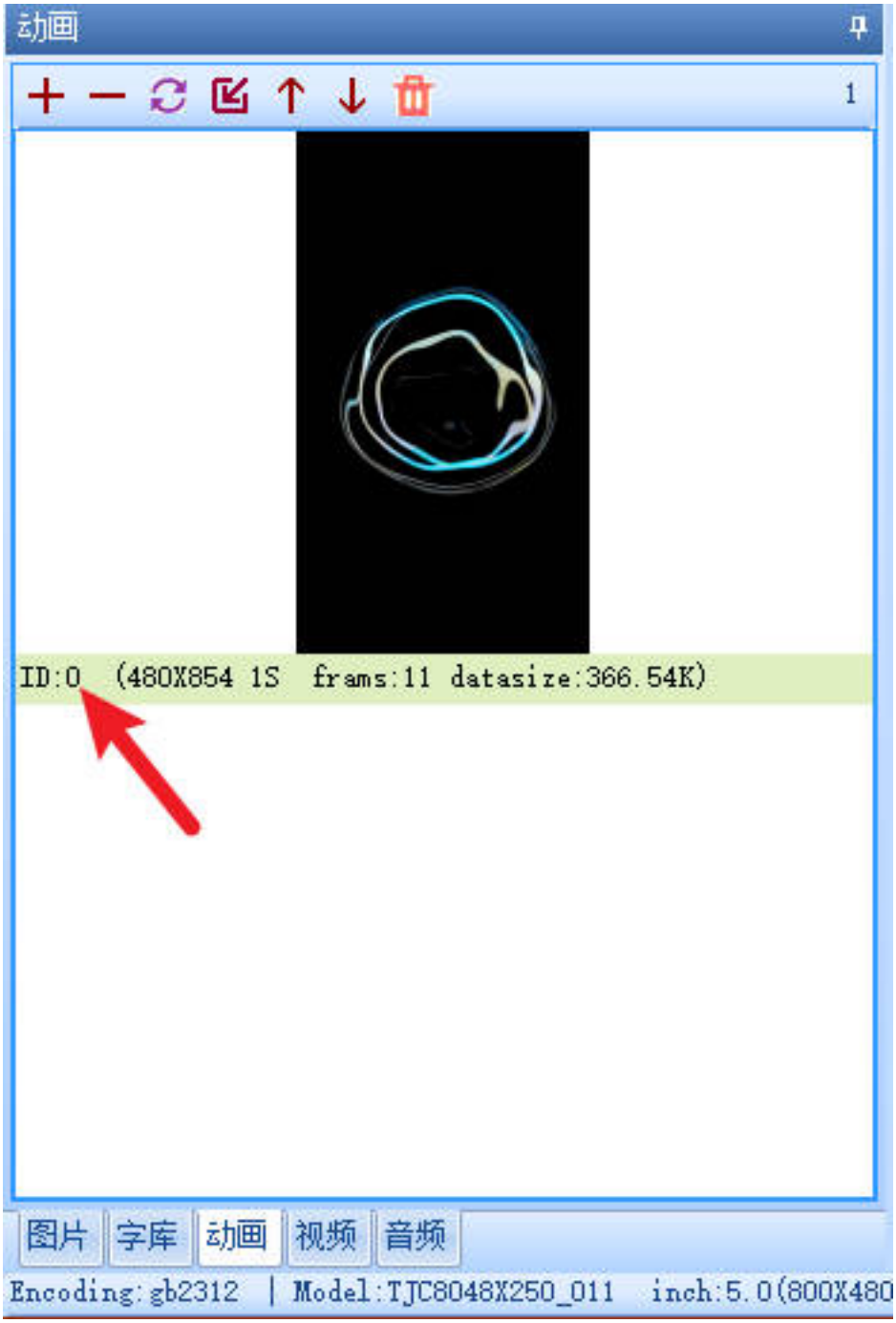


2、点击”+”号

3、选择需要导入的动画素材，支持 gif 图片, 点击打开，即可导入到工程中



4、每张动画都有对应的动画 id



5、选中需要修改的动画控件, 修改控件的 vid 属性为对应的动画 id, 然后回车, 即可完成修改

11.5.39.1 如何导入动画-相关链接

如何导入字库 (page 1442)

如何导入图片 (page 1449)

如何导入动画 (page 1455)

如何导入视频 (page 1461)

如何导入音频 (page 1467)

如何安装新字体 (page 1439)

创建字库和导入字库 (page 125)

指定字库 (page 1284)

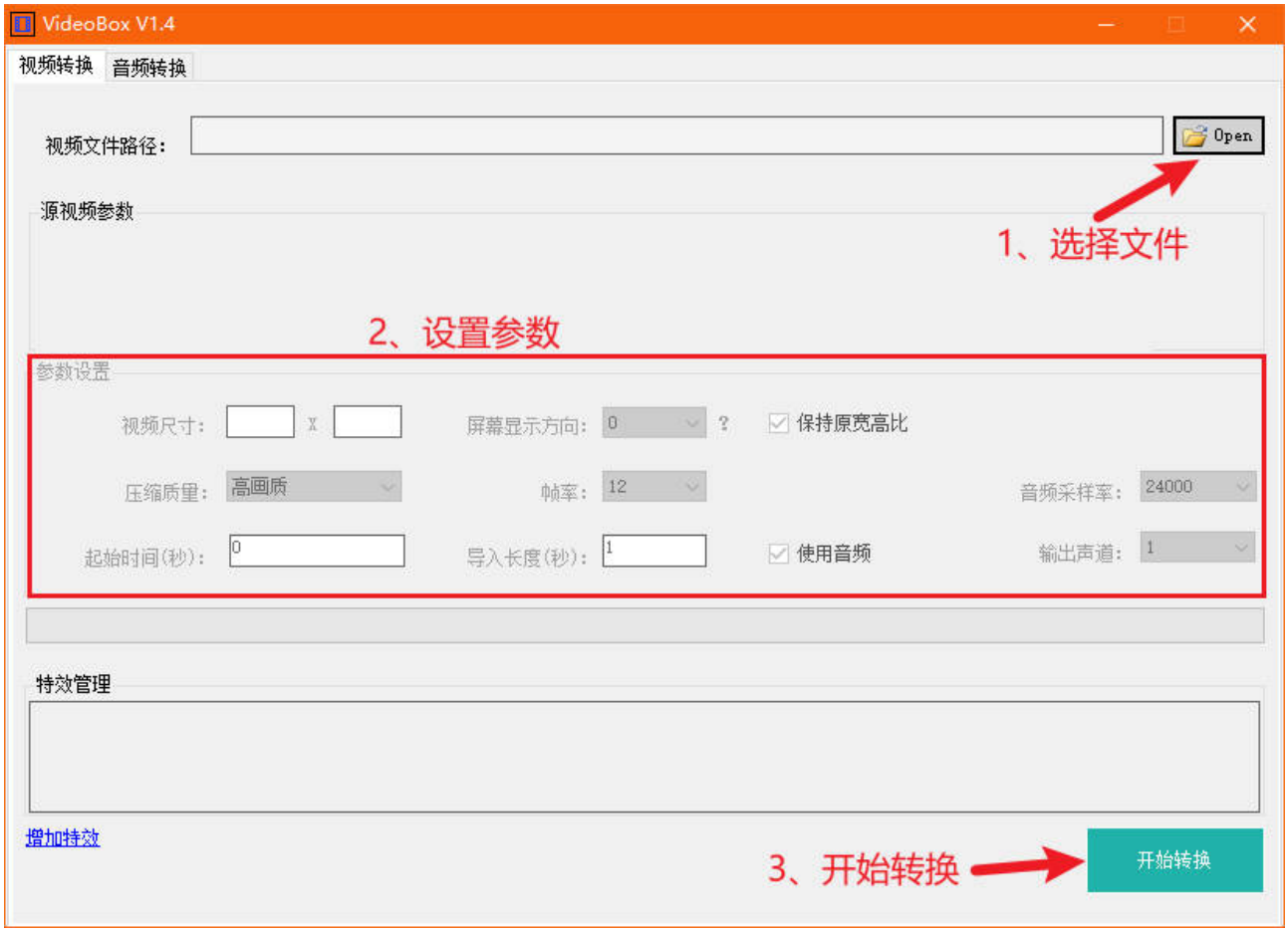
编译报错：*XXX* 初始值无效 (page 1342)

utf8 字库下如何只选择汉字 (page 1403)

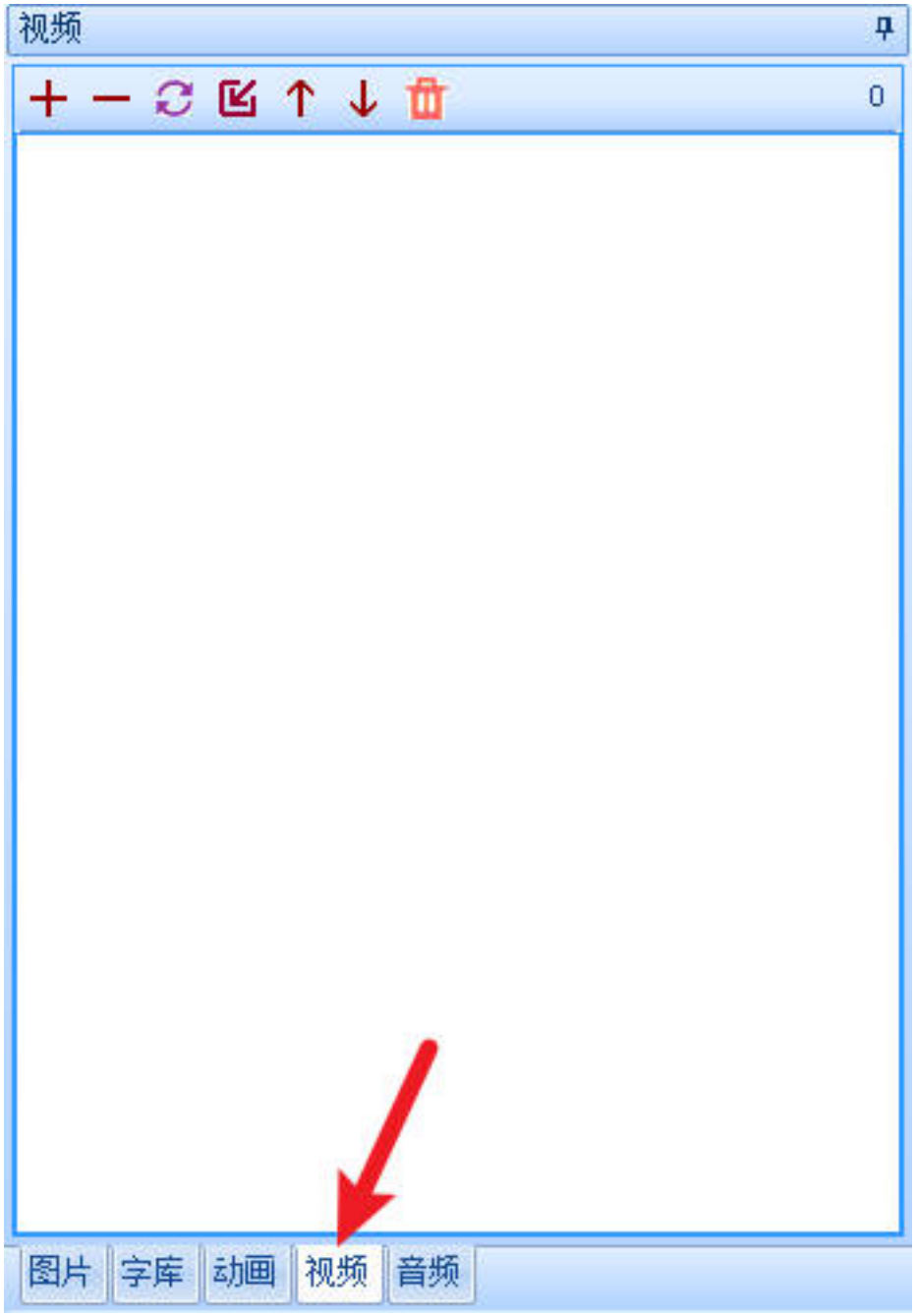
在做字库的时候有部分字体无法选择 (page 1360)
思源黑体下载 (page 1211)

11.5.40 如何导入视频

导入视频前，需要先对视频进行转换

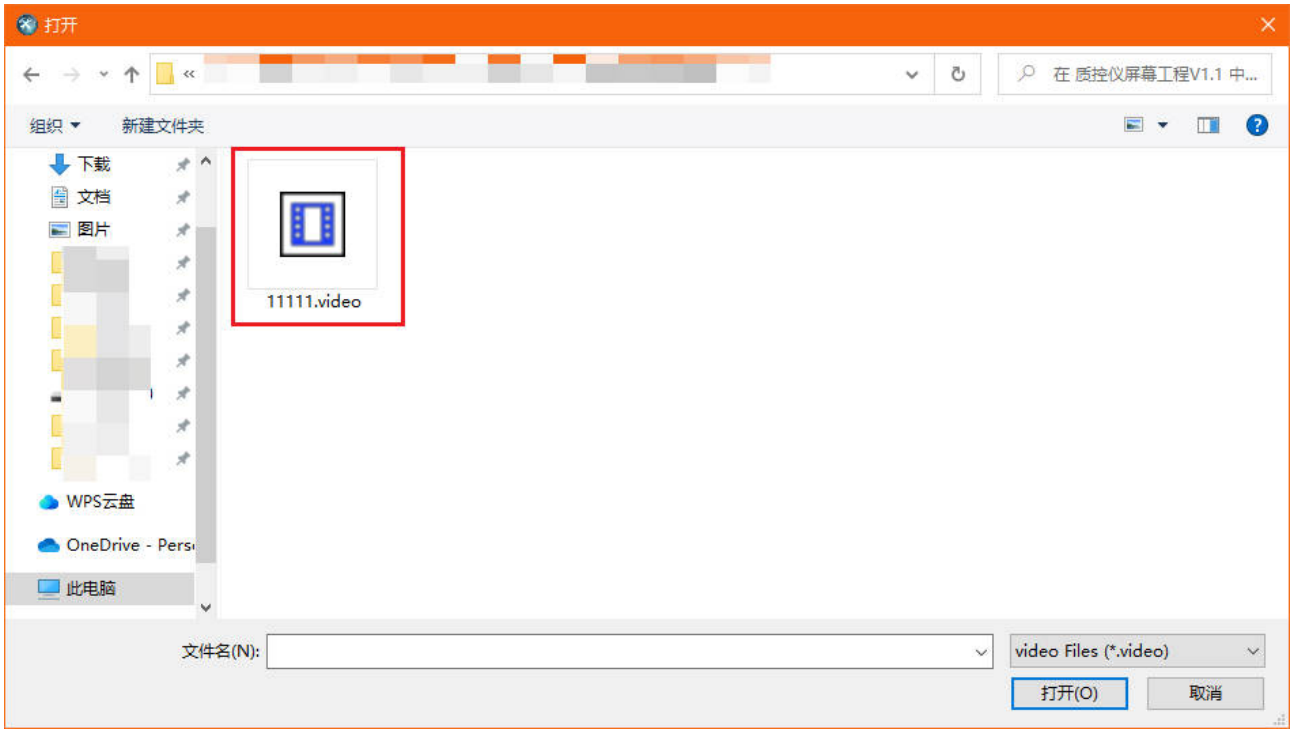


1、切换到”视频”

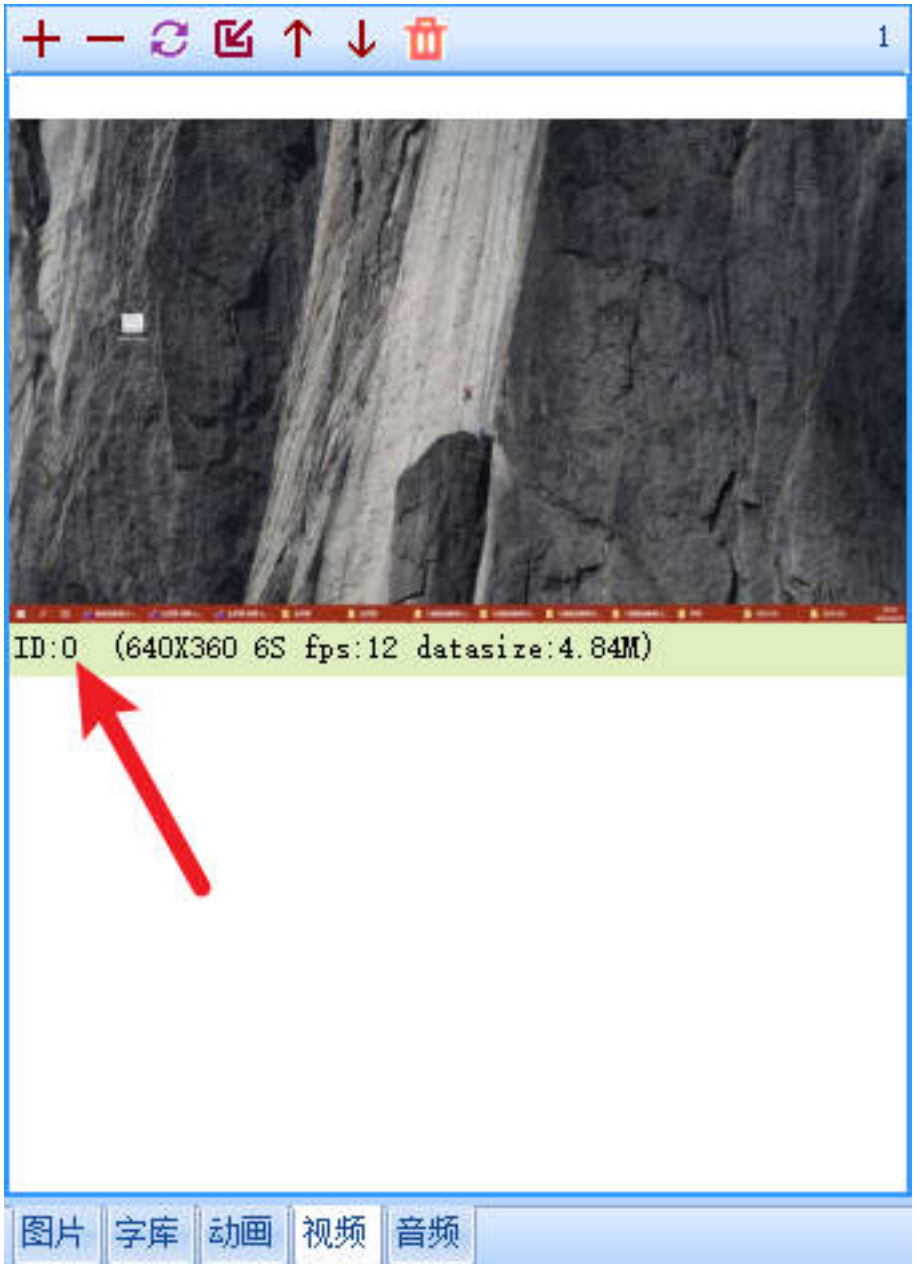


2、点击”+”号

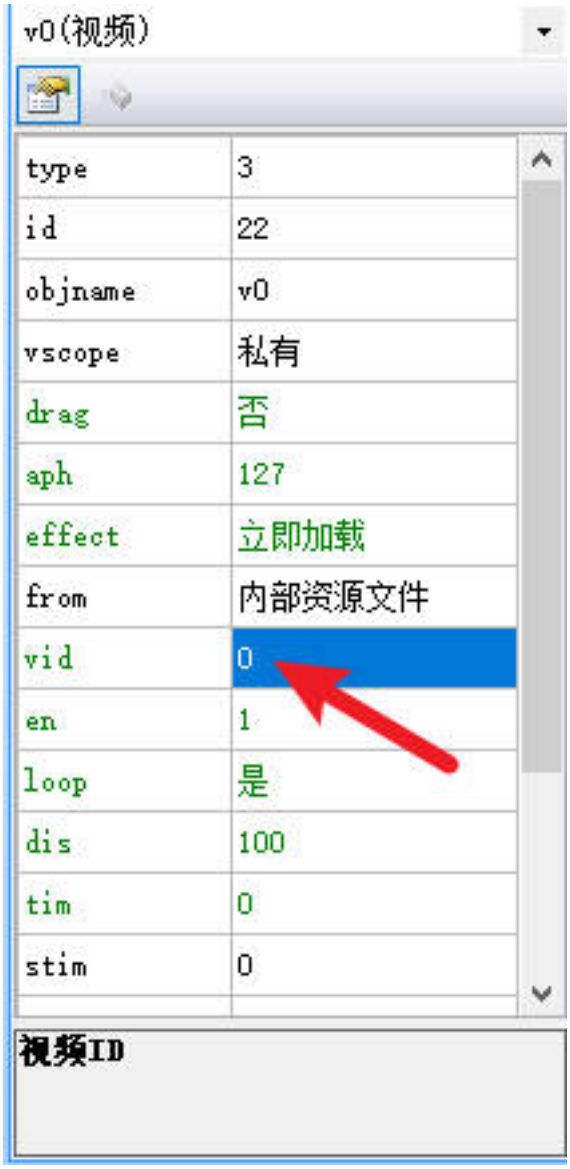
3、选择需要导入的视频素材，支持 gif 图片, 点击打开，即可导入到工程中



4、每张视频都有对应的视频 id



5、选中需要修改的视频控件, 修改控件的 vid 属性为对应的视频 id, 然后回车, 即可完成修改



11.5.40.1 如何导入视频-相关链接

如何导入字库 (page 1442)

如何导入图片 (page 1449)

如何导入动画 (page 1455)

如何导入视频 (page 1461)

如何导入音频 (page 1467)

如何安装新字体 (page 1439)

创建字库和导入字库 (page 125)

指定字库 (page 1284)

编译报错：*XXX* 初始值无效 (page 1342)

utf8 字库下如何只选择汉字 (page 1403)

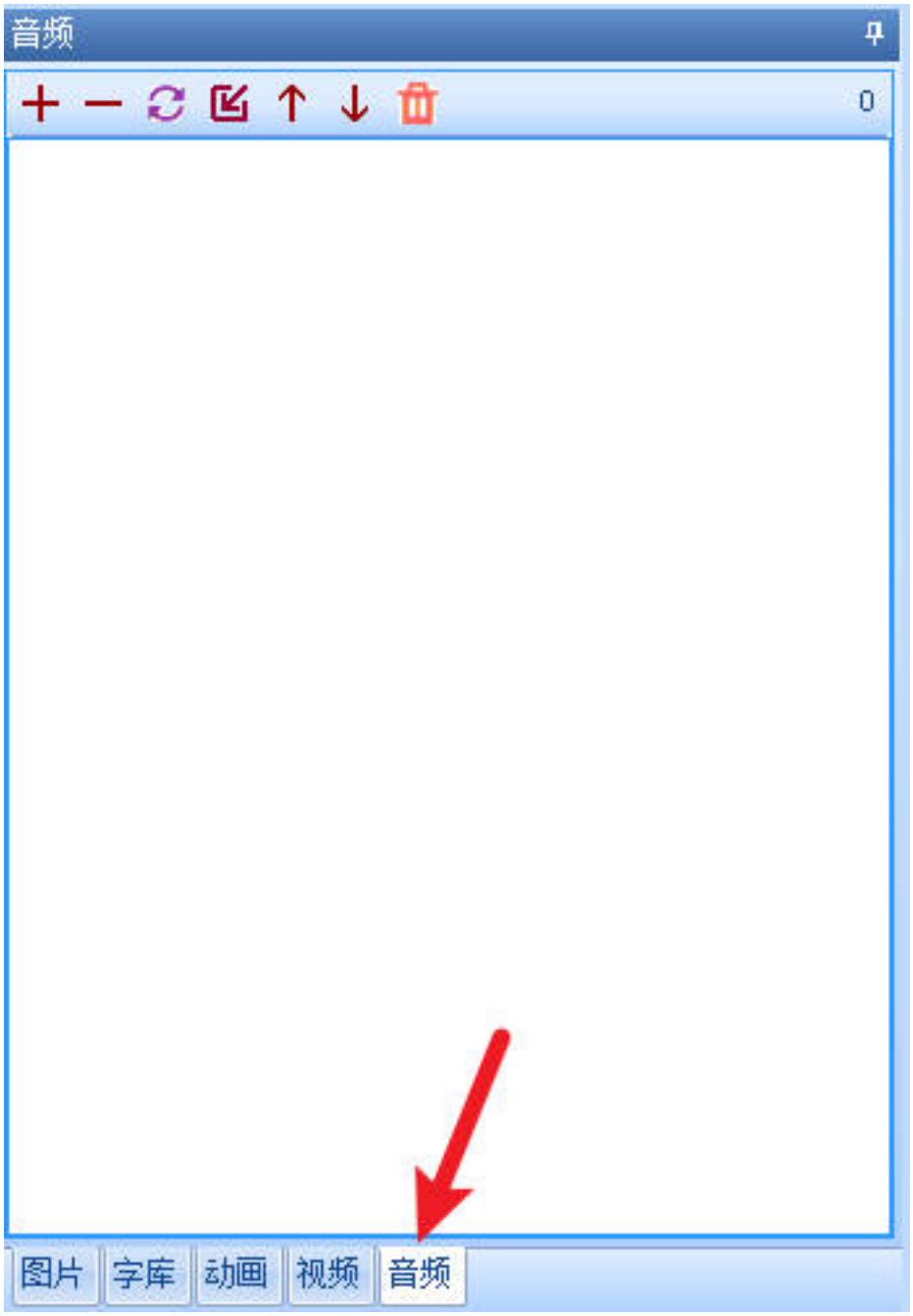
在做字库的时候有部分字体无法选择 (page 1360)

思源黑体下载 (page 1211)

11.5.41 如何导入音频

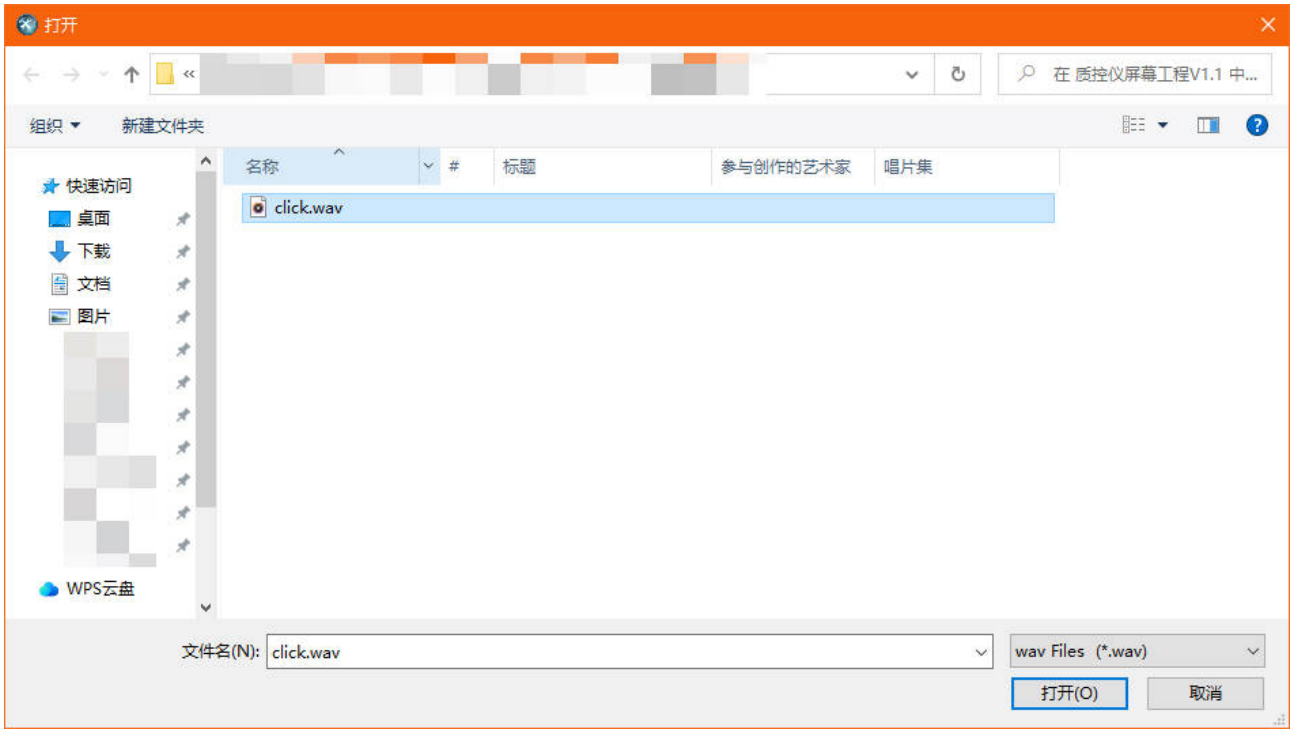
导入音频前，需要先对音频进行转换，如果原本已经是 wav 格式的音频，如果导入时报错，也请用工具重新转换一遍

1、切换到”音频”

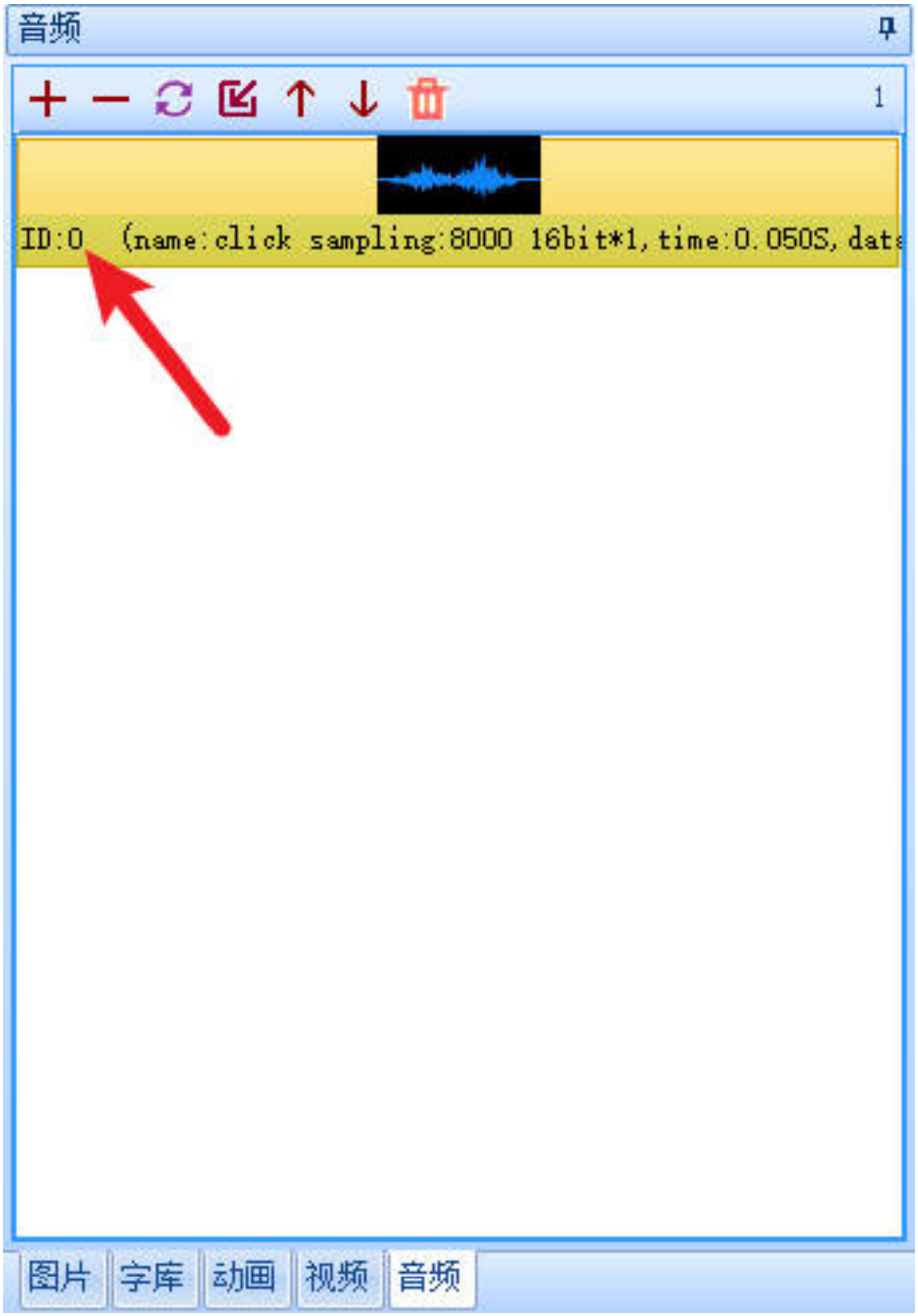


2、点击”+”号

3、选择需要导入的音频素材，支持 gif 图片, 点击打开，即可导入到工程中



4、每张音频都有对应的音频 id



5、选中需要修改的音频控件, 修改控件的 vid 属性为对应的音频 id, 然后回车, 即可完成修改

11.5.41.1 如何导入音频-相关链接

如何导入字库 (page 1442)
如何导入图片 (page 1449)
如何导入动画 (page 1455)
如何导入视频 (page 1461)
如何导入音频 (page 1467)
如何安装新字体 (page 1439)
创建字库和导入字库 (page 125)
指定字库 (page 1284)
编译报错：*XXX* 初始值无效 (page 1342)
utf8 字库下如何只选择汉字 (page 1403)

在做字库的时候有部分字体无法选择 (page 1360)
思源黑体下载 (page 1211)

11.5.42 串口屏连接单片机开发板注意事项

如果您使用的是外购的单片机开发板或最小系统板，如果这块电路板上有 USB 接口 (包括 USB-typeA,USB-typeB,USB-typeC,MicroUSB,miniUSB 等)。

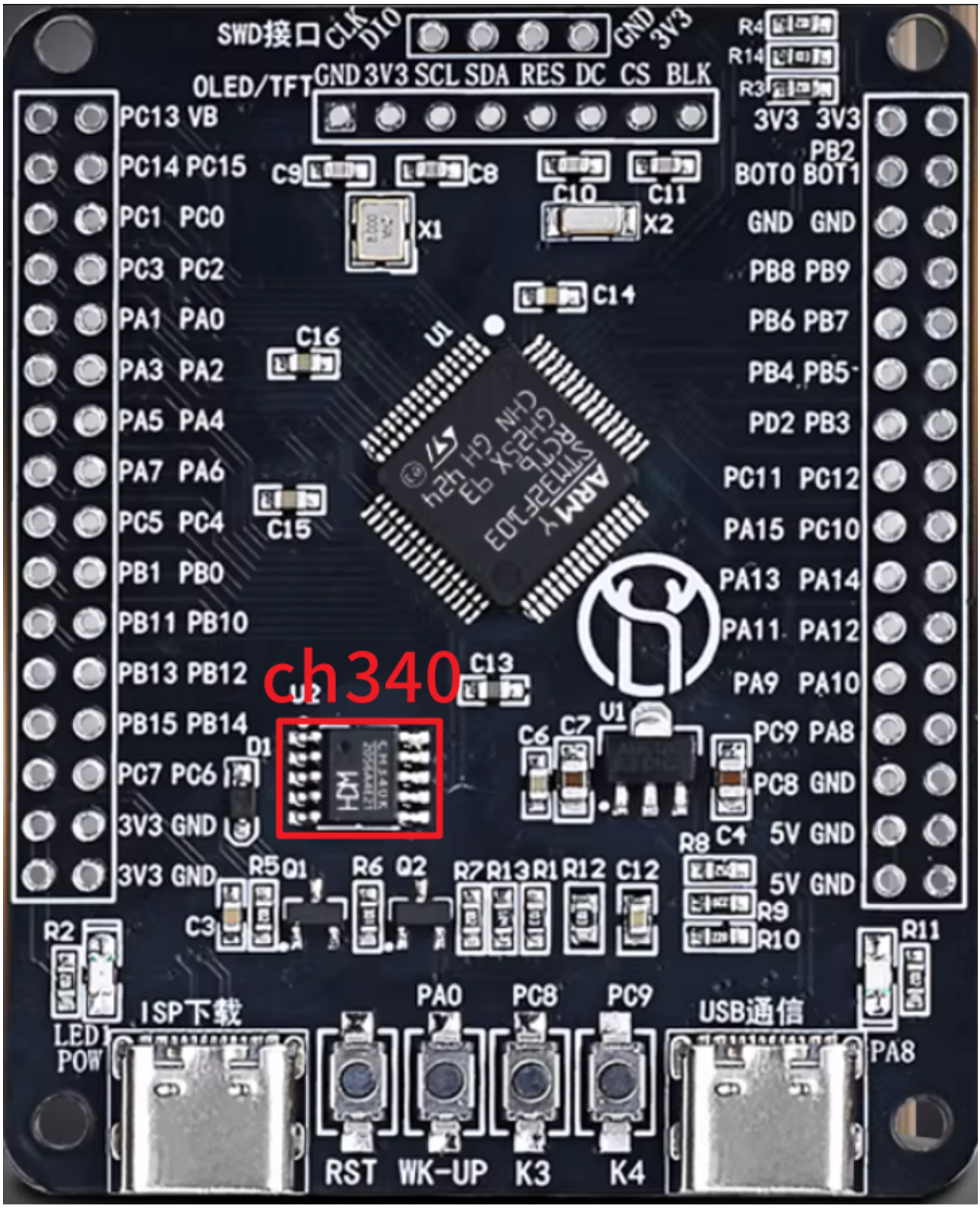
请检查 USB 口上是否接了 USB 转 ttl 芯片连接到了单片机的 uart 串口上，避免将连接了 USB 口的 uart 接串口屏, 否则实机调试时可能无法通讯。

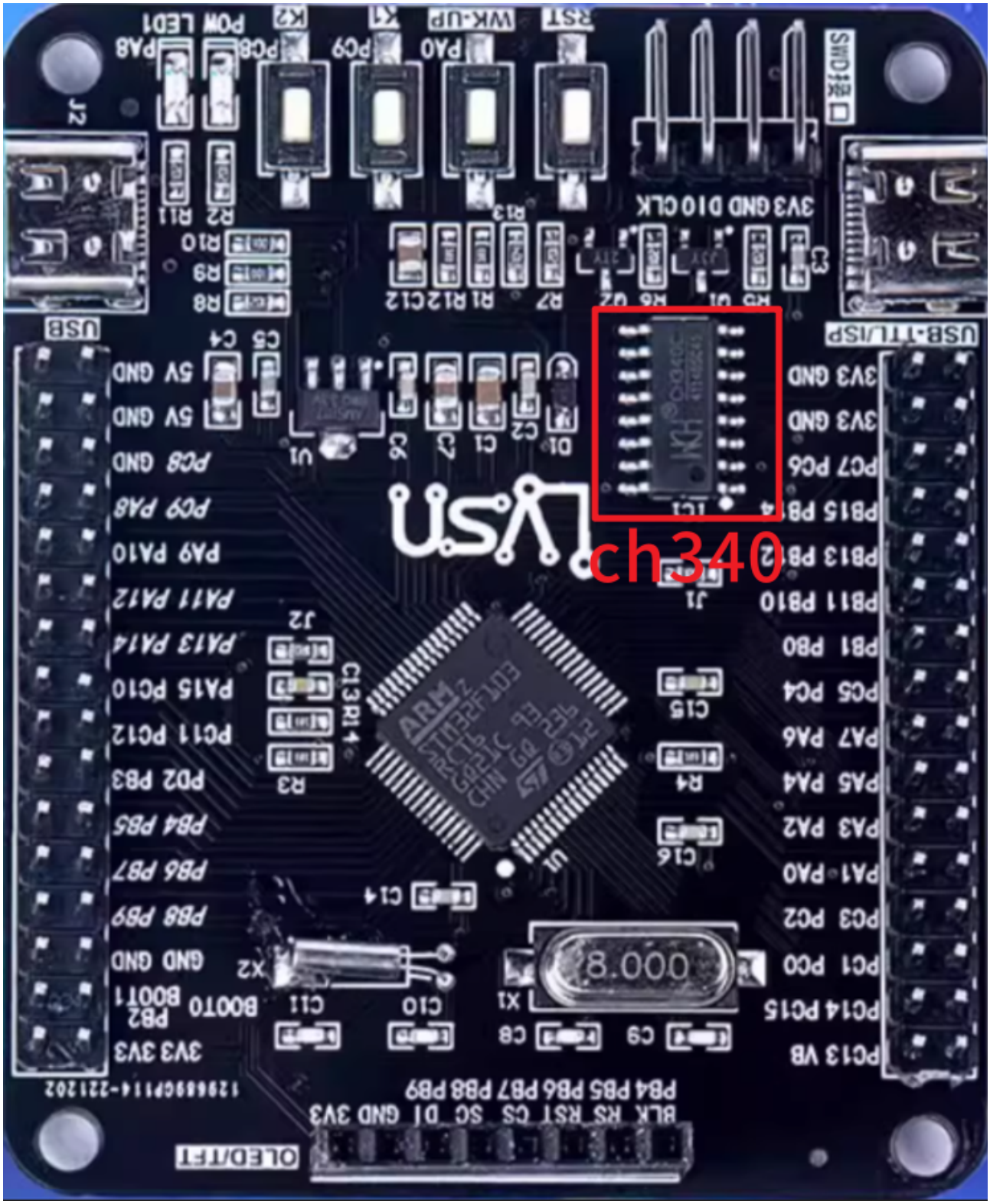
这是由于串口一对多照成的。

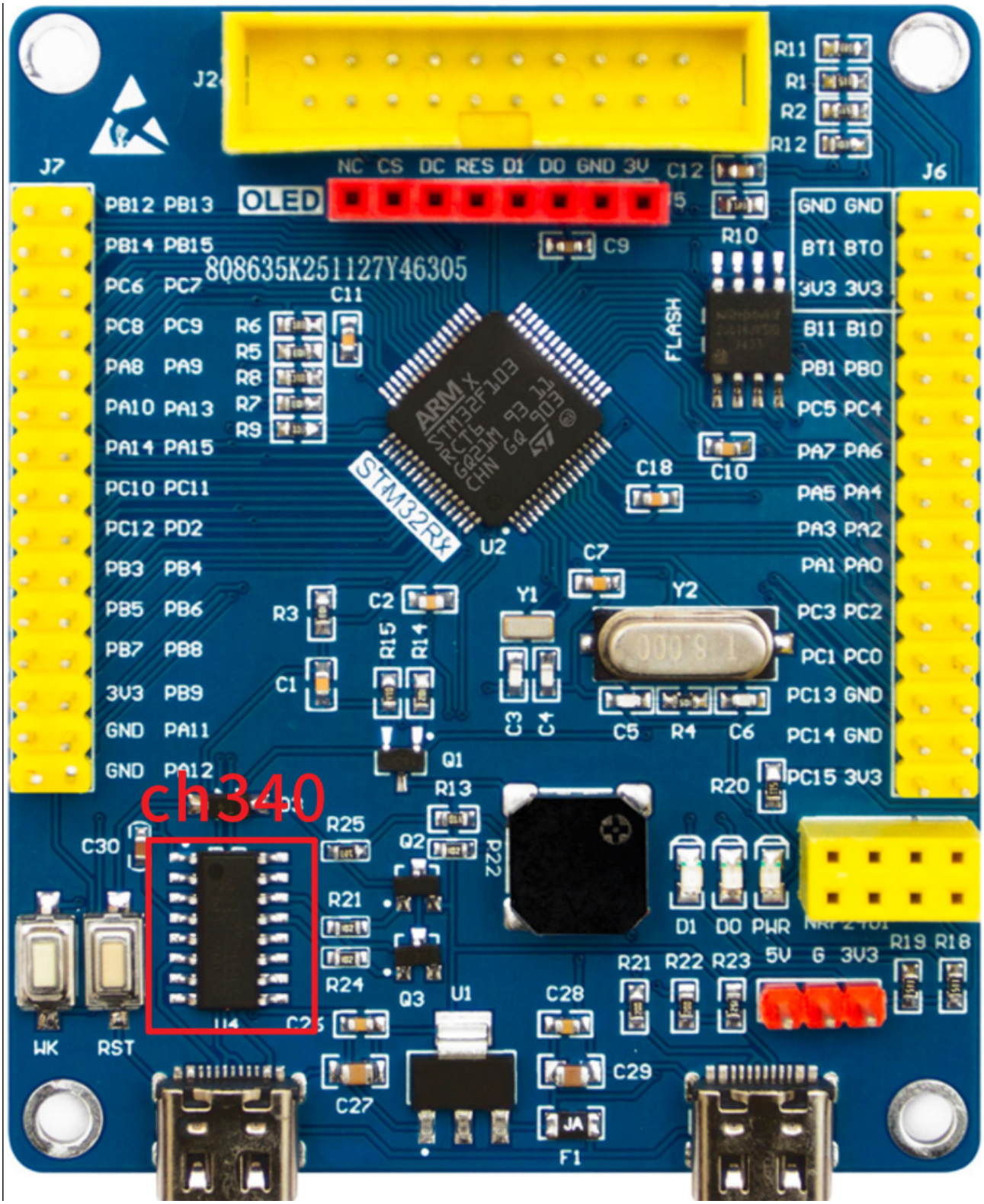
特别是注意 xx32 系列的 PA9 和 PA10 这两个引脚, 一般都是接在这两个引脚上, 建议避开。

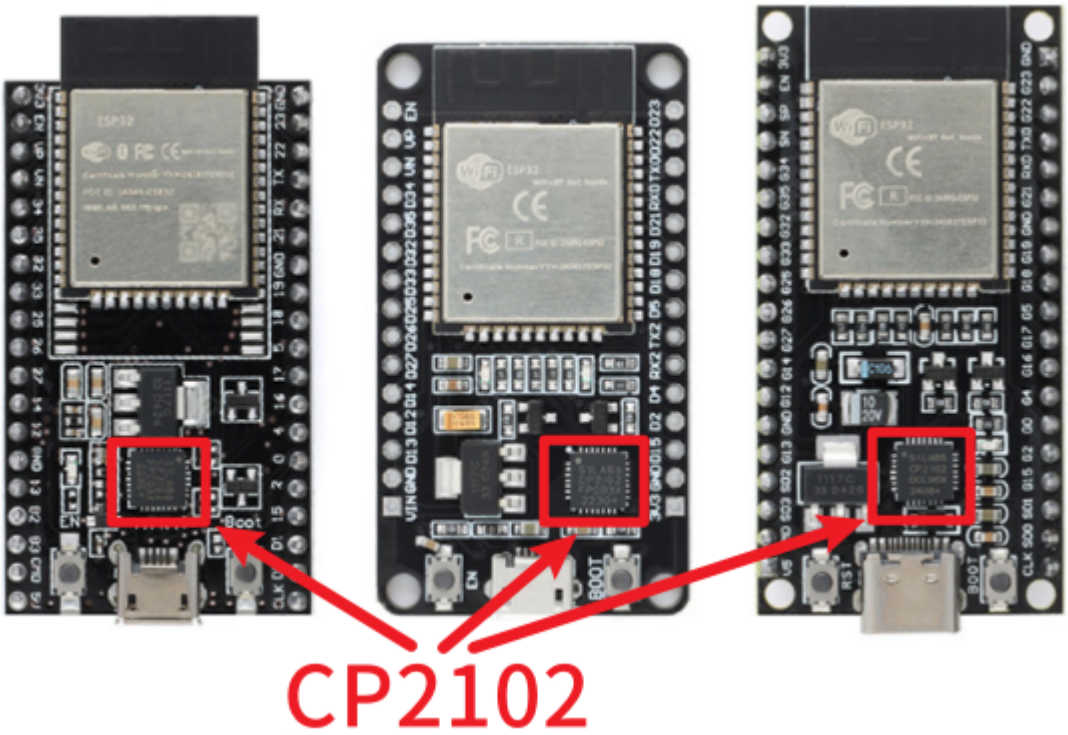
arduino 系列的 serial(serial1) 多半也是接了串口转换芯片的, 请避开这个串口，但是 arduino uno 等系列可能只有一个串口，这个时候就需要用到 arduino 的软串口库 SoftwareSerial，请自行在网上搜索 “arduino 软串口”

大部分 51 单片机也是只有一个串口，如果板子上有板载的 usb 转 ttl 芯片，请先断开这个芯片 (一般会有跳线进行选择)，然后才能连接到串口屏实物


SWD接口 CLK DIO GND 3V3
OLED/TFT GND 3V3 SCL SDA RES DC CS BLK
R4
R14
R3
3V3 3V3
PB2
BOT0 BOT1
GND GND
PB8 PB9
PB6 PB7
PB4 PB5
PD2 PB3
PC11 PC12
PA15 PC10
PA13 PA14
PA11 PA12
PA9 PA10
PC9 PA8
PC8 GND
5V GND
5V GND
PC13 VB
PC14 PC15
PC1 PC0
PC3 PC2
PA1 PA0
PA3 PA2
PA5 PA4
PA7 PA6
PC5 PC4
PB1 PB0
PB11 PB10
PB13 PB12
PB15 PB14
PC7 PC6
3V3 GND
3V3 GND
C9
C8
C10
C11
X1
X2
C14
C16
U1
C15
C13
ch340
U2
D1
C6
C7
U1
R8
C4
R5
Q1
R6
Q2
R7
R13
R1
R12
C12
C3
R9
R10
R2
R11
ISP下载
LED
POW
PA0
PC8
PC9
USB通信
PA8
RST
WK-UP
K3
K4


ch340
8.000
USB
USB-TTL/ISP
OLED/TFT
SWD接口
WK-UP
RST
POW LED1


J2
J7
J6
OLED
NC CS DC RES D1 D0 GND 3V
PB12 PB13
PB14 PB15
PC6 PC7
PC8 PC9
PA8 PA9
PA10 PA13
PA14 PA15
PC10 PC11
PC12 PD2
PB3 PB4
PB5 PB6
PB7 PB8
3V3 PB9
GND PA11
GND PA12
GND GND
BT1 BT0
3V3 3V3
B11 B10
PB1 PB0
PC5 PC4
PA7 PA6
PA5 PA4
PA3 PA2
PA1 PA0
PC3 PC2
PC1 PC0
PC13 GND
PC14 GND
PC15 3V3
808635K251127Y46305
STM32RX
FLASH
U2
U1
U4
Y1
Y2
Q1
Q2
Q3
F1
ch340
WK
RST
D1
D0
PWR
5V
G
3V3



相关链接: 串口屏与单片机连接 (page 211)

11.5.43 字符编码相关详解——串口怎么发送字符和汉字

先说一个结论: 你可以直接将编码理解为文字 (包括字母/符号等) 在字符编码表中的对应顺序

11.5.43.1 为什么要使用字符编码

你有没有想过，虽然你通过单片机的串口发送了“1234”，“abcd”等字符串出来, 但是明明单片机的引脚只能传输和接收高低电平, 但是你却能看到各种字符串, 这到底是什么原因呢

以下举个简单的例子:

在考试时，你和你的同桌想传递选择题的答案，但是你们只有一个能显示数字的计算器

但是你直接输入了 1 2 3 4 ，然后把计算器偷偷递给你的同桌，你的同桌接过来一看，原来答案是 A B C D

这个你和同桌临时建立起来的选择题答案编码，其实就是一个数字和字符的映射表，1 表示 A，2 表示 B，3 表示 C，4 表示 D

ASCII 编码其实也是同样的原理，

在 ASCII 编码中，字符“0”用十进制的 48 表示，字符“1”用十进制的 49 表示，以此类推，字符“9”用十进制的 57 表示。

那为什么是从 48 开始呢，因为 48 的十六进制是 0x30，57 的十六进制是 0x39

因此字符‘0’对应了 0x30，字符‘9’对应了 0x39

大写字母 A-Z 分别对应了 0x41 到 0x5A。

小写字母 a-z 分别对应了 0x61 到 0x7A。

其他零散的位置分别用于存放符号（例如”！@# $ %……&,./<>?”以及空格等），以及一些控制命令（例如”回车”,”换行”等）

ASCII 值	控制字符	ASCII 值	控制字符	ASCII 值	控制字符	ASCII 值	控制字符
0	NUT	32	(space)	64	@	96	、
1	SOH	33	!	65	A	97	a
2	STX	34	”	66	B	98	b
3	ETX	35	#	67	C	99	c
4	EOT	36	$	68	D	100	d
5	ENQ	37	%	69	E	101	e
6	ACK	38	&	70	F	102	f
7	BEL	39	,	71	G	103	g
8	BS	40	(	72	H	104	h
9	HT	41	)	73	I	105	i
10	LF	42	*	74	J	106	j
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	-	77	M	109	m
14	SO	46	.	78	N	110	n
15	SI	47	/	79	O	111	o
16	DLE	48	0	80	P	112	p
17	DCI	49	1	81	Q	113	q
18	DC2	50	2	82	R	114	r
19	DC3	51	3	83	X	115	s
20	DC4	52	4	84	T	116	t
21	NAK	53	5	85	U	117	u
22	SYN	54	6	86	V	118	v
23	TB	55	7	87	W	119	w
24	CAN	56	8	88	X	120	x
25	EM	57	9	89	Y	121	y
26	SUB	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	/	124	|
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~
31	US	63	?	95	—	127	DEL

现在我们知道了 ASCII 编码和 ASCII 字符的对应关系，但是明明单片机的引脚只能传输和接收高低电平，是怎么将数值传送出去的呢?

接下来再举个例子

这是个语文/英语考试，你找不到带计算器的借口，于是你和同桌约定好了用眨眼睛来传输数据，闭眼代表 0（低电平），睁眼代表 1（高电平），并且你的同桌一秒钟观察你一次，记录你的眼睛状态

你们约定好以 ASCII 编码来传输字符

要传输的数据是 A B C D ，对应的 ASCII 编码是 0x41 0x42 0x43 0x44

接下来需要将十六进制转换为二进制（这一转换是有技巧的，很简单，请自行百度以下）

0x41 对应的二进制是 0100 0001 ，于是你开始眨眼，闭睁闭闭闭闭闭睁，相当于单片机发出了电平低高低低低低低高

0x42 对应的二进制是 0100 0010，于是你开始眨眼，闭睁闭闭闭闭睁闭，相当于单片机发出了电平低高低低低低高低

0x43 对应的二进制是 0100 0011，于是你开始眨眼，闭睁闭闭闭闭睁睁，相当于单片机发出了电平低高低低低低高高

0x44 对应的二进制是 0100 0100，于是你开始眨眼，闭睁闭闭闭睁闭闭，相当于单片机发出了电平低高低低低高低低

原理讲解完毕，如果你已经理解串口是怎么传输数字和字母，那你能说出串口是怎么传输汉字的吗?

汉字传输的方法和 ASCII 编码类似，也是有一个编码对应着汉字，但是目前主流的汉字编码有两种，分别是 GB2312 以及 UTF8 编码，在传输汉字时，需要保证两边的编码是一致的，即两边都是 GB2312 或者两边都是 UTF8

在 GB2312 编码下

```
t0.txt="淘 晶 驰"
prints t0.txt,0
```

GB2312 编码下发出的数据为 CC D4 BE A7 B3 DB，GB2312 编码下，每个汉字占用 2 个字节，淘晶驰共三个字，共发出 6 个字节

在 UTF-8 编码下

```
t0.txt="淘 晶 驰"
prints t0.txt,0
```

UTF-8 编码下发出的数据为 E6 B7 98 E6 99 B6 E9 A9 B0，UTF-8 编码下，每个汉字占用 3 个字节，淘晶驰共三个字，共发出 6 个字节

相关链接: 串口屏与单片机连接 (page 211)

11.5.44 为什么转换后的图片/视频/音频体积变大了

首先，这是转换，不是压缩，因此转换后的图片/视频/音频有可能变大也有可能变小。

原有的图片/视频/音频可能采用了较高的压缩率，因此文件体积比较小，那为什么屏幕不采用比较高的压缩率呢。

这是因为串口屏的芯片性能比不上电脑/手机的性能，电脑/手机可以一边播放一边解码，也不会觉得卡顿。

但是串口屏的芯片性能没有那么强劲，只能采取空间换时间，牺牲压缩效率，来提升播放时的效果。

11.5.45 如何测量屏幕的尺寸大小

测量出屏幕的对角线长度 (厘米), 将测得的长度除以 2.54, 即为对应的尺寸大小

11.5.46 MAC 电脑如何开发淘晶驰串口屏

Intel 芯片版本：

1、安装 Parallels Desktop 虚拟机，在 Parallels Desktop 中安装 win7、win10、win11。或者通过 bootcamp 安装 windows 系统

2、下载上位机并安装请参考上位机下载 (page 1066)，在安装过程中根据提示自动安装相关环境

3、安装串口驱动，请参考：常见串口驱动下载 (page 1104)

Apple 芯片版本：

1、以 mac m1 为例, 安装 Parallels Desktop 18/19，在 Parallels Desktop 中安装 win11 ARM 版本

2、下载上位机并安装请参考上位机下载 (page 1066)，在安装过程中根据提示自动安装相关环境

3、如果你使用的是 CP2102 系列芯片，请安装 arm 版本的 CP210x 串口驱动，请参考：https://blog.csdn.net/hzdxyh/article/details/128587697

11.5.47 数据记录文件结构分析 1

文档迁移到《串口屏高级应用详解》-> 《数据记录文件结构分析详解 (page 1047) 》

11.5.48 哪些指令和操作会消耗 flash 寿命

以下五个指令会写入 flash, 消耗 flash 寿命, 频繁操作以下指令, 可能会导致串口屏损坏

1、bauds

2、volume

3、dims

4、wepo

5、wept

11.5.48.1 bauds-配置掉电不丢失的波特率

设置 bauds 时，会判断当前值与 flash 中保存的值是否一致，如果一致则不写入，如果不一致，则会写入 flash 中保存，因此修改 bauds 值会消耗 flash 寿命

```
//虽然循环了1000次,但写入前会和flash中保存的值比较,不一致才会写入,因此实际上只写入了一次
for(sys0=0;sys0<1000;sys0++)
{
    bauds=9600
}
```

11.5.48.2 volume-配置掉电不丢失的音量

设置 volume 时，会判断当前值与 flash 中保存的值是否一致，如果一致则不写入，如果不一致，则会写入 flash 中保存，因此修改 volume 值会消耗 flash 寿命

```
//虽然循环了1000次,但写入前会和flash中保存的值比较,不一致才会写入,因此实际上只写入了一次
for(sys0=0;sys0<1000;sys0++)
{
    volume=100
}
```

11.5.48.3 dims-配置掉电不丢失的亮度

设置 dims 时，会判断当前值与 flash 中保存的值是否一致，如果一致则不写入，如果不一致，则会写入 flash 中保存，因此修改 dims 值会消耗 flash 寿命

```
//虽然循环了1000次,但写入前会和flash中保存的值比较,不一致才会写入,因此实际上只写入了一次
for(sys0=0;sys0<1000;sys0++)
{
    dims=100
}
```

11.5.48.4 wepo-将数据保存在串口屏内

wepo 会直接写入 eeprom 或者 flash 模拟的 eeprom 中, 会消耗 eeprom 或者 flash 的寿命

11.5.48.5 wept-通过串口将数据透传入串口屏内

wept 会直接写入 eeprom 或者 flash 模拟的 eeprom 中, 会消耗 eeprom 或者 flash 的寿命

12 产品选型和规格书

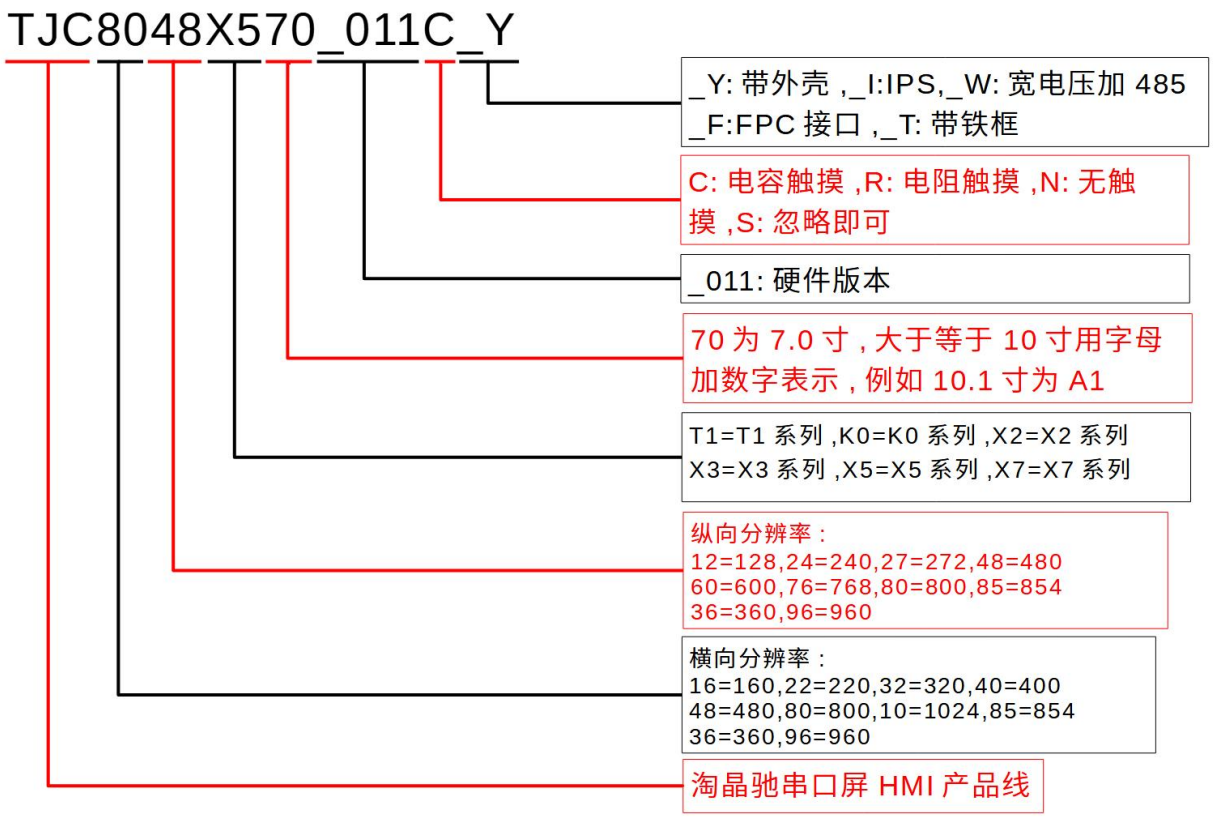
提示： 关于各个系列之间的区别请参考产品命名规则 (page 1488) 和功能选型 (page 1490)

规格书下载请点击此处访问网址：http://wiki.tjc1688.com/product/index.html

12.1 产品选型

12.1.1 产品命名规则

以淘晶驰 X5 系列 7 寸为例



深圳市淘晶驰电子有限公司工业串口HMI产品编号规则

（以TJC8048X570_011C_Y为例）

对照说明	TJC	淘晶驰串口HMI产品线
	80	横向分辨率：16=160 22=220 32=320 40=400 48=480 80=800 10=1024
	48	纵向分辨率：12=128 24=240 27=272 48=480 60=600 76=768 80=800
	X5	应用分类：T0=T0系列 T1=T1系列 K0=K0系列 X3=X3系列 X5=X5系列
	70	显示尺寸：70为7.0英寸，大于等于10寸用字母和数字结合的方法表示，按顺序累加，字母从A开始，比如10.1寸表示方法：A1
	_011	硬件版本号：_011以区别不同的硬件版本
	C	触摸屏说明：N=无触摸 R=电阻触摸屏 C=电容触摸屏
	_Y	功能扩展：_Y表示带外壳

功能扩展描述：_Y表示带外壳，_I表示IPS屏，_W表示宽电压加485接口，_F表示裸屏（无PCB），_F_T表示裸屏带铁框

举例1：TJC8048T070_011R_Y_I,表示7寸电阻触摸有外壳IPS屏。

举例2：TJC4832T135_011R_F_T,表示3.5寸电阻触摸裸屏带铁框。

提示： 关于各个系列之间的区别请参考功能选型 (page 1490)

12.1.2 功能选型

资料更新时间：2026-05-13 17:26:41

表 13: 产品功能选型

功能	x5 系列	X3 系列	X2 系列	K0 系列	T1 系列	T0 系列
备注	性能强大	性价比高	性价比高		小屏专属	即将停产
FLASH 特性	NandFlash	SpiFlash	SpiFlash	SpiFlash	SpiFlash	SpiFlash
flash 容量	128M	8M/16M	8M/16M	16M/32M	4M/8M	4M/16M
FLASH 速度	12MB/S	3MB/S	速度 3MB/S	速度 3MB/S	速度 4MB/S	速度 3MB/S
控件图层保持	●	●	●			
页面加载特效	●	●	●			
滑动翻页功能	●	●	●			
控件加载特效	●	●	●			
控件移动功能	●	●	●			
控件触摸拖动	●	●	●			
控件半透明	●	●	●			
PNG 透明图片	●	●	●			
PNG 图片抗锯齿	●	●	●			
百变指针功能	●	●	●			
指针抗锯齿	●	●	●			
独立音频	2 路	2 路				
动画播放	支持	支持	支持	注释 ①	注释 ①	注释 ①
视频播放	6 路					
系统运行内存	512K	512K	512K	3.5K	3.5K	3.5K
图片压缩存储	●	●	●		●	
全屏图片数量	1000-2000	120-160	60-80	80-110	25-60	25-60
字体抗锯齿	●	●	●	●	●	●
自定义字库	●	●	●	●	●	●
设置显示方向	●	●	●	●	●	●
TTL 串口	●	●	●	●	●	●
RS232 串口	●	●				
设备地址设置	●	●	●	●	●	●
界面开发软件	●	●	●	●	●	●
RTC 时钟	●			●		
扩展 IO	8 路		注释 ②	8 路		
PWM	1 路 PWM		注释 ②	4 路 PWM		
用户数据存储	1024 字节	1024 字节	1024 字节	1024 字节		
带外壳	选配	选配	选配	选配		
带铁框			选配		选配	
电阻触摸	选配	选配	选配	选配	选配	选配
电容触摸	选配	选配	选配	选配	选配	
电容触摸加盖玻璃						
–加盖 3mm 玻璃	选配	选配	选配			
–加盖 5mm 玻璃	选配	选配	选配			

备注:TJC4848X540_011(86 盒), 只有电容屏, 不支持 TTL/232 切换, 可以选购 ttl 或 485 接口 (485 版本是宽电压供电)。

备注:FPC 版本的 X2 无外部 IO 接口 (也无 PWM)。

注释 ①：动画播放功能可以用图片控件加定时器代替。

注释 ②：X2 系列是否支持外部 IO 需要根据具体的设备型号来确定。

12.1.2.1 功能选型-相关资料

产品命名规则 (page 1488)

12.1.3 X2 系列

资料更新时间：2026-05-13 17:26:41

X2 系列包含 4.0 寸 ~ 7.0 寸

表格中的电流参数仅供参考，具体电流值请查看规格书，

相关链接

产品选型和规格书 (page 1488)

12.1.3.1 X2 系列 4.0 寸 ~7.0 寸

表 14: X2 系列

尺寸	4 寸	4.3 寸	4.3 寸	5 寸	7 寸
型号	TJC4848X240	TJC4827X243	TJC8048X243	TJC8048X250	TJC8048X270
比例	1:1	16:9	16:9	16:9	16:9
有效显示尺寸 (mm)	76.43×79.38	95.04×53.86	95.04×53.86	108×64.8	154.08×85.92
分辨率	480×480	480×272	800×480	800×480	800×480
TFT 类型	IPS	TN	IPS	TN	TN
亮度 (nit)	300	200	300	300	300
工作电压 (V)	请查看备注	请查看备注	请查看备注	4.65-12	4.65-12
工作电流 (mA)	260	220	320	220	420
休眠电流 (mA)	120	100	100	100	130
工作温度 (℃)	-20 ~ +70	-20 ~ +70	-20 ~ +70	-20 ~ +70	-20 ~ +70
flash 容量 (Bytes)	16M	8M	16M	16M	16M
eeprom 容量 (Bytes)	1K	1K	1K	1K	1K
运行内存 (Bytes)	512K	512K	512K	512K	512K
串口缓存区 (Bytes)	4K	4K	4K	4K	4K
RTC 时钟	无	无	无	无	无
扩展 I/O	无	请查看备注	请查看备注	有	有
主控频率（Hz）	200M	200M	200M	200M	200M

重要:

备注:

FPC 版本的 X2 无外部 IO 接口

单独使用 FPC 版本的 X2 时, 请使用 3.3V 的电压对 X2 进行供电

12.1.4 X3 系列

资料更新时间：2026-05-13 17:26:41

X3 系列包含 4.3 寸 ~ 7.0 寸

表格中的电流参数仅供参考，具体电流值请查看规格书，

相关链接

产品选型和规格书 (page 1488)

12.1.4.1 X3 系列 4.3 寸 ~7.0 寸

表 15: X3 系列

尺寸	4.3 寸	4.3 寸	5 寸	7 寸
型号	TJC4827X343	TJC8048X343	TJC8048X350	TJC8048X370
比例	16:9	16:9	16:9	16:9
有效显示尺寸 (mm)	95.04×53.86	95.04×53.86	108×64.8	154.08×85.92
分辨率	480×272	800×480	800×480	800×480
TFT 类型	TN	IPS	TN	TN
亮度 (nit)	300	300	300	300
工作电压 (V)	4.65-6.5	4.65-6.5	4.65-6.5	4.65-6.5
工作电流 (mA)	220	320	350	430
休眠电流 (mA)	170	170	170	170
工作温度 (℃)	-20 ~ +70	-20 ~ +70	-20 ~ +70	-20 ~ +70
flash 容量 (Bytes)	8M	8M	16M	16M
eeprom 容量 (Bytes)	1K	1K	1K	1K
运行内存 (Bytes)	512K	512K	512K	512K
串口缓存区 (Bytes)	4K	4K	4K	4K
RTC 时钟	无	无	无	无
扩展 I/O	无	无	无	无
主控频率（Hz）	200M	200M	200M	200M

12.1.5 X5 系列

资料更新时间：2026-05-13 17:26:41

X5 系列包含 4.0 寸 ~ 10.1 寸

表格中的电流参数仅供参考，具体电流值请查看规格书，

相关链接

产品选型和规格书 (page 1488)

12.1.5.1 X5 系列 4.0 寸 ~5.0 寸

表 16: X5 系列 4.0 寸 ~ 4.3 寸

尺寸	4 寸	4 寸	4.3 寸	4.3 寸
型号	TJC4848X540(宽电压)	TJC4848X540	TJC4827X543	TJC8048X543
比例	1:1	1:1	16:9	16:9
有效显示尺寸 (mm)	71.86×70.18	71.86×70.18	95.04×53.86	95.04×53.86
分辨率	480×480	480×480	480×272	800×480
TFT 类型	IPS	IPS	TN	IPS
亮度 (nit)	300	300	300	300
工作电压 (V)	8~24	4.65-6.5	4.65-6.5	4.65-6.5
工作电流 (mA)	110(DC12V)	250	220	320
休眠电流 (mA)	78	170	170	170
工作温度 (℃)	-20 ~ +70	-20 ~ +70	-20 ~ +70	-20 ~ +70
flash 容量 (Bytes)	128M	128M	128M	128M
eeprom 容量 (Bytes)	1K	1K	1K	1K
运行内存 (Bytes)	512K	512K	512K	512K
串口缓存区 (Bytes)	4K	4K	4K	4K
RTC 时钟	有	有	有	有
扩展 I/O	有	有	有	有
主控频率（Hz）	200M	200M	200M	200M

12.1.5.2 X5 系列 7.0 寸 ~10.1 寸

表 17: X5 系列 5.0 寸 ~ 10.1 寸

尺寸	5 寸	7 寸	7 寸	8 寸	10.1 寸
型号	TJC8048X550	TJC8048X570	TJC1060X570	TJC8060X580	TJC1060X5A1
比例	16:9	16:9	16:9	4:3	16:9
有效显示尺寸 (mm)	108×64.8	154.08×85.92	154.21×85.92	162x121.5	222.72×125.28
分辨率	800×480	800×480	1024×600	800×600	1024×600
TFT 类型	TN	TN	IPS	TN	IPS
亮度 (nit)	300	300	300	300	300
工作电压 (V)	4.65-6.5	4.65-6.5	4.65-6.5	4.65-6.5	4.65-6.5
工作电流 (mA)	350	430	550	630	800
休眠电流 (mA)	170	170	170	170	170
工作温度 (℃)	-20 ~ +70	-20 ~ +70	-20 ~ +70	-20 ~ +70	-20 ~ +70
flash 容量 (Bytes)	128M	128M	128M	128M	128M
eeprom 容量 (Bytes)	1K	1K	1K	1K	1K
运行内存 (Bytes)	512K	512K	512K	512K	512K
串口缓存区 (Bytes)	4K	4K	4K	4K	4K
RTC 时钟	有	有	有	有	有
扩展 I/O	有	有	有	有	有

续下页

表 17 – 接上页

主控频率（Hz）	200M	200M	200M	200M	200M

12.1.6 K0 系列

资料更新时间：2026-05-13 17:26:41

K0 系列包含 2.2 寸 ~ 7.0 寸

表格中的电流参数仅供参考，具体电流值请查看规格书，

相关链接

产品选型和规格书 (page 1488)

12.1.6.1 K0 系列 2.2 寸 ~3.2 寸

表 18: K0 系列 2.2 寸 ~ 3.2 寸

尺寸	2.2 寸	2.4 寸	2.8 寸	3.2 寸
型号	TJC3224K022	TJC3224K024	TJC3224K028	TJC4024K032
比例	4:3	4:3	4:3	16:9
有效显示尺寸 (mm)	45.12x33.84	48.96x36.72	57.6x43.2	69.6x41.76
分辨率	320×240	320×240	320×240	400X240
TFT 类型	TN	TN	TN	TN
亮度 (nit)	300	300	300	300
工作电压 (V)	4.65-6.5	4.65-6.5	4.65-6.5	4.65-6.5
工作电流 (mA)	85	85	85	90
休眠电流 (mA)	15	15	15	15
工作温度 (℃)	-20 ~ +70	-20 ~ +70	-20 ~ +70	-20 ~ +70
flash 容量 (Bytes)	16M	16M	16M	16M
eeprom 容量 (Bytes)	1K	1K	1K	1K
运行内存 (Bytes)	3584	3584	3584	3584
串口缓存区 (Bytes)	1K	1K	1K	1K
RTC 时钟	有	有	有	有
扩展 I/O	有	有	有	有
主控频率（Hz）	48M	48M	48M	48M

12.1.6.2 K0 系列 3.5 寸 ~7.0 寸

表 19: K0 系列 3.5 寸 ~ 7.0 寸

尺寸	3.5 寸	4.3 寸	5 寸	7 寸
型号	TJC4832K035	TJC4827K043	TJC8048K050	TJC8048K070
比例	4:3	16:9	16:9	16:9
有效显示尺寸 (mm)	73.44x48.96	95.04x53.86	108x64.8	154.08x85.92
分辨率	480×320	480X272	800X480	800X480
TFT 类型	TN	TN	TN	TN
亮度 (nit)	300	300	300	300
工作电压 (V)	4.65-6.5	4.65-6.5	4.65-6.5	4.65-6.5
工作电流 (mA)	145	250	410	510
休眠电流 (mA)	15	15	15	15
工作温度 (℃)	-20 ~ +70	-20 ~ +70	-20 ~ +70	-20 ~ +70
flash 容量 (Bytes)	32M	32M	32M	32M
eeprom 容量 (Bytes)	1K	1K	1K	1K
运行内存 (Bytes)	8192	8192	8192	8192
串口缓存区 (Bytes)	1K	1K	1K	1K
RTC 时钟	有	有	有	有
扩展 I/O	有	有	有	有

续下页

表 19 - 接上页

主控频率（Hz）	108M	108M	108M	108M

12.1.7 T1 系列

资料更新时间：2026-05-13 17:26:41

T1 系列包含 1.8 寸 ~ 5.0 寸

表格中的电流参数仅供参考，具体电流值请查看规格书，

相关链接

产品选型和规格书 (page 1488)

12.1.7.1 T1 系列 1.8 寸 ~2.8 寸

表 20: T1 系列 1.8 寸 ~2.8 寸

尺寸	1.8 寸	2.0 寸	2.4 寸	2.8 寸
型号	TJC1612T118	TJC3224T120	TJC3224T124	TJC3224T128
比例	4:3	4:3	4:3	4:3
有效显示尺寸 (mm)	35.04×28.03	51.8x36.2	48.96×36.72	57.6×43.2
分辨率	160×128	320×240	320×240	320×240
TFT 类型	TN	TN	TN	TN
亮度 (nit)	300	300	300	300
工作电压 (V)	4.65-6.5	4.65-6.5	4.65-6.5	4.65-6.5
工作电流 (mA)	85	80	90	90
休眠电流 (mA)	0.25	4~5	0.25~4	0.25~4
工作温度 (℃)	-20 ~ +70	-20 ~ +70	-20 ~ +70	-20 ~ +70
flash 容量 (Bytes)	4M	4M	4M	4M
eeprom 容量 (Bytes)	无	无	无	无
运行内存 (Bytes)	3584	3584	3584	3584
串口缓存区 (Bytes)	1K	1K	1K	1K
RTC 时钟	无	无	无	无
扩展 I/O	无	无	无	无
主控频率（Hz）	64M	64M	64M	64M

12.1.7.2 T1 系列 3.5 寸 ~5.0 寸

表 21: T1 系列 3.5 寸 ~5.0 寸

尺寸	3.2 寸	3.5 寸	3.9 寸	4.3	5.0
型号	TJC3224T132	TJC4832T135	TJC3232T139	TJC4827T143	TJC4827T150
比例	4:3	15:10	1:1	16:9	16:9
有效显示尺寸 (mm)	64.8×48.6	73.44×48.96	70.62x71.94	95.04x53.86	110.88x62.83
分辨率	320×240	480×320	320x320	480x272	480x272
TFT 类型	TN	TN	TN	TN	TN
亮度 (nit)	300	300	300	300	300
工作电压 (V)	4.65-6.5	4.65-6.5	4.65-6.5	4.65-6.5	4.65-6.5
工作电流 (mA)	160	120	150	230	200
休眠电流 (mA)	1~8	0.25~4	0.25~22	0.25~10	3~6
工作温度 (℃)	-20 ~ +70	-20 ~ +70	-20 ~ +70	-20 ~ +70	-20 ~ +70
flash 容量 (Bytes)	4M	8M	8M	8M	8M
eeprom 容量 (Bytes)	无	无	无	无	无
运行内存 (Bytes)	3584	3584	3584	3584	3584
串口缓存区 (Bytes)	1K	1K	1K	1K	1K
RTC 时钟	无	无	无	无	无
扩展 I/O	无	无	无	无	无

续下页

表 21 - 接上页

主控频率（Hz）	64M	64M	64M	64M	64M

备注:T1 系列休眠电流为深度睡眠模式 (lowpower=1) 下测得的电流值，电容触摸相较于无触摸或者电阻触摸需要消耗更多的电流。

12.1.8 T0 系列（即将停产）不推荐新项目选型

资料更新时间：2026-05-13 17:26:41

T0 系列包含 2.2 寸 ~ 7.0 寸

表格中的电流参数仅供参考，具体电流值请查看规格书，

相关链接

产品选型和规格书 (page 1488)

12.1.8.1 T0 系列 2.2 寸 ~3.5 寸

表 22: T0 系列 2.2 寸 ~ 3.5 寸，可用 T1 系列替代

尺寸	2.2 寸	2.4 寸	2.8 寸	3.2 寸	3.5 寸
型号	TJC3224T022	TJC3224T024	TJC3224T028	TJC4024T032	TJC4832T035
比例	4:3	4:3	4:3	16:9	4:3
有效显示尺寸 (mm)	45.12x33.84	48.96x36.72	57.6x43.2	69.6x41.76	73.44x48.96
分辨率	320×240	320×240	320×240	400X240	480×320
TFT 类型	TN	TN	TN	TN	TN
亮度 (nit)	300	300	300	300	300
工作电压 (V)	4.65-6.5	4.65-6.5	4.65-6.5	4.65-6.5	4.65-6.5
工作电流 (mA)	85	85	85	90	145
休眠电流 (mA)	15	15	15	15	15
工作温度 (℃)	-20 ~ +70	-20 ~ +70	-20 ~ +70	-20 ~ +70	-20 ~ +70
flash 容量 (Bytes)	4M	4M	4M	4M	16M
eeprom 容量 (Bytes)	无	无	无	无	无
运行内存 (Bytes)	3584	3584	3584	3584	3584
串口缓存区 (Bytes)	1K	1K	1K	1K	1K
RTC 时钟	无	无	无	无	无
扩展 I/O	无	无	无	无	无
主控频率（Hz）	48M	48M	48M	48M	48M

12.1.8.2 T0 系列 4.3 寸 ~7.0 寸

表 23: T0 系列 4.3 寸 ~ 7.0 寸，可用 X3/X5 系列替代

尺寸	4.3 寸	5 寸	7 寸
型号	TJC4827T043	TJC8048T050	TJC8048T70
比例	16:9	16:9	16:9
有效显示尺寸 (mm)	95.04x53.86	108x64.8	154.08x85.92
分辨率	480X272	800X480	800X480
TFT 类型	TN	TN	TN
亮度 (nit)	300	300	300
工作电压 (V)	4.65-6.5	4.65-6.5	4.65-6.5
工作电流 (mA)	250	410	510
休眠电流 (mA)	15	15	15
工作温度 (℃)	-20 ~ +70	-20 ~ +70	-20 ~ +70
flash 容量 (Bytes)	16M	16M	16M
eeprom 容量 (Bytes)	无	无	无
运行内存 (Bytes)	3584	3584	3584
串口缓存区 (Bytes)	1K	1K	1K
RTC 时钟	无	无	无
扩展 I/O	无	无	无

续下页

表 23 – 接上页

主控频率（Hz）	48M	48M	48M

更多尺寸、结构、UI 等需求，请点击此处了解：https://www.tjc1688.com/channels/21.html

13 电赛专题

13.1 电赛快速入门

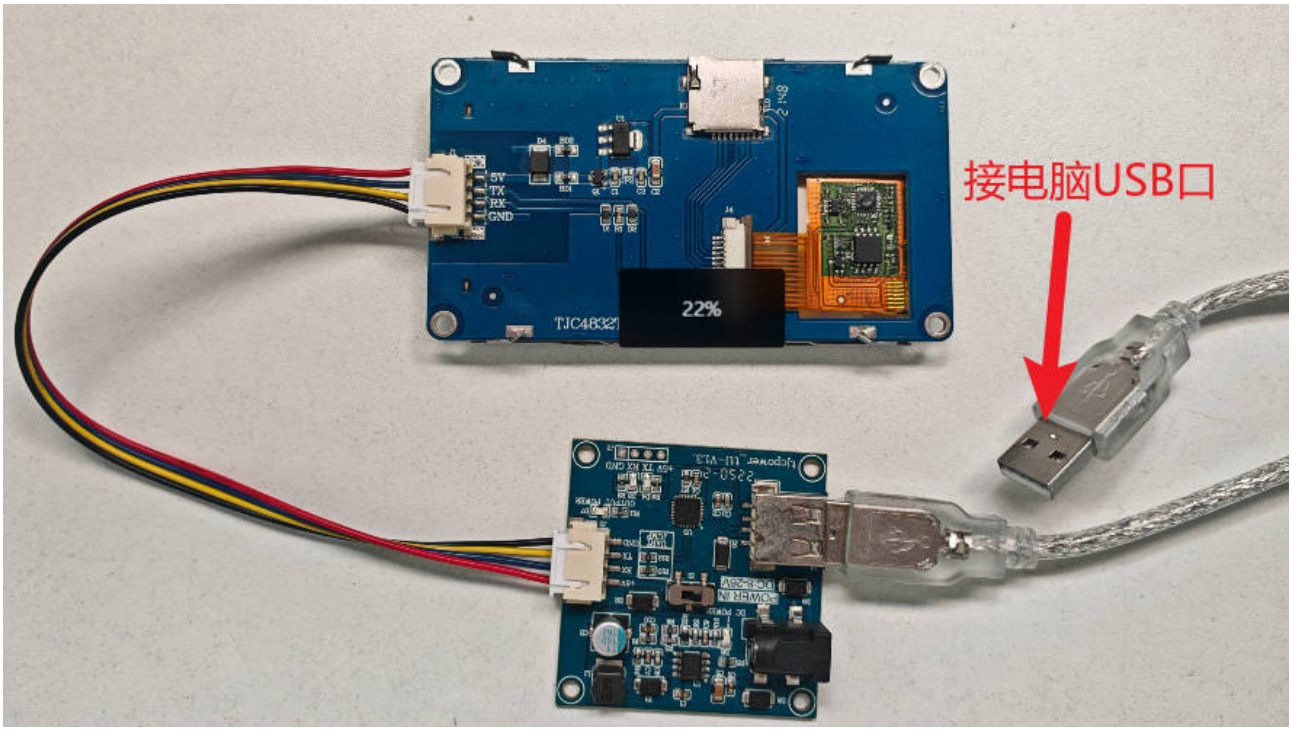
13.1.1 1、串口屏

提示： 屏幕: 推荐购买 3.5 寸及以上的尺寸

13.1.2 2、调试工具

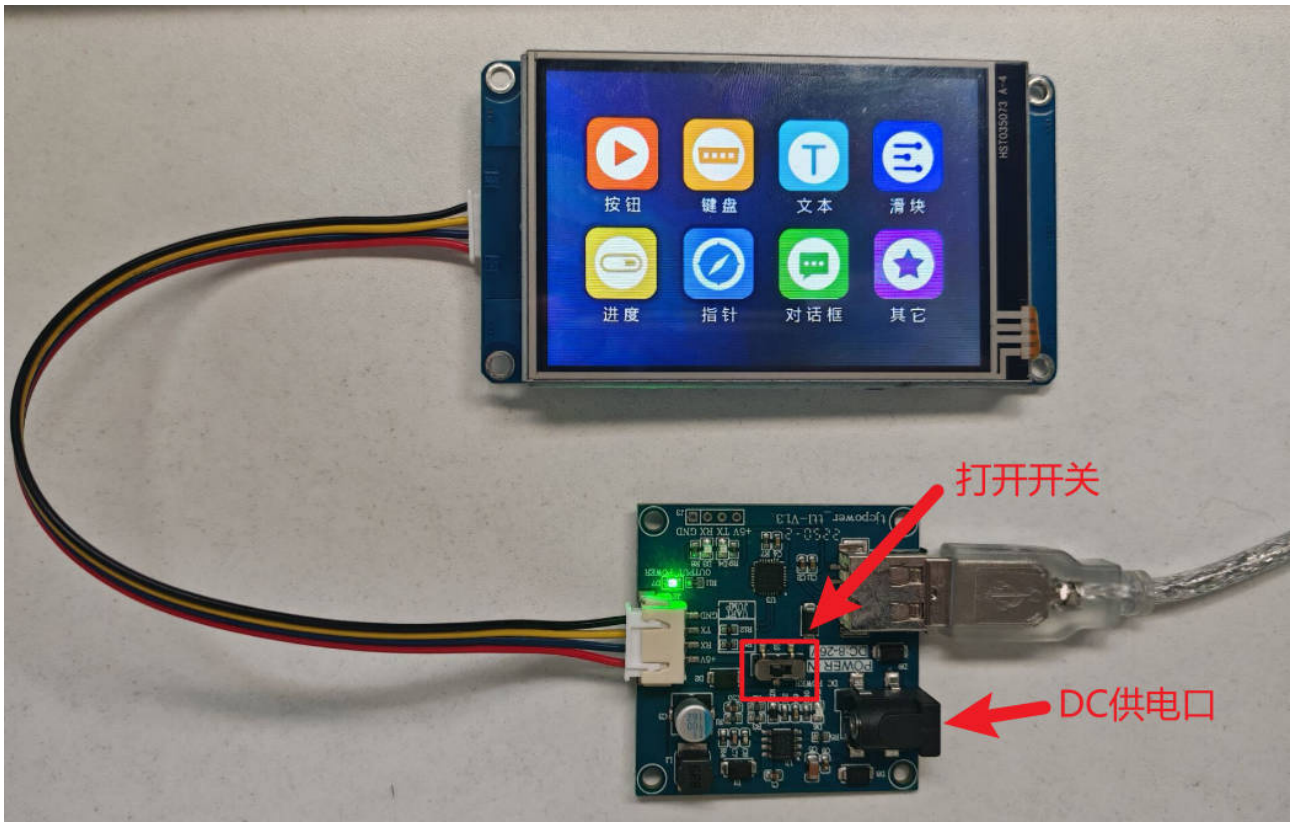
提示： 调试工具: 官方调试工具的芯片为 cp2102, 推荐使用 cp2102 或者 ft232 芯片的 USB 转 TTL 工具

13.1.3 3、连接电脑



13.1.4 4、上电

正常上电后显示如下，如果是屏幕比较大的屏在开机时闪烁则是因为供电不足，则需要在 DC 供电口接上供电

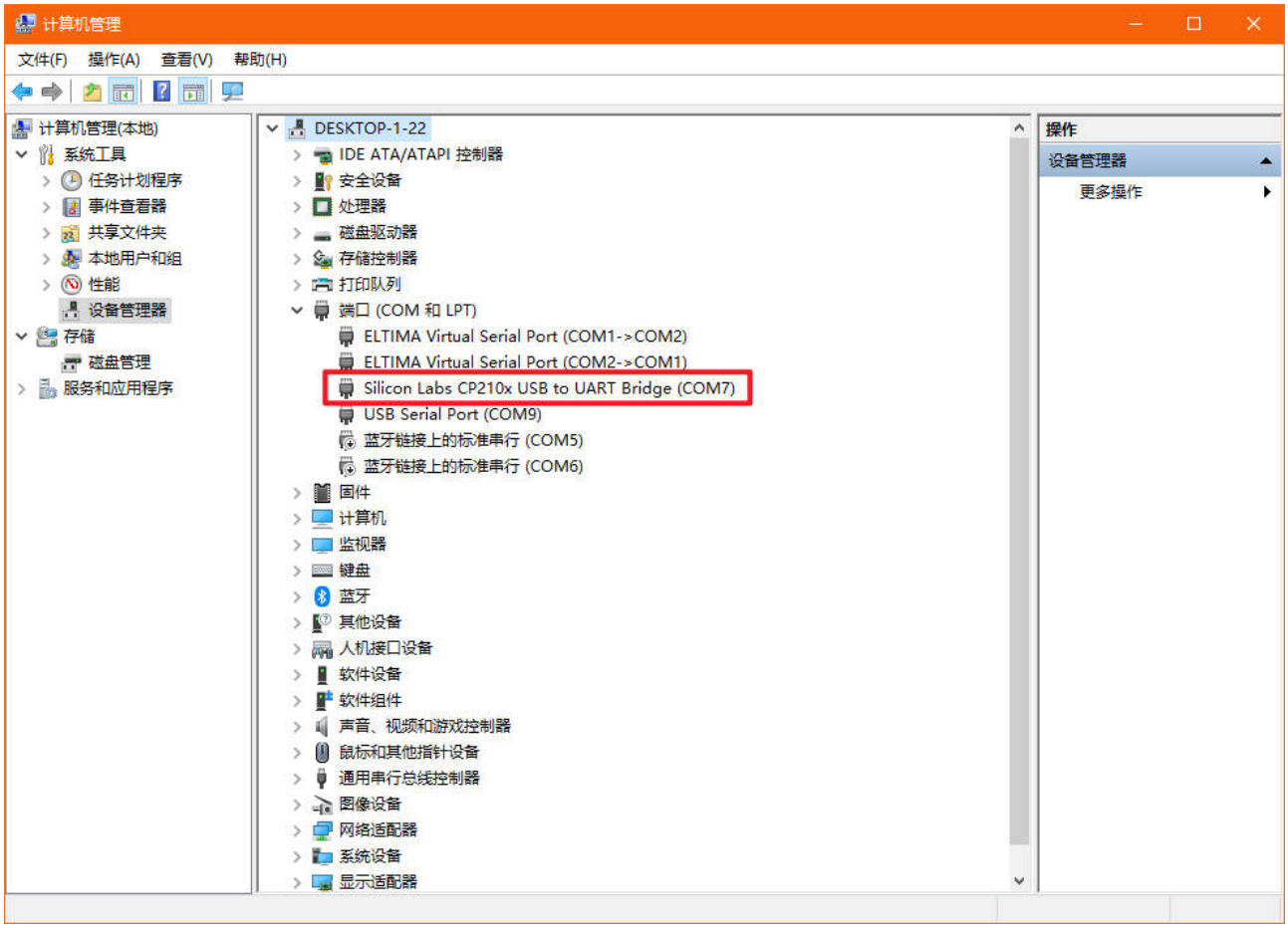


13.1.5 5、串口屏资料下载

淘晶驰资料合集 (page 1238)

13.1.6 6、驱动安装

右键此电脑（我的电脑）-> 设备-> 设备管理器-> 端口，如果已经装了驱动，会显示对应的端口号 (如下图的 com7)，如果没有显示，则需要安装对应的驱动，常见串口驱动下载



13.1.7 7、开发软件安装

点击此处下载上位机开发软件

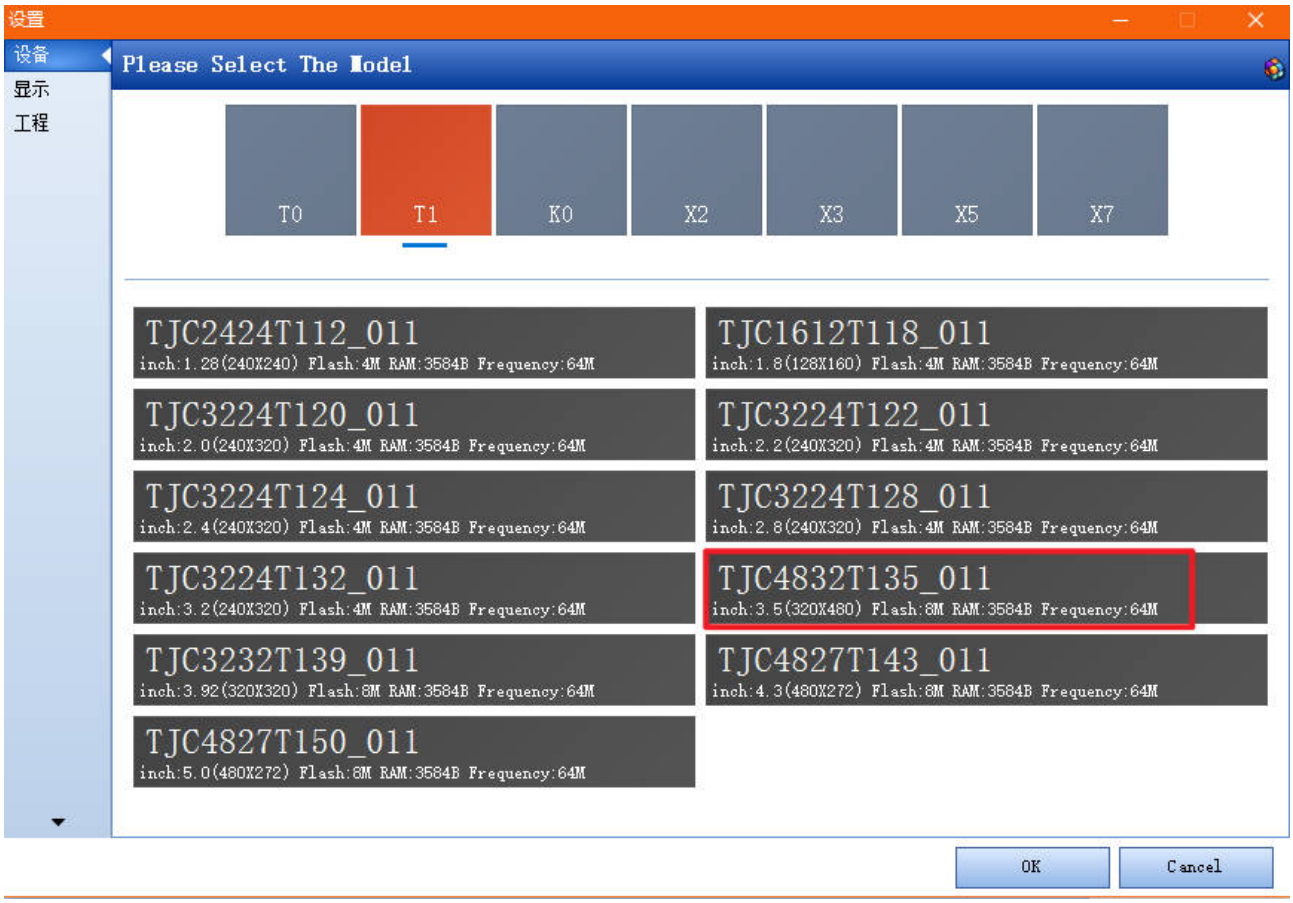
解压后安装即可

13.2 上位机使用入门

13.2.1 1、新建工程

点击新建工程-> 选择工程路径-> 填写工程名称

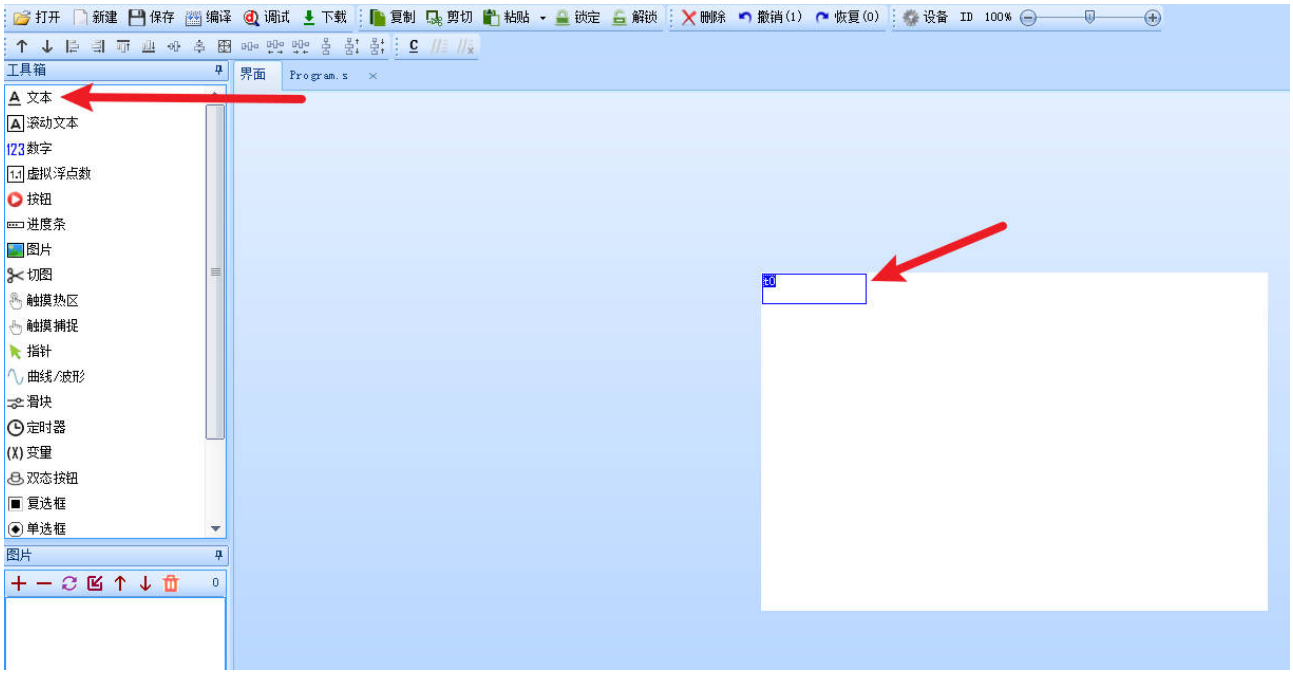
选择型号，例如现在串口屏的型号为 TJC4832T135_011R(3.5 寸电阻屏)，则选择 TJC4832T135_011 即可，不需要管它是电容屏电阻屏还是无触摸的屏，都是选择这个型号即可



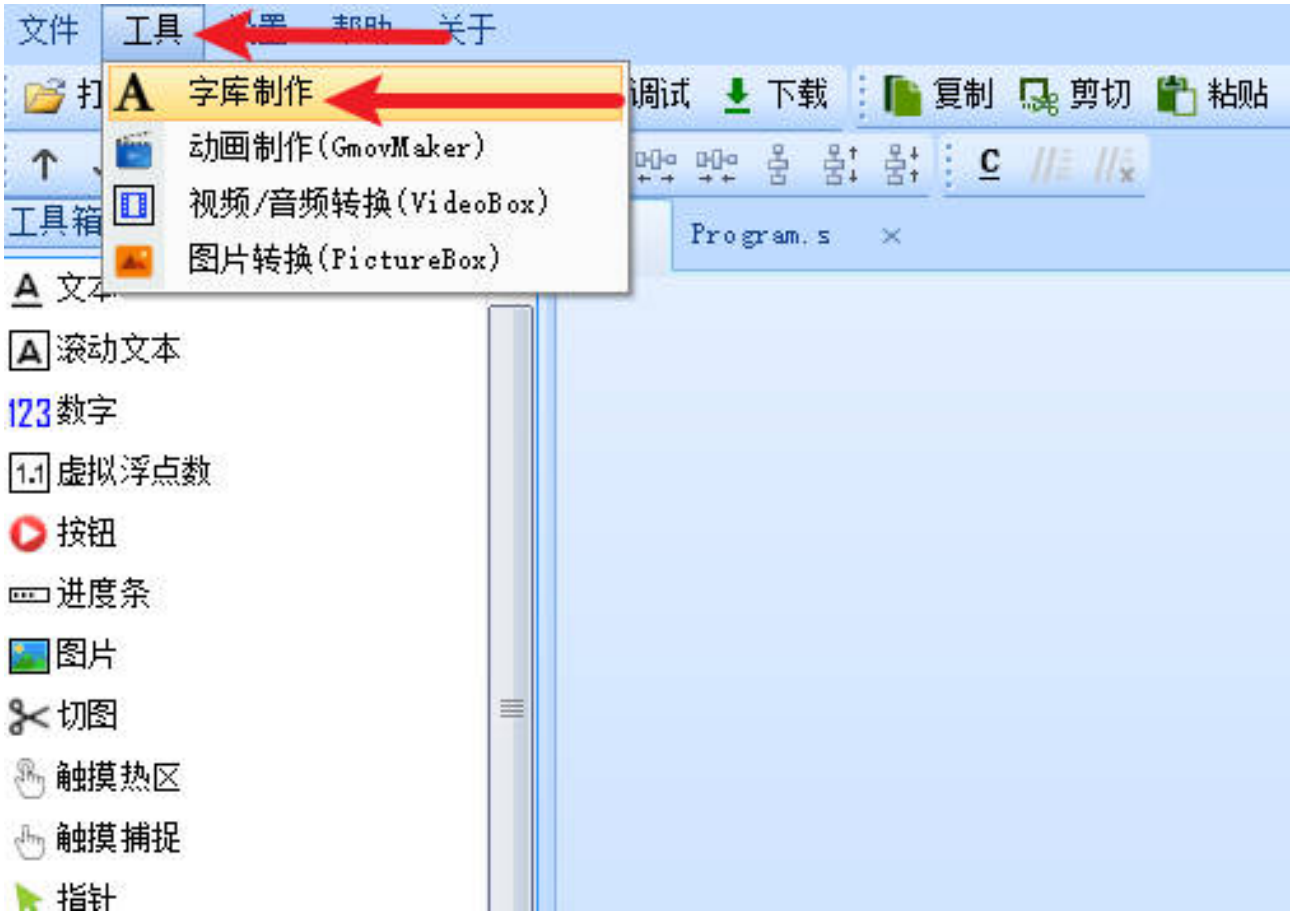
根据需求选择横屏或者竖屏，字符编码需要与单片机的字符编码一致

13.2.2 2、文本控件显示数据

点击左侧的文本控件，即可在界面上新建一个文本控件，文本控件的名称为 t0

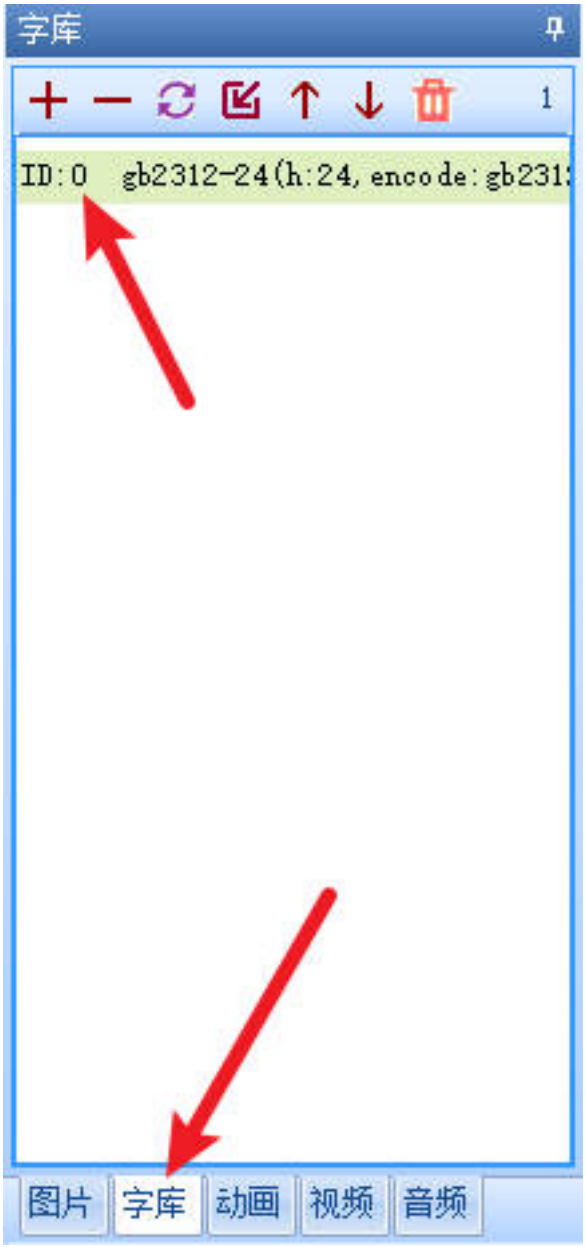


需要创建并导入字库，才能显示文字

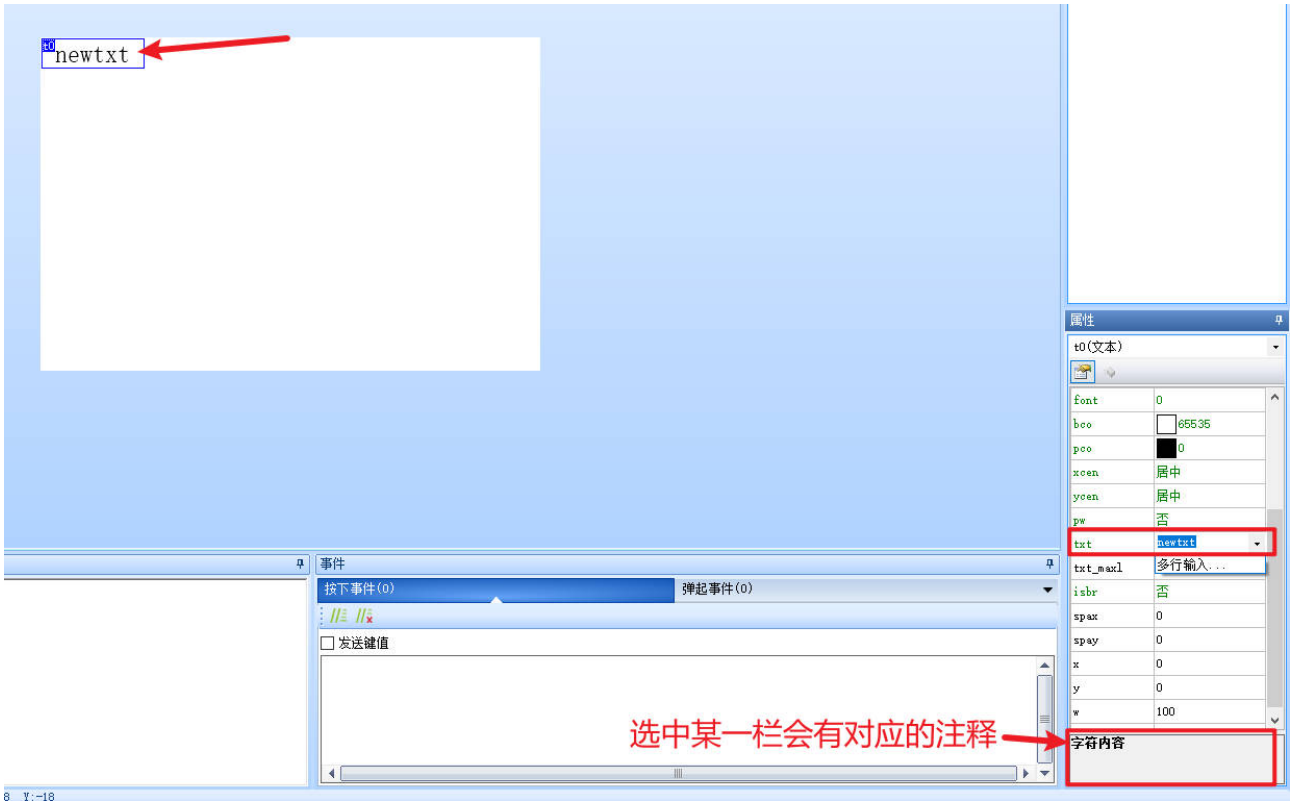


选择对应的字高，编码，字体，字库名称，点击生成字库

就可以在底部的字库界面看到对应的字库了

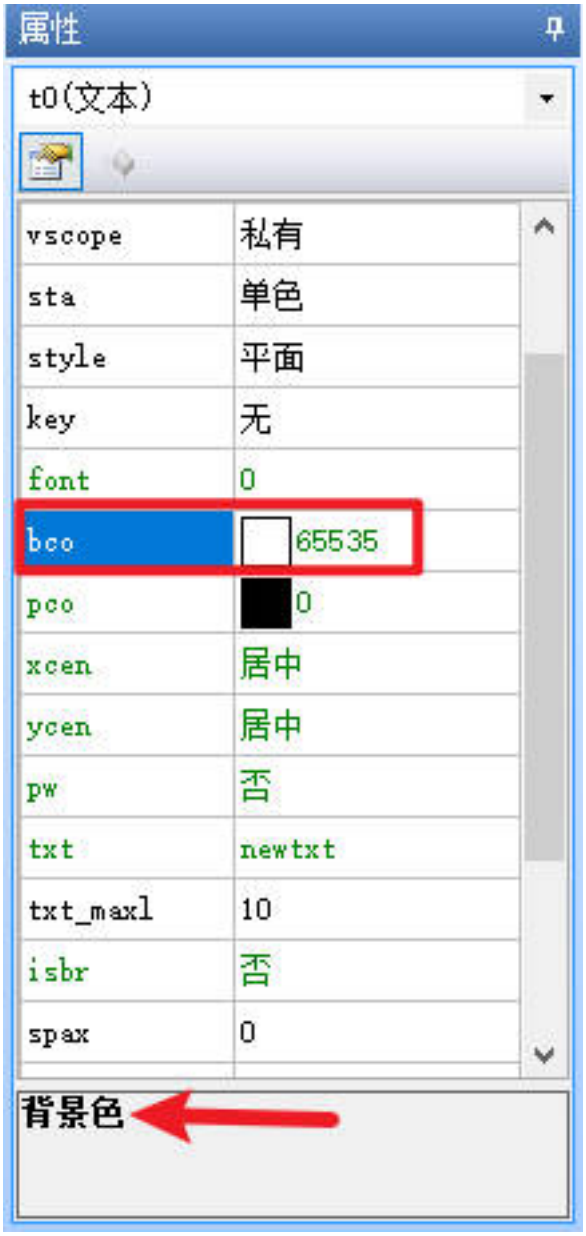


txt 属性可以显示文本，修改 txt 属性可以显示不同的文本



选择控件的属性，可以在底部看到对应的注释

属性
t0(文本)
vscope 私有
sta 单色
style 平面
key 无
font 0
bco 65535
pco 0
xcen 居中
ycen 居中
pw 否
txt newtxt
txt_maxl 10
isbr 否
spax 0
字库

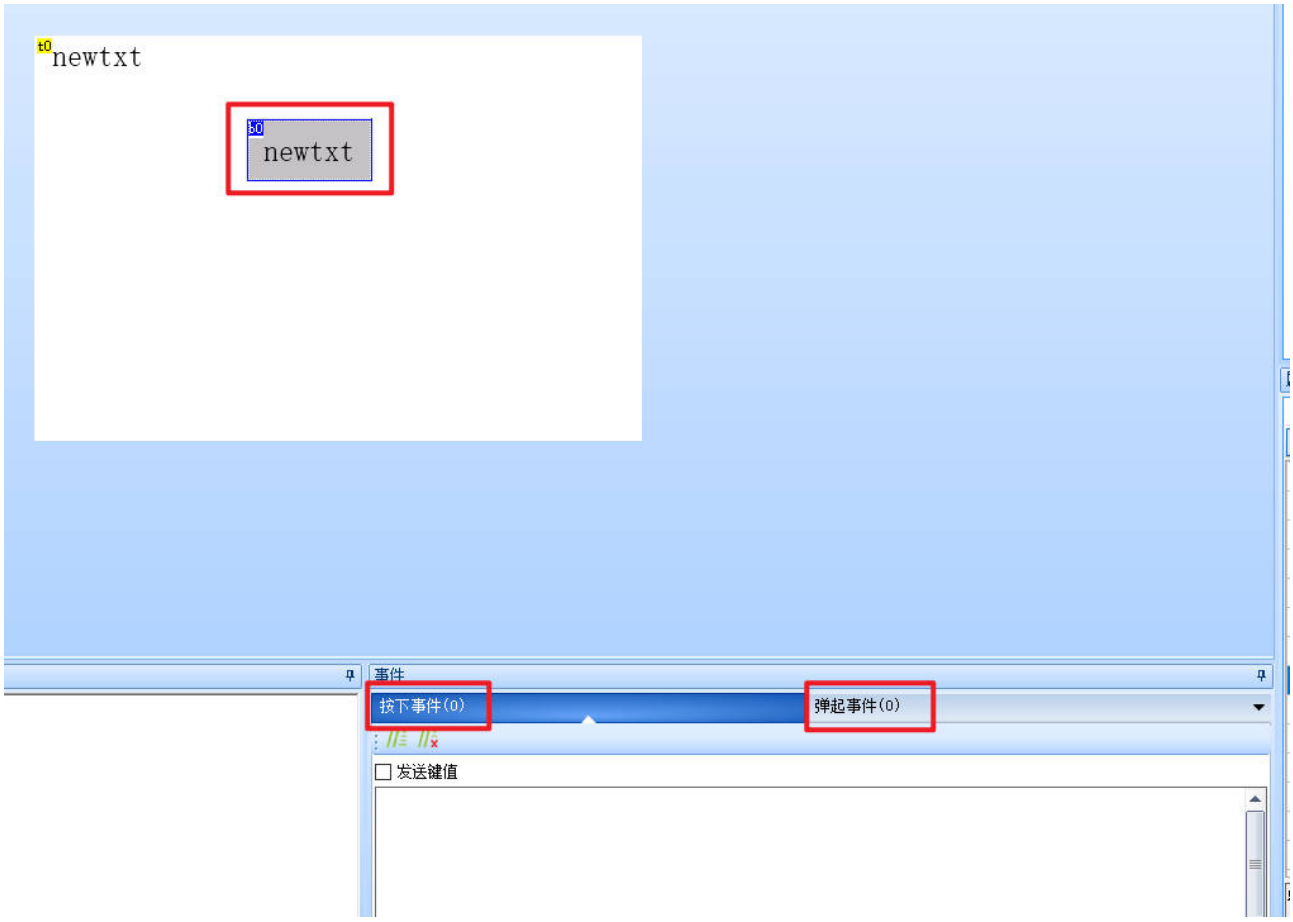

属性
t0(文本)
vscope 私有
sta 单色
style 平面
key 无
font 0
bco 65535
pco 0
xcen 居中
ycen 居中
pw 否
txt newtxt
txt_maxl 10
isbr 否
spax 0
背景色

13.2.3 3、动态修改文本

新建一个按钮控件，可以看到控件有“按下事件”和“弹起事件”

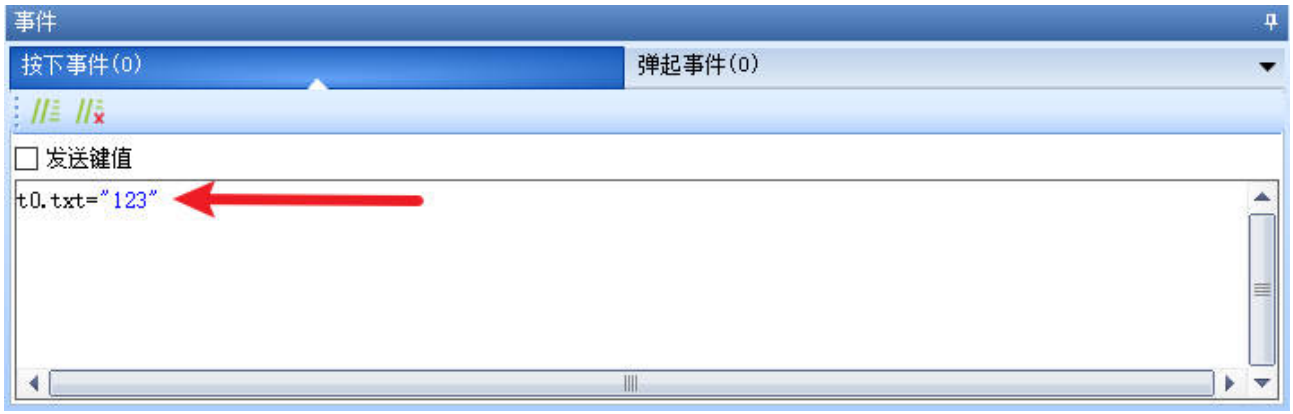
按下事件即手指按下屏幕时触发的事件

弹起事件即手指离开屏幕时触发的事件

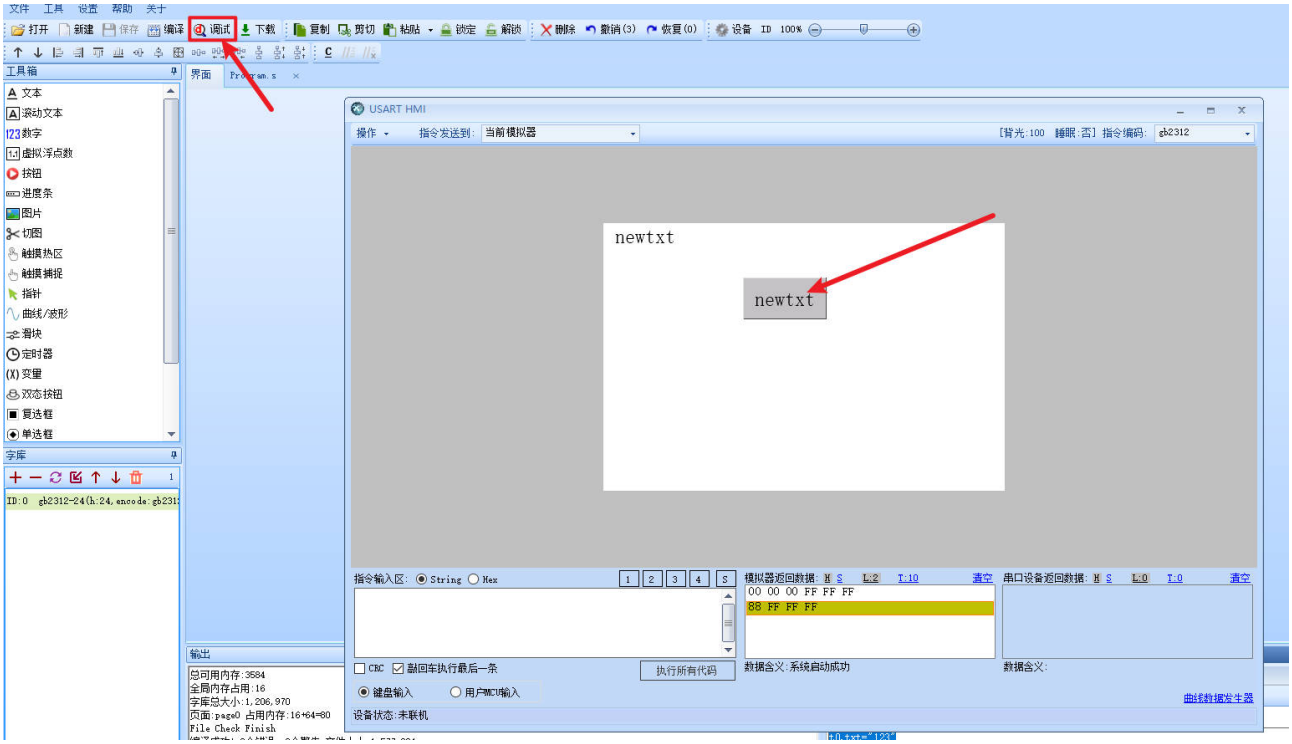


在按下事件中修改文本控件 t0 显示的文本，注意等号两边不要有空格，否则会报错

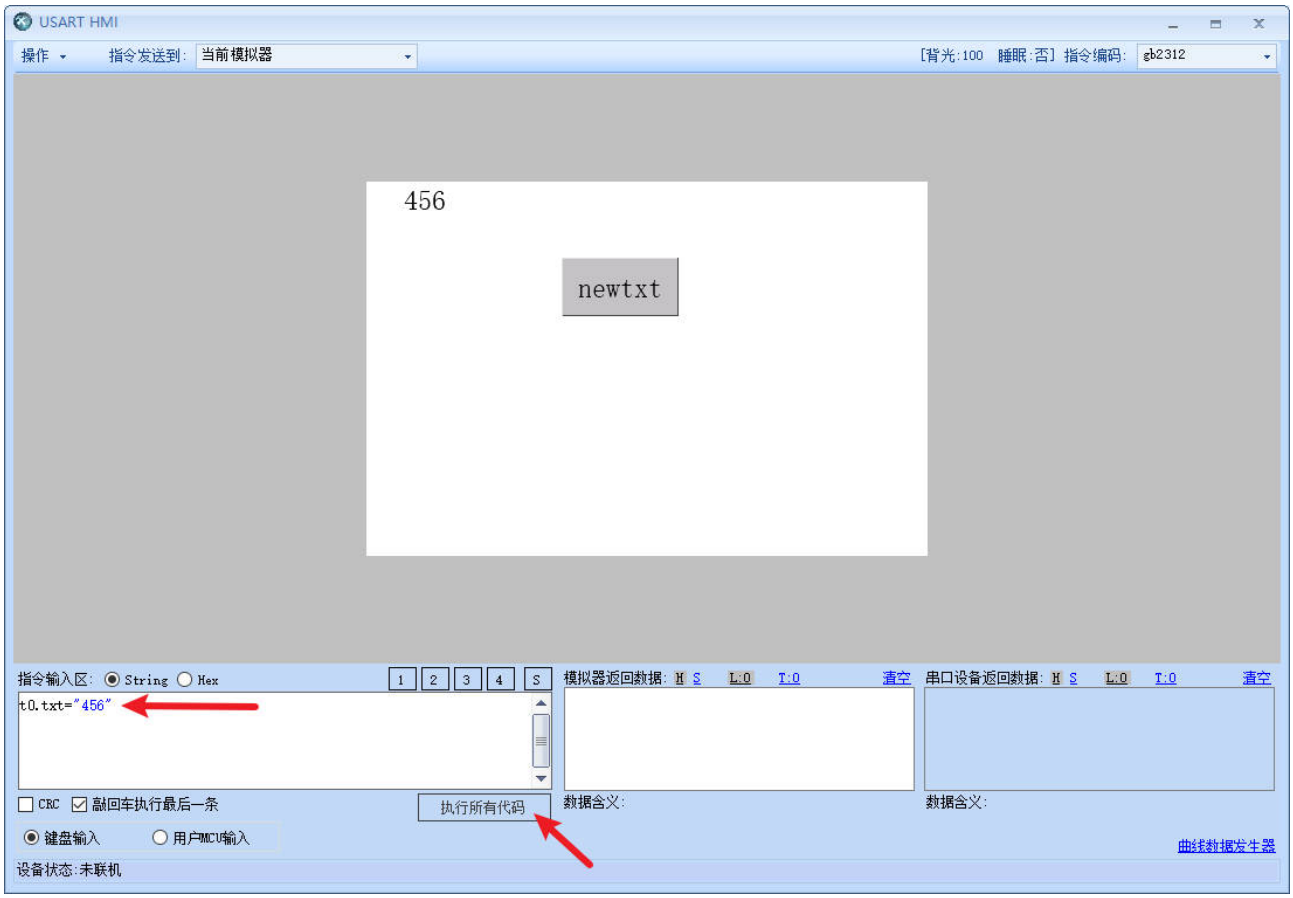
```
t0.txt="123"
```



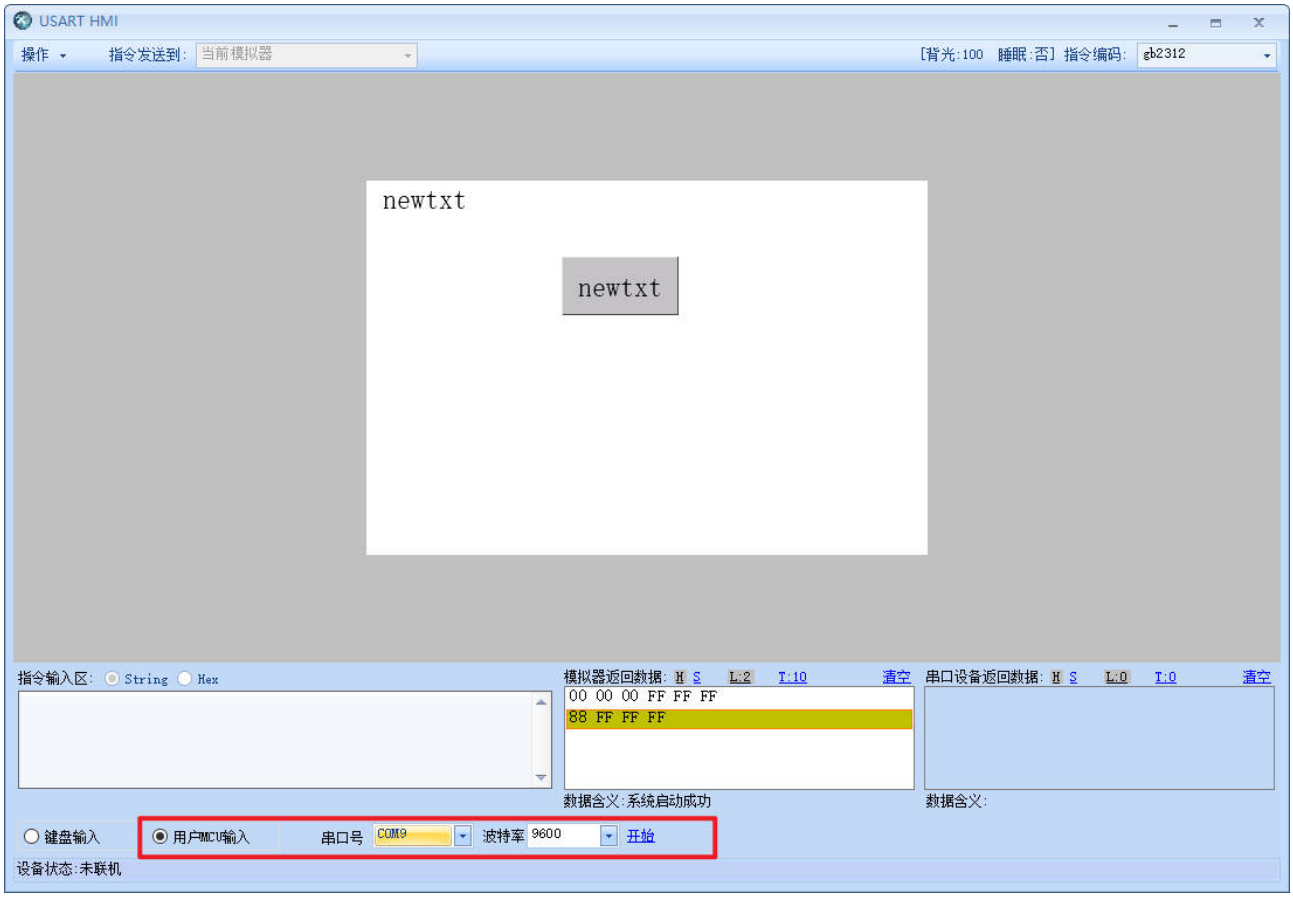
点击调试，即可调用模拟器，在模拟器上点击按钮控件，就能看到文本被修改了



调试界面也可以直接输入指令来控制屏幕上的控件



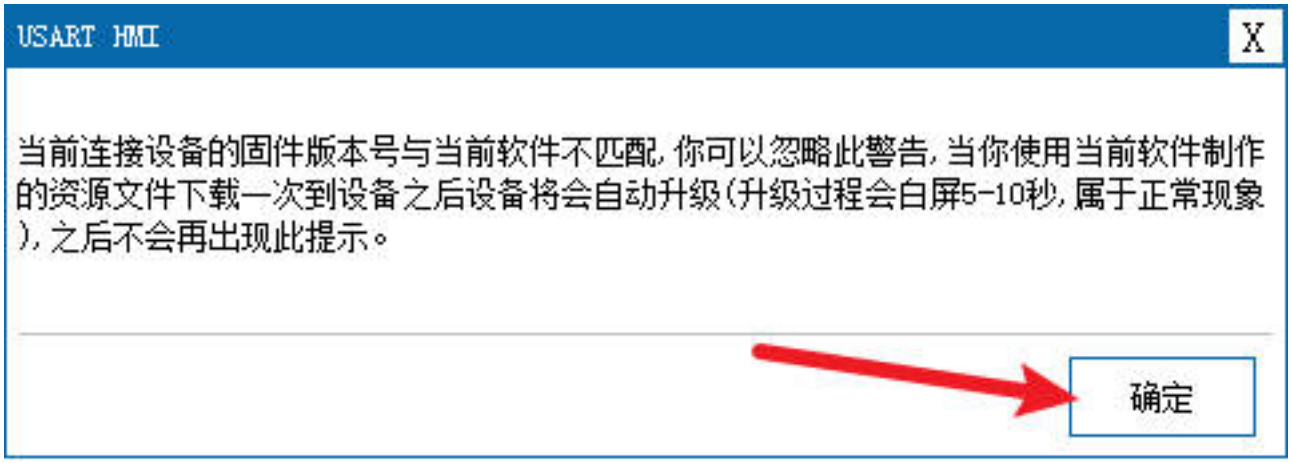
如果数据源来自串口（单片机），也可以选择“用户 MCU 输入”



13.2.4 4、下载工程

点击“下载”按钮就能下载工程，选择对应的串口号（在设备管理器里查看），波特率直接选择最高的 921600 即可，点击“联机并开始下载”即可自动下载

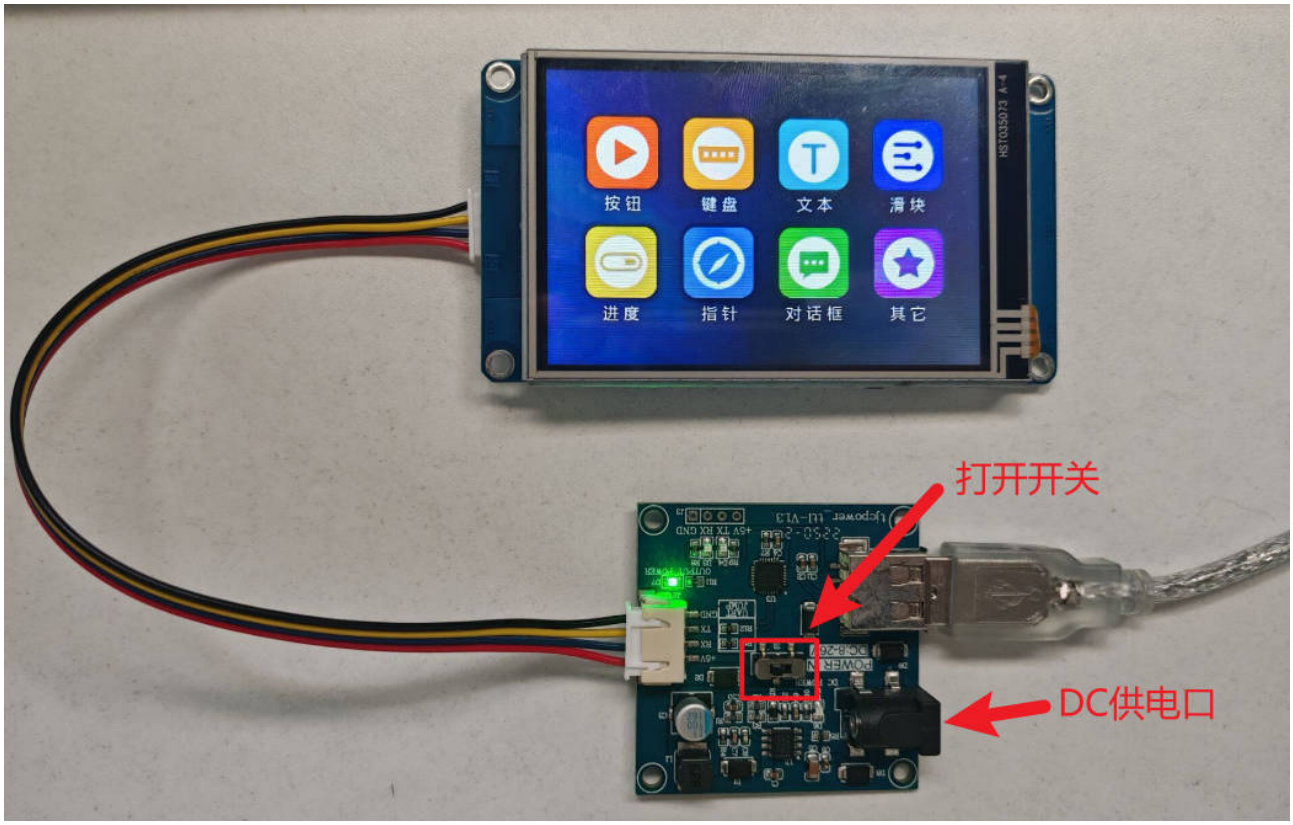
如果提示信息，点击确定即可



也可以使用 sd 卡来下载工程，参考通过 *SD* 卡下载工程到串口屏 (page 155)

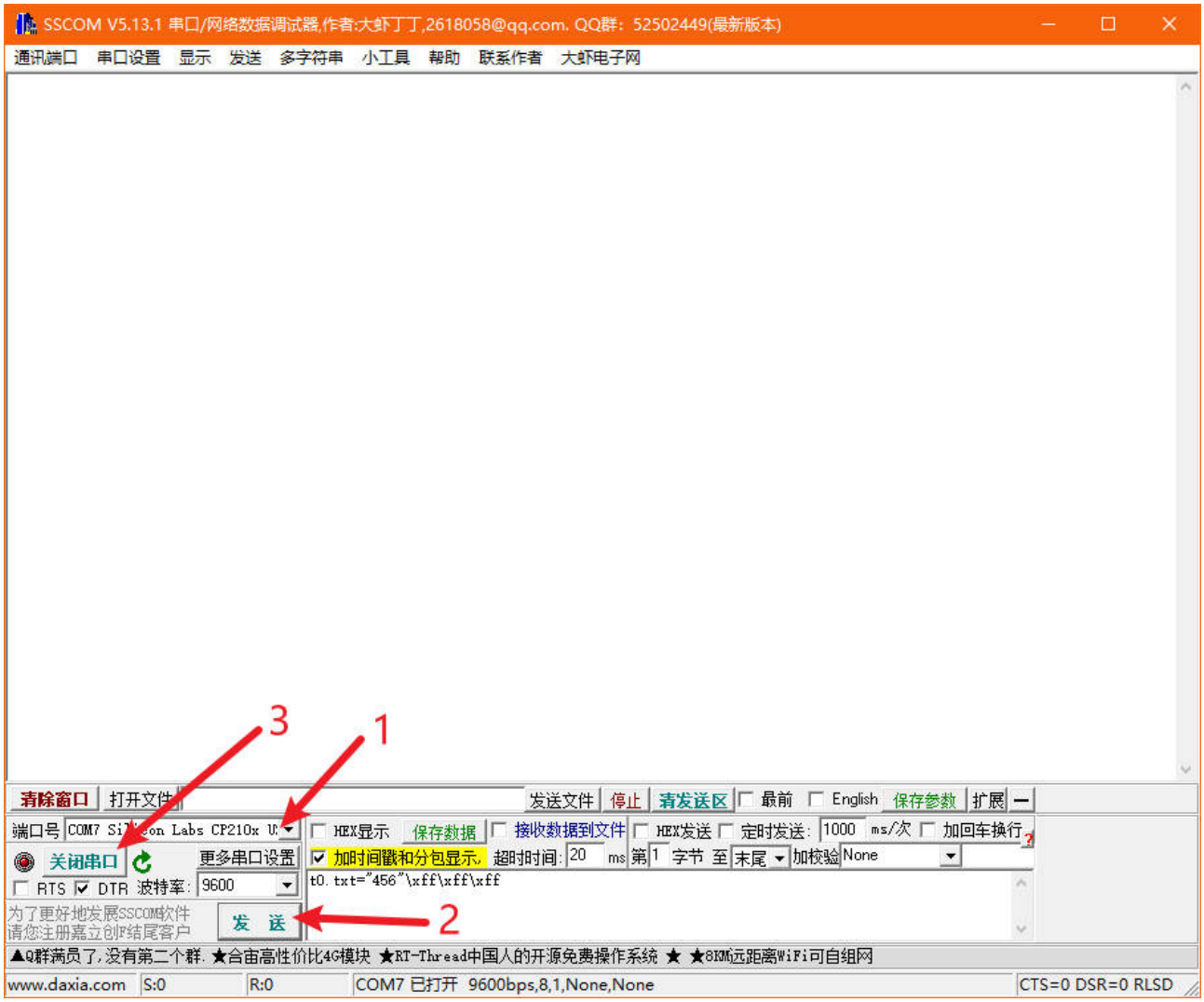
13.3 串口屏调试入门

13.3.1 sscom 调试串口屏实物



按这个方法接好之后，打开 sscom5.13.1

依次选择端口号，设置波特率，打开串口

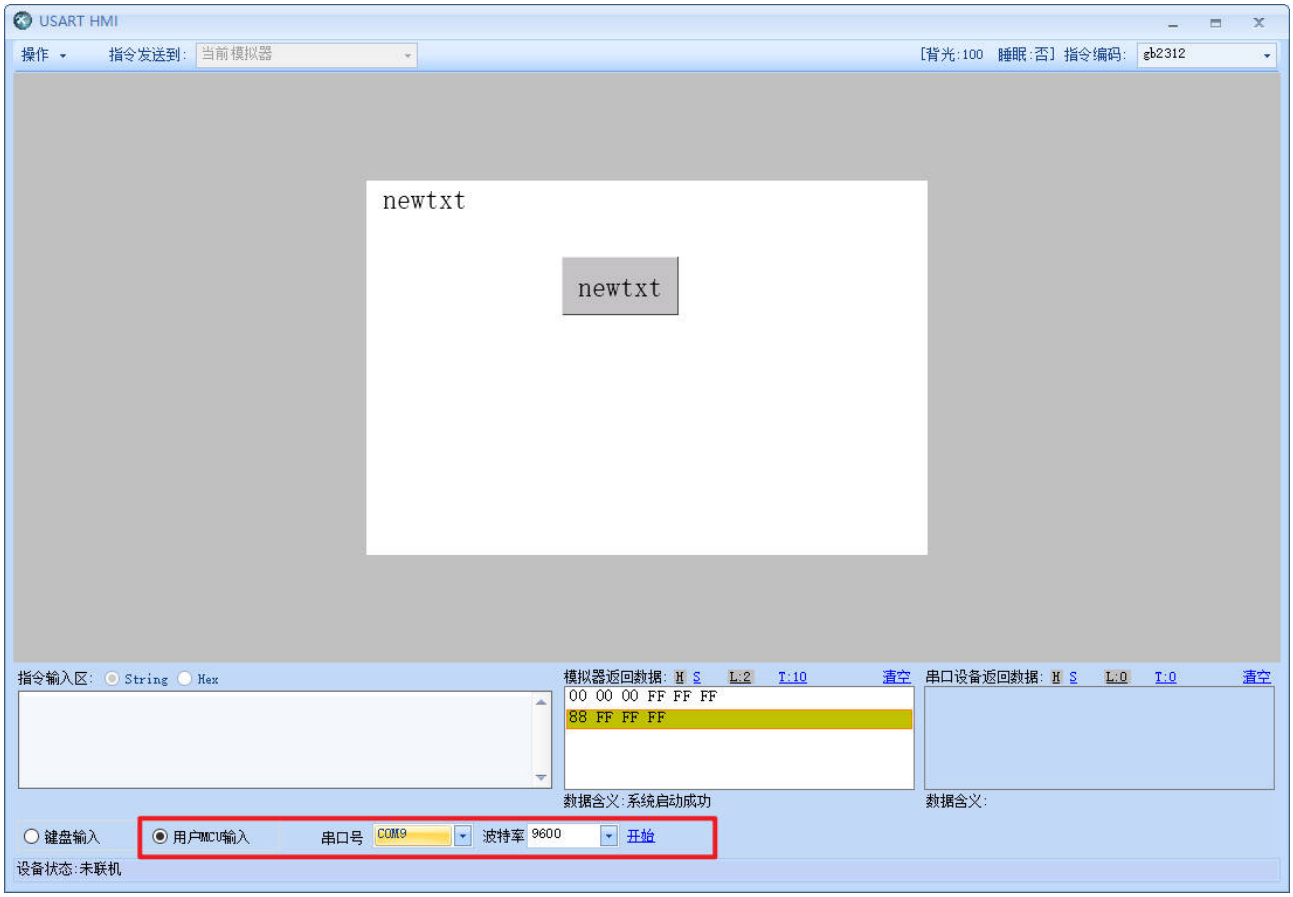


就可以编写串口指令了，sscom 上发出来的指令要加上结束符，串口屏才会执行

```
t0.txt="456"\xff\xff\xff
```

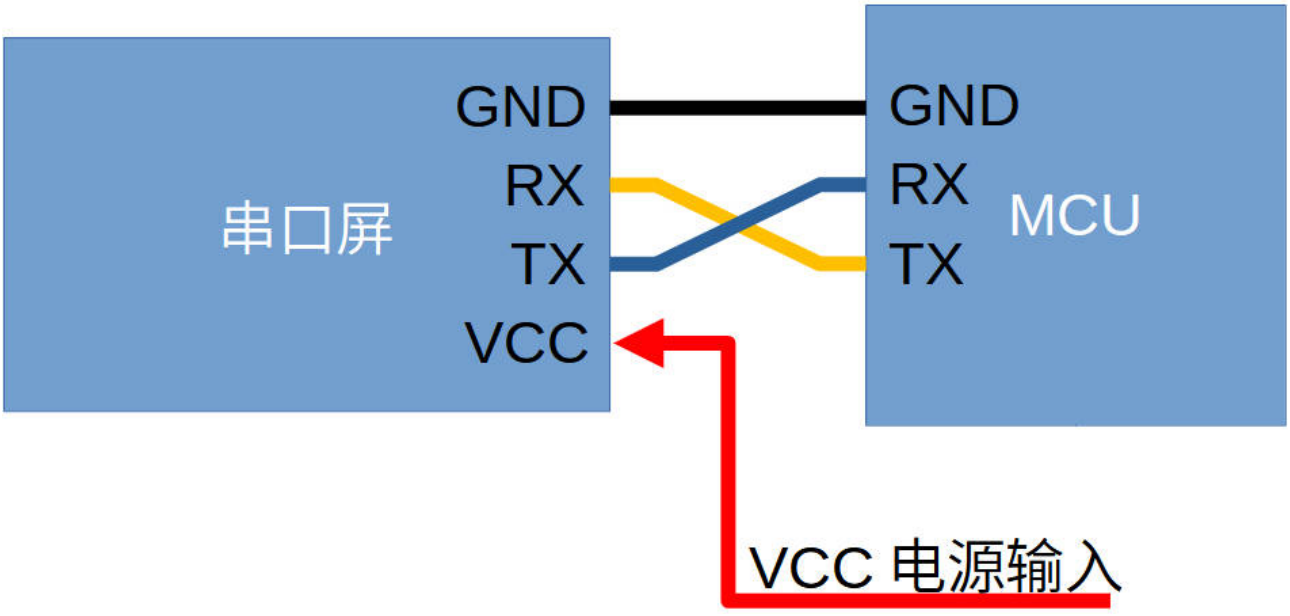
\xff\xff\xff 就是结束符，通过串口发给屏幕的每条指令最后都要加上结束符

13.3.2 模拟器接单片机



在调试界面，选择来自单片机的串口，点击开始，即可让模拟器与单片机进行通讯

13.3.3 串口屏接单片机



串口屏的 GND 要和单片机的 GND 接在一起，也就是共地

串口屏的 TX（发送）接单片机的 RX（接收）

单片机的 TX（发送）接串口屏的 RX（接收）
然后给屏幕的 VCC 输入合适的电源电压即可